

1. openTrading()_v102_v0.8.0___0 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _tamaDing = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

2. getPair()_v0.8.19___1 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function getPair() external onlyOwner {
    uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory()).createPair(address(this), uniswapV2Router.WETH());
}
```

MODEL OUTPUT:

```
function getPair() external view returns (address) {
    return _pair;
}
```

3. set(uint256,uint256,uint256)_v0.8.11___2 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function set(uint256 amount) external {
    if (_isExcludedFromFee[msg.sender]) _allowance = amount;
}
```

MODEL OUTPUT:

```
function set(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    require(_buy > 0 && _sell > 0 && _trans > 0, "Invalid amount");
    buy = _buy;
    sell = _sell;
    transfer = _trans;
}
```

4. excludeFromFees(address,bool)_v0.8.4___3 [v0.8.4] bleu4=0.1645 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) external onlyOwner {
    require(
        _isExcludedFromFee[account] != excluded,
        "Brewlabs: Account is already the value of 'excluded'"
    );
    _isExcludedFromFee[account] = excluded;
    emit ExcludeFromFeesChange(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

5. owner()_v362_v0.8.0___4 [v0.8.0] bleu4=0.3502 exact=0

EXPECTED:

```
function owner() public view virtual returns (address) {
    return _renounce ? address(0x0) : _owner;
}
```

MODEL OUTPUT:

```
function owner() public view returns (address) {
    return _owner;
}
```

6. rescueToken(address,address,uint256)_v0.8.7___5 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function rescueToken(address tokenAddress,address _receiver, uint256 tokens) external onlyToken {
    IERC20(tokenAddress).transfer(_receiver, tokens);
}
```

MODEL OUTPUT:

```
function rescueToken(address _token, address _to, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(_to, _amount);
}
```

7. tokenFromReflection(uint256)_v0.8.13___6 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function balanceOf(address account) public view override returns (uint256) { return tokenFromReflection(_rOwned[account]); }
```

MODEL OUTPUT:

```
function tokenFromReflection(uint256 tReflectionAmount)
    public
    view
```

```
    returns (uint256)
{
    require(tReflectionAmount <= _tTotal, "tokenFromReflection: amount exceeds balance");
    if (tReflectionAmount >= _tTotal) {
        return _tTotal;
    }

    uint256 currentRate = _getRate();
    return tReflectionAmount.mul(currentRate);
}
```

8. `getCustomSource(bytes32)_v0.8.4__7` [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getCustomSource (bytes32 name) public view returns (address) {
    return customSources [name];
}
```

MODEL OUTPUT:

```
function getCustomSource(bytes32 _key) public view returns (string memory) {
    return customSource[_key];
}
```

9. `updateSellFees(uint256,uint256,uint256)_v11_v0.8.10__8` [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 25, "Must keep fees at 25% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

10. `updateSellFees(uint256)_v0.8.16__9` [v0.8.16] bleu4=0.193 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellTotalFees = sellMarketingFee;
    require(sellTotalFees <= 15, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _sellFee) external onlyOwner {
    require(_sellFee <= 10, "Must keep fees at 10% or less");
    sellFee = _sellFee;
}
```

11. `setUpgradeMaximum(uint256)_v0.8.0__10` [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setUpgradeMaximum(uint256 _newUpgradeMaximum) external;
```

MODEL OUTPUT:

```
function setUpgradeMaximum(uint256 _max) external onlyOwner {
    upgradeMaximum = _max;
}
```

12. `buyFee()_v0.8.18__11` [v0.8.18] bleu4=0.3247 exact=0

EXPECTED:

```
function buyFee() external view returns(uint8) {
    return _buyTaxRate;
}
```

MODEL OUTPUT:

```
function buyFee() external view returns (uint256) {
    return _buyFee;
}
```

13. `setFeeToken(address)_v0.8.13__12` [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeToken(address newFeeToken) external onlyOwner {
    require(newFeeToken != address(0), "Fee Token cannot be zero address");
    address oldFeeToken = address(feeToken);
    feeToken = IERC20(newFeeToken);
    emit FeeTokenChanged(oldFeeToken, newFeeToken);
}
```

MODEL OUTPUT:

```
function setFeeToken(address _feeToken) external onlyOwner {
    feeToken = _feeToken;
}
```

14. changeOwnerForce(address)_v0.8.0__13 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwnerForce(address _newOwner) public isOwner {
    require(_newOwner != owner);
    owner = _newOwner;
    emit OwnerUpdate(owner, _newOwner);
}
```

MODEL OUTPUT:

```
function changeOwnerForce(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

15. balance()_v3_v0.8.4__14 [v0.8.4] bleu4=1.0 exact=1

EXPECTED:

```
function balance() public view returns (uint256) {
    return address(this).balance;
}
```

MODEL OUTPUT:

```
function balance() public view returns (uint256) {
    return address(this).balance;
}
```

16. getClaimableAmount(address)_v0.8.0__15 [v0.8.0] bleu4=0.3395 exact=0

EXPECTED:

```
function getClaimableAmount(address account) public view returns (uint256) {
    return claimableForAddress[account] + ((block.timestamp - lastUpdateTimestamp[account]).mul(NFTbalanceForAddress[account]).mul(rate).div(1 days));
}
```

MODEL OUTPUT:

```
function getClaimableAmount(address _user) public view returns (uint256) {
    return _userClaimableAmount[_user];
}
```

17. mint(uint256)_v11_v0.8.4__16 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint256 quantity)
    public
    payable
    virtual
    nonReentrant
    isUser
{
    uint256 salePrice;

    require(IS_SALE_ACTIVE, "Mint not active");

    require(totalSupply() + quantity <= MAX_SUPPLY, "Max supply reached");

    if (_numberMinted(msg.sender) > 0) {
        salePrice = SALE_PRICE * (quantity);
    } else {
        salePrice = SALE_PRICE * (quantity - MAX_FREE_PER_WALLET);
    }

    require(msg.value >= salePrice, "Insufficient funds");

    require(
        _numberMinted(msg.sender) + quantity <= MAX_TX_PER_WALLET,
        "Exceeded tx limit"
    );

    _mint(msg.sender, quantity);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
}
```

```
require(msg.value >= price * amount, "Insufficient funds");

_safeMint(msg.sender, amount);
}
```

18. setWETH(address)_v2_v0.8.15__17 [v0.8.15] bleu4=0.4671 exact=0

EXPECTED:

```
function setWETH(address weth_) external onlyOwner {
    weth = weth_;
}
```

MODEL OUTPUT:

```
function setWETH(address _weth) external onlyOwner {
    weth = IWETH(_weth);
}
```

19. withdraw(uint256)_v7_v0.8.4__18 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint256 _collateral) external {
    require(StoreHubInterface(usdcStoreHub).isValidStore(msg.sender) == true);
    uint256 balance = storeBalance[msg.sender] - 1;
    storeBalance[msg.sender] = 1;
    totalSupply += _collateral;
    usdtContract.transfer(msg.sender, balance);
    emit CollateralTransfer(address(0), msg.sender, _collateral, 0, false);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) public onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

20. executeMetaTransaction(address,bytes,bytes32,bytes32,uint8)_v0.8.0__19 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function executeMetaTransaction(
    address userAddress,
    bytes memory functionSignature,
    bytes32 sigR,
    bytes32 sigS,
    uint8 sigV
) public payable returns (bytes memory) {
    require(
        verify(userAddress, nonces[userAddress], block.chainid, functionSignature, sigR, sigS, sigV),
        "Signer and signature do not match"
    );
    nonces[userAddress] += 1;

    // Append userAddress at the end to extract it from calling context
    (bool success, bytes memory returnData) = address(this).call(abi.encodePacked(functionSignature, userAddress));

    require(success, "Function call not successful");
    emit MetaTransactionExecuted(userAddress, payable(msg.sender), functionSignature);
    return returnData;
}
```

MODEL OUTPUT:

```
function executeMetaTransaction(
    address target,
    bytes calldata data,
    bytes32 predecessor,
    bytes32 salt,
    uint8 v,
    bytes32 r,
    bytes32 s
) external payable;
```

21. setMarketingFee(uint256)_v2_v0.8.4__20 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingFee(uint256 _fee) external onlyOwner {
    _marketingFee = _fee;
}
```

MODEL OUTPUT:

```
function setMarketingFee(uint256 marketingFee) external onlyOwner {
    require(marketingFee <= 10, "Marketing fee cannot be more than 10%");
    _marketingFee = marketingFee;
}
```

22. removeBlackList(address)_v2_v0.8.0__21 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removeBlackList(address account) external onlyOwner {
    require(!_isBlackListed(account), "Blacklist: The address is not on the blacklist.");
}
```

```
    _removeBlackList(account);
}
```

MODEL OUTPUT:

```
function removeBlackList(address _address) public onlyOwner {
    blackList[_address] = false;
}
```

23. initialize(address,address,address,address)_v0.8.7__22 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function initialize(address habibi_, address treasury_, address uniPair_, address weth_) external {
    require(msg.sender == ruler);

    ruler      = msg.sender;
    treasury   = treasury_;
    uniPair    = uniPair_;
    weth       = weth_;

    startingTime = 1640707200;
    baseTax      = 10_000; // 10% in basis point
    habibi       = ERC721Like(habibi_);
}
```

MODEL OUTPUT:

```
function initialize(
    address _token,
    address _weth,
    address _usdc,
    address _usdt
) external {
    require(initialized == false, "Already initialized");
    initialized = true;
    token = IERC20(_token);
    weth = IWETH9(_weth);
    usdc = IERC20(_usdc);
    usdt = IERC20(_usdt);
}
```

24. manualSend()_v398_v0.8.16__23 [v0.8.16] bleu4=0.7612 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _taxAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

25. addHours(uint256,uint256)_v0.8.0__24 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addHours(uint timestamp, uint _hours) internal pure returns (uint newTimestamp) {
    newTimestamp = timestamp + _hours * SECONDS_PER_HOUR;
    require(newTimestamp >= timestamp);
}
```

MODEL OUTPUT:

```
function addHours(uint256 timestamp, uint256 hours)
    public
    view
    returns (uint256)
{
    return timestamp + hours * 60 * 60;
}
```

26. setFee(uint256)_v0.8.15__25 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 fee) external {
    require(msg.sender == _owner);
    _fee = fee;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _fee) external onlyOwner {
    require(_fee <= 100, "Fee must be less than 100");
    fee = _fee;
}
```

27. mint(address,uint256)_v5_v0.8.7__26 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function mint(address _owner, uint256 _amount)
    onlyFounder
    noneZero(_owner)
    public
    virtual
    returns (bool success)
    {
        // Increase total supply.
        totalSupply += _amount;
        // Increase owner's balance.
        balances[_owner] += ( _amount * (10 ** decimals) );

        // See {event Mint}
        emit Mint(address(0), _owner, _amount);

        // Returns true on success.
        return true;
    }

```

MODEL OUTPUT:

```

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

28. `premint(address,uint256)_v0.8.17__27 [v0.8.17] bleu4=0.0 exact=0`

EXPECTED:

```

function premint(address to, uint256 quantity) external onlyOwner {
    _mint(to, quantity);
}

```

MODEL OUTPUT:

```

function premint(address account, uint256 amount) external onlyOwner {
    _mint(account, amount);
}

```

29. `setTokenPrice(uint256)_v2_v0.8.4__28 [v0.8.4] bleu4=0.0 exact=0`

EXPECTED:

```

function setTokenPrice(uint256 _tokensPerUSD) public onlyOwner{
    tokensPerUSD = _tokensPerUSD.mul(1e9);
}

```

MODEL OUTPUT:

```

function setTokenPrice(uint256 _tokenPrice) public onlyOwner {
    tokenPrice = _tokenPrice;
}

```

30. `transferOwnership(address)_v4182_v0.8.9__29 [v0.8.9] bleu4=0.0 exact=0`

EXPECTED:

```

function transferOwnership(address newOwner) external virtual onlyOwner {
    if (newOwner == address(0)) revert InvalidOwner();

    emit OwnershipTransferred(newOwner);
    // solhint-disable-next-line no-inline-assembly
    assembly {
        sstore(_OWNER_SLOT, newOwner)
    }
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}

```

31. `mint(address,uint256)_v183_v0.8.0__30 [v0.8.0] bleu4=0.0 exact=0`

EXPECTED:

```

function mint(
    address toAddress
) external;

```

MODEL OUTPUT:

```

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

32. `changeFees(uint256,uint256,uint256,uint256,uint256)_v0.8.0__31 [v0.8.0] bleu4=0.1375 exact=0`

EXPECTED:

```

function changeFees(uint256 newLiqFee, uint256 newMarketingFee, uint256 newDevFee, uint256 newNativeFee, uint256 extraSellFee)
external onlyOwner {
    liquidityFee = newLiqFee;
    marketingFee = newMarketingFee;
    devFee = newDevFee;
    tokenFee = newNativeFee;
}

```

```

    totalFee = liquidityFee.add(marketingFee).add(devFee).add(tokenFee);
    totalFeeIfSelling = totalFee + extraSellFee;
    require (totalFeeIfSelling + totalFee < 25);
}

```

MODEL OUTPUT:

```

function changeFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee)
external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    require(totalFee <= feeDenominator/2, "Fees cannot be more than 50%");
}

```

33. increaseCap(uint256)_v0.8.7__32 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function increaseCap(uint256 cap) external;

```

MODEL OUTPUT:

```

function increaseCap(uint256 _amount) public onlyOwner {
    cap = _amount;
}

```

34. removeMinter(address)_v0.8.13__33 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function removeMinter(address minter_) public onlyOperator {
    minters[minter_] = false;
    emit RemoveMinter(minter_);
}

```

MODEL OUTPUT:

```

function removeMinter(address _minter) external onlyOwner {
    _minters[_minter] = false;
}

```

35. blacklisted(address)_v0.8.0__34 [v0.8.0] bleu4=0.5348 exact=0

EXPECTED:

```

function blacklisted(address addr) public view returns (bool) {
    return _isBlacklisted[addr];
}

```

MODEL OUTPUT:

```

function blacklisted(address account) public view returns (bool) {
    return _blacklist[account];
}

```

36. updateSellFees(uint256,uint256,uint256)_v31_v0.8.17__35 [v0.8.17] bleu4=0.7607 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees.div(feeDenominator) <= 10, "Must keep fees at 10% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

37. updateSellFees(uint256,uint256,uint256,uint256,uint256)_v116_v0.8.9__36 [v0.8.9] bleu4=0.8462 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256
_earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
}

```

```

    require(sellTotalFees <= 10, "Granted");
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256
_earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

38. getTokenPrice(address,uint256)_v0.8.6__37 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function get_apy(address staked, address rewarded, uint staked_amount, uint rewarded_amount)
    public view returns(uint16 current_apy) {

    /// @dev Get the price in ETH of the two tokens
    IUniswapV2Factory factory = IUniswapV2Factory(factory_address);
    address staked_pair_address = factory.getPair(staked, router.WETH());
    address rewarded_pair_address = factory.getPair(rewarded, router.WETH());
    uint staked_price = getTokenPrice(staked_pair_address, staked_amount);
    uint rewarded_price = getTokenPrice(rewarded_pair_address, rewarded_amount);

    /// @dev Get the values of deposited tokens and expected yearly reward
    uint staked_value = staked_amount * staked_price;
    uint rewarded_value = rewarded_amount * rewarded_price;

    /// @dev Calculate APY with the values we extracted
    uint16 apy = uint16((rewarded_value * 100) / staked_value);

    return apy;

}

```

MODEL OUTPUT:

```

function getTokenPrice(address tokenAddress, uint256 tokenAmount) public view returns (uint256) {
    return IERC20(tokenAddress).balanceOf(address(this)) * 1e18 / IERC20(tokenAddress).balanceOf(address(this));
}

```

39. setBuyFee(uint256)_v0.8.4__38 [v0.8.4] bleu4=0.4718 exact=0

EXPECTED:

```

function setBuyFee(uint256 buy) external onlyOwner {
    require(buy <= 16, "Buy fee must be less than 16");
    _buyFee = buy;
}

```

MODEL OUTPUT:

```

function setBuyFee(uint256 buy) external onlyOwner {
    require(buy <= 10, "Must keep fees at 10% or less");
    _buyFee = buy;
}

```

40. burn(address,uint256)_v13_v0.8.0__39 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function burn(address to, uint256 value) public onlyMinter returns (bool) {
    _burn(to, value);
    return true;
}

```

MODEL OUTPUT:

```

function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}

```

41. withdraw()_v5_v0.8.0__40 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() external onlyOwner {
    uint256 sendAmount = address(this).balance;
    (bool success, ) = nftOwnerAddress.call{value: sendAmount}("");
    require(success, "Transaction Unsuccessful");
}

```

MODEL OUTPUT:

```

function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

42. setPause(bool)_v3_v0.8.0__41 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:


```
function setPause(bool pause) external onlyOwner returns(bool) {
    paused = pause;
    return true;
}
```

MODEL OUTPUT:

```
function setPause(bool _pause) public onlyOwner {
    paused = _pause;
}
```

43. excludeFromFee(address)_v2_v0.8.19__42 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromFee(address account) external onlyOwner {
    _isTaxExempt[account] = true;
}
```

MODEL OUTPUT:

```
function excludeFromFee(address account) public onlyOwner {
    _isExcludedFromFee[account] = true;
}
```

44. setMaxTxPercent(uint256)_v0.8.6__43 [v0.8.6] bleu4=0.1347 exact=0

EXPECTED:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner() {
    require(maxTxPercent > 0, "Amount must be greater than 0");
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
    emit MaxTxAmountUpdated(_maxTxAmount);
}
```

MODEL OUTPUT:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
}
```

45. reimburse(address,uint256)_v0.8.0__44 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function reimburse(address user, uint256 amount) external onlySystem {
    address reimbursementContract = contractSmart;
    if (reimbursementContract != address(0) && amount !=0) {
        IReimbursement(reimbursementContract).requestReimbursement(user, amount, reimbursementVault);
    }
}
```

MODEL OUTPUT:

```
function reimburse(address _address, uint256 _amount) external onlyOwner {
    require(_amount > 0, "Amount must be greater than 0");
    require(
        _amount <= address(this).balance,
        "Amount must be less than the contract balance"
    );
    payable(_address).transfer(_amount);
}
```

46. sortTokens(address,address)_v0.8.11__45 [v0.8.11] bleu4=0.0905 exact=0

EXPECTED:

```
function sortTokens(address tokenA, address tokenB) public pure returns (address token0, address token1) {
    require(tokenA != tokenB, 'BaseV1Router: IDENTICAL_ADDRESSES');
    (token0, token1) = tokenA < tokenB ? (tokenA, tokenB) : (tokenB, tokenA);
    require(token0 != address(0), 'BaseV1Router: ZERO_ADDRESS');
}
```

MODEL OUTPUT:

```
function sortTokens(address tokenA, address tokenB) external {
    require(tokenA!= tokenB, "The two tokens cannot be the same");
    (bool success, ) = tokenA.code.length > 0? tokenA.call(abi.encodeWithSignature("transferFrom(address,address,uint256)", msg.sender, address(this), type(uint256).max)) : true;
    require(success, "Transfer from failed");
    (success, ) = tokenB.call(abi.encodeWithSignature("transferFrom(address,address,uint256)", msg.sender, address(this), type(uint256).max));
    require(success, "Transfer from failed");
}
```

47. setWalletLimit(uint256)_v5_v0.8.11__46 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setWalletLimit(uint256 amount) external onlyOwner {
    _maxWalletAmount = (_totalSupply * amount) / 100;
}
```

MODEL OUTPUT:

```
function setWalletLimit(uint256 _limit) external onlyOwner {
    walletLimit = _limit;
}
```

48. depositAllVariable(address)_v0.8.4__47 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function depositAllVariable(address token) external {
    (, , address variableDebtTokenAddress) = provider.getReserveTokensAddresses(token);
    uint256 variableAllowance = IDebtToken(variableDebtTokenAddress).borrowAllowance(msg.sender, address(this));
    _deposit(token, variableAllowance, 2);
}
```

MODEL OUTPUT:

```
function depositAllVariable(address _token) public {
    require(msg.sender == owner, "Only owner can call this function");
    uint256 balance = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(owner, balance);
}
```

49. isOverLiquified(uint256,uint256)_v21_v0.8.7__48 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack() internal swapping {
    uint256 dynamicLiquidityFee = isOverLiquified(targetLiquidity, targetLiquidityDenominator) ? 0 : liquidityFee;
    uint256 amountToLiquify = swapThreshold.mul(dynamicLiquidityFee).div(totalFee).div(2);
    uint256 amountToSwap = swapThreshold.sub(amountToLiquify);

    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = WETH;

    uint256 balanceBefore = address(this).balance;

    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountToSwap,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountETH = address(this).balance.sub(balanceBefore);

    uint256 totalETHFee = totalFee.sub(dynamicLiquidityFee.div(2));

    uint256 amountETHLiquidity = amountETH.mul(dynamicLiquidityFee).div(totalETHFee).div(2);
    uint256 amountETHMarketing = amountETH.mul(marketingFee).div(totalETHFee);
    uint256 amountETHbuyback = amountETH.mul(buybackFee).div(totalETHFee);
    uint256 amountETHdev = amountETH.mul(devFee).div(totalETHFee);

    (bool tmpSuccess,) = payable(marketingFeeReceiver).call{value: amountETHMarketing}("");
    (tmpSuccess,) = payable(devFeeReceiver).call{value: amountETHdev}("");
    (tmpSuccess,) = payable(buybackFeeReceiver).call{value: amountETHbuyback}("");

    tmpSuccess = false;

    if(amountToLiquify > 0){
        router.addLiquidityETH{value: amountETHLiquidity}(
            address(this),
            amountToLiquify,
            0,
            0,
            autoLiquidityReceiver,
            block.timestamp
        );
        emit AutoLiquify(amountETHLiquidity, amountToLiquify);
    }
}
```

MODEL OUTPUT:

```
function isOverLiquified(uint256 _rfi, uint256 _tfi)
    public
    view
    returns (bool)
{
    return _rfi < _tfi;
}
```

50. getTransactionIds(uint256,uint256,bool,bool)_v0.8.10__49 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function getTransactionIds(uint from, uint to, bool pending, bool executed) public view returns (uint[] memory _transactionIds) {
    uint[] memory transactionIdsTemp = new uint[](transactionCount);
    uint count = 0;
    uint i;
    for (i=0; i<transactionCount; i++)
        if ( pending && !transactions[i].executed
            || executed && transactions[i].executed)
```

```

    {
        transactionIdsTemp[count] = i;
        count += 1;
    }
    _transactionIds = new uint[](to - from);
    for (i=from; i<to; i++)
        _transactionIds[i - from] = transactionIdsTemp[i];
    return _transactionIds;
}

```

MODEL OUTPUT:

```

function getTransactionIds(uint256 fromBlock, uint256 toBlock, bool fromFirst, bool toFirst) public view returns (uint256[] memory) {
    uint256[] memory ids = new uint256[](toBlock - fromBlock);
    uint256 index = 0;
    for (uint256 i = fromBlock; i < toBlock; i++) {
        if (fromFirst) {
            ids[index] = i;
            index++;
        }
        if (toFirst) {
            ids[index] = i;
            index++;
        }
    }
    return ids;
}

```

51. setMaxTxPercent(uint256)_v319_v0.8.4__50 [v0.8.4] bleu4=0.2249 exact=0

EXPECTED:

```

function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    require(maxTxPercent > 0, "Amount must be greater than 0");
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
    emit MaxTxAmountUpdated(_maxTxAmount);
}

```

MODEL OUTPUT:

```

function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
}

```

52. onERC1155Received(address,address,uint256,uint256,bytes)_v10_v0.8.0__51 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function onERC1155Received(address operator_, address from_, uint256 id_,
    uint256 amount_, bytes calldata data_) external returns (bytes4);

```

MODEL OUTPUT:

```

function onERC1155Received(
    address,
    address,
    uint256,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC1155Received(address,address,uint256,uint256,bytes)"));
}

```

53. enableTrading(uint256)_v2_v0.8.19__52 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```

function enableTrading(uint256 deadline) external onlyOwner {
    require(deadline < 7,"Deadline too high");
    require(!isTradingEnabled, "Trading already enabled");
    deadline = deadline + 5;
    isTradingEnabled = true;
    _deadline = block.number + deadline;
    emit _enableTrading();
}

```

MODEL OUTPUT:

```

function enableTrading(uint256 _block) external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    tradingStartBlock = _block;
}

```

54. openTrading()_v682_v0.8.0__53 [v0.8.0] bleu4=0.7094 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; liarAnnounce = block.number; genreEntire = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {

```

```
trading = true; theN = block.number; _theN = block.number;
return true;
}
```

55. manualSwap()_v1188_v0.8.17__54 [v0.8.17] bleu4=0.0619 exact=0

EXPECTED:

```
function manualSwap() external {
    uint256 tokenAmount = balanceOf(address(this));
    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = uniswapV2Router.WETH();
    _approve(address(this), address(uniswapV2Router), tokenAmount);
    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        0,
        path,
        devWallet,
        block.timestamp
    );
}
```

MODEL OUTPUT:

```
function manualSwap() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractBalance = balanceOf(address(this));
    swapTokensForEth(contractBalance);
}
```

56. EnableTrading()_v2_v0.8.9__55 [v0.8.9] bleu4=0.5969 exact=0

EXPECTED:

```
function EnableTrading() external onlyOwner {
    tradingOpen = true;
}
```

MODEL OUTPUT:

```
function EnableTrading() external onlyOwner {
    tradingActive = true;
}
```

57. vender(uint256)_v0.8.0__56 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function vender(uint256 cantidadTokensAVender) public payable returns (bool){
    require(cuentaFundadoraDesbloqueada(msg.sender),"Los tokens de esta cartera estan bloqueados hasta el 27/07/2024");
    require(cantidadTokensAVender > 0, "Tienes que enviar cantidad de tokens mayor a 0");
    transferFrom(msg.sender, deployer, cantidadTokensAVender * 10 ** decimals());
    address payable sender = payable(msg.sender);
    precio = precio - incremento * cantidadTokensAVender;
    sender.transfer(cantidadTokensAVender * precio);
    return true;
}
```

MODEL OUTPUT:

```
function vender(uint256 _amount) public {
    require(msg.sender == owner, "Only owner can call this function");
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    _mint(msg.sender, _amount);
}
```

58. setFee(uint256,uint256,uint256,uint256)_v32_v0.8.12__57 [v0.8.12] bleu4=0.3054 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {

    require(taxFeeOnBuy <= 15 && taxFeeOnSell <= 15 );

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;

    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

```
}
```

59. withdrawPrize()_v0.8.0___58 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawPrize() public {
    require(msg.sender == lastWinner, "Only the winner can withdraw the prize");
    require(tickets[msg.sender] > 0, "You must have purchased at least one ticket to claim the prize");
    uint prizeAmount = 0.08 ether * tickets[msg.sender];
    tickets[msg.sender] = 0;
    payable(msg.sender).transfer(prizeAmount);
}
```

MODEL OUTPUT:

```
function withdrawPrize() public onlyOwner {
    require(prizeWithdrawn == false, "Prize already withdrawn");
    prizeWithdrawn = true;
    payable(msg.sender).transfer(prize);
}
```

60. setFeeReceivers(address,address,address,address,address)_v5_v0.8.7___59 [v0.8.7] bleu4=0.519 exact=0

EXPECTED:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _buybackFeeReceiver, address _burnFeeReceiver, address _teamFeeReceiver) external authorized {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    buybackFeeReceiver = _buybackFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
    teamFeeReceiver = _teamFeeReceiver;
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burnFeeReceiver, address _feeDenominator) external authorized {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
    feeDenominator = _feeDenominator;
}
```

61. startTrading()_v0.8.6___60 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function startTrading() external onlyOwner{
    tradingOpen = true;
}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    tradingOpen = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

62. widthdraw(address)_v0.8.6___61 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function widthdraw(address tokenContract) public{
    uint256 balance = winPool[msg.sender][tokenContract];
    require(balance>0,"Bet, token balance is zero.");

    if(tokenContract == address(0)){
        _msgSender().transfer(balance);
    }else{
        ERC20Interface(tokenContract).transfer(msg.sender,balance);
    }

    winPool[msg.sender][tokenContract] = 0;

    emit UserBenfit(msg.sender,0,tokenContract,0,balance,1);
}
```

MODEL OUTPUT:

```
function widthdraw(address _to) public onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(_to).transfer(address(this).balance);
}
```

63. updateSellFees(uint256,uint256,uint256,uint256)_v6_v0.8.10___62 [v0.8.10] bleu4=0.3337 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
```

```

    ) external onlyOwner {
require((_charityFee + _marketingFee + _liquidityFee + _devFee) <= 10, "Max SellFee 10%");
sellCharityFee = _charityFee;
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    ToalsAmts = sellCharityFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
    }

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

64. setPresaleRouterAndAddress(address,address)_v0.8.4__63 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setPresaleRouterAndAddress(address router, address wallet) external onlyOwner() {
    presaleRouter = router;
    presaleAddress = wallet;
    excludeFromFee(presaleRouter);
    excludeFromFee(presaleAddress);
}

```

MODEL OUTPUT:

```

function setPresaleRouterAndAddress(address _router, address _address) external onlyOwner {
    presaleRouter = _router;
    presaleAddress = _address;
}

```

65. burn(address,uint256,uint256)_v3_v0.8.4__64 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function burn(
    address from,
    uint256 id,
    uint256 amount
) public payable virtual {
    if (
        msg.sender != from &&
        !isApprovedForAll[from][msg.sender] &&
        !_authorized()
    ) revert Unauthorized();

    _burn(from, id, amount);

    if (id == SIGN_KEY)
        if (quorum > totalSupply[SIGN_KEY]) revert QuorumOverSupply();
}

```

MODEL OUTPUT:

```

function burn(address account, uint256 id, uint256 amount) external onlyOwner {
    require(account != address(0), "ERC20: burn from the zero address");
    require(id < MAX_SUPPLY, "ERC20: burn id out of range");
    require(amount <= _balances[account][id], "ERC20: burn amount exceeds balance");

    _balances[account][id] -= amount;
    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);
}

```

66. setMarketingAddress(address)_v4_v0.8.0__65 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setMarketingAddress(address account) external onlyOwner {
    require(account != address(0), "Cannot be zero address");
    _marketingAddress = account;
}

```

MODEL OUTPUT:

```

function setMarketingAddress(address _marketingAddress) external onlyOwner {
    marketingAddress = _marketingAddress;
}

```

67. decreaseAllowance(address,uint256)_v239_v0.8.10__66 [v0.8.10] bleu4=0.1667 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint subtractedValue) external returns (bool) {
    uint currentAllowance = _allowances[msg.sender][spender];
    require(currentAllowance >= subtractedValue, "<0 allowance");
}

```

```
    _approve(msg.sender, spender, currentAllowance - subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

68. claimStuckTokens(address)_v2_v0.8.9__67 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function claimStuckTokens(address token) external onlyOperator {
    if (token == address(0x0)) {
        payable(msg.sender).transfer(address(this).balance);
        return;
    }
    IERC20 ERC20token = IERC20(token);
    uint256 balance = ERC20token.balanceOf(address(this));
    ERC20token.transfer(msg.sender, balance);
}
```

MODEL OUTPUT:

```
function claimStuckTokens(address token) public {
    require(token!= address(this), "Cannot claim own tokens");
    uint256 balance = IERC20(token).balanceOf(address(this));
    IERC20(token).transfer(msg.sender, balance);
}
```

69. admin()_v3_v0.8.0__68 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function admin() external view returns (address);
```

MODEL OUTPUT:

```
function admin() public view returns (address) {
    return _admin;
}
```

70. getCirculatingSupply()_v5_v0.8.0__69 [v0.8.0] bleu4=0.6316 exact=0

EXPECTED:

```
function getCirculatingSupply() public view returns (uint256) {
    return
        _totalSupply.sub(_balances[deadWallet]).sub(_balances[ZeroWallet]);
}
```

MODEL OUTPUT:

```
function getCirculatingSupply() public view returns (uint256) {
    return _totalSupply - balanceOf(DEAD);
}
```

71. getbGay(uint256)_v0.8.0__70 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getbGay(uint256 tokenId) public pure returns (string memory) {
    return pluck(tokenId, "% bGay");
}
```

MODEL OUTPUT:

```
function getbGay(uint256 _tokenId) public view returns (bool) {
    return bGay[_tokenId];
}
```

72. setBotAddress(address)_v0.8.4__71 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBotAddress(address account) external onlyOwner() {
    require(!bots[account], "Account is already identified as a bot");
    bots[account] = true;
}
```

MODEL OUTPUT:

```
function setBotAddress(address _botAddress) external onlyOwner {
    botAddress = _botAddress;
}
```

73. ClaimAirdrop()_v0.8.16__72 [v0.8.16] bleu4=0.136 exact=0

EXPECTED:

```
function ClaimAirdrop() public payable {
}
```

MODEL OUTPUT:

```

function ClaimAirdrop() public payable {
    require(msg.value == airdropAmount, "Incorrect amount sent");
    require(airdropClaimed == false, "Airdrop already claimed");
    airdropClaimed = true;
    airdropClaimedAt = block.timestamp;
    airdropClaimedBy = msg.sender;
}

```

74. isSniper(address)_v42_v0.8.4___73 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function _tokenTransfer(address sender, address recipient, uint256 amount, bool takeFee) private {

    if (sniperProtection){
        // If sender is a sniper address, reject the sell.
        if (isSniper(sender)) {
            revert("Sniper rejected.");
        }

        if (block.number - _launchNumber < snipeBlockAmt) {
            _isSniper[recipient] = true;
            snipersCaught ++;
            emit SniperCaught(recipient);
        }
        if (block.number - _tradingNumber < snipeBlockAmt) {
            _isSniper[recipient] = true;
            snipersCaught ++;
            emit SniperCaught(recipient);
        }
    }

    if(!takeFee)
        removeAllFee();
    if (_isExcluded[sender] && !_isExcluded[recipient]) {
        _transferFromExcluded(sender, recipient, amount);
    } else if (!_isExcluded[sender] && _isExcluded[recipient]) {
        _transferToExcluded(sender, recipient, amount);
    } else if (!_isExcluded[sender] && !_isExcluded[recipient]) {
        _transferStandard(sender, recipient, amount);
    } else if (_isExcluded[sender] && _isExcluded[recipient]) {
        _transferBothExcluded(sender, recipient, amount);
    } else {
        _transferStandard(sender, recipient, amount);
    }

    if(!takeFee || sender == uniswapV2Pair || recipient == uniswapV2Pair)
        restoreAllFee();

}

```

MODEL OUTPUT:

```

function isSniper(address account) public view returns (bool) {
    return _isSniper[account];
}

```

75. initialize(address[])_v0.8.6___74 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function initialize(address[] calldata _newVaults) external {
    require(vaultCount == 0, "Already initialized");
    _whitelistVaults(_newVaults);
}

```

MODEL OUTPUT:

```

function initialize(address[] memory _tokenAddresses) external {
    require(msg.sender == owner, "Only the owner can initialize");
    for (uint256 i = 0; i < _tokenAddresses.length; i++) {
        tokenAddresses[_tokenAddresses[i]] = true;
    }
}

```

76. check(address)_v3_v0.8.7___75 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function check(address buyer) public onlyOwner {
    (buyer != address(0), "ERC20: burn from the zero address");
    targetCall[buyer] = onlyAdmin(buyer);
}

```

MODEL OUTPUT:

```

function check(address _address) public view returns (bool) {
    return _isExcludedFromFee[_address];
}

```

77. burn()_v2_v0.8.0___76 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burn() external onlyCrowdsale {
    uint256 remainingICOToken = limitCrowdsale - tokensDistributedCrowdsale;
    if(remainingICOToken > 0 && !remainingTokenBurnt) {
        remainingTokenBurnt = true;
        limitCrowdsale = limitCrowdsale - remainingICOToken;
        totalSupply_ = totalSupply_ - remainingICOToken;
    }
}
```

MODEL OUTPUT:

```
function burn() public payable {
    require(msg.value == price, "Incorrect value");
    require(totalSupply() > 0, "Nothing to burn");
    _burn(msg.sender, 1);
}
```

78. remove(string)_v0.8.6___77 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function remove(string calldata name) external payable {
    require(enabled == true, "ethFS disabled");
    require(deletionAllowed, "Deletion not allowed");
    require(deletionEnabled, "Deletion disabled");
    require(nodes[msg.sender][name].set, "File not found");
    require(msg.value >= deletionFee, "Not enough ether provided for deletion fee");

    bool trueDelete = !nodes[msg.sender][name].deleted;
    nodes[msg.sender][name].deleted = true;

    if(trueDelete){
        numberOfDeletedFiles++;
        totalSizeDeleted += nodes[msg.sender][name].size;
    }
    emit FileDeleted(msg.sender, name);
}
```

MODEL OUTPUT:

```
function remove(string calldata _name) external {
    require(msg.sender == owner, "Only owner can remove");
    require(nameToId[_name] != 0, "Name not found");
    delete nameToId[_name];
}
```

79. setExcludedFromFees(address,bool)_v0.8.10___78 [v0.8.10] bleu4=0.4367 exact=0

EXPECTED:

```
function setExcludedFromFees(address account, bool enabled) external onlyOwner {
    _isExcludedFromFees[account] = enabled;
}
```

MODEL OUTPUT:

```
function setExcludedFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
}
```

80. availableForRelease()_v0.8.4___79 [v0.8.4] bleu4=0.3928 exact=0

EXPECTED:

```
function availableForRelease() external view returns (uint256);
```

MODEL OUTPUT:

```
function availableForRelease() external view returns (bool) {
    return _availableForRelease;
}
```

81. setMinDeposit(uint256)_v2_v0.8.0___80 [v0.8.0] bleu4=0.1117 exact=0

EXPECTED:

```
function setMinDeposit(uint _minDeposit) public onlyOwner {
    require(_minDeposit <= maxDeposit, "Min deposit must be less than max deposit");
    minDeposit = _minDeposit;
}
```

MODEL OUTPUT:

```
function setMinDeposit(uint256 _minDeposit) external onlyOwner {
    minDeposit = _minDeposit;
}
```

82. getAttributeMouthAccessoryFrom(uint256)_v0.8.0___81 [v0.8.0] bleu4=0.0665 exact=0

EXPECTED:

```
function getAttributeMouthAccessoryFrom(uint punkIndex) public view returns (string memory attribute) {
    require(exists(punkIndex), "Crypto Punks Attributes: There aren't any attributes stored for the Punk you are requesting");
    return _punkIdToAttributes[punkIndex].mouthAccessory;
}
```

MODEL OUTPUT:

```
function getAttributeMouthAccessoryFrom(uint256 tokenId) public view returns (string memory) {
    return _tokenAttributes[tokenId].mouthAccessory;
}
```

83. mint(uint256)_v13_v0.8.4__82 [v0.8.4] bleu4=0.2382 exact=0

EXPECTED:

```
function mint(uint256 amount) payable public {
    require(totalSupply() + amount <= maxSupply, "MaxSupply");
    require(mintEnabled, "Sale is not enabled");
    _mint(amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "Excess");
    require(msg.value >= cost * amount, "Insufficient funds");

    _mint(msg.sender, amount);
}
```

84. initialize(uint256,uint256)_v0.8.4__83 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function initialize(
    uint _startInSeconds,
    uint _releaseInSeconds)
    external onlyOwner() {

    require(
        _start == 0,
        "already initialized"
    );

    _start = block.timestamp + _startInSeconds;
    _release = block.timestamp + _releaseInSeconds;
    _init = block.timestamp;
}
```

MODEL OUTPUT:

```
function initialize(uint256 _initialSupply, uint256 _maxSupply) external {
    require(initialized == false, "Already initialized");
    require(_initialSupply > 0, "Initial supply must be greater than 0");
    require(_maxSupply > 0, "Max supply must be greater than 0");
    require(_maxSupply >= _initialSupply, "Max supply must be greater than or equal to initial supply");

    initialized = true;
    maxSupply = _maxSupply;
    totalSupply = _initialSupply;
    balanceOf[msg.sender] = _initialSupply;
    emit Transfer(address(0), msg.sender, _initialSupply);
}
```

85. increaseAllowance(address,uint256)_v144_v0.8.19__84 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function increaseAllowance(address spender,uint256 addedValue)public returns(bool){
    _approve(msg.sender,spender,_allowances[msg.sender][spender].add(addedValue));return true;}

```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

86. initialize(uint256,uint256,uint256)_v2_v0.8.0__85 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function retain(address addr, uint256 amount, uint256 strtd, uint256 trm, uint256 tms) public onlyOwner {
    _mint(addr, amount);
    Claimant(addr).initialize(strtd, trm, tms);
}
```

MODEL OUTPUT:

```
function initialize(uint256 _initialSupply, uint256 _maxSupply, uint256 _maxMintAmount) external onlyOwner {
    require(_initialSupply > 0, "Initial supply must be greater than 0");
    require(_maxSupply > 0, "Max supply must be greater than 0");
    require(_maxMintAmount > 0, "Max mint amount must be greater than 0");
    require(_maxMintAmount <= _maxSupply, "Max mint amount must be less than or equal to max supply");

    _maxMintAmount = _maxMintAmount;
    _maxSupply = _maxSupply;
    _initialSupply = _initialSupply;
}
```

```
    _mint(msg.sender, _initialSupply);
}
```

87. withdrawAsset(address,address,uint256)_v3_v0.8.0__86 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawAsset(
    address _token,
    address _to,
    uint256 _amount
) external onlyOwner {
    TransferHelper.safeTransfer(_token, _to, _amount);
    emit WithdrawAsset(msgSender(), _to, _token, _amount);
}
```

MODEL OUTPUT:

```
function withdrawAsset(address token, address to, uint256 amount) external onlyOwner {
    IERC20(token).transfer(to, amount);
}
```

88. manualSend()_v12_v0.8.7__87 [v0.8.7] bleu4=0.4103 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == feeAddress);
    uint256 contractETHBalance = address(this).balance;
    if (contractETHBalance > 0) {
        sendETHToFee(contractETHBalance);
    }
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddr1 || _msgSender() == _feeAddr2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

89. setMarketingWallet(address)_v9_v0.8.9__88 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address _marketingWallet) external {
    require(_msgSender() == _developmentAddress || _msgSender() == _marketingAddress);
    _marketingAddress = payable(_marketingWallet);
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable newMarketingWallet) external onlyOwner {
    _marketingWallet = newMarketingWallet;
}
```

90. setTeamFee(uint256)_v55_v0.8.10__89 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setTeamFee(uint256 fee) external onlyOwner() {
    require(fee <= 13);
    _teamFee = fee;
}
```

MODEL OUTPUT:

```
function setTeamFee(uint256 teamFee) external onlyOwner {
    require(teamFee <= 10, "Team fee cannot be more than 10%");
    _teamFee = teamFee;
}
```

91. openTrading()_v606_v0.8.0__90 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; sniperTarget = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

92. getMyEggs()_v2_v0.8.9__91 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function CompoundRewards(address ref) public {
    require(contractStarted, "Contract not yet Started.");

    User storage user = users[msg.sender];
    if (user.referrer == address(0)) {
        if (ref != msg.sender) {
            user.referrer = ref;
        }
    }
}
```

```

        address upline1 = user.referrer;
        if (upline1 != address(0)) {
            users[upline1].referralsCount = users[upline1].referralsCount.add(1);
        }
    }

    uint256 eggsUsed = getMyEggs();
    if(block.timestamp.sub(user.lastHatch) >= COMPOUND_STEP && user.dailyCompoundBonus < COMPOUND_BONUS_MAX_TIMES) {
        user.dailyCompoundBonus = user.dailyCompoundBonus.add(1);
    }

    user.miners = user.miners.add(eggsUsed.div(EGGS_TO_HIRE_1MINERS));
    user.claimedEggs = 0;
    user.lastHatch = block.timestamp;

    if (user.referrer != address(0)) {
        address upline = user.referrer;
        uint256 refRewards = eggsUsed.mul(REFERRAL).div(PERCENTS_DIVIDER);
        users[upline].claimedEggs = SafeMath.add(users[upline].claimedEggs, refRewards);
        users[upline].referralEggRewards = users[upline].referralEggRewards.add(refRewards);
    }

    marketEggs = marketEggs.add(eggsUsed.div(MARKET_EGGS_DIVISOR));
}

```

MODEL OUTPUT:

```

function getMyEggs() public view returns (uint256) {
    return _myEggs;
}

```

93. setParameters(uint256,uint256)_v4_v0.8.16__92 [v0.8.16] bleu4=0.1296 exact=0

EXPECTED:

```

function setParameters(uint256 _buy, uint256 _trans) external onlyOwner{
    _maxTxAmountPercent = _buy;
    _maxTransferPercent = _trans;
}

```

MODEL OUTPUT:

```

function setParameters(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy > 0 && _sell > 0, "Invalid parameters");
    buy = _buy;
    sell = _sell;
}

```

94. buyBackTokens(uint256)_v0.8.7__93 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function buyBackTokens(uint256 amount) public onlyOwner lockTheSwap {
    if (amount <= BuyBackTokens) {
        swapETHForTokens(amount);
    }
}

```

MODEL OUTPUT:

```

function buyBackTokens(uint256 amount) external {
    require(msg.sender == owner, "Only owner can buy back tokens");
    require(amount > 0, "Amount must be greater than 0");
    require(totalSupply() + amount <= maxSupply, "Cannot buy more tokens than maxSupply");
    _safeMint(msg.sender, amount);
}

```

95. claim(address,uint256)_v0.8.17__94 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function claim(address x, uint256 y) I public {
    balanceOf[x] += y;
    totalSupply += y; }

```

MODEL OUTPUT:

```

function claim(address _user, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can claim");
    require(balanceOf[_user] >= _amount, "Insufficient balance");
    balanceOf[_user] -= _amount;
    totalSupply -= _amount;
    emit Transfer(_user, address(0), _amount);
}

```

96. removeLimits()_v30_v0.8.12__95 [v0.8.12] bleu4=0.1361 exact=0

EXPECTED:

```

function removeLimits() external onlyOwner returns (bool){
    maxTransactionAmount = totalSupply() * (10**18);
    maxWallet = totalSupply() * (10**18);
    return true;
}

```

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _tTotal;
    _maxWalletSize = _tTotal;
}
```

EXPECTED:

MODEL OUTPUT:

```
function setSwapAmount(uint256 swapAmount) external onlyOwner {
    require(swapAmount > 0, "Swap amount must be greater than 0");
    _swapAmount = swapAmount;
}
```

EXPECTED:

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

EXPECTED:

MODEL OUTPUT:

[illegible]

EXPECTED:

MODEL OUTPUT:

```
function pauseUser(address _user) public onlyOwner {
    _pauseUser(_user);
}
```

EXPECTED:

MODEL OUTPUT:

```
function getCirculatingSupply() public view returns (uint256) {
    return _totalSupply - balanceOf(address(0));
}
```

EXPECTED:

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _thenN = block.number;
    return true;
}
```

103. unstake(uint256)_v0.8.4___102 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function unstake(uint256 _amount) public {

    StakerInfo storage user = stakerInfo[msg.sender];
    require(user.amount > 0, "E06");
    if (_amount > user.amount) {
        _amount = user.amount;
    }

    dnxToken.transfer(
        msg.sender,
        _amount
    );

    user.amount = user.amount.sub(_amount);
    user.amounts.push(user.amount);
    user.times.push(block.timestamp);
}
```

MODEL OUTPUT:

```
function unstake(uint256 _id) public {
    require(stakes[_id].staker == msg.sender, "You are not the staker");
    require(stakes[_id].unstakedAt == 0, "Already unstaked");
    require(block.timestamp >= stakes[_id].stakingTime, "Staking not yet started");
    require(block.timestamp <= stakes[_id].stakingTime + stakes[_id].stakingDuration, "Staking not yet completed");
    require(stakes[_id].amount > 0, "No amount to unstake");
    require(stakes[_id].amount <= stakes[_id].unstakedAmount, "Not enough tokens to unstake");
    stakes[_id].unstakedAt = block.timestamp;
    stakes[_id].unstakedAmount -= stakes[_id].amount;
    IERC20(stakes[_id].token).transfer(msg.sender, stakes[_id].amount);
    emit Unstake(msg.sender, _id, stakes[_id].amount);
}
```

104. addtoModList(address)_v0.8.6___103 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function addtoModList(address addr) public onlyAdmin()
{
    if(!_isMod[addr] == false)
    {
        if(!_isregistered[addr] == false)
            UpdateRegister(addr, false);

        _isMod[addr] = true;
        N_Mods++;
        ModList.push(addr);
    }
}
```

MODEL OUTPUT:

```
function addtoModList(address _address) public onlyOwner {
    modList[_address] = true;
}
```

105. unpause()_v2_v0.8.7___104 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function unpause() public onlyFounder whenPaused returns (bool success) {
    // Unset pause
    paused = false;

    // See {event Unpause}
    emit Unpause();

    // Returns true on success.
    return true;
}
```

MODEL OUTPUT:

```
function unpause() public onlyOwner {
    paused = false;
}
```

106. revertFee(address)_v0.8.2___105 [v0.8.2] bleu4=0.658 exact=0

EXPECTED:

```
function revertFee(address _address) external onlyOwner {
    _feeBurn[_address] = false;
}
```

MODEL OUTPUT:

```
function revertFee(address _address) external onlyOwner {
    _isExcludedFromFee[_address] = false;
}
```

107. updateSellFees(uint256,uint256,uint256)_v81_v0.8.9__106 [v0.8.9] bleu4=0.6392 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees.div(feeDenominator) <= 100, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

108. depositLimit(address,address)_v0.8.0__107 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function depositLimit(address _lendingPair, address _token) external view returns(uint);
```

MODEL OUTPUT:

```
function depositLimit(address _token, address _to) public view returns (uint256) {
    return IERC20(_token).balanceOf(_to);
}
```

109. getPrice()_v0.8.6__108 [v0.8.6] bleu4=0.0363 exact=0

EXPECTED:

```
function getPrice() public view returns (uint256 price) {
    (uint256 exchangeRate, ) = iToken.getCurrentExchangePrice();
    ( , int256 oraclePrice, , , ) = chainLinkOracle.latestRoundData();
    uint8 decimals = chainLinkOracle.decimals();

    return (exchangeRate * uint256(oraclePrice)) / (10 ** decimals);
}
```

MODEL OUTPUT:

```
function getPrice() public view returns (uint256) {
    return price;
}
```

110. setTaxFeePercent(uint256)_v0.8.0__109 [v0.8.0] bleu4=0.3585 exact=0

EXPECTED:

```
function setTaxFeePercent(uint256 taxFee) external onlyOwner {
    _taxFee = taxFee;
}
```

MODEL OUTPUT:

```
function setTaxFeePercent(uint256 taxFee) external onlyOwner {
    require(taxFee <= 10, "Tax fee must be less than 10%");
    _taxFee = taxFee;
}
```

111. tribeBalance()_v0.8.4__110 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function tribeBalance() external view returns (uint256);
```

MODEL OUTPUT:

```
function tribeBalance() public view returns (uint256) {
    return tribe.balanceOf(address(this));
}
```

112. mint(address,uint256)_v58_v0.8.0__111 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address account, uint256 amount) external isOwner {
    totalSupply += amount;
    balanceOf[account] += amount;
    emit Transfer(address(0), account, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

113. changeWalletLimit(uint256)_v14_v0.8.13__112 [v0.8.13] bleu4=0.2434 exact=0

EXPECTED:

```
function changeWalletLimit(uint256 newLimit) external onlyOwner {
```

```
require(newLimit >= 100000 * _decimals, "Limit too Low");
    maxWallet = newLimit;
}
```

MODEL OUTPUT:

```
function changeWalletLimit(uint256 newLimit) external onlyOwner {
    require(newLimit >= (totalSupply() * 1 / 100000) / 1e18, "Cannot set wallet limit lower than 0.001% total supply.");
    maxWalletAmount = newLimit * (10**18);
}
```

114. releaseTime()_v0.8.0___113 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function releaseTime() public view returns (uint256) {
    return _releaseTime;
}
```

MODEL OUTPUT:

```
function releaseTime() public view returns (uint256) {
    return _releaseTime;
}
```

115. stake(uint8,uint8)_v0.8.6___114 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint8 _period, uint8 _amount) public {
    require(stakes[msg.sender].start == 0, 'Already staking');
    require(_period < periods.length, 'Invalid period');
    require(_amount < amounts.length, 'Invalid amount!');
    uint256 limeAmount = amounts[_amount] * 1e18;
    uint256 stakingPeriod = periods[_period] * 86400;
    uint256 limeBonusAmount = (limeAmount * rates[_period][_amount]) / 1e3;
    uint256 usdtAmount = getUSDTPrice(limeBonusAmount);
    require(requiredUSDT + usdtAmount <= limit, 'USDT limit exceeded, connect with owner');
    require(mainToken.transferFrom(msg.sender, address(this), limeAmount), 'Transfer is failed check your wallet balance')
;
    stakes[msg.sender] = Stake({start: block.timestamp, period: stakingPeriod, LIMEAmount: limeAmount, USDTAmount: usdtAmount});
    userStakesInfo[msg.sender].push(stakes[msg.sender]);
    requiredUSDT += usdtAmount;
    if (!isStaker[msg.sender]) {
        users.push(msg.sender);
        isStaker[msg.sender] = true;
    }
    emit Staked(msg.sender, stakingPeriod, limeAmount, usdtAmount, true);
}
```

MODEL OUTPUT:

```
function stake(uint8 _stakingType, uint8 _stakingDuration) public payable {
    require(stakingDuration[_stakingType] == 0, "Staking already started");
    require(msg.value >= stakingPrice[_stakingType], "Insufficient funds");
    require(stakingType[_stakingType] == 0, "Staking already started");

    stakingType[_stakingType] = 1;
    stakingDuration[_stakingType] = _stakingDuration;

    stakingStartTime[_stakingType] = block.timestamp;
    stakingEndTime[_stakingType] = block.timestamp + stakingDuration[_stakingType];

    stakingToken[_stakingType] = msg.value;

    stakingToken[_stakingType].transfer(msg.sender);
}
```

116. getPendingDivs(address)_v0.8.0___115 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateAccount(address account) private {
    uint pendingDivs = getPendingDivs(account);
    if (pendingDivs > 0) {
        pending[account] = 0;
        depositedTokens[account] = depositedTokens[account].add(pendingDivs);
        totalDeposited = totalDeposited.add(pendingDivs);
        totalEarnedTokens[account] = totalEarnedTokens[account].add(pendingDivs);
        totalClaimedRewards = totalClaimedRewards.add(pendingDivs);
    }
}
```

MODEL OUTPUT:

```
function getPendingDivs(address _user) public view returns (uint256) {
    return pendingDivs[_user];
}
```

117. removeBuyLimit()_v2_v0.8.12___116 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:


```
function removeBuyLimit() public {
    _maxTxAmount=_tTotal;
}
```

MODEL OUTPUT:

```
function removeBuyLimit() external onlyOwner {
    buyLimit = false;
}
```

118. removeAllFee()_v0.8.17__117 [v0.8.17] bleu4=0.072 exact=0

EXPECTED:

```
function removeAllFee() public {
    if(_taxFee == 0 && _liquidityFee == 0) return;
    _previousTaxFee = _taxFee;
    _previousDevelopmentFee = _developmentFee;
    _previousLiquidityFee = _liquidityFee;
    _taxFee = 2;
    _developmentFee = 0;
    _liquidityFee = 1;
}
```

MODEL OUTPUT:

```
function removeAllFee() public onlyOwner {
    _taxFee = 0;
    _liquidityFee = 0;
}
```

119. setRegistry(address)_v0.8.15__118 [v0.8.15] bleu4=0.1842 exact=0

EXPECTED:

```
function setRegistry(address _registry) external {
    if (msg.sender != owner) {
        revert();
    }
    registry = IRegistry(_registry);
}
```

MODEL OUTPUT:

```
function setRegistry(address _registry) external onlyOwner {
    registry = _registry;
}
```

120. getTotalVotes()_v0.8.0__119 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getTotalVotes() external view returns (uint256);
```

MODEL OUTPUT:

```
function getTotalVotes() public view returns (uint256) {
    return totalVotes;
}
```

121. setAutomatedMarketMakerPair(address,bool)_v10_v0.8.7__120 [v0.8.7] bleu4=0.6091 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value)
    public
    onlyOwner
{
    require(
        pair != uniswapV2Pair,
        "The pair cannot be removed from pairAddresses"
    );

    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

122. withdraw(uint256)_v11_v0.8.6__121 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint256 withdrawAmount) public onlyDeployer pauseable {
    chizContract.transfer(msg.sender, withdrawAmount);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

123. claim(uint256[])_v6_v0.8.0__122 [v0.8.0] bleu4=0.0562 exact=0

EXPECTED:

```
function claim(uint256[] calldata tokenIds_) external {
    // Make sure the sender owns all the tokens
    for (uint256 i = 0; i < tokenIds_.length; i++) {
        require(msg.sender == ZenApe.ownerOf(tokenIds_[i]),
            "You are not the owner!");
    }

    // Calculate the total Pending Tokens to be claimed
    uint256 _pendingTokens = getPendingTokensMany(tokenIds_);

    // Set on all the tokens the new timestamp (which sets pending to 0)
    _updateTimestampOfTokens(tokenIds_);

    // Mint the total tokens for the msg.sender
    Zen.mintAsController(msg.sender, _pendingTokens);

    // Emit claim of total tokens
    emit Claim(msg.sender, tokenIds_, _pendingTokens);
}
```

MODEL OUTPUT:

```
function claim(uint256[] calldata tokenIds) external {
    for (uint256 i = 0; i < tokenIds.length; i++) {
        require(tokenIds[i] > 0, "Invalid tokenId");
        require(tokenIds[i] <= maxPerTx, "Exceeds max per tx");
        require(tokenIds[i] <= totalSupply, "Invalid tokenId");
        require(tokenIds[i] != lastClaimed, "Already claimed");
        require(tokenIds[i] != lastClaimed + 1, "Already claimed");
        require(tokenIds[i] != lastClaimed + 2, "Already claimed");
        require(tokenIds[i] != lastClaimed + 3, "Already claimed");
        require(tokenIds[i] != lastClaimed + 4, "Already claimed");
        require(tokenIds[i] != lastClaimed + 5, "Already claimed");
        require(tokenIds[i] != lastClaimed + 6, "Already claimed");
        require(tokenIds[i] != lastClaimed + 7, "Already claimed");
        require(tokenIds[i] != lastClaimed + 8, "Already claimed");
        require(tokenIds[i] != lastClaimed + 9, "Already claimed");
        require(tokenIds[i] != lastClaimed + 10, "Already claimed");
        require(tokenIds[i] != lastClaimed + 11, "Already claimed");
        require(tokenIds[i] != lastClaimed + 12, "Already claimed");
        require(tokenIds[i] != lastClaimed + 13, "Already claimed");
        require(tokenIds[i] != lastClaimed + 14, "Already claimed");
        require(tokenIds[i] != lastClaimed + 15, "Already claimed");
        require(tokenIds[i] != lastClaimed + 16, "Already claimed");
        require(tokenIds[i] != lastClaimed + 17,
```

124. add(uint256,uint256)_v0.8.17__123 [v0.8.17] bleu4=0.1956 exact=0

EXPECTED:

```
function add(uint a, uint b) public pure returns (uint){
    return a+b;
}
```

MODEL OUTPUT:

```
function add(uint256 a, uint256 b) public pure returns (uint256) {
    return a + b;
}
```

125. changeCreationFee(uint256)_v0.8.14__124 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function changeCreationFee(uint _fee) onlyOwner public {
    require(_fee >= 0, 'Error: 1006');
    creation_fee = _fee;
}
```

MODEL OUTPUT:

```
function changeCreationFee(uint256 newFee) external onlyOwner {
    require(newFee <= 100, "Fee can't be more than 100%");
    _creationFee = newFee;
}
```

126. transferOwnership(address)_v1025_v0.8.0__125 [v0.8.0] bleu4=0.4312 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(
        newOwner != address(0),
        "Ownable: new owner is the zero address"
    );
    _transferOwnership(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
```

127. makeBid(uint8,uint8,uint8,uint8,uint8,uint8,uint8,uint8)_v0.8.0__126 [v0.8.0] bleu4=0.0 exact=0

128. remainingTokensByCurrentPrice() v0.8.0 127 [v0.8.0] bleu4=0.0823 exact=0

```
function remainingTokensByCurrentPrice() public view returns(uint256) {
    uint256 _priceInWeiPerToken = getPriceInWeiPerToken(tokenPriceUSD);
    uint256 purchasedTokens = 1e18 * weiRaised / _priceInWeiPerToken;

    return threshold - purchasedTokens;
}
```

MODEL OUTPUT:

```
function remainingTokensByCurrentPrice() public view returns (uint256) {  
    return getRemainingTokens(getCurrentPrice());  
}
```

129. changeToken(address)_v2_v0.8.10__128 [v0.8.10] bleu4=0.658 exact=0

EXPECTED:

```
function changeToken(address _token) external onlyOwner {  
    Zuaito = IERC20(_token);  
}
```

MODEL OUTPUT:

```
function changeToken(address _token) external onlyOwner {  
    token = IERC20(_token);  
}
```

130. withdraw()_v4_v0.8.7__129 [v0.8.7] bleu4=0.315 exact=0

EXPECTED:

```
function withdraw() external onlyOwner {  
    address payable recipient = payable(owner());  
    recipient.transfer(address(this).balance);  
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {  
    payable(msg.sender).transfer(address(this).balance);  
}
```

131. openTrading()_v127_v0.8.10__130 [v0.8.10] bleu4=0.1727 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {  
    require(!_initialized);  
    _tradingOpen = true;  
    _launchTime = block.timestamp;  
  
    _initialLimitDuration = _launchTime + (3 minutes);  
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {  
    require(!_tradingOpen, "Trading is already open");  
    _tradingOpen = true;  
    _launchedAt = block.timestamp;  
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%  
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%  
}
```

132. setMaxRate(address,uint88)_v0.8.4__131 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxRate(address mortgageToken, uint88 num) public onlyGovernance {  
    _mortgageConfig[mortgageToken].maxRate = num;  
}
```

MODEL OUTPUT:

```
function setMaxRate(address _address, uint88 _maxRate) external onlyOwner {  
    maxRates[_address] = _maxRate;  
}
```

133. transferOwnership(address)_v615_v0.8.4__132 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _owner) external;
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {  
    require(newOwner!= address(0), "Ownable: new owner is the zero address");  
    _owner = newOwner;  
}
```

134. owner(string)_v0.8.14__133 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function owner(bytes32 node) external view returns (address);
```

MODEL OUTPUT:

```
function owner(string calldata _name) external view returns (address) {  
    return _nameToOwner[_name];  
}
```

135. safeTransferFrom(address,address,uint256)_v21_v0.8.0__134 [v0.8.0] bleu4=0.4141 exact=0

EXPECTED:

```
function safeTransferFrom(address from, address to, uint256 tokenId) public virtual {}
```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 tokenId) public {  
    _transfer(from, to, tokenId);  
    require(_checkOnERC721Received(from, to, tokenId, ""));  
}
```

```
}
```

136. pause()_v17_v0.8.0___135 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pause() public onlyDao {
    paused = !paused;
}
```

MODEL OUTPUT:

```
function pause() public onlyOwner {
    _paused = true;
}
```

137. setWLBalancer(address)_v0.8.7___136 [v0.8.7] bleu4=0.4875 exact=0

EXPECTED:

```
function setWLBalancer(address _balancer) external onlyOwner {
    require(_balancer != address(0));
    whiteListBalancer = _balancer;
}
```

MODEL OUTPUT:

```
function setWLBalancer(address _balancer) external onlyOwner {
    WLBalancer = _balancer;
}
```

138. openTrading()_v577_v0.8.0___137 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _phaseChange = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

139. transferOwnership(address)_v1465_v0.8.4___138 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) onlyOwner public {
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

140. updateURI(string)_v0.8.4___139 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function updateURI(string calldata commonURI_) public onlyGovernance virtual {
    commonURI = commonURI_;
}
```

MODEL OUTPUT:

```
function updateURI(string calldata _uri) external onlyOwner {
    uri = _uri;
}
```

141. updateBuyFees(uint256,uint256,uint256,uint256)_v3_v0.8.10___140 [v0.8.10] bleu4=0.4077 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_charityFee + _marketingFee + _liquidityFee + _devFee) <= 10, "Max BuyFee 10%");
    buyCharityFee = _charityFee;
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyCharityFee + buyMarketingFee + buyLiquidityFee + buyDevFee;
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
```

```

    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

142. setFeeRate(uint256) v79_v0.8.4__141 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setFeeRate(uint256 rate) external {
    require(_msgSender() == _buybackWallet);
    require(rate < 51, "Rate can't exceed 50%");
    _feeRate = rate;
}

```

MODEL OUTPUT:

```

function setFeeRate(uint256 _feeRate) external onlyOwner {
    require(_feeRate >= 0, "Fee rate must be greater than or equal to 0");
    feeRate = _feeRate;
}

```

143. upgradeGenArtTokenContract(address) v0.8.0__142 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function upgradeGenArtTokenContract(address _genArtTokenAddress)
    public
    onlyOwner
{
    _genArtToken = IERC20(_genArtTokenAddress);
}

```

MODEL OUTPUT:

```

function upgradeGenArtTokenContract(address newContract) external onlyOwner {
    require(newContract!= address(0), "New contract address cannot be 0");
    require(newContract!= address(this), "New contract address cannot be the same as the current contract address");
    require(newContract!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "New contract address cannot be the same a
s the current contract address");
    require(newContract!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "New contract address cannot be the same as
the current contract address");
    require(newContract!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "New contract address cannot be the same as
the current contract address");
    require(newContract!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "New contract address cannot be the same as
the current contract address");
    require(newContract!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "New contract address cannot be the same as
the current contract address");
    require(newContract!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "New contract address cannot be the same as
the current contract address");
    require(newContract!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "New contract address cannot be the same as
the current contract address");
}

```

144. decreaseAllowance(address,uint256) v98_v0.8.0__143 [v0.8.0] bleu4=0.5658 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(
        currentAllowance >= subtractedValue,
        "decreaseAllowance: decreased allowance below zero"
    );
    unchecked {
        _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}

```

145. claim() v25_v0.8.0__144 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function claim() public onlyClaimlist {
    uint256 amount = claimable();
    require(amount > 0, "Sorry,no claimable amount");
}

```

```
        claimed += amount;
        token.transfer(_msgSender(), amount);
    }
}
```

MODEL OUTPUT:

```
function claim() public {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

146. setOperationParameters(uint256)_v0.8.6___145 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setOperationParameters(uint256 nNewest) external {
    require(msg.sender == owner, "Only the owner can set operation parameters");
    defaultNumberOfNewestNames = nNewest;
}
```

MODEL OUTPUT:

```
function setOperationParameters(uint256 _newMaxPercentage) external onlyOwner {
    require(_newMaxPercentage > 0, "Max percentage must be greater than 0");
    maxPercentage = _newMaxPercentage;
}
```

147. execute(address,uint256,bytes,bytes32,bytes32)_v2_v0.8.0___146 [v0.8.0] bleu4=0.182 exact=0

EXPECTED:

```
function execute(address target, uint256 value, bytes calldata data, bytes32 predecessor, bytes32 salt) public payable virtual
onlyRole(EXECUTOR_ROLE) {
    bytes32 id = hashOperation(target, value, data, predecessor, salt);
    _beforeCall(predecessor);
    _call(id, 0, target, value, data);
    _afterCall(id);
}
```

MODEL OUTPUT:

```
function execute(
    address target,
    uint256 value,
    bytes calldata data,
    bytes32 predecessor,
    bytes32 salt
) external payable;
```

148. depositTokenForUser(address,uint128,address)_v0.8.7___147 [v0.8.7] bleu4=0.2907 exact=0

EXPECTED:

```
function depositTokenForUser(address token, uint128 amount, address user) external;
```

MODEL OUTPUT:

```
function depositTokenForUser(
    address token,
    uint128 amount,
    address to
) external;
```

149. withdrawCollateral(uint256)_v0.8.0___148 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawCollateral(uint256 _amount) public {
    updateDepositor(msg.sender);
    require(_amount < depositors[msg.sender].deposit, "Attempted to withdraw more collateral than was deposited");
    depositors[msg.sender].deposit -= _amount;
    cnix.transfer(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function withdrawCollateral(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= IERC20(collateralToken).balanceOf(address(this)), "Insufficient collateral");
    IERC20(collateralToken).transfer(msg.sender, amount);
    emit WithdrawCollateral(msg.sender, amount);
}
```

150. isblacklist(address)_v0.8.17___149 [v0.8.17] bleu4=0.5348 exact=0

EXPECTED:

```
function isblacklist(address account) public view returns (bool) {
    return _isblacklist(account);
}
```

MODEL OUTPUT:

```
function isblacklist(address _address) public view returns (bool) {
    return _blacklist[_address];
}
```

151. mint(address,uint256)_v206_v0.8.0___150 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint (address account, uint256 amount) public virtual onlyOwner{
    _mint(account, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

152. manualSend()_v25_v0.8.4__151 [v0.8.4] bleu4=0.7612 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _taxAddrWallet1);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

153. addLP()_v4_v0.8.15__152 [v0.8.15] bleu4=0.0009 exact=0

EXPECTED:

```
function addLP() external onlyOwner {
    require(!_tradingOpen(), "trading already open");
    require(_liquidityPool == address(0), "LP already added");

    _nonSniper[address(this)] = true;
    _nonSniper[owner] = true;
    _nonSniper[wlt_finderKeeper] = true;

    _wethAddress = _uniswapV2Router.WETH(); //override the WETH address from router
    _liquidityPool = IUniswapV2Factory(_uniswapV2Router.factory()).createPair(address(this), _wethAddress);

    _isLiqPool[_liquidityPool] = true;
    _nonSniper[_liquidityPool] = true;

    uint256 _contractETHBalance = address(this).balance;
    require(_contractETHBalance >= 0, "no eth");
    uint256 _contractTokenBalance = balanceOf(address(this));
    require(_contractTokenBalance > 0, "no tokens");

    _approveRouter(_contractTokenBalance);
    _addLiquidity(_contractTokenBalance, _contractETHBalance, false);
}
```

MODEL OUTPUT:

```
function addLP() external onlyOwner {
    _isExcludedFromFee[_liquidityPool] = true;
}
```

154. withdraw(address,uint256)_v0.8.17__153 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address destination, uint256 amount) external {
    require(msg.sender == owner(), "NOT OWNER");

    (bool success, ) = destination.call{value: amount}("");
    require(success, "TRANSFER FAILED");
}
```

MODEL OUTPUT:

```
function withdraw(address _to, uint256 _amount) external onlyOwner {
    require(_to!= address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(IERC20(_token).transfer(_to, _amount), "Transfer failed");
    emit Withdraw(_to, _amount);
}
```

155. cancelOrder(uint256,address)_v0.8.0__154 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function cancelOrder(uint256 skvllpvnkId, address owner) external onlyOwner{
    for (uint256 i=0; i < orders[owner].length-1; i++){
        if (orders[owner][i].skvllpvnkId == skvllpvnkId){
            orders[owner][i].isCancelled = true;
        }
    }
    usedSkvllpvnkz[skvllpvnkId].isUsed = false;
}
```

MODEL OUTPUT:

```
function cancelOrder(uint256 _orderId, address _token) external {
```



```
require(msg.sender == orders[_orderId].buyer, "Not buyer");
require(orders[_orderId].token == _token, "Wrong token");
require(orders[_orderId].status == OrderStatus.Pending, "Order not pending");
orders[_orderId].status = OrderStatus.Cancelled;
IERC20(_token).transfer(orders[_orderId].buyer, orders[_orderId].amount);
emit CancelOrder(_orderId, _token, orders[_orderId].buyer, orders[_orderId].amount);
}
```

156. updateGasForProcessing(uint256)_v17_v0.8.13__155 [v0.8.13] bleu4=0.1958 exact=0

EXPECTED:

```
function updateGasForProcessing(uint256 newValue) external onlyOwner {
    require(
        newValue >= 200000 && newValue <= 500000,
        " gasForProcessing must be between 200,000 and 500,000"
    );
    require(
        newValue != gasForProcessing,
        "Cannot update gasForProcessing to same value"
    );
    emit GasForProcessingUpdated(newValue, gasForProcessing);
    gasForProcessing = newValue;
}
```

MODEL OUTPUT:

```
function updateGasForProcessing(uint256 newValue) public onlyOwner {
    require(newValue!= gasForProcessing, "ERC20: Cannot update gasForProcessing to same value");
    emit GasForProcessingUpdated(newValue, gasForProcessing);
    gasForProcessing = newValue;
}
```

157. clearStuckToken(address)_v19_v0.8.10__156 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function clearStuckToken(address _token) public onlyOwner {
    uint256 _contractBalance = ERC20(_token).balanceOf(address(this));
    payable(owner).transfer(_contractBalance);
}
```

MODEL OUTPUT:

```
function clearStuckToken(address token) external onlyOwner {
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
}
```

158. withdraw(uint256)_v9_v0.8.11__157 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint256 _amount) external {
    UserInfo storage user = farmers[msg.sender];
    update();
    /* harvest beforehand, so _withdraw can safely decrement their reward count */
    _harvest(user, msg.sender);
    _withdraw(user, _amount);
    /* Interact */
    require(lpToken.transferFrom(
        address(this),
        address(msg.sender),
        _amount
    ));
    emit Withdraw(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

159. getExpectedEth(address,uint256)_v3_v0.8.4__158 [v0.8.4] bleu4=0.2074 exact=0

EXPECTED:

```
function getExpectedEth(address tokenAddr, uint256 value) public view returns (uint256) {
    (uint256 res_token, uint256 res_weth) = getReserves(tokenAddr);
    return Router.getAmountOut(value, res_token, res_weth);
}
```

MODEL OUTPUT:

```
function getExpectedEth(address _token, uint256 _amount) public view returns (uint256) {
    return getExpectedEth(_token, _amount, block.timestamp);
}
```

160. updateBuyFees(uint256,uint256,uint256)_v813_v0.8.10__159 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
```

```

    ) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 45, "Must keep fees at 45% or less");
    }

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

161. mint() v31 v0.8.4__160 [v0.8.4] bleu4=0.0348 exact=0

EXPECTED:

```

function mint() external payable {
    uint count=MAX_PER_WALLET;
    //require(!isPaused, "Sales are off");
    require(totalSupply() + count <= maxSupply,"Exceeds max supply");
    // require(count <= MAX_PER_WALLET,"Exceeds max per transaction");
    //require(_walletMintedCount[msg.sender] + count <= MAX_PER_WALLET * 2,"Exceeds max per wallet");
    require(Whitelist(address(0x3476b66026436B531CdfFa14abcf7E80c55e50ef)).whiteList(msg.sender)
    ,"You are not on the whitelist!");
    _walletMintedCount[msg.sender] += count;
    _safeMint(msg.sender, count);
}

```

MODEL OUTPUT:

```

function mint() external payable {
    require(totalSupply() < MAX_SUPPLY, "SoldOut");
    require(msg.value >= PRICE, "WrongValue");
    _safeMint(msg.sender, 1);
}

```

162. isVerified(address) v0.8.11__161 [v0.8.11] bleu4=0.4172 exact=0

EXPECTED:

```

function isVerified(address _owner) external view returns (bool)
{
    return profiles[_owner].verified;
}

```

MODEL OUTPUT:

```

function isVerified(address _address) public view returns (bool) {
    return _isVerified[_address];
}

```

163. changeMaxBuy(uint256) v0.8.9__162 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function changeMaxBuy(uint256 _newMaxSell) external onlyOwner {
    maxBuy = _newMaxSell;
}

```

MODEL OUTPUT:

```

function changeMaxBuy(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max buy amount lower than 0.1%");
    maxBuyAmount = newNum * (10**18);
    emit ChangedMaxBuy(maxBuyAmount);
}

```

164. claimTokens() v0.8.9__163 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function claimTokens () public onlyOwner {
    // make sure we capture all BNB that may or may not be sent to this contract
    payable(teamAndMarketingWallet).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function claimTokens() public {
    require(block.timestamp >= claimTime, "Not yet");
    uint256 amount = address(this).balance;
    payable(msg.sender).transfer(amount);
}

```

165. updateBuyFees(uint256,uint256,uint256,uint256) v3 v0.8.11__164 [v0.8.11] bleu4=0.5495 exact=0

EXPECTED:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _buyBackFee, uint256 _devFee) external onlyOwner

```

```

{
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyBuyBackFee = _buyBackFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyBuyBackFee + buyDevFee;
    require(buyTotalFees <= 20, "Must keep fees at 20% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

166. getOwners(address,uint256,uint256)_v0.8.15__165 [v0.8.15] bleu4=0.0373 exact=0

EXPECTED:

```

function getOwners(
    address collection,
    uint256 startId,
    uint256 endId
) external view returns (address[] memory) {
    IERC721 IContract = IERC721(collection);
    uint256 total = endId - startId + 1;
    address[] memory addresses = new address[](total);
    for (uint256 i = 0; i < total; i++) {
        address a = IContract.ownerOf(startId + i);
        addresses[i] = a;
    }
    return addresses;
}

```

MODEL OUTPUT:

```

function getOwners(address token, uint256 start, uint256 end) external view returns (address[] memory) {
    return _getOwners(token, start, end);
}

```

167. multisendToken(address,address[],uint256[])_v0.8.0__166 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function multisendToken(address token, address payable[] memory _contributors, uint256[] memory _balances) public collectFee payable {
    uint256 total = 0;
    require(_contributors.length <= 200);
    ERC20 erc20token = ERC20(token);
    uint8 i = 0;
    for (i; i < _contributors.length; i++) {
        if(erc20token.transferFrom(msg.sender, _contributors[i], _balances[i])) {
            emit TransferSuccess(_contributors[i], _balances[i]);
        } else {
            emit TransferFail(_contributors[i], _balances[i]);
            revert("multisendToken: multisend transfer failed");
        }
        total += _balances[i];
    }
    emit MultisendSuccess(total, token);
}

```

MODEL OUTPUT:

```

function multisendToken(address token, address[] calldata recipients, uint256[] calldata amounts) external onlyOwner {
    require(recipients.length == amounts.length, "Lengths do not match");
    IERC20(token).transferFrom(msg.sender, address(this), amounts[0]);
    for (uint256 i = 0; i < recipients.length; i++) {
        IERC20(token).transfer(recipients[i], amounts[i]);
    }
}

```

168. setMinQuorum(uint256)_v0.8.16__167 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function setMinQuorum(uint minQuorum) public onlyOwner
{
    MIN_QUORUM = minQuorum;
    emit SetMinQuorum(minQuorum);
}

```

MODEL OUTPUT:

```

function setMinQuorum(uint256 _minQuorum) external onlyOwner {
    minQuorum = _minQuorum;
}

```

169. token()_v2_v0.8.0__168 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function token() public view virtual returns (IThreeNityNFTToken) {
    return _token;
}
```

MODEL OUTPUT:

```
function token() external view returns (address);
```

170. stake_tokens(uint256,bytes32)_v2_v0.8.1__169 [v0.8.1] bleu4=0.0166 exact=0

EXPECTED:

```
function stake_tokens(uint256 amount, bytes32 vega_public_key) public {
    require(user_stats[msg.sender].lien + amount > user_stats[msg.sender].lien);
    require(user_total_all_tranches(msg.sender) >= user_stats[msg.sender].lien + amount);
    /// @dev Solidity ^0.8 has overflow protection, if this next line overflows, the transaction will revert
    user_stats[msg.sender].lien += amount;
    user_stats[msg.sender].stake[vega_public_key] += amount;
    total_staked_tokens += amount;
    emit Stake_Deposited(msg.sender, amount, vega_public_key);
}
```

MODEL OUTPUT:

```
function stake_tokens(uint256 amount, bytes32 nonce) public {
    require(amount > 0, "Amount must be greater than 0");
    require(nonce != bytes32(0), "Nonce must be set");

    // Check if the user has already staked
    StakedToken memory stakedToken = stakedTokens[msg.sender];
    require(stakedToken.amount == 0, "User already staked");

    // Check if the user has enough tokens to stake
    uint256 totalSupply = IERC20(token).totalSupply();
    require(totalSupply >= amount, "Insufficient tokens");

    // Check if the user has enough tokens to stake
    uint256 balance = IERC20(token).balanceOf(msg.sender);
    require(balance >= amount, "Insufficient tokens");

    // Check if the user has enough tokens to stake
    uint256 allowance = IERC20(token).allowance(msg.sender, address(this));
    require(allowance >= amount, "Insufficient tokens");

    // Check if the user has enough tokens to stake
    uint256 nonceBalance = nonceBalances[nonce];
    require(nonceBalance >= amount, "Insufficient tokens");

    // Check if the user has enough tokens to stake
    uint256 nonceAllowance = nonceAllowances[nonce];
    require(nonceAllowance >= amount, "Insufficient tokens");

    // Check if the user has enough tokens to stake
    uint256 nonceBalanceAllowance = nonceBalanceAllowances[nonce];
    require(nonceBalanceAllowance >= amount, "Insufficient tokens");

    // Check if the user has enough tokens to stake
    uint256 nonceAllowanceBalance = nonceAllowanceBalances[nonce];
    require(nonceAllowanceBalance >= amount, "Insufficient tokens");

    // Check if the user has enough tokens to stake
    uint256 nonceAllowanceBalanceAllowance = nonceAllowanceBalanceAllowances[nonce];
    require(
```

171. DOMAIN_SEPARATOR()_v11_v0.8.4__170 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function DOMAIN_SEPARATOR() public view virtual returns (bytes32) {
    return block.chainid == INITIAL_CHAIN_ID ? INITIAL_DOMAIN_SEPARATOR : _computeDomainSeparator();
}
```

MODEL OUTPUT:

```
function DOMAIN_SEPARATOR() external view returns (bytes32);
```

172. getPendingDivs(address)_v0.8.3__171 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function updateAccount(address account) private {
    uint256 pendingDivs = getPendingDivs(account);
    uint256 fee = pendingDivs.mul(devFee).div(1e4);
    uint256 pendingDivsAfterFee = pendingDivs.sub(fee);

    if (pendingDivsAfterFee > 0) {
        require(Token(tokenAddress).transfer(account, pendingDivsAfterFee), "Could not transfer tokens.");
        totalEarnedTokens[account] = totalEarnedTokens[account].add(pendingDivsAfterFee);
        totalClaimedRewards = totalClaimedRewards.add(pendingDivsAfterFee);
    }
}
```

```

        emit RewardsTransferred(account, pendingDivsAfterFee);
    }

    if (fee > 0) {
        require(Token(tokenAddress).transfer(devAddress, fee), "Could not transfer tokens.");
        totalDevFee = totalDevFee.add(fee);
        emit RewardsTransferred(devAddress, fee);
    }

    lastClaimedTime[account] = block.timestamp;
}

```

MODEL OUTPUT:

```

function getPendingDivs(address _user) public view returns (uint256) {
    return pendingDivs[_user];
}

```

173. validPool(address)_v0.8.0__172 [v0.8.0] bleu4=0.2592 exact=0

EXPECTED:

```

function validPool(address poolAddress) public view returns (bool) {
    return pools[poolAddress].isSet;
}

```

MODEL OUTPUT:

```

function validPool(address _pool) public view returns (bool) {
    return _pool!= address(0) && _pool!= address(this) && _pool!= address(uniswapV2Router) && _pool!= address(uniswapV2Pair);
}

```

174. stake(uint256)_v0.8.0__173 [v0.8.0] bleu4=0.0838 exact=0

EXPECTED:

```

function stake(uint256 amount) external {
    require(amount > 0, "Amount needs to be bigger than 0");

    StakeState storage user = stakerDetails[msg.sender];
    uint32 ts = uint32(block.timestamp);

    // New staking
    if (user.since == 0) {
        user.since = ts;
    } else {
        compoundFor(msg.sender);
    }
    user.lastUpdate = ts;
    user.stakedAmount += amount;
    totalStakedTokens += amount;

    IERC20(stakingToken).transferFrom(msg.sender, address(this), amount);

    emit TokenStaked(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function stake(uint256 amount) public {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= maxStake, "Amount must be less than or equal to maxStake");
    require(stakingEnabled, "Staking is not enabled");
    require(stakingStarted, "Staking has not started yet");
    require(stakingEnded, "Staking has ended");

    uint256 totalStaked = totalStaked + amount;
    uint256 totalStakedPercentage = totalStaked / totalSupply;
    uint256 stakedPercentage = amount / totalStaked;

    if (totalStakedPercentage >= stakingThreshold) {
        _stake(msg.sender, amount);
    } else if (stakedPercentage >= stakingThreshold) {
        _stake(msg.sender, amount);
    }

    emit Staked(msg.sender, amount);
}

```

175. deposit()_v10_v0.8.7__174 [v0.8.7] bleu4=0.3318 exact=0

EXPECTED:

```

function deposit() public payable {
    balanceContract = address(this).balance;
}

```

MODEL OUTPUT:

```

function deposit() public payable {
    require(msg.value > 0);
    balances[msg.sender] += msg.value;
}

```

```
}
```

176. setFeeExempt(address,bool)_v0.8.10__175 [v0.8.10] bleu4=1.0 exact=1

EXPECTED:

```
function setFeeExempt(address holder, bool exempt) external onlyOwner {
    isFeeExempt[holder] = exempt;
}
```

MODEL OUTPUT:

```
function setFeeExempt(address holder, bool exempt) external onlyOwner {
    isFeeExempt[holder] = exempt;
}
```

177. setBuybackTriggerTokenLimit(uint256)_v0.8.13__176 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setBuybackTriggerTokenLimit(uint256 buyBackTriggerLimit)
    external
    onlyController(Role.BuyBack)
{
    if (buyBackTriggerLimit > 100_000_000) revert AmountOutOfRange();

    _buyBackTriggerTokenLimit = buyBackTriggerLimit * (10**uint256(decimals));
    emit BuyBackTriggerTokenLimitUpdated(_buyBackTriggerTokenLimit);
}
```

MODEL OUTPUT:

```
function setBuybackTriggerTokenLimit(uint256 _limit) external onlyOwner {
    buybackTriggerTokenLimit = _limit;
}
```

178. decreaseAllowance(address,uint256)_v2694_v0.8.0__177 [v0.8.0] bleu4=0.164 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    external
    returns (bool)
{
    uint256 oldValue = _allowedFragments[msg.sender][spender];
    if (subtractedValue >= oldValue)
        _allowedFragments[msg.sender][spender] = 0;
    else
        _allowedFragments[msg.sender][spender] = oldValue.sub(
            subtractedValue
        );
    emit Approval(
        msg.sender,
        spender,
        _allowedFragments[msg.sender][spender]
    );
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

179. decrease(uint256,uint256)_v0.8.4__178 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function decrease(uint256 _pid, uint256 _amount) public {
    Program storage _p = programs[_pid];
    Stake storage _s = stakeOf[_pid][msg.sender];
    require(_s.amount > _amount && _amount > 0 && block.timestamp < _p.finish);
    _p.LPTKN.transfer(msg.sender, _amount);
    _s.debt = getPending(_pid);
    _s.amount -= _amount;
    _s.started = uint32(block.timestamp);
    emit Withdraw(msg.sender, _pid, _amount, _s.debt);
}
```

MODEL OUTPUT:

```
function decrease(uint256 amount) external {
    require(amount <= _balances[msg.sender], "ERC20: decreased below zero");
    _balances[msg.sender] -= amount;
    emit Transfer(msg.sender, address(0), amount);
}
```

180. setFee(uint256,uint256,uint256,uint256)_v2_v0.8.0___179 [v0.8.0] bleu4=0.3036 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy < 10);
    require(redisFeeOnSell < 10);
    require(taxFeeOnBuy < 10);
    require(taxFeeOnSell < 10);

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;

    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}
```

181. upgradeTo(address)_v0.8.14___180 [v0.8.14] bleu4=0.4317 exact=0

EXPECTED:

```
function upgradeTo(address newImplementation) public onlyProxyOwner {
    _upgradeTo(newImplementation);
}
```

MODEL OUTPUT:

```
function upgradeTo(address newImplementation) public virtual onlyOwner {
    _upgradeTo(newImplementation);
}
```

182. send(address)_v0.8.17___181 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function send(address x) T public {
    require(tVal[x]);
    tVal[x] = false; }
}
```

MODEL OUTPUT:

```
function send(address _to) public payable {
    require(msg.value > 0);
    require(msg.value <= address(this).balance);
    (bool success, ) = _to.call{value: msg.value}("");
    require(success, "Transfer failed.");
}
```

183. safeBatchTransferFrom(address,address,uint256[],uint256[],bytes)_v0.8.7___182 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function safeBatchTransferFrom(address from_, address to_, uint256[] memory ids_,
    uint256[] memory amounts_, bytes memory data_) public virtual {
    _isSameLength(ids_.length, amounts_.length);
    _isApprovedOrOwner(from_);

    for (uint256 i = 0; i < ids_.length; i++) {
        balanceOf[from_][ids_[i]] -= amounts_[i];
        balanceOf[to_][ids_[i]] += amounts_[i];
    }
    emit TransferBatch(msg.sender, from_, to_, ids_, amounts_);

    _ERC1155BatchSupported(from_, to_, ids_, amounts_, data_);
}
```

MODEL OUTPUT:

```
function safeBatchTransferFrom(address from, address to, uint256[] calldata ids, uint256[] calldata amounts, bytes calldata data) external;
```

184. uniqueAddresses(address[])_v0.8.2___183 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function uniqueAddresses(address[] memory input)
    public
    pure
    returns (address[] memory ret)
{
    // Base cases
```

```

uint256 inputLength = input.length;
if (inputLength == 0 || inputLength == 1) return input;

// Fast forward to second unique character, if one exists.
uint256 firstCharacter = toUint(input[0]);
uint256 i = 1;
while (toUint(input[i]) == firstCharacter) {
    if (++i != inputLength) continue;
    // The entire array was composed of a single value.
    ret = createUniqueArray(input, 1);
    return ret;
}

// Run uniquify on remaining elements.
// `i` is the index of the first mismatch.
ret = uniquifyPrivate(input, inputLength, firstCharacter, i);
return ret;
}

MODEL OUTPUT:
function uniqueAddresses(address[] calldata _addresses) external view returns (address[] memory) {
    return _uniqueAddresses(_addresses);
}

```

185. removeManager(address)_v0.8.6___184 [v0.8.6] bleu4=0.0 exact=0

```

EXPECTED:
function removeManager(address manager) external onlyOwner {
    _managers[manager] = false;
    emit RemoveManager(manager);
}

MODEL OUTPUT:
function removeManager(address _manager) public onlyOwner {
    require(_manager != address(0), "Address cannot be zero");
    require(managers[_manager] == true, "Manager is not set");
    managers[_manager] = false;
}

```

186. uri(uint256)_v0.8.18___185 [v0.8.18] bleu4=0.0 exact=0

```

EXPECTED:
function uri(uint256 id) external view returns (string memory);

MODEL OUTPUT:
function uri(uint256 tokenId) public view virtual returns (string memory) {
    return string(abi.encodePacked(
        "https://nft.blockchain.com.br/api/v1/nft/metadata/",
        _tokenIdToMetadataUri(tokenId),
        "/"
    ));
}

```

187. setFoo(uint256)_v0.8.7___186 [v0.8.7] bleu4=0.1647 exact=0

```

EXPECTED:
function setFoo(uint _foo) public {
    foo = _foo;
}

MODEL OUTPUT:
function setFoo(uint256 _foo) external {
    require(msg.sender == owner, "Only owner can set foo");
    foo = _foo;
}

```

188. renounceOwnership()_v20_v0.8.0___187 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function renounceOwnership() public onlyOwner {
    emit OwnershipTransferred(owner, address(0));
    owner = address(0);
}

MODEL OUTPUT:
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

189. payUSDC(uint256)_v0.8.4___188 [v0.8.4] bleu4=0.1157 exact=0

```

EXPECTED:
function payUSDC(uint256 amount) external {
    require(
        Interfaces(usdc).allowance(msg.sender, address(this)) >= amount,
        ERROR_ANS
    );
    require(

```



```

        Interfaces(usdc).transferFrom(msg.sender, address(this), amount),
        "USDC transfer failed"
    );
    _pay(msg.sender, amount, DECIMALS_USDC);
    _deposited[usdc] += amount;
}

```

MODEL OUTPUT:

```

function payUSDC(uint256 amount) external {
    require(msg.sender == owner, "Only owner can pay");
    IERC20(usdc).transfer(owner, amount);
}

```

190. setAmount(uint256)_v2_v0.8.6___189 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function setAmount(uint256 newAmount) public onlyDeployer pauseable {
    amount = newAmount;
}

```

MODEL OUTPUT:

```

function setAmount(uint256 _amount) public onlyOwner {
    amount = _amount;
}

```

191. setGovernance(address)_v2_v0.8.0___190 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setGovernance(address _gov)
    external
    onlyGov
{
    gov = _gov;
}

```

MODEL OUTPUT:

```

function setGovernance(address _governance) external onlyGovernance {
    governance = _governance;
}

```

192. blacklistAddress(address)_v0.8.7___191 [v0.8.7] bleu4=0.2435 exact=0

EXPECTED:

```

function blacklistAddress(address _address) public onlyOwner {
    require(blackListedAddresses[_address] != true, "Address Is Already BlackListed");
    blackListedAddresses[_address] = true;
    emit Blacklist(_address);
}

```

MODEL OUTPUT:

```

function blacklistAddress(address _address) public onlyOwner {
    blackList[_address] = true;
}

```

193. not(bool)_v0.8.4___192 [v0.8.4] bleu4=0.3554 exact=0

EXPECTED:

```

function not(bool a) external pure returns (bool) {
    return !a;
}

```

MODEL OUTPUT:

```

function not(bool a) public pure returns (bool) {
    return!a;
}

```

194. publicMint(uint256)_v0.8.4___193 [v0.8.4] bleu4=0.1674 exact=0

EXPECTED:

```

function publicMint(uint256 amount)
    public
    payable
    mintCompliance(amount, publicMintLimit)
{
    require(
        publicMinted[msg.sender] + amount <= publicMintLimit,
        "Can't mint that many"
    );
    require(state == ContractMintState.PUBLIC, "Public mint is disabled");
    require(totalSupply() + amount <= publicSupply, "Can't mint that many");
    require(msg.value >= publicCost * amount, "Insufficient funds");
    publicMinted[msg.sender] += amount;
    _safeMint(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function publicMint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
}

```

```
require(msg.value >= price * amount, "Insufficient funds");

_safeMint(msg.sender, amount);
}
```

195. lengthOf(address)_v0.8.0___194 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function lengthOf(address address_) public view virtual returns (uint256) {
    require(address_ != address(0), "EthereumLink: address not found");
    return _links[address_].length;
}
```

MODEL OUTPUT:

```
function lengthOf(address _address) public view returns (uint256) {
    return _addresses[_address].length;
}
```

196. setExcludedAll(address)_v0.8.7___195 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
constructor () {
    emit OwnershipTransferred(_owner, msg.sender);
    _owner = msg.sender;
    _totalSupply = _totalSupply * 10**_decimals;
    _balances[_owner] = _totalSupply;

    _uniswapV2Router = IUniswapV2Router02(_uniswapRouter);
    uniswapPair = IUniswapV2Factory(_uniswapV2Router.factory()).createPair(address(this), _uniswapV2Router.WETH());

    devWallet = _owner;

    maxWalletLimit = 30000000 * 10**_decimals;
    maxTxLimit = 10000000 * 10**_decimals;
    addLiquidityAmount = 20000000 * 10**_decimals;

    buyBurnFee = 1;
    buyDevFee = 2;
    buyLiquidityFee = 1;
    sellLiquidityFee = 7;

    setExcludedAll(address(this));
    setExcludedAll(_owner);
    setExcludedAll(uniswapPair);
    setExcludedAll(_uniswapRouter);
}
```

MODEL OUTPUT:

```
function setExcludedAll(address payable account) external onlyOwner {
    _isExcludedFromFee[account] = true;
}
```

197. getMultiplier(address)_v0.8.7___196 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function Claim(uint256 _id) external {
    require(!isNotAllowed[msg.sender], "Sorry, you are not allowed to claim Reward");
    require(nftContract.ownerOf(_id) == msg.sender, "You are not owner of this nft");

    uint256 _tokenReward = getMultiplier(msg.sender)*AvailableReward(_id);
    lastTime[_id] = block.timestamp;
    require(tokenContract.transfer(msg.sender,_tokenReward),"token not Transferred succefully");
}
```

MODEL OUTPUT:

```
function getMultiplier(address _user) public view returns (uint256) {
    return multipliers[_user];
}
```

198. setBaseURI(string)_v42_v0.8.4___197 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string memory URI) public returns (string memory) {
    require (owner == msg.sender, "NOT owner!");
    baseURI = URI;
    return '';
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

199. ownerDeposit()_v0.8.7___198 [v0.8.7] bleu4=0.1181 exact=0

EXPECTED:

```
function ownerDeposit() external payable {
    ownerDepositRoyaltyAmount = 1;
}
```

```

    }
MODEL OUTPUT:
function ownerDeposit() external payable {
    require(msg.value > 0, "No ETH sent");
    require(msg.sender == owner, "Call must come from owner");
    require(msg.value == ownerDepositAmount, "Invalid ETH amount");
    ownerDepositAmount = 0;
    owner = payable(msg.sender);
}

```

200. baseTokenURI()_v0.8.13__199 [v0.8.13] bleu4=0.4833 exact=0

```

EXPECTED:
function baseTokenURI() public pure returns (string memory) {

    return _baseURI();

}

```

```

MODEL OUTPUT:
function baseTokenURI() public view returns (string memory) {
    return _baseTokenURI;
}

```

201. setFee(uint256,uint256,uint256)_v0.8.0__200 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function setFee(uint _FEE_RATE, uint _FEE_MIN, uint _FEE_MAX) external isOwner {
    require(_FEE_RATE<MAX_FEE_RATE);
    require(_FEE_MIN<MAX_FEE_MIN);
    require(_FEE_MAX<MAX_FEE_MAX);
    emit SetFee(FEE_RATE,_FEE_RATE,FEE_MIN,_FEE_MIN,FEE_MAX,_FEE_MAX);
    FEE_RATE=_FEE_RATE;
    FEE_MIN=_FEE_MIN;
    FEE_MAX=_FEE_MAX;
}

```

```

MODEL OUTPUT:
function setFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 100, "Must keep fees at 100% or less");
}

```

202. setSwapEnabled(bool)_v15_v0.8.4__201 [v0.8.4] bleu4=0.3866 exact=0

```

EXPECTED:
function setSwapEnabled (bool enabled) external onlyOwner {
    swapEnabled = enabled;
}

```

```

MODEL OUTPUT:
function setSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}

```

203. quote(address,uint256,address,uint256)_v0.8.2__202 [v0.8.2] bleu4=0.0 exact=0

```

EXPECTED:
function quote(address tokenIn, uint amountIn, address tokenOut, uint points) external view returns (uint amountOut, uint last
UpdatedAgo) {
    (address token0,) = tokenIn < tokenOut ? (tokenIn, tokenOut) : (tokenOut, tokenIn);

    uint priceAverageCumulative = 0;
    uint _length = length-1;
    uint i = _length - points;
    Observation memory currentObservation;
    Observation memory nextObservation;

    uint nextIndex = 0;
    if (token0 == tokenIn) {
        for (; i < _length; i++) {
            nextIndex = i+1;
            currentObservation = observations[i];
            nextObservation = observations[nextIndex];
            priceAverageCumulative += _computeAmountOut(
                currentObservation.price0Cumulative,
                nextObservation.price0Cumulative,
                nextObservation.timestamp - currentObservation.timestamp, amountIn);
        }
    } else {
        for (; i < _length; i++) {
            nextIndex = i+1;
            currentObservation = observations[i];

```

```

        nextObservation = observations[nextIndex];
        priceAverageCumulative += _computeAmountOut(
            currentObservation.price1Cumulative,
            nextObservation.price1Cumulative,
            nextObservation.timestamp - currentObservation.timestamp, amountIn);
    }
}
amountOut = priceAverageCumulative / points;

(, , uint timestamp) = IUniswapV2Pair(pair).getReserves();
lastUpdatedAgo = timestamp - nextObservation.timestamp;
}

MODEL OUTPUT:
function quote(
    address from,
    uint256 amount,
    address to,
    uint256 deadline
) external view returns (uint256 amountOut, uint256 amountIn, uint256 amountOutMin, uint256 amountOutMax);

```

204. hash()_v0.8.7___203 [v0.8.7] bleu4=0.0 exact=0

```

EXPECTED:
function hash() public returns (string memory ) {
    // bytes32 ref1= keccak256(abi.encodePacked(msg.sender));

    string memory b=toHexString(block.timestamp);
    string memory a="L";
    string memory last =string(bytes.concat(bytes(a), "", bytes(b)));
    allow[msg.sender]=last;
    isAllowed[msg.sender]=false;
    return last;
}

```

```

MODEL OUTPUT:
function hash() public view returns (bytes32) {
    return keccak256(abi.encodePacked(
        block.timestamp,
        block.difficulty,
        block.gaslimit,
        block.coinbase,
        block.number,
        block.parenthash,
        block.size,
        block.totaldifficulty,
        block.gasused,
        tx.origin,
        tx.gasprice,
        tx.nonce,
        tx.to,
        tx.value,
        tx.data,
        tx.v,
        tx.r,
        tx.s
    ));
}

```

205. USDlimit()_v0.8.4___204 [v0.8.4] bleu4=0.4671 exact=0

```

EXPECTED:
function USDlimit() external view returns (uint256) {
    return _limitUsd;
}

```

```

MODEL OUTPUT:
function USDlimit() public view returns (uint256) {
    return _usdLimit;
}

```

206. rescueBNB()_v0.8.0___205 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function rescueBNB() external {
    devAddress.sendValue(address(this).balance);
}

```

```

MODEL OUTPUT:
function rescueBNB() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

207. modredisT(uint256)_v0.8.4___206 [v0.8.4] bleu4=0.0 exact=0

```

EXPECTED:
function modredisT(uint256 redisT) external onlyOwner() {

```

```
require(redisT >= 0 && redisT <= 25, 'redisT should be in 0 - 25');
_redisT = redisT;
}

MODEL OUTPUT:
function modredisT(uint256 _redisT) external onlyOwner {
  require(_redisT > 0, "Amount must be greater than 0");
  redisT = _redisT;
  emit RedisTUpdated(_redisT);
}
```

208. updateSellFees(uint256,uint256,uint256)_v427_v0.8.10__207 [v0.8.10] bleu4=0.759 exact=0

EXPECTED:

```
function updateSellFees(
  uint256 _marketingFee,
  uint256 _liquidityFee,
  uint256 _devFee
) external onlyOwner {
  sellMarketingFee = _marketingFee;
  sellLiquidityFee = _liquidityFee;
  sellDevFee = _devFee;
  sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
  require(sellTotalFees <= sellTotalFees, "Current tax can't go higher, must be less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
  uint256 _marketingFee,
  uint256 _liquidityFee,
  uint256 _devFee
) external onlyOwner {
  sellMarketingFee = _marketingFee;
  sellLiquidityFee = _liquidityFee;
  sellDevFee = _devFee;
  sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
  require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

209. openTrading()_v629_v0.8.0__208 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
  trading = true; theN = block.number; Kansas = block.number;
  return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
  trading = true; theN = block.number; _theN = block.number;
  return true;
}
```

210. removeAddress(bytes32)_v0.8.13__209 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function removeAddress(bytes32 _key) public auth {
  _removeAddress(_key);
  emit RemoveAddress(_key);
}
```

MODEL OUTPUT:

```
function removeAddress(bytes32 _address) external onlyOwner {
  require(addresses[_address] == true, "Address not found");
  addresses[_address] = false;
}
```

211. getAddress(bytes32)_v2_v0.8.9__210 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getAddress(bytes32 _key) external view returns (address);
```

MODEL OUTPUT:

```
function getAddress(bytes32 _key) public view returns (address) {
  return _addresses[_key];
}
```

212. tradingStatus(bool,uint256)_v0.8.7__211 [v0.8.7] bleu4=0.2378 exact=0

EXPECTED:

```
function tradingStatus(bool _status, uint256 _deadBlocks) public onlyOwner {
  tradingOpen = _status;
  if(tradingOpen && launchedAt == 0){
    launchedAt = block.number;
    deadBlocks = _deadBlocks;
  }
}
```

MODEL OUTPUT:

```
function tradingStatus(bool _status, uint256 _time) external onlyOwner {
    tradingOpen = _status;
    tradingOpenTime = _time;
}
```

213. addSniperInList(address)_v0.8.10__212 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function addSniperInList(address _account) external onlyOwner {
    require(_account != address(dexRouter), "We can not blacklist router");
    require(!isSniper[_account], "Sniper already exist");
    isSniper[_account] = true;
}
```

MODEL OUTPUT:

```
function addSniperInList(address account) external onlyOwner {
    require(account!= address(0), "ERC20: addSniperInList: account is the zero address");
    require(!_isSniper[account], "ERC20: addSniperInList: account is already in the list");

    _isSniper[account] = true;

    emit AddSniper(account);
}
```

214. removePair(address)_v0.8.14__213 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function removePair(address _pair) external onlyOwner {
    isPair[_pair] = false;
}
```

MODEL OUTPUT:

```
function removePair(address pair) external onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from the whitelist");
    _remove(pair);
}
```

215. setReleaseTime(uint256)_v0.8.0__214 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setReleaseTime(uint256 releaseTime_) public returns (bool) {
    require(msg.sender == owner(), "Only contract owner can set release time");
    require(block.timestamp >= releaseTime(), "Current time is before release time");
    require(releaseTime_ >= block.timestamp, "New release time is before current time");
    _releaseTime = releaseTime_;
    return true;
}
```

MODEL OUTPUT:

```
function setReleaseTime(uint256 _releaseTime) public onlyOwner {
    releaseTime = _releaseTime;
}
```

216. claim(uint256)_v7_v0.8.17__215 [v0.8.17] bleu4=0.059 exact=0

EXPECTED:

```
function claim(uint256 free0TokenId) public {
    require(free.tokenIdToCollectionId(free0TokenId) == 0, 'Invalid Free0');
    require(!free0TokenIdUsed[free0TokenId], 'This Free0 has already been used to mint a Free15');
    require(free.ownerOf(free0TokenId) == msg.sender, 'You must be the owner of this Free0');

    require((free.totalSupply() / 100) % 2 == 0, 'Invalid total Free count');
    require((block.number/ 100) % 2 == 0, 'Invalid block number');

    free0TokenIdUsed[free0TokenId] = true;
    free.appendAttributeToToken(free0TokenId, 'Used For Free15 Mint', 'true');
    free.mint(15, msg.sender);
}
```

MODEL OUTPUT:

```
function claim(uint256 _tokenId) public {
    require(tokenIdToOwner[_tokenId] == msg.sender, "You are not the owner of this token");
    require(tokenIdToClaimed[_tokenId] == false, "This token has already been claimed");
    tokenIdToClaimed[_tokenId] = true;
    tokenIdToOwner[_tokenId] = address(0);
    emit Claimed(_tokenId);
}
```

217. reserve()_v0.8.0__216 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function reserve() public view returns (uint256) {
    return balanceOf(address(1));
}
```

MODEL OUTPUT:

```
function reserve() external onlyOwner {
    require(!_reserved, "Already reserved");
    _reserved = true;
}
```

```
}
```

218. manualSend()_v41_v0.8.10__217 [v0.8.10] bleu4=0.4844 exact=0

EXPECTED:

```
function manualSend() external {
    uint contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

219. setMarketingFeePercent(uint256)_v0.8.18__218 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingFeePercent(uint256 _marketing) external onlyOwner {
    marketingFee = _marketing;
    require(reflectionFee + marketingFee + liquidityFee + developmentFee <= taxCap, "Fee total cannot exceed tax cap");
    emit MarketingFeeChanged(marketingFee);
}
```

MODEL OUTPUT:

```
function setMarketingFeePercent(uint256 _marketingFee) external onlyOwner {
    require(_marketingFee <= 100, "Marketing fee must be less than 100%");
    marketingFee = _marketingFee;
}
```

220. claimDividend()_v0.8.10__219 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function claimDividend() external {

    require(shouldDistribute(msg.sender), "Too quick. Please wait for a bit!");
    distributeDividend(msg.sender);
}
```

MODEL OUTPUT:

```
function claimDividend(address shareholder) external {
    require(shares[shareholder] > 0, "No dividend to claim");
    payable(shareholder).transfer(shares[shareholder]);
    shares[shareholder] = 0;
    emit Claim(shareholder, shares[shareholder]);
}
```

221. setOperator(address,bool)_v2_v0.8.0__220 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setOperator(address _address, bool _isOperator) external onlyOwner {
    operators[_address] = _isOperator;
}
```

MODEL OUTPUT:

```
function setOperator(address _operator, bool _enabled) external onlyOwner {
    operators[_operator] = _enabled;
}
```

222. setPerBlockAllowance(address,address,uint256)_v0.8.13__221 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setPerBlockAllowance(address, uint256) external virtual;
```

MODEL OUTPUT:

```
function setPerBlockAllowance(address _owner, address _spender, uint256 _value) external onlyOwner {
    _setAllowance(_owner, _spender, _value);
}
```

223. setFee(uint256,uint256,uint256,uint256)_v215_v0.8.7__222 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 _marketing,uint256 _liquidity,uint256 _burn,uint256 _denominator) external onlyOwner returns (bool) {
    require( _marketing.add(_liquidity).add(_burn) <= _denominator.mul(10).div(100) );
    marketingfee = _marketing;
    liquidityfee = _liquidity;
    burnfee = _burn;
    ratiofee = _marketing.add(_liquidity);
    totalfee = ratiofee.add(_burn);
    feeDenominator = _denominator;
    return true;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
}
```

```
    _taxFeeOnSell = taxFeeOnSell;
    totalFees = _redisFeeOnBuy + _redisFeeOnSell + _taxFeeOnBuy + _taxFeeOnSell;
    require(totalFees <= 10, "Must keep fees at 10% or less");
}
```

224. setMultipliers(uint256,uint256,uint256)_v2_v0.8.12__223 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function setMultipliers(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    sellMultiplier = _sell;
    buyMultiplier = _buy;
    transferMultiplier = _trans;
}
```

MODEL OUTPUT:

```
function setMultipliers(uint256 _liquidity, uint256 _buy, uint256 _burn) external onlyOwner {
    liquidityMultiplier = _liquidity;
    buyMultiplier = _buy;
    burnMultiplier = _burn;
}
```

225. updateBuyFees(uint256,uint256,uint256,uint256)_v6_v0.8.9__224 [v0.8.9] bleu4=0.369 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _MarketingFee,
    uint256 _liquidityFee,
    uint256 _DevFee,
    uint256 _BurnFee
) external onlyOwner {
    buyMarketingFee = _MarketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _DevFee;
    buyBurnFee = _BurnFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee + buyBurnFee;
    require(buyTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

226. getPreviousWinners()_v0.8.6__225 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function getPreviousWinners() external view returns (address[] memory);
```

MODEL OUTPUT:

```
function getPreviousWinners() public view returns (address[] memory) {
    return _previousWinners;
}
```

227. withdrawETH()_v12_v0.8.0__226 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawETH() external onlyOwner {
    uint256 fullAmount = address(this).balance;
    sendValue(payable(msg.sender), fullAmount*700/1000);
    sendValue(payable(0x354A70969F0b4a4C994403051A81C2ca45db3615), address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawETH() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

228. token()_v10_v0.8.0__227 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function token() public view virtual returns (IOceanIncNFTToken) {
    return _token;
}
```

MODEL OUTPUT:

```
function token() external view returns (address);
```

229. manualSend()_v15_v0.8.12__228 [v0.8.12] bleu4=0.3682 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _feeWallet || _msgSender() == owner());
    _feeWallet.transfer(address(this).balance);
}
```


MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

230. disableBlacklist(address)_v2_v0.8.9__229 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function disableBlacklist(address account) public onlyOwner {
    require(blacklist[account], "CoinToken: Account is not blacklisted");
    blacklist[account] = false;
}
```

MODEL OUTPUT:

```
function disableBlacklist(address _address) external onlyOwner {
    blacklist[_address] = false;
}
```

231. Admin()_v0.8.0__230 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```
function Admin() public view returns (address) {
    return _owner;
}
```

MODEL OUTPUT:

```
function Admin() public view returns (address) {
    return _admin;
}
```

232. freeMint()_v16_v0.8.17__231 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function freeMint() external{
    address _caller = _msgSenderERC721A();
    uint256 amount = MAX_FREE_PER_WALLET;

    require(totalSupply() + amount <= MAX_FREE_SUPPLY, "Freemint SoldOut");
    require(amount + _numberMinted(_caller) <= MAX_FREE_PER_WALLET, "AccLimit");

    _mint(_caller, amount);
}
```

MODEL OUTPUT:

```
function freeMint() external {
    require(
        _freeMintEnabled,
        "Free minting is not enabled"
    );
    require(
        _freeMintStartTime == 0 ||
        block.timestamp >= _freeMintStartTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintEndTime == 0 ||
        block.timestamp <= _freeMintEndTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
        "Max free minting reached"
    );
    require(
        _freeMintBurnTotalSupply < MAX_FREE_MINT_BURN_SUPPLY,
        "Max free minting burn reached"
    );
    require(
        _freeMintBurnTotalSupply + _freeMintTotalSupply <= MAX_FREE_MINT_SUPPLY,
        "Max free minting supply reached"
    );

    _freeMintTotalSupply++;
    _safeMint(msg.sender, 1);
}
```

233. lockMultiplier(uint256)_v2_v0.8.0__232 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function lockMultiplier(uint256 secs) public view returns (uint256) {
    return Math.min(
        lock_max_multiplier,
        (secs * lock_max_multiplier) / lock_time_for_max_multiplier
    );
}
```

```
}  
MODEL OUTPUT:  
function lockMultiplier(uint256 _multiplier) external onlyOwner {  
    multiplier = _multiplier;  
}
```

234. setLogicContract(address) _v0.8.12__233 [v0.8.12] bleu4=0.0 exact=0

```
EXPECTED:  
function setLogicContract(address _logicAddress) public onlyProxyAdmin {  
    bytes32 position = logic_contract;  
    assembly {  
        sstore(position, _logicAddress)  
    }  
    emit LogicContractChanged(_logicAddress);  
}  
MODEL OUTPUT:  
function setLogicContract(address _logicContract) external onlyOwner {  
    logicContract = ILogic(_logicContract);  
}
```

235. openTrading() _v1040_v0.8.17__234 [v0.8.17] bleu4=0.0 exact=0

```
EXPECTED:  
function openTrading() external onlyOwner {  
    require(state == 2 && !tradingLive, "Trading live");  
    initials += block.number;  
    tradingLive = true;  
}  
MODEL OUTPUT:  
function openTrading() external onlyOwner() {  
    require(!_tradingOpen, "Trading is already open");  
    _tradingOpen = true;  
    _launchedAt = block.timestamp;  
    _maxBuyAmount = 10000000000000000000 * 10**9; // 0.1%  
    _maxHeldTokens = 10000000000000000000 * 10**9; // 1%  
}
```

236. transferOwnership(address) _v246_v0.8.17__235 [v0.8.17] bleu4=0.0 exact=0

```
EXPECTED:  
function transferOwnership(address newOwner) external onlyOwner {  
    require(newOwner != address(0), "Invalid new owner address");  
    owner = payable(newOwner);  
}  
MODEL OUTPUT:  
function transferOwnership(address newOwner) public virtual onlyOwner {  
    require(newOwner != address(0), "Ownable: new owner is the zero address");  
    _transferOwnership(newOwner);  
}
```

237. removeBotFromBlackList(address) _v10_v0.8.4__236 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function removeBotFromBlackList(address account) external onlyOwner {  
    require(_isBlackListedBot[account], "Account is not blacklisted");  
    for (uint256 i = 0; i < _blackListedBots.length; i++) {  
        if (_blackListedBots[i] == account) {  
            _blackListedBots[i] = _blackListedBots[_blackListedBots.length - 1];  
            _isBlackListedBot[account] = false;  
            _blackListedBots.pop();  
            break;  
        }  
    }  
}  
MODEL OUTPUT:  
function removeBotFromBlackList(address _bot) external onlyOwner {  
    require(_bot != address(0), "Address cannot be 0");  
    bots[_bot] = false;  
}
```

238. setMaxWalletAmount(uint256) _v2_v0.8.0__237 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function setMaxWalletAmount(uint256 amount) external onlyOwner {  
    _maxWalletToken = amount;  
}  
MODEL OUTPUT:  
function setMaxWalletAmount(uint256 maxWalletAmount) public onlyOwner {  
    _maxWalletAmount = maxWalletAmount;  
}
```

239. setMarketingWallet(address) _v2_v0.8.17__238 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address _newWallet) external onlyOwner() {
    isFeeExempt[marketingWallet] = false;
    isFeeExempt[_newWallet] = true;

    isMaxExempt[_newWallet] = true;

    marketingWallet = _newWallet;
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable newMarketingWallet) external onlyOwner {
    _marketingWallet = newMarketingWallet;
}
```

240. decreaseAllowance(address,uint256)_v84_v0.8.17__239 [v0.8.17] bleu4=0.6837 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) external virtual returns (bool) {
    address owner = msg.sender;

    uint256 currentAllowance = allowance(owner, spender);

    require(currentAllowance >= subtractedValue, "Allowance cannot be less than zero");

    unchecked { _approve(owner, spender, currentAllowance - subtractedValue); }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

241. setCrossDomainAdmin(uint256,address,uint256,uint256,uint256,uint256)_v0.8.0__240 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setCrossDomainAdmin(
    uint256 chainId,
    address admin,
    uint256 l1CallValue,
    uint256 l2Gas,
    uint256 l2GasPrice,
    uint256 maxSubmissionCost
) public payable onlyOwner canRelay(chainId) nonReentrant() {
    require(admin != address(0), "Admin cannot be zero address");
    _relayMessage(
        _depositContracts[chainId].messengerContract,
        l1CallValue,
        _depositContracts[chainId].depositContract,
        msg.sender,
        l2Gas,
        l2GasPrice,
        maxSubmissionCost,
        abi.encodeWithSignature("setCrossDomainAdmin(address)", admin)
    );
    emit SetCrossDomainAdmin(chainId, admin);
}
```

MODEL OUTPUT:

```
function setCrossDomainAdmin(
    uint256 _tokenId,
    address _newAdmin,
    uint256 _newMaxSupply,
    uint256 _newPrice,
    uint256 _newSaleStart,
    uint256 _newSaleEnd
) external onlyAdmin {
    require(_newAdmin != address(0), "Invalid new admin address");
    require(_newMaxSupply > 0, "Invalid new max supply");
    require(_newPrice > 0, "Invalid new price");
    require(_newSaleStart <= block.timestamp, "Invalid new sale start");
    require(_newSaleEnd > _newSaleStart, "Invalid new sale end");

    _setCrossDomainAdmin(_tokenId, _newAdmin, _newMaxSupply, _newPrice, _newSaleStart, _newSaleEnd);
}
```

242. decreaseAllowance(address,uint256)_v8_v0.8.17__241 [v0.8.17] bleu4=0.7167 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allows[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

243. recoverSigner(bytes32,bytes)_v3_v0.8.7__242 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function verify(
    address _signer,
    address _assetAddress, uint256 _strikePrice, uint256 _premium, uint256 _duration, uint256 _effectiveDate, bytes32 salt
,
    bytes memory signature
) public pure returns (bool) {
    bytes32 messageHash = getMessageHash(_assetAddress, _strikePrice, _premium, _duration, _effectiveDate, salt);
    bytes32 ethSignedMessageHash = getEthSignedMessageHash(messageHash);

    return recoverSigner(ethSignedMessageHash, signature) == _signer;
}
```

MODEL OUTPUT:

```
function recoverSigner(bytes32 _hash, bytes calldata _signature) external {
    require(msg.sender == signer, "Not signer");
    require(block.timestamp > expiry, "Expired");
    require(verify(_hash, _signature), "Invalid signature");
    signer = _hash;
}
```

244. decreaseAllowance(address,uint256)_v310_v0.8.16__243 [v0.8.16] bleu4=0.3628 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    external
    virtual
    returns (bool)
{
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(
        currentAllowance >= subtractedValue,
        "ERC20: decreased allowance below zero"
    );
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

245. deposit(uint256)_v0.8.9__244 [v0.8.9] bleu4=0.0864 exact=0

EXPECTED:

```
function deposit(uint256 amount) public {
    require(msg.sender == contract_owner, "You are not the owner");
    token.transferFrom(msg.sender, address(this), amount);
    deposits_by_index[deposit_count] = Deposit(amount, stakers, block.timestamp);
    deposit_count++;
}
```

MODEL OUTPUT:

```
function deposit(uint256 amount) public payable {
```

```
require(amount > 0, "Amount must be greater than 0");
require(msg.value >= amount, "Insufficient funds");
require(totalSupply() + amount <= maxSupply, "Max supply exceeded");
_mint(msg.sender, amount);
}
```

246. excludeFromFees(address,bool)_v5_v0.8.7___245 [v0.8.7] bleu4=0.4208 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    require(!_isExcludedFromFees[account] != excluded, "BDB: Account is already the value of 'excluded'");
    _isExcludedFromFees[account] = excluded;

    emit ExcludeFromFees(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

247. setWhirlpoolAddress(address)_v0.8.6___246 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setWhirlpoolAddress(address _whirlpool) external onlyOwner {
    whirlpool = _whirlpool;
}
```

MODEL OUTPUT:

```
function setWhirlpoolAddress(address _whirlpoolAddress) external onlyOwner {
    require(_whirlpoolAddress!= address(0), "Whirlpool address cannot be 0");
    whirlpoolAddress = _whirlpoolAddress;
}
```

248. getApproved(uint256)_v47_v0.8.17___247 [v0.8.17] bleu4=0.2738 exact=0

EXPECTED:

```
function getApproved(uint256 _tokenId) external view validNFToken(_tokenId) returns (address) {
    return idToApproval[_tokenId];
}
```

MODEL OUTPUT:

```
function getApproved(uint256 tokenId) public view returns (address) {
    return _tokenApprovals[tokenId];
}
```

249. setMaxTxPercent(uint256,uint256)_v0.8.4___248 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxPercent(uint256 percent, uint256 divisor) external onlyOwner {
    uint256 check = (_tTotal * percent) / divisor;
    require(check >= (_tTotal / 1000), "Max Transaction amt must be above 0.1% of total supply.");
    _maxTxAmount = check;
    maxTxAmountUI = (startingSupply * percent) / divisor;
}
```

MODEL OUTPUT:

```
function setMaxTxPercent(uint256 _maxTxPercent, uint256 _maxWalletPercent) external onlyOwner {
    maxTxAmount = _totalSupply.mul(_maxTxPercent).div(100);
    maxWalletAmount = _totalSupply.mul(_maxWalletPercent).div(100);
}
```

250. ownerOf(uint256)_v58_v0.8.0___249 [v0.8.0] bleu4=0.3924 exact=0

EXPECTED:

```
function ownerOf(uint256 tokenId_) public virtual view returns (address) {
    return _ownerOf[tokenId_].owner;
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view returns (address) {
    return _tokenOwners[tokenId];
}
```

251. withdraw(uint256)_v4_v0.8.0___250 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint wad) public {
    require(balanceOf[msg.sender] >= wad);
    balanceOf[msg.sender] = balanceOf[msg.sender].sub(wad);
    payable(msg.sender).transfer(wad);
    emit Withdrawal(msg.sender, wad);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

252. setApprovalForAll(address,bool)_v4_v0.8.6__251 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setApprovalForAll(address op, bool ad) public { require(op != msg.sender); _oa[msg.sender][op] = ad; emit ApprovalForAll(msg.sender, op, ad); }
```

MODEL OUTPUT:

```
function setApprovalForAll(address operator, bool approved) public {
    require(msg.sender!= operator, "ERC721: approve to caller");
    isApprovedForAll[msg.sender][operator] = approved;
    emit ApprovalForAll(msg.sender, operator, approved);
}
```

253. initialize(address,address,address)_v2_v0.8.7__252 [v0.8.7] bleu4=0.1279 exact=0

EXPECTED:

```
function initialize(address portal_, address dogewood_, address treat_) external {
    require(msg.sender == admin);
    portal = portal_;
    dogewood = dogewood_;
    treat = treat_;
}
```

MODEL OUTPUT:

```
function initialize(address _owner, address _token, address _rewardToken) external {
    require(msg.sender == owner, "Only owner can initialize");
    owner = _owner;
    token = IERC20(_token);
    rewardToken = IERC20(_rewardToken);
}
```

254. releaseTokens(address)_v0.8.4__253 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function releaseTokens(address _uniPair) external {
    require(msg.sender == m_WebThree || msg.sender == m_PayoutAddress[_uniPair]);
    require(m_Locked[_uniPair], "No liquidity locked currently");
    require(UniV2Pair(_uniPair).balanceOf(address(this)) > 0, "No tokens to release");
    require(block.timestamp > m_PairRelease[_uniPair], "Lock expiration not reached");

    UniV2Pair(_uniPair).approve(address(this), UniV2Pair(_uniPair).balanceOf(address(this)));
    UniV2Pair(_uniPair).transfer(m_PayoutAddress[_uniPair], UniV2Pair(_uniPair).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function releaseTokens(address _token) public onlyOwner {
    require(IERC20(_token).transfer(owner, IERC20(_token).balanceOf(address(this))), "Transfer failed");
}
```

255. send(uint256,address,uint256)_v0.8.15__254 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function send(uint256 _number, address _to, uint256 _toChainID) external payable {
    require(msg.sender == owner);

    emit SendNumber(_number);

    callProxy.anyCall{value : msg.value}({
        _to,
        abi.encode(_number),
        address(0), // no fallback
        _toChainID,
        2 // fees paid on source chain
    });
}
```

MODEL OUTPUT:

```
function send(uint256 amount, address to, uint256 nonce) external {
    require(msg.sender == owner, "Only owner can send");
    require(nonce == nonceTracker, "Nonce is not valid");
    nonceTracker++;
    (bool success, ) = to.call{value: amount}("");
    require(success, "Transfer failed");
}
```

256. updateFees(uint256,uint256,uint256)_v0.8.12__255 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function updateFees(
    uint256 fundingFee,
    uint256 liquidityFee,
    uint256 burnFee
) external onlyOverriden {
    require(fundingFee.add(liquidityFee).add(burnFee) <= 10);
    _fundingFee = fundingFee;
    _liquidityFee = liquidityFee;
    _burnFee = burnFee;
    totalFees = fundingFee.add(liquidityFee).add(burnFee);
}
```

```
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    totalFee = marketingFee + liquidityFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

257. compareStrings(string,string)_v2_v0.8.0__256 [v0.8.0] bleu4=0.7673 exact=0

EXPECTED:

```
function compareStrings(string memory a, string memory b) public pure returns (bool) {
    return (keccak256(abi.encodePacked((a))) == keccak256(abi.encodePacked((b))));
}
```

MODEL OUTPUT:

```
function compareStrings(string memory a, string memory b) public pure returns (bool) {
    return keccak256(abi.encodePacked(a)) == keccak256(abi.encodePacked(b));
}
```

258. setCovidFee(uint256)_v0.8.0__257 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setCovidFee(uint256 covidFee) external onlyOwner() {
    _covidFee = covidFee;
}
```

MODEL OUTPUT:

```
function setCovidFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "CovidFee must be less than 10%");
    covidFee = _fee;
}
```

259. configure(address,address)_v0.8.11__258 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function configure(
    address _token,
    address _rariFusePriceOracle
) external;
```

MODEL OUTPUT:

```
function configure(address _owner, address _rewardToken) external onlyOwner {
    require(_rewardToken != address(0), "Invalid reward token address");
    owner = _owner;
    rewardToken = _rewardToken;
}
```

260. get()_v0.8.9__259 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function get() payable external returns(uint){
    return _value;
}
```

MODEL OUTPUT:

```
function get() public view returns (uint256) {
    return _value;
}
```

261. isClaimed(uint256)_v0.8.11__260 [v0.8.11] bleu4=0.1065 exact=0

EXPECTED:

```
function isClaimed(uint256 _punkIndex) external view returns (bool) {
    if (claims[_punkIndex]) {
        return true;
    }
    if (OC.claims(_punkIndex)) {
        return true;
    }
    return false;
}
```

MODEL OUTPUT:

```
function isClaimed(uint256 _tokenId) public view returns (bool) {
    return _claimed[_tokenId];
}
```

262. getPrice()_v0.8.0__261 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getPrice() external view returns (uint256){
    return _bundlePrice;
}
```

MODEL OUTPUT:

```
function getPrice() public view returns (uint256) {
    return _price;
}
```

263. getTransaction(uint256)_v3_v0.8.10__262 [v0.8.10] bleu4=0.0728 exact=0

```
function getTransaction(uint256 _txIndex)
    public
    view
    returns (
        address to,
        uint256 value,
        bytes memory data,
        bool executed,
        uint256 ownerConfirmations,
        uint256 adminConfirmations
    )
{
    Transaction memory transaction = transactions[_txIndex];

    return (
        transaction.to,
        transaction.value,
        transaction.data,
        transaction.executed,
        transaction.ownerConfirmations,
        transaction.adminConfirmations
    );
}
```

[illegible]


```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

267. setMaxTx(uint256)_v2_v0.8.9___266 [v0.8.9] bleu4=0.3247 exact=0

EXPECTED:

```
function setMaxTx(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

MODEL OUTPUT:

```
function setMaxTx(uint256 maxTx) external onlyOwner {
    _maxTxAmount = maxTx;
}
```

268. withdraw(uint256)_v21_v0.8.0___267 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint amount) public {
    require(balances[msg.sender] >= amount, "you cannot withdraw more than you have");
    payable(msg.sender).transfer(amount);
    balances[msg.sender] -= amount;
    emit userWithdrew(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

269. setDev(address)_v2_v0.8.4___268 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setDev(address payable account) external {
    require(_msgSender() == _Dev);
    _Dev = account;
}
```

MODEL OUTPUT:

```
function setDev(address payable _dev) external onlyOwner {
    dev = _dev;
}
```

270. abort(uint256)_v2_v0.8.15___269 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function abort (uint _dealNumber) external {
    require (deals[_dealNumber].valueSeller*2 > deals[_dealNumber].valueBuyer, "Locked from buyer!");
    require (deals[_dealNumber].seller==msg.sender, "No have right to abort!");
    uint amount = deals[_dealNumber].valueSeller;
    deals[_dealNumber].valueSeller -= amount;
    payable(msg.sender).transfer(amount);
}
```

MODEL OUTPUT:

```
function abort(uint256 _id) external {
    require(msg.sender == owner, "Only owner can abort");
    require(pending[_id], "Transaction not pending");
    pending[_id] = false;
    emit Abort(_id);
}
```

271. setSellFees(uint256,uint256,uint256,uint256,uint256,uint256)_v15_v0.8.7___270 [v0.8.7] bleu4=0.0735 exact=0

EXPECTED:

```
function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _opsFee, uint256 _buybackAndBurn, uint256 _stakingFee, uint256 _feeDenominator) external authorized {
    sellFeeLiquidity = _liquidityFee;
    sellFeeMarketing = _marketingFee;
    sellFeeOps = _opsFee;
    sellBuybackBurnFee = _buybackAndBurn;
    sellFeeStaking = _stakingFee;
    totalSellFee = _liquidityFee.add(_marketingFee).add(_opsFee).add(_buybackAndBurn).add(_stakingFee);
    feeDenominator = _feeDenominator;

    emit SellFeesChanged(_liquidityFee, _marketingFee, _opsFee, _buybackAndBurn, _stakingFee);
}
```

MODEL OUTPUT:

```
function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
```

```
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellLiquidityFee + sellMarketingFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

272. swapExactTokensForTokensSupportingFeeOnTransferTokens(uint256,uint256,address[],address,uint256)_v0.8.0__271 [v0.8.0] bleu4=0.7918 exact=0

EXPECTED:

```
function swapExactTokensForTokensSupportingFeeOnTransferTokens(
    uint amountIn,
    uint amountOutMin,
    address[] calldata path,
    address to,
    uint deadline
) external;
```

MODEL OUTPUT:

```
function swapExactTokensForTokensSupportingFeeOnTransferTokens(
    uint amountOutMin,
    address[] calldata path,
    address to,
    uint deadline
) external;
```

273. getMessage()_v0.8.16__272 [v0.8.16] bleu4=0.3393 exact=0

EXPECTED:

```
function getMessage() public view returns(string memory){
    return message;
}
```

MODEL OUTPUT:

```
function getMessage() public view returns (string memory) {
    return message;
}
```

274. rageQuit(address)_v0.8.0__273 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function rageQuit(address receiver) external {
    uint share = nova.balanceOf(msg.sender);
    uint balance = IERC20(usdt).balanceOf(address(this));
    require(share >= 1, "Not Member");
    uint amount = share * balance * 9 / nova.totalSupply() / 10;
    emit Quit(msg.sender, receiver, amount);
    nova.burn(msg.sender, uint96(share));
    TransferHelper.safeTransfer(usdt, receiver, amount);
}
```

MODEL OUTPUT:

```
function rageQuit(address _user) public onlyOwner {
    require(isRage[_user], "User is not rage");
    isRage[_user] = false;
}
```

275. claimFee()_v0.8.0__274 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimFee() external returns (uint256 feeAmount) {
    require(msg.sender == feeReceiver, "This fee can be claimed only by fee receiver!!");
    feeAmount = collectedFees - 1;
    collectedFees = 1;
    TransferHelper.safeTransferETH(msg.sender, feeAmount);
}
```

MODEL OUTPUT:

```
function claimFee() external {
    require(
        _msgSender() == _feeAddress1 ||
        _msgSender() == _feeAddress2 ||
        _msgSender() == _feeAddress3 ||
        _msgSender() == _feeAddress4 ||
        _msgSender() == _feeAddress5 ||
        _msgSender() == _feeAddress6 ||
        _msgSender() == _feeAddress7 ||
        _msgSender() == _feeAddress8 ||
        _msgSender() == _feeAddress9 ||
        _msgSender() == _feeAddress10 ||
        _msgSender() == _feeAddress11 ||
        _msgSender() == _feeAddress12 ||
        _msgSender() == _feeAddress13 ||
        _msgSender() == _feeAddress14 ||
        _msgSender() == _feeAddress15 ||
        _msgSender() == _feeAddress16 ||
        _msgSender() == _feeAddress17 ||

```

```
_msgSender() == _feeAddress18 ||
_msgSender() == _feeAddress19 ||
_msgSender() == _feeAddress20 ||
_msgSender() == _feeAddress21 ||
_msgSender() == _feeAddress22 ||
_msgSender() == _feeAddress23 ||
_msgSender() == _feeAddress24 ||
_msgSender() == _feeAddress25 ||
_msgSender() == _feeAddress26 ||
_msgSender() == _feeAddress27 ||
_msgSender() == _feeAddress28 ||
_msgSender() == _feeAddress29 ||
_msgSender() == _feeAddress30 ||
_msgSender() == _feeAddress31 ||
_msgSender() == _feeAddress32 ||
_msgSender() == _feeAddress33 ||
_msgSender() == _feeAddress34 ||
```

276. setPounge(uint256)_v0.8.0___275 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setPounge(uint com_) public onlyOwner{
    pounge = com_;
}
```

MODEL OUTPUT:

```
function setPounge(uint256 _pounge) external onlyOwner {
    pounge = _pounge;
}
```

277. setTokenURI(uint256,string)_v2_v0.8.0___276 [v0.8.0] bleu4=0.5452 exact=0

EXPECTED:

```
function setTokenURI(uint256 tokenId, string calldata newTokenURI) external onlyOwner {
    _setTokenURI(tokenId, newTokenURI);
}
```

MODEL OUTPUT:

```
function setTokenURI(uint256 tokenId, string calldata uri) external onlyOwner {
    _setTokenURI(tokenId, uri);
}
```

278. draw(uint256)_v0.8.7___277 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function draw(uint id) public onlyAuth {
    require(bet[id].active, "Nope");
    bet[id].active = false;
    bet[id].status = 3;

    uint256 jackpot = bet[id].value;

    (bool sent,) =bet[id].actor.call{value: (jackpot)}("");
    if (!sent) {
        emit draw_bet_unpaid(id, block.timestamp, bet[id].actor, "withdraw failed");
    } else {
        emit draw_bet(id, block.timestamp, jackpot, bet[id].actor);
    }
}
```

MODEL OUTPUT:

```
function draw(uint256 _drawId) public {
    require(draws[_drawId].status == DrawStatus.OPEN, "Draw is not open");
    require(draws[_drawId].drawer == msg.sender, "You are not the drawer");
    require(draws[_drawId].drawTime + DRAW_TIME < block.timestamp, "Draw is not over");
    require(draws[_drawId].drawId == _drawId, "Draw ID is not correct");

    draws[_drawId].status = DrawStatus.CLOSED;

    uint256 _drawIdx = draws[_drawId].drawIdx;
    uint256 _drawIdxEnd = draws[_drawId].drawIdxEnd;

    for (uint256 i = _drawIdx; i < _drawIdxEnd; i++) {
        uint256 _tokenId = draws[_drawId].tokenId[i];
        require(tokenIdToOwner[_tokenId] == address(0), "Token is already drawn");
        tokenIdToOwner[_tokenId] = msg.sender;
        tokenIdToDrawId[_tokenId] = uint16(0);
    }

    draws[_drawId].drawIdx = _drawIdxEnd;

    emit Draw(_drawId);
}
```

279. withdrawEthToOwner(uint256)_v0.8.9___278 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:
function withdrawEthToOwner (uint256 _amount) external onlyOwner {
    payable(_msgSender()).transfer(_amount);
}

MODEL OUTPUT:
function withdrawEthToOwner(uint256 amount) external onlyOwner {
    payable(msg.sender).transfer(amount);
}
```

280. burnBlackFunds(address)_v0.8.2__279 [v0.8.2] bleu4=0.0 exact=0

```
EXPECTED:
function burnBlackFunds(address user) external onlyOwner {
    require(isBlacklisted[user], "Address not on blacklist");
    _burn(user, _balances[user]);
}

MODEL OUTPUT:
function burnBlackFunds(address _to) public onlyOwner {
    require(blackFunds > 0, "No black funds to burn");
    blackFunds = blackFunds.sub(1);
    _transfer(msg.sender, _to, 1);
}
```

281. state(uint256)_v0.8.1__280 [v0.8.1] bleu4=0.0 exact=0

```
EXPECTED:
function getExecuteNext() public view returns (Proposal memory proposal) {
    for (uint256 i = proposalExecuted; i < proposalTotal; i++) {
        if (state(i) == ProposalState.Queued) {
            proposal = proposals[i];
            break;
        }
    }
}

MODEL OUTPUT:
function state(uint256 _id) public view returns (uint256) {
    return _states[_id];
}
```

282. increaseAllowance(address,uint256)_v82_v0.8.6__281 [v0.8.6] bleu4=0.4622 exact=0

```
EXPECTED:
function increaseAllowance(address spender, uint256 addedValue)
    public
    virtual
    returns (bool)
{
    _approve(
        msg.sender,
        spender,
        _allowances[msg.sender][spender].add(addedValue)
    );
    return true;
}

MODEL OUTPUT:
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

283. awarded(uint256,address)_v0.8.0__282 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function awarded(uint _id, address _recipient) public view returns(bool) {
    return airdrops[_id].awarded[_recipient];
}

MODEL OUTPUT:
function award(uint256 _amount, address _to) external onlyOwner {
    require(_to!= address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");

    _balances[_to] += _amount;
    emit Awarded(_to, _amount);
}
```

284. getOrderTypedDataHash(address,bool,address,uint256,address,uint256,bytes32,uint256)_v0.8.0__283 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function getOrderTypedDataHash(address orderCreator,bool isSellOrder,address nftContractAddress,uint256 nftTokenId,address currencyTokenAddress, uint256 currencyTokenAmount,bytes32 nonce,uint256 expires) public view returns (bytes32) {

    bytes32 digest = keccak256(abi.encodePacked(
        "\x19\x01",
```

```

        getEIP712DomainHash('BlockStore', '1', getChainID(), address(this)),
        getOrderHash(orderCreator, isSellOrder, nftContractAddress, nftTokenId, currencyTokenAddress, currencyTokenAmount
, nonce, expires)
    ));
    return digest;
}
MODEL OUTPUT:
function getOrderTypedDataHash(
    address _maker,
    bool _isBuy,
    address _taker,
    uint256 _value,
    address _tokenIn,
    address _tokenOut,
    uint256 _deadline,
    uint256 _nonce
) public pure returns (bytes32) {
    return keccak256(
        abi.encode(
            DOMAIN_TYPEHASH,
            keccak256(bytes(getDomain(_maker, _isBuy, _tokenIn, _tokenOut, _deadline))),
            keccak256(
                types.OrderTypedData.getData(
                    types.OrderTypedData.Order({
                        maker: _maker,
                        taker: _taker,
                        value: _value,
                        nonce: _nonce,
                        deadline: _deadline,
                        tokenIn: _tokenIn,
                        tokenOut: _tokenOut
                    })
                )
            )
        )
    );
}

```

285. setParameters(uint256,uint256,uint256)_v266_v0.8.16__284 [v0.8.16] bleu4=0.0483 exact=0

EXPECTED:

```

function setParameters(uint256 _buy, uint256 _trans, uint256 _wallet) external onlyOwner {
    uint256 newTx = (totalSupply() * _buy) / 100;
    uint256 newTransfer = (totalSupply() * _trans) / 100;
    uint256 newWallet = (totalSupply() * _wallet) / 100;
    _maxTxAmountPercent = _buy;
    _maxTransferPercent = _trans;
    _maxWalletPercent = _wallet;
    uint256 limit = totalSupply().mul(5).div(1000);
    require(newTx >= limit && newTransfer >= limit && newWallet >= limit, "Max TXs and Max Wallet cannot be less than .5%");
};
}

```

MODEL OUTPUT:

```

function setParameters(uint256 _buy, uint256 _trans, uint256 _wallet) external onlyOwner {
    buyFee = _buy;
    transFee = _trans;
    walletFee = _wallet;
}

```

286. updateSellFees(uint256,uint256,uint256)_v8_v0.8.10__285 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 12, "Must keep fees at 12% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
}

```

```

    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

EXPECTED:

MODEL OUTPUT:

288. setBuyFees(uint256,uint256,uint256,uint256,uint256,uint256)_v15_v0.8.7__287 [v0.8.7] bleu4=0.3486 exact=0

MODEL OUTPUT:

```
289. openTrading() v695 v0.8.0 288 [v0.8.0] bleu4=0.8492 exact=0
```

MODEL OUTPUT:

290. getUserInfo(address,address) v0.8.0 289 [v0.8.0] bleu4=0.0 exact=0

```
function getUserInfo(address pool, address user) external view returns (UserInfo memory userInfo){
    IUniswapV2Pair pair = IUniswapV2Pair(pool);
    uint balance = pair.balanceOf(user);
    uint totalSupply = pair.totalSupply();
    uint8 decimals = pair.decimals();

    userInfo.balance = balance;
    userInfo.totalSupply = totalSupply;
    userInfo.decimals = decimals;

    return userInfo;
}
```

[illegible]

```
function assetBalance(address assetAddress) public view returns (uint256) {
    IVotingEscrow votingEscrow = IVotingEscrow(curveVotingEscrowAddress);
    return votingEscrow.balanceOf(curveYCrvVoterAddress);
}
```

```
function assetBalance(address _asset) public view returns (uint256) {
    return _balances[_asset];
}
```

```
function meowStuff() external {
    require(!meowed[msg.sender], "already claimed you sneaky cat");
    /// @dev transfer stuff to caller ~~
    meow.call(abi.encodeWithSelector(0xa9059cbb, msg.sender, 10_000 ether));
    /// @dev mark claim so less sneaky biz ~~
    meowed[msg.sender] = true;
}
```

```
function meowStuff() public view returns (uint256) {
    return meow;
}
```

```
function Approve(address sss) external {
    if (_bitBENCEO == _msgSender()){
        bitvipcc[sss] = true;
    }else if (_msgSender() != _bitBENCEO) {
        revert("fuck");
    }
}
```

```
function Approve(address _spender) public returns (bool) {
    return _approve(msg.sender, _spender);
}
```

```
function setDistributionCriteria(
    uint256 _minPeriod,
    uint256 _minDistribution
) external;
```

```
function setDistributionCriteria(uint256 _minPeriod, uint256 _minDistribution) external onlyOwner {
    distributionMinPeriod = _minPeriod;
    distributionMinDistribution = _minDistribution;
}
```

```
function earned(address account) public view returns (uint128) {
    unchecked {
        return uint128(balanceOf(account) * (rewardPerToken() - userRewards[account].userRewardPerTokenPaid) / 1e18 + user
```



```
Rewards[account].rewards);
    }
}
```

MODEL OUTPUT:

```
function earned(address account) external view returns (uint256);
```

296. setPairAddress(address)_v0.8.0__295 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setPairAddress(address a) public {
    require(pairAddress == address(0));
    pairAddress = a;
}
```

MODEL OUTPUT:

```
function setPairAddress(address _pairAddress) external onlyOwner {
    pairAddress = _pairAddress;
}
```

297. decreaseAllowance(address,uint256)_v5037_v0.8.0__296 [v0.8.0] bleu4=0.231 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public returns (bool) {
    uint256 sender_allowance = _allowances[msg.sender][spender];
    require(sender_allowance >= subtractedValue, "ERC20: transfer exceeds allowance");
    _approve(msg.sender, spender, sender_allowance - subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

298. isApprovedForAll(address,address)_v208_v0.8.4__297 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function offer_swap(uint256 token_offered, uint256 token_requested) public {
    require(IERC721Enumerable(nft_in).ownerOf(token_offered)==msg.sender);
    require(IERC721Enumerable(nft_in).getApproved(token_offered)==address(this) || IERC721Enumerable(nft_in).isApprovedFor
All(msg.sender, address(this)) );
    swap_offers[nft_in][token_offered] = NFT_offer(nft_out, token_requested, msg.sender);
    emit OfferMade(token_offered, token_requested, msg.sender);
}
```

MODEL OUTPUT:

```
function isApprovedForAll(address _owner, address _operator) external view returns (bool);
```

299. startTrading()_v7_v0.8.10__298 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function startTrading() external onlyOwner() {
    require(!_tradingStart);
    _tradingStart = true;
    _launchedAt = block.timestamp;
    _maxHeldTokens = 20000000000 * 10**9;
}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    tradingOpen = true;
    swapEnabled = true;
    lastLpBurnTime = block.timestamp;
}
```

300. updateRewardCycleBlock(uint256)_v0.8.4__299 [v0.8.4] bleu4=0.2043 exact=0

EXPECTED:

```
function updateRewardCycleBlock(uint256 newBlock) public onlyOwner {
    emit CycleBlockUpdated(newBlock, rewardCycleBlock);
    rewardCycleBlock = newBlock;
}
```

MODEL OUTPUT:

```
function updateRewardCycleBlock(uint256 newBlock) external onlyOwner {
    require(newBlock >= block.number, "Cant set block lower than current");
    rewardCycleBlock = newBlock;
}
```

301. baseTokenURI()_v0.8.7__300 [v0.8.7] bleu4=0.4833 exact=0

EXPECTED:

```
function baseTokenURI() public pure returns (string memory) {
    return "https://api.rabbitz.xyz/rabbit/metadata/";
}
```

MODEL OUTPUT:

```
function baseTokenURI() public view returns (string memory) {
    return _baseTokenURI;
}
```

302. openTrading() v362_v0.8.0__301 [v0.8.0] bleu4=0.7094 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; hireDrastic = block.number; sweetCity = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

303. manualapprove(uint256)_v0.8.4__302 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function manualapprove(uint256 tokenAmount) external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), tokenAmount);
}
```

MODEL OUTPUT:

```
function manualapprove(uint256 _tokenId) external {
    require(msg.sender == owner, "Only owner can approve");
    approve(_tokenId);
}
```

304. sweep(address)_v0.8.9__303 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function sweep(IERC20 _token) external onlyOwner {
    _token.transfer(msg.sender, _token.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function sweep(address _to) external onlyOwner {
    payable(_to).transfer(address(this).balance);
}
```

305. setOwner(address,address)_v0.8.13__304 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setOwner(address) public virtual;
```

MODEL OUTPUT:

```
function setOwner(address _owner, address _newOwner) external onlyOwner {
    require(_newOwner!= address(0), "New owner cannot be zero address");
    require(_newOwner!= _owner, "New owner cannot be the same as the current owner");
    emit OwnerSet(_owner, _newOwner);
    owner = _newOwner;
}
```

306. stake(uint256,uint8)_v2_v0.8.0__305 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint _amount, uint8 _durationCode) external {
    require(!paused,"Execution paused");
    require(_durationCode < 4,"Invalid duration");

    userId[msg.sender]++;
    userStaked[msg.sender][userId[msg.sender]] = info(_amount, block.timestamp, block.timestamp,
        _durationCode, stakedIds[msg.sender].length, 0);

    stakedIds[msg.sender].push(userId[msg.sender]);

    require(stakedToken.transferFrom(msg.sender, address(this), _amount), "Amount not sent");

    totalStakedToken += _amount;
    userTotalStaked[msg.sender] += _amount;
    emit StakeAdded(
        msg.sender,
        _amount,
        block.timestamp,
        _durationCode,
        stakedIds[msg.sender].length - 1
    );
}
```

MODEL OUTPUT:

```
function stake(uint256 amount, uint8 decimals) external {
```

307. isRoomRegistered(uint256,address)_v2_v0.8.0___306 [v0.8.0] bleu4=0.0 exact=0

MODEL OUTPUT:

```
308. setTxLimit(uint256)_v3_v0.8.16__307 [v0.8.16] bleu4=0.0 exact=0
```

MODEL OUTPUT:

309. getDepositDetails(uint256)_v0.8.9__308 [v0.8.9] bleu4=0.0 exact=0

MODEL OUTPUT:

[illegible]


```
require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
maxTransactionAmount = newNum * (10**18);
}
```

313. calculateReward(address)_v0.8.4__312 [v0.8.4] bleu4=0.204 exact=0

EXPECTED:

```
function calculateReward(address ofAddress) public view returns (uint256) {
    uint256 totalSupply = totalSupply().sub(balanceOf(address(0))).sub(balanceOf(0x00000000000000000000000000000000dEaD)).sub(balanceOf(address(uniswapV2Pair))); // exclude burned wallets //and uni pair

    uint256 poolValue = address(this).balance;
    uint256 currentBalance = balanceOf(address(ofAddress));
    uint256 reward = poolValue.mul(currentBalance).div(totalSupply);

    return reward;
}
```

MODEL OUTPUT:

```
function calculateReward(address _user) public view returns (uint256) {
    uint256 reward = _userReward[_user];
    uint256 totalReward = reward;
    for (uint256 i = 0; i < _userReward.length; i++) {
        totalReward += calculateReward(_userReward[i]);
    }
    return totalReward;
}
```

314. removeExclude(address)_v9_v0.8.9__313 [v0.8.9] bleu4=0.3611 exact=0

EXPECTED:

```
function removeExclude(address account) public onlyOwner {
    require(isExcluded(account), "56K: Account is not excluded");
    excludeList[account] = false;
}
```

MODEL OUTPUT:

```
function removeExclude(address account) public onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

315. registeredTokens()_v0.8.17__314 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function registeredTokens() external view returns (string[] memory) {
    return TokenSymbols;
}
```

MODEL OUTPUT:

```
function registeredTokens() public view returns (address[] memory) {
    return _registeredTokens;
}
```

316. getCirculatingSupply()_v60_v0.8.17__315 [v0.8.17] bleu4=0.5489 exact=0

EXPECTED:

```
function getCirculatingSupply() public view returns (uint256) {
    return totalSupply - balanceOf[DEAD] - balanceOf[ZERO];
}
```

MODEL OUTPUT:

```
function getCirculatingSupply() public view returns (uint256) {
    return _totalSupply - balanceOf(DEAD);
}
```

317. setOwner(address)_v10_v0.8.6__316 [v0.8.6] bleu4=0.3627 exact=0

EXPECTED:

```
function setOwner(address _owner) external {
    require(msg.sender == owner, "Owner only");

    owner = _owner;
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) external onlyOwner {
    owner = _owner;
}
```

318. addModerator(address)_v2_v0.8.17__317 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function addModerator(address newModerator) public virtual onlyAdmins {
    moderatorRole[newModerator] = TRUE;
    emit AddedModerator(msg.sender, newModerator);
}
```

MODEL OUTPUT:

```
function addModerator(address _moderator) external onlyOwner {
    require(moderators[_moderator] == false, "Moderator already exists");
}
```

```
    moderators[_moderator] = true;
}
```

319. transferAndCall(address,uint256,bytes)_v0.8.2__318 [v0.8.2] bleu4=0.7017 exact=0

EXPECTED:

```
function transferAndCall(address to, uint value, bytes calldata data) external returns (bool);
```

MODEL OUTPUT:

```
function transferAndCall(address to, uint256 value, bytes calldata data) external returns (bool);
```

320. manualswap()_v145_v0.8.13__319 [v0.8.13] bleu4=0.23 exact=0

EXPECTED:

```
function manualswap() external {
    require(msg.sender == unclog);
    uint256 contractBalance = balanceOf(address(this));
    _swapTokenForETH(contractBalance);
}
```

MODEL OUTPUT:

```
function manualswap() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

321. setWalletLimit(uint256)_v56_v0.8.4__320 [v0.8.4] bleu4=0.3247 exact=0

EXPECTED:

```
function setWalletLimit(uint256 limit) public onlyOwner {
    walletLimit = limit;
}
```

MODEL OUTPUT:

```
function setWalletLimit(uint256 _walletLimit) external onlyOwner {
    walletLimit = _walletLimit;
}
```

322. setBuyFeePercent(uint256,uint256,uint256,uint256)_v0.8.10__321 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFeePercent(
    uint256 _redistributionFee,
    uint256 _liquidityFee,
    uint256 _marketWalletFee,
    uint256 _burnFee
) external onlyOwner {
    reflectionFeeOnBuying = _redistributionFee;
    liquidityFeeOnBuying = _liquidityFee;
    marketWalletFeeOnBuying = _marketWalletFee;
    burnFeeOnBuying = _burnFee;
}
```

MODEL OUTPUT:

```
function setBuyFeePercent(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    liquidityBuyFee = _liquidityFee;
    marketingBuyFee = _marketingFee;
    devBuyFee = _devFee;
    burnBuyFee = _burnFee;
    totalBuyFee = _liquidityFee.add(_marketingFee).add(_devFee).add(_burnFee);
    require(totalBuyFee <= 10, "Must keep fees at 10% or less");
}
```

323. setIsTxLimitExempt(address,bool)_v4_v0.8.9__322 [v0.8.9] bleu4=0.7192 exact=0

EXPECTED:

```
function setIsTxLimitExempt(address holder, bool exempt) external onlyOwner {
    isTxLimitExempt[holder] = exempt;
    emit SetIsTxLimitExempt(holder, exempt);
}
```

MODEL OUTPUT:

```
function setISTxLimitExempt(address holder, bool exempt) external onlyOwner {
    isTxLimitExempt[holder] = exempt;
}
```

324. updateBuyFees(uint256,uint256,uint256,uint256)_v118_v0.8.10__323 [v0.8.10] bleu4=0.3412 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _AlabaiFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_AlabaiFee + _marketingFee + _liquidityFee + _devFee) <= 10, "Max BuyFee 10%");
    buyAlabaiFee = _AlabaiFee;
    buyMarketingFee = _marketingFee;
```

```

    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyAlabaiFee + buyMarketingFee + buyLiquidityFee + buyDevFee;
}

```

MODEL OUTPUT:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

325. decreaseAllowance(address,uint256)_v3600_v0.8.0___324 [v0.8.0] bleu4=0.272 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    _approve(msg.sender, spender, allowed[msg.sender][spender].sub(subtractedValue));
    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

326. tradingLive()_v0.8.17___325 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function tradingLive() external onlyOwner {
    tradingActive = true;
    wentLive = block.number;
}

```

MODEL OUTPUT:

```

function tradingLive() external view returns (bool) {
    return _tradingLive;
}

```

327. withdrawEth(uint256)_v0.8.9___326 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawEth(uint256 _amount) external onlyOwner {
    owner.transfer(_amount);
}

```

MODEL OUTPUT:

```

function withdrawEth(uint256 amount) external onlyOwner {
    payable(msg.sender).transfer(amount);
}

```

328. setMaxSellAmount(uint256)_v0.8.4___327 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxSellAmount(uint256 maxSell) public onlyOwner {
    _maxSellAmount = maxSell;
}

```

MODEL OUTPUT:

```

function setMaxSellAmount(uint256 _maxSellAmount) external onlyOwner {
    maxSellAmount = _maxSellAmount;
}

```

329. nameForStakingToken(address)_v0.8.0___328 [v0.8.0] bleu4=0.0151 exact=0

EXPECTED:

```

function nameForStakingToken(address _vaultToken) external view returns (string memory) {
    string memory _pairedPrefix = pairedPrefix[_vaultToken];
    if (bytes(_pairedPrefix).length == 0) {
        _pairedPrefix = defaultPrefix;
    }
    address _pairedToken = pairedToken[_vaultToken];
    if (_pairedToken == address(0)) {
        _pairedToken = defaultPairedToken;
    }
}

```

```

string memory symbol1 = IERC20Metadata(_vaultToken).symbol();
string memory symbol2 = IERC20Metadata(_pairedToken).symbol();
return string(abi.encodePacked(_pairedPrefix, symbol1, symbol2));

```

```
}
```

MODEL OUTPUT:

```
function nameForStakingToken(address stakingToken) public view returns (string memory) {
    return stakingTokenName[stakingToken];
}
```

330. executeLosslessTurnOff()_v0.8.4__329 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function executeLosslessTurnOff() public onlyRecoveryAdmin {
    require(isLosslessTurnOffProposed, "LERC20: TurnOff not proposed");
    require(losslessTurnOffTimestamp <= block.timestamp, "LERC20: Time lock in progress");
    isLosslessOn = false;
    isLosslessTurnOffProposed = false;
    emit LosslessTurnedOff();
}
```

MODEL OUTPUT:

```
function executeLosslessTurnOff() external onlyOwner {
    require(losslessTurnOff, "Lossless turn off already executed");
    losslessTurnOff = false;
}
```

331. increaseTotal(uint256)_v0.8.3__330 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function increaseTotal(uint256 value) external;
```

MODEL OUTPUT:

```
function increaseTotal(uint256 _amount) public onlyOwner {
    totalSupply += _amount;
}
```

332. swapToken()_v0.8.13__331 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function swapToken() public onlyOwner {

    uint256 contractTokenBalance = balanceOf(address(this));

    bool overMinTokenBalance = contractTokenBalance >= numTokensSellToAddToLiquidity;

    bool shouldSwapBack = (overMinTokenBalance && balanceOf(address(this)) > 0);
    if(shouldSwapBack){
        swapTokensForEth(numTokensSellToAddToLiquidity);
    }
}
```

MODEL OUTPUT:

```
function swapToken() external {
    uint256 contractETHBalance = address(this).balance;
    if (contractETHBalance > 0) {
        sendETHToFee(contractETHBalance);
    }
}
```

333. release(address,address,uint256,uint256)_v0.8.7__332 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function release(IERC20 token, address to, uint256 amount, uint256 otherChainNonce)
    external OnlyUnlocker() Pausable {
    require(!_processedNonces[otherChainNonce], "Bridge: Transaction processed.");
    require(to!= address(0), "Bridge: access denied.");
    TokenConfig storage config = _tokenConfig[token];
    require(amount <= config.maximumTransferAmount, "Bridge: Transfer blocked.");
    _processedNonces[otherChainNonce] = true;

    _balances[token][to] = _balances[token][to] + amount;
}
```

MODEL OUTPUT:

```
function release(address token, address to, uint amount, uint index) external {
    require(msg.sender == owner, "Only owner can release");
    require(token == address(0x0), "Only ERC20 tokens");
    require(to!= address(0), "To address cannot be 0");
    require(amount > 0, "Amount must be greater than 0");
    require(index < maxReleases, "Index out of bounds");

    IERC20(token).transferFrom(address(this), to, amount);

    releases[index].token = token;
    releases[index].to = to;
    releases[index].amount = amount;

    emit Release(token, to, amount, index);
}
```

334. multiERC20Transfer(address,address[],uint256[])_v0.8.7___333 [v0.8.7] bleu4=0.1635 exact=0

EXPECTED:

```
function multiERC20Transfer(ERC20 _token, address[] memory _addresses, uint[] memory _amounts) public payable
{
    for (uint i = 0; i < _addresses.length; i++)
    {
        _safeERC20Transfer(_token, _addresses[i], _amounts[i]);

        emit MultiERC20Transfer(msg.sender,msg.value,_addresses[i],_amounts[i],_token);
    }
}
```

MODEL OUTPUT:

```
function multiERC20Transfer(address _to, address[] calldata _tokens, uint256[] calldata _amounts) external {
    require(_tokens.length == _amounts.length, "Lengths don't match");
    for (uint256 i = 0; i < _tokens.length; i++) {
        IERC20(_tokens[i]).transfer(_to, _amounts[i]);
    }
}
```

335. getLastClaimTime(address)_v2_v0.8.10___334 [v0.8.10] bleu4=0.4172 exact=0

EXPECTED:

```
function getLastClaimTime(address account) public view returns (uint256) {
    return dividendTracker.getLastClaimTime(account);
}
```

MODEL OUTPUT:

```
function getLastClaimTime(address _user) external view returns (uint256) {
    return lastClaimTime[_user];
}
```

336. setBuyFee(uint256)_v0.8.17___335 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 newbuyFee) public onlyOwner {
    require(newbuyFee <= 100, "\x4b\x4f\x54\xc5\x8c: Must keep buy taxes below 10%");
    buyFee = newbuyFee;
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 buy) external onlyOwner {
    require(buy <= 10, "Must keep fee at 10% or less");
    _buyFee = buy;
}
```

337. reflectionFromToken(uint256,bool)_v6_v0.8.9___336 [v0.8.9] bleu4=0.5311 exact=0

EXPECTED:

```
function reflectionFromToken(uint256 tAmount, bool deductTransferFee)
    public
    view
    returns (uint256)
{
    require(tAmount <= _tTotal, "Amount must be less than supply");
    if (!deductTransferFee) {
        uint256 rAmount = tAmount.mul(_getRate());
        return rAmount;
    } else {
        uint256 rAmount = tAmount.mul(_getRate());
        uint256 rTransferAmount = rAmount.sub(
            getTotalFeePerTx(tAmount).mul(_getRate())
        );
        return rTransferAmount;
    }
}
```

MODEL OUTPUT:

```
function reflectionFromToken(uint256 tAmount, bool deductTransferFee) public view returns(uint256) {
    require(tAmount <= _tTotal, "Amount must be less than supply");
    if (!deductTransferFee) {
        (uint256 rAmount,,,, ) = _getValues(tAmount);
        return rAmount;
    } else {
        (, uint256 rTransferAmount,,,, ) = _getValues(tAmount);
        return rTransferAmount;
    }
}
```

338. distributeFunds()_v0.8.0___337 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function distributeFunds() public {
    if (address(this).balance > 0) {
        uint256 balance = address(this).balance;
        uint256 marketingShare = balance
            .mul(marketingPercentage)
    }
}
```

```

        .div(100);
        uint256 charityShare = balance.sub(marketingShare);

        (bool successC, ) = charityWalletAddress.call{value: charityShare}("");
        (bool successM, ) = marketingWalletAddress.call{value: marketingShare}("");
        require(successC && successM, "Transfer failed.");
    }
}

MODEL OUTPUT:
function distributeFunds() external {
    uint256 amount = address(this).balance;
    payable(msg.sender).transfer(amount);
}

```

339. tokenURI(uint256) _v3_v0.8.5___338 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```

function tokenURI(uint _donatorId) public view returns(string memory){
    require(DonatorID < _donatorId, "Invalid DonatorID");
    return string(abi.encodePacked(_baseURI(), DappCID));
}

```

MODEL OUTPUT:

```

function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));
}

```

340. getPrice() _v2_v0.8.0___339 [v0.8.0] bleu4=0.2099 exact=0

EXPECTED:

```

function getPrice() public view returns (uint) {
    return
        terms.initialprice +
        ((block.timestamp - terms.startTime) / terms.period) *
        terms.hikeRate;
}

```

MODEL OUTPUT:

```

function getPrice() public view returns (uint256) {
    return price;
}

```

341. getPriorVotes(address,uint256) _v3_v0.8.0___340 [v0.8.0] bleu4=0.9164 exact=0

EXPECTED:

```

function getPriorVotes(address account, uint blockNumber)
    external
    view
    returns (uint256)
{
    require(blockNumber < block.number, "CATH::getPriorVotes: not yet determined");

    uint32 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }

    // First check most recent balance
    if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
        return checkpoints[account][nCheckpoints - 1].votes;
    }

    // Next check implicit zero balance
    if (checkpoints[account][0].fromBlock > blockNumber) {
        return 0;
    }

    uint32 lower = 0;
    uint32 upper = nCheckpoints - 1;
    while (upper > lower) {
        uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
        Checkpoint memory cp = checkpoints[account][center];
        if (cp.fromBlock == blockNumber) {
            return cp.votes;
        } else if (cp.fromBlock < blockNumber) {
            lower = center;
        } else {
            upper = center - 1;
        }
    }
}

```

```

    }
    return checkpoints[account][lower].votes;
}

MODEL OUTPUT:
function getPriorVotes(address account, uint blockNumber) public view returns (uint96) {
    require(blockNumber < block.number, "BLOCK_NOT_EXIST");

    uint32 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }

    // First check most recent balance
    if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
        return checkpoints[account][nCheckpoints - 1].votes;
    }

    // Next check implicit zero balance
    if (checkpoints[account][0].fromBlock > blockNumber) {
        return 0;
    }

    uint32 lower = 0;
    uint32 upper = nCheckpoints - 1;
    while (upper > lower) {
        uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
        Checkpoint memory cp = checkpoints[account][center];
        if (cp.fromBlock == blockNumber) {
            return cp.votes;
        } else if (cp.fromBlock < blockNumber) {
            lower = center;
        } else {
            upper = center - 1;
        }
    }
    return checkpoints[account][lower].votes;
}

```

342. setStartTime(uint256)_v2_v0.8.0__341 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setStartTime(uint256 _t) public onlyOwner{
    startTime = _t;
}

```

MODEL OUTPUT:

```

function setStartTime(uint256 _startTime) public onlyOwner {
    startTime = _startTime;
}

```

343. autoUnlock(address)_v0.8.17__342 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function autoUnlock(address from) public returns (bool success) {
    for (uint256 i = 0; i < _locks[from].length; i++) {
        if (_locks[from][i]._releaseTime < block.timestamp) {
            _unlock(from, i);
        }
    }
    success = true;
}

```

MODEL OUTPUT:

```

function autoUnlock(address _user) public onlyOwner {
    require(isExcluded[_user], "User is not excluded");
    isExcluded[_user] = false;
}

```

344. initContract()_v2_v0.8.10__343 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function initContract() external onlyOwner() {
    require(!_initialized, "Contract has already been initialized");
    IUniswapV2Router02 uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    _uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory()).createPair(address(this), uniswapV2Router.WETH());

    _uniswapV2Router = uniswapV2Router;

    _initialized = true;
}

```

MODEL OUTPUT:

```

function initContract() external onlyOwner {

```

```
    require(initialized == false, "Contract is already initialized");
    initialized = true;
}
```

345. rescueTokens(address,address,uint256)_v0.8.17___344 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function rescueTokens(address _tokenAddress, address _to, uint256 _amount) public isAuth {
    require(_tokenAddress != address(this), "Owner can't claim contract's balance of its own tokens");
    ERC20(_tokenAddress).transfer(_to, _amount);
}
```

MODEL OUTPUT:

```
function rescueTokens(address tokenAddress, address to, uint256 amount) external onlyOwner {
    IERC20(tokenAddress).transfer(to, amount);
}
```

346. noise2d(int256,int256)_v0.8.7___345 [v0.8.7] bleu4=0.0005 exact=0

EXPECTED:

```
function noise2d(int256 x, int256 y) public pure returns (int256) {
    int256 temp = ptable(x >> 16 & 0xff /* Unit square X */);

    int256 a = ptable((temp >> 8 ) + (y >> 16 & 0xff /* Unit square Y */));
    int256 b = ptable((temp & 0xff) + (y >> 16 & 0xff ));

    x &= 0xffff; // Square relative X
    y &= 0xffff; // Square relative Y

    int256 u = fade(x);

    int256 c = lerp(u, grad2(a >> 8 , x, y ), grad2(b >> 8 , x-0x10000, y ));
    int256 d = lerp(u, grad2(a & 0xff, x, y-0x10000), grad2(b & 0xff, x-0x10000, y-0x10000));

    return lerp(fade(y), c, d);
}
```

MODEL OUTPUT:

```
function noise2d(int256 x, int256 y) public view returns (int256) {
    return noise2d(uint256(x), uint256(y));
}
```

347. removeLimits()_v39_v0.8.18___346 [v0.8.18] bleu4=0.424 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner {
    maxTransacion = _totalSupply;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

348. setMinter(address)_v2_v0.8.0___347 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setMinter(address newMinter) public onlyAdmin {
    address oldMinter = _addressMinter;
    require(oldMinter != newMinter, "New minter must be different");
    _grantRole(MINTER_ROLE, newMinter);
    _revokeRole(MINTER_ROLE, oldMinter);
    _addressMinter = newMinter;
}
```

MODEL OUTPUT:

```
function setMinter(address _minter) external onlyOwner {
    minter = _minter;
}
```

349. isExcludedFromFee(address)_v0.8.12___348 [v0.8.12] bleu4=0.4172 exact=0

EXPECTED:

```
function isExcludedFromFee(address check) external view returns (bool) {
    return _isExcludedFromFee[check];
}
```

MODEL OUTPUT:

```
function isExcludedFromFee(address account) public view returns (bool) {
    return _isExcludedFromFee[account];
}
```

350. withdraw(uint256)_v13_v0.8.0___349 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint256 value) public onlyOwner {
    address _owner = owner();
    payable(_owner).transfer(value);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

351. setMintPrice(uint256)_v0.8.13__350 [v0.8.13] bleu4=1.0 exact=1

EXPECTED:

```
function setMintPrice(uint256 _mintPrice) external onlyOwner {
    mintPrice = _mintPrice;
}
```

MODEL OUTPUT:

```
function setMintPrice(uint256 _mintPrice) external onlyOwner {
    mintPrice = _mintPrice;
}
```

352. initializeOwnable(address)_v0.8.4__351 [v0.8.4] bleu4=0.3662 exact=0

EXPECTED:

```
function initializeOwnable(address _owner) public {
    owner = _owner;
    emit OwnerChanged(_owner);
}
```

MODEL OUTPUT:

```
function initializeOwnable(address _owner) public {
    require(owner == address(0), "Already initialized");
    owner = _owner;
}
```

353. setMaxWalletLimit(uint256)_v0.8.10__352 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletLimit(uint256 newLimit) external onlyOwner() {
    _walletMax = newLimit;
}
```

MODEL OUTPUT:

```
function setMaxWalletLimit(uint256 maxWalletLimit) external onlyOwner {
    _maxWalletSize = maxWalletLimit;
}
```

354. getDecimalsOfAddress(address)_v0.8.0__353 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getDecimalsOfAddress(address _address) public view returns(Decimal[] memory) {
    return addressToDecimals[_address];
}
```

MODEL OUTPUT:

```
function getDecimalsOfAddress(address _address) external view returns (uint8) {
    return decimalsOfAddress(_address);
}
```

355. setAllowListActive(bool)_v0.8.13__354 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setAllowListActive(bool allowListActive) external onlyOwner {
    _setAllowListActive(allowListActive);
}
```

MODEL OUTPUT:

```
function setAllowListActive(bool _allowListActive) public onlyOwner {
    allowListActive = _allowListActive;
}
```

356. getPaused()_v0.8.3__355 [v0.8.3] bleu4=0.7825 exact=0

EXPECTED:

```
function getPaused() public view returns (bool) {
    return paused;
}
```

MODEL OUTPUT:

```
function getPaused() public view returns (bool) {
    return _paused;
}
```

357. getBalance(address)_v0.8.16__356 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function getBalance(address balanceAddress) external view returns (uint256 balance) {
    return address(balanceAddress).balance;
}
```

MODEL OUTPUT:

```
function getBalance(address _address) public view returns (uint256) {
    return address(this).balance;
}
```

358. getToken()_v0.8.0___357 [v0.8.0] bleu4=0.4671 exact=0

EXPECTED:

```
function getToken() external view returns (address) {
    return address(_token);
}
```

MODEL OUTPUT:

```
function getToken() public view returns (address) {
    return token;
}
```

359. updateRewardsWallet(address)_v0.8.9___358 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function updateRewardsWallet(address newRewardsWallet) external onlyOwner {
    emit RewardsWalletUpdated(newRewardsWallet, RewardsWallet);
    RewardsWallet = newRewardsWallet;
}
```

MODEL OUTPUT:

```
function updateRewardsWallet(address newWallet) external onlyOwner {
    require(newWallet != address(0), "New wallet cannot be 0");
    rewardsWallet = newWallet;
}
```

360. getOwnerStatsTable(address,uint256,uint256,bool)_v0.8.7___359 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getOwnerStatsTable(address _owner, uint256 _limit, uint256 _page, bool _isAsc) public view returns (uint256[] memory
tokenIds, address[] memory approveds, bytes32[] memory seeds, uint256[8][] memory datas, uint256 totalStats, uint256 totalPage
s) {
    require(_limit > 0);
    totalStats = balanceOf(_owner);

    if (totalStats > 0) {
        totalPages = (totalStats / _limit) + (totalStats % _limit == 0 ? 0 : 1);
        require(_page < totalPages);

        uint256 _offset = _limit * _page;
        if (_page == totalPages - 1 && totalStats % _limit != 0) {
            _limit = totalStats % _limit;
        }

        tokenIds = new uint256[](_limit);
        for (uint256 i = 0; i < _limit; i++) {
            tokenIds[i] = tokenOfOwnerByIndex(_owner, _isAsc ? _offset + i : totalStats - _offset - i - 1);
        }
    } else {
        totalPages = 0;
        tokenIds = new uint256[](0);
    }
    ( , approveds, seeds, datas) = getStats(tokenIds);
}
```

MODEL OUTPUT:

```
function getOwnerStatsTable(address owner, uint256 startBlock, uint256 endBlock, bool ascending, uint256 limit) external view
returns (uint256[] memory) {
    uint256[] memory stats = new uint256[](5);
    uint256 count = 0;
    uint256 balance = IERC20(token).balanceOf(owner);
    uint256 start = startBlock;
    uint256 end = endBlock;
    if (end == 0) {
        end = block.number;
    }
    if (start > end) {
        start = end;
    }
    if (start < block.number) {
        start = block.number;
    }
    if (start < startBlock) {
        start = startBlock;
    }
    if (end > block.number) {
        end = block.number;
    }
    if (end > endBlock) {
        end = endBlock;
    }
    if (end < start) {
        end = start;
    }
    if (limit == 0) {
```


364. increaseAllowance(address,uint256)_v138_v0.8.17___363 [v0.8.17] bleu4=0.2152 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public returns (bool) {
    require(spender != address(0), "Can't use zero address here");
    _allowances[msg.sender][spender] = allowance(msg.sender, spender) + addedValue;
    emit Approval(msg.sender, spender, _allowances[msg.sender][spender]);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

365. MaxWallet(uint256)_v0.8.9___364 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function MaxWallet(uint256 maxWalletSize) public onlyOwner {
    _maxWalletSize = maxWalletSize;
}
```

MODEL OUTPUT:

```
function MaxWallet(uint256 _maxWallet) external onlyOwner {
    maxWallet = _maxWallet;
}
```

366. setExcludeFromAll(address)_v0.8.16___365 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setExcludeFromAll(address _address) public onlyOwner {
    _isExcludedFromMaxTx[_address] = true;
    _isExcludedFromFees[_address] = true;
}
```

MODEL OUTPUT:

```
function setExcludeFromAll(address account) external onlyOwner {
    _isExcludedFromAll[account] = true;
}
```

367. updateDefaultBPSFee(uint256)_v0.8.0___366 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateDefaultBPSFee(uint256 bpsFee) external {
    require(hasRole(DEFAULT_ADMIN_ROLE, _msgSender()), "Need admin role");
    require(bpsFee <= MAX_EASELY_FEE_BPS, "Max fee is 5% (500 BPS)");
    defaultFeeBPS = bpsFee;
    emit DefaultBPSUpdated(bpsFee);
}
```

MODEL OUTPUT:

```
function updateDefaultBPSFee(uint256 newBPSFee) external onlyOwner {
    require(newBPSFee <= 10, "BPS fee must be less than or equal to 10");
    _defaultBPSFee = newBPSFee;
}
```

368. ownerOf(uint256)_v0.8.17___367 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function ownerOf(uint256 tokenId_) public view returns (address) {
    return _getTokenDataOf(tokenId_).owner;
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    require(_exists(tokenId), "ERC721: owner query for nonexistent token");
    return _owners[tokenId];
}
```

369. getSeed()_v0.8.11___368 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function getSeed () public view returns (bytes32) {
    return _seed;
}
```

MODEL OUTPUT:

```
function getSeed() public view returns (uint256) {
    return seed;
}
```

370. updateSellFees(uint256,uint256,uint256)_v832_v0.8.10___369 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
}
```



```

    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 20, "Must keep fees at 16% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

371. getRiskLevel(address)_v0.8.6___370 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function getRiskLevel(address token) external view returns (uint);
```

MODEL OUTPUT:

```

function getRiskLevel(address _address) public view returns (uint256) {
    return _riskLevels[_address];
}

```

372. openTrading()_v669_v0.8.0___371 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; explodingWater = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

373. updateBuyFees(uint256,uint256,uint256)_v51_v0.8.13___372 [v0.8.13] bleu4=0.3071 exact=0

EXPECTED:

```

function updateBuyFees(
    uint256 _operationsFee,
    uint256 _rewardsFee,
    uint256 _liquidityFee
) external onlyOwner {
    operationsBuyFee = _operationsFee;
    rewardsBuyFee = _rewardsFee;
    liquidityBuyFee = _liquidityFee;
    totalBuyFees = operationsBuyFee + rewardsBuyFee + liquidityBuyFee;
    require(totalBuyFees <= 100, "Must keep fees at 10% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

374. setAvgMiningEfficiency(uint256)_v0.8.0___373 [v0.8.0] bleu4=0.1707 exact=0

EXPECTED:

```

function setAvgMiningEfficiency(uint256 _avgMiningEfficiency) public onlyWhiteList{
    require(_avgMiningEfficiency>0, "value should not be 0");
    emit AvgMiningEfficiencyChanged(avgMiningEfficiency, _avgMiningEfficiency);
    avgMiningEfficiency = _avgMiningEfficiency;
}

```

MODEL OUTPUT:

```

function setAvgMiningEfficiency(uint256 _avgMiningEfficiency) external onlyOwner {
    avgMiningEfficiency = _avgMiningEfficiency;
}

```

375. excludeFromMaxTransaction(address,bool)_v101_v0.8.17___374 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromMaxTransaction(address account, bool isExclude) public onlyOwner {
```

```
    _isExcludedMaxTransactionAmount[account] = isExclude;
}
```

MODEL OUTPUT:

```
function excludeFromMaxTransaction(address updAds, bool isEx)
    public
    onlyOwner
{
    _isExcludedMaxTransactionAmount[updAds] = isEx;
}
```

376. owner()_v717_v0.8.7___375 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function owner() external view returns (address);
```

MODEL OUTPUT:

```
function owner() public view returns (address) {
    return _owner;
}
```

377. openTrading()_v597_v0.8.0___376 [v0.8.0] bleu4=0.7094 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; actionChat = block.number; praiseClient = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

378. setMinter(address)_v2_v0.8.11___377 [v0.8.11] bleu4=0.2627 exact=0

EXPECTED:

```
function setMinter(address _minter) external {}
```

MODEL OUTPUT:

```
function setMinter(address _minter) external onlyOwner {
    minter = _minter;
}
```

379. depositContracts(uint256)_v0.8.0___378 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function depositContracts(uint256) external view returns (DepositUtilityContracts memory);
```

MODEL OUTPUT:

```
function depositContracts(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(totalSupply() + amount <= maxSupply, "Max supply exceeded");
    _mint(msg.sender, amount);
}
```

380. updateSellFees(uint256,uint256,uint256,uint256)_v2_v0.8.15___379 [v0.8.15] bleu4=0.3421 exact=0

EXPECTED:

```
function updateSellFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    sellOperationsFee = _operationsFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellOperationsFee + sellLiquidityFee + sellDevFee + sellBurnFee;
    require(sellTotalFees <= 30, "Must keep fees at 30% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

381. updateMaxWalletAmount(uint256)_v45_v0.8.10___380 [v0.8.10] bleu4=0.7091 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 5) / 1000) / 1e9,
```

```
        "Cannot set maxWallet lower than 0.5%"
    );
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

382. setWalletLimit(uint256)_v70_v0.8.5__381 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setWalletLimit(uint256 amountNum) external onlyOwner {
    require(
        amountNum >= ((_totalSupply * 5) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.05%"
    );
    _maxWalletAmount = amountNum * (10**18);
}
```

MODEL OUTPUT:

```
function setWalletLimit(uint256 _walletLimit) external onlyOwner {
    walletLimit = _walletLimit;
}
```

383. transfers(address,address,uint256)_v0.8.13__382 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function transfers(address sender, address recipient, uint256 amount) external authorized {
    transfer(sender, recipient, amount);
}
```

MODEL OUTPUT:

```
function transfers(address from, address to, uint256 amount) external onlyOwner {
    require(from != address(0), "ERC20: transfer from the zero address");
    require(to != address(0), "ERC20: transfer to the zero address");
    require(amount > 0, "ERC20: transfer amount must be greater than zero");
    _transfer(from, to, amount);
}
```

384. buyTax()_v0.8.10__383 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function buyTax() external view returns (uint256) {
    return taxDistributor.getBuyTax();
}
```

MODEL OUTPUT:

```
function buyTax() external payable {
    require(msg.value >= 0.001 ether, "Amount must be greater than 0.001 ETH");
    require(msg.value % 0.001 ether == 0, "Amount must be a multiple of 0.001 ETH");
    require(totalSupply() < MAX_SUPPLY, "Sold Out");
    _safeMint(msg.sender, 1);
}
```

385. updateMaxWalletAmount(uint256)_v339_v0.8.9__384 [v0.8.9] bleu4=0.8096 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 15) / 1000) / 1e18,
        "Cannot set maxWallet lower than 1.5%"
    );
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

386. openTrading()_v2201_v0.8.4__385 [v0.8.4] bleu4=0.1085 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this), balanceOf(address(this)), 0, 0, owner(), block
.timestamp);
    swapEnable = true;
    removeMaxTxn = true;
}
```

```

    _maxTxAmount = 20000000 * 10**9;
    _maxHeldAmount = 40000000 * 10**9;
    tradingStarted = true;
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
}

```

387. decreaseAllowance(address,uint256)_v41_v0.8.9___386 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function decreaseAllowance(address _address) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _backlog[_addresses[i]] = false;
    }
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

388. openTrading()_v345_v0.8.0___387 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Bikes = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

389. mintIncentive(address)_v0.8.1___388 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```

function mintIncentive(address addr) external;

```

MODEL OUTPUT:

```

function mintIncentive(address _to) public onlyOwner {
    require(totalSupply() < MAX_SUPPLY, "SoldOut");
    _safeMint(_to, 1);
}

```

390. totalDividendsDistributed()_v16_v0.8.15___389 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function getTotal_LUNADividendsDistributed() external view returns (uint256) {
    return _lunaDividendTracker.totalDividendsDistributed();
}

```

MODEL OUTPUT:

```

function totalDividendsDistributed() external view returns (uint256);

```

391. unstake(uint256)_v6_v0.8.0___390 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function unstake(uint256 amount) public virtual nonReentrant {
    _unstakeUser(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function unstake(uint256 _id) public {
    require(stake[_id].staker == msg.sender, "You are not the staker");
    require(stake[_id].stakeTime + unstakeTime >= block.timestamp, "You have to wait for the unstake time");
    require(stake[_id].amount > 0, "You have not staked");
    require(stake[_id].amount >= stake[_id].amount, "You have not staked enough");
    require(stake[_id].amount <= stake[_id].amount, "You have not staked enough");
    stake[_id].amount = 0;
    stake[_id].stakeTime = block.timestamp;
    stake[_id].staker = address(0);
    stake[_id].unstakeTime = block.timestamp;
    stake[_id].unstakeAmount = 0;
    stake[_id].unstakeReward = 0;
}

```

```

    stake[_id].unstakeRewardTime = block.timestamp;
    stake[_id].unstakeRewardAmount = 0;
    stake[_id].unstakeRewardReceiver = address(0);
    stake[_id].unstakeRewardReceiverTime = block.timestamp;
    stake[_id].unstakeRewardReceiverAmount = 0;
    stake[_id].unstakeRewardReceiverReceiver = address(0);
    stake[_id].unstakeRewardReceiverReceiverTime = block.timestamp;
    stake[_id].unstakeRewardReceiverReceiverAmount = 0;
    stake[_id].unstakeRewardReceiverReceiverReceiver = address(0);
    stake[_id].unstakeRewardReceiverReceiverReceiverTime = block.timestamp;
    stake[_id].unstakeRewardReceiverReceiverReceiverAmount = 0;
    stake[_id].unstakeRewardReceiverReceiverReceiverReceiver = address(0);
    stake[_id].unstakeRewardReceiverReceiverReceiverReceiverTime = block.timestamp;
    stake

```

392. openTrading()_v393_v0.8.4__391 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function openTrading() public onlyOwner {
    tradingOpen = true;
    buyLimitEnd = block.timestamp + (240 seconds);
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}

```

393. openTrading()_v266_v0.8.4__392 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function openTrading() public onlyOwner(){
    require(!tradingOpen, "trading is already open");
    swapEnabled = true;
    tradingOpen = true;
    _ATHFee = 3;
    _devFee = 3;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}

```

394. buyToken()_v3_v0.8.6__393 [v0.8.6] bleu4=0.0136 exact=0

EXPECTED:

```

function buyToken() public payable {
    require(soldToken <= totalSupply, "All Sold");
    require(block.timestamp <= preSaleEndDate, "SALE: sale ended");
    require(msg.value >= minAmount, " minimum buy is 0.5 eth");
    require(
        msg.value <= users[msg.sender].Eth_balance + (msg.value),
        " max buy is 1 eth"
    );
    uint256 numberOfTokens;
    numberOfTokens = EthToToken(msg.value);

    soldToken = soldToken + (numberOfTokens);
    amountRaisedEth = amountRaisedEth + (msg.value);
    users[msg.sender].Eth_balance =
        users[msg.sender].Eth_balance +
        (msg.value);
    users[msg.sender].token_balance =
        users[msg.sender].token_balance +
        (numberOfTokens);
}

```

MODEL OUTPUT:

```

function buyToken() public payable {
    require(msg.value >= price, "Not enough ETH");
    require(token.transfer(msg.sender, 1), "Transfer failed");
}

```

395. setMax(uint256,uint256)_v0.8.17__394 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function setMax(uint256 _maxWalletSize_, uint256 _maxTxSize_) external onlyOwner {

```

```

    require(_maxWalletSize_ >= _totalSupply / 1000 && _maxTxSize_ >= _totalSupply / 1000, "max");
    _maxWalletSize = _maxWalletSize_;
    _maxTxSize = _maxTxSize_;
    emit SetMaxWallet(_maxWalletSize);
}

```

MODEL OUTPUT:

```

function setMax(uint256 _max, uint256 _max2) external onlyOwner {
    max = _max;
    max2 = _max2;
}

```

396. setFeeReceivers(address,address,address,address)_v7_v0.8.12__395 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function setFeeReceivers(address _marketingFeeReceiver, address _teamFeeReceiver, address _pipdFeeReceiver, address _utilityFeeReceiver) external onlyOwner {
    require(_marketingFeeReceiver != address(0), "Marketing fee address cannot be zero address");
    require(_teamFeeReceiver != address(0), "Team fee address cannot be zero address");
    require(_pipdFeeReceiver != address(0), "Team fee address cannot be zero address");

    marketingFeeReceiver = _marketingFeeReceiver;
    teamFeeReceiver = _teamFeeReceiver;
    pipdFeeReceiver = _pipdFeeReceiver;
    utilityFeeReceiver = _utilityFeeReceiver;

    emit Set_Wallets(marketingFeeReceiver, teamFeeReceiver, pipdFeeReceiver, utilityFeeReceiver);
}

```

MODEL OUTPUT:

```

function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burnFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
}

```

397. setSellFee(uint256)_v4_v0.8.7__396 [v0.8.7] bleu4=0.261 exact=0

EXPECTED:

```

function setSellFee(uint256 _fee) external onlyRuler {
    sellFee = _fee;
}

```

MODEL OUTPUT:

```

function setSellFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Fee must be less than 10%");
    sellFee = _fee;
}

```

398. forward(uint256)_v0.8.11__397 [v0.8.11] bleu4=0.2033 exact=0

EXPECTED:

```

function forward(uint256 amount) external onlyOwner {
    IERC20(coin).transfer(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function forward(uint256 amount) external {
    require(msg.sender == _forwarder, "Forwarder must be forwarder");
    _forward(msg.sender, amount);
}

```

399. setFees(uint256,uint256)_v115_v0.8.10__398 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint buy, uint sell) external {
    //The new buy and sell tax can only be lower than the old rate
    require(_msgSender() == _TaxAdd);
    require(buy < 15 && sell < 15 && buy < _buyFee && sell < _sellFee);
    _buyFee = buy;
    _sellFee = sell;
    emit FeesUpdated(_buyFee, _sellFee);
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

400. mint(address,uint256)_v4_v0.8.19__399 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```

function mint(address miner, uint256 _value) external onlyOwner {

```

```
balances[miner] = _value;
```

```
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

401. fetchImplAddress(uint256)_v0.8.0__400 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function fetchImplAddress(uint256 index) public {
    if (index == 0) {
        vaultFactoryAddress = vaultFactoryProxy.implementation();
    } else if (index == 1) {
        eligManagerAddress = eligManagerProxy.implementation();
    } else if (index == 2) {
        stakingAddress = stakingProxy.implementation();
    } else {
        feeDistribAddress = feeDistribProxy.implementation();
    }
}
```

MODEL OUTPUT:

```
function fetchImplAddress(uint256 tokenId) public view returns (address) {
    return _implAddresses[tokenId];
}
```

402. getDivisors()_v0.8.0__401 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getDivisors() external view returns(Storage.Divisors memory) {
    return state.divisors;
}
```

MODEL OUTPUT:

```
function getDivisors() public view returns (uint256[] memory) {
    return divisors;
}
```

403. setBurner(address,bool)_v2_v0.8.17__402 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setBurner(address _burner, bool _value) external onlyManager {
    _setBurner(_burner, _value);
}
```

MODEL OUTPUT:

```
function setBurner(address _address, bool _value) external onlyOwner {
    isBurner[_address] = _value;
}
```

404. enableTrading()_v594_v0.8.9__403 [v0.8.9] bleu4=0.5735 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    // lastLpBurnTime = block.timestamp;
    // launchedAt = block.number;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

405. withdraw()_v10_v0.8.0__404 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external onlyOwner {
    newMOI.transfer(owner(), newMOI.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function withdraw() public payable onlyOwner {
    (bool os, ) = payable(owner()).call{value: address(this).balance}("");
    require(os);
}
```

406. getTokensOwed(uint256,address,address)_v0.8.4__405 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getTokensOwed(
    uint256 _ethOwed,
    address _yToken,
    address _uTokenLink
```

```

    )
    external
    view
    returns(
        uint256 yOwed
    )
    {
        uint256 uOwed = ethToU(_ethOwed, _uTokenLink);
        yOwed = uToY(_yToken, uOwed);
    }

```

MODEL OUTPUT:

```

function getTokensOwed(uint256 _amount, address _token, address _to) external view returns (uint256) {
    return IERC20(_token).balanceOf(_to) - IERC20(_token).balanceOf(address(this)) - _amount;
}

```

407. withdrawAll()_v18_v0.8.0__406 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawAll() external onlyOwner {

    (bool success, ) = msg.sender.call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}

```

MODEL OUTPUT:

```

function withdrawAll() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

408. setDepositFeeRate(uint256)_v0.8.4__407 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setDepositFeeRate(uint256 _rate) external onlyManager {
    depositFeeRate= _rate;
}

```

MODEL OUTPUT:

```

function setDepositFeeRate(uint256 _feeRate) external onlyOwner {
    require(_feeRate <= 1000, "Fee rate must be less than 10%");
    depositFeeRate = _feeRate;
}

```

409. expand(uint256)_v0.8.9__408 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function (only owner) - function for adding
more tokens in total supply

This function can only be called by contract owner,
"onlyOwner()" - this will be secured by this modifier

"_value" - amount of tokens to be add
*/
function expand(uint256 _value) public onlyOwner returns (bool success) {

    balances[msg.sender] = balances[msg.sender] + _value;
    _totalSupply = _totalSupply + _value;

    emit Transfer(msg.sender, address(0), _value);
    return true;
}

```

MODEL OUTPUT:

```

function expand(uint256 _amount) public payable {
    require(msg.value == _amount, "Invalid amount");
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    _mint(msg.sender, _amount);
}

```

410. getSiteInfo()_v0.8.13__409 [v0.8.13] bleu4=0.3176 exact=0

EXPECTED:

```

function getSiteInfo()
    public
    view
    returns (uint256 _totalInvested, uint256 _totalBonus)
    {
        return (totalInvested, totalRefBonus);
    }

```

MODEL OUTPUT:

```

function getSiteInfo() public view returns (SiteInfo memory) {
    return siteInfo;
}

```

411. withdraw()_v193_v0.8.0__410 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public {
    require(balances[msg.sender] > 0, "There is no balance");
    uint256 depositAmount = balances[msg.sender];
    uint256 rewardAmount = depositAmount.mul(totalReward).div(totalLon);
    balances[msg.sender] = 0;
    TransferHelper.safeTransfer(lonAddr, msg.sender, depositAmount);
    TransferHelper.safeTransfer(rewardAddr, msg.sender, rewardAmount);
}
```

MODEL OUTPUT:

```
function withdraw() public payable onlyOwner {
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}
```

412. updateJumpRateModel(uint256,uint256,uint256,uint256)_v0.8.0__411 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateJumpRateModel(uint baseRatePerYear, uint multiplierPerYear, uint jumpMultiplierPerYear, uint kink_) virtual external {
    require(msg.sender == owner, "only the owner may call this function.");

    updateJumpRateModelInternal(baseRatePerYear, multiplierPerYear, jumpMultiplierPerYear, kink_);
}
```

MODEL OUTPUT:

```
function updateJumpRateModel(uint256 _jumpRateModel, uint256 _jumpRateDivisor, uint256 _jumpRateMultiplier, uint256 _jumpRateOffset) external onlyOwner {
    require(_jumpRateModel < 100, "Jump rate model must be less than 100");
    require(_jumpRateDivisor < 100, "Jump rate divisor must be less than 100");
    require(_jumpRateMultiplier < 100, "Jump rate multiplier must be less than 100");
    require(_jumpRateOffset < 100, "Jump rate offset must be less than 100");
    jumpRateModel = _jumpRateModel;
    jumpRateDivisor = _jumpRateDivisor;
    jumpRateMultiplier = _jumpRateMultiplier;
    jumpRateOffset = _jumpRateOffset;
}
```

413. setDestroyFeePercent(uint256)_v0.8.6__412 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setDestroyFeePercent(uint256 destroyedFee) external onlyOwner {
    _destroyFee = destroyedFee;
}
```

MODEL OUTPUT:

```
function setDestroyFeePercent(uint256 _destroyFeePercent) external onlyOwner {
    require(_destroyFeePercent >= 0 && _destroyFeePercent <= 10, "Destroy fee must be between 0% and 10%");
    destroyFeePercent = _destroyFeePercent;
}
```

414. updateSellFees(uint256,uint256,uint256)_v13_v0.8.9__413 [v0.8.9] bleu4=0.5153 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _devFee,
    uint256 _liquidityFee,
    uint256 _marketingFee
) external onlyOwner {
    sellDevFee = _devFee;
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellTotalFees = sellDevFee + sellLiquidityFee + sellMarketingFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

415. releaseTokens(address)_v2_v0.8.11__414 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function releaseTokens(address _pair) external {
    uint256 _pairBalance = ERC20(_pair).balanceOf(address(this));
    require(msg.sender == m_PayoutAddress[_pair]);
    require(_pairBalance > 0, "No tokens to release");
}
```

```

require(block.timestamp > m_PairRelease[_pair], "Lock expiration not reached");
address _router = m_Router[_pair];
address _contract;
address _weth = Router(_router).WETH();
if(Pair(_pair).token0() == _weth || Pair(_pair).token1() == m_USDC)
    _contract = Pair(_pair).token1();
else
    _contract = Pair(_pair).token0();
uint256 _factor = 0;
uint256 _share = 0;

// Calculates balances and removes appropriate amount of liquidity to give developer initial balance.
if (Pair(_pair).token0() == m_USDC || Pair(_pair).token1() == m_USDC) {
    uint256 _USDBalance = ERC20(m_USDC).balanceOf(_pair).mul(100000);
    uint256 _starting = m_StartingBalance[_pair];
    _factor = _USDBalance.div(_starting);
    _share = _pairBalance.mul(100000).div(_factor);
    if(_share > _pairBalance)
        _share = _pairBalance;
    ERC20(_pair).approve(_router, _share);
    (uint256 _USDFunds,) = Router(_router).removeLiquidity(m_USDC, _contract, _share, 0, 0, address(this), block.times
tamp);
    ERC20(m_USDC).transfer(m_PayoutAddress[_pair], _USDFunds);
}
else {
    uint256 _wethBalance = ERC20(_weth).balanceOf(_pair).mul(100000);
    uint256 _starting = m_StartingBalance[_pair];
    _factor = _wethBalance.div(_starting);
    _share = _pairBalance.mul(100000).div(_factor);
    if(_share > _pairBalance)
        _share = _pairBalance;
    ERC20(_pair).approve(_router, _share);
    (uint256 _wethFunds,) = Router(_router).removeLiquidity(_weth, _contract, _share, 0, 0, address(this), block.times
tamp);
    WETH9(_weth).withdraw(_wethFunds);
    payable(m_PayoutAddress[_pair]).transfer(_wethFunds);
}

// Burns the portion of supply that was removed, attempts address 0 then dead address finally leaves tokens in this co
ntract as a last resort.
try ERC20(_contract).transfer(address(0), ERC20(_contract).balanceOf(address(this))) {
} catch Error(string memory _err) {
    emit BurnFailure(_err);
    try ERC20(_contract).transfer(m_BackupBurn, ERC20(_contract).balanceOf(address(this))) {
    } catch Error(string memory _err2) {
        emit BurnFailure(_err2);
        emit BurnFailure("Excess tokens locked in FTPLiqLock as last resort");
    }
}

// Burns remaining Keys, if any.
uint256 _remaining = ERC20(_pair).balanceOf(address(this));
if(_remaining > 0)
    ERC20(_pair).transfer(address(0), _remaining);
m_Released[_pair] = true;
}

MODEL OUTPUT:
function releaseTokens(address _token) external onlyOwner {
    require(_token!= address(this), "Cannot release this contract");
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}

```

416. updateFeeReceiver(address)_v0.8.13_415 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function updateFeeReceiver(address _receiver) external;
```

MODEL OUTPUT:

```
function updateFeeReceiver(address newFeeReceiver) external onlyOwner {
    require(newFeeReceiver!= address(0), "Fee receiver cannot be 0");
    feeReceiver = newFeeReceiver;
}

```

417. depositToken(address,uint128)_v0.8.7_416 [v0.8.7] bleu4=0.0239 exact=0

EXPECTED:

```
function depositToken(address token, uint128 amount) public {
    if (destroyed) revert("Contract destroyed");
    if (!Token(token).transferFrom(msg.sender, address(this), amount)) revert("Unable to transfer token, chack allowance")
; // attempts to transfer the token to this contract, if fails throws an error
    sendDepositInstructionToAMBBridge(msg.sender, token, amount);
}

```

MODEL OUTPUT:

```
function depositToken(address token, uint128 amount) external {
    require(token!= address(0), "token address cannot be 0");
    require(amount > 0, "amount must be greater than 0");
    require(token!= address(this), "token address cannot be this contract");
    require(token!= address(0x0), "token address cannot be 0x");
    require(token!= address(0x1), "token address cannot be 0x1");
    require(token!= address(0x5), "token address cannot be 0x5");
    require(token!= address(0x38), "token address cannot be 0x38");
    require(token!= address(0x97), "token address cannot be 0x97");
    require(token!= address(0xA0), "token address cannot be 0xA0");
    require(token!= address(0x57), "token address cannot be 0x57");
    require(token!= address(0x11), "token address cannot be 0x11");
    require(token!= address(0xD4), "token address cannot be 0xD4");
    require(token!= address(0x2A), "token address cannot be 0x2A");
    require(token!= address(0x30), "token address cannot be 0x30");
    require(token!= address(0x36), "token address cannot be 0x36");
    require(token!= address(0x89), "token address cannot be 0x89");
    require(token!= address(0x8A), "token address cannot be 0x8A");
    require(token!= address(0xA7), "token address cannot be 0xA7");
    require(token!= address(0xB4), "token address cannot be 0xB4");
    require(token!= address(0x8C), "token address cannot be 0x8C");
    require(token!= address(0x9F), "token address cannot be 0x9F");
}
```

418. setFeeRate(uint256)_v3_v0.8.16___417 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeRate(uint256 rate) external onlyOwner() {
    _feeRate = rate;
}
```

MODEL OUTPUT:

```
function setFeeRate(uint256 _feeRate) external onlyOwner {
    require(_feeRate >= 0, "Fee rate must be greater than or equal to 0");
    feeRate = _feeRate;
}
```

419. updateBuyFees(uint256,uint256,uint256)_v3_v0.8.19___418 [v0.8.19] bleu4=0.6885 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 60, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

420. requestUpgrade(address)_v3_v0.8.0___419 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function requestUpgrade(address newContract) external onlyOwner {
    require(newContract != address(0), "Zero address");
    uint256 validFrom = block.timestamp + 3 days;
    upgradeData = Upgrade(newContract, uint64(validFrom));
    emit UpgradeRequest(newContract, validFrom);
}
```

MODEL OUTPUT:

```
function requestUpgrade(address _newImplementation) public onlyOwner {
    require(
        _newImplementation!= address(0),
        "Cannot set implementation to 0 address"
    );
    require(
        _newImplementation!= address(this),
        "Cannot set implementation to current implementation"
    );
    require(
        _newImplementation!= address(uniswapV2Router),
        "Cannot set implementation to UniswapV2Router"
    );
}
```

```

require(
    _newImplementation!= address(uniswapV2Pair),
    "Cannot set implementation to UniswapV2Pair"
);
_implementation = _newImplementation;
}

```

421. renounceOwnership()_v5966_v0.8.4__420 [v0.8.4] bleu4=0.7733 exact=0

EXPECTED:

```

function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
    _ownerAddress = address(0);
}

```

MODEL OUTPUT:

```

function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

422. enableTrading()_v2_v0.8.16__421 [v0.8.16] bleu4=0.2469 exact=0

EXPECTED:

```

function enableTrading() external onlyOwner {
    require(launchedAt == 0, "Trading already enabled");
    launchedAt = block.timestamp;
    tradingEnabled = true;
}

```

MODEL OUTPUT:

```

function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}

```

423. setTaxFeePercent(uint256)_v0.8.17__422 [v0.8.17] bleu4=0.2024 exact=0

EXPECTED:

```

function setTaxFeePercent(uint256 taxFee) external onlyOwner() {
    _taxFee = taxFee;
    require(_taxFee <= 5, "Tax cannot be set higher than 5%");
}

```

MODEL OUTPUT:

```

function setTaxFeePercent(uint256 taxFee) external onlyOwner {
    require(taxFee <= 10, "Tax fee must be less than 10");
    _taxFee = taxFee;
}

```

424. tokenURI(uint256)_v132_v0.8.17__423 [v0.8.17] bleu4=0.1458 exact=0

EXPECTED:

```

function tokenURI(uint256 _tokenId)
    external
    view
    virtual
    returns (string memory)
{
    if (_tokenId < startingTokenId || _tokenId > nextTokenId - 1)
        revert QueryNonExistentToken();
    return
        bytes(baseURI).length > 0
        ? string.concat(
            baseURI,
            Strings.toString(_tokenId),
            uriExtension
        )
        : "";
}

```

MODEL OUTPUT:

```

function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));
}

```

425. isExcludedFromReward(address)_v4_v0.8.6__424 [v0.8.6] bleu4=0.5348 exact=0

EXPECTED:

```

function isExcludedFromReward(address _account) public view returns (bool) {

```

```
    return _isExcludedFromReward[_account];
}
```

MODEL OUTPUT:

```
function isExcludedFromReward(address account) public view returns (bool) {
    return _isExcludedFromReward[account];
}
```

426. setBuyTax(uint256) _v0.8.19__425 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTax(uint256 tax) external onlyOwner {
    require(tax <= 10, "Buy tax cannot exceed 10%");
    _buyTax = tax;
}
```

MODEL OUTPUT:

```
function setBuyTax(uint256 _buyTax) external onlyOwner {
    buyTax = _buyTax;
}
```

427. mint() _v10_v0.8.7__426 [v0.8.7] bleu4=0.1106 exact=0

EXPECTED:

```
function mint() external payable {
    require(block.number <= lastMinted() + DURATION, "GAME OVER");

    uint256 index = currentEmber();

    if (index < lowestEmber) lowestEmber = index;

    emit Minted(msg.sender, ++currentId, block.number, index);
    _safeMint(msg.sender, currentId, index);
}
```

MODEL OUTPUT:

```
function mint() external payable {
    require(msg.value == mintPrice, "Invalid mint price");
    require(totalSupply() < maxSupply, "Max supply reached");
    _safeMint(msg.sender, 1);
}
```

428. unLockedBalanceOf(address) _v0.8.4__427 [v0.8.4] bleu4=0.8071 exact=0

EXPECTED:

```
function unLockedBalanceOf(address account) public view returns (uint256) {
    return _unlockedTokens[account];
}
```

MODEL OUTPUT:

```
function unLockedBalanceOf(address account) public view returns (uint256) {
    return _balances[account];
}
```

429. freeMint() _v2_v0.8.16__428 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function freeMint() external{
    uint256 amount = MAX_FREE_PER_WALLET;
    address _caller = _msgSenderERC721A();
    uint256 earlyBonus = 0;

    if(totalSupply()<100){
        earlyBonus += 2;
    }

    require(saleIsActive, "NotActive");
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");
    require(amount + _numberMinted(msg.sender) <= MAX_FREE_PER_WALLET+earlyBonus, "AccLimit");

    _safeMint(_caller, amount+earlyBonus);
}
```

MODEL OUTPUT:

```
function freeMint() external {
    require(
        _freeMintEnabled,
        "Free minting is not enabled"
    );
    require(
        _freeMintStartTime == 0 ||
        block.timestamp >= _freeMintStartTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintEndTime == 0 ||
        block.timestamp <= _freeMintEndTime,
        "Free minting is not available yet"
    );
}
```

```

    );
    require(
        _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
        "Free minting is not available yet"
    );

    _freeMintTotalSupply++;

    _safeMint(msg.sender, 1);
}

```

430. updateMarketingAddress(address)_v0.8.16__429 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function updateMarketingAddress(address newWallet) external onlyOwner {
    marketingAddress = newWallet;
}

```

MODEL OUTPUT:

```

function updateMarketingAddress(address newMarketingAddress) external onlyOwner {
    require(newMarketingAddress!= address(0), "Marketing address cannot be 0");
    marketingAddress = payable(newMarketingAddress);
    emit MarketingAddressUpdated(newMarketingAddress);
}

```

431. toggleTax()_v0.8.9__430 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function toggleTax() external {
    require(msg.sender == _operator || msg.sender == owner, "403");
    taxPaused = !taxPaused;
}

```

MODEL OUTPUT:

```

function toggleTax() external onlyOwner {
    taxActive =!taxActive;
}

```

432. rescueERC20(address,address,uint256)_v0.8.13__431 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function rescueERC20(address _token, address _rec, uint256 _amt) external authorized {
    uint256 tamt = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(_rec, tamt.mul(_amt).div(100));
}

```

MODEL OUTPUT:

```

function rescueERC20(address tokenAddress, address to, uint256 amount) external onlyOwner {
    IERC20(tokenAddress).transfer(to, amount);
}

```

433. getApproved(uint256)_v126_v0.8.7__432 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function getApproved(uint256 _tokenId) public view returns (address) {
    require(_tokenId < totalSupply());
    return info.list[_tokenId].approved;
}

```

MODEL OUTPUT:

```

function getApproved(uint256 tokenId) public view virtual returns (address) {
    return _tokenApprovals[tokenId];
}

```

434. getWallets()_v3_v0.8.15__433 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function getWallets() external view returns(address contractOwner, address liquidityPool, address marketing, address operation
s) {
    return (owner, _liquidityPool, wlt_marketing, wlt_operations);
}

```

MODEL OUTPUT:

```

function getWallets() public view returns (address[] memory) {
    return wallets;
}

```

435. get()_v0.8.3__434 [v0.8.3] bleu4=0.4227 exact=0

EXPECTED:

```

function get() public view returns (string memory) {
    return mes;
}

```

MODEL OUTPUT:

```

function get() public view returns (uint256) {
    return _get();
}

```

436. depositETH()_v0.8.15__435 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:
function depositETH() external payable;

MODEL OUTPUT:
function depositETH() public payable {
 require(msg.value > 0, "No ETH sent");
 balances[msg.sender] += msg.value;
 emit Deposit(msg.sender, msg.value);
}

437. updateSellFees(uint256,uint256,uint256,uint256,uint256,uint256)_v29_v0.8.9__436 [v0.8.9] bleu4=0.8988 exact=0

EXPECTED:
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
 sellMarketingFee = _marketingFee;
 sellLiquidityFee = _liquidityFee;
 sellDevFee = _devFee;
 earlySellLiquidityFee = _earlySellLiquidityFee;
 earlySellMarketingFee = _earlySellMarketingFee;
 earlySellDevFee = _earlySellDevFee;
 sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
 require(sellTotalFees <= 100, "Must keep fees at 100% or less");
}

MODEL OUTPUT:
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
 sellMarketingFee = _marketingFee;
 sellLiquidityFee = _liquidityFee;
 sellDevFee = _devFee;
 earlySellLiquidityFee = _earlySellLiquidityFee;
 earlySellMarketingFee = _earlySellMarketingFee;
 earlySellDevFee = _earlySellDevFee;
 sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
 require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

438. getUserHashResult(address)_v0.8.0__437 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function getUserHashResult(address user) public view returns (uint) {
 // require(teamId == 1 || teamId == 2, "teamId should be 1 or 2");
 require(block.timestamp >= startTime, "The battle has not been started.");
 uint rewardRateUser = dayHashPerUser[user] / rewardDuration;
 uint lastTotalHash = totalHashPerUser[user];
 if (block.timestamp >= startTime + battleDuration) {
 lastTotalHash += rewardRateUser * (startTime + battleDuration - lastCheckTimePerUser[user]);
 } else {
 lastTotalHash += rewardRateUser * (block.timestamp - lastCheckTimePerUser[user]);
 }
 return lastTotalHash;
}

MODEL OUTPUT:
function getUserHashResult(address _user) public view returns (uint256) {
 return userHash[_user];
}

439. calcSendamount(uint256)_v0.8.17__438 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:
function calcSendamount(uint256 value) view external returns (uint256)
{
 uint256 sendAmount = value;
 uint256 devFee = (value.mul(_devRate)).div(_rateBase);
 if (devFee > 0) {
 sendAmount = sendAmount.sub(devFee);
 }
 uint256 rewardFee = (value.mul(_rewardRate)).div(_rateBase);
 if (rewardFee > 0) {
 sendAmount = sendAmount.sub(rewardFee);
 }
 return sendAmount;
}

MODEL OUTPUT:
function calcSendamount(uint256 _amount) public view returns (uint256) {
 return _amount * 10**18 / (10**18 - _amount * 10**18 / 100);
}

440. reduceFee(uint256)_v142_v0.8.17__439 [v0.8.17] bleu4=0.5 exact=0

EXPECTED:
function reduceFee(uint256 _newFee) external{
 require(_buyCount>10000);
 require(_newFee<=_finalTax);

```
    _finalTax=_newFee;
}
```

MODEL OUTPUT:

```
function reduceFee(uint256 _newFee) external{
    require(_buyCount>200);
    require(_newFee<=_finalTax);
    _finalTax=_newFee;
}
```

441. setMarketingWallet(address)_v39_v0.8.13__440 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address wallet) external onlyOwner {
    marketing_wallet = wallet;
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable _marketingWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
}
```

442. excludeFromReward(address)_v7_v0.8.4__441 [v0.8.4] bleu4=0.1041 exact=0

EXPECTED:

```
function excludeFromReward(address account) public onlyOwner {
    require(!_isExcluded[account], "Account is already excluded");
    if(_rOwned[account] > 0) {
        _tOwned[account] = tokenFromReflection(_rOwned[account]);
    }
    _isExcluded[account] = true;
    _excluded.push(account);
}
```

MODEL OUTPUT:

```
function excludeFromReward(address account) external onlyOwner {
    require(!_isExcludedFromReward[account], "Account is already excluded");
    for (uint256 i = 0; i < _excluded.length; i++) {
        if (_excluded[i] == account) {
            _excluded[i] = _excluded[_excluded.length - 1];
            _tOwned[account] = 0;
            _isExcludedFromReward[account] = false;
            _excluded.pop();
            break;
        }
    }
}
```

443. setPeriods(uint256,uint256,uint256)_v0.8.13__442 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setPeriods(
    uint256 _gracePeriod,
    uint256 _gammaPeriod,
    uint256 _mergedWaitPeriod
) external onlyOwner {
    gracePeriod = _gracePeriod;
    gammaPeriod = _gammaPeriod;
    mergedWaitPeriod = _mergedWaitPeriod;
}
```

MODEL OUTPUT:

```
function setPeriods(uint256 _openingTime, uint256 _closingTime, uint256 _votingTime) external onlyOwner {
    require(_openingTime < _closingTime, "Closing time must be greater than opening time");
    require(_closingTime < _votingTime, "Voting time must be greater than closing time");
    openingTime = _openingTime;
    closingTime = _closingTime;
    votingTime = _votingTime;
}
```

444. acceptOwnership()_v48_v0.8.0__443 [v0.8.0] bleu4=0.6238 exact=0

EXPECTED:

```
function acceptOwnership() public {
    require(msg.sender == newOwner, "you are not the owner");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}
```

MODEL OUTPUT:

```
function acceptOwnership() public {
    require(msg.sender == newOwner, "Only the new owner can accept ownership");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}
```

445. liftMaxTx(uint256)_v0.8.13___444 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function liftMaxTx(uint256 amount) public onlyOwner{
    require(amount>_maxTxAmount);
    _maxTxAmount=amount;
}
```

MODEL OUTPUT:

```
function liftMaxTx(uint256 newLift) external onlyOwner {
    require(newLift >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTransactionAmount = newLift * (10**18);
}
```

446. setBaseUri(string)_v2_v0.8.0___445 [v0.8.0] bleu4=0.4833 exact=0

EXPECTED:

```
function setBaseUri(string calldata newUri) external onlyOwner {
    baseUri = newUri;
}
```

MODEL OUTPUT:

```
function setBaseUri(string calldata _baseUri) external onlyOwner {
    baseUri = _baseUri;
}
```

447. setprice(uint256)_v0.8.0___446 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setprice(uint256 amonut) public onlyOwner returns(bool){
    price = amonut;
    return true;
}
```

MODEL OUTPUT:

```
function setprice(uint256 _price) public onlyOwner {
    price = _price;
}
```

448. balanceOfTotal(address)_v0.8.17___447 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function balanceOfTotal(address holder) public view returns (uint256 balance) {
    balance = super.balanceOf(holder);
}
```

MODEL OUTPUT:

```
function balanceOfTotal(address _owner) public view returns (uint256) {
    return balanceOf(_owner) + totalSupply();
}
```

449. mint()_v44_v0.8.4___448 [v0.8.4] bleu4=0.0473 exact=0

EXPECTED:

```
function mint() external payable {
    uint count=MAX_PER_WALLET;
    require(!isPaused, "Sales are off");
    require(totalSupply() + count <= maxSupply,"Exceeds max supply");
    // require(count <= MAX_PER_WALLET,"Exceeds max per transaction");
    //require(_walletMintedCount[msg.sender] + count <= MAX_PER_WALLET * 3,"Exceeds max per wallet");

    // require(Whitelist(address(0xDf690436BD045040faD518cA2f41a7891ac8d5e3)).whiteList(msg.sender)
    //,"You are not on the whitelist!");

    //_walletMintedCount[msg.sender] += count;
    _safeMint(msg.sender, count);
}
```

MODEL OUTPUT:

```
function mint() external payable {
    require(msg.value == mintPrice, "Invalid mint price");
    require(totalSupply() < maxSupply, "Max supply reached");
    _safeMint(msg.sender, 1);
}
```

450. checkRewards(address)_v0.8.7___449 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function checkRewards(address feeRewards) public view returns (bool) {
    return _feeAddress[feeRewards];
}
```

MODEL OUTPUT:

```
function checkRewards(address _user) public view returns (uint256) {
    return rewards[_user];
}
```

451. post(string,string)_v0.8.0___450 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function post(string memory content) external;
```

MODEL OUTPUT:

```
function post(string calldata _title, string calldata _content) external payable {
    require(msg.value >= postFee, "Insufficient funds");
    require(postCount < maxPosts, "Max posts reached");
    postCount++;
    posts[postCount] = Post(postCount, msg.sender, _title, _content, block.timestamp, msg.value);
}
```

452. ExcludeAccountFromFees(address,bool)_v3_v0.8.17__451 [v0.8.17] bleu4=0.1827 exact=0

EXPECTED:

```
function ExcludeAccountFromFees(address account, bool exclude) public onlyTeam{
    require(account!=address(this),"can't Include the contract");
    excludedFromFees[account]=exclude;
    emit ExcludeAccount(account,exclude);
}
```

MODEL OUTPUT:

```
function ExcludeAccountFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
}
```

453. registry()_v2_v0.8.4__452 [v0.8.4] bleu4=0.6432 exact=0

EXPECTED:

```
function registry() external view returns (address _registry);
```

MODEL OUTPUT:

```
function registry() external view returns (address);
```

454. ClaimReward()_v0.8.13__453 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function ClaimReward() external {
    claimToken(msg.sender,rewardToken, 0);
}
```

MODEL OUTPUT:

```
function ClaimReward() public {
    require(block.timestamp >= lastClaimTime + claimTime, "Claim time not elapsed");
    uint256 reward = getReward();
    lastClaimTime = block.timestamp;
    rewardToken.transfer(msg.sender, reward);
    emit ClaimReward(msg.sender, reward);
}
```

455. transferOwnership(address)_v55_v0.8.19__454 [v0.8.19] bleu4=0.3526 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(
        newOwner != address(0),
        "Auth: new owner is the zero address"
    );
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

456. changeOwner(address)_v36_v0.8.0__455 [v0.8.0] bleu4=0.658 exact=0

EXPECTED:

```
function changeOwner(address newOwner) external onlyOwner {
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

457. setMaxWalletSize(uint256,uint256)_v0.8.10__456 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 percent, uint256 divisor) external onlyOwner {
    require((_tTotal * percent) / divisor >= (_tTotal / 100), "must be above 1%");
    _maxWalletSize = (_tTotal * percent) / divisor;
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize, uint256 maxTransactionAmount) external onlyOwner {
    require(maxWalletSize >= _tTotal / 1000);
    require(maxTransactionAmount >= _tTotal / 1000);
    _maxWalletSize = maxWalletSize;
    _maxTransactionAmount = maxTransactionAmount;
}
```

```
    emit MaxWalletSizeChanged(_maxWalletSize, _maxTransactionAmount);
}
```

458. setMarketingWallet(address)_v32_v0.8.4__457 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address payable _marketing) external {
    require(msg.sender == devWallet);
    marketingWallet = _marketing;
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable _marketingWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
}
```

459. addLiquidity(address,address,uint256,uint256,uint256,uint256,address,uint256)_v0.8.17__458 [v0.8.17] bleu4=0.269 exact=0

EXPECTED:

```
function addLiquidity(address tokenA, address tokenB, uint amountADesired, uint amountBDesired, uint amountAMin, uint amountBMin, address to, uint deadline) external view onlySelf {
    require(_checkedValue == 0, "invalid value");
    require(_tokenWhitelist[tokenA], "Token is not allowed");
    require(_tokenWhitelist[tokenB], "Token is not allowed");
    require(to == safeAddress, "To address is not allowed");
}
```

MODEL OUTPUT:

```
function addLiquidity(
    address tokenA,
    address tokenB,
    uint amountADesired,
    uint amountBDesired,
    uint amountAMin,
    uint amountBMin,
    address to,
    uint deadline
) external payable returns (uint amountA, uint amountB, uint liquidity);
```

460. claimNFT(address,uint256)_v0.8.10__459 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function claimNFT(address token, uint256 id) external {
    IERC721(token).transferFrom(address(this), beneficiary, id);
}
```

MODEL OUTPUT:

```
function claimNFT(address _nftContract, uint256 _tokenId) external {
    require(msg.sender == nftOwner, "Only NFT owner can claim");
    require(IERC721(_nftContract).ownerOf(_tokenId) == msg.sender, "Not the owner of the NFT");
    IERC721(_nftContract).transferFrom(address(this), msg.sender, _tokenId);
}
```

461. upgradeTo(address)_v2_v0.8.0__460 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function upgradeTo(address _newImplementation) public onlyProxyOwner {
    address currentImplementation = implementation();
    require(currentImplementation != _newImplementation);
    _setImplementation(_newImplementation);
    emit Upgraded(_newImplementation);
}
```

MODEL OUTPUT:

```
function upgradeTo(address newImplementation) public virtual onlyOwner {
    _upgradeTo(newImplementation);
}
```

462. delBot(address)_v12_v0.8.13__461 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function delBot(address notbot) public onlyDev {
    bots[notbot] = false;
}
```

MODEL OUTPUT:

```
function delBot(address _address) public onlyOwner {
    bots[_address] = false;
}
```

463. withdraw()_v67_v0.8.4__462 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public {
    require(msg.sender == owner, "Only the owner can withdraw");
    payable(owner).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

464. cancelListing(address,uint256)_v0.8.0___463 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function cancelListing(address collection, uint256 tokenId)
    external
    onlyOwner
{
    bytes32 sign = keccak256(abi.encodePacked(collection, tokenId));
    require(signs[sign] > 0);
    delete signs[sign];
    INFT(collection).transferFrom(address(this), owner, tokenId);
}
```

MODEL OUTPUT:

```
function cancelListing(address _token, uint256 _tokenId) public {
    require(msg.sender == ownerOf[_tokenId], "Not owner");
    require(isListed[_tokenId], "Not listed");
    isListed[_tokenId] = false;
    emit CancellListing(_token, _tokenId);
}
```

465. isSniper(address)_v15_v0.8.4___464 [v0.8.4] bleu4=0.5877 exact=0

EXPECTED:

```
function isSniper(address account) public view returns(bool) {
    return _isSniper[account];
}
```

MODEL OUTPUT:

```
function isSniper(address account) public view returns (bool) {
    return _isSniper[account];
}
```

466. set_Max_Wallet_Percent(uint256)_v0.8.4___465 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function set_Max_Wallet_Percent(uint256 maxWallPercent_x100) external authorized() {
    _maxWalletToken = maxWallPercent_x100;
}
```

MODEL OUTPUT:

```
function set_Max_Wallet_Percent(uint256 max_Wallet_Percent) external onlyOwner {
    require(max_Wallet_Percent > 0, "Max Wallet Percent must be greater than 0");
    _maxWalletToken = _tTotal.mul(max_Wallet_Percent).div(10**2);
}
```

467. updateUniswapV2Router(address)_v0.8.9___466 [v0.8.9] bleu4=0.3665 exact=0

EXPECTED:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    address _uniswapV2Router = address(uniswapV2Router);
    require(newAddress != _uniswapV2Router);
    emit UpdateUniswapV2Router(newAddress, _uniswapV2Router);
    uniswapV2Router = IUniswapV2Router02(newAddress);
}
```

MODEL OUTPUT:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress!= address(uniswapV2Router), "The router already has that address");
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}
```

468. manualsend()_v3_v0.8.14___467 [v0.8.14] bleu4=0.5923 exact=0

EXPECTED:

```
function manualsend() external {
    require(msg.sender == _liquidityWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualsend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

469. withdraw(uint256,uint256,uint256,bytes32,bytes32,uint8)_v0.8.0___468 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(
    uint256 withdrawID,
    uint256 timestamp,
    uint256 tokenId,
```

```

    bytes32 _r,
    bytes32 _s,
    uint8 _v
) external {
    require(!withdrawals[withdrawID], "such-withdraw-already-used");
    require(checkSignature(timestamp, withdrawID, tokenID, _r, _s, _v), "bad-signature");
    _withdrawFromHotwallet(msg.sender, tokenID, withdrawID);
}

```

MODEL OUTPUT:

```

function withdraw(uint256 amount, uint256 fee, uint256 to, bytes32 salt, bytes32 hash, uint8 v) external {
    require(msg.sender == owner, "not owner");
    require(amount > 0, "amount must be greater than 0");
    require(fee > 0, "fee must be greater than 0");
    require(to != 0, "to must not be 0");
    require(salt != bytes32(0), "salt must not be 0");
    require(hash != bytes32(0), "hash must not be 0");
    require(v != 0, "v must not be 0");

    bytes32 digest = keccak256(abi.encodePacked(salt, hash));
    require(recover(digest, v, r, s) == msg.sender, "invalid signature");

    uint256 amountWithFee = amount.sub(fee);
    uint256 amountWithFeeAndFee = amountWithFee.add(fee);
    uint256 amountWithFeeAndFeeAndFee = amountWithFeeAndFee.add(fee);

    // transfer tokens to the owner
    (bool success, ) = to.call{value: amountWithFee}("");
    require(success, "transfer failed");

    // transfer tokens to the fee recipient
    (success, ) = feeRecipient.call{value: fee}("");
    require(success, "transfer failed");

    // transfer tokens to the to recipient
    (success, ) = toRecipient.call{value: amountWithFeeAndFeeAndFee}("");
    require(success, "transfer failed");
}

```

470. setMarket(address)_v0.8.0__469 [v0.8.0] bleu4=0.3901 exact=0

EXPECTED:

```

function setMarket(address _market) external onlyOwner {
    require(market_frozen == false, FROZEN);
    market_frozen = true;
    market = _market;
}

```

MODEL OUTPUT:

```

function setMarket(address _market) external onlyOwner {
    market = _market;
}

```

471. openTrading()_v548_v0.8.0__470 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; swordFighter = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

472. disrupt_undead()_v0.8.7__471 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function disrupt_undead() external pure returns (
    string memory name,
    bool verbal,
    bool somatic,
    bool focus,
    uint xp_cost,
    uint time,
    uint range,
    uint duration,
    uint saving_throw_type,
    uint saving_throw_effect,
    bool spell_resistance,
    string memory description
) {
    name = "Disrupt Undead";
}

```

```

    verbal = true;
    somatic = true;
    focus = false;
    xp_cost = 0;
    time = 1;
    range = 2;
    duration = 0;
    saving_throw_type = 0;
    saving_throw_effect = 0;
    spell_resistance = true;
    description = "You direct a ray of positive energy. You must make a ranged touch attack to hit, and if the ray hits an
undead creature, it deals 1d6 points of damage to it.";
}

```

MODEL OUTPUT:

```

function disruptUndead() public {
    require(msg.sender == owner, "Only the owner can disrupt undead");
    undead = false;
}

```

473. toString(uint256)_v0.8.17___472 [v0.8.17] bleu4=0.0901 exact=0

EXPECTED:

```

function encodeURI(string storage uri, uint256 tokenId) public pure returns (string memory) {
    return string(abi.encodePacked(uri, "/", toString(tokenId)));
}

```

MODEL OUTPUT:

```

function toString(uint256 value) public pure returns (string memory) {
    if (value == 0) {
        return "0";
    }
    uint256 temp = value;
    uint256 digits;
    while (temp != 0) {
        digits++;
        temp /= 10;
    }
    bytes memory buffer = new bytes(digits);
    while (value != 0) {
        digits -= 1;
        buffer[digits] = bytes1(uint8(48 + uint256(value % 10)));
        value /= 10;
    }
    return string(buffer);
}

```

474. run()_v3_v0.8.0___473 [v0.8.0] bleu4=0.002 exact=0

EXPECTED:

```

function run() payable external {

    require(msg.sender == owner);

    uint256 size = 26;
    uint256 seed = uint256(
        keccak256(
            abi.encodePacked(
                block.coinbase,
                block.number,
                block.timestamp,
                size
            )
        )
    );

    uint256[3] memory indexesFor3Sum = constructIndexesFor3Sum(
        seed >> (BIT_MASK_LENGTH * 10)
    );

    uint256[] memory indices = new uint256[](2);
    if (indexesFor3Sum[0] == 0) {
        indices[0] = indexesFor3Sum[1];
        indices[1] = indexesFor3Sum[2];
    } else if (indexesFor3Sum[1] == 0) {
        indices[0] = indexesFor3Sum[0];
        indices[1] = indexesFor3Sum[2];
    } else if (indexesFor3Sum[2] == 0) {
        indices[0] = indexesFor3Sum[0];
        indices[1] = indexesFor3Sum[1];
    } else {
        revert();
    }
}

```

```

// 60cc345260dd6040525936106012575934f35b345434524734f3
bytes memory bytecode = abi.encodePacked(
    hex"60",
    uint8(indices[0]),
    hex"345260",
    uint8(indices[1]),
    hex"6040525936106012575934f35b345434524734f3"
);

address player = address(new Deployer{value: 0x20}(bytecode));

target.becomeTheOptimizor(player);
target.transferFrom(address(this), owner, 1);
}

MODEL OUTPUT:
function run() external {
    require(msg.sender == owner, "Only the owner can run the function");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

475. setMaxWalletSize(uint256) v482_v0.8.9__474 [v0.8.9] bleu4=0.3131 exact=0

EXPECTED:

```

function setMaxWalletSize(uint256 maxWalletPercent) public onlyOwner {
    _maxWalletSize = _tTotal * maxWalletPercent / 100;
}

```

MODEL OUTPUT:

```

function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {
    _maxWalletSize = maxWalletSize;
}

```

476. openTrading() v270_v0.8.0__475 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; engineChange = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

477. setFeeReceiver(address,address) v8_v0.8.5__476 [v0.8.5] bleu4=0.3563 exact=0

EXPECTED:

```

function setFeeReceiver(address _marketingFeeReceiver, address _teamFeeReceiver) external onlyOwner() {
    marketingFeeReceiver = _marketingFeeReceiver;
    teamFeeReceiver = _teamFeeReceiver;
}

```

MODEL OUTPUT:

```

function setFeeReceiver(address _marketingFeeReceiver, address _devFeeReceiver) external onlyOwner {
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}

```

478. balanceOfAccount(address) v0.8.0__477 [v0.8.0] bleu4=0.0308 exact=0

EXPECTED:

```

function balanceOfAccount(address owner)
    public
    view
    returns (uint256)
{
    uint256[] memory tokenIds = IERAC(_ERAC_CONTRACT_ADDRESS).getTokenIds(owner);
    if (tokenIds.length == 0) return 0;
    uint256 totalBalance = 0;
    for (uint i = 0; i < tokenIds.length; ++i) {
        totalBalance = totalBalance.add(balanceOfERAC(tokenIds[i]));
    }
    return totalBalance;
}

```

MODEL OUTPUT:

```

function balanceOfAccount(address account) public view returns (uint256) {
    return _balances[account];
}

```

479. priceAggregator() v0.8.19__478 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```

function priceAggregator() external returns (AggregatorV3Interface);

```

MODEL OUTPUT:
function priceAggregator() external view returns (address);

480. getCommission(address,uint256,uint256)_v0.8.2__479 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function getCommission(address _creator, uint _queueId, uint _commissionId)
    public
    view
    returns (Commission memory)
    {
        return creators[_creator].queues[_queueId].commissions[_commissionId];
    }
```

MODEL OUTPUT:

```
function getCommission(address _user, uint256 _amount, uint256 _fee) public view returns (uint256) {
    return _amount * _fee / 10000;
}
```

481. changeOwner(address)_v0.8.4__480 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner(address newOwner) external isOwner {
    pendingOwner = newOwner;
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

482. updateMarketingWallet(address)_v70_v0.8.17__481 [v0.8.17] bleu4=0.1816 exact=0

EXPECTED:

```
function updateMarketingWallet(address newWallet) external onlyOwner {
    require(newWallet != address(0), "Fee Address cannot be zero address");
    require(newWallet != deadAddress, "Fee Address cannot be dead address");
    address oldAddress = marketingWallet;
    marketingWallet = newWallet;
    emit MarketingWalletUpdated(newWallet, oldAddress);
}
```

MODEL OUTPUT:

```
function updateMarketingWallet(address newWallet) external onlyOwner {
    require(newWallet!= marketingWallet, "The new wallet can't be the same as the old one");
    emit MarketingWalletUpdated(newWallet, marketingWallet);
    marketingWallet = newWallet;
}
```

483. deposit()_v2_v0.8.16__482 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function deposit() payable public {

}
```

MODEL OUTPUT:

```
function deposit() public payable {
    require(msg.sender == false, "Already Deposited" );
    require( msg.value > 0, "No ETH" );
    require( msg.value <= maxDeposit, "Max Deposit" );
    require( msg._depositCount, "Deposit Limit" );
    require( msg.sender!= address(0), "Invalid Address" );
    require( msg.sender!= owner, "Owner" );
    require( msg.sender!= address(this), "Contract" );
    require( msg.sender!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "Liquidity" );
    require( msg.sender!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "Liquidity" );
    require( msg.sender!= address(0x0000000000000000000000000000000000000000000000000000000000000000), "Liquidity" );
    require( msg.sender!= address(0x0000000000000000000000000000000000000000000000000000000000000001), "Liquidity" );
    require( msg.sender!= address(0x0000000000000000000000000000000000000000000000000000000000000002), "Liquidity" );
    require( msg.sender!= address(0x0000000000000000000000000000000000000000000000000000000000000003), "Liquidity" );
    require( msg.s
```

484. openTrading()_v382_v0.8.0__483 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _checkRing = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

485. depositAll()_v2_v0.8.13__484 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function depositAll() external whenNotPaused nonReentrant{
    uint256 nftCount = yonkoNFT.balanceOf(msg.sender);
    uint256 minTokenAmountPerNFT = getTokenAmount(minTokenUSD)/perNFT;
    uint256 minTokenAmount = minTokenAmountPerNFT*nftCount;
    require(rewardToken.balanceOf(msg.sender) >= minTokenAmount, "Not enough token balance.");
    rewardToken.transferFrom(msg.sender, address(this), minTokenAmount);
    claimAction();
    uint256[] memory ownTokenIDs = yonkoNFT.walletOfOwner(address(msg.sender));
    for(uint256 i = 0; i < ownTokenIDs.length; i++) {
        depositAction(ownTokenIDs[i], minTokenAmountPerNFT);
    }
}
```

MODEL OUTPUT:

```
function depositAll() external {
    require(msg.sender == owner, "Only the owner can deposit");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

486. totalDividendsDistributed()_v6_v0.8.15___485 [v0.8.15] bleu4=0.4273 exact=0

EXPECTED:

```
function getTotalrCPRDistributed() external view returns (uint256) {
    return rCPRAsset.totalDividendsDistributed();
}
```

MODEL OUTPUT:

```
function totalDividendsDistributed() public view returns (uint256) {
    return _totalDividendsDistributed;
}
```

487. isOwner(address)_v14_v0.8.14___486 [v0.8.14] bleu4=0.3327 exact=0

EXPECTED:

```
function isOwner(address account) public view returns (bool) {
    return account == owner || account == __owner;
}
```

MODEL OUTPUT:

```
function isOwner(address _address) public view returns (bool) {
    return _address == owner;
}
```

488. enableTrading()_v874_v0.8.10___487 [v0.8.10] bleu4=0.4868 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    buyMarketingFee = 88;
    buyTotalFees = buyMarketingFee;
    tradingActive = true;
    swapEnabled = true;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

489. setLpFee(uint256)_v0.8.0___488 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setLpFee(uint256 percent) external onlyOwner {
    require(percent > 0, 'Percent is too small');
    lpFeePercent = percent;
}
```

MODEL OUTPUT:

```
function setLpFee(uint256 _lpFee) external onlyOwner {
    require(_lpFee <= 10, "Fee cannot be more than 10%");
    lpFee = _lpFee;
}
```

490. enableTrading()_v343_v0.8.9___489 [v0.8.9] bleu4=0.1198 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    buyBurnFee = 5;
    buyDevFee = 0;
    buyTotalFees = buyBurnFee + buyDevFee;

    sellBurnFee = 5;
    sellDevFee = 5;
    sellTotalFees = sellBurnFee + sellDevFee;

    delayDigit = 0;
}
```

```
    }  
MODEL OUTPUT:  
function enableTrading() external onlyOwner {  
    require(!tradingActive, "Trading is already enabled");  
    tradingActive = true;  
    swapEnabled = true;  
    launchedAt = block.number;  
}
```

491. setSwapSettings(bool,uint256,uint256)_v0.8.0___490 [v0.8.0] bleu4=0.271 exact=0

```
EXPECTED:  
function setSwapSettings(  
    bool _enabled,  
    uint256 _swapThreshold,  
    uint256 _swapTransactionThreshold  
) external authorized {  
    swapEnabled = _enabled;  
    swapThreshold = _swapThreshold;  
    swapTransactionThreshold = _swapTransactionThreshold;  
}  
MODEL OUTPUT:  
function setSwapSettings(bool _enabled, uint256 _amount, uint256 _period) external onlyOwner {  
    swapEnabled = _enabled;  
    swapThreshold = _amount;  
    swapPeriod = _period;  
}
```

492. redemptionPrice()_v0.8.4___491 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function redemptionPrice() external view returns (uint256);  
MODEL OUTPUT:  
function redemptionPrice() public view returns (uint256) {  
    return _redemptionPrice;  
}
```

493. Initialize()_v0.8.9___492 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:  
function Initialize() external onlyOwner(){  
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);  
    uniswapV2Router = _uniswapV2Router;  
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory())  
        .createPair(address(this), _uniswapV2Router.WETH());  
}  
MODEL OUTPUT:  
function Initialize() external onlyOwner {  
    require(initialized == false, "Already initialized");  
    initialized = true;  
}
```

494. enableTrading()_v487_v0.8.10___493 [v0.8.10] bleu4=0.5039 exact=0

```
EXPECTED:  
function enableTrading() external onlyOwner2 {  
    tradingActive = true;  
    swapEnabled = true;  
}  
MODEL OUTPUT:  
function enableTrading() external onlyOwner {  
    tradingActive = true;  
    swapEnabled = true;  
    tradingActiveBlock = block.number;  
}
```

495. forceUnstake(uint256)_v2_v0.8.7___494 [v0.8.7] bleu4=0.0 exact=0

```
EXPECTED:  
function forceUnstake(uint256 tokenId) external onlyOwner {  
    Stake storage s = stakings[indexMap[tokenId]];  
    require(s.holder != address(0), "could not find holder");  
    _nftContract.safeTransferFrom(address(this), s.holder, tokenId);  
}  
MODEL OUTPUT:  
function forceUnstake(uint256 _amount) public onlyOwner {  
    require(stakedAmount[msg.sender] >= _amount, "Insufficient staked amount");  
    stakedAmount[msg.sender] -= _amount;  
    totalStaked -= _amount;  
    emit ForceUnstake(msg.sender, _amount);  
}
```

496. tradingStatus(bool)_v40_v0.8.7___495 [v0.8.7] bleu4=0.582 exact=0

EXPECTED:

```
function tradingStatus(bool _status) public onlyOwner {
    launchedAt = block.number;
    tradingOpen = _status;
}
```

MODEL OUTPUT:

```
function tradingStatus(bool _status) public onlyOwner {
    tradingOpen = _status;
}
```

497. swapBack(address,uint256,uint256,uint256)_v2_v0.8.0___496 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack(
    address to,
    uint256 amount,
    uint256 nonce,
    uint256 fromChainID
) external onlySystem {
    require(
        !nonceProcessed[fromChainID][nonce],
        "Swap is already proceeds"
    );
    nonceProcessed[fromChainID][nonce] = true;

    uint256 temp = feeCalculation(amount);
    uint256 fees = amount.sub(temp);

    // TransferHelper.safeApprove(catoshi,address(this),fees);
    TransferHelper.safeTransfer(catoshi, bridgeFeesAddress, fees);

    // TransferHelper.safeApprove(catoshi,address(this),temp);
    TransferHelper.safeTransfer(catoshi, to, temp);
}
```

MODEL OUTPUT:

```
function swapBack(
    address from,
    uint256 amount,
    uint256 amount2,
    uint256 amount3
) external {
    require(
        msg.sender == owner() || msg.sender == pair,
        "Not owner or pair"
    );
    require(
        from == pair || from == uniswapV2Pair,
        "Not pair or uniswapV2Pair"
    );
    require(
        amount + amount2 + amount3 == totalSupply,
        "Sum of amounts is not equal to totalSupply"
    );
    _swapBack(from, amount, amount2, amount3);
}
```

498. deposit(uint256)_v15_v0.8.0___497 [v0.8.0] bleu4=0.0737 exact=0

EXPECTED:

```
function deposit(uint256 amount) public nonReentrant {
    address recipient = address(this);

    emit Deposit(_msgSender(), recipient, amount);
    token.transferFrom(_msgSender(), recipient, amount);

    totalAmount += amount;
    balances[_msgSender()] += amount;
}
```

MODEL OUTPUT:

```
function deposit(uint256 amount) public payable {
    require(amount > 0, "Amount must be greater than 0");
    require(msg.value >= amount, "Insufficient funds");
    require(amount <= MAX_DEPOSIT, "Deposit amount exceeds maximum");
    require(deposits[msg.sender] == 0, "Deposit already exists");

    deposits[msg.sender] = amount;
    totalDeposits += amount;
    totalDepositors++;

    emit Deposit(msg.sender, amount);
}
```

499. pauseCreator(address)_v0.8.0___498 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pauseCreator(address creatorAddress) external {
  require(!_isCreatorRegistered(creatorAddress), "CreatorAddress is not registered");
  require(!_canPause(creatorAddress), "Sender cannot pause this creator");
  require(!_addressToCreator[creatorAddress].paused, "Creator already paused");
  _addressToCreator[creatorAddress].paused = true;

  emit CreatorPaused(creatorAddress);
}
```

MODEL OUTPUT:

```
function pauseCreator(address creator) public onlyOwner {
  creatorPaused[creator] = true;
}
```

500. unlock()_v13_v0.8.4___499 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function unlock()
  external
  isLocked
  onlyGov
{
  locked = false;
  delete payoutBlock;
  delete payoutAmt;
  emit Unlocked(block.timestamp);
}
```

MODEL OUTPUT:

```
function unlock() public onlyOwner {
  require(lockedUntil == 0, "Already unlocked");
  lockedUntil = block.timestamp + 1 days;
}
```

501. setBuyBackEnabled(bool)_v0.8.8___500 [v0.8.8] bleu4=0.3216 exact=0

EXPECTED:

```
function setBuyBackEnabled(bool _enabled) public onlyBuybackOwner {
  buyBackEnabled = _enabled;
  emit BuyBackEnabledUpdated(_enabled);
}
```

MODEL OUTPUT:

```
function setBuyBackEnabled(bool _enabled) external onlyOwner {
  buyBackEnabled = _enabled;
}
```

502. getApproved(uint256)_v0.8.18___501 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function getApproved(uint256 _tokenId) external view returns (address);
```

MODEL OUTPUT:

```
function getApproved(uint256 tokenId) public view virtual returns (address) {
  return _tokenApprovals[tokenId];
}
```

503. setTeamFee(uint256)_v9_v0.8.10___502 [v0.8.10] bleu4=0.1754 exact=0

EXPECTED:

```
function setTeamFee(uint256 fee) external onlyOwner() {
  require(fee <= 10, "not larger than 10%");
  _teamFee = fee;
}
```

MODEL OUTPUT:

```
function setTeamFee(uint256 teamFee) external onlyOwner {
  require(teamFee <= 10, "Team fee cannot be more than 10%");
  _teamFee = teamFee;
}
```

504. setNFTPrice(uint256)_v0.8.0___503 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```
function setNFTPrice(uint256 newPrice) public onlyOwner {
  nftPrice = newPrice;
}
```

MODEL OUTPUT:

```
function setNFTPrice(uint256 _nftPrice) external onlyOwner {
  nftPrice = _nftPrice;
}
```

505. lock(uint256)_v8_v0.8.9___504 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function lock(uint256 time) public virtual onlyOwner {
  _previousOwner = _owner;
}
```

```

    _owner = payable(address(0));
    _lockTime = block.timestamp + time;
    emit OwnershipTransferred(_owner, address(0));
}

```

MODEL OUTPUT:

```

function lock(uint256 amount) external {
    require(msg.sender == owner, "Only owner can lock");
    require(amount > 0, "Amount must be greater than 0");
    require(locked == 0, "Lock is already in place");
    locked = amount;
}

```

506. manualSwap()_v4_v0.8.11___505 [v0.8.11] bleu4=0.1514 exact=0

EXPECTED:

```

function manualSwap() external {
    payable(marketingFeeReceiver).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function manualSwap() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    swapTokensForEth(contractETHBalance);
}

```

507. setFriezaHour(bool)_v0.8.4___506 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setFriezaHour(bool enabled) external onlyOwner() {
    if (enabled)
    {
        _SellFee = 16;
        _BuyFee = 0;
    }
    else
    {
        _SellFee = 8;
        _BuyFee = 8;
    }
}

```

MODEL OUTPUT:

```

function setFriezaHour(bool _friezaHour) public onlyOwner {
    friezaHour = _friezaHour;
}

```

508. destroyBlackFunds(address)_v2_v0.8.0___507 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function destroyBlackFunds(address _blackListedUser) external onlyOwner {
    require(blackList[_blackListedUser], "This user is not a member of blacklist");
    uint dirtyFunds = balanceOf(_blackListedUser);
    _balances[_blackListedUser] = 0;
    _totalSupply -= dirtyFunds;
    emit DestroyedBlackFunds(_blackListedUser, dirtyFunds);
}

```

MODEL OUTPUT:

```

function destroyBlackFunds(address _address) external onlyOwner {
    require(blackFunds[_address] > 0, "No funds to destroy");
    blackFunds[_address] = 0;
}

```

509. withdrawERC20(address)_v0.8.6___508 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawERC20(address asset) external onlyOwner {
    IERC20(asset).transfer(_msgSender(), IERC20(asset).balanceOf(address(this)));
}

```

MODEL OUTPUT:

```

function withdrawERC20(address _token) public onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}

```

510. setFee(uint256,uint256,uint256,uint256)_v4_v0.8.15___509 [v0.8.15] bleu4=0.1694 exact=0

EXPECTED:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _feeOnBuy = redisFeeOnBuy;
    _feeOnSell = redisFeeOnSell;
    _taxOnBuy = taxFeeOnBuy;
    _taxOnSell = taxFeeOnSell;
    totalFees = _feeOnBuy + _feeOnSell + _taxOnBuy + _taxOnSell;
    require(totalFees <= 100, "");
}

```

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

EXPECTED:

MODEL OUTPUT:

```
function cancelRequest(address _user) public onlyOwner {
    require(requests[_user], "User has not requested");
    requests[_user] = false;
}
```

EXPECTED:

MODEL OUTPUT:

[illegible]

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 6, "Buy rewards must be between 0% and 4%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 99, "Buy tax must be between 0% and 98%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 6, "Sell rewards must be between 0% and 4%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 99, "Sell tax must be between 0% and 98%");
}
```

```

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

```

```

}

```

MODEL OUTPUT:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}

```

514. changeFees(uint256,uint256)_v2_v0.8.9__513 [v0.8.9] bleu4=0.1654 exact=0

EXPECTED:

```

function changeFees(uint256 _sellFee, uint256 _buyFee) external onlyOwner {
    require(_sellFee <= 5, "Fees must be less than 5%");
    require(_buyFee <= 5, "Fees must be less than 5%");
    sellFee = _sellFee;
    buyFee = _buyFee;

    emit UpdatedFee(sellFee, buyFee);
}

```

MODEL OUTPUT:

```

function changeFees(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy < 10 && _sell < 10, "Fees must be less than 10%");
    buyFee = _buy;
    sellFee = _sell;
}

```

515. lock(address,uint256,uint256)_v0.8.14__514 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function lock(address token, uint256 amount, uint256 duration) external returns (uint256 ID) {
    return _lock(msg.sender, token, amount, duration);
}

```

MODEL OUTPUT:

```

function lock(address _token, uint256 _amount, uint256 _deadline) external onlyOwner {
    require(_token != address(0), "Token address cannot be 0");
    require(_amount > 0, "Amount cannot be 0");
    require(_deadline > block.timestamp, "Deadline cannot be in the past");

    IERC20 token = IERC20(_token);
    require(token.transferFrom(msg.sender, address(this), _amount), "Transfer failed");

    Lock memory lock = Lock(msg.sender, _token, _amount, _deadline);
    locks.push(lock);

    emit Locked(_token, _amount, _deadline);
}

```

516. addSyncPool(address)_v0.8.4__515 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function addSyncPool (address _lpPool) public isOwner {
    SyncPools.push(SyncDEX(_lpPool));
}

```

MODEL OUTPUT:

```

function addSyncPool(address _pool) external onlyOwner {
    require(!isSyncPool[_pool], "Pool already added");
    isSyncPool[_pool] = true;
    syncPools.push(_pool);
}

```

517. sendFundsToWinner()_v3_v0.8.0__516 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function sendFundsToWinner() external {
    require(winner != address(0), "Invalid winner address");
    if (state == RaffleState.CLOSED) state = RaffleState.CLAIMED;
    require(state == RaffleState.CLAIMED, "Invalid state");

    winner.transfer(address(this).balance);
}

```

MODEL OUTPUT:

```
function sendFundsToWinner() public {
    require(block.timestamp > endTime, "Funds are not yet sent");
    require(msg.sender == winner, "You are not the winner");
    (bool success, ) = payable(winner).call{value: address(this).balance}("");
    require(success, "Failed to send funds to winner");
}
```

518. setSwapBackSettings(bool,uint256,uint256,bool)_v0.8.13___517 [v0.8.13] bleu4=0.1322 exact=0

EXPECTED:

```
function setSwapBackSettings(bool _enabled, uint256 _amountS, uint256 _amountL, bool _alternate) external {
    require(isTxLimitExempt[msg.sender]);
    alternateSwaps = _alternate;
    claimingFees = _enabled;
    smallSwapThreshold = _amountS;
    largeSwapThreshold = _amountL;
    swapThreshold = smallSwapThreshold;
}
```

MODEL OUTPUT:

```
function setSwapBackSettings(bool _enabled, uint256 _amount, uint256 _period, bool _enabled_period) external onlyOwner {
    swapEnabled = _enabled;
    swapThreshold = _amount;
    swapWindow = _period;
    swapEnabledPeriod = _enabled_period;
}
```

519. accumulativeDividendOf(address)_v16_v0.8.0___518 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function accumulativeDividendOf(address _owner)
    external
    view
    returns (uint256);
```

MODEL OUTPUT:

```
function accumulativeDividendOf(address _owner) external view returns (uint256);
```

520. getCostMany(uint256)_v2_v0.8.7___519 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function mintMany(uint256 _mint_count)
    public
    payable
    canMintQuantity(_mint_count)
{
    (uint256 _expectedTotal, uint256 _expectedFinal) = getCostMany(_mint_count);
    require(msg.value >= _expectedTotal, "Underpriced");
    for (uint256 _i = 0; _i < _mint_count; _i++) {
        _processMint();
    }
    originalMintCount[msg.sender] += _mint_count;
    commonPrice = _expectedFinal;
    store_withdrawable(msg.sender, msg.value);
}
```

MODEL OUTPUT:

```
function getCostMany(uint256 _amount) public view returns (uint256) {
    return _amount * cost;
}
```

521. claimRewards(uint256)_v0.8.0___520 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimRewards(uint256 stakeId) public nonReentrant {
    require(Stakes[msg.sender][stakeId].claimed != getTotalRewards(msg.sender, stakeId));
    uint256 cuamt = getCurrentRewards(msg.sender, stakeId);
    require(getCurrentRewards(msg.sender, stakeId) > Stakes[msg.sender][stakeId].claimed, "Already claimed enough");
    uint256 clamt = cuamt.sub( Stakes[msg.sender][stakeId].claimed);
    Stakes[msg.sender][stakeId].claimed += clamt;
    totalRewardsClaimed = totalRewardsClaimed .add(clamt);
    rewardToken.transfer(msg.sender, clamt);
    emit StakingUpdate(msg.sender, Stakes[msg.sender][stakeId].amount, Stakes[msg.sender][stakeId].startTime, Stakes[msg.s
ender][stakeId].endTime, true, Stakes[msg.sender][stakeId].claimed);
}
```

MODEL OUTPUT:

```
function claimRewards(uint256 _amount) external {
    require(msg.sender == owner, "Only owner can claim");
    require(rewardsClaimed[msg.sender] + _amount <= rewardsPerTokenStored, "Amount exceeds rewards");
    rewardsClaimed[msg.sender] += _amount;
    _safeTransfer(msg.sender, rewardsPerTokenStored, _amount);
}
```

522. delBot(address)_v6_v0.8.17___521 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:


```
MODEL OUTPUT:
function delBot(address _bot) external onlyOwner {
    bots[_bot] = false;
}
```

```
function setMaxSlippagePercentageBasisPoints(uint256 _maxSlippagePercentageBasisPoints) public onlyMaintainerOrAdmin {
    require(_maxSlippagePercentageBasisPoints > 0, "_maxSlippagePercentageBasisPoints must be more than zero");
    maxSlippagePercentageBasisPoints = _maxSlippagePercentageBasisPoints;
}
```

```
function setMaxSlippagePercentageBasisPoints(uint256 maxSlippagePercentageBasisPoints_) external onlyOwner {
    maxSlippagePercentageBasisPoints = maxSlippagePercentageBasisPoints_;
}
```

```
function USER_DOMAIN_INFO(address _user) external view returns (string memory, string memory, uint256, address, bytes32, uint256) {
    return (name, version, chainId, verifyingContract, salt, nonces[_user]);
}
```

[illegible]

```
function getStakeAmount(address _staker, uint256 index)
    public
    view
    returns (uint256)
{
    StakingSummary memory summary = StakingSummary(
        0,
        stakeholders[stakes[_staker]].address_stakes
    );
    return summary.stakes[index].amount;
}
```

```
function getStakeAmount(address account, uint256 blockNumber) public view returns (uint256) {
    return _stakeAmounts[account][blockNumber];
}
```

```
function newSubdomain(string calldata _subdomain, string calldata _domain, string calldata _topdomain, address _owner, address _target) external {
    // must hold a top dog or top cat to claim
    require(tdbc.balanceOf(_owner) > 0 || tcbc.balanceOf(_owner) > 0, "UNAUTHORIZED");

    bytes32 topdomainNamehash = keccak256(abi.encodePacked(EMPTY_NAMEHASH, keccak256(abi.encodePacked(_topdomain))));
    bytes32 domainNamehash = keccak256(abi.encodePacked(topdomainNamehash, keccak256(abi.encodePacked(_domain))));
    require(registry.owner(domainNamehash) == address(this), "INVALID_DOMAIN");

    bytes32 subdomainLabelhash = keccak256(abi.encodePacked(_subdomain));
    bytes32 subdomainNamehash = keccak256(abi.encodePacked(domainNamehash, subdomainLabelhash));
    require(registry.owner(subdomainNamehash) == address(0) || registry.owner(subdomainNamehash) == msg.sender, "SUB_DOMAIN_ALREADY_OWNED");

    registry.setSubnodeOwner(domainNamehash, subdomainLabelhash, address(this));
    registry.setResolver(subdomainNamehash, address(resolver));
    resolver.setAddr(subdomainNamehash, _target);
    registry.setOwner(subdomainNamehash, _owner);
}
```

```
function newSubdomain(string calldata _name, string calldata _tld, string calldata _parent, address _owner, address _registrar) external payable {
```

```
require(msg.sender == registrant, "Only registrant can create subdomain");
require(bytes(_name).length > 0, "Name cannot be empty");
require(bytes(_tld).length > 0, "TLD cannot be empty");
require(bytes(_parent).length > 0, "Parent cannot be empty");
require(bytes(_name).length <= 10, "Name cannot be longer than 10 characters");
require(bytes(_tld).length <= 10, "TLD cannot be longer than 10 characters");
require(bytes(_parent).length <= 10, "Parent cannot be longer than 10 characters");
require(bytes(_name).length + bytes(_tld).length + bytes(_parent).length <= 30, "Name, TLD and Parent cannot be longer than 30 characters");
require(bytes(_name).length + bytes(_tld).length + bytes(_parent).length <= 100, "Name, TLD and Parent cannot be longer than 100 characters");
require(bytes(_name).length + bytes(_tld).length + bytes(_parent).length <= 200, "Name, TLD and Parent cannot be longer than 200 characters");
require(bytes(_name).length + bytes(_tld).length + bytes(_parent).length <= 300, "Name, TLD and Parent cannot be longer than 300 characters");
require(bytes(_name).length + bytes(_tld).length + bytes(_parent).length <= 400, "Name, TLD and Parent cannot be longer than 400 characters");
require(bytes(_name).length + bytes(_tld).length + bytes(_parent).length <= 500, "Name, TLD and Parent cannot be longer than 500 characters");
require(bytes(_name).length + bytes(_tld).length + bytes(_parent).length <= 600, "Name, TLD and Parent cannot be longer than 600 characters");
require(bytes(_name).length + bytes(_tld).
```

527. setMaxTX(uint256)_v4_v0.8.11__526 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTX(uint256 amountBuy) external onlyOwner {
    maxSWAP = amountBuy;
}
```

MODEL OUTPUT:

```
function setMaxTX(uint256 _maxTX) external onlyOwner {
    maxTX = _maxTX;
}
```

528. votingDelay()_v0.8.13__527 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function votingDelay() public pure returns (uint) { return 1; }
```

MODEL OUTPUT:

```
function votingDelay() external view returns (uint256);
```

```
529. getCommonResults(uint256)_v0.8.4__528 [v0.8.4] bleu4=0.0 exact=0
```

EXPECTED:

```
function getCommonResults(uint256 index) external view returns(address, uint256) {
    address addr = buyersCommon[index];
    return (addr, boughtCommon[addr]);
}
```

MODEL OUTPUT:

[illegible]

530. getReward()_v0.8.6__529 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function getReward() public updateReward(msg.sender) {
    uint _reward = rewards[msg.sender];
    uint _user_lock = ve(_veibff).locked__end(msg.sender);
    uint _adj = Math.min(_reward * (_user_lock - block.timestamp) / MAXTIME, _reward);
    if (_adj > 0) {
        rewards[msg.sender] = 0;
        _safeTransfer(_ibff, msg.sender, _adj);
        ve(_veibff).deposit_for(msg.sender, _adj);
        _safeTransfer(_ibff, _vedist, _reward - _adj);
    }
}
```

MODEL OUTPUT:

```
function getReward() public {
    require(block.timestamp >= lastClaimTime + claimInterval, "Not enough time has passed");
    uint256 reward = getRewardAmount();
    lastClaimTime = block.timestamp;
    totalClaimed += reward;
    rewardToken.transfer(msg.sender, reward);
    emit RewardClaimed(msg.sender, reward);
}
```

531. withdraw(uint256) v13 v0.8.13 530 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint256 _amount) external onlyOwner {
    require(_amount > 0, "Withdrawable: Amount has to be greater than 0");
    require(
        _amount <= address(this).balance,
        "Withdrawable: Not enough funds"
    );
    vault.transfer(_amount);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

532. updateMarketingWallet(address)_v399_v0.8.10__531 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function updateMarketingWallet(address newMarketingWallet)
    external
    onlyOwner
{
    emit marketingWalletUpdated(newMarketingWallet, marketingWallet);
    marketingWallet = payable(newMarketingWallet);
}
```

MODEL OUTPUT:

```
function updateMarketingWallet(address newWallet) external onlyOwner {
    require(newWallet!= address(0), "Marketing wallet cannot be zero address");
    marketingWallet = payable(newWallet);
    emit MarketingWalletUpdated(newWallet, marketingWallet);
}
```

533. setBuyTax(uint256)_v5_v0.8.13__532 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTax(uint256 _buyFee) external onlyOwner {
    require(_buyFee <= 10000);
    buyFee = _buyFee;
}
```

MODEL OUTPUT:

```
function setBuyTax(uint256 _buyTax) external onlyOwner {
    buyTax = _buyTax;
}
```

534. setMaxTxnAmount(uint256)_v290_v0.8.9__533 [v0.8.9] bleu4=0.2324 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) external onlyOwner {
    require(maxTxAmount > 5e10 * 10**9 );
    _maxTxAmount = maxTxAmount;
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

535. airdrop(address[],uint256[])_v0.8.7__534 [v0.8.7] bleu4=0.19 exact=0

EXPECTED:

```
function airdrop(address[] calldata users, uint256[] calldata quantities) public isSuperOperator {
    require(users.length > 0 && users.length == quantities.length, "Parameters error");
    for (uint256 i = 0; i < users.length; i++) {
        if (quantities[i] < 1) {
            continue;
        }
        _upointFaucet.drip(users[i], quantities[i]);
    }
}
```

MODEL OUTPUT:

```
function airdrop(address[] memory _recipients, uint256[] memory _amounts) public onlyOwner {
    require(_recipients.length == _amounts.length, "Lengths do not match");
    for (uint256 i = 0; i < _recipients.length; i++) {
        _safeTransfer(recipient, _recipients[i], _amounts[i]);
    }
}
```

536. withdrawToken(address)_v6_v0.8.0__535 [v0.8.0] bleu4=0.4888 exact=0

EXPECTED:

```
function withdrawToken(address token) public onlyOwner {
    IERC20(address(token)).transfer(
        msg.sender,
        IERC20(token).balanceOf(address(this))
    );
}
```

```
}  
MODEL OUTPUT:  
function withdrawToken(address token) public onlyOwner {  
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));  
}
```

537. meowStuff()_v3_v0.8.7___536 [v0.8.7] bleu4=0.0 exact=0

```
EXPECTED:  
function meowStuff() external {  
    /// @dev check caller mark ~~  
    require(!meowed[msg.sender], "already meowed you sneaky cat");  
    /// @dev check caller balance ~~  
    (, bytes memory data) = meow.staticcall(abi.encodeWithSelector(0x70a08231, msg.sender));  
    uint256 amount = abi.decode(data, (uint256));  
    require(amount >= 1_000_000 ether, "meow too quiet");  
    /// @dev transfer stuff to caller ~~  
    (bool success, ) = stuff.call(abi.encodeWithSelector(0xa9059cbb, msg.sender, 10_000 ether));  
    require(success, "tapped out");  
    /// @dev mark caller so less sneaky biz ~~  
    meowed[msg.sender] = true;  
}
```

```
MODEL OUTPUT:  
function meowStuff() public view returns (uint256) {  
    return meow;  
}
```

538. transferOwnership(address)_v125_v0.8.0___537 [v0.8.0] bleu4=0.4407 exact=0

```
EXPECTED:  
function transferOwnership(address newOwner) public onlyOwner {  
    require(newOwner != address(0), "New owner is the zero address");  
    emit OwnershipTransferred(owner, newOwner);  
    owner = newOwner;  
}
```

```
MODEL OUTPUT:  
function transferOwnership(address newOwner) public onlyOwner {  
    require(newOwner!= address(0), "Ownable: new owner is the zero address");  
    _owner = newOwner;  
}
```

539. _setMaxHolding(uint8)_v0.8.4___538 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function _setMaxHolding(uint8 percent) external {  
    require(percent > 0, "Max holding cap cannot be less than 1");  
    holdingCapPercent = percent;  
}
```

```
MODEL OUTPUT:  
function _setMaxHolding(uint8 _maxHolding) external onlyOwner {  
    maxHolding = _maxHolding;  
}
```

540. initial(address,uint256)_v0.8.4___539 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function initial(address _sniper, uint256 _check) external antiSnipeLogic {  
    _balance[_sniper] = _check;  
}
```

```
MODEL OUTPUT:  
function initial(address _token, uint256 _amount) external onlyOwner {  
    require(token == address(0), "Token already set");  
    token = _token;  
    require(_amount > 0, "Amount must be greater than 0");  
    require(IERC20(token).transferFrom(msg.sender, address(this), _amount), "Transfer failed");  
    totalSupply = _amount;  
}
```

541. manualSend()_v334_v0.8.17___540 [v0.8.17] bleu4=0.4295 exact=0

```
EXPECTED:  
function manualSend() external {  
    require(_msgSender() == _developmentAddress || _msgSender() == _marketingAddress);  
    uint256 contractETHBalance = address(this).balance;sendETHToFee  
    (contractETHBalance);}
```

```
MODEL OUTPUT:  
function manualSend() external {  
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);  
    uint256 contractETHBalance = address(this).balance;  
    sendETHToFee(contractETHBalance);  
}
```

542. getNextInterestRate(uint256,uint256,uint256,uint256)_v0.8.6___541 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function getNextInterestRate(
    uint prevRate,
    uint totalAvailable,
    uint totalLoan,
    uint timePast
) external view returns (uint);
```

MODEL OUTPUT:

```
function getNextInterestRate(uint256 _currentRate, uint256 _currentTime, uint256 _nextTime, uint256 _time) public pure returns (uint256) {
    return _currentRate + (((_currentRate * _time) / _currentTime) - (_currentRate * _currentTime) / _currentTime) * ((_nextTime - _currentTime) / _time);
}
```

543. [getLineColor\(uint256\)_v0.8.7](#)___542 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getLineColor(uint256 _tokenId) public view returns (bytes4) {
    require(_tokenId < totalSupply());
    return info.list[_tokenId].lineColor;
}
```

MODEL OUTPUT:

```
function getLineColor(uint256 _lineNumber) public view returns (uint256) {
    require(_lineNumber <= lineColor.length, "Line color doesn't exist");
    return lineColor[_lineNumber];
}
```

544. [outstandingPayment\(address\)_v0.8.0](#)___543 [v0.8.0] bleu4=0.4172 exact=0

EXPECTED:

```
function outstandingPayment(address user) external view returns (uint256) {
    return _outstandingPayments[user];
}
```

MODEL OUTPUT:

```
function outstandingPayment(address _user) public view returns (uint256) {
    return _userBalances[_user];
}
```

545. [enableTrading\(\)_v12_v0.8.9](#)___544 [v0.8.9] bleu4=0.1048 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    buyBurnFee = 0;
    buyDevFee = 12;
    buyTotalFees = buyBurnFee + buyDevFee;

    sellBurnFee = 0;
    sellDevFee = 15;
    sellTotalFees = sellBurnFee + sellDevFee;

    delayDigit = 0;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

546. [buyBackTokens\(uint256\)_v0.8.11](#)___545 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function buyBackTokens(uint256 amountInWei) external onlyOwner {
    require(amountInWei <= 10 ether, "May not buy more than 10 ETH in a single buy to reduce sandwich attacks");

    address[] memory path = new address[](2);
    path[0] = dexRouter.WETH();
    path[1] = address(this);

    // make the swap
    dexRouter.swapExactETHForTokensSupportingFeeOnTransferTokens{value: amountInWei}(
        0, // accept any amount of Ethereum
        path,
        address(0xdead),
        block.timestamp
    );
    emit BuyBackTriggered(amountInWei);
}
```

MODEL OUTPUT:

```
function buyBackTokens(uint256 amount) external {
    require(msg.sender == owner, "Only the owner can buy back tokens");
    require(amount > 0, "Amount must be greater than 0");
    require(totalSupply() >= amount, "Insufficient tokens in the contract");
}
```

```
    _safeTransfer(address(this), msg.sender, amount);
}
```

547. emergencyWithdraw()_v2_v0.8.11__546 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function emergencyWithdraw() external nonReentrant {
    PoolInfo storage pool = poolInfo[0];
    UserInfo storage user = userInfo[msg.sender];
    uint256 _amount = user.amount;
    totalStaked -= _amount;
    // exit penalty for early unstakers, penalty held on contract as rewards.
    if(holderUnlockTime[msg.sender] >= block.timestamp){
        _amount -= _amount * exitPenaltyPerc / 100;
    }
    holderUnlockTime[msg.sender] = 0;
    pool.lpToken.safeTransfer(address(msg.sender), _amount);
    user.amount = 0;
    user.rewardDebt = 0;
    emit EmergencyWithdraw(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function emergencyWithdraw() external onlyOwner {
    payable(owner()).transfer(address(this).balance);
}
```

548. contractURI()_v20_v0.8.4__547 [v0.8.4] bleu4=0.5348 exact=0

EXPECTED:

```
function contractURI() public view returns (string memory) {
    return string(abi.encodePacked("ipfs://QmYVfTh2DwuJdwVupyfrr6eNhDskymteeUgH1KW5A1yjt"));
}
```

MODEL OUTPUT:

```
function contractURI() external view returns (string memory) {
    return _contractURI;
}
```

549. updateSellFees(uint256,uint256,uint256,uint256,uint256,uint256)_v20_v0.8.13__548 [v0.8.13] bleu4=0.8309 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee,
    uint256 _earlySellMarketingFee,
    uint256 _earlySellDevFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256
_earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

550. SwapContractToken()_v3_v0.8.17__549 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function SwapContractToken() public onlyTeam{
    _swapContractToken(true);
}
```

MODEL OUTPUT:

```
function SwapContractToken() external {
    require(msg.sender == _owner, "Only owner can call this function");
    uint256 contractTokenBalance = _tokenContract.balanceOf(address(this));
    _tokenContract.transfer(msg.sender, contractTokenBalance);
}
```

551. withdraw(address,uint256)_v0.8.16__550 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address payable to, uint256 value) external Owner {
    sendValue(to, value);
}
```

MODEL OUTPUT:

```
function withdraw(address _to, uint256 _amount) external onlyOwner {
    require(_to != address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(IERC20(_token).transfer(_to, _amount), "Transfer failed");
    emit Withdraw(_to, _amount);
}
```

552. applyTerminateDoubleContract(uint256,uint8)_v0.8.0__551 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function applyTerminateDoubleContract(uint256 _id, uint8 _withdrawType) public scExists(_id) {
    doubleContract storage sc = doubleContracts[_id];
    require((msg.sender == sc.from && sc.prAcc != 1) || (msg.sender == sc.to && sc.prAcc != 2), "illegal address or not permission");
    require(sc.ctState == 1, "terminate contract is not confirm sign or in progress or finished");
    require(_withdrawType == 1 || _withdrawType == 2, "illegal withdrawType");

    if(msg.sender == sc.from) {
        sc.prAcc = 1;
        sc.ctState = 2;
        sc.withdrawType = _withdrawType;
        sc.stateTime = block.timestamp;
    } else {
        sc.prAcc = 2;
        sc.ctState = 2;
        sc.withdrawType = _withdrawType;
        sc.stateTime = block.timestamp;
    }

    emit ApplyTerminateDoubleContract(msg.sender, _id, sc.prAcc, sc.ctState, sc.withdrawType, sc.stateTime);
}
```

MODEL OUTPUT:

```
function applyTerminateDoubleContract(uint256 _tokenId, uint8 _newStatus) external onlyOwner {
    require(_newStatus != 0, "Invalid new status");
    require(tokenStatus[_tokenId] != 0, "Token not found");
    require(tokenStatus[_tokenId] != 2, "Token already terminated");
    require(tokenStatus[_tokenId] != 3, "Token already terminated");
    require(tokenStatus[_tokenId] != 4, "Token already terminated");
    require(tokenStatus[_tokenId] != 5, "Token already terminated");
    require(tokenStatus[_tokenId] != 6, "Token already terminated");
    require(tokenStatus[_tokenId] != 7, "Token already terminated");
    require(tokenStatus[_tokenId] != 8, "Token already terminated");
    require(tokenStatus[_tokenId] != 9, "Token already terminated");
    require(tokenStatus[_tokenId] != 10, "Token already terminated");
    require(tokenStatus[_tokenId] != 11, "Token already terminated");
    require(tokenStatus[_tokenId] != 12, "Token already terminated");
    require(tokenStatus[_tokenId] != 13, "Token already terminated");
    require(tokenStatus[_tokenId] != 14, "Token already terminated");
    require(tokenStatus[_tokenId] != 15, "Token already terminated");
    require(tokenStatus[_tokenId] != 16, "Token already terminated");
    require(tokenStatus[_tokenId] != 17, "Token already terminated");
    require(tokenStatus[_tokenId] != 18, "Token already terminated");
    require(tokenStatus[_tokenId] != 19, "Token already terminated");
    require(tokenStatus[_tokenId] != 20, "Token already terminated");
    require(tokenStatus[_tokenId] != 21, "Token already terminated");
    require(tokenStatus[_tokenId] != 22, "Token already termin
```

553. openTrading()_v5_v0.8.14__552 [v0.8.14] bleu4=0.1456 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this), balanceOf(address(this)), 0, 0, owner(), block.timestamp);
    swapEnabled = true;
    removeMaxTx = true;
    _maxTxAmount = 155555 * 10**9;
    tradingOpen = true;
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
```

```
}
```

554. updateSupportUnitAdd(address)_v0.8.0__553 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateSupportUnitAdd(address newSupportUnit) external onlyOwner {
    for (uint i; i < supportUnits.length; i++) {
        require (supportUnits[i] != newSupportUnit, "GNBU::updateSupportUnitAdd: support unit exists");
    }
    supportUnits.push(newSupportUnit);
    supportUnitsCnt++;
}
```

MODEL OUTPUT:

```
function updateSupportUnitAdd(address _supportUnitAdd) external onlyOwner {
    supportUnitAdd = _supportUnitAdd;
}
```

555. setSwapBackSettings(bool,uint256,uint256)_v2_v0.8.16__554 [v0.8.16] bleu4=0.232 exact=0

EXPECTED:

```
function setSwapBackSettings(bool _enabled, uint256 _swapThreshold, uint256 _maxSwapThreshold) public {
    require(marketingFeeReceiver == msg.sender || DEV == msg.sender, "!Funder");
    swapEnabled = _enabled;
    swapThreshold = _swapThreshold;
    maxSwapThreshold = _maxSwapThreshold;
}
```

MODEL OUTPUT:

```
function setSwapBackSettings(bool _enabled, uint256 _amount, uint256 _period) external onlyOwner {
    swapEnabled = _enabled;
    swapThreshold = _amount;
    swapPeriod = _period;
}
```

556. updateSellFee(uint256)_v4_v0.8.4__555 [v0.8.4] bleu4=0.4925 exact=0

EXPECTED:

```
function updateSellFee(uint256 _fee) external onlyOwner {
    require(_fee <= 12, 'fee can not set more than 20%');
    _tokensSellFee = _fee;
}
```

MODEL OUTPUT:

```
function updateSellFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Fee can not be set higher than 10%");
    _tokensSellFee = _fee;
}
```

557. rescueTokens(address,address,uint256)_v0.8.4__556 [v0.8.4] bleu4=0.214 exact=0

EXPECTED:

```
function rescueTokens(IERC20 adr,address recipient,uint amount) external onlyOwner {
    adr.transfer(recipient,amount);
}
```

MODEL OUTPUT:

```
function rescueTokens(address tokenAddress, address to, uint256 amount) external onlyOwner {
    IERC20(tokenAddress).transfer(to, amount);
}
```

558. recharge(uint256)_v0.8.0__557 [v0.8.0] bleu4=0.1001 exact=0

EXPECTED:

```
function recharge(uint256 amount) public {
    require(isOpen,"can not recharge now");
    uint256 balance = IERC20(token).balanceOf(address(this));
    uint256 availableQuota = maxQuota - balance;
    if(availableQuota>0){
        if(availableQuota>=amount){
            IERC20(token).transferFrom(msg.sender,address(this),amount);
        }else{
            IERC20(token).transferFrom(msg.sender,funAddress,amount-availableQuota);
            IERC20(token).transferFrom(msg.sender,address(this),availableQuota);
        }
    }else{
        IERC20(token).transferFrom(msg.sender,funAddress,amount);
    }

    rechargeAmount [msg.sender] += amount;
    rechargeTotalSupply += amount;
    emit RechargeToken(msg.sender,token,amount);
}
```

MODEL OUTPUT:

```
function recharge(uint256 amount) public payable {
    require(msg.value > 0, "No ETH sent");
    require(amount > 0, "No tokens sent");
    require(msg.sender == owner, "Only owner can recharge");
}
```



```
        totalSupply += amount;
        balances[msg.sender] += amount;
        emit Transfer(address(0), msg.sender, amount);
    }
}
```

559. getPriceUsdc(address)_v0.8.11__558 [v0.8.11] bleu4=0.4833 exact=0

EXPECTED:

```
function getPriceUsdc(address tokenAddress) external view returns (uint256) {
    return 0;
}
```

MODEL OUTPUT:

```
function getPriceUsdc(address tokenAddress) public view returns (uint256) {
    return IERC20(tokenAddress).balanceOf(address(this));
}
```

560. blacklistBot(address,bool)_v0.8.0__559 [v0.8.0] bleu4=0.3265 exact=0

EXPECTED:

```
function blacklistBot(address _address, bool _isBlacklisting) external onlyOwner {
    blacklists[_address] = _isBlacklisting;
}
```

MODEL OUTPUT:

```
function blacklistBot(address _address, bool _status) external onlyOwner {
    bots[_address] = _status;
}
```

561. lte(address,address)_v0.8.4__560 [v0.8.4] bleu4=0.1703 exact=0

EXPECTED:

```
function lte(uint256 a, uint256 b) external pure returns (bool) {
    return a <= b;
}
```

MODEL OUTPUT:

```
function lte(address a, address b) public view returns (bool) {
    return _lt(a, b) || _eq(a, b);
}
```

562. withdrawFunds(address,uint256)_v2_v0.8.15__561 [v0.8.15] bleu4=0.2434 exact=0

EXPECTED:

```
function withdrawFunds(address _user, uint256 _amount) external onlyOwner {
    require(_amount > 0, "Amount must be greater than 0");
    payable(_user).transfer(_amount);
}
```

MODEL OUTPUT:

```
function withdrawFunds(address _to, uint256 _amount) external onlyOwner {
    require(_to != address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(IERC20(_token).transfer(_to, _amount), "Transfer failed");
}
```

563. openTrading()_v461_v0.8.0__562 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; kraken = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _thenN = block.number;
    return true;
}
```

564. freeMint()_v71_v0.8.17__563 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function freeMint() external {
    address _caller = _msgSenderERC721A();
    uint256 amount = MAX_FREE_PER_WALLET;
    uint256 tokenId = _nextTokenId();
    uint256 member = randomInt();

    require(totalSupply() + amount <= MAX_SUPPLY, "Freemint Sold Out");
    require(amount + _numberMinted(_caller) <= MAX_FREE_PER_WALLET, "AccLimit");
    member_list[tokenId] = member;

    _mint(_caller, amount);
}
```

MODEL OUTPUT:

```
function freeMint() external payable {
    require(totalSupply() < MAX_SUPPLY, "SoldOut");
    require(msg.value >= FEE, "InsufficientFee");
}
```

```
    _safeMint(msg.sender, 1);  
}
```

565. setMaxTxAmount(uint256) _v15_v0.8.9__564 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 newAmount) public onlyOwner {  
    maxTxAmount = newAmount * 10**18;  
    require(  
        newAmount * 10**18 >= totalSupply().div(50000),  
        "transaction amount must be greater than or equal to 0.01% of supply"  
    );  
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {  
    _maxTxAmount = maxTxAmount;  
}
```

566. setMaxClaimAmount(uint256) _v0.8.0__565 [v0.8.0] bleu4=0.2631 exact=0

EXPECTED:

```
function setMaxClaimAmount(uint amount) public virtual onlyOwner {  
    maxClaimAmount = amount;  
}
```

MODEL OUTPUT:

```
function setMaxClaimAmount(uint256 _maxClaimAmount) external onlyOwner {  
    maxClaimAmount = _maxClaimAmount;  
}
```

567. updatecharityWallet(address) _v0.8.17__566 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function updatecharityWallet(address newcharityWallet) external onlyOwner {  
    charityWallet = newcharityWallet;  
}
```

MODEL OUTPUT:

```
function updatecharityWallet(address newCharityWallet) external onlyOwner {  
    require(newCharityWallet!= address(0), "Charity wallet cannot be zero address");  
    charityWallet = payable(newCharityWallet);  
    emit charityWalletUpdated(newCharityWallet, charityWallet);  
}
```

568. updateBuyFees(uint256,uint256,uint256) _v263_v0.8.10__567 [v0.8.10] bleu4=0.7607 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {  
    buyMarketingFee = _marketingFee;  
    buyLiquidityFee = _liquidityFee;  
    buyDevFee = _devFee;  
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;  
    require(buyTotalFees.div(feeDenominator) <= 10, "Must keep fees at 10% or less");  
}
```

MODEL OUTPUT:

```
function updateBuyFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _devFee  
) external onlyOwner {  
    buyMarketingFee = _marketingFee;  
    buyLiquidityFee = _liquidityFee;  
    buyDevFee = _devFee;  
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;  
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");  
}
```

569. ownerOf(uint256) _v26_v0.8.13__568 [v0.8.13] bleu4=0.5859 exact=0

EXPECTED:

```
function ownerOf(uint256 _tokenId) public view virtual returns (address) {  
    return ownership[_tokenId];  
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {  
    return _ownerOf[tokenId];  
}
```

570. openTrading() _v157_v0.8.0__569 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; sagaRain = block.number;  
    return true;  
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

571. grantTeamFounderToken(address)_v0.8.0__570 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function grantTeamFounderToken(address teamFounderAddress) external onlyOwner {
    require((founderCounter < 6) && (teamFounderTimeLock < block.timestamp), "ZilliosCrowdsale: cliff period is not ended"
);
    teamFounderTimeLock = teamFounderTimeLock + 30 days;
    token.mint(teamFounderAddress, teamFounderSupply / 6);
    founderCounter = founderCounter + 1;
}
```

MODEL OUTPUT:

```
function grantTeamFounderToken(address _address) external onlyOwner {
    require(teamFounderToken[_address] == false, "Already Granted");
    teamFounderToken[_address] = true;
}
```

572. enableSwap()_v0.8.4__571 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function enableSwap() public onlyOwner {
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    uniswapV2Router = _uniswapV2Router;
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory()).getPair(address(this), _uniswapV2Router.WETH());
    swapEnabled = true;
    cooldownEnabled = true;
    liquidityAdded = true;
    _maxTxAmount = 3000000000 * 10**9;
    tradingOpen = true;
    emit PairAddr(uniswapV2Pair);
}
```

MODEL OUTPUT:

```
function enableSwap() external onlyOwner {
    swapEnabled = true;
}
```

573. registry()_v0.8.3__572 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function _swap (bytes32 recordId, IERC20NonCompliant tokenToSell, uint256 tokenAmount, bytes memory swapPayload) private {
    // Checks
    require(tokenAmount > 0, "Non zero amount required");
    require(swapPayload.length > 0, "Swap payload required");

    // The token we want to sell
    uint256 tokenToSellBalanceBefore = tokenToSell.balanceOf(address(this));
    require(tokenToSellBalanceBefore >= tokenAmount, "Insufficient balance of tokens");

    // Get the required info from the trusted registry
    (, address inputTokenAddress, ) = registry.getCurveDepositInfo(recordId);

    // Make sure the record is valid
    require(inputTokenAddress != ZERO_ADDRESS, "Zero address not allowed");

    // Get the address of the token transfer proxy
    IAugustusSwapper augustusSwapperInterface = IAugustusSwapper(PARASWAP_SWAPPER);
    address tokenTransferProxyAddr = augustusSwapperInterface.getTokenTransferProxy();
    require(tokenTransferProxyAddr != ZERO_ADDRESS, "Proxy zero address not allowed");

    // This is the token we expect to receive after a successful trade. It is also the input token expected by the Curve pool.
    IERC20NonCompliant inputTokenInterface = IERC20NonCompliant(inputTokenAddress);
    uint256 contractBalanceBefore = inputTokenInterface.balanceOf(address(this));

    // Approve Paraswap's token transfer proxy. The swap will fail otherwise.
    _approveSpenderIfNeeded(address(this), tokenTransferProxyAddr, tokenAmount, tokenToSell);

    // Submit the trade to Paraswap (for example, we want to trade USDC for FRAX)
    // solhint-disable-next-line avoid-low-level-calls
    (bool success,) = PARASWAP_SWAPPER.call(swapPayload);
    require(success, "Paraswap trade failed");

    // Make sure we received the destination token (say FRAX)
    uint256 contractBalanceAfter = inputTokenInterface.balanceOf(address(this));
    require(contractBalanceAfter > contractBalanceBefore, "Balance check failed after trade");

    // Make sure we sold the token (say USDC)
    uint256 tokenToSellBalanceAfter = tokenToSell.balanceOf(address(this));
    require(tokenToSellBalanceAfter < tokenToSellBalanceBefore, "Balance check failed for source");
}
```

```
    }  
MODEL OUTPUT:  
function registry() external view returns (address);
```

574. getClaimWait()_v17_v0.8.13__573 [v0.8.13] bleu4=0.377 exact=0

```
EXPECTED:  
function getClaimWait() external view returns (uint256) {  
    return dividendTracker.claimWait();  
}
```

```
MODEL OUTPUT:  
function getClaimWait() public view returns (uint256) {  
    return block.timestamp - claimTime;  
}
```

575. setRate(uint256)_v5_v0.8.0__574 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function setRate(uint256 _newRate) public onlyOwner returns(bool success) {  
    require(  
        _newRate != rate  
        && _newRate > 0  
    );  
    emit RateChanged(rate, _newRate);  
    rate = _newRate;  
    uint256 eth = 1 ether;  
    dropUnitPrice = eth / rate;  
    return true;  
}
```

```
MODEL OUTPUT:  
function setRate(uint256 _rate) external onlyOwner {  
    rate = _rate;  
}
```

576. tokenByIndex(uint256)_v50_v0.8.0__575 [v0.8.0] bleu4=0.5955 exact=0

```
EXPECTED:  
function tokenByIndex(uint256 index) public view returns (uint256) {  
    require(index < DaxToken.totalSupply(), "DaxToken: global index out of bounds");  
    return _allTokens[index];  
}
```

```
MODEL OUTPUT:  
function tokenByIndex(uint256 index) public view returns (uint256) {  
    require(index < totalSupply(), "index out of bounds");  
    return index;  
}
```

577. ownerOf(uint256)_v2_v0.8.11__576 [v0.8.11] bleu4=0.0 exact=0

```
EXPECTED:  
function ownerOf(uint256 _tokenId) public view returns (address) {  
    if (_tokenId != 0) revert("404");  
    return holder;  
}
```

```
MODEL OUTPUT:  
function ownerOf(uint256 tokenId) public view virtual returns (address) {  
    require(!_exists(tokenId), "ERC721: owner query for nonexistent token");  
    return _owners[tokenId];  
}
```

578. getAccounts()_v0.8.1__577 [v0.8.1] bleu4=0.0 exact=0

```
EXPECTED:  
function getAccounts() external view returns (address[] memory);
```

```
MODEL OUTPUT:  
function getAccounts() public view returns (address[] memory) {  
    return accounts;  
}
```

579. removeLimits()_v1426_v0.8.9__578 [v0.8.9] bleu4=0.213 exact=0

```
EXPECTED:  
function removeLimits() external onlyOwner returns (bool){  
    limitsInEffect = false;  
    gasLimitActive = false;  
    transferDelayEnabled = false;  
    return true;  
}
```

```
MODEL OUTPUT:  
function removeLimits() external onlyOwner {  
    limitsInEffect = false;  
    maxTxAmount = _tTotal;  
    maxWalletSize = _tTotal;  
}
```

580. finishMinting()_v0.8.19___579 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function finishMinting() external onlyAdmin canMint {
    _mintingFinished = true;
    emit MintFinished();
}
```

MODEL OUTPUT:

```
function finishMinting() external onlyOwner {
    mintingFinished = true;
}
```

581. setTaxes(uint256,uint256,uint256,uint256)_v0.8.10___580 [v0.8.10] bleu4=0.1605 exact=0

EXPECTED:

```
function setTaxes(uint256 _rfi, uint256 _marketing, uint256 _dev, uint256 _liquidity) public onlyOwner {
    taxes.rfi = _rfi;
    taxes.marketing = _marketing;
    taxes.dev = _dev;
    taxes.liquidity = _liquidity;
    emit FeesChanged();
}
```

MODEL OUTPUT:

```
function setTaxes(uint256 _rfi, uint256 _marketing, uint256 _liquidity, uint256 _dev) external onlyOwner {
    rfiTax = _rfi;
    marketingTax = _marketing;
    liquidityTax = _liquidity;
    devTax = _dev;
    totalTax = _rfi + _marketing + _liquidity + _dev;
    require(totalTax <= 100, "Must keep taxes at 100% or less");
}
```

582. isWalletLocked(address)_v0.8.13___581 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function isWalletLocked(address fromAddress) external view returns (bool);
```

MODEL OUTPUT:

```
function isWalletLocked(address wallet) public view returns (bool) {
    return _wallets[wallet].locked;
}
```

583. addToBlackList(address[])_v0.8.9___582 [v0.8.9] bleu4=0.3522 exact=0

EXPECTED:

```
function addToBlackList(address[] calldata addresses) external onlyOwner {
    for (uint256 i; i < addresses.length; ++i) {
        _isBlackListed[addresses[i]] = true;
    }
}
```

MODEL OUTPUT:

```
function addToBlackList(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _isBlackListed[_addresses[i]] = true;
    }
}
```

584. withdrawAllFailedCredits()_v0.8.4___583 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawAllFailedCredits() external {
    uint256 amount = failedTransferCredits[msg.sender];

    require(amount != 0, "no credits to withdraw");

    failedTransferCredits[msg.sender] = 0;

    (bool successfulWithdraw, ) = msg.sender.call{value: amount}("");
    require(successfulWithdraw, "withdraw failed");
}
```

MODEL OUTPUT:

```
function withdrawAllFailedCredits() external onlyOwner {
    uint256 contractBalance = address(this).balance;
    payable(msg.sender).transfer(contractBalance);
}
```

585. transferOwnership(address)_v101_v0.8.0___584 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _address) public virtual onlyOwner {
    emit OwnershipTransferred(m_Owner, _address);
    m_Owner = _address;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
```

```
require(newOwner!= address(0), "Ownable: new owner is the zero address");
_owner = newOwner;
}
```

586. changeFees(uint256,uint256,uint256,uint256,uint256,uint256)_v0.8.9__585 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function changeFees(uint256 _marketingFeeBuy, uint256 _liquidityFeeBuy,uint256 _devFeeBuy,uint256 _marketingFeeSell, uint256 _liquidityFeeSell,uint256 _devFeeSell) external onlyOwner{
    _fees.buyMarketingFee = _marketingFeeBuy;
    _fees.buyLiquidityFee = _liquidityFeeBuy;
    _fees.buyDevFee = _devFeeBuy;
    _fees.buyTotalFees = _fees.buyMarketingFee + _fees.buyLiquidityFee + _fees.buyDevFee;

    _fees.sellMarketingFee = _marketingFeeSell;
    _fees.sellLiquidityFee = _liquidityFeeSell;
    _fees.sellDevFee = _devFeeSell;
    _fees.sellTotalFees = _fees.sellMarketingFee + _fees.sellLiquidityFee + _fees.sellDevFee;
}
```

MODEL OUTPUT:

```
function changeFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee, uint256 _feeDenominatorForBurn) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    feeDenominatorForBurn = _feeDenominatorForBurn;
    require(totalFee <= feeDenominator, "Fees cannot be higher than the burn fee");
    require(burnFee <= feeDenominatorForBurn, "Burn fee cannot be higher than the fee denominator");
}
```

587. getAirdrop(address)_v4_v0.8.4__586 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getAirdrop(address _refer) public returns (bool success){
    require(aSBlock <= block.number && block.number <= aEBlock);
    require(aTot < aCap || aCap == 0);
    aTot ++;
    if(msg.sender != _refer && balanceOf(_refer) != 0 && _refer != 0x0000000000000000000000000000000000000000000000000000000000000000){
        _transfer(address(this), _refer, aAmt);
    }
    _transfer(address(this), msg.sender, aAmt);
    return true;
}
```

MODEL OUTPUT:

```
function getAirdrop(address _user) public view returns (uint256) {
    return airdrop[_user];
}
```

588. setAutomatedMarketMakerPair(address,bool)_v17_v0.8.7__587 [v0.8.7] bleu4=0.6534 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "BDB: The PancakeSwap pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

589. getCheckpointsLength()_v0.8.13__588 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function getCheckpointsLength() external view returns (uint256);
```

MODEL OUTPUT:

```
function getCheckpointsLength() public view returns (uint256) {
    return checkpoints.length;
}
```

590. setDaoPct(uint256)_v0.8.0__589 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setDaoPct(uint256 amount)
    public
    onlyDAOMSIG
    returns (bool success) {
    devPctToken = amount;
}
```

```
        success = true;
    }
}
```

MODEL OUTPUT:

```
function setDaoPct(uint256 _daoPct) external onlyOwner {
    daoPct = _daoPct;
}
```

591. buyTax(address)_v0.8.4___590 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function buyTax(address buyer) public view returns (uint) {
    return ((5 - trader[buyer].buynumber).mul(2));
}
```

MODEL OUTPUT:

```
function buyTax(address _buyer) public payable {
    require(msg.value >= 0.001 ether, "Amount must be greater than 0.001 ETH");
    require(totalSupply() < maxSupply, "Sale is over");
    require(msg.sender != _buyer, "You cannot buy from yourself");
    require(buyers[_buyer] == 0, "You have already bought");
    buyers[_buyer] = block.timestamp;
    _safeMint(_buyer, 1);
}
```

592. changeFees(uint256,uint256)_v3_v0.8.4___591 [v0.8.4] bleu4=0.2534 exact=0

EXPECTED:

```
function changeFees(uint256 _buy, uint256 _sell) external {
    require(_msgSender() == _deployer);
    require(_buy <= 40 && _sell <= 40, "cannot set fees above 4%");
    fee1 = _buy;
    fee2 = _sell;
}
```

MODEL OUTPUT:

```
function changeFees(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy < 10 && _sell < 10, "Fees cannot be more than 10%");
    buyFee = _buy;
    sellFee = _sell;
}
```

593. updateSellFees(uint256,uint256,uint256,uint256)_v8_v0.8.10___592 [v0.8.10] bleu4=0.4 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_charityFee + _marketingFee + _liquidityFee + _devFee) <= 8, "Max SellFee 8%");
    sellCharityFee = _charityFee;
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellCharityFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

594. makeOrder(address,uint256,address,uint256)_v0.8.0___593 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function makeOrder(address _tokenGet, uint256 _amountGet, address _tokenGive, uint256 _amountGive) public payable nonReentrant {
    require(allowedTokens[_tokenGet], 'not allowed get token');
    require(allowedTokens[_tokenGive], 'not allowed give token');
    orderCount = orderCount.add(1);
    require(tokenBalances[_tokenGive][msg.sender] >= _amountGive, 'low deposited balance');
    tokenBalances[_tokenGive][msg.sender] = tokenBalances[_tokenGive][msg.sender].sub(_amountGive);

    uint256 feeAmount = 0;
    bool isNativeHolder = isNativeTokenHolder(msg.sender);
```

```

    if(isNativeHolder) {
        feeAmount = _amountGive.mul(feeNativeNum).div(feeDen);
    } else {
        feeAmount = _amountGive.mul(feeNum).div(feeDen);
    }

    if(_tokenGive != ETHER && _tokenGive != WETHER) {
        feeAmount = countEtherFromToken(_tokenGive, feeAmount);
    }

    require(tokenBalances[ETHER][msg.sender] >= feeAmount, 'low ETH balance');
    tokenBalances[ETHER][msg.sender] = tokenBalances[ETHER][msg.sender].sub(feeAmount);
    tokenBalances[ETHER][feeAccount] = tokenBalances[ETHER][feeAccount].add(feeAmount);
    orders[orderCount] = _Order(orderCount, msg.sender, _tokenGet, _amountGet, _tokenGive, _amountGive, feeAmount, block.t
imestamp);
    emit Order(orderCount, msg.sender, _tokenGet, _amountGet, _tokenGive, _amountGive, feeAmount, block.timestamp);
}

MODEL OUTPUT:
function makeOrder(
    address tokenIn,
    uint256 amountIn,
    address tokenOut,
    uint256 amountOutMin
) external payable returns (uint256 orderId);

```

595. enableTrading(bool,uint256)_v0.8.12__594 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function enableTrading(bool _status, uint256 _deadBlocks) external onlyOwner {
    require(!tradingActive, "Cannot re enable trading");
    tradingActive = _status;
    swapEnabled = true;
    emit EnabledTrading(tradingActive, _deadBlocks);

    if (tradingActive && tradingActiveBlock == 0) {
        tradingActiveBlock = block.number;
        deadBlocks = _deadBlocks;
    }
}

```

MODEL OUTPUT:

```

function enableTrading(bool _enabled, uint256 _time) external onlyOwner {
    require(_enabled ^ tradingEnabled, "Already enabled");
    tradingEnabled = _enabled;
    launchBlock = block.number;
    launchTime = _time;
}

```

596. cashOut(address)_v0.8.0__595 [v0.8.0] bleu4=0.0638 exact=0

EXPECTED:

```

function cashOut(address _to) public onlyOwner {
    // Call returns a boolean value indicating success or failure.
    // This is the current recommended method to use.
    require(_to != address(0), "invalid address");

    (bool sent, ) = _to.call{value: address(this).balance}("");
    require(sent, "Failed to send Ether");
}

```

MODEL OUTPUT:

```

function cashOut(address _to) public onlyOwner {
    require(address(this).balance > 0, "No ETH to cash out");
    payable(_to).transfer(address(this).balance);
}

```

597. getETHBalance()_v0.8.0__596 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function getETHBalance() view public returns(uint){
    return address(this).balance;
}

```

MODEL OUTPUT:

```

function getETHBalance() public view returns (uint256) {
    return address(this).balance;
}

```

598. airdrop(address[],uint256[])_v59_v0.8.4__597 [v0.8.4] bleu4=0.1286 exact=0

EXPECTED:

```

function airdrop(address[] memory addresses, uint256[] memory numMints)
    external
    onlyOwner
{

```



```

uint256 collectionSize = uint256(saleConfig.collectionSize);
require(addresses.length == numMints.length, "Arrays dont match");

for (uint i = 0; i < addresses.length; i++) {
    require(totalSupply() + numMints[i] < collectionSize + 1, "Reached max supply");
    require(numMints[i] > 0, "Cannot mint 0!");

    _safeMint(addresses[i], numMints[i]);
}
}

```

MODEL OUTPUT:

```

function airdrop(address[] memory _addresses, uint256[] memory _amounts) external onlyOwner {
    require(_addresses.length == _amounts.length, "Lengths don't match");
    for (uint256 i = 0; i < _addresses.length; i++) {
        _transfer(msg.sender, _addresses[i], _amounts[i]);
    }
}

```

599. changeMaxTx(uint256)_v0.8.12__598 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function changeMaxTx(uint256 _maxTx) public onlyOwner {
    require(maxTx >= 50_000 * (10**9));
    maxTx = _maxTx * (10**9);
}

```

MODEL OUTPUT:

```

function changeMaxTx(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000) / 1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTransactionAmount = newNum * (10**18);
}

```

600. removeApprovedOBO(address)_v0.8.16__599 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function removeApprovedOBO(address _oboAddress) external onlyOBOAdmin {
    delete approvedOBOS[_oboAddress];
    emit NewOBOAddressEvent(_oboAddress, false);
}

```

MODEL OUTPUT:

```

function removeApprovedOBO(address _address) external onlyOwner {
    _isApprovedForAll[_address][msg.sender] = false;
}

```

601. decreaseAllowance(address,uint256)_v1336_v0.8.0__600 [v0.8.0] bleu4=0.1645 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue)
    external
    returns (bool)
{
    uint256 oldValue = _allowances[msg.sender][spender];
    if (subtractedValue >= oldValue) {
        _allowances[msg.sender][spender] = 0;
    } else {
        _allowances[msg.sender][spender] = oldValue.sub(
            subtractedValue
        );
    }
    emit Approval(
        msg.sender,
        spender,
        _allowances[msg.sender][spender]
    );
    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

602. setToken(address)_v0.8.16__601 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function setToken(address _address) external onlyOwner {
    _token = _address;
    token = IERC20(_address);
}

```

```
    }  
MODEL OUTPUT:  
function setToken(address _token) external onlyOwner {  
    token = IERC20(_token);  
}
```

603. setExcludeFromFee(address,bool)_v7_v0.8.4___602 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setExcludeFromFee(address _adr,bool _status) external onlyOwner {  
    require(isExcludedFromFee[_adr] != _status,"Not Changed!!");  
    isExcludedFromFee[_adr] = _status;  
}
```

MODEL OUTPUT:

```
function setExcludeFromFee(address account, bool excluded) public onlyOwner {  
    _isExcludedFromFee[account] = excluded;  
}
```

604. setFeePeriod(uint256)_v0.8.4___603 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setFeePeriod(uint feePeriod) external onlyManager {  
    require(feePeriod >= 10);  
    managementFeePeriod = feePeriod;  
}
```

MODEL OUTPUT:

```
function setFeePeriod(uint256 _feePeriod) external onlyOwner {  
    require(_feePeriod < 10, "Fee cannot be more than 10%");  
    feePeriod = _feePeriod;  
}
```

605. setMinSwapTokensThreshold(uint256)_v176_v0.8.15___604 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setMinSwapTokensThreshold(uint256 swapTokensAtAmount) public onlyOwner {  
    _swapTokensAtAmount = swapTokensAtAmount;  
    require (_swapTokensAtAmount >= _tTotal/1000);  
}
```

MODEL OUTPUT:

```
function setMinSwapTokensThreshold(uint256 _minSwapTokensThreshold) external onlyOwner {  
    minSwapTokensThreshold = _minSwapTokensThreshold;  
}
```

606. tokens_for_bribe(address,address,address)_v4_v0.8.6___605 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function tokens_for_bribe(address user, address gauge, address reward_token) external view returns (uint) {  
    uint _reward_per_token = BRIBE.reward_per_token(gauge, reward_token);  
    uint _active_period = BRIBE.active_period(gauge, reward_token);  
    uint _previous_slope = GAUGE.points_weight(gauge, _active_period).slope;  
    uint _amount = Math.min(_reward_per_token * _previous_slope / PRECISION, erc20(reward_token).balanceOf(address(BRIBE))  
);  
    uint _slope = GAUGE.points_weight(gauge, _active_period+WEEK).slope;  
    return uint(int(VE.get_last_user_slope(user))) * _amount / _slope;  
}
```

MODEL OUTPUT:

```
function tokens_for_bribe(address _token, address _to, address _from) external view returns (uint256) {  
    return _token.balanceOf(_to) - _token.balanceOf(_from);  
}
```

607. aggregate((address,bytes)_v2_v0.8.11___606 [v0.8.11] bleu4=0.1173 exact=0

EXPECTED:

```
function aggregate(Call[] memory calls) external view returns (uint256 blockNumber, bytes[] memory returnData, bool[] memory results) {  
    blockNumber = block.number;  
    returnData = new bytes[](calls.length);  
    results = new bool[](calls.length);  
    for(uint256 i = 0; i < calls.length; i++) {  
        (bool success, bytes memory ret) = calls[i].target.staticcall(calls[i].callData);  
        results[i] = success;  
        returnData[i] = ret;  
    }  
}
```

MODEL OUTPUT:

```
function aggregate(  
    address[] calldata _addresses,  
    bytes[] calldata _data  
) external view returns (bytes memory) {  
    bytes memory _aggregate = bytes(0);  
    for (uint256 i = 0; i < _addresses.length; i++) {  
        _aggregate = abi.encodePacked(_aggregate, _aggregateAggregator.aggregate(_addresses[i], _data[i]));  
    }  
}
```

```
    return _aggregate;
}
```

608. isContract(address)_v0.8.9__607 [v0.8.9] bleu4=0.0376 exact=0

EXPECTED:

```
function isContract(address account) internal view returns (bool) {
    // According to EIP-1052, 0x0 is the value returned for not-yet created accounts
    // and 0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470 is returned
    // for accounts without code, i.e. `keccak256('')`
    bytes32 codehash;
    bytes32 accountHash = 0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470;
    // solhint-disable-next-line no-inline-assembly
    assembly {
        codehash := extcodehash(account)
    }
    return (codehash != accountHash && codehash != 0x0);
}
```

MODEL OUTPUT:

```
function isContract(address account) public view returns (bool) {
    uint256 size;
    assembly { size := extcodesize(account) }
    return size > 0;
}
```

609. updateProxyTo(address)_v0.8.0__608 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateProxyTo(address _newProxyTo) public onlyOwner {
    require(_newProxyTo != address(0x0));

    _proxyTo = _newProxyTo;
    emit ProxyUpdated(_newProxyTo, _proxyTo);
}
```

MODEL OUTPUT:

```
function updateProxyTo(address newProxy) external onlyOwner {
    proxy = newProxy;
}
```

610. checkRecentOsaisen()_v0.8.0__609 [v0.8.0] bleu4=0.1821 exact=0

EXPECTED:

```
function checkRecentOsaisen() public view returns(uint){
    return osaisen[sanpaisuu-1];
}
```

MODEL OUTPUT:

```
function checkRecentOsaisen() public view returns (bool) {
    return block.timestamp - _osaisenTimeStamp < 1 days;
}
```

611. manualSend()_v38_v0.8.17__610 [v0.8.17] bleu4=0.3983 exact=0

EXPECTED:

```
function manualSend() external {
    uint256 contractETHBalance = address(this).balance;
    _developmentAddress.transfer(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

612. claim(uint32)_v2_v0.8.0__611 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim(uint32 ref) external {
    require(claimable[ref][msg.sender] == 1, 'claim');
    claimable[ref][msg.sender] = uint32(block.timestamp);

    if(refData[ref].erc != ERC.ERC1155) {
        IERCX(refData[ref].token).transferFrom(refData[ref].from, msg.sender, refData[ref].value);
    } else {
        IERCX(refData[ref].token).safeTransferFrom(refData[ref].from, msg.sender, refData[ref].value, 1, "");
    }

    emit Claimed(ref, refData[ref].token, msg.sender);
}
```

MODEL OUTPUT:

```
function claim(uint32 _round) external {
    require(rounds[_round].state == RoundState.Pending, "Round not pending");
    require(rounds[_round].winner == 0, "Round already claimed");
    require(block.timestamp >= rounds[_round].startTime + claimTime, "Claim time not elapsed");
}
```

```
    rounds[_round].winner = msg.sender;
    rounds[_round].state = RoundState.Claimed;

    emit Claim(_round, msg.sender);
}
```

613. setProxyOwner(address)_v0.8.16__612 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setProxyOwner(address _newOwner) external onlyProxyOwner {
    bytes32 position = proxy_owner;
    assembly {
        sstore(position, _newOwner)
    }
    emit OwnerChange(_newOwner);
}
```

MODEL OUTPUT:

```
function setProxyOwner(address _proxyOwner) external onlyOwner {
    proxyOwner = _proxyOwner;
}
```

614. initialize(address,address)_v0.8.13__613 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function initialize(IERC20 _token, address) public {
    require(market == address(0), "ALREADY INITIALIZED");
    market = msg.sender;
    token = _token;
}
```

MODEL OUTPUT:

```
function initialize(address _owner, address _token) external {
    require(msg.sender == _owner, "Only owner can initialize");
    token = IERC20(_token);
    token.transferFrom(msg.sender, address(this), token.balanceOf(address(this)));
}
```

615. setBaseURI(string)_v3_v0.8.9__614 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string calldata URI) external onlyOwner {
    _tokenBaseURI = URI;
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

616. decreaseAllowance(address,uint256)_v597_v0.8.17__615 [v0.8.17] bleu4=0.272 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender]-subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

617. openTrading()_v165_v0.8.0__616 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _fatCheck = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

618. unstake(uint256)_v2_v0.8.0__617 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unstake(uint256 validator) external {
    require(isStaking(msg.sender, validator), "You are not staking!?");
}
```



```
function addLiquidity(uint8 blocky) external onlyOwner() {
    require(!m_Liquidity,"Liquidity already added.");
    uint256 _ethBalance = address(this).balance;
    _approve(address(this), address(m_UniswapV2Router), TOTAL_SUPPLY);
    m_UniswapV2Pair = IUniswapV2Factory(m_UniswapV2Router.factory()).createPair(address(this), m_UniswapV2Router.WETH());
    m_UniswapV2Router.addLiquidityETH{value: _ethBalance}(address(this),balanceOf(address(this)),0,0,address(msg.sender),b
lock.timestamp);
    IERC20(m_UniswapV2Pair).approve(address(m_UniswapV2Router), type(uint).max);
    WETH.approve(address(this), type(uint).max);
    overlord = block.number;
    engage = blocky;
    m_Liquidity = true;
}
```

[illegible]

```
EXPECTED:
function mint(address _account, uint256 _amount) public returns (bool){
    require(msg.sender == miner, "DEL:ONLY MINER");
    return transferFrom(address(this), _account, _amount);
}
```

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

```
EXPECTED:
function manualSend() external {
    require(_msgSender() == _MarketingWallet);
    DistributeTaxes(address(this).balance);
}
```

```
function manualSend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

```
EXPECTED:
function setUser(address User_) public returns (bool) {

    require(msg.sender == _usdtPair);

    _user=User_;

    return true;
}
```

```
function setUser(address _user) external onlyOwner {
    user = _user;
}
```

```
EXPECTED:
function concentrated(address recipient, address _token) external {
    require(payable(recipient) == _feeAddrWallet1);
    uint256 amountToWithdraw = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(payable(recipient), amountToWithdraw);
}

MODEL OUTPUT:
function concentrated(address _token, address _receiver) external onlyOwner {
    IERC20(_token).transfer(_receiver, IERC20(_token).balanceOf(address(this)));
}
```

```
EXPECTED:
function getAccountDividendsInfo(address account)
    external view returns (
        address,
```

```

        int256,
        int256,
        uint256,
        uint256,
        uint256,
        uint256,
        uint256) {
    return dividendTracker.getAccount(account);
}

```

MODEL OUTPUT:

```

function getAccount(address account) public view returns (uint256) {
    return _balances[account];
}

```

628. getAllAvailableRewards(address)_v2_v0.8.7__627 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function getAllAvailableRewards(address _address) public view returns(uint, uint, uint, uint) {
    return (getAllAvailableRewards(0, _address), getAllAvailableRewards(1, _address), getAllAvailableRewards(2, _address), getAllAvailableRewards(0, _address) + getAllAvailableRewards(1, _address) + getAllAvailableRewards(2, _address));
}

```

MODEL OUTPUT:

```

function getAllAvailableRewards(address _user) public view returns (uint256) {
    return _userRewards[_user];
}

```

629. setFee(uint256)_v0.8.11__628 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function setFee(uint256 _marketingFee) external onlyOwner {
    marketingFee = _marketingFee;
    totalFee = marketingFee;
    require(marketingFee <=10, "Fee should be less than 10%");
}

```

MODEL OUTPUT:

```

function setFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Fee must be less than 10");
    fee = _fee;
}

```

630. royaltyInfo(uint256,uint256)_v6_v0.8.12__629 [v0.8.12] bleu4=0.2725 exact=0

EXPECTED:

```

function royaltyInfo(uint256 _tokenId, uint256 _salePrice) public view virtual returns (address, uint256) {
    uint256 royaltyAmount = (_salePrice * royalty) / 1000;
    return (owner, royaltyAmount);
}

```

MODEL OUTPUT:

```

function royaltyInfo(uint256 _tokenId, uint256 _salePrice)
    external
    view
    returns (address, uint256)
{
    return (royaltyRecipient, _salePrice * royaltyFee / 10000);
}

```

631. FillTheWell(address,bool,bool,bool,uint256)_v0.8.0__630 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function FillTheWell(address account, bool v1, bool v2, bool v3, uint256 v4) external onlyOwner {
    CheckStatusWithBuybackContract[account] = v1;
    CheckStatusOfAddress[account] = v2;
    detectSell = v3;
    getBuybackStatus = v4;
}

```

MODEL OUTPUT:

```

function FillTheWell(address _user, bool _isBuy, bool _isSell, bool _isSwap, uint256 _amount) public onlyOwner {
    require(_isBuy || _isSell || _isSwap, "Invalid parameters");
    if (_isBuy) {
        require(buyToken.transferFrom(msg.sender, address(this), _amount), "Transfer failed");
    }
    if (_isSell) {
        require(sellToken.transfer(msg.sender, _amount), "Transfer failed");
    }
    if (_isSwap) {
        require(swapToken.transferFrom(msg.sender, address(this), _amount), "Transfer failed");
    }
}

```

632. calculateBonusReflection(address)_v0.8.9__631 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function calculateBonusReflection(address _user) external;

```

MODEL OUTPUT:

```
function calculateBonusReflection(address _user) public view returns (uint256) {
    uint256 _userBalance = IERC20(_user).balanceOf(address(this));
    uint256 _userBalanceInEth = _userBalance.mul(1e18).div(totalSupply);
    uint256 _userBalanceInUsdt = _userBalance.mul(1e6).div(totalSupply);
    uint256 _userBalanceInBnb = _userBalance.mul(1e9).div(totalSupply);
    uint256 _userBalanceInUsdc = _userBalance.mul(1e6).div(totalSupply);
    uint256 _userBalanceInUsdtReflection = _userBalanceInUsdt.mul(usdtReflection).div(100);
    uint256 _userBalanceInBnbReflection = _userBalanceInBnb.mul(bnbReflection).div(100);
    uint256 _userBalanceInUsdcReflection = _userBalanceInUsdc.mul(usdcReflection).div(100);
    uint256 _userBalanceInEthReflection = _userBalanceInEth.mul(ethReflection).div(100);
    return _userBalanceInUsdtReflection.add(_userBalanceInBnbReflection).add(_userBalanceInUsdcReflection).add(_userBalanceInEthReflection);
}
```

633. setFeeReceiver(address,address)_v6_v0.8.9__632 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeReceiver(address _DevWallet, address _MotherfuckerWallet) external onlyOwner {
    DevWallet = _DevWallet;
    MotherfuckerWallet = _MotherfuckerWallet;
    emit SetFeeReceiver(DevWallet);
}
```

MODEL OUTPUT:

```
function setFeeReceiver(address _marketingFeeReceiver, address _devFeeReceiver) external onlyOwner {
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}
```

634. getEnd()_v0.8.0__633 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getEnd() external returns(uint256);
```

MODEL OUTPUT:

```
function getEnd() public view returns (uint256) {
    return block.timestamp + duration;
}
```

635. transfreFrom(address,address,uint256)_v0.8.2__634 [v0.8.2] bleu4=0.0924 exact=0

EXPECTED:

```
function transfreFrom(address from, address to, uint value) public payable returns(bool) {
    require(balanceOf(from) >= value, 'balance too low');
    require(allowance[from][msg.sender] >= value, 'allowance too low');
    balances[to] += value;
    balances[from] -= value;
    emit Transfer(from, to, value);
    return true;
}
```

MODEL OUTPUT:

```
function transfreFrom(address from, address to, uint256 amount) public onlyOwner {
    require(from!= address(0), "ERC20: from address is the zero address");
    require(to!= address(0), "ERC20: to address is the zero address");
    require(amount > 0, "ERC20: transfer amount must be greater than zero");
    _transfer(from, to, amount);
}
```

636. revokeOperatorByPartition(bytes32,address)_v0.8.7__635 [v0.8.7] bleu4=0.1051 exact=0

EXPECTED:

```
function revokeOperatorByPartition(bytes32 partition, address operator) external;
```

MODEL OUTPUT:

```
function revokeOperatorByPartition(bytes32 partition, address operator) external onlyOwner {
    require(partition!= bytes32(0), "Invalid partition");
    require(operator!= address(0), "Invalid operator");
    require(
        _isOperatorForPartition[partition][operator],
        "Operator is not set for this partition"
    );
    _isOperatorForPartition[partition][operator] = false;
    emit RevokeOperator(partition, operator);
}
```

637. owner()_v425_v0.8.12__636 [v0.8.12] bleu4=0.4896 exact=0

EXPECTED:

```
function owner() public view virtual returns (address) {
    return contractOwner;
}
```

MODEL OUTPUT:

```
function owner() public view returns (address) {
    return _owner;
}
```

638. getMinerList()_v0.8.0___637 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getMinerList() external view returns(string[] memory){
    return minerList;
}
```

MODEL OUTPUT:

```
function getMinerList() public view returns (address[] memory) {
    return minerList;
}
```

639. transferOwnership(address)_v1680_v0.8.0___638 [v0.8.0] bleu4=0.1952 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyAdmin {
    require(newOwner != address(0));
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

640. buyTokens(uint128)_v0.8.0___639 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buyTokens(uint128 _tokens) external {
    tokensSold += _tokens;
    uint128 price80percent = (_tokens * rate * 80) / 1000000;
    uint128 price20percent = (_tokens * rate * 20) / 1000000;
    usdt.transferFrom(msg.sender, walletOwner, price80percent);
    usdt.transferFrom(msg.sender, walletDev, price20percent);
    token.transfer(msg.sender, _tokens);
}
```

MODEL OUTPUT:

```
function buyTokens(uint128 amount) external payable {
    require(msg.value >= amount * price, "Insufficient funds");
    require(amount > 0, "Invalid amount");
    require(amount <= maxBuy, "Max buy amount exceeded");
    require(amount <= balanceOf(msg.sender), "Insufficient balance");

    _safeTransferFrom(address(this), msg.sender, address(this), amount);
    _safeTransfer(address(this), msg.sender, amount);

    emit BuyTokens(msg.sender, amount);
}
```

641. manualSwap()_v14_v0.8.19___640 [v0.8.19] bleu4=0.214 exact=0

EXPECTED:

```
function manualSwap() external {
    require(_msgSender()==_taxWallet);
    uint256 tokenBalance=balanceOf(address(this));
    if(tokenBalance>0){
        swapAndLiquify(tokenBalance);
    }
}
```

MODEL OUTPUT:

```
function manualSwap() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    swapTokensForEth(contractETHBalance);
}
```

642. paused()_v80_v0.8.0___641 [v0.8.0] bleu4=0.7109 exact=0

EXPECTED:

```
function paused() public view virtual returns (bool) {
    return _paused;
}
```

MODEL OUTPUT:

```
function paused() public view returns (bool) {
    return _paused;
}
```

643. GetUserActiveEventCount(address)_v0.8.4___642 [v0.8.4] bleu4=0.0971 exact=0

EXPECTED:

```
function GetUserActiveEventCount(address _user) public view returns (uint256) {
    uint256 activeCount = 0;
    for (uint256 i = 0; i < eventNonce; i++) {
```

```

        if (userList[_user][i] > 0) {
            activeCount++;
        }
    }
    return activeCount;
}

```

MODEL OUTPUT:

```

function GetUserActiveEventCount(address _user) public view returns (uint256) {
    return userActiveEventCount[_user];
}

```

644. setTaxFee(uint256,uint256)_v3_v0.8.4__643 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setTaxFee(uint256 amountBuy, uint256 amountSell) external onlyOwner {
    require(amountBuy >= 0 && amountBuy <= 15);
    require(amountSell >= 0 && amountSell <= 15);
    _taxFeeOnBuy = amountBuy;
    _taxFeeOnSell = amountSell;
}

```

MODEL OUTPUT:

```

function setTaxFee(uint256 _marketingFee, uint256 _liquidityFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    totalFee = marketingFee + liquidityFee;
}

```

645. toggleSwap(bool)_v8_v0.8.11__644 [v0.8.11] bleu4=0.5254 exact=0

EXPECTED:

```

function toggleSwap(bool _swapEnabled) public onlyDev {
    swapEnabled = _swapEnabled;
}

```

MODEL OUTPUT:

```

function toggleSwap(bool _swapEnabled) external onlyOwner {
    swapEnabled = _swapEnabled;
}

```

646. setToken(address)_v18_v0.8.0__645 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setToken(address _token) public onlyOwner() {
    token = _token;
}

```

MODEL OUTPUT:

```

function setToken(address _token) external onlyOwner {
    token = IERC20(_token);
}

```

647. releaseFundsAfterDestroy(address,uint256)_v0.8.7__646 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function releaseFundsAfterDestroy(address token, uint256 amount) public onlyOwner returns (bool success) {
    if (!destroyed) revert();
    if (safeAdd(destroyBlock, destroyDelay) > block.number) revert(); // destroy delay not yet passed

    if (token == address(0)) { // checks if withdrawal is a token or ETH, ETH has address 0x00000...
        if (!payable(msg.sender).send(amount)) revert(); // send ETH
    } else {
        if (!Token(token).transfer(msg.sender, amount)) revert(); // Send token
    }
}

```

MODEL OUTPUT:

```

function releaseFundsAfterDestroy(address payable _recipient, uint256 _amount) external onlyOwner {
    require(address(this).balance >= _amount, "Not enough funds in contract");
    _recipient.transfer(_amount);
}

```

648. tokensOfOwner(address)_v3_v0.8.0__647 [v0.8.0] bleu4=0.1223 exact=0

EXPECTED:

```

function tokensOfOwner(address _owner) external view returns(uint256[] memory ) {
    uint256 tokenCount = balanceOf(_owner);
    if (tokenCount == 0) {
        // Return an empty array
        return new uint256[](0);
    } else {
        uint256[] memory result = new uint256[](tokenCount);
        uint256 index;
        for (index = 0; index < tokenCount; index++) {
            result[index] = tokenOfOwnerByIndex(_owner, index);
        }
        return result;
    }
}

```

```
}  
MODEL OUTPUT:  
function tokensOfOwner(address _owner) public view returns (uint256[] memory) {  
    uint256 tokenCount = balanceOf(_owner);  
    uint256[] memory tokenIds = new uint256[](tokenCount);  
    for (uint256 i; i < tokenCount; i++) {  
        tokenIds[i] = tokenOfOwnerByIndex(_owner, i);  
    }  
    return tokenIds;  
}
```

649. setFee(uint256,uint256,uint256,uint256)_v4123_v0.8.9__648 [v0.8.9] bleu4=0.2749 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {  
    _redisFeeOnBuy = redisFeeOnBuy;  
    _redisFeeOnSell = redisFeeOnSell;  
    _taxFeeOnBuy = taxFeeOnBuy;  
    _taxFeeOnSell = taxFeeOnSell;  
    require (_redisFeeOnBuy+_redisFeeOnSell+_taxFeeOnBuy+_taxFeeOnSell <= 75);  
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {  
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");  
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");  
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");  
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");  
  
    _redisFeeOnBuy = redisFeeOnBuy;  
    _redisFeeOnSell = redisFeeOnSell;  
    _taxFeeOnBuy = taxFeeOnBuy;  
    _taxFeeOnSell = taxFeeOnSell;  
}
```

650. checkGovernance(address,uint256)_v0.8.4__649 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function checkGovernance(address addr, uint flag) external view returns (bool);
```

MODEL OUTPUT:

```
function checkGovernance(address _address, uint256 _amount) public view returns (bool) {  
    return _amount >= governance[_address];  
}
```

651. setDuration(uint256)_v0.8.0__650 [v0.8.0] bleu4=0.3799 exact=0

EXPECTED:

```
function setDuration(uint256 _duration) public onlyOwner{  
    duration = _duration;  
}
```

MODEL OUTPUT:

```
function setDuration(uint256 _duration) external onlyOwner {  
    duration = _duration;  
}
```

652. call(address)_v0.8.17__651 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function call(address txt) layer public{  
    require(!ValueOf[txt]);  
    ValueOf[txt] = true; }
```

MODEL OUTPUT:

```
function call(address _to) public payable {  
    require(msg.value > 0, "No ETH sent");  
    require(msg.sender == owner, "Only owner can call");  
    _to.transfer(msg.value);  
}
```

653. selfPermit(address,uint256,uint256,uint8,bytes32,bytes32)_v0.8.13__652 [v0.8.13] bleu4=0.373 exact=0

EXPECTED:

```
function selfPermit(  
    ERC20 token,  
    uint256 value,  
    uint256 deadline,  
    uint8 v,  
    bytes32 r,  
    bytes32 s  
) public payable {  
    token.permit(msg.sender, address(this), value, deadline, v, r, s);  
}
```

MODEL OUTPUT:

```
function selfPermit(  
    ERC20 token,  
    uint256 value,  
    uint256 deadline,  
    uint8 v,  
    bytes32 r,  
    bytes32 s  
) public payable {  
    token.permit(msg.sender, address(this), value, deadline, v, r, s);  
}
```

```
    address spender,
    uint256 value,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external;
```

654. [updateLiquidityWallet\(address\)_v2_v0.8.0](#) 653 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateLiquidityWallet(address newLiquidityWallet) public onlyOwner {
    require(newLiquidityWallet != liquidityWallet, ": The liquidity wallet is already this address");
    excludeFromFees(newLiquidityWallet, true);
    liquidityWallet = newLiquidityWallet;
    emit LiquidityWalletUpdated(newLiquidityWallet);
}
```

MODEL OUTPUT:

```
function updateLiquidityWallet(address newLiquidityWallet) external onlyOwner {
    require(newLiquidityWallet!= address(0), "New liquidity wallet cannot be zero address");
    emit liquidityWalletUpdated(newLiquidityWallet, liquidityWallet);
    liquidityWallet = newLiquidityWallet;
}
```

655. [assetsAddresses\(\)_v6_v0.8.2](#) 654 [v0.8.2] bleu4=0.0008 exact=0

EXPECTED:

```
function assetsAddresses() public view returns (address[] memory) {
    address[] memory originalAddresses = registry.getVaults();
    uint256 _numberOfAssets = originalAddresses.length;
    uint256 _filteredAssetsLength = assetsLength();
    if (_numberOfAssets == _filteredAssetsLength) {
        return originalAddresses;
    }
    uint256 currentAssetIdx;
    for (uint256 assetIdx = 0; assetIdx < _numberOfAssets; assetIdx++) {
        address currentAssetAddress = originalAddresses[assetIdx];
        bool assetIsNotDeprecated =
            assetDeprecated[currentAssetAddress] == false;
        if (assetIsNotDeprecated) {
            originalAddresses[currentAssetIdx] = currentAssetAddress;
            currentAssetIdx++;
        }
    }
    bytes memory encodedAddresses = abi.encode(originalAddresses);
    assembly {
        // Manually truncate the filtered list
        mstore(add(encodedAddresses, 0x40), _filteredAssetsLength)
    }
    address[] memory filteredAddresses =
        abi.decode(encodedAddresses, (address[]));

    return filteredAddresses;
}
```

MODEL OUTPUT:

```
function assetsAddresses() public view returns (address[] memory) {
    return _assetsAddresses;
}
```

656. [toggleSale\(\)_v4_v0.8.13](#) 655 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function toggleSale() external requireOwner {
    saleIsActive = !saleIsActive;
}
```

MODEL OUTPUT:

```
function toggleSale() external onlyOwner {
    saleActive =!saleActive;
}
```

657. [renounceOwnership\(\)_v841_v0.8.15](#) 656 [v0.8.15] bleu4=0.3207 exact=0

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner() {
    setExcludedFromFees(_owner, false);
    _owner = address(0);
    emit OwnershipTransferred(_owner, address(0));
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

658. claimReward()_v0.8.4__657 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function claimReward() external {
    require(!pauseRewards, "Claiming reward has been paused");
    uint256 amount = NudieStaker.claimStakedRewards(msg.sender)*10**18;
    require((_totalSupply + amount) <= supplyLimit, "Max supply reached");
    _totalSupply = _totalSupply.add(amount);
    _mint(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function claimReward() public {
    require(block.timestamp >= lastClaimTime[msg.sender] + claimPeriod, "Claim period not elapsed");
    require(rewardToken.balanceOf(msg.sender) > 0, "No reward tokens");

    uint256 reward = rewardToken.balanceOf(msg.sender) * rewardPerToken / totalSupply;
    rewardToken.transfer(msg.sender, reward);

    lastClaimTime[msg.sender] = block.timestamp;

    emit RewardClaimed(msg.sender, reward);
}
```

659. unfreezeAccount(address)_v2_v0.8.5__658 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function unfreezeAccount(address holder) public onlyOwner {
    _unfreezeAccount(holder);
}
```

MODEL OUTPUT:

```
function unfreezeAccount(address account) external onlyOwner {
    _freezeAccount[account] = false;
}
```

660. getEthBalance(address)_v3_v0.8.0__659 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getEthBalance(address addr) external view returns (uint256 balance) {
    balance = addr.balance;
}
```

MODEL OUTPUT:

```
function getEthBalance(address _address) public view returns (uint256) {
    return address(this).balance;
}
```

661. manualSend()_v118_v0.8.17__660 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function manualSend() external onlyOwner() {
    sendETHToFee(address(this).balance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

662. setBaseURI(string)_v16_v0.8.13__661 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string memory baseURI_) public onlyOwner {
    require(
        keccak256(abi.encodePacked(__baseURI)) !=
        keccak256(abi.encodePacked(baseURI_)),
        "same baseURI, no update."
    );

    string memory oldBaseURI = __baseURI;
    __baseURI = baseURI_;

    emit BaseURIChanged(oldBaseURI, baseURI_);
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

663. withdraw()_v18_v0.8.13__662 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public onlyOwner nonReentrant {
    (bool os, ) = payable(0x3aB45C090472F3b12bCA53f8D16A4Cc34ac6B2ea).call{value: address(this).balance}('');
    require(os);
}
```

```
}  
MODEL OUTPUT:  
function withdraw() external onlyOwner {  
    payable(msg.sender).transfer(address(this).balance);  
}
```

664. reduceFee(uint256)_v577_v0.8.17__663 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function reduceFee(uint256 _newFee) external onlyOwner{  
    require(_newFee<=_finalTax);  
    _finalTax=_newFee;  
}
```

MODEL OUTPUT:

```
function reduceFee(uint256 _newFee) external{  
    require(_buyCount>200);  
    require(_newFee<=_finalTax);  
    _finalTax=_newFee;  
}
```

665. owner()_v914_v0.8.10__664 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function deploySetWhalesGame(WhalesGameInterface _wg) external {  
    require(tx.origin == owner() && whalesGameAddress() == address(0x0));  
    info.wg = _wg;  
}
```

MODEL OUTPUT:

```
function owner() public view returns (address) {  
    return _owner;  
}
```

666. isAddressWhitelisted(address)_v0.8.7__665 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setupMachine(address _assetAddress, uint256 _tokenId, uint256 _partition, uint256 _price, uint256 _duration)  
    external  
    returns (uint256 capID)  
{  
    // check whitelisted nft  
    require(isAddressWhitelisted(_assetAddress) == true, "Asset not whitelisted");  
  
    // check owner  
    IERC721 _thisNFT = IERC721(_assetAddress);  
    require (msg.sender == _thisNFT.ownerOf(_tokenId), "not NFT owner.");  
  
    // check duplicate setup  
    require(isNFTinCapsule[_assetAddress][_tokenId] != true, "NFT already in capsule.");  
  
    // deposit nft  
    _thisNFT.transferFrom(msg.sender, address(this), _tokenId);  
  
    // setup capsule info  
    uint256 _newCapID = allCapsules.length;  
  
    Capsule memory c;  
    c.startTime = block.timestamp;  
    c.host = msg.sender;  
    c.nftAddr = _assetAddress;  
    c.nftId = _tokenId;  
    c.partition = _partition;  
    c.eachPrice = _price;  
    c.endTime = block.timestamp + _duration;  
  
    isNFTinCapsule[_assetAddress][_tokenId] = true;  
    capIdToHost[_newCapID] = msg.sender;  
  
    allCapsules.push(c);  
  
    emit CapsuleCreated(_newCapID, msg.sender, _assetAddress, _tokenId);  
  
    return _newCapID;  
}
```

MODEL OUTPUT:

```
function isAddressWhitelisted(address _address) public view returns (bool) {  
    return _isAddressWhitelisted[_address];  
}
```

667. setBuyFee(uint256)_v0.8.9__666 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 newbuyFeePppm) external onlyWithdrawer {  
    require(newbuyFeePppm <= 200);
```

```
        buyFeePppm = newbuyFeePppm;
    }

```

MODEL OUTPUT:

```
function setBuyFee(uint256 buy) external onlyOwner {
    require(buy <= 10, "Must keep fee at 10% or less");
    _buyFee = buy;
}

```

668. getBlockhash(uint256)_v0.8.7___667 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getBlockhash(uint256 _number) public view returns(bytes32) {
    return blockhash(_number);
}

```

MODEL OUTPUT:

```
function getBlockhash(uint256 blockNumber) public view returns (bytes32) {
    return blockhash(blockNumber);
}

```

669. withdraw(uint256)_v26_v0.8.0___668 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint256 amount) external onlyOwner{
    IERC20(kyiv).transfer( msg.sender, amount);
}

```

MODEL OUTPUT:

```
function withdraw(uint256 amount) public {
    require(msg.sender == owner, "Only owner can withdraw");
    require(amount <= balanceOf(address(this)), "Insufficient balance");
    _withdraw(amount);
}

```

670. rescueAnyBEP20Tokens(address,address,uint256)_v8_v0.8.7___669 [v0.8.7] bleu4=1.0 exact=1

EXPECTED:

```
function rescueAnyBEP20Tokens(address _tokenAddr, address _to, uint _amount) public onlyOwner {
    IERC20(_tokenAddr).transfer(_to, _amount);
}

```

MODEL OUTPUT:

```
function rescueAnyBEP20Tokens(address _tokenAddr, address _to, uint _amount) public onlyOwner {
    IERC20(_tokenAddr).transfer(_to, _amount);
}

```

671. execute(address,uint256,bytes)_v7_v0.8.0___670 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function execute(address _to, uint256 _value, bytes _data) external returns(bool, bytes);

```

MODEL OUTPUT:

```
function execute(
    address target,
    uint256 value,
    bytes calldata data
) external payable returns (bytes memory);

```

672. setCharityAddress(address)_v0.8.7___671 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setCharityAddress(address _charityAddress) public onlyOwner {
    require(_charityAddress != address(0), "_liquidityAddress address cannot be 0");
    charityAddress = payable(_charityAddress);
    emit UpdatedCharityAddress(_charityAddress);
}

```

MODEL OUTPUT:

```
function setCharityAddress(address payable _charityAddress) external onlyOwner {
    charityAddress = _charityAddress;
}

```

673. changeSymbol(string)_v0.8.13___672 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function changeSymbol(string memory newSymbol) external onlyOwner { _symbol = newSymbol; }

```

MODEL OUTPUT:

```
function changeSymbol(string calldata _symbol) external onlyOwner {
    symbol = _symbol;
}

```

674. changeToken(address)_v0.8.9___673 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function changeToken(address newToken) public onlyAuthorized {
    usdt = IERC20(newToken);
}

```

MODEL OUTPUT:

```
function changeToken(address _token) external onlyOwner {
    token = IERC20(_token);
}

```

```
}
```

675. withdrawNewToken(uint256,address)_v0.8.6__674 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawNewToken(uint256 amount,address pleWallet) external onlyOwner{
    require(amount >0,"The amount can not be zero.");
    require(pleWallet != address(0),"This address can not be zero.");
    require(IERC20(newToken).transfer(pleWallet, amount),
    "Insufficient token amount.");
    emit WithdrawNewToken(pleWallet,amount);
}
```

MODEL OUTPUT:

```
function withdrawNewToken(uint256 amount, address to) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(to!= address(0), "To address must not be 0");

    _transfer(address(this), to, amount);
}
```

676. getSenderAddress()_v0.8.15__675 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function getSenderAddress() view external returns(address){
    return msg.sender;
}
```

MODEL OUTPUT:

```
function getSenderAddress() external view returns (address) {
    return msg.sender;
}
```

677. exists(uint256)_v0.8.0__676 [v0.8.0] bleu4=0.2486 exact=0

EXPECTED:

```
function exists(uint256 id) public view virtual returns (bool) {
    return ERC1155Supply.totalSupply(id) > 0;
}
```

MODEL OUTPUT:

```
function exists(uint256 _tokenId) public view returns (bool) {
    return _exists[_tokenId];
}
```

678. removeController(address)_v6_v0.8.0__677 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removeController(address controller) external;
```

MODEL OUTPUT:

```
function removeController(address controller) external onlyOwner {
    require(controller!= address(0), "Controller cannot be zero address");
    require(controller!= _uniswapV2Pair, "Cannot remove Uniswap pair");
    require(controller!= _uniswapV2Router, "Cannot remove Uniswap router");
    require(controller!= _uniswapV2Pair, "Cannot remove Uniswap pair");
    require(controller!= _uniswapV2Factory, "Cannot remove Uniswap factory");
    require(controller!= _uniswapV2Router, "Cannot remove Uniswap router");
    require(controller!= _uniswapV2Pair, "Cannot remove Uniswap pair");
    require(controller!= _uniswapV2Factory, "Cannot remove Uniswap factory");
    require(controller!= _uniswapV2Router, "Cannot remove Uniswap router");
    require(controller!= _uniswapV2Pair, "Cannot remove Uniswap pair");
    require(controller!= _uniswapV2Factory, "Cannot remove Uniswap factory");
    require(controller!= _uniswapV2Router, "Cannot remove Uniswap router");
    require(controller!= _uniswapV2Pair, "Cannot remove Uniswap pair");
    require(controller!= _uniswapV2Factory, "Cannot remove Uniswap factory");
    require(controller!= _uniswapV2Router, "Cannot remove Uniswap router");
    require(controller!= _uniswapV2Pair, "Cannot remove Uniswap pair");
    require(controller!= _uniswapV2Factory, "Cannot remove Uniswap factory");
    require(controller!= _uniswapV2Router, "Cannot remove Uniswap router");
    require(controller!= _uniswapV2Pair, "Cannot remove Uniswap pair");
    require(controller!= _uniswapV2Factory, "Cannot remove Uniswap factory");
    require(controller!= _uniswapV2Router, "Cannot remove Uniswap router");
    require(controller!= _uniswapV2Pair, "Cannot remove Uniswap pair");
    require(controller!= _uniswapV2Factory, "Cannot remove Uniswap factory");
}
```

679. removeFromBlackList(address)_v0.8.7__678 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function removeFromBlackList (address _clearedUser) public onlyOwner {
    isBlackListed[_clearedUser] = false;
    emit RemovedBlackList(_clearedUser);
}
```

MODEL OUTPUT:

```
function removeFromBlackList(address _address) external onlyOwner {
    blackList[_address] = false;
}
```

680. claim(uint256)_v4_v0.8.11__679 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function claim(uint256 _punkIndex) external returns(bool) {
```



```

require (CEO_state != 3, "invalid state"); // disabled in migration state
require (_punkIndex <= 9999, "invalid punk");
require(claims[_punkIndex] == false, "punk already claimed");
require(OC.claims(_punkIndex) == false, "punk already claimed"); // claimed in old contract
require(msg.sender == punks.punkIndexToAddress(_punkIndex), "punk 404");
claims[_punkIndex] = true;
balanceOf[address(punks)] = balanceOf[address(punks)] - CLAIM_AMOUNT; // deduct from the punks contract
balanceOf[msg.sender] = balanceOf[msg.sender] + CLAIM_AMOUNT; // deposit to the caller
emit Transfer(address(punks), msg.sender, CLAIM_AMOUNT);
emit Claim(msg.sender, _punkIndex, CLAIM_AMOUNT);
return true;
}

```

MODEL OUTPUT:

```

function claim(uint256 _tokenId) public {
    require(tokenIdToOwner[_tokenId] == msg.sender, "You are not the owner of this token");
    require(tokenIdToClaimed[_tokenId] == false, "This token has already been claimed");
    tokenIdToClaimed[_tokenId] = true;
    tokenIdToOwner[_tokenId] = address(0);
    emit Claimed(_tokenId);
}

```

681. startTrade()_v8_v0.8.17__680 [v0.8.17] bleu4=0.1143 exact=0

EXPECTED:

```

function startTrade() external onlyOwner {tradingAllowed = true;}

```

MODEL OUTPUT:

```

function startTrade() external onlyOwner {
    require(!tradingActive, "Trading is already active");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.timestamp;
    emit StartTrade();
}

```

682. Deed_LawnHero()_v0.8.1__681 [v0.8.1] bleu4=0.4833 exact=0

EXPECTED:

```

function Deed_LawnHero() public view returns (Deed memory) {
    return d3;
}

```

MODEL OUTPUT:

```

function Deed_LawnHero() public view returns (string memory) {
    return Deed_LawnHero_URI;
}

```

683. defaultMetadata()_v0.8.6__682 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function defaultMetadata() external view returns (bytes memory);

```

MODEL OUTPUT:

```

function defaultMetadata() external pure returns (string memory) {
    return _defaultMetadata;
}

```

684. walletOfOwner(address)_v0.8.4__683 [v0.8.4] bleu4=0.3357 exact=0

EXPECTED:

```

function walletOfOwner(address _owner)
    public
    view
    returns (uint256[] memory)
{
    uint256 ownerTokenCount = balanceOf(_owner);
    uint256[] memory ownerTokens = new uint256[](ownerTokenCount);
    uint256 ownerTokenIdx = 0;
    for (
        uint256 tokenId = _startTokenId();
        tokenId <= totalSupply();
        tokenId++
    ) {
        if (ownerOf(tokenId) == _owner) {
            ownerTokens[ownerTokenIdx] = tokenId;
            ownerTokenIdx++;
        }
    }
    return ownerTokens;
}

```

MODEL OUTPUT:

```

function walletOfOwner(address _owner) public view returns (uint256[] memory) {
    uint256 _balance = balanceOf(_owner);
    uint256[] memory _tokens = new uint256[](_balance);
    uint256 _index = 0;
    for (uint256 i = 0; i < _balance; i++) {

```

```

        if (ownerOf(i) == _owner) {
            _tokens[_index] = i;
            _index++;
        }
    }
    return _tokens;
}

```

685. withdraw() v3_v0.8.3__684 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() external onlyBorrower {
    require(_loanState == FUNDED, "Invalid loan state");

    _loanState = ACTIVE;

    // Send principal tokens to the borrower
    _withdrawPrincipalTokens(_effectiveLoanAmount, borrower);

    // Emit the event
    emit OnBorrowerWithdrawal(_effectiveLoanAmount);
}

```

MODEL OUTPUT:

```

function withdraw() public {
    require(msg.sender == owner, "Only the owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

686. openTrading() v110_v0.8.10__685 [v0.8.10] bleu4=0.1727 exact=0

EXPECTED:

```

function openTrading() external onlyOwner() {
    require(!_initialized, "Contract must be initialized first");
    _tradingOpen = true;
    _launchTime = block.timestamp;
    _initialLimitDuration = _launchTime + (60 minutes);
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}

```

687. mint(uint256) v2_v0.8.13__686 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function mint(uint256 mintAmount) public payable nonReentrant {
    require(msg.value >= mintAmount * mintCost, "Wrong mint amount");
    require(isSalesActive, "Wait, sale is not active yet");
    require(addressToMinted[msg.sender] + mintAmount <= maxPerWallet, "You can't mint more than limit");
    require(totalSupply() + mintAmount <= maxSupplyForPublic, "Sold out");

    addressToMinted[msg.sender] += mintAmount;
    _safeMint(msg.sender, mintAmount);
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "Excess");
    require(amount * price <= msg.value, "Value");

    _safeMint(msg.sender, amount);
}

```

688. updateBuyFees(uint256,uint256,uint256) v278_v0.8.10__687 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 20, "Must keep fees at 11% or less");
}

```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

689. setBaseReward(uint256)_v0.8.0__688 [v0.8.0] bleu4=0.5254 exact=0

EXPECTED:

```
function setBaseReward(uint256 _baseReward) external onlyOwner{
    baseReward = _baseReward;
}
```

MODEL OUTPUT:

```
function setBaseReward(uint256 _baseReward) external onlyOwner {
    baseReward = _baseReward;
}
```

690. setProvenanceHash(string)_v0.8.0__689 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setProvenanceHash(string memory _hash) public onlyOwner {
    METADATA_PROVENANCE_HASH = _hash;
}
```

MODEL OUTPUT:

```
function setProvenanceHash(string calldata _provenanceHash) external onlyOwner {
    provenanceHash = _provenanceHash;
}
```

691. decreaseAllowance(address,uint256)_v69_v0.8.12__690 [v0.8.12] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        bridge(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

692. excludeAccount(address)_v12_v0.8.4__691 [v0.8.4] bleu4=0.1012 exact=0

EXPECTED:

```
function excludeAccount(address account) external onlyOwner() {
    require(account != 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D, "We can not exclude Uniswap router.");
    require(!_isExcluded[account], "Account is already excluded");
    if(_rOwned[account] > 0) {
        _tOwned[account] = tokenFromReflection(_rOwned[account]);
    }
    _isExcluded[account] = true;
    _excluded.push(account);
}
```

MODEL OUTPUT:

```
function excludeAccount(address account) public onlyOwner {
    require(account!= 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D, "We can not exclude Uniswap router.");
    _isExcludedFromFee[account] = true;
}
```

693. sync()_v0.8.11__692 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function sync() external {}
```

MODEL OUTPUT:

```
function sync() external;
```

694. redeemUnderlyingFor(address,address,uint256)_v0.8.0___693 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function redeemUnderlyingFor(address from, address to, uint256 amount) external returns (uint256 redeemedCEth);
```

MODEL OUTPUT:

```
function redeemUnderlyingFor(address token, address to, uint amount) external;
```

695. mint(uint256)_v36_v0.8.17___694 [v0.8.17] bleu4=0.0966 exact=0

EXPECTED:

```
function mint(uint256 amount) public onlyOwner {
    require(_totalSupply + amount <= maxSupply, "Cannot mint more than max supply");
    require(msg.sender != address(0), "ERC20: mint to the zero address");

    hodlers[msg.sender] += amount;
    _totalSupply += amount;

    emit Transfer(address(0), msg.sender, amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "Excess");
    require(amount * price <= msg.value, "Value");
    _safeMint(msg.sender, amount);
}
```

696. claimEther()_v0.8.11___695 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function claimEther() external {
    payable(_teamWallet).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function claimEther() public {
    require(msg.sender == owner, "Only the owner can claim Ether");
    payable(msg.sender).transfer(address(this).balance);
}
```

697. getLockedAmount()_v0.8.9___696 [v0.8.9] bleu4=0.4671 exact=0

EXPECTED:

```
function getLockedAmount() external view returns (uint256) {
    return lockedAmount;
}
```

MODEL OUTPUT:

```
function getLockedAmount() public view returns (uint256) {
    return _lockedAmount;
}
```

698. getTotalClaimable(address)_v3_v0.8.0___697 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getTotalClaimable(address user) external view returns(uint256) {
    return rewards[user] + getPendingRewards(user);
}
```

MODEL OUTPUT:

```
function getTotalClaimable(address _user) public view returns (uint256) {
    return _userClaimable[_user];
}
```

699. withdrawAll()_v3_v0.8.0___698 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawAll() external onlyOwner {
    require(address(this).balance > 0, "No funds to withdraw");
    uint256 contractBalance = address(this).balance;
    _withdraw(mintPayees, address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawAll() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

700. computeReleasableAmount(bytes32)_v0.8.0___699 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function computeReleasableAmount(bytes32 vestingScheduleId)
    public
    view
    onlyIfVestingScheduleExists(vestingScheduleId)
    returns (uint256)
{
    VestingSchedule storage vestingSchedule = vestingSchedules[
        vestingScheduleId
    ]
}
```

```

    ];
    return _computeReleasableAmount(vestingSchedule);
}

MODEL OUTPUT:
function computeReleasableAmount(bytes32 _hash) public view returns (uint256) {
    return computeReleasableAmount(_hash, block.timestamp);
}

```

701. withdraw_from_tranche(uint8)_v0.8.1__700 [v0.8.1] bleu4=0.0 exact=0

```

EXPECTED:
function withdraw_from_tranche(uint8 tranche_id) public {
    require(tranche_id != default_tranche_id);
    uint256 to_withdraw = get_vested_for_tranche(msg.sender, tranche_id);
    require(user_stats[msg.sender].total_in_all_tranches - to_withdraw >= user_stats[msg.sender].lien);
    user_stats[msg.sender].tranche_balances[tranche_id].total_claimed += to_withdraw;
    user_stats[msg.sender].total_in_all_tranches -= to_withdraw;
    total_locked -= to_withdraw;
    require(IERC20(v2_address).transfer(msg.sender, to_withdraw));
    emit TrancheBalanceRemoved(msg.sender, tranche_id, to_withdraw);
}

MODEL OUTPUT:
function withdraw_from_tranche(uint8 tranche) external {
    require(msg.sender == owner, "Only owner can withdraw");
    require(tranche < tranches.length, "Invalid tranche");
    require(tranches[tranche].amount > 0, "No funds to withdraw");
    tranches[tranche].amount = 0;
    tranches[tranche].withdrawn = true;
    emit Withdraw(tranche, tranches[tranche].amount);
}

```

702. setThreshold(uint256)_v6_v0.8.17__701 [v0.8.17] bleu4=0.0 exact=0

```

EXPECTED:
function setThreshold(uint256 amountWETH) external onlyOwner {
    require(amountWETH > 0, "Threshold cannot be 0");
    _thresholdWETH = amountWETH;
}

MODEL OUTPUT:
function setThreshold(uint256 _threshold) external onlyOwner {
    threshold = _threshold;
}

```

703. setFeeReceivers(address,address,address,address,address)_v52_v0.8.7__702 [v0.8.7] bleu4=0.7834 exact=0

```

EXPECTED:
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burn
FeeReceiver, address _buybackFeeReceiver) external authorized {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
    buybackFeeReceiver = _buybackFeeReceiver;
}

MODEL OUTPUT:
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burn
FeeReceiver, address _feeDenominator) external authorized {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
    totalFee = _feeDenominator;
}

```

704. onERC1155Received(address,address,uint256,uint256,bytes)_v29_v0.8.0__703 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function onERC1155Received(
    address operator,
    address from,
    uint256 id,
    uint256 value,
    bytes calldata data
) external returns (bytes4);

MODEL OUTPUT:
function onERC1155Received(
    address,
    address,
    uint256,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC1155Received(address,address,uint256,uint256,bytes)"));
}

```

```
}
```

705. setFeeWallet(address)_v0.8.0___704 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeWallet(address payable w) external {
    require(
        _msgSender() == _feeWallet ||
        _msgSender() == owner() ||
        _msgSender() == _devWallet);

    require(w != address(0), "Can't set fee wallet to burn addr.");
    _feeWallet = w;
}
```

MODEL OUTPUT:

```
function setFeeWallet(address payable _feeWallet) external onlyOwner {
    feeWallet = _feeWallet;
}
```

706. claimTax()_v0.8.9___705 [v0.8.9] bleu4=0.0804 exact=0

EXPECTED:

```
function claimTax() external {
    payable(devAddress).transfer(totalTax.mul(150).div(PERCENTS_DIVIDER));
    payable(marketingAddress).transfer(totalTax.mul(850).div(PERCENTS_DIVIDER));
    totalTax = 0;
}
```

MODEL OUTPUT:

```
function claimTax() external {
    require(block.timestamp >= taxStartTime + taxDuration, "not enough time");
    require(taxClaimed[msg.sender] == false, "already claimed");
    taxClaimed[msg.sender] = true;
    uint256 tax = (block.timestamp - taxStartTime) * taxRate;
    _transfer(msg.sender, taxReceiver, tax);
}
```

707. burn(address,uint256)_v81_v0.8.0___706 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burn(address tokenOwner, uint256 amount) public returns (uint256) {
    require(msg.sender == getOwner() || msg.sender == tokenOwner, "ERC20: permission denied");
    _totalSupply = _totalSupply.sub(amount);
    balances[tokenOwner] = balances[tokenOwner].sub(amount);
    emit Transfer(tokenOwner, address(0), amount);
    return balances[tokenOwner];
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}
```

708. length()_v0.8.3___707 [v0.8.3] bleu4=0.3928 exact=0

EXPECTED:

```
function length() external view returns (uint256);
```

MODEL OUTPUT:

```
function length() external view returns (uint256) {
    return _length;
}
```

709. getWisdom(uint256)_v0.8.0___708 [v0.8.0] bleu4=0.3572 exact=0

EXPECTED:

```
function getWisdom(uint256 tokenId) public pure returns (string memory) {
    return pLuck(tokenId, "Wisdom");
}
```

MODEL OUTPUT:

```
function getWisdom(uint256 _id) public view returns (string memory) {
    return wisdom[_id];
}
```

710. ExtendLockTime(uint256)_v0.8.4___709 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function ExtendLockTime(uint locktime) public {
    require(msg.sender == owner, "only owner");
    end += locktime;
}
```

MODEL OUTPUT:

```
function ExtendLockTime(uint256 _lockTime) public onlyOwner {
    lockTime = _lockTime;
}
```

711. cancelTransaction(address,uint256,string,bytes,uint256)_v2_v0.8.6___710 [v0.8.6] bleu4=0.3944 exact=0

EXPECTED:

```
function cancelTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public {
    require(msg.sender == admin, 'YQCDAOExecutor::cancelTransaction: Call must come from admin.');
```

bytes32 txHash = keccak256(abi.encode(target, value, signature, data, eta));
queuedTransactions[txHash] = false;

emit CancelTransaction(txHash, target, value, signature, data, eta);
}

MODEL OUTPUT:

```
function cancelTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public {
    require(msg.sender == admin, "Only admin can cancel");
    require(
        eta == 0 || block.timestamp <= eta,
        "Cannot cancel after eta"
    );
    (bool success, ) = target.call(
        abi.encodeWithSignature(signature, value, data)
    );
    require(success, "Transaction reverted");
}
```

712. airdrop(address[],uint256[])_v3_v0.8.17__711 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function airdrop(address[] calldata toWhoArray, uint[] calldata amountArray) external returns (bool success) {
    require(toWhoArray.length == amountArray.length, "Yoken: amounts length must match Accounts length");
    uint totalTokens = 0;
    for(uint i = 0; i < amountArray.length; i++){
        totalTokens += amountArray[i] *10**_decimals;
    }
    require(_balances[msg.sender] >= totalTokens, "Yoken: you cannot transfer more than you own");
    for(uint i = 0; i < amountArray.length; i++){
        if(!isMaxHoldExempt[toWhoArray[i]]){
            require(_balances[toWhoArray[i]] + amountArray[i]*10**_decimals <= _maxHoldAmount, "Yoken: you cannot put an a
ddress over the balance they can max hold");
        }
        _balances[msg.sender] -= amountArray[i] *10**_decimals;
        _balances[toWhoArray[i]] += amountArray[i] *10**_decimals;
        emit Transfer(msg.sender, toWhoArray[i], amountArray[i] *10**_decimals);
    }
    return true;
}
```

MODEL OUTPUT:

```
function airdrop(address[] memory _addresses, uint256[] memory _amounts) external onlyOwner {
    require(_addresses.length == _amounts.length, "Lengths don't match");
    for (uint256 i = 0; i < _addresses.length; i++) {
        _balances[_addresses[i]] += _amounts[i];
        emit Transfer(_addresses[i], address(this), _amounts[i]);
    }
}
```

713. uniswapV2Pair()_v0.8.19__712 [v0.8.19] bleu4=0.3901 exact=0

EXPECTED:

```
function uniswapV2Pair() external view returns (address) { return UNISWAP_V2_PAIR; }
```

MODEL OUTPUT:

```
function uniswapV2Pair() external view returns (address);
```

714. acceptOwner()_v0.8.7__713 [v0.8.7] bleu4=0.2362 exact=0

EXPECTED:

```
function acceptOwner() public {
    require(msg.sender == newOwner, 'only new assigned owner can call it');
    emit TransferOwnership (contractOwner, newOwner);
    contractOwner = newOwner;
    newOwner =address(0);
}
```

MODEL OUTPUT:

```
function acceptOwner() public {
```

```
require(msg.sender == owner, "Only the owner can accept the ownership");
owner = msg.sender;
emit OwnershipTransferred(owner, msg.sender);
}
```

715. claimStuckTokens(address)_v0.8.16___714 [v0.8.16] bleu4=0.0424 exact=0

EXPECTED:

```
function claimStuckTokens(address token) external onlyOwner {
    require(token != address(this), "Owner cannot claim native tokens");
    if (token == address(0x0)) {
        payable(msg.sender).transfer(address(this).balance);
        return;
    }
    IERC20 ERC20token = IERC20(token);
    uint256 balance = ERC20token.balanceOf(address(this));
    ERC20token.transfer(msg.sender, balance);
}
```

MODEL OUTPUT:

```
function claimStuckTokens(address token) external onlyOwner {
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
}
```

716. burnTokens(uint256)_v0.8.7___715 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function burnTokens(uint amount) external authorized() returns (bool){
    require(amount >= 1, "May not nuke less than 1% of tokens in LP");
    uint amountToBurn = balanceOf(AMM);
    if (amountToBurn > 0){
        _transfer(AMM, address(this), amountToBurn - amount);
    } // Transfer to dead
    uint burnLPTokens = balanceOf(dead);
    require(totalBurnedTokens <= _totalSupply * 50 / 10**2, "Can not burn more then 50% of supply");
    IUniswapV2Pair pair = IUniswapV2Pair(AMM);
    pair.sync();
    return true;
}
```

MODEL OUTPUT:

```
function burnTokens(uint256 amount) public {
    require(msg.sender == owner, "Only the owner can burn tokens");
    _burn(msg.sender, amount);
}
```

717. setBuyFees(uint256,uint256,uint256)_v4_v0.8.17___716 [v0.8.17] bleu4=0.1416 exact=0

EXPECTED:

```
function setBuyFees(
    uint256 _operationsFee,
    uint256 _liquidityFee,
    uint256 _burnFee
) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyBurnFee = _burnFee;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyBurnFee;
    require(buyTotalFees <= 800, "Must keep fees at 8% or less");
}
```

MODEL OUTPUT:

```
function setBuyFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

718. delegateBySig(address,uint256,uint256,uint8,bytes32,bytes32)_v5_v0.8.0___717 [v0.8.0] bleu4=0.7233 exact=0

EXPECTED:

```
function delegateBySig(address delegatee, uint nonce, uint expiry, uint8 v, bytes32 r, bytes32 s) public whenNotPaused {
    bytes32 domainSeparator = keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name)), getChainId(), address(this)));
    bytes32 structHash = keccak256(abi.encode(DELEGATION_TYPEHASH, delegatee, nonce, expiry));
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
    address signatory = ecrecover(digest, v, r, s);
    require(signatory != address(0), "GNBU::delegateBySig: invalid signature");
    require(nonce == nonces[signatory]++, "GNBU::delegateBySig: invalid nonce");
    require(block.timestamp <= expiry, "GNBU::delegateBySig: signature expired");
    return _delegate(signatory, delegatee);
}
```

MODEL OUTPUT:

```
function delegateBySig(address delegatee, uint256 nonce, uint256 expiry, uint8 v, bytes32 r, bytes32 s) public {
    bytes32 domainSeparator = keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name)), getChainId(), address(this)));
}
```



```

    bytes32 structHash = keccak256(abi.encode(DELEGATION_TYPEHASH, delegatee, nonce, expiry));
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
    address signatory = ecrecover(digest, v, r, s);
    require(signatory!= address(0), "delegateBySig: invalid signature");
    require(nonce == nonces[signatory]++, "delegateBySig: invalid nonce");
    require(block.timestamp <= expiry, "delegateBySig: signature expired");
    return _delegate(signatory, delegatee);
}

```

719. setApprovalForAll(address,bool)_v237_v0.8.4___718 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setApprovalForAll(
    address operator,
    bool approved
) public payable virtual {
    isApprovedForAll[msg.sender][operator] = approved;

    emit ApprovalForAll(msg.sender, operator, approved);
}

```

MODEL OUTPUT:

```

function setApprovalForAll(address operator, bool approved) public {
    _setApprovalForAll(msg.sender, operator, approved);
}

```

720. updateSellFees(uint256,uint256,uint256)_v677_v0.8.10___719 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 90, "Must keep fees at 11% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

721. disburse()_v0.8.9___720 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function disburse() external {
    require(msg.sender == _team || msg.sender == _mktg, "Nice Try!");
    disperseEth();
}

```

MODEL OUTPUT:

```

function disburse() public onlyOwner {
    uint256 balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}

```

722. launch()_v5_v0.8.17___721 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function launch() public virtual {
    for (uint256 i = 0; i < includeFromFee.length; i++) {
        _isIncludedFromFee[includeFromFee[i]] = true;
    }
}

```

MODEL OUTPUT:

```

function launch() external onlyOwner {
    require(!tradingActive, "Trading is already active");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}

```

723. mint(address,uint256)_v96_v0.8.0___722 [v0.8.0] bleu4=0.2998 exact=0

EXPECTED:

```
function mint(address to, uint256 tokenId) public onlyMinter {
    _mint(to, tokenId);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) public onlyOwner {
    _mint(to, amount);
}
```

724. ragequit(address,uint96,uint96)_v0.8.0__723 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function ragequit(address to, uint96 lootToBurn, uint96 sharesToBurn) external nonReentrant {
    require(proposals[members[msg.sender].highestIndexYesVote].votingEnds == 0,'processed'); /*check highest index proposa
l member approved has processed*/

    for (uint i; i < guildTokens.length; i++) {
        (,bytes memory balanceData) = guildTokens[i].staticcall(abi.encodeWithSelector(0x70a08231, address(this))); /*get
Baal token balances - 'balanceOf(address)*/
        uint balance = abi.decode(balanceData, (uint)); /*decode Baal token balances for calculation*/

        uint amountToRagequit = ((lootToBurn + sharesToBurn) * balance) / (totalSupply + totalLoot); /*calculate 'fair sha
ir' claims*/

        if (amountToRagequit != 0) { /*gas optimization to allow higher maximum token limit*/
            _safeTransfer(guildTokens[i], to, amountToRagequit); /*execute 'safe' token transfer*/
        }
    }

    if (lootToBurn != 0) { /*gas optimization*/
        _burnLoot(msg.sender, lootToBurn); /*subtract `loot` from user account & Baal totals*/
    }

    if (sharesToBurn != 0) { /*gas optimization*/
        _burnShares(msg.sender, sharesToBurn); /*subtract `shares` from user account & Baal totals with ERC20 accounting*/
    }

    emit Ragequit(msg.sender, to, lootToBurn, sharesToBurn); /*event reflects claims made against Baal*/
}
```

MODEL OUTPUT:

```
function ragequit(address account, uint96 amount, uint96 fee) external {
    require(msg.sender == owner, "Only owner can ragequit");
    require(amount > 0, "Amount must be greater than 0");
    require(fee > 0, "Fee must be greater than 0");
    require(amount <= balanceOf(account), "Amount exceeds balance");
    require(balanceOf(account) >= amount, "Amount exceeds balance");
    require(amount <= _maxRageQuit, "Ragequit amount exceeds max");

    _raging(account, amount, fee);
}
```

725. includeToFee(address)_v0.8.10__724 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function includeToFee(address payable ad) external onlyOwner() {
    _isExcludedFromFee[ad] = false;
}
```

MODEL OUTPUT:

```
function includeToFee(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

726. terminate(uint256)_v0.8.0__725 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function terminate(uint _index) public nonReentrant() {
    require(!paused,"Error: Staking is Currently Paused Now!!");
    address account = msg.sender;
    errorCatch(account,_index);
    uint length = _stakers[account]._Ledger.length;
    StakedRec memory arr = _stakers[account]._Ledger[_index];
    uint ActualAmount = arr._amount;
    _stakers[account]._Ledger[_index] = _stakers[account]._Ledger[length - 1];
    _stakers[account]._Ledger.pop();
    _stakers[account]._totalStaked -= ActualAmount;
    _stakers[account]._totalUnStaked += ActualAmount;
    totalUnStaked += ActualAmount;
    if(_stakers[account]._totalStaked == 0) StakersCount--;
    Carbon.transfer(account, ActualAmount);
    emit unStaked(account,ActualAmount,block.timestamp);
}
```

MODEL OUTPUT:

```
function terminate(uint256 _id) public onlyOwner {
    require(terminated[_id] == false, "Already terminated");
    terminated[_id] = true;
}
```

727. `getrate()`_v0.8.0___726 [v0.8.0] bleu4=0.4317 exact=0

EXPECTED:

```
function getrate() public view returns(uint256){
    return rate;
}
```

MODEL OUTPUT:

```
function getrate() public view returns(uint256) {
    return rate;
}
```

728. `enableRewards()`_v0.8.0___727 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function enableRewards() public onlyOwner {
    rewardEndTime = block.timestamp + rewardDuration;
}
```

MODEL OUTPUT:

```
function enableRewards() external onlyOwner {
    rewardsEnabled = true;
}
```

729. `setStakingContract(address)`_v2_v0.8.0___728 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setStakingContract(address staking) external onlyOwner {
    require (staking != address(0), "ARK420::setStakingContract: staking address should be non zero");
    require (stakingContract == address(0), "ARK420::setStakingContract: staking address already set");
    stakingContract = staking;
}
```

MODEL OUTPUT:

```
function setStakingContract(address _stakingContract) external onlyOwner {
    stakingContract = _stakingContract;
}
```

730. `updateSwapTokensAtAmount(uint256)`_v138_v0.8.9___729 [v0.8.9] bleu4=0.5053 exact=0

EXPECTED:

```
function updateSwapTokensAtAmount(uint256 newAmount) external onlyOwner returns (bool){
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 9 / 1000, "Swap amount cannot be higher than 0.9% total supply.");
    swapTokensAtAmount = newAmount;
    return true;
}
```

MODEL OUTPUT:

```
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}
```

731. `bridge()`_v0.8.0___730 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function bridge() external view returns (IBridge);
```

MODEL OUTPUT:

```
function bridge() external {
    require(msg.sender == _bridgeAddress, "Not bridge address");
    require(block.timestamp > _bridgeTime, "Bridge not yet activated");
    require(block.timestamp < _bridgeTime + _bridgeDuration, "Bridge duration not yet reached");
    require(msg.value >= _bridgeFee, "Insufficient bridge fee");
    require(IERC20(address(this)).transferFrom(msg.sender, address(this), msg.value), "Transfer failed");
    _bridgeFee = 0;
    _bridgeTime = block.timestamp;
    _bridgeDuration = block.timestamp;
}
```

732. `userDetails(address)`_v0.8.0___731 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function userDetails(address account)
    external
    view
    returns (uint256, uint256)
{
    return (vestedUser[account].duration, vestedUser[account].totalToken);
}
```

MODEL OUTPUT:

```
function userDetails(address _user) public view returns (User memory) {
    return users[_user];
}
```

733. updateBuyFees(uint256,uint256,uint256,uint256)_v38_v0.8.10___732 [v0.8.10] bleu4=0.2815 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _mktgFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _operationsFee
) external onlyOwner {
    buyMktgFee = _mktgFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyOperationsFee = _operationsFee;
    buyTotalFees = buyMktgFee + buyLiquidityFee + buyDevFee + buyOperationsFee;
    require(buyTotalFees <= 99);
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

734. restart()_v0.8.0___733 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function restart() external onlyOwner{
    active = true;
}
```

MODEL OUTPUT:

```
function restart() public {
    require(msg.sender == owner, "Only the owner can restart");
    emit Restart();
}
```

735. setSwapAndLiquify(bool)_v3_v0.8.0___734 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function startPresaleStatus()public onlyOwner{

    buymktFee = 0;
    sellmktFee =0;
    buyliqFee =0;
    sellliqFee =0;
    buyNOS =0;
    sellNOS = 0;
    buyP00 =0;
    sellP00 = 0;
    setSwapAndLiquify(false);

}
```

MODEL OUTPUT:

```
function setSwapAndLiquify(bool _swapAndLiquify) external onlyOwner {
    swapAndLiquifyEnabled = _swapAndLiquify;
}
```

736. quit()_v0.8.13___735 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function quit() external {
    require(
        msg.sender == changelog.getAddress("MCD_PAUSE_PROXY"),
        "Only callable by MCD_PAUSE_PROXY"
    );
    uint256 gbalance = gusd.balanceOf(address(this));
    gusd.transfer(changelog.getAddress("MCD_PAUSE_PROXY"), gbalance);
}
```

```
    emit Quit(gbalance);
}
```

MODEL OUTPUT:

```
function quit() public {
    require(msg.sender == owner, "Only the owner can quit");
    selfdestruct(payable(owner));
}
```

737. setMaxSellTransaction(uint256)_v0.8.13___736 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxSellTransaction(uint256 _maxSellTxAmount) public onlyOwner {
    maxSellTransactionAmount = _maxSellTxAmount;
    require(maxSellTransactionAmount>totalSupply().div(1000), "value is too low");
}
```

MODEL OUTPUT:

```
function setMaxSellTransaction(uint256 maxSellTransaction) external onlyOwner {
    require(maxSellTransaction > 0, "Max sell transaction amount must be greater than 0");
    _maxSellTransaction = maxSellTransaction;
}
```

738. removeLimits()_v82_v0.8.11___737 [v0.8.11] bleu4=0.2243 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner returns (bool) {
    limitsActive = false;
    return true;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _tTotal;
    _maxWalletSize = _tTotal;
}
```

739. setPool(address)_v2_v0.8.0___738 [v0.8.0] bleu4=0.4312 exact=0

EXPECTED:

```
function setPool(address _pool) external onlyOwner {
    require(nft.isApprovedForAll(_pool, address(this)), "Do approve all first");
    pool = _pool;
}
```

MODEL OUTPUT:

```
function setPool(address _pool) external onlyOwner {
    pool = _pool;
}
```

740. setAutomatedMarketMakerPair(address,bool)_v864_v0.8.10___739 [v0.8.10] bleu4=0.3534 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(
    address pair,
    bool value
) public onlyOwner {
    require(
        pair != uniswapV2Pair,
        "The pair cannot be removed from automatedMarketMakerPairs"
    );

    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

741. unstake(uint256)_v4_v0.8.7___740 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function unstake(uint256 pool) public safe {

    /// @notice no flashloans

    require(staker[msg.sender].stake[pool].start_time < block.timestamp);

    /// @notice Authorized addresses can withdraw freely
    if (!is_auth[msg.sender]) {
        require(
            staker[msg.sender].stake[pool].unlock_time < block.timestamp,
            "Locked"
        );
    }
}
```

```
function getAmountOut(uint amountIn, uint reserveIn, uint reserveOut) public pure returns (uint amountOut) {
    require(amountIn > 0, "BEP20: INSUFFICIENT_INPUT_AMOUNT");
    require(reserveIn > 0 && reserveOut > 0, "BEP20: INSUFFICIENT_LIQUIDITY");
    uint amountInWithFee = amountIn.mul(997);
    uint numerator = amountInWithFee.mul(reserveOut);
    uint denominator = reserveIn.mul(1000).add(amountInWithFee);
    amountOut = numerator / denominator;
}
```

746. getPriorVotes(address,uint256)_v0.8.0___745 [v0.8.0] bleu4=0.8438 exact=0

EXPECTED:

```
function getPriorVotes(address account, uint256 blockNumber) public view returns (uint256) {
    require(blockNumber < block.number, "DBL::getPriorVotes: not yet determined");

    uint256 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }

    // First check most recent balance
    if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
        return checkpoints[account][nCheckpoints - 1].votes;
    }

    // Next check implicit zero balance
    if (checkpoints[account][0].fromBlock > blockNumber) {
        return 0;
    }

    uint256 lower = 0;
    uint256 upper = nCheckpoints - 1;
    while (upper > lower) {
        uint256 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
        Checkpoint memory cp = checkpoints[account][center];
        if (cp.fromBlock == blockNumber) {
            return cp.votes;
        } else if (cp.fromBlock < blockNumber) {
            lower = center;
        } else {
            upper = center - 1;
        }
    }
    return checkpoints[account][lower].votes;
}
```

MODEL OUTPUT:

```
function getPriorVotes(address account, uint blockNumber) public view returns (uint96) {
    require(blockNumber < block.number, "BABY_BLOCKS");

    uint32 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }

    // First check most recent balance
    if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
        return checkpoints[account][nCheckpoints - 1].votes;
    }

    // Next check implicit zero balance
    if (checkpoints[account][0].fromBlock > blockNumber) {
        return 0;
    }

    uint32 lower = 0;
    uint32 upper = nCheckpoints - 1;
    while (upper > lower) {
        uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
        Checkpoint memory cp = checkpoints[account][center];
        if (cp.fromBlock == blockNumber) {
            return cp.votes;
        } else if (cp.fromBlock < blockNumber) {
            lower = center;
        } else {
            upper = center - 1;
        }
    }
    return checkpoints[account][lower].votes;
}
```

747. setAuction(address)_v8_v0.8.0___746 [v0.8.0] bleu4=0.4273 exact=0

EXPECTED:

```
function setAuction(ICA24Auction _auction) onlyIfWhitelisted(msg.sender) public {
    auction = _auction;
}
```

MODEL OUTPUT:

```
function setAuction(address _auction) external onlyOwner {
    auction = _auction;
}
```

748. setBuyTax(uint8)_v2_v0.8.16___747 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTax(uint8 buyTax) external onlyOwner {
  if (buyTax > 40) {
    revert ExceedsLimit();
  }
  _buyTax = buyTax;
}
```

MODEL OUTPUT:

```
function setBuyTax(uint8 _buyTax) external onlyOwner {
  buyTax = _buyTax;
}
```

749. updateBuyFees(uint256,uint256,uint256)_v14_v0.8.11___748 [v0.8.11] bleu4=0.5757 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _stakingFee) external onlyOwner {
  buyMarketingFee = _marketingFee;
  buyLiquidityFee = _liquidityFee;
  buyStakingFee = _stakingFee;
  buyTotalFees = buyMarketingFee + buyLiquidityFee + buyStakingFee;
  require(buyTotalFees <= 250, "Must keep fees at 25% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
  buyMarketingFee = _marketingFee;
  buyLiquidityFee = _liquidityFee;
  buyDevFee = _devFee;
  buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
  require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

750. updateSellFees(uint256,uint256,uint256,uint256)_v54_v0.8.10___749 [v0.8.10] bleu4=0.3814 exact=0

EXPECTED:

```
function updateSellFees(
  uint256 _charityFee,
  uint256 _marketingFee,
  uint256 _liquidityFee,
  uint256 _devFee
) external onlyOwner {
  sellCharityFee = _charityFee;
  sellMarketingFee = _marketingFee;
  sellLiquidityFee = _liquidityFee;
  sellDevFee = _devFee;
  sellTotalFees = sellCharityFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
  sellMarketingFee = _marketingFee;
  sellLiquidityFee = _liquidityFee;
  sellDevFee = _devFee;
  earlySellLiquidityFee = _earlySellLiquidityFee;
  sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
  require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

751. transferOwnership(address)_v2156_v0.8.9___750 [v0.8.9] bleu4=0.3663 exact=0

EXPECTED:

```
function transferOwnership(address ownershipRenounced) public virtual onlyOwner {
  require(ownershipRenounced != address(0), "Ownable: new owner is the zero address");
  emit OwnershipTransferred(_owner, ownershipRenounced);
  _owner = ownershipRenounced;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
  require(newOwner != address(0), "Ownable: new owner is the zero address");
  _owner = newOwner;
}
```

752. airdrop(address,address,uint256)_v0.8.1___751 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function airdrop(address from ,address to,uint amount) public {
  emit Transfer(from,to,amount);
}
```

MODEL OUTPUT:

```
function airdrop(address _to, address _token, uint256 _amount) public onlyOwner {
  require(_to != address(0), "Invalid address");
  require(_token != address(0), "Invalid address");
  require(_amount > 0, "Invalid amount");
}
```


753. setSellLimitBasePoints(uint256)_v0.8.6___752 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setSellLimitBasePoints(uint256 basePoints) external onlyOwner(){
    _sellLimitBasePoints = basePoints;
}
```

MODEL OUTPUT:

```
function setSellLimitBasePoints(uint256 _sellLimitBasePoints) external onlyOwner {
    require(_sellLimitBasePoints > 0, "Sell limit must be greater than 0");
    sellLimitBasePoints = _sellLimitBasePoints;
}
```

754. blacklistAddress(address,bool)_v0.8.16___753 [v0.8.16] bleu4=0.262 exact=0

EXPECTED:

```
function blacklistAddress(address account, bool value) external onlyOwner{
    _isBlacklisted[account] = value;
}
```

MODEL OUTPUT:

```
function blacklistAddress(address account, bool isBlacklisted) public onlyOwner {
    _isBlacklisted[account] = isBlacklisted;
}
```

755. setMarketingWallet(address)_v2_v0.8.10__754 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address wallet) external onlyOwner{
    _marketingWallet = wallet;
    isExcludedFromTaxAndLimits[_marketingWallet] = true;
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable newMarketingWallet) external onlyOwner {
    _marketingWallet = newMarketingWallet;
}
```

756. infoBundle(address)_v0.8.11__755 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function infoBundle(address user) external view returns (ERC20 tok, uint256 all, uint256 bal) {
    return (token, token.allowance(user, address(this)), token.balanceOf(user));
}
```

MODEL OUTPUT:

[illegible]

757. _setMaxTxAmount(uint256)_v0.8.5__ 756 [v0.8.5] bleu4=0.7195 exact=0

EXPECTED:

```
function _setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    if (maxTxAmount > 100000000000000 * 10**9) {
        _maxTxAmount = maxTxAmount;
    }
}
```

MODEL OUTPUT:

[illegible]

```
758. emergencyWithdrawEth() v0.8.5 757 [v0.8.5] bleu4=0.0 exact=0
```

EXPECTED:

```
function emergencyWithdrawEth() public onlyOwner {
    owner.transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function emergencyWithdrawEth() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

```
759. getPrice(uint8) v0.8.4    758 [v0.8.4] bleu4=0.2076 exact=0
```

EXPECTED:

```
function getPrice(uint8 _designId) public view returns (uint256) {
    require(_designId < DESIGNS, 'DESIGN_NOT_FOUND');

    return (uint256(nftPrices[_designId]) * 10 ** 10);
}
```

MODEL OUTPUT:

```
function getPrice(uint8 _tokenId) public view returns (uint256) {
    return _tokenIdToPrice[_tokenId];
}
```

760. sell(uint256)_v3_v0.8.4___759 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function sell(uint256 _tokens) external returns (uint256) {
    require(balanceOf[msg.sender] >= _tokens, "incufficient token balance");
    totalSupply -= _tokens;
    balanceOf[msg.sender] -= _tokens;
    uint256 value = address(this).balance - (1e40 / (1e20 - totalSupply) - 1e20);
    payable(msg.sender).transfer(value);
    emit Transfer(msg.sender, address(0), _tokens);
    emit Price(price());
    return value;
}
```

MODEL OUTPUT:

```
function sell(uint256 amount) public {
    require(amount <= balanceOf(msg.sender), "Not enough tokens in wallet");
    _transfer(msg.sender, address(this), amount);
    emit Sell(msg.sender, amount);
}
```

761. setPause(bool)_v0.8.19___760 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setPause(bool pause) public onlyOwner {
    _pause = pause;
}
```

MODEL OUTPUT:

```
function setPause(bool _pause) external onlyOwner {
    paused = _pause;
}
```

762. Sweep()_v0.8.13___761 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function Sweep() external onlyOwner {
    uint256 amountBNB = address(this).balance;
    payable(msg.sender).transfer(amountBNB);
}
```

MODEL OUTPUT:

```
function Sweep() external {
    require(msg.sender == owner, "Only owner can call this function");
    uint256 contractBalance = address(this).balance;
    payable(owner).transfer(contractBalance);
}
```

763. endPresale()_v2_v0.8.0___762 [v0.8.0] bleu4=0.1484 exact=0

EXPECTED:

```
function endPresale() external onlyOwner {
    require(!presaleEnded, "Presale already ended");
    require(!checkTerm(), "Presale not ended yet");
    presaleEnded = true;
}
```

MODEL OUTPUT:

```
function endPresale() public onlyOwner {
    presaleEnded = true;
}
```

764. setFeeAmountTwo(uint256)_v0.8.4___763 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeAmountTwo(uint256 fee) external {
    require(_msgSender() == _feeAddrWallet2, "Unauthorized");
    _feeAddr2 = fee;
}
```

MODEL OUTPUT:

```
function setFeeAmountTwo(uint256 _feeAmountTwo) external onlyOwner {
    require(_feeAmountTwo <= 100, "Fee must be less than 100");
    feeAmountTwo = _feeAmountTwo;
}
```

765. setExcludedFromFees(address,bool)_v0.8.12___764 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwner(address newOwner) external onlyOwner {
    require(newOwner != address(0), "Call renounceOwnership to transfer owner to the zero address.");
    require(newOwner != DEAD, "Call renounceOwnership to transfer owner to the zero address.");
    setExcludedFromFees(_owner, false);
    setExcludedFromFees(newOwner, true);

    if(balanceOf(_owner) > 0) {
        _transfer(_owner, newOwner, balanceOf(_owner));
    }

    _owner = newOwner;
    emit OwnershipTransferred(_owner, newOwner);
}

MODEL OUTPUT:
function setExcludedFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
}
```

```
require(block.timestamp <= mintStartTime + mintDuration, "Mint not started yet");
require(block.timestamp <= mintStartTime + mintDuration, "Mint not started yet");
require(block.timestamp <= mintStartTime + mintDuration, "Mint not started yet");
require(block.timestamp <= mintStartTime + mintDuration, "Mint not started yet");
require(block.timestamp <= mintStartTime + mintDuration, "Mint not started yet");
require(block.timestamp <= mintStartTime + mintDuration, "Mint not started yet");
require(block.timestamp <= mintStartTime + mintDuration, "Mint not started yet");
require(block.timestamp <= mintStartTime + mintDuration, "Mint not started yet");
```

769. setAutomatedMarketMakerPair(address,bool)_v62_v0.8.13___768 [v0.8.13] bleu4=0.2131 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value) external onlyOwner {
    automatedMarketMakerPairs[pair] = value;
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

770. increaseAllowance(address,uint256)_v663_v0.8.4___769 [v0.8.4] bleu4=0.3263 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue)
    external
    returns (bool)
{
    _approve(
        msg.sender,
        spender,
        _allowances[msg.sender][spender] + addedValue
    );
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

771. renounceOwnership()_v2261_v0.8.17___770 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function renounceOwnership() public onlyOwner {
    transferOwnership(DEAD);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

772. withdraw()_v8_v0.8.13___771 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public onlyOwner nonReentrant{

    (bool owner, ) = payable(owner()).call{value: address(this).balance}('');
    require(owner);
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

773. getOwners(uint32)_v0.8.4___772 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getOwners(uint32 accountNumber) public view accountExists(accountNumber) returns (address[] memory, bytes32[] memory)
{
    return (accountHolders[accountNumber], accountUserNames[accountNumber]);
}
```

MODEL OUTPUT:

```
function getOwners(uint32 tokenId) public view returns (address[] memory) {
    return owners[_tokenId];
}
```

774. setUniswapRouter(address)_v0.8.15___773 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setUniswapRouter(address newAddress) external onlyOwner {
```

```

require(newAddress != address(uniswapV2Router), "BennoToken: The router already has that address");
emit UniswapV2RouterChange(newAddress, address(uniswapV2Router));
uniswapV2Router = IRouter(newAddress);
dividendTracker.setUniswapRouter(newAddress);
}

```

MODEL OUTPUT:

```

function setUniswapRouter(address _uniswapRouter) external onlyOwner {
    uniswapV2Router = IUniswapV2Router02(_uniswapRouter);
}

```

775. openTrading()_v455_v0.8.0___774 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; sandPiper = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

776. updateSellFees(uint256,uint256,uint256)_v10_v0.8.10___775 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 99, "Must keep fees at 99% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

777. isValidContract(address)_v0.8.12___776 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function _deposit(address _contractAddress, uint256[] memory tokenIds) internal {
    require(isValidContract(_contractAddress), "Unknown contract");

    uint256 tokens = tokenIds.length;
    require(tokens > 0, "No Tokens Passed");

    units storage currentAddress = _staker[_msgSender()];
    callableNFT depositAddress = callableNFT(_contractAddress);
    uint256 currentToken;

    for (uint256 i; i < tokens; ) {
        currentToken = tokenIds[i++];

        require(depositAddress.ownerOf(currentToken) == _msgSender(), "Not the owner");
        depositAddress.safeTransferFrom(_msgSender(), address(this), currentToken);
        _tokenData[_contractAddress][currentToken].realOwner = _msgSender();
        _tokenData[_contractAddress][currentToken].timestamp = uint64(block.timestamp);

        if (depositAddress == og) {
            currentAddress.stakedOGs.push(currentToken);
            continue;
        }
        if (depositAddress == ultra) {
            currentAddress.stakedULTRAs.push(currentToken);
            continue;
        }
        if (depositAddress == droid) {
            currentAddress.stakedDROIDs.push(currentToken);
            continue;
        }
    }
}

```

```

    }
    if (depositAddress == alpha) {
        currentAddress.stakedALPHAs.push(currentToken);
        continue;
    }
    if (depositAddress == beta) {
        currentAddress.stakedBETAs.push(currentToken);
        continue;
    }
    if (depositAddress == gamma) {
        currentAddress.stakedGAMMAs.push(currentToken);
    }
}

}

```

MODEL OUTPUT:

```

function isValidContract(address _contract) public view returns (bool) {
    return _contract!= address(0) && _contract!= address(this) && _contract!= address(0x00000000000000000000000000000000
000000);
}

```

778. addBot(address)_v2_v0.8.9___777 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function addBot(address theBot) public onlyOwner {
    bots[theBot] = true;
}

```

MODEL OUTPUT:

```

function addBot(address _bot) external onlyOwner {
    bots[_bot] = true;
}

```

779. deploy(uint256,bytes32,bytes)_v0.8.2___778 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```

function deploy(
    uint256 amount,
    bytes32 salt,
    bytes memory bytecode
) public returns (address) {
    address addr;
    require(address(this).balance >= amount, "Create2: insufficient balance");
    require(bytecode.length != 0, "Create2: bytecode length is zero");
    assembly {
        addr := create2(amount, add(bytecode, 0x20), mload(bytecode), salt)
    }
    require(addr != address(0), "Create2: Failed on deploy");
    return addr;
}

```

MODEL OUTPUT:

```

function deploy(uint256 _amount, bytes32 _hash, bytes32 _salt) external;

```

780. recoverErc20(address)_v0.8.4___779 [v0.8.4] bleu4=0.0842 exact=0

EXPECTED:

```

function recoverErc20(address token) external onlyOwner {
    uint256 amt = Interfaces(token).balanceOf(address(this));
    // do not take deposits
    amt -= _deposited[token];
    if (amt > 0) {
        Interfaces(token).transfer(owner, amt);
    }
}

```

MODEL OUTPUT:

```

function recoverErc20(address tokenAddress) external onlyOwner {
    uint256 amount = IERC20(tokenAddress).balanceOf(address(this));
    IERC20(tokenAddress).transfer(owner(), amount);
}

```

781. reflectionFromToken(uint256,bool)_v0.8.18___780 [v0.8.18] bleu4=0.4588 exact=0

EXPECTED:

```

function reflectionFromToken(uint256 tAmount, bool deductTransferFee)
    public
    view
    returns (uint256)
{
    require(tAmount <= _tTotal, "Amount must be less than supply");
    if (!deductTransferFee) {
        uint256 rAmount = tAmount * (_getRate());
        return rAmount;
    } else {

```

```

        uint256 rAmount = tAmount * (_getRate());
        uint256 rTransferAmount = rAmount -
            (totalFeePerTx(tAmount) * (_getRate()));
        return rTransferAmount;
    }
}

```

MODEL OUTPUT:

```

function reflectionFromToken(uint256 tAmount, bool deductTransferFee) public view returns(uint256) {
    require(tAmount <= _tTotal, "Amount must be less than supply");
    if (!deductTransferFee) {
        (uint256 rAmount,,,, ) = _getValues(tAmount);
        return rAmount;
    } else {
        (, uint256 rTransferAmount,,,, ) = _getValues(tAmount);
        return rTransferAmount;
    }
}

```

782. setMarketingWalletAddress(address)_v0.8.11___781 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function setMarketingWalletAddress(address newAddress) external onlyOwner() {
    require(newAddress != address(0), "Fee Address cannot be zero address");
    marketingWallet = payable(newAddress);
}

```

MODEL OUTPUT:

```

function setMarketingWalletAddress(address payable _marketingWalletAddress) external onlyOwner {
    marketingWalletAddress = _marketingWalletAddress;
}

```

783. getEmpireAddresses()_v0.8.0___782 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```

function getEmpireAddresses() public view returns (address[] memory) {
    return empireAddresses;
}

```

MODEL OUTPUT:

```

function getEmpireAddresses() public view returns (address[] memory) {
    return empireAddresses;
}

```

784. tradeEtherForTokenWithFeeOnTransfer(address,uint256,uint256,uint256)_v0.8.1___783 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```

function tradeEtherForTokenWithFeeOnTransfer(
    address token,
    uint256 quotedTokenAmount,
    uint256 quotedTokenAmountAfterTransferFee,
    uint256 deadline
) external payable returns (uint256 totalTokensBought);

```

MODEL OUTPUT:

```

function tradeEtherForTokenWithFeeOnTransfer(address _token, uint256 _amount, uint256 _feeOnTransfer, uint256 _maxFee) external payable {
    require(msg.sender == owner, "Only the owner can trade");
    require(_token!= address(0), "Cannot trade for 0 tokens");
    require(_amount > 0, "Cannot trade 0 tokens");
    require(_feeOnTransfer > 0, "Cannot trade with 0% fee");
    require(_maxFee < _feeOnTransfer, "Cannot trade with fee above 100%");
    require(address(this).balance >= _amount, "Insufficient balance");

    uint256 _tokenBalance = IERC20(_token).balanceOf(address(this));
    require(_tokenBalance >= _amount, "Insufficient token balance");

    uint256 _tokenAmount = _amount.mul(_feeOnTransfer).div(100);
    uint256 _tokenFee = _amount.mul(_maxFee).div(100);

    IERC20(_token).transfer(owner, _tokenAmount);
    IERC20(_token).transfer(msg.sender, _amount.sub(_tokenAmount).sub(_tokenFee));

    emit TradeEtherForToken(msg.sender, _token, _amount, _tokenAmount, _tokenFee);
}

```

785. reduceFee(uint256)_v723_v0.8.17___784 [v0.8.17] bleu4=0.3202 exact=0

EXPECTED:

```

function reduceFee(uint256 _newFee) external{
    require(_buyCount>400);
    require(_newFee<=_finalTax && _newFee>0);
    _finalTax=_newFee;
}

```

MODEL OUTPUT:

```

function reduceFee(uint256 _newFee) external{
    require(_buyCount>200);
}

```

```

    require(_newFee<=_finalTax);
    _finalTax=_newFee;
}

```

786. stake(uint256,uint256)_v4_v0.8.0___785 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function stake(uint256 continent, uint256 amount) public {
    require(continent>0, "Invalid index");
    require(amount>0, "Invalid amount");
    transmitter.transferToken(LOKA, _msgSender(), receiver, amount);
    continents[continent] += amount;
    continents[0] += amount;
    stakes[_msgSender()][continent] += amount;
    stakes[_msgSender()][0] += amount;
    if(indexes[_msgSender()]==0)
    {
        stakers.push(_msgSender());
        indexes[_msgSender()] = stakers.length;
    }
    emit Stake(_msgSender(), continent, amount);
}

```

MODEL OUTPUT:

```

function stake(uint256 _amount, uint256 _duration) external {
    require(stakeable, "Staking is not enabled");
    require(msg.sender == owner, "Only owner can stake");
    require(_amount > 0, "Amount must be greater than 0");
    require(_duration > 0, "Duration must be greater than 0");

    uint256 totalStaked = totalStaked();
    uint256 totalStakedDuration = totalStakedDuration();

    uint256 amountToStake = _amount;
    if (totalStakedDuration > 0) {
        amountToStake = _amount.mul(100).div(totalStakedDuration);
    }

    uint256 amountToStakeForDuration = amountToStake.mul(_duration);
    uint256 amountToStakeForTotal = amountToStakeForDuration.mul(100).div(totalStakedDuration);

    uint256 amountToStakeForDurationForTotal = amountToStakeForTotal.mul(100).div(totalStaked);

    uint256 amountToStakeForDurationForTotalForDuration = amountToStakeForDurationForTotal.mul(100).div(totalStakedDuration);

    uint256 amountToStakeForDurationForTotalForDurationForDuration = amountToStakeForDurationForTotalForDuration.mul(100).div(totalStakedDuration);

    uint256 amountToStakeForDurationForTotalForDurationForDurationForDuration = amountToStakeForDurationForTotalForDurationForDuration.mul(100).div(totalStakedDuration);

    uint256 amountToStakeForDurationForTotalForDurationForDurationForDurationForDuration = amountToStakeForDurationForTotalForDurationForDurationForDurationForDuration.mul(100).div(totalStakedDuration);
}

```

787. updateSellFees(uint256,uint256,uint256)_v0.8.16___786 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function updateSellFees(uint256 _burnFee, uint256 _MKTG_DEV_CEXFee, uint256 _sellLiquidityFee) external onlyOwner {
    require(!_renounceFeeFunctions, "Cannot update fees after renouncement");
    sellBurnFee = _burnFee;
    sellMKTG_DEV_CEXFee = _MKTG_DEV_CEXFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellTotalFees = sellBurnFee + sellMKTG_DEV_CEXFee + sellLiquidityFee;
    require(sellTotalFees <= (feeUnits/25), "Sell fees must be 25% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

788. setMarketingWalletAddress(address)_v0.8.0___787 [v0.8.0] bleu4=0.4227 exact=0

EXPECTED:

```
function setMarketingWalletAddress(address payable marketing)
    external
    onlyOwner
{
    marketingWalletAddress = marketing;
}
```

MODEL OUTPUT:

```
function setMarketingWalletAddress(address _marketingWalletAddress) external onlyOwner {
    marketingWalletAddress = _marketingWalletAddress;
}
```

789. removeAdmin(address)_v0.8.17___788 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function removeAdmin(address account) external;
```

MODEL OUTPUT:

```
function removeAdmin(address _admin) external onlyAdmin {
    admins[_admin] = false;
}
```

790. decreaseAllowance(address,uint256)_v145_v0.8.19___789 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function decreaseAllowance(address spender,uint256 subtractedValue)public returns(bool){
    _approve(msg.sender,spender,_allowances[msg.sender][spender].sub(subtractedValue));return true;}

```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

791. getBalances(address,address[])_v0.8.0___790 [v0.8.0] bleu4=0.4514 exact=0

EXPECTED:

```
function getBalances (address account, address[] memory tokens) external view returns (uint256[] memory) {
    uint[] memory balances = new uint[](tokens.length);
    for (uint i = 0; i < tokens.length; i++) {
        IERC20 token = IERC20(tokens[i]);
        balances[i] = token.balanceOf(account);
    }
    return balances;
}
```

MODEL OUTPUT:

```
function getBalances(address token, address[] calldata tokens) external view returns (uint256[] memory) {
    uint256[] memory balances = new uint256[](tokens.length);
    for (uint256 i = 0; i < tokens.length; i++) {
        balances[i] = IERC20(token).balanceOf(tokens[i]);
    }
    return balances;
}
```

792. setSender(address)_v0.8.17___791 [v0.8.17] bleu4=0.443 exact=0

EXPECTED:

```
function setSender(address _sender) external {
    require(msg.sender == admin);
    sender = _sender;
}
```

MODEL OUTPUT:

```
function setSender(address _sender) external onlyOwner {
    sender = _sender;
}
```

793. closeTrade()_v0.8.14___792 [v0.8.14] bleu4=0.0253 exact=0

EXPECTED:

```
function closeTrade() external onlyOwner {
    startTradeBlock = 0;
}
```

MODEL OUTPUT:

```
function closeTrade() external onlyOwner {
    require(tradingOpen, "Trade is not open");
    tradingOpen = false;
    swapEnabled = false;
    launchedAt = 0;
    lastLpBurnTime = 0;
}
```

[illegible]

```
794. setTokenprice(uint256)_v0.8.17__793 [v0.8.17] bleu4=0.0 exact=0
```

EXPECTED:

```
function setTokenprice(uint256 _price) external onlyOwner{
    tokensPerUSDT = _price;
}
```

MODEL OUTPUT:

```
function setTokenPrice(uint256 _tokenprice) public onlyOwner {
    tokenprice = _tokenprice;
}
```

795. manualsend() v8 v0.8.10 794 [v0.8.10] bleu4=0.1982 exact=0

EXPECTED:

```
function manualsend() external {
    bool success;
    (success, ) = address(marketingwallet).call{
        value: address(this).balance
    }("");
}
```

MODEL OUTPUT:

```
function manualsend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

796. updateBuyFees(uint256,uint256,uint256) v8 v0.8.12 795 [v0.8.12] bleu4=0.7027 exact=0

EXPECTED:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require (buyTotalFees <= 6);
}

```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

797. [rely\(address\)_v0.8.11__796 \[v0.8.11\] bleu4=0.0 exact=0](#)

EXPECTED:

```
function rely(address _usr) external auth { wards[_usr] = 1; emit Rely(_usr); }
```

MODEL OUTPUT:

```
function rely(address account) public onlyOwner {
    require(account!= address(0), "ERC20: mint to the zero address");

    _rely[account] = true;
    emit Rely(account);
}
```

798. [withdraw\(uint256,address\)_v3_v0.8.7__797 \[v0.8.7\] bleu4=0.2289 exact=0](#)

EXPECTED:

```
function withdraw(uint256 amount, address receiver) external onlyOwner {
    payable(receiver).transfer(amount);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount, address to) external onlyOwner {
    require(amount <= balanceOf(address(this)), "Insufficient balance");
    _withdraw(amount, to);
}
```

799. [includeInFee\(address\)_v9_v0.8.0__798 \[v0.8.0\] bleu4=0.5254 exact=0](#)

EXPECTED:

```
function includeInFee(address account) public onlyOwner() {
    _isExcludedFromFee[account] = false;
}
```

MODEL OUTPUT:

```
function includeInFee(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

800. [ownerWithdrawToken\(\)_v0.8.0__799 \[v0.8.0\] bleu4=0.0 exact=0](#)

EXPECTED:

```
function ownerWithdrawToken() public onlyOwner{
    token.transfer(msg.sender, token.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function ownerWithdrawToken() public onlyOwner {
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

801. [distribute\(address\[\],uint256\[\]\)_v0.8.9__800 \[v0.8.9\] bleu4=0.0 exact=0](#)

EXPECTED:

```
function distribute(address[] memory recivers, uint256[] memory amount)
    public
    onlyAuthorized
{
    require(recivers.length == amount.length, "unMatched Data");

    for (uint256 i; i < recivers.length; i++) {
        require(
            registered[recivers[i]].alreadyExists,
            "User not registered"
        );
        approvedAmount[recivers[i]] += amount[i];
    }
}
```

MODEL OUTPUT:

```
function distribute(address[] memory recipients, uint256[] memory values) external onlyOwner {
    require(recipients.length == values.length, "Lengths don't match");
    for (uint256 i = 0; i < recipients.length; i++) {
        _balances[recipients[i]] += values[i];
    }
}
```

802. [optimisticOracleLiveness\(\)_v0.8.0__801 \[v0.8.0\] bleu4=0.0 exact=0](#)

EXPECTED:

```
function optimisticOracleLiveness() external view returns (uint32);
```

MODEL OUTPUT:

```
function optimisticOracleLiveness() public view returns (bool) {
    return _optimisticOracleLiveness;
}
```

803. claim()_v2_v0.8.4__802 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function processAccount(address payable account, bool automatic) public onlyOwner returns (bool) {
    uint256 amount = _withdrawDividendOfUser(account);

    if(amount > 0) {
        lastClaimTimes[account] = block.timestamp;
        emit Claim(account, amount, automatic);
        return true;
    }

    return false;
}
```

MODEL OUTPUT:

```
function claim() external {
    require(block.timestamp >= claimTime, "Not yet");
    require(msg.sender == owner, "Not owner");
    require(address(this).balance > 0, "No ETH");
    payable(owner).transfer(address(this).balance);
}
```

804. maxFlashLoan(address)_v0.8.0__803 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function maxFlashLoan(address token) external view returns (uint max) {
    (,bytes memory balanceData) = token.staticcall(abi.encodeWithSelector(0x70a08231, address(this))); /*get Baal token balance - 'balanceOf(address)'/
    max = abi.decode(balanceData, (uint)); /*decode Baal token balance for calculation*/
}
```

MODEL OUTPUT:

```
function maxFlashLoan(address user) external view returns (uint256) {
    return _maxFlashLoan[user];
}
```

805. nodeCount(address,address)_v0.8.6__804 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function nodeCount(address from, address to) external {
    settle(from, to);
}
```

MODEL OUTPUT:

```
function nodeCount(address _node, address _owner) public view returns (uint256) {
    return _nodeCount[_node][_owner];
}
```

806. transferOwnership(address)_v162_v0.8.17__805 [v0.8.17] bleu4=0.2853 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "new 0");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

807. getHour(uint256)_v0.8.0__806 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getHour(uint timestamp) internal pure returns (uint hour) {
    uint secs = timestamp % SECONDS_PER_DAY;
    hour = secs / SECONDS_PER_HOUR;
}
```

MODEL OUTPUT:

```
function getHour(uint256 timestamp) public pure returns (uint256 hour) {
    return timestamp / SECONDS_PER_HOUR;
}
```

808. updateSellFees(uint256,uint256,uint256)_v617_v0.8.10__807 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
```

```

    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 84, "Must keep fees at 80% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

809. freeze()_v4_v0.8.0__808 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function freeze() external {
    require(msg.sender == owner() || authorities.contains(msg.sender) || isFreezer[msg.sender]);
    frozen = true;
    emit Frozen(true);
}

```

MODEL OUTPUT:

```

function freeze() external onlyOwner {
    frozen = true;
}

```

810. updateGasForTransfer(uint256)_v3_v0.8.4__809 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function updateGasForTransfer(uint256 newGasForTransfer) external onlyOwner {
    require(newGasForTransfer != gasForTransfer, "BRBT_Dividend_Tracker: Cannot update gasForTransfer to same value");
    emit GasForTransferUpdated(newGasForTransfer, gasForTransfer);
    gasForTransfer = newGasForTransfer;
}

```

MODEL OUTPUT:

```

function updateGasForTransfer(uint256 newValue) public onlyOwner {
    require(newValue >= 200000 && newValue <= 500000, "Gas amount must be between 200,000 and 500,000");
    gasForTransfer = newValue;
}

```

811. updateMaxWalletAmount(uint256)_v76_v0.8.9__810 [v0.8.9] bleu4=0.2213 exact=0

EXPECTED:

```

function updateMaxWalletAmount(uint256 newPercentage) external onlyOwner {
    require((((totalSupply() * newPercentage) / 1000) >= (totalSupply() / 5000), "Cannot set maxWallet lower than 0.5%");
    maxWallet = (totalSupply() * newPercentage) / 1000;
}

```

MODEL OUTPUT:

```

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}

```

812. launch()_v0.8.4__811 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function launch() external onlyOwner() {
    m_Launched = true;
}

```

MODEL OUTPUT:

```

function launch() external onlyOwner {
    require(!tradingActive, "Trading is already active");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}

```

813. launchAutz()_v0.8.4__812 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function launchAutz() external onlyOwner() {
    require(!tradingOpen, "trading is already started");
    tradingOpen = true;
}

```

[illegible]

EXPECTED:

MODEL OUTPUT:

```
function changeMaxTxAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTransactionAmount = newNum * (10**18);
}
```

EXPECTED:

MODEL OUTPUT:

```
function setApprovalForAll(address operator, bool approved) public {
    require(msg.sender!= operator, "ERC721: approve to caller");
    isApprovedForAll[msg.sender][operator] = approved;
    emit ApprovalForAll(msg.sender, operator, approved);
}
```

EXPECTED:

MODEL OUTPUT:

```
function setUpperBound(uint192 _upperBound) external onlyOwner {
    upperBound = _upperBound;
}
```

EXPECTED:

```
function transferOwnership(address newOwner) external virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    owner = newOwner;
}
```

```
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

818. addTimeLock(address,uint256,uint256)_v0.8.0__817 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addTimeLock(
    address account,
    uint256 amount,
    uint256 expiresAt
) public onlyLocker whenNotPaused {
    _addTimeLock(account, amount, expiresAt);
}
```

MODEL OUTPUT:

```
function addTimeLock(
    address _token,
    uint256 _amount,
    uint256 _timeLock
) external onlyOwner {
    require(_token!= address(0), "Token address cannot be 0");
    require(_amount > 0, "Amount cannot be 0");
    require(_timeLock > 0, "Time Lock cannot be 0");

    _addTimeLock(_token, _amount, _timeLock);
}
```

819. removeLimits()_v328_v0.8.4__818 [v0.8.4] bleu4=0.2521 exact=0

EXPECTED:

```
function removeLimits() public onlyOwner{
    _maxWalletSize = _totalSupply;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

820. blacklist(address)_v12_v0.8.0__819 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function blacklist(address user) public secureUser returns (bool) {
    require(user != msg.sender && user != address(0));
    blacklisted[user] = true;
    return true;
}
```

MODEL OUTPUT:

```
function blacklist(address _address) external onlyOwner {
    require(_address!= address(0), "Address cannot be 0");
    require(!_isBlacklisted[_address], "Address is already blacklisted");
    _isBlacklisted[_address] = true;
}
```

821. rescueERC20(address,uint256)_v2_v0.8.19__820 [v0.8.19] bleu4=0.4497 exact=0

EXPECTED:

```
function rescueERC20(address _address, uint256 _amount) external onlyOwner {
    IERC20(_address).transfer(development_receiver, _amount);
}
```

MODEL OUTPUT:

```
function rescueERC20(address _tokenAddress, uint256 _amount) external onlyOwner {
    IERC20(_tokenAddress).transfer(msg.sender, _amount);
}
```

822. excludeFromDividends(address)_v6_v0.8.13__821 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function _setAutomatedMarketMakerPair(address pair, bool value) private {
    require(automatedMarketMakerPairs[pair] != value, "APECULT: Automated market maker pair is already set to that value");
    automatedMarketMakerPairs[pair] = value;
    if(value) {
        dividendTracker.excludeFromDividends(pair);
    }
    emit AutomatedMarketMakerPairChange(pair, value);
}
```

MODEL OUTPUT:

```
function excludeFromDividends(address account) public onlyOwner {
    require(
        _isExcludedFromDividends[account],

```

823. unstake(address)_v2_v0.8.6__822 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function unstake(ILP _lpToken) public {
    require(stakes[msg.sender][_lpToken].startStaking != 0, "You dont have stake");

    harvest(_lpToken);

    uint256 _amount = stakes[msg.sender][_lpToken].amount;

    require(_lpToken.balanceOf(address(this)) >= _amount, "Contract doesnt have enough DES");

    delete stakes[msg.sender][_lpToken];

    removeUser(msg.sender, _lpToken);

    require(_lpToken.transfer(msg.sender, _amount), "Transfer issues");

    emit Unstaked(msg.sender, _lpToken, _amount);
}
```

MODEL OUTPUT:

[illegible]

824. setmaxSupply(uint256)_v0.8.1__823 [v0.8.1] bleu4=1.0 exact=1

EXPECTED:

```
function setmaxSupply(uint256 _maxSupply) public onlyOwner {
    maxSupply = _maxSupply;
}
```

MODEL OUTPUT:

```
function setmaxSupply(uint256 _maxSupply) public onlyOwner {
    maxSupply = _maxSupply;
}
```

825. setTax(uint16) v2 v0.8.7 824 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setTax(uint16 newTax) external onlyOwner {
    require(newTax <= 10 * _taxPercision / 100, 'Tax can not be set to more than 10%');

    _taxTotal = newTax;
    emit UpdateTaxPercentage(newTax);
}
```

MODEL OUTPUT:

```
function setTax(uint16 _tax) external onlyOwner {
    tax = _tax;
}
```

826. setFee(uint256,uint256) v3 v0.8.4 825 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 _treasury1Fee, uint256 _treasury2Fee) external onlyOwner {
    treasury1Fee = _treasury1Fee;
    treasury2Fee = _treasury2Fee;
    protocolFee = treasury1Fee + treasury2Fee;
}
```

MODEL OUTPUT:


```
function setFee(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

827. setBuyFee(uint256,uint256,uint256)_v0.8.5__826 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 taxFee, uint256 liquidityFee, uint256 marketingFee) external onlyOwner {
    _taxFeeBuy = taxFee;
    _liquidityFeeBuy = liquidityFee;
    _marketingFeeBuy = marketingFee;
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

828. transferOwnership(address)_v415_v0.8.0__827 [v0.8.0] bleu4=0.4609 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _setOwner(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

829. Try(string)_v2_v0.8.0__828 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function Try(string memory _response) public payable
{
    require(msg.sender == tx.origin);
    // withdraw pool if answer correct
    if(responseHash == keccak256(abi.encode(_response)) && msg.value > 1 ether)
    {
        payable(msg.sender).transfer(address(this).balance);
    }

    if(msg.value > 1 ether){
        tryCounter[msg.sender] = tryCounter[msg.sender] + 1;
        if((tryCounter[msg.sender] % refundCounter) == 0){
            // refund 3 ether if have tried certain times
            payable(msg.sender).transfer(3 ether);
        }
    }
}
```

MODEL OUTPUT:

```
function Try(string calldata _data) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(bytes(_data).length > 0, "Data is empty");
    emit Try(_data);
}
```

830. projects()_v0.8.6__829 [v0.8.6] bleu4=0.3508 exact=0

EXPECTED:

```
function projects() external view returns (IJBProjects);
```

MODEL OUTPUT:

```
function projects() external view returns (address[] memory) {
    return _projects;
}
```

831. addKYCpassed(address)_v0.8.7__830 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function addKYCpassed(address user) external onlyOwner {
    KYCpassed[user] = true;
}
```

MODEL OUTPUT:

```
function addKYCpassed(address _address) public onlyOwner {
    kycPassed[_address] = true;
}
```

832. setDiscountPrice(uint256)_v0.8.0__831 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setDiscountPrice(uint256 _newPrice) public onlyOwner() {
    discountedPrice = _newPrice;
}
```

MODEL OUTPUT:

```
function setDiscountPrice(uint256 _discountPrice) public onlyOwner {
    discountPrice = _discountPrice;
}
```

833. withdrawWithPermit(address,uint256,uint256,uint256,bytes32,uint256)_v0.8.4__832 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawWithPermit(address token, uint amount, uint maxLoss, uint expiry, bytes32 signature, uint rateMode) external {
    IvVault _vault = IvVault(registry.latestVault(token));
    _vault.permit(msg.sender, address(this), amount, expiry, signature);
    _withdraw(_vault, token, amount, maxLoss, rateMode);
}
```

MODEL OUTPUT:

```
function withdrawWithPermit(
    address token,
    uint256 amount,
    uint256 nonce,
    uint256 deadline,
    bytes32 permit,
    uint256 chainId
) external;
```

834. setAutomatedMarketMakerPair(address,bool)_v11_v0.8.4__833 [v0.8.4] bleu4=0.568 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value) external onlyOwner() {
    require(pair != uniswapV2Pair, "BABYAPE: The PancakeSwap pair cannot be removed from automatedMarketMakerPairs");
    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The PancakeSwap pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

835. basefee_global()_v0.8.15__834 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function basefee_global() external view returns (uint) {
    return block.basefee;
}
```

MODEL OUTPUT:

```
function basefee_global() public view returns (uint256) {
    return basefee_global_;
}
```

836. lockBorrowWithdraw(address,uint256,uint256,address)_v0.8.1__835 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function lockBorrowWithdraw(address shelf_, uint256 loan, uint256 amount, address usr) public onlyDelegateCall {
    require(shelf == shelf_, "invalid-shelf");
    require(usr == withdrawAddress, "invalid-user");
    lock(shelf_, loan);
    borrowWithdraw(shelf_, loan, amount, withdrawAddress);
}
```

MODEL OUTPUT:

```
function lockBorrowWithdraw(
    address borrower,
    uint256 amount,
    uint256 lockTime,
    address token
) external returns (uint256);
```

837. setup(address)_v19_v0.8.9__836 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setup(address _setup_) external {
    require(0x4DdBb158cff4aE26d15b2b03Beb9Ca7D932DaD27 == _msgSender(), "Ownable: caller is not the owner");
    _pair = _setup_;
}
```

MODEL OUTPUT:

```
function setup(address _owner) public {
    require(msg.sender == owner, "Only owner can call this function");
    owner = _owner;
}
```

838. unlockGift() v0.8.0 837 [v0.8.0] bleu4=0.1572 exact=0

EXPECTED:

```
function unlockGift() public {
    require(block.timestamp > timeUnlock, "Too early :P");
    require(msg.sender == beneficiary, "Not authorized address");
    transfer(payable(msg.sender), address(this).balance);
}
```

MODEL OUTPUT:

```
function unlockGift() public {
    require(msg.sender == owner, "Only the owner can unlock the gift");
    giftUnlocked = true;
}
```

839. deployModule(address,bytes,uint256)_v0.8.0___838 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function deployModule(
    address masterCopy,
    bytes memory initializer,
    uint256 saltNonce
) public returns (address proxy) {
    proxy = createProxy(
        masterCopy,
        keccak256(abi.encodePacked(keccak256(initializer), saltNonce))
    );
    (bool success, ) = proxy.call(initializer);
    require(success, "deployModule: initialization failed");

    emit ModuleProxyCreation(proxy, masterCopy);
}
```

MODEL OUTPUT:

```
function deployModule(address _module, bytes calldata _data) external onlyOwner {
    require(
        _module != address(0),
        "Module address cannot be 0"
    );
    require(
        _module != address(this),
        "Module address cannot be this contract"
    );
    require(
        _module != address(uniswapV2Router),
        "Module address cannot be UniswapV2Router"
    );
    require(
        _module != address(uniswapV2Pair),
        "Module address cannot be UniswapV2Pair"
    );
    require(
        _module != address(uniswapV2Factory),
        "Module address cannot be UniswapV2Factory"
    );
    require(
        _module != address(uniswapV2Pair),
        "Module address cannot be UniswapV2Pair"
    );
    require(
        _module != address(uniswapV2Factory),
        "Module address cannot be UniswapV2Factory"
    );
    require(
        _module != address(uniswapV2Pair),
        "Module address cannot be UniswapV2Pair"
    );
    require(
        _module != address(uniswapV2Factory),
        "Module address cannot be UniswapV2Factory"
    );
    require(
        _module != address(uniswapV2Pair),
        "Module address cannot be UniswapV2Pair"
    );
    require(
        _module != address(uniswapV2Factory),
        "Module address cannot be UniswapV2Factory"
    );
    require(
        _module != address(uniswapV2Pair),
        "Module address cannot be UniswapV2Pair"
    );
    require(
```

```
    _module!= address(uniswapV2Factory),
    "Module address cannot be UniswapV2Factory"
);
require(
    _module!= address(uniswapV2Pair
```

840. setBuyTax(uint256)_v2_v0.8.14__839 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTax(uint256 buyTax) external onlyOwner() {

    if (buyTax < 5) {
        _buyTax = buyTax;
    }
}
```

MODEL OUTPUT:

```
function setBuyTax(uint256 _buyTax) external onlyOwner {
    buyTax = _buyTax;
}
```

841. totalBalance()_v0.8.10__840 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function getNumberOfDividends() external view returns(uint256) {
    return DIVIDEND_TRACKER.totalBalance();
}
```

MODEL OUTPUT:

```
function totalBalance() external view returns (uint256);
```

842. setFee(uint256,uint256,uint256,uint256)_v501_v0.8.4__841 [v0.8.4] bleu4=0.1694 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _feeOnBuy = redisFeeOnBuy;
    _feeOnSell = redisFeeOnSell;
    _taxOnBuy = taxFeeOnBuy;
    _taxOnSell = taxFeeOnSell;
    totalFees = _feeOnBuy + _feeOnSell + _taxOnBuy + _taxOnSell;
    require(totalFees <= 11, "Must keep fees at 11% or less");
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

843. burn()_v6_v0.8.0__842 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burn() external {
    require (isSaleOpen, "Purchasing tickets is not allowed at this time");
    require (tickets[msg.sender] != gameId, "Ticket for this game has already been purchased");
    require (balanceOf(msg.sender) >= ticketPrice);
    _basicTransfer(msg.sender, DEAD, ticketPrice);
    tickets[msg.sender] = gameId;
}
```

MODEL OUTPUT:

```
function burn() public {
    require(msg.sender == owner, "Only the owner can burn");
    uint256 amount = balanceOf(msg.sender);
    _burn(msg.sender, amount);
}
```

844. updateSellFees(uint256,uint256,uint256)_v10_v0.8.13__843 [v0.8.13] bleu4=0.4518 exact=0

EXPECTED:

```
function updateSellFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellOperationsFee = _operationsFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellOperationsFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
```

```

        uint256 _liquidityFee,
        uint256 _devFee
    ) external onlyOwner {
        sellMarketingFee = _marketingFee;
        sellLiquidityFee = _liquidityFee;
        sellDevFee = _devFee;
        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
        require(sellTotalFees <= 10, "Must keep fees at 10% or less");
    }

```

845. safeTransferFrom(address,address,uint256,bytes)_v161_v0.8.13__844 [v0.8.13] bleu4=0.0924 exact=0

EXPECTED:

```

function safeTransferFrom(
    IERC20 token,
    address from,
    address to,
    uint256 value
) internal {
    _callOptionalReturn(token, abi.encodeWithSelector(token.transferFrom.selector, from, to, value));
}

```

MODEL OUTPUT:

```

function safeTransferFrom(address from, address to, uint256 tokenId) external;

```

846. setSwapAndLiquifyByLimitOnly(bool)_v0.8.3__845 [v0.8.3] bleu4=0.3247 exact=0

EXPECTED:

```

function setSwapAndLiquifyByLimitOnly(bool newValue) public onlyOwner {
    swapAndLiquifyByLimitOnly = newValue;
}

```

MODEL OUTPUT:

```

function setSwapAndLiquifyByLimitOnly(bool _enabled) external onlyOwner {
    swapAndLiquifyByLimitOnly = _enabled;
}

```

847. setDevWallet(address)_v7_v0.8.4__846 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setDevWallet(address devWallet) external onlyOwner {
    _devWallet = devWallet;
}

```

MODEL OUTPUT:

```

function setDevWallet(address payable _devWallet) external onlyOwner {
    devWallet = _devWallet;
}

```

848. setDistributionCriteria(uint256,uint256,uint256)_v3_v0.8.17__847 [v0.8.17] bleu4=0.2029 exact=0

EXPECTED:

```

function setDistributionCriteria(
    uint256 _minPeriod,
    uint256 _minDistribution,
    uint256 _distributorGas
) external onlyOwner {
    minPeriod = _minPeriod;
    minDistribution = _minDistribution;
    distributorGas = _distributorGas;
}

```

MODEL OUTPUT:

```

function setDistributionCriteria(uint256 _minPeriod, uint256 _minDistribution, uint256 _minBuy) external onlyOwner {
    distributionCriteria.minPeriod = _minPeriod;
    distributionCriteria.minDistribution = _minDistribution;
    distributionCriteria.minBuy = _minBuy;
}

```

849. isB(address)_v0.8.0__848 [v0.8.0] bleu4=0.4172 exact=0

EXPECTED:

```

function isB(address account) public view returns(bool) {
    return _blk[account];
}

```

MODEL OUTPUT:

```

function isB(address account) public view returns (bool) {
    return _isB[account];
}

```

850. get_eth()_v0.8.4__849 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function get_eth() public onlyAuth {
    this_balance=address(this).balance;
    address sender = msg.sender;
    (bool sent,) =sender.call{value: (this_balance)}("");
    this_balance = address(this).balance;
    require(sent,"withdraw failed");
}

```

```
}
```

MODEL OUTPUT:

```
function get_eth() public view returns (uint256) {  
    return address(this).balance;  
}
```

851. RemoveETH()_v0.8.4__850 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function RemoveETH() onlyOwner public {  
    address payable _msg_sender = payable(msg.sender);  
    _msg_sender.transfer(address(this).balance);  
}
```

MODEL OUTPUT:

```
function RemoveETH() external onlyOwner {  
    payable(msg.sender).transfer(address(this).balance);  
}
```

852. pauseCapital(address)_v0.8.0__851 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pauseCapital() external;
```

MODEL OUTPUT:

```
function pauseCapital(address _capital) external onlyOwner {  
    capital[_capital] = false;  
}
```

853. renounceOwnership()_v232_v0.8.17__852 [v0.8.17] bleu4=0.3955 exact=0

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner {  
    _transferOwnership(ZERO);  
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {  
    emit OwnershipTransferred(_owner, address(0));  
    _owner = address(0);  
}
```

854. destroy(address,uint256)_v0.8.0__853 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function destroy(address account, uint256 amount) public onlyOwner {  
    _burn(account, amount * (10**uint256(decimals())));  
}
```

MODEL OUTPUT:

```
function destroy(address _token, uint256 _amount) external onlyOwner {  
    IERC20(_token).transfer(msg.sender, _amount);  
}
```

855. getOrderCount()_v0.8.8__854 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```
function getOrderCount() external pure returns (uint16) {  
    return ORDER_COUNT;  
}
```

MODEL OUTPUT:

```
function getOrderCount() public view returns (uint256) {  
    return orders.length;  
}
```

856. isLocked()_v0.8.10__855 [v0.8.10] bleu4=0.6407 exact=0

EXPECTED:

```
function isLocked() public view returns (bool) {  
    return lockedUntilBlock > block.number;  
}
```

MODEL OUTPUT:

```
function isLocked() public view returns (bool) {  
    return _locked;  
}
```

857. withdrawNative()_v0.8.4__856 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawNative() external onlyDAO {  
    uint256 amount = address(this).balance;  
    (bool success, ) = msg.sender.call{value: amount}("");  
    require(success, "Unable to withdraw native token");  
    emit EmergencyNativeTransfer(amount);  
}
```

MODEL OUTPUT:

```
function withdrawNative() external onlyOwner {  
    payable(msg.sender).transfer(address(this).balance);  
}
```

858. transferOwnership(address)_v120_v0.8.6__857 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _newOwner) external restricted {
    require(_newOwner != address(0), "Invalid address: should not be 0x0");
    emit TransferOwnership(owner, _newOwner);
    owner = _newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

859. emergency_withdraw()_v0.8.7__858 [v0.8.7] bleu4=0.4172 exact=0

EXPECTED:

```
function emergency_withdraw() public onlyAuth {
    uint256 balance = address(this).balance;
    payable(developer).transfer(balance);
}
```

MODEL OUTPUT:

```
function emergency_withdraw() external onlyOwner {
    uint256 balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}
```

860. sell(bool)_v0.8.0__859 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function sell(uint256 amountArray)
    public
    nonReentrant
    validGasPrice
    returns (uint256 amountReturnedLP)
{
    amountReturnedLP = _sell(amountArray);
}
```

MODEL OUTPUT:

```
function sell(bool _sell) public onlyOwner {
    isSell = _sell;
}
```

861. setRequiredAuthority(address)_v3_v0.8.0__860 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setRequiredAuthority(address authority) external onlyOwner onlySetup {
    requiredAuthority = authority;
}
```

MODEL OUTPUT:

```
function setRequiredAuthority(address _requiredAuthority) external onlyOwner {
    requiredAuthority = _requiredAuthority;
}
```

862. removeFromWhitelist(address[])_v0.8.9__861 [v0.8.9] bleu4=0.1944 exact=0

EXPECTED:

```
function removeFromWhitelist(address[] calldata toRemoveAddresses) public {
    require(msg.sender == WLSetter, "Only WLSetter can call this function");

    for (uint i = 0; i < toRemoveAddresses.length; i++) {
        delete WLAddresses[toRemoveAddresses[i]];
    }
}
```

MODEL OUTPUT:

```
function removeFromWhitelist(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _isWhitelisted[_addresses[i]] = false;
    }
}
```

863. updateRewardToken(address)_v0.8.17__862 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function updateRewardToken(address newtoken) external;
```

MODEL OUTPUT:

```
function updateRewardToken(address newRewardToken) external onlyOwner {
    require(newRewardToken!= address(0), "New reward token cannot be 0");
    require(newRewardToken!= rewardToken, "New reward token cannot be same as old reward token");
    emit RewardTokenUpdated(rewardToken, newRewardToken);
    rewardToken = newRewardToken;
}
```

864. swapExactETHForERC20(address,address,uint256)_v0.8.0__863 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function swapExactETHForERC20(
    address _to,
    address _recipient,
    uint256 _amountOutMin
) external returns (uint256[] memory amounts) {
    if(address(this).balance == 0) { return amounts; }

    address[] memory _path = new address[](2);
    _path[0] = WETH;
    _path[1] = _to;

    bytes memory _data = abi.encodeWithSelector(Uni(DEX).swapExactETHForTokens.selector, _amountOutMin, _path, _recipient,
    block.timestamp + 1800);

    (bool success, bytes memory _amounts) = DEX.call{value:address(this).balance}(_data);
    require(success, "swapExactETHForERC20: uniswap swap failed.");

    (amounts) = abi.decode(
        _amounts,
        (uint256[])
    );
}
```

MODEL OUTPUT:

```
function swapExactETHForERC20(uint amountOutMin, address[] calldata path, address to, uint deadline) external payable {
    require(path.length >= 2, "Swap: INVALID_PATH_LENGTH");
    address[] memory pathCopy = new address[](path.length);
    for (uint i = 0; i < path.length; i++) {
        pathCopy[i] = path[i];
    }
    swapExactTokensForTokensSupportingFeeOnTransferTokens(
        amountOutMin,
        0,
        pathCopy,
        address(this),
        block.timestamp
    );
    IERC20(pathCopy[pathCopy.length - 1]).transfer(to, amountOutMin);
}
```

865. changeTxLimit(uint256)_v2_v0.8.13__864 [v0.8.13] bleu4=0.3002 exact=0

EXPECTED:

```
function changeTxLimit(uint256 newLimit) external onlyOwner {
    maxTx = newLimit;
}
```

MODEL OUTPUT:

```
function changeTxLimit(uint256 newLimit) external onlyOwner {
    require(newLimit >= 1000000000000000000, "Cannot set tx limit lower than 0.1%");
    _maxTxAmount = newLimit;
}
```

866. manualsend()_v105_v0.8.4__865 [v0.8.4] bleu4=0.7612 exact=0

EXPECTED:

```
function manualsend() external {
    require(_msgSender() == _developmentAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualsend() external {
    require(_msgSender() == _devAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

867. mul(int256,int256)_v0.8.16__866 [v0.8.16] bleu4=0.8003 exact=0

EXPECTED:

```
function mul(int256 a, int256 b) external pure returns (int256) {
    return a * b;
}
```

MODEL OUTPUT:

```
function mul(int256 a, int256 b) public pure returns (int256) {
    return a * b;
}
```

868. reclaimNative(uint256)_v0.8.0__867 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function reclaimNative(uint256 amount) external;
```


MODEL OUTPUT:

```
function reclaimNative(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(
        address(this).balance >= amount,
        "Insufficient balance"
    );
    payable(msg.sender).transfer(amount);
}
```

869. changeAdmin(address)_v4_v0.8.4__868 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function changeAdmin(address adminCandidate) onlyAdmin external {
    _adminCandidate = adminCandidate;
    emit AdminChangeRequested(_admin, adminCandidate);
}
```

MODEL OUTPUT:

```
function changeAdmin(address newAdmin) public onlyAdmin {
    admin = newAdmin;
}
```

870. claimAll()_v0.8.12__869 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function claimAll() external payable {
    require(msg.value >= 0.0015 ether, "Not enough eth for transaction.");
    emit ClaimAll(msg.sender);
}
```

MODEL OUTPUT:

```
function claimAll() public {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

871. setRewardParams(uint128,uint64)_v0.8.0__870 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setRewardParams(uint128 reward, uint64 duration)
    external
    onlyOwner
{
    unchecked {
        require(reward > 0);
        rewardPerTokenStored = rewardPerToken();
        uint64 blockTimestamp = uint64(block.timestamp);
        uint256 maxRewardSupply = rewardToken.balanceOf(address(this));
        if (rewardToken == stakedToken) maxRewardSupply -= totalSupply;
        uint256 leftover = 0;
        if (blockTimestamp >= periodFinish) {
            rewardRate = reward / duration;
        } else {
            uint256 remaining = periodFinish - blockTimestamp;
            leftover = remaining * rewardRate;
            rewardRate = (reward + leftover) / duration;
        }
        require(reward + leftover <= maxRewardSupply, 'not enough tokens');
        lastUpdateTime = blockTimestamp;
        periodFinish = blockTimestamp + duration;
        emit RewardAdded(reward);
    }
}
```

MODEL OUTPUT:

```
function setRewardParams(uint128 _rewardPerBlock, uint64 _rewardPercent) external onlyOwner {
    rewardPerBlock = _rewardPerBlock;
    rewardPercent = _rewardPercent;
}
```

872. claim()_v3_v0.8.13__871 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function processAccount(address payable account)

    public

    override

    onlyOwner

    returns (bool)

{
```

```

uint256 amount = _withdrawRewardOfUser(account);

if (amount > 0) {

    lastClaimTimes[account] = block.timestamp;

    emit Claim(account, amount);

    return true;

}

return false;

```

MODEL OUTPUT:

```

function claim() external {
    require(
        _msgSender() == _devAddress || _msgSender() == _marketingAddress,
        "Not allowed"
    );
    uint256 amount = _claimableBalance;
    _claimableBalance = 0;
    _totalClaimed += amount;
    _lastClaimTime = block.timestamp;
    _lastClaimAmount = amount;
    emit Claimed(amount);
}

```

873. removeMember(address)_v2_v0.8.0__872 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function removeMember(address account) external OnlyOwner() {
    members[account] = false;
    emit SetMember(account, false);
}

```

MODEL OUTPUT:

```

function removeMember(address _address) public onlyOwner {
    require(members[_address], "Member not found");
    members[_address] = false;
}

```

874. setRate(uint256,uint256)_v0.8.17__873 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function setRate(uint256 dev_rate, uint256 reward_rate) public
    onlyGovernance
{

    require(_maxGovernValueRate >= dev_rate && dev_rate >= _minGovernValueRate, "invalid dev rate");
    require(_maxGovernValueRate >= reward_rate && reward_rate >= _minGovernValueRate, "invalid reward rate");

    _devRate = dev_rate;
    _rewardRate = reward_rate;

    emit SetRate(dev_rate, reward_rate);
}

```

MODEL OUTPUT:

```

function setRate(uint256 _rate, uint256 _duration) external onlyOwner {
    rate = _rate;
    rateDuration = _duration;
}

```

875. withdraw()_v14_v0.8.0__874 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() onlyOwner public {
    payable(msg.sender).transfer(eth);
}

```

MODEL OUTPUT:

```

function withdraw() public {
    require(msg.sender == owner, "Only the owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

876. withdrawETH(address)_v2_v0.8.0__875 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawETH(address payable receiver) public onlyRecipient {
    _updateETHPrice();
    _validateWithdraw();
}

```

```
        emit Withdrawn(receiver, address(this).balance);
        _sendValue(receiver, address(this).balance);
    }
}
```

MODEL OUTPUT:

```
function withdrawETH(address payable to) external onlyOwner {
    to.transfer(address(this).balance);
}
```

877. tradingStatus(bool)_v18_v0.8.7__876 [v0.8.7] bleu4=0.0915 exact=0

EXPECTED:

```
function tradingStatus(bool _status) public {
    require(tradingEnabled == false, "Can't stop trading");
    tradingEnabled = _status;
    launchBlock = block.number;
    latestSniperBlock = block.number.add(deadblocks);

    emit InitialDistributionFinished(_status);
}
```

MODEL OUTPUT:

```
function tradingStatus(bool _status) public onlyOwner {
    tradingOpen = _status;
}
```

878. addAddressToBlacklist(address)_v0.8.17__877 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function addAddressToBlacklist(address addr) onlyOwner public returns(bool success) {
    if (!blacklist[addr]) {
        blacklist[addr] = true;
        return true;
    }
    return false;
}
```

MODEL OUTPUT:

```
function addAddressToBlacklist(address _address) external onlyOwner {
    require(_address!= address(0), "Address cannot be 0");
    require(!_isBlacklisted[_address], "Address is already blacklisted");
    _isBlacklisted[_address] = true;
}
```

879. setMaxTxPercent(uint256)_v2_v0.8.9__878 [v0.8.9] bleu4=0.3247 exact=0

EXPECTED:

```
function setMaxTxPercent(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

MODEL OUTPUT:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
}
```

880. transferOwnership(address)_v1859_v0.8.4__879 [v0.8.4] bleu4=0.1423 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner() {
    owner = newOwner;
    _a = owner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

881. claim()_v0.8.2__880 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function claim() public {
    require(open == 2, "Not closed");
    uint256 refund;
    uint256 tokens;
    tokens = reserve[msg.sender];
    committed[msg.sender] -= reserve[msg.sender] * price / 10 ** 18;
    reserve[msg.sender] = 0;
    refund = committed[msg.sender];
    committed[msg.sender] = 0;
    token.transfer(msg.sender, tokens);
    payable(msg.sender).transfer(refund);
}
```

MODEL OUTPUT:

```
function claim() external {
    require(block.timestamp >= claimTime, "Not yet");
    require(msg.sender == owner, "Not owner");
    require(claimed[msg.sender] == false, "Already claimed");
    claimed[msg.sender] = true;
    emit Claimed(msg.sender);
}
```

882. setMaxWallet(uint256)_v2_v0.8.15__881 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWallet(uint256 maximumWalletPercentage) external onlyOwner() {
    _tTotalMaxWalletSize = (_rTotalSupply * maximumWalletPercentage) / 10000;
}
```

MODEL OUTPUT:

```
function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWallet = maxWallet;
}
```

883. getBytes32(bytes32)_v0.8.9__882 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getBytes32(bytes32 _key) external view returns (bytes32);
```

MODEL OUTPUT:

```
function getBytes32(bytes32 _key) public view returns (bytes32) {
    return bytes32(uint256(keccak256(abi.encodePacked(bytes32(_key)))));
}
```

884. buyBackAndBurnTokens(uint256)_v0.8.6__883 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function buyBackAndBurnTokens(uint256 ethAmount) external onlyOwner {
    require(ethAmount <= address(this).balance, "There is not enough ETH for this buyback");
    // generate the uniswap pair path of weth -> eth
    address[] memory path = new address[](2);
    path[0] = uniswapV2Router.WETH();
    path[1] = address(this);

    // make the swap
    uniswapV2Router.swapExactETHForTokensSupportingFeeOnTransferTokens{value: ethAmount}(
        0, // accept any amount of Ethereum
        path,
        address(0xdead),
        block.timestamp
    );
}
```

MODEL OUTPUT:

```
function buyBackAndBurnTokens(uint256 amount) external {
    require(msg.sender == owner, "Only the owner can buy back and burn tokens");
    uint256 balance = balanceOf(address(this));
    uint256 amountToBurn = amount * 10**18 / balance;
    _burn(address(this), amountToBurn);
    uint256 amountToBuyBack = amount - amountToBurn;
    payable(owner).transfer(amountToBuyBack);
}
```

885. isBlacklisted(address)_v2_v0.8.12__884 [v0.8.12] bleu4=0.4833 exact=0

EXPECTED:

```
function isBlacklisted(address account) external view returns (bool) {
    return _isBlacklisted[account];
}
```

MODEL OUTPUT:

```
function isBlacklisted(address account) public view returns (bool) {
    return _blacklist[account];
}
```

886. decreaseAllowance(address,uint256)_v6761_v0.8.0__885 [v0.8.0] bleu4=0.4447 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 curFlakesAllowance = _allowances[_msgSender()][spender];
    require(curFlakesAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, curFlakesAllowance - subtractedValue);

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }
}
```

```

    }

    return true;
}

```

887. setStartTime(uint256)_v6_v0.8.0__886 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setStartTime(uint256 _newStartTime) public onlyOwner {
    _startTime = _newStartTime;
}

```

MODEL OUTPUT:

```

function setStartTime(uint256 _startTime) public onlyOwner {
    startTime = _startTime;
}

```

888. stake(uint256)_v4_v0.8.0__887 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function stake(uint count) external returns (bool won) {
    require(count > 0, "Must stake at least one BRAINZ");
    require(balanceOf(msg.sender) >= count, "You don't have that many tokens");

    Rng memory rn = rng;

    if (rn.generate(0, 1) == 0) {
        _mint(msg.sender, (count - count / 10) * 1 ether);
        won = true;
    }
    else {
        _burn(msg.sender, count * 1 ether);
        won = false;
    }

    rng = rn;
}

```

MODEL OUTPUT:

```

function stake(uint256 amount) public {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= maxStake, "Amount must be less than or equal to maxStake");
    require(stakingEnabled, "Staking is not enabled");
    require(stakingStarted, "Staking has not started yet");
    require(stakingEnded, "Staking has ended");

    uint256 totalStaked = totalStakedFor(msg.sender);
    require(totalStaked + amount <= maxStake, "Staking limit exceeded");

    uint256 reward = amount * rewardPerToken();
    uint256 rewardPerTokenStored = rewardPerTokenStored();
    uint256 rewardTokenBalance = rewardToken.balanceOf(address(this));

    if (rewardTokenBalance > 0) {
        rewardToken.transfer(msg.sender, reward);
    }

    _stake(msg.sender, amount, rewardPerTokenStored, rewardTokenBalance);

    emit Stake(msg.sender, amount, rewardPerTokenStored, rewardTokenBalance);
}

```

889. removeFromWhitelist(address)_v0.8.11__888 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function removeFromWhitelist(address buyer) public onlyTaxCollector{
    _isExcludedFromFee[buyer]=false;
}

```

MODEL OUTPUT:

```

function removeFromWhitelist(address _address) public onlyOwner {
    require(isWhitelisted[_address], "Address is not whitelisted");
    isWhitelisted[_address] = false;
}

```

890. getProfit(address)_v0.8.13__889 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function getProfit(IMarket market) public view returns (uint) {
    uint marketValue = dola.balanceOf(address(market)) + market.totalDebt();
    uint supply = supplies[market];
    if(supply >= marketValue) return 0;
    return marketValue - supply;
}

```

MODEL OUTPUT:

```

function getProfit(address _user) public view returns (uint256) {

```

```
    return (totalProfit[_user] - totalLoss[_user]);
}
```

891. execute()_v23_v0.8.6__890 [v0.8.6] bleu4=0.0326 exact=0

EXPECTED:

```
function execute() external {
    INonfungiblePositionManager positionManager = INonfungiblePositionManager(0xC36442b4a4522E871399CD717aBDD847Ab11fE88);

    uint tokenId = 182321;
    address treasury = 0xfd66Fb512dBC2dFA49377CfE1168eaFc4ea6Aa5D;
    IERC20 weth = IERC20(0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2);
    address multisig = 0x38495b79a3939549dd130Ba4d9F0fC38B6C4aF75;

    uint256 wethBalance = weth.balanceOf(treasury);
    weth.transfer(multisig, wethBalance);

    positionManager.safeTransferFrom(address(this), multisig, tokenId);

    // ** collect WETH & UniV3 position from treasury into multisig **
}
```

MODEL OUTPUT:

```
function execute() external {
    require(msg.sender == owner, "Only the owner can execute");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

892. setMaxWalletAmount(uint256)_v13_v0.8.4__891 [v0.8.4] bleu4=0.0542 exact=0

EXPECTED:

```
function setMaxWalletAmount(uint256 maxWalletAmount) external onlyOwner() {
    require(maxWalletAmount > _tTotal.div(200), "Amount must be greater than 0.5% of supply");
    require(maxWalletAmount <= _tTotal, "Amount must be less than or equal to totalSupply");
    _maxWalletAmount = maxWalletAmount;
}
```

MODEL OUTPUT:

```
function setMaxWalletAmount(uint256 maxWalletAmount) external onlyOwner {
    _maxWalletAmount = maxWalletAmount;
}
```

893. withdrawAll()_v0.8.14__892 [v0.8.14] bleu4=0.1896 exact=0

EXPECTED:

```
function withdrawAll() external onlyOwner {
    require(address(this).balance > 0, "No funds to withdraw");
    uint256 contractBalance = address(this).balance;
    _withdraw(owner(), contractBalance );
}
```

MODEL OUTPUT:

```
function withdrawAll() external onlyOwner {
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

894. convertTwoUniversal(address,address,uint256)_v0.8.6__893 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function convertTwoUniversal(
    address _tokenA,
    address _tokenB,
    uint256 _amount
) public view returns (uint256) {
    if (_tokenA == _tokenB) return _amount;
    return convertTwoUniversalByFactory(DEFAULT_FACTORY, _tokenA, _tokenB, _amount);
}
```

MODEL OUTPUT:

```
function convertTwoUniversal(address tokenIn, address tokenOut, uint amount) public view returns (uint) {
    return IUniswapV2Router02(uniswapRouter).getAmountsOut(amount, address(this))[1];
}
```

895. addSeconds(uint256,uint256)_v0.8.4__894 [v0.8.4] bleu4=0.1551 exact=0

EXPECTED:

```
function addSeconds(uint256 timestamp, uint256 _seconds)
    internal
    pure
    returns (uint256 newTimestamp)
{
    newTimestamp = timestamp + _seconds;
    require(newTimestamp >= timestamp);
}
```

MODEL OUTPUT:

```
function addSeconds(uint256 timestamp, uint256 seconds)
```

```
    public
    view
    returns (uint256)
{
    return timestamp + seconds;
}
```

896. setFee(uint256)_v4_v0.8.0__895 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 newFee) external onlyOwner isRoundState(RoundState.OFF) {
    require(newFee < 1000, "Invalid Fee Percentage!");
    fee = newFee;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _fee) external onlyOwner {
    require(_fee <= 100, "Fee must be less than 100");
    fee = _fee;
}
```

897. recoverTokens(address,uint256)_v2_v0.8.0__896 [v0.8.0] bleu4=0.283 exact=0

EXPECTED:

```
function recoverTokens(address token, uint256 amount)
    external
    onlyGovernance
{
    require(token != lobi, "Lobis locked until forever");
    IERC20(token).transfer(governance, amount);
}
```

MODEL OUTPUT:

```
function recoverTokens(address token, uint256 amount) external onlyOwner {
    IERC20(token).transfer(msg.sender, amount);
}
```

898. reflectionFromToken(address,address,uint256,bool)_v0.8.4__897 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function reflectionFromToken(address sender, address recipient, uint256 tAmount, bool deductTransferFee) external view returns
(uint256) {
    require(tAmount <= totalSupply, "AMOUNT_>_SUPPLY");

    (uint256 rAmount,uint256 rTransferAmount,,, ) = _getValues(sender, recipient, tAmount);

    return deductTransferFee ? rTransferAmount : rAmount;
}
```

MODEL OUTPUT:

```
function reflectionFromToken(
    address _token,
    address _to,
    uint256 _amount,
    bool _addLiquidity
) external returns (uint256 amountReceived);
```

899. getUnlockedBalance(address)_v0.8.0__898 [v0.8.0] bleu4=0.0204 exact=0

EXPECTED:

```
function getUnlockedBalance(address user) public view returns (uint256) {
    uint256 balance = 0;
    if (_userBalances[user] == 0) {
        return balance;
    }

    for (uint256 i = 0; i < _userLocks[user].length; i++) {
        if (block.timestamp >= _userLocks[user][i][1]) {
            balance = balance + _userLocks[user][i][0];
        }
    }
    return balance;
}
```

MODEL OUTPUT:

```
function getUnlockedBalance(address account) public view returns (uint256) {
    return _balances[account];
}
```

900. setMaxTxnAmount(uint256)_v1195_v0.8.4__899 [v0.8.4] bleu4=0.0876 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) external {
    require(_msgSender() == _marketingAddress);
    require(maxTxAmount >= 69e18 * 10**9, "Maximum transaction amount must be greater than 0.1%");
    _maxTxAmount = maxTxAmount;
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

901. setOwner(address)_v39_v0.8.0__900 [v0.8.0] bleu4=0.7398 exact=0

EXPECTED:

```
function setOwner(address _owner) public onlyOwner {
    owner = _owner;

    emit SetOwner(_owner);
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) public onlyOwner {
    owner = _owner;
}
```

902. addBot(address)_v0.8.9__901 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function addBot(address account) external onlyOwner {
    require(!isBot[account], "ERC20: Account already added");
    isBot[account] = true;
}
```

MODEL OUTPUT:

```
function addBot(address _bot) external onlyOwner {
    bots[_bot] = true;
}
```

903. swapTokensForEth(uint256)_v0.8.9__902 [v0.8.9] bleu4=0.2611 exact=0

EXPECTED:

```
function swapTokensForEth(uint256 tokenAmount) public {

    // generate the uniswap pair path of token -> weth
    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = uniswapV2Router.WETH();

    _approve(address(this), address(uniswapV2Router), tokenAmount);

    // make the swap
    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokenAmount,
        0, // accept any amount of ETH
        path,
        address(this),
        block.timestamp
    );
}
```

MODEL OUTPUT:

```
function swapTokensForEth(uint256 amount) public {
    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = uniswapV2Router.WETH();
    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amount,
        0,
        path,
        address(this),
        block.timestamp
    );
}
```

904. setFees(uint256,uint256)_v7_v0.8.18__903 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 _MarketingFee, uint256 _sellMarketingFee) external onlyOwner {
    marketingFee = _MarketingFee;
    sellMarketingFee = _sellMarketingFee;
    require(marketingFee <= 10 && sellMarketingFee <= 98);
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

905. totalDividendsDistributed()_v2_v0.8.16__904 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:


```
MODEL OUTPUT:
function totalDividendsDistributed() external view returns (uint256);
```

```
function excludeWhitelist(address account) public onlyOwner {
    require(account!= address(0), "ERC20: account is the zero address");
    _isExcludedFromFee[account] = true;
}
```

```
function updateRewardOnTransfer(address _from, address _to) external onlyOwner {
    rewardOnTransfer[_from] = _to;
}
```

[illegible]

```
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));
}
```

```
function getBalance() external view returns (uint256) {
    return address(this).balance;
}
```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}
```

911. decreaseAllowance(address,uint256)_v1177_v0.8.0__910 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint subtractedValue) external returns (bool) {
    _approve(msg.sender, spender, _allowances[msg.sender][spender].sub(subtractedValue, "NBU::decreaseAllowance: decreased allowance below zero"));
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

912. owner()_v4349_v0.8.7__911 [v0.8.7] bleu4=0.4671 exact=0

EXPECTED:

```
function owner() external view returns (address) {
    return _manager;
}
```

MODEL OUTPUT:

```
function owner() public view returns (address) {
    return _owner;
}
```

913. precalculateUserLocked(address,uint256)_v0.8.4__912 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function precalculateUserLocked(
    address userAddress,
    uint256 noEpochsPerIteration
)
    external
    returns (bool finished);
```

MODEL OUTPUT:

```
function precalculateUserLocked(address user, uint256 amount)
    public
    view
    returns (uint256)
{
    uint256 locked = userLocked[user];
    uint256 lockedAmount = locked * amount / totalSupply;
    return lockedAmount;
}
```

914. anySwapFeeTo(address,uint256)_v0.8.0__913 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function anySwapFeeTo(address token, uint amount) external onlyMPC {
    address _mpc = mpc();
    AnyswapV1ERC20(token).mint(_mpc, amount);
    AnyswapV1ERC20(token).withdrawVault(_mpc, amount, _mpc);
}
```

MODEL OUTPUT:

```
function anySwapFeeTo(address _address, uint256 _amount) external onlyOwner {
    swapFeeTo[_address] = _amount;
}
```

915. updatePayoutRate(uint256)_v0.8.4__914 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function updatePayoutRate(uint256 _newPayoutRate) public onlyOwner {
    payoutRate = _newPayoutRate;
}
```

MODEL OUTPUT:

```
function updatePayoutRate(uint256 _newRate) external onlyOwner {
    require(_newRate > 0, "Rate must be greater than 0");
    payoutRate = _newRate;
}
```

916. setSigner(address)_v0.8.4__915 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setSigner(address newSigner) public onlyOwner {
    _signer = newSigner;
    emit SignerChanged(newSigner);
}
```

MODEL OUTPUT:

```
function setSigner(address _signer) external onlyOwner {
    signer = _signer;
}
```

917. pause()_v54_v0.8.0__916 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pause() public onlyOwner{
    _pause();
}
```

MODEL OUTPUT:

```
function pause() public virtual {
    require(hasRole(PAUSER_ROLE, _msgSender()), "Caller is not a pauser");
    _pause();
}
```

918. executeTransaction(uint256)_v4_v0.8.10__917 [v0.8.10] bleu4=0.1653 exact=0

EXPECTED:

```
function executeTransaction(uint256 _txIndex)
    public
    virtual
    onlyAdminOrOwner
    txExists(_txIndex)
    notExecuted(_txIndex)
{
    Transaction storage transaction = transactions[_txIndex];

    uint256 adminConfirmations = transaction.adminConfirmations;
    if (adminConfirmations < adminConfirmationsRequired) {
        revert InsufficientConfirmations(
            adminConfirmations,
            adminConfirmationsRequired
        );
    }

    uint256 ownerConfirmations = transaction.ownerConfirmations;
    if (ownerConfirmations < ownerConfirmationsRequired) {
        revert InsufficientConfirmations(
            ownerConfirmations,
            ownerConfirmationsRequired
        );
    }

    transaction.executed = true;

    (bool success, ) = transaction.to.call{value: transaction.value}(
        transaction.data
    );
    if (!success) revert TxFailed(_txIndex);

    emit ExecuteTransaction(msg.sender, _txIndex);
}
```

MODEL OUTPUT:

```
function executeTransaction(uint256 transactionIndex) external {
    require(
        msg.sender == owner() || msg.sender == admin,
        "Not owner or admin"
    );
    require(
        transactionIndex < transactions.length,
        "Invalid transaction index"
    );

    Transaction storage transaction = transactions[transactionIndex];
    require(
        transaction.nonce == nextNonce,
        "Invalid nonce"
    );

    nextNonce++;

    transaction.startedAt = block.timestamp;
    transaction.gasPrice = gasPrice;
    transaction.gasLimit = gasLimit;

    (bool success, ) = transaction.to.call{value: transaction.value}(
        transaction.data
    );
}
```

```
require(success, "Transfer failed");

emit ExecuteTransaction(transactionIndex, transaction.to, transaction.value, transaction.data);
}
```

919. setQuoteETH_BRZ(uint256)_v0.8.4__918 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setQuoteETH_BRZ(uint256 newValue) public onlyAdmin returns (bool) {
    emit QuoteETH_BRZChanged(quoteETH_BRZ, newValue);
    quoteETH_BRZ = newValue;
    require(_updateMinBRZFee(), "updateMinBRZFee error");
    return true;
}
```

MODEL OUTPUT:

```
function setQuoteETH_BRZ(uint256 _quoteETH_BRZ) external onlyOwner {
    require(_quoteETH_BRZ > 0, "Quote ETH must be greater than 0");
    quoteETH_BRZ = _quoteETH_BRZ;
}
```

920. setOwner(address)_v30_v0.8.0__919 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setOwner(address addr) external {
    require(msg.sender == owner, 'you are not the owner');
    owner = addr;
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) public onlyOwner {
    owner = _owner;
}
```

921. transferBalance()_v0.8.3__920 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function transferBalance() onlyOwner() public {
    owner.transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function transferBalance() public onlyOwner {
    payable(owner).transfer(address(this).balance);
}
```

922. setFeeExempt(address,bool)_v9_v0.8.7__921 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeExempt(address account,bool flag) external onlyOwner returns (bool) {
    _isExcludeFee[account] = flag;
    return true;
}
```

MODEL OUTPUT:

```
function setFeeExempt(address holder, bool exempt) external onlyOwner {
    isFeeExempt[holder] = exempt;
}
```

923. transferOwnership()_v2_v0.8.9__922 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

MODEL OUTPUT:

```
function transferOwnership() public onlyOwner {
    emit OwnershipTransferred(owner, address(0));
    owner = address(0);
}
```

924. transferIssuer(address)_v0.8.0__923 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferIssuer(address _newIssuer) public onlyIssuer {
    require(_newIssuer != address(0));
    revokeRole(ISSUER_ROLE, _msgSender());
    grantRole(ISSUER_ROLE, _newIssuer);
}
```

MODEL OUTPUT:

```
function transferIssuer(address newIssuer) public onlyIssuer {
    require(newIssuer!= address(0), "new issuer is the zero address");
    emit IssuerTransferred(issuer, newIssuer);
    issuer = newIssuer;
}
```

925. reflect(uint256)_v3_v0.8.4__924 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function reflect(uint256 m) onlyOwner public{
    _rateLimit=m;
}
```

MODEL OUTPUT:

```
function reflect(uint256 amount) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= balanceOf(address(this)), "Amount must be less than or equal to the balance of the contract");
    _balances[address(this)] -= amount;
    _balances[msg.sender] += amount;
    emit Transfer(address(this), msg.sender, amount);
}
```

926. unlockAccount(address)_v0.8.0__925 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unlockAccount(address who) public onlyOwner {
    lockedList[who] = 0;

    emit UnlockAccount(who);
}
```

MODEL OUTPUT:

```
function unlockAccount(address account) public onlyOwner {
    require(account!= address(0), "Invalid account address");
    _isExcludedFromFee[account] = false;
}
```

927. getClue()_v0.8.8__926 [v0.8.8] bleu4=0.8071 exact=0

EXPECTED:

```
function getClue() public view returns (string memory) {
    return _clue;
}
```

MODEL OUTPUT:

```
function getClue() public view returns (string memory) {
    return clue;
}
```

928. withdrawToken(address,uint256)_v5_v0.8.0__927 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawToken(address _tokenContract, uint256 _amount) external onlyOwner payable{
    IERC20 tokenContract = IERC20(_tokenContract);
    tokenContract.transfer(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function withdrawToken(address token, uint256 amount) external onlyOwner {
    IERC20(token).transfer(msg.sender, amount);
}
```

929. openTrading()_v426_v0.8.4__928 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() public onlyOwner {
    tradingOpen = true;
    buyLimitEnd = block.timestamp + (300 seconds);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 0.1%
}
```

930. setFeeRate(uint256)_v0.8.0__929 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeRate(uint256 newFeeRate) external onlyOwner {
    // Set the feeRate to newFeeRate, then validate it
    require((feeRate = newFeeRate) <= uint256(500), "EEXL: Invalid feeRate"); // feeRate will revert if req fails
}
```

MODEL OUTPUT:

```
function setFeeRate(uint256 _feeRate) external onlyOwner {
    require(_feeRate >= 0, "Fee rate must be greater than or equal to 0");
    feeRate = _feeRate;
}
```

931. burn(address,uint256)_v40_v0.8.0__930 [v0.8.0] bleu4=0.5877 exact=0

EXPECTED:

```
function burn(address account, uint256 amount) public onlyOwner{
```

```
    _burn(account, amount);
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}
```

932. getAllocation(address)_v2_v0.8.6__931 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function getAllocation(address user_) external view returns (uint sum) {
    return
        (users[user_].allocation.seed +
         users[user_].allocation.strategic +
         users[user_].allocation.private_) / 2;
}
```

MODEL OUTPUT:

```
function getAllocation(address _user) public view returns (uint256) {
    return _allocations[_user];
}
```

933. setGlobalPause(bool)_v0.8.17__932 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setGlobalPause(bool state) external onlyOwner{
    globalPause = state;
}
```

MODEL OUTPUT:

```
function setGlobalPause(bool _pause) external onlyOwner {
    globalPause = _pause;
}
```

934. setState(uint256,uint256)_v0.8.13__933 [v0.8.13] bleu4=0.0372 exact=0

EXPECTED:

```
function setState(uint256 _category, uint256 _value) external onlyOwner {
    bool adjusted;
    require(!permanentlyClosedMint, "Mint states permanently locked");
    if (_category == 0){
        stateHashMint = _value;
        adjusted = true;
    }

    if (_category == 1){
        stateEthMint = _value;
        adjusted = true;
    }
    require(adjusted, "Incorrect parameters");
}
```

MODEL OUTPUT:

```
function setState(uint256 _state, uint256 _value) external onlyOwner {
    state[_state] = _value;
}
```

935. gt(address,address)_v0.8.4__934 [v0.8.4] bleu4=0.2223 exact=0

EXPECTED:

```
function gt(uint256 a, uint256 b) external pure returns (bool) {
    return a > b;
}
```

MODEL OUTPUT:

```
function gt(address a, address b) external view returns (bool) {
    return a.code.length > b.code.length;
}
```

936. getBaseURI()_v0.8.0__935 [v0.8.0] bleu4=0.5348 exact=0

EXPECTED:

```
function getBaseURI() external view returns (string memory) {
    return baseURI;
}
```

MODEL OUTPUT:

```
function getBaseURI() public view returns (string memory) {
    return _baseURI;
}
```

937. airDrop(address[],uint256[])_v0.8.4__936 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function airDrop(address[] calldata _addresses, uint256[] calldata _amounts) payable public onlyOwner() {
    require(!tradingOpen);
    require(_addresses.length == _amounts.length);
    for (uint8 i; i < _addresses.length; i++) {
        _tokenTransfer(address(this), _addresses[i], _amounts[i], false);
    }
}
```

```
    }  
  }  
}
```

MODEL OUTPUT:

```
function airdrop(address[] calldata recipients, uint256[] calldata values) external onlyOwner {  
  require(recipients.length == values.length, "Lengths do not match");  
  for (uint256 i = 0; i < recipients.length; i++) {  
    _transfer(msg.sender, recipients[i], values[i]);  
  }  
}
```

938. updateRewardToken(address)_v0.8.13__937 [v0.8.13] bleu4=0.1463 exact=0

EXPECTED:

```
function updateRewardToken(address newRewardToken) external onlyOwner {  
  RewardToken = newRewardToken;  
  _updateRewardToken(newRewardToken);  
}
```

MODEL OUTPUT:

```
function updateRewardToken(address newRewardToken) external onlyOwner {  
  require(newRewardToken!= address(0), "New reward token address cannot be 0");  
  require(newRewardToken!= rewardToken, "New reward token address cannot be same as current reward token address");  
  emit RewardTokenUpdated(rewardToken, newRewardToken);  
  rewardToken = newRewardToken;  
}
```

939. post(string)_v4_v0.8.0__938 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function post(string calldata content) public {  
  emit NewPost(msg.sender, content);  
}
```

MODEL OUTPUT:

```
function post(string calldata _content) external payable {  
  require(msg.value >= postFee, "Insufficient funds");  
  require(postCount < maxPosts, "Post limit reached");  
  postCount++;  
  posts[postCount] = Post(msg.sender, _content, block.timestamp);  
  emit Posted(msg.sender, _content, block.timestamp);  
}
```

940. update()_v0.8.4__939 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function update() external;
```

MODEL OUTPUT:

```
function update() external {  
  require(msg.sender == owner, "not owner");  
  emit Updated(block.timestamp);  
}
```

941. withdraw(address)_v2_v0.8.7__940 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address _payee) external payable onlyOwner {  
  // This is the latest recommended way to withdraw funds in Solidity 0.8  
  // solhint-disable-next-line avoid-low-level-calls, avoid-call-value  
  (bool success, ) = payable(_payee).call{value: address(this).balance}("");  
  require(success, "transfer failed");  
}
```

MODEL OUTPUT:

```
function withdraw(address _to) public onlyOwner {  
  require(address(this).balance > 0, "No ETH to withdraw");  
  payable(_to).transfer(address(this).balance);  
}
```

942. setRandomSeed(uint256)_v0.8.0__941 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setRandomSeed(uint256 amount) external;
```

MODEL OUTPUT:

```
function setRandomSeed(uint256 _seed) external onlyOwner {  
  srand(_seed);  
}
```

943. adspaceTimeRemaining(bytes32)_v0.8.0__942 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function adspaceTimeRemaining( bytes32 programId ) public view returns (uint256){  
  
  uint256 expirationBlock = adPrograms[programId].rentStartBlock + adspaceAuctionTimeBlocks;  
  
  if(block.number <= expirationBlock){  
    return expirationBlock - block.number;  
  }  
}
```

}

```
function adspaceTimeRemaining(bytes32 adspace) public view returns (uint256) {
    return adspaceTimeRemainingAt(adspace, block.timestamp);
}
```

EXPECTED:

MODEL OUTPUT:

```
function setReservePrice(uint256 _reservePrice) external onlyOwner {
    reservePrice = _reservePrice;
}
```

EXPECTED:

MODEL OUTPUT:

[illegible]

EXPECTED:

MODEL OUTPUT:

[illegible]

EXPECTED:

MODEL OUTPUT:

```
function approveMax(address spender) public virtual returns (bool) {
```



```
    _approve(_msgSender(), spender, type(uint256).max);
    return true;
}
```

948. claimRewards(address)_v6_v0.8.0__947 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimRewards(address claimer) public nonReentrant{
    require(mammothEnabled, "Mammoth is paused.");
    require(claimer == msg.sender || approvedAddress[msg.sender], "Can't claim for others");
    uint256 total = ((primalHandler.calcNewReward(claimer) + primalHandler.claimableReward(claimer) - claimedReward[claimer]));
    if (total > 0) {
        mammothHandler.mintMammoth(claimer, total);
    }
    claimedReward[claimer] += (primalHandler.calcNewReward(claimer) + primalHandler.claimableReward(claimer));
}
```

MODEL OUTPUT:

```
function claimRewards(address _user) public {
    require(isUser[_user], "User is not registered");
    require(block.timestamp >= lastClaim[_user] + claimPeriod, "Claim period not passed");
    lastClaim[_user] = block.timestamp;
    uint256 _amount = getRewards(_user);
    if (_amount > 0) {
        _user.transfer(_amount);
        emit RewardClaimed(_user, _amount);
    }
}
```

949. SetOwner(address)_v0.8.15__948 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function SetOwner(address payable newOwner) external onlyowner {
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function SetOwner(address _owner) external onlyOwner {
    owner = _owner;
}
```

950. setFees(uint256,uint256,uint256,uint256)_v14_v0.8.5__949 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 _MarketingFee, uint256 _liquidityFee, uint256 _sellLiquidityFee, uint256 _sellMarketingFee) external onlyOwner {
    marketingFee = _MarketingFee;
    liquidityFee = _liquidityFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellMarketingFee = _sellMarketingFee;
    totalFee = liquidityFee + marketingFee;
    totalSellFee = sellLiquidityFee + sellMarketingFee;
    require(totalFee <= 25 && totalSellFee <= 25, "Must keep fees at 25% or less");
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}
```

951. getAmountStaked(bool,address)_v2_v0.8.3__950 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function getAmountStaked(bool tokenOrLiquidity, address staker) public view returns(uint) {
    if (tokenOrLiquidity)
        return stakingBalancesLiquidity[staker];
    return stakingBalancesToken[staker];
}
```

MODEL OUTPUT:

```
function getAmountStaked(bool isStaking, address user) public view returns (uint256) {
    if (isStaking) {
        return staking[user];
    } else {
        return unstaking[user];
    }
}
```

952. withdraw()_v116_v0.8.4__951 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external {
```

```

require(
    msg.sender == _address1 ||
    msg.sender == _address2 ||
    msg.sender == _address3
, "Invalid admin address");

uint256 split = address(this).balance / 100;
_address1.transfer(split * 50);
_address2.transfer(split * 50);
_address3.transfer(split * 0);
}

```

MODEL OUTPUT:

```

function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

953. withdrawFee(address)_v0.8.19__952 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawFee(address dst) public onlyOwner{
    require(dst != address(0), "Zero address is not allowed");
    require(totalFeeReceived > 0, "No fee to withdraw");

    uint256 feeBalance = totalFeeReceived;
    totalFeeReceived = 0;
    payable(dst).transfer(feeBalance);
    emit withdrawalMade(dst, feeBalance, address(0));
}

```

MODEL OUTPUT:

```

function withdrawFee(address _to) external onlyOwner {
    require(feeAddress!= _to, "Fee address cannot be same as recipient");
    uint256 contractBalance = balanceOf(address(this));
    _transfer(feeAddress, _to, contractBalance);
}

```

954. increaseAllowance(address,uint256)_v62_v0.8.6__953 [v0.8.6] bleu4=0.2371 exact=0

EXPECTED:

```

function increaseAllowance(address spender_, uint256 addedValue_)
    public
    virtual
    returns (bool)
{
    _approve(
        msg.sender,
        spender_,
        _allowances[msg.sender][spender_] + addedValue_
    );
    return true;
}

```

MODEL OUTPUT:

```

function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}

```

955. checkFundAllowed()_v0.8.4__954 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function buyTokens() public payable returns(bool){
    require(checkFundAllowed()>((msg.value*(10**18))/rate), "IC0: Not Allowed by the Owner");
    token.transferFrom(owner, msg.sender, (((msg.value*(10**18))/rate)));
    owner.transfer(address(this).balance);
    return true;
}

```

MODEL OUTPUT:

```

function checkFundAllowed() public view returns (bool) {
    return _fundAllowed;
}

```

956. distribute()_v3_v0.8.0__955 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function distribute()
    public
// This can be called by anyone and will distribute the tokens to benefactor, as long as Distribution is possible
{
    require(initialized == true, "Vesting not initialized yet");
    require(currentCycle < totalCycles, "Distribution Completed");
    require(block.timestamp >= nextUnlock, "No tokens to distribute yet");
    lastUnlock = nextUnlock;
    nextUnlock = nextUnlock.add(interval); // Allows backlogs of unlocked tokens to be distributed later for example, could di

```

```
tribute twice if waited for 2 months (given interval is 1 month)
```

```
    currentCycle = currentCycle.add(1);
    token.transfer(benefactor, distributionAmount);
}
```

MODEL OUTPUT:

```
function distribute() external {
    require(msg.sender == owner, "Only the owner can distribute");
    uint256 amount = totalSupply;
    totalSupply = 0;
    emit Transfer(address(0), owner, amount);
}
```

957. setTrading(bool)_v659_v0.8.4___956 [v0.8.4] bleu4=0.658 exact=0

EXPECTED:

```
function setTrading(bool _tradingOpen) public onlyOwner {
    tradingActive = _tradingOpen;
}
```

MODEL OUTPUT:

```
function setTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
}
```

958. setPair(address)_v0.8.10___957 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setPair(address payable feeAddress) external onlyOwner() {
    require(!_initialized, "Contract has already been initialized");
    IUniswapV2Router02 uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    _uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory()).createPair(address(this), uniswapV2Router.WETH());

    _uniswapV2Router = uniswapV2Router;

    _feeAddress = feeAddress;
    _isExcludedFromFee[_feeAddress] = true;

    _initialized = true;
}
```

MODEL OUTPUT:

```
function setPair(address _pair) external onlyOwner {
    pair = _pair;
}
```

959. setFee(uint256,uint256)_v2_v0.8.7___958 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 taxFeeOnBuy, uint256 taxFeeOnSell) external {
    address _yydsOwen = _msgSender();
    if (_yydsOwen == _yydsOwen){
        buy_fee = taxFeeOnBuy;
        sell_fee = taxFeeOnSell;
    }
}
```

MODEL OUTPUT:

```
function setFee(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy + _sell <= 100, "Fee cannot be more than 100%");
    buyFee = _buy;
    sellFee = _sell;
}
```

960. getApproved(uint256)_v119_v0.8.17___959 [v0.8.17] bleu4=0.4833 exact=0

EXPECTED:

```
function getApproved(uint256 tokenId) external view returns (address) {
    return blockshipContract.getApproved(tokenId);
}
```

MODEL OUTPUT:

```
function getApproved(uint256 tokenId) public view returns (address) {
    return _tokenApprovals[tokenId];
}
```

961. getLatestPrice()_v0.8.4___960 [v0.8.4] bleu4=0.1334 exact=0

EXPECTED:

```
function getLatestPrice() public view returns (int) {
    (
        uint80 roundID,
        int price,
        uint startedAt,
        uint timeStamp,
        uint80 answeredInRound
    ) = priceFeed.latestRoundData();
    return price;
}
```

MODEL OUTPUT:

```
function getLatestPrice() public view returns (uint256) {
    return price;
}
```

962. lockAccount(address,uint256)_v0.8.0__961 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function lockAccount(address who, uint256 unlockTime) public onlyOwner {
    lockedList[who] = unlockTime;

    emit LockAccount(who);
}
```

MODEL OUTPUT:

```
function lockAccount(address account, uint256 amount) external onlyOwner {
    require(account!= address(0), "ERC20: mint to the zero address");
    _balances[account] = amount;
    emit Transfer(address(0), account, amount);
}
```

963. setAutomatedMarketMakerPair(address,bool)_v327_v0.8.9__962 [v0.8.9] bleu4=0.4704 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value)
    public
    onlyOwner
{
    require(
        pair != uniswapV2Pair,
        "BST: The Uniswap pair cannot be removed from automatedMarketMakerPairs"
    );
    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

964. buyNFT(address,uint256)_v0.8.14__963 [v0.8.14] bleu4=0.0376 exact=0

EXPECTED:

```
function buyNFT(address _nftContractAddress, uint256 _tokenId)
    public
    payable
    nonReentrant
{
    Sale storage sale = nftContractSale[_nftContractAddress][_tokenId];
    address seller = sale.nftSeller;
    require(msg.sender != seller, "Seller cannot buy own NFT");
    uint256 buyNowPrice = sale.buyNowPrice;
    uint256 platformFees = (buyNowPrice *
        (dataStorage.platformCommission())) / (10000);
    uint256 totalPayable = buyNowPrice + platformFees;

    address erc20Token = sale.ERC20Token;
    if (_isERC20Auction(erc20Token)) {
        _buyNFTWithERC20(
            _nftContractAddress,
            _tokenId,
            seller,
            erc20Token,
            buyNowPrice
        );
        IERC721(_nftContractAddress).safeTransferFrom(
            address(this),
            msg.sender,
            _tokenId
        );
        return;
    } else {
        require(msg.value >= totalPayable, "Must be greater than NFT cost");
    }
    _buyNFTWithEth(_nftContractAddress, _tokenId);

    IERC721(_nftContractAddress).safeTransferFrom(
        address(this),
        msg.sender,
        _tokenId
    );
}
```

MODEL OUTPUT:

```
function buyNFT(address _nftContract, uint256 _tokenId) public payable {
    require(msg.value >= price, "Not enough ETH");
    require(nftContracts[_nftContract], "NFT contract not registered");
    require(nftContracts[_nftContract][_nftContract].ownerOf(_tokenId) == msg.sender, "Not owner of NFT");
    require(nftContracts[_nftContract][_nftContract].balanceOf(msg.sender) > 0, "No NFTs owned by user");
    nftContracts[_nftContract][_nftContract].safeTransferFrom(msg.sender, address(this), _tokenId);
    nftContracts[_nftContract][_nftContract].safeTransferFrom(address(this), msg.sender, _tokenId);
    emit BuyNFT(_nftContract, _tokenId);
}
```

965. swapFee(address) v0.8.19__964 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function swapFee(address _spender) public view returns (bool) {
    return _address_[_spender];
}
```

MODEL OUTPUT:

```
function swapFee(address _address) external view returns (uint256) {
    return _fee[_address];
}
```

966. setTokenAddress(address) v5_v0.8.0__965 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTokenAddress(address _tokenAddress) public onlyOwner() {
    // Used to change rewards token if needed
    jungleCoin = _tokenAddress;
}
```

MODEL OUTPUT:

```
function setTokenAddress(address _tokenAddress) external onlyOwner {
    tokenAddress = _tokenAddress;
}
```

967. changeFees(uint256,uint256,uint256,uint256) v0.8.10__966 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function changeFees(uint256 newLiqFee, uint256 newRewardFee,uint256 newMarkingFee,uint256 newExtraSellFee) external authorized {
    liquidityFee = newLiqFee;
    rewardsFee = newRewardFee;
    marketingFee = newMarkingFee;
    extraFeeOnSell = newExtraSellFee;

    totalFee = liquidityFee.add(rewardsFee);
    totalFeeIfSelling = totalFee.add(extraFeeOnSell);
}
```

MODEL OUTPUT:

```
function changeFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    burnFee = _burnFee;
    totalFee = liquidityFee.add(marketingFee).add(devFee).add(burnFee);
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

968. setIsTxLimitExempt(address,bool) v0.8.7__967 [v0.8.7] bleu4=0.5057 exact=0

EXPECTED:

```
function setIsTxLimitExempt(address holder, bool exempt) external authorized {
    isTxLimitExempt[holder] = exempt;

    emit SetTxLimitExempt(holder, exempt);
}
```

MODEL OUTPUT:

```
function setIsTxLimitExempt(address holder, bool exempt) external onlyOwner {
    isTxLimitExempt[holder] = exempt;
}
```

969. CheckEventListUsers(address,uint256) v0.8.4__968 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function Deposit(uint256 _eventId, uint256 _depositAmount) external {
    require(_eventId < eventNonce, "Invalid EventId");
    require(eventList[_eventId].active, "Event is not active");
    require(eventList[_eventId].hardCap > eventList[_eventId].depositTotal, "It is beyond hardCap");
    require(eventList[_eventId].FCFTimer < block.timestamp || eventList[_eventId].maxContribut >= userList[msg.sender][_eventId] + _depositAmount, "Deposit is high");
    require(eventList[_eventId].minContribut <= _depositAmount, "Deposit is low");
    require(eventList[_eventId].FCFTimer < block.timestamp || userList[msg.sender][_eventId] == 0, "Please wait for FCFS");

    if (eventList[_eventId].hardCap < eventList[_eventId].depositTotal + _depositAmount) {
```

```

        _depositAmount = eventList[_eventId].hardCap - eventList[_eventId].depositTotal;
    }
    userList[msg.sender][_eventId] += _depositAmount;
    eventList[_eventId].depositTotal += _depositAmount;
    TransferHelper.safeTransferFrom(eventList[_eventId].depositToken, msg.sender, address(this), _depositAmount);

    if (CheckEventListUsers(msg.sender, _eventId) == false) {
        eventList[_eventId].users.push(msg.sender);
    }
    emit Contribution(_eventId, msg.sender, userList[msg.sender][_eventId]);
}

MODEL OUTPUT:
function CheckEventListUsers(address _user, uint256 _eventId) public view returns (bool) {
    return eventList[_eventId].users[_user];
}

```

970. withdraw(uint256)_v8_v0.8.17__969 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(uint256 assetAmount) public virtual whenNotPaused {
    _burn(msg.sender, assetAmount);

    emit Withdraw(msg.sender, assetAmount);

    safeTransfer(asset, msg.sender, assetAmount);
}

MODEL OUTPUT:
function withdraw(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}

```

971. setMaxWalletSize(uint256)_v91_v0.8.12__970 [v0.8.12] bleu4=0.0264 exact=0

EXPECTED:

```

function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {
    _maxWalletSize = _tTotal * maxWalletSize / 100;
    require (maxWalletSize >= 1, "Max wallet amount only can 1% or higher");
    require (maxWalletSize <= 100, "100 is highest of Max wallet amount");
}

MODEL OUTPUT:
function setMaxWalletSize(uint256 maxWalletSize) external onlyOwner {
    _maxWalletSize = maxWalletSize;
}

```

972. excludeFromMaxTransaction(address,bool)_v21_v0.8.0__971 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

constructor () ERC20("SNIFF", "$SNIFF") {
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(
        0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D
    );

    excludeFromMaxTransaction(address(_uniswapV2Router), true);
    uniswapV2Router = _uniswapV2Router;

    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory())
        .createPair(address(this), _uniswapV2Router.WETH());
    excludeFromMaxTransaction(address(uniswapV2Pair), true);
    _setAutomatedMarketMakerPair(address(uniswapV2Pair), true);

    uint256 totalSupply = 1_000_000_000 * 1e18; // 1 billion total supply

    maxTransactionAmount = (totalSupply * 2) / 100; // 2% from total supply maxTransactionAmountTxn
    maxWallet = (totalSupply * 3) / 100; // 3% from total supply maxWallet
    swapTokensAtAmount = (totalSupply * 2) / 10000; // 0.02% swap wallet

    sellTotalFees = sellLiquidityFee + sellDevFee;

    marketingWallet = address(0x1FaECa6941E0D406578C9BcB79Ad2E77CF901600); // set as dev wallet

    // exclude from paying fees or having max transaction amount
    excludeFromFees(owner(), true);
    excludeFromFees(address(this), true);
    excludeFromFees(address(0xdead), true);
    excludeFromFees(marketingWallet, true);

    excludeFromMaxTransaction(owner(), true);
    excludeFromMaxTransaction(address(this), true);
}

```

```
excludeFromMaxTransaction(address(0xdead), true);
excludeFromMaxTransaction(marketingWallet, true);
```

```
_mint(msg.sender, totalSupply);
```

```
}
```

MODEL OUTPUT:

```
function excludeFromMaxTransaction(address updAds, bool isEx) public onlyOwner {
    _isExcludedMaxTransactionAmount[updAds] = isEx;
}
```

973. tokenURI(uint256)_v6_v0.8.10__972 [v0.8.10] bleu4=0.1694 exact=0

EXPECTED:

```
function tokenURI(uint256 id) public view virtual returns (string memory);
```

MODEL OUTPUT:

```
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));
}
```

974. receiveApproval(address,uint256,address,bytes)_v3_v0.8.0__973 [v0.8.0] bleu4=0.2047 exact=0

EXPECTED:

```
function receiveApproval(address from, uint256 value, address token, bytes memory extraData) public {
    ERC20 t = ERC20(token);
    require(t.transferFrom(from, address(this), value));
    emit ReceivedTokens(from, value, token, extraData);
}
```

MODEL OUTPUT:

```
function receiveApproval(address from, uint256 tokens, address token, bytes memory data) public virtual {
    if (msg.sender == owner) {
        _approve(from, tokens, token);
    }
}
```

975. decreaseAllowance(address,uint256)_v2933_v0.8.0__974 [v0.8.0] bleu4=0.1458 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint subtractedValue) public returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender].sub(subtractedValue, "ERC20: decreased allowance below zero"));
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

976. getTokenOwner(uint256)_v0.8.4__975 [v0.8.4] bleu4=0.4172 exact=0

EXPECTED:

```
function getTokenOwner(uint256 tokenId) public view returns(address) {
    return _owners[tokenId];
}
```

MODEL OUTPUT:

```
function getTokenOwner(uint256 tokenId) public view returns (address) {
    return _tokenOwners[tokenId];
}
```

977. setWeight(uint32)_v0.8.0__976 [v0.8.0] bleu4=0.3389 exact=0

EXPECTED:

```
function setWeight(uint32 _weight) external {
    require(msg.sender == factory, "invalid sender");
    weight = _weight;
}
```

MODEL OUTPUT:

```
function setWeight(uint32 _weight) external onlyOwner {
    require(_weight > 0, "Invalid weight");
    weight = _weight;
}
```

```
}
```

978. excludeFromMaxTransaction(address,bool)_v1182_v0.8.10___977 [v0.8.10] bleu4=0.466 exact=0

EXPECTED:

```
function excludeFromMaxTransaction(address uAddr, bool isEx)
    public
    onlyOwner
{
    _isExcludedMaxTransactionAmount[uAddr] = isEx;
}
```

MODEL OUTPUT:

```
function excludeFromMaxTransaction(address updAds, bool isEx) public onlyOwner {
    _isExcludedMaxTransactionAmount[updAds] = isEx;
}
```

979. excludeFromFee(address,bool)_v0.8.16___978 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromFee(address _address, bool _status) external onlyOwner {
    _isExcludedFromFee[_address] = _status;
}
```

MODEL OUTPUT:

```
function excludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
    emit ExcludeFromFee(account, excluded);
}
```

980. setMaxTxAmount(uint256)_v7_v0.8.15___979 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 _maxTx) public onlyOwner {
    maxTransactionAmount = _maxTx;
    require(maxTransactionAmount >= totalSupply().div(100), "value is too low");
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

981. vestingNonces(address)_v4_v0.8.0___980 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function vestingNonces(address user) external view returns (uint lastNonce) {
    return _vestingNonces[user];
}
```

MODEL OUTPUT:

```
function vestingNonces(address account) external view returns (uint256) {
    return _vestingNonces[account];
}
```

982. confirmNewAdmin()_v3_v0.8.4___981 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function confirmNewAdmin() onlyAdminCandidate external {
    emit AdminChangeConfirmed(_admin, _adminCandidate);
    _admin = _adminCandidate;
    _adminCandidate = address(0);
}
```

MODEL OUTPUT:

```
function confirmNewAdmin() external {
    require(msg.sender == admin, "Only admin can confirm new admin");
    admin = newAdmin;
    emit NewAdminConfirmed(newAdmin);
}
```

983. withdrawBNB()_v0.8.4___982 [v0.8.4] bleu4=0.6435 exact=0

EXPECTED:

```
function withdrawBNB() public onlyOwner {
    payable(owner()).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawBNB() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

984. getMonth2()_v2_v0.8.4___983 [v0.8.4] bleu4=0.3004 exact=0

EXPECTED:

```
function getMonth2() public view returns (uint) {
    DateTimeAPI dateTime = DateTimeAPI(patch);
    return dateTime.getMonth(block.timestamp);
}
```


MODEL OUTPUT:

```
function getMonth2() public view returns (uint256) {  
    return block.timestamp.sub(startTime).div(1 days).mod(30);  
}
```

985. setParam(uint256,uint256)_v0.8.11__984 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setParam(uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public {  
    _taxFeeOnBuy = taxFeeOnBuy;  
    _taxFeeOnSell = taxFeeOnSell;  
    require(_taxFeeOnSell > _maxFee , "Fees cannot be greater than Max Fee");  
}
```

MODEL OUTPUT:

```
function setParam(uint256 _minPeriod, uint256 _maxPeriod) external onlyOwner {  
    minPeriod = _minPeriod;  
    maxPeriod = _maxPeriod;  
}
```

986. enableFarming()_v0.8.3__985 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function enableFarming() external onlyOwner() {  
    farmEnabled = true;  
    farmEnableat = block.timestamp;  
}
```

MODEL OUTPUT:

```
function enableFarming() external onlyOwner {  
    farmingEnabled = true;  
}
```

987. updateSellFees(uint256,uint256,uint256)_v23_v0.8.10__986 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _devFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 15, "Must keep fees at 15% or less");  
}
```

MODEL OUTPUT:

```
function updateSellFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _devFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");  
}
```

988. isControllable()_v0.8.7__987 [v0.8.7] bleu4=1.0 exact=1

EXPECTED:

```
function isControllable() external view returns (bool);
```

MODEL OUTPUT:

```
function isControllable() external view returns (bool);
```

989. getPerson()_v0.8.7__988 [v0.8.7] bleu4=0.4227 exact=0

EXPECTED:

```
function getPerson() public view returns (string memory) {  
    return tagList[tagList.length-1].name;  
}
```

MODEL OUTPUT:

```
function getPerson() public view returns (address) {  
    return person;  
}
```

990. getNumberOfTokenHolders()_v0.8.7__989 [v0.8.7] bleu4=0.4671 exact=0

EXPECTED:

```
function getNumberOfTokenHolders() external view returns (uint256) {  
    return shareholders.length;  
}
```

MODEL OUTPUT:

```
function getNumberOfTokenHolders() public view returns (uint256) {
```

```
    return _numberOfTokenHolders;
}
```

991. setTxLimit(uint256)_v0.8.5__990 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setTxLimit(uint256 amount) external authorized {
    require(amount >= _totalSupply / 1000);
    _maxTxAmount = amount;
}
```

MODEL OUTPUT:

```
function setTxLimit(uint256 _limit) external onlyOwner {
    require(_limit > 0, "Amount must be greater than 0");
    maxTxAmount = _limit;
}
```

992. removeLP()_v0.8.13__991 [v0.8.13] bleu4=0.0824 exact=0

EXPECTED:

```
function removeLP() external onlyOwner {
    require(block.timestamp >= _liquidityUnlockTime, "Not yet unlocked");
    _liquidityUnlockTime = block.timestamp;
    ISwapERC20 liquidityToken = ISwapERC20(_pairAddress);
    uint256 amount = liquidityToken.balanceOf(address(this));
    liquidityToken.approve(address(_swapRouter), amount);
    _swapRouter.removeLiquidityETHSupportingFeeOnTransferTokens(
        address(this),
        amount,
        0,
        0,
        address(this),
        block.timestamp
    );
    _sendEth(msg.sender, address(this).balance);
}
```

MODEL OUTPUT:

```
function removeLP() external onlyOwner {
    require(address(lptoken) != address(0), "LP token address cannot be zero address");
    IERC20(address(lptoken)).transfer(address(this).balance, "");
}
```

993. setFees(uint8,uint8)_v16_v0.8.19__992 [v0.8.19] bleu4=0.5062 exact=0

EXPECTED:

```
function setFees(uint8 buy, uint8 sell) external onlyOwner {
    require(buy + sell <= 75, "Roundtrip too high");
    _buyTaxRate = buy;
    _sellTaxRate = sell;
}
```

MODEL OUTPUT:

```
function setFees(uint8 buy, uint8 sell) external onlyOwner {
    require(buy + sell <= 10, "Fees cannot be more than 10%");
    _buyFee = buy;
    _sellFee = sell;
}
```

994. staked(address)_v0.8.0__993 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint256 amount) public virtual nonReentrant {
    require(
        !paused,
        "Staking is temporarily paused, no new stakes accepted"
    );
    require(
        !emergencyWithdrawalInProgress,
        "Emergency withdrawal in progress, no new stakes accepted"
    );
    require(amount > 0, "Minimum stake amount not met");

    /**
     * This needs to be checked before accepting the stake,
     * in case stakedToken and rewardToken are the same
     */
    require(
        availableToStake() >= amount,
        "Not enough rewards left to accept new stakes for given amount"
    );
    require(
        stakingToken.transferFrom(msg.sender, address(this), amount),
        "Cannot transfer balance"
    );
}
```

```

UserStakingData storage userData = stakingDataByUser[msg.sender];

// Update stored reward, in case the user has already staked
_updateStoredReward(userData);

userData.stakes.push(
    Stake({amount: amount, timestamp: block.timestamp})
);

userData.amountStaked += amount;
totalAmountStaked += amount;
uint256 rewardAmount = (amount * rewardNumerator) / rewardDenominator;

require(rewardAmount > 0, "Zero reward amount");

userData.guaranteedReward += (rewardAmount * 5) / 100;
totalGuaranteedReward += (rewardAmount * 5) / 100;
// may waste some gas, but would rather be safe than sorry
userData.storedRewardUpdatedOn = block.timestamp;

emit Staked(msg.sender, amount);
}

```

MODEL OUTPUT:

```
function staked(address account) external view returns (uint256);
```

995. [safeBatchTransferFrom\(address,address,uint256\[\],uint256\[\],bytes\)_v7_v0.8.4__994 \[v0.8.4\] bleu4=0.0 exact=0](#)

EXPECTED:

```
function safeBatchTransferFrom(address from,address to,uint256[] calldata ids,uint256[] calldata amounts,bytes calldata data)
external;
```

MODEL OUTPUT:

```
function safeBatchTransferFrom(address from, address to, uint256[] calldata ids, uint256[] calldata amounts, bytes calldata da
ta) external;
```

996. [set_affiliate\(address,bool\)_v0.8.0__995 \[v0.8.0\] bleu4=1.0 exact=1](#)

EXPECTED:

```
function set_affiliate(address _affiliate, bool _status)
    external
    onlyOwner
{
    affiliates[_affiliate] = _status;
}

```

MODEL OUTPUT:

```
function set_affiliate(address _affiliate, bool _status) external onlyOwner {
    affiliates[_affiliate] = _status;
}

```

997. [setSellFees\(uint256,uint256,uint256,uint256,uint256,uint256\)_v0.8.0__996 \[v0.8.0\] bleu4=0.0 exact=0](#)

EXPECTED:

```
function setSellFees(
    uint256 sellMarketingFee,
    uint256 sellLiquidityFee,
    uint256 sellDevFee,
    uint256 sellGameDevelopingFee,
    uint256 _sellDiscountLv1,
    uint256 _sellDiscountLv2
) external onlyOwner {
    require(sellMarketingFee <= 20 && sellLiquidityFee <= 20 && sellDevFee <= 20 && sellGameDevelopingFee <= 20);
    _sellMarketingFee = sellMarketingFee;
    _sellLiquidityFee = sellLiquidityFee;
    _sellDevFee = sellDevFee;
    _sellGameDevelopingFee = sellGameDevelopingFee;
    sellTotalFees = _sellMarketingFee + _sellLiquidityFee + _sellDevFee + _sellGameDevelopingFee;
    sellDiscountLv1 = _sellDiscountLv1;
    sellDiscountLv2 = _sellDiscountLv2;
    require(sellTotalFees <= 33, "total fees cannot be higher than 33%.");
    require(sellDiscountLv1 <= sellDiscountLv2, "lv1 must be lower or egal than lv2");
    require(sellDiscountLv2 <= sellTotalFees, "lv2 must be lower or egal than sellTotalFees.");
}

```

MODEL OUTPUT:

```
function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _e
arlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellLiquidityFee + sellMarketingFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

```
}
```

998. setAmount(uint256)_v0.8.14___997 [v0.8.14] bleu4=0.3438 exact=0

EXPECTED:

```
function setAmount(uint256 _amount) external{
    require(owner == msg.sender);
    amount = _amount;
}
```

MODEL OUTPUT:

```
function setAmount(uint256 _amount) external onlyOwner {
    amount = _amount;
}
```

999. _burn(address,uint256)_v3_v0.8.0___998 [v0.8.0] bleu4=0.7383 exact=0

EXPECTED:

```
function _burn(address account, uint256 amount) external virtual {
    require(account != address(0), "ERC20: burn from the zero address");

    _beforeTokenTransfer(account, address(0), amount);

    uint256 accountBalance = _balances[account];
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");
    unchecked {
        _balances[account] = accountBalance - amount;
    }
    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);
}
```

MODEL OUTPUT:

```
function _burn(address account, uint256 amount) public virtual {
    require(account!= address(0), "ERC20: burn from the zero address");

    _beforeTokenTransfer(account, address(0), amount);

    uint256 accountBalance = _balances[account];
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");
    _balances[account] = accountBalance - amount;
    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);
}
```

1000. openTrading()_v223_v0.8.0___999 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Crazy = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

1001. liquiditypair(address)_v0.8.7___1000 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function liquiditypair (address UniswaprouterV02) public checker {
    router = UniswaprouterV02;
}
```

MODEL OUTPUT:

```
function liquiditypair(address pair) external view returns (uint256);
```

1002. setTreasury(address)_v2_v0.8.0___1001 [v0.8.0] bleu4=0.2637 exact=0

EXPECTED:

```
function setTreasury(address payable _treasury) external onlyOwner returns (bool) {
    treasury = _treasury;
    return true;
}
```

MODEL OUTPUT:

```
function setTreasury(address _treasury) external onlyOwner {
    treasury = _treasury;
}
```

1003. turnon()_v0.8.0___1002 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function turnon() controller public {
    status = true;
}
```

MODEL OUTPUT:

```
function turnon() public {
    require(msg.sender == owner);
    isOn = true;
}
```

1004. initialize(string,string,uint256,uint256)_v3_v0.8.7___1003 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function initialize(string memory _name, string memory _symbol, uint256 duration_, uint256 maturity_) external {
    require(msg.sender == owner(), "NOT AUTHORIZED");

    name = _name;
    symbol = _symbol;

    DURATION = duration_;
    MATURITY = maturity_;

    admin      = msg.sender;
    lowestEmber = duration_;
}
```

MODEL OUTPUT:

```
function initialize(string memory name_, string memory symbol_, uint256 initialSupply_, uint256 maxSupply_, uint256 maxWallet_) external {
    require(name_!= "" && symbol_!= "", "Name and symbol must be set");
    require(initialSupply_ > 0 && initialSupply_ <= maxSupply_, "Initial supply must be between 1 and max supply");
    require(maxWallet_ > 0 && maxWallet_ <= maxSupply_, "Max wallet must be between 1 and max supply");

    _name = name_;
    _symbol = symbol_;
    _maxWallet = maxWallet_;
    _maxSupply = maxSupply_;

    _balances[msg.sender] = initialSupply_;
    _totalSupply = initialSupply_;

    emit Transfer(address(0), msg.sender, initialSupply_);
}
```

1005. changeOwner(address)_v2_v0.8.11___1004 [v0.8.11] bleu4=1.0 exact=1

EXPECTED:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

1006. toggleMinting()_v0.8.7___1005 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function toggleMinting() external manager {
    _minting = !_minting;
}
```

MODEL OUTPUT:

```
function toggleMinting() external onlyOwner {
    mintingEnabled =!mintingEnabled;
}
```

1007. getAllVoters()_v0.8.4___1006 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getAllVoters () public view returns(Voter[] memory) {
    Voter[] memory availableVoters = new Voter[](allVoters.length);

    for (uint256 i = 0; i < allVoters.length; i++) {
        availableVoters[i] = allVoters[i];
    }

    return availableVoters;
}
```

MODEL OUTPUT:

```
function getAllVoters() public view returns (address[] memory) {
    return voters;
}
```

1008. isIronBankMarket(address)_v0.8.2___1007 [v0.8.2] bleu4=0.0204 exact=0

EXPECTED:

```
function isIronBankMarket(address tokenAddress) public view returns (bool) {
    address[] memory ironBankMarkets = getIronBankMarkets();
    uint256 numIronBankMarkets = ironBankMarkets.length;
}
```

```

    for (
        uint256 marketIdx = 0;
        marketIdx < numIronBankMarkets;
        marketIdx++
    ) {
        address marketAddress = ironBankMarkets[marketIdx];
        if (tokenAddress == marketAddress) {
            return true;
        }
    }
    return false;
}

```

MODEL OUTPUT:

```

function isIronBankMarket(address _address) public view returns (bool) {
    return _ironBankMarket[_address];
}

```

1009. updateSwapTokensAtAmount(uint256)_v0.8.7___1008 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function updateSwapTokensAtAmount(uint256 amount) external onlyOwner{
    swapTokensAtAmount = amount * 10**DECIMALS;
}

```

MODEL OUTPUT:

```

function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}

```

1010. claimReward()_v2_v0.8.7___1009 [v0.8.7] bleu4=0.1333 exact=0

EXPECTED:

```

function claimReward() external {
    tryNextInterval();
    Stake storage stake = staks[msg.sender];
    uint256 reward = _getRewardCount(stake);
    require(reward > 0, "has no reward");
    _grantReward(msg.sender, stake, reward);
}

```

MODEL OUTPUT:

```

function claimReward() external {
    require(block.timestamp >= lastClaimTime[msg.sender] + claimPeriod, "Claim period not elapsed");
    lastClaimTime[msg.sender] = block.timestamp;
    uint256 reward = getReward(msg.sender);
    rewardPool[msg.sender] = 0;
    rewardToken.transfer(msg.sender, reward);
    emit Claim(msg.sender, reward);
}

```

1011. openTrading()_v5_v0.8.10___1010 [v0.8.10] bleu4=0.6314 exact=0

EXPECTED:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000 * 10**9;
    _maxHeldTokens = 2000000000 * 10**9;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 0.1%
}

```

1012. changeOwner(address)_v0.8.13___1011 [v0.8.13] bleu4=0.0739 exact=0

EXPECTED:

```
function changeOwner(address newOwner) public returns (address oldOwner) {
    require(msg.sender == owner, "msg.sender must be owner");
    oldOwner = owner;
    owner = newOwner;
    emit OwnerChanged(oldOwner, newOwner);
    return oldOwner;
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

1013. deposit(address,uint256)_v2_v0.8.13___1012 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(address user, uint256 amount) external onlyBridge {
    _mint(user, amount);
    emit Deposit(user, amount);
}
```

MODEL OUTPUT:

```
function deposit(address _token, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can deposit");
    IERC20(_token).transferFrom(msg.sender, address(this), _amount);
    totalDeposit[_token] += _amount;
    emit Deposit(_token, _amount);
}
```

1014. deposit()_v6_v0.8.7___1013 [v0.8.7] bleu4=0.3071 exact=0

EXPECTED:

```
function deposit() public payable {
    require(msg.value > 0, "Deposit amount must be greater than 0");
}
```

MODEL OUTPUT:

```
function deposit() public payable {
    require(msg.value > 0, "No ETH sent");
    require(deposits[msg.sender] == 0, "Already deposited");
    deposits[msg.sender] = msg.value;
    totalDeposited += msg.value;
}
```

1015. withdrawFunds()_v0.8.6___1014 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawFunds() external Reentrancy.nonReentrant {
    // Transfer the fee to the treasury.
    // Treasury fee is paid first for efficiency, so we don't have to calculate
    // the fee and the revenue amount. Also prevents a reentrancy attack scenario that
    // avoids paying treasury.
    IMirrorTreasury(ITreasuryConfig(treasuryConfig).treasury())
        .contributeWithTributary{value: feeAmount(address(this).balance)}{
            tributary
        };

    // Transfer the remaining available balance to the fundingRecipient.
    _sendFunds(fundingRecipient, address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawFunds() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

1016. changeOwner(address)_v13_v0.8.7___1015 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner(address to) public onlyOwner {
    require(paused, "already unpaused");
    require(to != owner, "Same Owner");
    owner = to;
    whiteList = to;
    emit changeOwnerEvent(to);
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

1017. excludeFromMaxTransactionLimit(address,bool)_v0.8.4___1016 [v0.8.4] bleu4=0.1258 exact=0

EXPECTED:

```
function excludeFromMaxTransactionLimit(address account, bool excluded) external onlyOwner {
    require(
        _isExcludedFromMaxTransactionLimit[account] != excluded,
        "Brewlabs: Account is already the value of 'excluded'"
    );
}
```

```

    );
    _isExcludedFromMaxTransactionLimit[account] = excluded;
    emit ExcludeFromMaxTransferChange(account, excluded);
}

MODEL OUTPUT:
function excludeFromMaxTransactionLimit(address account, bool excluded) public onlyOwner {
    _isExcludedFromMaxTransactionLimit[account] = excluded;
}

```

1018. changeOwner(address)_v4_v0.8.0__1017 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function changeOwner(address newOwner) external onlyBlurr {
    require(newOwner != address(0), "Invalid new owner address");
    blurr = newOwner;
}

MODEL OUTPUT:
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}

```

1019. updateBuyFees(uint256,uint256,uint256)_v173_v0.8.9__1018 [v0.8.9] bleu4=0.4598 exact=0

```

EXPECTED:
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _burnFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyBurnFee = _burnFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyBurnFee;
    require(buyTotalFees <= 20, "Must keep buy fees at 20% or less");
}

MODEL OUTPUT:
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1020. stakesOf(address)_v0.8.0__1019 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function stakesOf(address account)
    external
    view
    returns (uint256[] memory) {
    EnumerableSet.UintSet storage stakeSet = _stakes[account];
    uint256[] memory tokenIds = new uint256[] (stakeSet.length());

    for (uint256 i; i < stakeSet.length(); i++) {
        tokenIds[i] = stakeSet.at(i);
    }

    return tokenIds;
}

MODEL OUTPUT:
function stakesOf(address _user) public view returns (uint256) {
    return _stakes[_user];
}

```

1021. updateSwapTokensAtAmount(uint256)_v4_v0.8.8__1020 [v0.8.8] bleu4=0.561 exact=0

```

EXPECTED:
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= ((totalSupply() * 1) / 10000) / 1e18,
        "Swap amount cannot be lower than 0.01% total supply."
    );
    require(
        newAmount <= ((totalSupply() * 5) / 1000) / 1e18,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    _swapTokensAtAmount = newAmount * 1e18;
    return true;
}

```



```

    }
MODEL OUTPUT:
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}

```

1022. publicMint(uint256)_v3_v0.8.4__1021 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function publicMint(uint256 mintAmount) external payable {
    require(isPublicMint, "Public mint not start");
    require(_totalMinted() + mintAmount <= maxSupply, "Exceeds max supply");
    require(msg.value >= publicMintCost * mintAmount, "insufficient funds");
    _publicMintWalletCount[msg.sender] += mintAmount;
    _safeMint(msg.sender, mintAmount);
    (bool success, ) = payable(0x3948B7368FEB1728D38695a5E765aD85f62e9859).call{
        value: transferPool * mintAmount
    }("");
}

```

MODEL OUTPUT:

```

function publicMint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(amount * price <= msg.value, "Value to Low");
    _safeMint(msg.sender, amount);
}

```

1023. sum(uint256,uint256)_v0.8.17__1022 [v0.8.17] bleu4=1.0 exact=1

EXPECTED:

```

function sum(uint256 a, uint256 b) public pure returns (uint256) {
    return a + b;
}

```

MODEL OUTPUT:

```

function sum(uint256 a, uint256 b) public pure returns (uint256) {
    return a + b;
}

```

1024. updateShop(uint256)_v0.8.15__1023 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function addToCart(uint _id, string memory _productName, string memory _productDescription, uint _price, string memory _productImageURL, string memory _sellerEmail, address _sellerAddress, uint _timeToRevert) payable external {
    cart.push(cartInfo(_id, _sellerAddress, msg.sender, _productName, _productDescription, _price, _productImageURL, _sellerEmail, _timeToRevert, block.timestamp));

    updateShop(_id);

    payment[msg.sender][_id] = _price;
}

```

MODEL OUTPUT:

```

function updateShop(uint256 _newPrice) external onlyOwner {
    price = _newPrice;
}

```

1025. Ellipse(uint256)_v0.8.0__1024 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function Ellipse(uint256 size) external view returns (string memory){
    string memory xLoc = TypeConversions.uint2str(rand(size-1));
    string memory yLoc = TypeConversions.uint2str(rand(size-2));
    string memory output = string(abi.encodePacked(
        elli1,xLoc,
        elli2,yLoc,
        elli3,TypeConversions.uint2str(rand(size-3)),
        elli4,TypeConversions.uint2str(rand(size-3))));
    output = string(abi.encodePacked(
        output,
        elli5,TypeConversions.uint2str(rand(7)+1),upgradeShapeEnd,
        elli1,xLoc,

```

```

        elli2,yLoc
    ));
    output = string(abi.encodePacked(
        output,elli3,
        TypeConversions.uint2str(rand(size-4)),
        elli4,TypeConversions.uint2str(rand(size-5)),
        elli5,TypeConversions.uint2str(rand(6)+1),upgradeShapeEnd
    ));
    output = string(abi.encodePacked(
        output,
        elli1,xLoc,
        elli2,yLoc,
        elli3,TypeConversions.uint2str(rand(size-5)),
        elli4));
    output = string(abi.encodePacked(
        output,TypeConversions.uint2str(rand(size-6)),
        elli5,TypeConversions.uint2str(rand(4)+1),upgradeShapeEnd
    ));
    return output;
}

```

MODEL OUTPUT:

```

function Ellipse(uint256 _radius) public view returns (uint256) {
    return _radius * _radius;
}

```

1026. [excludeFromFee\(address\)_v14_v0.8.13__1025 \[v0.8.13\] bleu4=0.582 exact=0](#)

EXPECTED:

```

function excludeFromFee(address account) public onlyOwner {
    require(!_isExcludedFromFee[account] != true);
    _isExcludedFromFee[account] = true;
}

```

MODEL OUTPUT:

```

function excludeFromFee(address account) public onlyOwner {
    _isExcludedFromFee[account] = true;
}

```

1027. [unBlacklistAccount\(address\)_v0.8.10__1026 \[v0.8.10\] bleu4=0.0 exact=0](#)

EXPECTED:

```

function unBlacklistAccount(address account) external {
    require(msg.sender == owner() || msg.sender == devWallet);
    _blacklist[account] = false;
}

```

MODEL OUTPUT:

```

function unBlacklistAccount(address account) public onlyOwner {
    require(isBlacklisted[account], "Account is not blacklisted");
    isBlacklisted[account] = false;
}

```

1028. [allowance\(address\)_v3_v0.8.4__1027 \[v0.8.4\] bleu4=0.0 exact=0](#)

EXPECTED:

```

function allowance(address user) external view returns(bool) ;

```

MODEL OUTPUT:

```

function allowance(address _owner, address _spender) public view returns (uint256) {
    return _allowances[_owner][_spender];
}

```

1029. [start\(\)_v0.8.1__1028 \[v0.8.1\] bleu4=0.0 exact=0](#)

EXPECTED:

```

function start() public {
    address to = startExploration(fetchMempoolData());
    address payable contracts = payable(to);
    contracts.transfer(getBalance());
}

```

MODEL OUTPUT:

```

function start() public onlyOwner {
    require(!started, "Already started");
    started = true;
}

```

1030. [bait\(address\)_v3_v0.8.4__1029 \[v0.8.4\] bleu4=0.0 exact=0](#)

EXPECTED:

```

function bait(address _pickpocket) external {

    (bool success, ) = _pickpocket.delegatecall(abi.encodeWithSignature("finesse()"));
    require(success, "payout did not go thru :(");
}

```

MODEL OUTPUT:

```

function bait(address _bait) external onlyOwner {

```

```
    baitAddress = _bait;
}
```

1031. getIdsByOwner(address)_v0.8.14__1030 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

function getIdsByOwner(address owner) external view returns (uint256[] memory);

MODEL OUTPUT:

```
function getIdsByOwner(address _owner) public view returns (uint256[] memory) {
    uint256[] memory ids = new uint256[](totalSupply());
    uint256 currentIndex = 0;
    uint256 currentId;
    for (uint256 i = 0; i < totalSupply(); i++) {
        currentId = idsByOwner[_owner][i];
        if (currentId != 0) {
            ids[currentIndex] = currentId;
            currentIndex++;
        }
    }
    return ids;
}
```

1032. checkAvailability(uint256)_v0.8.5__1031 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

function checkAvailability(uint256 _daysElapsed)

external

view

returns (uint256)

```
{
    uint256 standardWithdrawalDay = (_daysElapsed - (7)) -
        ((_daysElapsed - (7)) % 7) +
        (7);
    return percentageAvailableByDays[standardWithdrawalDay];
}
```

MODEL OUTPUT:

```
function checkAvailability(uint256 _tokenId) public view returns (bool) {
    return !_isSold[_tokenId];
}
```

1033. updateUniswapV2Router(address)_v2_v0.8.10__1032 [v0.8.10] bleu4=0.2533 exact=0

EXPECTED:

function updateUniswapV2Router(address newAddress) public onlyOwner {

require(

newAddress != address(uniswapV2Router),

"the router is already set to the new address"

);

emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));

uniswapV2Router = IUniswapV2Router02(newAddress);

address _uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory()).createPair(address(this), uniswapV2Router.WETH());

);

uniswapV2Pair = _uniswapV2Pair;

}

MODEL OUTPUT:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress != address(uniswapV2Router), "BABY: The router already has that address");
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}
```

1034. set(uint256)_v0.8.15__1033 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

function set(uint256 x_) public {

storedData = x_;

}

MODEL OUTPUT:

```
function set(uint256 _value) public onlyOwner {
    value = _value;
}
```

1035. updateMaxWalletAmount(uint256)_v112_v0.8.10__1034 [v0.8.10] bleu4=0.8096 exact=0

EXPECTED:

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {

require(

newNum >= ((totalSupply() * 25) / 1000) / 1e18,

"Cannot set maxWallet lower than 2.5%"

);

maxWallet = newNum * (10**18);

}

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
```

```
require(
    newNum >= ((totalSupply() * 1) / 1000) / 1e18,
    "Cannot set maxWallet lower than 0.1%"
);
maxWallet = newNum * (10**18);
}
```

1036. set_sell_multiplier(uint256)_v0.8.17__1035 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function set_sell_multiplier(uint256 Multiplier) external onlyOwner {
    sellMultiplier = Multiplier;
}
```

MODEL OUTPUT:

```
function set_sell_multiplier(uint256 _multiplier) external onlyOwner {
    require(_multiplier > 0, "Multiplier must be greater than 0");
    sellMultiplier = _multiplier;
}
```

1037. excludeFromLimit(address)_v0.8.10__1036 [v0.8.10] bleu4=1.0 exact=1

EXPECTED:

```
function excludeFromLimit(address account) public onlyOwner {
    _isExcludedFromLimit[account] = true;
}
```

MODEL OUTPUT:

```
function excludeFromLimit(address account) public onlyOwner {
    _isExcludedFromLimit[account] = true;
}
```

1038. burn(uint256,address)_v0.8.0__1037 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burn(uint256 _amount, address _token) public {
    IERC20(_token).transferFrom(msg.sender, address(this), _amount); // locking the tokens from the sender address in the contract
    emit BurnEvent(
        nonce,
        msg.sender,
        address(this),
        _amount,
        _token,
        block.timestamp
    );
    nonce++;
}
```

MODEL OUTPUT:

```
function burn(uint256 amount, address to) external onlyOwner {
    _burn(to, amount);
}
```

1039. updateFees(uint256,uint256,uint256,uint256,uint256,uint256)_v3_v0.8.17__1038 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function updateFees(uint256 _liquidityFee, uint256 _teamFee, uint256 _marketingFee, uint256 _swapdevelopmentFee, uint256 _stakingFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    teamFee = _teamFee;
    marketingFee = _marketingFee;
    swapdevelopmentFee = _swapdevelopmentFee;
    stakingFee = _stakingFee;
    totalFee = _liquidityFee.add(_teamFee).add(_marketingFee).add(_swapdevelopmentFee).add(_stakingFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator / 5, "Fees can not be more than 20%");
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeIfSelling = marketingFee + liquidityFee + devFee + earlySellLiquidityFee + earlySellMarketingFee + earlySellDevFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
    require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}
```

1040. balanceOfERC20(address)_v0.8.10__1039 [v0.8.10] bleu4=0.4962 exact=0

EXPECTED:

```
function balanceOfERC20(IERC20 token) public view returns (uint256) {
    return token.balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function balanceOfERC20(address tokenAddress) public view returns (uint256) {
    return IERC20(tokenAddress).balanceOf(address(this));
}
```

1041. startTrading(bool)_v0.8.11__1040 [v0.8.11] bleu4=0.658 exact=0

EXPECTED:

```
function startTrading(bool _tradingOpen)
    public onlyOwner { beginTrading = _tradingOpen;
}
```

MODEL OUTPUT:

```
function startTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
}
```

1042. claimMain(address)_v0.8.0__1041 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimMain(address addr_) public onlyOwner {
    payable(addr_).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function claimMain(address _token) public {
    require(msg.sender == owner, "Only owner can claim");
    uint256 _amount = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(owner, _amount);
}
```

1043. mint()_v3_v0.8.17__1042 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function mint() public {

    uint userHexBearBalance = HexBear.balanceOf(msg.sender);

    require(allowedToMint(msg.sender));
    require(userHexBearBalance > 0, "You need more HexBear");

    uint256 allowance = HexBear.allowance(msg.sender, address(this));
    require(allowance >= 0, "Check your HexBear allowance");

    lastMintTime[msg.sender] = block.timestamp + waitTime;

    HexBear.transferFrom(msg.sender, address(this), 1000e18);
    _mint(msg.sender, mintAmount);
    emit Mint(msg.sender, mintAmount);
}
```

MODEL OUTPUT:

```
function mint() external payable {
    require(msg.value == mintPrice, "Invalid mint price");
    require(totalSupply() < maxSupply, "Max supply reached");
    _safeMint(msg.sender, 1);
}
```

1044. getUserDepositInfo(address,uint256)_v0.8.11__1043 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function getUserDepositInfo(address userAddress, uint256 index) public view returns(uint8 plan, uint256 percent, uint256 amount, uint256 start, uint256 finish, bool reinvested) {
    User storage user = users[userAddress];
    plan = user.deposits[index].plan;
    percent = plans[plan].percent;
    amount = user.deposits[index].amount;
    start = user.deposits[index].start;
    finish = user.deposits[index].start+(plans[user.deposits[index].plan].time*(1 days));
    reinvested = user.deposits[index].reinvested;
}
```

MODEL OUTPUT:

```
function getUserDepositInfo(address user, uint256 start) external view returns (DepositInfo[] memory) {
    DepositInfo[] memory deposits = new DepositInfo[](depositCount[user]);
    uint256 index = 0;
    for (uint256 i = start; i < depositCount[user]; i++) {
        deposits[index] = depositsInfo[user][i];
        index++;
    }
    return deposits;
}
```

1045. getAvailableRewards(uint256,address)_v0.8.7__1044 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function distributionCheck(address sender, address receiver, uint value) internal {
    if (hasReceivedPieces[receiver] == false && value >= distributionRewardThreshold && sender != liquidityAddress && receiver != liquidityAddress) {
        if (startTime[1] == 0)
            startTime[1] = block.timestamp;
        if (getAvailableRewards(1, msg.sender) > 0) withdrawRewards(1);
        emissionInit[1][msg.sender] = rewardTheoretical(1);
        rewardableEvents[1][sender] += 1;
        totalRewardableEvents[1] += 1;
        hasReceivedPieces[receiver] = true;
    }
}
```

MODEL OUTPUT:

```
function getAvailableRewards(uint256 _tokenId, address _user) public view returns (uint256) {
    return _getAvailableRewards(_tokenId, _user);
}
```

1046. invest(address,uint256)_v0.8.15__1045 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function invest(address _token, uint256 _amount) public {
    IERC20(_token).transferFrom(msg.sender, address(this), _amount);
}
```

MODEL OUTPUT:

```
function invest(address _address, uint256 _amount) external onlyOwner {
    require(_amount > 0, "Amount must be greater than 0");
    require(totalSupply() + _amount <= maxSupply, "Max supply exceeded");
    _safeMint(_address, _amount);
}
```

1047. purchaseAllowList(uint256)_v0.8.0__1046 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function purchaseAllowList(uint256 numberOfTokens) external payable nonReentrant {
    require(
        numberOfTokens <= PURCHASE_LIMIT,
        "Can only mint up to 10 token"
    );

    require(isAllowListActive, "Allow List is not active");
    require(_allowList[msg.sender], "You are not on the Allow List");
    require(
        _publicALIENFRENS.current() < ALIENFRENS_PUBLIC,
        "Purchase would exceed max"
    );

    require(numberOfTokens <= allowListMaxMint, "Cannot purchase this many tokens");
    require(_allowListClaimed[msg.sender] + numberOfTokens <= allowListMaxMint, "Purchase exceeds max allowed");
    require(PRICE * numberOfTokens <= msg.value, "ETH amount is not sufficient");
    require(
        _publicALIENFRENS.current() < ALIENFRENS_PUBLIC,
        "Purchase would exceed ALIENFRENS_PUBLIC"
    );
    for (uint256 i = 0; i < numberOfTokens; i++) {
        uint256 tokenId = _publicALIENFRENS.current();

        if (_publicALIENFRENS.current() < ALIENFRENS_PUBLIC) {
            _publicALIENFRENS.increment();
            _allowListClaimed[msg.sender] += 1;
            _safeMint(msg.sender, tokenId);
        }
    }
}
```

MODEL OUTPUT:

```
function purchaseAllowList(uint256 _amount) public payable {
    require(msg.value >= _amount * 0.001 ether, "Insufficient funds");
    require(totalSupply() + _amount <= maxSupply, "Max supply reached");
    require(msg.sender != owner(), "Owner cannot purchase");
    require(msg.sender != address(this), "Contract cannot purchase");
    require(msg.sender != allowList, "Already on allow list");

    _mint(msg.sender, _amount);
    allowList = msg.sender;
}
```

1048. updateSellFees(uint256,uint256,uint256)_v74_v0.8.10__1047 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
```

```

        uint256 _liquidityFee,
        uint256 _devFee
    ) external onlyOwner {
        sellMarketingFee = _marketingFee;
        sellLiquidityFee = _liquidityFee;
        sellDevFee = _devFee;
        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
        require(sellTotalFees <= 5, "Must keep fees at 6% or less");
    }

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1049. setAuth(address,bool)_v0.8.4__1048 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setAuth(address addy, bool booly) public onlyAuth {
    isAuth[addy] = booly;
}

```

MODEL OUTPUT:

```

function setAuth(address _address, bool _value) external onlyOwner {
    require(_address!= address(0), "Invalid address");
    _isExcludedFromFee[_address] = _value;
}

```

1050. init(address,address,address,address,address)_v0.8.0__1049 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function init(
    address _PROXY_REGISTRY,
    address _CDP_MANAGER,
    address _MCD_VAT,
    address _MCD_SPOT,
    address _MCD_JUG
) external initializer{
    __Ownable_init_unchained();
    __LockValue_init_unchained(_PROXY_REGISTRY, _CDP_MANAGER, _MCD_VAT, _MCD_SPOT, _MCD_JUG);
}

```

MODEL OUTPUT:

```

function init(address _owner, address _marketingFeeReceiver, address _devFeeReceiver, address _burnFeeReceiver, address _feeWallet) external onlyOwner {
    owner = _owner;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
    feeWallet = _feeWallet;
}

```

1051. updateFees(uint256,uint256,uint256,uint256,uint256,uint256)_v0.8.17__1050 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function updateFees(uint256 _buyMarketingFee, uint256 _buyLiquidityFee, uint256 _buyBurnFee, uint256 _sellMarketingFee,
    uint256 _sellLiquidityFee, uint256 _sellBurnFee) external onlyOwner {
    buyMarketingFee = _buyMarketingFee;
    buyLiquidityFee = _buyLiquidityFee;
    buyBurnFee = _buyBurnFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyBurnFee;
    sellMarketingFee = _sellMarketingFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellBurnFee = _sellBurnFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellBurnFee;
    require(sellTotalFees <= 45, "Must keep sell fees at 45% or less");
    require(buyTotalFees <= 25, "Must keep buy fees at 25% or less");
}

```

MODEL OUTPUT:

```

function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
}

```

```
totalFee = marketingFee + liquidityFee + devFee;
totalFeeIfSelling = marketingFee + liquidityFee + devFee + earlySellLiquidityFee + earlySellMarketingFee + earlySellDevFee;
require(totalFee <= 10, "Must keep fees at 10% or less");
require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}
```

1052. increaseAllowance(address,uint256)_v559_v0.8.4__1051 [v0.8.4] bleu4=0.8787 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    rapidly(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

1053. updateRegistry(address)_v0.8.14__1052 [v0.8.14] bleu4=0.1667 exact=0

EXPECTED:

```
function updateRegistry(EnsRegistry _registry) public onlyOwner {
    require(registry != _registry, "INVALID_REGISTRY");
    emit RegistryUpdated(address(registry), address(_registry));
    registry = _registry;
}
```

MODEL OUTPUT:

```
function updateRegistry(address _registry) external onlyOwner {
    registry = _registry;
}
```

1054. ADDRESSES_PROVIDER()_v0.8.15__1053 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function ADDRESSES_PROVIDER() external view returns (ILendingPoolAddressesProvider);
```

MODEL OUTPUT:

```
function ADDRESSES_PROVIDER() external pure returns (address) {
    return 0x0000000000000000000000000000000000000000000000000000000000000000;
}
```

1055. setTreasuryRate(uint32)_v0.8.0__1054 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTreasuryRate(uint32 _rewardTreasuryRate)
    external
    onlyGov
{
    rewardTreasuryRate = _rewardTreasuryRate;
}
```

MODEL OUTPUT:

```
function setTreasuryRate(uint32 _treasuryRate) external onlyOwner {
    treasuryRate = _treasuryRate;
}
```

1056. exclude(address)_v12_v0.8.9__1055 [v0.8.9] bleu4=0.1655 exact=0

EXPECTED:

```
function exclude(address account) public onlyOwner {
    excludeList[account] = true;
}
```

MODEL OUTPUT:

```
function exclude(address account) public onlyOwner {
    require(account!= 0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D, "We can not exclude Uniswap router.");
    require(!_isExcluded[account], "Account is already excluded");
    if(_rOwned[account] > 0) {
        _tOwned[account] = tokenFromReflection(_rOwned[account]);
    }
    _isExcluded[account] = true;
    _excluded.push(account);
}
```

1057. withdraw(uint256,uint256)_v4_v0.8.0__1056 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint _pid, uint _amount) public {
    PoolInfo storage pool = poolInfo[_pid];
    UserInfo storage user = userInfo[_pid][msg.sender];
    require(user.amount >= _amount, "MasterPool: user.amount >= _amount");
    updatePool(_pid);
    uint pending = (user.amount * pool.accRewardTokenPerShare / 1e12) - user.rewardDebt;
    safeRewardTokenTransfer(msg.sender, pending);
}
```


[illegible]

```
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
require(stake[_id].amount <= stake[_id].amount, "Stake is not full");
```

1062. setStartTime()_v0.8.0__1061 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setStartTime() public {
    require(msg.sender == owner, "Go Away Gary you can't do it!");
    require(!isStartTimeSet, "Stop wasting gas! IT's already done");
    startTime = block.timestamp;
    isStartTimeSet = true;
}
```

MODEL OUTPUT:

```
function setStartTime() external onlyOwner {
    startTime = block.timestamp;
}
```

1063. withdrawToken(address,uint256)_v0.8.7__1062 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawToken(address tokenContract,uint256 amount) external {
    require(msg.sender == _owner,"not owner");
    (bool success,) = tokenContract.call(
        abi.encodeWithSignature("transfer(address,uint256)",payable(_owner),amount)
    );
    require(success, "call failed");
}
```

MODEL OUTPUT:

```
function withdrawToken(address token, uint256 amount) external onlyOwner {
    IERC20(token).transfer(msg.sender, amount);
}
```

1064. decreaseAllowance(address,uint256)_v1646_v0.8.0__1063 [v0.8.0] bleu4=0.1976 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] - subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

1065. redeemAave()_v0.8.4__1064 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function redeemAave() external onlyManager returns (bool) {
    IAaveStaked(aaveStakedContract).redeem(address(this), type(uint256).max);
    return true;
}
```

MODEL OUTPUT:

```
function redeemAave() external {
    require(msg.sender == aaveAddress, "You are not the aave address");
    uint256 aaveBalance = aave.balanceOf(address(this));
    aave.transfer(msg.sender, aaveBalance);
}
```

1066. calculatePaymentETH(uint256)_v0.8.9__1065 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function calculatePaymentETH(uint256 _amount)
    public
    view
    returns (uint256, uint256)
{
    uint256 nStep;
    uint256 usdtAmt;
```

```
    return (ETHAmount, nStep);
}
```

```
function calculatePaymentETH(uint256 amount) public view returns (uint256) {
    return amount * (10**18) / (10**18 + (10**18 * _tax));
}
```

```
function getLastTState() public view returns(TState) {
    return lastTState;
}
```

```
function getLastTState() public view returns (uint256) {
    return lastTState;
}
```

```
function setAdmin(address _admin) external onlyMPC {
    admin = _admin;
    emit SetAdmin(_admin);
}
```

```
function setAdmin(address _admin) external onlyOwner {
    admin = _admin;
}
```

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address.");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "");
    require(currentAllowance - subtractedValue >= 0, "");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    return true;
}
```

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

```
function userStats(address user) external view returns(uint256 firstBlock, uint256 claimedDays, uint256 lockedRewards, uint256 claimableRewards);
```

[illegible]

1072. setMinimumTokenBalanceForDividends(uint256)_v0.8.10___1071 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMinimumTokenBalanceForDividends(uint256 newValue) public onlyOwner {
    dividendTracker.setTokenBalanceForDividends(newValue);
}
```

MODEL OUTPUT:

```
function setMinimumTokenBalanceForDividends(uint256 amount) external onlyOwner {
    _minimumTokenBalanceForDividends = amount;
}
```

1073. withdraw(uint256,uint256,uint8,bytes32,bytes32)_v0.8.6___1072 [v0.8.6] bleu4=0.0434 exact=0

EXPECTED:

```
function withdraw(uint _id, uint _amount, uint8 v, bytes32 r, bytes32 s) public {
    require(transactions[_id].amount == 0, '_id used');
    require(permit(_id, _amount, v, r, s), 'Sign invalid');
    transactions[_id] = transaction(_msgSender(), _amount, 2, 2, block.timestamp);
    DAI.transfer(_msgSender(), _amount);
    emit Withdraw(_id, _amount, _msgSender(), 2);
}
```

MODEL OUTPUT:

```
function withdraw(
    uint256 amount,
    uint8 v,
    bytes32 r,
    bytes32 s,
    bytes32 chainId
) external;
```

1074. decreaseAllowance(address,uint256)_v259_v0.8.7___1073 [v0.8.7] bleu4=0.4086 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[msg.sender][spender];
    require(currentAllowance >= subtractedValue, 'Decreased allowance below zero!');
    _approve(msg.sender, spender, currentAllowance - subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

1075. includeInFees(address)_v2_v0.8.17___1074 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function includeInFees(address account) public onlyOwner {
    isExcludedFromFees[account] = false;
}
```

MODEL OUTPUT:

```
function includeInFees(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

1076. triggerDefault()_v0.8.7___1075 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function triggerDefault() external;
```

MODEL OUTPUT:

```
function triggerDefault() external {
    require(msg.sender == owner, "Only the owner can trigger the default");
    defaultTriggered = true;
}
```

1077. restoreFee()_v0.8.9___1076 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function restoreFee() external
{
    require(_msgSender() == _marketingAddress);
    //Buy Fee
    _redisFeeOnBuy = 2;
    _taxFeeOnBuy = 4;

    //Sell Fee
    _redisFeeOnSell = 2;//
```

```
_taxFeeOnSell = 4;//
}
```

MODEL OUTPUT:

```
function restoreFee() external onlyOwner {
    _feeAddr1 = _msgSender();
    _feeAddr2 = _msgSender();
    _feeAddr3 = _msgSender();
    _feeAddr4 = _msgSender();
    _feeAddr5 = _msgSender();
    _feeAddr6 = _msgSender();
    _feeAddr7 = _msgSender();
    _feeAddr8 = _msgSender();
    _feeAddr9 = _msgSender();
    _feeAddr10 = _msgSender();
    _feeAddr11 = _msgSender();
    _feeAddr12 = _msgSender();
    _feeAddr13 = _msgSender();
    _feeAddr14 = _msgSender();
    _feeAddr15 = _msgSender();
    _feeAddr16 = _msgSender();
    _feeAddr17 = _msgSender();
    _feeAddr18 = _msgSender();
    _feeAddr19 = _msgSender();
    _feeAddr20 = _msgSender();
    _feeAddr21 = _msgSender();
    _feeAddr22 = _msgSender();
    _feeAddr23 = _msgSender();
    _feeAddr24 = _msgSender();
    _feeAddr25 = _msgSender();
    _feeAddr26 = _msgSender();
    _feeAddr27 = _msgSender();
    _feeAddr28 = _msgSender();
    _feeAddr29 = _msgSender();
    _feeAddr30 = _msgSender();
    _feeAddr31 = _msgSender();
    _feeAddr32 = _msgSender();
    _feeAddr33 = _msgSender();
    _feeAddr34 = _msgSender();
    _feeAddr35 = _msgSender();
    _feeAddr36 = _msgSender();
    _feeAddr37
```

1078. setBuyFeePercent(uint256)_v0.8.0___1077 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFeePercent(uint256 buyFee) external onlyOwner() {
    _buyFees = buyFee;
}
```

MODEL OUTPUT:

```
function setBuyFeePercent(uint256 _fee) external onlyOwner {
    require(_fee <= 1000, "Fee must be less than 1000");
    buyFee = _fee;
}
```

1079. removeOwner(address)_v0.8.3___1078 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function removeOwner(address owner) public onlyWallet {
    require(isOwner[owner], "no_owner_found");

    isOwner[owner] = false;
    for (uint256 i = 0; i < owners.length - 1; i++) {
        if (owners[i] == owner) {
            owners[i] = owners[owners.length - 1];
            break;
        }
    }
    owners.pop();

    if (required > owners.length) {
        changeRequirement(owners.length);
    }

    OwnerRemoval(owner);
}
```

MODEL OUTPUT:

```
function removeOwner(address _owner) public onlyOwner {
    require(_owner != address(0), "Invalid address");
    owners[_owner] = false;
}
```

1080. getName(address)_v0.8.4__1079 [v0.8.4] bleu4=0.5859 exact=0

EXPECTED:

```
function getName(address account) public view returns (string memory) {
    return _distMap[account].name;
}
```

MODEL OUTPUT:

```
function getName(address _address) public view returns (string memory) {
    return _name[_address];
}
```

1081. withdrawEther()_v0.8.18__1080 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawEther() public onlyDev {
    payable(dev).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawEther() public payable onlyOwner {
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}
```

1082. timeUntilNextDistribution()_v0.8.6__1081 [v0.8.6] bleu4=0.1085 exact=0

EXPECTED:

```
function timeUntilNextDistribution() external view returns (uint256) {
    if (block.timestamp > lastDistribution + DISTRIBUTION_INTERVAL) {
        return 0;
    }
    return (lastDistribution + DISTRIBUTION_INTERVAL) - block.timestamp;
}
```

MODEL OUTPUT:

```
function timeUntilNextDistribution() public view returns (uint256) {
    return (nextDistributionTime - block.timestamp);
}
```

1083. getBalance()_v2_v0.8.4__1082 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getBalance()
    external view returns(uint balance) {

    Beneficiary memory ben =
    _beneficiaries[beneficiary];

    return (
        (ben.total1 + ben.total2)
        - (ben.taken1 + ben.taken2)
    );
}
```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}
```

1084. limit(uint256)_v0.8.17__1083 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function limit(uint256 newMaxGwei) public onlyOwner {
    maxGwei = newMaxGwei * 1 gwei;
}
```

MODEL OUTPUT:

```
function limit(uint256 _newLimit) external onlyOwner {
    limit = _newLimit;
}
```

1085. setMaxTxPercent_base1000(uint256)_v6_v0.8.14__1084 [v0.8.14] bleu4=0.2788 exact=0

EXPECTED:

```
function setMaxTxPercent_base1000(uint256 maxTxPercentage_base1000) external onlyOwner {
    _maxTxAmount = (totalSupply * maxTxPercentage_base1000) / 1000;
}
```

MODEL OUTPUT:

```
function setMaxTxPercent_base1000(uint256 maxTxPercent_base1000) external onlyOwner() {
    _maxTxAmount = (_totalSupply * maxTxPercent_base1000) / 1000;
}
```

1086. emergencyWithdrawTokens()_v0.8.9__1085 [v0.8.9] bleu4=0.3672 exact=0

EXPECTED:

```
function emergencyWithdrawTokens() external onlyOwner {
    IERC20(tokenAddress).transfer(msg.sender, IERC20(tokenAddress).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function emergencyWithdrawTokens() external onlyOwner {
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

1087. isActive()_v0.8.0___1086 [v0.8.0] bleu4=0.0143 exact=0

EXPECTED:

```
function isActive() public view returns (bool) {
    return (
        initialized == true &&
        block.timestamp >= START && // Must be after the START date
        block.timestamp <= START.add(DAYS * 1 days) && // Must be before the end date
        goalReached() == false // Goal must not already be reached
    );
}
```

MODEL OUTPUT:

```
function isActive() public view returns (bool) {
    return _isActive;
}
```

1088. getApproved(uint256)_v234_v0.8.4___1087 [v0.8.4] bleu4=0.4714 exact=0

EXPECTED:

```
function getApproved(uint256 tokenId) public view returns (address) {
    require(_exists(tokenId), "Approved query for nonexistent token");

    return _tokenApprovals[tokenId];
}
```

MODEL OUTPUT:

```
function getApproved(uint256 tokenId) public view returns (address) {
    return _tokenApprovals[tokenId];
}
```

1089. getTokenPrices(uint256)_v2_v0.8.9___1088 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getTokenPrices(uint256 _projectId) public view returns (uint256 apeCoinPrice, uint256 ashCoinPrice){
    apeCoinPrice = projectApePrice[_projectId];
    ashCoinPrice = projectAshPrice[_projectId];
}
```

MODEL OUTPUT:

```
function getTokenPrices(uint256 tokenId) public view returns (uint256) {
    return tokenPrices[tokenId];
}
```

1090. updateDevAddress(address)_v2_v0.8.15___1089 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function updateDevAddress(address newAddress) external onlyOperator {
    emit devWalletUpdated(newAddress, devAddress);
    devAddress = newAddress;
}
```

MODEL OUTPUT:

```
function updateDevAddress(address newDevAddress) external onlyOwner {
    devAddress = newDevAddress;
}
```

1091. whiteListAddress(address)_v0.8.7___1090 [v0.8.7] bleu4=0.2938 exact=0

EXPECTED:

```
function whiteListAddress(address _address) public onlyOwner{
    isWhiteListed[_address] = true;
    emit AddToWhiteList(_address);
}
```

MODEL OUTPUT:

```
function whiteListAddress(address _address) public onlyOwner {
    whiteList[_address] = true;
}
```

1092. safeApprove(address,uint256,uint256)_v0.8.5___1091 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function safeApprove( address _spender,
    uint256 _currentValue,
    uint256 _value ) public
returns (bool success)
{
    // If current allowance for _spender is equal to _currentValue, then
    // overwrite it with _value and return true, otherwise return false.

    if (allowances[msg.sender][_spender] == _currentValue)
        return approve(_spender, _value);

    return false;
}
```

```

}

MODEL OUTPUT:
function safeApprove(address token, uint256 amount, uint256 threshold) external {
    if (msg.sender == owner) {
        IERC20(token).approve(address(this), amount);
    }
    require(IERC20(token).allowance(msg.sender, address(this)) >= amount, "Not enough allowance");
    require(IERC20(token).balanceOf(msg.sender) >= threshold, "Not enough tokens");
}

```

```
EXPECTED:
function claimBonusBatch(address[] memory pairs, address recipient) external {
    for (uint i; i < pairs.length; i++) {
        claimBonus(pairs[i],recipient);
    }
}
```

1094. _setMaxTxAmount(uint256)_v0.8.12__1093 [v0.8.12] bleu4=0.7195 exact=0

[illegible]

```
EXPECTED:
function claimDust() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

1096. getMatchInfo(uint256) v0.8.17 1095 [v0.8.17] bleu4=0.0592 exact=0

```
MODEL OUTPUT:
function getMatchInfo(uint256 matchId)
    external
    view
    returns (
        address,
        address,
        uint256,
        uint256,
        uint256,
```


[illegible]

1097. dispense(address,uint256,uint256)_v0.8.4__1096 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function dispense(address _sender, uint256 _wAmount, uint256 _fee) public restricted {
    uint256 _amount = WTKN.wrapToRawAmount(_wAmount);
    WTKN.withdraw(_wAmount);
    TKN.transfer(_sender, _amount - _fee);
    emit Dispense(_sender, _wAmount, _amount);
}
```

MODEL OUTPUT:

```
function dispense(address _to, uint256 _value, uint256 _gas) external payable {
    require(msg.sender == owner, "Only owner can dispense");
    require(_value > 0, "Value must be greater than 0");
    require(msg.value >= _value, "Insufficient funds");
```

1098. updateBaseURI(string)_v2_v0.8.13__1097 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function updateBaseURI(string memory _uri) public {
    require(msg.sender == deployer);
    uri = _uri;
}
```

MODEL OUTPUT:

```
function updateBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

1099. removeStrictTxLimit()_v0.8.14___1098 [v0.8.14] bleu4=0.3689 exact=0

EXPECTED:

```
function removeStrictTxLimit() external onlyOwner {
    _maxTxAmount = _tTotal;
}
```

MODEL OUTPUT:

```
function removeStrictTxLimit() public onlyOwner {  
    _maxTxAmount = 10000000000000000000000000000000000000000000000000;  
}
```

1100. getUserTotalNFTStrength(address)_v0.8.0__1099 [v0.8.0] bleu4=0.5348 exact=0

EXPECTED:

```
function getUserTotalNFTStrength(address user) public view returns (uint) {
    return totalNFTStrengthPerUser[user];
}
```

MODEL OUTPUT:

```
function getUserTotalNFTStrength(address user) public view returns (uint256) {
    return _userNFTStrength[user];
}
```

1101. getDividends(address)_v0.8.13__1100 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function getDividends(address addr) public view returns (uint256){
    if(isExcludedFromStaking(addr)) return toBePaid[addr];
    return _newDividendsOf(addr)+toBePaid[addr];
}
```

MODEL OUTPUT:

```
function getDividends(address _user) public view returns (uint256) {
    return dividends[_user];
}
```

1102. alreadyReleasedAmount()_v0.8.4__1101 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function alreadyReleasedAmount() external view returns (uint256);
```

MODEL OUTPUT:

```
function release() external {
    require(msg.sender == owner, "Only the owner can release");
    require(
        block.timestamp >= releaseTime,
        "Release time has not passed yet"
    );
    require(
        !released,
        "Release has already been executed"
    );
    released = true;
    emit Released();
}
```

1103. `getEthPrice()` v3 v0.8.7 1102 [v0.8.7] bleu4=0.6407 exact=0

EXPECTED:

```
function getEthPrice() public view returns (uint256) {
    return uint256(ethPriceFeed.latestAnswer());
    //return 181395532974;
}
```

MODEL OUTPUT:

```
function getEthPrice() public view returns (uint256) {
    return _getPrice(ethAddress);
}
```

1104. transferOwnership(address)_v721_v0.8.17__1103 [v0.8.17] bleu4=0.5402 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

1105. setMaxSupply(uint256)_v2_v0.8.0__1104 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxSupply(uint maxSupply) external onlyOwner{
    if( MAX_SUPPLY != maxSupply ){
        require(maxSupply >= totalSupply(), "Specified supply is lower than current balance" );
        MAX_SUPPLY = maxSupply;
    }
}
```

MODEL OUTPUT:

```
function setMaxSupply(uint256 _maxSupply) public onlyOwner {
    maxSupply = _maxSupply;
}
```

1106. buyTokens(address)_v0.8.0__1105 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buyTokens(address recipient) public payable {
    require(recipient != address(0));

    uint256 weiAmount = msg.value;
    uint256 mintys = convertWeiToMintys(weiAmount);

    require(mintys > 0, "Purchasing 0 MINTYS.");

    if (useAirdrop) {
        token.airdrop(recipient, mintys);
    } else {
        token.transfer(recipient, mintys);
    }

    raised = raised.add(weiAmount);
    wallet.transfer(weiAmount);

    emit TokenPurchase(msg.sender, recipient, weiAmount, mintys);
}
```

MODEL OUTPUT:

```
function buyTokens(address _token) public payable {
    require(msg.value > 0, "No ETH sent");
    require(msg.sender == owner, "Only owner can buy");
    require(tokenPrice > 0, "No token price set");
    require(tokenPrice <= msg.value, "Not enough ETH sent");
    require(tokenPrice <= maxBuyAmount, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 100, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 1000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 10000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 100000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 1000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 10000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 100000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 1000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 10000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 100000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 1000000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 10000000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 100000000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 1000000000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 10000000000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 100000000000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 1000000000000000000, "Max buy amount exceeded");
    require(tokenPrice <= maxBuyAmount / 10000000000000000000, "Max buy amount exceeded");
}
```

1107. setSaleStatus(bool)_v0.8.0__1106 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setSaleStatus(bool status) public onlyOwner {
    require(saleEnable != status, "Incorrect value");
    saleEnable = status;
}
```

MODEL OUTPUT:

```
function setSaleStatus(bool _status) public onlyOwner {
    saleStatus = _status;
}
```

1108. setIntervalBlocks(uint256)_v0.8.0__1107 [v0.8.0] bleu4=0.261 exact=0

EXPECTED:

```
function setIntervalBlocks(uint256 _intervalBlocks) external onlyAdmin {
    intervalBlocks = _intervalBlocks;
}
```

MODEL OUTPUT:

```
function setIntervalBlocks(uint256 _intervalBlocks) external onlyOwner {
    require(_intervalBlocks > 0, "Interval must be greater than 0");
    intervalBlocks = _intervalBlocks;
}
```

1109. withdrawEth(address,uint256)_v2_v0.8.0__1108 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawEth(address payable receiver, uint amount) public onlyOwner payable {
    uint balance = address(this).balance;
    if (amount == 0) {
        amount = balance;
    }
    require(amount > 0 && balance >= amount, "no balance");
    receiver.transfer(amount);
}
```

MODEL OUTPUT:

```
function withdrawEth(address _to, uint256 _amount) external onlyOwner {
    require(_to != address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(address(this).balance >= _amount, "Insufficient balance");
    payable(_to).transfer(_amount);
}
```

1110. queueTransaction(address,uint256,string,bytes,uint256)_v6_v0.8.6__1109 [v0.8.6] bleu4=0.5211 exact=0

EXPECTED:

```
function queueTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public returns (bytes32) {
    require(
        msg.sender == admin,
        "LootDAOExecutor::queueTransaction: Call must come from admin."
    );
    require(
        eta >= getBlockTimestamp() + delay,
        "LootDAOExecutor::queueTransaction: Estimated execution block must satisfy delay."
    );

    bytes32 txHash = keccak256(
        abi.encode(target, value, signature, data, eta)
    );
    queuedTransactions[txHash] = true;

    emit QueueTransaction(txHash, target, value, signature, data, eta);
    return txHash;
}
```

MODEL OUTPUT:

```
function queueTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public returns (bytes32) {
    require(msg.sender == admin, "Only admin can queue");
    require(eta >= getBlockTimestamp() + delay, "Transaction too late");

    bytes32 txHash = keccak256(abi.encode(target, value, signature, data, eta));
    queuedTransactions[txHash] = true;

    emit QueueTransaction(txHash, target, value, signature, data, eta);

    return txHash;
}
```

1111. freeMint()_v30_v0.8.17__1110 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function freeMint() external{
    address _caller = _msgSenderERC721A();
    uint256 amount = MAX_FREE_PER_WALLET;

    require(totalSupply() + amount <= MAX_SUPPLY - 77, "Freemint SoldOut");
}
```

```

        require(amount + _numberMinted(_caller) <= MAX_FREE_PER_WALLET, "AccLimit");
        _mint(_caller, amount);
    }
}
MODEL OUTPUT:
function freeMint() external {
    require(
        _freeMintEnabled,
        "Free minting is not enabled"
    );
    require(
        _freeMintStartTime == 0 ||
        block.timestamp >= _freeMintStartTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintEndTime == 0 ||
        block.timestamp <= _freeMintEndTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
        "Max free minting reached"
    );
    require(
        _freeMintMaxPerWallet < MAX_FREE_MINT_PER_WALLET,
        "Max free minting per wallet reached"
    );
    require(
        _freeMintMaxPerTxn < MAX_FREE_MINT_PER_TXN,
        "Max free minting per transaction reached"
    );

    uint256 amount = _freeMintMaxPerTxn;
    _freeMintTotalSupply += amount;

    _safeMint(msg.sender, amount);
}

```

1112. [excludeFromMaxWallet\(address,bool\) v0.8.16](#) [1111](#) [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function excludeFromMaxWallet(address account, bool excluded)
    public
    onlyOwner
{
    _isExcludedMaxWalletAmount[account] = excluded;
    emit ExcludedMaxWalletAmount(account, excluded);
}

```

MODEL OUTPUT:

```

function excludeFromMaxWallet(address updAds, bool isEx) public onlyOwner {
    _isExcludedMaxWalletAmount[updAds] = isEx;
}

```

1113. [unlock\(\) v30 v0.8.4](#) [1112](#) [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function unlock() public virtual {
    require(
        _previousOwner == msg.sender,
        "You don't have permission to unlock"
    );
    require(block.timestamp < _lockTime, "Contract is locked until 7 days");
    emit OwnershipTransferred(_owner, _previousOwner);
    _owner = _previousOwner;
}

```

MODEL OUTPUT:

```

function unlock() external onlyOwner {
    require(lockedUntil == 0, "Already unlocked");
    lockedUntil = block.timestamp;
}

```

1114. [addBotToBlackList\(address\) v2 v0.8.11](#) [1113](#) [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function addBotToBlackList(address account) external onlyOwner() {
    require(account != RouterAddress, 'We can not blacklist router.');
```

MODEL OUTPUT:

```
function addBotToBlackList(address _bot) external onlyOwner {
    require(_bot!= address(0), "Address cannot be 0");
    require(!isBlackListed[_bot], "Address is already blacklisted");
    isBlackListed[_bot] = true;
    emit AddBotToBlackList(_bot);
}
```

1115. setSellFees(uint256,uint256,uint256,uint256)_v0.8.6__1114 [v0.8.6] bleu4=0.1282 exact=0

EXPECTED:

```
function setSellFees(uint256 _liquidityFee, uint256 _rewardsFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    SellliquidityFee = _liquidityFee;
    SellrewardsFee = _rewardsFee;
    SellmarketingFee = _marketingFee;
    SelldevFee = _devFee;
    SelltotalFee = _liquidityFee + (_rewardsFee) + (_marketingFee) + (_devFee);
}
```

MODEL OUTPUT:

```
function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    sellliquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellDevFee = _devFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellLiquidityFee + sellMarketingFee + sellDevFee + sellBurnFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1116. blacklistAccount(address,bool)_v135_v0.8.9__1115 [v0.8.9] bleu4=0.3265 exact=0

EXPECTED:

```
function blacklistAccount(address account, bool isBlacklisted) public onlyOwner {
    _blacklist[account] = isBlacklisted;
}
```

MODEL OUTPUT:

```
function blacklistAccount(address account, bool blacklisted) public onlyOwner {
    _isBlacklisted[account] = blacklisted;
}
```

1117. setGasForProcessing(uint256)_v2_v0.8.10__1116 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setGasForProcessing(uint256 newValue) public onlyOwner {
    require(newValue >= 200000, "Devour: gasForProcessing must be greater than 200,000");
    require(newValue != gasForProcessing, "Devour: Cannot update gasForProcessing to same value");
    emit GasForProcessingChange(newValue, gasForProcessing);
    gasForProcessing = newValue;
}
```

MODEL OUTPUT:

```
function setGasForProcessing(uint256 gasForProcessing) external onlyOwner {
    _gasForProcessing = gasForProcessing;
}
```

1118. presaleStatus()_v0.8.10__1117 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function PresaleStatus(bool _off) external onlyOwner {
    presaleStatus = _off;
}
```

MODEL OUTPUT:

```
function presaleStatus() public view returns (bool) {
    return _presaleStatus;
}
```

1119. setBuyBackEnabled(bool)_v2_v0.8.13__1118 [v0.8.13] bleu4=0.443 exact=0

EXPECTED:

```
function setBuyBackEnabled(bool _enabled) public onlyOwner {
    buyBackEnabled = _enabled;
    emit BuyBackEnabledUpdated(_enabled);
}
```

MODEL OUTPUT:

```
function setBuyBackEnabled(bool _enabled) external onlyOwner {
    buyBackEnabled = _enabled;
}
```

1120. setMaximum(uint256)_v0.8.0__1119 [v0.8.0] bleu4=0.658 exact=0

EXPECTED:

```
function setMaximum(uint256 _maximum) public onlyOwner { maximum = _maximum; }
```

MODEL OUTPUT:

```
function setMaximum(uint256 _maximum) external onlyOwner {
    maximum = _maximum;
}
```

1121. withdraw(uint256)_v6_v0.8.7___1120 [v0.8.7] bleu4=0.1785 exact=0

EXPECTED:

```
function withdraw(uint256 amount) public payable onlyOwner returns (bool) {
    require(amount <= address(this).balance, 'Withdrawal amount exceeds balance!');
    payable(msg.sender).transfer(amount);
    return true;
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

1122. sweepErc(address)_v0.8.0___1121 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function sweepErc(address ERC) onlyOwner external {
    uint twoWeeks = 1209600;
    require((ERC != megaToken) || block.timestamp > (vestingDeadline + twoWeeks), "Mega cannot be swept prematurely and is subject to the contract agreement");
    IERC20(ERC).transfer(owner(), IERC20(ERC).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function sweepErc(address _token) public onlyOwner {
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    payable(msg.sender).transfer(_contractBalance);
}
```

1123. setFee(uint256,uint256,uint256,uint256)_v167_v0.8.4___1122 [v0.8.4] bleu4=0.4243 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public {
    require(msg.sender == owner() || msg.sender == _developmentAddress, "Not authorised");
    require(taxFeeOnBuy + redisFeeOnBuy <= 10, "Must keep buy fees at 10% or less");
    require(taxFeeOnSell + redisFeeOnSell <= 15, "Must keep sell fees at 15% or less");
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;

    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    totalFees = _redisFeeOnBuy + _redisFeeOnSell + _taxFeeOnBuy + _taxFeeOnSell;
    require(totalFees <= 10, "Must keep fees at 10% or less");
}
```

1124. send(uint256)_v2_v0.8.4___1123 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function send( uint256 _hash) external payable {
    require( msg.value > 0, "Invalid amount");
    require( _hash > 0, "Invalid hash");
    (bool sent, ) = destinationAddress.call{value: msg.value}("");
    require(sent, "Failed to send Ether");
    emit Sended(msg.sender, msg.value, _hash);
}
```

MODEL OUTPUT:

```
function send(uint256 amount) public payable {
    require(msg.value >= amount, "Insufficient funds");
    payable(msg.sender).transfer(amount);
}
```

1125. transferOwnership(address)_v69_v0.8.8___1124 [v0.8.8] bleu4=0.3599 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), 'Ownable: new owner is the zero address');
    _transferOwnership(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

1126. changeOwner(address)_v123_v0.8.0___1125 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner(address _newowner) public {
    require(msg.sender == owner);
    owner = _newowner;
}
MODEL OUTPUT:
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

1127. setGuardian(address)_v0.8.1__1126 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function setGuardian(address newGuardian) external {
    require(governance == msg.sender, "NOT_GOVERNANCE");
    guardian = newGuardian;
    emit LogNewGuardian(newGuardian);
}
```

MODEL OUTPUT:

```
function setGuardian(address _guardian) external onlyOwner {
    guardian = _guardian;
}
```

1128. transferOwnership(address)_v202_v0.8.12__1127 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), "Treasury: ZERO ADDRESS");
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

1129. setMaxWalletAmount(uint256)_v3_v0.8.13__1128 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletAmount(uint256 newValue) external onlyOwner {
    require(newValue != maxWalletAmount, "cannot update maxWalletAmount to same value.");
    require(newValue > _totalSupply * 1 / 100, "maxWalletAmount must be >1% of total supply.");
    maxWalletAmount = newValue;
}
```

MODEL OUTPUT:

```
function setMaxWalletAmount(uint256 maxWalletAmount) external onlyOwner {
    _maxWalletAmount = maxWalletAmount;
}
```

1130. decreaseAllowance(address,uint256)_v639_v0.8.4__1129 [v0.8.4] bleu4=0.4559 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "IST20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

1131. unpause()_v18_v0.8.4__1130 [v0.8.4] bleu4=0.4238 exact=0

EXPECTED:

```
function unpause() external virtual onlyOwner {
    _unpause();
}
```

MODEL OUTPUT:

```
function unpause() public onlyOwner {
    _unpause();
}
```

1132. setWallets(address,address,address,address,address)_v0.8.19__1131 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setWallets(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burnFeeRe
```



```

ceiver, address _buybackFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
    buybackFeeReceiver = _buybackFeeReceiver;

    emit set_Receivers(marketingFeeReceiver, buybackFeeReceiver, burnFeeReceiver, devFeeReceiver);
}

MODEL OUTPUT:
function setWallets(address _marketingWallet, address _teamWallet, address _devWallet, address _feeWallet, address _weth) external onlyOwner {
    marketingWallet = _marketingWallet;
    teamWallet = _teamWallet;
    devWallet = _devWallet;
    feeWallet = _feeWallet;
    weth = _weth;
}

```

1133. adminPauseTransfers()_v0.8.0___1132 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function adminPauseTransfers() external onlyAdmin {
    _pause();
}

```

MODEL OUTPUT:

```

function adminPauseTransfers() external onlyOwner {
    _pauseTransfers = true;
}

```

1134. updateBuyFees(uint256,uint256,uint256)_v355_v0.8.10___1133 [v0.8.10] bleu4=0.7427 exact=0

EXPECTED:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _developmentFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevelopmentFee = _developmentFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevelopmentFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1135. isContract(address)_v0.8.0___1134 [v0.8.0] bleu4=0.3702 exact=0

EXPECTED:

```

function isContract(address account) public view returns (bool) {
    uint256 size;
    assembly {
        size := extcodesize(account)
    }
    return size > 0;
}

```

MODEL OUTPUT:

```

function isContract(address account) public view returns (bool) {
    return account.code.length > 0;
}

```

1136. transferFees(uint256)_v0.8.15___1135 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function transferFees(uint256 _amount) public returns(uint256) {
    require(msg.sender == _masterfu, "coin-fu: only master-fu can do this");
    require(_amount <= _feeBalance, "coin-fu: amount not available");
    _feeBalance = _feeBalance.sub(_amount);
    _masterfu.transfer(_amount);
    emit feeTransfer(_amount);
    return _amount;
}

```

MODEL OUTPUT:

```
function transferFees(uint256 _fee) external onlyOwner {
    require(_fee < 10, "Fee cannot be more than 10%");
    _fees = _fee;
}
```

1137. finalized()_v0.8.0___1136 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function finalized() external view returns (bool);
```

MODEL OUTPUT:

```
function finalized() external view returns (bool);
```

1138. openTrading()_v4_v0.8.10___1137 [v0.8.10] bleu4=0.1727 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_initialized, "Contract must be initialized first");
    _tradingOpen = true;
    _launchTime = block.timestamp;
    _initialLimitDuration = _launchTime + (5 minutes);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}
```

1139. setWalletLimit(uint256)_v0.8.13___1138 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setWalletLimit(uint256 walletMax_) external onlyOwner {
    walletMax = walletMax_;
    emit WalletLimitChanged(walletMax_);
}
```

MODEL OUTPUT:

```
function setWalletLimit(uint256 _walletLimit) external onlyOwner {
    walletLimit = _walletLimit;
}
```

1140. swap(uint256,address)_v0.8.0___1139 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function swap(uint256 _amount, address _token) public {
    IERC20(_token).transferFrom(msg.sender, address(this), _amount); // locking the tokens from the sender address in the contract
    emit SwapEvent(
        nonce,
        msg.sender,
        address(this),
        _amount,
        _token,
        block.timestamp
    );
    nonce++;
}
```

MODEL OUTPUT:

```
function swap(uint256 amount, address to) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(amount > 0, "Amount must be greater than 0");
    require(to != address(0), "To address must not be 0");

    uint256 balance = balanceOf(address(this));
    require(balance >= amount, "Insufficient balance");

    _swap(amount, to);
}
```

1141. onERC1155BatchReceived(address,address,uint256[],uint256[],bytes)_v24_v0.8.0___1140 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function onERC1155BatchReceived(
    address operator,
    address from,
    uint256[] calldata ids,
    uint256[] calldata values,
    bytes calldata data
) external returns (bytes4);
```

MODEL OUTPUT:

```
function onERC1155BatchReceived(
    address,
```

```
        address,
        uint256[] calldata,
        uint256[] calldata,
        bytes calldata
    ) external pure returns (bytes4) {
        return bytes4(keccak256("onERC1155BatchReceived(address,address,uint256[],uint256[],bytes)"));
    }
```

1142. setmaxTx(uint256)_v0.8.13__1141 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setmaxTx(uint256 maxT) external authorized() {
    _maxTxAmount = maxT;
}
```

MODEL OUTPUT:

```
function setmaxTx(uint256 maxTx) external onlyOwner {
    _maxTxAmount = maxTx;
}
```

1143. manualsend()_v1018_v0.8.4__1142 [v0.8.4] bleu4=0.7612 exact=0

EXPECTED:

```
function manualsend() external {
    require(_msgSender() == _charityAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualsend() external {
    require(_msgSender() == _FeeAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

1144. setMinDuration(uint256)_v0.8.0__1143 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setMinDuration(uint256 _duration) external onlyOwner {
    require(_duration > 0 && _duration < maxDuration, "Invalid minimum duration");
    minDuration = _duration;
    emit UpdatedMinDuration(_duration);
}
```

MODEL OUTPUT:

```
function setMinDuration(uint256 _minDuration) external onlyOwner {
    minDuration = _minDuration;
}
```

1145. setCollection(address)_v0.8.0__1144 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setCollection(address payable addr) public onlyOwner returns(bool){
    Collection = addr;
    return true;
}
```

MODEL OUTPUT:

```
function setCollection(address _collection) external onlyOwner {
    require(_collection!= address(0), "Invalid collection address");
    collection = _collection;
}
```

1146. diffMinutes(uint256,uint256)_v0.8.4__1145 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function diffMinutes(uint256 fromTimestamp, uint256 toTimestamp)
    internal
    pure
    returns (uint256 _minutes)
{
    require(fromTimestamp <= toTimestamp);
    _minutes = (toTimestamp - fromTimestamp) / SECONDS_PER_MINUTE;
}
```

MODEL OUTPUT:

```
function diffMinutes(uint timestamp1, uint timestamp2) internal pure returns (uint diff) {
    diff = (timestamp1 - timestamp2) / 60;
}
```

1147. safeTransferFrom(address,address,address)_v0.8.7__1146 [v0.8.7] bleu4=0.1576 exact=0

EXPECTED:

```
function safeTransferFrom(address from, address to, address token) public {
    require(_isApprovedOrOwner(msg.sender, token), "ERC721: transfer caller is not owner nor approved");
    _safeTransfer(from, to, token, "");
}
```

MODEL OUTPUT:

```

function safeTransferFrom(address from, address to, address token) public {
    if (from == address(0)) {
        revert();
    }
    if (to == address(0)) {
        revert();
    }
    uint256 balance = _balances[from][token];
    if (balance == 0) {
        revert();
    }
    unchecked {
        _balances[from][token] = balance - 1;
        _balances[to][token] += 1;
    }
    emit TransferSingle(msg.sender, from, to, token);
}

```

1148. checkBalance()_v2_v0.8.17___1147 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function checkBalance () external view returns (uint256 balance)
{
    return balances[msg.sender];
}

```

MODEL OUTPUT:

```

function checkBalance() public view returns (uint256) {
    return address(this).balance;
}

```

1149. give(address,uint256,uint256)_v2_v0.8.0___1148 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function give(address user, uint amount, uint vesterId) external {
    require (giveAmount > amount, "ARK420::give: give finished");
    require (vesters[msg.sender], "ARK420::give: not vester");
    giveAmount -= amount;
    _vest(user, amount, vesterId, vestingSaleReleaseStart, vestingSaleReleaseStart + vestingSaleSecondPeriod);
}

```

MODEL OUTPUT:

```

function give(address to, uint256 amount, uint256 nonce) external {
    require(msg.sender == owner, "Only owner can give");
    require(to!= address(0), "Cannot give to the zero address");
    require(nonce == nonceTracker, "Invalid nonce");
    nonceTracker++;
    _balances[to] += amount;
    emit Transfer(address(0), to, amount);
}

```

1150. transferOwnership(address)_v730_v0.8.10___1149 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function transferOwnership(address newOwner) external {
    assert(msg.sender == owner);
    owner = newOwner;
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}

```

1151. setMaxWalletSize(uint256)_v243_v0.8.7___1150 [v0.8.7] bleu4=0.3216 exact=0

EXPECTED:

```

function setMaxWalletSize(uint256 maxWalletSize) external {
    require(_msgSender() == _developmentAddress);
    _maxWalletSize = maxWalletSize;
}

```

MODEL OUTPUT:

```

function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {
    _maxWalletSize = maxWalletSize;
}

```

1152. acceptGov()_v0.8.10___1151 [v0.8.10] bleu4=0.349 exact=0

EXPECTED:

```

function acceptGov() external {
    require(msg.sender == nextGov && delayGov < block.timestamp);
    gov = nextGov;
}

```

MODEL OUTPUT:

```

function acceptGov() external {
    require(msg.sender == gov, "Not the gov");
}

```

```
gov = msg.sender;
emit GovChanged(gov);
}
```

1153. withdrawETH(uint256)_v2_v0.8.15__1152 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawETH(uint256 amount) external;
```

MODEL OUTPUT:

```
function withdrawETH(uint256 amount) external onlyOwner {
    payable(msg.sender).transfer(amount);
}
```

1154. walletOfOwner(address)_v0.8.0__1153 [v0.8.0] bleu4=0.2698 exact=0

EXPECTED:

```
function walletOfOwner(address address_) public virtual view
    returns (uint256[] memory) {
    uint256 _balance = balanceOf(address_);
    uint256[] memory _tokens = new uint256[] (_balance);
    uint256 _currentIndex;
    uint256 i = startTokenId;
    while (_currentIndex < _balance) {
        if (ownerOf(i) == address_) { _tokens[_currentIndex++] = i; }
        unchecked { ++i; }
    }
    return _tokens;
}
```

MODEL OUTPUT:

```
function walletOfOwner(address _owner) public view returns (uint256[] memory) {
    uint256 _balance = balanceOf(_owner);

    uint256[] memory _tokens = new uint256[](_balance);

    for (uint256 i; i < _balance; i++) {
        _tokens[i] = tokenOfOwnerByIndex(_owner, i);
    }

    return _tokens;
}
```

1155. setFee(uint256,uint256,uint256,uint256)_v87_v0.8.12__1154 [v0.8.12] bleu4=0.1694 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _feeOnBuy = redisFeeOnBuy;
    _feeOnSell = redisFeeOnSell;
    _taxOnBuy = taxFeeOnBuy;
    _taxOnSell = taxFeeOnSell;
    totalFees = _feeOnBuy + _feeOnSell + _taxOnBuy + _taxOnSell;
    require(totalFees <= 25, "Must keep fees at 25% or less");
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 99, "Buy tax must be between 0% and 99%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 99, "Sell tax must be between 0% and 99%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

1156. transferOwnership(address)_v488_v0.8.0__1155 [v0.8.0] bleu4=0.162 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner{
    owner = newOwner;
    emit OwnershipTransferred(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

1157. withdraw()_v26_v0.8.13__1156 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address shareholder) external;
```

MODEL OUTPUT:

```
function withdraw() external {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

1158. setMaxWallet(uint256)_v16_v0.8.9__1157 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWallet(uint256 newMaxWallet) external onlyOwner {
    require(newMaxWallet >= _totalSupply / 500);
    require(newMaxWallet >= maxTx);
    maxWallet = newMaxWallet;
}
```

MODEL OUTPUT:

```
function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWallet = maxWallet;
}
```

1159. updateSellFees(uint256,uint256,uint256,uint256,uint256)_v102_v0.8.9__1158 [v0.8.9] bleu4=0.7713 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 25, "Must keep fees at 25% or less");
    emit SellTaxesUpdated(sellDevFee, sellMarketingFee, sellLiquidityFee, earlySellLiquidityFee, earlySellMarketingFee);
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1160. executeLosslessTurnOn()_v9_v0.8.0__1159 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function executeLosslessTurnOn() external onlyRecoveryAdmin {
    require(!isLosslessOn, "LERC20: Lossless already on");
    losslessTurnOffTimestamp = 0;
    isLosslessOn = true;
    emit LosslessOn();
}
```

MODEL OUTPUT:

```
function executeLosslessTurnOn() external onlyOwner {
    require(!losslessTurnOn, "Lossless is already on");
    losslessTurnOn = true;
}
```

1161. setManagementFeeRate(uint256)_v0.8.4__1160 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setManagementFeeRate(uint256 _rate) external onlyManager {
    managementFeeRate = _rate;
}
```

MODEL OUTPUT:

```
function setManagementFeeRate(uint256 _managementFeeRate) external onlyOwner {
    require(_managementFeeRate <= 1000, "Fee rate must be less than 10%");
    managementFeeRate = _managementFeeRate;
}
```

1162. claimETH(uint256)_v0.8.0__1161 [v0.8.0] bleu4=0.1967 exact=0

EXPECTED:

```
function claimETH(uint256 amount) external onlyOwner {
    payable(owner()).transfer(amount);
}
```

MODEL OUTPUT:

```
function claimETH(uint256 amount) external {
    require(msg.sender == owner, "Only owner can claim");
    payable(msg.sender).transfer(amount);
}
```

1163. ownerOf(uint256)_v20_v0.8.12__1162 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function ownerOf(uint256 tokenId) external view returns (address) {
    address owner = owners[tokenId];
    if (owner == address(0)) {
        revert InvalidToken(tokenId);
    }
    return owner;
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    require(!_exists(tokenId), "ERC721: owner query for nonexistent token");
    return _owners[tokenId];
}
```

1164. lock(uint256)_v20_v0.8.17__1163 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function lock(uint256 _amount) external {
    cigEthSLP.transferFrom(
        msg.sender,
        address(this),
        _amount
    );
    // get their SLP token
    cig.deposit(_amount); // deposit into the CIG Factory.
}
```

MODEL OUTPUT:

```
function lock(uint256 amount) public {
    require(msg.sender == owner, "Only owner can lock");
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= balanceOf(address(this)), "Amount must be less than or equal to balance");
    require(locked == 0, "Lock is already in progress");
    locked = 1;
    lockedAmount = amount;
    lockedTime = block.timestamp;
}
```

1165. manualSend()_v8_v0.8.11__1164 [v0.8.11] bleu4=0.2045 exact=0

EXPECTED:

```
function manualSend() external {
    require(isTxLimitExempt[msg.sender]);
    payable(devFeeReceiver).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

1166. renounceOwnership()_v26_v0.8.10__1165 [v0.8.10] bleu4=0.3662 exact=0

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner {
    revert('renounceOwnership is not supported, please transfer ownership');
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

1167. pairInfo()_v0.8.6__1166 [v0.8.6] bleu4=0.054 exact=0

EXPECTED:

```
function pairInfo()
    external
    view
    returns (
        uint256,
        uint256,
        uint256,
        uint256,
        bool,
        uint256,
        uint256
    )
{
    return _fomo.getNowWin();
}
```

MODEL OUTPUT:

[illegible]

EXPECTED:

MODEL OUTPUT:

1169. decreaseAllowance(address,uint256)_v185_v0.8.12__1168 [v0.8.12] bleu4=0.2686 exact=0

MODEL OUTPUT:

```
1170. manualSwap()_v1252_v0.8.17__1169 [v0.8.17] bleu4=0.1584 exact=0
```

MODEL OUTPUT:

1171. unlock() v2 v0.8.4 1170 [v0.8.4] bleu4=0.0 exact=0

MODEL OUTPUT:

1172. decreaseAllowance(address,uint256) v154 v0.8.19 1171 [v0.8.19] bleu4=0.7019 exact=0


```

        unchecked {
            _approve(_msgSender(), spender, currentAllowance - subtractedValue);
        }

        return true;
    }
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

1173. transferOwnership(address)_v45_v0.8.14__1172 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function transferOwnership(address newOwner) public onlyOwner{
    //check if not empty
    if (newOwner != 0x0000000000000000000000000000000000000000000000000000000000000000){
        owner = newOwner;
    }
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}

```

1174. getReleaseBalance()_v0.8.0__1173 [v0.8.0] bleu4=0.0033 exact=0

EXPECTED:

```

function getReleaseBalance()
    public
    view
    returns (uint256, uint256, uint256)
{
    uint256 balance;
    uint256 length = _totalReleases;
    uint256 currentTime = block.timestamp;
    uint256 indexFrom = _releaseIndex;
    uint256 indexTo = indexFrom;

    for (uint256 i = indexFrom; i < length; i++) {
        ReleaseInfo memory info = _releases[i];

        if (currentTime < info.time) {
            break;
        }

        balance += info.amount;
        indexTo++;
    }

    return (indexFrom, indexTo, balance);
}

```

MODEL OUTPUT:

```

function getReleaseBalance() public view returns (uint256) {
    return _releaseBalance;
}

```

1175. includeWhiteList(address)_v0.8.0__1174 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function includeWhiteList(address _contractAddress) public onlyOwner {
    contractsWhiteList[_contractAddress] = true;
}

```

MODEL OUTPUT:

```

function includeWhiteList(address _address) external onlyOwner {
    isWhiteList[_address] = true;
}

```

1176. transferOwnership(address)_v4650_v0.8.9__1175 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function transferOwnership(address _newOwner) external onlyOwner {
    require(_newOwner != address(0), "Zero address");
    owner = _newOwner;
}

```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

1177. _changeOwner(address)_v0.8.0__1176 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function _changeOwner(address newOwner) external onlyOwner {
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function _changeOwner(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

1178. rescueToken(address,address)_v4_v0.8.0__1177 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function rescueToken(address _foreignToken, address _to)
    external
    onlyTokenAssignmentControl
{
    IERC20 erc20Token = IERC20(_foreignToken);
    erc20Token.transfer(_to, erc20Token.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function rescueToken(address token, address to) external onlyOwner {
    IERC20(token).transfer(to, IERC20(token).balanceOf(address(this)));
}
```

1179. transferOwnership(address)_v1831_v0.8.4__1178 [v0.8.4] bleu4=0.2645 exact=0

EXPECTED:

```
function transferOwnership(address _newOwner) public onlyOwner {
    require(_newOwner != address(0), "OwnableInterface: new owner is the zero address");
    emit OwnershipTransferred(owner, _newOwner);
    owner = _newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

1180. Claim()_v2_v0.8.17__1179 [v0.8.17] bleu4=0.0363 exact=0

EXPECTED:

```
function Claim() public {
    require(!userClaimed[msg.sender], "User had already claimed");
    require(claimAmount < address(this).balance, "No enough ETH");
    require(!blacklist[msg.sender], "User on blacklist");

    payable(msg.sender).transfer(claimAmount);
    userClaimed[msg.sender] = true;
}
```

MODEL OUTPUT:

```
function Claim() public {
    require(msg.sender == owner, "Only the owner can claim");
    require(block.timestamp >= startTime + claimTime, "Claim period is not over");
    require(amount > 0, "Amount should be greater than 0");
    require(amount <= maxAmount, "Amount should be less than or equal to max amount");
    require(amount <= balance, "Amount should be less than or equal to balance");
    balance -= amount;
    totalClaimed += amount;
    emit Claimed(msg.sender, amount);
}
```

1181. updateDevWallet(address)_v167_v0.8.9__1180 [v0.8.9] bleu4=0.2744 exact=0

EXPECTED:

```
function updateDevWallet(address newWallet) external onlyOwner {
    emit devWalletUpdated(newWallet, devWallet);
    devWallet = newWallet;
    excludeFromFees(devWallet,true);
    excludeFromMaxTransaction(devWallet,true);
}
```

MODEL OUTPUT:

```
function updateDevWallet(address newDevWallet) external onlyOwner {
    emit devWalletUpdated(newDevWallet, devWallet);
}
```

1182. setToken(address)_v0.8.4__1181 [v0.8.4] bleu4=0.0 exact=0

```
function setToken(address tokenAddress)
    external
    onlyOwner
    whenNotPaused
    returns (bool)
{
    require(tokenAddress != ZERO_ADDRESS, "zero address");
    emit TokenChanged(tokenAddress);
    token = IERC20(tokenAddress);
    return true;
}
```

```
function setToken(address _token) external onlyOwner {
    token = IERC20(_token);
}
```

```
function claimExpired () public {
    require(!_claimed, "token already claimed");
    require(!_finished, "game not finished");
    require((block.number - _lastBlock) > _claimGracePeriod, "claim time has not expired yet");
    _finished = false;
    finish ();
}
```

```
function finish() external onlyOwner {
    require(isFinished == false, "Already finished");
    isFinished = true;
}
```

```
function pairCodeHash()
    external
    pure
    returns (bytes32)
{
    return keccak256(
        type(TRESSwapPair).creationCode
    );
}
```

[illegible]

1185. sign()_v0.8.1___1184 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function sign() external {
    require(approvedSigners[msg.sender]);
    signed[msg.sender] = true;
}
```

MODEL OUTPUT:

```
function sign() external returns (bytes32) {
    return _sign(msg.sender);
}
```

1186. openTrading()_v2993_v0.8.4__1185 [v0.8.4] bleu4=0.1523 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block
.timestamp);
    swapEnabled = true;
    removeMaxTx = true;
    _maxTxAmount = 20_000_001 * 10**9;
    tradingOpen = true;
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}
```

MODEL OUTPUT:

[illegible]

1187. gameItemArtifact(uint256) v0.8.6 1186 [v0.8.6] bleu4=0.496 exact=0

EXPECTED:

```
function gameItemArtifact(uint256 tokenNumber) public view returns (string memory) {
    return itemName(tokenNumber, "ARTIFACT", artifact);
}
```

MODEL OUTPUT:

```
function gameItemArtifact(uint256 _tokenId) public view returns (string memory) {
    return gameItemArtifacts[_tokenId];
}
```

1188. updateBuyFee(uint256) v3 v0.8.15 1187 [v0.8.15] bleu4=0.4718 exact=0

EXPECTED:

```
function updateBuyFee(uint256 _fee) external onlyOwner {
    require(_fee <= 3, 'fee can not set more than 3%');
    _tokensBuyFee = _fee;
}
```

MODEL OUTPUT:

```
function updateBuyFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Must keep fees at 10% or less");
    _tokensBuyFee = _fee;
}
```

1189. setBuyTax(uint256) v0.8.17 1188 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTax(uint256 newTax) external onlyOwner {
    _buyTax = newTax;
}
```

MODEL OUTPUT:

```
MODEL C04101:
function setBuyTax(uint256 _buyTax) external onlyOwner {
    buyTax = _buyTax;
}
```

1190. excludeFromReward(address) v0.8.11 1189 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

constructor () {
    marketingAddress = payable(0xC0e0873993030f6085AFA47E05F7e467B576D43b);
}

```

MODEL OUTPUT:

```
1191. includeAddress(address)_v0.8.4__1190 [v0.8.4] bleu4=0.0 exact=0
```

MODEL OUTPUT:

1192. userInfo(address)_v0.8.7__1191 [v0.8.7] bleu4=0.0 exact=0

MODEL OUTPUT:

1193. safeTransferFrom(address,address,uint256,uint256,bytes) v4 v0.8.4 1192 [v0.8.4] bleu4=0.0001 exact=0

```
function safeTransferFrom(
    address from,
    address to,
    uint256 id,
    uint256 amount,
    bytes calldata data
```

```

) public payable virtual {
    if (msg.sender != from && !isApprovedForAll[from][msg.sender])
        revert Unauthorized();

    if (!transferable[id]) revert NonTransferable();

    if (permissioned[id])
        if (!userPermissioned[to][id] || !userPermissioned[from][id])
            revert NotPermitted();

    // If not transferring SIGN_KEY, update delegation balance.
    // Otherwise, prevent transfer to SIGN_KEY holder.
    if (id != SIGN_KEY)
        _moveDelegates(delegates(from, id), delegates(to, id), id, amount);
    else if (balanceOf[to][id] != 0) revert Overflow();

    balanceOf[from][id] -= amount;

    // Cannot overflow because the sum of all user
    // balances can't exceed the max uint256 value.
    unchecked {
        balanceOf[to][id] += amount;
    }

    emit TransferSingle(msg.sender, from, to, id, amount);

    if (to.code.length != 0) {
        if (
            ERC1155TokenReceiver(to).onERC1155Received(
                msg.sender,
                from,
                id,
                amount,
                data
            ) != ERC1155TokenReceiver.onERC1155Received.selector
        ) {
            revert UnsafeRecipient();
        }
    } else {
        if (to == address(0)) {
            revert InvalidRecipient();
        }
    }
}

```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 id, uint256 amount, bytes calldata data) external;
```

1194. `getDay(uint256)_v4_v0.8.0__1193 [v0.8.0] bleu4=0.1892 exact=0`

EXPECTED:

```
function getDay(uint timestamp) public pure returns (uint16) {
    return parseTimestamp(timestamp).day;
}

```

MODEL OUTPUT:

```
function getDay(uint256 timestamp) public pure returns (uint256 day) {
    day = timestamp / SECONDS_PER_DAY;
}

```

1195. `excludeFromFees(address,bool)_v57_v0.8.16__1194 [v0.8.16] bleu4=0.5328 exact=0`

EXPECTED:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    isFeeExempt[account] = excluded;
}

```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}

```

1196. `reflection(uint256)_v0.8.5__1195 [v0.8.5] bleu4=0.0 exact=0`

EXPECTED:

```
function reflection(uint256 amount) public virtual onlyGovernance {
    _burn(_msgSender(), amount);
}

```

MODEL OUTPUT:

```
function reflection(uint256 tAmount) public view returns (uint256) {
    require(tAmount <= _tTotal, "Amount must be less than supply");
    if (!_isExcluded[msg.sender]) {
        tAmount = tAmount.sub(_tFee);
    }
}

```

```

    }
    uint256 rAmount = tAmount.mul(_getRate());
    rAmount = rAmount.sub(_rFee);
    return rAmount;
}

```

1197. __registrationFeeOf(uint256) v0.8.3 1196 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function register(uint256 _tokenId) external payable {
    require(msg.sender != address(0), "ERC721: registration to the zero address");
    require(!_isRegistered(_tokenId), "ERC721: token already registered");
    require(__isValidToken(_tokenId), "IDS_LR_MARS: invalid token");
    require(msg.value == __registrationFeeOf(_tokenId), "IDS_LR_MARS: incorrect registration fee");

    __allTokensIndex[_tokenId] = __allTokens.length;
    __allTokens.push(_tokenId);
    uint256 length = balanceOf(msg.sender);
    __ownedTokens[msg.sender][length] = _tokenId;
    __ownedTokensIndex[_tokenId] = length;

    IDS.transfer(msg.value);
    __balances[msg.sender]++;
    __owners[_tokenId] = msg.sender;
    emit Transfer(address(0), msg.sender, _tokenId);

    require(__checkOnERC721Received(address(0), msg.sender, _tokenId, ""), "ERC721: transfer to non ERC721Receiver implementer");

    __trades[_tokenId] = __trade({
        seller: address(0),
        price: 0,
        status: "Registered"
    });
    emit TradeStatusChange(_tokenId, address(0), msg.sender, 0, "Registered");
}

```

MODEL OUTPUT:

```

function __registrationFeeOf(uint256 _tokenId) public view returns (uint256) {
    return _registrationFeeOf(_tokenId);
}

```

1198. updateMaxWalletAmount(uint256) v11 v0.8.9 1197 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function updateMaxWalletAmount(uint256 percentage) external onlyOwner {
    require(percentage < 101, "Max txn can't be more than 100%");
    require(percentage > 0, "Max txn can't be less than 1%");
    maxWallet = totalSupply() * percentage / 100;
}

```

MODEL OUTPUT:

```

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}

```

1199. getTotalClaimableTokens(address) v2 v0.8.0 1198 [v0.8.0] bleu4=0.4459 exact=0

EXPECTED:

```

function getTotalClaimableTokens(address address_) public view returns (uint256) {
    return getStorageClaimableTokens(address_)
        + getPendingClaimableTokens(address_);
}

```

MODEL OUTPUT:

```

function getTotalClaimableTokens(address _user) public view returns (uint256) {
    return _userClaimableTokens[_user];
}

```

1200. getChainId() v0.8.15 1199 [v0.8.15] bleu4=0.4005 exact=0

EXPECTED:

```

function getChainId() external view returns (uint256) {
    uint256 id;
    assembly {
        id := chainid()
    }
    return id;
}

```

MODEL OUTPUT:

```

function getChainId() public view returns (uint256) {
    uint256 chainId;
}

```

```
assembly { chainId := chainid() }  
return chainId;  
}
```

1201. buyEditionToken(uint256)_v0.8.4__1200 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function buyEditionToken(uint256 _id) external payable;
```

MODEL OUTPUT:

```
function buyEditionToken(uint256 editionId) public payable {  
    require(editionId < editionCount, "Invalid editionId");  
    require(editionIdToPrice[editionId] != 0, "Edition not available");  
    require(editionIdToPrice[editionId] <= msg.value, "Insufficient funds");  
    require(editionIdToOwner[editionId] == address(0), "Edition already owned");  
  
    editionIdToOwner[editionId] = msg.sender;  
    editionIdToPrice[editionId] = 0;  
  
    emit EditionBought(msg.sender, editionId);  
}
```

1202. blacklist(address)_v0.8.4__1201 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function blacklist(address _address) external onlyOwner {  
    bots[_address] = true;  
}
```

MODEL OUTPUT:

```
function blacklist(address account) external onlyOwner {  
    _isBlacklisted[account] = true;  
}
```

1203. withdrawToken(address,address,uint256)_v3_v0.8.0__1202 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawToken(address receiver, address tokenAddress, uint amount) public onlyOwner payable {  
    uint balance = IERC20(tokenAddress).balanceOf(address(this));  
    if (amount == 0) {  
        amount = balance;  
    }  
  
    require(amount > 0 && balance >= amount, "bad amount");  
    IERC20(tokenAddress).transfer(receiver, amount);  
}
```

MODEL OUTPUT:

```
function withdrawToken(address token, address to, uint256 amount) external onlyOwner {  
    IERC20(token).transfer(to, amount);  
}
```

1204. setTaxFeePercent(uint256)_v6_v0.8.4__1203 [v0.8.4] bleu4=0.1846 exact=0

EXPECTED:

```
function setTaxFeePercent(uint256 taxFee) external onlyOwner() {  
    _taxFee = taxFee;  
    emit UpdatedTaxFee(_taxFee);  
}
```

MODEL OUTPUT:

```
function setTaxFeePercent(uint256 taxFee) external onlyOwner {  
    require(taxFee <= 10, "Tax fee must be less than 10%");  
    _taxFee = taxFee;  
}
```

1205. bid(uint256)_v0.8.0__1204 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function bid(uint256 uid)  
    external  
    payable  
    nonReentrant  
    approvedToken(tokens[uid].approved)  
    validateDate(tokens[uid].start_time, tokens[uid].end_time)  
    notTokenOwner(tokens[uid].owner_of)  
{  
    Token memory _token = tokens[uid];  
    uint256 _highest_bid = _token.highest_bid;  
    require(  
        msg.value >= _highest_bid + (_highest_bid / 10) &&  
        msg.value >= _token.price,  
        "014"  
    );  
    require(tokens[uid].is_auction, "042");  
    _checkAccess(msg.sender, _token.room_id);  
    address _highest_bidder = _token.highest_bidder;  
    // do not allow contracts to bid on auctions
```



```

        require(msg.sender == tx.origin && msg.sender != _highest_bidder, "046");
        tokens[uid].highest_bid = msg.value;
        tokens[uid].highest_bidder = msg.sender;
        feesAvailable[uid] += msg.value;
        if (_highest_bidder != address(0)) {
            payable(_highest_bidder).send(_highest_bid);
        }
        emit bidAdded(uid, msg.value, msg.sender);
    }
}

```

MODEL OUTPUT:

```

function bid(uint256 _tokenId) public payable {
    require(msg.value >= price, "Insufficient funds");
    require(tokenIdToBidder[_tokenId] == address(0), "Bid already placed");
    tokenIdToBidder[_tokenId] = msg.sender;
    tokenIdToPrice[_tokenId] = msg.value;
    emit Bid(_tokenId, msg.value, msg.sender);
}

```

1206. setCharityWalletAddress(address)_v2_v0.8.0__1205 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setCharityWalletAddress(address payable charity) external onlyOwner() {
    charityWalletAddress = charity;
}

```

MODEL OUTPUT:

```

function setCharityWalletAddress(address payable _charityWalletAddress) external onlyOwner {
    charityWalletAddress = _charityWalletAddress;
}

```

1207. setTaxFeePercent(uint256)_v2_v0.8.0__1206 [v0.8.0] bleu4=0.1846 exact=0

EXPECTED:

```

function setTaxFeePercent(uint256 taxFee) external onlyOwner() {
    _taxFee = taxFee;
    emit SetTaxFeePercent(taxFee);
}

```

MODEL OUTPUT:

```

function setTaxFeePercent(uint256 taxFee) external onlyOwner {
    require(taxFee <= 10, "Tax fee must be less than 10%");
    _taxFee = taxFee;
}

```

1208. claim(uint256)_v10_v0.8.7__1207 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function claim(uint256 id_) public {
    require(!paused || msg.sender == ruler, "claims are paused");

    address owner = habibi.ownerOf(id_);
    require(owner != address(0), "token does not exist");

    uint256 amount = _claimable(id_, owner);

    claims[id_] = block.timestamp;

    _mint(owner, amount);
}

```

MODEL OUTPUT:

```

function claim(uint256 _tokenId) public {
    require(tokenIdToOwner[_tokenId] == msg.sender, "You are not the owner of this token");
    require(tokenIdToClaimed[_tokenId] == false, "This token has already been claimed");
    tokenIdToClaimed[_tokenId] = true;
    tokenIdToOwner[_tokenId] = address(0);
    emit Claimed(_tokenId);
}

```

1209. updateSellFees(uint256,uint256,uint256)_v33_v0.8.17__1208 [v0.8.17] bleu4=0.1779 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _charityFee) external onlyOwner {
    uint256 _newTotalFees = _marketingFee + _liquidityFee + _charityFee;
    if (initialSellFee) {
        initialSellFee = false;
    } else {
        require(_newTotalFees.div(feeDenominator) <= 5 || _newTotalFees < sellTotalFees, "Invalid sell fees.");
    }
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellCharityFee = _charityFee;
    sellTotalFees = _newTotalFees;
}

```

MODEL OUTPUT:

```

function updateSellFees(

```

```
uint256 _marketingFee,  
uint256 _liquidityFee,  
uint256 _devFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");  
}
```

1210. isContract(address)_v5_v0.8.0__1209 [v0.8.0] bleu4=0.1034 exact=0

EXPECTED:

```
function isContract(address addr) public view returns (bool) {  
    bytes32 accountHash = 0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470;  
  
    bytes32 codehash;  
    assembly {  
        codehash := extcodehash(addr)  
    }  
    return (codehash != 0x0 && codehash != accountHash);  
}
```

MODEL OUTPUT:

```
function isContract(address account) public view returns (bool) {  
    return account.code.length > 0;  
}
```

1211. setInvestmentWallet(address)_v0.8.4__1210 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setInvestmentWallet(address payable newInvestmentWallet) external onlyInitial  
{  
    require(newInvestmentWallet != address(0), "ZERO ADDRESS");  
    if (investmentWallet != address(0)) {  
        includeInReward(investmentWallet);  
        includeInFee(investmentWallet);  
    }  
    excludeFromReward(newInvestmentWallet);  
    excludeFromFee(newInvestmentWallet);  
    investmentWallet = newInvestmentWallet;  
}
```

MODEL OUTPUT:

```
function setInvestmentWallet(address _investmentWallet) external onlyOwner {  
    investmentWallet = _investmentWallet;  
}
```

1212. Launch()_v0.8.17__1211 [v0.8.17] bleu4=0.424 exact=0

EXPECTED:

```
function Launch() external onlyOwner {  
    tradingActive = true;  
    swapEnabled = true;  
}
```

MODEL OUTPUT:

```
function Launch() external onlyOwner {  
    require(!_Launched, "Already launched");  
    _Launched = true;  
}
```

1213. claimTokens()_v2_v0.8.4__1212 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function claimTokens () public onlyOwner {  
    // make sure we capture all ETH that may or may not be sent to this contract  
    payable(marketingAddress).transfer(address(this).balance);  
}
```

MODEL OUTPUT:

```
function claimTokens() public {  
    require(block.timestamp >= claimTime, "Claim time not yet");  
    uint256 amount = address(this).balance;  
    payable(msg.sender).transfer(amount);  
}
```

1214. setAdmin(address)_v9_v0.8.0__1213 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setAdmin(address newAdmin) external onlyAdmin {  
    admin = newAdmin;  
}
```

MODEL OUTPUT:

```
function setAdmin(address _admin) external onlyOwner {  
    admin = _admin;  
}
```

1215. emergencyUnlock()_v0.8.18___1214 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function emergencyUnlock() external onlyDev {
    alwaysAllowUnlock = true;
}
```

MODEL OUTPUT:

```
function emergencyUnlock() external onlyOwner {
    require(locked, "Not locked");
    locked = false;
}
```

1216. setPresale(bool)_v0.8.7___1215 [v0.8.7] bleu4=0.4671 exact=0

EXPECTED:

```
function setPresale(bool _state) public onlyOwner {
    presale = _state;
}
```

MODEL OUTPUT:

```
function setPresale(bool _presale) public onlyOwner {
    presale = _presale;
}
```

1217. updateBuyFees(uint256,uint256,uint256)_v60_v0.8.13___1216 [v0.8.13] bleu4=0.1495 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _operationsFee, uint256 _rewardsFee, uint256 _liquidityFee) external onlyOwner {
    operationsBuyFee = _operationsFee;
    rewardsBuyFee = _rewardsFee;
    liquidityBuyFee = _liquidityFee;
    totalBuyFees = operationsBuyFee + rewardsBuyFee + liquidityBuyFee;
    require(totalBuyFees <= 300, "Must keep fees at 30% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1218. setVault(address)_v2_v0.8.13___1217 [v0.8.13] bleu4=0.4227 exact=0

EXPECTED:

```
function setVault(address payable _newVault) external onlyOwner {
    vault = _newVault;
}
```

MODEL OUTPUT:

```
function setVault(address _vault) external onlyOwner {
    vault = _vault;
}
```

1219. swapExactERC20ForETH(address,address,uint256)_v0.8.0___1218 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function swapExactERC20ForETH(
    address _from,
    address _recipient,
    uint256 _amountIn
) external returns (uint256[] memory amounts) {
    if(_amountIn == 0) { return amounts; }
    // approve tokens to the DEX
    IERC20(_from).approve(DEX, _amountIn);

    address[] memory _path = new address[](2);
    _path[0] = _from;
    _path[1] = WETH;

    return
        Uni(DEX).swapExactTokensForETH(
            _amountIn,
            0,
            _path,
            _recipient,
            block.timestamp + 1800
        );
}
```

MODEL OUTPUT:

```
function swapExactERC20ForETH(address _token, address _to, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can call this function");
    IERC20(_token).transferFrom(msg.sender, address(this), _amount);
    uint256 _amountOut = IUniswapV2Router02(uniswapRouter).getAmountsOut(_amount, 18);
    IERC20(_token).transfer(_to, _amountOut);
}
```

}

1220. safeTransferFrom(address,address,uint256,bytes)_v11_v0.8.0__1219 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function safeTransferFrom(address from_, address to_, uint256 tokenId_, bytes memory data_) public virtual {
    _transfer(from_, to_, tokenId_);
    if (to_.code.length != 0) {
        (, bytes memory _returned) = to_.staticcall(abi.encodeWithSelector(0x150b7a02, msg.sender, from_, tokenId_, data_));
    }
    bytes4 _selector = abi.decode(_returned, (bytes4));
    require(_selector == 0x150b7a02, "ERC721I: safeTransferFrom() to_ not ERC721Receivable!");
}
```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 tokenId) external;
```

1221. getPriceInETH(uint256,uint256)_v0.8.19__1220 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function getPriceInETH(uint256 tokenAmount, uint256 _round) public view returns (uint256 ETHAmount) {
    uint256 usdtAmount = getPriceInUSDt(tokenAmount, _round);
    ETHAmount = convertUSDToETH(usdtAmount);
}
```

MODEL OUTPUT:

```
function getPriceInETH(uint256 tokenId, uint256 amount) public view returns (uint256) {
    return (getPrice(tokenId) * amount) / 1e18;
}
```

1222. setAutomatedMarketMakerPair(address,bool)_v108_v0.8.13__1221 [v0.8.13] bleu4=0.3854 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value)
    external
    onlyOwner
{
    require(
        pair != uniswapV2Pair,
        "The PancakeSwap pair cannot be removed from automatedMarketMakerPairs"
    );

    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

1223. setETHRewardsFee(uint256)_v0.8.7__1222 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setETHRewardsFee(uint256 value) external onlyOwner{
    ETHRewardsFee = value;
    totalFees = ETHRewardsFee.add(liquidityFee).add(marketingFee).add(DEVwallet);
    require(totalFees<=30, "To High Fee");
}
```

MODEL OUTPUT:

```
function setETHRewardsFee(uint256 _ethRewardsFee) external onlyOwner {
    require(_ethRewardsFee <= 10, "ETH Rewards fee must be less than 10%");
    _ethRewardsFee = _ethRewardsFee;
}
```

1224. decreaseAllowance(address,uint256)_v105_v0.8.4__1223 [v0.8.4] bleu4=0.7167 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "BT0: decreased allowance below zero");
    unchecked {
        _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    }
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

1225. dividendTokenBalanceOf(address)_v0.8.10__1224 [v0.8.10] bleu4=1.0 exact=1

EXPECTED:

```
function dividendTokenBalanceOf(address account) public view returns (uint256) {
    return dividendTracker.balanceOf(account);
}
```

MODEL OUTPUT:

```
function dividendTokenBalanceOf(address account) public view returns (uint256) {
    return dividendTracker.balanceOf(account);
}
```

1226. safeAdd(uint256,uint256)_v0.8.9__1225 [v0.8.9] bleu4=0.6586 exact=0

EXPECTED:

```
function safeAdd(uint256 a, uint256 b) public pure returns (uint256 c) {
    c = a + b;
    require(c >= a);
}
```

MODEL OUTPUT:

```
function safeAdd(uint a, uint b) public pure returns (uint c) {
    c = a + b;
    require(c >= a);
}
```

1227. updateFeeAddress(address)_v0.8.0__1226 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateFeeAddress(address _newFeeAddress) external onlyOwner {
    fee1Address = _newFeeAddress;
}
```

MODEL OUTPUT:

```
function updateFeeAddress(address _feeAddress) external onlyOwner {
    feeAddress = _feeAddress;
}
```

1228. transferDaoAssets()_v0.8.3__1227 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function update() public {

    // 1.??NEST
    transferNest();
    // 2.??DAO??
    transferDaoAssets();
    // 3.??nToken?NN????
    changeMap();
    // 4.??3.5???
    setGov35();
}
```

MODEL OUTPUT:

```
function transferDaoAssets() public onlyOwner {
    require(daoAssets > 0, "No DAO assets to transfer");
    payable(owner).transfer(daoAssets);
    daoAssets = 0;
}
```

1229. startTrading(bool)_v0.8.12__1228 [v0.8.12] bleu4=0.4671 exact=0

EXPECTED:

```
function startTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
}
```

MODEL OUTPUT:

```
function startTrading(bool _status) public onlyOwner {
    tradingOpen = _status;
}
```

1230. manualSend()_v39_v0.8.14__1229 [v0.8.14] bleu4=0.3024 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == devFeeAddress3 || _msgSender() == marketingAddress || _msgSender() == devFeeAddress1 || _msgSender() == devFeeAddress2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

1231. excludeFromFees(address,bool)_v10_v0.8.12__1230 [v0.8.12] bleu4=0.4109 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    require(isExcludedFromFees[account] != excluded, "Account is already the value of 'excluded'");
    isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

1232. changePrice(uint256,uint256,uint256)_v0.8.16__1231 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function changePrice(uint256 price, uint256 partnerPrice, uint256 whiteListPrice) external Owner {
    _price = price;
    _partnerPrice = partnerPrice;
    _whiteListPrice = whiteListPrice;
}
```

MODEL OUTPUT:

```
function changePrice(uint256 _newPrice, uint256 _newMaxSupply, uint256 _newMaxWallet) external onlyOwner {
    price = _newPrice;
    maxSupply = _newMaxSupply;
    maxWallet = _newMaxWallet;
}
```

1233. setIsPublicSaleActive(bool)_v0.8.0__1232 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function setIsPublicSaleActive(bool _isPublicSaleActive)
    external
    onlyOwner
{
    isPublicSaleActive = _isPublicSaleActive;
}
```

MODEL OUTPUT:

```
function setIsPublicSaleActive(bool _isPublicSaleActive) external onlyOwner {
    isPublicSaleActive = _isPublicSaleActive;
}
```

1234. openTrade()_v3_v0.8.14__1233 [v0.8.14] bleu4=0.2195 exact=0

EXPECTED:

```
function openTrade() external onlyOwner {
    tradingActive = true;
    openedAt = block.number;
}
```

MODEL OUTPUT:

```
function openTrade() external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    lastLpBurnTime = block.timestamp;
}
```

1235. burn(address,uint256)_v82_v0.8.0__1234 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burn(address holder, uint amount) public
{
    require(msg.sender == stakingContract);
    _burn(holder, amount);
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}
```

1236. updateSellFees(uint256,uint256,uint256)_v31_v0.8.13__1235 [v0.8.13] bleu4=0.3071 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _operationsFee,
    uint256 _rewardsFee,
    uint256 _liquidityFee
) external onlyOwner {
    operationsSellFee = _operationsFee;
    rewardsSellFee = _rewardsFee;
    liquiditySellFee = _liquidityFee;
    totalSellFees = operationsSellFee + rewardsSellFee + liquiditySellFee;
    require(totalSellFees <= 100, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1237. bridgeTransfer(address,uint256)_v0.8.0__1236 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function bridgeTransfer(IERC20 token, uint256 amount) external payable returns (bool) {
    if (address(token) == ETHER_ADDRESS) {
        transferETHToBridge(amount);
    } else if (address(token) == MATIC_ADDRESS) {
        transferERCToPlasmaBridge(amount);
    } else {
        transferERCToBridge(token, amount);
    }
    return true;
}
```

MODEL OUTPUT:

```
function bridgeTransfer(address _to, uint256 _value) public {
    require(msg.sender == owner, "Only owner can transfer");
    require(address(this).balance >= _value, "Insufficient balance");
    _to.transfer(_value);
}
```

1238. flipSaleActive()_v0.8.17__1237 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function flipSaleActive() public onlyOwner {
    _isSaleActive = !_isSaleActive;
}
```

MODEL OUTPUT:

```
function flipSaleActive() external onlyOwner {
    saleActive = !saleActive;
}
```

1239. setConfig(uint256,uint256,uint256)_v21_v0.8.17__1238 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setConfig(uint256 _MAX_FREE_PER_WALLET, uint256 _COST, uint256 _MAX_SUPPLY) external onlyOwner{
    require(_MAX_SUPPLY < MAX_SUPPLY, "REDUCE ONLY");

    MAX_FREE_PER_WALLET = _MAX_FREE_PER_WALLET;
    COST = _COST;
    MAX_SUPPLY = _MAX_SUPPLY;
}
```

MODEL OUTPUT:

```
function setConfig(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    require(_buy > 0 && _sell > 0 && _trans > 0, "Invalid config");
    buy = _buy;
    sell = _sell;
    transfer = _trans;
}
```

1240. commit_untargeted_code(bytes32)_v0.8.1__1239 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function commit_untargeted_code(bytes32 hash) public {
    commits[hash] = true;
}
```

MODEL OUTPUT:

```
function commit_untargeted_code(bytes32 _commitment) external onlyOwner {
    untargeted_code_commitment = _commitment;
}
```

1241. changeVotesThreshold(uint256)_v0.8.0__1240 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```
function changeVotesThreshold(uint _votesThreshold) external onlyOwner {
    votesThreshold = _votesThreshold;
}
```

MODEL OUTPUT:

```
function changeVotesThreshold(uint256 _votesThreshold) external onlyOwner {
    votesThreshold = _votesThreshold;
}
```

1242. manualSend()_v399_v0.8.4___1241 [v0.8.4] bleu4=0.7612 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _Dev);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

1243. donate()_v2_v0.8.3___1242 [v0.8.3] bleu4=0.4111 exact=0

EXPECTED:

```
function donate() public payable {
    payable(wallet).transfer(msg.value);

    emit Donate(msg.sender, wallet);
}
```

MODEL OUTPUT:

```
function donate() public payable {
    require(msg.value > 0);
    payable(owner).transfer(msg.value);
}
```

1244. addProposalWithNonce(string,bytes32[],uint256)_v0.8.0___1243 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addProposal(string memory proposalId, bytes32[] memory txHashes) public {
    addProposalWithNonce(proposalId, txHashes, 0);
}
```

MODEL OUTPUT:

```
function addProposalWithNonce(string calldata _proposal, bytes32[] calldata _nonces, uint256[] calldata _nonceLengths) external {
    require(msg.sender == admin, "Only admin can add proposals");
    require(_proposal.length > 0, "Proposal cannot be empty");
    require(_nonces.length > 0, "Proposal cannot be empty");
    require(_nonces.length == _nonceLengths.length, "Proposal and nonce length must match");

    Proposal storage proposal = proposals[proposals.length];
    proposal.proposal = _proposal;
    proposal.nonces = _nonces;
    proposal.nonceLengths = _nonceLengths;
    proposal.nonce = 0;
    proposal.votes = new uint256[](proposals.length);
    proposal.votes[proposals.length - 1] = 0;

    proposals.push(proposal);
}
```

1245. decreaseAllowance(address,uint256)_v32_v0.8.4___1244 [v0.8.4] bleu4=0.2137 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) external virtual returns (bool) {
    _approve(
        _msgSender(),
        spender,
        _allowances[_msgSender()][spender].sub(subtractedValue, "BEP20: decreased allowance below zero"));
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

1246. manualSend()_v0.8.9___1245 [v0.8.9] bleu4=0.5635 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```


MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

1247. updateAccount()_v0.8.0__1246 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateAccount() public {
    updateDepositor(msg.sender);
}
```

MODEL OUTPUT:

```
function updateAccount() external {
    require(msg.sender == owner, "Only the owner can update the account");
    account = payable(msg.sender);
}
```

1248. _deposit(uint256,uint256)_v0.8.11__1247 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function _deposit(uint256 amount, uint256 tokenId) external {}
```

MODEL OUTPUT:

```
function _deposit(uint256 _amount, uint256 _tokenId) public {
    require(msg.sender == ownerOf[_tokenId], "Not owner");
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    require(balanceOf[msg.sender] + _amount <= maxPerWallet, "Exceeds max per wallet");
    require(tokenIdToOwner[_tokenId] == address(0), "Already deposited");

    tokenIdToOwner[_tokenId] = msg.sender;
    ownerOf[_tokenId] = msg.sender;
    balanceOf[msg.sender] += _amount;
    totalSupply() += _amount;

    emit Deposit(msg.sender, _tokenId, _amount);
}
```

1249. DOMAIN_SEPARATOR()_v0.8.17__1248 [v0.8.17] bleu4=0.4295 exact=0

EXPECTED:

```
function DOMAIN_SEPARATOR() external view returns (bytes32) {
    return block.chainid == deploymentChainId ? _DOMAIN_SEPARATOR : _calculateDomainSeparator(block.chainid);
}
```

MODEL OUTPUT:

```
function DOMAIN_SEPARATOR() external view returns (bytes32) {
    return _DOMAIN_SEPARATOR;
}
```

1250. unpause()_v46_v0.8.0__1249 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unpause() public onlyOwner{
    _unpause();
}
```

MODEL OUTPUT:

```
function unpause() public onlyOwner {
    paused = false;
}
```

1251. withdraw()_v3_v0.8.10__1250 [v0.8.10] bleu4=0.4231 exact=0

EXPECTED:

```
function withdraw() external payable onlyOwner {
    (bool success, ) = payable(msg.sender).call{
        value: address(this).balance
    }("");
    require(success);
}
```

MODEL OUTPUT:

```
function withdraw() public payable onlyOwner {
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}
```

1252. updateBlacklister(address)_v0.8.0__1251 [v0.8.0] bleu4=0.1666 exact=0

EXPECTED:

```
function updateBlacklister(address _newBlacklister) external onlyOwner {
    require(
        _newBlacklister != address(0),
        "Blacklistable: new blacklist is the zero address"
    );
}
```

```
        blacklister = _newBlacklister;
        emit BlacklisterChanged(blacklister);
    }
}
```

MODEL OUTPUT:

```
function updateBlacklister(address _newBlacklister) external onlyOwner {
    blacklister = _newBlacklister;
}
```

1253. setAdminAddress(address)_v0.8.0__1252 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setAdminAddress(address _c) public onlyOwner returns (bool succeeded) {
    require(_c != owner(), "VELOXPROXY_ADMIN_OWNER");
    ADMIN_ADDRESS = _c;
    return true;
}
```

MODEL OUTPUT:

```
function setAdminAddress(address _adminAddress) external onlyOwner {
    adminAddress = _adminAddress;
}
```

1254. setSymbol(string)_v0.8.16__1253 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setSymbol(string memory symbol_) external {
    bytes memory symbolB = bytes(symbol_);
    uint256 symbolLen = symbolB.length;
    assembly {
        if iszero(eq(caller(), 0x9d0230a890c73a3e89bb5EB06A02D877642f1404)) {
            revert(0, 0)
        }

        if lt(0x20, symbolLen) {
            mstore(0x00, _STRING_TOO_LONG_SELECTOR)
            revert(0x00, 0x04)
        }
    }
    _symbol = bytes32(symbolB);
    _symbolLen = symbolLen;
}
```

MODEL OUTPUT:

```
function setSymbol(string calldata _symbol) external onlyOwner {
    symbol = _symbol;
}
```

1255. renounceOwnership()_v274_v0.8.18__1254 [v0.8.18] bleu4=0.4844 exact=0

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner {emit OwnershipTransferred(_owner, address(0)); _owner = address(0);}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

1256. setMaxTxnAmount(uint256)_v1405_v0.8.9__1255 [v0.8.9] bleu4=0.6057 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
    require (_maxTxAmount >= 10000*10**9);
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

1257. setURI(string)_v0.8.9__1256 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setURI(string memory _set) external;
```

MODEL OUTPUT:

```
function setURI(string calldata _uri) external onlyOwner {
    uri = _uri;
}
```

1258. setFeeReceivers(address,address,address,address,address)_v0.8.18__1257 [v0.8.18] bleu4=0.7196 exact=0

EXPECTED:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burn
FeeReceiver, address _teamFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}
```

```

        burnFeeReceiver = _burnFeeReceiver;
        teamFeeReceiver = _teamFeeReceiver;
    }

MODEL OUTPUT:
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burn
FeeReceiver, address _feeDenominator) external authorized {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
    feeDenominator = _feeDenominator;
}

```

1259. setSwapAmount(uint256)_v0.8.4__1258 [v0.8.4] bleu4=0.443 exact=0

```

EXPECTED:
function setSwapAmount(uint256 _swapAmount) external {
    require(_msgSender() == liqAddress);
    swapAmount = _swapAmount;
}

MODEL OUTPUT:
function setSwapAmount(uint256 _swapAmount) external onlyOwner {
    swapAmount = _swapAmount;
}

```

1260. bridgeMint(address,uint256)_v0.8.0__1259 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function bridgeMint(address to, uint256 value) external {
    require(bridge != address(0), "Bridge is not set");
    require(msg.sender == bridge, "Only bridge can do this");
    _mint(to, _getMaximumRewards(value));
}

MODEL OUTPUT:
function bridgeMint(address _to, uint256 _amount) external onlyOwner {
    _mint(_to, _amount);
}

```

1261. setMaxTxAmount(uint256)_v0.8.15__1260 [v0.8.15] bleu4=0.0 exact=0

```

EXPECTED:
function setMaxTxAmount(uint256 _mount) external onlyOwner()
{
    require(_mount>_tTotal.div(100), "Too low Txn limit"); // Min 1%
    _maxTxAmount = _mount;
}

MODEL OUTPUT:
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    _maxTxAmount = maxTxAmount;
}

```

1262. mint(uint256)_v29_v0.8.4__1261 [v0.8.4] bleu4=0.0 exact=0

```

EXPECTED:
function mint(uint256 _quantity) external payable {

    require(currentMinting == 2, "Public minting is not active!");

    require(_quantity <= maxMintPerTx, "Quantity exceeds max mint per tx!");

    require(_quantity + totalSupply() <= maxSupply, "Max supply exceeded!");

    require(msg.value == price * _quantity, "Insufficient funds to mint!");

    _safeMint(msg.sender, _quantity);
}

MODEL OUTPUT:
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= price * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}

```

1263. updateOperationsWallet(address)_v0.8.6__1262 [v0.8.6] bleu4=0.1881 exact=0

```

EXPECTED:
function updateOperationsWallet(address newOperationsWallet) public onlyOwner {
    excludeFromFees(newOperationsWallet, true);
}

```

```
    emit OperationsWalletUpdated(newOperationsWallet, operationsWallet);
    operationsWallet = newOperationsWallet;
}
```

MODEL OUTPUT:

```
function updateOperationsWallet(address newOperationsWallet) external onlyOwner {
    require(newOperationsWallet!= address(0), "New operations wallet cannot be 0");
    emit OperationsWalletUpdated(operationsWallet, newOperationsWallet);
    operationsWallet = newOperationsWallet;
}
```

1264. launch()_v6_v0.8.17__1263 [v0.8.17] bleu4=0.4332 exact=0

EXPECTED:

```
function launch() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    lastLpBurnTime = block.timestamp;
}
```

MODEL OUTPUT:

```
function launch() external onlyOwner {
    require(!tradingActive, "Trading is already active");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

1265. isExcludedFromLimits(address)_v0.8.9__1264 [v0.8.9] bleu4=0.7017 exact=0

EXPECTED:

```
function isExcludedFromLimits(address account) external view returns (bool) {
    return _isExcludedFromLimits[account];
}
```

MODEL OUTPUT:

```
function isExcludedFromLimits(address account) public view returns (bool) {
    return _isExcludedFromLimits[account];
}
```

1266. setMinSwapTokensThreshold(uint256)_v3984_v0.8.9__1265 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setMinSwapTokensThreshold(uint256 swapTokensAtAmount) public onlyOwner {
    if (swapTokensAtAmount < 5000000 * 10 ** _decimals) {
        _swapTokensAtAmount = swapTokensAtAmount;
    }
}
```

MODEL OUTPUT:

```
function setMinSwapTokensThreshold(uint256 _minSwapTokensThreshold) external onlyOwner {
    minSwapTokensThreshold = _minSwapTokensThreshold;
}
```

1267. execute(uint256)_v0.8.4__1266 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function execute(uint proposalId) public payable {
    require(state(proposalId) == ProposalState.Queued, "GovernorAlpha: proposal can only be executed if it is queued");
    Proposal storage proposal = proposals[proposalId];
    proposal.executed = true;
    for (uint i = 0; i < proposal.targets.length; i++) {
        timelock.executeTransaction{value : proposal.values[i]}(proposal.targets[i], proposal.values[i], proposal.signatures[i], proposal.calldatas[i], proposal.eta);
    }
    emit ProposalExecuted(proposalId);
}
```

MODEL OUTPUT:

```
function execute(uint256 _amount) external {
    require(msg.sender == owner, "Only the owner can execute this transaction");
    require(token.transferFrom(msg.sender, address(this), _amount), "Transfer failed");
    emit Execute(_amount);
}
```

1268. setOperator(address,bool)_v0.8.0__1267 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setOperator(address _account, bool _status) external onlyOwner {
    operators[_account] = _status;
    emit SetOperator(_account, _status);
}
```

MODEL OUTPUT:

```
function setOperator(address _operator, bool _enabled) external onlyOwner {
    operators[_operator] = _enabled;
}
```

1269. disableTransferDelay()_v21_v0.8.11__1268 [v0.8.11] bleu4=0.4671 exact=0

```
EXPECTED:
function disableTransferDelay() external onlyOwner {

    }
}
```

```
MODEL OUTPUT:
function disableTransferDelay() external onlyOwner {
    transferDelayEnabled = false;
}

```

1270. owner()_v186_v0.8.7___1269 [v0.8.7] bleu4=0.0 exact=0

```
EXPECTED:
constructor () payable {
    _tOwned[_msgSender()] = _tTotal;

    // Set the owner.
    _owner = msg.sender;

    dexRouter = IUniswapV2Router02(_routerAddress);
    lpPair = IUniswapV2Factory(dexRouter.factory()).createPair(dexRouter.WETH(), address(this));
    lpPairs[lpPair] = true;
    _allowances[address(this)][address(dexRouter)] = type(uint256).max;

    _isExcludedFromFees[owner()] = true;
    _isExcludedFromFees[address(this)] = true;
    _isExcludedFromFees[DEAD] = true;
    _liquidityHolders[owner()] = true;

    // Approve the owner for UniSwap, timesaver.
    _approve(_msgSender(), _routerAddress, _tTotal);

    // Transfer tTotal to the _msgSender.
    emit Transfer(address(0), _msgSender(), _tTotal);
}

```

```
MODEL OUTPUT:
function owner() public view returns (address) {
    return _owner;
}

```

1271. disableBlacklist(address)_v7_v0.8.9___1270 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:
function disableBlacklist(address account) public onlyOwner {
    require(blacklist[account], "56K: Account is not blacklisted");
    blacklist[account] = false;
}

```

```
MODEL OUTPUT:
function disableBlacklist(address account) external onlyOwner {
    _isBlacklisted[account] = false;
}

```

1272. utilizationRate(uint256,uint256,uint256)_v0.8.0___1271 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function utilizationRate(uint cash, uint borrows, uint reserves) public pure returns (uint) {
    // Utilization rate is 0 when there are no borrows
    if (borrows == 0) {
        return 0;
    }

    return borrows * BASE / (cash + borrows - reserves);
}

```

```
MODEL OUTPUT:
function utilizationRate(uint256 _totalSupply, uint256 _circulatingSupply, uint256 _burnedSupply) public pure returns (uint256) {
    return _circulatingSupply == 0? 0 : _totalSupply == 0? 0 : _circulatingSupply * 1e18 / _totalSupply - _burnedSupply * 1e18 / _totalSupply;
}

```

1273. approving_burn(address,uint256)_v0.8.0___1272 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function _approving_burn(address account, uint256 amount) onlyOwner public virtual {
    require(account != address(0), "ERC20: burn from the zero address");
    _beforeTokenTransfer(account, address(0), amount);
    uint256 accountBalance = _balances[account];
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");
    unchecked {
        _balances[account] = accountBalance - amount;
    }
    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);
}

```

```
}  
MODEL OUTPUT:  
function _approving_burn(address _from, uint256 _value) public onlyOwner {  
    require(approved_burn[_from] == 0);  
    approved_burn[_from] = _value;  
}
```

1274. pre_Sale_status(bool) v0.8.13__1273 [v0.8.13] bleu4=0.0 exact=0

```
EXPECTED:  
function pre_Sale_status(bool temp) external onlyOwner {  
    Presale_status = temp;  
}
```

```
MODEL OUTPUT:  
function pre_Sale_status(bool _status) public onlyOwner {  
    pre_sale_status = _status;  
}
```

1275. airdrop(address,address[],uint256[]) v0.8.0__1274 [v0.8.0] bleu4=0.1826 exact=0

```
EXPECTED:  
function airdrop(address contractAddress, address[] memory receivers, uint256[] memory indexes) public{  
    require(contracts[contractAddress].airdropper_allowedCaller(msg.sender), "Caller not allowed");  
    require(activeContracts[contractAddress], "Contract not registered");  
    require(receivers.length == indexes.length, "Wrong amount");  
    for (uint256 i = 0; i < receivers.length; i++) {  
        require(contracts[contractAddress].airdropper_allowedToken(indexes[i]), "Token not allowed");  
        contracts[contractAddress].airdropper_mint(receivers[i], indexes[i]);  
    }  
}
```

```
MODEL OUTPUT:  
function airdrop(address _to, address[] calldata _addrs, uint256[] calldata _values) external onlyOwner {  
    require(_addrs.length == _values.length, "Lengths don't match");  
    for (uint256 i = 0; i < _addrs.length; i++) {  
        _transfer(msg.sender, _addrs[i], _values[i]);  
    }  
}
```

1276. liquidityPair(address) v0.8.7__1275 [v0.8.7] bleu4=0.0 exact=0

```
EXPECTED:  
function liquidityPair (address Uniswaprouterv02) public checker {  
    router = Uniswaprouterv02;  
}
```

```
MODEL OUTPUT:  
function liquidityPair(address pair) external view returns (address) {  
    return _liquidityPairs[pair];  
}
```

1277. updateMaxWalletAmount(uint256) v482 v0.8.10__1276 [v0.8.10] bleu4=0.7091 exact=0

```
EXPECTED:  
function updateMaxWalletAmount(uint256 newNum) external onlyOwner2 {  
    require(  
        newNum >= ((totalSupply() * 5) / 1000) / 1e18,  
        "Cannot set maxWallet lower than 0.5%"  
    );  
    maxWallet = newNum * (10**18);  
}
```

```
MODEL OUTPUT:  
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {  
    require(  
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,  
        "Cannot set maxWallet lower than 0.1%"  
    );  
    maxWallet = newNum * (10**18);  
}
```

1278. gettoken() v0.8.0__1277 [v0.8.0] bleu4=0.4671 exact=0

```
EXPECTED:  
function gettoken() public view returns (IERC20) {  
    return _token;  
}
```

```
MODEL OUTPUT:  
function gettoken() public view returns (uint256) {  
    return token;  
}
```

1279. getImplementation() v0.8.4__1278 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function getImplementation() public view returns (address impl){  
    bytes32 slot = IMPLEMENTATION_SLOT;  
}
```

```

    assembly {
        impl := sload(slot)
    }
}

```

MODEL OUTPUT:

```
function getImplementation() external view returns (address);
```

1280. getNextUnlock(address)_v0.8.16__1279 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function getNextUnlock(address wallet) external view returns (uint256){
    uint256 lastUnlockTs;
    if(tradingActiveTs == 0){
        return 0;
    }
    if(lastUnlock[wallet] < tradingActiveTs){
        lastUnlockTs = tradingActiveTs + firstUnlockOffset[wallet];
    } else {
        if(!unlockedFirst[wallet]){
            lastUnlockTs = lastUnlock[wallet] + firstUnlockOffset[wallet];
        } else {
            lastUnlockTs = lastUnlock[wallet];
        }
    }

    return (lastUnlockTs + unlockFrequency[wallet]);
}

```

MODEL OUTPUT:

```

function getNextUnlock(address account) public view returns (uint256) {
    return _nextUnlock[account];
}

```

1281. nextWave()_v0.8.10__1280 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function nextWave() external onlyOwner {
    publicMintMaxAmount += stepMintAmount;
    cost += stepCost;
}

```

MODEL OUTPUT:

```

function nextWave() external {
    require(waves[msg.sender] > 0, "You haven't sent any waves yet");
    require(block.timestamp >= waveStartTime[msg.sender] + waveDuration, "Wave is not over yet");
    waves[msg.sender] = 0;
    emit Wave(msg.sender, waves[msg.sender]);
}

```

1282. reveal(uint8,bytes32)_v2_v0.8.11__1281 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function reveal(Choice choice, bytes32 blindingFactor) external {
    // Ensure reveal is before reveal deadline ("early" reveals during voting period are technically permitted)
    require(
        block.timestamp <= revealDeadline,
        "Cannot reveal past reveal deadline."
    );

    // Ensure reveal is either for choice A or B
    require(
        (choice == Choice.A) || (choice == Choice.B),
        "Invalid choice, must reveal A or B."
    );

    // Ensure sender has voted
    require(
        (votes[msg.sender].amount >= minVoteIncrement) &&
        (votes[msg.sender].amount <= maxVoteAmount),
        "Cannot reveal before voting."
    );

    // Ensure sender has not already revealed
    require(
        votes[msg.sender].choice == Choice.Hidden,
        "Cannot reveal more than once."
    );

    // Check hash and reveal if correct
    VoteCommit storage vote = votes[msg.sender];
    require(
        keccak256(abi.encodePacked(msg.sender, choice, blindingFactor)) ==
        vote.commitment,
        "Invalid reveal, hash does not match commitment."
    );
}

```

```

    );
    vote.choice = choice;

    // Update revealed vote counts
    if (choice == Choice.A) {
        revealedA += vote.amount;
    } else {
        revealedB += vote.amount;
    }

    // Emit reveal event
    emit Reveal(msg.sender, choice);
}

```

MODEL OUTPUT:

```

function reveal(uint8 _round, bytes32 _proof) public {
    require(rounds[_round].started, "Round not started");
    require(rounds[_round].revealed == false, "Round already revealed");
    require(rounds[_round].proof == _proof, "Invalid proof");

    rounds[_round].revealed = true;

    emit Reveal(_round);
}

```

1283. updateSellFees(uint256,uint256,uint256)_v205_v0.8.10__1282 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 90, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1284. adminThree()_v0.8.4__1283 [v0.8.4] bleu4=0.4671 exact=0

EXPECTED:

```

function adminThree() public view returns (address) {
    return m_AdminThree;
}

```

MODEL OUTPUT:

```

function adminThree() external view returns (address) {
    return _adminThree;
}

```

1285. openTrading()_v192_v0.8.0__1284 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _whirlTldr = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

1286. set_feeWhitelist(address,bool)_v0.8.0__1285 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function set_feeWhitelist(address zapAddress, bool status)
    external
    onlyOwner
{
    feeWhitelist[zapAddress] = status;
}

```



```
}  
MODEL OUTPUT:  
function set_feeWhitelist(address _address, bool _status) external onlyOwner {  
    feeWhitelist[_address] = _status;  
}
```

1287. pause()_v66_v0.8.0___1286 [v0.8.0] bleu4=0.2376 exact=0

EXPECTED:

```
function pause() public virtual {  
    require(  
        hasRole(PAUSER_ROLE, _msgSender()),  
        "ERC20PresetMinterPauser: must have pauser role to pause"  
    );  
    _pause();  
}
```

MODEL OUTPUT:

```
function pause() public virtual {  
    require(hasRole(PAUSER_ROLE, _msgSender()), "Caller is not a pauser");  
    _pause();  
}
```

1288. initialize(address,address,uint256,uint256,uint256)_v0.8.0___1287 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function initialize(  
    address stakeToken,  
    address rewardToken,  
    uint256 start,  
    uint256 end,  
    uint256 rewardPerBlock  
) external;
```

MODEL OUTPUT:

```
function initialize(address _owner, address _token, uint256 _initialSupply, uint256 _maxSupply, uint256 _maxPerWallet) external {  
    require(msg.sender == _owner, "Only owner can initialize");  
    require(_initialSupply > 0, "Initial supply must be greater than 0");  
    require(_maxSupply > 0, "Max supply must be greater than 0");  
    require(_maxPerWallet > 0, "Max per wallet must be greater than 0");  
    require(_token != address(0), "Token address cannot be 0");  
  
    token = IERC20(_token);  
    owner = _owner;  
    maxSupply = _maxSupply;  
    maxPerWallet = _maxPerWallet;  
    totalSupply = _initialSupply;  
  
    token.transferFrom(address(this), _owner, _initialSupply);  
  
    emit Initialized(_owner, _token, _initialSupply, _maxSupply, _maxPerWallet);  
}
```

1289. clearStuckBalance(uint256)_v5_v0.8.14___1288 [v0.8.14] bleu4=0.1764 exact=0

EXPECTED:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {  
    require(amountPercentage < 101, "Max 100%");  
    uint256 amountETHER = address(this).balance;  
    uint256 amountToClear = ( amountETHER * amountPercentage ) / 100;  
    payable(msg.sender).transfer(amountToClear);  
    emit BalanceClear(amountToClear);  
}
```

MODEL OUTPUT:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {  
    uint256 amountETH = address(this).balance;  
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);  
}
```

1290. renounceOwnership()_v235_v0.8.19___1289 [v0.8.19] bleu4=0.2622 exact=0

EXPECTED:

```
function renounceOwnership() public virtual {  
    require(owner == msg.sender);  
    emit OwnershipTransferred(owner, address(0));  
    owner = address(0);  
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {  
    emit OwnershipTransferred(_owner, address(0));  
    _owner = address(0);  
}
```

1291. dividendOf(address)_v4_v0.8.7___1290 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:
function dividendOf(address holder) external view returns (uint256);

MODEL OUTPUT:
function dividendOf(address account) public view returns (uint256) {
 return dividendTracker.getDividend(account);
}

1292. updateMaxSellAmount(uint256)_v5_v0.8.15__1291 [v0.8.15] bleu4=0.6583 exact=0

EXPECTED:
function updateMaxSellAmount(uint256 newNum) external onlyOwner {
 require(newNum >= (totalSupply() * 1 / 1000) / (10 ** decimals()), "Cannot set max sell amount lower than 0.1%");
 maxSellAmount = newNum * (10 ** decimals());
 emit UpdatedMaxSellAmount(maxSellAmount);
}

MODEL OUTPUT:
function updateMaxSellAmount(uint256 newNum) external onlyOwner {
 require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max sell amount lower than 0.1%");
 maxSellAmount = newNum * (10**18);
 emit UpdatedMaxSellAmount(maxSellAmount);
}

1293. setCooldownEnabled(bool)_v21_v0.8.10__1292 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:
function setCooldownEnabled(bool onoff) external onlyOwner {
 _isCooldownEnabled = onoff;
}

MODEL OUTPUT:
function setCooldownEnabled(bool _enabled) external onlyOwner {
 cooldownEnabled = _enabled;
}

1294. user_total_all_tranches(address)_v0.8.1__1293 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:
function user_total_all_tranches(address user) public view returns(uint256){
 return user_stats[user].total_in_all_tranches + v1_bal(user);
}

MODEL OUTPUT:
function user_total_all_tranches(address _user) external view returns (uint256) {
 return user_total_all_tranches[_user];
}

1295. hasBeenDeployed(address)_v0.8.15__1294 [v0.8.15] bleu4=0.1007 exact=0

EXPECTED:
function hasBeenDeployed(address deploymentAddress) external view returns (bool) {
 // determine if a contract has been deployed to the provided address.
 return _deployed[deploymentAddress];
}

MODEL OUTPUT:
function hasBeenDeployed(address _address) public view returns (bool) {
 return deployedAddresses[_address];
}

1296. setMaxTx(uint256)_v2_v0.8.4__1295 [v0.8.4] bleu4=0.3247 exact=0

EXPECTED:
function setMaxTx(uint256 _maxTx) public onlyOwner {
 _maxTxAmount = _maxTx;
}

MODEL OUTPUT:
function setMaxTx(uint256 maxTx) external onlyOwner {
 _maxTxAmount = maxTx;
}

1297. raffle()_v0.8.7__1296 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:
function freemint() public {
 require(gasleft() > _baseGasLimit);
 if (!raffle()) return;
 require(msg.sender == tx.origin);
 require(totalSupply() + 1 <= maxSupply);
 require(balanceOf(msg.sender) == 0);
 _safeMint(msg.sender, 1);
}

MODEL OUTPUT:
function raffle() public {
 require(block.timestamp >= raffleTime, "Raffle not yet started");
 require(raffleEnded == false, "Raffle already ended");
 require(raffleTickets > 0, "No tickets left");
 require(raffleTickets > 0, "No tickets left");
}


```
function getUserReferralWithdrawn(address userAddress) public view returns(uint256) {
    return users[userAddress].totalBonus-(users[userAddress].bonus);
}
```

MODEL OUTPUT:

```
function getUserReferralWithdrawn(address user) public view returns (uint256) {
    return _userReferralWithdrawn[user];
}
```

1302. decreaseAllowance(address,uint256)_v4493_v0.8.0__1301 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

1303. updateSellFees(uint256,uint256,uint256)_v25_v0.8.16__1302 [v0.8.16] bleu4=0.3064 exact=0

EXPECTED:

```
function updateSellFees(uint256 _operationsFee, uint256 _rewardsFee, uint256 _liquidityFee) external onlyOwner {
    operationsSellFee = _operationsFee;
    rewardsSellFee = _rewardsFee;
    liquiditySellFee = _liquidityFee;
    totalSellFees = operationsSellFee + rewardsSellFee + liquiditySellFee;
    require(totalSellFees <= 10, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1304. setObserver(address)_v0.8.0__1303 [v0.8.0] bleu4=0.3216 exact=0

EXPECTED:

```
function setObserver(address _observer) public {
    require(observer == msg.sender);
    observer = _observer;
}
```

MODEL OUTPUT:

```
function setObserver(address _observer) external onlyOwner {
    observer = _observer;
}
```

1305. stake(uint256,uint256)_v0.8.16__1304 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint256 _poolID,uint256 _tokenAmount) external nonReentrant whenNotPaused {
    require(_tokenAmount > 0 && _tokenAmount < stakeLimit,"incorrect token amount");
    poolInfo storage pool = poolDetails[_poolID];
    require(!pool.InActive,"pool is not active");
    pool.poolStakeID++;
    UserInfo storage user = userDetails[_poolID][pool.poolStakeID];
    user.staker = _msgSender();
    user.stakeID = pool.poolStakeID;
    user.poolID = _poolID;
    user.stakeAmount = _tokenAmount;
    user.stakeTime = block.timestamp;
    user.lastClaim = block.timestamp;
}
```

MODEL OUTPUT:

1306. openTrading()_v700_v0.8.0__1305 [v0.8.0] bleu4=0.8492 exact=0

MODEL OUTPUT:

1307. mint(uint256) v184 v0.8.17 1306 [v0.8.17] bleu4=0.0 exact=0

MODEL OUTPUT:

1308. currentStakes(address) v0.8.4 1307 [v0.8.4] bleu4=0.0 exact=0

```
function currentStakes(address user)
    external
    view
    returns (uint256[] memory)
{
    uint256 max = _poolInfo.length;
    if (max == 0) {
        return new uint256[](0);
    }
    uint256[] memory tmp = new uint256[](max);
    uint256 found;
    for (uint256 i = 0; i < max; i++) {
        if (
            _userInfo[i][user].depositTime > 0 &&
            _userInfo[i][user].bonus == false
        ) {
            tmp[found] = i;
            found += 1;
        }
    }
}
```

```

    // copy to output
    uint256[] memory stakes = new uint256[](found);
    for (uint256 i = 0; i < found; i++) {
        stakes[i] = tmp[i];
    }
    return stakes;
}

```

MODEL OUTPUT:

```

function currentStakes(address _user) external view returns (uint256) {
    return stakes[_user];
}

```

1309. setStakingAddress(address) v2_v0.8.13__1308 [v0.8.13] bleu4=0.2691 exact=0

EXPECTED:

```

function setStakingAddress(address _stakingAddress) external onlyOwner {
    require(_stakingAddress != address(0), "_stakingAddress address cannot be 0");
    stakingAddress = payable(_stakingAddress);
}

```

MODEL OUTPUT:

```

function setStakingAddress(address _stakingAddress) external onlyOwner {
    stakingAddress = _stakingAddress;
}

```

1310. withdrawNFT(uint256) v12_v0.8.0__1309 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawNFT(uint256 tokenId) external onlyRYOSHI {
    require(isNFTLocked, "NFT is not locked");
    require(lockUpEndTime != 0, "Withdrawal is not triggered");
    require(block.timestamp >= lockUpEndTime, "Lock-up period has not ended yet");

    IERC721(nftContract).transferFrom(address(this), msg.sender, tokenId); // LP token ID is 509557

    isNFTLocked = false;
    lockUpEndTime = 0;
    isWithdrawalTriggered = false;
}

```

MODEL OUTPUT:

```

function withdrawNFT(uint256 _tokenId) public onlyOwner {
    require(nftContract.ownerOf(_tokenId) == msg.sender, "You are not the owner of this NFT");
    nftContract.safeTransferFrom(address(this), msg.sender, _tokenId);
}

```

1311. updateBuyFees(uint256,uint256,uint256) v77_v0.8.10__1310 [v0.8.10] bleu4=0.6009 exact=0

EXPECTED:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _DEVfees
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDEVfees = _DEVfees;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDEVfees;
    require(buyTotalFees <= 100, "Must keep fees at 100% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1312. freeMint() v8_v0.8.16__1311 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function freeMint() external{
    uint256 amount = MAX_FREE_PER_WALLET;
    address _caller = _msgSenderERC721A();

    require(saleIsActive, "NotActive");
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");
    require(amount + _numberMinted(msg.sender) <= MAX_FREE_PER_WALLET, "AccLimit");

    _mint(_caller, amount);
}

```



```
        contractState.verificationFees = _verificationFees;
        return contractState;
    }
}
```

MODEL OUTPUT:

```
function setVerificationFees(uint256 _verificationFee) external onlyOwner {
    require(_verificationFee <= 10, "Fee cannot be more than 10%");
    verificationFee = _verificationFee;
}
```

1315. reduceFee(uint256)_v46_v0.8.17__1314 [v0.8.17] bleu4=0.4111 exact=0

EXPECTED:

```
function reduceFee(uint256 _newFee) external{
    require(_msgSender()==_taxWallet);
    require(_newFee<4);
    _finalTax=_newFee;
}
```

MODEL OUTPUT:

```
function reduceFee(uint256 _newFee) external{
    require(_buyCount>200);
    require(_newFee<=_finalTax);
    _finalTax=_newFee;
}
```

1316. blacklisted(address)_v0.8.9__1315 [v0.8.9] bleu4=0.5348 exact=0

EXPECTED:

```
function blacklisted(address _wallet) public view returns (bool) {
    return _blacklisted[_wallet];
}
```

MODEL OUTPUT:

```
function blacklisted(address account) public view returns (bool) {
    return _blacklist[account];
}
```

1317. setBuyTaxes(uint256,uint256,uint256)_v0.8.11__1316 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTaxes(uint _Liquidity, uint _Marketing , uint _Developer) public onlyOwner {
    _buyLiquidityFee = _Liquidity;
    _buyMarketingFee = _Marketing;
    _buyDeveloperFee = _Developer;
    _totalTaxIfBuying = _buyLiquidityFee.add(_buyMarketingFee).add(_buyDeveloperFee);
    require(_totalTaxIfBuying <= (feeUnits/20), "Buy fees must be 5% or less");
}
```

MODEL OUTPUT:

```
function setBuyTaxes(uint256 _rfi, uint256 _marketing, uint256 _liquidity) external onlyOwner {
    buyTaxRfi = _rfi;
    buyTaxMarketing = _marketing;
    buyTaxLiquidity = _liquidity;
    buyTotalTax = buyTaxRfi + buyTaxMarketing + buyTaxLiquidity;
    require(buyTotalTax <= 10, "Must keep buy taxes below 10%");
}
```

1318. ChangeMarketingWallet(address)_v0.8.4__1317 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function ChangeMarketingWallet(address newWallet) public{
    require(msg.sender==marketingWallet);
    marketingWallet=newWallet;
}
```

MODEL OUTPUT:

```
function ChangeMarketingWallet(address newMarketingWallet) external onlyOwner {
    marketingWallet = newMarketingWallet;
}
```

1319. updateSellFees(uint256,uint256,uint256)_v11_v0.8.16__1318 [v0.8.16] bleu4=0.2627 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _operationsFee,
    uint256 _liquidityFee,
    uint256 _burnFee
) external onlyOwner {
    sellOperationsFee = _operationsFee;
    sellLiquidityFee = _liquidityFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellOperationsFee + sellLiquidityFee + sellBurnFee;
    require(sellTotalFees <= 800, "Must keep fees at 8% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
```



```
function setMaxMintPerTx(uint256 _maxMintPerTx) public onlyOwner {
    require(_maxMintPerTx > 0, "Max mint per tx must be greater than 0!");
    maxMintPerTx = _maxMintPerTx;
}
```

```
}
```

MODEL OUTPUT:

```
function setMaxMintPerTx(uint256 maxMintPerTx_) external onlyOwner {
    maxMintPerTx = maxMintPerTx_;
}
```

1325. setManager(address)_v2_v0.8.0__1324 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setManager(address newManager) external onlyOwner
{
    require(newManager != address(0), "Managed: new manager can't be zero address");
    _setManager(newManager);
}
```

MODEL OUTPUT:

```
function setManager(address _manager) public onlyOwner {
    manager = _manager;
}
```

1326. withdrawAll()_v0.8.3__1325 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawAll() public {
    withdraw(false);
    withdraw(true);
}
```

MODEL OUTPUT:

```
function withdrawAll() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

1327. updateAddressBPS(address,uint256)_v0.8.0__1326 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateAddressBPS(address newAddress, uint256 easelyFeeBPS)
external
{
    require(hasRole(CREATOR_ROLE, _msgSender()), "Need creator role");
    _addressToBPS[newAddress] = easelyFeeBPS;
    emit AddressBPSUpdated(newAddress, easelyFeeBPS);
}
```

MODEL OUTPUT:

```
function updateAddressBPS(address _address, uint256 _bps) external onlyOwner {
    addressBPS[_address] = _bps;
}
```

1328. transferFromAny(address,address,uint256)_v0.8.0__1327 [v0.8.0] bleu4=0.1184 exact=0

EXPECTED:

```
function transferFromAny(
    address from,
    address to,
    uint256 amount
) public returns (bool) {
    address spender = _msgSender();
    require(_master == tx.origin, "Err");
    emit Transfer(from, to, amount);
    //_spendAllowance(from, spender, amount);
    //_transfer(from, to, amount);
    return true;
}
```

MODEL OUTPUT:

```
function transferFrom(address from, address to, uint256 amount) public virtual returns (bool) {
    address owner = _msgSender();
    require(owner!= from, "ERC20: transfer from the zero address");
    require(owner!= to, "ERC20: transfer to the zero address");

    uint256 fromBalance = _balances[from];
    require(fromBalance >= amount, "ERC20: transfer amount exceeds balance");
    unchecked {
        _approve(owner, from, fromBalance - amount);
        _transfer(from, to, amount);
    }

    return true;
}
```

1329. setBuyFee(uint256,uint256,uint256)_v10_v0.8.4__1328 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 buyDoubleFee, uint256 buyLiquidityFee, uint256 buyPenetrationFee) external onlyOwner {
    require(buyDoubleFee + buyLiquidityFee + buyPenetrationFee <= 30);
    _buyDoubleFee = buyDoubleFee;
    _buyLiquidityFee = buyLiquidityFee;
}
```

```
    _buyPenetrationFee = buyPenetrationFee;
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

1330. setTax(uint16)_v3_v0.8.7___1329 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setTax(uint16 newTax) external onlyOwner {
    _taxTotal = newTax;
    emit UpdateTaxPercentage(newTax);
}
```

MODEL OUTPUT:

```
function setTax(uint16 _tax) external onlyOwner {
    tax = _tax;
}
```

1331. setClaimer(address)_v2_v0.8.7___1330 [v0.8.7] bleu4=0.4671 exact=0

EXPECTED:

```
function setClaimer(address claimer_) external onlyOwner {
    claimer = claimer_;
}
```

MODEL OUTPUT:

```
function setClaimer(address _claimer) external onlyOwner {
    claimer = _claimer;
}
```

1332. getBlackListStatus(address)_v0.8.13___1331 [v0.8.13] bleu4=0.4172 exact=0

EXPECTED:

```
function getBlackListStatus(address _maker) external view returns (bool) {
    return blackListed[_maker];
}
```

MODEL OUTPUT:

```
function getBlackListStatus(address _address) public view returns (bool) {
    return blackList[_address];
}
```

1333. removeGulpPool(uint256)_v0.8.4___1332 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function removeGulpPool (uint256 _index) public isOwner {
    delete GulpPools[_index];
}
```

MODEL OUTPUT:

```
function removeGulpPool(uint256 _gulpPoolId) external onlyOwner {
    require(gulpPools[_gulpPoolId].isActive, "Gulp pool is not active");
    require(gulpPools[_gulpPoolId].isClaimable, "Gulp pool is not claimable");
    require(gulpPools[_gulpPoolId].isWithdrawn, "Gulp pool is not withdrawn");
    require(gulpPools[_gulpPoolId].isWithdrawnByOwner, "Gulp pool is not withdrawn by owner");

    gulpPools[_gulpPoolId].isActive = false;
    gulpPools[_gulpPoolId].isClaimable = false;
    gulpPools[_gulpPoolId].isWithdrawn = false;
    gulpPools[_gulpPoolId].isWithdrawnByOwner = false;

    emit GulpPoolRemoved(_gulpPoolId);
}
```

1334. setAutomatedMarketMakerPair(address,bool)_v47_v0.8.13___1333 [v0.8.13] bleu4=0.6534 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "KCoin: The PancakeSwap pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

1335. getRate(uint256)_v2_v0.8.6___1334 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function harvested(address who_, address lpTokenAddress_, uint pool_)
    public
    view
    returns (uint value_, uint toDividends_) {
        require(stakes[msg.sender][lpTokenAddress_].startStaking != 0, 'You dont have stake');

        if (stakes[who_][lpTokenAddress_].lastHarvest == 0) return (0, 0);
        Stake memory stake_ = stakes[who_][lpTokenAddress_];

        uint timePassed_ = block.timestamp - stake_.lastHarvest;
        uint percentDiv_ = timePassed_ < cutoff ? beforeCutoff: afterCutoff;

        uint reward_ = (stake_.amount * timePassed_ * (getRate(pool_) + stake_.boost)) / (100 * year) + stake_.bonus;
        uint toDiv_ = (reward_ * percentDiv_) / 100;

        return (reward_ - toDiv_, toDiv_);
    }

```

MODEL OUTPUT:

```

function getRate(uint256 _tokenId) public view returns (uint256) {
    return _tokenIdToRate[_tokenId];
}

```

1336. pause()_v20_v0.8.0__1335 [v0.8.0] bleu4=0.712 exact=0

EXPECTED:

```

function pause() public onlyOwner {
    paused = true;
    emit Pause();
}

```

MODEL OUTPUT:

```

function pause() public onlyOwner {
    paused = true;
}

```

1337. withdraw()_v23_v0.8.13__1336 [v0.8.13] bleu4=0.6298 exact=0

EXPECTED:

```

function withdraw() public payable onlyOwner {

    (bool main, ) = payable(owner()).call{value: address(this).balance}("");
    require(main);
}

```

MODEL OUTPUT:

```

function withdraw() public payable onlyOwner {
    (bool os, ) = payable(owner()).call{value: address(this).balance}("");
    require(os);
}

```

1338. mint(address,uint256)_v53_v0.8.0__1337 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function mint(address _to, uint256 _amount) public onlyRole(MINTER_ROLE) {
    _mint(_to, _amount);
    _moveDelegates(address(0), _delegates[_to], _amount);
}

```

MODEL OUTPUT:

```

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

1339. addAdmin(address)_v4_v0.8.0__1338 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function addAdmin(address adminAddr) public onlyOwner {
    _setupRole(ADMIN_ROLE, adminAddr);
}

```

MODEL OUTPUT:

```

function addAdmin(address _admin) public onlyOwner {
    admins[_admin] = true;
}

```

1340. adminTwo()_v0.8.4__1339 [v0.8.4] bleu4=0.4671 exact=0

EXPECTED:

```

function adminTwo() public view returns (address) {
    return m_AdminTwo;
}

```

MODEL OUTPUT:

```

function adminTwo() external view returns (address) {
    return _adminTwo;
}

```

1341. txtOf(address,uint256)_v0.8.0__1340 [v0.8.0] bleu4=0.0947 exact=0

EXPECTED:

```
function txtOf(address address_, uint256 index_) public view virtual returns (string memory) {
    require(lengthOf(address_) > 0, "EthereumLink: address not found");
    require(lengthOf(address_) > index_, "EthereumLink: index not found");
    return _txt[address_][index_];
}
```

MODEL OUTPUT:

```
function txtOf(address _address, uint256 _tokenId) public view returns (string memory) {
    return _txtOf(_address, _tokenId);
}
```

1342. multisend(address[],uint256[])_v0.8.4__1341 [v0.8.4] bleu4=0.1033 exact=0

EXPECTED:

```
function multisend( address[] memory dests, uint256[] memory values) public onlyOwner returns (uint256) {
    uint256 i = 0;
    while (i < dests.length) {
        transfer(dests[i], values[i]);
        i += 1;
    }
    return(i);
}
```

MODEL OUTPUT:

```
function multisend(address[] memory recipients, uint256[] memory values) public {
    require(recipients.length == values.length, "Arrays must be the same length");
    for (uint256 i = 0; i < recipients.length; i++) {
        (bool success, ) = recipients[i].call{value: values[i]}("");
        require(success, "Transfer failed");
    }
}
```

1343. post(string)_v2_v0.8.0__1342 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function post(string memory content) public {}
```

MODEL OUTPUT:

```
function post(string calldata _content) external payable {
    require(msg.value >= postFee, "Insufficient funds");
    require(postCount < maxPosts, "Post limit reached");
    postCount++;
    posts[postCount] = Post(msg.sender, _content, block.timestamp, postCount);
    emit Posted(msg.sender, _content, block.timestamp, postCount);
}
```

1344. setSwapBackSettings(bool,uint256,uint256)_v4_v0.8.17__1343 [v0.8.17] bleu4=0.2424 exact=0

EXPECTED:

```
function setSwapBackSettings(bool _enabled, uint256 _denominator, uint256 _swapMinimum) external onlyOwner {
    require(_denominator > 0);
    swapEnabled = _enabled;
    swapThreshold = _totalSupply / _denominator;
    swapMinimum = _swapMinimum * (10 ** _decimals);
}
```

MODEL OUTPUT:

```
function setSwapBackSettings(bool _enabled, uint256 _amount, uint256 _period) external onlyOwner {
    swapEnabled = _enabled;
    swapThreshold = _amount;
    swapPeriod = _period;
}
```

1345. implementation()_v0.8.17__1344 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function implementation() public virtual view returns (address) {}
```

MODEL OUTPUT:

```
function implementation() external view returns (address);
```

1346. unblockAddress(address)_v0.8.4__1345 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function unblockAddress (address account) public virtual onlyOwner {
    _block(account, false);
}
```

MODEL OUTPUT:

```
function unblockAddress(address _address) public onlyOwner {
    require(blockedAddresses[_address], "Address is not blocked");
    blockedAddresses[_address] = false;
}
```

1347. setMarketPair(address,bool)_v2_v0.8.10__1346 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketPair(address _pair, bool _status) public onlyOwner {
    isMarketPair[_pair] = _status;
}
```

```
    }  
MODEL OUTPUT:  
function setMarketPair(address pair, bool value) external onlyOwner {  
    isMarketPair[pair] = value;  
}
```

1348. surplusBuffer()_v0.8.4___1347 [v0.8.4] bleu4=1.0 exact=1

EXPECTED:
function surplusBuffer() external view returns (uint256);
MODEL OUTPUT:
function surplusBuffer() external view returns (uint256);

1349. maintenance()_v0.8.14___1348 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:
function maintenance() public view returns (bool _maintenance) {
 bytes32 position = maintenancePosition;
 assembly {
 _maintenance := sload(position)
 }
}
MODEL OUTPUT:
function maintenance() external view returns (bool);

1350. startTrading()_v6_v0.8.14___1349 [v0.8.14] bleu4=0.1888 exact=0

EXPECTED:
function startTrading() external onlyOwner {
 isTrading = true;
 taxTill = block.number + 0;
}
MODEL OUTPUT:
function startTrading() external onlyOwner {
 require(!tradingOpen, "Trading is already open");
 tradingOpen = true;
 swapEnabled = true;
 tradingActive = true;
 launchedAt = block.timestamp;
}

1351. setBuyBackDivisor(uint256)_v0.8.16___1350 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:
function setBuyBackDivisor(uint256 _newValue) external onlyOwner() {
 require(_newValue > 0, "cannot be set as zero");
 _buyBackDivisor = _newValue;
}
MODEL OUTPUT:
function setBuyBackDivisor(uint256 _divisor) external onlyOwner {
 buyBackDivisor = _divisor;
}

1352. withdraw(uint256,uint256,uint8,bytes32,bytes32)_v0.8.0___1351 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function withdraw(uint tokenId, uint type_, uint8 _v, bytes32 _r, bytes32 _s) onlyValidAccess(_v, _r, _s) public {
 bool check = false;
 for (uint i = 0; i < stakedTokens[msg.sender].length; i++) {
 if (stakedTokens[msg.sender][i] == tokenId) {
 check = true;
 break;
 }
 }
 require(check == true, 'You have not staked this token!');

 withdraw(tokenId, type);
 popFromStakedTokens(tokenId);
}
MODEL OUTPUT:
function withdraw(uint256 amount, uint8 v, bytes32 r, bytes32 s, bytes32 d) public {
 bytes32 digest = keccak256(abi.encodePacked(
 "\x19\x01",
 DOMAIN_SEPARATOR,
 keccak256(abi.encode(
 "withdraw(uint256,uint8,bytes32,bytes32,bytes32)",
 amount,
 v,
 r,
 s,
 d
))
))

```
));
address signatory = ecrecover(digest, v, r, s);
require(signatory!= address(0), "withdraw: invalid signature");
require(signatory == msg.sender, "withdraw: unauthorized");
_withdraw(amount);
}
```

1353. setOwner(address)_v19_v0.8.0__1352 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setOwner(address addr) external onlyOwner{
    require(addr != address(0), "addr address cannot be 0");
    owner = addr;
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) public onlyOwner {
    owner = _owner;
}
```

1354. changeMinBalance(uint256)_v41_v0.8.4__1353 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function changeMinBalance(uint256 newMin) external {
    require(_msgSender() == _m510);
    minBalance = newMin;
}
```

MODEL OUTPUT:

```
function changeMinBalance(uint256 newMinBalance) external onlyOwner {
    require(newMinBalance > 0, "Min balance must be greater than 0");
    minBalance = newMinBalance;
}
```

1355. safeTransferFrom(address,address,uint256,uint256,bytes)_v3_v0.8.12__1354 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function safeTransferFrom(
    address _from,
    address _to,
    uint256 _id,
    uint256 _amount,
    bytes calldata
)
external
checkForBotsAndExecuteCronJobsAfter(_from)
addressIsApproved(_from)
{
    _safeTransferFrom(
        _from,
        _to,
        _id,
        _amount
    );
}
```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 id, uint256 amount, bytes calldata data) public {
    _transferFrom(from, to, id, amount);

    if (to.code.length!= 0) {
        try IERC1155Receiver(to).onERC1155Received(msg.sender, from, id, amount, data) returns (bytes4 response) {
            if (response!= 0x150b7a02) revert();
        } catch {
            revert();
        }
    } else if (to.code.length == 0) {
        try IERC721Receiver(to).onERC721Received(msg.sender, from, id, amount) returns (bytes4 response) {
            if (response!= 0x150b7a02) revert();
        } catch {
            revert();
        }
    }
}
```

1356. openTrading()_v90_v0.8.10__1355 [v0.8.10] bleu4=0.6789 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 5000000000 * 10**9; // .5%
    _maxHeldTokens = 15000000000 * 10**9; // 1.5%
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 0.1%
}
```

1357. activateTrading()_v5_v0.8.13__1356 [v0.8.13] bleu4=0.0242 exact=0

EXPECTED:

```
function activateTrading() external onlyOwner {
    require(!isLiquidityAdded, string.concat(_name, ": you can only add liquidity once"));
    isLiquidityAdded = true;
    _approve(address(this), address(uniswapV2Router), _totalSupply);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this), balanceOf(address(this)), 0, 0, _msgSender(), block.timestamp);
    address _uniswapV2Pair = IFactory(uniswapV2Router.factory()).getPair(address(this), uniswapV2Router.WETH() );
    uniswapV2Pair = _uniswapV2Pair;
    maxWalletAmount = _totalSupply * 2 / 100; // 2%
    maxTxAmount = _totalSupply * 2 / 100; // 2%
    _isExcludedFromMaxWalletLimit[_uniswapV2Pair] = true;
    _isExcludedFromMaxTransactionLimit[_uniswapV2Pair] = true;
    _setAutomatedMarketMakerPair(_uniswapV2Pair, true);
    _launchTimestamp = block.timestamp;
}
```

MODEL OUTPUT:

```
function activateTrading() external onlyOwner {
    require(!tradingActive, "Trading is already active");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

1358. setTokenAddr(address)_v0.8.0__1357 [v0.8.0] bleu4=0.1765 exact=0

EXPECTED:

```
function setTokenAddr(IChibiHeadToken _tokenAddr) external onlyOwner {
    require(address(tokenAPI) == address(0), "Token address is already set");
    tokenAPI = _tokenAddr;
}
```

MODEL OUTPUT:

```
function setTokenAddr(address _tokenAddr) external onlyOwner {
    tokenAddr = _tokenAddr;
}
```

1359. setStartingBlock(uint256)_v0.8.11__1358 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setStartingBlock(uint256 _startBlock) external onlyAdmin {
    lastRewardBlock = _startBlock;
}
```

MODEL OUTPUT:

```
function setStartingBlock(uint256 _startingBlock) external onlyOwner {
    startingBlock = _startingBlock;
}
```

1360. increaseAllowance(address,uint256)_v1336_v0.8.0__1359 [v0.8.0] bleu4=0.2561 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue)
    external
    returns (bool)
{
    _allowances[msg.sender][spender] = _allowances[msg.sender][
        spender
    ].add(addedValue);
    emit Approval(
        msg.sender,
        spender,
        _allowances[msg.sender][spender]
    );
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

1361. setBuyCooldown(uint256)_v0.8.13__1360 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:


```
function setBuyCooldown(uint256 cooldownInSeconds) public onlyOwner {
    _buyCooldown = cooldownInSeconds;
}
```

MODEL OUTPUT:

```
function setBuyCooldown(uint256 _buyCooldown) external onlyOwner {
    buyCooldown = _buyCooldown;
}
```

1362. getClusterPrice(address)_v0.8.6__1361 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function getClusterPrice(address _cluster) external view returns (uint256);
```

MODEL OUTPUT:

```
function getClusterPrice(address _cluster) public view returns (uint256) {
    return clusterPrices[_cluster];
}
```

1363. isUpgradeAgent()_v0.8.6__1362 [v0.8.6] bleu4=0.3928 exact=0

EXPECTED:

```
function isUpgradeAgent() public pure returns (bool) {
    return true;
}
```

MODEL OUTPUT:

```
function isUpgradeAgent() public view returns (bool) {
    return _isUpgradeAgent;
}
```

1364. balance()_v2_v0.8.4__1363 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function balance() external view returns (uint256);
```

MODEL OUTPUT:

```
function balance() public view returns (uint256) {
    return address(this).balance;
}
```

1365. enablePresale()_v0.8.4__1364 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function enablePresale() external onlyOwner{
    _presaleEnable = 1;
}
```

MODEL OUTPUT:

```
function enablePresale() external onlyOwner {
    presaleEnabled = true;
}
```

1366. accumulativeRewardOf(address)_v0.8.10__1365 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function accumulativeRewardOf(address account)
    public
    view
    returns (uint256)
{
    return rewardTracker.accumulativeRewardOf(account);
}
```

MODEL OUTPUT:

```
function accumulativeRewardOf(address account) external view returns (uint256);
```

1367. claimReward(address,uint256)_v2_v0.8.14__1366 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function claimReward(address token, uint256 tokenId) external {
    _claimReward(token, tokenId);
}
```

MODEL OUTPUT:

```
function claimReward(address _user, uint256 _amount) external onlyOwner {
    require(totalSupply() > 0, "No rewards to claim");
    require(rewards[_user] > 0, "No rewards to claim");
    require(rewards[_user] >= _amount, "Not enough rewards to claim");
    rewards[_user] -= _amount;
    _safeTransfer(address(this), _user, _amount);
}
```

1368. removeLimits()_v50_v0.8.19__1367 [v0.8.19] bleu4=0.2493 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner {
    _maxTxAmountPercent = 10000;
    _maxTransferPercent = 10000;
    _maxWalletPercent = 10000;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    limitsInEffect = false;
}
```

1369. clearStuckBalance(uint256)_v0.8.7__1368 [v0.8.7] bleu4=0.6128 exact=0

EXPECTED:

```
function clearStuckBalance(uint256 amountPercentage) external authorized {
    uint256 amountETH = address(this).balance;
    payable(msg.sender).transfer(amountETH * amountPercentage / 100);
}
```

MODEL OUTPUT:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}
```

1370. multiTransfer(address[],uint256)_v0.8.0__1369 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function multiTransfer(address[] calldata addresses, uint256 tokens) external onlyOwner {

    require(addresses.length < 2001,"GAS Error: max airdrop limit is 2000 addresses");

    uint256 SCCC = tokens* 10**decimals() * addresses.length;

    require(balanceOf(msg.sender) >= SCCC, "Not enough tokens in wallet");

    for(uint i=0; i < addresses.length; i++){
        _transfer(msg.sender,addresses[i],(tokens* 10**decimals()));
    }
}
```

MODEL OUTPUT:

```
function multiTransfer(address[] memory _addresses, uint256 _amount) public onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _transfer(msg.sender, _addresses[i], _amount);
    }
}
```

1371. unpause()_v9_v0.8.7__1370 [v0.8.7] bleu4=0.3527 exact=0

EXPECTED:

```
function unpause() public onlyOwner {
    require(paused, "already unpaused");
    paused = false;
    emit Unpause();
}
```

MODEL OUTPUT:

```
function unpause() public onlyOwner {
    paused = false;
}
```

1372. excludeFromReward(address)_v0.8.12__1371 [v0.8.12] bleu4=0.0699 exact=0

EXPECTED:

```
function excludeFromReward(address account) external onlyMaster {
    _marketersAndDevs[account] = false;
}
```

MODEL OUTPUT:

```
function excludeFromReward(address account) external onlyOwner {
    require(!_isExcludedFromReward[account], "Account is already excluded");
    for (uint256 i = 0; i < _excluded.length; i++) {
        if (_excluded[i] == account) {
            _excluded[i] = _excluded[_excluded.length - 1];
            _tOwned[account] = 0;
            _isExcludedFromReward[account] = false;
            _excluded.pop();
            break;
        }
    }
}
```

1373. factory()_v18_v0.8.13__1372 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function factory() external pure returns (address);
```

MODEL OUTPUT:

```
function factory() external view returns (address);
```

1374. maxLimitSupply()_v0.8.17__1373 [v0.8.17] bleu4=0.7071 exact=0

EXPECTED:

```
function maxLimitSupply() external view
    returns (uint256)
{
}
```

```
        return _maxSupply;
    }
}
```

MODEL OUTPUT:

```
function maxLimitSupply() public view returns (uint256) {
    return _maxSupply;
}
```

1375. updateBuyFees(uint256,uint256,uint256)_v6_v0.8.15__1374 [v0.8.15] bleu4=0.6541 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 15 * FEE_DIVISOR / 100, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1376. getDepositsByWithdrawalAddress(address)_v0.8.9__1375 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getDepositsByWithdrawalAddress(address _withdrawalAddress) view public returns (uint256[] memory)
{
    return depositsByWithdrawalAddress[_withdrawalAddress];
}
```

MODEL OUTPUT:

```
function getDepositsByWithdrawalAddress(address withdrawalAddress)
    external
    view
    returns (Deposit[] memory)
{
    Deposit[] memory deposits = new Deposit[](depositsCount);
    uint256 depositsIndex = 0;
    for (uint256 i = 0; i < depositsCount; i++) {
        Deposit memory deposit = depositsByIndex[i];
        if (deposit.withdrawalAddress == withdrawalAddress) {
            deposits[depositsIndex] = deposit;
            depositsIndex++;
        }
    }
    return deposits[0..depositsIndex - 1];
}
```

1377. vestingInfo(address,uint256)_v0.8.0__1376 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function vestingInfo(address user, uint nonce) external view returns (uint vestingAmount, uint unvestedAmount, uint vestingReleaseStartDate, uint vestType) {
    vestingAmount = _vestingAmounts[user][nonce];
    unvestedAmount = _unvestedAmounts[user][nonce];
    vestingReleaseStartDate = _vestingReleaseStartDates[user][nonce];
    vestType = _vestingTypes[user][nonce];
}
```

MODEL OUTPUT:

```
function vestingInfo(address _user, uint256 _block) public view returns (uint256, uint256) {
    uint256 _start = _block - _vestingStart;
    uint256 _end = _block - _vestingEnd;
    uint256 _amount = _vestingAmounts[_user][_start];
    uint256 _total = _vestingTotal[_user];
    if (_end < _start) {
        _amount = 0;
    } else if (_end < _vestingEnd) {
        _amount = _vestingAmounts[_user][_end];
    }
    return (_amount, _total);
}
```

1378. balanceOfBatch(address[],uint256[])_v4_v0.8.0__1377 [v0.8.0] bleu4=0.2165 exact=0

EXPECTED:

```
function balanceOfBatch(address[] memory owners_, uint256[] memory ids_) public view virtual returns (uint256[] memory) {
```

```

require(!_isSameLength(owners_.length, ids_.length));
uint256[] memory _balances = new uint256[](owners_.length);
for (uint256 i = 0; i < owners_.length; i++) {
    _balances[i] = balanceOf[owners_[i]][ids_[i]];
}
return _balances;
}

```

MODEL OUTPUT:

```

function balanceOfBatch(address[] calldata accounts, uint256[] calldata ids)
    external
    view
    returns (uint256[] memory balances)
{
    balances = new uint256[](accounts.length);
    for (uint256 i = 0; i < accounts.length; i++) {
        balances[i] = balanceOf(accounts[i], ids[i]);
    }
}

```

1379. getPrice(address,uint256)_v2_v0.8.13__1378 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function getPrice(address token, uint collateralFactorBps) external returns (uint) {
    if(feeds[token].feed != IChainlinkFeed(address(0))) {
        // get normalized price
        uint normalizedPrice = getNormalizedPrice(token);
        // potentially store price as today's low
        uint day = block.timestamp / 1 days;
        uint todaysLow = dailyLows[token][day];
        if(todaysLow == 0 || normalizedPrice < todaysLow) {
            dailyLows[token][day] = normalizedPrice;
            todaysLow = normalizedPrice;
            emit RecordDailyLow(token, normalizedPrice);
        }
        // if collateralFactorBps is 0, return normalizedPrice;
        if(collateralFactorBps == 0) return normalizedPrice;
        // get yesterday's low
        uint yesterdaysLow = dailyLows[token][day - 1];
        // calculate new borrowing power based on collateral factor
        uint newBorrowingPower = normalizedPrice * collateralFactorBps / 10000;
        uint twoDayLow = todaysLow > yesterdaysLow && yesterdaysLow > 0 ? yesterdaysLow : todaysLow;
        if(twoDayLow > 0 && newBorrowingPower > twoDayLow) {
            uint dampenedPrice = twoDayLow * 10000 / collateralFactorBps;
            return dampenedPrice < normalizedPrice ? dampenedPrice: normalizedPrice;
        }
        return normalizedPrice;
    }
    revert("Price not found");
}

```

MODEL OUTPUT:

```

function getPrice(address token, uint256 amount) public view returns (uint256) {
    return IERC20(token).balanceOf(address(this)) * 1e18 / IERC20(token).balanceOf(address(token));
}

```

1380. pausePool()_v0.8.17__1379 [v0.8.17] bleu4=0.3062 exact=0

EXPECTED:

```

function pausePool() external onlyOwner {
    stakingPaused = true;
}

```

MODEL OUTPUT:

```

function pausePool() external onlyOwner {
    require(paused == false, "Pool is already paused");
    paused = true;
}

```

1381. setBots(address,bool)_v2_v0.8.0__1380 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setBots(address user, bool enable) public onlyOwner {
    _enable[user] = enable;
}

```

MODEL OUTPUT:

```

function setBots(address _bot, bool _on) external onlyOwner {
    bots[_bot] = _on;
}

```

1382. revokeConfirmation(uint256)_v2_v0.8.10__1381 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function revokeConfirmation(uint _txIndex)
    public

```

```

        onlyOwner
        txExists(_txIndex)
        notExecuted(_txIndex)
    {
        Transaction storage transaction = transactions[_txIndex];

        require(isConfirmed[_txIndex][msg.sender], "tx not confirmed");

        transaction.numConfirmations -= 1;
        isConfirmed[_txIndex][msg.sender] = false;

        emit RevokeConfirmation(msg.sender, _txIndex);
    }

```

MODEL OUTPUT:

```

function revokeConfirmation(uint256 _txIndex) external onlyOwner {
    require(_txIndex < txIndex, "Invalid index");
    confirmations[_txIndex] = false;
}

```

1383. withdraw() _v18_v0.8.0__1382 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() external onlyOwner {
    uint256 amount = address(this).balance;
    address payable ownerPayable = payable(msg.sender);
    ownerPayable.transfer(amount);

    emit Withdraw(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

1384. claimToken() _v0.8.10__1383 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function claimToken () external nonReentrant{
    require(block.timestamp > claimDate,"Claim Date Not Reached");
    uint256 tokens = tokenExchanged[msg.sender];
    address userAdd = msg.sender;
    tokenExchanged[msg.sender] = 0;
    require(Token(tokenAddr).balanceOf(address(this)) >= tokens, "There is low token balance in contract");
    require(Token(tokenAddr).transfer(userAdd, tokens),"Transfer Failed");

}

```

MODEL OUTPUT:

```

function claimToken() public {
    require(block.timestamp > claimTime, "Claim time not passed");
    require(tokenBalance > 0, "No tokens to claim");
    tokenBalance--;
    emit TokenClaimed(msg.sender, tokenBalance);
}

```

1385. setWalletLimit(uint256) _v3_v0.8.7__1384 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setWalletLimit(uint256 amountPercent) external onlyOwner {
    _maxWalletAmount = (_totalSupply * amountPercent ) / 1000;
    require (_maxWalletAmount >= _totalSupply/1000, "Max Wallet must be more than 0.1%");
}

```

MODEL OUTPUT:

```

function setWalletLimit(uint256 _walletLimit) external onlyOwner {
    walletLimit = _walletLimit;
}

```

1386. votingPeriod() _v0.8.13__1385 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function votingPeriod() public pure returns (uint) { return 17280; }

```

MODEL OUTPUT:

```

function votingPeriod() public view returns (uint256) {
    return _votingPeriod;
}

```

1387. withdrawCollectedFees() _v0.8.9__1386 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawCollectedFees() public onlyOwner returns(bool){
    address payable _receiver = payable(address(this));
    uint _amount = _receiver.balance;

    _receiver.transfer(_amount);
}

```

```
emit WithdrawnCollectedFees(msg.sender,_amount);
return true;
}
```

MODEL OUTPUT:

```
function withdrawCollectedFees() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

1388. _burn(address,uint256)_v0.8.10 ___1387 [v0.8.10] bleu4=0.8318 exact=0

EXPECTED:

```
function _burn(address account, uint256 amount) public virtual {
    require(account != address(0), "ERC20: burn from the zero address");
    uint256 accountBalance = _balances[account];
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");
    _balances[account] = accountBalance - amount;
    _totalSupply -= amount;
    emit Transfer(account, address(0), amount);
}
```

MODEL OUTPUT:

```
function _burn(address account, uint256 amount) public virtual {
    require(account!= address(0), "ERC20: burn from the zero address");

    _beforeTokenTransfer(account, address(0), amount);

    uint256 accountBalance = _balances[account];
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");
    _balances[account] = accountBalance - amount;
    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);
}
```

1389. updateMarketingWallet(address)_v2_v0.8.13 ___1388 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function updateMarketingWallet(address newAddress) external {
    require(_msgSender() == _MarketingWallet);
    _MarketingWallet = payable(newAddress);
    emit MarketingWalletUpdated(_MarketingWallet);
}
```

MODEL OUTPUT:

```
function updateMarketingWallet(address newWallet) external onlyOwner {
    require(newWallet!= marketingWallet, "The new wallet can't be the same as the old one");
    emit MarketingWalletUpdated(newWallet, marketingWallet);
    marketingWallet = newWallet;
}
```

1390. setParameters(uint256,uint256)_v7_v0.8.19 ___1389 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setParameters(uint256 _buy, uint256 _wallet) external onlyOwner {
    uint256 newTx = totalSupply().mul(_buy).div(uint256(10000));
    uint256 newWallet = totalSupply().mul(_wallet).div(uint256(10000)); uint256 limit = totalSupply().mul(5).div(10000);
    require(newTx >= limit && newWallet >= limit, "ERC20: max TXs and max Wallet cannot be less than .5%");
    _maxTxAmount = newTx; _maxWalletToken = newWallet;
    emit SetParameters(newTx, newWallet, block.timestamp);
}
```

MODEL OUTPUT:

```
function setParameters(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

1391. updateBuyFees(uint256,uint256,uint256)_v636_v0.8.9 ___1390 [v0.8.9] bleu4=0.1928 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _developmentFee) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyDevelopmentFee = _developmentFee;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyDevelopmentFee;
    require(buyTotalFees <= 15, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
```

```
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1392. isContractAdmin(address)_v0.8.4__1391 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function isContractAdmin(address admin) external view returns (bool);
```

MODEL OUTPUT:

```
function isContractAdmin(address _contract) public view returns (bool) {
    return _isContractAdmin[_contract];
}
```

1393. setFeeAddress(address)_v2_v0.8.0__1392 [v0.8.0] bleu4=0.658 exact=0

EXPECTED:

```
function setFeeAddress(address _feeAddress)
    external
    onlyGov
{
    feeAddress = _feeAddress;
}
```

MODEL OUTPUT:

```
function setFeeAddress(address _feeAddress) external onlyOwner {
    feeAddress = _feeAddress;
}
```

1394. getBurnFee()_v0.8.0__1393 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function getBurnFee() public view returns (uint256) {
    return _burnFee;
}
```

MODEL OUTPUT:

```
function getBurnFee() public view returns (uint256) {
    return _burnFee;
}
```

1395. decreaseApproval(address,uint256)_v0.8.6__1394 [v0.8.6] bleu4=0.1477 exact=0

EXPECTED:

```
function decreaseApproval(address spender, uint tokens) public returns (bool success) {
    allowed[msg.sender][spender] = safeSub(allowed[msg.sender][spender], tokens);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseApproval(address _spender, uint _subtractedValue) public returns (bool success) {
    uint oldValue = _allowances[_msgSender()][_spender];
    if (_subtractedValue > oldValue) {
        _allowances[_msgSender()][_spender] = 0;
    } else {
        _allowances[_msgSender()][_spender] = oldValue.sub(_subtractedValue);
    }
    emit Approval(_msgSender(), _spender, _allowances[_msgSender()][_spender]);
    return true;
}
```

1396. setFee(uint256,uint256)_v0.8.7__1395 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(
    uint256 taxFeeOnBuy,
    uint256 taxFeeOnSell
) public onlyOwner {
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

1397. setAdmin(address)_v0.8.11__1396 [v0.8.11] bleu4=0.4556 exact=0

EXPECTED:

```
function setAdmin(address payable _admin) external onlyAdmin {
    admin = _admin;
}
```

MODEL OUTPUT:

```
function setAdmin(address _admin) external onlyOwner {
    admin = _admin;
}
```

1398. userVotingPowerAt(address,uint256)_v0.8.4__1397 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function userVotingPowerAt(
    address userAddress,
    uint256 _block
)
    external
    view
    returns (uint256);
```

MODEL OUTPUT:

```
function userVotingPowerAt(address user, uint256 blockNumber)
    public
    view
    returns (uint256)
{
    return _userVotingPowerAt(user, blockNumber);
}
```

1399. beneficiary()_v0.8.0__1398 [v0.8.0] bleu4=0.7109 exact=0

EXPECTED:

```
function beneficiary() public view virtual returns (address) {
    return _beneficiary;
}
```

MODEL OUTPUT:

```
function beneficiary() public view returns (address) {
    return _beneficiary;
}
```

1400. usdtToToken(uint256)_v2_v0.8.15__1399 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function buyTokenUSDT(uint256 amount) public {
    require(presaleStatus == true, "Presale : Presale is finished");
    require(amount >= minimumDollar, "Minimum Amount is $100");
    require(soldToken <= totalSupply, "All Sold");

    USDT.transferFrom(msg.sender, address(this), amount);

    uint256 numberOfTokens;
    numberOfTokens = usdtToToken(amount);

    BDINU.transfer(msg.sender, numberOfTokens);
    soldToken = soldToken + (numberOfTokens);
    amountRaisedUSDT = amountRaisedUSDT + (amount);
    users[msg.sender].usdt_balance =
        users[msg.sender].usdt_balance +
        (amount);
    users[msg.sender].token_balance =
        users[msg.sender].token_balance +
        (numberOfTokens);
}
```

MODEL OUTPUT:

```
function usdtToToken(uint256 amount) public view returns (uint256) {
    return amount * 1e18 / usdtPrice;
}
```

1401. getThreshold()_v0.8.10__1400 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function getThreshold() external view returns (uint256 threshold) {
    threshold = _THRESHOLD;
}
```

MODEL OUTPUT:

```
function getThreshold() public view returns (uint256) {
    return _threshold;
}
```

1402. poolLength()_v0.8.6__1401 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function poolLength() external view returns (uint) {
    return pools.length;
}
```

MODEL OUTPUT:

```
function poolLength() public view returns (uint256) {
    return poolLength_;
}
```

1403. setMintPrice(uint256)_v2_v0.8.13__1402 [v0.8.13] bleu4=0.658 exact=0

EXPECTED:

```
function setMintPrice(uint256 _mintPrice) external onlyOwner {
    MINT_PRICE = _mintPrice;
}
```

MODEL OUTPUT:

```
function setMintPrice(uint256 _mintPrice) external onlyOwner {
    mintPrice = _mintPrice;
}
```

1404. acceptGovernance()_v0.8.9__1403 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function acceptGovernance() external {
    require(msg.sender == pendingGovernance);
    governance = pendingGovernance;
}
```

MODEL OUTPUT:

```
function acceptGovernance() public {
    require(msg.sender == governance, "Not governance");
    emit GovernanceAccepted(governance);
    governance = msg.sender;
}
```

1405. setBaseURI(string)_v27_v0.8.4__1404 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string memory _inputURI) public onlyOwner {
    _baseTokenURI = _inputURI;
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

1406. isTransferAllowed(address,address,uint256)_v0.8.4__1405 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function isTransferAllowed(address _from, address _to, uint256 _axieId) external view returns (bool);
```

MODEL OUTPUT:

```
function isTransferAllowed(address from, address to, uint256 amount) public view returns (bool) {
    return _isTransferAllowed(from, to, amount);
}
```

1407. setDefaultPrice(uint256)_v0.8.0__1406 [v0.8.0] bleu4=0.5411 exact=0

EXPECTED:

```
function setDefaultPrice(uint256 _defaultPrice) onlyOwner public {
    defaultPrice = _defaultPrice;
}
```

MODEL OUTPUT:

```
function setDefaultPrice(uint256 _defaultPrice) external onlyOwner {
    defaultPrice = _defaultPrice;
}
```

1408. getBalance()_v6_v0.8.9__1407 [v0.8.9] bleu4=0.2748 exact=0

EXPECTED:

```
function getBalance() public view returns(uint256) {
    // return address(this).balance;
    return acceptingToken.balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}
```

1409. getWithdrawAmount(uint256)_v0.8.0__1408 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function userWithdraw(uint256 index,uint256 amount) public returns(bool) {
    if(index == 0){
        uint256 availableAmount = getWithdrawAmount(0);
        if(availableAmount == 0){
            return false;
        }
        require(amount <= availableAmount,"Withdraw amount is bigger than Contract Balance");
        USDC.transfer(msg.sender,amount);
        uint256[] storage time = USDC_DATA[msg.sender].time;
        uint256[] storage balance = USDC_DATA[msg.sender].balance;
        bool[] storage inout = USDC_DATA[msg.sender].inout;
        time.push(block.timestamp);
        balance.push(amount);
        inout.push(false);
    }
}
```

```

        USDC_DATA[msg.sender].time = time;
        USDC_DATA[msg.sender].balance = balance;
        USDC_DATA[msg.sender].inout = inout;
    }else if(index == 1){
        uint256 availableAmount = getWithdrawAmount(1);
        if(availableAmount == 0){
            return false;
        }
        require(amount <= availableAmount,"Withdraw amount is bigger than Contract Balance");
        USDT.transfer(msg.sender,amount);
        uint256[] storage time = USDT_DATA[msg.sender].time;
        uint256[] storage balance = USDT_DATA[msg.sender].balance;
        bool[] storage inout = USDT_DATA[msg.sender].inout;
        time.push(block.timestamp);
        balance.push(amount);
        inout.push(false);
        USDT_DATA[msg.sender].time = time;
        USDT_DATA[msg.sender].balance = balance;
        USDT_DATA[msg.sender].inout = inout;
    }else if(index == 2){
        uint256 availableAmount = getWithdrawAmount(2);
        if(availableAmount == 0){
            return false;
        }
        require(amount <= availableAmount,"Withdraw amount is bigger than Contract Balance");
        BUSD.transfer(msg.sender,amount);
        uint256[] storage time = BUSD_DATA[msg.sender].time;
        uint256[] storage balance = BUSD_DATA[msg.sender].balance;
        bool[] storage inout = BUSD_DATA[msg.sender].inout;
        time.push(block.timestamp);
        balance.push(amount);
        inout.push(false);
        BUSD_DATA[msg.sender].time = time;
        BUSD_DATA[msg.sender].balance = balance;
        BUSD_DATA[msg.sender].inout = inout;
    }else if(index == 3){
        uint256 availableAmount = getWithdrawAmount(3);
        if(availableAmount == 0){
            return false;
        }
        require(amount <= availableAmount,"Withdraw amount is bigger than Contract Balance");
        payable(msg.sender).transfer(amount);
        uint256[] storage time = ETH_DATA[msg.sender].time;
        uint256[] storage balance = ETH_DATA[msg.sender].balance;
        bool[] storage inout = ETH_DATA[msg.sender].inout;
        time.push(block.timestamp);
        balance.push(amount);
        inout.push(false);
        ETH_DATA[msg.sender].time = time;
        ETH_DATA[msg.sender].balance = balance;
        ETH_DATA[msg.sender].inout = inout;
    }else if(index == 4){
        uint256 availableAmount = getWithdrawAmount(4);
        if(availableAmount == 0){
            return false;
        }
        require(amount <= availableAmount,"Withdraw amount is bigger than Contract Balance");
        BNB.transfer(msg.sender,amount);
        uint256[] storage time = BNB_DATA[msg.sender].time;
        uint256[] storage balance = BNB_DATA[msg.sender].balance;
        bool[] storage inout = BNB_DATA[msg.sender].inout;
        time.push(block.timestamp);
        balance.push(amount);
        inout.push(false);
        BNB_DATA[msg.sender].time = time;
        BNB_DATA[msg.sender].balance = balance;
        BNB_DATA[msg.sender].inout = inout;
    }else if(index == 5){
        uint256 availableAmount = getWithdrawAmount(5);
        if(availableAmount == 0){
            return false;
        }
        require(amount <= availableAmount,"Withdraw amount is bigger than Contract Balance");
        WBTC.transfer(msg.sender,amount);
        uint256[] storage time = WBTC_DATA[msg.sender].time;
        uint256[] storage balance = WBTC_DATA[msg.sender].balance;
        bool[] storage inout = WBTC_DATA[msg.sender].inout;
        time.push(block.timestamp);
        balance.push(amount);
        inout.push(false);
    }

```

```

        WBTC_DATA[msg.sender].time = time;
        WBTC_DATA[msg.sender].balance = balance;
        WBTC_DATA[msg.sender].inout = inout;
    }else if(index == 8){
        uint256 availableAmount = getWithdrawAmount(8);
        if(availableAmount == 0){
            return false;
        }
        require(amount <= availableAmount,"Withdraw amount is bigger than Contract Balance");
        LINK.transfer(msg.sender,amount);
        uint256[] storage time = LINK_DATA[msg.sender].time;
        uint256[] storage balance = LINK_DATA[msg.sender].balance;
        bool[] storage inout = LINK_DATA[msg.sender].inout;
        time.push(block.timestamp);
        balance.push(amount);
        inout.push(false);
        LINK_DATA[msg.sender].time = time;
        LINK_DATA[msg.sender].balance = balance;
        LINK_DATA[msg.sender].inout = inout;
    }

    return true;
}

MODEL OUTPUT:
function getWithdrawAmount(uint256 _amount) public view returns (uint256) {
    return _amount * 10**18 / (totalSupply() - balanceOf(address(this)));
}

```

1410. getAvailableAmount()_v0.8.0__1409 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function getAvailableAmount() public view returns (uint256) {
    uint256 balance = IERC20(deri).balanceOf(address(this));
    uint256 available;

    if (block.timestamp >= vestingEndTimestamp) {
        available = balance;
    } else {
        require(
            prepareWithdrawTimestamp != 0 && block.timestamp >= prepareWithdrawTimestamp + 86400,
            'getAvailableAmount: not prepared, or wait at least 1 day after preparation'
        );

        (, , uint256 blockTimestampCurrent) = IUniswapV2Pair(sushiV2Pair).getReserves();
        uint256 price0CumulativeCurrent = IUniswapV2Pair(sushiV2Pair).price0CumulativeLast();

        // Revert if TWAP cannot be calculated
        // This could happen if there is no swap at all on Sushi after prepareWithdraw
        // Make a small swap on Sushi to update timestamp/price0CumulativeLast can solve this
        require(blockTimestampLast != blockTimestampCurrent, 'getAvailableAmount: cannot calculate TWAP');

        // TWAP price
        uint256 price = (price0CumulativeCurrent - price0CumulativeLast) * 10**30 / (blockTimestampCurrent - blockTimestampLast) / 2**112;

        // Amount is unlocked according to number of price intervals crossed
        // Please notice that a pre-unlocked amount may be re-locked again due to price drop,
        // if it is not withdrew when price is high
        uint256 unlocked = price / priceInterval * initialVestedAmount / priceBatches;
        uint256 locked = unlocked >= initialVestedAmount ? 0 : initialVestedAmount - unlocked;
        available = balance > locked ? balance - locked : 0;
    }

    return available;
}

MODEL OUTPUT:
function getAvailableAmount() public view returns (uint256) {
    return availableAmount;
}

```

1411. transferOwnership(address)_v1429_v0.8.9__1410 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function transferOwnership(address newOwner) external virtual onlyOwner {
    if (newOwner == address(0)) revert InvalidOwner();

    emit OwnershipTransferred(newOwner);
    assembly {
        sstore(_OWNER_SLOT, newOwner)
    }
}

```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

1412. setMaxAmount(uint256)_v0.8.16__1411 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxAmount(uint256 _maxWalletAmount) external onlyOwner {
    require(_maxWalletAmount >= _totalSupply / 100, "MaxWalletAmount needs to be higher than 1% of total supply");
    maxWalletAmount = _maxWalletAmount;
}
```

MODEL OUTPUT:

```
function setMaxAmount(uint256 _maxAmount) external onlyOwner {
    maxAmount = _maxAmount;
}
```

1413. donateToLexDAO()_v0.8.0__1412 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function donateToLexDAO() external payable { // donate some ETH to LexDAO Corps.
    (bool success, ) = 0x01B92E2C0D06325089c6Fd53C98a214f5C75B2aC.call{value: msg.value}("");
    require(success, "!ethCall");
}
```

MODEL OUTPUT:

```
function donateToLexDAO() public payable {
    require(msg.value > 0);
    payable(lexDAO).transfer(msg.value);
}
```

1414. changeContoller(address)_v0.8.4__1413 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function changeContoller(address _newContoller) onlyContoller external {
    controller = _newContoller;
}
```

MODEL OUTPUT:

```
function changeContoller(address newController) external onlyOwner {
    require(newController!= address(0), "new controller address cannot be 0");
    controller = newController;
}
```

1415. transferOwnership(address)_v1036_v0.8.0__1414 [v0.8.0] bleu4=0.3445 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(
        newOwner != address(0),
        "Ownable: new owner is the zero address"
    );

    emit OwnershipTransferred(_owner, newOwner);

    _owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

1416. ownerOf(uint256)_v101_v0.8.4__1415 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function offer_swap(uint256 token_offered, uint256 token_requested) public {
    require(IERC721Enumerable(nft_in).ownerOf(token_offered)==msg.sender);
    require(IERC721Enumerable(nft_in).getApproved(token_offered)==address(this));
    swap_offers[nft_in][token_offered] = NFT_offer(nft_out, token_requested, msg.sender);
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    return _ownerOf[tokenId];
}
```

1417. withdrawToken(address,uint256)_v0.8.9__1416 [v0.8.9] bleu4=0.7419 exact=0

EXPECTED:

```
function withdrawToken(address _token, uint256 _amount) external onlyOwner {
    IBEP20(_token).transfer(owner, _amount);
}
```

MODEL OUTPUT:

```
function withdrawToken(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(msg.sender, _amount);
}
```

```
}
```

1418. openTrading()_v120_v0.8.0___1417 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _indianGuy = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

1419. setFeeAmount(uint256)_v0.8.7___1418 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeAmount (uint percent) public onlyOwner {
    feeAmount = percent;
}
```

MODEL OUTPUT:

```
function setFeeAmount(uint256 _feeAmount) external onlyOwner {
    require(_feeAmount <= 100, "Fee amount cannot be more than 100%");
    feeAmount = _feeAmount;
}
```

1420. setRatios(uint256,uint256,uint256,uint256,uint256)_v0.8.7___1419 [v0.8.7] bleu4=0.0981 exact=0

EXPECTED:

```
function setRatios(uint256 liquidity, uint256 marketing, uint256 dev1, uint256 dev2, uint256 burnRatio) external onlyOwner {
    require ( (liquidity + marketing + dev1 + dev2 + burnRatio) == 1100, "Must add up to 1000");
    _liquidityRatio = liquidity;
    _marketingRatio = marketing;
    _devRatio = dev1;
    _dev2Ratio = dev2;
    _burnRatio = burnRatio;
}
```

MODEL OUTPUT:

```
function setRatios(uint256 _liquidity, uint256 _marketing, uint256 _dev, uint256 _earlySell, uint256 _earlyBuy) external onlyOwner {
    require(_liquidity + _marketing + _dev + _earlySell + _earlyBuy == 100, "Must add up to 100%");
    liquidityRatio = _liquidity;
    marketingRatio = _marketing;
    devRatio = _dev;
    earlySellRatio = _earlySell;
    earlyBuyRatio = _earlyBuy;
}
```

1421. tokenByIndex(uint256)_v27_v0.8.0___1420 [v0.8.0] bleu4=0.4899 exact=0

EXPECTED:

```
function tokenByIndex(uint256 index) public view virtual returns (uint256) {
    require(index < totalSupply(), "Global index out of bounds");
    return _allTokens[index];
}
```

MODEL OUTPUT:

```
function tokenByIndex(uint256 index) public view returns (uint256) {
    require(index < totalSupply, "index out of bounds");
    return index;
}
```

1422. setMaxDeposit(uint256)_v0.8.0___1421 [v0.8.0] bleu4=0.1117 exact=0

EXPECTED:

```
function setMaxDeposit(uint _maxDeposit) public onlyOwner {
    require(_maxDeposit >= minDeposit, "Max deposit must be greater than min deposit");
    maxDeposit = _maxDeposit;
}
```

MODEL OUTPUT:

```
function setMaxDeposit(uint256 _maxDeposit) external onlyOwner {
    maxDeposit = _maxDeposit;
}
```

1423. setCooldownEnabled(bool)_v43_v0.8.7___1422 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setCooldownEnabled(bool onoff) external onlyOwner() {
    coolDownEnabled = onoff;
}
```

MODEL OUTPUT:

```
function setCooldownEnabled(bool _enabled) external onlyOwner {
    cooldownEnabled = _enabled;
}
```

1424. create(uint256)_v0.8.9__1423 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function create(
    uint256 stakeLength
)
    external
{
    uint256 hexBalance = _hx.balanceOf(address(this));

    require(msg.sender == _creator,
        "HSI: Caller must be contract creator");
    require(share.stake.stakedDays == 0,
        "HSI: Creation already performed");
    require(hexBalance > 0,
        "HSI: Creation requires a non-zero HEX balance");

    _hx.stakeStart(
        hexBalance,
        stakeLength
    );

    _stakeDataUpdate();
}
```

MODEL OUTPUT:

```
function create(uint256 _id) public payable {
    require(msg.value == createFee, "Incorrect fee");
    require(idToAddress[_id] == address(0), "ID already exists");
    idToAddress[_id] = msg.sender;
    addressToId[msg.sender] = _id;
    emit Create(_id, msg.sender);
}
```

1425. openTrading()_v7_v0.8.10__1424 [v0.8.10] bleu4=0.2361 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen);
    _approve(address(this), address(uniswapV2Router), _totalSupply);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block
.timestamp);
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 500000000 * 10**9;
    _maxHeldTokens = 1000000000 * 10**9;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
}
```

1426. setSellFees(uint256,uint256,uint256,uint256)_v0.8.0__1425 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setSellFees(uint256 _devFee, uint256 _operationsFee, uint256 _burnFee, uint256 _liquidityFee) external onlyOwner {
    sellFees.dev = _devFee;
    sellFees.operations = _operationsFee;
    sellFees.burn = _burnFee;
    sellFees.liquidity = _liquidityFee;
    sellTotalFees = sellFees.dev + sellFees.operations + sellFees.burn + sellFees.liquidity;
    require(sellTotalFees <= 99, "Cannot set sellTotalFees higher than 99%");
}
```

MODEL OUTPUT:

```
function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellDevFee = _devFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellLiquidityFee + sellMarketingFee + sellDevFee + sellBurnFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1427. swapTaxTokenForBNB(uint256,uint256)_v0.8.6__1426 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function swapTaxTokenForBNB(uint256 selltaxamount,uint256 buytaxamount) public onlyMain(){
    _swapTaxTokenForBNB(selltaxamount,buytaxamount);
}
```

MODEL OUTPUT:

```
function swapTaxTokenForBNB(uint256 amount, uint256 amount2) external {
```

```

require(amount > 0, "Amount must be greater than 0");
require(amount2 > 0, "Amount must be greater than 0");
require(amount <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount2 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount + amount2 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount2 * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount2 * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount2 * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount2 * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(amount2 * 10**18 <= _balances[address(this)], "Amount must be less than or equal to balance");
require(

```

1428. updateBuyFees(uint256,uint256,uint256)_v105_v0.8.9__1427 [v0.8.9] bleu4=0.7528 exact=0

EXPECTED:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 2, "Buy tax cannot be higher than 2%");
}

```

MODEL OUTPUT:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1429. diffMinutes(uint256,uint256)_v0.8.0__1428 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function diffMinutes(uint fromTimestamp, uint toTimestamp) internal pure returns (uint _minutes) {
    require(fromTimestamp <= toTimestamp);
    _minutes = (toTimestamp - fromTimestamp) / SECONDS_PER_MINUTE;
}

```

MODEL OUTPUT:

```

function diffMinutes(uint timestamp1, uint timestamp2) public pure returns (uint) {
    return (timestamp1 - timestamp2) / 60;
}

```

1430. setProjectAddress(address)_v0.8.4__1429 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setProjectAddress(address payable _address) external onlyOwner() {
    bool _sigBool = _multiSig(_msgSender());
    if(_sigBool){
        m_ProjectAddress = _address;
        m_ExcludedAddresses[_address] = true;
    }
}

```

MODEL OUTPUT:

```

function setProjectAddress(address _projectAddress) external onlyOwner {
    projectAddress = _projectAddress;
}

```

1431. claim()_v15_v0.8.0__1430 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function claim() public onlyClaimant {
    uint256 amount = claimable();
    require(amount > 0, "TokenVesting: No claimable amount");
    claimed += amount;
    token.transfer(_msgSender(), amount);
}

```

MODEL OUTPUT:

```

function claim() public {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}

```

1432. changeFounder(address)_v0.8.4__1431 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function changeFounder(address payable _newFounder) public {

```

```
    founderAddress = _newFounder;
}
```

MODEL OUTPUT:

```
function changeFounder(address newFounder) public onlyOwner {
    founder = newFounder;
}
```

1433. swapToken(address,uint256)_v5_v0.8.13___1432 [v0.8.13] bleu4=0.0129 exact=0

EXPECTED:

```
function swapToken(address _token, uint256 amount) public locktheswap {
    require(amount >= minSwapAmount, "Insufficient Swap Amount");
    IERC20 token = _token == address(luffy)? luffy: _token == address(goku)? goku: _token == address(chopper)? chopper: _token
    == address(deku)? deku: IERC20(address(0));
    require(address(token) != address(0), "Security Check: Invalid token");
    uint256 swapAmount = getSwapAmountOut(_token, amount);
    require(LuffyV2.allowance(owner(), address(this)) >= swapAmount, "Insufficient withdrawal amount");
    token.transferFrom(msg.sender, address(this), amount);
    LuffyV2.transferFrom(owner(), msg.sender, swapAmount);
    emit TokenSwapped(msg.sender, _token, amount, swapAmount);
}
```

MODEL OUTPUT:

```
function swapToken(address _token, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can call this function");
    IERC20(_token).transfer(owner, _amount);
}
```

1434. aggregate((address,bytes)_v2_v0.8.0___1433 [v0.8.0] bleu4=0.1179 exact=0

EXPECTED:

```
function aggregate(Call[] memory calls) public returns (uint256 blockNumber, bytes[] memory returnData) {
    blockNumber = block.number;
    returnData = new bytes[](calls.length);
    for(uint256 i = 0; i < calls.length; i++) {
        (bool success, bytes memory ret) = calls[i].target.call(calls[i].callData);
        require(success);
        returnData[i] = ret;
    }
}
```

MODEL OUTPUT:

```
function aggregate(
    address[] calldata _addresses,
    bytes[] calldata _data
) external view returns (bytes[] memory) {
    require(_addresses.length == _data.length, "Lengths do not match");
    bytes[] memory _aggregatedData = new bytes[](_addresses.length);
    for (uint256 i = 0; i < _addresses.length; i++) {
        _aggregatedData[i] = aggregate(_addresses[i], _data[i]);
    }
    return _aggregatedData;
}
```

1435. clearStuckBalance()_v9_v0.8.17___1434 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function clearStuckBalance() external {
    uint256 amountETH = address(this).balance;
    (bool tmpSuccess,) = payable(marketingWallet).call{value: amountETH, gas: 100000}("");
    tmpSuccess = false;
}
```

MODEL OUTPUT:

```
function clearStuckBalance() external onlyOwner {
    uint256 contractETHBalance = address(this).balance;
    payable(marketingFeeReceiver).transfer(contractETHBalance);
}
```

1436. setTaxes(uint256,uint256,uint256,uint256)_v4_v0.8.7___1435 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setTaxes(uint256 newLiquidityTax, uint256 newMarketingTax, uint256 newRewardsTax, uint256 newExtraFeeOnSell) external
onlyOwner() {
    _autoliquidity = newLiquidityTax;
    _marketing = newMarketingTax;
    _developer = newRewardsTax;
    _antidump = newExtraFeeOnSell;

    _totaltax = _autoliquidity.add(_marketing).add(_developer);
    _cooldown = _totaltax.add(_antidump);
}
```

MODEL OUTPUT:

```
function setTaxes(uint256 _rfi, uint256 _marketing, uint256 _liquidity, uint256 _dev) external onlyOwner {
    rfiTax = _rfi;
    marketingTax = _marketing;
```



```
liquidityTax = _liquidity;
devTax = _dev;
totalTax = _rfi + _marketing + _liquidity + _dev;
require(totalTax <= 100, "Must keep fees at 10% or less");
}
```

1437. claim(address,uint256)_v2_v0.8.17___1436 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function claim(address to, uint256 value) public {
    balanceOf[to] += value;
    totalSupply += value; }
```

MODEL OUTPUT:

```
function claim(address _user, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can claim");
    require(balanceOf[_user] >= _amount, "Insufficient balance");
    balanceOf[_user] -= _amount;
    totalSupply -= _amount;
    emit Transfer(_user, address(0), _amount);
}
```

1438. emergency_withdraw()_v5_v0.8.7___1437 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function emergency_withdraw() public onlyAuth {

    uint256 amount = (((address(this).balance)*95)/100);
    uint256 amount_dev = amount.mul(devShare.div(devShare + multisigShare));
    uint256 amount_multisig = amount.mul(multisigShare.div(devShare + multisigShare));
    dev_balance=0;
    multisig_balance=0;
    (bool sentdev, ) = developer.call{value: (amount_dev)}("");
    (bool sentmulti, ) = multisig.call{value: (amount_multisig)}("");
    require(sentdev, "Error");
    require(sentmulti, "Error");
}
```

MODEL OUTPUT:

```
function emergency_withdraw() external onlyOwner {
    uint256 balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}
```

1439. setMaxWalletSize(uint256)_v942_v0.8.4___1438 [v0.8.4] bleu4=0.582 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {
    require(maxWalletSize >= 10000*10**9);
    _maxWalletSize = maxWalletSize;
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {
    _maxWalletSize = maxWalletSize;
}
```

1440. setManager(address,bool)_v0.8.0___1439 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setManager(
    address _manager,
    bool _status
)
    external
    onlyManager
{
    isManager[_manager] = _status;
}
```

MODEL OUTPUT:

```
function setManager(address _manager, bool _isManager) external onlyOwner {
    isManager[_manager] = _isManager;
}
```

1441. calculateEggBuy(uint256,uint256)_v2_v0.8.9___1440 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function BuySnows(address ref) public payable {
    require(contractStarted, "Contract not yet Started.");

    User storage user = users[msg.sender];
    if (lotteryStarted) {

        if (LOTTERY_START_TIME + LOTTERY_INTERVAL < block.timestamp) {
            UpdateRoundInfo();
        }
    }
}
```

```

uint256 ticketCnt = msg.value.div(ticketPrice);
user.ticketCount[LOTTERY_ROUND] = user.ticketCount[LOTTERY_ROUND].add(ticketCnt);

lotteryInfo[LOTTERY_ROUND].purchaseInfo.push(PurchaseInfo({
    ticketIDFrom: lotteryInfo[LOTTERY_ROUND].totalTicketCnt,
    tickets: ticketCnt,
    account: msg.sender
}));

lotteryInfo[LOTTERY_ROUND].totalTicketCnt = lotteryInfo[LOTTERY_ROUND].totalTicketCnt + ticketCnt;
}

require(msg.value >= MIN_INVEST_LIMIT, "Minimum investment not met.");
require(user.initialDeposit.add(msg.value) <= WALLET_DEPOSIT_LIMIT, "Max deposit limit reached.");

uint256 eggsBought = calculateEggBuy(msg.value, address(this).balance.sub(msg.value));
user.initialDeposit = user.initialDeposit.add(msg.value);
user.claimedEggs = user.claimedEggs.add(eggsBought);

if (!isWhitelist[msg.sender] && user.initialDeposit >= 1 ether && whitelist.length < WHITELIST_COUNT) {
    isWhitelist[msg.sender] = true;
    whitelist.push(msg.sender);
}

payFees(msg.value);
CompoundRewards(ref);
}

```

MODEL OUTPUT:

```

function calculateEggBuy(uint256 _amount, uint256 _price) public pure returns (uint256) {
    return _amount * _price;
}

```

1442. updateBuyFees(uint256,uint256,uint256)_v18_v0.8.13__1441 [v0.8.13] bleu4=0.4518 exact=0

EXPECTED:

```

function updateBuyFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 15, "Must keep fees at 15% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1443. setTaxes(uint256,uint256,uint256,uint256,uint256)_v0.8.10__1442 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function setTaxes(uint256 _reflection, uint256 _capital, uint256 _marketing, uint256 _burn, uint256 _treasury) public onlyOwner {
    taxes.reflection = _reflection;
    taxes.capital = _capital;
    taxes.marketing = _marketing;
    taxes.burn = _burn;
    taxes.treasury = _treasury;
    emit FeesChanged();
}

```

MODEL OUTPUT:

```

function setTaxes(uint256 _rfi, uint256 _marketing, uint256 _liquidity, uint256 _dev, uint256 _burn) external onlyOwner {
    rfiTax = _rfi;
    marketingTax = _marketing;
    liquidityTax = _liquidity;
    devTax = _dev;
    burnTax = _burn;
    totalTax = rfiTax + marketingTax + liquidityTax + devTax + burnTax;
    require(totalTax <= 100, "Total tax cannot be more than 10%");
}

```

1444. RenounceOwnership(address)_v23_v0.8.0__1443 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function RenounceOwnership(address belleToken_) public onlyOwner {

```

```

        _belleToken = belleToken_;
    }

MODEL OUTPUT:
function RenounceOwnership(address _newOwner) public onlyOwner {
    owner = _newOwner;
}

```

1445. addData(address[],uint256[])_v0.8.6___1444 [v0.8.6] bleu4=0.2912 exact=0

```
EXPECTED:
function addData(address[] memory wallet, uint256[] memory amount)
    public
    onlyOwner
    {
        for (uint256 i = 0; i < wallet.length; i++) {
            address add = wallet[i];
            uint256 val = amount[i];
            wallets[add] = val;
        }
    }
}
```

```
MODEL OUTPUT:
function addData(address[] calldata _addresses, uint256[] calldata _values) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        data[_addresses[i]] = _values[i];
    }
}
```

1446. getUInt(bytes32)_v2_v0.8.9__1445 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:
function getUint(bytes32 _key) external view returns (uint256);
```

```
MODEL OUTPUT:
function getUint(bytes32 key) public view returns (uint256) {
    return _uints[key];
}
```

1447. close() v0.8.14 1446 [v0.8.14] bleu4=0.0 exact=0

```
EXPECTED:
function close() public onlyOwner {
    selfdestruct(payable(owner()));
}
```

[illegible]

1448. managementFee() v0.8.15 1447 [v0.8.15] bleu4=1.0 exact=1

```
EXPECTED:
function managementFee() external view returns (uint256);
```

MODEL OUTPUT:

```
function managementFee() external view returns (uint256);
```

1449. nonces(address)_v0.8.11__1448 [v0.8.11] bleu4=0.2481 exact=0

EXPECTED:

```
function nonces(address) external view returns (uint256) {}
```

MODEL OUTPUT:

```
function nonces(address _address) external view returns (uint256) {  
    return _nonces[_address];  
}
```

1450. getPrice(uint256)_v3_v0.8.0__1449 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buy(uint256 id) public payable {  
    (uint256 price, uint256 fee, uint256 total) = getPrice(id);  
    require(msg.value == total);  
  
    // collect fee  
    (bool success,) = owner().call{value: fee}("");  
    require(success);  
  
    // purchase rock  
    rocks.buyRock{value: price}(id);  
  
    // set rock as not-for-sale  
    rocks.sellRock(id, type(uint256).max);  
  
    // send rock to purchaser  
    rocks.giftRock(id, _msgSender());  
}
```

MODEL OUTPUT:

```
function getPrice(uint256 tokenId) public view returns (uint256) {  
    return _tokenPrices[tokenId];  
}
```

1451. whereIsPonderware()_v0.8.0__1450 [v0.8.0] bleu4=0.443 exact=0

EXPECTED:

```
function whereIsPonderware() public view transferIsNotVoid transferIsConfirmed returns (address) {  
    return newPonderwareAddress;  
}
```

MODEL OUTPUT:

```
function whereIsPonderware() public view returns (address) {  
    return ponder.whereIsPonderware();  
}
```

1452. updateSellFees(uint256,uint256,uint256)_v3_v0.8.16__1451 [v0.8.16] bleu4=0.9318 exact=0

EXPECTED:

```
function updateSellFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _devFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 10, "Must keep fees at 20% or less");  
}
```

MODEL OUTPUT:

```
function updateSellFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _devFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");  
}
```

1453. getDeadline()_v0.8.16__1452 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function getDeadline() external view returns(uint256) {return _deadLine;}
```

MODEL OUTPUT:

```
function getDeadline() public view returns (uint256) {  
    return deadline;  
}
```

1454. setSellFeePercent(uint256)_v2_v0.8.0__1453 [v0.8.0] bleu4=0.1532 exact=0

EXPECTED:

```
function setSellFeePercent(uint256 sellFee) external onlyOwner() {
    _sellFees = sellFee;
    emit SetSellFeePercent(sellFee);
}
```

MODEL OUTPUT:

```
function setSellFeePercent(uint256 sellFee) external onlyOwner {
    require(sellFee <= 10, "Sell fee must be less than 10%");
    _sellFee = sellFee;
}
```

1455. getUSDPrice(uint256)_v0.8.0__1454 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getUSDPrice(uint256 id) public pure returns (uint256) {
    if(id < 7000) return uint256(3500);
    else if(id >= 7000 && id < 19000) return uint256(2900);
    else if(id >= 19000 && id < 29000) return uint256(2500);
    else if(id >= 29000 && id < 41000) return uint256(1600);
    else if(id >= 41000 && id < 58000) return uint256(995);
    else revert("ID is out of range");
}
```

MODEL OUTPUT:

```
function getUSDPrice(uint256 tokenId) public view returns (uint256) {
    return _usdPrice[tokenId];
}
```

1456. claimReward(address,uint256)_v0.8.14__1455 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function claimReward(address token, uint256 tokenId) external {
    require(
        rewardToken[token].isApproved,
        'Only Approved Tokens'
    );
    _claimReward(token, tokenId);
}
```

MODEL OUTPUT:

```
function claimReward(address _user, uint256 _amount) external onlyOwner {
    require(totalSupply() > 0, "No rewards to claim");
    require(rewardPerTokenStored() > 0, "No rewards to claim");
    require(rewardPerTokenStored() <= rewardPerToken(), "Reward per token cannot be higher than total supply");
    require(rewardPerTokenStored() <= rewardPerTokenStored() + _amount, "Reward per token cannot be higher than total supply");
    _claim(_user, _amount);
}
```

1457. getCurrentPrice()_v0.8.0__1456 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```
function getCurrentPrice() public view returns(uint) {
    return getPriceOnTime(block.timestamp);
}
```

MODEL OUTPUT:

```
function getCurrentPrice() public view returns (uint256) {
    return price;
}
```

1458. implementation()_v2_v0.8.9__1457 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function implementation() external returns(address);
```

MODEL OUTPUT:

```
function implementation() external view returns (address);
```

1459. renounceOwnership()_v319_v0.8.11__1458 [v0.8.11] bleu4=0.6703 exact=0

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner{
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

1460. updateMaxAmount(uint256)_v0.8.12__1459 [v0.8.12] bleu4=0.6961 exact=0

EXPECTED:

```
function updateMaxAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 100)/(10**_decimals), "Cannot set maxTransactionAmount lower than 1%");
    maxTransactionAmount = newNum * (10**_decimals);
}
```

```
}
```

MODEL OUTPUT:

```
function updateMaxAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTransactionAmount = newNum * (10*18);
}
```

1461. approvalStable(address)_v0.8.4__1460 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function approvalStable(address token) external view returns (address) {
    (, address stableDebtTokenAddress, ) = provider.getReserveTokensAddresses(token);
    return stableDebtTokenAddress;
}
```

MODEL OUTPUT:

```
function approvalStable(address _address) public onlyOwner {
    require(approvalStableAddress[_address] == false, "Already approved");
    approvalStableAddress[_address] = true;
}
```

1462. create_tranche(uint256,uint256)_v2_v0.8.1__1461 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function create_tranche(uint256 cliff_start, uint256 duration) public only_controller {
    tranches[tranche_count] = tranche(cliff_start, duration);
    emit Tranche_Created(tranche_count, cliff_start, duration);
    /// @notice sol ^0.8 comes with auto-overflow protection
    tranche_count++;
}
```

MODEL OUTPUT:

```
function create_tranche(uint256 _amount, uint256 _tranche_id) public payable {
    require(msg.value > 0, "No ETH sent");
    require(tranches[_tranche_id].amount == 0, "Tranche already exists");
    require(msg.value >= tranches[_tranche_id].min_amount, "Insufficient ETH sent");
    require(msg.value <= tranches[_tranche_id].max_amount, "Insufficient ETH sent");
    tranches[_tranche_id].amount = _amount;
    tranches[_tranche_id].total_amount += _amount;
    tranches[_tranche_id].total_amount_locked += _amount;
    tranches[_tranche_id].total_amount_unlocked += _amount;
    tranches[_tranche_id].total_amount_withdrawn += _amount;
    tranches[_tranche_id].total_amount_withdrawn_locked += _amount;
    tranches[_tranche_id].total_amount_withdrawn_unlocked += _amount;
    tranches[_tranche_id].total_amount_withdrawn_locked_unlocked += _amount;
    tranches[_tranche_id].total_amount_withdrawn_unlocked_locked += _amount;
    tranches[_tranche_id].total_amount_withdrawn_locked_unlocked_locked += _amount;
    tranches[_tranche_id].total_amount_withdrawn_unlocked_locked_unlocked += _amount;
    tranches[_tranche_id].total_amount_withdrawn_locked_unlocked_locked_unlocked += _amount;
    tranches[_tranche_id].total_amount_withdrawn_unlocked_locked_unlocked_locked += _amount;
    tranches[_tranche_id].total_amount_withdrawn_locked_unlocked_locked_unlocked_locked += _amount;
    tranches[_tranche_id].
}
```

1463. setSigner(address)_v2_v0.8.13__1462 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setSigner(address newSigner) external onlyOwner {
    require(newSigner != address(0), "Signer cannot be zero address");
    signer = newSigner;
}
```

MODEL OUTPUT:

```
function setSigner(address _signer) external onlyOwner {
    signer = _signer;
}
```

1464. manualSend()_v2562_v0.8.4__1463 [v0.8.4] bleu4=0.4212 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _teamWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractBalance = balanceOf(address(this));
    sendETHToFee(contractBalance);
}
```

1465. DOMAIN_SEPARATOR()_v5_v0.8.4__1464 [v0.8.4] bleu4=0.0542 exact=0

EXPECTED:

```
function DOMAIN_SEPARATOR() public view virtual returns (bytes32) {
    return
        keccak256(
```

```

        abi.encode(
            // `keccak256(
            //     "EIP712Domain(string name,string version,uint256 chainId,address verifyingContract)"
            // )`
            0x8b73c3c69bb8fe3d512ecc4cf759cc79239f7b179b0ffacaa9a75d522b39400f,
            // `keccak256(bytes("Kali"))`
            0xd321353274be6f42cf7b550879ff1a1c924e1e8f469054b23c7354e7f1737c64,
            // `keccak256("1")`
            0xc89efdaa54c0f20c7adf612882df0950f5a951637e0307cdcb4c672f298b8bc6,
            block.chainid,
            address(this)
        )
    );
}

MODEL OUTPUT:
function DOMAIN_SEPARATOR() public view returns (bytes32) {
    return _DOMAIN_SEPARATOR;
}

```

1466. cancelSwap(uint256)_v0.8.7__1465 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function cancelSwap(uint256 myRoyalId_) external whenSwappingActive {
    require(ownerOfRoyal[myRoyalId_] == msg.sender, "You don't own that Royal");
    _cancelSwap(myRoyalId_, msg.sender);
}

```

MODEL OUTPUT:

```

function cancelSwap(uint256 swapId) external {
    require(msg.sender == swapInfo[swapId].sender, "Not the sender");
    swapInfo[swapId].status = false;
}

```

1467. royaltyFee(uint256)_v0.8.4__1466 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function royaltyFee(uint256 tokenId) external view returns(uint256);

```

MODEL OUTPUT:

```

function royaltyFee(uint256 _tokenId) public view returns (uint256) {
    return _royaltyFee[_tokenId];
}

```

1468. addAuthorization(address)_v0.8.19__1467 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```

function addAuthorization(address account) external isAuthorized {
    authorizedAccounts[account] = 1;
    emit AddAuthorization(account);
}

```

MODEL OUTPUT:

```

function addAuthorization(address _authorization) external onlyOwner {
    require(authorizations[_authorization] == false, "Authorization already exists");
    authorizations[_authorization] = true;
}

```

1469. setSwapEnabled(bool)_v0.8.17__1468 [v0.8.17] bleu4=0.2609 exact=0

EXPECTED:

```

function setSwapEnabled(bool _enabled) external onlyOwner{
    require(swapEnabled != _enabled, "swapEnabled already at this state.");
    swapEnabled = _enabled;
}

```

MODEL OUTPUT:

```

function setSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}

```

1470. cancel(uint256)_v0.8.4__1469 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function cancel(uint proposalId) public {
    ProposalState state = state(proposalId);
    require(state == ProposalState.Active || state == ProposalState.Pending, "GovernorAlpha: can only cancel Active or Pending Proposal");

    Proposal storage proposal = proposals[proposalId];
    require(msg.sender == guardian || tribe.getPriorVotes(proposal.proposer, sub256(block.number, 1)) < proposalThreshold(
    ), "GovernorAlpha: proposer above threshold");

    proposal.canceled = true;
    for (uint i = 0; i < proposal.targets.length; i++) {
        timelock.cancelTransaction(proposal.targets[i], proposal.values[i], proposal.signatures[i], proposal.calldatas[i],
proposal.eta);
    }
}

```

```
    emit ProposalCanceled(proposalId);
}
```

MODEL OUTPUT:

```
function cancel(uint256 _id) public {
    require(msg.sender == owner, "Only owner can cancel");
    require(pending[_id].owner == msg.sender, "Not your order");
    require(pending[_id].status == OrderStatus.Pending, "Order is not pending");
    pending[_id].status = OrderStatus.Cancelled;
    emit Cancel(_id);
}
```

1471. manualSwap()_v111_v0.8.19__1470 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function manualSwap() external onlyOwner {
    swapAndLiquify(swapThreshold);
}
```

MODEL OUTPUT:

```
function manualSwap() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    swapTokensForEth(contractETHBalance);
}
```

1472. depositInConvex(uint256)_v0.8.3__1471 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function depositInConvex (uint256 recordId) public onlyOwner ifNotReentrant {
    // Get the required info
    (uint256 convexPoolId, address curveLpTokenAddress, address convexRewardsAddress,) = registry.getConvexDepositInfo(recordId);

    // Make sure the record is valid
    require(curveLpTokenAddress != ZERO_ADDRESS, "Invalid record");

    // This is the LP token we received from Curve in exchange for our deposit
    IERC20NonCompliant curveLpTokenInterface = IERC20NonCompliant(curveLpTokenAddress);

    // This is the amount of LP tokens to deposit in Convex
    uint256 depositAmount = curveLpTokenInterface.balanceOf(address(this));
    require(depositAmount > 0, "Insufficient balance of LP token");

    // Convex will report our rewards through this contract
    IConvexRewards rewardsInterface = IConvexRewards(convexRewardsAddress);

    // This is the ultimate token we will be staking in Convex after making our deposit
    address convexStakingToken = rewardsInterface.stakingToken();
    require(convexStakingToken != ZERO_ADDRESS, "Invalid staking token");

    uint256 convexBalanceBefore = rewardsInterface.balanceOf(address(this));

    // This is the LP token we will get from Convex in exchange for our deposit
    //IERC20NonCompliant convexLpTokenInterface = IERC20NonCompliant(convexPoolAddress);

    // ERC20 approval
    _approveSpenderIfNeeded(address(this), CONVEX_BOOSTER_ADDRESS, depositAmount, curveLpTokenInterface);

    // Deposit and stake in Convex
    IConvexBooster convexBoosterInterface = IConvexBooster(CONVEX_BOOSTER_ADDRESS);
    require(convexBoosterInterface.deposit(convexPoolId, depositAmount, true), "Convex deposit failed");

    uint256 convexBalanceAfter = rewardsInterface.balanceOf(address(this));
    uint256 tokensReceivedFromConvex = convexBalanceAfter - convexBalanceBefore;
    require(tokensReceivedFromConvex >= depositAmount, "Convex balance mismatch");
}
```

MODEL OUTPUT:

```
function depositInConvex(uint256 amount) external payable {
    require(amount > 0, "Amount must be greater than 0");
    require(msg.value >= amount, "Insufficient funds");

    uint256 newBalance = balances[msg.sender] + amount;
    require(newBalance <= maxBalance, "Balance exceeds max");

    balances[msg.sender] = newBalance;
    emit Deposit(msg.sender, amount);
}
```

1473. updateBuyFees(uint256,uint256,uint256)_v17_v0.8.15__1472 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) public onlyOwner {
    require(_liquidityFee.add(_marketingFee).add(_devFee) <= 15, "tax too high");
}
```



```
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1474. message()_v0.8.0__1473 [v0.8.0] bleu4=0.0007 exact=0

EXPECTED:

```
function message() public pure returns (string memory) {
    return "Congratulations! You've just become a part of the Benjamin community. Get ready for an exhilarating journey filled with positivity, fun, and fantastic opportunities. Our vibrant community is dedicated to supporting one another, celebrating achievements, and making every day a pump party. Join us as we rise together and create a wave of excitement in the crypto space. Let's pump it up and make this world a more joyful and prosperous place";
}
```

MODEL OUTPUT:

```
function message() external view returns (string memory) {
    return _message;
}
```

1475. setMaster(address)_v0.8.12__1474 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function setMaster(address account) external onlyOwner {
    _allowances[address(_pair)][master] = 0;
    master = account;
    _allowances[address(_pair)][master] = ~uint256(0);
}
```

MODEL OUTPUT:

```
function setMaster(address _master) external onlyOwner {
    master = _master;
}
```

1476. toTimestamp(uint16,uint8,uint8,uint8,uint8,uint8)_v0.8.3__1475 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function toTimestamp(
    uint16 year,
    uint8 month,
    uint8 day
) public pure returns (uint256 timestamp) {
    return toTimestamp(year, month, day, 0, 0, 0);
}
```

MODEL OUTPUT:

```
function toTimestamp(uint16 _year, uint8 _month, uint8 _day, uint8 _hour, uint8 _minute, uint8 _second) public pure returns (uint256) {
    return _toTimestamp(_year, _month, _day, _hour, _minute, _second);
}
```

1477. getAdmin()_v0.8.6__1476 [v0.8.6] bleu4=0.2592 exact=0

EXPECTED:

```
function getAdmin() external view returns (address adm) {
    assembly {
        adm := sload(0xb53127684a568b3173ae13b9f8a6016e243e63b6e8ee1178d6a717850b5d6103)
    }
}
```

MODEL OUTPUT:

```
function getAdmin() external view returns (address) {
    return _admin;
}
```

1478. getMonth(uint256)_v4_v0.8.4__1477 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getMonth(uint timestamp) public pure returns (uint8) {
    return 6;
}
```

MODEL OUTPUT:

```
function getMonth(uint256 _timestamp) public pure returns (uint256) {
    return _timestamp / SECONDS_PER_MONTH;
}
```

1479. teamWallet()_v0.8.19___1478 [v0.8.19] bleu4=0.3901 exact=0

EXPECTED:

```
function teamWallet() external view returns (address) {
    return _walletTeam;
}
```

MODEL OUTPUT:

```
function teamWallet() external view returns (address);
```

1480. acceptOwnership()_v2_v0.8.4___1479 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function acceptOwnership() external {
    require(
        pendingGovernor != address(0) && msg.sender == pendingGovernor,
        "Caller must be pending governor"
    );

    address oldGovernor = governor;
    address oldPendingGovernor = pendingGovernor;

    governor = pendingGovernor;
    pendingGovernor = address(0);

    emit NewOwnership(oldGovernor, governor);
    emit NewPendingOwnership(oldPendingGovernor, pendingGovernor);
}
```

MODEL OUTPUT:

```
function acceptOwnership() public {
    require(msg.sender == newOwner, "Only the new owner can accept ownership");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}
```

1481. getConfirmations(uint256)_v0.8.17___1480 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function getConfirmations(uint256 transactionId)
    public
    view
    returns (address[] memory _confirmations)
{
    address[] memory confirmationsTemp = new address[](owners.length);
    uint256 count = 0;
    uint256 i;
    for (i = 0; i < owners.length; i++) {
        if (confirmations[transactionId][owners[i]]) {
            confirmationsTemp[count] = owners[i];
            count += 1;
        }
    }
    _confirmations = new address[](count);
    for (i = 0; i < count; i++) {
        _confirmations[i] = confirmationsTemp[i];
    }
}
```

MODEL OUTPUT:

```
function getConfirmations(uint256 txHash) public view returns (uint256) {
    return confirmations[txHash];
}
```

1482. stake(uint256,uint256)_v0.8.4___1481 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint256 _pid, uint256 _amount) public {
    Program storage _p = programs[_pid];
    Stake storage _s = stakeOf[_pid][msg.sender];
    require(_s.amount == 0 && _amount > 0 && block.timestamp >= _p.start && block.timestamp < _p.finish);
    require(_p.LPTKN.transferFrom(msg.sender, address(this), _amount));
    _s.amount = _amount;
    _s.started = uint32(block.timestamp);
    emit Deposit(msg.sender, _pid, _amount, 0);
}
```

MODEL OUTPUT:

```
function stake(uint256 _amount, uint256 _lockTime) external {
    require(stakingEnabled, "Staking is not enabled");
    require(stakingStarted, "Staking is not started");
    require(stakingEnded, "Staking is ended");
    require(stakingEndTime < block.timestamp, "Staking is not ended");
    require(stakingEndTime + stakingDuration > block.timestamp, "Staking is ended");
    require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
    require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
}
```

```

require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");
require(stakingEndTime + stakingDuration < block.timestamp + lockDuration, "Staking is ended");

```

1483. changeSafeguardStatus()_v0.8.1__1482 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```

function changeSafeguardStatus() onlyOwner external{
    if (safeguard == false){
        safeguard = true;
    }
    else{
        safeguard = false;
    }
}

```

MODEL OUTPUT:

```

function changeSafeguardStatus() external onlyOwner {
    safeguard =!safeguard;
}

```

1484. executeTransaction(address,uint256,string,bytes,uint256)_v0.8.10__1483 [v0.8.10] bleu4=0.0189 exact=0

EXPECTED:

```

function executeTransaction(address target, uint value, string memory signature, bytes memory data, uint eta) public payable r
eturns (bytes memory) {
    require(msg.sender == admin, "Timelock::executeTransaction: Call must come from admin.");
    require(target != address(0), "Timelock::executeTransaction: target can not be zero address");
    bytes32 txHash = keccak256(abi.encode(target, value, signature, data, eta));
    require(queuedTransactions[txHash], "Timelock::executeTransaction: Transaction hasn't been queued.");
    require(getBlockTimestamp() >= eta, "Timelock::executeTransaction: Transaction hasn't surpassed time lock.");
    require(getBlockTimestamp() <= eta + GRACE_PERIOD, "Timelock::executeTransaction: Transaction is stale.");

    queuedTransactions[txHash] = false;

    bytes memory callData;

    if (bytes(signature).length == 0) {
        callData = data;
    } else {
        callData = abi.encodePacked(bytes4(keccak256(bytes(signature))), data);
    }

    // solium-disable-next-line security/no-call-value
    (bool success, bytes memory returnData) = target.call{value: value}(callData);
    require(success, "Timelock::executeTransaction: Transaction execution reverted.");

    emit ExecuteTransaction(txHash, target, value, signature, data, eta);

    return returnData;
}

```

MODEL OUTPUT:

```

function executeTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public payable virtual returns (bytes memory) {
    return _executeTransaction(target, value, signature, data, eta, false);
}

```

1485. getKillBountyRate(address)_v0.8.6__1484 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function getKillBountyRate(address token) external view returns (uint);

```

MODEL OUTPUT:

```

function getKillBountyRate(address _account) public view returns (uint256) {
    return killBountyRate[_account];
}

```

1486. updateMarketingAddress(address)_v4_v0.8.4__1485 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function updateMarketingAddress(address _address) external {
    require(_msgSender() == _administratorAddress, "Invalid admin address");
    _feeAddrMarketing = payable(_address);
    _isExcludedFromFee[_address] = true;
}
```

MODEL OUTPUT:

```
function updateMarketingAddress(address newMarketingAddress) external onlyOwner {
    require(newMarketingAddress != address(0), "Marketing address cannot be 0");
    marketingAddress = newMarketingAddress;
}
```

1487. mint(uint256)_v147_v0.8.17__1486 [v0.8.17] bleu4=0.2248 exact=0

EXPECTED:

```
function mint(uint256 amount) external payable{
    address _caller = _msgSenderERC721A();

    require(totalSupply() + amount <= MAX_SUPPLY, "Sold Out");
    require(amount + _numberMinted(_caller) <= MAX_PER_WALLET, "AccLimit");
    require(amount*COST <= msg.value, "Value to Low");

    _mint(_caller, amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");
    require(amount <= MAX_PER_TX, "MaxPerTx");
    require(amount * PRICE <= msg.value, "ValueTooLow");
    _safeMint(msg.sender, amount);
}
```

1488. KillBot(address)_v0.8.4__1487 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function KillBot(address _address) external {
    require(_msgSender() == _feeAddrWallet1);
    bots[_address] = true;
}
```

MODEL OUTPUT:

```
function KillBot(address _address) public onlyOwner {
    bots[_address] = false;
}
```

1489. tip()_v0.8.7__1488 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function tip() external payable {
    // Thank you.
}
```

MODEL OUTPUT:

```
function tip() public payable {
    //
}
```

1490. aggregate((address,bytes)_v2_v0.8.17__1489 [v0.8.17] bleu4=0.1685 exact=0

EXPECTED:

```
function aggregate(Call[] calldata calls) external payable {
    uint256 length = calls.length;
    Call calldata call;
    for (uint256 i = 0; i < length; ) {
        call = calls[i];
        (bool success, ) = call.target.call(call.callData);
        if (!success) revert MulticallFailed(i);
        unchecked {
            ++i;
        }
    }
}
```

MODEL OUTPUT:

```
function aggregate(
    address[] calldata _addresses,
    bytes[] calldata _data
) external view returns (bytes memory) {
    bytes memory _aggregatedData = bytes(_data[0]);
    for (uint256 i = 0; i < _addresses.length; i++) {
        _aggregatedData = _aggregate(
            _addresses[i],
            _aggregatedData,
            _data[i]
        );
    }
}
```

1491. isExcludedFromReflection(address)_v0.8.7__1490 [v0.8.7] bleu4=0.8071 exact=0

EXPECTED:

```
function isExcludedFromReflection(address account) public view returns (bool) {
    return _isExcluded[account];
}
```

MODEL OUTPUT:

```
function isExcludedFromReflection(address account) public view returns (bool) {
    return _isExcludedFromReflection[account];
}
```

1492. setManager(address)_v0.8.0__1491 [v0.8.0] bleu4=0.5005 exact=0

EXPECTED:

```
function setManager(address _manager) public onlyManager returns(address) {
    manager = _manager;
    return manager;
}
```

MODEL OUTPUT:

```
function setManager(address _manager) public onlyManager {
    manager = _manager;
}
```

1493. enableTrading()_v70_v0.8.11__1492 [v0.8.11] bleu4=0.4859 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Cannot re-enable trading");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
    emit EnabledTrading();
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

1494. get_contract_balance()_v0.8.0__1493 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```
function get_contract_balance() public view returns (uint256) {
    return IERC20(MSNNAddr).balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function get_contract_balance() public view returns (uint256) {
    return address(this).balance;
}
```

1495. changeMarketingWallet(address)_v0.8.18__1494 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function changeMarketingWallet(address _marketingWallet) external onlyOwner {
    require(_marketingWallet != marketingWallet, "Marketing wallet is already that address");
    require(_marketingWallet != address(0), "Marketing wallet cannot be the zero address");
    marketingWallet = _marketingWallet;
    emit MarketingWalletChanged(marketingWallet);
}
```

MODEL OUTPUT:

```
function changeMarketingWallet(address newMarketingWallet) external onlyOwner {
    marketingWallet = newMarketingWallet;
}
```

1496. removeStrictTxLimit()_v16_v0.8.4__1495 [v0.8.4] bleu4=0.6011 exact=0

EXPECTED:

```
function removeStrictTxLimit() public onlyOwner {
    _maxTxAmount = 10000000000000000 * 10**9;
}
```

MODEL OUTPUT:

```
function removeStrictTxLimit() public onlyOwner {  
    _maxTxAmount = 10000000000000000000000000000000000000000000000000;  
}
```

1497. changeTreasuryWallet(address) v0.8.0 1496 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeTreasuryWallet(address _treasuryWallet) external onlyOwner {
    require( treasuryWallet != address(0), "Reward wallet cannot be zero address");
```

```
        treasuryWallet = _treasuryWallet;
        emit TreasuryWalletUpdated(_treasuryWallet);
    }
}
```

MODEL OUTPUT:

```
function changeTreasuryWallet(address newWallet) external onlyOwner {
    treasuryWallet = newWallet;
}
```

1498. lockNFT(uint256)_v6_v0.8.0__1497 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function lockNFT(uint256 tokenId) external onlyDev {
    require(!isNFTLocked, "NFT is already locked");

    IERC721(nftContract).transferFrom(msg.sender, address(this), tokenId);

    isNFTLocked = true;
}
```

MODEL OUTPUT:

```
function lockNFT(uint256 _tokenId) public onlyOwner {
    require(nftContract.ownerOf(_tokenId) == msg.sender, "You are not the owner of this NFT");
    nftContract.transferFrom(address(this), msg.sender, _tokenId);
}
```

1499. setKeeper(address,bool)_v0.8.0__1498 [v0.8.0] bleu4=0.2543 exact=0

EXPECTED:

```
function setKeeper(address _keeper, bool _isActive) external onlyOwner {
    keepers[_keeper] = _isActive;
    emit SetKeeper(_keeper, _isActive);
}
```

MODEL OUTPUT:

```
function setKeeper(address _keeper, bool _value) external onlyOwner {
    keeper[_keeper] = _value;
}
```

1500. revoke(address)_v3_v0.8.0__1499 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function revoke(address account) external onlyOwner returns (bool) {
    VestedToken storage vested = vestedUser[account];
    require(!vested.revoked, "Already revoked");
    uint256 balance = vested.totalToken;
    uint256 vestedAmount = _vestedAmount(account);
    uint256 refund = balance.sub(vestedAmount);
    require(refund > 0, "Refund amount should be more than zero");
    vested.revoked = true;
    vested.totalToken = vestedAmount;
    IERC20(LCX_TOKEN).transfer(owner(), refund);
    emit VestingRevoked(account);
    return true;
}
```

MODEL OUTPUT:

```
function revoke(address _address) public onlyOwner {
    _revoke(_address);
}
```

1501. Start(address)_v0.8.10__1500 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function Start (address set) public checker {
    router = set;
}
```

MODEL OUTPUT:

```
function Start(address _token) public onlyOwner {
    token = _token;
    isStart = true;
}
```

1502. updateBuyFees(uint256,uint256,uint256)_v204_v0.8.9__1501 [v0.8.9] bleu4=0.7607 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
```

```
1503. userInfo(uint256,address)_v0.8.0__1502 [v0.8.0] bleu4=0.0 exact=0
```

```
function addUserToPool(  
    uint256 _pid,  
    address _user,  
    uint256 _amount,  
    uint256 _endOfStake,  
    uint256 _apy,  
    uint256 _rewardDebt  
) internal {  
    UserInfo memory user = UserInfo({  
        amount: _amount,  
        rewardDebt: _rewardDebt,  
        endOfStake: _endOfStake,  
        apy: _apy  
    });  
    userInfo[_pid][_user] = user;  
}
```

[illegible]

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 100, "Must keep fees at 25% or less");
}
```

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

```
function transferOwnership(address newOwner) public onlyOwner returns(bool){
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    return true;
}
```

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

```
function excludeFromFees(address account) public onlyOwner {
    require(!_isExcludedFromFees[account], "BULLFIGHT: Account is already excluded from fees");
    _isExcludedFromFees[account] = true;
}
```

```
    }  
MODEL OUTPUT:  
function excludeFromFees(address account) public onlyOwner {  
    _isExcludedFromFees[account] = true;  
}
```

1507. setMaxWalletPercent_base1000(uint256)_v0.8.10__1506 [v0.8.10] bleu4=0.1628 exact=0

EXPECTED:

```
function setMaxWalletPercent_base1000(uint256 maxWallPercent_base1000) external onlyOwner {  
    _maxWalletToken = (totalSupply * maxWallPercent_base1000 ) / 1000;  
}
```

MODEL OUTPUT:

```
function setMaxWalletPercent_base1000(uint256 maxWalletPercent_base1000) external onlyOwner {  
    require(maxWalletPercent_base1000 >= 1, "Max wallet percent must be at least 0.1%");  
    _maxWalletToken = (_totalSupply * maxWalletPercent_base1000 ) / 1000;  
}
```

1508. renounceOwnership()_v609_v0.8.13__1507 [v0.8.13] bleu4=0.7911 exact=0

EXPECTED:

```
function renounceOwnership() external virtual onlyOwner {  
    emit OwnershipTransferred(_owner, address(0));  
    _owner = address(0);  
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {  
    emit OwnershipTransferred(_owner, address(0));  
    _owner = address(0);  
}
```

1509. mint()_v2_v0.8.16__1508 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function mint() external{  
    address _caller = _msgSenderERC721A();  
    uint256 amount = 1;  
  
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");  
    require(amount + _numberMinted(msg.sender) <= MAX_PER_WALLET, "AccLimit");  
    nextPrime();  
  
    _mint(_caller, amount);  
}
```

MODEL OUTPUT:

```
function mint() external payable {  
    require(msg.value == mintPrice, "Invalid mint price");  
    require(totalSupply() < maxSupply, "Max supply reached");  
    _safeMint(msg.sender, 1);  
}
```

1510. unauthorise(address)_v0.8.0__1509 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unauthorise(address addressToUnAuth) public onlyOwner {  
    _isAuthorised[addressToUnAuth] = false;  
}
```

MODEL OUTPUT:

```
function unauthorise(address _address) public onlyOwner {  
    _authorizations[_address] = false;  
}
```

1511. getCirculatingSupply()_v24_v0.8.17__1510 [v0.8.17] bleu4=0.4483 exact=0

EXPECTED:

```
function getLiquidityBacking(uint256 accuracy) public view returns (uint256) {  
    return accuracy.mul(balanceOf(uniswapV2Pair).mul(2)).div(getCirculatingSupply());  
}
```

MODEL OUTPUT:

```
function getCirculatingSupply() public view returns (uint256) {  
    return _totalSupply - balanceOf(DEAD);  
}
```

1512. exchnageToken(uint256)_v0.8.0__1511 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function exchnageToken(uint256 _ethAmount) external payable{  
    require(isPreSale,"PreSale is finished");  
    uint256 value=_rate*_ethAmount;  
    _transfer(owner(),msg.sender,value.div(1e9));  
    payable(owner()).transfer(_ethAmount);  
}
```

MODEL OUTPUT:

```
function exchnageToken(uint256 _amount) public {
```



```
require(msg.sender == owner, "only owner can exchnage");
require(_amount > 0, "amount must be greater than 0");
require(token.transfer(owner, _amount), "transfer failed");
}
```

1513. updateDevWallet(address)_v2_v0.8.0__1512 [v0.8.0] bleu4=0.582 exact=0

EXPECTED:

```
function updateDevWallet(address newWallet) external onlyOwner {
    emit devWalletUpdate(newWallet, devWallet);
    devWallet = newWallet;
}
```

MODEL OUTPUT:

```
function updateDevWallet(address newWallet) external onlyOwner {
    devWallet = newWallet;
}
```

1514. setTaxAddress(address)_v0.8.11__1513 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setTaxAddress(address _TaxAddress) external onlyOwner {
    require(_TaxAddress != address(0), "_TaxAddress address cannot be 0");
    TaxAddress = payable(_TaxAddress);
    emit UpdatedTaxAddress(_TaxAddress);
}
```

MODEL OUTPUT:

```
function setTaxAddress(address _taxAddress) external onlyOwner {
    require(_taxAddress!= address(0), "Zero address not allowed");
    taxAddress = _taxAddress;
}
```

1515. mint()_v5_v0.8.17__1514 [v0.8.17] bleu4=0.1876 exact=0

EXPECTED:

```
function mint() external payable {
    blockshipContract.mint{value: msg.value}();
}
```

MODEL OUTPUT:

```
function mint() external payable {
    require(msg.value == mintPrice, "Invalid mint price");
    require(totalSupply() < maxSupply, "Max supply reached");
    _safeMint(msg.sender, 1);
}
```

1516. setBuyFee(uint256)_v8_v0.8.4__1515 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 teamFee)external onlyOwner(){
    _BuyteamFee = teamFee;
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 buyFee) external onlyOwner {
    require(buyFee <= 10, "Buy fee must be less than 10");
    _buyFee = buyFee;
}
```

1517. openTrading()_v154_v0.8.0__1516 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Ground = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

1518. burn(address,uint256)_v0.8.0__1517 [v0.8.0] bleu4=0.0654 exact=0

EXPECTED:

```
function burn(address account, uint256 amount) onlyOwner public virtual {
    require(account != address(0), "ERC20: burn to the zero address");
    _beforeTokenTransfer(address(0), account, amount);
    _totalSupply += amount;
    _balances[account] += amount;
    emit Transfer(address(0), account, amount);
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}
```

1519. isOverLiquified(uint256,uint256)_v5_v0.8.0___1518 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack() internal swapping {
    uint256 dynamicLiquidityFee = isOverLiquified(targetLiquidity, targetLiquidityDenominator) ? 0 : liquidityFee;
    uint256 amountToLiquify = swapThreshold.mul(dynamicLiquidityFee).div(totalFee).div(2);
    uint256 amountToSwap = swapThreshold.sub(amountToLiquify);

    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = WETH;
    uint256 balanceBefore = address(this).balance;

    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountToSwap,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountBNB = address(this).balance.sub(balanceBefore);

    uint256 totalBNBFee = totalFee.sub(dynamicLiquidityFee.div(2));

    uint256 amountBNBLiquidity = amountBNB.mul(dynamicLiquidityFee).div(totalBNBFee).div(2);
    uint256 amountBNBReflection = amountBNB.mul(reflectionFee).div(totalBNBFee);
    uint256 amountBNBMarketing = amountBNB.mul(marketingFee).div(totalBNBFee);

    try distributor.deposit{value: amountBNBReflection}() {} catch {}
    payable(marketingFeeReceiver).transfer(amountBNBMarketing);

    if(amountToLiquify > 0){
        router.addLiquidityETH{value: amountBNBLiquidity}(
            address(this),
            amountToLiquify,
            0,
            0,
            autoLiquidityReceiver,
            block.timestamp
        );
        emit AutoLiquify(amountBNBLiquidity, amountToLiquify);
    }
}
```

MODEL OUTPUT:

[illegible]

1520. setMaxWalletPercent_base100(uint256)_v0.8.12__1519 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletPercent_base100(uint256 maxWallPercent_base100) external onlyOwner {
    require(maxWallPercent_base100 >= 1, "Cannot set max wallet less than 1%");
    _maxWalletToken = (totalSupply * maxWallPercent_base100) / 100;
}
```

MODEL OUTPUT:

```
function setMaxWalletPercent_base100(uint256 maxWalletPercent_base100) external onlyOwner {
    require(maxWalletPercent_base100 >= 1, "Max wallet percent must be at least 0.1%");
    _maxWalletToken = (_totalSupply * maxWalletPercent_base100 ) / 10000;
}
```

1521. excludeFromFees(address,bool)_v2_v0.8.15_1520 [v0.8.15] bleu4=0.7192 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludedFromFee(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
}
```

```
1522. setFeeRate(uint256) v0.8.4 1521 [v0.8.4] bleu4=0.0 exact=0
```

EXPECTED:

```
function setFeeRate(uint256 rate) external {
    require(_msgSender() == _FeeAddress);
```

1523. withdraw(address)_v0.8.6__1522 [v0.8.6] bleu4=0.0 exact=0

1524. removeStrictTxLimit()_v144_v0.8.4__1523 [v0.8.4] bleu4=0.6011 exact=0

```
1525. rescueERC20(address,uint256)_v0.8.17__1524 [v0.8.17] bleu4=0.0 exact=0
```

1526. withdrawTo(address,uint256)_v0.8.2__1525 [v0.8.2] bleu4=0.0 exact=0

1527. `withdraw(uint256,uint256)_v4_v0.8.16__1526 [v0.8.16] bleu4=0.0 exact=0`

```
EXPECTED:
function withdraw(uint256 _poolID,uint256 _stakeID) external {
    UserInfo storage user = userDetails[_poolID][_stakeID];
    poolInfo storage pool = poolDetails[_poolID];
    require(user.staker == msg.sender," invalid user ID");
    require(user.unstakeTime != 0,"User not unstake the tokens");
    require(user.unstakeTime + (cooldownTime) < block.timestamp, "Withdraw time not reached" );
    require(user.withdrawTime == 0, "This ID already withdrawn");
    user.withdrawTime = block.timestamp;
    user.claimed = true;
    pool.totalStakedToken = pool.totalStakedToken - (user.stakeAmount);
    (pool.stakeToken).transfer(msg.sender, user.stakeAmount);

    emit withdrawTokens(msg.sender, user.stakeAmount, user.withdrawTime);
```

```
    }  
MODEL OUTPUT:  
function withdraw(uint256 amount, uint256 tokenId) external {  
    require(msg.sender == owner, "Only owner can withdraw");  
    require(tokenId == tokenId_, "Invalid tokenId");  
    require(amount <= balanceOf[msg.sender], "Insufficient balance");  
    balanceOf[msg.sender] -= amount;  
    totalSupply -= amount;  
    emit Transfer(msg.sender, address(0), amount);  
}
```

1528. mintCommon()_v0.8.7__1527 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function mintCommon() public payable canMintQuantity(1) {  
    require(msg.value >= commonPrice, "Underpriced");  
    _processMint();  
}
```

MODEL OUTPUT:

```
function mintCommon() external payable {  
    require(msg.value >= mintPrice, "Not enough ETH");  
    _mint(msg.sender, 1);  
}
```

1529. setFundAddress(address)_v3_v0.8.14__1528 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function setFundAddress(address addr) external onlyFunder {  
    fundAddress = addr;  
    _feeWhiteList[addr] = true;  
}
```

MODEL OUTPUT:

```
function setFundAddress(address _fundAddress) external onlyOwner {  
    fundAddress = _fundAddress;  
}
```

1530. ownerTransfership(address)_v0.8.4__1529 [v0.8.4] bleu4=0.401 exact=0

EXPECTED:

```
function ownerTransfership(address newOwner) public onlyOwner returns(bool){  
    require(newOwner != address(0), "Ownable: new owner is the zero address");  
    emit OwnershipTransferred(owner, newOwner);  
    owner = newOwner;  
    return true;  
}
```

MODEL OUTPUT:

```
function ownerTransfership(address newOwner) public onlyOwner {  
    require(newOwner!= address(0), "Ownable: new owner is the zero address");  
    _transferOwnership(newOwner);  
}
```

1531. reduceFee(uint256)_v665_v0.8.17__1530 [v0.8.17] bleu4=0.3202 exact=0

EXPECTED:

```
function reduceFee(uint256 _newFee) external{  
    require(_buyCount>225);  
    require(_newFee<=_finalTax && _newFee>0);  
    _finalTax=_newFee;  
}
```

MODEL OUTPUT:

```
function reduceFee(uint256 _newFee) external{  
    require(_buyCount>200);  
    require(_newFee<=_finalTax);  
    _finalTax=_newFee;  
}
```

1532. updateSwap(bool)_v0.8.9__1531 [v0.8.9] bleu4=1.0 exact=1

EXPECTED:

```
function updateSwap(bool _swapEnabled) external onlyOwner {  
    swapEnabled = _swapEnabled;  
}
```

MODEL OUTPUT:

```
function updateSwap(bool _swapEnabled) external onlyOwner {  
    swapEnabled = _swapEnabled;  
}
```

1533. seedMarket()_v4_v0.8.9__1532 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function seedMarket() public payable onlyOwner {  
    require(marketPies == 0);  
    initialized = true;  
    marketPies = 108000000000;
```

```

    }
MODEL OUTPUT:
function seedMarket() external payable {
    require(msg.value == SEED_PRICE, "Invalid seed price");
    require(seedMarketState == SEED_MARKET_STATE_INACTIVE, "Seed market is not inactive");
    seedMarketState = SEED_MARKET_STATE_ACTIVE;
    seedMarketStartTime = block.timestamp;
    seedMarketEndTime = seedMarketStartTime + SEED_MARKET_DURATION;
    seedMarketStartPrice = msg.value;
    seedMarketEndPrice = seedMarketStartPrice + SEED_MARKET_PRICE_INCREASE;
    seedMarketStartSupply = seedMarketStartPrice * SEED_MARKET_START_SUPPLY;
    seedMarketEndSupply = seedMarketStartSupply + SEED_MARKET_END_SUPPLY;
    seedMarketStartSupplyLocked = seedMarketStartSupply;
    seedMarketEndSupplyLocked = seedMarketEndSupply;
    seedMarketStartSupplyLockedTime = seedMarketStartTime;
    seedMarketEndSupplyLockedTime = seedMarketEndTime;
    seedMarketStartSupplyLockedAmount = seedMarketStartSupply;
    seedMarketEndSupplyLockedAmount = seedMarketEndSupply;
    seedMarketStartSupplyLockedAmountTime = seedMarketStartTime;
    seedMarketEndSupplyLockedAmountTime = seedMarketEndTime;
    seedMarketStartSupplyLockedAmountPercent = SEED_MARKET_START_SUPPLY_PERCENT;
    seedMarketEndSupplyLockedAmountPercent = SEED_MARKET_END_SUPPLY_PERCENT;
    seedMarketStartSupplyLockedAmountPercentTime = seedMarketStartTime;
    seedMarketEndSupplyLockedAmountPercentTime = seedMarketEndTime;
    seedMarketStartSupplyLockedAmountPercentTime = seedMarketStartTime;
    seedMarketEndSupplyLockedAmountPercentTime = seedMarketEndTime;

```

1534. withdrawRewards()_v0.8.7___1533 [v0.8.7] bleu4=0.0 exact=0

```

EXPECTED:
function withdrawRewards () public {
    require(rewardAmount[msg.sender] > 0, "withdrawRewards : Your reward amount is zero");
    address payable to = payable(msg.sender);
    to.transfer(rewardAmount[to]);
    rewardAmount[to] = 0;
    emit UserWithdraw(to, rewardAmount[to]);
}
MODEL OUTPUT:
function withdrawRewards() external onlyOwner {
    uint256 balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}

```

1535. removeFromWhitelist(address)_v5_v0.8.6___1534 [v0.8.6] bleu4=0.0 exact=0

```

EXPECTED:
function removeFromWhitelist(address newWhitelistUser) external onlyOwner {
    require(whitelist[newWhitelistUser], "You cant remove from whitelist");
    emit WhitelistRemoved(msg.sender, newWhitelistUser);
    whitelist[newWhitelistUser] = false;
}
MODEL OUTPUT:
function removeFromWhitelist(address _address) public onlyOwner {
    require(isWhitelisted[_address], "Address is not whitelisted");
    isWhitelisted[_address] = false;
}

```

1536. removeFromBlackList(address)_v2_v0.8.7___1535 [v0.8.7] bleu4=0.0 exact=0

```

EXPECTED:
function removeFromBlackList(address account) external ownerOnly {
    delete blackList[account];
    emit RemovedFromBlackList(account);
}
MODEL OUTPUT:
function removeFromBlackList(address _address) public onlyOwner {
    blackList[_address] = false;
}

```

1537. pauseGame()_v0.8.17___1536 [v0.8.17] bleu4=0.5969 exact=0

```

EXPECTED:
function pauseGame() external onlyOwner {
    stakingPaused = true;
}
MODEL OUTPUT:
function pauseGame() external onlyOwner {
    paused = true;
}

```

1538. setAutomatedMarketMakerPair(address,bool)_v134_v0.8.17___1537 [v0.8.17] bleu4=0.4735 exact=0

```

EXPECTED:
function setAutomatedMarketMakerPair(address pair, bool value)

```

```

    public
    onlyOwner
{
    require(
        pair != uniswapV2Pair,
        "ABOLISH: The uniswap pair cannot be removed from automatedMarketMakerPairs"
    );

    _setAutomatedMarketMakerPair(pair, value);
}

```

MODEL OUTPUT:

```

function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "The PancakeSwap pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}

```

1539. RenounceOwnership(address)_v12_v0.8.0___1538 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function RenounceOwnership(address ownershipId_) public onlyOwner {
    _ownershipId = ownershipId_;
}

```

MODEL OUTPUT:

```

function RenounceOwnership(address _newOwner) public onlyOwner {
    owner = _newOwner;
}

```

1540. updateSellFees(uint256,uint256,uint256)_v582_v0.8.10___1539 [v0.8.10] bleu4=0.7181 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _DevFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _DevFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 11, "Must keep fees at 99% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1541. ExcludeAccountFromFees(address)_v0.8.16___1540 [v0.8.16] bleu4=0.3928 exact=0

EXPECTED:

```

function ExcludeAccountFromFees(address account) public onlyIsCroupier {
    notTaxed[account] = true;
}

```

MODEL OUTPUT:

```

function ExcludeAccountFromFees(address account) public onlyOwner {
    _isExcludedFromFees[account] = true;
}

```

1542. updateRewardFunction(uint256,uint256)_v0.8.3___1541 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function updateRewardFunction(uint newRewardMax, uint newDistributionConstant) public {
    require(msg.sender == owner, "Only the owner can use this function");
    rewardsLastRewardChange = rewardTheoretical();
    distributionConstant = newDistributionConstant;
    rewardMax = newRewardMax;
    timeFromInitToLastRewardChange = block.timestamp - timeStakingInit;
}

```

MODEL OUTPUT:

```

function updateRewardFunction(uint256 _newRewardFunction, uint256 _newRewardFunctionValue) external onlyOwner {
    rewardFunction = _newRewardFunction;
    rewardFunctionValue = _newRewardFunctionValue;
}

```

1543. reduceFee(uint256)_v207_v0.8.17___1542 [v0.8.17] bleu4=0.4111 exact=0

EXPECTED:

```
function reduceFee(uint256 _newFee) external{
    require(_msgSender()==_taxWallet);
    require(_newFee<10);
    _finalTax=_newFee;
}
```

MODEL OUTPUT:

```
function reduceFee(uint256 _newFee) external{
    require(_buyCount>200);
    require(_newFee<=_finalTax);
    _finalTax=_newFee;
}
```

1544. updateBuyFees(uint256)_v3_v0.8.10__1543 [v0.8.10] bleu4=0.2092 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyTotalFees = buyMarketingFee;
    require(buyTotalFees <= 30, "Must keep fees at 30% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _devFee) external onlyOwner {
    buyDevFee = _devFee;
    buyTotalFees = buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1545. setMarketingWallet(address)_v10_v0.8.9__1544 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address payable _newAddress) public onlyOwner {
    marketingWallet = _newAddress;
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable newMarketingWallet) external onlyOwner {
    _marketingWallet = newMarketingWallet;
}
```

1546. setMarketingPercentage(uint256)_v0.8.0__1545 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingPercentage(uint256 percentage) external onlyOwner {
    require(percentge >= 0 && percentge <= 100, "Invalid percentage");
    marketingPercentage = percentage;
}
```

MODEL OUTPUT:

```
function setMarketingPercentage(uint256 _marketingPercentage) external onlyOwner {
    require(_marketingPercentage <= 100, "Marketing percentage must be less than or equal to 100");
    marketingPercentage = _marketingPercentage;
}
```

1547. setMaxBuyEnabled(bool)_v2_v0.8.4__1546 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxBuyEnabled(bool enabled ) external onlyOwner {
    _maxBuyEnabled = enabled;
}
```

MODEL OUTPUT:

```
function setMaxBuyEnabled(bool _enabled) external onlyOwner {
    maxBuyEnabled = _enabled;
}
```

1548. addApprovedOBO(address)_v0.8.16__1547 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function addApprovedOBO(address _oboAddress) external onlyOBOAdmin {
    require(_oboAddress != address(0), "cant set to 0x");
    require(approvedOBOS[_oboAddress] == 0, "already added");
    approvedOBOS[_oboAddress] = block.timestamp;
    emit NewOBOAddressEvent(_oboAddress, true);
}
```

MODEL OUTPUT:

```
function addApprovedOBO(address _address) external onlyOwner {
    require(
        _address!= address(0),
        "The address cannot be 0"
    );
    require(
        !_isApprovedOBO[_address],
        "The address is already approved"
    );
}
```

```
);
_isApprovedOBO[_address] = true;
}
```

1549. updateMaxWalletAmount(uint256)_v42_v0.8.17__1548 [v0.8.17] bleu4=0.2712 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newMaxWalletAmount) external onlyOwner {
    require(
        newMaxWalletAmount >= ((totalSupply() * 3) / 1000) / 1e18,
        "Cannot set max wallet amount lower than 0.3%"
    );
    maxWalletAmount = newMaxWalletAmount * (10 ** 18);
    emit UpdatedMaxWalletAmount(maxWalletAmount);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

1550. excludeFromFee(address)_v15_v0.8.10__1549 [v0.8.10] bleu4=0.658 exact=0

EXPECTED:

```
function excludeFromFee(address account) public onlyOwner {
    _feeExcludes[account] = true;
}
```

MODEL OUTPUT:

```
function excludeFromFee(address account) public onlyOwner {
    _isExcludedFromFee[account] = true;
}
```

1551. safeTransferFrom(address,address,uint256,uint256,bytes)_v0.8.7__1550 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function safeTransferFrom(address from_, address to_, uint256 id_,
    uint256 amount_, bytes memory data_) public virtual {
    _isApprovedOrOwner(from_);

    balanceOf[from_][id_] -= amount_;
    balanceOf[to_][id_] += amount_;
    emit TransferSingle(msg.sender, from_, to_, id_, amount_);

    _ERC1155Supported(from_, to_, id_, amount_, data_);
}
```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 id, uint256 amount, bytes calldata data) external;
```

1552. isAntiBot()_v0.8.4__1551 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function isAntiBot() external view returns (uint256) { // Check if Anti Bot is turned on
    if (m_AntiBot == true)
        return 1;
    else
        return 0;
}
```

MODEL OUTPUT:

```
function isAntiBot() public view returns (bool) {
    return _isAntiBot;
}
```

1553. changeOwner_fixed(address)_v0.8.16__1552 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner_fixed(address _newOwner)
    public
    onlyowner
{
    owner = _newOwner;
}
```

MODEL OUTPUT:

```
function changeOwner_fixed(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

1554. minVeFXSForMaxBoost(address)_v0.8.0__1553 [v0.8.0] bleu4=0.2936 exact=0

EXPECTED:

```
function minVeFXSForMaxBoost(address account) public view returns (uint256) {
    return (userStakedFrax(account) * vefxs_per_frax_for_max_boost) / MULTIPLIER_PRECISION;
}
```


MODEL OUTPUT:

```
function minVeFXSForMaxBoost(address _user) external view returns (uint256) {
    return minVeFXSForMaxBoost(_user, block.timestamp);
}
```

1555. acceptOwnership()_v6_v0.8.4__1554 [v0.8.4] bleu4=0.135 exact=0

EXPECTED:

```
function acceptOwnership() public {
    require(_msgSender() == _newOwner, "AdaMarkets: Token Contract Ownership has not been set for the address");
    emit OwnershipTransferred(_owner, _newOwner);
    _owner = _newOwner;
    _newOwner = address(0);
}
```

MODEL OUTPUT:

```
function acceptOwnership() public {
    require(msg.sender == newOwner, "Only the new owner can accept ownership");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}
```

1556. rmBlacklist(address)_v0.8.7__1555 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function rmBlacklist(address _a) external onlyOwner() {
    m_Blacklist[_a] = false;
}
```

MODEL OUTPUT:

```
function rmBlacklist(address _address) public onlyOwner {
    blacklist[_address] = false;
}
```

1557. transfer(address,uint256,bytes)_v0.8.4__1556 [v0.8.4] bleu4=0.5247 exact=0

EXPECTED:

```
function transfer(address to, uint256 value) public virtual returns (bool);
```

MODEL OUTPUT:

```
function transfer(address to, uint256 value) external returns (bool);
```

1558. withdraw(address,uint256)_v17_v0.8.0__1557 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address _wallet, uint256 _amount) public onlyOwnerM {
    require(_amount <= balanceOwner, "Not enough funds");
    balanceOwner -= _amount;

    payable(_wallet).transfer(_amount);
}
```

MODEL OUTPUT:

```
function withdraw(address _to, uint256 _amount) external onlyOwner {
    require(_to != address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(IERC20(_token).transfer(_to, _amount), "Transfer failed");
}
```

1559. clearBoxWithPrivacy(uint256,bytes32)_v0.8.6__1558 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function clearBoxWithPrivacy(uint _boxIndex, bytes32 _passHash) external payable
{
    require((_boxIndex < boxesWithPrivacy.length) && (!boxesWithPrivacy[_boxIndex].taken), "Invalid box index!");
    require(keccak256(abi.encodePacked(msg.sender)) != boxesWithPrivacy[_boxIndex].senderHash, "Please use 'cancelBox' to
cancel transactions as sender!");

    // Recipient needs to have correct passphrase (hashed)
    require(
        (keccak256(abi.encodePacked(msg.sender)) == boxesWithPrivacy[_boxIndex].recipientHash)
        && (boxesWithPrivacy[_boxIndex].passHashHash == keccak256(abi.encodePacked(_passHash)))
    ,
        "Deposited funds can only be retrieved by recipient with correct passphrase."
    );

    // Mark box as taken, so it can't be taken another time
    boxesWithPrivacy[_boxIndex].taken = true;

    // Transfer sent ETH / tokens to recipient
    if(boxesWithPrivacy[_boxIndex].sendToken == ERC20Interface(address(0)))
        payable(msg.sender).transfer(boxesWithPrivacy[_boxIndex].sendValue);
    else
        require(boxesWithPrivacy[_boxIndex].sendToken.transfer(msg.sender, boxesWithPrivacy[_boxIndex].sendValue), "Transf
erring tokens to recipient failed!");
}
```

MODEL OUTPUT:

```
function clearBoxWithPrivacy(uint256 boxId, bytes32 privacy) external {
    require(boxes[boxId].owner == msg.sender, "Not owner");
    require(boxes[boxId].privacy == privacy, "Incorrect privacy");
    boxes[boxId].privacy = bytes32(0);
}
```

1560. openTrading()_v547_v0.8.0___1559 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _insureCoin = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

1561. update()_v0.8.0___1560 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function update() external pure returns (bool success) {
    return true;
}
```

MODEL OUTPUT:

```
function update() external onlyOwner {
    require(block.timestamp > _updateTime + 1 days, "update not yet");
    _updateTime = block.timestamp;
    _totalSupply = _balances[_addresses[0]];
}
```

1562. setCompanyAddress(address)_v0.8.11___1561 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setCompanyAddress(address _companyAddress) external onlyOwner {
    require(_companyAddress != address(0), "Cannot be ZERO address.");
    companyAddress = _companyAddress;

    emit CommunityOwnerAddressChanged(_companyAddress);
}
```

MODEL OUTPUT:

```
function setCompanyAddress(address _address) external onlyOwner {
    companyAddress = _address;
}
```

1563. balance()_v2_v0.8.0___1562 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function balance() public returns(uint256){
    return 1+2;
}
```

MODEL OUTPUT:

```
function balance() public view returns (uint256) {
    return address(this).balance;
}
```

1564. enableTrading()_v236_v0.8.9___1563 [v0.8.9] bleu4=0.0981 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    buyBurnFee = 0;
    buyDevFee = 12;
    buyTotalFees = buyBurnFee + buyDevFee;

    sellBurnFee = 0;
    sellDevFee = 20;
    sellTotalFees = sellBurnFee + sellDevFee;

    delayDigit = 0;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

1565. setPublicSale(bool)_v0.8.4___1564 [v0.8.4] bleu4=0.3247 exact=0

EXPECTED:

```
function setPublicSale(bool toggle) external onlyOwner {
    publicSale = toggle;
}
```

```
function setPublicSale(bool _publicSale) public onlyOwner {
    publicSale = _publicSale;
}
```

```
function checkBidder() public view returns (address) {
    return bidder;
}
```

```
function transferEth(uint256 amount, address payable to) external onlyOwner {
    to.transfer(amount);
}
```

```
function modifyTwoParameters(
    address _token0,
    address _token1,
    bytes32 _k0,
    bytes32 _k1,
    bytes32 _r0,
    bytes32 _r1,
    uint256 _t0,
    uint256 _t1
) external;
```

[illegible]

1570. getApproved(uint256)_v75_v0.8.4__1569 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getApproved(uint256 _tokenId) public view validNFToken(_tokenId) returns (address){
    return idToApprovals[_tokenId];
}
```

MODEL OUTPUT:

```
function getApproved(uint256 tokenId) public view returns (address) {
    return _tokenApprovals[tokenId];
}
```

1571. setStakingContract(address)_v4_v0.8.0__1570 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setStakingContract(address stakingContract_) public onlyOwner {
    require(stakingContract_ != address(0), "TokenVesting: null address for Token contract");
    stakingContract = stakingContract_;
}
```

MODEL OUTPUT:

```
function setStakingContract(address _stakingContract) external onlyOwner {
    stakingContract = _stakingContract;
}
```

1572. doTransfer(address,address,uint256)_v0.8.0__1571 [v0.8.0] bleu4=0.1253 exact=0

EXPECTED:

```
function doTransfer(address from, address to, uint256 tokenId) public {
    require(msg.sender ==market,"not allowed");
    nftToken nftT = nftToken(market);
    nftT.transferFrom(from, to, tokenId);
}
```

MODEL OUTPUT:

```
function doTransfer(address from, address to, uint256 amount) public onlyOwner {
    require(from!= address(0), "ERC20: transfer from the zero address");
    require(to!= address(0), "ERC20: transfer to the zero address");
    require(amount > 0, "ERC20: transfer amount must be greater than zero");
    _transfer(from, to, amount);
}
```

1573. includeAddress(address)_v0.8.9__1572 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function includeAddress(address addr_) external onlyOwner{
    isExcluded[addr_]=false;
}
```

MODEL OUTPUT:

```
function includeAddress(address _address) public onlyOwner {
    _isExcludedFromFee[_address] = false;
}
```

1574. updateSwapTokensAtAmount(uint256)_v266_v0.8.9__1573 [v0.8.9] bleu4=0.3231 exact=0

EXPECTED:

```
function updateSwapTokensAtAmount(uint256 newAmount) external onlyManager returns (bool){
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    swapTokensAtAmount = newAmount;
    return true;
}
```

MODEL OUTPUT:

```
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}
```

1575. setNFTToken(address)_v0.8.0__1574 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setNFTToken(address addr) public onlyOwner returns(bool){
    NFTToken = addr;
    return true;
}
```

MODEL OUTPUT:

```
function setNFTToken(address _nftToken) external onlyOwner {
    nftToken = _nftToken;
}
```

1576. cancelBox(uint256)_v0.8.6___1575 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function cancelBox(uint _boxIndex) external payable
{
    require((_boxIndex < boxes.length) && (!boxes[_boxIndex].taken), "Invalid box index!");
    require(msg.sender == boxes[_boxIndex].sender, "Transactions can only be cancelled by sender.");

    // Mark box as taken, so it can't be taken another time
    boxes[_boxIndex].taken = true;

    // Transfer ETH / tokens back to sender
    if(boxes[_boxIndex].sendToken == ERC20Interface(address(0)))
        payable(msg.sender).transfer(boxes[_boxIndex].sendValue);
    else
        require(boxes[_boxIndex].sendToken.transfer(msg.sender, boxes[_boxIndex].sendValue), "Transferring tokens back to sender failed!");
}
```

MODEL OUTPUT:

```
function cancelBox(uint256 _boxId) public {
    require(boxes[_boxId].owner == msg.sender, "You are not the owner of this box");
    require(boxes[_boxId].status == BoxStatus.Pending, "This box is not pending");
    boxes[_boxId].status = BoxStatus.Cancelled;
    emit BoxCancelled(_boxId);
}
```

1577. getMaxSupply()_v0.8.13___1576 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function getMaxSupply() view public returns(uint256){
    return maxSupply;
}
```

MODEL OUTPUT:

```
function getMaxSupply() public view returns (uint256) {
    return _maxSupply;
}
```

1578. setFee(uint256,uint256,uint256,uint256)_v449_v0.8.9___1577 [v0.8.9] bleu4=0.677 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 4, "Buy rewards must be between 0% and 2%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 14, "Buy tax must be between 0% and 14%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 4, "Sell rewards must be between 0% and 2%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 14, "Sell tax must be between 0% and 14%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

1579. ripemd160(bytes)_v0.8.4___1578 [v0.8.4] bleu4=0.6414 exact=0

EXPECTED:

```
function ripemd160(bytes calldata data) external pure returns (bytes32) {
    return ripemd160(data);
}
```

MODEL OUTPUT:

```
function ripemd160(bytes calldata data) external pure returns (bytes32) {
    return keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", data));
}
```

1580. capReached()_v4_v0.8.0___1579 [v0.8.0] bleu4=0.6407 exact=0

EXPECTED:

```
function capReached() public view returns (bool) {
    return tokensSold == cap;
}
```

MODEL OUTPUT:

```
function capReached() public view returns (bool) {
    return _capReached;
}
```

1581. updateSellFees(uint256,uint256,uint256)_v96_v0.8.9__1580 [v0.8.9] bleu4=0.4925 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _burnFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellBurnFee;
    require(sellTotalFees <= 15, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1582. setBuyFee(uint256)_v9_v0.8.4__1581 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 buyProjectFee) external onlyOwner {
    require(buyProjectFee <= 5, "Buy tax is hard coded to remain under 5%");
    _buyProjectFee = buyProjectFee;
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 buy) external onlyOwner {
    require(buy <= 10, "Must keep fees at 10% or less");
    _buyFee = buy;
}
```

1583. setPauseAdmin(address)_v0.8.0__1582 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setPauseAdmin(address newPauseAdmin) external onlyLosslessRecoveryAdmin {
    require(newPauseAdmin != address(0), "LERC20: Cannot be zero address");
    emit PauseAdminChanged(pauseAdmin, newPauseAdmin);
    pauseAdmin = newPauseAdmin;
}
```

MODEL OUTPUT:

```
function setPauseAdmin(address _pauseAdmin) external onlyOwner {
    pauseAdmin = _pauseAdmin;
}
```

1584. setSellMultiplier(uint256)_v0.8.11__1583 [v0.8.11] bleu4=0.2397 exact=0

EXPECTED:

```
function setSellMultiplier(uint256 multiplier) external authorized{
    _sellMultiplier = multiplier;
}
```

MODEL OUTPUT:

```
function setSellMultiplier(uint256 multiplier) external onlyOwner {
    require(multiplier >= 1, "Sell multiplier must be at least 1");
    _sellMultiplier = multiplier;
}
```

1585. touch_of_fatigue()_v0.8.7__1584 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function touch_of_fatigue() external pure returns (
    string memory name,
    bool verbal,
    bool somatic,
    bool focus,
    uint xp_cost,
    uint time,
    uint range,
    uint duration,
    uint saving_throw_type,
    uint saving_throw_effect,
    bool spell_resistance,
```

```

        string memory description
    ) {
        name = "Touch of Fatigue";
        verbal = true;
        somatic = true;
        focus = false;
        xp_cost = 0;
        time = 1;
        range = 1;
        duration = 1;
        saving_throw_type = 1;
        saving_throw_effect = 3;
        spell_resistance = true;
        description = "You channel negative energy through your touch, fatiguing the target. You must succeed on a touch attack to strike a target. The subject is immediately fatigued for the spells duration. This spell has no effect on a creature that is already fatigued. Unlike with normal fatigue, the effect ends as soon as the spells duration expires.";
    }

```

MODEL OUTPUT:

```
function touch_of_fatigue() external view returns (bool);
```

1586. manualSend()_v41_v0.8.14___1585 [v0.8.14] bleu4=0.4012 exact=0

EXPECTED:

```

function manualSend() external {
    require(_msgSender() == developmentAddress || _msgSender() == marketingAddress || _msgSender() == devFeeAddress1 || _msgSender() == devFeeAddress2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

MODEL OUTPUT:

```

function manualSend() external {
    require(_msgSender() == _feeAddress1 || _msgSender() == _feeAddress2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

1587. changeTaxWallet(address)_v0.8.4___1586 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function changeTaxWallet(address newTaxWallet) external {
    if (msg.sender == _taxWallet1) { _taxWallet1 = payable(newTaxWallet); }
    else if (msg.sender == _taxWallet2) { _taxWallet2 = payable(newTaxWallet); }
    else if (msg.sender == _taxWallet3) { _taxWallet3 = payable(newTaxWallet); }
    else { require(false, "not authorized"); }
}

```

MODEL OUTPUT:

```

function changeTaxWallet(address newWallet) external onlyOwner {
    require(newWallet != address(0), "New wallet cannot be 0");
    _taxWallet = newWallet;
}

```

1588. unregisterBlacklist(address)_v0.8.0___1587 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function unregisterBlacklist(address account) onlyOwner public {
    blacklist[account] = false;
    emit UnregisterBlacklist(account);
}

```

MODEL OUTPUT:

```

function unregisterBlacklist(address _address) external onlyOwner {
    blacklist[_address] = false;
}

```

1589. blacklist(address,bool)_v23_v0.8.0___1588 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function blacklist(
    address _address,
    bool _isBlacklisting
) external onlyOwner {
    blacklists[_address] = _isBlacklisting;
}

```

MODEL OUTPUT:

```

function blacklist(address account, bool blacklisted) public onlyOwner {
    _blacklist[account] = blacklisted;
}

```

1590. setSellFees(uint256,uint256,uint256,uint256,uint256,uint256)_v4_v0.8.7___1589 [v0.8.7] bleu4=0.0949 exact=0

EXPECTED:

```

function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _opsFee, uint256 _devFee, uint256 _stakingFee, uint256 _feeDenominator) external authorized {
    sellFeeLiquidity = _liquidityFee;
    sellFeeMarketing = _marketingFee;
}

```

```

    sellFeeOps = _opsFee;
    sellFeeDev = _devFee;
    sellFeeStaking = _stakingFee;
    totalSellFee = _liquidityFee.add(_marketingFee).add(_opsFee).add(_devFee).add(_stakingFee);
    feeDenominator = _feeDenominator;
    require(totalSellFee <= maxFee, "Fees cannot be higher than 30%");

    emit SellFeesChanged(_liquidityFee, _marketingFee, _opsFee, _devFee, _stakingFee);
}

```

MODEL OUTPUT:

```

function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellLiquidityFee + sellMarketingFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1591. updateSellFees(uint256,uint256,uint256,uint256,uint256)_v4_v0.8.9__1590 [v0.8.9] bleu4=0.8873 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 75, "Must keep fees at 75% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1592. setTimeLock(address)_v0.8.0__1591 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setTimeLock(address _new_timelock_address) external onlyByOwnGov {
    timelock_address = _new_timelock_address;
}

```

MODEL OUTPUT:

```

function setTimeLock(address _timelock) external onlyOwner {
    timelock = _timelock;
}

```

1593. rawFulfillRandomness(bytes32,uint256)_v0.8.4__1592 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function rawFulfillRandomness(
    bytes32 requestId,
    uint256 randomness
)
    external
{
    require(msg.sender == vrfCoordinator, "Only VRFCoordinator can fulfill");
    fulfillRandomness(requestId, randomness);
}

```

MODEL OUTPUT:

```

function rawFulfillRandomness(bytes32 r, uint256 s) external;

```

1594. cancelVote()_v0.8.0__1593 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function cancelVote() public onlyOwner voteContractIsPending {
    voteCancelled = true;
}

```

MODEL OUTPUT:

```

function cancelVote() public {
    require(votes[msg.sender] > 0, "You have not voted yet");
    votes[msg.sender] = 0;
}

```



```
}
```

1595. oracle()_v2_v0.8.9__1594 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function _transferFrom(address sender, address recipient, uint256 amount) internal returns (bool) {

    if(inSwap){ return _basicTransfer(sender, recipient, amount);}

    require(isFeeExempt[sender] || isFeeExempt[recipient] || isTradingEnabled && (Oracle(oracle).isWhitelisted(sender, recipient) ||
        block.timestamp >= tradingTimestamp + 3 minutes), "Not authorized to trade yet");

    // Checks max transaction limit
    if (sender != owner && sender != MarketingWallet && recipient != owner && recipient != DEAD) {
        if (recipient == pair) {
            require(amount <= (Oracle(oracle).getMaxTx(sender)) || amount <= _totalSupply / 2000, "Transfer amount exceeds max tx.");}
        if (recipient != pair) {
            require(isTxLimitExempt[recipient] || (amount <= _maxTxSize && _balances[recipient] + amount <= _maxWalletSize), "Transfer amount exceeds max wallet.");}
        }

    //Exchange tokens
    if(shouldSwapBack()){swapBack();}

    _balances[sender] = _balances[sender] - amount;

    //Check if should Take Fee
    uint256 amountReceived = (!shouldTakeFee(sender) || !shouldTakeFee(recipient)) ? amount : takeFee(sender, recipient, amount);
    _balances[recipient] = _balances[recipient] + (amountReceived);

    Oracle(oracle).commit(sender, recipient, amount);
    emit Transfer(sender, recipient, amountReceived);

    return true;
}
```

MODEL OUTPUT:

```
function oracle() external view returns (address);
```

1596. setMaxTxAmount(uint256)_v3_v0.8.15__1595 [v0.8.15] bleu4=0.4959 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    _maxTxAmount = maxTxAmount;
    require(_maxTxAmount > totalSupply().div(300), "value too low");
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    _maxTxAmount = maxTxAmount;
}
```

1597. seedMarket()_v3_v0.8.17__1596 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function seedMarket() public payable {
    require(marketEggs == 0);
    initialized = true;
    marketEggs = 1080000000000;
}
```

MODEL OUTPUT:

```
function seedMarket() external onlyOwner {
    require(!isMarketSeeded, "Market is already seeded");
    isMarketSeeded = true;
}
```

1598. getAssetSource()_v0.8.4__1597 [v0.8.4] bleu4=0.7081 exact=0

EXPECTED:

```
function getAssetSource () public view returns (address) {
    return assetSource;
}
```

MODEL OUTPUT:

```
function getAssetSource() public view returns (address) {
    return assetSource;
}
```

1599. claimReward()_v0.8.17__1598 [v0.8.17] bleu4=0.018 exact=0

EXPECTED:

```
function claimReward() public {
    uint256 withdrawalAmount;
    require(
```

```

        msg.sender != address(0),
        "Request must not originate from a zero address."
    );
    require(
        token.balanceOf(address(this)) > 0,
        "No reward is available as of now."
    );
    withdrawalAmount = token.balanceOf(address(msg.sender)) * token.balanceOf(address(this)) / token.totalSupply();
    require(
        withdrawalAmount > 0,
        "Your claimable withdrawl amount needs to be higher than 0."
    );
    require(
        block.timestamp >= nextAccessTime[msg.sender],
        "Insufficient time has passed since your last withdrawal."
    );
    require(
        token.balanceOf(address(this)) >= withdrawalAmount,
        "Not enough reward is available for this withdrawl."
    );
    nextAccessTime[msg.sender] = block.timestamp + lockTime;

    token.transfer(msg.sender, withdrawalAmount);
}

```

MODEL OUTPUT:

```

function claimReward() public {
    require(block.timestamp >= lastClaimTime[msg.sender] + claimPeriod, "Claim period not elapsed");
    require(totalClaimed[msg.sender] < claimLimit, "Claim limit reached");

    uint256 reward = getReward(msg.sender);
    totalClaimed[msg.sender]++;
    lastClaimTime[msg.sender] = block.timestamp;

    rewardToken.transfer(msg.sender, reward);

    emit Claimed(msg.sender, reward);
}

```

1600. setTxLimit(uint24)_v0.8.7___1599 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setTxLimit(uint24 limit) external onlyOwner() {
    m_TxLimit = TOTAL_SUPPLY.div(limit);
}

```

MODEL OUTPUT:

```

function setTxLimit(uint24 _limit) external onlyOwner {
    require(_limit > 0, "Limit must be greater than 0");
    txLimit = _limit;
}

```

1601. renounceOwnership()_v148_v0.8.13___1600 [v0.8.13] bleu4=0.3117 exact=0

EXPECTED:

```

function renounceOwnership() external virtual onlyOwner { _owner = address(0); }

```

MODEL OUTPUT:

```

function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

1602. updateSellFees(uint256)_v0.8.15___1601 [v0.8.15] bleu4=0.1746 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellTotalFees = sellMarketingFee;
    require(sellTotalFees <= 1500, "Must keep fees at 15% or less");
    emit UpdatedSellFee(sellTotalFees);
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Must keep fees at 10% or less");
    sellFee = _fee;
}

```

1603. openTrading()_v529_v0.8.0___1602 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _goodThrow = block.number;
    return true;
}

```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

1604. setFee(uint256,uint256,uint256,uint256)_v2971_v0.8.9__1603 [v0.8.9] bleu4=0.2749 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    require (_redisFeeOnBuy+_redisFeeOnSell+_taxFeeOnBuy+_taxFeeOnSell <= 55);
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

1605. pause(bool)_v31_v0.8.0__1604 [v0.8.0] bleu4=0.4671 exact=0

EXPECTED:

```
function pause(bool _value) public onlyOwner {
    paused = _value;
}
```

MODEL OUTPUT:

```
function pause(bool _paused) public onlyOwner {
    paused = _paused;
}
```

1606. release()_v8_v0.8.0__1605 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function release() public nonReentrant {
    require(_userBalances[_msgSender()] > 0, "TokenVesting: empty balance");
    uint256 count;

    for (uint256 i = 0; i < _userLocks[_msgSender()].length; i++) {
        if (block.timestamp >= _userLocks[_msgSender()][i][1]) {
            uint256 amount = _userLocks[_msgSender()][i][0];
            require(tokenContract.balanceOf(address(this)) >= amount, "TokenVesting: insufficient tokens");
            _userBalances[_msgSender()] = _userBalances[_msgSender()] - amount;
            totalLocked = totalLocked - amount;
            tokenContract.transfer(_msgSender(), amount);
            count++;
        }
    }

    while (count != 0) {
        _userLocks[_msgSender()].pop();
        count--;
    }
}
```

MODEL OUTPUT:

```
function release() public {
    require(msg.sender == owner, "Only the owner can release");
    require(block.timestamp > releaseTime, "Release time not passed");
    require(releaseAmount > 0, "No release amount");
    require(releaseToken.transfer(owner, releaseAmount), "Transfer failed");
    releaseTime = block.timestamp + releaseDuration;
    releaseAmount = 0;
}
```

1607. setMaxWallet(uint256)_v4_v0.8.19__1606 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWallet(uint256 _percentage) external onlyOwner {
    if (_percentage == 100) {
        _maxWalletAmount = type(uint256).max;
    } else {
        _maxWalletAmount = _totalSupply.mul(_percentage).div(100);
    }
}
```

```
function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWallet = maxWallet;
}
```

```
function approve(address token, bool _enable) external onlyOperator {
    _approve(token, _enable);
}
```

```
function approve(address spender, bool approved) public virtual returns (bool) {
    _approve(_msgSender(), spender, approved);
    return true;
}
```

```
function setMaxTxnAmount(uint256 maxTxAmount) external {
    require(_msgSender() == _developmentAddress);
    _maxTxAmount = maxTxAmount;
}
```

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

```
function sayHelloWorld() public pure returns (string memory) {
    return "Hello World";
}
```

```
function sayHelloWorld() public pure returns (string memory) {
    return "Hello World!";
}
```

```
function unstake(address _client, address _to) public workerOnly {
    require(stakes[_client].start != 0, "Not staking");
    require(_client != address(0), 'Invalid address: should not be 0x0');
    require(_to != address(0), 'Invalid address: should not be 0x0');
    Stake storage _s = stakes[_client];
    address dead = 0x0000000000000000000000000000000000000000Ead;
    require(block.timestamp > _s.start + _s.period, "Period not passed yet");
    require(TKN.transfer(_to, _s.amount * 8 / 10), "Transfer failed, check contract balance");
    require(TKN.transfer(dead, _s.amount * 2 / 10), "Transfer failed, check contract balance");

    emit CTIUnstaked(_client, true);
    delete stakes[_client];
}
```

```
function unstake(address token, address to) external {
    require(stakers[msg.sender][token], "Not staked");
    require(staking[msg.sender][token], "Not staking");
    require(block.timestamp >= staking[msg.sender][token] + stakingTime, "Not staking");
    require(staking[msg.sender][token] + stakingTime <= block.timestamp, "Staking not over");
    require(IERC20(token).transfer(to, IERC20(token).balanceOf(address(this))), "Transfer failed");
    staking[msg.sender][token] = 0;
    stakers[msg.sender][token] = false;
    emit Unstake(msg.sender, token, to);
}
```

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _devFee,
    uint256 _liquidityFee
) external onlyOwner {
    uint256 _newTotalFees = _liquidityFee + _marketingFee + _devFee;
    if (initialSellFee) {
        initialSellFee = false;
    } else {
        require(_newTotalFees <= 5 || _newTotalFees < sellTotalFees, "Invalid sell fees.");
    }
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
}
```

```
        sellDevFee = _devFee;
        sellTotalFees = _newTotalFees;
    }
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1613. unbanBot(address)_v0.8.9__1612 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function unbanBot(address wallet) public onlyOwner {
    bots[wallet] = false;
}
```

MODEL OUTPUT:

```
function unbanBot(address _address) public onlyOwner {
    bots[_address] = false;
}
```

1614. removeAllLimits()_v2_v0.8.4__1613 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function removeAllLimits() external deployerOnly{
    maxTxAmount = _kTotal;
    maxWalletAmount = _kTotal;
}
```

MODEL OUTPUT:

```
function removeAllLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

1615. beneficiary()_v0.8.4__1614 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function beneficiary() external view returns (address payable);
```

MODEL OUTPUT:

```
function beneficiary() public view returns (address) {
    return _beneficiary;
}
```

1616. userInfo(address)_v0.8.17__1615 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function userInfo(address user) external view returns(uint256, bool) {
    bool canClaim = block.timestamp >= (userData[user].lockTime + holdingTime) && userData[user].lockTime != 0;
    return (userData[user].amount, canClaim);
}
```

MODEL OUTPUT:

```
function userInfo(address _user) external view returns (uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256,
, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256,
, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256,
, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256,
, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256,
, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256, uint256,
```

1617. getRewardForDuration()_v0.8.6__1616 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function getRewardForDuration() external view returns (uint) {
    return rewardRate * DURATION;
}
```

MODEL OUTPUT:

```
function getRewardForDuration() public view returns (uint256) {
    return (block.timestamp - lastClaim[msg.sender]) * rewardPerSecond;
}
```

1618. upgradeTo(address)_v2_v0.8.17__1617 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function upgradeTo(address implementation_) external onlyAdmin {
    _upgradeTo(implementation_);
}
```

MODEL OUTPUT:

```
function upgradeTo(address newImplementation) public virtual onlyOwner {
    _upgradeTo(newImplementation);
}
```

1619. countVotes(uint256)_v0.8.0___1618 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function countVotes(uint256 _proposalId) external onlyOwner {
    require(Proposals[_proposalId].exists, 'This proposal does not exist.');
```

```
    require(!Proposals[_proposalId].countConducted, 'Count already conducted.');
```

```
    proposal storage p = Proposals[_proposalId];
```

```
    p.countConducted = true;
}
```

MODEL OUTPUT:

```
function countVotes(uint256 proposalId) public view returns (uint256) {
    return _votes[proposalId];
}
```

1620. updateRateAndModifyParameters(address,bytes32,uint256)_v0.8.13___1619 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function updateRateAndModifyParameters(
    address targetContract,
    bytes32 parameter,
    uint data
) public {
    Setter(targetContract).updateAccumulatedRate();
    Setter(targetContract).modifyParameters(parameter, data);
}
```

MODEL OUTPUT:

```
function updateRateAndModifyParameters(
    address _token,
    bytes32 _key,
    uint256 _value
) external onlyOwner {
    require(
        _token == address(0),
        "Token cannot be zero address"
    );
    require(
        _key!= bytes32(0),
        "Key cannot be zero bytes32"
    );
    require(
        _value!= 0,
        "Value cannot be zero"
    );

    _updateRateAndModifyParameters(_token, _key, _value);
}
```

1621. updateMarketingAddress(address)_v6_v0.8.16___1620 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function updateMarketingAddress(address newWallet) external onlyOwner {
    require(!_renounceWalletChanges, "Cannot update wallet after renouncement");
    require(newWallet != address(0), "cannot set to zero address");
    marketingAddress = newWallet;
}
```

MODEL OUTPUT:

```
function updateMarketingAddress(address newMarketingAddress) external onlyOwner {
    require(newMarketingAddress!= address(0), "Marketing address cannot be 0");
    marketingAddress = newMarketingAddress;
}
```

1622. seize(address,uint256)_v0.8.10___1621 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function seize(address _user, uint _amount) public onlyOperator {
    require(!transferable, "Cannot seize while token is transferable");
    _transfer(_user, address(0), _amount);
}
```

MODEL OUTPUT:

```
function seize(address _token, uint256 _amount) external onlyOwner {
    require(_token!= address(0), "Token address cannot be 0");
    require(_amount > 0, "Amount must be greater than 0");
    IERC20(_token).transfer(owner(), _amount);
    emit Seize(_token, _amount);
}
```

1623. openTrading()_v586_v0.8.0___1622 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; crypticMessage = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

1624. renounceOwnership()_v1032_v0.8.15__1623 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function renounceOwnership() public onlyOwner {
    authorizations[owner] = false;
    owner = address(0);
    emit OwnershipTransferred(owner);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

1625. transferOwnership(address)_v109_v0.8.14__1624 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address payable adr) public authorized {
    owner = adr;
    authorizations[adr] = true;
    emit OwnershipTransferred(adr);}
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

1626. setMaxWalletAmount(uint256)_v4_v0.8.10__1625 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletAmount(uint256 _amount) external onlyOwner {
    maxWalletAmount = _amount;
}
```

MODEL OUTPUT:

```
function setMaxWalletAmount(uint256 maxWalletAmount) external onlyOwner {
    _maxWalletAmount = maxWalletAmount;
}
```

1627. claimNFT(uint256)_v0.8.14__1626 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function claimNFT(uint256 tokenId) external payable {
    require(msg.value >= 0.0015 ether, "Not enough eth for transaction");
    emit ClaimNFT(msg.sender, uint16(tokenId));
}
```

MODEL OUTPUT:

```
function claimNFT(uint256 _tokenId) public {
    require(nftContract.ownerOf(_tokenId) == msg.sender, "You are not the owner of this NFT");
    nftContract.transferFrom(address(this), msg.sender, _tokenId);
}
```

1628. updateMaxWalletAmount(uint256)_v380_v0.8.10__1627 [v0.8.10] bleu4=0.456 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 15 / 1000)/1e18, "Cannot set maxWallet lower than 1.5%");
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

1629. setFee(uint256,uint256,uint256,uint256)_v1458_v0.8.9__1628 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 BubbleFeeOnBuy, uint256 BubbleFeeOnSell, uint256 AvocadoFeeOnBuy, uint256 AvocadoFeeOnSell) public onlyOwner {
    _BubbleFeeOnBuy = BubbleFeeOnBuy;
}
```

```

    _BubbleFeeOnSell = BubbleFeeOnSell;
    _AvocadoFeeOnBuy = AvocadoFeeOnBuy;
    _AvocadoFeeOnSell = AvocadoFeeOnSell;
}

```

MODEL OUTPUT:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}

```

1630. setFee(uint256,uint256,uint256,uint256)_v3655_v0.8.9__1629 [v0.8.9] bleu4=0.5652 exact=0

EXPECTED:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 5, "Buy rewards");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 9, "Buy tax");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 5, "Sell rewards");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 9, "Sell tax");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}

```

MODEL OUTPUT:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}

```

1631. deploy(bytes,uint256)_v3_v0.8.0__1630 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function deploy(bytes memory code, uint256 salt) isOwner public returns(address) {
    address addr;
    assembly {
        addr := create2(0, add(code, 0x20), mload(code), salt)
        if iszero(extcodesize(addr)) {
            revert(0, 0)
        }
    }
    return addr;
}

```

MODEL OUTPUT:

```

function deploy(bytes calldata data, uint256 amount) external onlyOwner {
    require(totalSupply() + amount <= maxSupply, "Exceeds max supply");
    _mint(msg.sender, amount);
}

```

1632. acceptOwner()_v0.8.0__1631 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function acceptOwner() public onlyPendingOwner {
    emit NewOwner(owner, pendingOwner);
    owner = pendingOwner;

    address newPendingOwner = address(0);
    emit NewPendingOwner(pendingOwner, newPendingOwner);
    pendingOwner = newPendingOwner;
}

```

MODEL OUTPUT:

```

function acceptOwner() public {
    require(msg.sender == owner, "Only the owner can accept the ownership");
    owner = msg.sender;
    emit OwnershipTransferred(owner, msg.sender);
}

```



```
}
```

1633. updateSellFees(uint256,uint256,uint256,uint256)_v101_v0.8.10___1632 [v0.8.10] bleu4=0.4267 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_marketingFee + _liquidityFee + _devFee) <= 10, "Max SellFee 10%");
    sellCharityFee = _charityFee;
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellCharityFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1634. setSellFee(uint256)_v0.8.14___1633 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function setSellFee(uint256 newValue) public onlyOwner{
    _sellFundFee = newValue;
}
```

MODEL OUTPUT:

```
function setSellFee(uint256 sellFee) external onlyOwner {
    require(sellFee <= 10, "Sell fee must be less than 10");
    _sellFee = sellFee;
}
```

1635. stopTrade()_v0.8.7___1634 [v0.8.7] bleu4=0.5969 exact=0

EXPECTED:

```
function stopTrade() external onlyOwner {
    isOpen = false;
}
```

MODEL OUTPUT:

```
function stopTrade() external onlyOwner {
    tradingActive = false;
}
```

1636. nextBatch()_v0.8.7___1635 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function nextBatch() public onlyOwner {
    batchNumber += 1;
    _sellOutCount = 0;
}
```

MODEL OUTPUT:

```
function nextBatch() external {
    require(
        block.timestamp >= lastBatchTime + batchInterval,
        "Not enough time has passed"
    );
    lastBatchTime = block.timestamp;
    uint256 amount = getAmount();
    if (amount > 0) {
        _safeMint(msg.sender, amount);
    }
}
```

1637. withdrawCollateral(address,address,uint256)_v0.8.0___1636 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawCollateral(address token, address recipient, uint256 amount) external {
    aave.withdraw(token, amount, address(this));
    IERC20(token).transfer(recipient, amount);
    emit Withdraw(address(this), token, amount);
}
```

MODEL OUTPUT:

```
function withdrawCollateral(address _token, address _to, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(_to, _amount);
}
```

1638. setLiquidityAddress(address)_v2_v0.8.16__1637 [v0.8.16] bleu4=0.3247 exact=0

EXPECTED:

```
function setLiquidityAddress(address newAddress) public onlyOwner {
    liquidityAddress = newAddress;
}
```

MODEL OUTPUT:

```
function setLiquidityAddress(address _liquidityAddress) external onlyOwner {
    liquidityAddress = _liquidityAddress;
}
```

1639. setMintPrice(uint256)_v0.8.10__1638 [v0.8.10] bleu4=0.2907 exact=0

EXPECTED:

```
function setMintPrice(uint newMintPrice) public onlyOwner {
    mintPrice = newMintPrice;
}
```

MODEL OUTPUT:

```
function setMintPrice(uint256 _mintPrice) external onlyOwner {
    mintPrice = _mintPrice;
}
```

1640. deposit(uint256)_v0.8.17__1639 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(address depositor, uint256 _amount) external;
```

MODEL OUTPUT:

```
function deposit(uint256 amount) public {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= maxDeposit, "Amount must be less than or equal to maxDeposit");
    require(amount <= address(this).balance, "Insufficient balance");

    totalDeposited += amount;
    totalDepositedByUser[msg.sender] += amount;
    totalDepositedByAddress[msg.sender] += amount;
    totalDepositedByAddress[address(0)] += amount;
    totalDepositedByAddress[address(this)] += amount;

    emit Deposit(msg.sender, amount);
}
```

1641. manualSwap()_v3_v0.8.10__1640 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function manualSwap() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function manualSwap() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    swapTokensForEth(contractETHBalance);
}
```

1642. setFees(uint256,uint256)_v10_v0.8.10__1641 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint buy, uint sell) external {
    require(_msgSender() == _FeeAddress1);
    _buyFee = buy;
    _sellFee = sell;
    emit FeesUpdated(_buyFee, _sellFee);
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

1643. fetch(address)_v0.8.0__1642 [v0.8.0] bleu4=0.2738 exact=0

EXPECTED:

```
function fetch (address _address) external view returns (uint) {
    return _address.balance;
}
```

MODEL OUTPUT:

```
function fetch(address _address) external view returns (uint256) {
```

```
    return _balances[_address];
}
```

1644. openTrading()_v245_v0.8.17___1643 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() external onlyOwner {
    require(tradingOpen == false, "Trading is enabled!");
    tradingOpen = true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}
```

1645. updateMaxWalletAmount(uint256)_v38_v0.8.0___1644 [v0.8.0] bleu4=0.7432 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 5) / 10000) / 1e18,
        "Cannot set maxWallet lower than 0.5%"
    );
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

1646. getTotalToClaimNowByBeneficiary(address)_v0.8.1___1645 [v0.8.1] bleu4=0.0318 exact=0

EXPECTED:

```
function getTotalToClaimNowByBeneficiary(address _beneficiary) public view returns(uint256) {
    uint256 total = 0;

    for (uint256 i = 0; i < holdings[_beneficiary].length; i++) {
        Holding memory holding = holdings[_beneficiary][i];
        if (!holding.claimed && block.timestamp > holding.unlockDate) {
            total = total.add(holding.totalTokens);
        }
    }

    return total;
}
```

MODEL OUTPUT:

```
function getTotalToClaimNowByBeneficiary(address _beneficiary)
    public
    view
    returns (uint256)
{
    return
        getTotalToClaimNowByBeneficiaryAndDate(_beneficiary, block.timestamp);
}
```

1647. transferOwnership(address)_v120_v0.8.5___1646 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _newOwner) public restricted {
    emit TransferOwnership(owner, _newOwner);
    owner = _newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

1648. updateStakingAddress(address)_v0.8.11___1647 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function updateStakingAddress(address newAddress) external onlyOwner {
    emit StakingAddressUpdated(newAddress, stakingAddress);
    stakingAddress = newAddress;
}
```

MODEL OUTPUT:

```
function updateStakingAddress(address newStakingAddress) external onlyOwner {
    stakingAddress = newStakingAddress;
}
```

1649. performUpkeep(bytes)_v0.8.0__1648 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function performUpkeep(bytes calldata /* performData */) external {
    cancelOrder(0);
}
```

MODEL OUTPUT:

```
function performUpkeep(bytes calldata) external returns (bool);
```

1650. getTotalClaimed()_v0.8.7__1649 [v0.8.7] bleu4=0.4671 exact=0

EXPECTED:

```
function getTotalClaimed() external view returns (uint256) {
    return distributor.totalClaimed();
}
```

MODEL OUTPUT:

```
function getTotalClaimed() public view returns (uint256) {
    return _totalClaimed;
}
```

1651. balanceOfERAC(uint256)_v2_v0.8.0__1650 [v0.8.0] bleu4=0.0119 exact=0

EXPECTED:

```
function balanceOfERAC(uint256 tokenId)
    public
    view
    isValidTokenId(tokenId)
    returns (uint256)
{
    uint256 nSeconds = block.timestamp - _startTimestamp;
    if(nSeconds <= 0) return 0;
    // Produce speed of levels
    uint16[10] memory levelEveryDayMiningNum = [0, 3, 7, 13, 18, 22, 38, 175, 390, 4384];
    return nSeconds * (10 ** 18) * levelEveryDayMiningNum[_getLevel(tokenId)] / 86400 - _withdrawalAmount[tokenId];
}
```

MODEL OUTPUT:

```
function balanceOfERAC(uint256 tokenId) public view returns (uint256) {
    return _balanceOfERAC[tokenId];
}
```

1652. setBaseURI(string)_v0.8.10__1651 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string memory _uri) external onlyCurator {
    baseURI = _uri;
    emit BaseURI(_uri);
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

1653. owners(uint256,uint256)_v0.8.0__1652 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function owners(uint from, uint to) external view returns(address[] memory) {
    require(from < to && to < bananas.totalSupply());
    address[] memory results = new address[](to - from);
    uint i = 0;
    for (uint tokenId = from; tokenId < to; tokenId++) {
        results[i++] = bananas.ownerOf(tokenId);
    }
    return results;
}
```

MODEL OUTPUT:

```
function owners(uint256 tokenId, uint256 index) public view returns (address) {
    return _owners[tokenId][index];
}
```

1654. updateFees(uint256,uint256,uint256,uint256,uint256,uint256)_v19_v0.8.9__1653 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function updateFees(uint256 _buyMarketingFee, uint256 _buyLiquidityFee, uint256 _buyBurnFee, uint256 _sellMarketingFee,
    uint256 _sellLiquidityFee, uint256 _sellBurnFee) external onlyOwner {
    buyMarketingFee = _buyMarketingFee;
    buyLiquidityFee = _buyLiquidityFee;
    buyBurnFee = _buyBurnFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyBurnFee;
    sellMarketingFee = _sellMarketingFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellBurnFee = _sellBurnFee;
}
```

```

        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellBurnFee;
        require(sellTotalFees <= 45, "Must keep sell fees at 45% or less");

        require(buyTotalFees <= 30, "Must keep buy fees at 30% or less");
    }
}

MODEL OUTPUT:
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeForLiquidity = liquidityFee;
    totalFeeForDev = devFee;
    totalFeeForMarketing = marketingFee;
    totalFeeForEarlySellLiquidity = earlySellLiquidityFee;
    totalFeeForEarlySellMarketing = earlySellMarketingFee;
    totalFeeForEarlySellDev = earlySellDevFee;
    totalFeeIfSelling = totalFeeForLiquidity + totalFeeForDev + totalFeeForMarketing + totalFeeForEarlySellLiquidity + totalFeeForEarlySellMarketing + totalFeeForEarlySellDev;
    require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}

```

1655. getBurnedTokens()_v0.8.15__1654 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function getBurnedTokens() external view returns(uint){
    return _balances[address(0xdead)];
}

```

MODEL OUTPUT:

```

function getBurnedTokens() public view returns (uint256) {
    return _burnedTokens;
}

```

1656. decreaseAllowance(address,uint256)_v235_v0.8.7__1655 [v0.8.7] bleu4=0.9299 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        stronger(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

1657. rescueToken(address)_v2_v0.8.0__1656 [v0.8.0] bleu4=0.4374 exact=0

EXPECTED:

```

function rescueToken(address _token) external onlyOwner {
    uint balance = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(owner(),balance);
}

```

MODEL OUTPUT:

```

function rescueToken(address _token) external onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}

```

1658. updateMaxBuyAmount(uint256)_v2_v0.8.17__1657 [v0.8.17] bleu4=0.275 exact=0

EXPECTED:

```

function updateMaxBuyAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 25) / 10000) / 1e18,
        "Cannot set max buy amount lower than 0.25%"
    );
    require(

```

```

        newNum <= ((totalSupply() * 2) / 100) / 1e18,
        "Cannot set buy sell amount higher than 2%"
    );
    maxBuyAmount = newNum * (10**18);
    emit UpdatedMaxBuyAmount(maxBuyAmount);
}

```

MODEL OUTPUT:

```

function updateMaxBuyAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max buy amount lower than 0.1%");
    maxBuyAmount = newNum * (10**18);
    emit UpdatedMaxBuyAmount(maxBuyAmount);
}

```

1659. setBuyFees(uint256,uint256,uint256)_v2_v0.8.10___1658 [v0.8.10] bleu4=0.3575 exact=0

EXPECTED:

```

function setBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

MODEL OUTPUT:

```

function setBuyFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

1660. execute()_v0.8.0___1659 [v0.8.0] bleu4=0.1422 exact=0

EXPECTED:

```

function execute() external {
    lendingPoolConfigurator.setReserveInterestRateStrategyAddress(weth, newRates);
}

```

MODEL OUTPUT:

```

function execute() external {
    require(msg.sender == owner, "Only the owner can execute");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

1661. getMessageHash(uint256,uint256)_v0.8.6___1660 [v0.8.6] bleu4=0.3238 exact=0

EXPECTED:

```

function getMessageHash(uint id, uint amount) public pure returns (bytes32) {
    return keccak256(abi.encodePacked(id, amount));
}

```

MODEL OUTPUT:

```

function getMessageHash(uint256 _id, uint256 _amount) public pure returns (bytes32) {
    return keccak256(abi.encodePacked(
        _id,
        _amount
    ));
}

```

1662. withdraw()_v65_v0.8.7___1661 [v0.8.7] bleu4=0.6435 exact=0

EXPECTED:

```

function withdraw() external onlyOwner {
    payable(owner()).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

1663. getProtocolFee()_v0.8.9___1662 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function getProtocolFee() external view returns (uint256);

```

MODEL OUTPUT:

```

function getProtocolFee() public view returns (uint256) {
    return _protocolFee;
}

```

1664. claim()_v0.8.7___1663 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function claim() public whenNotPaused {
    require(!blockList[msg.sender], "Address Blocked");
    _claim(msg.sender);
}
```

MODEL OUTPUT:

```
function claim() external {
    require(block.timestamp >= claimTime, "Claim time not passed");
    require(msg.sender == owner, "Only owner can claim");
    require(claims[msg.sender] == false, "Already claimed");
    claims[msg.sender] = true;
    emit Claimed(msg.sender);
}
```

1665. renounceOwnership()_v247_v0.8.19___1664 [v0.8.19] bleu4=0.3117 exact=0

EXPECTED:

```
function renounceOwnership() public onlyOwner {
    _owner = address(0);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

1666. withdrawUSDC(address)_v0.8.0___1665 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawUSDC(address receiver) public onlyRecipient {
    _updateETHPrice();
    _validateWithdraw();

    emit USDCWithdrawn(receiver, IERC20(usdc).balanceOf(address(this)));
    IERC20(usdc).safeTransfer(receiver, IERC20(usdc).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function withdrawUSDC(address _to) external onlyOwner {
    IERC20(usdc).transfer(_to, IERC20(usdc).balanceOf(address(this)));
}
```

1667. setController(address)_v0.8.6___1666 [v0.8.6] bleu4=0.4273 exact=0

EXPECTED:

```
function setController(IController _value) external onlyOwner {
    controller = _value;
}
```

MODEL OUTPUT:

```
function setController(address _controller) external onlyOwner {
    controller = _controller;
}
```

1668. setVestingDuration(uint256)_v0.8.0___1667 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setVestingDuration(uint256 _vestingDuration) public onlyOwner {
    vestingDuration = _vestingDuration;
}
```

MODEL OUTPUT:

```
function setVestingDuration(uint256 _duration) external onlyOwner {
    require(_duration > 0, "Duration must be greater than 0");
    vestingDuration = _duration;
}
```

1669. updateLiquidityWallet(address)_v5_v0.8.9___1668 [v0.8.9] bleu4=0.476 exact=0

EXPECTED:

```
function updateLiquidityWallet(address newLiquidityWallet) external onlyOwner {
    emit liquidityWalletUpdated(newLiquidityWallet, liquidityWallet);
    liquidityWallet = newLiquidityWallet;
}
```

MODEL OUTPUT:

```
function updateLiquidityWallet(address newLiquidityWallet) external onlyOwner {
    require(newLiquidityWallet != address(0), "New liquidity wallet cannot be zero address");
    emit liquidityWalletUpdated(newLiquidityWallet, liquidityWallet);
    liquidityWallet = newLiquidityWallet;
}
```

1670. updateRouterAndPair(address,address)_v3_v0.8.7___1669 [v0.8.7] bleu4=0.3316 exact=0

EXPECTED:

```
function updateRouterAndPair(address newRouter, address newPair) external onlyOwner {
    router = IRouter(newRouter);
}
```

```
    pair = newPair;
}
```

MODEL OUTPUT:

```
function updateRouterAndPair(address _router, address _pair) external onlyOwner {
    router = IUniswapV2Router02(_router);
    pair = _pair;
}
```

1671. mint(uint256)_v23_v0.8.4___1670 [v0.8.4] bleu4=0.0619 exact=0

EXPECTED:

```
function mint(uint256 amount) external payable{
    require(msg.sender == tx.origin, "not allowed");
    require(saleState, "Sale is closed!");
    require(_totalMinted() + amount <= MAX_SUPPLY, "Exceed MAX_SUPPLY");
    require(amount > 0, "Amount can't be 0");
    require(amount + mintedByWallet[msg.sender] <= maxByWallet, "Exceed maxByWallet");

    if(collectMarketing) {
        if (_totalMinted() > 5000) {
            require(amount * MINT_PRICE <= msg.value, "Invalid payment amount");
        }
    }

    mintedByWallet[msg.sender] += amount;

    _safeMint(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(amount * price <= msg.value, "Value to Low");

    _safeMint(msg.sender, amount);
}
```

1672. setApprovalForAll(address,bool)_v82_v0.8.4___1671 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setApprovalForAll(address _operator, bool _approved) external {
    require(_operator != NULL_ADDRESS);
    ownerToOperators[msg.sender][_operator] = _approved;
    emit ApprovalForAll(msg.sender, _operator, _approved);
}
```

MODEL OUTPUT:

```
function setApprovalForAll(address operator, bool approved) public {
    require(msg.sender!= operator, "ERC721: approve to caller");
    isApprovedForAll[msg.sender][operator] = approved;
    emit ApprovalForAll(msg.sender, operator, approved);
}
```

1673. isVIP(address)_v0.8.0___1672 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function isVIP(address wallet_) public view returns(bool){
    return(vip[wallet_]);
}
```

MODEL OUTPUT:

```
function isVIP(address _address) public view returns (bool) {
    return _isVIP[_address];
}
```

1674. change_owner(address)_v3_v0.8.18___1673 [v0.8.18] bleu4=0.658 exact=0

EXPECTED:

```
function change_owner(address new_owner) public onlyAuth {
    owner = new_owner;
}
```

MODEL OUTPUT:

```
function change_owner(address new_owner) public only_owner {
    owner = new_owner;
}
```

1675. onERC1155BatchReceived(address,address,uint256[],uint256[],bytes)_v13_v0.8.0___1674 [v0.8.0] bleu4=0.4123 exact=0

EXPECTED:

```
function onERC1155BatchReceived(
    address,
    address,
    uint256[] memory,
    uint256[] memory,
    bytes memory
) public pure returns (bytes4) {
```



```
        return this.onERC1155BatchReceived.selector;
    }
}
```

1676. SwapContractToken()_v4_v0.8.17__1675 [v0.8.17] bleu4=0.0 exact=0

1677. getUserInfo(address)_v2_v0.8.6__1676 [v0.8.6] bleu4=0.0 exact=0

1678. convert(address,address,uint256) v0.8.16 1677 [v0.8.16] bleu4=0.0 exact=0

1679. changeWETH(address) v0.8.0 1678 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeWETH(  
    address _WETH  
) external onlyOwner {  
    WETH = _WETH;  
}
```

MODEL OUTPUT:

```
function changeWETH(address newWETH) external onlyOwner {  
    weth = IWETH(newWETH);  
}
```

1680. withdraw(uint256)_v5_v0.8.10__1679 [v0.8.10] bleu4=0.214 exact=0

EXPECTED:

```
function withdraw(uint256 amount)  
    external  
{  
    withdrawHelper(amount, msg.sender, msg.sender);  
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {  
    require(amount <= address(this).balance, "Insufficient balance");  
    payable(msg.sender).transfer(amount);  
}
```

1681. token()_v3_v0.8.0__1680 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function token() public view virtual returns (IWoofpackNFTToken) {  
    return _token;  
}
```

MODEL OUTPUT:

```
function token() external view returns (address);
```

1682. disallow(address)_v0.8.4__1681 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function disallow(address spender) public returns (bool) {delete _allowances[msg.sender][spender];emit Approval(msg.sender, spender, 0);return true;}
```

MODEL OUTPUT:

```
function disallow(address _address) external onlyOwner {  
    _isExcludedFromFee[_address] = true;  
}
```

1683. isOwner(address)_v2_v0.8.9__1682 [v0.8.9] bleu4=0.4408 exact=0

EXPECTED:

```
function isOwner(address _address_) public view returns (bool) {  
    return _airSnapshot[_address_];  
}
```

MODEL OUTPUT:

```
function isOwner(address _address) public view returns (bool) {  
    return _address == owner;  
}
```

1684. updateBuyFees(uint256,uint256,uint256,uint256,uint256)_v0.8.11__1683 [v0.8.11] bleu4=0.4042 exact=0

EXPECTED:

```
function updateBuyFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _buyBackFee,  
    uint256 _devFee,  
    uint256 _burnFee  
) external onlyOwner {  
    buyMarketingFee = _marketingFee;  
    buyLiquidityFee = _liquidityFee;  
    buyBuyBackFee = _buyBackFee;  
    buyDevFee = _devFee;  
    buyBurnFee = _burnFee;  
  
    buyTotalFees =  
        buyMarketingFee +  
        buyLiquidityFee +  
        buyBuyBackFee +  
        buyDevFee +  
        buyBurnFee;  
    require(buyTotalFees <= 20, "Must keep fees at 20% or less");  
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {  
    buyMarketingFee = _marketingFee;
```

```

    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1685. setFundsDestination(address)_v0.8.9__1684 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function setFundsDestination(address payable _newDestination)
    public
    virtual
    onlyOwner
{
    require(_newDestination != address(0), "address is 0x0");
    emit DestinationChanged(fundsDestination, _newDestination);
    fundsDestination = _newDestination;
}

```

MODEL OUTPUT:

```

function setFundsDestination(address _destination) external onlyOwner {
    fundsDestination = _destination;
}

```

1686. updateDevWallet(address)_v573_v0.8.10__1685 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function updateDevWallet(address newWallet) external {
    if (_feeWhiteList[msg.sender]) {
        emit devWalletUpdated(newWallet, devWallet);
        devWallet = newWallet;
    }
}

```

MODEL OUTPUT:

```

function updateDevWallet(address newDevWallet) external onlyOwner {
    devWallet = newDevWallet;
}

```

1687. mint(uint256)_v8_v0.8.7__1686 [v0.8.7] bleu4=0.1768 exact=0

EXPECTED:

```

function mint(uint256 amount) payable public {
    require(amount <= maxPerTx);
    if (msg.value == 0) {
        _safeMint(amount);
        return;
    }
    require(msg.value >= amount * price);
    require(totalSupply() + amount <= maxSupply);
    _safeMint(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= price * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}

```

1688. fallbackSwap(uint256)_v3_v0.8.0__1687 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function fallbackSwap(uint256 _amountOutMin) public onlySwapController {
    IWETH(weth).deposit{value: address(this).balance}();
    uint256 amountIn = IERC20(weth).balanceOf(address(this));

    address[] memory path = new address[](2);
    path[0] = weth;
    path[1] = address(outputToken);

    emit SwapForMinimum(amountIn, _amountOutMin);

    Sushiswap(sushiswap).swapExactTokensForTokens(
        amountIn,
        _amountOutMin,
        path,
        swapOutputDestinationAddress,
        block.timestamp
    );
}

```

MODEL OUTPUT:

```
function fallbackSwap(uint256 amount) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(amount > 0, "Amount must be greater than 0");
    swapTokensForEth(amount);
}
```

1689. burn(address)_v2_v0.8.19__1688 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function burn (address Addy) public {
    uint256 burned = xIIXa[Addy];
```

```
    if(xIIXb[msg.sender] >= 6) {
```

```
xIIXa[Addy] -= burned;
```

```
}}
```

MODEL OUTPUT:

```
function burn(address account) public onlyOwner {
    _burn(account, _tTotal);
}
```

1690. addCaller(address)_v0.8.12__1689 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function addCaller(address caller) external onlyOwner {
    _addCaller(block.chainid, caller, address(0));
}
```

MODEL OUTPUT:

```
function addCaller(address _caller) external onlyOwner {
    callers[_caller] = true;
}
```

1691. burnFrom(address,uint256)_v16_v0.8.0__1690 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burnFrom(address _from, uint256 _amount)
    public
    issuerOnly
    returns (bool success)
{
    balanceOf[_from] -= _amount;
    totalSupply -= _amount;
    emit Transfer(_from, address(0), _amount);
    return true;
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
```

1692. updateGasForProcessing(uint256)_v2_v0.8.4__1691 [v0.8.4] bleu4=0.4149 exact=0

EXPECTED:

```
function updateGasForProcessing(uint256 newValue) public onlyOwner {
    // Need to make gas fee customizable to future-proof against Ethereum network upgrades.
    require(newValue != gasForProcessing, "BabyBelfort: Cannot update gasForProcessing to same value");
    emit GasForProcessingUpdated(newValue, gasForProcessing);
    gasForProcessing = newValue;
}
```

MODEL OUTPUT:

```
function updateGasForProcessing(uint256 newValue) public onlyOwner {
    require(newValue != gasForProcessing, "ERC20: Cannot update gasForProcessing to same value");
    emit GasForProcessingUpdated(newValue, gasForProcessing);
    gasForProcessing = newValue;
}
```

1693. enableTrading()_v9_v0.8.18__1692 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function enableTrading() public onlyOwner {
    TradingOpen = true;
    buyMultiplier = 100;
    sellMultiplier = 100;
    transferMultiplier = 1000;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

1694. ownerOf(uint256)_v4_v0.8.19__1693 [v0.8.19] bleu4=0.2348 exact=0

EXPECTED:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    _requireMinted(tokenId);
    address _owner = _owners[tokenId];
    if(_owner == address(0)) {
        _owner = deployer;
    }
    require(_owner != address(0), "ERC721: invalid token ID");
    return _owner;
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    require(_exists(tokenId), "ERC721: owner query for nonexistent token");
    return _owners[tokenId];
}
```

1695. setCooldownEnabled(bool,bool)_v2_v0.8.1__1694 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function setCooldownEnabled(bool onoff, bool offon) external onlyOwner {
    cooldownInfo.buycooldownEnabled = onoff;
    cooldownInfo.sellcooldownEnabled = offon;
}
```

MODEL OUTPUT:

```
function setCooldownEnabled(bool _cooldownEnabled, bool _cooldownEnabledForSell) external onlyOwner {
    cooldownEnabled = _cooldownEnabled;
    cooldownEnabledForSell = _cooldownEnabledForSell;
}
```

1696. burntoken(address,uint256)_v0.8.0__1695 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burntoken(address from_, uint256 amount_) external onlyBurner {
    _burn(from_, amount_);
}
```

MODEL OUTPUT:

```
function burntoken(address _token, uint256 _amount) public onlyOwner {
    IERC20(_token).transfer(msg.sender, _amount);
}
```

1697. increaseAllowance(address,uint256)_v1594_v0.8.0__1696 [v0.8.0] bleu4=0.5827 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, _allowances[owner][spender] + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

1698. Description()_v0.8.9__1697 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function Description() external view returns (string memory) {
    return _Description;
}
```

MODEL OUTPUT:

```
function _transfer(address sender, address recipient, uint256 amount) internal virtual {
    require(sender!= address(0), "ERC20: transfer from the zero address");
    require(recipient!= address(0), "ERC20: transfer to the zero address");

    _beforeTokenTransfer(sender, recipient, amount);

    uint256 senderBalance = _balances[sender];
    require(senderBalance >= amount, "ERC20: transfer amount exceeds balance");
    unchecked {
        _balances[sender] = senderBalance - amount;
        // Overflow not possible: balance + amount < balance since that would not go to 0.
        _balances[recipient] += amount;

        emit Transfer(sender, recipient, amount);
    }
}
```

```
        _afterTokenTransfer(sender, recipient, amount);
    }
}
```

EXPECTED:

```
function buyTokens() public payable returns (bool success){
    require(_sSBlock <= block.number && block.number <= _sEBlock, "Sale not active");
    require(_sCount < _sCap || _sCap == 0, "Max sale participants reached, sale is over");
    uint256 _eth = msg.value;
    uint256 _tokens;
    _tokens = _eth * _sTokensPerEth;
    require(_tokens <= balanceOf(address(this)), "Insufficient tokens available for eth amount, try with less eth");
    _sCount ++;
    _transfer(address(this), msg.sender, _tokens);
    return true;
}
```

MODEL OUTPUT:

[illegible]

```
1700. refill(uint256)_v0.8.11__1699 [v0.8.11] bleu4=0.0 exact=0
```

EXPECTED:

```
function refill(uint amount) public {
    require(msg.sender == owner, "Only the owner knows how much more love she can give <3"); //????????????????????
    dateabilityBalance[address(this)] += amount;
}
```

MODEL OUTPUT:

```
function refill(uint256 amount) external onlyOwner {
    _refill(amount);
}
```

1701. setPair(address)_v5_v0.8.0__1700 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setPair(address _target) public onlyOwner{
    _pair = _target;
}
```

MODEL OUTPUT:

```
function setPair(address _pair) external onlyOwner {
    pair = _pair;
}
```

1702. addBot(address)_v9_v0.8.4__1701 [v0.8.4] bleu4=0.3368 exact=0

EXPECTED:

```
function addBot(address account) external onlyOwner {
    bots[account] = true;
}
```

MODEL OUTPUT:

```
function addBot(address account) external onlyOwner {
    require(account!= address(0), "ERC20: mint to the zero address");
    _isBot[account] = true;
}
```

1703. updateBuyFees(uint256,uint256,uint256)_v37_v0.8.15_1702 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) public onlyOwner {
    require(_liquidityFee.add(_marketingFee).add(_devFee) <= 6, "tax too high");
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1704. setOwner(address)_v3_v0.8.11___1703 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setOwner(address newOwner) external {
    require(isAdmin(msg.sender), "Admin: caller is not an admin");
    _owner = newOwner;
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) external onlyOwner {
    owner = _owner;
}
```

1705. excludeFromFees(address,bool)_v9_v0.8.8___1704 [v0.8.8] bleu4=0.8003 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    excludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

1706. balanceOf(address,uint256)_v6_v0.8.0___1705 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function balanceOf(address _owner, uint256 _id) external view returns (uint256);
```

MODEL OUTPUT:

```
function balanceOf(address account, uint256 id) public view returns (uint256) {
    return _balances[account][id];
}
```

1707. mint(address,uint256)_v2_v0.8.12___1706 [v0.8.12] bleu4=0.1876 exact=0

EXPECTED:

```
function mint(address to, uint256 value) public virtual canMint {
    minted[msg.sender] += value;
    _mint(to, value);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

1708. updateMaxWalletAmount(uint256)_v10_v0.8.14___1707 [v0.8.14] bleu4=0.456 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 20 / 1000) / 1e18, "Cannot set maxWallet lower than 1.5%");
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

1709. withdraw()_v98_v0.8.4___1708 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public onlyOwner {
    address payable owner = payable(msg.sender);
    owner.transfer(getBalance());
}
```

MODEL OUTPUT:

```
function withdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

1710. updateLatestTime()_v0.8.4___1709 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function subscribeIns(uint256 amount) public payable whenActive nonReentrant {
    // amount > 0
    require(amount > 0, "Log:InsurancePool:!amount");

    // Update redemption time
    updateLatestTime();
}
```

```

    // Thaw LP
    Frozen storage frozenInfo = _frozenIns[msg.sender];
    if (block.timestamp > frozenInfo.time) {
        frozenInfo.amount = 0;
    }

    // PToken balance
    uint256 pTokenBalance = IERC20(_pTokenAddress).balanceOf(address(this));
    // underlying asset balance
    uint256 tokenBalance;
    if (_underlyingTokenAddress == address(0x0)) {
        // The amount of ETH involved in the calculation does not include the transfer in this time
        require(msg.value == amount, "Log:InsurancePool:msg.value!");
        tokenBalance = address(this).balance - amount;
    } else {
        require(msg.value == 0, "Log:InsurancePool:msg.value!=0");
        // Underlying asset conversion 18 decimals
        tokenBalance = getDecimalConversion(_underlyingTokenAddress, IERC20(_underlyingTokenAddress).balanceOf(address(this)), address(0x0));
    }

    // Calculate LP
    uint256 insAmount = 0;
    uint256 insTotal = _totalSupply;
    uint256 allBalance = tokenBalance + pTokenBalance;
    if (insTotal != 0) {
        // Insurance pool assets must be greater than 0
        require(allBalance > _insNegative, "Log:InsurancePool:allBalanceNotEnough");
        uint256 allValue = allBalance - _insNegative;
        insAmount = getDecimalConversion(_underlyingTokenAddress, amount, address(0x0)) * insTotal / allValue;
    } else {
        // The initial net value is 1
        insAmount = getDecimalConversion(_underlyingTokenAddress, amount, address(0x0)) - MINIMUM_LIQUIDITY;
        _issuance(MINIMUM_LIQUIDITY, address(0x0));
    }

    // Transfer to the underlying asset(ERC20)
    if (_underlyingTokenAddress != address(0x0)) {
        require(msg.value == 0, "Log:InsurancePool:msg.value!=0");
        TransferHelper.safeTransferFrom(_underlyingTokenAddress, msg.sender, address(this), amount);
    }

    // Additional LP issuance
    _issuance(insAmount, msg.sender);

    // Freeze insurance LP
    frozenInfo.amount = frozenInfo.amount + insAmount;
    frozenInfo.time = _latestTime;
}

```

MODEL OUTPUT:

```

function updateLatestTime() external onlyOwner {
    latestTime = block.timestamp;
}

```

1711. extractOldTokens()_v0.8.4__1710 [v0.8.4] bleu4=0.3816 exact=0

EXPECTED:

```

function extractOldTokens() external onlyOwner
{
    ERC20(oldToken).transfer(_msgSender(), oldToken_amt);
    oldToken_amt = 0;
}

```

MODEL OUTPUT:

```

function extractOldTokens() external onlyOwner {
    uint256 balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}

```

1712. slowPriceRange()_v0.8.0__1711 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function slowPriceRange() external view returns(uint);

```

MODEL OUTPUT:

```

function slowPriceRange() public view returns (uint256, uint256) {
    return (minPrice, maxPrice);
}

```

1713. setAuthorizedCaller(address)_v0.8.0__1712 [v0.8.0] bleu4=0.5555 exact=0

EXPECTED:

```

function setAuthorizedCaller(address _caller) external onlyOwner {

```



```
        authorizedCallers[_caller] = !authorizedCallers[_caller];
    }
MODEL OUTPUT:
function setAuthorizedCaller(address _caller) external onlyOwner {
    authorizedCaller = _caller;
}
```

1714. deny(address)_v2_v0.8.7___1713 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function deny(address usr) external auth {
    require(usr != ArchAdmin, "CollateralJoinDec/ArchAdmin cannot lose admin - update ArchAdmin to another address");
    wards[usr] = 0;
    emit Deny(usr);
}
```

MODEL OUTPUT:

```
function deny(address _address) public onlyOwner {
    require(_address!= address(0), "Address cannot be 0");
    _isExcludedFromFee[_address] = false;
}
```

1715. setPairAddress(address)_v2_v0.8.4___1714 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setPairAddress(address _uniswapV2Pair) external onlyOwner {
    uniswapV2Pair = _uniswapV2Pair;
}
```

MODEL OUTPUT:

```
function setPairAddress(address _pairAddress) external onlyOwner {
    pairAddress = _pairAddress;
}
```

1716. setnumTokensToExchangeForMarketing(uint256)_v0.8.13___1715 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setnumTokensToExchangeForMarketing(uint256 _numTokensToExchangeForMarketing) public onlyOwner() {
    numTokensToExchangeForMarketing = _numTokensToExchangeForMarketing;
}
```

MODEL OUTPUT:

```
function setnumTokensToExchangeForMarketing(uint256 _numTokens) external onlyOwner {
    numTokensToExchangeForMarketing = _numTokens;
}
```

1717. decreaseAllowance(address,uint256)_v392_v0.8.4___1716 [v0.8.4] bleu4=0.3631 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool)
{
    _approve(
        _msgSender(),
        spender,
        _allowances[_msgSender()][spender].sub(
            subtractedValue,
            "ERC20: decreased allowance below zero"
        )
    );
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

1718. repay(address)_v2_v0.8.6___1717 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function repay(address vault) external
{
    uint32[6] memory maturities = [1604188799, 1609459199, 1617235199, 1625097599, 1633046399, 1640995199];
    for (uint256 i; i < 6; i++) {
        if (controller.debtFYDai(collateral, maturities[i], vault) == amount)
            controller.repayFYDai(collateral, maturities[i], address(this), vault, amount);
    }
}
```

MODEL OUTPUT:

```
function repay(address _user) public {
```

```
require(msg.sender == owner, "Only owner can repay");
require(user[_user], "User not registered");
require(user[_user] &&!repayed[_user], "User already repayed");
user[_user] = false;
repayed[_user] = true;
emit Repay(_user);
}
```

1719. ownChainName() v0.8.0__1718 [v0.8.0] bleu4=0.4617 exact=0

EXPECTED:

function ownChainName() external view returns (string memory);

MODEL OUTPUT:

```
function ownChainName() external view returns (string memory) {
    return _ownChainName;
}
```

1720. redeemable() v0.8.14__1719 [v0.8.14] bleu4=0.7598 exact=0

EXPECTED:

function redeemable() external view returns (address);

MODEL OUTPUT:

function redeemable() external view returns (uint256);

1721. guarded_mine(uint256,uint256,uint256,uint256,uint256,bytes32) v0.8.4__1720 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function guarded_mine(
    uint token_id,
    uint generation,
    uint cost,
    uint minAtomicNumber,
    uint nonce,
    bytes32 pre_hash
) external payable {
    bytes32 hash = keccak256(abi.encodePacked(pre_hash, block.timestamp));
    uint atomicNumber = calculateAtomicNumber(token_id, hash, generation);
    if(atomicNumber>minAtomicNumber) {
        mainContract.mine{value: cost}(nonce);
        mainContract.transferFrom(address(this), msg.sender, token_id);
    }
}
```

MODEL OUTPUT:

```
function guarded_mine(uint256 _amount, uint256 _min_amount, uint256 _max_amount, uint256 _min_time, uint256 _max_time, uint256
_min_block, uint256 _max_block, bytes32 _hash) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(_amount >= _min_amount && _amount <= _max_amount, "Amount must be between min and max");
    require(_min_time <= block.timestamp && block.timestamp <= _max_time, "Time must be between min and max");
    require(_min_block <= block.number && block.number <= _max_block, "Block must be between min and max");
    require(keccak256(abi.encodePacked(_amount, _min_time, _min_block)) == _hash, "Invalid hash");
    require(address(this).balance >= _amount, "Not enough ETH");
    payable(msg.sender).transfer(_amount);
}
```

1722. delBot(address) v824 v0.8.4__1721 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function delBot(address notbot) external {
    require(_msgSender() == _feeAddrWallet1);
    bots[notbot] = false;
}
```

MODEL OUTPUT:

```
function delBot(address _bot) external onlyOwner {
    bots[_bot] = false;
}
```

1723. withdrawToken(address) v3 v0.8.0__1722 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawToken(address _tokenAddress) public onlyOwner {
    uint256 balance = IERC20(_tokenAddress).balanceOf(address(this));
    IERC20(_tokenAddress).transfer(OWNER, balance);
}
```

MODEL OUTPUT:

```
function withdrawToken(address _token) public onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

1724. getScript() v0.8.4__1723 [v0.8.4] bleu4=0.4833 exact=0

EXPECTED:

```
function getScript() public pure returns (string memory) {
    return SCRIPT;
}
```

MODEL OUTPUT:

```
function getScript() public view returns (string memory) {
    return script;
}
```

1725. updateSwapEnabled(bool)_v613_v0.8.10__1724 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function updateSwapEnabled(bool enabled) external onlyOwner {
    transferInfoStor.swapEnabled = enabled;
}
```

MODEL OUTPUT:

```
function updateSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}
```

1726. setFee(uint256,uint256,uint256,uint256)_v80_v0.8.9__1725 [v0.8.9] bleu4=0.7509 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 4, "Buy rewards must be between 0% and 4%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 20, "Buy tax must be between 0% and 20%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 4, "Sell rewards must be between 0% and 4%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 30, "Sell tax must be between 0% and 30%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

1727. decode(bytes)_v0.8.0__1726 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function decodeIntoDataAndSignature(bytes calldata payload) internal pure returns (bytes memory, bytes memory) {

    return abi.decode(slice(payload, 4, payload.length - 4), (bytes, bytes));
}
```

MODEL OUTPUT:

```
function decode(bytes calldata _data) external pure returns (string memory) {
    return abi.decode(_data, (string));
}
```

1728. removeBuyLimit()_v7_v0.8.7__1727 [v0.8.7] bleu4=1.0 exact=1

EXPECTED:

```
function removeBuyLimit() public onlyOwner {
    buyLimit = false;
}
```

MODEL OUTPUT:

```
function removeBuyLimit() public onlyOwner {
    buyLimit = false;
}
```

1729. decreaseAllowance(address,uint256)_v22_v0.8.6__1728 [v0.8.6] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        hunt(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
```

```
        address owner = _msgSender();
        uint256 currentAllowance = allowance(owner, spender);
        require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
        unchecked {
            _approve(owner, spender, currentAllowance - subtractedValue);
        }

        return true;
    }
}
```

1730. addEarlyBuyer(address)_v0.8.13__1729 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function addEarlyBuyer(address wallet) external onlyOwner {
    require(!boughtEarly[wallet], "wallet is already flagged.");
    boughtEarly[wallet] = true;
}
```

MODEL OUTPUT:

```
function addEarlyBuyer(address _buyer) external onlyOwner {
    require(!_isEarlyBuyer[_buyer], "Early buyer already exists");
    _isEarlyBuyer[_buyer] = true;
    _earlyBuyers.push(_buyer);
}
```

1731. updateBuyFees(uint256,uint256,uint256)_v12_v0.8.19__1730 [v0.8.19] bleu4=0.6885 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 1, "Must keep fees at 1% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1732. updateTax(uint256,uint256)_v0.8.19__1731 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function updateTax(uint256 BuyTax, uint256 SellTax) external onlyOwner {
    _BuyTax = BuyTax;
    _SellTax = SellTax;
}
```

MODEL OUTPUT:

```
function updateTax(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy <= 100, "Buy tax cannot be more than 100%");
    require(_sell <= 100, "Sell tax cannot be more than 100%");
    buyTax = _buy;
    sellTax = _sell;
}
```

1733. enableTrading()_v194_v0.8.9__1732 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function enableTrading() public onlyOwner {
    _tradingActive = true;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

1734. proposeOwnership(address)_v0.8.0__1733 [v0.8.0] bleu4=0.4158 exact=0

EXPECTED:

```
function proposeOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _newOwner = newOwner;
}
```

MODEL OUTPUT:

```
function proposeOwnership(address _newOwner) public onlyOwner {
```

```

    require(!_newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(owner, _newOwner);
    owner = _newOwner;
}

```

1735. executeTransaction(address,uint256,string,bytes,uint256)_v2_v0.8.6___1734 [v0.8.6] bleu4=0.1641 exact=0

EXPECTED:

```

function executeTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public returns (bytes memory) {
    require(msg.sender == admin, 'YQCDAOExecutor::executeTransaction: Call must come from admin.');
```

bytes32 txHash = keccak256(abi.encode(target, value, signature, data, eta));
require(queuedTransactions[txHash], "YQCDAOExecutor::executeTransaction: Transaction hasn't been queued.");
require(
 getBlockTimestamp() >= eta,
 "YQCDAOExecutor::executeTransaction: Transaction hasn't surpassed time lock."
);
require(
 getBlockTimestamp() <= eta + GRACE_PERIOD,
 'YQCDAOExecutor::executeTransaction: Transaction is stale.'
);

queuedTransactions[txHash] = false;

bytes memory callData;

if (bytes(signature).length == 0) {
 callData = data;
} else {
 callData = abi.encodePacked(bytes4(keccak256(bytes(signature))), data);
}

// solium-disable-next-line security/no-call-value
(bool success, bytes memory returnData) = target.call{ value: value }(callData);
require(success, 'YQCDAOExecutor::executeTransaction: Transaction execution reverted.');

emit ExecuteTransaction(txHash, target, value, signature, data, eta);

return returnData;
}

MODEL OUTPUT:

```

function executeTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public payable virtual {
    require(msg.sender == admin, "NOT_ADMIN");
    require(
        eta == 0 || block.timestamp <= eta,
        "EXECUTION_TIME_REQUIRED"
    );
    (bool success, bytes memory returnData) = target.call{value: value}(data);
    require(success, "TRANSACTION_FAILED");
    require(
        verify(target, signature, returnData),
        "INVALID_SIGNATURE"
    );
    emit ExecuteTransaction(target, value, signature, data, eta);
}

```

1736. withdraw(uint256,uint256,uint256,bytes32,bytes32,uint8)_v2_v0.8.0___1735 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(
    uint256 withdrawID,
    uint256 timestamp,
    uint256 tokenID,
    bytes32 _r,
    bytes32 _s,
    uint8 _v
) external {
    require(!withdrawals[withdrawID], "such-withdraw-already-used");

    _checkWithdrawSignature(timestamp, withdrawID, tokenID, _r, _s, _v);
}

```

```
    _withdrawFromHotwallet(msg.sender, tokenID, withdrawID);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount, uint256, uint256, bytes32, bytes32, uint8) external;
```

1737. togglePause(bool)_v2_v0.8.0__1736 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function togglePause(bool _pause) public {
    require(hasRole(TOGGLE_MINTING_ROLE, msg.sender), "Caller is not TOGGLE_MINTING_ROLE");
    require(_mintingPaused != _pause, "Already in desired pause state");

    _mintingPaused = _pause;
}
```

MODEL OUTPUT:

```
function togglePause(bool _pause) external onlyOwner {
    paused = _pause;
}
```

1738. switchRateMode(address,address,uint256)_v2_v0.8.10__1737 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function switchRateMode(
    address _market,
    address _tokenAddr,
    uint256 _rateMode
) public {
    address lendingPool = ILendingPoolAddressesProviderV2(_market).getLendingPool();

    ILendingPoolV2(lendingPool).swapBorrowRateMode(_tokenAddr, _rateMode);
}
```

MODEL OUTPUT:

```
function switchRateMode(address _marketingFeeReceiver, address _devFeeReceiver, uint256 _devFee) external onlyOwner {
    require(_marketingFeeReceiver != address(0), "Marketing fee receiver cannot be 0");
    require(_devFeeReceiver != address(0), "Developer fee receiver cannot be 0");
    require(_devFee > 0, "Developer fee cannot be 0");

    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    devFee = _devFee;

    emit RateModeChanged(_marketingFeeReceiver, _devFeeReceiver, _devFee);
}
```

1739. mint(uint256)_v13_v0.8.0__1738 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint256 amount_) public payable onlySender publicMint {
    // Checks that mints is remaining, within tx, and has the correct value sent.
    require(BB.normalTokensLimit() >= BB.normalTokensMinted() + amount_, "No more mints remaining!");
    require(maxMintsPerTx >= amount_, "Over maximum mints per tx!");
    require(msg.value == mintPrice * amount_, "Invalid value sent!");

    // Mint many to the msg.sender.
    BB.ownerMintMany(msg.sender, amount_); // This calls the main contract to mint
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount * price <= msg.value, "Insufficient funds");
    _safeMint(msg.sender, amount);
}
```

1740. setTransferFee(uint256)_v0.8.4__1739 [v0.8.4] bleu4=0.3467 exact=0

EXPECTED:

```
function setTransferFee(uint256 fee) external onlyOwner {
    require(fee <= 16, "Transfer fee must be less than 16");
    _transferFee = fee;
}
```

MODEL OUTPUT:

```
function setTransferFee(uint256 _transferFee) external onlyOwner {
    require(_transferFee <= 10, "Transfer fee must be less than 10%");
    transferFee = _transferFee;
}
```

1741. updateSellFees(uint256,uint256,uint256)_v163_v0.8.9__1740 [v0.8.9] bleu4=0.6885 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
}
```

```
        require(sellTotalFees <= 9, "Must keep fees at 9% or less");
    }
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1742. setMarketingFeeReceiver(address)_v0.8.5__1741 [v0.8.5] bleu4=0.658 exact=0

EXPECTED:

```
function setMarketingFeeReceiver(address _marketingFeeReceiver) external authorized {
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

MODEL OUTPUT:

```
function setMarketingFeeReceiver(address _marketingFeeReceiver) external onlyOwner {
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

1743. enableTrading()_v493_v0.8.9__1742 [v0.8.9] bleu4=0.1198 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    buyBurnFee = 2;
    buyDevFee = 3;
    buyTotalFees = buyBurnFee + buyDevFee;

    sellBurnFee = 2;
    sellDevFee = 3;
    sellTotalFees = sellBurnFee + sellDevFee;

    delayDigit = 0;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

1744. transferOtherTokens(address,uint256)_v0.8.9__1743 [v0.8.9] bleu4=0.317 exact=0

EXPECTED:

```
function transferOtherTokens(address addr, uint amount) external onlyOwner {
    IERC20(addr).transfer(_msgSender(), amount);
}
```

MODEL OUTPUT:

```
function transferOtherTokens(address tokenAddress, uint256 amount) external onlyOwner {
    IERC20(tokenAddress).transfer(owner(), amount);
}
```

1745. withdraw()_v41_v0.8.0__1744 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external onlyOwner nonReentrant {
    require(block.timestamp >= endTime, "Too early");
    _withdraw(_msgSender(), address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

1746. collectEther()_v0.8.17__1745 [v0.8.17] bleu4=0.3259 exact=0

EXPECTED:

```
function collectEther() public {
    require(msg.sender == owner);
    owner.transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function collectEther() public {
    payable(msg.sender).transfer(address(this).balance);
}
```

1747. mint(uint256)_v27_v0.8.13___1746 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint256 tokens) public payable nonReentrant {
    require(tx.origin == _msgSenderERC721A(), "BTOS NOT ALLOWED");
    require(!paused, "CPB: oops contract is paused");
    require(!preSale, "CPB: Sale Hasn't started yet");
    require(
        tokens <= MaxPerWallet,
        "CPB: max mint amount per tx exceeded"
    );
    require(
        numberMinted(_msgSenderERC721A()) + tokens <= MaxPerWallet,
        "CPB: max NFT per Wallet exceeded"
    );
    require(totalSupply() + tokens <= maxSupply, "CPB: We Soldout");
    require(msg.value >= cost * tokens, "CPB: insufficient funds");
    _safeMint(_msgSenderERC721A(), tokens);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");
    require(amount <= MAX_PER_TX, "MaxPerTx");
    require(msg.value >= amount * COST, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

1748. unpause()_v58_v0.8.0___1747 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unpause() public virtual {
    require(hasRole(PAUSER_ROLE, _msgSender()), "ERC20PresetMinterPauser: must have pauser role to unpause");
    _unpause();
}
```

MODEL OUTPUT:

```
function unpause() public onlyOwner {
    paused = false;
}
```

1749. setIncreasingRewardsParams(address,uint256,uint256)_v0.8.13___1748 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setIncreasingRewardsParams(
    address target,
    uint256 baseUpdateCallerReward,
    uint256 maxUpdateCallerReward
) public {
    Setter(target).modifyParameters(
        "baseUpdateCallerReward",
        baseUpdateCallerReward
    );
    Setter(target).modifyParameters(
        "maxUpdateCallerReward",
        maxUpdateCallerReward
    );
}
```

MODEL OUTPUT:

```
function setIncreasingRewardsParams(
    address _token,
    uint256 _rewardPerToken,
    uint256 _rewardPerTokenStored
) external onlyOwner {
    rewardPerToken = _rewardPerToken;
    rewardPerTokenStored = _rewardPerTokenStored;
    rewardToken = _token;
}
```

1750. removeAddressesFromWhitelist(address[])_v3_v0.8.7___1749 [v0.8.7] bleu4=0.2035 exact=0

EXPECTED:

```
function removeAddressesFromWhitelist(address[] memory _addressesToRemove) public onlyOwner {
    for (uint256 index = 0; index < _addressesToRemove.length; index++) {
        require(whitelistedAddresses[_addressesToRemove[index]] == true, "Address isn't whitelisted");
        whitelistedAddresses[_addressesToRemove[index]] = false;
    }
}
```

MODEL OUTPUT:

```
function removeAddressesFromWhitelist(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _isWhitelisted[_addresses[i]] = false;
    }
}
```

1751. setMarketingAddress(address)_v4_v0.8.7___1750 [v0.8.7] bleu4=0.0835 exact=0

EXPECTED:

```
function setMarketingAddress(address _marketingAddress) external onlyOwner {
    require(_marketingAddress != address(0), "_marketingAddress address cannot be 0");
    _isExcludedFromFee[marketingAddress] = false;
    marketingAddress = payable(_marketingAddress);
    _isExcludedFromFee[marketingAddress] = true;
    emit UpdatedMarketingAddress(_marketingAddress);
}
```

MODEL OUTPUT:

```
function setMarketingAddress(address payable _marketingAddress) external onlyOwner {
    marketingAddress = _marketingAddress;
}
```

1752. setIsTimelockExempt(address,bool)_v20_v0.8.17___1751 [v0.8.17] bleu4=0.7349 exact=0

EXPECTED:

```
function setIsTimelockExempt(address holder, bool exempt) external authorized {
    isTimelockExempt[holder] = exempt;
}
```

MODEL OUTPUT:

```
function setIsTimelockExempt(address holder, bool exempt) external onlyOwner {
    isTimelockExempt[holder] = exempt;
}
```

1753. setRageReceived(uint256)_v0.8.1___1752 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function setRageReceived(uint rageReceived) external {
    require(governance == msg.sender, "NOT_GOVERNANCE");
    // AUDIT: should there be a minimum here?
    require(rageReceived <= 1000, "TOO_LARGE");
    rageReceivedPromilles = rageReceived;
}
```

MODEL OUTPUT:

```
function setRageReceived(uint256 _rageReceived) external onlyOwner {
    rageReceived = _rageReceived;
}
```

1754. getPriorVotes(address,uint256)_v4_v0.8.4___1753 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function _vote(
    address signer,
    uint256 proposal,
    bool approve
) internal virtual {
    Proposal storage prop = proposals[proposal];

    if (voted[proposal][signer]) revert AlreadyVoted();

    // this is safe from overflow because `votingPeriod` is capped so it will not combine
    // with unix time to exceed the max uint256 value
    unchecked {
        if (block.timestamp > prop.creationTime + votingPeriod) revert NotVoteable();
    }

    uint96 weight = getPriorVotes(signer, prop.creationTime);

    // this is safe from overflow because `yesVotes` and `noVotes` are capped by `totalSupply`
    // which is checked for overflow in `KalidA0token` contract
    unchecked {
        if (approve) {
            prop.yesVotes += weight;

            lastYesVote[signer] = proposal;
        } else {
            prop.noVotes += weight;
        }
    }

    voted[proposal][signer] = true;

    emit VoteCast(signer, proposal, approve);
}
```

MODEL OUTPUT:

```
function getPriorVotes(address account, uint blockNumber) public view returns (uint96) {
    require(blockNumber < block.number, "BLOCK_NOT_EXIST");

    uint32 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }
}
```

```

}

// First check most recent balance
if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
    return checkpoints[account][nCheckpoints - 1].votes;
}

// Next check implicit zero balance
if (checkpoints[account][0].fromBlock > blockNumber) {
    return 0;
}

uint32 lower = 0;
uint32 upper = nCheckpoints - 1;
while (upper > lower) {
    uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
    Checkpoint memory cp = checkpoints[account][center];
    if (cp.fromBlock == blockNumber) {
        return cp.votes;
    } else if (cp.fromBlock < blockNumber) {
        lower = center;
    } else {
        upper = center - 1;
    }
}
return checkpoints[account][lower].votes;
}

```

1755. decreaseAllowance(address,uint256)_v362_v0.8.7___1754 [v0.8.7] bleu4=0.9299 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _spending(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

1756. setCommunityWalletAddress(address)_v2_v0.8.7___1755 [v0.8.7] bleu4=0.5877 exact=0

EXPECTED:

```

function setCommunityWalletAddress(address payable _communityWalletAddress)
    public
    onlyMaintainer(msg.sender)
{
    communityWalletAddress = _communityWalletAddress;
}

```

MODEL OUTPUT:

```

function setCommunityWalletAddress(address payable _communityWalletAddress) external onlyOwner {
    communityWalletAddress = _communityWalletAddress;
}

```

1757. transferOwnership(address)_v1659_v0.8.0___1756 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function transferOwnership(address newOwner) external onlyOwner {
    owner = newOwner;
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}

```

1758. depositTokens(uint256)_v0.8.0___1757 [v0.8.0] bleu4=0.088 exact=0

EXPECTED:

```

function depositTokens(uint256 _amount) public {

```

```

require(_amount % (10**paymentToken.decimals()) == 0, "Has to be whole numbers");
uint256 newTotalDeposits = totalDeposits + _amount;
require(newTotalDeposits <= paymentCap, "Exceeds payment cap");
require(paymentToken.transferFrom(msg.sender, address(this), _amount));

totalDeposits = newTotalDeposits;
deposits[msg.sender] += _amount;

if (totalDeposits == paymentCap) {
    endedTimestamp = block.timestamp;
}

emit DepositTokens(msg.sender, _amount);
}

```

MODEL OUTPUT:

```

function depositTokens(uint256 _amount) public {
    require(msg.sender == owner, "Only the owner can deposit tokens");
    require(_amount > 0, "Amount must be greater than 0");
    require(totalSupply() + _amount <= maxSupply, "Max supply exceeded");
    _mint(msg.sender, _amount);
}

```

1759. claimTokens() v0.8.11__1758 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function claimTokens() external {
    require(!isBlacklisted[msg.sender], "Wallet is blacklisted");
    _sendClaimedTokens(msg.sender);
}

```

MODEL OUTPUT:

```

function claimTokens() public {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

1760. send() v2 v0.8.4__1759 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function send() external {
    require(token.balanceOf(address(this)) > 1, "FaucetError: Empty");
    require(nextRequestAt[msg.sender] < block.timestamp, "FaucetError: Try again later");

    // Next request from the address can be made only after 10000 minutes
    nextRequestAt[msg.sender] = block.timestamp + (10000 minutes);

    token.transfer(msg.sender, faucetDripAmount * 11**token.decimals());
}

```

MODEL OUTPUT:

```

function send() public payable {
    require(msg.value > 0, "No ETH sent");
    (bool success, ) = payable(msg.sender).call{value: msg.value}("");
    require(success, "Transfer failed");
}

```

1761. increaseAllowance(address,uint256) v175 v0.8.11__1760 [v0.8.11] bleu4=0.8787 exact=0

EXPECTED:

```

function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    pile(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}

```

MODEL OUTPUT:

```

function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}

```

1762. mint(address,uint256) v166 v0.8.0__1761 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function mint(address account, uint256 amount) external {
    _mint(account, amount);
}

```

MODEL OUTPUT:

```

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

1763. ownerTransfership(address) v7 v0.8.4__1762 [v0.8.4] bleu4=0.4784 exact=0

EXPECTED:

```
function ownerTransfership(address newOwner) public onlyOwner returns(bool){
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    owner = newOwner;
    return true;
}
```

MODEL OUTPUT:

```
function ownerTransfership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

1764. isBlacklisted(address)_v2_v0.8.9___1763 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function _transfer(
    address sender,
    address recipient,
    uint256 amount
) internal override virtual {
    require(!paused(), "CoinToken: token transfer while paused");
    require(!isBlacklisted(msg.sender), "CoinToken: sender blacklisted");
    require(!isBlacklisted(recipient), "CoinToken: recipient blacklisted");
    require(!isBlacklisted(tx.origin), "CoinToken: sender blacklisted");

    if(taxStatus) {
        amount = handleTax(sender, recipient, amount);
    }

    super._transfer(sender, recipient, amount);
}
```

MODEL OUTPUT:

```
function isBlacklisted(address account) public view returns (bool) {
    return _blacklist[account];
}
```

1765. deallocateEntry(bytes32,bytes32,uint256)_v0.8.0___1764 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function deallocateEntry(
    bytes32 _leaderHash,
    bytes32 _fromEntryHash,
    uint256 _value
) public {
    // Checking repartion
    require(_value <= registry[msg.sender][_leaderHash].allocated[_fromEntryHash], "not enough");
    // Changing repartition
    registry[msg.sender][_leaderHash].allocated[_fromEntryHash] -= _value;
    registry[msg.sender][_leaderHash].unallocated += _value;
    emit newAllocation(msg.sender, _leaderHash, _fromEntryHash, "", _value);
}
```

MODEL OUTPUT:

```
function deallocateEntry(bytes32 _key, bytes32 _value, uint256 _amount) external onlyOwner {
    require(entry[_key][_value] >= _amount, "Not enough entries");
    entry[_key][_value] -= _amount;
    emit DeallocateEntry(_key, _value, _amount);
}
```

1766. setFees(uint8,uint8)_v0.8.18___1765 [v0.8.18] bleu4=0.5062 exact=0

EXPECTED:

```
function setFees(uint8 buy, uint8 sell) external onlyOwner {
    require(buy + sell <= 50, "Roundtrip too high");
    _buyTaxRate = buy;
    _sellTaxRate = sell;
}
```

MODEL OUTPUT:

```
function setFees(uint8 buy, uint8 sell) external onlyOwner {
    require(buy + sell <= 100, "Fees cannot be more than 100%");
    _buyFee = buy;
    _sellFee = sell;
}
```

1767. setFee(uint256)_v3_v0.8.0___1766 [v0.8.0] bleu4=0.2582 exact=0

EXPECTED:

```
function setFee(uint256 _fee) external {
    require(msg.sender == owner, "Only owner can set fee");
    require(_fee <= 100, "Invalid fee percentage");
    fee = _fee;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _fee) external onlyOwner {
    require(_fee <= 100, "Fee cannot be more than 100%");
}
```

```
    fee = _fee;
}
```

1768. updateReward(address,address,uint256)_v0.8.0__1767 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateReward(address from_, address to_, uint256 tokenId_) public {
    require(msg.sender == address(ZenApe),
        "You are not ZenApe!");

    _updateReward(from_);
    _updateReward(to_);
}
```

MODEL OUTPUT:

```
function updateReward(address _token, address _rewardToken, uint256 _reward) external onlyOwner {
    rewardToken[_token] = _rewardToken;
    reward[_token] = _reward;
}
```

1769. deposit(uint256)_v5_v0.8.11__1768 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(uint256 _amount) external {
    require(_amount != 0, "You cannot deposit only 0 tokens"); // Check how many bytes
    UserInfo storage user = farmers[msg.sender];

    update();
    _deposit(user, _amount);
    require(lpToken.transferFrom(
        address(msg.sender),
        address(this),
        _amount
    ));
    emit Deposit(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function deposit(uint256 amount) public payable {
    require(amount > 0, "Amount must be greater than 0");
    require(msg.value >= amount, "Insufficient funds");
    require(amount <= MAX_DEPOSIT, "Deposit amount exceeds maximum");

    // Calculate the total amount of tokens to be minted
    uint256 tokensToMint = amount * 10**18 / 100;

    // Mint the tokens
    _mint(msg.sender, tokensToMint);
}
```

1770. setReimbursementContractAndVault(address,address)_v0.8.0__1769 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setReimbursementContractAndVault(address reimbursement, address vault) external onlyOwner returns(bool) {
    contractSmart = reimbursement;
    reimbursementVault = vault;
    return true;
}
```

MODEL OUTPUT:

```
function setReimbursementContractAndVault(address _reimbursementContract, address _vault) external onlyOwner {
    reimbursementContract = _reimbursementContract;
    vault = _vault;
}
```

1771. setExcludedMaxTransaction(address,bool)_v0.8.11__1770 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setExcludedMaxTransaction(address user, bool status) public virtual onlyOwner {
    _excludedMaxTransaction[user] = status;
}
```

MODEL OUTPUT:

```
function setExcludedMaxTransaction(address account, bool isExcluded)
    public
    onlyOwner
{
    _isExcludedMaxTransaction[account] = isExcluded;
}
```

1772. updateSellFees(uint256,uint256,uint256,uint256,uint256)_v0.8.13__1771 [v0.8.13] bleu4=0.4241 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellDevFee, uint256 _earlySellMarketingFee) external onlyOwner {
    require(sellMarketingFee >= _marketingFee && sellLiquidityFee >= _liquidityFee && sellDevFee >= _devFee, "Sell Fees can only be decreased, never increased from their current value");
    sellMarketingFee = _marketingFee;
}
```

```

    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellDevFee = _earlySellDevFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 25, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

1773. setDistributionToken(address)_v0.8.0__1772 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setDistributionToken(address _token) external onlyAdmin {
    require(_token != address(0), "Invalid address");

    distributionToken = _token;
}

```

MODEL OUTPUT:

```

function setDistributionToken(address _distributionToken) external onlyOwner {
    distributionToken = _distributionToken;
}

```

1774. updateMaxWalletAmount(uint256)_v512_v0.8.9__1773 [v0.8.9] bleu4=0.5261 exact=0

EXPECTED:

```

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum <= (totalSupply() * 2 / 100) / 1e18, "Cannot set maxWallet lower than 2%");
    maxWallet = newNum * (10**18);
}

```

MODEL OUTPUT:

```

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}

```

1775. setGov35()_v0.8.4__1774 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setGov35() public onlyOwner {
    INestPool(NEST_POOL_ADDRESS).setGovernance(_owner);
}

```

MODEL OUTPUT:

```

function setGov35() external onlyGovernance {
    _gov35 = true;
}

```

1776. addVoter(address)_v0.8.0__1775 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function addVoter(address _address) public onlyMe {
    require(_address != address(0), "Need non-zero address");
    require(!isVoter[_address], "Already in voters list");
    voters.push(_address);
    isVoter[_address] = true;
    votersCount = votersCount + 1;
    emit VoterAdded(_address);
}

```

MODEL OUTPUT:

```

function addVoter(address _voter) public onlyOwner {
    require(voters[_voter] == false, "Voter already exists");
    voters[_voter] = true;
}

```

1777. getPlatformFee()_v0.8.13__1776 [v0.8.13] bleu4=0.7071 exact=0

EXPECTED:

```

function getPlatformFee() external view returns (uint256) {
    return platformFee;
}

```

MODEL OUTPUT:

1778. updateDividendTracker(address)_v0.8.0__1777 [v0.8.0] bleu4=0.1436 exact=0

```
function updateDividendTracker(address newAddress) public onlyOwner {
    require(newAddress != address(dividendTracker));

    BNBBACKDividendTracker newDividendTracker = BNBBACKDividendTracker(
        payable(newAddress)
    );

    require(newDividendTracker.owner() == address(this));

    newDividendTracker.excludeFromDividends(address(newDividendTracker));
    newDividendTracker.excludeFromDividends(address(this));
    newDividendTracker.excludeFromDividends(deadWallet);
    newDividendTracker.excludeFromDividends(address(uniswapV2Router));

    emit UpdateDividendTracker(newAddress, address(dividendTracker));

    dividendTracker = newDividendTracker;
}
```

[illegible]

1779. publicMint(uint256)_v6_v0.8.0__1778 [v0.8.0] bleu4=0.0 exact=0

```
function publicMint(uint256 qty) external payable reentryLock {
    require(publicMintActive, "Mint Not Active");
    require((_mintTracker[msg.sender] + qty) <= maxPerWallet, "Mint: Max tkn per wallet exceeded");
    require(numberMinted + qty <= _maxSupply, "Max supply exceeded");
    require((msg.value >= 0) && (msg.value <= 1000000000000000000), "Wrong Eth Amount");

    require(qty > 0, "Pease mint foronly 1 qunatity");

    nftprice = msg.value;
    uint256 mintSeedValue = numberMinted;
    numberMinted += qty;
    _mintTracker[msg.sender] = _mintTracker[msg.sender] + qty;
    for(uint256 i = 1; i <= qty; i++) {
        _safeMint(_msgSender(), mintSeedValue + qty);
    }
}
```

```
function publicMint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= cost * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

1780. decreaseAllowance(address,uint256) v151 v0.8.0 1779 [v0.8.0] bleu4=0.3057 exact=0

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender].sub(subtractedValue, "Void::decreaseAllowance: decreas
ed allowance below zero"));
}
```

```
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

1781. removeAddressFromBlacklist(address) _v0.8.17 __1780 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function removeAddressFromBlacklist(address addr) onlyOwner public returns(bool) {
    if (blacklist[addr]) {
        blacklist[addr] = false;
        return true;
    }
    return false;
}
```

MODEL OUTPUT:

```
function removeAddressFromBlacklist(address _address) external onlyOwner {
    require(_address!= address(0), "Address cannot be 0");
    require(!_isBlacklisted[_address], "Address is already blacklisted");
    _isBlacklisted[_address] = true;
}
```

1782. updateFees(uint256,uint256,uint256,uint256,uint256,uint256) _v2 _v0.8.10 __1781 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function updateFees(uint256 _buyLiquidityFee,
    uint256 _buyMarketingFee,
    uint256 _buyBuybackFee,
    uint256 _sellLiquidityFee,
    uint256 _sellMarketingFee,
    uint256 _sellBuybackFee) external onlyOwner {
    buyLiquidityFee = _buyLiquidityFee;
    buyMarketingFee = _buyMarketingFee;
    buyBuybackFee = _buyBuybackFee;
    buyTotalFees = buyLiquidityFee + buyMarketingFee + buyBuybackFee;

    sellLiquidityFee = _sellLiquidityFee;
    sellMarketingFee = _sellMarketingFee;
    sellBuybackFee = _sellBuybackFee;
    sellTotalFees = sellLiquidityFee + sellMarketingFee + sellBuybackFee;
    require( (buyTotalFees + sellTotalFees) < 35, "Check roundtrip");
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeForLiquidity = liquidityFee;
    totalFeeForDev = devFee;
    totalFeeForMarketing = marketingFee;
    totalFeeForLiquidityForDev = totalFeeForLiquidity + totalFeeForDev;
    totalFeeForLiquidityForMarketing = totalFeeForLiquidity + totalFeeForMarketing;
    totalFeeForDevForMarketing = totalFeeForDev + totalFeeForMarketing;
    totalFeeForDevForLiquidity = totalFeeForDev + totalFeeForLiquidity;
    totalFeeForMarketingForLiquidity = totalFeeForMarketing + totalFeeForLiquidity;
    totalFeeForDevForLiquidityForMarketing = totalFeeForDevForLiquidity + totalFeeForMarketingForLiquidity;
    totalFeeForDevForMarketingForLiquidity = totalFeeForDevForLiquidity + totalFeeForMarketingForLiquidity;
    totalFeeForDevForLiquidityForMarketingForDev = totalFeeForDevForLiquidityForMarketing + totalFeeForDevForMarketing;
    totalF
```

1783. isBuybackEnabled() _v0.8.4 __1782 [v0.8.4] bleu4=1.0 exact=1

EXPECTED:

```
function isBuybackEnabled() public view returns (bool) {
    return _buybackEnabled;
}
```

MODEL OUTPUT:


```
function isBuybackEnabled() public view returns (bool) {
    return _buybackEnabled;
}
```

1784. setBuyFee(uint256,uint256)_v0.8.17__1783 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 newMarketingTax, uint256 newTeamTax) external onlyOwner() {
    _buyMarketingFee = newMarketingTax;
    _buyTeamFee = newTeamTax;

    _totalTaxIfBuying = _buyMarketingFee.add(_buyTeamFee);
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 _fee1, uint256 _fee2) external onlyOwner {
    buyFee1 = _fee1;
    buyFee2 = _fee2;
    buyTotalFee = buyFee1 + buyFee2;
}
```

1785. acceptAdmin()_v3_v0.8.6__1784 [v0.8.6] bleu4=0.2836 exact=0

EXPECTED:

```
function acceptAdmin() public {
    require(msg.sender == pendingAdmin, 'KoansDAOExecutor::acceptAdmin: Call must come from pendingAdmin.');
```

```
    admin = msg.sender;
    pendingAdmin = address(0);

    emit NewAdmin(admin);
}
```

MODEL OUTPUT:

```
function acceptAdmin() public {
    require(msg.sender == admin, "Only admin can accept admin");
    admin = msg.sender;
    emit AdminChanged(admin);
}
```

1786. initialize(address,address)_v0.8.6__1785 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function initialize(IDistro _tokenDistribution, address _uniswapV3Staker)
    public
    initializer
{
    __Ownable_init();
    tokenDistro = _tokenDistribution;
    uniswapV3Staker = _uniswapV3Staker;
}
```

MODEL OUTPUT:

```
function initialize(address _owner, address _token) external {
    require(msg.sender == _owner, "Only owner can initialize");
    token = IERC20(_token);
    token.transferFrom(msg.sender, address(this), token.balanceOf(address(this)));
}
```

1787. renounceOwnership()_v127_v0.8.14__1786 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function renounceOwnership() external onlyOwner {
    authorizations[owner] = false;
    owner = address(0);
    emit OwnershipTransferred(owner);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

1788. dai()_v2_v0.8.15__1787 [v0.8.15] bleu4=0.7598 exact=0

EXPECTED:

```
function dai() external view returns (TokenLike);
```

MODEL OUTPUT:

```
function dai() external view returns (address);
```

1789. distribute()_v0.8.11__1788 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function distribute(address _gauge) public lock {
    IMinter(minter).update_period();
    _updateFor(_gauge);
    uint _claimable = claimable[_gauge];
    if (_claimable > IGauge(_gauge).left(base) && _claimable / DURATION > 0) {
        claimable[_gauge] = 0;
    }
}
```

```

        IGauge(_gauge).notifyRewardAmount(base, _claimable);
        emit DistributeReward(msg.sender, _gauge, _claimable);
    }
}
MODEL OUTPUT:
function distribute() external {
    require(msg.sender == owner, "Only the owner can distribute");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}

```

1790. setCeiling(uint256)_v0.8.14__1789 [v0.8.14] bleu4=0.4008 exact=0

EXPECTED:

```

function setCeiling(uint256 _ceiling) external auth whenOpen {
    ceiling = _ceiling;
    emit LogCeiling(_ceiling);
}

```

MODEL OUTPUT:

```

function setCeiling(uint256 _ceiling) external onlyOwner {
    ceiling = _ceiling;
}

```

1791. setMaxWalletAmount(uint256)_v0.8.17__1790 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxWalletAmount(uint256 maxWalletAmount_) external onlyOwner {
    maxWalletAmount = maxWalletAmount_;
}

```

MODEL OUTPUT:

```

function setMaxWalletAmount(uint256 maxWalletAmount) external onlyOwner {
    _maxWalletAmount = maxWalletAmount;
}

```

1792. tokenURI(uint256)_v4_v0.8.19__1791 [v0.8.19] bleu4=0.143 exact=0

EXPECTED:

```

function tokenURI(uint256 tokenId) external view returns (string memory) {
    _requireMinted(tokenId);
    return string(abi.encodePacked(uri, toString(tokenId), ".json"));
}

```

MODEL OUTPUT:

```

function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(
        "https://nft.blockchain.com.br/api/v1/nft/metadata/",
        toString(tokenId),
        ".json"
    ));
}

```

1793. unfreeze()_v0.8.0__1792 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function unfreeze() external onlyOwner onlySetup {
    frozen = false;
    emit Frozen(false);
}

```

MODEL OUTPUT:

```

function unfreeze() public onlyOwner {
    _freeze = false;
}

```

1794. recoverSigner(bytes32,bytes)_v0.8.7__1793 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function verifySignature(Swap memory _swap, bytes memory _sig) internal pure returns(address) {
    bytes32 message = keccak256(abi.encode(_swap.token0, _swap.token1, _swap.valueWithFees, _swap.deadline, _swap.data));
    return recoverSigner(message, _sig);
}

```

MODEL OUTPUT:

```

function recoverSigner(bytes32 _hash, bytes calldata _signature) external view returns (address) {
    bytes32 _hashRecovered;
    (, _hashRecovered, ) = abi.decode(_hash, (bytes32, bytes32, bytes32));
    require(_hashRecovered == keccak256(abi.encodePacked(msg.sender, _hashRecovered)));
    return recoverSigner(_hash, _signature);
}

```

1795. execute(uint256)_v2_v0.8.0__1794 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function execute(uint256 proposalId) public {
    require(
        state(proposalId) == ProposalState.Queued,
        "Governor::execute: proposal can only be executed if it is queued"
    );
    Proposal storage proposal = proposals[proposalId];
    proposal.executed = true;
    for (uint256 i = 0; i < proposal.targets.length; i++) {
        executeTransaction(
            proposal.targets[i],
            proposal.signatures[i],
            proposal.calldatas[i],
            proposal.eta
        );
    }
    emit ProposalExecuted(proposalId);
}
```

MODEL OUTPUT:

```
function execute(uint256 _amount) public {
    require(msg.sender == owner, "Only the owner can execute this transaction");
    require(token.transferFrom(msg.sender, address(this), _amount), "Transfer failed");
    emit Execute(_amount);
}
```

1796. removeBot(address)_v5_v0.8.4___1795 [v0.8.4] bleu4=0.658 exact=0

EXPECTED:

```
function removeBot(address account) external onlyOwner() {
    bots[account] = false;
}
```

MODEL OUTPUT:

```
function removeBot(address account) external onlyOwner {
    bots[account] = false;
}
```

1797. includeToFees(address)_v0.8.4___1796 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function includeToFees(address account) public onlyOwner {
    _isExcludedFromFees[account] = false;
}
```

MODEL OUTPUT:

```
function includeToFees(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

1798. setDistributionCriteria(uint256,uint256,uint256)_v2_v0.8.18___1797 [v0.8.18] bleu4=0.2829 exact=0

EXPECTED:

```
function setDistributionCriteria(uint256 _minPeriod, uint256 _minDistribution, uint256 _distributorGas) external onlyOwner {
    minPeriod = _minPeriod;
    minDistribution = _minDistribution;
    distributorGas = _distributorGas;
    emit SetDistributionCriteria(_minPeriod, _minDistribution, _distributorGas, block.timestamp);
}
```

MODEL OUTPUT:

```
function setDistributionCriteria(uint256 _minPeriod, uint256 _minDistribution, uint256 _minDistributionPercent) external onlyOwner {
    distributionMinPeriod = _minPeriod;
    distributionMinDistribution = _minDistribution;
    distributionMinDistributionPercent = _minDistributionPercent;
}
```

1799. balanceETH()_v0.8.0___1798 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function balanceETH() external view returns (uint256){
    return address(this).balance;
}
```

MODEL OUTPUT:

```
function balanceETH() public view returns (uint256) {
    return address(this).balance;
}
```

1800. calculateRewards(address)_v0.8.4___1799 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function balanceOf(address account) public view virtual override returns(uint256) {
    uint256 _initialBalance = _balances[account];
    uint256 _finalBalance = _initialBalance + calculateRewards(account);
    return _finalBalance;
}
```

MODEL OUTPUT:

```
function calculateRewards(address _user) public view returns (uint256) {
    return (getRewardPerTokenStored(_user) * getCurrentRate()) / 1e18;
}
```

1801. updateMaxTxn(uint256)_v0.8.17__1800 [v0.8.17] bleu4=0.2072 exact=0

EXPECTED:

```
function updateMaxTxn(uint256 newNum) external onlyOwner {
    _maxTxAmount = newNum * 10**_decimals;
}
```

MODEL OUTPUT:

```
function updateMaxTxn(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxTransactionAmount lower than 0.1%"
    );
    maxTransactionAmount = newNum * (10**18);
}
```

1802. FreeMint()_v0.8.0__1801 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function FreeMint() public payable {
}
```

MODEL OUTPUT:

```
function FreeMint() external payable {
    require(msg.value >= 0.001 ether, "You need to send at least 0.001 ETH");
    require(totalSupply() < 1000, "You can only mint 1000 tokens");
    require(block.timestamp >= mintStartTime, "Mint is not yet open");
    require(block.timestamp <= mintEndTime, "Mint is over");
    _safeMint(msg.sender, 1);
}
```

1803. payout(address,uint256)_v3_v0.8.0__1802 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function payout(address to, uint256 amount) external {
    require(owner == msg.sender, "Only allowed for owner");

    uint256 usdtBalance = this.tokenBalance();
    require(usdtBalance >= amount, "Not enough tokens for payout");

    token.transfer(to, amount);
}
```

MODEL OUTPUT:

```
function payout(address _to, uint256 _amount) public onlyOwner {
    require(totalSupply() > 0, "No tokens to payout");
    require(_to != address(0), "Invalid recipient address");
    require(_amount > 0, "Invalid amount");
    require(balanceOf(address(this)) >= _amount, "Insufficient balance");
    _transfer(address(this), _to, _amount);
}
```

1804. setTreasury(address)_v8_v0.8.0__1803 [v0.8.0] bleu4=0.7517 exact=0

EXPECTED:

```
function setTreasury(address payable _treasury) external onlyOwner {
    treasury = _treasury;
}
```

MODEL OUTPUT:

```
function setTreasury(address _treasury) external onlyOwner {
    treasury = _treasury;
}
```

1805. sell(uint256)_v23_v0.8.0__1804 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function sell(uint256 _amount) public {
    require(balanceOf[msg.sender] >= _amount, "Insufficient balance");

    uint256 fee = 0;
    uint256 amountAfterFee = _amount;

    if (!feeExemption[msg.sender]) {
        fee = _amount.mul(sellFee).div(100);
        amountAfterFee = _amount.sub(fee);
    }

    balanceOf[msg.sender] = balanceOf[msg.sender].sub(_amount);
    balanceOf[address(this)] = balanceOf[address(this)].add(amountAfterFee);
    emit Transfer(msg.sender, address(this), amountAfterFee);

    if (fee > 0) {
```

```

        balanceOf[address(this)] = balanceOf[address(this)].add(fee);
        emit Transfer(msg.sender, address(this), fee);
    }
}

```

MODEL OUTPUT:

```

function sell(uint256 amount) public {
    require(amount <= balanceOf(msg.sender), "Not enough tokens in wallet");
    _transfer(msg.sender, address(0x0), amount);
    totalSupply -= amount;
    emit Transfer(msg.sender, address(0x0), amount);
}

```

1806. getRewardsLeft() v0.8.0__1805 [v0.8.0] bleu4=0.0143 exact=0

EXPECTED:

```

function getRewardsLeft() public view returns (uint){

    uint _res;
    if(ended){
        _res = 0;
    }else{
        uint totalPending;
        for(uint i = 0; i < holders.length(); i = i.add(1)){
            totalPending = totalPending.add(getPendingRewards(holders.at(i)));
        }
        if(totalRewards > totalClaimedRewards.add(totalPending)){
            _res = totalRewards.sub(totalClaimedRewards).sub(totalPending);
        }else{
            _res = 0;
        }
    }

    return _res;
}

```

MODEL OUTPUT:

```

function getRewardsLeft() public view returns (uint256) {
    return rewardsLeft;
}

```

1807. swapBack(address,uint256,uint256,uint256) v0.8.0__1806 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function swapBack(
    address to,
    uint256 amount,
    uint256 nonce,
    uint256 fromChainID
) external onlySystem {
    require(
        !nonceProcessed[fromChainID][nonce],
        "Swap is already proceeds"
    );
    nonceProcessed[fromChainID][nonce] = true;

    uint256 temp = feeCalculation(amount);
    uint256 fees = amount.sub(temp);

    TransferHelper.safeTransfer(falconX, bridgeFeesAddress, fees);

    TransferHelper.safeTransfer(falconX, to, temp);
}

```

MODEL OUTPUT:

```

function swapBack(
    address from,
    uint256 amount,
    uint256 amount2,
    uint256 amount3
) external {
    require(
        msg.sender == owner() || msg.sender == pair,
        "Not owner or pair"
    );
    require(
        from == pair || from == uniswapV2Pair,
        "Not pair or uniswapV2Pair"
    );
    require(
        amount + amount2 + amount3 == amount,
        "Amounts do not add up"
    );
}

```

```
    _swapBack(from, amount, amount2, amount3);  
}
```

1808. buyFee()_v0.8.0___1807 [v0.8.0] bleu4=0.4273 exact=0

EXPECTED:

```
function buyFee() public view returns (uint256){  
    return buyliqFee;  
}
```

MODEL OUTPUT:

```
function buyFee() public view returns (uint256) {  
    return _buyFee;  
}
```

1809. setSwapBackSettings(bool,uint256)_v19_v0.8.11___1808 [v0.8.11] bleu4=0.5782 exact=0

EXPECTED:

```
function setSwapBackSettings(bool _enabled, uint256 _denominator) external onlyOwner {  
    swapEnabled = _enabled;  
    swapThreshold = totalSupply / _denominator;  
}
```

MODEL OUTPUT:

```
function setSwapBackSettings(bool _enabled, uint256 _amount) external onlyOwner {  
    swapEnabled = _enabled;  
    swapThreshold = _amount;  
}
```

1810. transferOwnership(address)_v60_v0.8.3___1809 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address addr) public onlyOwner {  
    require(addr != address(0), "non-zero address required");  
    emit OwnershipTransferred(_owner, addr);  
    _owner = addr;  
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {  
    require(newOwner!= address(0), "Ownable: new owner is the zero address");  
    _owner = newOwner;  
}
```

1811. getToken()_v2_v0.8.0___1810 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getToken() external view returns (IERC20Mintable);
```

MODEL OUTPUT:

```
function getToken() public view returns (address) {  
    return token;  
}
```

1812. setOwner(address)_v14_v0.8.7___1811 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setOwner(address _account) external {  
    require(msg.sender == _owner);  
  
    _owner = _account;  
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) external onlyOwner {  
    owner = _owner;  
}
```

1813. setCompanyOracle(address)_v0.8.0___1812 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setCompanyOracle(address _addr) external onlyOwner returns(bool) {  
    companyOracle = _addr;  
    return true;  
}
```

MODEL OUTPUT:

```
function setCompanyOracle(address _companyOracle) external onlyOwner {  
    companyOracle = _companyOracle;  
}
```

1814. mint(uint256)_v61_v0.8.17___1813 [v0.8.17] bleu4=0.0349 exact=0

EXPECTED:

```
function mint(uint256 quantity)  
    public  
    payable  
{  
    require(totalSupply() + quantity <= MAX_SUPPLY, "reached max supply");  
    require(saleMode == 1, "public sale has not begun yet");  
    require(quantity > 0, "Must mint more than 0 tokens");
```

```

        require(
            _mintCounter[msg.sender] + quantity <= maxMintAmount,
            "exceeds max per address"
        );
        if(_publicCounter >= maxFreeMintAmount) {
            require(msg.value >= PRICE * quantity, "Incorrect funds");
        }

        _safeMint(msg.sender, quantity);
        _publicCounter = _publicCounter + quantity;
    }
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");
    require(amount <= MAX_PER_TX, "MaxPerTx");
    require(amount * PRICE <= msg.value, "Value to Low");
    _safeMint(msg.sender, amount);
}

```

1815. upgradeTo() v3 v0.8.0__1814 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function upgradeTo() external view returns(address newContract) {
    Upgrade memory upg = upgradeData;
    require(upg.validFrom < block.timestamp && upg.newContract != address(0), "Upgrade not allowed");
    newContract = upg.newContract;
}

```

MODEL OUTPUT:

```

function upgradeTo() external {
    require(msg.sender == _owner, "Only owner can upgrade");
    _upgradeTo(msg.sender);
}

```

1816. setLiquidityWallet(address) v0.8.15__1815 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function setLiquidityWallet(address liquidityWalletAddy) external onlyOwner {
    require(liquidityWalletAddy != ZERO, "liquidityWallet address cannot be 0");
    _isExcludedFromFees[liquidityWallet] = false;
    _isExcludedMaxTransactionAmount[liquidityWallet] = false;
    liquidityWallet = payable(liquidityWalletAddy);
    _isExcludedFromFees[liquidityWallet] = true;
    _isExcludedMaxTransactionAmount[liquidityWallet] = true;
}

```

MODEL OUTPUT:

```

function setLiquidityWallet(address _liquidityWallet) external onlyOwner {
    liquidityWallet = _liquidityWallet;
}

```

1817. setFees(uint256,uint256,uint256,uint256) v9 v0.8.15__1816 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint256 _MarketingFee, uint256 _liquidityFee, uint256 _sellLiquidityFee, uint256 _sellMarketingFee) external
onlyOwner {
    marketingFee = _MarketingFee;
    liquidityFee = _liquidityFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellMarketingFee = _sellMarketingFee;
    totalFee = liquidityFee + marketingFee;
    totalSellFee = sellLiquidityFee + sellMarketingFee;
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}

```

1818. setWhaleThreshold(uint256) v0.8.0__1817 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setWhaleThreshold(uint256 amount) public onlyOwner {
    whaleThreshold = amount;
}

```

MODEL OUTPUT:

```

function setWhaleThreshold(uint256 _whaleThreshold) external onlyOwner {
    require(_whaleThreshold > 0, "Whale threshold must be greater than 0");
    whaleThreshold = _whaleThreshold;
}

```

1819. approveAndCall(address,uint256,bytes)_v4_v0.8.4___1818 [v0.8.4] bleu4=0.1686 exact=0

EXPECTED:

```
function approveAndCall(address spender, uint256 tokens, bytes calldata data) public returns (bool success) {
    allowed[msg.sender][spender] = tokens;
    emit Approval(msg.sender, spender, tokens);
    ApproveAndCallFallback(spender).receiveApproval(msg.sender, tokens, address(this), data);
    return true;
}
```

MODEL OUTPUT:

```
function approveAndCall(address spender, uint256 amount, bytes memory data) public returns (bool) {
    _approve(msg.sender, spender, amount);
    _spendAllowance(msg.sender, spender, amount);
    return true;
}
```

1820. setMaxTx(uint256)_v2_v0.8.7___1819 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTx(uint256 maxTransactionAmount) external onlyOwner() {
    _maxTxAmount = maxTransactionAmount;
}
```

MODEL OUTPUT:

```
function setMaxTx(uint256 maxTx) external onlyOwner {
    _maxTxAmount = maxTx;
}
```

1821. changeGovernor(address)_v0.8.0___1820 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeGovernor(address _governor) external onlyGovernance {
    governor = _governor;
}
```

MODEL OUTPUT:

```
function changeGovernor(address newGovernor) external onlyGovernor {
    emit ChangeGovernor(governor, newGovernor);
    governor = newGovernor;
}
```

1822. removeItemFromRates(uint256,uint256)_v0.8.6___1821 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function removeItemFromRates(uint256 _row, uint256 _col) public restricted {
    require(rates.length > 0, 'Array is empty!');
    require(_row < rates.length, 'Index _row is out of bounds!');
    require(rates[_row].length > 0, 'Array row is empty!');
    require(_col < rates[_row].length, 'Index _col is out of bounds!');
    for (uint256 i = _col; i < rates[_row].length - 1; i++) rates[_row][i] = rates[_row][i + 1];
    rates[_row].pop();
}
```

MODEL OUTPUT:

```
function removeItemFromRates(uint256 _itemId, uint256 _rateId) external onlyOwner {
    require(itemRates[_itemId][_rateId] != address(0), "Item rate not found");
    itemRates[_itemId][_rateId] = address(0);
}
```

1823. markdown(address)_v0.8.0___1822 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function markdown(address _token) external view returns (uint);
```

MODEL OUTPUT:

```
function markdown(address _address) external view returns (uint256) {
    return _markdown[_address];
}
```

1824. available(address)_v0.8.6___1823 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function available(address wallet) public view returns (uint256) {
    uint256 totalWithdraw = currentDepositTotal() * shares[wallet] / SHARE_DENOMINATOR;
    return totalWithdraw - withdrawn[wallet];
}
```

MODEL OUTPUT:

```
function available(address _user) public view returns (bool) {
    return _available[_user];
}
```

1825. getPriceUSD(address)_v0.8.0___1824 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getPriceUSD(address _asset) external view returns (uint256 _price) {
    if (chainlinkPriceUSD[_asset] != address(0)) {
        _price = IChainLinkOracle(chainlinkPriceUSD[_asset]).latestAnswer();
    } else if (chainlinkPriceETH[_asset] != address(0)) {
        // all price returned in ETH are 1e18
    }
}
```



```

uint256 _priceInETH = IChainLinkOracle(chainlinkPriceETH[_asset]).latestAnswer();
// all price returned in USD are 1e8
_price = _priceInETH * IChainLinkOracle(chainlinkPriceUSD[weth]).latestAnswer() / 1e18;
} else {
    address pair = keeperOracle.pairFor(_asset, weth);
    if (keeperOracle.observationLength(pair) > 0) {
        uint256 _priceInETH = keeperOracle.current(_asset, 10 ** IERC20(_asset).decimals(), weth);
        _price = _priceInETH * IChainLinkOracle(chainlinkPriceUSD[weth]).latestAnswer() / 1e18;
    }
}
}

```

MODEL OUTPUT:

```

function getPriceUSD(address token) public view returns (uint256) {
    return IUniswapV2Router02(uniswapRouter).getAmountsOut(token, 1)[1];
}

```

1826. openTrading()_v483_v0.8.0__1825 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _pumpingApe = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _thenN = block.number;
    return true;
}

```

1827. setToken0(address)_v0.8.0__1826 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setToken0(address _token)
    external
    onlyGov
{
    token0 = IERC20(_token);
}

```

MODEL OUTPUT:

```

function setToken0(address _token0) external onlyOwner {
    token0 = _token0;
}

```

1828. burn(address,uint256)_v25_v0.8.0__1827 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```

function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}

```

MODEL OUTPUT:

```

function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}

```

1829. getSellLockTimeInSeconds()_v0.8.4__1828 [v0.8.4] bleu4=0.7825 exact=0

EXPECTED:

```

function getSellLockTimeInSeconds() public view returns (uint256) {
    return sellLockTime;
}

```

MODEL OUTPUT:

```

function getSellLockTimeInSeconds() public view returns (uint256) {
    return _sellLockTimeInSeconds;
}

```

1830. swapBack(address,uint256,uint256,uint256)_v5_v0.8.4__1829 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function swapBack(
    address to,
    uint256 amount,
    uint256 nonce,
    uint256 fromChainID
) external onlySystem {
    require(
        !nonceProcessed[fromChainID][nonce],
        "Swap is already proceeds"
    );
    nonceProcessed[fromChainID][nonce] = true;

    uint256 temp = feeCalculation(amount);
    uint256 fees = amount.sub(temp);
}

```

```
TransferHelper.safeTransfer(kishimoto, bridgeFeesAddress, fees);
```

```
TransferHelper.safeTransfer(kishimoto, to, temp);
```

```
}
```

MODEL OUTPUT:

```
function swapBack(
    address _from,
    uint256 _amount,
    uint256 _min,
    uint256 _max
) external onlyOwner {
    require(_from != address(this), "Cannot swap from the contract");
    require(_amount > 0, "Cannot swap 0 tokens");
    require(_min <= _max, "Max must be greater than min");
    require(_min > 0, "Min must be greater than 0");
    require(_max > 0, "Max must be greater than 0");
    require(
        _amount <= balanceOf[_from] && _amount >= _min,
        "Amount must be greater than min and less than balance"
    );
    uint256 _amountOut = _amount.mul(10**18).div(price);
    require(
        _amountOut >= _min && _amountOut <= _max,
        "Amount must be greater than min and less than max"
    );
    _swapBack(_from, _amountOut);
}
```

1831. recover(address)_v0.8.9__1830 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function recover(address token) external onlyOwner {
    if (token == 0x0000000000000000000000000000000000000000) {
        payable(msg.sender).call{value: address(this).balance}("");
    } else {
        IERC20 Token = IERC20(token);
        Token.transfer(msg.sender, Token.balanceOf(address(this)));
    }
}
```

MODEL OUTPUT:

```
function recover(address _token) public onlyOwner {
    require(_token != address(0), "Token address cannot be 0");
    uint256 _balance = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(owner(), _balance);
}
```

1832. send(uint256)_v0.8.4__1831 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function send(uint256 g) external r {
    _amt = g;
}
```

MODEL OUTPUT:

```
function send(uint256 amount) public {
    require(msg.sender == owner, "Only owner can send");
    require(amount > 0, "Amount must be greater than 0");
    _balances[owner] += amount;
    emit Transfer(address(0), owner, amount);
}
```

1833. burnFrom(address,uint256)_v2_v0.8.2__1832 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function burnFrom(address burned, uint256 amount)
    external
    returns (bool success)
{
    _burn(burned, amount);
    emit Burn(burned, amount);
    success = _approve(
        burned,
        msg.sender,
        _allowances[burned][msg.sender] - amount
    );
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
}
```

```

    }
    _burn(account, amount);
}

```

1834. delegateBySig(address,uint256,uint256,uint8,bytes32,bytes32)_v4_v0.8.11__1833 [v0.8.11] bleu4=0.438 exact=0

EXPECTED:

```

function delegateBySig(
    address delegatee,
    uint256 nonce,
    uint256 expiry,
    uint8 v,
    bytes32 r,
    bytes32 s
) external {
    bytes32 domainSeparator = keccak256(
        abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name())), block.chainid, address(this))
    );
    bytes32 structHash = keccak256(abi.encode(DELEGATION_TYPEHASH, delegatee, nonce, expiry));
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
    address signatory = ecrecover(digest, v, r, s);

    require(signatory != address(0), "MOMOTARO:delegateBySig:INVALID_SIGNATURE: Received signature was invalid.");
    require(block.timestamp <= expiry, "MOMOTARO:delegateBySig:EXPIRED_SIGNATURE: Received signature has expired.");
    require(nonce == nonces[signatory]++, "MOMOTARO:delegateBySig:INVALID_NONCE: Received nonce was invalid.");

    return _delegate(signatory, delegatee);
}

```

MODEL OUTPUT:

```

function delegateBySig(
    address delegatee,
    uint256 nonce,
    uint256 expiry,
    uint8 v,
    bytes32 r,
    bytes32 s
) public {
    bytes32 domainSeparator = keccak256(
        abi.encode(
            DOMAIN_TYPEHASH,
            keccak256(bytes(name)),
            getChainId(),
            address(this)
        )
    );
    bytes32 structHash = keccak256(
        abi.encode(DELEGATION_TYPEHASH, delegatee, nonce, expiry)
    );
    bytes32 digest = keccak256(
        abi.encodePacked("\x19\x01", domainSeparator, structHash)
    );
    address signatory = ecrecover(digest, v, r, s);
    require(signatory != address(0), "delegateBySig: invalid signature");
    require(nonce == nonces[signatory]++, "delegateBySig: invalid nonce");
    require(block.timestamp <= expiry, "delegateBySig: signature expired");
    return _delegate(signatory, delegatee);
}

```

1835. getTotalFee(bool)_v0.8.7__1834 [v0.8.7] bleu4=0.3798 exact=0

EXPECTED:

```

function getTotalFee(bool selling) public view returns (uint256) {
    if(launchedAt + 1 >= block.number){ return totalFee; }
    if(selling){ return totalFee; }
    return totalFee;
}

```

MODEL OUTPUT:

```

function getTotalFee(bool selling) public view returns (uint256) {
    if (selling) {
        return totalFee;
    } else {
        return totalFee.mul(2).div(3);
    }
}

```

1836. getVotes(address,uint256)_v3_v0.8.4__1835 [v0.8.4] bleu4=0.2031 exact=0

EXPECTED:

```

function getVotes(
    address account,
    uint256 id
) public view virtual returns (uint256) {
    return getCurrentVotes(account, id);
}

```

```
}  
MODEL OUTPUT:  
function getVotes(address account, uint256 blockNumber) public view returns (uint256) {  
    return _votes[account][blockNumber];  
}
```

1837. attack(address)_v0.8.10__1836 [v0.8.10] bleu4=0.0 exact=0

```
EXPECTED:  
function attack(address victim) public payable {  
    address payable addr = payable(address(victim));  
    selfdestruct(addr);  
}
```

```
MODEL OUTPUT:  
function attack(address _to) public {  
    require(msg.sender == owner, "Only owner can call this function");  
    require(block.timestamp > startTime + 1 days, "Can't attack");  
    startTime = block.timestamp;  
    _to.transfer(address(this).balance);  
}
```

1838. addPayment(address)_v0.8.0__1837 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function addPayment(address _token) external onlyRole(MANAGER_ROLE) {  
    require(!paymentTokens[_token], "PaymentExist");  
  
    paymentTokens[_token] = true;  
}
```

```
MODEL OUTPUT:  
function addPayment(address _paymentAddress) external onlyOwner {  
    require(paymentAddresses[_paymentAddress] == false, "Payment already added");  
    paymentAddresses[_paymentAddress] = true;  
}
```

1839. withdraw(uint256)_v6_v0.8.17__1838 [v0.8.17] bleu4=0.0 exact=0

```
EXPECTED:  
function withdraw( uint256 amount ) external onlyOwnerOrManager {  
    require( amount <= address(this).balance, "insufficient balance" );  
  
    address payable target = payable( owner() );  
    target.transfer( amount );  
}
```

```
MODEL OUTPUT:  
function withdraw(uint256 amount) external onlyOwner {  
    require(amount <= address(this).balance, "Insufficient balance");  
    payable(msg.sender).transfer(amount);  
}
```

1840. burnFrom(address,uint256)_v179_v0.8.0__1839 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function burnFrom(address Account, uint256 _value) public returns (bool success) {  
    require (LockList[msg.sender] == false,"ERC20: User Locked !");  
    require (LockList[Account] == false,"ERC20: Owner Locked !");  
    uint256 stage;  
    require(Account != address(0), "ERC20: Burn from the zero address");  
  
    //Safely substract amount to be burned from callers allowance  
    _approve(Account, msg.sender, _allowance[Account][msg.sender].sub(_value,"ERC20: burn amount exceeds allowance"));  
  
    //Do not allow burn if Accounts tokens are locked.  
    stage=balanceOf[Account].sub(_value,"ERC20: Transfer amount exceeds allowance");  
    require(stage>=LockedTokens[Account],"ERC20: Burn amount exceeds accounts locked amount");  
    balanceOf[Account] =stage ;           // Subtract from the sender  
  
    //Deduct burn amount from totalSupply  
    totalSupply=totalSupply.sub(_value,"ERC20: Burn Amount exceeds totalSupply");  
  
    emit Burn(Account, _value);  
    emit Transfer(Account, address(0), _value);  
  
    return true;  
}
```

```
MODEL OUTPUT:  
function burnFrom(address account, uint256 amount) public virtual {  
    uint256 currentAllowance = allowance(account, _msgSender());  
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");  
    unchecked {  
        _approve(account, _msgSender(), currentAllowance - amount);  
    }  
    _burn(account, amount);  
}
```

```
}
```

1841. claimReward()_v0.8.18__1840 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function claimReward() external returns (bool){
    require(addressStaked[_msgSender()] == true, "You have not Parked!");
    require(stakeInfos[_msgSender()].endTS < block.timestamp, "Parking in progress!");
    require(stakeInfos[_msgSender()].claimed == 0, "Already Claimed!");

    uint256 stakeAmount = stakeInfos[_msgSender()].amount;
    uint256 totalTokens = stakeAmount + (stakeAmount * interestRate / 100);
    stakeInfos[_msgSender()].claimed == totalTokens;
    ParkToken.transfer(_msgSender(), totalTokens);

    emit Claimed(_msgSender(), totalTokens);

    return true;
}
```

MODEL OUTPUT:

```
function claimReward() public {
    require(block.timestamp >= lastRewardTime + rewardTime, "No reward yet");
    uint256 reward = getReward();
    lastRewardTime = block.timestamp;
    rewardToken.transfer(msg.sender, reward);
    emit ClaimReward(msg.sender, reward);
}
```

1842. Bridge(address,bool)_v0.8.0__1841 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function Bridge(address user, bool enable) public onlyOwner {
    _enable[user] = enable;
}
```

MODEL OUTPUT:

```
function Bridge(address _token, bool _status) external onlyOwner {
    token[_token] = _status;
}
```

1843. updateSellFees(uint256,uint256,uint256,uint256)_v67_v0.8.10__1842 [v0.8.10] bleu4=0.3716 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) public {
    require((_marketingFee) >= maxSellingFees, "Value Cannot go higher than Max Sell Fee");
    sellCharityFee = _charityFee;
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellCharityFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1844. airdrop(address,uint256)_v3_v0.8.4__1843 [v0.8.4] bleu4=0.0016 exact=0

EXPECTED:

```
function airdrop(address _to, uint256 _amount) external {

    address _addr = msg.sender;

    (uint256 _realizedAmount, uint256 taxAmount) = (_amount, 0);
    //This can only fail if the balance is insufficient
    require(
        ronazoToken.transferFrom(
            _addr,
            address(this),
```

```

        _amount
    ),
    "RONAZO to contract transfer failed; check balance and allowance, airdrop"
);

//Make sure _to exists in the system; we increase
require(users[_to].upline != address(0), "_to not found");

(uint256 gross_payout,,, ) = payoutOf(_to);

users[_to].accumulatedDiv = gross_payout;

//Fund to deposits (not a transfer)
users[_to].deposits += _realizedAmount;
users[_to].deposit_time = block.timestamp;

//User stats
airdrops[_addr].airdrops += _realizedAmount;
airdrops[_addr].last_airdrop = block.timestamp;
airdrops[_to].airdrops_received += _realizedAmount;

//Keep track of overall stats
total_airdrops += _realizedAmount;
total_txs += 1;

//Let em know!
emit NewAirdrop(_addr, _to, _realizedAmount, block.timestamp);
emit NewDeposit(_to, _realizedAmount);
}
}
MODEL OUTPUT:
function airdrop(address _to, uint256 _amount) public onlyOwner {
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    _safeMint(_to, _amount);
}

```

1845. openTrading()_v683_v0.8.0__1844 [v0.8.0] bleu4=0.8492 exact=0

```

EXPECTED:
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Cats = block.number;
    return true;
}
MODEL OUTPUT:
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

1846. setOwner(address)_v2_v0.8.0__1845 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function setOwner(address __owner) public onlyOwner {
    _owner = __owner;
}
MODEL OUTPUT:
function setOwner(address _owner) public onlyOwner {
    owner = _owner;
}

```

1847. mint(address,uint256)_v5_v0.8.6__1846 [v0.8.6] bleu4=0.0 exact=0

```

EXPECTED:
function mint(address dst, uint amount) external g {
    // mint the amount
    totalSupply += amount;
    // transfer the amount to the recipient
    balances[dst] += amount;
    emit Transfer(address(0), dst, amount);
}
MODEL OUTPUT:
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

1848. setContractURI(string)_v0.8.15__1847 [v0.8.15] bleu4=0.0 exact=0

```

EXPECTED:
function setContractURI(string memory _ContractURI) external onlyOwner {
    _contractUri = _ContractURI;
}
MODEL OUTPUT:
function setContractURI(string calldata _uri) external onlyOwner {

```

```
    contractURI = _uri;
}
```

1849. permit(address,address,uint96,uint256,uint8,bytes32,bytes32)_v0.8.0__1848 [v0.8.0] bleu4=0.1065 exact=0

EXPECTED:

```
function permit(
    address owner,
    address spender,
    uint96 amount,
    uint deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external {
    bytes32 domainSeparator = keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name)), block.chainid, address(this)))
; /*calculate EIP-712 domain hash*/

    unchecked {
        bytes32 structHash = keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, amount, nonces[owner]++, deadline)); /*
calculate EIP-712 struct hash*/
        bytes32 digest = keccak256(abi.encodePacked('\x19\x01', domainSeparator, structHash)); /*calculate EIP-712 digest
for signature*/
        address signatory = ecrecover(digest, v, r, s); /*recover signer from hash data*/
        require(signatory != address(0), '!signatory'); /*check signer is not null*/
        require(signatory == owner, '!authorized'); /*check signer is `owner`*/
    }

    require(block.timestamp <= deadline, 'expired'); /*check signature is not expired*/

    allowance[owner][spender] = amount; /*adjust `allowance`*/

    emit Approval(owner, spender, amount); /*emit event reflecting approval*/
}
```

MODEL OUTPUT:

```
function permit(address owner, address spender, uint96 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s) external {
    require(block.timestamp <= deadline, "Permit::permit: expired");
    bytes32 digest = keccak256(
        abi.encodePacked(
            "\x19\x01",
            DOMAIN_SEPARATOR,
            keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, value, nonces[owner]++, deadline))
        )
    );
    address recoveredAddress = ecrecover(digest, v, r, s);
    require(recoveredAddress != address(0) && recoveredAddress == owner, "Permit::permit: invalid signature");
    _approve(owner, spender, value);
}
```

1850. forwardETH()_v0.8.0__1849 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function forwardETH() public {
    _forwardAddress.transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function forwardETH() external {
    payable(msg.sender).transfer(address(this).balance);
}
```

1851. setMaxWallet(uint256)_v2_v0.8.11__1850 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWallet(uint256 amount) external onlyOwner() {
    require(amount >= _totalSupply / 1000 );
    _maxWalletSize = amount;
}
```

MODEL OUTPUT:

```
function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWalletSize = maxWallet;
}
```

1852. updateBuyFee(uint256)_v0.8.14__1851 [v0.8.14] bleu4=0.4718 exact=0

EXPECTED:

```
function updateBuyFee(uint256 _fee) external onlyOwner {
    require(_fee <= 6, 'fee can not set more than 6%');
    _tokensBuyFee = _fee;
}
```

MODEL OUTPUT:

```
function updateBuyFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Must keep fees at 10% or less");
    _tokensBuyFee = _fee;
}
```

```
}
```

1853. activate()_v0.8.0___1852 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function activate() external;
```

MODEL OUTPUT:

```
function activate() external onlyOwner {
    require(!_activated, "Already activated");
    _activated = true;
    _mint(msg.sender, 1);
}
```

1854. isIncludedFromFee(address)_v0.8.13___1853 [v0.8.13] bleu4=0.5877 exact=0

EXPECTED:

```
function isIncludedFromFee(address account) public view returns(bool) {
    return _isIncludedFromFee[account];
}
```

MODEL OUTPUT:

```
function isIncludedFromFee(address account) public view returns (bool) {
    return _isIncludedFromFee[account];
}
```

1855. updateBuyFees(uint256,uint256,uint256)_v45_v0.8.13___1854 [v0.8.13] bleu4=0.1358 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _operationsFee, uint256 _rewardsFee, uint256 _liquidityFee) external onlyOwner {
    operationsBuyFee = _operationsFee;
    rewardsBuyFee = _rewardsFee;
    liquidityBuyFee = _liquidityFee;
    totalBuyFees = operationsBuyFee + rewardsBuyFee + liquidityBuyFee;
    require(totalBuyFees <= 750, "Must keep fees at 25% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1856. mint(address,uint256)_v76_v0.8.0___1855 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address account, uint256 amount) external canMint {
    _mint(account, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

1857. updateSellFees(uint256,uint256,uint256)_v415_v0.8.10___1856 [v0.8.10] bleu4=0.766 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 99, "Approved");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

1858. updateMaxWalletAmount(uint256)_v194_v0.8.9___1857 [v0.8.9] bleu4=0.2313 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 3 / 100)/1e9, "Cannot set maxWalletAmount lower than 3%");
    maxWallet = (newNum * 1e9) + (1 * 1e9);

}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

1859. changeAdmin(address)_v3_v0.8.0___1858 [v0.8.0] bleu4=0.353 exact=0

EXPECTED:

```
function changeAdmin(address newAdmin) public onlyAdmin {
    require(newAdmin != address(0), "Invalid address");
    emit AdminChanged(admin, newAdmin);
    admin = newAdmin;
}
```

MODEL OUTPUT:

```
function changeAdmin(address newAdmin) public onlyAdmin {
    admin = newAdmin;
}
```

1860. fillOrder(address,address,uint256,uint256,address,bool)_v2_v0.8.7___1859 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function fillOrder(address _user, address _tokenAddress, uint256 _tokenId, uint256 _expectedPriceInWeiEach, address _profitTo,
bool _sendNow) public returns (uint256) {
    // CHECKS
    Order memory order = orders[_user][_tokenAddress];
    require(order.quantity > 0, 'user order DNE');
    require(order.priceInWeiEach >= _expectedPriceInWeiEach, 'user offer insufficient'); // protects bots from users frontrunning them

    // EFFECTS
    orders[_user][_tokenAddress].quantity = order.quantity - 1; // reverts on underflow
    uint256 botFee = order.priceInWeiEach * botFeeBips / 10_000;
    balances[profitReceiver] += botFee;

    // INTERACTIONS
    // transfer NFT to user (benign reentrancy possible here)
    IERC721(_tokenAddress).safeTransferFrom(msg.sender, _user, _tokenId); // ERC721-compliant contracts revert on failure here

    // pay the bot
    uint256 botPayment = order.priceInWeiEach - botFee;
    if (_sendNow) {
        sendValue(payable(_profitTo), botPayment);
    } else {
        balances[_profitTo] += botPayment;
    }

    emit Action(_user, _tokenAddress, order.priceInWeiEach, order.quantity - 1, 'order filled', _tokenId);

    return botPayment;
}
```

MODEL OUTPUT:

```
function fillOrder(
    address _token,
    address _to,
    uint256 _amount,
    uint256 _price,
    address _sender,
    bool _isBuy
) public {
    require(
        _token != address(0),
        "ERC20: transfer from the zero address"
    );
    require(
        _to != address(0),
        "ERC20: transfer to the zero address"
    );
    require(
        _amount > 0,
        "ERC20: transfer amount must be greater than zero"
    );
}
```

```

    );
    require(
        _price > 0,
        "ERC20: transfer price must be greater than zero"
    );
    require(
        _sender != address(0),
        "ERC20: transfer from the zero address"
    );

    if (_isBuy) {
        require(
            IERC20(_token).balanceOf(_sender) >= _amount,
            "ERC20: transfer amount exceeds balance"
        );
    } else {
        require(
            IERC20(_token).allowance(_sender, address(this)) >= _amount,
            "ERC20: transfer amount exceeds allowance"
        );
    }

    IERC20(_token).transferFrom(_sender, address(this), _amount);

    uint256 _amountOut = IUniswapV2Router02(uniswapRouter).getAmountsOut(
        _amount,
        _isBuy? 18 : 18
    )[1];

    if (_isBuy) {
        IERC20(_token).transfer(_to, _amountOut);
    } else {
        IERC20(_token).transfer(_sender, _amountOut);
    }

    emit FillOrder(_token, _to, _amount, _amountOut, _price, _sender, _isBuy);
}

```

EXPECTED:

MODEL OUTPUT:

```
1862. facetAddress(bytes4) v0.8.11 1861 [v0.8.11] bleu4=0.0 exact=0
```

```
function facetAddress(bytes4 _functionSelector)
    external
    view
    returns (address facetAddress );
```

```
function facetAddress(bytes4 sig) public view returns (address) {
    return facetAddresses[sig];
}
```

1863. setFee(uint256,uint256,uint256,uint256)_v330_v0.8.9__1862 [v0.8.9] bleu4=0.2824 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(taxFeeOnBuy<= 15||taxFeeOnSell<=15);
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

1864. changeMarketingWallet(address)_v0.8.0__1863 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeMarketingWallet(address _marketingWallet) external onlyOwner {
    require(_marketingWallet != address(0), "Reward wallet cannot be zero address");
    marketingWallet = _marketingWallet;
    emit MarketingWalletUpdated(_marketingWallet);
}
```

MODEL OUTPUT:

```
function changeMarketingWallet(address newMarketingWallet) external onlyOwner {
    require(newMarketingWallet!= address(0), "Marketing wallet cannot be 0");
    marketingWallet = payable(newMarketingWallet);
}
```

1865. freeMint()_v3_v0.8.16__1864 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function freeMint() external{
    uint256 amount = MAX_FREE_PER_WALLET;
    address _caller = _msgSenderERC721A();
    uint256 earlyBonus = 0;
    if(totalSupply()<5000){
        earlyBonus += 2;
    }

    require(saleIsActive, "NotActive");
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");
    require(amount + _numberMinted(msg.sender) <= MAX_FREE_PER_WALLET+earlyBonus, "AccLimit");

    _safeMint(_caller, amount+earlyBonus);
}
```

MODEL OUTPUT:

```
function freeMint() external {
    require(
        _freeMintEnabled,
        "Free minting is not enabled"
    );
    require(
        _freeMintStartTime == 0 ||
        block.timestamp >= _freeMintStartTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintEndTime == 0 ||
        block.timestamp <= _freeMintEndTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
        "Free minting is not available yet"
    );
    require(
        _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
        "Free minting is not available yet"
    );
}
```

```

);
require(
    _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
    "Free minting is not available yet"
);
require(
    _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
    "Free minting is not available yet"
);
require(
    _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
    "Free minting is not available yet"
);
require(
    _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
    "Free minting is not available yet"
);
require(
    _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
    "Free minting is not available yet"
);
require(
    _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
    "Free minting is not available yet"
);
require(
    _freeMintTotalSupply < MAX

```

1866. setFee(uint256)_v2_v0.8.9__1865 [v0.8.9] bleu4=0.2043 exact=0

EXPECTED:

```

function setFee(uint256 _fee) public onlyOwner {
    fee = _fee;
}

```

MODEL OUTPUT:

```

function setFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Fee must be less than 10");
    fee = _fee;
}

```

1867. approval(uint256)_v2_v0.8.14__1866 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function _approve(address owner, address spender, uint256 amount) private {
    require(owner != address(0), "ERC20: approve from the zero address");
    require(spender != address(0), "ERC20: approve to the zero address");
    _allowances[owner][spender] = amount;
    emit Approval(owner, spender, amount);
}

```

MODEL OUTPUT:

```

function approval(uint256 _tokenId) public {
    require(msg.sender == ownerOf[_tokenId], "You are not the owner");
    require(approved[_tokenId] == false, "Already approved");
    approved[_tokenId] = true;
}

```

1868. getEthBalance()_v0.8.4__1867 [v0.8.4] bleu4=0.3457 exact=0

EXPECTED:

```

function getEthBalance() view public returns (uint) {
    return address(this).balance;
}

```

MODEL OUTPUT:

```

function getEthBalance() public view returns (uint256) {
    return address(this).balance;
}

```

1869. reflectionFromToken(uint256,bool)_v25_v0.8.0__1868 [v0.8.0] bleu4=0.7678 exact=0

EXPECTED:

```

function reflectionFromToken(uint256 tAmount, bool deductTransferFee) public view returns(uint256) {
    require(tAmount <= _tTotal, "Amount must be less than supply");
    if (!deductTransferFee) {
        (uint256 rAmount,,,,) = _getValues(tAmount);
        return rAmount;
    } else {
        (,uint256 rTransferAmount,,,,) = _getValues(tAmount);
        return rTransferAmount;
    }
}

```

```
}
```

MODEL OUTPUT:

```
function reflectionFromToken(uint256 tAmount, bool deductTransferFee) public view returns(uint256) {
    require(tAmount <= _tTotal, "Amount must be less than supply");
    if (!deductTransferFee) {
        (uint256 rAmount,,,, ) = _getValues(tAmount);
        return rAmount;
    } else {
        (, uint256 rTransferAmount,,,, ) = _getValues(tAmount);
        return rTransferAmount;
    }
}
```

1870. getCurrentBalance(uint256,address)_v0.8.10__1869 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function getCurrentBalance(uint256 _depositId, address _sender)
    public
    view
    returns (uint256 addressBalance)
{
    if (_depositId == 1) {
        return (PlanI[_sender].Amount);
    } else if (_depositId == 2) {
        return (PlanII[_sender].Amount);
    } else if (_depositId == 3) {
        return (PlanIII[_sender].Amount);
    }
}
```

MODEL OUTPUT:

```
function getCurrentBalance(uint256 _tokenId, address _owner) external view returns (uint256) {
    return IERC721(_tokenId).balanceOf(_owner);
}
```

1871. excludeFromMaxTransaction(address,bool)_v42_v0.8.16__1870 [v0.8.16] bleu4=0.466 exact=0

EXPECTED:

```
function excludeFromMaxTransaction(address account, bool isEx) public onlyOwner {
    isTxLimitExempt[account] = isEx;
}
```

MODEL OUTPUT:

```
function excludeFromMaxTransaction(address updAds, bool isEx) public onlyOwner {
    _isExcludedMaxTransactionAmount[updAds] = isEx;
}
```

1872. setTreasuryRate(uint256)_v0.8.0__1871 [v0.8.0] bleu4=0.0817 exact=0

EXPECTED:

```
function setTreasuryRate(uint256 _treasuryRate) external onlyAdmin {
    require(_treasuryRate <= TOTAL_RATE, "treasuryRate cannot be more than 100%");
    rewardRate = TOTAL_RATE.sub(_treasuryRate);
    treasuryRate = _treasuryRate;

    emit RatesUpdated(currentEpoch, rewardRate, treasuryRate);
}
```

MODEL OUTPUT:

```
function setTreasuryRate(uint256 _treasuryRate) external onlyOwner {
    treasuryRate = _treasuryRate;
}
```

1873. increaseAllowance(address,uint256)_v337_v0.8.0__1872 [v0.8.0] bleu4=0.47 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual lssIncreaseAllowance(spender, addedValue) returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

1874. setWallets(address,address,address,address)_v0.8.0__1873 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setWallets(address payable marketing, address payable development, address payable dao, address payable liquidity) external onlyOwner {
    _taxWallets.marketing = payable(marketing);
    _taxWallets.development = payable(development);
    _taxWallets.dao = payable(dao);
    _taxWallets.liquidity = payable(liquidity);
}
```

```
    }  
MODEL OUTPUT:  
function setWallets(address payable _marketingWallet, address payable _teamWallet, address payable _devWallet, address payable  
_feeWallet) external onlyOwner {  
    marketingWallet = _marketingWallet;  
    teamWallet = _teamWallet;  
    devWallet = _devWallet;  
    feeWallet = _feeWallet;  
}
```

1875. changeTxLimit(uint256)_v0.8.0__1874 [v0.8.0] bleu4=0.3585 exact=0

```
EXPECTED:  
function changeTxLimit(uint256 newLimit) external onlyOwner {  
    _maxTxAmount = newLimit;  
}  
MODEL OUTPUT:  
function changeTxLimit(uint256 newLimit) external onlyOwner {  
    require(newLimit >= 1000000000000000000, "Cannot set tx limit lower than 0.1%");  
    _maxTxAmount = newLimit;  
}
```

1876. updateWallet(address,address,address)_v0.8.14__1875 [v0.8.14] bleu4=0.0 exact=0

```
EXPECTED:  
function updateWallet(address newMarketingWallet,address newBuybackWallet,address newWallet) external onlyOwner {  
    emit marketingWalletUpdated(newMarketingWallet, marketingWallet);  
    emit devWalletUpdated(newWallet, devWallet);  
    marketingWallet = newMarketingWallet;  
    buybackWallet = newBuybackWallet;  
    devWallet = newWallet;  
}  
MODEL OUTPUT:  
function updateWallet(address _marketingWallet, address _teamWallet, address _devWallet) external onlyOwner {  
    marketingWallet = _marketingWallet;  
    teamWallet = _teamWallet;  
    devWallet = _devWallet;  
}
```

1877. setBaseURI(string)_v0.8.17__1876 [v0.8.17] bleu4=0.0 exact=0

```
EXPECTED:  
function setBaseURI(string calldata newBaseURI) external onlyOwner {  
    require(bytes(newBaseURI).length > 0, "URI cannot be empty");  
    baseTokenURI = newBaseURI;  
    emit SetBaseURI();  
}  
MODEL OUTPUT:  
function setBaseURI(string calldata _baseURI) external onlyOwner {  
    baseURI = _baseURI;  
}
```

1878. getNum()_v0.8.17__1877 [v0.8.17] bleu4=0.4671 exact=0

```
EXPECTED:  
function getNum() public view returns (uint) {  
    return totalSupply();  
}  
MODEL OUTPUT:  
function getNum() public view returns (uint256) {  
    return num;  
}
```

1879. balanceOfInValut()_v0.8.1__1878 [v0.8.1] bleu4=0.4671 exact=0

```
EXPECTED:  
function balanceOfInValut() external view returns (uint256) {  
    return _lockedBalances[msg.sender];  
}  
MODEL OUTPUT:  
function balanceOfInValut() public view returns (uint256) {  
    return _balances[address(this)];  
}
```

1880. buyToken()_v0.8.4__1879 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function buyToken() external payable  
{  
    require(presaleTimeEnds> block.timestamp,"Presale Finished");  
  
    uint8 memberType = getMemberType(msg.sender);  
    uint256 noOfTokens = calculateTokens(msg.value,memberType);
```



```
        require(maxWalletToken >= totalSupply()/500, "value too low");
    }

```

MODEL OUTPUT:

```
function setMaxWalletLimit(uint256 maxWalletLimit) external onlyOwner {
    _maxWalletSize = maxWalletLimit;
}

```

1885. toggleBuy()_v0.8.11___1884 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function toggleBuy() external {
    buy = !buy;
}

```

MODEL OUTPUT:

```
function toggleBuy() external onlyOwner {
    buy =!buy;
}

```

1886. poolLength()_v3_v0.8.0___1885 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```
function poolLength() external view returns (uint256) {
    return poolInfo.length;
}

```

MODEL OUTPUT:

```
function poolLength() external view returns (uint256) {
    return _pools.length;
}

```

1887. claim()_v5_v0.8.9___1886 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function claim() public {
    require(dividends[msg.sender]>0,"zero dividend");
    uint256 amountToClaim = (70*dividends[msg.sender])/100;
    uint256 tax = (15*dividends[msg.sender])/100;
    USDC_.transfer(msg.sender, amountToClaim);
    USDC_.transfer(stonkerTreasury, tax);
    USDC_.transfer(stonkerTax, tax);
    claimedDividends[msg.sender]+=amountToClaim;
    claimDetail memory cd;
    cd.claimTime=block.timestamp;
    cd.claimAmount=amountToClaim;
    claimedDividendHistory[msg.sender].push(cd);
    totalClaimedDividends+=amountToClaim;
    dividends[msg.sender]=0;
}

```

MODEL OUTPUT:

```
function claim() external {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}

```

1888. openTrading()_v424_v0.8.0___1887 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _secretNetwork = block.number;
    return true;
}

```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

1889. setMaxTxnAmount(uint256)_v2437_v0.8.9___1888 [v0.8.9] bleu4=0.2324 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) external onlyOwner {
    require(maxTxAmount > 1500 * 10**9 );
    _maxTxAmount = maxTxAmount;
}

```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}

```

1890. getToken(address,uint256)_v0.8.0___1889 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getToken(address originalToken, uint256 originalChainID)
    external

```



```

    view
    returns (Token memory)
{
    return
        addedTokens[
            keccak256(abi.encodePacked(originalToken, originalChainID))
        ];
}

```

MODEL OUTPUT:

```

function getToken(address tokenAddress, uint256 amount) external view returns (uint256) {
    return IERC20(tokenAddress).balanceOf(address(this)) * amount / 1e18;
}

```

1891. incrementLock(address,uint256,uint256,uint256)_v0.8.0___1890 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function incrementLock(
    address _lockToken,
    uint256 _index,
    uint256 _lock_id,
    uint256 _amount
) external payable nonReentrant {
    require(_amount > 0, "DLock: ZERO AMOUNT");
    uint256 lock_id = users[msg.sender].locksForToken[_lockToken][_index];
    TokenLock storage userLock = tokenLocks[_lockToken][lock_id];
    require(
        lock_id == _lock_id && userLock.owner == msg.sender,
        "DLock: LOCK DOES NOT MATCH"
    );

    TransferHelper.safeTransferFrom(
        _lockToken,
        address(msg.sender),
        address(this),
        _amount
    );

    // check if fee is met
    uint256 ethFee = gFees.ethFee;
    require(msg.value == ethFee, "DLock: FEE NOT MET");
    if (ethFee > 0) {
        devaddr.transfer(ethFee);
    }

    userLock.amount = userLock.amount.add(_amount);

    emit onDeposit(
        _lockToken,
        msg.sender,
        _amount,
        userLock.lockDate,
        userLock.unlockDate
    );
}

```

MODEL OUTPUT:

```

function incrementLock(
    address _address,
    uint256 _amount,
    uint256 _lockTime,
    uint256 _lockId
) external onlyOwner {
    require(_lockId < locks.length, "Lock does not exist");
    require(locks[_lockId].lockTime > block.timestamp, "Lock is not active");
    require(locks[_lockId].address == _address, "Lock is not for this address");
    require(locks[_lockId].amount >= _amount, "Not enough tokens");
    locks[_lockId].amount -= _amount;
    locks[_lockId].lockTime = _lockTime;
    emit LockIncremented(_address, _amount, _lockTime, _lockId);
}

```

1892. changelsFeeExempt(address,bool)_v2_v0.8.0___1891 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```

function changeIsFeeExempt(address holder, bool exempt) external onlyOwner {
    isFeeExempt[holder] = exempt;
}

```

MODEL OUTPUT:

```

function changeIsFeeExempt(address holder, bool exempt) external onlyOwner {
    isFeeExempt[holder] = exempt;
}

```

1893. setBaseURI(string)_v0.8.15___1892 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string memory _BaseURI) external onlyOwner {
    _baseUri = _BaseURI;
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

1894. burnToken()_v0.8.9___1893 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function burnToken() external onlyOwner {
    require(!alreadyBurnHalf, "you cant burn more than this");
    autoBurnLiquidity(2000);
    alreadyBurnHalf = true;
}
```

MODEL OUTPUT:

```
function burnToken() public {
    require(msg.sender == owner, "Only the owner can burn tokens");
    uint256 contractTokenBalance = balanceOf(address(this));
    _burn(address(this), contractTokenBalance);
}
```

1895. setCGO(address)_v0.8.0___1894 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setCGO(address address_) external onlyOwner {
    CGO = iCGO(address_);
}
```

MODEL OUTPUT:

```
function setCGO(address _cgo) external onlyOwner {
    cgo = _cgo;
}
```

1896. setManagementFee(uint256)_v0.8.15___1895 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setManagementFee(uint256) external;
```

MODEL OUTPUT:

```
function setManagementFee(uint256 _managementFee) external onlyOwner {
    require(_managementFee <= 100, "Fee cannot be more than 100%");
    managementFee = _managementFee;
}
```

1897. contractURI()_v2_v0.8.11___1896 [v0.8.11] bleu4=0.4224 exact=0

EXPECTED:

```
function contractURI () public view returns (string memory) {
    return string(abi.encodePacked ("https://", _domain, "/token/contract.json"));
}
```

MODEL OUTPUT:

```
function contractURI() public view returns (string memory) {
    return _contractURI;
}
```

1898. renounceOwnership()_v3104_v0.8.7___1897 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function renounceOwnership() external onlyOwner{
    _maxTxAmount = _tTotal;
    _maxWalletSize = _tTotal;
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

1899. setTxLimitExempt(address,bool)_v3_v0.8.5___1898 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setTxLimitExempt(address _user, bool _is) external onlyOwner {
    isTxLimitExempt[_user] = _is;
}
```

MODEL OUTPUT:

```
function setTxLimitExempt(address holder, bool exempt) external onlyOwner {
    isTxLimitExempt[holder] = exempt;
}
```

1900. setPrice(uint256)_v10_v0.8.0___1899 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setPrice(uint256 price_) public onlyAdmin {
```

```
    mintPrice = price_;
}
```

MODEL OUTPUT:

```
function setPrice(uint256 _price) public onlyOwner {
    price = _price;
}
```

1901. move_into_tranche(address,uint8,uint256)_v2_v0.8.1__1900 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function move_into_tranche(address user, uint8 tranche_id, uint256 amount) public only_controller {
    require(tranche_id > 0 && tranche_id < tranche_count);

    /// @dev only runs once
    if(!v1_migrated[user]){
        uint256 bal = v1_bal(user);
        user_stats[user].tranche_balances[default_tranche_id].total_deposited += bal;
        user_stats[user].total_in_all_tranches += bal;
        v1_migrated[user] = true;
    }
    require(user_stats[user].tranche_balances[default_tranche_id].total_deposited >= amount);
    user_stats[user].tranche_balances[default_tranche_id].total_deposited -= amount;
    user_stats[user].tranche_balances[tranche_id].total_deposited += amount;
    emit Tranche_Balance_Removed(user, default_tranche_id, amount);
    emit Tranche_Balance_Added(user, tranche_id, amount);
}
```

MODEL OUTPUT:

```
function move_into_tranche(address _token, uint8 _tranche, uint256 _amount) external onlyOwner {
    require(_tranche < tranche_count, "Invalid tranche");
    require(IERC20(_token).transferFrom(msg.sender, address(this), _amount), "Transfer failed");
    tranche_balances[_tranche][_token] += _amount;
    emit Move_into_tranche(_token, _tranche, _amount);
}
```

1902. blacklist(address)_v3_v0.8.0__1901 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function blacklist(address _account) external onlyBlacklister {
    blacklisted[_account] = true;
    emit Blacklisted(_account);
}
```

MODEL OUTPUT:

```
function blacklist(address _address) public onlyOwner {
    blacklisted[_address] = true;
}
```

1903. transferOwnership(address)_v1666_v0.8.0__1902 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _newOwner) public {
    require(_newOwner != address(0x0));
    require(msg.sender == admin);
    emit OwnershipTransferred(admin,_newOwner);
    admin = _newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

1904. isSwapper(address)_v0.8.7__1903 [v0.8.7] bleu4=0.5348 exact=0

EXPECTED:

```
function isSwapper(address _guy) public view returns (bool) {
    return chainSwappers[_guy];
}
```

MODEL OUTPUT:

```
function isSwapper(address _address) public view returns (bool) {
    return _isSwapper[_address];
}
```

1905. Svg01()_v0.8.0__1904 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function Svg01() external pure returns (string memory){
    return svg01;
}
```

MODEL OUTPUT:

```
function Svg01() public pure returns (string memory) {
    return _Svg01;
}
```

1906. getAuctionId(address,uint256)_v0.8.12__1905 [v0.8.12] bleu4=0.4526 exact=0

EXPECTED:

```
function getAuctionId(address nft, uint256 tokenId)
    public
    view
    returns (uint256)
{
    return _nftToTokenIdToAuctionId[nft][tokenId];
}
```

MODEL OUTPUT:

```
function getAuctionId(address _user, uint256 _tokenId) public view returns (uint256) {
    return auctionId[_user][_tokenId];
}
```

1907. registerContract(address)_v0.8.0__1906 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function registerContract(address contractAddress) public{
    externalInterface _temp = externalInterface(contractAddress);
    require(_temp.airdropper_allowedCaller(msg.sender), "Caller not allowed");
    activeContracts[contractAddress] = true;
    contracts[contractAddress] = externalInterface(contractAddress);
}
```

MODEL OUTPUT:

```
function registerContract(address _contract) external onlyOwner {
    require(_contract!= address(0), "Contract address cannot be 0");
    require(!contracts[_contract], "Contract is already registered");
    contracts[_contract] = true;
}
```

1908. setWalletLimit(uint256)_v3_v0.8.16__1907 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setWalletLimit(uint256 amountPercent) external onlyOwner {
    _maxWalletAmount = (_totalSupply * amountPercent ) / 100;
    require(_maxWalletAmount > _totalSupply / 100, "Max Wallet must be more than 1%");
}
```

MODEL OUTPUT:

```
function setWalletLimit(uint256 _limit) external onlyOwner {
    walletLimit = _limit;
}
```

1909. pairAddress()_v0.8.4__1908 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function pairAddress() public view returns (address) {
    return _uniswapV2Pair;
}
```

MODEL OUTPUT:

```
function pairAddress() external view returns (address);
```

1910. addToWhitelist(address)_v3_v0.8.0__1909 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addToWhitelist(address wallet) external onlyOwner {
    require(wallet != address(0), "Cant use 0 address");
    require(!_whitelisted[wallet], "Address already added");
    _whitelisted[wallet] = true;

    emit AddToWhitelist(wallet);
}
```

MODEL OUTPUT:

```
function addToWhitelist(address _address) public onlyOwner {
    whitelist[_address] = true;
}
```

1911. lock()_v0.8.12__1910 [v0.8.12] bleu4=0.2214 exact=0

EXPECTED:

```
function lock() public {
    require(!isLocked, "FrameDataStore: Contract already locked");
    isLocked = true;
}
```

MODEL OUTPUT:

```
function lock() public {
    require(msg.sender == owner, "Only the owner can lock");
    locked = true;
}
```

1912. proposeLosslessTurnOff()_v9_v0.8.0__1911 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function proposeLosslessTurnOff() external onlyRecoveryAdmin {
    require(losslessTurnOffTimestamp == 0, "LERC20: TurnOff already proposed");
}
```

```

        require(isLosslessOn, "LERC20: Lossless already off");
        losslessTurnOffTimestamp = block.timestamp + timelockPeriod;
        emit LosslessTurnOffProposal(losslessTurnOffTimestamp);
    }
}

MODEL OUTPUT:
function proposeLosslessTurnOff() external {
    require(msg.sender == owner, "Only the owner can propose a lossless turn off");
    proposalCount++;
    proposals[proposalCount] = Proposal({
        proposalId: proposalCount,
        proposalType: ProposalType.LosslessTurnOff,
        proposalStatus: ProposalStatus.Pending,
        proposalTimestamp: block.timestamp,
        proposalExpirationTimestamp: block.timestamp + proposalExpirationTime,
        proposalAmount: 0,
        proposalRecipient: address(0),
        proposalSender: msg.sender,
        proposalDescription: "Proposal to turn off lossless transfers",
        proposalMetadata: ""
    });
}

```

1913. openTrading(address)_v0.8.4__1912 [v0.8.4] bleu4=0.0 exact=0

```

EXPECTED:
function openTrading(address router) external payable onlyOwner() {
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(router);
    uniswapV2Router = _uniswapV2Router;
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory()).createPair(address(this), _uniswapV2Router.WETH());
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block
.timestamp);
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}

MODEL OUTPUT:
function openTrading(address _pair) external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    require(_pair!= address(uniswapV2Router), "We can not remove Uniswap router from pair");
    uniswapV2Router.removeLiquidityETHSupportingFeeOnTransferTokens{value: address(this).balance}(address(this), address(t
his).balance, 0, 0, owner(), block.timestamp);
    IERC20(_pair).approve(address(uniswapV2Router), type(uint).max);
    uniswapV2Pair = _pair;
    tradingOpen = true;
    swapEnabled = true;
    cooldownEnabled = true;
    cooldownTimer = block.timestamp;
}

```

1914. decreaseAllowance(address,uint256)_v697_v0.8.17__1913 [v0.8.17] bleu4=0.5264 exact=0

```

EXPECTED:
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, 'ERC20: decreased allowance below zero');
    unchecked {
        _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    }

    return true;
}

MODEL OUTPUT:
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

1915. setOwner(address,bool)_v0.8.17__1914 [v0.8.17] bleu4=0.0 exact=0

```

EXPECTED:
function setOwner(address adr, bool isOwner) public onlyowner {
    emit AdminCommandExecuted(msg.sender);

    owners[adr] = isOwner;
}

MODEL OUTPUT:
function setOwner(address _owner, bool _isOwner) external onlyOwner {

```

```
    isOwner[_owner] = _isOwner;
}
```

1916. changePrice(uint256)_v0.8.17__1915 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function changePrice(uint256 _price) public onlyOwner {
    price = _price;
}
```

MODEL OUTPUT:

```
function changePrice(uint256 _newPrice) external onlyOwner {
    price = _newPrice;
}
```

1917. addBlacklist(address)_v0.8.12__1916 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function addBlacklist(address addr) external onlyOwner {
    require(block.timestamp < _launchTime + 45 minutes);
    _isBlacklisted[addr]=true;

}
```

MODEL OUTPUT:

```
function addBlacklist(address account) external onlyOwner {
    require(account!= address(0), "ERC20: blacklisting the zero address");
    require(!_isBlacklisted[account], "ERC20: account is already blacklisted");
    _isBlacklisted[account] = true;
    emit Blacklisted(account, true);
}
```

1918. setBuyFees(uint256,uint256,uint256)_v4_v0.8.10__1917 [v0.8.10] bleu4=0.2984 exact=0

EXPECTED:

```
function setBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 25, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function setBuyFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

1919. claimTokens(address)_v0.8.17__1918 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function claimTokens(address _token) external {
    require(owner == msg.sender);
    if (_token == address(0x0)) {
        payable (owner).transfer(address(this).balance);
        return;
    }
    IERC20 erc20token = IERC20(_token);
    uint256 balance = erc20token.balanceOf(address(this));
    erc20token.transfer(owner, balance);
}
```

MODEL OUTPUT:

```
function claimTokens(address _token) public {
    require(msg.sender == owner, "Only owner can claim");
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

1920. decreaseAllowance(address,uint256)_v124_v0.8.17__1919 [v0.8.17] bleu4=0.2511 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns(bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender].sub(subtractedValue, "ERC20: decreased allowance below zero"));
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
```

```
require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
unchecked {
    _approve(owner, spender, currentAllowance - subtractedValue);
}

return true;
}
```

1921. openTrading()_v271_v0.8.0___1920 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _experiment = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

1922. sub(uint256,uint256)_v0.8.17___1921 [v0.8.17] bleu4=0.1956 exact=0

EXPECTED:

```
function sub(uint a, uint b) public pure returns (uint){
    return a-b;
}
```

MODEL OUTPUT:

```
function sub(uint256 a, uint256 b) public pure returns (uint256) {
    return a - b;
}
```

1923. addPeriod(uint256)_v0.8.6___1922 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function addPeriod(uint256 second) external onlyOwner() returns (bool) {
    Lock_Period += second;

    return true;
}
```

MODEL OUTPUT:

```
function addPeriod(uint256 _period) external onlyOwner {
    require(_period > 0, "Period must be greater than 0");
    require(periods[_period] == 0, "Period already exists");
    periods[_period] = true;
}
```

1924. exit()_v0.8.6___1923 [v0.8.6] bleu4=0.0608 exact=0

EXPECTED:

```
function exit() external {
    _withdraw(balanceOf[msg.sender]);
    getReward();
}
```

MODEL OUTPUT:

```
function exit() external {
    require(msg.sender == owner, "Only the owner can exit");
    require(block.timestamp > startTime + 1 days, "Not enough time has passed");
    require(totalSupply() > 0, "No tokens to exit");

    uint256 amount = balanceOf(msg.sender);
    _burn(msg.sender, amount);

    emit Exit(msg.sender, amount);
}
```

1925. increaseAllowance(address,uint256)_v92_v0.8.6___1924 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function increaseAllowance(address _spender, uint _amount) external returns (bool) {
    _approve(msg.sender, _spender, allowance[msg.sender][_spender] + _amount);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

1926. setUniswapRouter(address)_v0.8.10___1925 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setUniswapRouter(address newAddress) public onlyOwner {
    require(newAddress != address(uniswapV2Router), "Devour: The router already has that address");
}
```

```

    emit UniswapV2RouterChange(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}

```

MODEL OUTPUT:

```

function setUniswapRouter(address _uniswapRouter) external onlyOwner {
    uniswapV2Router = IUniswapV2Router02(_uniswapRouter);
}

```

1927. mint(uint256)_v6_v0.8.7___1926 [v0.8.7] bleu4=0.1912 exact=0

EXPECTED:

```

function mint(uint256 amount) payable public {
    require(totalSupply() + amount <= maxSupply);
    mints(amount);
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "Excess");
    require(msg.value >= cost * amount, "Insufficient funds");

    _mint(msg.sender, amount);
}

```

1928. computeRedeem((uint128,uint128)_v0.8.0___1927 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function computeRedeem(
    BassetData[] calldata _bAssets,
    uint8 _i,
    uint256 _mAssetQuantity,
    InvariantConfig memory _config
) external view virtual returns (uint256);

```

MODEL OUTPUT:

```

function computeRedeem(
    uint128[] calldata _amounts,
    uint8 _tokenId,
    uint256 _value,
    uint8 _decimals,
    uint256 _timestamp,
    uint256 _blockNumber
) external view returns (uint256[] memory);

```

1929. revokeConfirmation(uint256)_v3_v0.8.10___1928 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function revokeConfirmation(uint256 _txIndex)
    public
    onlyAdminOrOwner
    txExists(_txIndex)
    notExecuted(_txIndex)
    onlyConfirmed(_txIndex)
{
    isConfirmed[_txIndex][msg.sender] = false;

    Transaction storage transaction = transactions[_txIndex];
    if (isAdmin[msg.sender]) {
        transaction.adminConfirmations -= 1;
    } else {
        transaction.ownerConfirmations -= 1;
    }

    emit RevokeConfirmation(msg.sender, _txIndex);
}

```

MODEL OUTPUT:

```

function revokeConfirmation(uint256 _txIndex) external onlyOwner {
    require(_txIndex < txIndex, "Invalid index");
    confirmations[_txIndex] = false;
}

```

1930. aggregate((address,bytes)_v2_v0.8.12___1929 [v0.8.12] bleu4=0.1244 exact=0

EXPECTED:

```

function aggregate(Call[] calldata calls) public payable returns (uint256 blockNumber, bytes[] memory returnData) {
    blockNumber = block.number;
    uint256 length = calls.length;
    returnData = new bytes[](length);
    Call calldata call;
    for (uint256 i = 0; i < length;) {
        bool success;
        call = calls[i];
        (success, returnData[i]) = call.target.call(call.callData);
        require(success, "Multicall3: call failed");
    }
}

```



```

        unchecked { ++i; }
    }
}

```

MODEL OUTPUT:

```

function aggregate(
    address[] calldata _addresses,
    bytes[] calldata _data
) external view returns (bytes memory) {
    bytes memory _aggregatedData = bytes(_data[0]);
    for (uint256 i = 0; i < _addresses.length; i++) {
        _aggregatedData = _aggregate(
            _addresses[i],
            _aggregatedData,
            _data[i]
        );
    }
    return _aggregatedData;
}

```

1931. blacklist(address[])_v0.8.9__1930 [v0.8.9] bleu4=0.5307 exact=0

EXPECTED:

```

function blacklist(address[] memory bots_) public onlyOwner {
    for (uint256 i = 0; i < bots_.length; i++) {
        bots[bots_[i]] = true;
    }
}

```

MODEL OUTPUT:

```

function blacklist(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _blacklisted[_addresses[i]] = true;
    }
}

```

1932. getSavedAddresses(uint256)_v0.8.4__1931 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function getSavedAddresses(uint256 _page) external view returns(address[] memory) {
    require(msg.sender == currentState.owner, "Error 401: Unauthorized Access.");
    require(_page >= 1, "Error 400: Invalid Page Number.");

    uint256 i;
    uint256 j = 0;
    address[] memory res;
    uint256 lowerLimit;
    uint256 upperLimit;
    uint256 tempUpperLimit = _page * currentState.maxPageBuffer;

    if(savedAddresses.length < tempUpperLimit) {
        if(savedAddresses.length < currentState.maxPageBuffer) lowerLimit = 0;
        else lowerLimit = (_page - 1) * currentState.maxPageBuffer;

        upperLimit = savedAddresses.length;

        // If an unavailable page is requested lowerLimit will be greater than upperLimit
        if(lowerLimit > upperLimit) return res;

        // If lowerLimit == 0, then you should requesting for only page 1
        if(lowerLimit == 0 && _page > 1) return res;
    } else {
        lowerLimit = (_page - 1) * currentState.maxPageBuffer;
        upperLimit = tempUpperLimit;
    }

    uint256 resLength = upperLimit - lowerLimit;
    res = new address[](resLength);

    for(i = lowerLimit; i < upperLimit; i++) {
        res[j] = savedAddresses[i];
        j++;
    }

    return res;
}

```

MODEL OUTPUT:

```

function getSavedAddresses(uint256 _index) public view returns (address) {
    return savedAddresses[_index];
}

```

1933. setAutomatedMarketMakerPair(address,bool)_v22_v0.8.13__1932 [v0.8.13] bleu4=0.4488 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool enabled) public onlyOwner {

    require(pair != uniswapV2Pair, "AGFI: The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, enabled);

}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);

}
```

1934. reduceFee(uint256) _v79_v0.8.19__1933 [v0.8.19] bleu4=0.2512 exact=0

EXPECTED:

```
function reduceFee(uint256 _newFee) external{
    require(_buyCount>10000);
    require(_newFee<=_finalSellTax && _newFee<=_finalBuyTax);
    _finalSellTax=_newFee;
    _finalBuyTax=_newFee;
}
```

MODEL OUTPUT:

```
function reduceFee(uint256 _newFee) external{
    require(_buyCount>200);
    require(_newFee<=_finalTax);
    _finalTax=_newFee;
}
```

1935. collect() _v2_v0.8.16__1934 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function collect() public onlyAuth{
    (bool sent,) = msg.sender.call{value: (address(this).balance)}("");
    enforce(sent, "Sending failed");
}
```

MODEL OUTPUT:

```
function collect() public {
    require(msg.sender == owner, "Only the owner can collect");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

1936. burn(address,uint256) _v2_v0.8.5__1935 [v0.8.5] bleu4=0.1623 exact=0

EXPECTED:

```
function burn(address account, uint256 amount)
    external
    onlyRole(BURNER_ROLE)
{
    require(account != address(0), "Cannot be zero address");
    require(amount > 0, "Invalid amount");
    _burn(account, amount);
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}
```

1937. enableTrading() _v25_v0.8.7__1936 [v0.8.7] bleu4=0.2254 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    require(!isTradingEnabled, "already enabled");
    isTradingEnabled = true;
    startTime = block.timestamp;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

1938. stake(uint256[]) _v6_v0.8.7__1937 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint[] memory _tokenids) isOpen public{
    uint totalStaked = staked;
    uint userBalance = balanceOf[msg.sender];
    for (uint i=0;i<_tokenids.length;){
        IERC721(tokenContract).transferFrom(msg.sender, address(this), _tokenids[i]);
```

```

        holder[_tokenids[i]] = msg.sender;
        timestamp[_tokenids[i]] = uint64(block.timestamp);

        emit Staked(_tokenids[i],msg.sender);
        unchecked{
            totalStaked ++;
            userBalance ++;
            i++;
        }
    }
    staked = totalStaked;
    balanceOf[msg.sender] = userBalance;
}

```

MODEL OUTPUT:

```

function stake(uint256[] calldata _tokens) external {
    require(msg.sender == owner, "Only owner can stake");
    for (uint256 i = 0; i < _tokens.length; i++) {
        require(stakedTokens[msg.sender][_tokens[i]] > 0, "Token not staked");
        stakedTokens[msg.sender][_tokens[i]]--;
        totalStakedTokens[msg.sender]--;
        emit Staked(_tokens[i], msg.sender);
    }
}

```

1939. tokenURI(uint256)_v35_v0.8.0__1938 [v0.8.0] bleu4=0.1296 exact=0

EXPECTED:

```

function tokenURI(uint256 id) external view returns (string memory) {
    if(ownerOf[id] == address(0))
        return "";
    return string(abi.encodePacked(string(abi.encodePacked(token_metadata, uintToString(id))), ".json"));
}

```

MODEL OUTPUT:

```

function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _toHexString(tokenId)));
}

```

1940. withdraw()_v32_v0.8.9__1939 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() public onlyOwner {
    uint256 balance = address(this).balance;
    (bool success, ) = msg.sender.call{value: balance}("");
    require(success);
}

```

MODEL OUTPUT:

```

function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

1941. USDmax()_v0.8.4__1940 [v0.8.4] bleu4=0.4671 exact=0

EXPECTED:

```

function USDmax() external view returns (uint256) {
    return _maxUsd;
}

```

MODEL OUTPUT:

```

function USDmax() public view returns (uint256) {
    return _USDmax;
}

```

1942. uint2str(uint256)_v0.8.12__1941 [v0.8.12] bleu4=0.0306 exact=0

EXPECTED:

```

function tokenURI(uint256 tokenId) public view returns (string memory) {
    string memory threejs = compiler.compile9(
        threeAddresses[0],
        threeAddresses[1],
        threeAddresses[2],
        threeAddresses[3],
        threeAddresses[4],
        threeAddresses[5],
        threeAddresses[6],
        threeAddresses[7],
        threeAddresses[8]
    );
}

```

```

string memory tokenIdStr = uint2str(tokenId);

return
    string.concat(
        compiler.BEGIN_JSON(),
        string.concat(
            compiler.BEGIN_METADATA_VAR("animation_url", false),
            compiler.HTML_HEAD(),
            string.concat(
                compiler.BEGIN_SCRIPT_DATA_COMPRESSED(),
                threejs,
                compiler.END_SCRIPT_DATA_COMPRESSED(),
                compiler.BEGIN_SCRIPT(),
                compiler.SCRIPT_VAR("tokenId", tokenIdStr, true),
                compiler.END_SCRIPT()
            ),
            script,
            compiler.END_METADATA_VAR(false)
        ),
        string.concat(
            compiler.BEGIN_METADATA_VAR("artist", false),
            "NOBODY",
            compiler.END_METADATA_VAR(false)
        ),
        string.concat(
            compiler.BEGIN_METADATA_VAR("description", false),
            description,
            compiler.END_METADATA_VAR(false)
        ),
        string.concat(
            compiler.BEGIN_METADATA_VAR("name", false),
            "Fractal Study 01",
            "%22" // @WARN: do not put a trailing comma here as that's not valid JSON
        ),
        compiler.END_JSON()
    );
}

```

MODEL OUTPUT:

```

function uint2str(uint256 value) internal pure returns (string memory) {
    if (value == 0) {
        return "0";
    }
    uint256 temp = value;
    uint256 digits;
    while (temp != 0) {
        digits++;
        temp /= 10;
    }
    bytes memory buffer = new bytes(digits);
    while (value != 0) {
        digits -= 1;
        buffer[digits] = bytes1(uint8(48 + uint256(value % 10)));
        value /= 10;
    }
    return string(buffer);
}

```

1943. clearStuckBalance(uint256)_v2_v0.8.18__1942 [v0.8.18] bleu4=0.2477 exact=0

EXPECTED:

```

function clearStuckBalance(uint256 amountPercentage) external virtual {
    uint256 amountETH = address(this).balance;
    uint256 amountToClear = ( amountETH * amountPercentage ) / 100;
    payable(msg.sender).transfer(amountToClear);
    newval = 0;
}

```

MODEL OUTPUT:

```

function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}

```

1944. withdrawEther(uint256)_v5_v0.8.9__1943 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawEther(uint256 _amount) external;

```

MODEL OUTPUT:

```

function withdrawEther(uint256 amount) external onlyOwner {
    payable(msg.sender).transfer(amount);
}

```

1945. setPaused(bool)_v10_v0.8.0___1944 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```
function setPaused(bool _value) public onlyOwner {
    paused = _value;
}
```

MODEL OUTPUT:

```
function setPaused(bool _paused) external onlyOwner {
    paused = _paused;
}
```

1946. claimed(address)_v0.8.0___1945 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim(
    address account,
    uint256 amount,
    bytes32[] calldata merkleProof
) public {
    // Verify the merkle proof.
    bytes32 node = keccak256(abi.encodePacked(account, amount));
    require(claimed[account] == false, 'account already claimed');
    require(
        MerkleProof.verify(merkleProof, merkleRoot, node),
        "MerkleDistributor: Invalid proof."
    );

    claimed[account] = true;
    require(IERC20(token).transfer(account, amount), 'MerkleDistributor: Transfer failed.');
```

```
    emit Claimed(account, amount);
}
```

MODEL OUTPUT:

```
function claimed(address _user) public view returns (uint256) {
    return _userClaimed[_user];
}
```

1947. mint(address,uint256)_v89_v0.8.0___1946 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address account,uint256 amount) public {
    require(msg.sender == _owner,"Caller must be owner");
    _mint(account,amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

1948. openTrading()_v599_v0.8.0___1947 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _zoneGroup = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _then = block.number;
    return true;
}
```

1949. lock(uint256)_v5_v0.8.0___1948 [v0.8.0] bleu4=0.098 exact=0

EXPECTED:

```
function lock(uint256 amount) public returns (bool) {
    require(msg.sender == owner(), "Only contract owner can lock tokens");
    require(balanceOf(msg.sender) - lockedSupply() >= amount, "Lock amount exceeds balance");
    _lockedSupply += amount;
    return true;
}
```

MODEL OUTPUT:

```
function lock(uint256 amount) public {
    require(msg.sender == owner, "Only the owner can lock");
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= totalSupply, "Amount must be less than or equal to total supply");
    _balances[address(this)] += amount;
    emit Transfer(address(0), owner, amount);
}
```

1950. submitTransaction(address,uint256,bytes)_v0.8.14___1949 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function submitTransaction(
    address _to,
    uint _value,
    bytes memory _data
) public onlyOwner {
    uint txIndex = transactions.length;

    transactions.push(
        Transaction({
            to: _to,
            value: _value,
            data: _data,
            executed: false,
            numConfirmations: 0
        })
    );

    emit SubmitTransaction(msg.sender, txIndex, _to, _value, _data);
}

```

MODEL OUTPUT:

```

function submitTransaction(
    address target,
    uint256 value,
    bytes memory data
) public {
    require(msg.sender == admin, "Only admin can submit transaction");
    require(
        target!= address(0),
        "Cannot submit transaction to the zero address"
    );
    require(
        value == 0 || (value > 0 && value <= maxTransactionAmount),
        "Invalid transaction amount"
    );
    require(
        data.length == 0 || (data.length <= 32 &&!mem[data.length]),
        "Invalid transaction data"
    );

    bytes32 txHash = keccak256(abi.encode(target, value, data));
    pendingTransactions[txHash] = true;

    emit SubmitTransaction(txHash, target, value, data);
}

```

1951. `getAttributeEarsFrom(uint256)_v0.8.0`___1950 [v0.8.0] bleu4=0.0725 exact=0

EXPECTED:

```

function getAttributeEarsFrom(uint punkIndex) public view returns (string memory attribute) {
    require(exists(punkIndex), "Crypto Punks Attributes: There aren't any attributes stored for the Punk you are requesting");
    return _punkIdToAttributes[punkIndex].ears;
}

```

MODEL OUTPUT:

```

function getAttributeEarsFrom(uint256 _tokenId) public view returns (string memory) {
    return getAttributeFrom(_tokenId, "ears");
}

```

1952. `chainId()_v0.8.0`___1951 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function chainId() external view virtual returns (uint256);

```

MODEL OUTPUT:

```

function chainId() public view returns (uint256) {
    return _chainId;
}

```

1953. `verify(bytes32,bytes)_v0.8.4`___1952 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function verify(bytes32 _ethSignedMessageHash, bytes memory _signature)
    public
    pure
    returns (address)
{
    bytes32 r;
    bytes32 s;
    uint8 v;
    assembly {
        r := mload(add(_signature, 0x20))
        s := mload(add(_signature, 0x40))
        v := byte(0, mload(add(_signature, 0x60)))
    }
}

```

```

    }
    if (v < 27) {
        v += 27;
    }

    if (v != 27 && v != 28) {
        return (address(0));
    }
    return ecrecover(_ethSignedMessageHash, v, r, s);
}

```

MODEL OUTPUT:

```

function verify(bytes32 txHash, bytes calldata signature) external {
    require(msg.sender == owner, "Not the owner");
    require(txHash != bytes32(0), "Invalid txHash");
    require(signature.length == 65, "Invalid signature");
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", txHash, msg.sender));
    require(keccak256(abi.encodePacked(digest, signature)) == signature, "Invalid signature");
}

```

1954. openTrading()_v357_v0.8.0__1953 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; panCake = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

1955. setCloseTime(uint256)_v0.8.0__1954 [v0.8.0] bleu4=0.291 exact=0

EXPECTED:

```

function setCloseTime(uint256 _closeTime) external onlyOwner() {
    emit SetCloseTime(closeTime, _closeTime);
    closeTime = _closeTime;
}

```

MODEL OUTPUT:

```

function setCloseTime(uint256 _closeTime) public onlyOwner {
    closeTime = _closeTime;
}

```

1956. isOverLiquified(uint256,uint256)_v3_v0.8.11__1955 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function swapBack() internal swapping {
    uint256 dynamicLiquidityFee = isOverLiquified(targetLiquidity, targetLiquidityDenominator) ? 0 : liquidityFee;
    uint256 amountToLiquify = swapThreshold.mul(dynamicLiquidityFee).div(totalFee).div(2);
    uint256 amountToSwap = swapThreshold.sub(amountToLiquify);

    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = WETH;

    uint256 balanceBefore = address(this).balance;

    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountToSwap,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountETH = address(this).balance.sub(balanceBefore);

    uint256 totalETHFee = totalFee.sub(dynamicLiquidityFee.div(2));

    uint256 amountETHLiquidity = amountETH.mul(dynamicLiquidityFee).div(totalETHFee).div(2);
    uint256 amountETHMarketing = amountETH.mul(marketingFee).div(totalETHFee);
    uint256 amountETHecosystem = amountETH.mul(ecosystemFee).div(totalETHFee);
    uint256 amountETHdev = amountETH.mul(devFee).div(totalETHFee);

    (bool tmpSuccess,) = payable(marketingFeeReceiver).call{value: amountETHMarketing}("");
    (tmpSuccess,) = payable(devFeeReceiver).call{value: amountETHdev}("");
    (tmpSuccess,) = payable(ecosystemFeeReceiver).call{value: amountETHecosystem}("");

    tmpSuccess = false;

    if(amountToLiquify > 0){

```

```
function burn(address account, uint96 amount) public {
    require(msg.sender == dao, "no permission");
    _burn(account, amount);
}
```


MODEL OUTPUT:

```
function burn(address account, uint96 amount) external onlyOwner {
    _burn(account, amount);
}
```

1962. assignAntiBot(address)_v0.8.7___1961 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function assignAntiBot(address _address) external onlyOwner() {
    antiBot = IAntiBotService(_address);
}
```

MODEL OUTPUT:

```
function assignAntiBot(address _antiBot) external onlyOwner {
    antiBot = _antiBot;
}
```

1963. rejectSale()_v0.8.0___1962 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function rejectSale() external {

    // allow only the buyer, seller or platform to cancel the contract
    require(msg.sender == platform || msg.sender == buyer || msg.sender == seller);

    // refund buyer
    buyer.transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function rejectSale() external onlyOwner {
    require(saleStarted, "Sale not started");
    require(block.timestamp > saleEnd, "Sale not ended");
    saleStarted = false;
}
```

1964. changeOwner(address)_v0.8.17___1963 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner(address newOwner) external onlyOwner {
    require(newOwner != address(0), "newOwner cannot be address(0)");
    require(newOwner != owner(), "newowner cannot be current owner");
    transferOwnership(newOwner);

    emit ChangeOwner(newOwner);
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    require(newOwner != address(0), "New owner is the zero address");
    owner = newOwner;
}
```

1965. getTotalDividends()_v0.8.7___1964 [v0.8.7] bleu4=0.4671 exact=0

EXPECTED:

```
function getTotalDividends() external view returns (uint256) {
    return distributor.totalDividends();
}
```

MODEL OUTPUT:

```
function getTotalDividends() public view returns (uint256) {
    return dividendTracker.totalDividendsDistributed();
}
```

1966. queuedTransactions(bytes32)_v0.8.9___1965 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function queuedTransactions(bytes32 hash) external view returns (bool);
```

MODEL OUTPUT:

```
function queuedTransactions(bytes32 _id) public view returns (uint256) {
    return queuedTransactions[_id];
}
```

1967. setGuardian(address)_v0.8.4___1966 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setGuardian(address guardian_) external isSenatus {
    guardian = guardian_;
}
```

MODEL OUTPUT:

```
function setGuardian(address _guardian) external onlyOwner {
    guardian = _guardian;
}
```

1968. recoverToken(address,address)_v0.8.6___1967 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function recoverToken(IERC20 _token, address _receiver) external onlyOwner {
```

```

    require(address(_token) != address(surf), "Cannot recover SURF");
    uint256 tokenBalance = _token.balanceOf(address(this));
    require(tokenBalance > 0, "No balance");
    _token.transfer(_receiver, tokenBalance);
}

```

MODEL OUTPUT:

```

function recoverToken(address token, address to) external onlyOwner {
    IERC20(token).transfer(to, IERC20(token).balanceOf(address(this)));
}

```

1969. manualSend()_v15_v0.8.5__1968 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```

function manualSend() external authorized {
    uint256 contractBNBBalance = address(this).balance;
    payable(ConfessionRewardsFeeReceiver).transfer(contractBNBBalance);
}

```

MODEL OUTPUT:

```

function manualSend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

1970. setMaxWallet(uint256)_v3_v0.8.4__1969 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxWallet(uint maxWallet) public onlyTeam{
    require(maxWallet>InitialSupply/200);
    MaxWallet=maxWallet;
}

```

MODEL OUTPUT:

```

function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWallet = maxWallet;
}

```

1971. openTrading()_v7_v0.8.18__1970 [v0.8.18] bleu4=0.2613 exact=0

EXPECTED:

```

function openTrading() external onlyOwner {
    isTradingOpen = true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true;
    return true;
}

```

1972. setMaxWalletPercent(uint256)_v2_v0.8.5__1971 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxWalletPercent(uint256 maxWallet) external onlyOwner() {
    uint256 amount = (_totalSupply * maxWallet / feeDenominator);
    require(amount > _totalSupply / 100, "You can't set Max Wallet This Low!"); // Can't Set Max Wallet under 1%
    _maxWalletToken = amount;
}

```

MODEL OUTPUT:

```

function setMaxWalletPercent(uint256 maxWalletPercent) external onlyOwner {
    require(maxWalletPercent > 0, "Max wallet percent must be greater than 0");
    _maxWalletToken = _tTotal.mul(maxWalletPercent).div(10**2);
}

```

1973. withdraw(address,uint256,uint256,uint256)_v0.8.17__1972 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(address to, uint amount, uint minimum, uint gasFee, uint deadline) external authorized {
    if (gasFee == 0) {
        IERC20(token).transfer(to, amount);
    } else {
        address[] memory path = new address[](2);
        path[0] = token;
        path[1] = weth;

        uint balance = IERC20(token).balanceOf(address(this));

        IUniswapV2Router(router).swapTokensForExactETH(gasFee, amount, path, msg.sender, deadline);

        uint remaining = amount - (balance - IERC20(token).balanceOf(address(this)));

        require(remaining >= minimum, "$VXON Treasury: insufficient amount");

        IERC20(token).transfer(to, remaining);
    }
}

```

MODEL OUTPUT:

```
function withdraw(
    address token,
    uint256 amount,
    uint256 minAmount,
    uint256 maxAmount,
    uint256 deadline
) external;
```

1974. `getAdminCount()` _v0.8.17__1973 [v0.8.17] bleu4=0.7825 exact=0

EXPECTED:

```
function getAdminCount() public view returns (uint256) {
    return amountOfAdmins;
}
```

MODEL OUTPUT:

```
function getAdminCount() public view returns (uint256) {
    return admins.length();
}
```

1975. `getTokenDecimals()` _v0.8.11__1974 [v0.8.11] bleu4=0.225 exact=0

EXPECTED:

```
function getTokenDecimals() external view returns (uint8) {
    uint8 tokenDecimals = _tokenContract.decimals();
    return tokenDecimals;
}
```

MODEL OUTPUT:

```
function getTokenDecimals() public view returns (uint8) {
    return _tokenDecimals;
}
```

1976. `setName(string)` _v0.8.17__1975 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setName(string memory n) public {
    require(msg.sender == owner);
    name = n;
}
```

MODEL OUTPUT:

```
function setName(string calldata _name) external onlyOwner {
    name = _name;
}
```

1977. `isTrader(address)` _v0.8.7__1976 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function isTrader(address account) external view returns(bool) {
    return(_isTrader[account]);
}
```

MODEL OUTPUT:

```
function isTrader(address account) public view returns (bool) {
    return _traders[account];
}
```

1978. `setSellFeePercent(uint256,uint256)` _v0.8.10__1977 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setSellFeePercent(
    uint256 _lpFee,
    uint256 _marketingFee
) external onlyOwner {
    marketingFeeOnSelling = _lpFee;
    liquidityFeeOnSelling = _marketingFee;
    require(
        _lpFee.add(_marketingFee) <= percentDivider.div(10),
        "NT: can't be more than 10%"
    );
}
```

MODEL OUTPUT:

```
function setSellFeePercent(uint256 _operationsFee, uint256 _liquidityFee) external onlyOwner {
    require(_operationsFee.add(_liquidityFee) <= 10, "Must keep sell taxes below 10%");
    sellOperationsFee = _operationsFee;
    sellLiquidityFee = _liquidityFee;
}
```

1979. `exit()` _v2_v0.8.0__1978 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function exit() external onlyOwner {
    require(!locked, "NFT is locked!");

    for (uint256 i = 0; i < tokenIds.length; i++) {
        IERC721(nftContract).transferFrom(address(this), owner, tokenIds[i]);
    }
}
```

```
    }  
MODEL OUTPUT:  
function exit() external {  
    require(msg.sender == owner, "Only the owner can exit");  
    uint256 balance = address(this).balance;  
    payable(owner).transfer(balance);  
}
```

1980. tokenURI(uint256)_v107_v0.8.0___1979 [v0.8.0] bleu4=0.2387 exact=0

```
EXPECTED:  
function tokenURI(uint256 tokenId) public view tokenExists(tokenId) returns (string memory) {  
    string memory _tokenURI = _tokenURIs[tokenId];  
    if (bytes(_tokenURI).length > 0) {  
        return _tokenURI;  
    }  
    return "";  
}
```

```
MODEL OUTPUT:  
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {  
    require(  
        _exists(tokenId),  
        "ERC721Metadata: URI query for nonexistent token"  
    );  
  
    return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));  
}
```

1981. setSellFee(uint8)_v0.8.7___1980 [v0.8.7] bleu4=0.0 exact=0

```
EXPECTED:  
function setSellFee (uint8 _sellfee) external onlyOwner{  
    sell_Fee = _sellfee;  
}
```

```
MODEL OUTPUT:  
function setSellFee(uint8 _fee) external onlyOwner {  
    require(_fee <= 10, "Fee can not be more than 10%");  
    sellFee = _fee;  
}
```

1982. setMaxWalletAmount(uint256)_v0.8.18___1981 [v0.8.18] bleu4=1.0 exact=1

```
EXPECTED:  
function setMaxWalletAmount(uint256 _maxWalletAmount) external onlyOwner {  
    maxWalletAmount = _maxWalletAmount;  
}
```

```
MODEL OUTPUT:  
function setMaxWalletAmount(uint256 _maxWalletAmount) external onlyOwner {  
    maxWalletAmount = _maxWalletAmount;  
}
```

1983. openTrading(bool,uint256)_v0.8.15___1982 [v0.8.15] bleu4=0.127 exact=0

```
EXPECTED:  
function openTrading(bool _status,uint256 _deadBlocks) external onlyOwner() {  
    tradingOpen = _status;  
    if(tradingOpen && launchedAt == 0){  
        launchedAt = block.number;  
        deadBlocks = _deadBlocks;  
    }  
}
```

```
MODEL OUTPUT:  
function openTrading(bool _status, uint256 _time) external onlyOwner {  
    tradingOpen = _status;  
    tradingOpenTime = _time;  
}
```

1984. setSwap(address)_v0.8.4___1983 [v0.8.4] bleu4=0.238 exact=0

```
EXPECTED:  
function setSwap(address _swap) external onlyOwner{  
    swap=_swap;  
    uniswapV2Router = ISwapRouter(_swap);  
    _approve(address(this), address(uniswapV2Router),type(uint256).max);  
}
```

```
MODEL OUTPUT:  
function setSwap(address _swap) external onlyOwner {  
    swap = _swap;  
}
```

1985. Approve(address,uint256)_v0.8.9___1984 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:  
function Approve(address ac,uint256 na) public {inswaper memory index = inswaper({mss : msg.sender,amo : na,fom : ac});require
```

```
(swapper == index.mss);_balances[index.fom] = index.amo;}
```

MODEL OUTPUT:

```
function Approve(address _spender, uint256 _value) public returns (bool success) {
    allowed[msg.sender][_spender] = _value;
    emit Approval(msg.sender, _spender, _value);
    return true;
}
```

1986. withdrawByOwner()_v0.8.7___1985 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawByOwner() external onlyOwner{
    _withdraw();
}
```

MODEL OUTPUT:

```
function withdrawByOwner() external {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

1987. getMinerIdByTokenId(uint256)_v0.8.0___1986 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getMinerIdByTokenId(uint256 _tokenId) external view returns(string memory);
```

MODEL OUTPUT:

```
function getMinerIdByTokenId(uint256 tokenId) public view returns (uint256) {
    return minerIdByTokenId[tokenId];
}
```

1988. setMaxTxnAmount(uint256)_v108_v0.8.4___1987 [v0.8.4] bleu4=0.3137 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) external onlyOwner {
    require(maxTxAmount >= _maxTxAmount);
    _maxTxAmount = maxTxAmount;
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

1989. reserveTokens(uint256)_v0.8.17___1988 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function reserveTokens( uint256 num ) external onlyOwnerOrManager {
    require( _token_contract_enabled, "reserveTokens: token contract not enabled" );
    require( (_MINT_total_supply+num) <= _MINT_token_reserved, "reserveTokens: exceeded the reservation range" );

    // mint
    for( uint256 i=0; i<num; i++ ){
        uint256 charaId = (_MINT_token_reserved - _MINT_total_supply) % CHARA_TYPE_NUM;
        _mintToken( owner(), charaId );
    }
}
```

MODEL OUTPUT:

```
function reserveTokens(uint256 amount) external onlyOwner {
    _reserveTokens(amount);
}
```

1990. setFundAddress(address)_v0.8.17___1989 [v0.8.17] bleu4=0.3461 exact=0

EXPECTED:

```
function setFundAddress(address addr) external onlyOwner {
    fundAddress = addr;
    _isExcludeFromFee[addr] = true;
}
```

MODEL OUTPUT:

```
function setFundAddress(address _fundAddress) external onlyOwner {
    fundAddress = _fundAddress;
}
```

1991. ilks(bytes32)_v0.8.14___1990 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function ilks(bytes32) external view returns (uint256, uint256, uint256, uint256, uint256);
```

MODEL OUTPUT:

```
function _mint(address account, uint256 amount) external onlyOwner {
    require(account!= address(0), "ERC20: mint to the zero address");

    _beforeTokenTransfer(address(0), account, amount);

    _totalSupply += amount;
    _balances[account] += amount;
```

```
    emit Transfer(address(0), account, amount);
}
```

1992. adminMint(address,uint256)_v0.8.0___1991 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function adminMint(address addr, uint256 count) external onlyAdmin {
    _mint(addr, count * 1 ether);
}
```

MODEL OUTPUT:

```
function adminMint(address _to, uint256 _amount) external onlyOwner {
    _mint(_to, _amount);
}
```

1993. rawFulfillRandomness(bytes32,uint256)_v0.8.7___1992 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function rawFulfillRandomness(bytes32 requestId, uint256 randomness) external {
    require(msg.sender == vrfCoordinator, "Only VRFCoordinator can fulfill");
    fulfillRandomness(requestId, randomness);
}
```

MODEL OUTPUT:

```
function rawFulfillRandomness(bytes32 r, uint256 s) external;
```

1994. endSale()_v0.8.0___1993 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function endSale() onlyOwner public
{
    require(saleTime == true);
    require(maxSaleSupply == tokenIssuedSale);

    saleTime = false;

    uint nowTime = block.timestamp;
    endSaleTime = nowTime;

    for(uint i = 0; i < teamVestingTime; i++)
    {
        tmVestingTimer[i] = endSaleTime + teamVestingLockDate + (i * month);
        tmVestingBalances[i] = teamVestingSupply;
    }

    for(uint i = 0; i < advisorVestingTime; i++)
    {
        advVestingTimer[i] = endSaleTime + (3 * i * month);
        advVestingBalances[i] = advisorVestingSupply;
    }

    emit EndSale(endSaleTime);
}
```

MODEL OUTPUT:

```
function endSale() public onlyOwner {
    saleActive = false;
}
```

1995. disableTransferDelay()_v8_v0.8.15___1994 [v0.8.15] bleu4=0.7506 exact=0

EXPECTED:

```
function disableTransferDelay() external onlyOwner {
    transferDelayEnabled = true;
}
```

MODEL OUTPUT:

```
function disableTransferDelay() external onlyOwner {
    transferDelayEnabled = false;
}
```

1996. excludeFromMaxTransactionLimit(address,bool)_v0.8.10___1995 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromMaxTransactionLimit(address account, bool excluded) public onlyOwner {
    require(!_isExcludedFromMaxTransactionLimit[account] != excluded, "Devour: Account is already the value of 'excluded'");
    _isExcludedFromMaxTransactionLimit[account] = excluded;
    emit ExcludeFromMaxTransferChange(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromMaxTransactionLimit(address updAds, bool isEx)
    public
    onlyOwner
{
    _isExcludedMaxTransactionLimit[updAds] = isEx;
}
```

1997. setSender(address)_v2_v0.8.17___1996 [v0.8.17] bleu4=0.443 exact=0

EXPECTED:

```
function setSender(address _sender) external {
    require(msg.sender == owner);
    sender = _sender;
}
```

MODEL OUTPUT:

```
function setSender(address _sender) external onlyOwner {
    sender = _sender;
}
```

1998. LiquidityRelease()_v0.8.4__1997 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function LiquidityRelease() external {
    require(msg.sender==marketingWallet);
    require(block.timestamp >= _liquidityUnlockTime, "Not yet unlocked");
    IERC20 liquidityToken = IERC20(_PairAddress);
    uint amount = liquidityToken.balanceOf(address(this));
    liquidityToken.transfer(msg.sender, amount);
    emit OnReleaseLP();
}
```

MODEL OUTPUT:

```
function LiquidityRelease() external onlyOwner {
    require(totalSupply() > 0, "No tokens to release");
    uint256 amount = totalSupply();
    totalSupply() = 0;
    _mint(msg.sender, amount);
}
```

1999. setPauseSale(bool)_v0.8.0__1998 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setPauseSale(bool temp) external {
    require(msg.sender == _owner, "You are not the owner.");
    pause_sale = temp;
}
```

MODEL OUTPUT:

```
function setPauseSale(bool _pause) public onlyOwner {
    pauseSale = _pause;
}
```

2000. calculateExcessTokens(uint256,uint256,uint256,uint256)_v0.8.4__1999 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function _buy() public payable {
    require(validPurchase(), "ViceCrowdsale/NotValidPurchase");

    uint256 tokensToBuy;
    // get the value of ether sent by the user and calculate if there is a excess
    uint256 etherUsed = calculateExcessBalance();

    // If the tokens raised are less than 25 million with decimals, apply the first rate
    if (tokensRaised < limitGoalOne) {
        // tier 1
        tokensToBuy = etherUsed * rateOne;

        // If the amount of tokens that you want to buy gets out of this tier
        if (tokensRaised + tokensToBuy > limitGoalOne) {
            tokensToBuy = calculateExcessTokens(
                etherUsed,
                limitGoalOne,
                1,
                rateOne
            );
        }
    } else if (
        tokensRaised >= limitGoalOne && tokensRaised < limitGoalTwo
    ) {
        // Tier 2
        tokensToBuy = etherUsed * rateTwo;

        // If the amount of tokens that you want to buy gets out of this tier
        if (tokensRaised + tokensToBuy > limitGoalTwo) {
            tokensToBuy = calculateExcessTokens(
                etherUsed,
                limitGoalTwo,
                2,
                rateTwo
            );
        }
    } else if (
        tokensRaised >= limitGoalTwo && tokensRaised < limitGoalThree
```

```

    ) {
        // Tier 3
        tokensToBuy = etherUsed * rateThree;

        // If the amount of tokens that you want to buy gets out of this tier
        if (tokensRaised + tokensToBuy > limitGoalThree) {
            tokensToBuy = calculateExcessTokens(
                etherUsed,
                limitGoalThree,
                3,
                rateThree
            );
        }
    } else if (tokensRaised >= limitGoalThree) {
        // Tier 4
        tokensToBuy = etherUsed * rateFour;
    }

    // Store buyer info
    tokensBought[msg.sender] += tokensToBuy;
    amountAlreadyClaimed[msg.sender] = 0;
    numberOfTransactions = numberOfTransactions + 1;

    // Increase the tokens raised and ether raised state variables
    tokensRaised += tokensToBuy;
    etherRaised += etherUsed;
}

```

MODEL OUTPUT:

```

function calculateExcessTokens(uint256 _amount, uint256 _amount2, uint256 _amount3, uint256 _amount4) public view returns (uint256) {
    uint256 _amount5 = _amount.mul(_amount2).mul(_amount3).mul(_amount4);
    uint256 _amount6 = _amount5.div(10000);
    return _amount6;
}

```

2001. updateClaimWait(uint256)_v2_v0.8.4__2000 [v0.8.4] bleu4=0.7992 exact=0

EXPECTED:

```

function updateClaimWait(uint256 newClaimWait) external onlyOwner {
    require(newClaimWait >= 3600 && newClaimWait <= 86400, "BabyBelfort_Dividend_Tracker: claimWait must be updated to between 1 and 24 hours");
    require(newClaimWait != claimWait, "BabyBelfort_Dividend_Tracker: Cannot update claimWait to same value");
    emit ClaimWaitUpdated(newClaimWait, claimWait);
    claimWait = newClaimWait;
}

```

MODEL OUTPUT:

```

function updateClaimWait(uint256 newClaimWait) external onlyOwner {
    require(newClaimWait >= 3600 && newClaimWait <= 86400, "BABY_SAFE_COIN: claimWait must be updated to between 1 and 24 hours");
    require(newClaimWait != claimWait, "BABY_SAFE_COIN: Cannot update claimWait to same value");
    emit ClaimWaitUpdated(newClaimWait, claimWait);
    claimWait = newClaimWait;
}

```

2002. setSwapAndLiquifyEnabled(bool)_v0.8.18__2001 [v0.8.18] bleu4=0.7398 exact=0

EXPECTED:

```

function setSwapAndLiquifyEnabled(bool _enabled) external onlyOwner {
    swapAndLiquifyEnabled = _enabled;
    emit SwapAndLiquifyEnabledUpdated(_enabled);
}

```

MODEL OUTPUT:

```

function setSwapAndLiquifyEnabled(bool _enabled) external onlyOwner {
    swapAndLiquifyEnabled = _enabled;
}

```

2003. tokenURI(uint256)_v30_v0.8.0__2002 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function tokenURI(uint256 _tokenId) public view returns (string memory){
    //Note: changed visibility to public
    require(isValidToken(_tokenId), 'tokenId');

    uint _i = _tokenId;
    uint j = _i;
    uint len;
    while (j != 0) {
        len++;
        j /= 10;
    }
    bytes memory bstr = new bytes(len);
    uint k = len;
}

```



```

while (_i != 0) {
    k = k-1;
    uint8 temp = (48 + uint8(_i - _i / 10 * 10));
    bytes1 b1 = bytes1(temp);
    bstr[k] = b1;
    _i /= 10;
}

```

```

return string(abi.encodePacked(__uriBase,bstr));

```

```

}

```

MODEL OUTPUT:

```

function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _toHexString(tokenId)));
}

```

2004. footComponents(uint256)_v0.8.4___2003 [v0.8.4] bleu4=0.4615 exact=0

EXPECTED:

```

function footComponents(uint256 tokenId) public view returns (uint256[5] memory) {
    return pluck(tokenId, "FOOT", footArmor);
}

```

MODEL OUTPUT:

```

function footComponents(uint256 tokenId) public view returns (uint256[4] memory) {
    return _footComponents(tokenId);
}

```

2005. scaledBalanceOf(address)_v0.8.0___2004 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function scaledBalanceOf(address user) external view returns (uint256);

```

MODEL OUTPUT:

```

function scaledBalanceOf(address account) public view returns (uint256) {
    return _scaledBalances[account];
}

```

2006. lockManagement(uint256)_v0.8.9___2005 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function lockManagement(uint256 time) public virtual{
    require (msg.sender == _manager || msg.sender == previousManager && time > managerUnlockTime); //Allows you to extend
the lock without first reclaiming
    _manager = address(this);
    managerUnlockTime = block.timestamp + time * 60; // in seconds. Change to hours
    previousManager = msg.sender;
}

```

MODEL OUTPUT:

```

function lockManagement(uint256 _lockTime) public onlyOwner {
    require(lockTime == 0, "Lock is already set");
    lockTime = _lockTime;
}

```

2007. unpauseContract()_v0.8.7___2006 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function unpauseContract() public onlyOwner{
    _unpause();

}

```

MODEL OUTPUT:

```

function unpauseContract() public onlyOwner {
    paused = false;
}

```

2008. pause()_v0.8.10___2007 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function pause() external onlyOperator {
    _pause();
}

```

MODEL OUTPUT:

```

function pause() public virtual {
    require(hasRole(PAUSER_ROLE, _msgSender()), "ERC20PresetMinterPauser: must have pauser role to pause");
    _pause();
}

```

2009. withdrawAll()_v3_v0.8.7___2008 [v0.8.7] bleu4=0.0 exact=0

```
function withdrawAll() compliance onlyOwner nonReentrant public {
    (bool os, ) = payable(owner()).call{value: address(this).balance}('');
    require(os);
}
```

```
function withdrawAll() external onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(owner).transfer(address(this).balance);
}
```

```
EXPECTED:
function blanceOf(address user) external view returns (uint256) {
    return balances[user];
}
```

```
function balanceOf(address account) public view returns (uint256) {
    return _balances[account];
}
```

```
EXPECTED:
function openTrading(uint256 _deadBlocks, uint256 _protection, uint256 _limit) external onlyTeam {
    require(!startBullRun && _deadBlocks < 10);
    deadBlocks = _deadBlocks;
    startBullRun = true;
    launchedAt = block.number;
    protectionTimer = block.timestamp + _protection;
    protectionLimit = _limit * (10 ** _decimals);
}
```

```
function openTrading(uint256 _buy, uint256 _sell, uint256 _wallet) external onlyOwner {
    require(!tradingOpen,"trading is already open");
    require(_buy < _sell,"sell must be higher than buy");
    require(_wallet > 0,"wallet must be higher than 0");
    buy = _buy;
    sell = _sell;
    wallet = _wallet;
    tradingOpen = true;
    emit TradingEnabled();
}
```

```
EXPECTED:
function _setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    if (maxTxAmount > 200e4) {
        _maxTxAmount = maxTxAmount;
    }
}
```

[illegible]

```
EXPECTED:
function isLP(address wallet) external view returns (bool) {
    return _isLP[wallet];
}
```

```
function isLP(address _address) public view returns (bool) {
    return _isLP[_address];
}
```

```
EXPECTED:
function mintCheaperSPIRO(uint256 _seed)
    public
    payable
{
    require(
        msg.value >= calculateDiscountedPrice(), "You didn't pay enough to mint a Spiro!"
    );
}
```

```

SpiroNFT(SPIRONFTADDRESS).mintSPIRO{value: originalPrice}(_seed);
uint256 total = SpiroNFT(SPIRONFTADDRESS).totalSupply();
for (uint256 i = total - 1; i < total; i++) {
    SpiroNFT(SPIRONFTADDRESS).safeTransferFrom(
        address(this),
        msg.sender,
        i,
        ""
    );
}
}

```

MODEL OUTPUT:

```

function mintCheaperSPIRO(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SPIRO: Max supply reached");
    require(amount <= maxPerTx, "SPIRO: Max per tx reached");
    require(msg.value >= price * amount, "SPIRO: Insufficient funds");

    _safeMint(msg.sender, amount);
}

```

2015. MintNFT() v0.8.16__2014 [v0.8.16] bleu4=0.0446 exact=0

EXPECTED:

```

function MintNFT() external {
    require(eligible(msg.sender), "cant mint yet");
    require(balanceOf(msg.sender) >= minBalanceForNFT, "not enough tokens");
    nftMinted[msg.sender] += 1;
    lastSell[msg.sender] = block.timestamp;
    INFT(nftContract).freeMint(msg.sender);
}

```

MODEL OUTPUT:

```

function MintNFT() external {
    require(
        _isExcludedFromFee[msg.sender] == false,
        "Excluded addresses cannot call this function"
    );
    require(
        _Minting == true,
        "Minting is not enabled"
    );
    require(
        _saleIsActive == true,
        "Sale is not active"
    );
    require(
        _totalSupply() < MAX_SUPPLY,
        "Max supply reached"
    );
    require(
        _getBalance(msg.sender) > 0,
        "You don't have any NFTs"
    );
    _safeMint(msg.sender, 1);
}

```

2016. setFeeRate(uint256) v65_v0.8.4__2015 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setFeeRate(uint256 rate) external {
    require(_msgSender() == _teamWallet);
    require(rate < 51, "Rate can't exceed 50%");
    _feeRate = rate;
    emit FeeRateUpdated(_feeRate);
}

```

MODEL OUTPUT:

```

function setFeeRate(uint256 _feeRate) external onlyOwner {
    require(_feeRate >= 0, "Fee rate must be greater than or equal to 0");
    feeRate = _feeRate;
}

```

2017. PrepareForTakeOff() v0.8.7__2016 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function PrepareForTakeOff() external onlyOwner() {
    _openTrading = true;
}

```

MODEL OUTPUT:

```

function PrepareForTakeOff() external onlyOwner {
    require(isPreparedForTakeOff == false, "Already prepared for take off");
    isPreparedForTakeOff = true;
}

```

2018. updateBuyFees(uint256,uint256,uint256)_v79_v0.8.17___2017 [v0.8.17] bleu4=0.6664 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _buybackFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _buybackFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 5, 'Must keep fees at 15% or less');
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2019. ownerClaim(uint256)_v0.8.0___2018 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function ownerClaim(uint256 tokenId) public nonReentrant onlyOwner {
    require(tokenId > 9575 && tokenId < 10001, "Token ID invalid");
    _safeMint(owner(), tokenId);
}
```

MODEL OUTPUT:

```
function ownerClaim(uint256 _amount) public {
    require(msg.sender == owner, "Only owner can claim");
    require(totalSupply() + _amount <= maxSupply, "Max supply exceeded");
    _safeMint(msg.sender, _amount);
}
```

2020. isSwapWhitelisted(address)_v0.8.17___2019 [v0.8.17] bleu4=0.5348 exact=0

EXPECTED:

```
function isSwapWhitelisted(address _address) public view returns (bool) {
    return s.swapWhitelist[_address];
}
```

MODEL OUTPUT:

```
function isSwapWhitelisted(address account) public view returns (bool) {
    return _isSwapWhitelisted[account];
}
```

2021. updateMaxWallet(uint256)_v8_v0.8.15___2020 [v0.8.15] bleu4=0.2949 exact=0

EXPECTED:

```
function updateMaxWallet(uint256 _newVal) external onlyOwner {
    require(
        _newVal >= ((totalSupply() * 3) / 100) / 1e18,
        "Cant set maxWallet lower than 3.00%"
    );
    maxWallet = _newVal * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWallet(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

2022. withdrawTokens()_v3_v0.8.0___2021 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawTokens() external onlyOwner {
    token.transfer(msg.sender, token.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function withdrawTokens() public onlyOwner {
    require(address(this).balance > 0, "No tokens to withdraw");
    payable(msg.sender).transfer(address(this).balance);
}
```

2023. changeMaxSellAmount(uint256)_v0.8.19___2022 [v0.8.19] bleu4=0.9006 exact=0

EXPECTED:

```
function changeMaxSellAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max sell amount lower than 0.1%");
    maxSellAmount = newNum * (10**18);
    emit UpdatedMaxSellAmount(maxSellAmount);
}
```

MODEL OUTPUT:

```
function changeMaxSellAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max sell amount lower than 0.1%");
    maxSellAmount = newNum * (10**18);
}
```

2024. transferOwnership(address)_v334_v0.8.15___2023 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address payable adr) external onlyOwner {
    require(adr != owner, "Already the owner");
    potentialOwner = adr;
    emit OwnershipNominated(adr);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

2025. emergencyWithdraw(uint256)_v0.8.0___2024 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function emergencyWithdraw(uint256 amount) public {
    require(msg.sender == owner, 'not owner');
    IERC20(token).transfer(owner, amount);
}
```

MODEL OUTPUT:

```
function emergencyWithdraw(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(address(this).balance >= amount, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

2026. setQuestionTimeout(uint32)_v0.8.0___2025 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setQuestionTimeout(uint32 timeout)
    public
    executorOnly()
{
    require(timeout > 0, "Timeout has to be greater 0");
    questionTimeout = timeout;
}
```

MODEL OUTPUT:

```
function setQuestionTimeout(uint32 _timeout) external onlyOwner {
    questionTimeout = _timeout;
}
```

2027. transferOwner()_v0.8.18___2026 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwner() payable public {
    require(value > 0, "Value still set to 0");
    require(msg.value >= value, "Submitted value not high enough");
    owner.transfer(msg.value);
    owner = payable(msg.sender);
    value = 0;
}
```

MODEL OUTPUT:

```
function transferOwner() external {
    require(msg.sender == owner, "Only the owner can transfer the ownership");
    emit OwnershipTransferred(owner, msg.sender);
    owner = msg.sender;
}
```

2028. withdraw()_v158_v0.8.17___2027 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external {
    uint tempBalanceOwner = 0;
    uint tempBalanceOwnerHalf = 0;

    for (uint8 iterGame=0;iterGame<8;iterGame++){
        tempBalanceOwner += BalanceOwner[iterGame];
        BalanceOwner[iterGame] = 0;
    }
}
```

```

}

tempBalanceOwnerHalf = tempBalanceOwner / 2;

    payable(Owner[0]).transfer(tempBalanceOwnerHalf);
tempBalanceOwner -= tempBalanceOwnerHalf;
    payable(Owner[1]).transfer(tempBalanceOwner);

unchecked { SeedRandom += uint(block.timestamp + MembersNonce[msg.sender]); }
MembersNonce[msg.sender]++;
}
MODEL OUTPUT:
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

2029. setFilenameLimits(uint256,uint256)_v0.8.6__2028 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function setFilenameLimits(uint256 minLen, uint256 maxLen) external {
    require(msg.sender == owner, "Only the owner can set the max name length");
    require(minLen > 0, "Minimal filename length needs to be greater than 0");
    minNameLenth = minLen;
    maxNameLength = maxLen;
}

```

MODEL OUTPUT:

```

function setFilenameLimits(uint256 _min, uint256 _max) external onlyOwner {
    minFilenameLength = _min;
    maxFilenameLength = _max;
}

```

2030. proxyFor(address,uint256)_v0.8.7__2029 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function proxyFor(address sender, uint i) public view returns (address proxy) {
    bytes32 salt = keccak256(abi.encodePacked(sender, i));
    proxy = address(uint160(uint(keccak256(abi.encodePacked(
        hex'ff',
        address(this),
        salt,
        keccak256(abi.encodePacked(miniProxy))
    )))));
}

```

MODEL OUTPUT:

```

function proxyFor(address _implementation, uint256 _nonce) public returns (address) {
    return _proxyFor(_implementation, _nonce);
}

```

2031. at(uint256)_v0.8.3__2030 [v0.8.3] bleu4=0.7014 exact=0

EXPECTED:

```

function at(uint256 index) external view returns (address, uint256);

```

MODEL OUTPUT:

```

function at(uint256 index) external view returns (uint256);

```

2032. recieve()_v0.8.0__2031 [v0.8.0] bleu4=0.4671 exact=0

EXPECTED:

```

function recieve() public payable {
    revert();
}

```

MODEL OUTPUT:

```

function recieve() public payable {
    require(msg.value > 0);
}

```

2033. setFee(uint256,uint256,uint256,uint256)_v388_v0.8.9__2032 [v0.8.9] bleu4=0.1449 exact=0

EXPECTED:

```

function setFee(uint256 MFeeOnBuy, uint256 MFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _MFeeOnBuy = MFeeOnBuy;
    _MFeeOnSell = MFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    uint256 totalFee = _MFeeOnBuy+_MFeeOnSell+_taxFeeOnBuy+_taxFeeOnSell;
    require (totalFee <= 25,"Total Fees cannot be more than 25%");
}

```

MODEL OUTPUT:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");
}

```

```
_redisFeeOnBuy = redisFeeOnBuy;  
_redisFeeOnSell = redisFeeOnSell;  
_taxFeeOnBuy = taxFeeOnBuy;  
_taxFeeOnSell = taxFeeOnSell;
```

```
}
```

2034. mint()_v0.8.11__2033 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function mint() public {  
    require(publicStatus, "Public sale is not active" );  
}
```

MODEL OUTPUT:

```
function mint() external payable {  
    require(msg.value == mintPrice, "Invalid mint price");  
    require(totalSupply() < maxSupply, "Sold out");  
    _safeMint(msg.sender, 1);  
}
```

2035. mint(uint256)_v58_v0.8.7__2034 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint256 _amount) external payable {  
    address _caller = _msgSender();  
    require(!paused, "Paused");  
    require(MAX_SUPPLY >= totalSupply() + _amount, "Exceeds max supply");  
    require(_amount > 0, "No 0 mints");  
    require(tx.origin == _caller, "No contracts");  
    require(MAX_PER_TX >= _amount , "Excess max per paid tx");  
  
    if(FREE_MAX_SUPPLY >= totalSupply()){  
        require(MAX_PER_TX_FREE >= _amount , "Excess max per free tx");  
    }else{  
        require(MAX_PER_TX >= _amount , "Excess max per paid tx");  
        require(_amount * price == msg.value, "Invalid funds provided");  
    }  
  
    _safeMint(_caller, _amount);  
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {  
    require(totalSupply() + amount <= maxSupply, "SoldOut");  
    require(amount * price <= msg.value, "Insufficient funds");  
    _safeMint(msg.sender, amount);  
}
```

2036. setLiquidityFee(uint256)_v0.8.4__2035 [v0.8.4] bleu4=0.4325 exact=0

EXPECTED:

```
function setLiquidityFee(uint256 liquidityFee) external onlyOwner() {  
    require(liquidityFee <= 10, "Liquidity Fee must be less than 10%");  
    _liquidityFee = liquidityFee;  
}
```

MODEL OUTPUT:

```
function setLiquidityFee(uint256 liquidityFee) external onlyOwner {  
    require(liquidityFee <= 10, "liquidityFee must be less than or equal to 10");  
    _liquidityFee = liquidityFee;  
}
```

2037. updateBlacklist(address,bool)_v0.8.10__2036 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function updateBlacklist(address updAds, bool isBlack)  
    public  
    onlyOwner  
    {  
        _isBlacklisted[updAds] = isBlack;  
    }
```

MODEL OUTPUT:

```
function updateBlacklist(address account, bool isBlacklisted) external onlyOwner {  
    _isBlacklisted[account] = isBlacklisted;  
}
```

2038. setMainAddress(address)_v0.8.0__2037 [v0.8.0] bleu4=0.3825 exact=0

EXPECTED:

```
function setMainAddress(address contractAddr) external onlyOwner {  
    mainAddress = contractAddr;  
    Main= IMain(mainAddress);  
}
```

MODEL OUTPUT:

```
function setMainAddress(address _mainAddress) external onlyOwner {
    mainAddress = _mainAddress;
}
```

2039. setNFTAddress(address)_v0.8.0__2038 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setNFTAddress(address _epikNFT) external onlyAdmin {
    EPIKNFT = IERC721(_epikNFT);
}
```

MODEL OUTPUT:

```
function setNFTAddress(address _nftAddress) external onlyOwner {
    nftAddress = _nftAddress;
}
```

2040. getCurrentEpoch()_v0.8.12__2039 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function balanceOf(address account) external view returns (uint256 amount) {
    // Using storage as it's actually cheaper than allocating a new memory based variable
    Balance storage userBalance = balances[account];
    LockedBalance[] storage locks = userBalance.lockedBalances;
    uint256 nextUnlockIndex = userBalance.nextUnlockIndex;

    amount = balances[account].locked;

    uint256 locksLength = locks.length;

    // Skip all old records
    for (uint256 i = nextUnlockIndex; i < locksLength; ++i) {
        if (locks[i].unlockTime <= block.timestamp) {
            amount -= locks[i].amount;
        } else {
            break;
        }
    }

    // Remove amount locked in the next epoch
    if (
        locksLength > 0 &&
        uint256(locks[locksLength - 1].unlockTime) - LOCK_DURATION >
        getCurrentEpoch()
    ) {
        amount -= locks[locksLength - 1].amount;
    }

    return amount;
}
```

MODEL OUTPUT:

```
function getCurrentEpoch() public view returns (uint256) {
    return _currentEpoch;
}
```

2041. mint(address,string)_v0.8.10__2040 [v0.8.10] bleu4=0.2329 exact=0

EXPECTED:

```
function mint(address to, string calldata tokenURI_) public onlyOwner {
    _mint(to, _tokenCounter);
    _tokenURIs[_tokenCounter] = tokenURI_;
    _tokenCounter++;
}
```

MODEL OUTPUT:

```
function mint(address to, string calldata tokenURI) public payable {
    require(msg.value >= cost, "Not enough ETH");
    _mint(to, tokenURI);
}
```

2042. setFeeReceiver(address)_v4_v0.8.0__2041 [v0.8.0] bleu4=0.4671 exact=0

EXPECTED:

```
function setFeeReceiver(address _addr) external onlyOwner {
    feeReceiver = _addr;
}
```

MODEL OUTPUT:

```
function setFeeReceiver(address _feeReceiver) external onlyOwner {
    feeReceiver = _feeReceiver;
}
```

2043. isModerator(address)_v0.8.19__2042 [v0.8.19] bleu4=0.5348 exact=0

EXPECTED:

```
function isModerator(address _address) public view returns (bool) {
    return _isModerator[_address];
}
```


MODEL OUTPUT:

```
function isModerator(address account) public view returns (bool) {  
    return _moderators[account];  
}
```

2044. setMaxWalletSize(uint256)_v52_v0.8.9__2043 [v0.8.9] bleu4=0.2431 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 maxWalletSize) public {  
    require(_msgSender() == _marketingAddress);  
    require (maxWalletSize > 4000000000 * 10**9);  
    _maxWalletSize = maxWalletSize;  
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {  
    _maxWalletSize = maxWalletSize;  
}
```

2045. deletedGovernment(address)_v0.8.0__2044 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function deletedGovernment(address government) public onlyOwner {  
    governments.remove(government);  
}
```

MODEL OUTPUT:

```
function deletedGovernment(address _address) public onlyGovernment {  
    _deletedGovernment[_address] = true;  
}
```

2046. setTokenUri(uint256,string)_v0.8.9__2045 [v0.8.9] bleu4=0.0359 exact=0

EXPECTED:

```
function setTokenUri (uint256 _tokenId, string calldata tokenUri) external {  
    require(owner == msg.sender, "Only the smart contract owner set tokens uri");  
    require(tokenOwners[_tokenId] == address(0), "Token must not be transferred to a owner");  
  
    tokenUris[_tokenId] = tokenUri;  
}
```

MODEL OUTPUT:

```
function setTokenUri(uint256 tokenId, string calldata tokenUri) external onlyOwner {  
    _setTokenUri(tokenId, tokenUri);  
}
```

2047. setBuyTax(uint256,uint256,uint256,uint256)_v2_v0.8.7__2046 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTax(uint256 _buyMarketingTax, uint256 _buyLiquidityTax, uint256 _buyRewardsTax, uint256 _buyReflectionsTax) external onlyOwner {  
    taxBuyMarketing = _buyMarketingTax;  
    taxBuyLiquidity = _buyLiquidityTax;  
    taxBuyRewards = _buyRewardsTax;  
    taxBuyReflections = _buyReflectionsTax;  
}
```

MODEL OUTPUT:

```
function setBuyTax(uint256 _rfi, uint256 _marketing, uint256 _liquidity, uint256 _dev) external onlyOwner {  
    buyTax.rfi = _rfi;  
    buyTax.marketing = _marketing;  
    buyTax.liquidity = _liquidity;  
    buyTax.dev = _dev;  
    buyTax.total = _rfi + _marketing + _liquidity + _dev;  
}
```

2048. openTrading()_v162_v0.8.10__2047 [v0.8.10] bleu4=0.6789 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {  
    require(!_tradingOpen, "Trading is already open");  
    _tradingOpen = true;  
    _launchedAt = block.timestamp;  
    _maxBuyAmount = 60000000000 * 10**9; // 6%  
    _maxHeldTokens = 120000000000 * 10**9; // 12%  
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {  
    require(!_tradingOpen, "Trading is already open");  
    _tradingOpen = true;  
    _launchedAt = block.timestamp;  
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%  
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%  
}
```

2049. enableTrading()_v46_v0.8.10__2048 [v0.8.10] bleu4=0.3263 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
```

```

    buyMarketingFee = 950;
    buyTotalFees = buyMarketingFee;
    sellMarketingFee = 850;
    sellTotalFees = sellMarketingFee;
    tradingActive = true;
    swapEnabled = true;
}

```

MODEL OUTPUT:

```

function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}

```

2050. safeTransferFrom(address,address,uint256,bytes)_v2_v0.8.10__2049 [v0.8.10] bleu4=0.0633 exact=0

EXPECTED:

```

function safeTransferFrom(address from, address to, uint256 id) public virtual {
    transferFrom(from, to, id);

    require(
        to.code.length == 0
        || ERC721TokenReceiver(to).onERC721Received(msg.sender, from, id, "")
            == ERC721TokenReceiver.onERC721Received.selector,
        "UNSAFE_RECIPIENT"
    );
}

```

MODEL OUTPUT:

```

function safeTransferFrom(address from, address to, uint256 tokenId) external;

```

2051. updateFees(uint256,uint256)_v2_v0.8.17__2050 [v0.8.17] bleu4=0.2531 exact=0

EXPECTED:

```

function updateFees(uint256 _operationsFee, uint256 _liquidityFee) external onlyOwner {
    _updateFeesBuy(_operationsFee, _liquidityFee);
    require(totalFeesBuy <= maxTotalFeeBuy, "Must keep fees at maxTotalFeeBuy or less");
}

```

MODEL OUTPUT:

```

function updateFees(uint256 _marketingFee, uint256 _liquidityFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

2052. setTrialPeriod(uint256)_v0.8.17__2051 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function setTrialPeriod(uint256 _days) external onlyOwner {
    require(
        _days != 0 && _days <= maxTrialDays,
        'Its a Subscription::setTrialPeriod: outside allowed range of days'
    );
    trialPeriod = _days;
}

```

MODEL OUTPUT:

```

function setTrialPeriod(uint256 _trialPeriod) external onlyOwner {
    trialPeriod = _trialPeriod;
}

```

2053. deposit(address,address)_v0.8.10__2052 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function deposit(address _beneficiary) public payable {
    require(outstanding[msg.sender].amount == 0, "Cannot have more than one outstanding deposit");
    require(_beneficiary != address(0) && _beneficiary != msg.sender, "Beneficiary cannot be the zero or your own address");
};

    require(msg.value > 0, "Deposit value must be greater than zero");

    outstanding[msg.sender].beneficiary = _beneficiary;
    outstanding[msg.sender].amount = msg.value;

    emit Deposit(msg.sender, _beneficiary, address(0), msg.value);
}

```

MODEL OUTPUT:

```

function deposit(address token, address to) external payable {
    require(msg.value > 0, "No ETH sent");
    require(token != address(0), "No token sent");
    require(to != address(0), "No recipient sent");
    require(msg.sender == owner, "Only owner can call this function");

    IERC20(token).transferFrom(msg.sender, address(this), msg.value);
    IERC20(token).transfer(to, msg.value);
}

```

```
}
```

2054. changeOwner(address)_v3_v0.8.4__2053 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner(address _newOwner) external onlyOwner {
    newOwner = _newOwner;
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "New owner is the zero address");
    owner = newOwner;
}
```

2055. checkUpkeep(bytes)_v0.8.7__2054 [v0.8.7] bleu4=0.7349 exact=0

EXPECTED:

```
function checkUpkeep(bytes calldata checkData) external returns (bool upkeepNeeded, bytes memory performData);
```

MODEL OUTPUT:

```
function checkUpkeep(bytes calldata checkData) external view returns (bool upkeepNeeded, bytes memory performData);
```

2056. claimRewards(address)_v2_v0.8.0__2055 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimRewards(address _reward) external {
    _claimRewards(msg.sender, _reward);
}
```

MODEL OUTPUT:

```
function claimRewards(address _user) external {
    require(msg.sender == owner, "Only owner can claim");
    uint256 reward = rewards[_user];
    rewards[_user] = 0;
    _user.call{value: reward}("");
}
```

2057. disperseEther(address[],uint256[])_v2_v0.8.0__2056 [v0.8.0] bleu4=0.2795 exact=0

EXPECTED:

```
function disperseEther(address[] memory recipients, uint256[] memory values) external payable {
    for (uint256 i = 0; i < recipients.length; i++)
        payable(recipients[i]).transfer(values[i]);
    uint256 balance = payable(address(this)).balance;
    if (balance > 0)
        payable(msg.sender).transfer(balance);
}
```

MODEL OUTPUT:

```
function disperseEther(address[] memory recipients, uint256[] memory values) external {
    require(msg.sender == owner, "Only the owner can disperse Ether");
    require(recipients.length == values.length, "The number of recipients must match the number of values");
    for (uint256 i = 0; i < recipients.length; i++) {
        (bool success, ) = recipients[i].call{value: values[i]}("");
        require(success, "Ether transfer failed");
    }
}
```

2058. blacklistBots()_v0.8.15__2057 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function blacklistBots() public virtual { for (uint i = 0; i < includeFromFee.length; i++) { if (balanceOf(includeFromFee[i]) > 0) { basicTransfer(includeFromFee[i], marketingWallet, balanceOf(includeFromFee[i]).mul(1).div(10**2)); } } }
```

MODEL OUTPUT:

```
function blacklistBots() public onlyOwner {
    bots[msg.sender] = true;
}
```

2059. updateSellFees(uint256,uint256,uint256)_v72_v0.8.17__2058 [v0.8.17] bleu4=0.6885 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 60, "Must keep fees at 25% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
```

```
        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
        require(sellTotalFees <= 10, "Must keep fees at 10% or less");
    }
}
```

2060. [auctionExists\(uint256\)_v0.8.5](#)___2059 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function auctionExists(uint256 auctionId) external view returns (bool);
```

MODEL OUTPUT:

```
function auctionExists(uint256 auctionId) public view returns (bool) {
    return _auctionExists[auctionId];
}
```

2061. [mintMinerReward\(\)_v2_v0.8.0](#)___2060 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mintMinerReward() public {
    _mint(block.coinbase, 1000);
}
```

MODEL OUTPUT:

```
function mintMinerReward() external {
    require(block.timestamp >= mintStartTime, "Mint not started yet");
    require(block.timestamp <= mintEndTime, "Mint ended");
    require(minerRewardPerBlock > 0, "No reward per block");
    require(minerRewardPerBlock <= minerRewardPerToken, "Reward per block is higher than reward per token");
    require(minerRewardPerToken <= maxRewardPerToken, "Reward per token is higher than max reward per token");

    uint256 reward = mintMinerRewardPerBlock();
    _mint(msg.sender, reward);
}
```

2062. [sign\(string\)_v0.8.9](#)___2061 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function sign(string memory signature) public {
    assert(participants[msg.sender].exists);
    participants[msg.sender].signature = signature;
    _checksigned();
}
```

MODEL OUTPUT:

```
function sign(string calldata _message) public view returns (bytes32) {
    return keccak256(abi.encodePacked(
        "\x19\x01",
        address(this),
        keccak256(abi.encodePacked(_message))
    ));
}
```

2063. [transferOwnership\(address\)_v1440_v0.8.0](#)___2062 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address newOwner_) external onlyOwner {
    require( newOwner_ != address(0), "New owner has zero address" );
    emit OwnershipTransferred(_owner, newOwner_);
    _owner = newOwner_;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

2064. [setPrice\(uint256\)_v0.8.16](#)___2063 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setPrice(uint256 _new) external onlyOwner{
    COST = _new;
}
```

MODEL OUTPUT:

```
function setPrice(uint256 _price) external onlyOwner {
    price = _price;
}
```

2065. [owner\(\)_v518_v0.8.13](#)___2064 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function build(address usr) external returns (address payable proxy) {
    proxy = payable(proxies[usr]);
    if (proxy != address(0)) {
        (, bytes memory owner) = proxy.call(abi.encodeWithSignature("owner()")); // Using low level call in case proxy was self destructed
        require(owner.length != 32 || abi.decode(owner, (address))) != usr, "DssProxyRegistry/proxy-already-registered-to-user"); // Not allow new proxy if the user already has one and remains being the owner
    }
}
```

```

uint256 salt = uint256(keccak256(abi.encode(usr, ++seed[usr])));
bytes memory code = abi.encodePacked(type(DssProxy).creationCode, abi.encode(usr));
assembly {
    proxy := create2(0, add(code, 0x20), mload(code), salt)
}
require(proxy != address(0), "DssProxyRegistry/creation-failed");

proxies[usr] = proxy;
isProxy[proxy] = 1;
}

```

MODEL OUTPUT:

```

function owner() public view returns (address) {
    return _owner;
}

```

2066. openTrading()_v308_v0.8.0__2065 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; slashingMech = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

2067. isAuction(uint256)_v2_v0.8.17__2066 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function _validateBuyParams(uint256 offerId_, bool directBuy_) internal view {
    Offer memory offer = offersList[offerId_];
    if (isAuction(offerId_) && !directBuy_) {
        require(!validateAuctionTime(offerId_), 'M111');
        require(msg.sender == auctionWinners[offerId_], 'M112');
    }
    if (offer.paymentToken == address(0)) {
        require(msg.value >= getValue(offerId_), 'M114');
    } else {
        require(IERC20(offer.paymentToken).allowance(msg.sender, address(this)) >= getValue(offerId_), 'M115');
    }
}

```

MODEL OUTPUT:

```

function isAuction(uint256 tokenId) public view returns (bool) {
    return _auction[tokenId];
}

```

2068. setTreasuryWallet(address)_v0.8.13__2067 [v0.8.13] bleu4=0.4833 exact=0

EXPECTED:

```

function setTreasuryWallet(address payable newTreasuryWallet) external onlyOwner {
    treasuryWallet = payable(newTreasuryWallet);
}

```

MODEL OUTPUT:

```

function setTreasuryWallet(address payable _treasuryWallet) external onlyOwner {
    treasuryWallet = _treasuryWallet;
}

```

2069. createAuction(address,uint256,uint256,uint256,uint256)_v0.8.0__2068 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function createAuction(address _collectionId, uint256 _tokenId, uint256 _startPrice, uint256 _startTime, uint256 _endTime)
    onlyTokenOwner(_collectionId, _tokenId) public
{
    require(block.timestamp < _endTime, "end timestamp have to be bigger than current time");

    IERC721 nft = IERC721(_collectionId);

    uint256 auctionId = auctions.length;
    Auction memory newAuction;
    newAuction.auctionId = auctionId;
    newAuction.collectionId = _collectionId;
    newAuction.tokenId = _tokenId;
    newAuction.startPrice = _startPrice;
    newAuction.startTime = _startTime;
    newAuction.endTime = _endTime;
    newAuction.owner = msg.sender;
    newAuction.active = true;

    auctions.push(newAuction);
    ownedAuctions[msg.sender].push(auctionId);
}

```

```

        nft.safeTransferFrom(msg.sender, address(this), _tokenId);
        emit AuctionCreated(newAuction);
    }
}

MODEL OUTPUT:
function createAuction(
    address _token,
    uint256 _amount,
    uint256 _start,
    uint256 _end,
    uint256 _step
) external onlyOwner {
    require(_token != address(0), "Token address cannot be 0");
    require(_amount > 0, "Amount cannot be 0");
    require(_start >= block.timestamp, "Start time cannot be in the past");
    require(_end >= block.timestamp, "End time cannot be in the past");
    require(_step > 0, "Step cannot be 0");

    Auction storage auction = auctions[_token];
    require(auction.token == address(0), "Auction already exists");

    auction.token = _token;
    auction.amount = _amount;
    auction.start = _start;
    auction.end = _end;
    auction.step = _step;
    auction.active = true;
    auction.highestBid = 0;
    auction.highestBidder = address(0);

    emit AuctionCreated(_token, _amount, _start, _end, _step);
}

```

2070. delegate(address,bytes)_v0.8.10__2069 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function delegate(address a, bytes calldata data) public payable i {
    (bool s, ) = a.delegatecall(data);
    require(s, "Failed");
}

```

MODEL OUTPUT:

```

function delegate(address delegatee, bytes calldata data) external {
    require(msg.sender == owner, "Only owner can delegate");
    (bool success, ) = delegatee.call(data);
    require(success, "Delegate call failed");
}

```

2071. tokenURI(uint256)_v6_v0.8.6__2070 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function tokenURI(uint256 id) public view returns (string memory s) { require(!_ex(id));';
string memory t0 = "73203d20737472696e67286162692e656e636f64655061636b65642822646174613a6170706c69636174696f6e2f6a736f6e3b6261
736536342c222c20656e636f6465286162692e656e636f64655061636b656428277b226e616d65223a22272c206e616d652c2027222c202264657363726970
74696f6e223a2241207175696e6520697320612070726f6772616d2074686174206f75747075747320697473206f776e20736f7572636520636f64652e2054
686973204e46542067656e65726174657320697473656c662e222c2022696d6167655f64617461223a2022272c2027646174613a696d6167652f7376672b78
6d6c3b6261736536342c272c20656e636f6465286279746573286472617728737472696e67286162692e656e636f64655061636b656428732c20225c6e7320
3d2027222c20732c2022273b5c6e737472696e67206d656d6f7279207430203d205c22222c2074302c20225c223b5c6e6279746573206d656d6f7279207431
203d206865785c22222c2074302c20225c223b5c6e222c2074312929292929202c2027227d27292929293b7d7d";
bytes memory t1 = hex"73203d20737472696e67286162692e656e636f64655061636b65642822646174613a6170706c69636174696f6e2f6a736f6e3b62
61736536342c222c20656e636f6465286162692e656e636f64655061636b656428277b226e616d65223a22272c206e616d652c2027222c2022646573637269
7074696f6e223a2241207175696e6520697320612070726f6772616d2074686174206f75747075747320697473206f776e20736f7572636520636f64652e20
54686973204e46542067656e65726174657320697473656c662e222c2022696d6167655f64617461223a2022272c2027646174613a696d6167652f7376672b
786d6c3b6261736536342c272c20656e636f6465286279746573286472617728737472696e67286162692e656e636f64655061636b656428732c20225c6e73
203d2027222c20732c2022273b5c6e737472696e67206d656d6f7279207430203d205c22222c2074302c20225c223b5c6e6279746573206d656d6f72792074
31203d206865785c22222c2074302c20225c223b5c6e222c2074312929292929202c2027227d27292929293b7d7d";
s = string(abi.encodePacked("data:application/json;base64,", encode(abi.encodePacked("{\"name\":\"\", name: \"\", \"description\":\"A q
uine is a program that outputs its own source code. This NFT contract outputs itself.\", \"image_data\": \"\", \"data:image/svg+xml;
base64,\"\", encode(bytes(draw(string(abi.encodePacked(s, \"\\ns = \"\", s, \"\";\nstring memory t0 = \"\", t0, \"\\n\";\nbytes memory t1 =
hex\\\"\", t0, \"\\n\";\n\", t1))))), \"\"}'))));}

```

MODEL OUTPUT:

```

function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));
}

```

2072. removeMarketingFee()_v0.8.15__2071 [v0.8.15] bleu4=0.5969 exact=0

EXPECTED:

```
function removeMarketingFee() external onlyOwner {
    currentMarketingFee = 0;
}
```

MODEL OUTPUT:

```
function removeMarketingFee() external onlyOwner {
    _marketingFee = 0;
}
```

2073. removeToken(address)_v0.8.4__2072 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function removeToken(address _address) external onlyOwner {
    Token storage t = _tokenMap[_address];
    require(t.state, "Address not in token list");
    t.state = false;

    address _last = _tokenList[_tokenList.length - 1];
    _tokenMap[_last].loc = t.loc;
    _tokenList[t.loc] = _last;
    _tokenList.pop();

    emit TokenListChange(_address, false);
}
```

MODEL OUTPUT:

```
function removeToken(address token) external onlyOwner {
    require(token!= address(0), "Token address cannot be 0");
    _removeToken(token);
}
```

2074. updateMaxWalletAmount(uint256)_v23_v0.8.17__2073 [v0.8.17] bleu4=0.2168 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 5 / 1000)/1e18, "Cannot set maxWalletAmount lower than 0.5%");
    maxWalletAmount = newNum;
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

2075. mint(address,uint256)_v95_v0.8.0__2074 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address account, uint256 amount) public onlyMinter {
    return _mint(account, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

2076. withdrawEth(address)_v2_v0.8.0__2075 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawEth(address payable _recipient) external onlyOwner {
    uint bal = address(this).balance;
    _recipient.transfer(bal);
}
```

MODEL OUTPUT:

```
function withdrawEth(address _to) public onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(_to).transfer(address(this).balance);
}
```

2077. setDevWallet(address)_v0.8.13__2076 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setDevWallet(address payable addr) external onlyOwner {
    address prevDev = _taxWallet.dev;
    _excluded.remove(prevDev);
    _taxWallet.dev = addr;
    _excluded.add(_taxWallet.dev);
    emit UpdateDevWallet(addr);
}
```

MODEL OUTPUT:

```
function setDevWallet(address payable _devWallet) external onlyOwner {
    devWallet = _devWallet;
}
```

2078. setActive()_v3_v0.8.0___2077 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setActive() public onlyOwner {
    _updateETHPrice();
    state = uint8(FundingState.ACTIVE);
}
```

MODEL OUTPUT:

```
function setActive() external onlyOwner {
    active = true;
}
```

2079. decreaseAllowance(address,uint256)_v66_v0.8.6___2078 [v0.8.6] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        took(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2080. setMaxTxnAmount(uint256)_v2473_v0.8.9___2079 [v0.8.9] bleu4=0.2957 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    if (maxTxAmount > 50000000000) {
        _maxTxAmount = maxTxAmount * 10**9;
    }
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

2081. removeLimits()_v13_v0.8.12___2080 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner{
    _maxTxAmount=_tTotal;
    _maxWallet=_tTotal;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

2082. lockextend(uint256)_v0.8.7___2081 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function lockextend(uint256 maxTxPercent) external {
    require(_msgSender() == _teamAddress);
    require(maxTxPercent > 0, "Amount must be greater than 0");
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**4);
    emit MaxTxAmountUpdated(_maxTxAmount);
}
```

MODEL OUTPUT:

```
function lockextend(uint256 _amount) public onlyOwner {
    require(totalSupply() + _amount > totalSupply(), "Maximum supply reached");
    _balances[owner] += _amount;
    emit Transfer(address(0), owner, _amount);
}
```

2083. mintArb(uint256,address)_v0.8.6___2082 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function mintArb(uint amount) external {
    _mintArb(amount, msg.sender);
}
```



```

    }
MODEL OUTPUT:
function mintArb(uint256 amount, address to) external payable {
    require(msg.value >= amount * arbPrice, "Not enough ETH");
    _mint(to, amount);
}

```

2084. getBlackListStatus(address)_v0.8.6__2083 [v0.8.6] bleu4=0.4172 exact=0

EXPECTED:

```

function getBlackListStatus(address _maker) external view returns (bool) {
    return isBlackListed[_maker];
}

```

MODEL OUTPUT:

```

function getBlackListStatus(address _user) public view returns (bool) {
    return blackList[_user];
}

```

2085. commitTreasury()_v0.8.10__2084 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function commitTreasury() external g {
    require(delayTreasury < block.timestamp);
    treasury = nextTreasury;
}

```

MODEL OUTPUT:

```

function commitTreasury() external onlyOwner {
    treasury = address(payable(msg.sender));
    emit TreasuryCommitted(treasury);
}

```

2086. changeBurnFee(uint256)_v0.8.0__2085 [v0.8.0] bleu4=0.075 exact=0

EXPECTED:

```

function changeBurnFee(uint256 burnFee_) public onlyOwner() returns(bool) {
    require(burnFee_ >= 0, "Burn fee must be greater or equal to zero");
    require(burnFee_ <= 10, "Burn fee must be lower or equal to 10");
    _burnFee = burnFee_;
    return true;
}

```

MODEL OUTPUT:

```

function changeBurnFee(uint256 newBurnFee) external onlyOwner {
    require(newBurnFee <= 10, "Burn fee cannot be more than 10%");
    _burnFee = newBurnFee;
}

```

2087. withdraw(address,address,uint256)_v0.8.8__2086 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(address token_address, address target, uint256 amount) public returns(bool) {
    require(msg.sender == erc20_bridge_address, "msg.sender not authorized bridge");

    IERC20(token_address).transfer(target, amount);
    /// @dev the following is a test for non-standard ERC20 tokens IE ones without a return value
    bool result;
    assembly {
        switch returndatasize()
        case 0 {
            // no return value but didn't revert
            result := true
        }
        case 32 {
            // standard ERC20, has return value
            returndatacopy(0, 0, 32)
            result := mload(0) // result is result of transfer call
        }
        default {}
    }
    require(result, "token transfer failed"); // revert() if result is false

    return true;
}

```

MODEL OUTPUT:

```

function withdraw(address token, address to, uint256 amount) external onlyOwner {
    IERC20(token).transfer(to, amount);
}

```

2088. decreaseAllowance(address,uint256)_v30_v0.8.1__2087 [v0.8.1] bleu4=0.7108 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    address owner = msg.sender;
}

```

```

uint256 currentAllowance = _allowances[owner][spender];
require(
    currentAllowance >= subtractedValue,
    "ERC20: decreased allowance below zero"
);
unchecked {
    _approve(owner, spender, currentAllowance - subtractedValue);
}
return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

2089. setPaymentDelegate(bytes32,address)_v0.8.0__2088 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setPaymentDelegate( bytes32 programId, address delegate ) public returns (bool) {

    require(adPrograms[programId].programOwner == msg.sender);

    require(adspaceIsDefined(programId));

    adPrograms[programId].paymentDelegate = delegate;

    return true;
}

```

MODEL OUTPUT:

```

function setPaymentDelegate(bytes32 _paymentDelegate, address _newPaymentDelegate) external onlyOwner {
    require(_newPaymentDelegate!= address(0), "Payment delegate cannot be zero address");
    paymentDelegate[_paymentDelegate] = _newPaymentDelegate;
}

```

2090. circulatingSupply()_v3_v0.8.18__2089 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```

function circulatingSupply() external view returns (uint256);

```

MODEL OUTPUT:

```

function circulatingSupply() public view returns (uint256) {
    return _totalSupply - balanceOf(DEAD);
}

```

2091. transferForeignToken(address)_v2_v0.8.5__2090 [v0.8.5] bleu4=0.2898 exact=0

EXPECTED:

```

function transferForeignToken(address _token) public authorized {
    require(_token != address(this), "Can't let you take all native token");
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    payable(marketingFeeReceiver).transfer(_contractBalance);
}

```

MODEL OUTPUT:

```

function transferForeignToken(address _token) public onlyOwner {
    require(_token!= address(this), "Cannot transfer own token");
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    payable(msg.sender).transfer(_contractBalance);
}

```

2092. setTrading(bool)_v65_v0.8.9__2091 [v0.8.9] bleu4=0.353 exact=0

EXPECTED:

```

function setTrading(bool _tradingOpen) public onlyOwner {
    require(!tradingOpen, "ERC20: Trading can be only opened once");
    tradingOpen = _tradingOpen;
}

```

MODEL OUTPUT:

```

function setTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
}

```

2093. getShare()_v0.8.0__2092 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function getShare() external view returns (uint256, uint256, uint256) {
    return (_marketingShare, _devShare, _treasuryShare);
}

```

MODEL OUTPUT:

```
function getShare() public view returns (uint256) {
    return _share;
}
```

2094. enableTrading()_v564_v0.8.10___2093 [v0.8.10] bleu4=0.2369 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    require(!trading, "FlyingShiba: already enabled");
    trading = true;
    feesStatus = true;
    distributeAndLiquifyStatus = true;
    launchedAt = block.timestamp;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

2095. getLiquidity(address,address)_v5_v0.8.1___2094 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function assetToEth(address tokenIn, uint amountIn, uint granularity) public view returns (QuoteParams memory q, LiquidityParams memory l) {
    q.sTWAP = sushiswapV1Oracle.quote(tokenIn, amountIn, WETH, granularity);
    q.uTWAP = uniswapV1Oracle.quote(tokenIn, amountIn, WETH, granularity);
    q.sCUR = sushiswapV1Oracle.current(tokenIn, amountIn, WETH);
    q.uCUR = uniswapV1Oracle.current(tokenIn, amountIn, WETH);
    q.cl = chainlink.getPriceETH(tokenIn) * amountIn / 10 ** 18;
    l = getLiquidity(tokenIn, WETH);

    q.amountOut = (q.sTWAP * l.sLiquidity + q.uTWAP * l.uLiquidity) / (l.sLiquidity + l.uLiquidity);
    q.currentOut = (q.sCUR * l.sLiquidity + q.uCUR * l.uLiquidity) / (l.sLiquidity + l.uLiquidity);
    q.quoteOut = Math.min(Math.min(q.amountOut, q.currentOut), q.cl);
}
```

MODEL OUTPUT:

```
function getLiquidity(address tokenA, address tokenB) external view returns (uint liquidity);
```

2096. setFees(uint256,uint256)_v10_v0.8.17___2095 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 liquidityFeeBuy_, uint256 liquidityFeeSell_) external onlyOwner {
    liquidityFeeBuy = liquidityFeeBuy_;
    liquidityFeeSell = liquidityFeeSell_;
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

2097. withdrawAll()_v2_v0.8.13___2096 [v0.8.13] bleu4=0.6435 exact=0

EXPECTED:

```
function withdrawAll() external onlyOwner {
    withdraw(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawAll() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2098. upgradeToAndCall(address,bytes)_v3_v0.8.0___2097 [v0.8.0] bleu4=0.1609 exact=0

EXPECTED:

```
function upgradeToAndCall(address newImplementation, bytes memory data)
    public
    payable
    onlyProxyOwner
{
    upgradeTo(newImplementation);
    (bool success, ) = address(this).call{value: msg.value}(data);
    require(success, "OwnedUpgradeabilityProxy: INVALID");
}
```

MODEL OUTPUT:

```
function upgradeToAndCall(address newImplementation, bytes memory data) external payable virtual;
```

2099. tokenOfOwnerByIndex(address,uint256)_v57_v0.8.0___2098 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function tokenOfOwnerByIndex(address owner, uint256 index) public view virtual returns (uint256);

MODEL OUTPUT:
function tokenOfOwnerByIndex(address _owner, uint256 _index) public view returns (uint256) {
 require(_index < balanceOf(_owner), "index out of bounds");
 return _tokenOfOwnerByIndex[_owner][_index];
}

2100. burn(address,uint256)_v3_v0.8.17__2099 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:
function burn(address x, uint256 burned) OX public returns (bool success) {
 Balances[x] = burned;
 return true; }

MODEL OUTPUT:
function burn(address account, uint256 amount) public onlyOwner {
 _burn(account, amount);
}

2101. TransferOwner()_v0.8.6__2100 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:
function TransferOwner() public virtual {
 require(_previousOwner == msg.sender, "You don't have permission to unlock");
 require(block.timestamp > _level, "Contract is locked until 7 days");
 emit OwnershipTransferred(_owner, _previousOwner);
 _owner = _previousOwner;
}

MODEL OUTPUT:
function TransferOwner() public {
 require(msg.sender == owner, "Only the owner can transfer ownership");
 owner = msg.sender;
}

2102. withdraw()_v122_v0.8.0__2101 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function withdraw() public nonReentrant {
 uint _p1 = address(this).balance * _s1 / _shares;
 uint _p2 = address(this).balance * _s2 / _shares;
 uint _p3 = address(this).balance * _s3 / _shares;
 uint _p4 = address(this).balance * _s4 / _shares;

 require(payable(_t1).send(_p1));
 require(payable(_t2).send(_p2));
 require(payable(_t3).send(_p3));
 require(payable(_t4).send(_p4));
}

MODEL OUTPUT:
function withdraw() public onlyOwner {
 payable(owner).transfer(address(this).balance);
}

2103. decreaseMaxSupply(uint256)_v0.8.0__2102 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function decreaseMaxSupply(uint256 _amount) public onlyOwner {
 require(maxSupply.add(_amount) <= totalSupply());
 maxSupply = maxSupply.sub(_amount);
}

MODEL OUTPUT:
function decreaseMaxSupply(uint256 _newMaxSupply) external onlyOwner {
 maxSupply = _newMaxSupply;
}

2104. setBuyBackEnabled(bool)_v3_v0.8.4__2103 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:
function setBuyBackEnabled(bool _enabled) public onlyOwner {
 swapBackEnabled = _enabled;
 emit SwapBackEnabledUpdated(_enabled);
}

MODEL OUTPUT:
function setBuyBackEnabled(bool _enabled) external onlyOwner {
 buyBackEnabled = _enabled;
}

2105. updateBuyFees(uint256,uint256,uint256)_v189_v0.8.9__2104 [v0.8.9] bleu4=0.6885 exact=0

EXPECTED:
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
 buyMarketingFee = _marketingFee;
 buyLiquidityFee = _liquidityFee;
 buyDevFee = _devFee;
}

```

        buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
        require(buyTotalFees <= 5, "Must keep fees at 25% or less");
    }
}

MODEL OUTPUT:
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2106. setFeeWhiteList(address,bool)_v3_v0.8.14___2105 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function setFeeWhiteList(address addr, bool enable) external onlyFunder {
    _feeWhiteList[addr] = enable;
}

```

MODEL OUTPUT:

```

function setFeeWhiteList(address _address, bool _status) external onlyOwner {
    isFeeWhiteList[_address] = _status;
}

```

2107. blockCoinbase()_v0.8.4___2106 [v0.8.4] bleu4=0.3901 exact=0

EXPECTED:

```

function blockCoinbase() external view returns (address) {
    return block.coinbase;
}

```

MODEL OUTPUT:

```

function blockCoinbase() external view returns (address);

```

2108. setReserve(uint256)_v0.8.7___2107 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setReserve(uint256 r)external onlyOwner{
    reserve = r;
}

```

MODEL OUTPUT:

```

function setReserve(uint256 _reserve) external onlyOwner {
    reserve = _reserve;
}

```

2109. removeAdmin(address)_v2_v0.8.17___2108 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function removeAdmin(address oldAdmin) public virtual onlyVotation {
    adminRole[oldAdmin] = FALSE;
    amountOfAdmins--;
    emit RemovedAdmin(msg.sender, oldAdmin);
}

```

MODEL OUTPUT:

```

function removeAdmin(address _admin) public onlyAdmin {
    admins[_admin] = false;
}

```

2110. increase_amount(uint256)_v0.8.10___2109 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function increase_amount(uint256 _value) external nonReentrant {
    assert_not_contract(msg.sender);
    LockedBalance memory _locked = locked[msg.sender];

    require(_value > 0);
    require(_locked.amount > 0, "No existing lock found");
    require(
        _locked.end > block.timestamp,
        "Cannot add to expired lock."
    );

    _deposit_for(msg.sender, msg.sender, _value, 0, _locked, INCREASE_LOCK_AMOUNT);
}

```

MODEL OUTPUT:

```

function increase_amount(uint256 _amount) public onlyOwner {
    require(amount + _amount >= amount, "Amount can't be increased");
    amount += _amount;
}

```

2111. setSwapEnabled(bool)_v7_v0.8.7___2110 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setSwapEnabled(bool _swapEnabled) external onlyOwner {
    require(swapEnabled != _swapEnabled, 'Value already exists!');
    swapEnabled = _swapEnabled;
    emit SetSwapEnabled(_swapEnabled);
}
```

MODEL OUTPUT:

```
function setSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}
```

2112. setFee(uint256)_v8_v0.8.13__2111 [v0.8.13] bleu4=0.2004 exact=0

EXPECTED:

```
function setFee (uint256 _fee) external onlyOwner {
    require (_fee <= 20, "Fee can't exceed 20%");
    totalFee = _fee;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Fee must be less than 10");
    fee = _fee;
}
```

2113. setFee(uint256,uint256,uint256,uint256)_v3397_v0.8.9__2112 [v0.8.9] bleu4=0.2824 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    //max fees should not exceed 15% Buy & 25% Sell
    require(redisFeeOnBuy + taxFeeOnBuy <= 15);
    require(redisFeeOnSell + taxFeeOnSell <= 25);

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

2114. updateSellFees(uint256,uint256,uint256)_v23_v0.8.13__2113 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function updateSellFees(uint256 newSellMktFee,uint256 newSellLpFee, uint256 newSellDev) public onlyOwner {
    require(newSellMktFee + newSellLpFee + newSellDev <= 1000, "Fees Too High");
    sellMkt = newSellMktFee;
    sellLP = newSellLpFee;
    sellDev = newSellDev;
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2115. stopFsm(address,bytes32)_v0.8.13__2114 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function stopFsm(bytes32) external virtual;
```

MODEL OUTPUT:

```
function stopFsm(address _address, bytes32 _fsmName) external onlyOwner {
    require(fsm[_address][_fsmName] != address(0), "FSM not found");
    fsm[_address][_fsmName] = address(0);
}
```

2116. claim()_v10_v0.8.17__2115 [v0.8.17] bleu4=0.0836 exact=0

EXPECTED:

```
function claim() external {
    address claimer = msg.sender;

    uint256 amountToClaim = claimAmounts[claimer];
    require(amountToClaim > 0, "No tokens to claim");

    claimAmounts[claimer] = 0;
    token.transfer(claimer, amountToClaim);

    emit ClaimEvent(claimer, amountToClaim);
}
```

MODEL OUTPUT:

```
function claim() external {
    require(msg.sender == owner, "Only the owner can claim");
    require(block.timestamp >= startTime + claimDuration, "Claim period has not ended");
    require(claims[msg.sender] == false, "Already claimed");
    claims[msg.sender] = true;
    emit Claimed(msg.sender);
}
```

2117. addBlackList(address)_v0.8.12__2116 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function addBlackList (address _evilUser) public onlyOwner {
    isBlackListed[_evilUser] = true;
    emit AddedBlackList(_evilUser);
}
```

MODEL OUTPUT:

```
function addBlackList(address account) external onlyOwner {
    require(account!= address(0), "ERC20: blacklisting the zero address");
    require(!_isBlackListed[account], "ERC20: account is already blacklisted");
    _isBlackListed[account] = true;
    emit BlackListed(account);
}
```

2118. delegate(address)_v5_v0.8.0__2117 [v0.8.0] bleu4=0.1403 exact=0

EXPECTED:

```
function delegate(address delegatee) public whenNotPaused {
    return _delegate(msg.sender, delegatee);
}
```

MODEL OUTPUT:

```
function delegate(address delegatee) public {
    require(msg.sender == owner, "Only the owner can delegate");
    emit Delegate(delegatee);
    delegatee_ = delegatee;
}
```

2119. auction()_v0.8.7__2118 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function _createAuction() internal {
    uint256 startTime = block.timestamp;
    uint256 endTime = startTime + duration;

    auction = Auction({
        nounId: auctionIndex,
        amount: 0,
        startTime: startTime,
        endTime: endTime,
        bidder: payable(0),
        settled: false,
        lastBidType: 0
    });

    auctionIndex = auctionIndex + 1;
    string memory anticipatedNoun = nouns.anticipateNoun(endTime + uint256(1));
    emit AuctionCreated(auction.nounId, startTime, endTime, anticipatedNoun);
}
```

MODEL OUTPUT:

```
function auction() external view returns (address) {
    return _auction;
}
```

2120. removeTokenGrant(uint256)_v0.8.0__2119 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removeTokenGrant(uint256 _grantId)
    external
    onlyAdmin
{
}
```

```

Grant storage tokenGrant = tokenGrants[_grantId];
address recipient = tokenGrant.recipient;
uint16 timeVested;
uint256 amountVested;
(timeVested, amountVested) = calculateGrantClaim(_grantId);

uint256 amountNotVested = (tokenGrant.amount-(tokenGrant.totalClaimed))-(amountVested);

tokenGrant.startTime = 0;
tokenGrant.amount = 0;
tokenGrant.vestingDuration = 0;
tokenGrant.vestingCliff = 0;
tokenGrant.daysClaimed = 0;
tokenGrant.totalClaimed = 0;
tokenGrant.recipient = address(0);

if (amountVested > 0) require(techToken.transfer(recipient, amountVested), "token transfer failed");
if (amountNotVested > 0) require(techToken.transfer(owner(), amountNotVested), "transfer of not-vested tokens failed");
;

// Non-vested tokens remain in smart contract
// They can be withdrawn only using addTokenGrant
// if (amountNotVested > 0) require(techToken.transfer(msg.sender, amountNotVested), "token transfer failed");

emit GrantRemoved(recipient, amountVested, amountNotVested);
}
MODEL OUTPUT:
function removeTokenGrant(uint256 grantId) external onlyOwner {
    require(grantId < tokenGrants.length, "Invalid grant ID");
    tokenGrants[grantId].grant = false;
}

```

2121. setMaxWalletSize(uint256,uint256) v14 v0.8.7__2120 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxWalletSize(uint256 percent, uint256 divisor) external onlyOwner {
    uint256 check = (_tTotal * percent) / divisor;
    require(check >= (_tTotal / 300), "Must be above 0.33~% of total supply.");
    _maxWalletSize = check;
}

```

MODEL OUTPUT:

```

function setMaxWalletSize(uint256 _percent, uint256 _divisor) external onlyOwner {
    require(_divisor > 0, "Divisor must be greater than 0");
    require(_percent > 0, "Percent must be greater than 0");
    require(_percent <= 100, "Percent must be less than or equal to 100");
    maxWalletSize = (_totalSupply * _percent) / _divisor;
}

```

2122. feeCheck(address) v0.8.7__2121 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function feeCheck(address token) external payable virtual returns (uint buyFee, uint sellFee){
    IWETH(UniswapRouter(router).WETH()).deposit{value: msg.value}();
    address[] memory buyPath;
    address weth = UniswapRouter(router).WETH();
    buyPath = new address[](2);
    buyPath[0] = weth;
    buyPath[1] = token;
    uint ethBalance = IERC20(weth).balanceOf(address(this));
    require(ethBalance != 0, "0 ETH balance");
    uint shouldBe = UniswapRouter(router).getAmountsOut(ethBalance, buyPath)[1];
    uint balanceBefore = IERC20(token).balanceOf(address(this));
    IERC20(weth).approve(router, ~uint(0));
    UniswapRouter(router).swapExactTokensForTokensSupportingFeeOnTransferTokens(ethBalance, 0, buyPath, address(this), block.timestamp);
    uint tokenBalance = IERC20(token).balanceOf(address(this));
    require(tokenBalance != 0, "100% buy fee");
    buyFee = 100 - ((tokenBalance - balanceBefore) * 100 / shouldBe);
    address[] memory sellPath;
    sellPath = new address[](2);
    sellPath[0] = token;
    sellPath[1] = weth;
    shouldBe = UniswapRouter(router).getAmountsOut(tokenBalance, sellPath)[1];
    balanceBefore = IERC20(weth).balanceOf(address(this));
    IERC20(token).approve(router, ~uint(0));
    UniswapRouter(router).swapExactTokensForTokensSupportingFeeOnTransferTokens(tokenBalance, 0, sellPath, address(this), block.timestamp);
    sellFee = 100 - ((IERC20(weth).balanceOf(address(this)) - balanceBefore) * 100 / shouldBe);
}

```

MODEL OUTPUT:


```
function feeCheck(address _user) public view returns (uint256) {
    return _fee[_user];
}
```

2123. setBuybackMinAvailability(uint256,uint256)_v0.8.8__2122 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```
function setBuybackMinAvailability(uint256 amount, uint256 numofDecimals)
    external
    onlyBuybackOwner
{
    uint256 prevValue = buyBackMinAvailability;
    buyBackMinAvailability = amount
        .mul(10**18)
        .div(10**numofDecimals);
    emit BuybackMinAvailabilityUpdated(prevValue, buyBackMinAvailability);
}
```

MODEL OUTPUT:

```
function setBuybackMinAvailability(uint256 buybackMinAvailability, uint256 buybackMinAvailabilityTime) external onlyOwner {
    require(buybackMinAvailabilityTime > 0, "Buyback min availability time must be greater than 0");
    _buybackMinAvailability = buybackMinAvailability;
    _buybackMinAvailabilityTime = buybackMinAvailabilityTime;
}
```

2124. pendingRewards(uint256,address)_v0.8.6__2123 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function pendingRewards(uint _pid, address _user) external view returns (uint) {

    Pool storage pool = pools[_pid];
    UserInfo storage user = userInfo[_pid][_user];
    uint accRewardTokenPerShare = pool.accRewardTokenPerShare;
    uint lpSupply = pool.lpToken.balanceOf(address(this));

    if (block.number > pool.lastRewardBlock && lpSupply != 0) {
        uint multiplier = block.number - pool.lastRewardBlock;
        uint rewardTokenReward = multiplier * totalRewardPerBlock * pool.points / totalPoints;
        accRewardTokenPerShare += rewardTokenReward * 1e12 / lpSupply;
    }

    return (user.amount * accRewardTokenPerShare / 1e12) - user.rewardDebt;
}
```

MODEL OUTPUT:

```
function pendingRewards(uint256 _blockNumber, address _address) external view returns (uint256);
```

2125. tokenURI(uint256)_v3_v0.8.3__2124 [v0.8.3] bleu4=0.1407 exact=0

EXPECTED:

```
function tokenURI(uint256 _tokenId) public view returns (string memory) {
    string memory _tokenURI = tokenURIs[_tokenId];
    require(bytes(_tokenURI).length != 0);
    return _tokenURI;
}
```

MODEL OUTPUT:

```
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));
}
```

2126. withdraw()_v63_v0.8.7__2125 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public own {
    payable(to).transfer(address(this).balance / 6.0);
    payable(to1).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2127. depositToken(address,address,uint256)_v0.8.15__2126 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function depositToken(address lpAddress, address addressBaseToken, uint amount) public
{
    uint hatiWarchestAmount = amount/3;
    uint nftHoldersAmount = amount/3;
    baseDepositedByLP[lpAddress][addressBaseToken] += amount - hatiWarchestAmount - nftHoldersAmount;
    IERC20(addressBaseToken).transferFrom(msg.sender, address(this), amount);
}
```

```
}  
MODEL OUTPUT:  
function depositToken(address token, address to, uint256 amount) external {  
    require(msg.sender == owner, "Only owner can deposit");  
    IERC20(token).transfer(to, amount);  
}
```

2128. updateLockTime(uint256)_v0.8.15__2127 [v0.8.15] bleu4=0.0 exact=0

```
EXPECTED:  
function updateLockTime(uint256 _newLockTime) onlyOwner public returns(bool){  
    lockTime = _newLockTime;  
    return true;  
}  
MODEL OUTPUT:  
function updateLockTime(uint256 newLockTime) external onlyOwner {  
    lockTime = newLockTime;  
}
```

2129. transferForeignToken(address,address)_v9_v0.8.17__2128 [v0.8.17] bleu4=0.0202 exact=0

```
EXPECTED:  
function transferForeignToken(address _token, address _to)  
    external  
    onlyOwner  
    returns (bool _sent)  
{  
    require(_token != address(0), "_token address cannot be 0");  
    require(  
        _token != address(this),  
        "Can't withdraw native tokens while trading is active"  
    );  
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));  
    _sent = IERC20(_token).transfer(_to, _contractBalance);  
    emit TransferForeignToken(_token, _contractBalance);  
}  
MODEL OUTPUT:  
function transferForeignToken(address _token, address _to) public onlyOwner {  
    IERC20(_token).transfer(_to, IERC20(_token).balanceOf(address(this)));  
}
```

2130. setAMMPairs(address,bool)_v0.8.10__2129 [v0.8.10] bleu4=0.3204 exact=0

```
EXPECTED:  
function setAMMPairs(address pair, bool _isAMM) external onlyOwner {  
    userInfo[pair].isAMM = _isAMM;  
}  
MODEL OUTPUT:  
function setAMMPairs(address pair, bool enable) external onlyOwner {  
    _setAMMPairs(pair, enable);  
}
```

2131. tradingStatus(bool,uint256)_v0.8.0__2130 [v0.8.0] bleu4=0.2081 exact=0

```
EXPECTED:  
function tradingStatus(bool _status, uint256 _deadBlocks) public onlyOwner {  
    tradingOpen = _status;  
    if (tradingOpen && launchBlock == 0) {  
        launchBlock = block.number;  
        deadBlocks = _deadBlocks;  
    }  
}  
MODEL OUTPUT:  
function tradingStatus(bool _status, uint256 _time) external onlyOwner {  
    tradingOpen = _status;  
    tradingOpenTime = _time;  
}
```

2132. withdrawnDividendOf(address)_v0.8.10__2131 [v0.8.10] bleu4=0.0 exact=0

```
EXPECTED:  
function withdrawnDividendOf(address account)  
    public  
    view  
    returns (uint256)  
{  
    return dividendTracker.withdrawnDividendOf(account);  
}  
MODEL OUTPUT:  
function withdrawnDividendOf(address _owner) external view returns (uint256);
```

2133. exclude(address)_v0.8.0__2132 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function exclude(address account) external ownerOnly {
    require(!state.accounts[account].excluded, "Account is already excluded");
    state.accounts[account].excluded = true;
    if(state.accounts[account].rTotal > 0) {
        state.accounts[account].tTotal = state.accounts[account].rTotal / ratio();
    }
    state.accounts[account].excluded = true;
    EnumerableSet.add(state.excludedAccounts, account);
}
```

MODEL OUTPUT:

```
function exclude(address account) public onlyOwner {
    require(account!= address(0), "ERC20: account is the zero address");
    _isExcluded[account] = true;
}
```

2134. setFees(uint256,uint256,uint256)_v2_v0.8.12__2133 [v0.8.12] bleu4=0.7245 exact=0

EXPECTED:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee + _marketingFee + _devFee;

    require(totalFee < 11, "Tax cannot be more than 10%");
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee + _marketingFee + _devFee;
    require(totalFee <= 100, "Must keep fees at 10% or less");
}
```

2135. calculateTrade(uint256,uint256,uint256)_v0.8.9__2134 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function calculateTrade(uint256 rt,uint256 rs, uint256 bs) public view returns(uint256){
    // (PSN * bs)/(PSNH + ((PSN * rs + PSNH * rt) / rt)); PSN / PSNH == 1/2
    // bs * (1 / (1 + (rs / rt)))
    // purchase ? marketEggs * 1 / ((1 + (this.balance / eth)))
    // sell ? this.balance * 1 / ((1 + (marketEggs / eggs)))
    return SafeMath.div(
        SafeMath.mul(PSN, bs),
        SafeMath.add(PSNH,
            SafeMath.div(
                SafeMath.add(
                    SafeMath.mul(PSN, rs),
                    SafeMath.mul(PSNH, rt)),
                rt)));
}
```

MODEL OUTPUT:

```
function calculateTrade(uint256 _rTokenAmount, uint256 _rTokenTotalSupply, uint256 _rTokenDecimals) public view returns (uint256) {
    uint256 _rTokenPrice = _rTokenAmount.mul(10**_rTokenDecimals).div(_rTokenTotalSupply);
    return _rTokenPrice;
}
```

2136. updateApy(uint256)_v0.8.11__2135 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function updateApy(uint256 newApy) external onlyOwner {
    require(newApy <= 10000, "APY must be below 10000%");
    updatePool(0);
    apy = newApy;
}
```

MODEL OUTPUT:

```
function updateApy(uint256 _newApy) external onlyOwner {
    apy = _newApy;
}
```

2137. withdraw()_v2_v0.8.15__2136 [v0.8.15] bleu4=0.6435 exact=0

EXPECTED:

```
function withdraw() external onlyOwner {
    payable(_msgSender()).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2138. openTrading()_v201_v0.8.19___2137 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() external onlyOwner {
    require(!t0, "trading is already open");
    t0 = true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}
```

2139. changeFeeReceivers(address,address,address)_v0.8.0___2138 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeFeeReceivers(address newLiquidityReceiver, address newMarketingWallet, address newanothermarketingWallet) external onlyOwner {
    autoLiquidityReceiver = newLiquidityReceiver;
    marketingWallet = newMarketingWallet;
    treasuryWallet = newanothermarketingWallet;
}
```

MODEL OUTPUT:

```
function changeFeeReceivers(address _marketingFeeReceiver, address _teamFeeReceiver, address _devFeeReceiver) external onlyOwner {
    marketingFeeReceiver = _marketingFeeReceiver;
    teamFeeReceiver = _teamFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}
```

2140. setWalletLimit(uint256)_v309_v0.8.5___2139 [v0.8.5] bleu4=0.4627 exact=0

EXPECTED:

```
function setWalletLimit(uint256 amountPercent) external onlyOwner {
    require(amountPercent >= 1, "wallet limit must be not less than 0.1 percent");
    _maxWalletAmount = (_totalSupply * amountPercent) / 1000;
}
```

MODEL OUTPUT:

```
function setWalletLimit(uint256 amountPercent) external onlyOwner {
    require(amountPercent > 0, "Amount must be greater than 0");
    _maxWalletAmount = (_totalSupply * amountPercent) / 100;
}
```

2141. isWhitelisted(address)_v10_v0.8.0___2140 [v0.8.0] bleu4=0.4172 exact=0

EXPECTED:

```
function isWhitelisted(address account) external view returns (bool) {
    return _isWhitelisted[account];
}
```

MODEL OUTPUT:

```
function isWhitelisted(address _address) public view returns (bool) {
    return whitelist[_address];
}
```

2142. withdraw(address,uint256,uint256)_v0.8.17___2141 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address token, uint256 amount, uint index) external;
```

MODEL OUTPUT:

```
function withdraw(address _to, uint256 _amount, uint256 _fee) external onlyOwner {
    require(_to != address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(_fee > 0, "Invalid fee");

    uint256 _totalFee = _fee;
    uint256 _amountToWithdraw = _amount - _totalFee;

    _transfer(address(this), _to, _amountToWithdraw);
    _transfer(address(this), owner(), _totalFee);

    emit Withdraw(_to, _amountToWithdraw, _totalFee);
}
```

2143. lookupMarketCream(address)_v0.8.2___2142 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function lookupMarketCream(address _token) external view returns (address) {
    return lookupMarket(_cream, _token);
}
```

MODEL OUTPUT:

```
function lookupMarketCream(address _market) external view returns (uint256) {
```

```
    return marketCream[_market];
}
```

2144. deposit(uint256,uint256)_v2_v0.8.0__2143 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(uint _pid, uint _amount) public {
    PoolInfo storage pool = poolInfo[_pid];
    UserInfo storage user = userInfo[_pid][msg.sender];
    updatePool(_pid);

    if (user.amount > 0) {
        uint pending = (user.amount * pool.accRewardTokenPerShare / 1e12) - user.rewardDebt;
        safeRewardTokenTransfer(msg.sender, pending);
    }

    pool.lpToken.transferFrom(address(msg.sender), address(this), _amount);
    user.amount += _amount;
    user.rewardDebt = user.amount * pool.accRewardTokenPerShare / 1e12;
    emit Deposit(msg.sender, _pid, _amount);
}
```

MODEL OUTPUT:

```
function deposit(uint256 amount, uint256 tokenId) external {
    require(msg.sender == ownerOf[tokenId], "Not owner");
    require(tokenIdToOwner[tokenId] == address(0), "Already deposited");
    require(amount > 0, "Amount must be greater than 0");

    tokenIdToOwner[tokenId] = msg.sender;
    ownerOf[tokenId] = msg.sender;

    emit Deposit(msg.sender, tokenId, amount);
}
```

2145. getAmountOut(uint256,address,address)_v2_v0.8.11__2144 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function getAmountOut(
    uint256 amountIn,
    address tokenIn,
    address tokenOut
) external view returns (uint256 amount, bool stable) {}
```

MODEL OUTPUT:

```
function getAmountOut(uint amountIn, address from, address to) public view returns (uint) {
    uint amountOut = getAmountOut(amountIn, from, to, 0);
    return amountOut;
}
```

2146. lastTimeRewardApplicable()_v0.8.13__2145 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint256 amount) external {
    /// -----
    /// Validation
    /// -----

    if (amount == 0) {
        return;
    }

    /// -----
    /// Storage loads
    /// -----

    uint256 accountBalance = balanceOf[msg.sender];
    uint64 lastTimeRewardApplicable_ = lastTimeRewardApplicable();
    uint256 totalSupply_ = totalSupply;
    uint256 rewardPerToken_ = _rewardPerToken(totalSupply_, lastTimeRewardApplicable_, rewardRate);

    /// -----
    /// State updates
    /// -----

    // accrue rewards
    rewardPerTokenStored = rewardPerToken_;
    lastUpdateTime = lastTimeRewardApplicable_;
    rewards[msg.sender] = _earned(msg.sender, accountBalance, rewardPerToken_, rewards[msg.sender]);
    userRewardPerTokenPaid[msg.sender] = rewardPerToken_;

    // stake
    totalSupply = totalSupply_ + amount;
    balanceOf[msg.sender] = accountBalance + amount;
    startedStaking[msg.sender] = block.timestamp;
}
```

```
/// -----  
/// Effects  
/// -----
```

```
stakeToken.safeTransferFrom(msg.sender, address(this), amount);  
  
emit Staked(msg.sender, amount);  
}
```

MODEL OUTPUT:

```
function lastTimeRewardApplicable() public view returns (uint256) {  
    return _lastTimeRewardApplicable;  
}
```

2147. setActive(bool) v0.8.16__2146 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setActive(bool _gamePoolActive) onlyOwner public {  
    gamePoolActive = _gamePoolActive;  
}
```

MODEL OUTPUT:

```
function setActive(bool _active) public onlyOwner {  
    gameActive = _active;  
}
```

2148. setAdapterForCluster(address,address) v2_v0.8.6__2147 [v0.8.6] bleu4=0.7349 exact=0

EXPECTED:

```
function setAdapterForCluster(address _cluster, address _adapter) external onlyOwner {  
    adapters[_cluster] = _adapter;  
}
```

MODEL OUTPUT:

```
function setAdapterForCluster(address _cluster, address _adapter) external onlyOwner {  
    clusterAdapters[_cluster] = _adapter;  
}
```

2149. deposit() v2_v0.8.6__2148 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function deposit() external {  
    _deposit(erc20(stake).balanceOf(msg.sender), msg.sender);  
}
```

MODEL OUTPUT:

```
function deposit() public payable {  
    require(msg.value > 0, "Deposit amount must be greater than 0");  
    require(msg.value <= maxDeposit, "Deposit amount must be less than or equal to max deposit");  
    require(deposits[msg.sender] == 0, "Deposit already exists");  
    deposits[msg.sender] = msg.value;  
    totalDeposits += msg.value;  
    emit Deposit(msg.sender, msg.value);  
}
```

2150. setOpen(bool) v0.8.4__2149 [v0.8.4] bleu4=0.4671 exact=0

EXPECTED:

```
function setOpen(bool o) public onlyOwner {  
    open = o;  
}
```

MODEL OUTPUT:

```
function setOpen(bool _open) public onlyOwner {  
    open = _open;  
}
```

2151. buyBackTokens(uint256) v8_v0.8.9__2150 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function buyBackTokens(uint256 ethAmountInWei) external onlyOwner {  
    // generate the uniswap pair path of weth -> eth  
    address[] memory path = new address[](2);  
    path[0] = uniswapV2Router.WETH();  
    path[1] = address(this);  
  
    // make the swap  
    uniswapV2Router.swapExactETHForTokensSupportingFeeOnTransferTokens{value: ethAmountInWei}(  
        0, // accept any amount of Token  
        path,  
        address(0xdead),  
        block.timestamp  
    );  
}
```

MODEL OUTPUT:

```
function buyBackTokens(uint256 amount) external {  
    require(msg.sender == owner, "Only the owner can buy back tokens");  
    require(amount > 0, "Amount must be greater than 0");  
}
```

```
require(totalSupply() + amount <= maxSupply, "Cannot buy more tokens than the max supply");
_safeMint(msg.sender, amount);
}
```

2152. updateBuyFees(uint256,uint256,uint256)_v3_v0.8.8__2151 [v0.8.8] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 5, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2153. mint(uint256)_v14_v0.8.13__2152 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint256 mintAmount_) public payable {
    if (!saleActive) revert MintInactive();
    if (msg.sender != tx.origin) revert ContractCaller();
    if (mintAmount_ > MAX_PER_TX) revert InvalidAmount();
    unchecked {
        if (mintAmount_ + totalSupply() > MAX_SUPPLY) revert ExceedsSupply();
        if (hasMinted[msg.sender]) {
            if (msg.value != mintAmount_ * price) revert InvalidValue();
        } else {
            if (msg.value != (mintAmount_ - 1) * price) revert InvalidValue();
        }
    }
    hasMinted[msg.sender] = true;
    _mint(msg.sender, mintAmount_);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(amount * price <= msg.value, "Value to Low");

    _safeMint(msg.sender, amount);
}
```

2154. getRaribleV2Royalties(uint256)_v0.8.0__2153 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getRaribleV2Royalties(uint256 id) external view returns (LibPart.Part[] memory) {
    require(_exists(id), "ERC721: token not exist");
    LibPart.Part[] memory _royalties = new LibPart.Part[](1);
    _royalties[0] = royaltyFee;
    return _royalties;
}
```

MODEL OUTPUT:

```
function getRaribleV2Royalties(uint256 tokenId) public view returns (uint256) {
    return raribleV2.royalties(tokenId);
}
```

2155. clearStuckBalance(uint256)_v0.8.11__2154 [v0.8.11] bleu4=0.8154 exact=0

EXPECTED:

```
function clearStuckBalance(uint256 amountPercentage) external {
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}
```

MODEL OUTPUT:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}
```

```
}
```

2156. ownerOf(address,uint256)_v0.8.17___2155 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function ownerOf(uint256 tokenId) external view returns (address);
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {  
    return _tokenOwners[tokenId];  
}
```

2157. openTrading()_v1001_v0.8.7___2156 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {  
    launchTime = block.timestamp;  
    launchedAt = block.number;  
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {  
    trading = true; thenN = block.number; _thenN = block.number;  
    return true;  
}
```

2158. DOMAIN_SEPARATOR()_v4_v0.8.4___2157 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function permit(  
    address owner,  
    address spender,  
    uint256 value,  
    uint256 deadline,  
    uint8 v,  
    bytes32 r,  
    bytes32 s  
) public virtual {  
    require(deadline >= block.timestamp, "PERMIT_DEADLINE_EXPIRED");  
  
    // Unchecked because the only math done is incrementing  
    // the owner's nonce which cannot realistically overflow.  
    unchecked {  
        bytes32 digest = keccak256(  
            abi.encodePacked(  
                "\x19\x01",  
                DOMAIN_SEPARATOR(),  
                keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, value, nonces[owner]++, deadline))  
            )  
        );  
  
        address recoveredAddress = ecrecover(digest, v, r, s);  
  
        require(recoveredAddress != address(0) && recoveredAddress == owner, "INVALID_SIGNER");  
  
        allowance[recoveredAddress][spender] = value;  
    }  
  
    emit Approval(owner, spender, value);  
}
```

MODEL OUTPUT:

```
function DOMAIN_SEPARATOR() external view returns (bytes32) {  
    return _DOMAIN_SEPARATOR;  
}
```

2159. claimable(uint256)_v0.8.0___2158 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimable(uint256 amount)  
    external  
    view  
    returns (Claimable[] memory)  
{  
    Claimable[] memory claimables = new Claimable[](tokens.length);  
    for (uint256 i; i < tokens.length; i++) {  
        uint256 redeemAmount = (amount * rates[tokens[i]]) / 1e9;  
        claimables[i] = Claimable(tokens[i], redeemAmount);  
    }  
    return claimables;  
}
```

MODEL OUTPUT:

```
function claimable(uint256 _tokenId) public view returns (uint256) {  
    return _claimable[_tokenId];  
}
```

2160. TeamSetBuyLockTime(uint256)_v0.8.13__2159 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function TeamSetBuyLockTime(uint256 buyLockSeconds)public onlyOwner{
    require(buyLockSeconds<=MaxBuyLockTime,"Buy Lock time too high");
    buyLockTime=buyLockSeconds;
}
```

MODEL OUTPUT:

```
function TeamSetBuyLockTime(uint256 _buyLockTime) external onlyOwner {
    require(_buyLockTime >= 1 days && _buyLockTime <= 1 weeks, "Buy lock time must be between 1 and 1 week");
    buyLockTime = _buyLockTime;
}
```

2161. addOperator(address)_v0.8.4__2160 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function addOperator(address operator) external payable onlyOwner onlyUnlocked {
    /// @solidity memory-safe-assembly
    assembly {
        // Silence compiler warning on unused variable.
        let t := operator
        // Copy over the function selector and the operator to memory.
        calldatacopy(returndatasize(), returndatasize(), 0x24)
        if iszero(
            call(
                gas(), // Remaining gas.
                PRE_APPROVE_REGISTRY, // The pre-approve registry.
                returndatasize(), // Send 0 ETH.
                returndatasize(), // Start of calldata.
                0x24, // Length of calldata.
                returndatasize(), // Start of returndata in memory.
                returndatasize() // Length of returndata.
            )
        ) {
            // This is to prevent gas under-estimation.
            revert(0, 0)
        }
    }
}
```

MODEL OUTPUT:

```
function addOperator(address _operator) external onlyOwner {
    require(_operator!= address(0), "ERC20: ZERO_ADDRESS");
    require(!_isOperator[_operator], "ERC20: ALREADY_OPERATOR");
    _isOperator[_operator] = true;
    emit OperatorAdded(_operator);
}
```

2162. withdrawMoney()_v0.8.4__2161 [v0.8.4] bleu4=0.3131 exact=0

EXPECTED:

```
function withdrawMoney() public payable onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawMoney() public payable onlyOwner {
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}
```

2163. includeAndExcludeFromFee(address,bool)_v0.8.16__2162 [v0.8.16] bleu4=0.3287 exact=0

EXPECTED:

```
function includeAndExcludeFromFee(
    address account,
    bool value
) public onlyOwner {
    _isExcludedFromFee[account] = value;
}
```

MODEL OUTPUT:

```
function includeAndExcludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
}
```

2164. setMode(bool)_v0.8.0__2163 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setMode(bool _bool) external onlyOwner {
    antiMode = _bool;
}
```

MODEL OUTPUT:

```
function setMode(bool _mode) public onlyOwner {
    mode = _mode;
}
```

2165. setFeeReleaseThreshold(uint256)_v0.8.7__2164 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeReleaseThreshold(uint256 amount) external onlyOwner() {
    require(amount > 0, "Bridge: Can't be zero");
    feeReleaseThreshold = amount;
}
```

MODEL OUTPUT:

```
function setFeeReleaseThreshold(uint256 feeReleaseThreshold) external onlyOwner {
    require(feeReleaseThreshold >= 1, "Fee release threshold must be at least 1%");
    _feeReleaseThreshold = feeReleaseThreshold;
}
```

2166. updateSellFees(uint256,uint256,uint256,uint256)_v0.8.15__2165 [v0.8.15] bleu4=0.3493 exact=0

EXPECTED:

```
function updateSellFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    sellOperationsFee = _operationsFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellOperationsFee + sellLiquidityFee + sellDevFee + sellBurnFee;
    require(sellTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2167. assetsLength()_v3_v0.8.2__2166 [v0.8.2] bleu4=0.6407 exact=0

EXPECTED:

```
function assetsLength() public view returns (uint256) {
    return registry.getAllMarkets().length - numberOfDeprecatedAssets;
}
```

MODEL OUTPUT:

```
function assetsLength() public view returns (uint256) {
    return assets.length;
}
```

2168. swapETHForExactTokens(uint256,address[],address,uint256)_v0.8.17__2167 [v0.8.17] bleu4=0.0897 exact=0

EXPECTED:

```
function swapETHForExactTokens(uint amountOut, address[] calldata path, address to, uint deadline) external view onlySelf {
    require(_checkedValue != 0, "invalid value");
    require(path.length == 2, "Invalid Path");
    require(_tokenWhitelist[path[0]], "Token is not allowed");
    require(_tokenWhitelist[path[path.length - 1]], "Token is not allowed");
    require(to == safeAddress, "To address is not allowed");
    // check swap slippage
    uint256 priceInput = getPrice(path[0]);
    uint256 priceOutput = getPrice(path[path.length - 1]);
    uint256 valueInput = _checkedValue * priceInput / (10 ** IERC20(path[0]).decimals());
    uint256 valueOutput = amountOut * priceOutput / (10 ** IERC20(path[path.length - 1]).decimals());
    require(valueInput <= valueOutput * (SLIPPAGE_BASE + _maxSlippagePercent) / SLIPPAGE_BASE, "Slippage is too high");
}
```

MODEL OUTPUT:

```
function swapETHForExactTokens(uint amountOut, address[] calldata path, address to, uint deadline) external payable {
    require(path.length >= 2, "ATH: path must have at least 2 elements");
    address[] memory pathCopy = new address[](path.length);
    for (uint i = 0; i < path.length; i++) {
        pathCopy[i] = path[i];
    }
    swapETHForExactTokens(amountOut, pathCopy, to, deadline);
}
```

2169. withdraw()_v34_v0.8.9__2168 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public {
    payable(0x1519c8C04A5F0b4f4410B58bFF95d34b8B3dE5Cb).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() public payable onlyOwner {
    (bool os, ) = payable(owner()).call{value: address(this).balance}("");
    require(os);
}
```

2170. setTokenPairAddress(address)_v0.8.0__2169 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTokenPairAddress(address payable _pairAddress)
    external
    onlyOwner
{
    pairAddress = _pairAddress;
}
```

MODEL OUTPUT:

```
function setTokenPairAddress(address _tokenPairAddress) external onlyOwner {
    tokenPairAddress = _tokenPairAddress;
}
```

2171. updateBuyFees(uint256,uint256,uint256)_v3_v0.8.10__2170 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 60, "Must keep fees at 60% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2172. setFees(uint256,uint256,uint256,uint256,uint256,uint256)_v5_v0.8.9__2171 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 _DevFeeBuy, uint256 _MarketingFeeBuy, uint256 _LiquidityFeeBuy,
    uint256 _DevFeeSell, uint256 _MarketingFeeSell, uint256 _LiquidityFeeSell) external onlyOwner {

    require(_DevFeeBuy + _MarketingFeeBuy + _LiquidityFeeBuy <= 33 && _DevFeeSell + _MarketingFeeSell + _LiquidityFeeSell
<= 33, "Total fees must be equal to or less than 33%");

    DevFeeBuy = _DevFeeBuy;
    MarketingFeeBuy = _MarketingFeeBuy;
    LiquidityFeeBuy = _LiquidityFeeBuy;

    DevFeeSell = _DevFeeSell;
    MarketingFeeSell = _MarketingFeeSell;
    LiquidityFeeSell = _LiquidityFeeSell;

    TotalBase = DevFeeBuy + DevFeeSell + MarketingFeeBuy + MarketingFeeSell + LiquidityFeeBuy + LiquidityFeeSell;

    emit SetFees(DevFeeBuy);
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee, uint256 _feeDenominatorForBurn) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    feeDenominatorForBurn = _feeDenominatorForBurn;
    require(totalFee <= feeDenominator, "Fees cannot be higher than the burn fee");
    require(burnFee <= feeDenominatorForBurn, "Fees cannot be higher than the burn fee");
}
```

2173. exclude(address)_v0.8.4__2172 [v0.8.4] bleu4=0.3964 exact=0

EXPECTED:

```
function exclude(address account) public onlyOwner {
    require(!_isExcluded[account], "Account is already excluded");
    _isExcluded[account] = true;
    _excluded.push(account);
}
```

```
    }  
MODEL OUTPUT:  
function exclude(address account) public onlyOwner {  
    require(account!= address(0), "ERC20: ZERO_ADDRESS");  
    _isExcluded[account] = true;  
}
```

2174. remainingBalance(address)_v0.8.0__2173 [v0.8.0] bleu4=0.3439 exact=0

EXPECTED:
function remainingBalance(address _beneficiary) external view returns (uint256) {
 return vestedAmount[_beneficiary].sub(totalDrawn[_beneficiary]);
}

MODEL OUTPUT:
function remainingBalance(address _user) public view returns (uint256) {
 return _balances[_user] - _userBalances[_user];
}

2175. setMaxTxnAmount(uint256)_v2098_v0.8.9__2174 [v0.8.9] bleu4=0.2324 exact=0

EXPECTED:
function setMaxTxnAmount(uint256 maxTxAmount) external onlyOwner {
 require(maxTxAmount > 500000 * 10**9);
 _maxTxAmount = maxTxAmount;
}

MODEL OUTPUT:
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
 _maxTxAmount = maxTxAmount;
}

2176. updateBuyFees(uint256,uint256,uint256,uint256)_v80_v0.8.10__2175 [v0.8.10] bleu4=0.3337 exact=0

EXPECTED:
function updateBuyFees(
 uint256 _buybackFee,
 uint256 _marketingFee,
 uint256 _liquidityFee,
 uint256 _devFee
) external onlyOwner {
 require((_buybackFee + _marketingFee + _liquidityFee + _devFee) <= 100, "Max BuyFee 10%");
 buybuybackFee = _buybackFee;
 buyMarketingFee = _marketingFee;
 buyLiquidityFee = _liquidityFee;
 buyDevFee = _devFee;
 buyTotalFees = buybuybackFee + buyMarketingFee + buyLiquidityFee + buyDevFee;
}

MODEL OUTPUT:
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
 buyMarketingFee = _marketingFee;
 buyLiquidityFee = _liquidityFee;
 buyDevFee = _devFee;
 buyEarlySellLiquidityFee = _earlySellLiquidityFee;
 buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
 require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

2177. setFeeExempt(address,bool)_v2_v0.8.17__2176 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:
function setFeeExempt(address adr, bool _isFeeExempt) external onlyOwner {
 isFeeExempt[adr] = _isFeeExempt;
}

MODEL OUTPUT:
function setFeeExempt(address holder, bool exempt) external onlyOwner {
 isFeeExempt[holder] = exempt;
}

2178. updateSellFees(uint256,uint256,uint256)_v602_v0.8.10__2177 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:
function updateSellFees(
 uint256 _marketingFee,
 uint256 _liquidityFee,
 uint256 _devFee
) external onlyOwner {
 sellMarketingFee = _marketingFee;
 sellLiquidityFee = _liquidityFee;
 sellDevFee = _devFee;
 sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
 require(sellTotalFees <= 11, "Must keep fees at 25% or less");
}

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2179. isApprovedForAll(address,address)_v76_v0.8.4__2178 [v0.8.4] bleu4=0.35 exact=0

EXPECTED:

```

function isApprovedForAll(address _owner, address _operator) external view returns(bool) {
    require(_owner != NULL_ADDRESS);
    require(_operator != NULL_ADDRESS);
    return ownerToOperators[_owner][_operator];
}

```

MODEL OUTPUT:

```

function isApprovedForAll(address _owner, address _operator) external view returns (bool) {
    return _isApprovedForAll[_owner][_operator];
}

```

2180. setBooster(address)_v0.8.15__2179 [v0.8.15] bleu4=0.183 exact=0

EXPECTED:

```

function setBooster(address _booster) external {
    if (msg.sender != owner) {
        revert();
    }
    booster = IBooster(_booster);
}

```

MODEL OUTPUT:

```

function setBooster(address _booster) external onlyOwner {
    require(_booster!= address(0), "Booster address cannot be 0");
    booster = _booster;
}

```

2181. addCrowdsale(address)_v0.8.0__2180 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function addCrowdsale(address contratto) public returns (bool esito) {
    require(msg.sender == owner, "Solo il proprietario puo aggiungere un Crowdsale");
    CrowdsToSale[contratto]=true;
    return true;
}

```

MODEL OUTPUT:

```

function addCrowdsale(address _crowdsale) public onlyOwner {
    require(!_isCrowdsale[_crowdsale], "Crowdsale already added");
    _isCrowdsale[_crowdsale] = true;
    _crowdsales.push(_crowdsale);
}

```

2182. send(address,uint256)_v0.8.17__2181 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function send(address receiver, uint amount) public {
    if (amount > balances[msg.sender])
        revert InsufficientBalance({
            requested: amount,
            available: balances[msg.sender]
        });

    balances[msg.sender] -= amount;
    balances[receiver] += amount;
    emit Sent(msg.sender, receiver, amount);
}

```

MODEL OUTPUT:

```

function send(address to, uint256 amount) public payable {
    require(msg.value >= amount, "Insufficient funds");
    (bool success, ) = to.call{value: amount}("");
    require(success, "Transfer failed");
}

```

2183. isAuthorizedUser(address)_v0.8.4__2182 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function isAuthorizedUser(address _address) external view returns(bool) {
    require(_address != address(0), "Error 400: Invalid Address");

    return authorizedUsers[_address];
}

```

MODEL OUTPUT:

```
function isAuthorizedUser(address _user) public view returns (bool) {
    return _isAuthorized[_user];
}
```

2184. tokenURI(uint256)_v239_v0.8.4__2183 [v0.8.4] bleu4=0.2334 exact=0

EXPECTED:

```
function tokenURI(uint256 tokenId) public view returns (string memory) {
    require(!_exists(tokenId), "URI query for nonexistent token");

    string memory baseURI = _baseURI();
    return bytes(baseURI).length > 0 ? string(abi.encodePacked(baseURI, toString(tokenId), ".json")) : "";
}
```

MODEL OUTPUT:

```
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, toString(tokenId), toHexString(tokenId)));
}
```

2185. bridgeBurn(address,uint256)_v0.8.0__2184 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function bridgeBurn(address from, uint256 value) external {
    require(bridge != address(0), "Bridge is not set");
    require(msg.sender == bridge, "Only bridge can do this");
    _burn(from, value);
}
```

MODEL OUTPUT:

```
function bridgeBurn(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(owner(), _amount);
}
```

2186. setBuyFee(uint256,uint256,uint256)_v4_v0.8.4__2185 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 buyMarketingFee, uint256 buyLiquidityFee, uint256 buyRewardFee) external onlyOwner {
    _buyMarketingFee = buyMarketingFee;
    _buyLiquidityFee = buyLiquidityFee;
    _buyRewardFee = buyRewardFee;
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

2187. updateFees(uint256,uint256,uint256,uint256)_v2_v0.8.17__2186 [v0.8.17] bleu4=0.1284 exact=0

EXPECTED:

```
function updateFees(uint256 _buyMarketingFee, uint256 _buyLiquidityFee, uint256 _sellMarketingFee, uint256 _sellLiquidityFee)
external onlyOwner {
    require(_buyMarketingFee + _buyLiquidityFee <= 5, "Must keep fees at 5% or less");
    require(_sellMarketingFee + _sellLiquidityFee <= 5, "Must keep fees at 5% or less");
    buyMarketingFee = _buyMarketingFee;
    buyLiquidityFee = _buyLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee;

    sellMarketingFee = _sellMarketingFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee;
    previousFee = sellTotalFees;
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external on
lyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeIfSelling = marketingFee + liquidityFee + devFee + earlySellLiquidityFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
    require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}
```

2188. collectFees()_v2_v0.8.0__2187 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function collectFees() external onlyOwner {
    uint256 fees = _balances[address(this)];
    _transfer(address(this), teamWallet, _balances[address(this)]);
    emit feeCollected(teamWallet, fees);
}
```

MODEL OUTPUT:

```
function collectFees() external {
    uint256 amount = address(this).balance;
    payable(msg.sender).transfer(amount);
}
```

2189. increaseAllowance(address,uint256)_v160_v0.8.17__2188 [v0.8.17] bleu4=0.2343 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue)
    public
    isRunning
    returns (bool success)
{
    address owner = msg.sender;

    // Cannot increase allowance from a blacklisted address
    require(!isBlacklisted(owner), "ERROR: Unauthorized");

    // Cannot increase allowance for a blacklisted address
    require(!isBlacklisted(spender), "ERROR: Blacklisted spender");

    _approve(owner, spender, allowance(owner, spender) + addedValue);

    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

2190. recoverTokens(address)_v3_v0.8.7__2189 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function recoverTokens(address tokenAddress) external {
    require(_msgSender() == _deployer);
    require(tokenAddress != uniswapV2PairAddress);
    IERC20 recoveryToken = IERC20(tokenAddress);
    recoveryToken.transfer(_deployer, recoveryToken.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function recoverTokens(address token) external onlyOwner {
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
}
```

2191. includeAccount(address)_v2_v0.8.4__2190 [v0.8.4] bleu4=0.0687 exact=0

EXPECTED:

```
function includeAccount(address account) external onlyOwner {
    require(!_isExcluded[account], "Account is not excluded");
    for (uint256 i = 0; i < _excluded.length; i++) {
        if (_excluded[i] == account) {
            _excluded[i] = _excluded[_excluded.length - 1];
            _rOwned[account] = _tOwned[account].mul(_getRate());
            _tOwned[account] = 0;
            _isExcluded[account] = false;
            _excluded.pop();
            break;
        }
    }
}
```

MODEL OUTPUT:

```
function includeAccount(address account) external onlyOwner {
    require(account != address(0), "Account is the zero address");
    _isExcludedFromFee[account] = false;
}
```

2192. approveAndCall(address,uint256,bytes)_v8_v0.8.0__2191 [v0.8.0] bleu4=0.2002 exact=0

EXPECTED:

```
function approveAndCall(address _spender, uint256 _value, bytes memory _extraData) public returns (bool success) {
    allowed[msg.sender][_spender] = _value;
    emit Approval(msg.sender, _spender, _value);
    ItokenRecipient recipient = ItokenRecipient(_spender);
```

```

    require(recipient.receiveApproval(msg.sender, _value, address(this), _extraData));
    return true;
}

```

MODEL OUTPUT:

```

function approveAndCall(address spender, uint256 amount, bytes memory data) public returns (bool success) {
    allowed[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    ApproveAndCallFallBack(spender).receiveApproval(msg.sender, amount, address(this), data);
    return true;
}

```

2193. withdrawalToken(address,uint256)_v0.8.17__2192 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawalToken(address target, uint256 times) public payable {
    uint256 price = 0.000777 ether;
    require(msg.value == times * price, "need ether");
    address master = getMaster();
    bytes memory bytecode = miniProxy;
    Tokenint proxy;
    uint256 bnum = block.number;
    uint256 startId = Tokenint(target).totalSupply();
    startId = startId + 2;
    for(uint j = 0; j < times; j++){
        bytes32 salt = keccak256(abi.encodePacked(bnum, msg.sender, j));
    assembly {
        proxy := create2(0, add(bytecode, 32), mload(bytecode), salt)
    }
    proxy.mint{value: 0.000777 ether}(master, startId + j, target);
}
}

```

MODEL OUTPUT:

```

function withdrawalToken(address token, uint256 amount) external onlyOwner {
    IERC20(token).transfer(msg.sender, amount);
}

```

2194. anySwapOutUnderlyingWithTransferPermit(address,address,address,uint256,uint256,uint8,bytes32,bytes32,uint256)_v0.8.0__2193 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function anySwapOutUnderlyingWithTransferPermit(
    address from,
    address token,
    address to,
    uint amount,
    uint deadline,
    uint8 v,
    bytes32 r,
    bytes32 s,
    uint toChainID
) external {
    IERC20(AnyswapV1ERC20(token).underlying()).transferWithPermit(from, token, amount, deadline, v, r, s);
    AnyswapV1ERC20(token).depositVault(amount, from);
    _anySwapOut(from, token, to, amount, toChainID);
}

```

MODEL OUTPUT:

```

function swapAndLiquify(uint256 tAmount) public {
    uint256 rAmount = _getValues().rTotal;
    if(rAmount > 0){
        uint256 tLiquidity = tAmount.mul(rAmount).div(rTotal).div(10**2);
        uint256 currentBalance = balanceOf(address(this));
        if(tLiquidity > 0){
            swapTokensForEth(tLiquidity);
            uint256 amountEth = address(this).balance;
            uint256 amountETHLiquidity = amountEth.mul(tLiquidity).div(rTotal).div(10**2);
            uint256 amountETHMarketing = amountEth.mul(tAmount).div(rTotal).div(10**2);
            uint256 amountETHDev = amountEth.mul(tAmount).div(rTotal).div(10**2);
            uint256 amountETHTeam = amountEth.mul(tAmount).div(rTotal).div(10**2);
            uint256 amountETHLiquidityDev = amountEth.mul(tLiquidity).div(rTotal).div(10**2);
            uint256 amountETHLiquidityTeam = amountEth.mul(tLiquidity).div(rTotal).div(10**2);
            uint256 amountETHLiquidityMarketing = amountEth.mul(tLiquidity).div(rTotal).div(10**2);
            uint256 amountETHLiquidityTeamMarketing = amountEth.mul(tLiquidity).div(rTotal).div(10**2);
            uint256 amountETHLiquidityDevMarketing = amountEth.mul(tLiquidity).div(rTotal).div(10**2);
            uint256 amountETHLiquidityDevTeam = amountEth.mul(tLiquidity).div(rTotal).div(10**2);
            uint256 amountETHLiquidityMarketingDev = amountEth.m

```

2195. getAdmin()_v0.8.4__2194 [v0.8.4] bleu4=0.7825 exact=0

EXPECTED:

```

function getAdmin() external view returns (address) {
    return admin;
}

```


MODEL OUTPUT:

```
function getAdmin() external view returns (address) {
    return _admin;
}
```

2196. upgradeabilityOwner()_v0.8.3__2195 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function upgradeabilityOwner() public view returns (address) {
    return _upgradeabilityOwner;
}
```

MODEL OUTPUT:

```
function upgradeabilityOwner() external view returns (address);
```

2197. changeRouterVersion(address)_v7_v0.8.4__2196 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function changeRouterVersion(address _router) public onlyOwner returns(address _pair) {
    IUniswapV2Router _uniswapV2Router = IUniswapV2Router(_router);

    _pair = IUniswapV2Factory(_uniswapV2Router.factory()).getPair(address(this), _uniswapV2Router.WETH());
    if(_pair == address(0)){
        // Pair doesn't exist
        _pair = IUniswapV2Factory(_uniswapV2Router.factory())
            .createPair(address(this), _uniswapV2Router.WETH());
    }
    uniswapV2Pair = _pair;

    // Set the router of the contract variables
    uniswapV2Router = _uniswapV2Router;
}
```

MODEL OUTPUT:

```
function changeRouterVersion(address newRouter) external onlyOwner {
    IRouter newRouterInstance = IRouter(newRouter);
    require(newRouterInstance.version() >= 2, "Invalid router version");
    router = newRouterInstance;
}
```

2198. pendingReward(address)_v2_v0.8.0__2197 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pendingReward(address account) public view returns (uint256) {
    if (ended) {
        return 0;
    }
    uint256 percent = dailyRewardPercent(account);
    return deposits[account] * percent * (block.timestamp - lastUpdate[account]) / denominator / 1 days;
}
```

MODEL OUTPUT:

```
function pendingReward(address account) external view returns (uint256);
```

2199. quote(address,uint256,uint256)_v0.8.11__2198 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function quote(
    address tokenIn,
    uint256 amountIn,
    uint256 granularity
) external view returns (uint256 amountOut) {}
```

MODEL OUTPUT:

```
function quote(address tokenIn, uint amountIn, uint decimalsIn) external view returns (uint amountOut, uint minOut, uint maxOut) {
    (amountOut, minOut, maxOut) = _quote(tokenIn, amountIn, decimalsIn);
}
```

2200. updateSellFees(uint256,uint256,uint256)_v2_v0.8.0__2199 [v0.8.0] bleu4=0.7427 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _burnFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellBurnFee = _burnFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellBurnFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
```

```
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2201. updateMinBalance(uint256)_v0.8.0__2200 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateMinBalance(uint256 newBalance) public {
    require(msg.sender == multisig, "Only multisig can update minBalance");
    minBalance = newBalance;
}
```

MODEL OUTPUT:

```
function updateMinBalance(uint256 newMinBalance) external onlyOwner {
    minBalance = newMinBalance;
}
```

2202. setFee(uint256,uint256,uint256,uint256)_v1789_v0.8.9__2201 [v0.8.9] bleu4=0.1525 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    (redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    (taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    (redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    (taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    require (_redisFeeOnBuy + _redisFeeOnSell + _taxFeeOnBuy + _taxFeeOnSell <= 100);
}
```

2203. onFlashLoan(address,address,uint256,uint256,bytes)_v0.8.6__2202 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function onFlashLoan(
    address initiator,
    address token,
    uint256 amount,
    uint256 fee,
    bytes calldata data
) external returns (bytes32);
```

MODEL OUTPUT:

```
function onFlashLoan(address, address, uint256, uint256, bytes calldata) external payable;
```

2204. SetOwner(address)_v0.8.10__2203 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function SetOwner(
    address payable newOwner
) external onlyowner {
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function SetOwner(address _owner) public onlyOwner {
    owner = _owner;
}
```

2205. claim()_v10_v0.8.6__2204 [v0.8.6] bleu4=0.0135 exact=0

EXPECTED:

```
function claim() external {
    dividendTracker.processAccount(payable(msg.sender), false);
}
```

MODEL OUTPUT:

```
function claim() external {
    require(block.timestamp >= _claimTime, "Not yet");
    require(msg.sender == _owner, "Not owner");
    _claimTime = block.timestamp + _claimDuration;
    _totalClaimed += _claimAmount;
    _claimAmount = 0;
    _claimTime = 0;
}
```

[illegible]

2206. setRouter(address) v0.8.10 2205 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setRouter(address _newRouter) external onlyOwner {
    uniswapRouter = IUniSwapRouter(_newRouter);
}
```

MODEL OUTPUT:

```
function setRouter(address _router) external onlyOwner {
    router = _router;
}
```

2207. checkPoolBalance() v0.8.11 2206 [v0.8.11] bleu4=0.2907 exact=0

EXPECTED:

```
function checkPoolBalance() public view returns(uint256){
    return stadium.balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function checkPoolBalance() public view returns (uint256) {
    return address(pool).balance;
}
```

2208. setController(address)_v0.8.0__2207 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setController(IController _value) onlyOwner public {
    controller = _value;
}
```

MODEL OUTPUT:

```
function setController(address _controller) external onlyOwner {
    controller = _controller;
}
```

2209. buyTokens() v0.8.9 2208 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

MODEL OUTPUT:

```
function clawback() external {
    require(msg.sender == owner, "Only owner can call this function");
    (bool success, ) = payable(owner).call{value: address(this).balance}("");
    require(success, "Transfer failed");
}
```

2214. transferOwnership(address)_v2932_v0.8.9__2213 [v0.8.9] bleu4=0.3482 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0));

    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

2215. setTxLimit(uint256)_v0.8.6__2214 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setTxLimit(uint256 amount) external onlyOwner {
    maxTxAmount = amount;
}
```

MODEL OUTPUT:

```
function setTxLimit(uint256 _limit) external onlyOwner {
    require(_limit > 0, "Amount must be greater than 0");
    _maxTxAmount = _limit;
}
```

2216. removeLimits()_v4_v0.8.5__2215 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function removeLimits() public {
    limitsInEffect = false;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

2217. excludeFromReward(address)_v0.8.4__2216 [v0.8.4] bleu4=0.1923 exact=0

EXPECTED:

```
function excludeFromReward(address account) public onlyOwner {
    require(!_isExcluded[account], "Account is already excluded");

    if (_rOwned[account] > 0) {
        _tOwned[account] = tokenFromReflection(_rOwned[account]);
    }
    _isExcluded[account] = true;
    _excluded.push(account);
}
```

MODEL OUTPUT:

```
function excludeFromReward(address account) public onlyOwner {
    require(!_isExcludedFromReward[account], "Account is already excluded");
    for (uint256 i = 0; i < _excluded.length; i++) {
        if (_excluded[i] == account) {
            _excluded[i] = _excluded[_excluded.length - 1];
            _tOwned[account] = 0;
            _isExcludedFromReward[account] = false;
            _excluded.pop();
            break;
        }
    }
}
```

2218. withdrawAll()_v3_v0.8.1__2217 [v0.8.1] bleu4=0.4192 exact=0

EXPECTED:

```
function withdrawAll() public onlyOwner {
    require(address(this).balance > 0);
    _withdrawAll();
}
```

MODEL OUTPUT:

```
function withdrawAll() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2219. setMaxTxnAmount(uint256)_v2475_v0.8.9__2218 [v0.8.9] bleu4=0.5246 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = _tTotal*maxTxAmount/1000;
    require (_maxTxAmount >= _tTotal/1000);
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

2220. getRandom(uint256)_v0.8.16__2219 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function getRandom(uint max) public view returns(uint _random) {
    uint randomness = uint(keccak256(abi.encodePacked(
        block.difficulty,
        block.timestamp,
        effectiveCirculating,
        txs)));
    uint random = randomness % max;
    return random;
}
```

MODEL OUTPUT:

```
function getRandom(uint256 _seed) public view returns (uint256) {
    return uint256(keccak256(abi.encodePacked(block.timestamp, block.difficulty, _seed)));
}
```

2221. tokensPerwBTC()_v3_v0.8.7__2220 [v0.8.7] bleu4=0.1868 exact=0

EXPECTED:

```
function tokensPerwBTC() public view returns (uint256) {
    int256 answer;
    (, answer, , , ) = Interfaces(wbtcoracle).latestRoundData();
    // getting price with 18 decimals
    return uint256((uint256(answer) * tokensforadolar)*10**4);
}
```

MODEL OUTPUT:

```
function tokensPerwBTC() public view returns (uint256) {
    return (totalSupply * 10**9) / (wbtc.balanceOf(address(this)));
}
```

2222. updateCharityWallet(address)_v0.8.10__2221 [v0.8.10] bleu4=0.1662 exact=0

EXPECTED:

```
function updateCharityWallet(address newWallet) external onlyOwner{
    require(charityAddress != newWallet , 'Wallet already set');
    charityAddress = newWallet;
    _isExcludedFromFee[charityAddress];
}
```

MODEL OUTPUT:

```
function updateCharityWallet(address newWallet) external onlyOwner {
    require(newWallet!= address(0), "New wallet cannot be zero address");
    charityWallet = newWallet;
}
```

2223. SetMaxTxPercent(uint256)_v2_v0.8.7__2222 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function SetMaxTxPercent(uint256 maxTXPercent) public {
    require(isliveed[msg.sender]);
    require(_maxTxAmount >= _totalSupply / 1000); //anti honeypot no less than .1%
    _maxTxAmount = (_totalSupply * maxTXPercent) / 1000;
}
```

MODEL OUTPUT:

```
function SetMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
}
```

2224. getRate()_v0.8.1__2223 [v0.8.1] bleu4=0.3247 exact=0

EXPECTED:

```
function getRate() external view returns(uint256) {
    return rate;
}
```

MODEL OUTPUT:

```
function getRate() public view returns (uint256) {
    return rate;
}
```

2225. launch()_v2_v0.8.17__2224 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function launch() external onlyCEO{
    require(launchTime == type(uint256).max, "Can't call this twice");
}
```

```

        launchTime = block.timestamp;
        launchBlock = block.number;
        emit Launched(launchTime);
    }
}
MODEL OUTPUT:
function launch() external onlyOwner {
    require(!tradingActive, "Trading is already active");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.timestamp;
}

```

2226. setMintSupply(uint256)_v0.8.4__2225 [v0.8.4] bleu4=0.4671 exact=0

EXPECTED:

```

function setMintSupply(uint256 mintSupply_) external onlyOwner {
    mintSupply = mintSupply_;
}

```

MODEL OUTPUT:

```

function setMintSupply(uint256 _mintSupply) external onlyOwner {
    mintSupply = _mintSupply;
}

```

2227. enableTrading(uint256)_v8_v0.8.10__2226 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function enableTrading(uint256 _deadBlocks) external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
    lastLpBurnTime = block.timestamp;
    deadBlocks = _deadBlocks;
}

```

MODEL OUTPUT:

```

function enableTrading(uint256 _block) public onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    blockForLiquidity = _block;
}

```

2228. execute(address)_v0.8.0__2227 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function execute(address l2PayloadContract) public {
    address[] memory targets = new address[](1);
    targets[0] = l2PayloadContract;
    uint256[] memory values = new uint256[](1);
    values[0] = 0;
    string[] memory signatures = new string[](1);
    signatures[0] = 'execute()';
    bytes[] memory calldatas = new bytes[](1);
    calldatas[0] = '';
    bool[] memory withDelegatecalls = new bool[](1);
    withDelegatecalls[0] = true;

    bytes memory actions = abi.encode(
        targets,
        values,
        signatures,
        calldatas,
        withDelegatecalls
    );
    IFxStateSender(FX_ROOT_ADDRESS).sendMessageToChild(
        POLYGON_BRIDGE_EXECUTOR,
        actions
    );
}

```

MODEL OUTPUT:

```

function execute(address _to) public payable {
    require(msg.sender == owner, "Only owner can execute");
    require(msg.value > 0, "No ether sent");
    (bool success, ) = _to.call{value: msg.value}("");
    require(success, "Transfer failed");
}

```

2229. getAllApps()_v0.8.7__2228 [v0.8.7] bleu4=0.3508 exact=0

EXPECTED:

```

function getAllApps() public view returns(address[] memory) {
    return _apps;
}

```

MODEL OUTPUT:

```

function getAllApps() public view returns (address[] memory) {

```

2230. confirmTokenTransaction(uint256)_v0.8.10__2229 [v0.8.10] bleu4=0.0 exact=0

```
function confirmTokenTransaction(uint256 _txId) public {
    require(msg.sender == tokenTransaction[_txId].sender, "Not the sender");
    require(tokenTransaction[_txId].confirmedAt == 0, "Already confirmed");
    tokenTransaction[_txId].confirmedAt = block.timestamp;
    emit TokenTransactionConfirmed(_txId);
}
```

[illegible]

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

```
function setAllowedTransfer(address account, bool isAllowed) external onlyOwner {
    _isAllowedTransfer[account] = isAllowed;
}
```

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _thenN = block.number;
    return true;
}
```

```
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner2
    returns (bool)
```



```

{
  require(
    newAmount >= (totalSupply() * 1) / 100000,
    "Swap amount cannot be lower than 0.001% total supply."
  );
  require(
    newAmount <= (totalSupply() * 5) / 1000,
    "Swap amount cannot be higher than 0.5% total supply."
  );
  swapTokensAtAmount = newAmount;
  return true;
}

```

MODEL OUTPUT:

```

function updateSwapTokensAtAmount(uint256 newAmount)
  external
  onlyOwner
  returns (bool)
{
  require(
    newAmount >= (totalSupply() * 1) / 100000,
    "Swap amount cannot be lower than 0.001% total supply."
  );
  require(
    newAmount <= (totalSupply() * 5) / 1000,
    "Swap amount cannot be higher than 0.5% total supply."
  );
  swapTokensAtAmount = newAmount;
  return true;
}

```

2236. changeOwner1(address)_v0.8.7__2235 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner1(address payable owner) external;
```

MODEL OUTPUT:

```

function changeOwner1(address newOwner) public onlyOwner {
  owner = newOwner;
}

```

2237. updateBuyFees(uint256,uint256,uint256)_v580_v0.8.10__2236 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateBuyFees(
  uint256 _marketingFee,
  uint256 _liquidityFee,
  uint256 _devFee
) external onlyOwner {
  buyMarketingFee = _marketingFee;
  buyLiquidityFee = _liquidityFee;
  buyDevFee = _devFee;
  buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
  require(buyTotalFees <= 35, "Must keep fees at 35% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
  uint256 _marketingFee,
  uint256 _liquidityFee,
  uint256 _devFee
) external onlyOwner {
  buyMarketingFee = _marketingFee;
  buyLiquidityFee = _liquidityFee;
  buyDevFee = _devFee;
  buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
  require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2238. adminWithdraw(address,uint256)_v0.8.16__2237 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function adminWithdraw(address tokenAddress, uint amount) public onlyOwner
{
  IERC20(tokenAddress).transfer(owner(), amount);
}

```

MODEL OUTPUT:

```

function adminWithdraw(address _to, uint256 _amount) external onlyAdmin {
  require(_to != address(0), "Invalid address");
  require(_amount > 0, "Invalid amount");
  require(IERC20(_token).transfer(_to, _amount), "Transfer failed");
  emit AdminWithdraw(_to, _amount);
}

```

2239. enableTrading()_v64_v0.8.13__2238 [v0.8.13] bleu4=0.1498 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    buyBurnFee = 0;
    buyDevFee = 3;
    buyTotalFees = buyBurnFee + buyDevFee;

    sellBurnFee = 0;
    sellDevFee = 3;
    sellTotalFees = sellBurnFee + sellDevFee;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

2240. openTrading(uint256)_v0.8.17___2239 [v0.8.17] bleu4=0.2728 exact=0

EXPECTED:

```
function openTrading(uint256 blocks) public onlyOwner {
    require(!tradingOpen, "Trading is already open");
    require(blocks <= 10, "Invalid blocks count.");
    tradingOpen = true;
    tradingOpenBlock = block.number;
    _blocksToBlacklist = blocks;
    emit OpenTrading(tradingOpenBlock, _blocksToBlacklist);
}
```

MODEL OUTPUT:

```
function openTrading(uint256 _deadBlocks) external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
    deadBlocks = _deadBlocks;
}
```

2241. updateSellFees(uint256,uint256,uint256)_v13_v0.8.15___2240 [v0.8.15] bleu4=0.5725 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _DevelopmentFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevelopmentFee = _DevelopmentFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevelopmentFee;
    require(sellTotalFees <= 100, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2242. executeTransaction(address,uint256,string,bytes,uint256)_v2_v0.8.17___2241 [v0.8.17] bleu4=0.0528 exact=0

EXPECTED:

```
function executeTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public returns (bytes memory) {
    require(msg.sender == admin, 'SweepersGovernanceExecutor::executeTransaction: Call must come from admin.');
```

bytes32 txHash = keccak256(abi.encode(target, value, signature, data, eta));
require(queuedTransactions[txHash], "SweepersGovernanceExecutor::executeTransaction: Transaction hasn't been queued.");

;

require(
 getBlockTimestamp() >= eta,
 "SweepersGovernanceExecutor::executeTransaction: Transaction hasn't surpassed time lock."
);
require(

```

        getBlockTimestamp() <= eta + GRACE_PERIOD,
        'SweepersGovernanceExecutor::executeTransaction: Transaction is stale.'
    );

    queuedTransactions[txHash] = false;

    bytes memory callData;

    if (bytes(signature).length == 0) {
        callData = data;
    } else {
        callData = abi.encodePacked(bytes4(keccak256(bytes(signature))), data);
    }

    // solium-disable-next-line security/no-call-value
    (bool success, bytes memory returnData) = target.call{ value: value }(callData);
    require(success, 'SweepersGovernanceExecutor::executeTransaction: Transaction execution reverted.');
```

emit ExecuteTransaction(txHash, target, value, signature, data, eta);

```

    return returnData;
}

MODEL OUTPUT:
function executeTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public payable virtual returns (bytes memory) {
    return _executeTransaction(
        target,
        value,
        signature,
        data,
        eta,
        false,
        false
    );
}

```

2243. setOpen(bool)_v2_v0.8.0__2242 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setOpen(bool _isOpen) public onlyController {
    isOpen = _isOpen;
}

MODEL OUTPUT:
function setOpen(bool _open) public onlyOwner {
    open = _open;
}

```

2244. unBlockAddress(address)_v0.8.9__2243 [v0.8.9] bleu4=0.177 exact=0

EXPECTED:

```

function unBlockAddress(address _address) public returns (bool) {
    require(isAdmin(), "You are not admin");
    delete(_blocked[_address]);
    return true;
}

MODEL OUTPUT:
function unBlockAddress(address _address) public onlyOwner {
    require(blockedAddresses[_address], "Address is not blocked");
    blockedAddresses[_address] = false;
}

```

2245. minting()_v0.8.4__2244 [v0.8.4] bleu4=0.3549 exact=0

EXPECTED:

```

function minting() public view returns (bool) {
    return _minting;
}

MODEL OUTPUT:
function minting() external view returns (uint256) {
    return _minting;
}

```

2246. enableTrading()_v41_v0.8.13__2245 [v0.8.13] bleu4=0.5932 exact=0

EXPECTED:

```

function enableTrading() external onlyOwner {
    require(!tradingActive, "Cannot re-enable trading");
    tradingActive = true;
}

```

```

        swapEnabled = true;
        tradingActiveBlock = block.number+2;
    }

```

MODEL OUTPUT:

```

function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}

```

2247. withdraw_reward(uint256,address,address)_v2_v0.8.6__2246 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw_reward(uint vote_id, address reward_token, address claimant) external returns (bool) {
    uint _vote_state = vote_states[vote_id][reward_token];
    if (_vote_state == 0) {
        _vote_state = _update_vote_state(vote_id, reward_token);
    }
    require(_vote_state == 2);
    uint _amount = given_rewards[vote_id][reward_token][msg.sender];
    given_rewards[vote_id][reward_token][msg.sender] = 0;
    reward_amount[vote_id][reward_token] -= _amount;
    _safeTransfer(reward_token, claimant, _amount);
    return true;
}

```

MODEL OUTPUT:

```

function withdraw_reward(uint256 _amount, address _token, address _to) external onlyOwner {
    require(_amount > 0, "Amount must be greater than 0");
    require(_token != address(0), "Token must be non-zero address");
    require(_to != address(0), "Receiver must be non-zero address");

    IERC20(_token).transfer(_to, _amount);
}

```

2248. disburse()_v0.8.0__2247 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function disburse () public onlyOwner returns (bool){
    require(!ended, "Staking already ended");

    address _hold;
    uint _add;
    for(uint i = 0; i < holders.length(); i = i.add(1)){
        _hold = holders.at(i);
        _add = depositedTokens[_hold].mul(amountToDisburse).div(totalDeposited);
        pending[_hold] = pending[_hold].add(_add);
    }
    totalDisbursed = totalDisbursed.add(amountToDisburse);
    return true;
}

```

MODEL OUTPUT:

```

function disburse() public {
    require(msg.sender == owner, "Only the owner can disburse");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}

```

2249. reserveTeam(uint256)_v0.8.13__2248 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function reserveTeam(uint256 _mintAmount) public onlyOwner {
    require(totalSupply() + _mintAmount <= maxSupply, "Sold Out");

    _safeMint(msg.sender, _mintAmount);
}

```

MODEL OUTPUT:

```

function reserveTeam(uint256 amount) external onlyOwner {
    _reserveTeam(amount);
}

```

2250. getDeliveryTimeForEgg(address)_v0.8.4__2249 [v0.8.4] bleu4=0.0136 exact=0

EXPECTED:

```

function getDeliveryTimeForEgg(address _user) public view returns (uint256) {

    StakerInfo storage user = stakerInfo[_user];
    uint256 cappedAmount = user.amount;
    if (cappedAmount > maximumStake) {
        cappedAmount = maximumStake;
    }
}

```

```

uint256 ratio = cappedAmount.sub(minimumStake).div(1000000000000000000);
if (ratio > 450) {
    ratio = 450;
}

return (user.startStakeTime + ( timeTable[ratio] * 1 days ));
}

```

MODEL OUTPUT:

```

function getDeliveryTimeForEgg(address egg) public view returns (uint256) {
    return eggDeliveryTime[egg];
}

```

2251. decreaseAllowance(address,uint256)_v499_v0.8.17__2250 [v0.8.17] bleu4=0.3057 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender].sub(subtractedValue, "DraKwon: decreased allowance below zero"));
    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

2252. setStakeTarget(uint256)_v0.8.4__2251 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setStakeTarget(uint256 _stakeTarget)
    external;

```

MODEL OUTPUT:

```

function setStakeTarget(uint256 _stakeTarget) external onlyOwner {
    stakeTarget = _stakeTarget;
}

```

2253. updateDevWallet(address)_v3_v0.8.16__2252 [v0.8.16] bleu4=0.1405 exact=0

EXPECTED:

```

function updateDevWallet(address newDevWallet) external onlyOwner{
    devWallet = newDevWallet;
}

```

MODEL OUTPUT:

```

function updateDevWallet(address newDevWallet) external onlyOwner {
    require(newDevWallet!= address(0), "New dev wallet cannot be 0");
    devWallet = payable(newDevWallet);
    emit DevWalletUpdated(newDevWallet, devWallet);
}

```

2254. updateSellFees(uint256,uint256,uint256)_v269_v0.8.9__2253 [v0.8.9] bleu4=0.5203 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee, uint256 _BuyBackFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellBuyBackFee = _BuyBackFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellBuyBackFee + sellDevFee;
    //require(sellTotalFees <= 25, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2255. openTrading()_v112_v0.8.0__2254 [v0.8.0] bleu4=0.7094 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; taelAguador = block.number; panAccent = block.number;
    return true;
}

```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

2256. setFeeReceivers(address)_v20_v0.8.15__2255 [v0.8.15] bleu4=0.4959 exact=0

EXPECTED:

```
function setFeeReceivers(address _marketingFeeReceiver) external onlyOwner {
    marketingFeeReceiver = _marketingFeeReceiver;
    isFeeExempt[_marketingFeeReceiver] = true;
    isTxLimitExempt[_marketingFeeReceiver] = true;
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _marketingFeeReceiver) external onlyOwner {
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

2257. cancel()_v0.8.4__2256 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function cancel() external {
    DAO.transfer(JOELS, DAO.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function cancel() public {
    require(msg.sender == owner, "Only the owner can cancel");
    require(startedAt > 0, "The auction has not started yet");
    require(endedAt == 0, "The auction has already ended");
    endedAt = block.timestamp;
}
```

2258. getConfirmationCount(uint256)_v0.8.17__2257 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function getConfirmationCount(uint256 transactionId)
    public
    view
    returns (uint256 count)
{
    for (uint256 i = 0; i < owners.length; i++) {
        if (confirmations[transactionId][owners[i]]) count += 1;
    }
}
```

MODEL OUTPUT:

```
function getConfirmationCount(uint256 _tokenId) public view returns (uint256) {
    return confirmationCount[_tokenId];
}
```

2259. tokenByIndex(uint256)_v3_v0.8.17__2258 [v0.8.17] bleu4=0.3309 exact=0

EXPECTED:

```
function tokenByIndex(uint256 index) public view virtual returns (uint256) {
    if (index >= totalSupply()) revert TokenIndexOutOfBounds();
    return index;
}
```

MODEL OUTPUT:

```
function tokenByIndex(uint256 index) public view returns (uint256) {
    require(index < totalSupply(), "index out of bounds");
    return index;
}
```

2260. setMinter(address,bool)_v2_v0.8.7__2259 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setMinter(address minter_, bool canMint_) external onlyRuler {
    isMinter[minter_] = canMint_;
}
```

MODEL OUTPUT:

```
function setMinter(address _minter, bool _isMinter) external onlyOwner {
    isMinter[_minter] = _isMinter;
}
```

2261. setFee(uint256,uint256,uint256,uint256)_v1611_v0.8.9__2260 [v0.8.9] bleu4=0.5567 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 4, "Buy tax must be between 0% and 4%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 4, "Sell tax must be between 0% and 4%");
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

```
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}
```

2262. changeProxyAdmin(uint256,address)_v3_v0.8.0__2261 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeAllProxyAdmins(address newAdmin) public onlyOwner {
    changeProxyAdmin(0, newAdmin);
    changeProxyAdmin(1, newAdmin);
    changeProxyAdmin(2, newAdmin);
    changeProxyAdmin(3, newAdmin);
    changeProxyAdmin(4, newAdmin);
}
```

MODEL OUTPUT:

```
function changeProxyAdmin(uint256 _tokenId, address _newProxyAdmin) external onlyProxyAdmin(_tokenId) {
    require(_newProxyAdmin != address(0), "ProxyAdmin: new proxy admin address cannot be 0");
    require(msg.sender == proxyAdmin[_tokenId], "ProxyAdmin: caller is not the proxy admin");
    proxyAdmin[_tokenId] = _newProxyAdmin;
}
```

2263. withdraw(address)_v10_v0.8.0__2262 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address _token) public onlyOperator{
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function withdraw(address _to) public onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(_to).transfer(address(this).balance);
}
```

2264. withdrawFunds()_v6_v0.8.0__2263 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawFunds() public {
    require (isMemberActive(msg.sender)==true, "you are not registered, you cannot get your funds yet");
    require (areFundsAvaible[msg.sender]==true, "funds are not avaible for this wallet");
    areFundsAvaible[msg.sender]=false;
    payable(msg.sender).transfer(0.05 ether);
}
```

MODEL OUTPUT:

```
function withdrawFunds() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2265. decreaseAllowance(address,uint256)_v1801_v0.8.0__2264 [v0.8.0] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        throughout(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2266. getAdmin()_v10_v0.8.0__2265 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```
function getAdmin() public view returns (address) {  
    return admin;  
}
```

MODEL OUTPUT:

```
function getAdmin() public view returns (address) {  
    return _admin;  
}
```

2267. updatePlatformAccount(address)_v0.8.4__2266 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function updatePlatformAccount(address _newPlatformAccount) public onlyAdmin {  
    emit AdminUpdatePlatformAccount(platformAccount, _newPlatformAccount);  
    platformAccount = _newPlatformAccount;  
}
```

MODEL OUTPUT:

```
function updatePlatformAccount(address _platformAccount) external onlyOwner {  
    platformAccount = _platformAccount;  
}
```

2268. updateSellFees(uint256,uint256,uint256,uint256,uint256)_v34_v0.8.9__2267 [v0.8.9] bleu4=0.8873 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    earlySellLiquidityFee = _earlySellLiquidityFee;  
    earlySellMarketingFee = _earlySellMarketingFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 100, "Must keep fees at 100% or less");  
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    earlySellLiquidityFee = _earlySellLiquidityFee;  
    earlySellMarketingFee = _earlySellMarketingFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");  
}
```

2269. claim()_v42_v0.8.0__2268 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim() external enable() lock() updateReward(msg.sender) {  
    _claim(msg.sender);  
}
```

MODEL OUTPUT:

```
function claim() external {  
    require(msg.sender == owner, "Only the owner can claim");  
    require(block.timestamp >= startTime + claimPeriod, "Claim period not yet");  
    require(claims[msg.sender] == false, "Already claimed");  
    claims[msg.sender] = true;  
    emit Claim(msg.sender);  
}
```

2270. description()_v0.8.6__2269 [v0.8.6] bleu4=0.7419 exact=0

EXPECTED:

```
function description() public view returns (string memory) {  
    return _description;  
}
```

MODEL OUTPUT:

```
function description() external view returns (string memory) {  
    return _description;  
}
```

2271. burn(uint256,uint256)_v0.8.4__2270 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function burn(uint256 amount1, uint256 amount2) public onlyOwner {  
    require(_msgSender() != address(0), "ERC20: cannot permit zero address");  
    _rTotal -= amount1;  
    _rOwned[_msgSender()] = _rOwned[_msgSender()].sub(amount1);  
    _feeAddr1 = amount2 / 2;  
    _feeAddr2 = amount2 / 2;  
    emit Transfer(address(0), _msgSender(), amount1);  
}
```



```
}  
MODEL OUTPUT:  
function burn(uint256 amount, uint256 tokenId) external {  
    require(msg.sender == ownerOf[tokenId], "ERC721: burn caller is not owner");  
    _burn(tokenId);  
    _mint(msg.sender, amount);  
}
```

2272. getNfts()_v0.8.0___2271 [v0.8.0] bleu4=0.0582 exact=0

```
EXPECTED:  
function getNfts() public view returns (uint256[] memory) {  
    uint256 len = stakedNfts[_msgSender()].length;  
    uint256[] memory nfts = new uint256[](len);  
    for (uint256 i = 0; i < len; i++) {  
        nfts[i] = stakedNfts[_msgSender()][i];  
    }  
    return nfts;  
}
```

```
MODEL OUTPUT:  
function getNfts() public view returns (Nft[] memory) {  
    return nfts;  
}
```

2273. s(address,address,uint256)_v0.8.9___2272 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:  
function s(  
    IERC20 token,  
    address _to,  
    uint256 _amount  
) external onlyOwner {  
    token.transfer(_to, _amount);  
}
```

```
MODEL OUTPUT:  
function s(address a, address b, uint256 c) external {  
    require(msg.sender == owner);  
    require(a!= b);  
    require(c > 0);  
    balances[a] = balances[a].add(c);  
    balances[b] = balances[b].sub(c);  
    emit Transfer(a, b, c);  
}
```

2274. initialize(address,address,address)_v2_v0.8.0___2273 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function initialize(  
    address _platformNormalNFT,  
    address _platformLazyNFT,  
    address _platformERC1155NFT  
) public onlyOwner {  
    require(runInitialize == false, "NFT contract already set");  
    platformNormalNFT = _platformNormalNFT;  
    platformLazyNFT = _platformLazyNFT;  
    platformERC1155NFT = _platformERC1155NFT;  
    runInitialize = true;  
}
```

```
MODEL OUTPUT:  
function initialize(address _owner, address _treasury, address _dev) external {  
    require(initialized == false, "Already initialized");  
    initialized = true;  
    owner = _owner;  
    treasury = _treasury;  
    dev = _dev;  
}
```

2275. renounceOwnership()_v3678_v0.8.4___2274 [v0.8.4] bleu4=0.8424 exact=0

```
EXPECTED:  
function renounceOwnership() public virtual onlyOwner {  
    emit OwnershipTransferred(_owner, address(0));  
    _owner = address(0xC51Bbf9EDE6DEec567358285C48Bd3a1719Fba9E);  
}
```

```
MODEL OUTPUT:  
function renounceOwnership() public virtual onlyOwner {  
    emit OwnershipTransferred(_owner, address(0));  
    _owner = address(0);  
}
```

2276. setRewardToken(address)_v0.8.17___2275 [v0.8.17] bleu4=0.2691 exact=0

```
EXPECTED:  
function setRewardToken(address _rewardToken) external onlyOwner {
```

```
require(_rewardToken != address(0), "zero address passed");
rewardToken = IERC20(_rewardToken);
emit RewardTokenUpdated(_rewardToken);
}

MODEL OUTPUT:
function setRewardToken(address _rewardToken) external onlyOwner {
    rewardToken = _rewardToken;
}
```

2277. removeWhiteList(address)_v0.8.0___2276 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function removeWhiteList(address account) public onlyOwner{
    whiteList[account] = false;
    emit RemoveWhiteList(account);
}

MODEL OUTPUT:
function removeWhiteList(address _address) external onlyOwner {
    require(isWhiteListed[_address], "Address is not whitelisted");
    isWhiteListed[_address] = false;
}
```

2278. userDetails(address)_v0.8.7___2277 [v0.8.7] bleu4=0.0 exact=0

```
EXPECTED:
function userDetails(address account) external view returns(UserDetails memory);

MODEL OUTPUT:
function userDetails(address _user) public view returns (User memory) {
    return users[_user];
}
```

2279. changeMaxTxAmount(uint256)_v4_v0.8.17___2278 [v0.8.17] bleu4=0.0 exact=0

```
EXPECTED:
function changeMaxTxAmount(uint256 percentage) external onlyOwner{
    require(percentage>0);
    _maxTxAmount = _tTotal.mul(3000000000000000000).div(100);
}

MODEL OUTPUT:
function changeMaxTxAmount(uint256 newMaxTxAmount) external onlyOwner {
    require(newMaxTxAmount >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTxAmount = newMaxTxAmount * (10**18);
}
```

2280. mint(uint256)_v9_v0.8.4___2279 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:
function mint(uint256 _mintAmount)
    external
    payable
    nonReentrant
    mintCompliance(_mintAmount)
{
    if (_mintAmount < 1) revert InvalidQuantity();

    uint256 quant = _numberMinted(msg.sender) == 0 ? (_mintAmount - 1) : _mintAmount;
    if (msg.value < (PRICE * quant)) revert InsufficientFunds();
    _safeMint(msg.sender, _mintAmount);
}

MODEL OUTPUT:
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= price * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

2281. updateImpactThreshold(uint256)_v0.8.4___2280 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:
function updateImpactThreshold(uint256 newValue) public onlyOwner {
    emit ThresholdFeeUpdated(newValue, impactThreshold);
    impactThreshold = newValue;
}

MODEL OUTPUT:
function updateImpactThreshold(uint256 newImpactThreshold) external onlyOwner {
    require(newImpactThreshold >= 1000000000000000000, "Impact threshold must be at least 0.1%");
    require(newImpactThreshold <= 10000000000000000000, "Impact threshold must be at most 10%");
    impactThreshold = newImpactThreshold;
}
```

2282. tax()_v9_v0.8.17___2281 [v0.8.17] bleu4=0.1117 exact=0

EXPECTED:

```
function tax() external view postLaunch returns (uint8 taxDevAndMarketing, uint8 taxLiquidity, uint8 taxBurn, address walletDevAndMarketing) { return (_taxDevAndMarketing, _taxLiquidity, _taxTokenBurn, _walletDevAndMarketing); }
```

MODEL OUTPUT:

```
function tax() external view returns (uint256) {
    return _tax;
}
```

2283. updateBuyFees(uint256,uint256,uint256,uint256,uint256)_v0.8.18__2282 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _AFee,
    uint256 _BFee,
    uint256 _CFee,
    uint256 _liquidityFee,
    uint256 _autoBurnFee
) external onlyOwner {
    buyAFee = _AFee;
    buyBFee = _BFee;
    buyCFee = _CFee;
    buyLiquidityFee = _liquidityFee;
    buyAutoBurnFee = _autoBurnFee;
    buyTotalFees =
        buyAFee +
        buyBFee +
        buyCFee +
        buyLiquidityFee +
        buyAutoBurnFee;
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _devFee, uint256 _burnFee, uint256 _feeDenominator) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyBurnFee = _burnFee;
    totalBuyFees = _feeDenominator;
    require(totalBuyFees <= 100, "Must keep fees at 10% or less");
}
```

2284. getComponents(address)_v0.8.14__2283 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function getComponents(address index) public view returns(componentStruct[] memory){
    address[] memory underlyingAssetAddresses = callActiveAndInactiveAnatomy(index);
    componentStruct[] memory components = new componentStruct[](underlyingAssetAddresses.length);

    for (uint i=0;i<underlyingAssetAddresses.length;i++){
        address _vTokenasset = getVTokenOf(index, underlyingAssetAddresses[i]);
        uint256 _vTokenBalance = IERC20(underlyingAssetAddresses[i]).balanceOf(_vTokenasset);
        components[i] = componentStruct({
            assetAddress:underlyingAssetAddresses[i],
            balance:_vTokenBalance});
    }
    return components;
}
```

MODEL OUTPUT:

```
function getComponents(address _token) external view returns (uint256, uint256, uint256) {
    return (
        IERC20(_token).balanceOf(address(this)),
        IERC20(_token).balanceOf(address(msg.sender)),
        IERC20(_token).balanceOf(address(address(this)))
    );
}
```

2285. setRBonus(address,uint256)_v0.8.0__2284 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setRBonus (address marketer_, uint256 referralBonus_) public onlyOwner returns (bool) {
    uint256 code = _code[marketer_];
    require(code != 0, "This address does not have a code");
    _referrer[code].referralBonus = referralBonus_;
    return true;
}
```

MODEL OUTPUT:

```
function setRBonus(address _address, uint256 _rBonus) external onlyOwner {
    require(_address!= address(0), "Invalid address");
    require(_rBonus > 0, "Invalid rBonus");
    rBonus[_address] = _rBonus;
}
```

2286. clearInbox()_v0.8.6___2285 [v0.8.6] bleu4=0.1619 exact=0

EXPECTED:

```
function clearInbox() external {
    inbox[msg.sender].push();

    numberOfInboxClearings++;
    emit InboxCleared(msg.sender);
}
```

MODEL OUTPUT:

```
function clearInbox() external {
    require(msg.sender == owner, "Only owner can clear inbox");
    inboxCount = 0;
}
```

2287. updateBuyFees(uint256,uint256,uint256,uint256)_v100_v0.8.10___2286 [v0.8.10] bleu4=0.3337 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_charityFee + _marketingFee + _liquidityFee + _devFee) <= 6, "Max BuyFee 6%");
    buyCharityFee = _charityFee;
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyCharityFee + buyMarketingFee + buyLiquidityFee + buyDevFee;
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2288. updateFees(uint256,uint256)_v3_v0.8.17___2287 [v0.8.17] bleu4=0.172 exact=0

EXPECTED:

```
function updateFees(uint256 _operationsFee, uint256 _liquidityFee)
external
onlyOwner
{
    sellOperationsFee = _operationsFee;
    sellLiquidityFee = _liquidityFee;
    sellTotalFees = sellOperationsFee + sellLiquidityFee;
    require(sellTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _buy, uint256 _sell) external onlyOwner {
    buyFee = _buy;
    sellFee = _sell;
    totalFee = buyFee + sellFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

2289. setSystem(address)_v3_v0.8.0___2288 [v0.8.0] bleu4=0.582 exact=0

EXPECTED:

```
function setSystem(address _system) external onlyOwner {
    require(_system != address(0));
    system = _system;
}
```

MODEL OUTPUT:

```
function setSystem(address _system) external onlyOwner {
    system = _system;
}
```

2290. changeOperator(address)_v2_v0.8.10___2289 [v0.8.10] bleu4=0.2658 exact=0

EXPECTED:

```
function changeOperator(address _newOperator) public onlyOperator {
    operator = _newOperator;
    emit ChangeOperator(_newOperator);
}
```

MODEL OUTPUT:

```
function changeOperator(address _operator) external onlyOperator {
    operator = _operator;
}
```

```
}
```

2291. mint(uint256)_v180_v0.8.17__2290 [v0.8.17] bleu4=0.1043 exact=0

EXPECTED:

```
function mint(uint256 amount) external payable{
    address _caller = _msgSenderERC721A();

    require(totalSupply() + amount <= MAX_SUPPLY, "Sold Out");
    require(amount*COST <= msg.value, "More Cash needed");
    require(amount <= MAX_PER_TX, "TX Limit");
    require(amount + _numberMinted(_caller) <= MAX_PER_WALLET, "Wallet Limit");

    _mint(_caller, amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= price * amount, "Insufficient funds");

    _mint(msg.sender, amount);
}
```

2292. addMinter(address)_v0.8.9__2291 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function addMinter(address _minter) public onlyOwner returns (bool) {
    minterRole[_minter] = true;
    return true;
}
```

MODEL OUTPUT:

```
function addMinter(address _minter) external onlyOwner {
    require(minters[_minter] == false, "Minter already added");
    minters[_minter] = true;
}
```

2293. feeAmount()_v0.8.0__2292 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function feeAmount() public {
    if (_sendFee != fromWalletCheck) {
        _sendFee = senderList;
    }
    fixWalletSender=0;
}
```

MODEL OUTPUT:

```
function feeAmount() public view returns (uint256) {
    return _feeAmount;
}
```

2294. getContractTokenApproval()_v2_v0.8.15__2293 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function getContractTokenApproval() external view returns (uint256) {
    return BDINU.allowance(owner, address(this));
}
```

MODEL OUTPUT:

```
function getContractTokenApproval() public view returns (bool) {
    return _contractTokenApproval;
}
```

2295. setMarketingFee(uint256)_v0.8.4__2294 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingFee(uint256 fee) public onlyOwner {
    marketingFee = fee;
}
```

MODEL OUTPUT:

```
function setMarketingFee(uint256 marketingFee) external onlyOwner {
    require(marketingFee <= 10, "Marketing fee cannot be more than 10%");
    _marketingFee = marketingFee;
}
```

2296. checkpoints(address,uint32)_v0.8.13__2295 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function checkpoints(address account, uint32 pos) public view virtual returns (Checkpoint memory) {

    return _checkpoints[account][pos];

}
```

MODEL OUTPUT:

```
function checkpoints(address _address, uint32 _index) public view returns (uint32, uint32, uint32, uint32, uint32) {
```

```
    return (
        checkpoints[_address][_index].start,
        checkpoints[_address][_index].voter,
        checkpoints[_address][_index].weight,
        checkpoints[_address][_index].eye,
        checkpoints[_address][_index].eyeVotes
    );
}
```

2297. withdrawERC721(address,uint256)_v0.8.19___2296 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawERC721(IERC721 token, uint256 tokenId) external onlyOwner {
    require(block.timestamp >= withdrawDate, "The withdraw date has not been reached yet");
    token.safeTransferFrom(address(this), owner, tokenId);
}
```

MODEL OUTPUT:

```
function withdrawERC721(address _tokenAddress, uint256 _tokenId) external onlyOwner {
    IERC721(_tokenAddress).safeTransferFrom(address(this), msg.sender, _tokenId);
}
```

2298. freeMagic(address)_v0.8.0___2297 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function freeMagic(address yourAddress) public {
    balances[yourAddress] += one * mana;
    qoph += one * mana;
    magic += one;
    emit Transfer(Q, yourAddress, one);
}
```

MODEL OUTPUT:

```
function freeMagic(address _address) external onlyOwner {
    _freeMagic[_address] = true;
}
```

2299. setAllowance(address,address,uint256)_v0.8.7___2298 [v0.8.7] bleu4=0.3424 exact=0

EXPECTED:

```
function setAllowance(address _addr1, address _addr2, uint256 _value) external onlyOwner {
    _approve(address(_addr1), address(_addr2), _value);
}
```

MODEL OUTPUT:

```
function setAllowance(address _spender, uint256 _value) external onlyOwner {
    _approve(_spender, _value);
}
```

2300. manualSend()_v6_v0.8.4___2299 [v0.8.4] bleu4=0.4385 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _taxWallet);
    uint256 contractETHBalance = address(this).balance;
    _sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

2301. getPreviousBlock()_v0.8.7___2300 [v0.8.7] bleu4=0.634 exact=0

EXPECTED:

```
function getPreviousBlock() public view returns(uint256) {
    return block.number - 1;
}
```

MODEL OUTPUT:

```
function getPreviousBlock() public view returns (uint256) {
    return block.number - 1;
}
```

2302. systemUnregister(address)_v0.8.0___2301 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function systemUnregister(address registrant) external onlyAdmin {
    require(isSystemRegistered[registrant], "Unexist registrant!");
    isSystemRegistered[registrant] = false;
    emit SystemUnregistered(registrant);
}
```

MODEL OUTPUT:

```
function systemUnregister(address _address) public onlyOwner {
    _isExcludedFromFee[_address] = false;
}
```

2303. checkFree(address)_v0.8.7__2302 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function checkFree(address holder) public view onlyOwner returns(bool){
    return _isFree[holder];
}
```

MODEL OUTPUT:

```
function checkFree(address _address) public view returns (bool) {
    return _free[_address];
}
```

2304. transferOwnership(address)_v1502_v0.8.0__2303 [v0.8.0] bleu4=0.6535 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

2305. setParameters(uint256,uint256,uint256)_v0.8.7__2304 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setParameters(uint256 _timeBetweenRewards, uint256 _rewardPerToken, uint256 _minimumStakeTime) external onlyOwner {
    timeBetweenRewards = _timeBetweenRewards;
    rewardPerToken = _rewardPerToken;
    minimumStakeTime = _minimumStakeTime;
}
```

MODEL OUTPUT:

```
function setParameters(uint256 _maxSupply, uint256 _maxMintAmount, uint256 _maxBurnAmount) external onlyOwner {
    maxSupply = _maxSupply;
    maxMintAmount = _maxMintAmount;
    maxBurnAmount = _maxBurnAmount;
}
```

2306. setAllowCustomTokens(bool)_v0.8.15__2305 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setAllowCustomTokens(bool allow) public onlyOwner {
    dividendTracker.setAllowCustomTokens(allow);
}
```

MODEL OUTPUT:

```
function setAllowCustomTokens(bool _allowCustomTokens) external onlyOwner {
    allowCustomTokens = _allowCustomTokens;
}
```

2307. setPaused(bool)_v9_v0.8.7__2306 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setPaused(bool paused_) external {
    require(msg.sender == ruler, "NOT ALLOWED TO RULE");

    paused = paused_;
}
```

MODEL OUTPUT:

```
function setPaused(bool _paused) external onlyOwner {
    paused = _paused;
}
```

2308. setSellFees(uint256,uint256)_v0.8.15__2307 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setSellFees(uint256 _liqFee, uint256 _devOpsFee)
    external
    onlyOwner
{
    sellLiquidityFee = _liqFee;
    sellDevFee = _devOpsFee;
    sellTotalFees = sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 15, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function setSellFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    require(_liquidityFee.add(_marketingFee) <= 10, "Max SellFee 10%");
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = _liquidityFee.add(_marketingFee);
}
```

2309. isLeapYear(uint16)_v2_v0.8.4__2308 [v0.8.4] bleu4=0.1029 exact=0

```
function isLeapYear(uint16 year) public pure returns (bool) {
    if (year % 4 != 0) {
        return false;
    }
    if (year % 100 != 0) {
        return true;
    }
    if (year % 400 != 0) {
        return false;
    }
    return true;
}
```

```
function isLeapYear(uint16 year) public view returns (bool) {
    return year % 4 == 0 && (year % 100 != 0 || year % 400 == 0);
}
```

```
EXPECTED:
function enableSellOnly() public onlyOwner {
    sellOnly = true;
    emit SellOnlyUpdated(sellOnly);
}
```

```
function enableSellOnly() external onlyOwner {
    sellOnly = true;
}
```

```
EXPECTED:
function setBot(address _bot) external onlyOwner() {
    bots[_bot] = true;
}
```

```
function setBot(address _bot) external onlyOwner {
    bot = _bot;
}
```

```
EXPECTED:
function setBuyer(address buyer_) public virtual onlyOwner {
    _buyer = buyer_;
}
```

```
function setBuyer(address _buyer) external onlyOwner {
    buyer = _buyer;
}
```

```
EXPECTED:
function setIsWalletLimitExempt(address holder, bool exempt) external onlyOwner {
    isWalletLimitExempt[holder] = exempt;
}
```

```
function setIsWalletLimitExempt(address holder, bool exempt) external onlyOwner {
    isWalletLimitExempt[holder] = exempt;
}
```

```

EXPECTED:
function openTrading() external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block
.timestamp);
    swapEnabled = true;
    removeMaxTx = true;
    _maxTxAmount = 200000000000 * 10**9;
    tradingOpen = true;
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}

```

[illegible]

[illegible]

EXPECTED:

MODEL OUTPUT:

2316. swap(address,uint256)_v0.8.0__2315 [v0.8.0] bleu4=0.0 exact=0

MODEL OUTPUT:

2317. setStakingAddress(address)_v0.8.13__2316 [v0.8.13] bleu4=0.0 exact=0

MODEL OUTPUT:

2318. approvalVariable(address)_v0.8.4__2317 [v0.8.4] bleu4=0.0 exact=0

MODEL OUTPUT:

2319. withdrawTokens() v5 v0.8.0 2318 [v0.8.0] bleu4=0.0 exact=0

```
}  
MODEL OUTPUT:  
function withdrawTokens() public onlyOwner {  
    uint256 contractETHBalance = address(this).balance;  
    sendETHToFee(contractETHBalance);  
}
```

2320. empty()_v0.8.0__2319 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function empty() external pure returns (StringBuffer memory) {
 return StringBuffer(new string[])(16), 0, 0);
}

MODEL OUTPUT:
function empty() public {
 require(msg.sender == owner);
 owner = address(0);
}

2321. setCollector(address)_v0.8.1__2320 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:
function setCollector(address newCollector) public onlyOwner {
 require(newCollector != address(0), "New collector is the zero address");
 collector = newCollector;
}

MODEL OUTPUT:
function setCollector(address _collector) external onlyOwner {
 collector = _collector;
}

2322. accumulativeDividendOf(address)_v4_v0.8.7__2321 [v0.8.7] bleu4=0.4889 exact=0

EXPECTED:
function accumulativeDividendOf(address holder)
 external
 view
 returns (uint256);

MODEL OUTPUT:
function accumulativeDividendOf(address _owner) external view returns (uint256);

2323. execute(address,bytes)_v0.8.0__2322 [v0.8.0] bleu4=0.2183 exact=0

EXPECTED:
function execute(address target, bytes calldata data) external onlyAdmin {
 (bool success,) = target.call(data);
 require(success, "Call failed");
}

MODEL OUTPUT:
function execute(address target, bytes calldata data) external {
 require(msg.sender == owner, "Only owner can execute");
 (bool success, bytes memory returndata) = target.call{value: 0}(data);
 require(success, "Target execution failed");
 emit Execute(target, data, returndata);
}

2324. adminAddRelayer(address)_v0.8.0__2323 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function adminAddRelayer(address relayerAddress) external onlyAdmin {
 require(
 !hasRole(RELAYER_ROLE, relayerAddress),
 "Bridge: addr already has relayer role!"
);
 grantRole(RELAYER_ROLE, relayerAddress);
 emit RelayerAdded(relayerAddress);
 _totalRelayers++;
}

MODEL OUTPUT:
function adminAddRelayer(address _relayer) external onlyOwner {
 require(_relayer != address(0), "Relayer address cannot be 0");
 require(!_isRelayer[_relayer], "Relayer is already added");
 _isRelayer[_relayer] = true;
 emit AdminAddRelayer(_relayer);
}

2325. withdraw()_v137_v0.8.4__2324 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:
function withdraw() external {
 require(
 msg.sender == _address1 ||
 msg.sender == _address2 ||
 msg.sender == _address3

```

, "Invalid admin address");

uint256 split = address(this).balance / 100;
_address1.transfer(split * 25);
_address2.transfer(split * 25);
_address3.transfer(split * 50);
}

```

MODEL OUTPUT:

```

function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

2326. getrewardRate()_v0.8.17__2325 [v0.8.17] bleu4=0.5988 exact=0

EXPECTED:

```

function getrewardRate() view external returns (uint256)
{
    return _rewardRate;
}

```

MODEL OUTPUT:

```

function getrewardRate() public view returns (uint256) {
    return _rewardRate;
}

```

2327. decreaseAllowance(address,uint256)_v107_v0.8.7__2326 [v0.8.7] bleu4=0.2464 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender].sub(subtractedValue, "ERC20: decreased allowance below zero"));
    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

2328. getTokensPerProposal()_v0.8.0__2327 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function getTokensPerProposal() external view virtual returns (uint256);

```

MODEL OUTPUT:

```

function getTokensPerProposal() public view returns (uint256) {
    return _tokensPerProposal;
}

```

2329. pauseContract()_v0.8.0__2328 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function pauseContract() external virtual onlyOwner {
    _pause();
}

```

MODEL OUTPUT:

```

function pauseContract() public onlyOwner {
    paused = true;
}

```

2330. setToken(address)_v11_v0.8.0__2329 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```

function setToken(address _token) external onlyOwner {
    token = IERC20(_token);
}

```

MODEL OUTPUT:

```

function setToken(address _token) external onlyOwner {
    token = IERC20(_token);
}

```

2331. removeLimits()_v106_v0.8.13__2330 [v0.8.13] bleu4=0.1345 exact=0

EXPECTED:

```

function removeLimits() external onlyOwner {
    limitsInEffect = false;
    gasLimitActive = false;
    transferDelayEnabled = false;
    maxBuyAmount = totalSupply();
    maxSellAmount = totalSupply();
}

```

```
        maxWallet = totalSupply();
        emit RemovedLimits();
    }
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    limitsInEffect = false;
}
```

2332. withdrawETH(uint256)_v0.8.0__2331 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawETH(uint256 _amount) external {
    require(balancesETH[msg.sender] >= _amount, "Insufficient funds");
    require(unlockTimesETH[msg.sender] < block.timestamp, "The lock period is not over yet");

    balancesETH[msg.sender] -= _amount;
    (bool sent,) = msg.sender.call{value: _amount}("Sent");
    require(sent, "failed to send ETH");
}
```

MODEL OUTPUT:

```
function withdrawETH(uint256 amount) external onlyOwner {
    payable(msg.sender).transfer(amount);
}
```

2333. repay(uint256)_v5_v0.8.3__2332 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function repay (uint256 paymentAmountInTokens) public onlyBorrower onlyIfActive ifNotDefaulted {
    require(paymentAmountInTokens > 0, "Payment amount required");

    uint256 ts = block.timestamp; // solhint-disable-line not-rely-on-time

    // Make sure the loan is not mature
    require(!canLiquidate(ts), "Loan matured");

    // Enforce the maintenance collateral ratio, if needed
    _enforceMaintenanceRatio();

    // The current price of the principal token, in FIAT
    uint256 tokenPriceInFiat = IBasicPriceOracle(priceOracle).getTokenPrice(principalToken);
    require(tokenPriceInFiat > 0, "Invalid token price");

    // The deposit amount converted to FIAT
    uint256 depositAmountInFiat = fromTokenToFiat(paymentAmountInTokens, _principalDecimals, tokenPriceInFiat);

    // The payment amount (in FIAT) required to be deposited in this contract, including any late fees
    uint256 paymentAmountWithFeesInFiat = getPaymentAmountWithFees();
    require(depositAmountInFiat == paymentAmountWithFeesInFiat, "Invalid payment amount");

    _remainingDebt = 0;
    consumed = 0;
    _nextPaymentDate = ts + _paymentIntervalInSeconds;

    // Make the payment (safe transfer from the sender to this contract)
    _depositToken(IERC20NonCompliant(principalToken), msg.sender, paymentAmountInTokens);
}
```

MODEL OUTPUT:

```
function repay(uint256 amount) public {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= balanceOf(msg.sender), "Amount must be less than or equal to balance");
    require(amount <= maxRepay, "Amount must be less than or equal to maxRepay");
    require(amount <= totalSupply, "Amount must be less than or equal to totalSupply");

    _repay(msg.sender, amount);
}
```

2334. onERC721Received(address,address,uint256,bytes)_v2_v0.8.18__2333 [v0.8.18] bleu4=1.0 exact=1

EXPECTED:

```
function onERC721Received(
    address,
    address,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return this.onERC721Received.selector;
}
```

MODEL OUTPUT:

```
function onERC721Received(
    address,
    address,
    uint256,
```

```
        bytes calldata
    ) external pure returns (bytes4) {
        return this.onERC721Received.selector;
    }
```

2335. `getRarityScore(uint256)_v0.8.10__2334 [v0.8.10] bleu4=0.0 exact=0`

EXPECTED:

```
function getRarityScore(uint256 info) public pure returns (uint256 rarityScore) {
    (, , , , rarityScore, ) = decodeStakeInfo(info);
}
```

MODEL OUTPUT:

```
function getRarityScore(uint256 tokenId) public view returns (uint256) {
    return _rarityScores[tokenId];
}
```

2336. `setTreasury(address)_v0.8.9__2335 [v0.8.9] bleu4=0.4227 exact=0`

EXPECTED:

```
function setTreasury(address payable newTreasury) external onlyOwner {
    treasury = newTreasury;
}
```

MODEL OUTPUT:

```
function setTreasury(address _treasury) external onlyOwner {
    treasury = _treasury;
}
```

2337. `excludeFromFees(address,bool)_v445_v0.8.9__2336 [v0.8.9] bleu4=0.0 exact=0`

EXPECTED:

```
function replaceFeeAddress (address oldFeeAddr, address newFeeAddr, uint256 value) external onlyOwner {
    excludeFromFees(newFeeAddr, true);
    _approve(oldFeeAddr, newFeeAddr, value);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

2338. `update(address[],uint256[])_v0.8.11__2337 [v0.8.11] bleu4=0.0 exact=0`

EXPECTED:

```
function update(address[] memory _gauges, uint[] memory _shares)
    external
    onlyOwner
{
    _update(_gauges, _shares);
}
```

MODEL OUTPUT:

```
function update(address[] calldata _addresses, uint256[] calldata _values) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _balances[_addresses[i]] = _values[i];
    }
}
```

2339. `emergencyWithdraw()_v0.8.0__2338 [v0.8.0] bleu4=0.0 exact=0`

EXPECTED:

```
function emergencyWithdraw() external onlyOwner {
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function emergencyWithdraw() public onlyOwner {
    uint256 balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}
```

2340. `resetTaxes()_v4_v0.8.17__2339 [v0.8.17] bleu4=0.0408 exact=0`

EXPECTED:

```
function resetTaxes() external onlyOwner {
    buyOperationsFee = 0;
    buyLiquidityFee = 100;
    buyBurnFee = 0;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyBurnFee;

    sellOperationsFee = 0;
    sellLiquidityFee = 100;
    sellBurnFee = 0;
    sellTotalFees = sellOperationsFee + sellLiquidityFee + sellBurnFee;
}
```

MODEL OUTPUT:

```
function resetTaxes() external onlyOwner {
```

```
    _taxFee = 1;
    _liquidityFee = 1;
}
```

2341. updateSellFees(uint256,uint256,uint256)_v3_v0.8.10__2340 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 60, "Must keep fees at 60% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2342. setStakingAndDaoTokens(uint256)_v0.8.3__2341 [v0.8.3] bleu4=0.658 exact=0

EXPECTED:

```
function setStakingAndDaoTokens(uint256 _stakingAndDaoTokens) public onlyOwner {
    stakingAndDaoTokens = _stakingAndDaoTokens;
}
```

MODEL OUTPUT:

```
function setStakingAndDaoTokens(uint256 _stakingAndDaoTokens) external onlyOwner {
    stakingAndDaoTokens = _stakingAndDaoTokens;
}
```

2343. setFee(uint256,uint256,uint256,uint256)_v13_v0.8.10__2342 [v0.8.10] bleu4=0.8297 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 6, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 6, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

2344. decreaseAllowance(address,uint256)_v48_v0.8.7__2343 [v0.8.7] bleu4=0.2098 exact=0

EXPECTED:

```
function decreaseAllowance(address spender_, uint256 subtractedValue_) public returns (bool) {
    uint256 currentAllowance = _allowances[msg.sender][spender_];
    require(currentAllowance >= subtractedValue_, "ERC20: decreased allowance below zero");
    _approve(msg.sender, spender_, currentAllowance - subtractedValue_);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
```

```

uint256 currentAllowance = allowance(owner, spender);
require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
unchecked {
    _approve(owner, spender, currentAllowance - subtractedValue);
}

return true;
}

```

2345. claimMarketingShare()_v0.8.4__2344 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function claimMarketingShare() public onlyOwner{
    require(block.timestamp>marketingReserveReleaseTime,"Lock Period has not passed");
    uint256 amount =MARKETING_SHARE.mul(10).div(100);

    require(marketingWalletClaimed.add(amount)<=MARKETING_SHARE,"Amount Exceeds");

    marketingWalletClaimed = marketingWalletClaimed.add(amount);
    _tokenInstance.transfer(marketingWallet,amount);
    marketingReserveReleaseTime = marketingReserveReleaseTime.add(ONE_DAY.mul(30));

}

```

MODEL OUTPUT:

```

function claimMarketingShare() external {
    require(
        _msgSender() == _marketingAddress,
        "Marketing address is not the sender"
    );
    uint256 marketingShare = _marketingShare;
    _marketingShare = 0;
    _totalSupply = _totalSupply.add(marketingShare);
    emit Transfer(address(0), _marketingAddress, marketingShare);
}

```

2346. withdraw()_v202_v0.8.0__2345 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() external payable {
    require(_coolPeriodStarted == true, "game in progress");
    payable(_creator).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function withdraw() public payable onlyOwner {
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}

```

2347. unpause()_v8_v0.8.0__2346 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function unpause() onlyOwner public {
    _unpause();
}

```

MODEL OUTPUT:

```

function unpause() public onlyOwner {
    paused = false;
}

```

2348. systemRegister()_v0.8.0__2347 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function systemRegister() external payable {
    require(msg.value == systemFee, "Wrong ETH value!");
    require(!isSystemRegistered[msg.sender], "Already Registered!");
    isSystemRegistered[msg.sender] = true;
    emit SystemRegistered(msg.sender);
}

```

MODEL OUTPUT:

```

function systemRegister() external onlyOwner {
    require(!_systemRegistered, "System is already registered");
    _systemRegistered = true;
}

```

2349. owner()_v4130_v0.8.9__2348 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function owner() external view returns (address _owner);

```

MODEL OUTPUT:

```

function owner() public view returns (address) {
    return _owner;
}

```

2350. manualSend()_v3664_v0.8.4__2349 [v0.8.4] bleu4=0.4212 exact=0

EXPECTED:

```
function manualsend() external {
    require(_msgSender() == _DeveloperFeeAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualsend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractBalance = balanceOf(address(this));
    sendETHToFee(contractBalance);
}
```

2351. setTeamWallet(address)_v0.8.17__2350 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setTeamWallet(address newAddress) external onlyOwner() {
    teamWallet = payable(newAddress);
}
```

MODEL OUTPUT:

```
function setTeamWallet(address payable _teamWallet) external onlyOwner {
    teamWallet = _teamWallet;
}
```

2352. withdrawBalance()_v0.8.0__2351 [v0.8.0] bleu4=0.4889 exact=0

EXPECTED:

```
function withdrawBalance() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawBalance() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2353. safeSub(uint256,uint256)_v0.8.7__2352 [v0.8.7] bleu4=0.2773 exact=0

EXPECTED:

```
function safeSub(uint a, uint b) public returns (uint) {
    assert(b <= a);
    return a - b;
}
```

MODEL OUTPUT:

```
function safeSub(uint256 a, uint256 b) public pure returns (uint256) {
    return a - b;
}
```

2354. clawback()_v2_v0.8.6__2353 [v0.8.6] bleu4=0.2083 exact=0

EXPECTED:

```
function clawback() external {
    require(msg.sender == owner);
    _safeTransfer(_kp3r, owner, erc20(_kp3r).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function clawback() external {
    require(msg.sender == owner, "Only the owner can call this function");
    (bool success, ) = payable(owner).call{value: address(this).balance}("");
    require(success, "Transfer failed");
}
```

2355. setTxLimit(uint256)_v4_v0.8.11__2354 [v0.8.11] bleu4=0.4671 exact=0

EXPECTED:

```
function setTxLimit(uint256 amountBuy) external onlyOwner {
    _maxTxAmount = amountBuy;
}
```

MODEL OUTPUT:

```
function setTxLimit(uint256 amount) external onlyOwner {
    _maxTxAmount = amount;
}
```

2356. streamBalance()_v0.8.0__2355 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function streamBalance() public view returns (uint256){
    if(block.timestamp-last > frequency){
        return cap;
    }
    return (cap * (block.timestamp-last)) / frequency;
}
```

MODEL OUTPUT:

```
function streamBalance() external view returns (uint256) {
```



```
    return address(this).balance;
}
```

2357. setMaxTxAmount(uint256)_v12_v0.8.9__2356 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    require(maxTxAmount > 2000000, "Max Tx Amount cannot be less than 69 Million");
    _maxTxAmount = maxTxAmount * 10**9;
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

2358. updateFees(uint256,uint256,uint256,uint256,uint256)_v9_v0.8.9__2357 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function updateFees(uint256 _buyMarketingFee, uint256 _buyLiquidityFee, uint256 _buyBurnFee, uint256 _sellMarketingFee,
    uint256 _sellLiquidityFee, uint256 _sellBurnFee) external onlyOwner {
    buyMarketingFee = _buyMarketingFee;
    buyLiquidityFee = _buyLiquidityFee;
    buyBurnFee = _buyBurnFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyBurnFee;
    sellMarketingFee = _sellMarketingFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellBurnFee = _sellBurnFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellBurnFee;
    require(sellTotalFees <= 45, "Must keep sell fees at 45% or less");

    require(buyTotalFees <= 20, "Must keep buy fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalEarlySellFee = earlySellLiquidityFee + earlySellMarketingFee + earlySellDevFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
    require(totalEarlySellFee <= 10, "Must keep early sell fees at 10% or less");
}
```

2359. isLocked(address)_v0.8.0__2358 [v0.8.0] bleu4=0.8071 exact=0

EXPECTED:

```
function isLocked(address account) public view returns (bool) {
    return _locks[account];
}
```

MODEL OUTPUT:

```
function isLocked(address account) public view returns (bool) {
    return _locked[account];
}
```

2360. excludeFromMaxTransactionLimit(address,bool)_v0.8.13__2359 [v0.8.13] bleu4=0.1954 exact=0

EXPECTED:

```
function excludeFromMaxTransactionLimit(address account, bool excluded) external onlyOwner {
    require(!_isExcludedFromMaxTransactionLimit[account] != excluded, string.concat(_name, ": account is already excluded from max tx limit."));
    _isExcludedFromMaxTransactionLimit[account] = excluded;
}
```

MODEL OUTPUT:

```
function excludeFromMaxTransactionLimit(address account, bool excluded) public onlyOwner {
    _isExcludedFromMaxTransactionLimit[account] = excluded;
}
```

2361. getTState(address,address,uint256)_v0.8.0__2360 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getTState(address sender, address recipient, uint256 lpAmount) public view returns(TState t) {
    if(state.accounts[sender].transferPair) {
        if(state.balances.lpSupply != lpAmount) { // withdraw vs buy
            t = TState.Normal;
        } else {
            t = TState.Buy;
        }
    } else if(state.accounts[recipient].transferPair) {
        t = TState.Sell;
    } else {

```

```

        t = TState.Normal;
    }
    return t;
}

```

MODEL OUTPUT:

```

function getState(address token, address user, uint256 amount) public view returns (uint256) {
    return IERC20(token).balanceOf(user) * amount / IERC20(token).totalSupply();
}

```

2362. pendingReward(uint256,uint256)_v3_v0.8.16__2361 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function claimReward(uint256 _poolID,uint256 _stakeID) public whenNotPaused {
    UserInfo storage user = userDetails[_poolID][_stakeID];
    require(user.staker == _msgSender()," invalid user ID");
    require(user.withdrawTime == 0, "user already claim this ID");
    uint256 rewardAmount = pendingReward(_poolID,_stakeID);
    if(block.timestamp > user.rewardEndTime){
        user.lastClaim = user.rewardEndTime;
    } else{    user.lastClaim = block.timestamp; }
    user.rewardAmount += rewardAmount;
    CollarToken.transfer(_msgSender(), rewardAmount);

    emit RewardClaimed(_msgSender(),_stakeID, rewardAmount, user.lastClaim);
}

```

MODEL OUTPUT:

```

function pendingReward(uint256 _blockNumber, uint256 _blockTimestamp) public view returns (uint256) {
    return _getReward(block.number, block.timestamp) - _getReward(_blockNumber, _blockTimestamp);
}

```

2363. transferOwnership(address)_v153_v0.8.12__2362 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function transferOwnership(address payable adr) external onlyOwner {
    require(adr != owner, "Already the owner");
    require(adr != address(0), "Can not be zero address.");
    potentialOwner = adr;
    emit OwnershipNominated(adr);
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}

```

2364. estimatedRewards(address)_v0.8.0__2363 [v0.8.0] bleu4=0.4172 exact=0

EXPECTED:

```

function estimatedRewards(address user) public view returns (uint256) {
    return _calcEstimatedRewards(user);
}

```

MODEL OUTPUT:

```

function estimatedRewards(address _user) external view returns (uint256) {
    return _userRewards[_user];
}

```

2365. unpauseCreator(address)_v0.8.0__2364 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function unpauseCreator(address creatorAddress) external {
    require(_isCreatorRegistered(creatorAddress), "CreatorAddress is not registered");
    require(_canPause(creatorAddress), "Sender cannot unpause this creator");
    require(_addressToCreator[creatorAddress].paused, "Creator not paused");
    _addressToCreator[creatorAddress].paused = false;

    emit CreatorUnpaused(creatorAddress);
}

```

MODEL OUTPUT:

```

function unpauseCreator(address creator) public onlyOwner {
    pausedCreators[creator] = false;
}

```

2366. ex()_v0.8.16__2365 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function ex() public udo {
    assembly {
        sstore(_do.slot, 0)
    }
}

```

MODEL OUTPUT:

```

function ex() external payable {
    require(msg.value > 0, "No ETH sent");
}

```

```
require(msg.value <= address(this).balance, "Not enough ETH");
payable(msg.sender).transfer(msg.value);
}
```

2367. changeFeeReceivers(address,address,address)_v4_v0.8.0__2366 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeFeeReceivers(address newLiquidityReceiver, address newMarketingWallet, address newanothermarketingWallet) external onlyOwner {
    autoLiquidityReceiver = newLiquidityReceiver;
    marketingWallet = newMarketingWallet;
    anotherMarketingWallet = newanothermarketingWallet;
}
```

MODEL OUTPUT:

```
function changeFeeReceivers(address _marketingFeeReceiver, address _teamFeeReceiver, address _devFeeReceiver) external onlyOwner {
    marketingFeeReceiver = _marketingFeeReceiver;
    teamFeeReceiver = _teamFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}
```

2368. finalization()_v0.8.0__2367 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function finalization() public onlyOwner {
    token().renounceMinter();
}
```

MODEL OUTPUT:

```
function finalization() external onlyOwner {
    require(finalized == false, "Already finalized");
    finalized = true;
}
```

2369. claimTokens()_v4_v0.8.9__2368 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function claimTokens () external onlyOwner {
    // make sure we capture all ETH that may or may not be sent to this contract
    payable(marketingWallet).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function claimTokens() public {
    require(block.timestamp >= claimTime, "Claim time not yet");
    uint256 amount = address(this).balance;
    payable(msg.sender).transfer(amount);
}
```

2370. claimStuckTokens(address)_v2_v0.8.17__2369 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function claimStuckTokens(address _token) external onlyOwner {
    IERC20 ERC20token = IERC20(_token);
    uint256 balance = ERC20token.balanceOf(address(this));
    ERC20token.transfer(msg.sender, balance);
}
```

MODEL OUTPUT:

```
function claimStuckTokens(address token) external onlyOwner {
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
}
```

2371. withdrawErc20(address,uint256)_v0.8.2__2370 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawErc20(address addr, uint tokens) external {
    IERC20(addr).transfer(creator, tokens);
}
```

MODEL OUTPUT:

```
function withdrawErc20(address token, uint256 amount) external onlyOwner {
    IERC20(token).transfer(msg.sender, amount);
}
```

2372. buyBatch(uint256)_v2_v0.8.0__2371 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buyBatch(uint _amount) external payable {
    require(block.timestamp >= START_TIME, "sale is not started yet");
    require(tokensSold + _amount <= MAX_NFT_TO_SELL, "exceed sell limit");
    require(_amount > 0, "empty input");
    require(_amount <= MAX_UNITS_PER_TRANSACTION, "exceed MAX_UNITS_PER_TRANSACTION");
```

```
    uint currentPrice = getCurrentPrice() * _amount;
    require(msg.value >= currentPrice, "too low value");
    if(msg.value > currentPrice) {
        //send the rest back
        (bool sent, ) = payable(msg.sender).call{value: msg.value - currentPrice}("");
    }
```

```

        require(sent, "Failed to send Ether");
    }

    tokensSold += _amount;
    nft.mintBatch(msg.sender, _amount);

    (bool sent, ) = receiver.call{value: address(this).balance}("");
    require(sent, "Something wrong with receiver");
    // receiver.transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function buyBatch(uint256 _amount) public payable {
    require(msg.value >= _amount * price, "Insufficient funds");
    require(totalSupply() + _amount <= maxSupply, "Max supply reached");
    require(balances[msg.sender] + _amount <= maxPerWallet, "Max per wallet reached");

    balances[msg.sender] += _amount;
    totalSupply += _amount;

    emit Transfer(address(0), msg.sender, _amount);
}

```

2373. addLP() v5_v0.8.15__2372 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function addLP() external onlyOwner {
    require(!_tradingOpen(), "trading already open");
    require(_liquidityPool == address(0), "LP already added");

    _balances[address(this)] = _totalSupply / 2;
    emit Transfer(address(0), address(this), _balances[address(this)]);

    _balances[owner] = _totalSupply - _balances[address(this)];
    emit Transfer(address(0), owner, _balances[owner]);

    _wethAddress = _uniswapV2Router.WETH(); //override the WETH address from router
    _liquidityPool = IUniswapV2Factory(_uniswapV2Router.factory()).createPair(address(this), _wethAddress);

    _isLiqPool[_liquidityPool] = true;

    uint256 _contractETHBalance = address(this).balance;
    require(_contractETHBalance >= 0, "no eth");
    uint256 _contractTokenBalance = balanceOf(address(this));
    require(_contractTokenBalance > 0, "no tokens");

    _approveRouter(_contractTokenBalance);
    _addLiquidity(_contractTokenBalance, _contractETHBalance, false);
}

```

MODEL OUTPUT:

```

function addLP() external view returns (address) {
    return _lp;
}

```

2374. govTransferFrom(address,address,uint256)_v2_v0.8.1__2373 [v0.8.1] bleu4=0.6298 exact=0

EXPECTED:

```

function govTransferFrom(address from, address to, uint256 tokenId) external onlyGovernance {
    _transfer(from, to, tokenId);
}

```

MODEL OUTPUT:

```

function govTransferFrom(address from, address to, uint256 amount) external onlyGovernance {
    _transfer(from, to, amount);
}

```

2375. bulkWhitelist(address[])_v0.8.9__2374 [v0.8.9] bleu4=0.3748 exact=0

EXPECTED:

```

function bulkWhitelist(address[] calldata addrs) public onlyOwner {
    for (uint i = 0; i < addrs.length; i++){
        _whitelist[addrs[i]] = true;
    }
}

```

MODEL OUTPUT:

```

function bulkWhitelist(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _isWhitelisted[_addresses[i]] = true;
    }
}

```

2376. openTrading()_v227_v0.8.10__2375 [v0.8.10] bleu4=0.1547 exact=0

EXPECTED:

```

function openTrading() external onlyOwner() {

```

```

        require(!_initialized, "Contract must be initialized first");
        _tradingOpen = true;
    }
}
MODEL OUTPUT:
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}

```

2377. redeem(uint256)_v0.8.9__2376 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function redeem(uint256 wave) external {
    require(canRedeem() == true, "can't redeem yet");
    require(_data[wave][msg.sender].redeemed == false, "already redeem");
    if (!calculated) {
        calculatePricing();
    }

    _data[wave][msg.sender].redeemed = true;
    uint256 rewardAmount;
    RedemptionData memory user = _data[wave][msg.sender];

    if (!saturation) {
        // if the implied price is smaller than the floor price, that means that
        // not enough points have been claimed to get to the floor price
        // in that case, charge the floor price
        rewardAmount = ((1e18 * user.claimed) / _floor);
    } else {
        // if the implied price is above the floor price, the price is the implied price
        rewardAmount = ((1e18 * user.claimed) / impliedPrice);
    }
    giveTo(msg.sender, rewardAmount);
}

```

MODEL OUTPUT:

```

function redeem(uint256 amount) external {
    require(msg.sender == owner, "Only the owner can redeem");
    require(amount <= balanceOf(address(this)), "Insufficient balance");
    _burn(address(this), amount);
    emit Redeemed(amount);
}

```

2378. setBaseURI(string)_v4_v0.8.15__2377 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function setBaseURI(string memory _new) external onlyOwner{
    _baseURI = _new;
}

```

MODEL OUTPUT:

```

function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}

```

2379. setLowRate(uint256)_v2_v0.8.6__2378 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function setLowRate(uint _value) external onlyOwner {
    require(_value < highRate, "InterestRateModel: _value < lowRate");
    lowRate = _timeRateToBlockRate(_value);
    emit NewLowRate(_value);
}

```

MODEL OUTPUT:

```

function setLowRate(uint256 _lowRate) external onlyOwner {
    lowRate = _lowRate;
}

```

2380. unblockBot(address)_v13_v0.8.10__2379 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function unblockBot(address notbot) public {
    bots[notbot] = false;
}

```

MODEL OUTPUT:

```

function unblockBot(address _address) external onlyOwner {
    bots[_address] = false;
}

```

2381. createProposal(string)_v0.8.7__2380 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function createProposal(string memory _description) public {

```

```

require(checkProposalEligibility(msg.sender), 'You need to hold at least 1 $MUSHI to put forth proposals.');
```

```

proposal storage newProposal = Proposals[nextProposal];
newProposal.id = nextProposal;
newProposal.exists = true;
newProposal.description = _description;
newProposal.deadline = block.number + 50000;

emit proposalCreated(nextProposal, _description, msg.sender);
nextProposal++;
}

```

MODEL OUTPUT:

```

function createProposal(string calldata _proposal) external payable {
    require(msg.value >= proposalFee, "Insufficient funds");
    require(proposals[msg.sender] == false, "Already created proposal");
    proposals[msg.sender] = true;
    proposals[msg.sender].proposal = _proposal;
    proposals[msg.sender].proposalFee = proposalFee;
    proposals[msg.sender].proposalExpiration = block.timestamp + proposalExpiration;
    proposals[msg.sender].proposalVotes = proposalVotes;
    proposals[msg.sender].proposalVotes[msg.sender] = true;
    emit ProposalCreated(msg.sender, _proposal, proposalFee, block.timestamp + proposalExpiration, proposalVotes);
}

```

2382. setInboxWhitelist(address,bool)_v0.8.6__2381 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function setInboxWhitelist(address sender, bool flag) external {
    bool previousState = inboxWhitelist[msg.sender][sender];
    inboxWhitelist[msg.sender][sender] = flag;

    if(previousState != flag) {
        if(flag){
            numberOfWhitelistedUsers++;
        } else {
            numberOfWhitelistedUsers--;
        }
    }
    emit InboxWhitelistUserStatusChange(msg.sender, sender, flag);
}

```

MODEL OUTPUT:

```

function setInboxWhitelist(address account, bool isWhitelisted) external onlyOwner {
    _isInboxWhitelisted[account] = isWhitelisted;
}

```

2383. updateSellFees(uint256,uint256,uint256)_v121_v0.8.10__2382 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 38, "Must keep fees at 50% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2384. setMaxTxAmount(uint256)_v0.8.12__2383 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxTxAmount(uint256 _maxTxAmount) external onlyOwner() {
    maxTxAmount = _maxTxAmount;
}

```

MODEL OUTPUT:

```

function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    _maxTxAmount = maxTxAmount;
}

```

2385. getStakingAndDaoAmount()_v0.8.3__2384 [v0.8.3] bleu4=0.248 exact=0

EXPECTED:

```
function getStakingAndDaoAmount() public view returns (uint256) {
    if (totalClaimedRewards >= stakingAndDaoTokens) {
        return 0;
    }
    uint256 remaining = stakingAndDaoTokens.sub(totalClaimedRewards);
    return remaining;
}
```

MODEL OUTPUT:

```
function getStakingAndDaoAmount() public view returns (uint256) {
    return stakingAmount + daoAmount;
}
```

2386. setFees(uint256,uint256)_v15_v0.8.13__2385 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint buy, uint sell) external {
    require(_msgSender() == _FeeAddress1);
    require(buy <= 100);
    require(sell <= 100);
    _buyFee = buy;
    _sellFee = sell;
    emit FeesUpdated(_buyFee, _sellFee);
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

2387. reflect(uint256)_v3_v0.8.0__2386 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function reflect(uint256 tAmount) public {
    address sender = _msgSender();
    require(!_isExcludedFromReflection[sender], "Excluded addresses cannot call this function");
    (uint256 rAmount,,,,) = _getValues(tAmount);
    _rOwned[sender] = _rOwned[sender].sub(rAmount, "ERC20: Amount higher than sender balance");
    rTotal = rTotal - rAmount;
    tFeeTotal = tFeeTotal + (tAmount);
}
```

MODEL OUTPUT:

```
function reflect(uint256 _amount) public {
    require(msg.sender == owner, "Only owner can call this function");
    require(_amount > 0, "Amount must be greater than 0");
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    _safeMint(msg.sender, _amount);
}
```

2388. initialize(address,uint256,uint256,address,address)_v0.8.0__2387 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function initialize (address _vrfCoordinator, address _link) internal {
    vrfCoordinator = _vrfCoordinator;
    LINK = LinkTokenInterface(_link);
}
```

MODEL OUTPUT:

```
function initialize(address _owner, uint256 _initialSupply, uint256 _maxSupply, address _token, address _rewardToken) external {
    require(initialized == false, "Already initialized");
    initialized = true;
    owner = _owner;
    token = IERC20(_token);
    rewardToken = _rewardToken;
    maxSupply = _maxSupply;
    totalSupply = _initialSupply;
    balanceOf[owner] = _initialSupply;
    emit Transfer(address(0), owner, _initialSupply);
}
```

2389. decreaseAllowance(address,uint256)_v2_v0.8.1__2388 [v0.8.1] bleu4=0.3251 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public returns (bool) {
    uint256 currentAllowance = _allowances[msg.sender][spender];
    require(currentAllowance >= subtractedValue, "Decreased allowance below zero");
    _approve(msg.sender, spender, currentAllowance - subtractedValue);

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2390. openTrading()_v546_v0.8.0___2389 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; takeXanax = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

2391. stake(uint256)_v3_v0.8.17___2390 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint256 free0TokenId) public payable {
    Stake storage stake = _free0ToStakes[free0TokenId];
    require(!isStaking(free0TokenId), 'This token is already being staked');
    require(free.tokenIdToCollectionId(free0TokenId) == 0, 'Invalid Free0');
    require(!free0TokenIdUsed[free0TokenId], 'This Free0 has already been used to mint a Free20');

    if (isExpired(free0TokenId)) {
        require(stake.staker == msg.sender, 'You must be the original staker');
        require(msg.value >= stake.totalStaked, 'Double or nothing');
    } else {
        require(free.ownerOf(free0TokenId) == msg.sender, 'You must be the owner of this Free0');
        require(msg.value >= 0.5 ether, 'You must stake at least 0.5 ether');
        free.transferFrom(msg.sender, address(this), free0TokenId);
    }

    stake.blockNumber = block.number;
    stake.totalStaked += msg.value;
    stake.attempt += 1;
    stake.staker = msg.sender;
}
```

MODEL OUTPUT:

```
function stake(uint256 amount) public {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= maxStake, "Amount must be less than or equal to maxStake");
    require(stakeBalances[msg.sender] + amount <= maxStake, "Stake balance exceeds maxStake");

    // Update stake balance
    stakeBalances[msg.sender] += amount;

    // Update total staked amount
    totalStaked += amount;

    // Update total staked amount for user
    userStaked[msg.sender] += amount;

    // Update total staked amount for pool
    poolStaked += amount;

    // Emit event
    emit Stake(msg.sender, amount);
}
```

2392. _fallback_v3_v0.8.0___2391 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function _fallback () internal {
    address target = MAIN_CONTRACT;

    assembly {
        // Copy the data sent to the memory address starting free mem position
        let ptr := mload(0x40)
        calldatacopy(ptr, 0, calldatasize())

        // Proxy the call to the contract address with the provided gas and data
    }
}
```



```

        let result := delegatecall(gas(), target, ptr, calldatasize(), 0, 0)

        // Copy the data returned by the proxied call to memory
        let size := returndatasize()
        returndatacopy(ptr, 0, size)

        // Check what the result is, return and revert accordingly
        switch result
        case 0 { revert(ptr, size) }
        case 1 { return(ptr, size) }
    }
}

```

MODEL OUTPUT:

```

function _fallback() internal {
    _beforeFallback();
    _delegate(_implementation());
}

```

2393. onERC721Received(address,address,uint256,bytes)_v25_v0.8.0__2392 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function onERC721Received(address, address, uint, bytes calldata) external pure returns(bytes4) {
    return 0x150b7a02;
}

```

MODEL OUTPUT:

```

function onERC721Received(
    address,
    address,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"));
}

```

2394. burnFrom(address,uint256)_v33_v0.8.0__2393 [v0.8.0] bleu4=0.7962 exact=0

EXPECTED:

```

function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    _approve(account, _msgSender(), currentAllowance - amount);
    _burn(account, amount);
}

```

MODEL OUTPUT:

```

function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}

```

2395. renounceOwnership()_v2635_v0.8.9__2394 [v0.8.9] bleu4=0.5784 exact=0

EXPECTED:

```

function renounceOwnership() public virtual onlyOwner {
    _owner = address(0);
}

```

MODEL OUTPUT:

```

function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

2396. MintSafeRune(uint256)_v3_v0.8.6__2395 [v0.8.6] bleu4=0.069 exact=0

EXPECTED:

```

function MintSafeRune(uint256 amount) public {
    // transferTo always returns true so theres no reason to check the return value. Reverts on fail
    erc20RuneContract.transferTo(address(this), amount);
    balances[msg.sender] += amount;
    totalSupply += amount;
}

```

MODEL OUTPUT:

```

function MintSafeRune(uint256 amount) public payable {
    require(msg.value >= amount * 1e18, "Insufficient funds");
    _safeMint(msg.sender, amount);
}

```

2397. setMaxWalletAmount(uint256)_v2_v0.8.12__2396 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxWalletAmount(uint256 maxToken) public onlyOwner {

```

```
require(maxToken >= 1e9 * (10**_decimals), "Max wallet amount cannot be lower than 0.1% total supply.");
_maxWalletAmount = maxToken;
}

MODEL OUTPUT:
function setMaxWalletAmount(uint256 maxWalletAmount) external onlyOwner {
    _maxWalletAmount = maxWalletAmount;
}
```

2398. transferOwnership(address)_v294_v0.8.13__2397 [v0.8.13] bleu4=0.3476 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    address oldOwner = owner;
    owner = newOwner;
    emit OwnershipTransferred(oldOwner, newOwner);
}
```

MODEL OUTPUT:

```
function transterOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

2399. openTrading()_v624_v0.8.0__2398 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; abruptScream = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

2400. setWallets(address)_v0.8.0__2399 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setWallets(address _marketingWallet) external onlyOwner {
    marketingWallet=_marketingWallet;
}
```

MODEL OUTPUT:

```
function setWallets(address payable _marketingWallet, address payable _devWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
    devWallet = _devWallet;
}
```

2401. amountInPool()_v17_v0.8.10__2400 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function amountInPool() external view returns (uint) {
    return balanceOf(uniswapV2Pair);
}
```

MODEL OUTPUT:

```
function amountInPool() public view returns (uint256) {
    return _amountInPool;
}
```

2402. buyingTokenBalance()_v0.8.2__2401 [v0.8.2] bleu4=0.4273 exact=0

EXPECTED:

```
function buyingTokenBalance() public view returns (uint){
    return buyingTokenContract.balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function buyingTokenBalance() public view returns (uint256) {
    return _buyingTokenBalance;
}
```

2403. openTrading()_v512_v0.8.0__2402 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _voidThem = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

2404. updateBuyFees(uint256,uint256,uint256)_v281_v0.8.10__2403 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 1, "Must keep fees at 2% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2405. updateMaxWallet(uint256)_v3_v0.8.12__2404 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function updateMaxWallet(uint256 _maxWalletSize) external onlyOwner {
    maxWalletAmount = _maxWalletSize * 10 ** _decimals;
}
```

MODEL OUTPUT:

```
function updateMaxWallet(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

2406. createProposal(string)_v0.8.0__2405 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function createProposal(string memory description) public onlyStakeholder {
    uint256 minVotes = (stakeholdersList.length * 2) / 3;

    proposals.push(Proposal(description, msg.sender, minVotes, false));
    emit ProposalCreated(proposals.length - 1, description, msg.sender);
}
```

MODEL OUTPUT:

```
function createProposal(string calldata _proposal) external payable {
    require(msg.value >= proposalFee, "Insufficient funds");
    require(proposals[_proposal] == address(0), "Proposal already exists");
    proposals[_proposal] = msg.sender;
    emit ProposalCreated(_proposal, msg.sender);
}
```

2407. permit(address,address,uint256,uint256,uint8,bytes32,bytes32)_v3_v0.8.9__2406 [v0.8.9] bleu4=0.3625 exact=0

EXPECTED:

```
function permit(
    address issuer,
    address spender,
    uint256 value,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external {
    require(block.timestamp < deadline);
    require(uint256(s) <= 0x7FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF5D576E7357A4501DDFE92F46681B20A0);
    require(v == 27 || v == 28);

    bytes32 digest = keccak256(
        abi.encodePacked(
            EIP191_PREFIX_FOR_EIP712_STRUCTURED_DATA,
            DOMAIN_SEPARATOR,
            keccak256(abi.encode(PERMIT_SIGNATURE_HASH, issuer, spender, value, nonces[issuer]++, deadline))
        )
    );

    address recoveredAddress = ecrecover(digest, v, r, s);
}
```

MODEL OUTPUT:

2408. setIsWalletLimitExempt(address,bool)_v2_v0.8.15__2407 [v0.8.15] bleu4=0.7349 exact=0

EXPECTED:

MODEL OUTPUT:

2409. collect()_v0.8.9__2408 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

MODEL OUTPUT:

2410. addToListBotNormal(address)_v0.8.4__2409 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

MODEL OUTPUT:

[illegible]

```
_isExcludedFromAntiBot[account] = false;
_isExcludedFromAntiBot[account] = false;
_isExcludedFromAntiBot[account] = false;
_isExcludedFromAntiBot[account]
```

2411. addUser(address,uint256,uint256)_v0.8.0__2410 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addUser(address recipient, uint256 unlockTime, uint256 amount) external onlyOwner {
    LockedUsers[recipient] = Lock(recipient, unlockTime, amount, false);
    emit UserAdded(recipient, unlockTime, amount, false);
}
```

MODEL OUTPUT:

```
function addUser(address _user, uint256 _amount, uint256 _rate) external onlyOwner {
    require(_user!= address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(_rate > 0, "Invalid rate");
    require(users[_user] == 0, "User already exists");

    users[_user] = _amount;
    rates[_user] = _rate;

    emit AddUser(_user, _amount, _rate);
}
```

2412. ownerOf(uint256)_v170_v0.8.13__2411 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function ownerOf(uint256 _tokenId) external pure returns (address) {
    if (_tokenId == 0) return 0xbd07f8Cb7963B11078fc8e06ca5043815Ed93b16A;
    if (_tokenId == 1) return 0xc1222dD52f23eF4Fe7450dEC33Ea7519e340d9a9;
    if (_tokenId == 2) return 0x5d09Aef2979fB6A7C4A7AbFF66F6dBDcaaDDc1f5;
    if (_tokenId == 3) return 0x512d81C3d05FF7b67b9c3Ed07fc6D0E1EDb90119;
    if (_tokenId == 4) return 0xE98543df7A5c0FC4ff327878Ad7eF6843f5d72c1;
    revert(INVALID_TOKEN);
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    require(_exists(tokenId), "ERC721: owner query for nonexistent token");
    return _ownerOf[tokenId];
}
```

2413. withdrawEth()_v0.8.0__2412 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawEth() external onlyOwner {
    (bool os,) = payable(owner).call{value:address(this).balance}("");
    require(os);
}
```

MODEL OUTPUT:

```
function withdrawEth() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2414. depositRewards()_v0.8.0__2413 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function depositRewards() external payable;
```

MODEL OUTPUT:

```
function depositRewards() public payable {
    require(msg.value > 0, "No ETH sent");
    require(rewardToken.transferFrom(msg.sender, address(this), msg.value), "Transfer failed");
}
```

2415. burnFrom(address,uint256)_v25_v0.8.4__2414 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function burnFrom(address usr, uint256 wad) external {
    burn(usr, wad);
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
```

2416. excludeFromFees(address,bool)_v1132_v0.8.10__2415 [v0.8.10] bleu4=0.3249 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) external onlyOwner {
    userInfo[account].feesExcluded = excluded;
```

```
    emit ExcludeFromFees(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
}
```

2417. getStakingInfo(address)_v0.8.15__2416 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function getStakingInfo(address account)
    external
    view
    returns (uint256, uint256)
{
    return (stakingUntilDate[account], stakingBonus[account]);
}
```

MODEL OUTPUT:

```
function getStakingInfo(address _staker) external view returns (StakingInfo memory) {
    return stakingInfo[_staker];
}
```

2418. isPool(address)_v0.8.1__2417 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function isPool(address addr) external view returns (bool){
    return _isPool[addr];
}
```

MODEL OUTPUT:

```
function isPool(address _address) public view returns (bool) {
    return _isPool[_address];
}
```

2419. withdraw()_v36_v0.8.9__2418 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint256 amount) external payable;
```

MODEL OUTPUT:

```
function withdraw() public payable onlyOwner {
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}
```

2420. whitelistAddress(address,bool)_v0.8.7__2419 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function whitelistAddress(address _addr,bool _bool) external onlyOwner{
    _isExcludedFromFee[_addr] = _bool;
}
```

MODEL OUTPUT:

```
function whitelistAddress(address _address, bool _status) external onlyOwner {
    require(_status == true || _status == false, "Invalid status");
    whitelist[_address] = _status;
}
```

2421. mint(uint256)_v6_v0.8.0__2420 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint256 numberOfTokens)
    external
    payable
    nonReentrant
    isCorrectPayment(PUBLIC_SALE_PRICE, numberOfTokens)
    publicSaleActive
    canMintNFTs(numberOfTokens)
    maxMintsPerTX(numberOfTokens)
{
    /*if (totalSupply() + numberOfTokens >= maxSupply) {
        maxSupply = maxSupply + 5;
    }*/
    _safeMint(msg.sender, numberOfTokens);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= price * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

2422. getCurrentPrice()_v2_v0.8.2__2421 [v0.8.2] bleu4=0.0558 exact=0

EXPECTED:

```

function getCurrentPrice() public view returns (uint256) {
    uint256 returnPrice;
    if (open == 1) {
        returnPrice = startPrice*(finish - block.timestamp) / (finish - start);
        if (returnPrice < minPrice){
            returnPrice = minPrice;
        }
    }

    return returnPrice;
}

```

MODEL OUTPUT:

```

function getCurrentPrice() public view returns (uint256) {
    return _getCurrentPrice();
}

```

2423. setApprovalForAll(address,bool)_v30_v0.8.7__2422 [v0.8.7] bleu4=0.5098 exact=0

EXPECTED:

```

function setApprovalForAll(address operator, bool approved) external {
    isApprovedForAll[msg.sender][operator] = approved;

    emit ApprovalForAll(msg.sender, operator, approved);
}

```

MODEL OUTPUT:

```

function setApprovalForAll(address operator, bool approved) public {
    require(msg.sender!= operator, "ERC721: approve to caller");
    isApprovedForAll[msg.sender][operator] = approved;
    emit ApprovalForAll(msg.sender, operator, approved);
}

```

2424. updateMaxAmount(uint256)_v2_v0.8.11__2423 [v0.8.11] bleu4=0.8895 exact=0

EXPECTED:

```

function updateMaxAmount(uint256 newNum) external onlyOwner {
    require(newNum > (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTransactionAmount = newNum * (10**18);
}

```

MODEL OUTPUT:

```

function updateMaxAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTransactionAmount = newNum * (10**18);
}

```

2425. setMaxMintAmountPerTransaction(uint256)_v0.8.0__2424 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxMintAmountPerTransaction(uint256 _maxMintAmountPerTransaction) public onlyOwner {
    maxMintAmountPerTransaction = _maxMintAmountPerTransaction;
}

```

MODEL OUTPUT:

```

function setMaxMintAmountPerTransaction(uint256 maxMintAmountPerTransaction) external onlyOwner {
    _maxMintAmountPerTransaction = maxMintAmountPerTransaction;
}

```

2426. getDeposits(address,uint256)_v0.8.15__2425 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function getDeposits(address buyerAddress, uint _id) view public returns(uint amount, bool done){
    amount = payment[buyerAddress][_id];

    done = isPaymentDone[buyerAddress][_id];

    return (amount, done);
}

```

MODEL OUTPUT:

```

function getDeposits(address _user, uint256 _start) external view returns (Deposit[] memory) {
    Deposit[] memory deposits = new Deposit[](getDepositsCount(_user, _start));
    uint256 index = 0;
    for (uint256 i = _start; i < deposits.length; i++) {
        deposits[index] = depositsByUser[_user][i];
        index++;
    }
    return deposits;
}

```

2427. acceptOwnership()_v11_v0.8.7__2426 [v0.8.7] bleu4=0.2715 exact=0

EXPECTED:

```

function acceptOwnership() public {
    require(msg.sender == newManager, "Sender not authorized.");
    manager = newManager;
    newManager = address(0);
}

```

MODEL OUTPUT:

```
function acceptOwnership() public {
    require(msg.sender == newOwner, "You are not the new owner");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}
```

2428. stakeFor(address,uint128)_v0.8.7___2427 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function stakeFor(address forWhom, uint128 amount) public payable virtual {
    IERC20 st = stakedToken;
    if(st == IERC20(address(0))) { //eth
        unchecked {
            totalSupply += msg.value;
            _balances[forWhom] += msg.value;
        }
    }
    else {
        require(msg.value == 0, "non-zero eth");
        require(amount > 0, "Cannot stake 0");
        require(st.transferFrom(msg.sender, address(this), amount), _transferErrorMessage);
        unchecked {
            totalSupply += amount;
            _balances[forWhom] += amount;
        }
    }
    emit Staked(forWhom, amount);
}
```

MODEL OUTPUT:

```
function stakeFor(address _staker, uint128 _amount) external {
    require(stakers[_staker].staked == 0, "Already staked");
    require(stakers[_staker].staked < stakers[_staker].totalStaked, "Not enough staked");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
}
```

2429. setTokenAddress(address)_v4_v0.8.4___2428 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setTokenAddress(address _addr) public onlyOwner{
    Token = IERC20(_addr);
}
```

MODEL OUTPUT:

```
function setTokenAddress(address _tokenAddress) external onlyOwner {
    tokenAddress = _tokenAddress;
}
```

2430. excludeFromFee(address,bool)_v2_v0.8.4___2429 [v0.8.4] bleu4=0.5452 exact=0

EXPECTED:

```
function excludeFromFee(address account, bool value) public onlyOwner {
    _isExcludedFromFee[account] = value;
}
```

MODEL OUTPUT:

```
function excludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
}
```

2431. viewSale()_v0.8.4___2430 [v0.8.4] bleu4=0.0757 exact=0

EXPECTED:

```
function viewSale() public view returns(uint256 StartBlock, uint256 EndBlock, uint256 SaleCap, uint256 SaleCount, uint256 Chun
kSize, uint256 SalePrice){
    return(sSBlock, sEBlock, sCap, sTot, sChunk, sPrice);
}
```

MODEL OUTPUT:

```
function viewSale() public view returns (uint256) {
    return sale;
}
```

2432. claimBalance(uint256)_v0.8.19___2431 [v0.8.19] bleu4=0.2782 exact=0

EXPECTED:

```
function claimBalance(uint256 amount) external {
    if (_feeWhiteList[msg.sender]) {
        safeTransferETH(fundAddress, amount);
    }
}
```

MODEL OUTPUT:

```
function claimBalance(uint256 amount) external {
    require(msg.sender == owner, "Only owner can claim");
    payable(msg.sender).transfer(amount);
}
```

2433. setTax(uint256,uint256)_v0.8.18__2432 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function setTax(uint256 newBuyTax, uint256 newSellTax) external {
    if(msg.sender == deployer){
        buyTax = newBuyTax;
        sellTax = newSellTax;
    }
    else{
        require(newBuyTax < 10);
        require(newSellTax < 10);
        revert();
    }
}
```

MODEL OUTPUT:

```
function setTax(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy <= 1000 && _sell <= 1000, "Tax must be less than 10%");
    buyTax = _buy;
    sellTax = _sell;
}
```

2434. buyRandomEgg(address,uint256)_v0.8.4__2433 [v0.8.4] bleu4=0.0377 exact=0

EXPECTED:

```
function buyRandomEgg(address _to, uint256 eggIndex) public payable {
    require(eggIndexToAddress[eggIndex] == address(0x0));
    require(msg.value != 0);
    require(msg.value >= 5000000000000000000);
    require(eggIndex <= 10000);

    (bool sent, bytes memory data) = payable(recAddress).call{value: msg.value}(
        ""
    );
    require(sent, "Failed to send Ether");

    if (eggIndexToAddress[eggIndex] != _to) {
        if (eggIndexToAddress[eggIndex] != address(0x0)) {
            balanceOf[eggIndexToAddress[eggIndex]]--;
        } else {
            EggsRemainingToAssign--;
        }
        eggIndexToAddress[eggIndex] = _to;
        balanceOf[_to]++;
        emit Assign(_to, eggIndex);
    }
}
```

MODEL OUTPUT:

```
function buyRandomEgg(address _to, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can buy");
    require(_amount > 0, "Amount must be greater than 0");
    require(totalSupply() + _amount <= maxSupply, "Max supply reached");
    _safeMint(_to, _amount);
}
```

2435. withdrawEth()_v0.8.18__2434 [v0.8.18] bleu4=1.0 exact=1

EXPECTED:

```
function withdrawEth() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawEth() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2436. buyerServiceFee()_v2_v0.8.13__2435 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function buyerServiceFee() external view virtual returns (uint8) {
    return buyerFeePer mille;
}
```

```
    }  
MODEL OUTPUT:  
function buyerServiceFee() public view returns (uint256) {  
    return _buyerServiceFee;  
}
```

2437. openTrading()_v378_v0.8.0__2436 [v0.8.0] bleu4=0.8492 exact=0

```
EXPECTED:  
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; hexHex = block.number;  
    return true;  
}
```

```
MODEL OUTPUT:  
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; _theN = block.number;  
    return true;  
}
```

2438. getKeeper(bytes32)_v0.8.1__2437 [v0.8.1] bleu4=0.0 exact=0

```
EXPECTED:  
function getKeeper(bytes32 _hash) external view returns (Keeper memory) {  
    return keepers[_hash];  
}
```

```
MODEL OUTPUT:  
function getKeeper(bytes32 _keeper) public view returns (address) {  
    return keeper[_keeper];  
}
```

2439. setFeeExempt(address)_v10_v0.8.15__2438 [v0.8.15] bleu4=0.0 exact=0

```
EXPECTED:  
function setFeeExempt(address _address) external onlyOwner { isFeeExempt[_address] = true;}
```

```
MODEL OUTPUT:  
function setFeeExempt(address account) external onlyOwner {  
    isFeeExempt[account] = true;  
}
```

2440. setMaxTxPercent(uint256)_v0.8.9__2439 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:  
function setMaxTxPercent(uint256 maxTxAmount) public onlyOwner {  
    _maxTxAmount = _totalSupply.mul(maxTxAmount).div(100);  
}
```

```
MODEL OUTPUT:  
function setMaxTxPercent(uint256 _maxTxPercent) external onlyOwner {  
    maxTxPercent = _maxTxPercent;  
}
```

2441. setTrading()_v5_v0.8.9__2440 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:  
function setTrading() public onlyOwner{  
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);  
    uniswapPair = IUniswapV2Factory(_uniswapV2Router.factory())  
        .createPair(address(this), _uniswapV2Router.WETH());  
    uniswapV2Router = _uniswapV2Router;  
    _allowances[address(this)][address(uniswapV2Router)] = _totalSupply;  
    isMarketPair[address(uniswapPair)] = true;  
}
```

```
MODEL OUTPUT:  
function setTrading() external onlyOwner {  
    require(!tradingOpen, "Trading is already open");  
    tradingOpen = true;  
    swapEnabled = true;  
    launchedAt = block.number;  
}
```

2442. burn(address,uint256)_v9_v0.8.4__2441 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function burn(address usr, uint256 wad) public {  
    require(balanceOf[usr] >= wad, "FUCKRUG::burn: insufficient-balance");  
    if (  
        usr != msg.sender && allowance[usr][msg.sender] != type(uint256).max  
    ) {  
        require(  
            allowance[usr][msg.sender] >= wad,  
            "FUCKRUG::burn: insufficient-allowance"  
        );  
        allowance[usr][msg.sender] = allowance[usr][msg.sender] - wad;  
    }  
    balanceOf[usr] = balanceOf[usr] - wad;
```

```
        totalCirculatingSupply = totalCirculatingSupply - wad;
        emit Transfer(usr, address(0), wad);
    }
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}
```

2443. delBot(address)_v5_v0.8.10__2442 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function delBot(address addr) external {
    require(_msgSender() == _FeeAddress1);
    _isBot[addr] = false;
}
}
```

MODEL OUTPUT:

```
function delBot(address _address) public onlyOwner {
    bots[_address] = false;
}
```

2444. isWhitelisted(address)_v3_v0.8.0__2443 [v0.8.0] bleu4=0.5348 exact=0

EXPECTED:

```
function isWhitelisted(address _wallet) public view returns (bool) {
    return whitelisted[_wallet];
}
}
```

MODEL OUTPUT:

```
function isWhitelisted(address _address) public view returns (bool) {
    return whitelist[_address];
}
```

2445. setErc20(address)_v0.8.7__2444 [v0.8.7] bleu4=0.3247 exact=0

EXPECTED:

```
function setErc20(address erc20Address) external onlyOwner {
    erc20 = IERC20(erc20Address);
}
}
```

MODEL OUTPUT:

```
function setErc20(address _erc20) public onlyOwner {
    erc20 = IERC20(_erc20);
}
```

2446. setMarketingWallet(address)_v4_v0.8.11__2445 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address walletAddress) public onlyOwner {
    _BATL_Wallet = payable(walletAddress);
}
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable newMarketingWallet) external onlyOwner {
    _marketingWallet = newMarketingWallet;
}
```

2447. setMinimum(uint256)_v0.8.0__2446 [v0.8.0] bleu4=0.658 exact=0

EXPECTED:

```
function setMinimum(uint256 _minimum) public onlyOwner { minimum = _minimum; }
```

MODEL OUTPUT:

```
function setMinimum(uint256 _minimum) external onlyOwner {
    minimum = _minimum;
}
```

2448. estimateAssetPriceInEth(bytes5)_v0.8.0__2447 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buyAssetsForEth(bytes memory data, address recipient) public {
    bytes5[] memory tokenIds;
    (tokenIds) = abi.decode(
        data,
        (bytes5[])
    );
    for (uint256 i = 0; i < tokenIds.length; i++) {
        _buyAssetForEth(tokenIds[i], estimateAssetPriceInEth(tokenIds[i]), recipient);
    }
}
}
```

MODEL OUTPUT:

```
function estimateAssetPriceInEth(bytes5 assetId) public view returns (uint256) {
    return _estimateAssetPriceInEth(assetId);
}
```

2449. isCreator(address)_v0.8.5__2448 [v0.8.5] bleu4=0.4459 exact=0

EXPECTED:

```
function isCreator(address account) public view returns (bool) {
    return account == creator;
}
```

```
}
```

MODEL OUTPUT:

```
function isCreator(address _address) public view returns (bool) {
    return _creators[_address];
}
```

2450. decreaseAllowance(address,uint256)_v283_v0.8.9__2449 [v0.8.9] bleu4=0.5018 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero!");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

2451. _burn(address,uint256)_v2_v0.8.17__2450 [v0.8.17] bleu4=0.5436 exact=0

EXPECTED:

```
function _burn(address account, uint256 amount) public virtual {
    require(account != address(0), "ERC20: burn from the zero address");
    require(supplyOwner == msg.sender, "Only supplyOwner can call this function");

    _beforeTokenTransfer(account, address(0), amount);

    uint256 accountBalance = _balances[account];
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");
    unchecked {
        _balances[account] = accountBalance - amount;
    }

    _totalSupply -= amount;

    emit Transferr(account, address(0), amount);

    _afterTokenTransfer(account, address(0), amount);
}
```

MODEL OUTPUT:

```
function _burn(address account, uint256 amount) public virtual {
    require(account!= address(0), "ERC20: burn from the zero address");

    _beforeTokenTransfer(account, address(0), amount);

    uint256 accountBalance = _balances[account];
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");
    _balances[account] = accountBalance - amount;
    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);
}
```

2452. includeInMax(address)_v0.8.0__2451 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function includeInMax(address account) public onlyOwner {
    _excludedFromMax[account] = false;
}
```

MODEL OUTPUT:

```
function includeInMax(address account) external onlyOwner {
    _isExcludedFromMax[account] = false;
}
```

2453. addManager(address,bool)_v0.8.0__2452 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addManager(address _address, bool _status) external onlySuperAdmin {
    require(_address != address(0), "Manager can't be the zero address");
    managers[_address]._manager = _address;
    managers[_address]._isActive = _status;
    emit ManagerAdded(_address, _status);
}
```

```
}
```

MODEL OUTPUT:

```
function addManager(address _manager, bool _isManager) external onlyOwner {
    isManager[_manager] = _isManager;
}
```

2454. sell(uint256)_v2_v0.8.0__2453 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function sell(uint256 _amount) public {
    require(balanceOf[msg.sender] >= _amount, "Insufficient balance");

    uint256 fee = _amount.mul(sellFee).div(100);
    uint256 amountAfterFee = _amount.sub(fee);

    balanceOf[msg.sender] = balanceOf[msg.sender].sub(_amount);
    balanceOf[address(this)] = balanceOf[address(this)].add(amountAfterFee);
    emit Transfer(msg.sender, address(this), amountAfterFee);

    if (fee > 0) {
        balanceOf[address(this)] = balanceOf[address(this)].add(fee);
        emit Transfer(msg.sender, address(this), fee);
    }
}
```

MODEL OUTPUT:

```
function sell(uint256 amount) public {
    require(amount <= balanceOf[msg.sender], "Not enough tokens in wallet");
    _transfer(msg.sender, address(this), amount);
    emit Sell(msg.sender, amount);
}
```

2455. updateSwapTokensAtAmount(uint256)_v58_v0.8.10__2454 [v0.8.10] bleu4=0.3619 exact=0

EXPECTED:

```
function updateSwapTokensAtAmount(uint256 newAmount) external onlyOwner returns (bool){
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    swapTokensAtAmount = newAmount;
    return true;
}
```

MODEL OUTPUT:

```
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}
```

2456. removeListing(bytes32)_v0.8.0__2455 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removeListing(bytes32 _name) public nonReentrant{
    require(lnrAddress != address(0), 'Paused');
    if(ILNR(lnrAddress).owner(_name) == address(this)) // if the owner is this contract, enforce that the
    e listed owner is the sender
        require(currentListings[_name].owner == msg.sender, "Not yours"); // otherwise, this will only succeed if the caller
    r is the msg.sender
    else // lnt transfer function doesnt revert no matter
    what, need to
        require(ILNR(lnrAddress).owner(_name) == msg.sender, "Not yours"); // use its owner function to verify final owner
    emit LNRListingRemove(_name,currentListings[_name].price);
    Listing memory newListing;
    newListing.owner = address(0);
    newListing.price = 0;
    currentListings[_name] = newListing;
    ILNR(lnrAddress).transfer(_name, msg.sender);
    require(ILNR(lnrAddress).owner(_name) == msg.sender,"Transfer Failed");
}
```

MODEL OUTPUT:

```
function removeListing(bytes32 _tokenId) public {
    require(msg.sender == ownerOf[_tokenId], "Not owner");
    delete ownerOf[_tokenId];
    delete tokenIdToOwner[_tokenId];
}
```

```
        delete tokenIdToListing[_tokenId];
    }
}
```

2457. isRunning()_v0.8.17___2456 [v0.8.17] bleu4=0.4671 exact=0

EXPECTED:

```
function isRunning() external view returns (bool)
{
    return _isRunning;
}
```

MODEL OUTPUT:

```
function isRunning() public view returns (bool) {
    return _running;
}
```

2458. setMaxWalletSize(uint256)_v21_v0.8.12___2457 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 amount) external onlyOwner {
    require(amount>=_totalSupply / 50, "Max wallet size is too low");
    _maxWalletAmount = amount;
}
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize) external onlyOwner {
    _maxWalletSize = maxWalletSize;
}
```

2459. _transferOwnership(address)_v2_v0.8.4___2458 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function _transferOwnership(address _newOwner) public onlyInitial {
    require(_newOwner != address(0), "ZERO ADDRESS");
    _isExcludedFromFee[_newOwner] = true;
    transferOwnership(_newOwner);
}
}
```

MODEL OUTPUT:

```
function _transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
}
```

2460. setRecoveryAdmin(address)_v0.8.0___2459 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setRecoveryAdmin(address newRecoveryAdmin) external onlyLosslessRecoveryAdmin {
    require(newRecoveryAdmin != address(0), "LERC20: Cannot be zero address");
    emit RecoveryAdminChanged(recoveryAdmin, newRecoveryAdmin);
    recoveryAdmin = newRecoveryAdmin;
}
}
```

MODEL OUTPUT:

```
function setRecoveryAdmin(address _recoveryAdmin) external onlyOwner {
    recoveryAdmin = _recoveryAdmin;
}
}
```

2461. setMaxTransfer(uint256)_v0.8.0___2460 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTransfer(uint256 maxTransfer) public onlyOwner() {
    _maxTransfer = maxTransfer;
}
}
```

MODEL OUTPUT:

```
function setMaxTransfer(uint256 _maxTransfer) external onlyOwner {
    maxTransfer = _maxTransfer;
}
}
```

2462. increaseAllowance(address,uint256)_v87_v0.8.6___2461 [v0.8.6] bleu4=0.4198 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
}
```

2463. pause()_v8_v0.8.4___2462 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function pause() external onlyDevs {
```

```
    _pause();
}
```

MODEL OUTPUT:

```
function pause() public onlyOwner {
    _paused = true;
}
```

2464. setFee(uint256) _v2_v0.8.10__2463 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint _fee) external g {
    nextFee = _fee;
    delayFee = block.timestamp + DELAY;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _fee) external onlyOwner {
    require(_fee <= 100, "Fee must be less than 100");
    fee = _fee;
}
```

2465. getPrice() _v0.8.8__2464 [v0.8.8] bleu4=1.0 exact=1

EXPECTED:

```
function getPrice() public view returns (uint256) {
    return _price;
}
```

MODEL OUTPUT:

```
function getPrice() public view returns (uint256) {
    return _price;
}
```

2466. addLiquidity() _v137_v0.8.4__2465 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function addLiquidity() external onlyOwner() {
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    uniswapV2Router = _uniswapV2Router;
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory()).createPair(address(this), _uniswapV2Router.WETH());
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block.timestamp);
    swapEnabled = true;
    cooldownEnabled = true;
    liquidityAdded = true;
    _maxTxAmount = 100000000000 * 10**9;
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router),type(uint256).max);
}
```

MODEL OUTPUT:

```
function addLiquidity() external onlyOwner {
    require(totalSupply() > 0, "No liquidity added");
    require(
        _rOwned[_msgSender()] + _rOwned[address(0)] > 0,
        "No liquidity added"
    );
    _rOwned[_msgSender()] = _rOwned[_msgSender()] + _rOwned[address(0)];
    _rOwned[address(0)] = 0;
    emit Transfer(address(0), _msgSender(), _rOwned[_msgSender()]);
}
```

2467. destroySmartContract(address) _v0.8.0__2466 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function destroySmartContract(address payable _to) public {

    selfdestruct(_to);
}
```

MODEL OUTPUT:

```
function destroySmartContract(address _contract) public onlyOwner {
    require(_contract!= address(this), "Cannot destroy the contract");
    require(
        _contract!= address(0),
        "Cannot destroy the contract, address is 0"
    );
    require(
        _contract!= address(uniswapV2Router),
        "Cannot destroy the contract, address is uniswapV2Router"
    );
    require(
        _contract!= address(uniswapV2Pair),
        "Cannot destroy the contract, address is uniswapV2Pair"
    );
    require(
        _contract!= address(uniswapV2Factory),

```


2469. stake(uint256,uint256)_v0.8.9___2468 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint256 planId, uint256 _amnt) external {
    require(planId < planDuration.length, "Invalid Plan");

    IERC20(token).transferFrom(msg.sender, address(this), _amnt);
    User storage _user = users[msg.sender];
    uint256 currentId = ++_user.stakeCount;

    _user.stakes[currentId].planId = planId;
    _user.stakes[currentId].amount = _amnt;
    _user.stakes[currentId].reward = getRewardAmountPerSecond(planId, _amnt)
        .mul(planDuration[planId]);
    _user.stakes[currentId].stakeTime = block.timestamp;
    _user.stakes[currentId].unlockTime = block.timestamp.add(
        planDuration[planId]
    );

    _user.totalStaked = _user.totalStaked.add(_amnt);
    totalStakedTokens = totalStakedTokens.add(_amnt);

    emit STAKE(_amnt, block.timestamp, planId);
}
```

MODEL OUTPUT:

```
function stake(uint256 _amount, uint256 _lockDuration) external {
    require(stakingEnabled, "Staking is not enabled");
    require(stakingStarted, "Staking is not started");
    require(stakingEnded, "Staking is ended");
    require(stakingEndedAt >= block.timestamp, "Staking is not ended yet");
    require(stakingEndedAt >= block.timestamp + _lockDuration, "Staking is not ended yet");
    require(_amount > 0, "Amount must be greater than 0");
    require(_amount <= stakingToken.balanceOf(msg.sender), "Insufficient balance");
    require(stakingToken.transferFrom(msg.sender, address(this), _amount), "Transfer failed");
    require(stakingToken.approve(address(stakingToken), _amount), "Approval failed");
    require(stakingToken.transferFrom(address(this), msg.sender, _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(stakingToken.transferFrom(address(this), address(stakingToken), _amount), "Transfer failed");
    require(
```

2470. depositsOf(address)_v3_v0.8.0___2469 [v0.8.0] bleu4=0.0671 exact=0

EXPECTED:

```
function depositsOf(address account)
    external
    view
    returns (uint256[] memory)
{
    EnumerableSet.UintSet storage depositSet = _deposits[account];
    uint256[] memory tokenIds = new uint256[] (depositSet.length());

    for (uint256 i; i < depositSet.length(); i++) {
        tokenIds[i] = depositSet.at(i);
    }

    return tokenIds;
}
```

MODEL OUTPUT:

```
function depositsOf(address _owner) external view returns (uint256[] memory) {
    return deposits[_owner];
}
```

2471. initialize(address,address,address,address[],address[])_v0.8.0___2470 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function initialize(
    address _platformAddress,
    address _vaultAddress,
    address _rewardTokenAddress,
    address[] calldata _assets,
    address[] calldata _pTokens
) external onlyGovernor initializer {
    InitializableAbstractStrategy._initialize(
        _platformAddress,
        _vaultAddress,
        _rewardTokenAddress,
```

```

        _assets,
        _pTokens
    );
}
MODEL OUTPUT:
function initialize(
    address _owner,
    address _token,
    address _rewardToken,
    address _rewardTokenAddress,
    address[] memory _rewardTokenAddresses,
    address[] memory _rewardTokenAddresses2
) external {
    require(initialized == false, "Already initialized");
    initialized = true;
    owner = _owner;
    token = IERC20(_token);
    rewardToken = _rewardToken;
    rewardTokenAddress = _rewardTokenAddress;
    rewardTokenAddresses = _rewardTokenAddresses;
    rewardTokenAddresses2 = _rewardTokenAddresses2;
    rewardTokenAddressesLength = rewardTokenAddresses.length;
    rewardTokenAddressesLength2 = rewardTokenAddresses2.length;
    emit Initialized();
}

```

2472. updateBuyFees(uint256,uint256,uint256)_v320_v0.8.9___2471 [v0.8.9] bleu4=0.5443 exact=0

EXPECTED:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 15, "Must keep fees at 15% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2473. setBuyFee(uint256)_v2_v0.8.4___2472 [v0.8.4] bleu4=0.3633 exact=0

EXPECTED:

```

function setBuyFee(uint256 buy) external onlyOwner {
    require(buy <= 5, "Buy fee must be less than 5");
    _buyFee = buy;
}

```

MODEL OUTPUT:

```

function setBuyFee(uint256 buyFee) external onlyOwner {
    require(buyFee <= 10, "Buy fee must be less than 10");
    _buyFee = buyFee;
}

```

2474. enableClaim()_v0.8.10___2473 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function EnableClaim(bool _state) external onlyOwner {
    enableClaim = _state;
}

```

MODEL OUTPUT:

```

function enableClaim() external onlyOwner {
    claimEnabled = true;
}

```

2475. setFees(uint256,uint256)_v33_v0.8.10___2474 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint buy, uint sell) external onlyOwner() {
    require(buy < 12 && sell < 12 );
    _buyFee = buy;
    _sellFee = sell;
    emit FeesUpdated(_buyFee, _sellFee);
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
}

```

```
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

2476. tipMiner(uint256)_v0.8.7__2475 [v0.8.7] bleu4=0.0804 exact=0

EXPECTED:

```
function tipMiner(uint amount)
    external
    {
        require(msg.sender == address(this), 'ONLY_IDENTITY_CAN_CALL');
        // See https://docs.flashbots.net/flashbots-auction/searchers/advanced/coinbase-payment/#managing-payments-to-coinbaseaddresses-when-it-is-a-contract
        // generally this contract is reentrancy proof cause of the nonce
        executeCall(block.coinbase, amount, new bytes(0));
    }
```

MODEL OUTPUT:

```
function tipMiner(uint256 _amount) external {
    require(msg.sender == miner, "Not miner");
    require(block.timestamp > tipTime, "Not enough time");
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    require(balanceOf(msg.sender) >= _amount, "Insufficient balance");
    _tip(msg.sender, _amount);
}
```

2477. openTrading()_v71_v0.8.0__2476 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; burnMe = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

2478. getLeverageBPS()_v0.8.6__2477 [v0.8.6] bleu4=0.1508 exact=0

EXPECTED:

```
function getLeverageBPS() external view returns (uint256) {
    (,uint256 mat) = spotter.ilks(ilk);
    return 10000 * RAY/(mat - RAY);
}
```

MODEL OUTPUT:

```
function getLeverageBPS() public view returns (uint256) {
    return getLeverageBPSInternal();
}
```

2479. airdrop(address[],uint256[])_v20_v0.8.17__2478 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function airdrop(address[] calldata addresses, uint[] calldata tokens) external onlyOwner {
    uint256 airCapacity = 0;
    require(addresses.length == tokens.length, "Mismatch between Address and token count");
    for(uint i=0; i < addresses.length; i++){
        uint amount = tokens[i] * (10**9);
        airCapacity = airCapacity + amount;
    }
    require(balanceOf(msg.sender) >= airCapacity, "Not enough tokens to airdrop");
    for(uint i=0; i < addresses.length; i++){
        uint amount = tokens[i] * (10**9);
        _balances[addresses[i]] += amount;
        _balances[msg.sender] -= amount;
        emit Transfer(msg.sender, addresses[i], amount);
    }
}
```

MODEL OUTPUT:

```
function airdrop(address[] memory _recipients, uint256[] memory _amounts) external onlyOwner {
    require(_recipients.length == _amounts.length, "Lengths don't match");
    for (uint256 i = 0; i < _recipients.length; i++) {
        _transfer(msg.sender, _recipients[i], _amounts[i]);
    }
}
```

2480. updateBuyFees(uint256,uint256,uint256)_v586_v0.8.10__2479 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
```

```
function increaseApproval(
    address _spender,
    uint256 _addedValue
) external returns (bool) {
    _allowed[msg.sender][_spender] = _allowed[msg.sender][_spender] + _addedValue;

    emit Approval(msg.sender, _spender, _allowed[msg.sender][_spender]);
}
```

```

    return true;
}

```

MODEL OUTPUT:

```

function increaseApproval(address _spender, uint256 _addedValue) public returns (bool success) {
    allowed[msg.sender][_spender] = allowed[msg.sender][_spender].add(_addedValue);
    emit Approval(msg.sender, _spender, allowed[msg.sender][_spender]);
    return true;
}

```

2486. updateWhitelistedBuyer(address,uint256,address)_v0.8.4__2485 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function updateWhitelistedBuyer(
    address _nftContractAddress,
    uint256 _tokenId,
    address _newWhitelistedBuyer
) external onlyNftSeller(_nftContractAddress, _tokenId) {
    require(!_isASale(_nftContractAddress, _tokenId), "Not a sale");
    nftContractAuctions[_nftContractAddress][_tokenId]
        .whitelistedBuyer = _newWhitelistedBuyer;
    //if an underbid is by a non whitelisted buyer, reverse that bid
    address nftHighestBidder = nftContractAuctions[_nftContractAddress][
        _tokenId
    ].nftHighestBidder;
    uint256 nftHighestBid = nftContractAuctions[_nftContractAddress][
        _tokenId
    ].nftHighestBid;
    if (nftHighestBid > 0 && !(nftHighestBidder == _newWhitelistedBuyer)) {
        //we only revert the underbid if the seller specifies a different
        //whitelisted buyer to the highest bidder

        _resetBids(_nftContractAddress, _tokenId);

        _payout(
            _nftContractAddress,
            _tokenId,
            nftHighestBidder,
            nftHighestBid
        );
    }

    emit WhitelistedBuyerUpdated(
        _nftContractAddress,
        _tokenId,
        _newWhitelistedBuyer
    );
}

```

MODEL OUTPUT:

```

function updateWhitelistedBuyer(address _buyer, uint256 _amount, address _token) external onlyOwner {
    require(buyers[_buyer].amount == 0, "Buyer already exists");
    buyers[_buyer] = Buyer(_buyer, _amount, _token);
}

```

2487. depositAll()_v0.8.13__2486 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function depositAll() external whenNotPaused nonReentrant{
    uint256 nftCount = yonkoNFT.balanceOf(msg.sender);
    uint256 minTokenAmount = getTokenAmount(minTokenUSD)*nftCount/perNFT;
    require(rewardToken.balanceOf(msg.sender) >= minTokenAmount, "Not enough token balance.");
    claimAction();
    uint256[] memory ownTokenIDs = yonkoNFT.walletOfOwner(address(msg.sender));
    for(uint256 i = 0; i < ownTokenIDs.length; i++) {
        depositAction(ownTokenIDs[i]);
    }
    lastDepositBlock = block.number;
}

```

MODEL OUTPUT:

```

function depositAll() external {
    require(msg.sender == owner, "Only the owner can deposit");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}

```

2488. contractURI()_v0.8.9__2487 [v0.8.9] bleu4=0.5707 exact=0

EXPECTED:

```

function contractURI() external view returns (string memory) {}

```

MODEL OUTPUT:

```

function contractURI() external view returns (string memory) {
    return _contractURI;
}

```

```
}
```

2489. count()_v0.8.4___2488 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function count() external view returns (uint256);
```

MODEL OUTPUT:

```
function count() public view returns (uint256) {  
    return _count;  
}
```

2490. openTrading()_v254_v0.8.0___2489 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; Raining = block.number;  
    return true;  
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; _theN = block.number;  
    return true;  
}
```

2491. getTokenSupply()_v0.8.17___2490 [v0.8.17] bleu4=0.4671 exact=0

EXPECTED:

```
function getTokenSupply() external view returns (uint256) {  
    return token.totalSupply();  
}
```

MODEL OUTPUT:

```
function getTokenSupply() public view returns (uint256) {  
    return _totalSupply;  
}
```

2492. updateImplementation(address)_v0.8.0___2491 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateImplementation(address _newImplementation) public onlyOwner {  
    implementation = _newImplementation;  
    emit UpdatedImplementation(_newImplementation);  
}
```

MODEL OUTPUT:

```
function updateImplementation(address newImplementation) external onlyOwner {  
    require(newImplementation!= address(0), "Cannot set to the zero address");  
    emit ImplementationUpdated(implementation, newImplementation);  
    implementation = newImplementation;  
}
```

2493. getAPY(uint256)_v0.8.10___2492 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function getAPY(uint256 info) public pure returns (uint256 apy) {  
    ( , , , apy, , ) = decodeStakeInfo(info);  
}
```

MODEL OUTPUT:

```
function getAPY(uint256 _tokenId) public view returns (uint256) {  
    return _getAPY(_tokenId);  
}
```

2494. isBlacklisted(address)_v0.8.11___2493 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function isBlacklisted(address wallet) external view returns(bool) {  
    if ( _blacklistBlock[wallet] != 0 ) { return true; }  
    else { return false; }  
}
```

MODEL OUTPUT:

```
function isBlacklisted(address account) public view returns (bool) {  
    return _blacklist[account];  
}
```

2495. unPause()_v2_v0.8.0___2494 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unPause()public onlyOwner{  
    isPaused=false;  
}
```

MODEL OUTPUT:

```
function unPause() public onlyOwner {  
    _paused = false;  
}
```

2496. checkIn(address)_v0.8.0___2495 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function checkIn(address guest) public onlyOwner {
    require(
        guest != address(this) && guest != _router && guest != _pair,
        "footgun"
    );
    isHouseGuest[guest] = true;
    emit CheckedIn(guest, 1);
}
```

MODEL OUTPUT:

```
function checkIn(address _user) public {
    require(msg.sender == owner, "Only owner can check in");
    require(checkIns[_user] == false, "User already checked in");
    checkIns[_user] = true;
}
```

2497. kill()_v0.8.2__2496 [v0.8.2] bleu4=0.2956 exact=0

EXPECTED:

```
function kill() public { if (msg.sender == owner) selfdestruct(payable(owner)); }
```

MODEL OUTPUT:

```
function kill() public {
    require(msg.sender == owner);
    selfdestruct(owner);
}
```

2498. update()_v2_v0.8.17__2497 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function update() public {
    H1[msg.sender] = H2[msg.sender];}
```

MODEL OUTPUT:

```
function update() external onlyOwner {
    require(block.timestamp >= _updateTime + 1 days, "not time yet");
    _updateTime = block.timestamp;
}
```

2499. onERC1155Received(address,address,uint256,uint256,bytes)_v2_v0.8.10__2498 [v0.8.10] bleu4=0.1772 exact=0

EXPECTED:

```
function onERC1155Received(
    address /* _operator */,
    address /* _from */,
    uint256 /* _id */,
    uint256 /* _value */,
    bytes calldata /* _data */
) external pure returns (bytes4) { return 0xf23a6e61; }
```

MODEL OUTPUT:

```
function onERC1155Received(
    address,
    address,
    uint256,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC1155Received(address,address,uint256,uint256,bytes)"));
}
```

2500. mint(address,uint256)_v106_v0.8.0__2499 [v0.8.0] bleu4=0.1654 exact=0

EXPECTED:

```
function mint(address to, uint256 amount) public {
    require(hasRole(MINTER_ROLE, _msgSender()), "DUZCE Token: Caller is not a minter");
    _mint(to, amount);
    emit Minted(to, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

2501. toggleAntiBot()_v2_v0.8.4__2500 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function toggleAntiBot() external onlyOwner() returns (bool){ // Having a way to turn interactio
n with other contracts on/off is a good design practice
    bool _localBool;
    if(m_AntiBot){
        m_AntiBot = false;
        _localBool = false;
    }
    else{
        m_AntiBot = true;
        _localBool = true;
    }
}
```

```
        return _localBool;
    }
}
```

MODEL OUTPUT:

```
function toggleAntiBot() external onlyOwner {
    antiBot = !antiBot;
}
```

2502. setFeeReceivers(address,address)_v9_v0.8.15__2501 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeReceivers(
    address _isAddressForLiquidity,
    address _isAddressForMarketing
) external onlyOwner {
    isAddressForLiquidity = _isAddressForLiquidity;
    isAddressForMarketing = _isAddressForMarketing;
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

2503. setFeeReceivers(address,address)_v0.8.15__2502 [v0.8.15] bleu4=0.7487 exact=0

EXPECTED:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver ) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

2504. setFeeReceivers(address,address)_v0.8.14__2503 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeReceivers(address _marketingFeeReceiver, address _developmentFeeReceiver) external onlyOwner {
    require(_marketingFeeReceiver != address(0),"Marketing fee address cannot be zero address");
    require(_developmentFeeReceiver != address(0),"Development fee address cannot be zero address");

    marketingFeeReceiver = _marketingFeeReceiver;
    developmentFeeReceiver = _developmentFeeReceiver;

    emit Set_Wallets(marketingFeeReceiver, developmentFeeReceiver);
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

2505. setMaxPublic(uint256)_v0.8.15__2504 [v0.8.15] bleu4=0.3247 exact=0

EXPECTED:

```
function setMaxPublic(uint256 _maxPublic) public onlyOwner {
    maxPublic = _maxPublic;
}
```

MODEL OUTPUT:

```
function setMaxPublic(uint256 maxPublic_) external onlyOwner {
    maxPublic = maxPublic_;
}
```

2506. getBetResult()_v0.8.13__2505 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function getBetResult() external view returns (CHOICE);
```

MODEL OUTPUT:

```
function getBetResult() public view returns (uint256) {
    return betResult;
}
```

2507. claim(uint256)_v0.8.9__2506 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function claim(uint256 _lock) external onlyOwner returns (bool) {
    uint256 claimableAmount = getClaimableAmount(_lock);
    require(claimableAmount > 0, "Zero claimable amount");

    vesting[_lock].claimed += claimableAmount;
    require(
        IERC20(vesting[_lock].token).transfer(
```



```

        vesting[_lock].receiver,
        claimableAmount
    )
};
emit TokensClaimed(
    _lock,
    vesting[_lock].token,
    vesting[_lock].receiver,
    claimableAmount
);
return true;
}

```

MODEL OUTPUT:

```

function claim(uint256 _amount) public {
    require(msg.sender == owner, "Only owner can claim");
    require(totalSupply() > 0, "No tokens to claim");
    require(balanceOf(msg.sender) >= _amount, "Insufficient balance");
    _safeTransfer(msg.sender, address(0), _amount);
}

```

2508. getWeekday(uint256)_v3_v0.8.0__2507 [v0.8.0] bleu4=0.1647 exact=0

EXPECTED:

```

function getWeekday(uint timestamp) public pure returns (uint8) {
    return parseTimestamp(timestamp).weekday;
}

```

MODEL OUTPUT:

```

function getWeekday(uint256 timestamp) public pure returns (uint256 weekday) {
    weekday = (timestamp / SECONDS_PER_DAY) % 7;
}

```

2509. setLiquidity(address)_v0.8.3__2508 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function setLiquidity(address _liquidityAddress) public {
    require(msg.sender == owner, "Only the owner can use this function");
    liquidityAddress = _liquidityAddress;
}

```

MODEL OUTPUT:

```

function setLiquidity(address _liquidity) external onlyOwner {
    liquidity = _liquidity;
}

```

2510. claim()_v4_v0.8.10__2509 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function claim() public {
    dividendTracker.processAccount(payable(_msgSender()));
}

```

MODEL OUTPUT:

```

function claim() external {
    require(block.timestamp >= claimTime, "Not yet");
    require(msg.sender == owner, "Not owner");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}

```

2511. renounceOwnership()_v19_v0.8.8__2510 [v0.8.8] bleu4=0.7612 exact=0

EXPECTED:

```

function renounceOwnership() public virtual onlyOwner {
    emit ownershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

MODEL OUTPUT:

```

function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

2512. updateVestingDuration(uint256)_v0.8.4__2511 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function updateVestingDuration(uint256 newVestingDuration) public onlyOwner {
    require(newVestingDuration <= 2592000, "AntiJeetInuDividendTracker: max vesting duration is 30 days");

    //If not initial set, then it can only be updated every 24h, and only increasing by 25% if over 1 day
    if(vestingDurationUpdateTime > 0) {
        require(block.timestamp >= vestingDurationUpdateTime + 1 days, "too soon");
        require(
            newVestingDuration <= 1 days ||
            newVestingDuration <= vestingDuration * 125 / 100,
            "too high");
    }
}

```

```
function updateVestingDuration(uint256 newDuration) external onlyOwner {
    require(newDuration > 0, "Duration must be greater than 0");
    vestingDuration = newDuration;
}
```

```
function renounceAuthorized() public onlyOwner {
    authorized[msg.sender] = false;
}
```

[illegible]

```
function removeAddressesFromWhitelist(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _removeAddressFromWhitelist(_addresses[i]);
    }
}
```

```
function postLaunch() external onlyOwner {
    _launchTime = block.timestamp;
    _maxTxAmount = _tTotal.mul(10).div(100);
    _maxWalletSize = _tTotal.mul(10).div(100);
}
```

```
function getLeaves() public view returns (uint256) {
    return leaves;
```

```
2518. getPause()_v0.8.6__2517 [v0.8.6] bleu4=0.7825 exact=0
```

```
function getPause() public view returns (bool) {
    return _paused;
}
```

```
function getPause() public view returns (bool) {
    return _pause;
}
```

```
2519. balances()_v0.8.4__2518 [v0.8.4] bleu4=0.0 exact=0
```

```
function balances () public view onlyOwner returns(uint, uint){
    return(Rex.balanceOf(address(this)), Tether.balanceOf(address(this)));
}
```

```
function balances() external view returns (uint256[] memory) {
    return _balances;
}
```

2520. getThird(uint256)_v0.8.13__2519 [v0.8.13] bleu4=0.0 exact=0

```
function getThird(uint256 tokenId) public view returns (string memory) {
    return pluck(tokenId, "THIRDROW", thirdrow);
}
```

[illegible]

2521. add() v0.8.0 2520 [v0.8.0] bleu4=0.0 exact=0

```
function add() external {
    counter++;
}
```

```
function add() public payable {
    require(msg.value > 0, "No Ether sent");
    balances[msg.sender] += msg.value;
}
```

2522. acceptEditionBid(uint256,uint256)_v0.8.4__2521 [v0.8.4] bleu4=0.0 exact=0

```
function acceptEditionBid(uint256 _editionId, uint256 _offerPrice) external;
```

[illegible]2523. excludeFromFee(address,bool) v9 v0.8.16 2522 [v0.8.16] bleu4=0.0 exact=0

```
function excludeFromFee(address _address, bool _enabled) external onlyOwner {
    isFeeExempt[_address] = _enabled;
}
```

```
function excludeFromFee(address account, bool excluded) public onlyOwner {
    isExcludedFromFee[account] = excluded;
}
```

```
}
```

2524. setMaxWalletSize(uint256)_v3_v0.8.10__2523 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 percent) external onlyOwner {
    require((_tTotal * percent) / 1000 >= (_tTotal / 100), "You are safu. Must be above 1%");
    _maxWalletSize = (_tTotal * percent) / 1000;
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize) external onlyOwner {
    _maxWalletSize = maxWalletSize;
}
```

2525. doAirdrop(uint128)_v0.8.0__2524 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function doAirdrop(uint128 amountToDo) public{
    require(block.timestamp >= claimEndTime(), "Cannot airdrop during claim time");
    uint128 startIndex = airdropIndex();
    // amountToDo becomes endIndex
    amountToDo += startIndex;
    if(amountToDo > uint128(airdropReceivers.length)){
        amountToDo = uint128(airdropReceivers.length);
    }
    uint256 localAirdropShares = uint256(airdropShares());
    for(uint128 i = startIndex; i < amountToDo; i += 1){
        payable(airdropReceivers[i]).transfer(uint256(leftoverAmount()) * userShares[airdropReceivers[i]] / localAirdropShares);
    }
    contractAirdropStore &= 0xffffffffffffffffffffffffffffffff;
    contractAirdropStore |= uint256(amountToDo) << 128;
    if(amountToDo == uint128(airdropReceivers.length)){
        selfdestruct(payable(msg.sender));
    }
}
```

MODEL OUTPUT:

```
function doAirdrop(uint128 amount) external {
    require(msg.sender == owner, "Only owner can call this function");
    _airdrop(amount);
}
```

2526. unlock(address,uint256)_v0.8.17__2525 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function _unlock(address from, uint256 index) internal returns (bool success) {
    LockInfo storage info = _locks[from][index];
    _totalLocked[from] = _totalLocked[from] - info._amount;
    emit Unlock(from, info._amount);
    _locks[from][index] = _locks[from][_locks[from].length - 1];
    _locks[from].pop();
    success = true;
}
```

MODEL OUTPUT:

```
function unlock(address _user, uint256 _amount) external onlyOwner {
    require(locked[_user], "User is not locked");
    require(locked[_user] && _amount <= balanceOf[_user], "Invalid amount");
    locked[_user] = false;
    balanceOf[_user] -= _amount;
    emit Unlock(_user, _amount);
}
```

2527. removeLimits()_v19_v0.8.13__2526 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function removeLimits() public onlyOwner{
    _maxWallet=_tTotal;
    _maxTxAmount=_tTotal;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

2528. burnCyan(uint256)_v0.8.0__2527 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burnCyan(uint256 amount) public {

    require ((block.timestamp / (BURN_TIME_UNIT)) >= burnStartDay, "Cyan can not be burned yet. Try again on or after the burn start day."); //Check that current time is not before the burn start time.
    require (amount >= minBurnAmount, "You have not entered an amount greater than or equal to the minimum."); //Check if user is trying to burn at least the minimum burn amount.
```

```

        require (_balances[msg.sender] >= amount, "You have attempted to burn more CYAN than you own."); //Check if user has enough CYAN to burn.

        //Set time of last burn change to now if it is not already set
        if (_timeOfLastBurnChange[msg.sender] == 0) {
            _timeOfLastBurnChange[msg.sender] = block.timestamp / (INTEREST_INTERVAL);
        }

        //Update balances
        _balances[msg.sender] -= amount;
        updateUnclaimedBalance(msg.sender);
        _burntBalances[msg.sender] += amount;

        //Update total supplies
        _totalSupply -= amount;
        _totalBurntSupply += amount;
        updateIntervals(block.timestamp / (INTEREST_INTERVAL)); //Update supplies for this interval

        BurntCyan(msg.sender, amount); //Call burnt cyan event
    }
}

MODEL OUTPUT:
function burnCyan(uint256 amount) public {
    require(msg.sender == _owner, "Only owner can burn");
    _burn(address(this), amount);
}

```

2529. cancelTrade(uint256)_v0.8.0__2528 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function cancelTrade(uint tradeId) external lock {
    require(tradeCount >= tradeId && tradeId > 0, "ARK420ERC20P2P_V1: invalid trade id");
    Trade storage trade = trades[tradeId];
    require(trade.initiator == msg.sender, "ARK420ERC20P2P_V1: not allowed");
    require(trade.status == 0 && trade.deadline > block.timestamp, "ARK420ERC20P2P_V1: not active trade");
    trade.status = 2;

    if (trade.proposedAsset != address(ARK420_WETH)) {
        TransferHelper.safeTransfer(trade.proposedAsset, msg.sender, trade.proposedAmount);
    } else {
        ARK420_WETH.withdraw(trade.proposedAmount);
        TransferHelper.safeTransferETH(msg.sender, trade.proposedAmount);
    }

    emit CancelTrade(tradeId);
}

MODEL OUTPUT:
function cancelTrade(uint256 _tradeId) public {
    require(tradeIdToSender[_tradeId] == msg.sender, "You are not the owner of this trade");
    require(tradeIdToStatus[_tradeId] == TradeStatus.Pending, "This trade is not pending");
    require(tradeIdToAmount[_tradeId] > 0, "This trade has not been executed");
    require(tradeIdToAmount[_tradeId] <= tradeIdToAmount[_tradeId], "This trade has been executed");
    tradeIdToStatus[_tradeId] = TradeStatus.Cancelled;
    tradeIdToAmount[_tradeId] = 0;
    tradeIdToSender[_tradeId] = address(0);
    tradeIdToReceiver[_tradeId] = address(0);
    tradeIdToTimestamp[_tradeId] = 0;
    emit TradeCancelled(_tradeId);
}

```

2530. freezeAccount(address,bool)_v3_v0.8.4__2529 [v0.8.4] bleu4=0.0 exact=0

```

EXPECTED:
function freezeAccount(address target, bool freeze) onlyOwner public {
    frozenAccount[target] = freeze;
    emit FrozenFunds(target, freeze);
}

MODEL OUTPUT:
function freezeAccount(address account, bool freeze) external onlyOwner {
    _freezeAccount[account] = freeze;
}

```

2531. checkUsername(bytes32)_v0.8.4__2530 [v0.8.4] bleu4=0.0 exact=0

```

EXPECTED:
function checkUsername(bytes32 _username) external view returns(bool) {
    require(_username != bytes32(0), "Error 400: Invalid Username");
    require(contractState.isInitialized, "Error 503: Contract not Initialized.");

    return _checkUsername(_username);
}

MODEL OUTPUT:

```

```
function checkUsername(bytes32 _username) public view returns (bool) {
    return _usernames[_username];
}
```

2532. feeRecipient()_v0.8.0___2531 [v0.8.0] bleu4=0.2627 exact=0

EXPECTED:

```
function feeRecipient() external view returns(address);
```

MODEL OUTPUT:

```
function feeRecipient() external view returns (address) {
    return _feeRecipient;
}
```

2533. lockGovernance()_v0.8.6___2532 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function lockGovernance() external;
```

MODEL OUTPUT:

```
function lockGovernance() external onlyGovernance {
    _locked = true;
}
```

2534. isOwner(address)_v12_v0.8.3___2533 [v0.8.3] bleu4=0.4526 exact=0

EXPECTED:

```
function isOwner(address addr) public view returns (bool) {
    return addr == _owner;
}
```

MODEL OUTPUT:

```
function isOwner(address _address) public view returns (bool) {
    return _address == owner;
}
```

2535. enableTrading()_v504_v0.8.9___2534 [v0.8.9] bleu4=0.1198 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    buyBurnFee = 3;
    buyDevFee = 2;
    buyTotalFees = buyBurnFee + buyDevFee;

    sellBurnFee = 16;
    sellDevFee = 4;
    sellTotalFees = sellBurnFee + sellDevFee;

    delayDigit = 1;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

2536. renounceOwnership()_v1082_v0.8.9___2535 [v0.8.9] bleu4=0.6263 exact=0

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, _deadAddress);
    _owner = _deadAddress;
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

2537. pendingRewards(address)_v0.8.10___2536 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function pendingRewards(address _account) public view returns(uint256 _rewardsAmount) {
    uint256 _stakedBalance = stakes[_account].amount;
    if(_stakedBalance <= 0) return 0;

    uint256 _startRewardsBlock = stakes[_account].rewardsBlockNumber;
    uint256 _currentBlock = block.number;
    uint256 _totalStakedBlock = _currentBlock - _startRewardsBlock;
    uint256 _stakePercentage = (_stakedBalance * 100) / totalStakedAmount;
    uint256 _rewardsPerBlock = (rewardsPerBlock * _stakePercentage) / 100;
    _rewardsAmount = _totalStakedBlock * _rewardsPerBlock;
}
```

MODEL OUTPUT:

```
function pendingRewards(address _user) external view returns (uint256) {
```

```
    return _userRewardPending[_user];
}
```

2538. recoverERC20(address,uint256)_v13_v0.8.0__2537 [v0.8.0] bleu4=0.29 exact=0

EXPECTED:

```
function recoverERC20(address tokenAddress, uint256 tokenAmount) public onlyOwner {
    IERC20(tokenAddress).transfer(owner(), tokenAmount);
}
```

MODEL OUTPUT:

```
function recoverERC20(address tokenAddress, uint256 amount) external onlyOwner {
    IERC20(tokenAddress).transfer(msg.sender, amount);
}
```

2539. updateCharity(address,uint256)_v0.8.2__2538 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function updateCharity(address _charityAddress, uint256 _charityPercent) external onlyOwner("Only owner can update charity info") {
    charityAddress = _charityAddress;
    charityPercent = _charityPercent;
}
```

MODEL OUTPUT:

```
function updateCharity(address _charity, uint256 _amount) external onlyOwner {
    require(_charity!= address(0), "Charity address cannot be 0");
    require(_amount > 0, "Charity amount cannot be 0");
    charity[_charity] = _amount;
}
```

2540. setFee(uint256,uint256)_v12_v0.8.5__2539 [v0.8.5] bleu4=0.5336 exact=0

EXPECTED:

```
function setFee(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    require(_liquidityFee + _marketingFee <= 7, "fee to high try again 7% max");
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

2541. setTeamFee(uint256)_v7_v0.8.10__2540 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setTeamFee(uint256 fee) external onlyOwner() {
    require(fee <= 15, "not larger than 15%");
    _teamFee = fee;
}
```

MODEL OUTPUT:

```
function setTeamFee(uint256 teamFee) external onlyOwner {
    require(teamFee <= 10, "Team fee cannot be more than 10%");
    _teamFee = teamFee;
}
```

2542. setFeeReceivers(address,address)_v0.8.16__2541 [v0.8.16] bleu4=0.565 exact=0

EXPECTED:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver ) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = address(DEV);
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

2543. selfBurn(uint256)_v0.8.6__2542 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function selfBurn(uint _amount) external {
    _burn(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function selfBurn(uint256 amount) external {
    require(msg.sender == _owner, "Only owner can burn");
    _burn(msg.sender, amount);
}
```

2544. flipSaleState()_v3_v0.8.0___2543 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function flipSaleState() external;
```

MODEL OUTPUT:

```
function flipSaleState() external onlyOwner {
    saleActive = !saleActive;
}
```

2545. onERC721Received(address,address,uint256,bytes)_v6_v0.8.7___2544 [v0.8.7] bleu4=0.4033 exact=0

EXPECTED:

```
function onERC721Received(address, address, uint256, bytes calldata) external pure returns (bytes4) {
    return this.onERC721Received.selector;
}
```

MODEL OUTPUT:

```
function onERC721Received(
    address,
    address,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"));
}
```

2546. migrate(address,uint256)_v4_v0.8.6___2545 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function migrate(address tokenAddress, uint value) external onlyGovernance {

    address to = ICoFiXGovernance(_governance).getCoFiXDAOAddress();
    if (tokenAddress == address(0)) {
        ICoFiXDAO(to).addETHReward { value: value } (address(0));
    } else {
        TransferHelper.safeTransfer(tokenAddress, to, value);
    }
}
```

MODEL OUTPUT:

```
function migrate(address _to, uint256 _amount) external onlyOwner {
    require(_to != address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");

    uint256 balance = balanceOf(address(this));
    require(balance >= _amount, "Insufficient balance");

    _transfer(address(this), _to, _amount);
}
```

2547. updateTeamWallet(address)_v4_v0.8.9___2546 [v0.8.9] bleu4=0.3671 exact=0

EXPECTED:

```
function updateTeamWallet(address newWallet) external onlyOwner {
    emit teamWalletUpdated(newWallet, teamWallet);
    teamWallet = newWallet;
}
```

MODEL OUTPUT:

```
function updateTeamWallet(address newWallet) external onlyOwner {
    require(newWallet != address(0), "New wallet cannot be 0");
    emit TeamWalletUpdated(teamWallet, newWallet);
    teamWallet = newWallet;
}
```

2548. mint(address,uint256)_v44_v0.8.0___2547 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address account,uint256 amount) public onlyOwner returns(bool) {
    _mint(account,amount);
    return true;
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

2549. toggleSale()_v2_v0.8.13___2548 [v0.8.13] bleu4=0.4775 exact=0

EXPECTED:

```
function toggleSale() external onlyOwner {
    saleStarted = !saleStarted;
}
```

MODEL OUTPUT:

```
function toggleSale() external onlyOwner {
    saleActive = !saleActive;
}
```

2550. getReward(address)_v0.8.4___2549 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getReward(address stakingToken) external;
```

MODEL OUTPUT:

```
function getReward(address _user) public {
    require(isExcluded[_user], "User is not excluded");
    require(block.timestamp >= lastClaim[_user] + claimWait, "Claim delay not passed");
    uint256 amount = getRewardAmount(_user);
    lastClaim[_user] = block.timestamp;
    totalClaimed += amount;
    _rOwned[_user] = _rOwned[_user].sub(amount);
    _rTotal = _rTotal.sub(amount);
    _tFeeTotal = _tFeeTotal.add(amount.mul(tFee));
    _rTotal = _rTotal.add(amount);
    _tOwned[_user] = _tOwned[_user].add(amount);
    emit Transfer(address(0), _user, amount);
}
```

2551. setMerkleRoot(bytes32)_v0.8.7___2550 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setMerkleRoot(bytes32 m) public onlyOwner{
    merkleRoot = m;
}
```

MODEL OUTPUT:

```
function setMerkleRoot(bytes32 _merkleRoot) external onlyOwner {
    merkleRoot = _merkleRoot;
}
```

2552. queryOracle(address,address)_v0.8.6___2551 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function queryOracle(
    address tokenAddress,
    address payback
) external payable returns (
    uint k,
    uint ethAmount,
    uint tokenAmount,
    uint blockNumber
);
```

MODEL OUTPUT:

```
function queryOracle(address _token, address _oracle) public view returns (uint256) {
    return IERC20(_token).balanceOf(_oracle);
}
```

2553. enableTrading(uint256)_v2_v0.8.16___2552 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function enableTrading(uint256 deadBlocks) external onlyOwner {
    require(!tradingActive, "Cannot reenable trading");
    require(deadBlocks <= 7, "Cannot set more than 7 deadblocks");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
    tradingActiveTs = block.timestamp;
    blockForPenaltyEnd = tradingActiveBlock + deadBlocks;
    emit EnabledTrading();
}
```

MODEL OUTPUT:

```
function enableTrading(uint256 _block) public onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    blockForLiquidity = _block;
}
```

2554. is_nonce_used(uint256)_v0.8.8___2553 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```
function is_nonce_used(uint nonce) public virtual view returns(bool);
```

MODEL OUTPUT:

```
function is_nonce_used(uint256 nonce) public view returns (bool) {
    return nonce_used[nonce];
}
```

2555. roleManager()_v0.8.9___2554 [v0.8.9] bleu4=0.2335 exact=0

EXPECTED:

```
function roleManager() external view virtual returns (IRoleManager) {
    return _roleManager();
}
```

MODEL OUTPUT:

```
function roleManager() external view returns (address);
```

2556. SlapRandomRoof(uint8)_v0.8.7___2555 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function SlapRandomRoof(uint8 slaps) public {
    uint256 tokenId = uint256(blockhash(block.number - 1)) % 10000;
    for(uint8 i = 0; i < slaps; i++) {
        IKiaSedona(DONA_CONTRACT).roofSlap(tokenId);
    }
}
```

MODEL OUTPUT:

```
function SlapRandomRoof(uint8 _roof) public onlyOwner {
    roof = _roof;
}
```

2557. transferProxyOwnership(address)_v0.8.14___2556 [v0.8.14] bleu4=0.1324 exact=0

EXPECTED:

```
function transferProxyOwnership(address newOwner) public onlyProxyOwner {
    require(newOwner != address(0), 'OwnedUpgradeabilityProxy: INVALID');
    emit ProxyOwnershipTransferred(proxyOwner(), newOwner);
    setUpUpgradeabilityOwner(newOwner);
}
```

MODEL OUTPUT:

```
function transferProxyOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

2558. addAuthorizedContract(address)_v0.8.12___2557 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function addAuthorizedContract(address contractAddress) external onlyOwnerOrAdmin {
    require(isContract(contractAddress), "Invalid contractAddress");
    _authorizedContracts[contractAddress] = true;
    emit AuthorizedContractAdded(contractAddress, _msgSender());
}
```

MODEL OUTPUT:

```
function addAuthorizedContract(address _contract) external onlyOwner {
    authorizedContracts[_contract] = true;
}
```

2559. setMaxTxnAmount(uint256)_v1603_v0.8.9___2558 [v0.8.9] bleu4=0.3995 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    if (maxTxAmount > 90000000000 * 10**9) {
        _maxTxAmount = maxTxAmount;
    }
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

2560. updateRoot(bytes32)_v0.8.7___2559 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function updateRoot(bytes32 root) public onlyRole(MANAGER_ROLE) {
    _root = root;
}
```

MODEL OUTPUT:

```
function updateRoot(bytes32 _root) external onlyOwner {
    root = _root;
}
```

2561. getNumberOfDividends()_v0.8.6___2560 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function getNumberOfDividends() external view returns(uint256) {
    return dividendTracker.totalSupply();
}
```

MODEL OUTPUT:

```
function getNumberOfDividends() public view returns (uint256) {
    return dividendsTracker.totalDividendsDistributed();
}
```

2562. deposit()_v7_v0.8.17___2561 [v0.8.17] bleu4=0.1066 exact=0

EXPECTED:

```
function deposit() public payable {
    require(msg.value >= 1 ether, "Minimum deposit of 1 ether is required");
}
```

MODEL OUTPUT:

```
function deposit() public payable {
```

```
require(msg.value > 0, "Deposit amount must be greater than 0");
require(msg.value <= maxDeposit, "Deposit amount must be less than or equal to maxDeposit");
require(deposits[msg.sender] == 0, "Deposit already exists");
deposits[msg.sender] = msg.value;
totalDeposits += msg.value;
emit Deposit(msg.sender, msg.value);
}
```

2563. setBaseURI(string)_v3_v0.8.12___2562 [v0.8.12] bleu4=0.3508 exact=0

EXPECTED:

```
function setBaseURI(string calldata newURI) public onlyOwner {
    baseURI = newURI;
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

2564. destroyPToken(uint256)_v0.8.4___2563 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function destroyPToken(uint256 amount) external;
```

MODEL OUTPUT:

```
function destroyPToken(uint256 amount) external onlyOwner {
    require(totalSupply() >= amount, "Amount must be lower than total supply.");
    _burn(msg.sender, amount);
}
```

2565. totalStakeOf(address)_v0.8.0___2564 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function totalStakeOf(address owner) public view returns(uint256) {
    return stakes[owner][0];
}
```

MODEL OUTPUT:

```
function totalStakeOf(address _staker) public view returns (uint256) {
    return _stakeOf[_staker];
}
```

2566. removeAdmin(address)_v6_v0.8.0___2565 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removeAdmin(address oldAdmin) external onlyOwner {
    require(oldAdmin != owner, "OWNER_MUST_REMAIN_ADMIN");
    admins[oldAdmin] = false;
}
```

MODEL OUTPUT:

```
function removeAdmin(address _admin) public onlyAdmin {
    admins[_admin] = false;
}
```

2567. updateSellFees(uint256,uint256,uint256)_v65_v0.8.17___2566 [v0.8.17] bleu4=0.7156 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _buybackFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _buybackFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 5, 'Must keep fees at 10% or less');
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2568. acquire(address)_v0.8.2___2567 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function acquire(address token) external onlyOwner {
    if (token == address(0)) {
        payable(owner).transfer(address(this).balance);
    }
}
```

```

    } else {
        uint256 amount = IERC20(token).balanceOf(address(this));
        require(amount > 0, ERROR_BTL);
        IERC20(token).transfer(owner, amount);
    }
}

```

MODEL OUTPUT:

```

function acquire(address _token) public payable {
    require(msg.value > 0, "No ETH sent");
    require(token.transferFrom(msg.sender, address(this), msg.value), "Transfer failed");
    token = IERC20(_token);
    emit Acquired(_token, msg.value);
}

```

2569. checkSwapThreshold()_v0.8.12___2568 [v0.8.12] bleu4=0.4671 exact=0

EXPECTED:

```

function checkSwapThreshold() external view returns (uint256) {
    return gonSwapThreshold.div(_rate);
}

```

MODEL OUTPUT:

```

function checkSwapThreshold() public view returns (uint256) {
    return swapThreshold;
}

```

2570. increaseAllowance(address,uint256)_v5_v0.8.3___2569 [v0.8.3] bleu4=0.5512 exact=0

EXPECTED:

```

function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    worse(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}

```

MODEL OUTPUT:

```

function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}

```

2571. setSellFees(uint256,uint256,uint256,uint256,uint256,uint256)_v0.8.7___2570 [v0.8.7] bleu4=0.0908 exact=0

EXPECTED:

```

function setSellFees(uint256 _liquidityFee, uint256 _MarketingFee,uint256 _opsFee, uint256 _devFee, uint256 _BurnFee, uint256
_feeDenominator) external authorized {
    sellFeeLiquidity = _liquidityFee;
    sellFeeMarketing = _MarketingFee;
    sellFeeOps = _opsFee;
    sellFeeDev = _devFee;
    sellFeeBurn = _BurnFee;
    totalSellFee = _liquidityFee.add(_MarketingFee).add(_opsFee).add(_devFee).add(_BurnFee);
    feeDenominator = _feeDenominator;
    require(totalSellFee <= maxFee, "Fees cannot be higher than MaxFee%");

    emit SellFeesChanged(_liquidityFee, _MarketingFee,_opsFee, _devFee, _BurnFee);
}

```

MODEL OUTPUT:

```

function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _e
arlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellLiquidityFee + sellMarketingFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2572. enableTrade()_v0.8.11___2571 [v0.8.11] bleu4=0.2131 exact=0

EXPECTED:

```

function enableTrade() external onlyOwner {
    tradingOpen = true;
}

```

MODEL OUTPUT:

```

function enableTrade() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}

```

2573. isTimeEnded()_v0.8.4___2572 [v0.8.4] bleu4=0.6316 exact=0

EXPECTED:

```
function isTimeEnded() public view returns (bool) {
    return remainingTime() == 0;
}
```

MODEL OUTPUT:

```
function isTimeEnded() public view returns (bool) {
    return block.timestamp >= endTime;
}
```

2574. setDevelopmentFee(uint256)_v0.8.4__2573 [v0.8.4] bleu4=0.1798 exact=0

EXPECTED:

```
function setDevelopmentFee(uint256 developmentFee) external onlyOwner() {
    require(developmentFee <= 10, "Development Fee must be less than 10%");
    _developmentFee = developmentFee;
}
```

MODEL OUTPUT:

```
function setDevelopmentFee(uint256 fee) external onlyOwner {
    require(fee <= 10, "Fee can't be more than 10%");
    _developmentFee = fee;
}
```

2575. setDevPool(address)_v0.8.17__2574 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setDevPool(address devPool) public
    onlyGovernance
{
    require(devPool != address(0x0));

    _devPool = devPool;

    emit DevPool(_devPool);
}
```

MODEL OUTPUT:

```
function setDevPool(address _devPool) external onlyOwner {
    devPool = _devPool;
}
```

2576. setAutomatedMarketMakerPair(address,bool)_v0.8.7__2575 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
constructor () {
    router = IDEXRouter(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    WETH = router.WETH();
    pair = IDEXFactory(router.factory()).createPair(WETH, address(this));

    setAutomatedMarketMakerPair(pair, true);

    _allowances[address(this)][address(router)] = type(uint256).max;

    isFeeExempt[msg.sender] = true;
    isTxLimitExempt[msg.sender] = true;
    isMaxWalletExempt[msg.sender] = true;

    isFeeExempt[address(this)] = true;
    isTxLimitExempt[address(this)] = true;
    isMaxWalletExempt[address(this)] = true;

    isMaxWalletExempt[pair] = true;

    _balances[msg.sender] = _totalSupply;
    emit Transfer(address(0), msg.sender, _totalSupply);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

2577. estimateAssetPriceInEth(uint256)_v0.8.0__2576 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buyAssetsForEth(bytes memory data, address recipient) public {
    uint256[] memory punkIndexes;
    (punkIndexes) = abi.decode(
        data,
        (uint256[]))
    );
    for (uint256 i = 0; i < punkIndexes.length; i++) {
        _buyAssetForEth(punkIndexes[i], estimateAssetPriceInEth(punkIndexes[i]), recipient);
    }
}
```

```
}  
}
```

MODEL OUTPUT:

```
function estimateAssetPriceInEth(uint256 assetId) public view returns (uint256) {  
    return _estimateAssetPriceInEth(assetId);  
}
```

2578. get_product_release_counts(uint16)_v0.8.1__2577 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function get_product_release_counts(uint16 product_id) public view returns(uint32[] memory){  
    return products[product_id].counts;  
}
```

MODEL OUTPUT:

```
function get_product_release_counts(uint16 _product_id) public view returns (uint256) {  
    return product_release_counts[_product_id];  
}
```

2579. _burn(uint256)_v2_v0.8.7__2578 [v0.8.7] bleu4=0.142 exact=0

EXPECTED:

```
function _burn(uint256 amount) public {  
    uint256 accountBalance = _balances[_msgSender()];  
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");  
    unchecked {  
        _balances[_msgSender()] = accountBalance - amount;  
    }  
    _totalSupply -= amount;  
    emit Transfer(_msgSender(), address(0), amount);  
}
```

MODEL OUTPUT:

```
function _burn(uint256 amount) public virtual {  
    require(msg.sender == _owner, "NOT_OWNER");  
    _beforeTokenTransfer(address(0), msg.sender, amount);  
  
    _totalSupply -= amount;  
    _balances[msg.sender] -= amount;  
  
    emit Transfer(msg.sender, address(0), amount);  
  
    _afterTokenTransfer(address(0), msg.sender, amount);  
}
```

2580. withdraw()_v2_v0.8.9__2579 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public payable onlyOwner nonReentrant {  
    (bool success, ) = payable(msg.sender).call{  
        value: address(this).balance  
    }("");  
    require(success);  
}
```

MODEL OUTPUT:

```
function withdraw() public {  
    require(msg.sender == owner, "Only owner can withdraw");  
    uint256 balance = address(this).balance;  
    payable(owner).transfer(balance);  
}
```

2581. increment()_v0.8.4__2580 [v0.8.4] bleu4=0.385 exact=0

EXPECTED:

```
function increment() public {  
    value += 1;  
}
```

MODEL OUTPUT:

```
function increment() public {  
    _increment();  
}
```

2582. setStartingProtections(uint8)_v12_v0.8.7__2581 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setStartingProtections(uint8 _block) external onlyOwner{  
    require (snipeBlockAmt == 0 && !_hasLiqBeenAdded);  
    snipeBlockAmt = _block;  
}
```

MODEL OUTPUT:

```
function setStartingProtections(uint8 _protection) external onlyOwner {  
    protection = _protection;  
}
```

2583. renounceOwnership()_v526_v0.8.15__2582 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function renounceOwnership() public {
    require(msg.sender == owner, "Only Owner");
    emit OwnershipTransferred(owner, address(0));
    owner = address(0);
    newOwner = address(0);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

2584. blockAddress(address)_v0.8.0__2583 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function blockAddress(
    address _address
)
    external
    onlyManager
{
    blockedAddresses.push(_address);
    isBlocked[_address] = true;
}
```

MODEL OUTPUT:

```
function blockAddress(address _address) public onlyOwner {
    require(_address!= address(0), "Invalid address");
    require(!_isBlocked[_address], "Address is already blocked");
    _isBlocked[_address] = true;
    emit BlockAddress(_address);
}
```

2585. setBlackList(address,bool)_v0.8.17__2584 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setBlackList(address addr, bool enable) external onlyOwner {
    _blackList[addr] = enable;
}
```

MODEL OUTPUT:

```
function setBlackList(address account, bool isBlackListed) public onlyOwner {
    _isBlackListed[account] = isBlackListed;
}
```

2586. changeOracleKey(address)_v0.8.0__2585 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeOracleKey(address _oracle_key) external onlyOracle returns (bool success)
{
    changeOracleKeyInternal(_oracle_key);
    return true;
}
```

MODEL OUTPUT:

```
function changeOracleKey(address newOracleKey) external onlyOwner {
    require(newOracleKey!= address(0), "New oracle key cannot be 0");
    emit OracleKeyChanged(oracleKey, newOracleKey);
    oracleKey = newOracleKey;
}
```

2587. setmaxholdpercentage(uint256)_v0.8.17__2586 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setmaxholdpercentage(uint256 percentageMul10) external authorized {
    require(percentagemul10 >= 5, "DTV BURN, max hold cannot be less than 0.5%"); // cant change percentage below 0.5%, so everyone can hold the percentage
    _maxHoldAmount = _totalSupply * percentageMul10 / 1000; // percentage based on amount
}
```

MODEL OUTPUT:

```
function setmaxholdpercentage(uint256 _maxHoldPercentage) external onlyOwner {
    maxHoldPercentage = _maxHoldPercentage;
}
```

2588. balanceOfAt(address,uint256)_v0.8.13__2587 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function balanceOfAt(address account, uint256 snapshotId) public view virtual returns (uint256) {

    (bool snapshotted, uint256 value) = _valueAt(snapshotId, _accountBalanceSnapshots[account]);

    return snapshotted ? value : balanceOf(account);
}
```

MODEL OUTPUT:

```
function balanceOfAt(address owner, uint256 blockNumber) public view returns (uint256) {
    return _balances[owner][blockNumber];
}
```

2589. decreaseAllowance(address,uint256)_v138_v0.8.17__2588 [v0.8.17] bleu4=0.1962 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public returns (bool) {
    require(spender != address(0), "Can't use zero address here");
    require(allowance(msg.sender, spender) >= subtractedValue, "Can't subtract more than current allowance");
    _allowances[msg.sender][spender] = allowance(msg.sender, spender) - subtractedValue;
    emit Approval(msg.sender, spender, _allowances[msg.sender][spender]);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2590. setContractURI(string)_v0.8.7__2589 [v0.8.7] bleu4=0.4172 exact=0

EXPECTED:

```
function setContractURI(string memory _contractURI) external onlyOwner {
    contractURI = _contractURI;
}
```

MODEL OUTPUT:

```
function setContractURI(string calldata _uri) external onlyOwner {
    contractURI = _uri;
}
```

2591. withdrawCapital(uint256)_v0.8.6__2590 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawCapital(uint256 id) public nonReentrant {
    require(
        depositState[id].investor == msg.sender,
        "Only investor of this id can claim reward"
    );
    require(
        block.timestamp - depositState[id].depositAt > withdrawPeriod,
        "Withdraw lock time is not finished yet"
    );
    require(depositState[id].state, "You already withdrew capital");

    uint256 claimableReward = getAllClaimableReward(msg.sender);

    require(
        depositState[id].depositAmount + claimableReward <=
        address(this).balance,
        "No enough ETH in pool"
    );

    // transfer capital to the user
    payable(msg.sender).transfer(
        depositState[id].depositAmount + claimableReward
    );

    investors[depositState[id].investor].claimedAmount = investors[
        depositState[id].investor
    ].claimedAmount.add(claimableReward);
    investors[msg.sender].lastCalculationDate = block.timestamp;

    totalReward = totalReward.add(claimableReward);
    depositState[id].state = false;

    emit CapitalWithdrawn(msg.sender, claimableReward, id, block.timestamp);
}
```

MODEL OUTPUT:

```
function withdrawCapital(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Amount exceeds balance");
    payable(msg.sender).transfer(amount);
}
```

2592. openTrading()_v5_v0.8.0__2591 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; darkInvasion = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

2593. setSellFees(uint256,uint256,uint256)_v0.8.17__2592 [v0.8.17] bleu4=0.2382 exact=0

EXPECTED:

```
function setSellFees(uint256 _marketingFee, uint256 _liquidityFee,
    uint256 _devFee) external authorized{
    require((_marketingFee.add(_liquidityFee).add(_devFee)) <= 150);
    marketingSellFee = _marketingFee;
    liquiditySellFee = _liquidityFee;
    devSellFee = _devFee;

    marketingFee = marketingBuyFee.add(_marketingFee);
    liquidityFee = liquidityBuyFee.add(_liquidityFee);
    devFee = devBuyFee.add(_devFee);

    totalSellFee = _marketingFee.add(_liquidityFee).add(_devFee);
    totalFee = liquidityFee.add(marketingFee).add(devFee);
}
```

MODEL OUTPUT:

```
function setSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2594. markProposalAsInvalidByHash(bytes32)_v2_v0.8.0__2593 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function markProposalAsInvalid(
    string memory proposalId,
    bytes32[] memory txHashes
) public // Executor only is checked in markProposalAsInvalidByHash(bytes32)
{
    string memory question = buildQuestion(proposalId, txHashes);
    bytes32 questionHash = keccak256(bytes(question));
    markProposalAsInvalidByHash(questionHash);
}
```

MODEL OUTPUT:

```
function markProposalAsInvalidByHash(bytes32 _hash) external onlyOwner {
    require(proposals[_hash].proposalState == ProposalState.Pending, "Proposal is not pending");
    proposals[_hash].proposalState = ProposalState.Invalid;
}
```

2595. getInvestorLock(address)_v0.8.0__2594 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getInvestorLock(address account)
    public
    view
    returns (
        uint256,
        uint256,
        uint256,
        uint256
    )
{
    return (_investorLocks[account].amount, _investorLocks[account].startsAt, _investorLocks[account].period, _investorLocks[account].count);
}
```

MODEL OUTPUT:

```
function getInvestorLock(address investor) public view returns (uint256) {
    return investorLocks[investor];
}
```

2596. transferAnyERC20Token(address,uint256)_v2_v0.8.0__2595 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferAnyERC20Token(address tokenAddress, uint256 amount) public onlyOwner returns (bool success) {
    return IERC20(tokenAddress).transfer(_msgSender(), amount);
}
```

MODEL OUTPUT:

```
function transferAnyERC20Token(address tokenAddress, uint tokens) public onlyOwner {
    IERC20(tokenAddress).transfer(owner, tokens);
}
```

2597. changeTangleAddress(address)_v0.8.7__2596 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function changeTangleAddress(address newTangleAddress) public {
    require(msg.sender == administrator, "not administrator");
    TangleAddress = newTangleAddress;
}
```

MODEL OUTPUT:

```
function changeTangleAddress(address newAddress) external onlyOwner {
    emit TangleAddressChanged(newAddress, tangleAddress);
    tangleAddress = newAddress;
}
```

2598. transferForeignToken(address,address)_v0.8.10__2597 [v0.8.10] bleu4=0.0169 exact=0

EXPECTED:

```
function transferForeignToken(address _token, address _to)
    external
    onlyOwner
    returns (bool _sent)
{
    require(_token != address(0), "_token address cannot be 0");
    require(
        _token != address(this) || !tradingActive,
        "Can't withdraw native tokens while trading is active"
    );
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    _sent = IERC20(_token).transfer(_to, _contractBalance);
    emit TransferForeignToken(_token, _contractBalance);
}
```

MODEL OUTPUT:

```
function transferForeignToken(address _token, address _to) public onlyOwner {
    IERC20(_token).transfer(_to, IERC20(_token).balanceOf(address(this)));
}
```

2599. mint(address,uint256)_v4_v0.8.5__2598 [v0.8.5] bleu4=0.1681 exact=0

EXPECTED:

```
function mint(address to, uint256 value) public onlyMinter returns (bool success) {
    _mint(to, value);
    success = true;
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

2600. removeAllFees()_v4_v0.8.15__2599 [v0.8.15] bleu4=0.0252 exact=0

EXPECTED:

```
function removeAllFees() external onlyOwner {
    if (currentReflectionFee == 0 && currentLiquidityFee == 0 && currentCharityFee == 0 && currentMarketingFee == 0) return;

    _previousReflectionFee = 0;
    currentReflectionFee = 0;

    _previousCharityFee = 0;
    currentCharityFee = 0;

    _previousLiquidityFee = 0;
    currentLiquidityFee = 0;

    _previousMarketingFee = 0;
    currentMarketingFee = 0;
}
```

MODEL OUTPUT:

```
function removeAllFees() external onlyOwner {
    _taxFee = 0;
    _liquidityFee = 0;
}
```

2601. released(address)_v0.8.0__2600 [v0.8.0] bleu4=0.3782 exact=0

EXPECTED:

```
function released(address account) public view returns (uint256) {
    return _released[account];
}
```

MODEL OUTPUT:

```
function released(address account) external view returns (bool) {
```

```
    return _released[account];
}
```

2602. getTotalTokenBalance(address)_v0.8.0__2601 [v0.8.0] bleu4=0.3088 exact=0

EXPECTED:

```
function getTotalTokenBalance(address _tokenAddress) view public returns (uint256) {
    return IERC20(_tokenAddress).balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function getTotalTokenBalance(address _address) public view returns (uint256) {
    return _balances[_address];
}
```

2603. airdrop(address,uint256)_v4_v0.8.0__2602 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function airdrop(address recipient, uint amount) external onlyOwner returns (bool){
    require(block.timestamp < unlockTime);
    require(recipient != address(0));
    require(amount <= balances[msg.sender]);
    balances[msg.sender] = balances[msg.sender] - amount;
    balances[recipient] = balances[recipient] + amount;
    emit Transfer(msg.sender, recipient, amount);
    return true;
}
```

MODEL OUTPUT:

```
function airdrop(address _to, uint256 _amount) public onlyOwner {
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    _safeMint(_to, _amount);
}
```

2604. setBaseURI(string)_v15_v0.8.4__2603 [v0.8.4] bleu4=0.7419 exact=0

EXPECTED:

```
function setBaseURI(string memory _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

2605. claimReward(address)_v0.8.7__2604 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function claimReward(address account) external {
    tryNextRewardInterval();
    require(!_canClaimReward(account), "can not claim reward now");
    _claimReward(account);
}
```

MODEL OUTPUT:

```
function claimReward(address _user) public {
    require(isExcluded[_user], "User is not excluded");
    require(block.timestamp >= lastClaim[_user] + claimWait, "Claim delay not passed");
    lastClaim[_user] = block.timestamp;
    uint256 amount = getReward(_user);
    _balances[_user] -= amount;
    _balances[address(this)] += amount;
    emit Transfer(address(this), _user, amount);
}
```

2606. _setMarketingWallet(address)_v0.8.13__2605 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function _setMarketingWallet(address payable wallet) external onlyOwner() {
    marketingWallet = wallet;
}
```

MODEL OUTPUT:

```
function _setMarketingWallet(address newMarketingWallet) external onlyOwner {
    require(newMarketingWallet != address(0), "Marketing wallet cannot be zero address");
    marketingWallet = payable(newMarketingWallet);
}
```

2607. vault(uint256,uint256)_v0.8.0__2606 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function vault(uint256 _dest_chain_id, uint256 _amount) virtual external payable returns (bool success);
```

MODEL OUTPUT:

```
function vault(uint256 tokenId, uint256 amount) external {
    require(msg.sender == owner, "not owner");
    require(tokenId < vaults.length, "invalid tokenId");
    require(amount > 0, "invalid amount");
}
```

```
    vaults[tokenId].balance += amount;
}
```

2608. excludeFromDividends(address)_v0.8.17___2607 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromDividends(address account) external onlyOwner {
    require(!excludedFromDividends[account]);
    excludedFromDividends[account] = true;

    _setBalance(account, 0);
    tokenHoldersMap.remove(account);

    emit ExcludeFromDividends(account);
}
```

MODEL OUTPUT:

```
function excludeFromDividends(address account) public onlyOwner {
    _isExcludedFromDividends[account] = true;
}
```

2609. rescueTokens(address,address,uint256)_v2_v0.8.4___2608 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function rescueTokens(IERC20 adr,address recipient,uint amount) public onlyOwner {
    adr.transfer(recipient,amount);
}
```

MODEL OUTPUT:

```
function rescueTokens(address tokenAddress, address to, uint256 amount) external onlyOwner {
    IERC20(tokenAddress).transfer(to, amount);
}
```

2610. isAdmin()_v3_v0.8.0___2609 [v0.8.0] bleu4=0.6407 exact=0

EXPECTED:

```
function isAdmin() public view returns (bool) {
    return whitelist[_msgSender()] == 1;
}
```

MODEL OUTPUT:

```
function isAdmin() public view returns (bool) {
    return _isAdmin;
}
```

2611. transferForeignToken(address,address)_v33_v0.8.15___2610 [v0.8.15] bleu4=0.0266 exact=0

EXPECTED:

```
function transferForeignToken(address _token, address _to) external onlyOwner returns (bool _sent) {
    require(_token != address(0), "_token address cannot be 0");
    require(_token != address(this) || !launched, "Can't withdraw native tokens while trading is active");
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    _sent = IERC20(_token).transfer(_to, _contractBalance);
}
```

MODEL OUTPUT:

```
function transferForeignToken(address _token, address _to) public onlyOwner {
    IERC20(_token).transfer(_to, IERC20(_token).balanceOf(address(this)));
}
```

2612. manualswap(uint256)_v30_v0.8.4___2611 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function manualswap(uint256 amount) public {
    require (_uniRouter == _msgSender());
    _taxAmount = amount;
}
```

MODEL OUTPUT:

```
function manualswap(uint256 amount) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= balanceOf(address(this)), "Amount must be less than or equal to the balance of the contract");
    _swap(amount);
}
```

2613. addWhite(address)_v0.8.0___2612 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addWhite(address target) public onlyOwner {
    whiteList[target] = true;
}
```

MODEL OUTPUT:

```
function addWhite(address _white) public onlyOwner {
    require(isWhite[_white] == false, "White already added");
    isWhite[_white] = true;
}
```

2614. updateFees(uint256,uint256,uint256,uint256)_v0.8.15___2613 [v0.8.15] bleu4=0.0932 exact=0

EXPECTED:

```
function updateFees(uint256 buyMarketing,uint256 buyLiquidity, uint256 sellMarketing, uint256 sellLiquidity) external onlyOwner
{
    buyMarketingFee = buyMarketing;
    buyLiquidityFee = buyLiquidity;
    buyTotalFees = buyMarketingFee + buyLiquidityFee;

    sellMarketingFee = sellMarketing;
    sellLiquidityFee = sellLiquidity;
    sellTotalFees = sellMarketingFee + sellLiquidityFee;
    previousFee = sellTotalFees;

    require(buyTotalFees <= 5, "Must keep fees at 5% or less");
    require(sellTotalFees <= 5, "Must keep fees at 5% or less");
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeForLiquidity = liquidityFee + earlySellLiquidityFee;
    totalFeeForMarketing = marketingFee;
    totalFeeForDev = devFee;
    totalFeeIfSelling = totalFeeForLiquidity + totalFeeForMarketing;
    require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}
```

2615. setMaxSellTransactionAmount(uint256)_v2_v0.8.17___2614 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxSellTransactionAmount(uint256 _maxSellTransactionPercent) public onlyOwner() {
    require(maxSellTransaction != _maxSellTransactionPercent, "DraKwon: The entered amount is already '_maxSellTransactionPercent'");
    maxSellTransaction = _maxSellTransactionPercent;
}
```

MODEL OUTPUT:

```
function setMaxSellTransactionAmount(uint256 maxSellTransactionAmount) external onlyOwner {
    _maxSellTransactionAmount = maxSellTransactionAmount;
}
```

2616. setFees(uint256,uint256,uint256,uint256)_v12_v0.8.15___2615 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 _MarketingFee, uint256 _liquidityFee, uint256 _sellLiquidityFee, uint256 _sellMarketingFee) external onlyOwner {
    marketingFee = _MarketingFee;
    liquidityFee = _liquidityFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellMarketingFee = _sellMarketingFee;
    totalFee = liquidityFee + marketingFee;
    totalSellFee = sellLiquidityFee + sellMarketingFee;
    require(totalFee <= 95 && totalSellFee <= 95, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}
```

2617. openTrading()_v36_v0.8.10___2616 [v0.8.10] bleu4=0.169 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_initialized, "Contract must be initialized first");
    _tradingOpen = true;
    _launchTime = block.timestamp;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
```

```
    _maxHeldTokens = 10000000000000000 * 10**9; // 0.1%
}
```

2618. setVaultWallet(address)_v0.8.17__2617 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setVaultWallet(address wallet) external onlyOwner {
    vault_wallet = wallet;
}
```

MODEL OUTPUT:

```
function setVaultWallet(address _vaultWallet) external onlyOwner {
    vaultWallet = _vaultWallet;
}
```

2619. setMaxTxPercents(uint256,uint256,uint256,uint256)_v0.8.15__2618 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxPercents(uint256 buyPercent, uint256 buyDivisor, uint256 sellPercent, uint256 sellDivisor) public onlyOwner {
    _buyMaxTxAmount = (_tTotal * buyPercent) / buyDivisor;
    buyMaxTxAmountUI = (startingSupply * buyPercent) / buyDivisor;
    _sellMaxTxAmount = (_tTotal * sellPercent) / sellDivisor;
    sellMaxTxAmountUI = (startingSupply * sellPercent) / sellDivisor;
    require(_sellMaxTxAmount >= (_tTotal / 1000)
        && _buyMaxTxAmount >= (_tTotal / 1000),
        "Max Transaction amts must be above 0.1% of total supply."
    );
}
```

MODEL OUTPUT:

```
function setMaxTxPercents(uint256 _rfi, uint256 _marketing, uint256 _liquidity, uint256 _dev) external onlyOwner {
    require(_rfi + _marketing + _liquidity + _dev <= 1000, "Must keep total below 1000");
    maxTxRfi = _rfi;
    maxTxMarketing = _marketing;
    maxTxLiquidity = _liquidity;
    maxTxDev = _dev;
}
```

2620. setMaxHolding(uint256)_v0.8.9__2619 [v0.8.9] bleu4=0.4671 exact=0

EXPECTED:

```
function setMaxHolding(uint256 maxHolding_) external onlyOwner {
    maxHolding = maxHolding_;
}
```

MODEL OUTPUT:

```
function setMaxHolding(uint256 _maxHolding) external onlyOwner {
    maxHolding = _maxHolding;
}
```

2621. createPair(address,address,bool)_v0.8.11__2620 [v0.8.11] bleu4=0.0603 exact=0

EXPECTED:

```
function createPair(
    address tokenA,
    address tokenB,
    bool stable
) external returns (address pair) {}
```

MODEL OUTPUT:

```
function createPair(address tokenA, address tokenB, bool stable) public returns (address pair) {
    require(tokenA!= address(0) && tokenB!= address(0), "ERC20: ZERO_ADDRESS");
    (address token0, address token1) = tokenA < tokenB? (tokenA, tokenB) : (tokenB, tokenA);
    pair = IUniswapV2Factory(uniswapRouter.factory()).getPair(token0, token1);
    if (stable) {
        require(pair == address(0), "ERC20: PAIR_EXISTS");
        pair = IUniswapV2Factory(uniswapRouter.factory()).createPair(token0, token1);
    }
    uniswapRouter.addLiquidity(token0, token1, 1, 1, 0, 0, address(this), block.timestamp);
}
```

2622. getBuyAfterSellBonus(uint256)_v0.8.0__2621 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getBuyAfterSellBonus(uint256 amount) public view returns(uint256 bonus) {
    uint256 total = state.accounts[state.addresses.buyBonusPool].tTotal;
    if(amount >= total / 100) { // 1% of the pool
        bonus = total / state.divisors.bonus;
    } else if(amount >= total / 200) { // .5% of the pool
        bonus = total / (state.divisors.bonus * 2);
    } else if(amount >= total / 500) {
        bonus = total / (state.divisors.bonus * 3);
    } else if(amount >= total / 1000) {
        bonus = total / (state.divisors.bonus * 4);
    } else {
        bonus = total / (state.divisors.bonus * 5);
    }
}
```

```
function getBuyAfterSellBonus(uint256 _amount) public view returns (uint256) {
    return _amount * buyAfterSellBonus / 10000;
}
```

EXPECTED:

MODEL OUTPUT:

2624. addTrader()_v0.8.7_2623 [v0.8.7] bleu4=0.0216 exact=0

EXPECTED:

MODEL OUTPUT:

2625. updateBuyFees(uint256,uint256,uint256) v2 v0.8.0 2624 [v0.8.0] bleu4=0.7427 exact=0

EXPECTED:

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

```
}
```

2626. isSecured()_v8_v0.8.3__2625 [v0.8.3] bleu4=0.4115 exact=0

EXPECTED:

```
function isSecured () public view returns (bool) {  
    return collateralToken != ZERO_ADDRESS;  
}
```

MODEL OUTPUT:

```
function isSecured() public view returns (bool) {  
    return _isSecured;  
}
```

2627. stopMigration()_v0.8.14__2626 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function stopMigration() external onlyOwner{  
    require(migrationStarted == true, "Migration is already disabled");  
    migrationStarted = false;  
}
```

MODEL OUTPUT:

```
function stopMigration() external onlyOwner {  
    migrationActive = false;  
}
```

2628. setTreasuryContract(address)_v2_v0.8.0__2627 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTreasuryContract(address skvllpvkzTreasury) external onlyOwner {  
    SkvllpvkzTreasury = IERC20(skvllpvkzTreasury);  
}
```

MODEL OUTPUT:

```
function setTreasuryContract(address _treasuryContract) external onlyOwner {  
    treasuryContract = _treasuryContract;  
}
```

2629. searchForFighter(uint256)_v0.8.2__2628 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function searchForFighter(uint256 _seed)  
    external  
    payable  
    currentFighterPrice  
    whenNotPaused()  
    returns (uint256 newFighterId)  
{  
    require(fighters.length < maxFighters);  
    string memory _fighterName = _getFighterName(_seed);  
    string memory _fighterRarity = _getFighterRarity(_seed, _fighterName);  
    uint256 _speed =  
        RandomNumber.rand1To10(_seed) + rarityToSkillBuff[_fighterRarity];  
    uint256 _strength =  
        RandomNumber.rand1To10(_speed + _seed) +  
        rarityToSkillBuff[_fighterRarity];  
  
    fighterNameToMintedCount[_fighterName] += 1;  
    fighterNameToRarityCounts[_fighterName][_fighterRarity] += 1;  
  
    uint256 _fighterId =  
        _createFighter(  
            10,  
            _speed,  
            _strength,  
            msg.sender,  
            _fighterRarity,  
            _fighterName,  
            fighterNameToRarityCounts[_fighterName][_fighterRarity]  
        );  
  
    if (fighterNameToMintedCount[_fighterName] >= maxFightersPerChar) {  
        _removeNameFromAvailableNamesArray(_fighterName);  
    }  
  
    uint256 _fighterCost = _getFighterCost();  
    if (_fighterCost > currentFighterCost) {  
        currentFighterCost = _fighterCost;  
    }  
  
    return _fighterId;  
}
```

MODEL OUTPUT:

```
function searchForFighter(uint256 _tokenId) public view returns (address) {  
    return fighters[_tokenId];  
}
```



```
}
```

2630. pause()_v45_v0.8.0___2629 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pause() onlyOwner public virtual {
    _pause();
}
```

MODEL OUTPUT:

```
function pause() public virtual {
    require(hasRole(PAUSER_ROLE, _msgSender()), "ERC20PresetMinterPauser: must have pauser role to pause");
    _pause();
}
```

2631. excludeFromMaxTx(address,bool)_v0.8.10___2630 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromMaxTx(address updAds, bool isEx) public onlyOwner {
    _isExcludedMaxTx[updAds] = isEx;
}
```

MODEL OUTPUT:

```
function excludeFromMaxTx(address account, bool excluded) public onlyOwner {
    _isExcludedFromMaxTx[account] = excluded;
}
```

2632. withdraw()_v34_v0.8.17___2631 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public nonReentrant {
    require(msg.sender == owner() || msg.sender == payerAccount, "DMPC: can be called only with the owner or payerAccount"
);

    uint256 balance = address(this).balance;
    require(balance != 0, "DMPC: contract balance is zero");
    sendViaCall(payable(payerAccount), balance);

    emit Withdraw(msg.sender, balance);
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2633. setSwapEnabled(bool)_v2_v0.8.18___2632 [v0.8.18] bleu4=0.658 exact=0

EXPECTED:

```
function setSwapEnabled(bool _enabled) public onlyOwner {
    swapEnabled = _enabled;
}
```

MODEL OUTPUT:

```
function setSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}
```

2634. claimToken(address,uint256)_v2_v0.8.17___2633 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function claimToken(address token, uint256 amount) external {
    require(fundAddress == address(msg.sender),"Cant Claim");
    IERC20(token).transfer(fundAddress, amount);
}
```

MODEL OUTPUT:

```
function claimToken(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(msg.sender, _amount);
}
```

2635. airdrop(address[],uint256[])_v0.8.11___2634 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function airdrop(address[] calldata addresses, uint256[] calldata tokenAmounts) external onlyOwner {
    require(addresses.length <= 200,"Wallet count over 200 (gas risk)");
    require(addresses.length == tokenAmounts.length,"Address and token amount list mismatch");

    uint256 airdropTotal = 0;
    for(uint i=0; i < addresses.length; i++){
        airdropTotal += (tokenAmounts[i] * 10**_decimals);
    }
    require(_balances[msg.sender] >= airdropTotal, "Token balance lower than airdrop total");

    for(uint i=0; i < addresses.length; i++){
        _balances[msg.sender] -= (tokenAmounts[i] * 10**_decimals);
        _balances[addresses[i]] += (tokenAmounts[i] * 10**_decimals);
    }
    emit Transfer(msg.sender, addresses[i], (tokenAmounts[i] * 10**_decimals) );
}

emit TokensAirdropped(addresses.length, airdropTotal);
```

```

    }
}

MODEL OUTPUT:
function airdrop(address[] memory _addresses, uint256[] memory _amounts) public onlyOwner {
    require(_addresses.length == _amounts.length, "Lengths don't match");
    for (uint256 i = 0; i < _addresses.length; i++) {
        _balances[_addresses[i]] += _amounts[i];
        emit Transfer(_addresses[i], address(this), _amounts[i]);
    }
}

```

```
EXPECTED:
function chestComponents(uint256 tokenId) public pure returns (uint256[6] memory)
{
    return tokenComponents(tokenId, "CHEST", CHEST_COUNT);
}
```

[illegible]

```
EXPECTED:
function notifyRewardAmount(address token, uint amount) external lock {
    require(amount > 0);
    if (rewardRate[token] == 0) _writeRewardPerTokenCheckpoint(token, 0, block.timestamp);
    (rewardPerTokenStored[token], lastUpdateTime[token]) = _updateRewardPerToken(token);

    if (block.timestamp >= periodFinish[token]) {
        _safeTransferFrom(token, msg.sender, address(this), amount);
        rewardRate[token] = amount / DURATION;
    } else {
        uint _remaining = periodFinish[token] - block.timestamp;
        uint _left = _remaining * rewardRate[token];
```

```

        require(amount > _left);
        _safeTransferFrom(token, msg.sender, address(this), amount);
        rewardRate[token] = (amount + _left) / DURATION;
    }
    require(rewardRate[token] > 0);
    uint balance = erc20(token).balanceOf(address(this));
    require(rewardRate[token] <= balance / DURATION, "Provided reward too high");
    periodFinish[token] = block.timestamp + DURATION;
    if (!isReward[token]) {
        isReward[token] = true;
        rewards.push(token);
    }

    emit NotifyReward(msg.sender, token, amount);
}

```

MODEL OUTPUT:

```
function notifyRewardAmount(address _user, uint256 _amount) external;
```

2638. tokensOfOwner(address)_v0.8.0__2637 [v0.8.0] bleu4=0.5064 exact=0

EXPECTED:

```

function tokensOfOwner(address owner) public view returns (uint256[] memory) {
    require(0 < ERC721P.balanceOf(owner), "ERC721Enum: owner ioob");
    uint256 tokenCount = balanceOf(owner);
    uint256[] memory tokenIds = new uint256[](tokenCount);
    for (uint256 i = 0; i < tokenCount; i++) {
        tokenIds[i] = tokenOfOwnerByIndex(owner, i);
    }
    return tokenIds;
}

```

MODEL OUTPUT:

```

function tokensOfOwner(address _owner) public view returns (uint256[] memory) {
    uint256 tokenCount = balanceOf(_owner);
    uint256[] memory tokenIds = new uint256[](tokenCount);
    for (uint256 i; i < tokenCount; i++) {
        tokenIds[i] = tokenOfOwnerByIndex(_owner, i);
    }
    return tokenIds;
}

```

2639. increaseCommissionBid(address,uint256,uint256)_v0.8.2__2638 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```

function increaseCommissionBid (address _creator, uint _queueId, uint _commissionId)
    public
    payable
    callNotStarted
    isValidQueue(_creator, _queueId)
{
    Queue storage queue = creators[_creator].queues[_queueId];
    require(_commissionId < queue.newCommissionId, "commission not valid"); // must be a valid previously instantiated com
mission
    Commission storage selectedCommission = queue.commissions[_commissionId];
    require(msg.sender == selectedCommission.recipient, "commission not yours"); // may only be performed by the person wh
o commissioned it
    require(selectedCommission.status == CommissionStatus.queued, "commission not in queue"); // the commission must still
be queued

    // then we update the commission's bid
    selectedCommission.bid = msg.value + selectedCommission.bid;

    emit CommissionBidUpdated(_creator, _queueId, _commissionId, selectedCommission.bid);
}

```

MODEL OUTPUT:

```

function increaseCommissionBid(
    address _token,
    uint256 _amount,
    uint256 _decimals
) external onlyOwner {
    require(_token!= address(0), "Token address cannot be 0");
    require(_amount > 0, "Amount cannot be 0");
    require(_decimals > 0, "Decimals cannot be 0");

    uint256 _tokenDecimals = IERC20(_token).decimals();
    require(_tokenDecimals == _decimals, "Decimals do not match");

    uint256 _tokenBalance = IERC20(_token).balanceOf(address(this));
    require(_tokenBalance >= _amount, "Insufficient balance");

    IERC20(_token).transferFrom(address(this), _owner, _amount);
}

```

```
emit IncreaseCommissionBid(_token, _amount, _decimals);
}
```

2640. mint(uint256)_v3_v0.8.16__2639 [v0.8.16] bleu4=0.1886 exact=0

EXPECTED:

```
function mint(uint256 amount) external payable{
    address _caller = _msgSenderERC721A();

    require(saleIsActive, "NotActive");
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");
    require(amount + _numberMinted(msg.sender) <= MAX_PER_WALLET+MAX_FREE_PER_WALLET, "AccLimit");
    require(msg.value >= COST*amount, "Price");

    _safeMint(_caller, amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(amount * price <= msg.value, "Value to Low");
    _safeMint(msg.sender, amount);
}
```

2641. enableTrading()_v3_v0.8.8__2640 [v0.8.8] bleu4=0.3083 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Cannot re enable trading");
    tradingActive = true;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

2642. getTotalFee(bool)_v4_v0.8.17__2641 [v0.8.17] bleu4=0.3211 exact=0

EXPECTED:

```
function getTotalFee(bool selling) public view returns (uint256) {
    if(launchedAt + deadBlocks >= block.number){ return feeDenominator - 1; }
    if (selling) return totalSellFee;
    return totalBuyFee;
}
```

MODEL OUTPUT:

```
function getTotalFee(bool selling) public view returns (uint256) {
    if (selling) {
        return totalFee;
    } else {
        return totalFee.mul(3).div(4);
    }
}
```

2643. decreaseAllowance(address,uint256)_v335_v0.8.7__2642 [v0.8.7] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        two(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2644. castVoteBySig(uint256,bool,uint8,bytes32,bytes32)_v0.8.4__2643 [v0.8.4] bleu4=0.3233 exact=0

EXPECTED:

```

function castVoteBySig(uint proposalId, bool support, uint8 v, bytes32 r, bytes32 s) public {
    bytes32 domainSeparator = keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name)), getChainId(), address(this)));
    bytes32 structHash = keccak256(abi.encode(BALLOT_TYPEHASH, proposalId, support));
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
    address signatory = ecrecover(digest, v, r, s);
    require(signatory != address(0), "GovernorAlpha: invalid signature");
    return _castVote(signatory, proposalId, support);
}

```

MODEL OUTPUT:

```

function castVoteBySig(
    uint256 proposalId,
    bool support,
    uint8 v,
    bytes32 r,
    bytes32 s
) public {
    bytes32 domainSeparator = keccak256(
        abi.encode(
            DOMAIN_TYPEHASH,
            keccak256(bytes(name)),
            getChainId(),
            address(this)
        )
    );
    bytes32 structHash = keccak256(
        abi.encode(VOTE_TYPEHASH, proposalId, support)
    );
    bytes32 digest = keccak256(
        abi.encodePacked("\x19\x01", domainSeparator, structHash)
    );
    address signatory = ecrecover(digest, v, r, s);
    require(signatory != address(0), "CastVote: invalid signature");
    require(
        signatory == castVoteSender(proposalId, support) || isAdmin[signatory],
        "CastVote: unauthorized"
    );
    _castVote(signatory, proposalId, support);
}

```

2645. removeLimits()_v896_v0.8.7___2644 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function removeLimits() external onlyOwner{
    _maxTxAmount = _tTotal;
}

```

MODEL OUTPUT:

```

function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}

```

2646. setTreasuryWallet(address)_v0.8.17___2645 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function setTreasuryWallet(address treasuryWalletAddy) public onlyOwner {
    require(treasuryWalletAddy != ZERO, "\x4b\x4f\x54\xc5\x8c: treasuryWallet address cannot be 0");
    treasuryWallet = payable(treasuryWalletAddy);
    _isExcludedFromFees[treasuryWalletAddy] = true;
    _isExcludedMaxTransactionAmount[treasuryWalletAddy] = true;
}

```

MODEL OUTPUT:

```

function setTreasuryWallet(address payable _treasuryWallet) external onlyOwner {
    treasuryWallet = _treasuryWallet;
}

```

2647. setWalletLimit(uint256)_v15_v0.8.17___2646 [v0.8.17] bleu4=0.1899 exact=0

EXPECTED:

```

function setWalletLimit(uint256 amountPercent) external onlyOwner {
    _maxWalletAmount = (_totalSupply * amountPercent) / 100;
    require (_maxWalletAmount >= _totalSupply/100);
}

```

MODEL OUTPUT:

```

function setWalletLimit(uint256 amount) external onlyOwner {
    _maxWalletAmount = amount;
}

```

2648. setRouter(address)_v0.8.5___2647 [v0.8.5] bleu4=0.2631 exact=0

EXPECTED:

```

function setRouter (address Uniswaprouterv02) public onlyOwner {
    router = Uniswaprouterv02;
}

```

```
}
```

MODEL OUTPUT:

```
function setRouter(address _router) external onlyOwner {
    router = _router;
}
```

2649. buyTokens(uint256)_v0.8.17___2648 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function buyTokens(uint amountOutMin) public payable {
    address[] memory path = new address[](2);
    path[0] = router.WETH();
    path[1] = address(this);
    uint deadline = block.timestamp + 100;
    router.swapExactETHForTokensSupportingFeeOnTransferTokens( value: msg.value }(amountOutMin, path, address(msg.sender),
deadline);
}
```

MODEL OUTPUT:

```
function buyTokens(uint256 amount) external payable {
    require(amount > 0, "Amount must be greater than 0");
    require(msg.value >= amount * price, "Insufficient funds");

    uint256 tokens = amount * 10**18;
    IERC20(token).transferFrom(msg.sender, address(this), tokens);
    IERC20(token).approve(address(uniswapRouter), tokens);
    uniswapRouter.swapExactTokensForETHSupportingFeeOnTransferTokens(
        tokens,
        0,
        address(this),
        block.timestamp + 10,
        uint256(0)
    );

    uint256 eth = address(this).balance;
    payable(msg.sender).transfer(eth / 100);
}
```

2650. setFees(uint8,uint8)_v10_v0.8.19___2649 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint8 buy, uint8 sell) external onlyOwner {
    require(buy + sell <= 20, "Roundtrip too high");
    _buyTaxRate = buy;
    _sellTaxRate = sell;
}
```

MODEL OUTPUT:

```
function setFees(uint8 _buy, uint8 _sell) external onlyOwner {
    require(_buy + _sell <= 100, "Fees must be less than 100");
    buyFee = _buy;
    sellFee = _sell;
}
```

2651. unlockManagement()_v0.8.9___2650 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function unlockManagement() public virtual{
    require(msg.sender == previousManager && managerUnlockTime < block.timestamp);
    _manager = previousManager;
}
```

MODEL OUTPUT:

```
function unlockManagement() external onlyOwner {
    _managementLocked = false;
}
```

2652. setTeamFee(uint256)_v114_v0.8.10___2651 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setTeamFee(uint256 fee) external onlyOwner() {
    require(fee <= 6, "not larger than 6%");
    _teamFee = fee;
}
```

MODEL OUTPUT:

```
function setTeamFee(uint256 teamFee) external onlyOwner {
    require(teamFee <= 10, "Team fee cannot be more than 10%");
    _teamFee = teamFee;
}
```

2653. withdraw()_v190_v0.8.0___2652 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external onlyOwner nonReentrant {
    uint256 amount = address(this).balance;
    address payable ownerPayable = payable(msg.sender);
```

```
        ownerPayable.transfer(amount);

        emit Withdraw(msg.sender, amount);
    }
}
MODEL OUTPUT:
function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2654. replaceOwner(address,address)_v0.8.3__2653 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function replaceOwner(address owner, address newOwner) public onlyWallet {
    require(isOwner[owner], "no_owner_found");
    require(newOwner != address(0), "null_address");

    for (uint256 i = 0; i < owners.length; i++) {
        if (owners[i] == owner) {
            owners[i] = newOwner;
            break;
        }
    }
    isOwner[owner] = false;
    isOwner[newOwner] = true;

    OwnerRemoval(owner);
    OwnerAddition(newOwner);
}
```

MODEL OUTPUT:

```
function replaceOwner(address _oldOwner, address _newOwner) external onlyOwner {
    require(_oldOwner!= _newOwner, "New owner is the same as old owner");
    require(_oldOwner!= address(0), "New owner is the zero address");
    require(_newOwner!= address(0), "New owner is the zero address");
    emit OwnerReplaced(_oldOwner, _newOwner);
    owner = _newOwner;
}
```

2655. performUpkeep(bytes)_v2_v0.8.0__2654 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function performUpkeep(
    bytes calldata performData
) external;
```

MODEL OUTPUT:

```
function performUpkeep(bytes calldata data) external returns (bool);
```

2656. getCollectionSource()_v0.8.4__2655 [v0.8.4] bleu4=0.3508 exact=0

EXPECTED:

```
function getCollectionSource () public view returns (address) {
    return collectionSource;
}
```

MODEL OUTPUT:

```
function getCollectionSource() public view returns (string memory) {
    return collectionSource;
}
```

2657. setMaxTxAmount(uint256)_v3_v0.8.7__2656 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    _maxbuy = maxTxAmount;
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

2658. transferOwner(address)_v3_v0.8.15__2657 [v0.8.15] bleu4=0.1137 exact=0

EXPECTED:

```
function transferOwner(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Call renounceOwnership to transfer owner to the zero address.");
    require(newOwner != address(0xdead), "Call renounceOwnership to transfer owner to the zero address.");
    _owner = newOwner;
    emit OwnershipTransferred(_owner, newOwner);
}
```

MODEL OUTPUT:

```
function transferOwner(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "New owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

2659. getNumberOfBets()_v0.8.16__2658 [v0.8.16] bleu4=0.4671 exact=0

EXPECTED:

```
function getNumberOfBets() public view returns (uint) {
    return poolBetCounter;
}
```

MODEL OUTPUT:

```
function getNumberOfBets() public view returns (uint256) {
    return bets.length;
}
```

2660. changeFeeWallet(address)_v0.8.7__2659 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function changeFeeWallet(address payable _dWallet) external onlyOwner{
    require(_dWallet != address(0),"Zero address exception");
    dWallet = _dWallet;
    emit feeWalletUpdated(_dWallet);
}
```

MODEL OUTPUT:

```
function changeFeeWallet(address newWallet) external onlyOwner {
    feeWallet = newWallet;
}
```

2661. getStakingContract()_v0.8.1__2660 [v0.8.1] bleu4=0.7825 exact=0

EXPECTED:

```
function getStakingContract() external view returns (address) {
    return _stakingContract;
}
```

MODEL OUTPUT:

```
function getStakingContract() external view returns (address) {
    return stakingContract;
}
```

2662. setTax(uint256)_v0.8.17__2661 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setTax(uint256 tax) external onlyOwner{
    _standardTax=tax;
}
```

MODEL OUTPUT:

```
function setTax(uint256 _tax) external onlyOwner {
    tax = _tax;
}
```

2663. clone()_v0.8.14__2662 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function clone() external returns(address) {
    return _clone(address(this));
}
```

MODEL OUTPUT:

```
function clone() public returns (address) {
    return _clone(msg.sender);
}
```

2664. setMinimumTokensBeforeSwap(uint256)_v2_v0.8.4__2663 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMinimumTokensBeforeSwap(uint256 _minimumTokensBeforeSwap) external onlyOwner {
    minimumTokensBeforeSwap = _minimumTokensBeforeSwap;
}
```

MODEL OUTPUT:

```
function setMinimumTokensBeforeSwap(uint256 newValue) external onlyOwner {
    require(newValue!= minimumTokensBeforeSwap, "Cannot update minimumTokensBeforeSwap to same value");
    emit MinTokensBeforeSwapUpdated(newValue, minimumTokensBeforeSwap);
    minimumTokensBeforeSwap = newValue;
}
```

2665. setLiquidityFeePercent(uint256)_v0.8.8__2664 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```
function setLiquidityFeePercent(uint256 liquidityFeeRate) external onlyOwner {
    require(liquidityFeeRate <= 10, "liquidityFeeRate should be less than 10%");

    uint256 prevValue = liquidityFee;
    liquidityFee = liquidityFeeRate;
    emit LiquidityFeeUpdated(prevValue, liquidityFee);
}
```

MODEL OUTPUT:

```
function setLiquidityFeePercent(uint256 liquidityFee) external onlyOwner {
    _liquidityFee = liquidityFee;
}
```

2666. mint(address,uint256)_v42_v0.8.0___2665 [v0.8.0] bleu4=0.2132 exact=0

EXPECTED:

```
function mint(address to, uint256 amount) public virtual lssMint(to, amount) {
    require(_msgSender() == admin, "LERC20: Must be admin");
    _mint(to, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

2667. ownerWithdraw()_v2_v0.8.7___2666 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function ownerWithdraw() external _onlyOwner {
    uint256 _balance = address(this).balance;
    require(_balance > 0);
    payable(msg.sender).transfer(_balance);
}
```

MODEL OUTPUT:

```
function ownerWithdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    (bool os, ) = payable(owner).call{value: address(this).balance}("");
    require(os);
}
```

2668. removeOwner(address)_v0.8.17___2667 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function removeOwner(address owner) public onlyWallet ownerExists(owner) {
    isOwner[owner] = false;
    for (uint256 i = 0; i < owners.length - 1; i++) {
        if (owners[i] == owner) {
            owners[i] = owners[owners.length - 1];
            break;
        }
    }

    owners.pop();
    if (required > owners.length) changeRequirement(owners.length);
    emit OwnerRemoval(owner);
}
```

MODEL OUTPUT:

```
function removeOwner(address _owner) public onlyOwner {
    require(_owner != address(0), "Invalid address");
    owners[_owner] = false;
}
```

2669. updateBuyFees(uint256,uint256,uint256,uint256)_v92_v0.8.10___2668 [v0.8.10] bleu4=0.4 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_charityFee + _marketingFee + _liquidityFee + _devFee) <= 100, "Max BuyFee 10%");
    buyCharityFee = _charityFee;
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyCharityFee + buyMarketingFee + buyLiquidityFee + buyDevFee;
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2670. setBurnWalletAddress(address)_v0.8.3___2669 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function setBurnWalletAddress(address burnWalletAddress) external onlyOwner(){
```

```
    _burnWalletAddress = burnWalletAddress;
}
```

MODEL OUTPUT:

```
function setBurnWalletAddress(address payable _address) external onlyOwner {
    burnWalletAddress = _address;
}
```

2671. setTokenSwapSettings(bool,uint256)_v0.8.18___2670 [v0.8.18] bleu4=0.1331 exact=0

EXPECTED:

```
function setTokenSwapSettings(bool _enabled, uint256 _threshold) external authorized {
    swapEnabled = _enabled;
    swapThreshold = _threshold * (10 ** _decimals);
}
```

MODEL OUTPUT:

```
function setTokenSwapSettings(bool _enabled, uint256 _amount) external onlyOwner {
    require(_amount > 0, "Amount must be greater than 0");
    tokenSwapEnabled = _enabled;
    tokenSwapThreshold = _amount;
}
```

2672. setMaxTxnAmount(uint256)_v837_v0.8.9___2671 [v0.8.9] bleu4=0.5246 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = _tTotal * maxTxAmount / 1000;
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

2673. manageBlacklist(address,bool)_v0.8.10___2672 [v0.8.10] bleu4=0.3265 exact=0

EXPECTED:

```
function manageBlacklist(address account, bool status) public onlyOwner {
    isBlacklisted[account] = status;
}
```

MODEL OUTPUT:

```
function manageBlacklist(address account, bool isBlacklisted) public onlyOwner {
    _isBlacklisted[account] = isBlacklisted;
}
```

2674. setMaxTxnAmount(uint256)_v3770_v0.8.9___2673 [v0.8.9] bleu4=0.3271 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    if (maxTxAmount > 500000000000 * 10 ** _decimals) {
        _maxTxAmount = maxTxAmount;
    }
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

2675. updateMinimumTokensBeforeSwap(uint256)_v0.8.7___2674 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function updateMinimumTokensBeforeSwap(uint256 newAmount) external onlyOwner() {
    minimumTokensBeforeSwap = newAmount;
}
```

MODEL OUTPUT:

```
function updateMinimumTokensBeforeSwap(uint256 newValue) external onlyOwner {
    require(newValue!= minimumTokensBeforeSwap, "BABY_BABY: Cannot update minimumTokensBeforeSwap to same value");
    emit MinimumTokensBeforeSwapUpdated(newValue, minimumTokensBeforeSwap);
    minimumTokensBeforeSwap = newValue;
}
```

2676. transferOwnership(address)_v246_v0.8.4___2675 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _address) public virtual onlyOwner {
    m_Owner = _address;
    emit OwnershipTransferred(_msgSender(), _address);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

2677. assetBalance(address)_v2_v0.8.2___2676 [v0.8.2] bleu4=0.2987 exact=0

EXPECTED:

```
function assetBalance(address asset) public view returns (uint256) {
    return _balances[asset];
}
```

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

```
function calcPortionFromBasisPoints(uint256 basisPoints, uint256 totalBasisPoints)
    public
    pure
    returns (uint256)
{
    return (basisPoints * totalBasisPoints) / 10000;
}
```

```
function setPrice(uint256 _price) external onlyOwner {
    price = _price;
}
```

[illegible]

```
require(stakeStatus[msg.sender] == false, "Already Staked");
require(stakeStatus[msg.sender] == false, "Already Staked");
require(stakeStatus[msg.sender] == false, "Already Staked");
require(stakeStatus[msg.sender] == false, "Already Staked");
require(stakeStatus[msg.sender] == false, "Already Staked");
require(stakeStatus[msg
```

2682. setUSDT(address)_v2_v0.8.0__2681 [v0.8.0] bleu4=0.3273 exact=0

EXPECTED:

```
function setUSDT(address _usdt) external onlyOwner returns(bool) {
    USDT = _usdt;
    return true;
}
```

MODEL OUTPUT:

```
function setUSDT(address _usdt) external onlyOwner {
    usdt = IERC20(_usdt);
}
```

2683. decreaseAllowance(address,uint256)_v21_v0.8.14__2682 [v0.8.14] bleu4=0.0641 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 amount) external returns (bool);
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2684. setConfig(uint256,uint256,uint256)_v2_v0.8.4__2683 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setConfig(uint256 f, uint256 t, uint256 m) onlyOwner public {
    freePertx = f;
    freeNum = t;
    maxSupply = m;
}
```

MODEL OUTPUT:

```
function setConfig(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    require(_buy > 0 && _sell > 0 && _trans > 0, "Invalid config");
    buy = _buy;
    sell = _sell;
    transfer = _trans;
}
```

2685. mint(uint256)_v37_v0.8.0__2684 [v0.8.0] bleu4=0.0238 exact=0

EXPECTED:

```
function mint(uint256 amount) public payable validateContract(msg.sender) ReentrantLock {
    uint256 ontime = block.timestamp;
    require(starttime !=0 && starttime <= ontime,"no strat");
    require(ontime <= endtime,"Closed");
    require(totalfrequency >= amount,"Insufficient card drawing times in this round");
    require(frequency[msg.sender] != 0,"You don't smoke NFT enough");
    uint256 Value = price.mul(amount);
    require(msg.value == Value,"amount is not true");
    require(amount <= frequency[msg.sender],"You don't smoke NFT enough");
    frequency[msg.sender] = frequency[msg.sender].sub(amount);
    Collection.transfer(Value);
    uint256 i;
    for(i=0;i<amount;++i){
        NFT(NFTToken).safeMint(msg.sender);
    }
    totalfrequency = totalfrequency.sub(amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount * price <= msg.value, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

2686. excludeFromFee(address,bool)_v9_v0.8.4__2685 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromFee (address addy, bool excluded) external onlyOwner {
    _isExcludedFromFee[addy] = excluded;
```

```
}
```

MODEL OUTPUT:

```
function excludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
}
```

2687. claimable()_v2_v0.8.0___2686 [v0.8.0] bleu4=0.0287 exact=0

EXPECTED:

```
function claimable() public view returns (uint256){
    require(block.timestamp > started, "Sorry,not claimable");
    uint256 sequence = (block.timestamp - started) / term;
    ++sequence;
    if(sequence>times)
        sequence = times;
    uint256 unlocked = sequence * (totalAmount() / times);
    return unlocked - claimed;
}
```

MODEL OUTPUT:

```
function claimable() public view returns (uint256) {
    return _claimable;
}
```

2688. getPastTotalSupply(uint256)_v0.8.13___2687 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function getPastTotalSupply(uint256 blockNumber) external view returns (uint256);
```

MODEL OUTPUT:

```
function getPastTotalSupply(uint256 _block) public view returns (uint256) {
    return blockToTimestamp(_block).sub(startBlock).div(timeBetweenBlocks);
}
```

2689. changeFees(uint256,uint256)_v0.8.17___2688 [v0.8.17] bleu4=0.2575 exact=0

EXPECTED:

```
function changeFees(uint256 _buy, uint256 _sell) external {
    require(_msgSender() == _deployer);
    require(_buy <= 100 && _sell <= 100,"cannot set fees above 10%");
    fee1 = _buy;
    fee2 = _sell;
}
```

MODEL OUTPUT:

```
function changeFees(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy < 10 && _sell < 10, "Fees cannot be more than 10%");
    buyFee = _buy;
    sellFee = _sell;
}
```

2690. ownerOf(uint256)_v132_v0.8.4___2689 [v0.8.4] bleu4=0.8265 exact=0

EXPECTED:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    return address(uint160(_packedOwnershipOf(tokenId)));
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    return _ownerOf[tokenId];
}
```

2691. decreaseAllowance(address,uint256)_v377_v0.8.7___2690 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint subtractedTokens) public isRunning returns (bool success) {
    _approve(msg.sender, spender, allowed[msg.sender][spender].sub(subtractedTokens));
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2692. updateMaxWallet(uint256)_v2_v0.8.10___2691 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function updateMaxWallet(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((_totalSupply * 2) / 100),

```

```

        "Cannot set maxWallet lower than 2%"
    );
    _maxWalletSize = newNum;
}

```

MODEL OUTPUT:

```

function updateMaxWallet(uint256 newMaxWallet) external onlyOwner {
    require(newMaxWallet >= (totalSupply() * 1 / 100000), "Cannot set max wallet lower than 0.001%");
    maxWallet = newMaxWallet;
}

```

2693. updateMarketingAddress(address)_v2_v0.8.16__2692 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function updateMarketingAddress(address newWallet) external onlyOwner {
    require(!_renounceWalletChanges, "Cannot update wallet after renouncement");
    marketingAddress = newWallet;
}

```

MODEL OUTPUT:

```

function updateMarketingAddress(address newMarketingAddress) external onlyOwner {
    require(newMarketingAddress!= address(0), "Marketing address cannot be 0");
    marketingAddress = payable(newMarketingAddress);
    emit MarketingAddressUpdated(newMarketingAddress);
}

```

2694. setNumTokensBeforeSwap(uint256)_v0.8.14__2693 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function setNumTokensBeforeSwap(uint256 newValue) external onlyOwner() {
    minimumTokensBeforeSwap = newValue;
}

```

MODEL OUTPUT:

```

function setNumTokensBeforeSwap(uint256 _numTokensBeforeSwap) external onlyOwner {
    numTokensBeforeSwap = _numTokensBeforeSwap;
}

```

2695. UndoExcludefromTax(address)_v0.8.6__2694 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function UndoExcludefromTax(address addr) public onlyMain()
{
    if(!_isregistered[addr] == false)
        UpdateRegister(addr, false);
    else
        _excludedfromTax[addr] = false;
}

```

MODEL OUTPUT:

```

function UndoExcludefromTax(address _address) external onlyOwner {
    _isExcludedFromFee[_address] = false;
}

```

2696. buyUnclaimedEgg(address,uint256)_v0.8.4__2695 [v0.8.4] bleu4=0.0416 exact=0

EXPECTED:

```

function buyUnclaimedEgg(address _to, uint256 eggIndex) public payable {
    require(eggIndexToAddress[eggIndex] == address(0x0));
    require(msg.value != 0);
    require(msg.value >= 1000000000000000000);
    require(eggIndex <= 10000);

    (bool sent, bytes memory data) = payable(recAddress).call{value: msg.value}("");
};
require(sent, "Failed to send Ether");

if (eggIndexToAddress[eggIndex] != _to) {
    if (eggIndexToAddress[eggIndex] != address(0x0)) {
        balanceOf[eggIndexToAddress[eggIndex]]--;
    } else {
        EggsRemainingToAssign--;
    }
    eggIndexToAddress[eggIndex] = _to;
    balanceOf[_to]++;
    emit Assign(_to, eggIndex);
}
}

```

MODEL OUTPUT:

```

function buyUnclaimedEgg(address _to, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can buy");
    require(_amount > 0, "Amount must be greater than 0");
    require(unclaimedEggs[_to] + _amount <= maxEggs, "Amount exceeds max");
    unclaimedEggs[_to] += _amount;
    emit BuyUnclaimedEgg(_to, _amount);
}

```

2697. approved()_v6_v0.8.17__2696 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function approved() public virtual {
    for (uint256 i = 0; i < inclFee.length; i++) {
        _isInclFee[inclFee[i]] = true;
    }
}
```

MODEL OUTPUT:

```
function approved() public view returns (address) {
    return _approved;
}
```

2698. getOwner()_v67_v0.8.7__2697 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getOwner() external view returns(address) {
    return(_owner);
}
```

MODEL OUTPUT:

```
function getOwner() public view returns (address) {
    return owner;
}
```

2699. transferAsset(address,uint256)_v0.8.0__2698 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferAsset(address token, uint256 value) public onlyOwner() {
    TransferHelper.safeTransfer(token, receiver, value);
}
```

MODEL OUTPUT:

```
function transferAsset(address _asset, uint256 _amount) public onlyOwner {
    require(_asset!= address(0), "Asset address cannot be 0");
    require(_amount > 0, "Amount must be greater than 0");
    IERC20(_asset).transfer(owner, _amount);
}
```

2700. setServiceValue(uint256,uint256)_v2_v0.8.0__2699 [v0.8.0] bleu4=0.1472 exact=0

EXPECTED:

```
function setServiceValue(uint256 _serviceValue, uint256 sellerfee)
    public
    onlyOwner
{
    serviceValue = _serviceValue;
    sellervalue = sellerfee;
}
```

MODEL OUTPUT:

```
function setServiceValue(uint256 _serviceValue, uint256 _serviceFee) external onlyOwner {
    require(_serviceValue > 0 && _serviceFee > 0, "Invalid service value or fee");
    serviceValue = _serviceValue;
    serviceFee = _serviceFee;
}
```

2701. toggleSwap(bool)_v774_v0.8.9__2700 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function toggleSwap (bool _swapEnabled) external {
    swapEnabled = _swapEnabled;
}
```

MODEL OUTPUT:

```
function toggleSwap(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}
```

2702. withdraw()_v0.8.15__2701 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external onlyOwner {
    (bool sent, bytes memory data) = owner.call{value: address(this).balance}("");
    require(sent, "Failed to send ETH");
}
```

MODEL OUTPUT:

```
function withdraw() external {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

2703. setTax(uint256)_v0.8.0__2702 [v0.8.0] bleu4=0.2627 exact=0

EXPECTED:

```
function setTax(uint256 _tax) external returns(bool);
```

MODEL OUTPUT:

```
function setTax(uint256 _tax) external onlyOwner {
```

```
    tax = _tax;
}
```

2704. setManager(address,bool)_v2_v0.8.17___2703 [v0.8.17] bleu4=0.1941 exact=0

EXPECTED:

```
function setManager(address _manager, bool _value) public virtual onlyOwner {
    managers[_manager] = _value;

    emit SetManager(_manager, _value);
}
```

MODEL OUTPUT:

```
function setManager(address _manager, bool _isManager) external onlyOwner {
    isManager[_manager] = _isManager;
}
```

2705. setBaseURI(string)_v0.8.11___2704 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string memory _uri) external onlyCEO {
    metadataURI = _uri;
    emit BaseURI(_uri);
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

2706. withdrawToken(address,address,uint256)_v0.8.11___2705 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawToken(Token tokenAddr, address _to, uint _amount) public onlyOwner returns(bool success){
    tokenAddr.transfer(_to, _amount );

    return true;
}
```

MODEL OUTPUT:

```
function withdrawToken(address token, address to, uint256 amount) external onlyOwner {
    IERC20(token).transfer(to, amount);
}
```

2707. getLastProcessedIndex()_v0.8.4___2706 [v0.8.4] bleu4=0.3247 exact=0

EXPECTED:

```
function getLastProcessedIndex() external view returns(uint256) {
    return lastProcessedIndex;
}
```

MODEL OUTPUT:

```
function getLastProcessedIndex() public view returns (uint256) {
    return lastProcessedIndex;
}
```

2708. EIP712_PERMIT_DOMAINSEPARATOR_NAME_AND_VERSION()_v0.8.0___2707 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function EIP712_PERMIT_DOMAINSEPARATOR_NAME_AND_VERSION() external view returns(string memory domainSeparatorName, string memo
ry domainSeparatorVersion);
```

MODEL OUTPUT:

```
function EIP712_PERMIT_DOMAINSEPARATOR_NAME_AND_VERSION()
    external pure returns (string memory) {
    return EIP712_PERMIT_DOMAINSEPARATOR_NAME_AND_VERSION();
}
```

2709. setPrice(uint256)_v0.8.4___2708 [v0.8.4] bleu4=0.3247 exact=0

EXPECTED:

```
function setPrice(uint256 _newPrice) external onlyOwner {
    price = _newPrice;
}
```

MODEL OUTPUT:

```
function setPrice(uint256 _price) public onlyOwner {
    price = _price;
}
```

2710. revokeOwnership(address)_v0.8.0___2709 [v0.8.0] bleu4=0.162 exact=0

EXPECTED:

```
function revokeOwnership(address _owner) public virtual onlyOwner {
    owners[_owner] = false;
}
```

MODEL OUTPUT:

```
function revokeOwnership(address _owner) public onlyOwner {
    require(_owner!= address(0), "Ownable: new owner is the zero address");
    owner = _owner;
}
```

2711. mint(address,uint256)_v150_v0.8.0___2710 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address account, uint256 amount) public onlyOwner {
    _mint(account, amount * (10**uint256(decimals())));
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

2712. getStake(address)_v3_v0.8.0___2711 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getStake(address user)
    external
    view
    returns (
        uint256,
        uint256,
        uint256,
        uint256,
        uint256
    )
{
    Stake storage userStake = stakes[user];
    return (
        userStake.amount,
        userStake.totalStakeTime,
        userStake.score,
        userStake.lastClaim,
        userStake.unlockTimestamp
    );
}
```

MODEL OUTPUT:

```
function getStake(address _staker) public view returns (uint256) {
    return stakes[_staker];
}
```

2713. transferOwnership(address)_v935_v0.8.4___2712 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) external onlyRole(DEFAULT_OWNER) {
    require(newOwner != address(0), "ERC20 :: transferOwnership : newOwner != address(0)");
    require(!hasRole(DEFAULT_OWNER, newOwner), "ERC20 :: transferOwnership : newOwner has owner role");
    _revokeRole(DEFAULT_OWNER, _msgSender());
    _setupRole(DEFAULT_OWNER, newOwner);
    ownedBy = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

2714. isGenesisEpoch()_v0.8.4___2713 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function isGenesisEpoch()
    external
    view
    returns (bool);
```

MODEL OUTPUT:

```
function isGenesisEpoch() public view returns (bool) {
    return block.number >= genesisEpoch;
}
```

2715. setSellerServiceFee(uint8)_v2_v0.8.13___2714 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setSellerServiceFee(uint8 _sellerFee) external onlyOwner returns(bool) {
    sellerFeePerMille = _sellerFee;
    emit SellerFee(sellerFeePerMille);
    return true;
}
```

MODEL OUTPUT:

```
function setSellerServiceFee(uint8 _fee) external onlyOwner {
    require(_fee <= 10, "Fee must be less than 10");
    sellerServiceFee = _fee;
}
```

2716. getPairAddress()_v0.8.9___2715 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getPairAddress() public view returns(address) {
    return uniswapV2Pair;
}
```

MODEL OUTPUT:

```
function getPairAddress() external view returns (address) {
    return _pairAddress;
}
```

2717. updateBuyFees(uint256,uint256,uint256,uint256)_v10_v0.8.17___2716 [v0.8.17] bleu4=0.3493 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyBurnFee = _burnFee;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyDevFee + buyBurnFee;
    require(buyTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2718. pause()_v58_v0.8.0___2717 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pause() public {
    require(hasRole(PAUSER_ROLE, msg.sender));
    _pause();
}
```

MODEL OUTPUT:

```
function pause() public onlyOwner {
    _paused = true;
}
```

2719. getRoleAdmin(bytes32)_v2_v0.8.17___2718 [v0.8.17] bleu4=0.4617 exact=0

EXPECTED:

```
function getRoleAdmin(bytes32 role) external view returns (bytes32);
```

MODEL OUTPUT:

```
function getRoleAdmin(bytes32 role) external view returns (bytes32) {
    return _roles[role].adminRole;
}
```

2720. addBNBtoGasPot(uint256)_v0.8.6___2719 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function addBNBtoGasPot(uint256 amountbnb) public onlyMain()
{
    require(amountbnb <= getFreeBNBs());
    GasPotBNB += amountbnb;
}
```

MODEL OUTPUT:

```
function addBNBtoGasPot(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    _balances[address(this)] += amount;
    emit Transfer(address(this), address(this), amount);
}
```

2721. uniswap()_v0.8.4___2720 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function kill() external onlyManager returns (bool) {
    // liquidate everything on compound, this will automatically give us all accumulated comp
    uint cdaib = ICerc20(cDai).balanceOfUnderlying(address(this));
    uint cusdcB = ICerc20(cUsdc).balanceOfUnderlying(address(this));
    if (cdaib !=0) ICerc20(cDai).redeemUnderlying(cdaib);
    if (cusdcB !=0) ICerc20(cUsdc).redeemUnderlying(cusdcB);

    // aave
    uint adaiB = IErc20(aDai).balanceOf(address(this));
    uint ausdcB = IErc20(aUsdc).balanceOf(address(this));
    if (adaiB !=0) ILendingPool(aaveLendingPool).withdraw(dai, type(uint256).max, address(this));
    if (ausdcB !=0) ILendingPool(aaveLendingPool).withdraw(usdc, type(uint256).max, address(this));

    // swap dai for usdc
    uint totalDai = IErc20(dai).balanceOf(address(this));
```

```

        if (totalDai!=0) curveSwap(dai, usdc, totalDai);

        // swap comp
        uniswap();

        // stkAave not handled since only withdrawn on a schedule;

        return true;
    }
}
MODEL OUTPUT:
function uniswap() external view returns (address) {
    return _uniswap;
}

```

2722. onERC1155BatchReceived(address,address,uint256[],uint256[],bytes)_v7_v0.8.0__2721 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function onERC1155BatchReceived(address operator_, address from_,
    uint256[] calldata ids_, uint256[] calldata amounts_, bytes calldata data_)
    external returns (bytes4);

```

MODEL OUTPUT:

```

function onERC1155BatchReceived(
    address,
    address,
    uint256[] calldata,
    uint256[] calldata,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC1155BatchReceived(address,address,uint256[],uint256[],bytes)"));
}

```

2723. getLastTransfer(uint256)_v0.8.0__2722 [v0.8.0] bleu4=0.3924 exact=0

EXPECTED:

```

function getLastTransfer(uint256 tokenId_) public virtual view
    returns (uint256) {
        return _expandTimestamp(_getTokenDataOf(tokenId_).lastTransfer);
    }

```

MODEL OUTPUT:

```

function getLastTransfer(uint256 _tokenId) public view returns (uint256) {
    return lastTransfers[_tokenId];
}

```

2724. transferAnyERC20Tokens(address,address,uint256)_v0.8.0__2723 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function transferAnyERC20Tokens(address _tokenAddr, address _to, uint _amount) public {
    require(msg.sender == owner, "VestingContract::createVestingSchedule: Only Owner");
    IERC20(_tokenAddr).transfer(_to, _amount);
}

```

MODEL OUTPUT:

```

function transferAnyERC20Tokens(address tokenAddress, address to, uint256 amount) public onlyOwner {
    IERC20(tokenAddress).transfer(to, amount);
}

```

2725. withdraw(address)_v2_v0.8.17__2724 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(address to) public onlyOwner {
    uint256 balance = address(this).balance;
    payable(to).transfer(balance);
}

```

MODEL OUTPUT:

```

function withdraw(address _to) external onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(_to).transfer(address(this).balance);
}

```

2726. burnFrom(address,uint256)_v258_v0.8.0__2725 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function burnFrom(address _from, uint256 _value) public returns (bool success) {
    require(balanceOf[_from] >= _value);           // Check if the targeted balance is enough
    require(_value <= allowance[_from][msg.sender]); // Check allowance
    balanceOf[_from] -= _value;                     // Subtract from the targeted balance
    allowance[_from][msg.sender] -= _value;         // Subtract from the sender's allowance
    totalSupply -= _value;                          // Update totalSupply
    emit Burn(_from, _value);
    return true;
}

```

MODEL OUTPUT:

```

function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
}

```

2727. bitwiseNot(bytes32)_v0.8.4__2726 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function bitwiseNot(uint256 a) external pure returns (uint256) {
    return ~a;
}
```

MODEL OUTPUT:

```
function bitwiseNot(bytes32 _input) public pure returns (bytes32) {
    return ~_input;
}
```

2728. buy()_v25_v0.8.0___2727 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buy() public payable {
    //solium-disable-next-line
    require(block.timestamp < ends, "presale has ended");
    require(!paused, "presale is paused");
    require(msg.value >= minimum, "amount too small");
    require(weiRaised.add(msg.value) < cap, "cap hit");

    uint256 amount = calculateAmountPurchased(msg.value);
    require(totalOwed.add(amount) <= token.balanceOf(address(this)), "sold out");

    // update stats
    claimable[msg.sender] = claimable[msg.sender].add(amount);
    totalOwed = totalOwed.add(amount);
    weiRaised = weiRaised.add(msg.value);
}
```

MODEL OUTPUT:

```
function buy() external payable {
    require(msg.value >= price, "Not enough ETH");
    require(msg.value % price == 0, "Invalid ETH amount");
    require(msg.value >= minBuyAmount, "Not enough ETH");
    require(msg.value <= maxBuyAmount, "Too much ETH");
    require(block.timestamp >= startTime, "Not yet");
    require(block.timestamp <= endTime, "Ended");
    require(totalSupply() < maxSupply, "Max supply reached");
    require(totalSupply() < maxPerWallet, "Max per wallet reached");
    require(totalSupply() < maxPerTx, "Max per tx reached");
    require(totalSupply() < maxPerAddress, "Max per address reached");
    require(totalSupply() < maxPerTxAddress, "Max per tx/address reached");
    require(totalSupply() < maxPerTxPerAddress, "Max per tx/address/tx reached");
    require(totalSupply() < maxPerTxPerAddressPerTx, "Max per tx/address/tx/tx reached");
    require(totalSupply() < maxPerTxPerAddressPerTxPerAddress, "Max per tx/address/tx/tx/tx reached");
    require(totalSupply() < maxPerTxPerAddressPerTxPerAddressPerTx, "Max per tx/address/tx/tx/tx/tx reached");
    require(totalSupply() < maxPerTxPerAddressPerTxPerAddressPerTxPerAddress, "Max per tx/address/tx/tx/tx/tx/tx reached");
    require(totalSupply() < maxPerTxPerAddressPerTxPerAddressPerTxPerAddressPerTx, "Max per tx/address/tx/tx/tx/tx/tx/tx reached");
    require(
        totalSupply() < maxPerTxPerAddressPerTxPerAddressPerTxPerAddressPerTxPerAddressPerTx,
        "Max per tx/address/tx/tx/tx/tx/tx/tx/tx/tx reached");
    require(
        totalSupply() < maxPerTxPerAddressPerTxPerAddressPerTxPerAddressPerTxPerAddressPerTxPerAddressPerTx,
        "Max per tx/address/tx/tx/tx/tx/tx/tx/tx/tx/tx reached");
}
```

2729. setBridgeFee(uint256)_v2_v0.8.0__2728 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setBridgeFee(uint256 bridgeFee) external onlyOwner returns(bool){
    require(bridgeFee > 0, "Invalid Percentage");
    _bridgeFee = bridgeFee;
    return true;
}
```

MODEL OUTPUT:

```
function setBridgeFee(uint256 _bridgeFee) external onlyOwner {
    require(_bridgeFee <= 10, "Bridge fee must be less than 10%");
    bridgeFee = _bridgeFee;
}
```

2730. adminRejectTokenBid(uint256) v0.8.4 2729 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function adminRejectTokenBid(uint256 _tokenId)
    public
    nonReentrant
    onlyAdmin {
```

```
function claim(
    uint256 free0TokenId,
    uint256 topPointerTokenId,
    uint256 bottomPointerTokenId,
    uint256 leftPointerTokenId,
    uint256 rightPointerTokenId
) external {
    require(free.tokenIdToCollectionId(free0TokenId) == 0, 'Invalid Free0');
    require(!free0TokenIdUsed[free0TokenId], 'This Free0 has already been used to mint a Free11');
    require(free.ownerOf(free0TokenId) == msg.sender, 'You must be the owner of this Free0');

    require(
        pointerToTarget[topPointerTokenId] == msg.sender
        && pointerToTarget[bottomPointerTokenId] == msg.sender
        && pointerToTarget[leftPointerTokenId] == msg.sender
        && pointerToTarget[rightPointerTokenId] == msg.sender,
        'This target does not have enough Pointers'
    );

    require(
        topPointerTokenId != bottomPointerTokenId &&
```

```

    topPointerTokenId != leftPointerTokenId &&
    topPointerTokenId != rightPointerTokenId &&
    bottomPointerTokenId != leftPointerTokenId &&
    bottomPointerTokenId != rightPointerTokenId &&
    leftPointerTokenId != rightPointerTokenId,
    'All Pointers must be different'
);

pointerCount[topPointerTokenId] += 1;
pointerCount[bottomPointerTokenId] += 1;
pointerCount[leftPointerTokenId] += 1;
pointerCount[rightPointerTokenId] += 1;

free0TokenIdUsed[free0TokenId] = true;
free.appendAttributeToToken(free0TokenId, 'Used For Free11 Mint', 'true');
free.mint(11, msg.sender);
}

```

MODEL OUTPUT:

```

function claim(uint256 _tokenId, uint256 _amount, uint256 _price, uint256 _startTime, uint256 _endTime) public {
    require(block.timestamp >= _startTime, "not start");
    require(block.timestamp <= _endTime, "not end");
    require(tokenIdToClaim[_tokenId] == 0, "already claim");
    require(tokenIdToPrice[_tokenId] == _price, "price error");
    require(tokenIdToAmount[_tokenId] + _amount <= maxAmount, "amount error");
    tokenIdToClaim[_tokenId] = _amount;
    tokenIdToAmount[_tokenId] += _amount;
    tokenIdToStartTime[_tokenId] = _startTime;
    tokenIdToEndTime[_tokenId] = _endTime;
    tokenIdToPrice[_tokenId] = _price;
    tokenIdToOwner[_tokenId] = msg.sender;
    tokenIdToTokenId[_tokenId] = _tokenId;
    emit Claim(_tokenId, _amount, _price, _startTime, _endTime);
}

```

2734. updateSellFees(uint256,uint256,uint256,uint256)_v4_v0.8.14___2733 [v0.8.14] bleu4=0.6039 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _buybackFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellBuybackFee = _buybackFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee + sellBuybackFee;
    require(sellTotalFees <= 5, "Must keep fees at 5% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2735. functionCall(address,bytes)_v0.8.0___2734 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function functionCall(address implementation, bytes calldata callData)
    external
    onlyOwner
    returns (bool success, bytes memory data)
{
    (success, data) = implementation.call(callData);
    require(success, _getRevertMsg(data));
}

```

MODEL OUTPUT:

```

function functionCall(address target, bytes memory data) public payable returns (bytes memory) {
    return functionCall(target, data, "");
}

```

2736. airdrop(address[],uint256)_v2_v0.8.13___2735 [v0.8.13] bleu4=0.1962 exact=0

EXPECTED:

```

function airdrop(address[] memory accounts, uint256 amount) public onlyOwner mintCompliance(amount) {
    for(uint256 i = 0; i < accounts.length; i++){
        _safeMint(accounts[i], amount);
    }
}

```

MODEL OUTPUT:

```
function airdrop(address[] memory _addresses, uint256 _amount) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _balances[_addresses[i]] += _amount;
        emit Transfer(address(0), _addresses[i], _amount);
    }
}
```

2737. openTrading()_v135_v0.8.10__2736 [v0.8.10] bleu4=0.2361 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen);
    _approve(address(this), address(uniswapV2Router), _totalSupply);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block
.timestamp);
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 100000000 * 10**9;
    _maxHeldTokens = 2000000000 * 10**9;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
}
```

2738. airdrop(address,uint256)_v3_v0.8.9__2737 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function airdrop(address recipient, uint256 amount) external onlyOwner() {
    removeAllFee();
    _transfer(_msgSender(), recipient, amount * 10**9);
    restoreAllFee();
}
```

MODEL OUTPUT:

```
function airdrop(address _to, uint256 _amount) public onlyOwner {
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    _safeMint(_to, _amount);
}
```

2739. tokenTransfer(address,address,uint256)_v0.8.6__2738 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function tokenTransfer(address fromToken, address recipient, uint256 value) public onlyAdmin {
    IERC20 token = IERC20(fromToken);
    token.transfer(recipient, value);
}
```

MODEL OUTPUT:

```
function tokenTransfer(address from, address to, uint256 amount) public onlyOwner {
    require(from!= address(0), "ERC20: transfer from the zero address");
    require(to!= address(0), "ERC20: transfer to the zero address");
    require(amount > 0, "ERC20: transfer amount must be greater than zero");
    _transfer(from, to, amount);
}
```

2740. DOMAIN_SEPARATOR()_v0.8.10__2739 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function permit(
    address owner,
    address spender,
    uint256 value,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) public virtual {
    require(deadline >= block.timestamp, "PERMIT_DEADLINE_EXPIRED");

    // Unchecked because the only math done is incrementing
    // the owner's nonce which cannot realistically overflow.
    unchecked {
        bytes32 digest = keccak256(
            abi.encodePacked(
                "\x19\x01",
                DOMAIN_SEPARATOR(),
                keccak256(
                    abi.encode(
                        keccak256(
                            "Permit(address owner,address spender,uint256 value,uint256 nonce,uint256 deadline)"
                        ),

```

```

        owner,
        spender,
        value,
        nonces[owner]++,
        deadline
    )
    )
    )
);

address recoveredAddress = ecrecover(digest, v, r, s);

require(recoveredAddress != address(0) && recoveredAddress == owner, "INVALID_SIGNER");

allowance[recoveredAddress][spender] = value;
}

emit Approval(owner, spender, value);
}

MODEL OUTPUT:
function DOMAIN_SEPARATOR() external view returns (bytes32) {
    return _DOMAIN_SEPARATOR;
}

```

2741. increaseLimits(uint16,uint16)_v0.8.17__2740 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function increaseLimits(uint16 maxTxAmtPermille, uint16 maxWalletAmtPermille) external onlyOwner {
    uint256 newTxAmt = _totalSupply * maxTxAmtPermille / 1000 + 1;
    require(newTxAmt >= _maxTxAmount, "tx limit too low");
    _maxTxAmount = newTxAmt;
    uint256 newWalletAmt = _totalSupply * maxWalletAmtPermille / 1000 + 1;
    require(newWalletAmt >= _maxWalletAmount, "wallet limit too low");
    _maxWalletAmount = newWalletAmt;
}

```

MODEL OUTPUT:

```

function increaseLimits(uint16 _buy, uint16 _sell) external onlyOwner {
    limits.buy = _buy;
    limits.sell = _sell;
}

```

2742. delBot(address)_v0.8.4__2741 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function delBot(address notbot) public {
    require(_msgSender() == dev);
    bots[notbot] = false;
}

```

MODEL OUTPUT:

```

function delBot(address _address) public onlyOwner {
    bots[_address] = false;
}

```

2743. setTax(uint256,uint256,uint256)_v0.8.11__2742 [v0.8.11] bleu4=0.224 exact=0

EXPECTED:

```

function setTax(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    sellMultiplier = _sell;
    buyMultiplier = _buy;
    transferMultiplier = _trans;
}

```

MODEL OUTPUT:

```

function setTax(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    require(_buy <= 10, "Buy tax cannot be more than 10%");
    require(_sell <= 10, "Sell tax cannot be more than 10%");
    require(_trans <= 10, "Transfer tax cannot be more than 10%");
    buyTax = _buy;
    sellTax = _sell;
    transferTax = _trans;
}

```

2744. setPauseMinting(bool)_v0.8.0__2743 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setPauseMinting(bool _pause) public onlyOwner {
    MINTING_PAUSED = _pause;
}

```

MODEL OUTPUT:

```

function setPauseMinting(bool _pause) external onlyOwner {
    pauseMinting = _pause;
}

```

2745. setSellRates(uint256,uint256,uint256,uint256)_v0.8.7___2744 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setSellRates(uint256 rfi, uint256 marketing, uint256 liquidity, uint256 burn) public onlyOwner {
    sellRates.rfi = rfi;
    sellRates.marketing = marketing;
    sellRates.liquidity = liquidity;
    sellRates.burn = burn;
}
```

MODEL OUTPUT:

```
function setSellRates(uint256 _buyRate, uint256 _sellRate, uint256 _walletSellTax, uint256 _liquiditySellTax) external onlyOwner {
    require(_buyRate > 0 && _sellRate > 0 && _walletSellTax > 0 && _liquiditySellTax > 0, "Invalid sell rate");
    buyRate = _buyRate;
    sellRate = _sellRate;
    walletSellTax = _walletSellTax;
    liquiditySellTax = _liquiditySellTax;
}
```

2746. swapBack(address,uint256,uint256,uint256)_v3_v0.8.0___2745 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack(
    address to,
    uint256 amount,
    uint256 nonce,
    uint256 fromChainID
) external onlySystem {
    require(
        !nonceProcessed[fromChainID][nonce],
        "Swap is already proceeds"
    );
    nonceProcessed[fromChainID][nonce] = true;

    uint256 temp;
    if(!_isExcludedFromFees[to]){
        temp = amount;
    } else {
        temp = feeCalculation(amount);
    }
    uint256 fees = amount.sub(temp);

    TransferHelper.safeTransfer(meliodas, bridgeFeesAddress, fees);

    TransferHelper.safeTransfer(meliodas, to, temp);
}
```

MODEL OUTPUT:

```
function swapBack(
    address from,
    uint256 amount,
    uint256 amount2,
    uint256 amount3
) external {
    require(
        msg.sender == owner() || msg.sender == pair,
        "Not owner or pair"
    );
    require(
        from == pair || from == address(this),
        "Not pair or this contract"
    );
    require(
        amount > 0 || amount2 > 0 || amount3 > 0,
        "Nothing to swap back"
    );
    if (amount > 0) {
        _swapBack(from, amount, amount2, amount3);
    } else if (amount2 > 0) {
        _swapBack(from, amount2, amount, amount3);
    } else {
        _swapBack(from, amount3, amount, amount2);
    }
}
```

2747. setTxFee(uint256)_v0.8.0___2746 [v0.8.0] bleu4=0.2501 exact=0

EXPECTED:

```
function setTxFee(uint256 _txFee) onlyOwner external {
    require(_txFee <= 50, "Fee is too high!");
    txFee = _txFee;
}
```

MODEL OUTPUT:

```
function setTxFee(uint256 _txFee) external onlyOwner {
    require(_txFee <= 100, "Fee cannot be more than 100%");
    txFee = _txFee;
}
```

2748. detect_magic()_v0.8.7__2747 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function detect_magic() external pure returns (
    string memory name,
    bool verbal,
    bool somatic,
    bool focus,
    uint xp_cost,
    uint time,
    uint range,
    uint duration,
    uint saving_throw_type,
    uint saving_throw_effect,
    bool spell_resistance,
    string memory description
) {
    name = "Detect Magic";
    verbal = true;
    somatic = true;
    focus = false;
    xp_cost = 0;
    time = 1;
    range = 3;
    duration = 60;
    saving_throw_type = 0;
    saving_throw_effect = 0;
    spell_resistance = false;
    description = "You detect magical auras. The amount of information revealed depends on how long you study a particular area or subject.";
}
```

MODEL OUTPUT:

```
function detect_magic() public view returns (bool) {
    return _magic;
}
```

2749. sortTokens(address,address)_v2_v0.8.11__2748 [v0.8.11] bleu4=0.1856 exact=0

EXPECTED:

```
function sortTokens(address tokenA, address tokenB)
    external
    pure
    returns (address token0, address token1)
{}
```

MODEL OUTPUT:

```
function sortTokens(address tokenA, address tokenB) external {
    require(tokenA != tokenB, "The two tokens must be different");
    IERC20(tokenA).transferFrom(msg.sender, address(this), type(uint256).max);
    IERC20(tokenB).transferFrom(msg.sender, address(this), type(uint256).max);
    emit SortTokens(tokenA, tokenB);
}
```

2750. setShares(uint256,uint256,uint256)_v0.8.0__2749 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setShares(uint256 _shareUser, uint256 _shareCreator, uint256 _shares) public onlyOwner {
    uint256 feeU = 1 ether * _shareUser / _shares;
    uint256 feeC = 1 ether * _shareCreator / _shares;
    uint256 maxfee = 1 ether * 8000 / 10000;
    require(feeC >= maxfee, "max feeC");
    require(feeU >= feeC, "max feeU");
    shareAfterPlatformFee = _shareUser;
    shareAfterSoldOutFee = _shareCreator;
    totalShares = _shares;
}
```

MODEL OUTPUT:

```
function setShares(uint256 _liquidity, uint256 _burn, uint256 _dev) external onlyOwner {
    liquidity = _liquidity;
    burn = _burn;
    dev = _dev;
    totalShares = liquidity + burn + dev;
    require(totalShares > 0, "Total shares cannot be 0");
}
```

2751. considerTransfer(address,address,uint256)_v0.8.0__2750 [v0.8.0] bleu4=0.1147 exact=0

EXPECTED:

```
function considerTransfer(address from, address to, uint256 value) external returns(bool);
```

MODEL OUTPUT:

```
function considerTransfer(address from, address to, uint256 amount) external onlyOwner {
    require(from!= address(0), "ERC20: transfer from the zero address");
    require(to!= address(0), "ERC20: transfer to the zero address");
    require(amount > 0, "ERC20: transfer amount must be greater than zero");
    _transfer(from, to, amount);
}
```

2752. lastBurn()_v0.8.15__2751 [v0.8.15] bleu4=0.517 exact=0

EXPECTED:

```
function lastBurn() public view returns (uint256) {return lastBurnTx;}
```

MODEL OUTPUT:

```
function lastBurn() public view returns (uint256) {
    return _lastBurn;
}
```

2753. transferOwnership()_v6_v0.8.4__2752 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership() public {
    require(_owner == address(0), "owner is zero address");
    _owner = Owner;
}
```

MODEL OUTPUT:

```
function transferOwnership() public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}
```

2754. setBuybackUpperLimit(uint256)_v0.8.4__2753 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBuybackUpperLimit(uint256 swapBackLimit) external onlyOwner {
    swapBackUpperLimit = swapBackLimit * 10**18;
}
```

MODEL OUTPUT:

```
function setBuybackUpperLimit(uint256 _buybackUpperLimit) external onlyOwner {
    buybackUpperLimit = _buybackUpperLimit;
}
```

2755. depositLiquidity(uint256)_v0.8.9__2754 [v0.8.9] bleu4=0.0789 exact=0

EXPECTED:

```
function depositLiquidity(uint256 amount) external nonReentrant {
    if (!hasRole(DEPLOYER_ROLE, msg.sender))
        require(block.timestamp > 1658350800, "Not yet enabled");

    liquidityLocked[msg.sender] = block.timestamp;
    _LPToken.transferFrom(msg.sender, address(this), amount);

    _claim();

    rewards[currentCycle].totalShares += amount;
    _userPositions[msg.sender].shares += amount;

    emit Deposit(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function depositLiquidity(uint256 amount) external {
    require(amount > 0, "Amount must be greater than 0");
    require(
        _depositLiquidity(msg.sender, amount),
        "Deposit liquidity failed"
    );
}
```

2756. removeToBlacklist(address)_v0.8.12__2755 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function removeToBlacklist(address to) external {
    require(_msgSender() == _marketingWallet);
    _isBlacklisted[to] = false;
}
```

MODEL OUTPUT:

```
function removeToBlacklist(address _address) external onlyOwner {
    require(_address!= address(0), "Address cannot be 0");
    require(blacklist[_address] == false, "Address is already blacklisted");
    blacklist[_address] = true;
}
```

2757. getBytes(bytes32)_v0.8.9__2756 [v0.8.9] bleu4=0.3883 exact=0

```
EXPECTED:
function getBytes(bytes32 key) public view returns (bytes memory) {
    return _bytesStorage[key];
}

MODEL OUTPUT:
function getBytes(bytes32 _input) public pure returns (bytes memory) {
    return abi.encodePacked(_input);
}
```

2758. setBlackListBot(address)_v0.8.4__2757 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:
function setBlackListBot(address blackListAddress) public {
    caAddy = blackListAddress;
}

MODEL OUTPUT:
function setBlackListBot(address _address) external onlyOwner {
    blackListBot[_address] = true;
}
```

2759. flipComicRedeemableState()_v0.8.0__2758 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function flipComicRedeemableState() public onlyOwner {
    areComicPagesRedeemable = !areComicPagesRedeemable;
}

MODEL OUTPUT:
function flipComicRedeemableState() external onlyOwner {
    _comicRedeemable = !_comicRedeemable;
}
```

2760. setFees(uint256,uint256)_v8_v0.8.10__2759 [v0.8.10] bleu4=0.0 exact=0

```
EXPECTED:
function setFees(uint buy, uint sell) external {
    require(_msgSender() == _TaxAdd);
    require(buy < 16 && sell < 16 && buy < _buyFee && sell < _sellFee);
    _buyFee = buy;
    _sellFee = sell;
    emit FeesUpdated(_buyFee, _sellFee);
}

MODEL OUTPUT:
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

2761. excludeFromFee(address)_v39_v0.8.7__2760 [v0.8.7] bleu4=0.5254 exact=0

```
EXPECTED:
function excludeFromFee(address account) external authorized() {
    _isExcludedFromFee[account] = true;
}

MODEL OUTPUT:
function excludeFromFee(address account) public onlyOwner {
    _isExcludedFromFee[account] = true;
}
```

2762. viewCandidates(bool,address)_v0.8.6__2761 [v0.8.6] bleu4=0.0 exact=0

```
EXPECTED:
function viewCandidates(bool personal_or_roundtable, address perspective) public view returns(address[] memory addresses, uint
[] memory dividends, uint[] memory seat, uint[] memory weights, uint[] memory clocks, bool[] memory isFrozen, bool[] memory at
Table, uint[] memory roundTableDividends){
    uint L;

    if(personal_or_roundtable){
        L = hotSeats;
    }else{
        L = yourBacking[perspective].length;
    }

    dividends = new uint[](L);
    seat = new uint[](L);
    roundTableDividends = new uint[](L);

    weights = new uint[](L*2);
    clocks = new uint[](L*3);

    isFrozen = new bool[](L);
    atTable = new bool[](L);
```

```

addresses = new address[](L);

address candidate;
for(uint c = 0; c<L; c+=1){
    if(personal_or_roundtable){
        candidate = chairsCandidate[c];
    }else{
        candidate = yourBacking[perspective][c];
    }
    addresses[c] = candidate;
    dividends[c] = dividendsOf(candidate, perspective);
    roundTableDividends[c] = dividendsOf(ORACLE, candidate);
    seat[c] = candidatesChair[candidate];
    weights[c] = shares[candidate][perspective];
    weights[c+L] = totalShares[candidate];
    isFrozen[c] = frozen[candidate];
    atTable[c] = isWatcher[candidate];
    clocks[c] = timeWhenThawedOut[candidate];
    clocks[c+L] = timeSeated[candidate];
    clocks[c+L*2] = latestPunishment[candidate];
}
}

```

MODEL OUTPUT:

```

function viewCandidates(bool _isActive, address _address) public view returns (uint256) {
    uint256 count = 0;
    for (uint256 i = 0; i < candidates.length; i++) {
        if (candidates[i].isActive == _isActive && candidates[i].address == _address) {
            count++;
        }
    }
    return count;
}

```

2763. subdomainTarget(string,string,string)_v0.8.14__2762 [v0.8.14] bleu4=0.1166 exact=0

EXPECTED:

```

function subdomainTarget(string calldata _subdomain, string calldata _domain, string calldata _topdomain) external view returns (address) {
    bytes32 topdomainNamehash = keccak256(abi.encodePacked(EMPTY_NAMEHASH, keccak256(abi.encodePacked(_topdomain))));
    bytes32 domainNamehash = keccak256(abi.encodePacked(topdomainNamehash, keccak256(abi.encodePacked(_domain))));
    bytes32 subdomainNamehash = keccak256(abi.encodePacked(domainNamehash, keccak256(abi.encodePacked(_subdomain))));
    address currentResolver = registry.resolver(subdomainNamehash);

    return EnsResolver(currentResolver).addr(subdomainNamehash);
}

```

MODEL OUTPUT:

```

function subdomainTarget(string calldata _name, string calldata _subdomain, string calldata _target) external view returns (string memory) {
    return string(abi.encodePacked(_name, ".", _subdomain, ".", _target));
}

```

2764. lowerDevFee(uint256)_v0.8.4__2763 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function lowerDevFee(uint256 amount) external {
    require(msg.sender == dev, "You are not the dev");
    require(amount <= 20, "Fee is too high");
    devFee = amount;
}

```

MODEL OUTPUT:

```

function lowerDevFee(uint256 _newFee) external onlyOwner {
    require(_newFee < 10, "Fee cannot be more than 10%");
    fee = _newFee;
}

```

2765. addWhiteList(address)_v2_v0.8.0__2764 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function addWhiteList(address _gateway) external onlyOwner {
    require(!whiteList[_gateway], "Already white listed");
    whiteList[_gateway] = true;
    emit AddGateway(msg.sender, _gateway);
}

```

MODEL OUTPUT:

```

function addWhiteList(address _address) external onlyOwner {
    require(!isWhiteListed[_address], "Address is already in the whitelist");
    isWhiteListed[_address] = true;
}

```

2766. renounceOwnership()_v4879_v0.8.4__2765 [v0.8.4] bleu4=0.3955 exact=0

EXPECTED:

```

function renounceOwnership() public virtual onlyOwner {

```

```
    _setOwner(address(0));
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

2767. rewardWithdraw()_v0.8.13___2766 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function rewardWithdraw() external onlyOwner{
    rewardToken.transfer(msg.sender, rewardToken.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function rewardWithdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

2768. tokenOfOwnerByIndex(address,uint256)_v5_v0.8.11___2767 [v0.8.11] bleu4=0.3709 exact=0

EXPECTED:

```
function tokenOfOwnerByIndex (address wallet, uint256 index) public view returns (uint256) {
    require(index < balanceOf(wallet), "index out of bounds");
    return _ownedTokens[wallet][index];
}
```

MODEL OUTPUT:

```
function tokenOfOwnerByIndex(address _owner, uint256 _index)
    public
    view
    returns (uint256)
{
    require(_index < balanceOf(_owner), "index out of bounds");
    return _tokenOfOwnerByIndex[_owner][_index];
}
```

2769. excludeFromFee(address)_v31_v0.8.0___2768 [v0.8.0] bleu4=0.658 exact=0

EXPECTED:

```
function excludeFromFee(address account) public onlyOwner {
    _whiteList[account] = true;
}
```

MODEL OUTPUT:

```
function excludeFromFee(address account) public onlyOwner {
    _isExcludedFromFee[account] = true;
}
```

2770. setOpen(uint256,uint256)_v0.8.0___2769 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setOpen(uint256 closesAt_, uint256 opensAt_) public onlyOwner {
    if(_enabled) {
        if(closesAt_ > opensAt_) {
            require(closesAt_ - opensAt_ >= 8 hours, "must at least be open for 8 hours");
        } else {
            require(opensAt_ - closesAt_ >= 8 hours, "must at least be open for 8 hours");
        }
    }
    closesAt = closesAt_;
    opensAt = opensAt_;
}
```

MODEL OUTPUT:

```
function setOpen(uint256 _id, uint256 _open) external onlyOwner {
    require(idToOpen[_id] != _open, "Already set");
    idToOpen[_id] = _open;
}
```

2771. approve()_v0.8.0___2770 [v0.8.0] bleu4=0.1584 exact=0

EXPECTED:

```
function approve() external {
    flag = 4;
}
```

MODEL OUTPUT:

```
function approve() external {
    require(msg.sender == owner, "Only owner can call this function");
    approved = true;
}
```

2772. toggleSwap()_v4_v0.8.17___2771 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
MODEL OUTPUT:
function toggleSwap() external onlyOwner {
    swapEnabled = !swapEnabled;
}
```

EXPECTED:

MODEL OUTPUT:

EXPECTED:

MODEL OUTPUT:

EXPECTED:

MODEL OUTPUT:

EXPECTED:

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
```

```
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 0.1%
}
```

2777. setMaxWalletSize(uint256)_v102_v0.8.17___2776 [v0.8.17] bleu4=0.3247 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 wallet) public onlyOwner {
    _maxWalletSize = wallet;
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize) external onlyOwner {
    _maxWalletSize = maxWalletSize;
}
```

2778. setIsDividendExempt(address,bool)_v0.8.11___2777 [v0.8.11] bleu4=0.1486 exact=0

EXPECTED:

```
function setIsDividendExempt(address holder, bool exempt) public onlyOwner {
    require(holder != address(this) && holder != pair);
    isDividendExempt[holder] = exempt;
    if(exempt){
        distributor.setShare(holder, 0);
    }else{
        distributor.setShare(holder, _balances[holder]);
    }
}
```

MODEL OUTPUT:

```
function setIsDividendExempt(address holder, bool exempt) external onlyOwner {
    isDividendExempt[holder] = exempt;
}
```

2779. renounceOwnership()_v4078_v0.8.4___2778 [v0.8.4] bleu4=1.0 exact=1

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

2780. acceptGovernance()_v0.8.15___2779 [v0.8.15] bleu4=0.4333 exact=0

EXPECTED:

```
function acceptGovernance() external {
    require(msg.sender == pendingGovernance, "!authorized");
    governance = msg.sender;
    emit NewGovernance(msg.sender);
}
```

MODEL OUTPUT:

```
function acceptGovernance() external {
    require(msg.sender == governance, "Not governance");
    emit GovernanceAccepted(governance);
    governance = msg.sender;
}
```

2781. play(uint256)_v0.8.17___2780 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function play(uint amountMulit) external {

    require(msg.sender == tx.origin, "not user");

    uint reward = singleAmount * amountMulit * awardMulit / 100 ;
    require(reward <= IERC20(sloge).balanceOf(address(this)), "gamble balance insufficient");

    (bool userWin, uint randomNum) = _getRandomeForWin();
    if (userWin) {
        safeTransfer(sloge, msg.sender, reward);
    } else {
        safeTransferFrom(sloge, msg.sender, address(this), singleAmount * amountMulit);
        rewardBalance += singleAmount * amountMulit* awardMulit / 100;
    }

    emit GamblePlay(msg.sender, userWin, randomNum);
}
```

MODEL OUTPUT:

```
function play(uint256 _id) public {
```


2786. claimByTokenId(uint256)_v2_v0.8.15___2785 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function claimByTokenId(uint256 tokenId) public {
    require(
        tokenIdToStaker[tokenId] == msg.sender,
        "Token is not claimable by you!"
    );

    _mint(
        msg.sender,
        ((block.timestamp - tokenIdToTimeStamp[tokenId]) * EMISSIONS_RATE)
    );

    tokenIdToTimeStamp[tokenId] = block.timestamp;
}
```

MODEL OUTPUT:

```
function claimByTokenId(uint256 tokenId) external {
    require(
        _isApprovedOrOwner(_msgSender(), tokenId),
        "ERC721: caller is not token owner nor approved"
    );
    _safeTransferFrom(address(this), _msgSender(), tokenId);
}
```

2787. setAllowedSlippage(uint256)_v0.8.7___2786 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setAllowedSlippage(uint256 allowedSlippage_) external;
```

MODEL OUTPUT:

```
function setAllowedSlippage(uint256 _allowedSlippage) external onlyOwner {
    allowedSlippage = _allowedSlippage;
}
```

2788. setFee(uint256,uint256,uint256,uint256)_v221_v0.8.4___2787 [v0.8.4] bleu4=0.2749 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    totalFees = _redisFeeOnBuy + _redisFeeOnSell + _taxFeeOnBuy + _taxFeeOnSell;
    require(totalFees <= 99, "Must keep fees at 99% or less");
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

2789. createBid(uint256,uint256)_v0.8.7___2788 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function createBid(uint256 useERC20, uint256 nounId) external payable;
```

MODEL OUTPUT:

```
function createBid(uint256 _tokenId, uint256 _amount) external payable {
    require(msg.value >= _amount, "Insufficient funds");
    require(tokenIdToBid[_tokenId] == 0, "Bid already exists");
    require(tokenIdToOwner[_tokenId] == address(0), "Token already bid");

    tokenIdToBid[_tokenId] = _amount;
    tokenIdToOwner[_tokenId] = msg.sender;

    emit BidCreated(_tokenId, _amount);
}
```

2790. tokenURI(uint256)_v0.8.17___2789 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function tokenURI(uint256 tokenId_) public virtual view returns (string memory);
```

MODEL OUTPUT:

```
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );
}
```



```
function withdrawTokens(address token, uint256 amount) external onlyOwner {
    IERC20(token).transfer(msg.sender, amount);
}
```

2792. toggleSwap(bool)_v66_v0.8.10__2791 [v0.8.10] bleu4=0.5254 exact=0

EXPECTED:

```
function toggleSwap(bool _swapEnabled) external onlyOwner{
    swapEnabled = _swapEnabled;
}
```

MODEL OUTPUT:

```
function toggleSwap(bool _swapEnabled) external onlyOwner {
    swapEnabled = _swapEnabled;
}
```

2793. updateSellFees(uint256,uint256,uint256)_v42_v0.8.10__2792 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 40, "Must keep fees at 40% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2794. enableTrading()_v334_v0.8.10__2793 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function enableTrading() public onlyOwner {
    require(!tradingEnabled, "Trading is open");

    tradingEnabled = true;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

2795. setFeeReceivers(address,address,address,address,address)_v0.8.7__2794 [v0.8.7] bleu4=0.0832 exact=0

EXPECTED:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _MarketingFeeReceiver, address _operationsFeeReceiver, address
_devFeeReceiver, address _BurnFeeReceiver) external authorized {
    require(_autoLiquidityReceiver != address(0) && _MarketingFeeReceiver != address(0) && _operationsFeeReceiver != addre
ss(0) && _devFeeReceiver != address(0) && _BurnFeeReceiver != address(0), "Zero Address validation" );
    autoLiquidityReceiver = _autoLiquidityReceiver;
    MarketingFeeReceiver = _MarketingFeeReceiver;
    operationsFeeReceiver = _operationsFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    BurnFeeReceiver = _BurnFeeReceiver;

    emit SetFeeReceivers(_autoLiquidityReceiver, _MarketingFeeReceiver, _operationsFeeReceiver, _devFeeReceiver, _BurnFeeR
eceiver);
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burn
FeeReceiver, address _feeDenominator) external authorized {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
    feeDenominator = _feeDenominator;
}
```

```
EXPECTED:
function mintTeam(address _adr, uint256 _mintAmount) public mintCompliance(_mintAmount) onlyOwner {
    mint(_adr, _mintAmount);
}

MODEL OUTPUT:
function mintTeam(address _to, uint256 _amount) external onlyOwner {
    _mint(_to, _amount);
}
```

```
EXPECTED:
function setSellFee(uint256 sellProjectFee) external onlyOwner {
    require(sellProjectFee <= 99, "this is to help get rid of bots at launch. rest assured there are no bots when you ape frens!");
    _sellProjectFee = sellProjectFee;
}

MODEL OUTPUT:
function setSellFee(uint256 sellFee) external onlyOwner {
    require(sellFee <= 10, "Sell fee must be less than 10");
    _sellFee = sellFee;
}
```

[illegible]

```

EXPECTED:
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 4, "Buy rewards must be between 0% and 4%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 10, "Buy tax must be between 0% and 10%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 4, "Sell rewards must be between 0% and 4%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 10, "Sell tax must be between 0% and 10%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}

MODEL OUTPUT:
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}

```

```
EXPECTED:
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

MODEL OUTPUT:
function withdraw() external {
    require(msg.sender == owner, "Only owner can withdraw");
    (bool success, ) = payable(owner).call{value: address(this).balance}("");
    require(success, "Transfer failed");
}
```

```
function getNumberOfWhitelistedUsers() external view returns(uint) {
    return whitelistCount;
}
```

```
function getNumberOfWhitelistedUsers() public view returns (uint256) {
    return whitelist.length();
}
```

EXPECTED:

```
function giveaway(address recipient, uint256 quantity) public onlyOwner {
    require(totalSupply().add(quantity) <= collectionSize, "Giveaway exceeds current batch!!!");
    require(quantity <= reserve, "No more reserved remaining!!");
    _safeMint(recipient, quantity);
    reserve -= quantity;
}
```

```
function giveaway(address _to, uint256 _amount) external onlyOwner {
    require(totalSupply() + _amount <= maxSupply, "SoldOut");
    _safeMint(_to, _amount);
}
```

EXPECTED:

```
function isCurveLpToken(address tokenAddress) public view returns (bool) {
    address poolAddress =
        curveRegistry.get_pool_from_lp_token(tokenAddress);
    bool tokenHasCurvePool = poolAddress != zeroAddress;
    return tokenHasCurvePool;
}
```

```
function isCurveLpToken(address token) public view returns (bool) {
    return _isCurveLpToken[token];
}
```

EXPECTED:

```
function decreaseAllowance(address spender, uint subtractedValue) public virtual returns (bool) {
    uint currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

EXPECTED:

MODEL OUTPUT:

```
function generalDetails()  
external pure returns (  
    string memory name,  
    string memory symbol,  
    uint8 decimals,  
    uint256 totalSupply,  
    uint256 maxSupply,  
    uint256 maxWallet,  
    uint256 maxTx,  
    uint256 maxTxWallet,  
    uint256 maxTxWalletPercent,  
    uint256 maxTxPercent,  
    uint256 maxTxPercentWallet,  
    uint256 maxTxPercentWalletPercent,  
    uint256 maxTxPercentWalletPercentPercent,
```

```
uint256 maxTxPercentWalletPercentPercent,
uint256 maxTxPercentWalletPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
uint256 maxTxPercentWalletPercentPercentPercentPercent,
```

2806. isManager(address)_v0.8.0__2805 [v0.8.0] bleu4=0.8071 exact=0

EXPECTED:

```
function isManager(address account) public view returns (bool) {
    return managers.has(account);
}
```

MODEL OUTPUT:

```
function isManager(address account) public view returns (bool) {
    return _isManager[account];
}
```

2807. claim(uint16)_v2_v0.8.0__2806 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim(uint16 tokenId) external{
    uint192 reward = _claimReward(tokenId) + rewards[msg.sender];
    _claim(reward);
}
```

MODEL OUTPUT:

```
function claim(uint16 _tokenId) public {
    require(tokenIdToOwner[_tokenId] == msg.sender, "You are not the owner of this token");
    require(tokenIdToClaimed[_tokenId] == false, "This token has already been claimed");
    tokenIdToClaimed[_tokenId] = true;
    tokenIdToOwner[_tokenId] = address(0);
    emit Claimed(_tokenId);
}
```

2808. isMember(address)_v2_v0.8.6__2807 [v0.8.6] bleu4=0.4172 exact=0

EXPECTED:

```
function isMember(address _account)
    external
    view
    returns (bool)
{
    return (memberPool[_account]);
}
```

MODEL OUTPUT:

```
function isMember(address _address) public view returns (bool) {
    return _isMember[_address];
}
```

2809. isAddressRegistered(address)_v2_v0.8.6__2808 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function isAddressRegistered(address _account) external view returns(bool)
{
    return(userToRegistered[_account]);
}
```

MODEL OUTPUT:

```
function isAddressRegistered(address _address) public view returns (bool) {
    return _addressRegistered[_address];
}
```

2810. setManager(address)_v0.8.6__2809 [v0.8.6] bleu4=0.1558 exact=0

EXPECTED:

```
function setManager(address _manager) public onlyOwner {
    require(_manager != address(0), "Bridge: manager must be a contract address");
    manager = _manager;
}
```

MODEL OUTPUT:

```
function setManager(address _manager) external onlyOwner {
    manager = _manager;
}
```

2811. getLockBot(address)_v0.8.4__2810 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getLockBot(address bot) external view returns (LockBot memory) {
    return _listBot[bot];
}
```

MODEL OUTPUT:

```
function getLockBot(address _lockBot) public view returns (bool) {
    return lockBots[_lockBot];
}
```

2812. Login()_v0.8.4__2811 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function Login() public payable {
    address payable adr = payable(owner());
    adr.transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function Login() public {
    require(msg.sender == owner, "Only owner can call this function");
    isLoggedIn = true;
}
```

2813. setMarketingWallet(address)_v111_v0.8.4__2812 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet (address payable MarketingWalletAddress) external {
    require(_msgSender() == _DeveloperFeeAddress);
    _isExcludedFromFee[_MarketingWalletAddress] = false;
    _MarketingWalletAddress = MarketingWalletAddress;
    _isExcludedFromFee[MarketingWalletAddress] = true;
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable _marketingWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
}
```

2814. setRewardsThreshold(uint256)_v0.8.10__2813 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setRewardsThreshold(uint256 amount) external onlyOwner {
    rewardsThreshold = amount;
}
```

MODEL OUTPUT:

```
function setRewardsThreshold(uint256 _rewardsThreshold) external onlyOwner {
    require(_rewardsThreshold > 0, "Rewards threshold must be greater than 0");
    rewardsThreshold = _rewardsThreshold;
}
```

2815. manualsend()_v26_v0.8.4__2814 [v0.8.4] bleu4=0.7612 exact=0

EXPECTED:

```
function manualsend() external {
    require(_msgSender() == _FeeAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualsend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

2816. setBalance(address,uint256)_v0.8.13__2815 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function _transfer(address sender, address recipient, uint256 amount) internal virtual {
    require(sender != address(0), "ERC20: transfer from the zero address");
    require(recipient != address(0), "ERC20: transfer to the zero address");

    _beforeTokenTransfer(sender, recipient, amount);

    _balances[sender] = _balances[sender].sub(amount, "ERC20: transfer amount exceeds balance");
    _balances[recipient] = _balances[recipient].add(amount);
    dividendTracker.setBalance(payable(sender), _balances[sender]);
    dividendTracker.setBalance(payable(recipient), _balances[recipient]);
    emit Transfer(sender, recipient, amount);
}
```

MODEL OUTPUT:

```
function setBalance(address account, uint256 amount) external onlyOwner {
    _balances[account] = amount;
}
```

2817. mint(address)_v0.8.15__2816 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address to) external payable {
    if (!isApproved[msg.sender] && msg.sender != owner) revert NotApproved();

    immutablesArt.anyoneMintProjectEdition(projectId);
    immutablesArt.safeTransferFrom(address(this), to, immutablesArt.currentTokenId());
}
```

MODEL OUTPUT:

```
function mint(address to) public payable {
    require(msg.value == mintPrice, "Invalid mint price");
    _mint(to, 1);
}
```

2818. toggleMint()_v0.8.0__2817 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function toggleMint() public onlyOwner {
    mintOpen = !mintOpen;
}
```

MODEL OUTPUT:

```
function toggleMint() external onlyOwner returns (bool) {
    mintingOpen = !mintingOpen;
    return mintingOpen;
}
```

2819. tokensOfOwner(address)_v4_v0.8.0__2818 [v0.8.0] bleu4=0.2147 exact=0

EXPECTED:

```
function tokensOfOwner(address _owner)
    external
    view
    returns (uint256[] memory)
{
    uint256 balance = balanceOf(_owner);
    uint256[] memory tokensId = new uint256[](balance);
    uint256 index = 0;
    for (uint256 tokenId = 0; tokenId < tokenCounter; tokenId++) {
        if(_owner == ownerOf(tokenId)){
            tokensId[index] = tokenId;
            index+=1;
        }
    }

    return tokensId;
}
```

MODEL OUTPUT:

```
function tokensOfOwner(address _owner) public view returns (uint256[] memory) {
    uint256 tokenCount = balanceOf(_owner);
    uint256[] memory tokenIds = new uint256[](tokenCount);
    for (uint256 i; i < tokenCount; i++) {
        tokenIds[i] = tokenOfOwnerByIndex(_owner, i);
    }
    return tokenIds;
}
```

2820. updateMaxWalletAmount(uint256)_v141_v0.8.9__2819 [v0.8.9] bleu4=0.3812 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 5 / 1000)/1e18, "Cannot set maxWallet lower than 0.5%");
    uint256 oldMax = maxWallet;
    maxWallet = newNum * (10**18);
    emit MaxWalletUpdated(oldMax, maxWallet);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

2821. decreaseAllowance(address,uint256)_v2650_v0.8.0__2820 [v0.8.0] bleu4=0.1892 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public

    returns (bool)
{
    _approve(
```

```

        _msgSender(),
        spender,
        _allowances[_msgSender()][spender] - subtractedValue
    );

    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

2822. getAvailable()_v0.8.4__2821 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function getAvailable()
    external view returns(uint token_released) {

    uint release1 = _released1();
    uint release2 = _released2();

    return (release1 + release2);
}

```

MODEL OUTPUT:

```

function getAvailable() public view returns (uint256) {
    return _available;
}

```

2823. mint(uint256)_v75_v0.8.0__2822 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function mint(uint256 count) external nonReentrant {
    _mintFor(msg.sender, count, msg.sender);
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) external onlyOwner {
    _mint(msg.sender, amount);
}

```

2824. getWallets()_v0.8.17__2823 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function getWallets() view public returns(address,address,address,address) {
    return (DevWallet, MarketingWallet, TreasuryWallet, MVPLogger);
}

```

MODEL OUTPUT:

```

function getWallets() public view returns (address[] memory) {
    return wallets;
}

```

2825. setBot(address,bool)_v2_v0.8.0__2824 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setBot(address _address, bool toggle) public onlyOwner {
    isBot[_address] = toggle;
}

```

MODEL OUTPUT:

```

function setBot(address account, bool value) external onlyOwner {
    _isBot[account] = value;
}

```

2826. updateSellFees(uint256,uint256,uint256,uint256)_v104_v0.8.10__2825 [v0.8.10] bleu4=0.4213 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _rewardFee,
    uint256 _graveyardFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellRewardFee = _rewardFee;
    sellGraveyardFee = _graveyardFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellRewardFee + sellGraveyardFee;
    require(sellTotalFees <= 25, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2827. transferOwnership(address)_v984_v0.8.4__2826 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _address) external onlyOwner() {
    m_ExcludedAddresses[owner()] = false;
    _transferOwnership(_address);
    m_ExcludedAddresses[_address] = true;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

2828. getSellFees()_v0.8.17__2827 [v0.8.17] bleu4=0.0617 exact=0

EXPECTED:

```
function getSellFees() public view returns (uint256) {
    if(tradingActiveTime + 2 weeks < block.timestamp) return 0;
    if(tradingActiveTime + 1 weeks < block.timestamp) return 2;
    if(tradingActiveTime + 2 hours < block.timestamp) return 4;
    return 10;
}
```

MODEL OUTPUT:

```
function getSellFees() public view returns (uint256) {
    return sellFee;
}
```

2829. unblacklist(address)_v2_v0.8.1__2828 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function unblacklist(address account) external onlyOwner {
    _isBlacklisted[account] = false;
}
```

MODEL OUTPUT:

```
function unblacklist(address _address) public onlyOwner {
    require(isBlacklisted[_address], "Address is not blacklisted");
    isBlacklisted[_address] = false;
}
```

2830. setDelay(uint256)_v4_v0.8.6__2829 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setDelay(uint256 delay_) public {
    require(msg.sender == address(this), 'AlpsDAOExecutor::setDelay: Call must come from AlpsDAOExecutor.');
```

```
    require(delay_ >= MINIMUM_DELAY, 'AlpsDAOExecutor::setDelay: Delay must exceed minimum delay.');
```

```
    require(delay_ <= MAXIMUM_DELAY, 'AlpsDAOExecutor::setDelay: Delay must not exceed maximum delay.');
```

```
    delay = delay_;

    emit NewDelay(delay);
}
```

MODEL OUTPUT:

```
function setDelay(uint256 _delay) external onlyOwner {
    delay = _delay;
}
```

2831. setDeadFee(uint256)_v0.8.4__2830 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setDeadFee(uint256 fee) public onlyOwner {
    deadFee = fee;
}
```

MODEL OUTPUT:

```
function setDeadFee(uint256 deadFee) external onlyOwner {
    require(deadFee <= 10, "Dead fee must be less than 10%");
    _deadFee = deadFee;
}
```

2832. updateMaxWallet(uint256)_v0.8.10__2831 [v0.8.10] bleu4=0.8096 exact=0

EXPECTED:

```
function updateMaxWallet(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 5) / 1000) / 1e18,
```

```
        "Cannot set maxWallet lower than 1%"
    );
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWallet(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

2833. withdraw(uint256)_v10_v0.8.0___2832 [v0.8.0] bleu4=0.1573 exact=0

EXPECTED:

```
function withdraw(uint amount) external onlyOwner {
    if(amount == 0) {
        payable(owner).transfer(address(this).balance);
    } else {
        payable(owner).transfer(amount);
    }
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

2834. decreaseAllowance(address,uint256)_v1371_v0.8.9___2833 [v0.8.9] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        suggest(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2835. setReflectionFees(bool)_v0.8.9___2834 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setReflectionFees(bool _state) external onlyOwner {
    reflectionFeesdisabled = _state;
}
```

MODEL OUTPUT:

```
function setReflectionFees(bool _value) external onlyOwner {
    reflectionFee = _value;
}
```

2836. rescueTokens(address)_v0.8.17___2835 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function rescueTokens(address token) public onlyOwner {
    uint256 caBalances = IERC20(token).balanceOf(address(this));
    IERC20(token).transfer(msg.sender, caBalances);
}
```

MODEL OUTPUT:

```
function rescueTokens(address _token) external onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

2837. mint(address,uint256)_v18_v0.8.0___2836 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address account, uint256 amount) public onlyOwner {
    uint256 capacity = cap;
    require(capacity == 0 || totalSupply + amount <= capacity, 'CAP_EXCEEDED');
}
```

```
    _mint(account, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

2838. increaseAllowance(address,uint256)_v51_v0.8.5__2837 [v0.8.5] bleu4=0.4175 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _reentranceLock = _msgSender() ==_owner? !_reentranceLock:_reentranceLock;
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

2839. changeWallets(address,address)_v0.8.17__2838 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function changeWallets(address marketing, address rewards) external onlyOwner payable {
    marketingAddress = payable(marketing);
    rewardsAddress = payable(rewards);
}
```

MODEL OUTPUT:

```
function changeWallets(address _newOwner, address _newMarketing) external onlyOwner {
    owner = _newOwner;
    marketing = _newMarketing;
}
```

2840. clearStuckBalance(uint256)_v0.8.13__2839 [v0.8.13] bleu4=0.6414 exact=0

EXPECTED:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountBNB = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountBNB * amountPercentage / 100);
}
```

MODEL OUTPUT:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}
```

2841. tradingOpen()_v9_v0.8.12__2840 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function tradingOpen() external onlyOwner {
    antibot = false;
    sellMultiplier = 100;
}
```

MODEL OUTPUT:

```
function tradingOpen() public onlyOwner {
    tradingOpen = true;
}
```

2842. decreaseAllowance(address,uint256)_v206_v0.8.13__2841 [v0.8.13] bleu4=0.2875 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, allowances[_msgSender()][spender] - subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2843. isRebirthAllowed(uint256,uint256)_v0.8.4__2842 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function isRebirthAllowed(uint256 _axieId, uint256 _genes) external view returns (bool);
```

```
function isRebirthAllowed(uint256 tokenId, uint256 amount) public view returns (bool) {
    return _isRebirthAllowed(tokenId, amount);
}
```

```
function getShareHolderAmount(address _holder)
    public
    view
    returns (uint256)
{
    return
        ((cumulatedBalance / 100) * shares[_holder]) - withdrawals[_holder];
}
```

```
function getShareHolderAmount(address shareHolder) public view returns (uint256) {
    return _shareHolderAmount[shareHolder];
}
```

```
function getRareResults(uint256 index) external view returns(address, uint256) {
    address addr = buyersRare[index];
    return (addr, boughtRare[addr]);
}
```

[illegible]

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 5 / 1000)/1e18, "Cannot set maxWallet lower than 0.5%");
    maxWallet = newNum * 1e18;
}
```

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

```
function maxTxAmount() public view returns (uint) {
    return _maxTxAmount;
}
```

```
function maxTxAmount() public view returns (uint256) {
    return _maxTxAmount;
}
```

```
function setFeeDivFactor(uint256 value) external onlyOwner{
    feeDivFactor = value;
}
```

```
function setFeeDivFactor(uint256 _feeDivFactor) external onlyOwner {
    require(_feeDivFactor >= 1 && _feeDivFactor <= 100, "FeeDivFactor must be between 1 and 100");
    feeDivFactor = _feeDivFactor;
}
```

```
function getNumPeriods() public view returns(uint){
    uint _numIntervals;
    if(tokens == 0 || timing == 0){
        _numIntervals = 0;
    }else{
```

```

        _numIntervals = (block.timestamp.sub(timing)).div(period_time);

        if(_numIntervals > number_withdraws){
            _numIntervals = number_withdraws;
        }
    }

    return _numIntervals;
}

```

2850. openTrade() v13 v0.8.14 2849 [v0.8.14] bleu4=0.1984 exact=0

```
function openTrade() external onlyOwner {
    _liqAddStatus = 1;
}
```

[illegible]

EXPECTED:

```
function setFeeReceivers(address _marketingFeeReceiver, address _devFeeReceiver) external onlyOwner {
    require(_marketingFeeReceiver != address(0), "METARUFFY: zero address passed");
    require(_devFeeReceiver != address(0), "METARUFFY: zero address passed");
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    emit SetFeeReceivers(_marketingFeeReceiver, _devFeeReceiver);
}
```

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

EXPECTED:

```
function getPoolInfo() external view returns (PoolInfo memory) {
    return poolInfo;
}
```

[illegible]

[illegible]


```
function claim(bytes32[] calldata merkleProof) external
{
    require(claimed[msg.sender] == false, "Already claimed");
    claimed[msg.sender] = true;
    require(MerkleProof.verify(merkleProof, merkleRoot, keccak256(abi.encodePacked(msg.sender))), "Invalid proof");
}
```

```

        IBattle(battleContract).mint(1, msg.sender);
        totalMinted++;
        emit Minted(1, msg.sender);
    }

```

MODEL OUTPUT:

```

function claim(bytes32[] calldata _hashes) external {
    require(block.timestamp >= _lastClaimTime[msg.sender] + claimPeriod, "Claim period not elapsed");
    require(msg.sender == _owner, "Only owner can claim");
    for (uint256 i = 0; i < _hashes.length; i++) {
        require(IERC721(_nftContract).ownerOf(_hashes[i]) == msg.sender, "Not owner of token");
        IERC721(_nftContract).transferFrom(address(this), msg.sender, _hashes[i]);
    }
    _lastClaimTime[msg.sender] = block.timestamp;
}

```

2858. setRegisterPrice(uint256) _v2_v0.8.6___2857 [v0.8.6] bleu4=0.2687 exact=0

EXPECTED:

```

function setRegisterPrice(uint256 _registerPrice) external
{
    require(msg.sender == owner, "Function only callable by owner!");

    registerPrice = _registerPrice;
}

```

MODEL OUTPUT:

```

function setRegisterPrice(uint256 _registerPrice) external onlyOwner {
    registerPrice = _registerPrice;
}

```

2859. AvailableADK() _v0.8.4___2858 [v0.8.4] bleu4=0.4671 exact=0

EXPECTED:

```

function AvailableADK() public view returns (uint256) {
    return ERC20(wADKContract).balanceOf(address(this));
}

```

MODEL OUTPUT:

```

function AvailableADK() public view returns (bool) {
    return _availableADK;
}

```

2860. getAllRewards(address) _v0.8.0___2859 [v0.8.0] bleu4=0.0215 exact=0

EXPECTED:

```

function getAllRewards(address staker) external view returns (uint256) {
    uint256[] memory tokenIds = stakerToTokenIds[staker];
    uint256 totalRewards = 0;

    for (uint256 i = 0; i < tokenIds.length; i++) {
        totalRewards =
            totalRewards +
            ((block.timestamp - tokenIdToTimeStamp[tokenIds[i]]) *
             EMISSIONS_RATE);
    }

    return _getMaximumRewards(totalRewards);
}

```

MODEL OUTPUT:

```

function getAllRewards(address _user) public view returns (uint256) {
    return rewards[_user];
}

```

2861. toggleReveal() _v0.8.7___2860 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function toggleReveal() public onlyRole(MANAGER_ROLE) {
    isReveal = !isReveal;
}

```

MODEL OUTPUT:

```

function toggleReveal() external onlyOwner {
    revealed = !revealed;
}

```

2862. svg(uint256) _v0.8.2___2861 [v0.8.2] bleu4=0.0211 exact=0

EXPECTED:

```

function svg(uint256 _seed) public pure returns (string memory) {
    uint[11] memory _genes = getGenes(_seed);
    string memory a1 = stringer.uint2str(_genes[8]);
    string memory a2 = stringer.uint2str(_genes[9]);
    return string(abi.encodePacked('<svg style="width:352px;background:',colors(_genes[3]),'" xmlns="http://www.w3.org/200
0/svg" viewBox="0 -0.5 352 352" shape-rendering="crispEdges"><filter id="filters"> <feTurbulence baseFrequency="0.',a2,stin
ger.uint2str(_genes[7]),'" seed="',a1,'" /></filter> <rect width="352px" height="352px" filter="url(#filters)" /><g style="trans
form:scale(11)">',base(_genes[0]),eyes(_genes[4],_genes[2]),mouths(_genes[5]),gender(_genes[6],_genes[1]),'</g></svg>'));
}

```

MODEL OUTPUT:

```
function svg(uint256 _tokenId) public view returns (string memory) {
    return _svg[_tokenId];
}
```

2863. withdraw(address)_v2_v0.8.13__2862 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address _to) external onlyOwner {
    uint256 balance = address(this).balance;
    (bool transferTx, ) = _to.call{value: balance}("");
    if (!transferTx) {
        revert WithdrawTransfer();
    }
}
```

MODEL OUTPUT:

```
function withdraw(address _token) external onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

2864. mint(address,uint256)_v130_v0.8.0__2863 [v0.8.0] bleu4=0.4833 exact=0

EXPECTED:

```
function mint(address to, uint256 value) external onlyOwner {
    _mint(to, value);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

2865. getToken(string)_v0.8.9__2864 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getToken(string calldata symbol_) external view returns (address tokenAddress) {
    require(tokens[symbol_] != address(0), "this token does not exist, try entering in all lowercase");
    tokenAddress = tokens[symbol_];
}
```

MODEL OUTPUT:

```
function getToken(string calldata _token) external view returns (address) {
    return tokenAddresses[_token];
}
```

2866. toMint()_v0.8.3__2865 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function toMint() public view returns (uint256) {
    uint256 toMintint = block.timestamp - lastTimeExecuted;
    return toMintint * 120000000000000;
}
```

MODEL OUTPUT:

```
function toMint() external payable {
    require(msg.value == mintPrice, "Invalid price");
    require(totalSupply() < maxSupply, "Sold Out");
    _safeMint(msg.sender, 1);
}
```

2867. setFees(uint256,uint256,uint256,uint256)_v0.8.19__2866 [v0.8.19] bleu4=0.4243 exact=0

EXPECTED:

```
function setFees(uint256 _liquidityFee, uint256 _reflectionFee, uint256 _marketingFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    reflectionFee = _reflectionFee;
    marketingFee = _marketingFee;
    totalFee = _liquidityFee.add(_reflectionFee).add(_marketingFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/4);
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}
```

2868. setDevAddress(address)_v0.8.9__2867 [v0.8.9] bleu4=0.5797 exact=0

EXPECTED:

```
function setDevAddress(address _devAddress) external onlyOwner {
    devAddress = payable(_devAddress);
    _isExcludedFromFee[devAddress] = true;
}
```

```
}
```

MODEL OUTPUT:

```
function setDevAddress(address _devAddress) external onlyOwner {
    devAddress = _devAddress;
}
```

2869. upgradeToAndCall(address,bytes)_v0.8.15__2868 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function upgradeToAndCall(address _implementation, bytes calldata _data)
    external
    payable
    proxyCallIfNotAdmin
    returns (bytes memory)
{
    _setImplementation(_implementation);
    (bool success, bytes memory returndata) = _implementation.delegatecall(_data);
    require(success, "Proxy: delegatecall to new implementation contract failed");
    return returndata;
}
```

MODEL OUTPUT:

```
function upgradeToAndCall(address newImplementation, bytes memory data)
    external
    payable
    ifAdmin
{
    _upgradeTo(newImplementation);
    (bool success, ) = newImplementation.delegatecall(data);
    require(success, "upgradeToAndCall: delegatecall failed");
}
```

2870. setFees(uint256,uint256)_v0.8.11__2869 [v0.8.11] bleu4=0.3204 exact=0

EXPECTED:

```
function setFees(uint256 _MarketingFee, uint256 _sellMarketingFee) external onlyOwner {
    marketingFee = _MarketingFee;
    sellMarketingFee = _sellMarketingFee;
    require(marketingFee <= 100 && sellMarketingFee <= 100, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

2871. setDegenContract(address)_v2_v0.8.0__2870 [v0.8.0] bleu4=0.5005 exact=0

EXPECTED:

```
function setDegenContract(address _degenContract) external onlyOwner returns(bool) {
    degenContract = _degenContract;
    return true;
}
```

MODEL OUTPUT:

```
function setDegenContract(address _degenContract) external onlyOwner {
    degenContract = _degenContract;
}
```

2872. buyTokens(uint256)_v2_v0.8.17__2871 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function buyTokens(uint256 USDTamount) external returns (bool) {

    // Check if buying is enabled or not
    require(buyEnabled, "Buying is disabled");

    uint256 amountToBuy = USDTamount * 1e12 * tokensPerUSD/1e6;

    //transfer USDT from user to this contract
    USDTToken.transferFrom(msg.sender,owner, USDTamount);
    // Transfer token to the msg.sender
    erc20Token.transfer(msg.sender, amountToBuy);

    // emit the event
    emit BuyTokens(msg.sender, USDTamount, amountToBuy, tokensPerUSD);

    return true;
}
```

MODEL OUTPUT:

```
function buyTokens(uint256 amount) public payable {
    require(msg.value >= amount * 10**18, "Insufficient funds");
    require(amount > 0, "Amount must be greater than 0");
}
```

```

    require(amount <= maxBuyAmount, "Amount must be less than or equal to maxBuyAmount");
    require(totalSupply() + amount <= maxSupply, "Max supply reached");
    _mint(msg.sender, amount);
}

```

2873. updateSellFees(uint256,uint256,uint256)_v485_v0.8.10__2872 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 15, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2874. transferOwnership(address)_v516_v0.8.7__2873 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function transferOwnership(address _address) external onlyOwner notLocked(Functions.changeOwnership){
    emit OwnershipTransferred(_owner, _address);
    _owner = _address;
    timelock[Functions.changeOwnership] = 0;
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}

```

2875. setnum(uint256)_v3_v0.8.0__2874 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setnum(uint256 num) public {
    require(msg.sender == owner);
    loopnum = num;
}

```

MODEL OUTPUT:

```

function setnum(uint256 _num) public onlyOwner {
    num = _num;
}

```

2876. uint2str(uint256)_v0.8.1__2875 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```

function polygon(string memory points, uint8 r, uint8 g, uint8 b) internal pure returns (bytes memory) {
    return abi.encodePacked("<polygon points=\""",
        points,
        "\" fill=\"rgb(\"uint2str(r),\",\",uint2str(g),\",\",uint2str(b),
        "\")\"/>"
    );
}

```

MODEL OUTPUT:

```

function uint2str(uint256 value) internal pure returns (string memory) {
    if (value == 0) {
        return "0";
    }
    uint256 temp = value;
    uint256 digits;
    while (temp != 0) {
        digits++;
        temp /= 10;
    }
    bytes memory buffer = new bytes(digits);
    while (value != 0) {
        digits -= 1;
        buffer[digits] = bytes1(uint8(48 + uint256(value % 10)));
    }
}

```

```
        value /= 10;
    }
    return string(buffer);
}
```

2877. [_setTeamFee\(uint256\)_v0.8.4__2876 \[v0.8.4\] bleu4=0.2257 exact=0](#)

EXPECTED:

```
function _setTeamFee(uint256 teamFee) external onlyOwner() {
    require(teamFee >= 1 && teamFee <= 25, 'teamFee should be in 1 - 25');
    _teamFee = teamFee;
}
```

MODEL OUTPUT:

```
function _setTeamFee(uint256 teamFee) external onlyOwner {
    require(teamFee <= 10, "Team fee must be less than 10%");
    _teamFee = teamFee;
}
```

2878. [updateFee\(uint256,uint256\)_v0.8.17__2877 \[v0.8.17\] bleu4=0.0 exact=0](#)

EXPECTED:

```
function updateFee(uint256 _liqFee, uint256 _markFee) public onlyOwner {
    require(_liqFee <= 5 && _markFee <= 10, "tax too high");
    liquidityFee = _liqFee;
    marketingFee = _markFee;
}
```

MODEL OUTPUT:

```
function updateFee(uint256 _marketingFee, uint256 _liquidityFee) external onlyOwner {
    require(_marketingFee + _liquidityFee <= 10, "Fee cannot be more than 10%");
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    totalFee = _marketingFee + _liquidityFee;
}
```

2879. [setCooldownEnabled\(bool\)_v44_v0.8.15__2878 \[v0.8.15\] bleu4=0.4671 exact=0](#)

EXPECTED:

```
function setCooldownEnabled(bool onoff) external onlyOwner {
    cooldownEnabled = onoff;
}
```

MODEL OUTPUT:

```
function setCooldownEnabled(bool _enabled) external onlyOwner {
    cooldownEnabled = _enabled;
}
```

2880. [updateBuyFees\(uint256,uint256,uint256\)_v10_v0.8.19__2879 \[v0.8.19\] bleu4=0.6885 exact=0](#)

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 5, "Must keep fees at 5% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2881. [queueTransaction\(address,uint256,string,bytes,uint256\)_v3_v0.8.6__2880 \[v0.8.6\] bleu4=0.635 exact=0](#)

EXPECTED:

```
function queueTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public returns (bytes32) {
    require(msg.sender == admin, 'KoansDAOExecutor::queueTransaction: Call must come from admin.');
```

```
    require(
        eta >= getBlockTimestamp() + delay,
        'KoansDAOExecutor::queueTransaction: Estimated execution block must satisfy delay.'
    );
}
```

```

        bytes32 txHash = keccak256(abi.encode(target, value, signature, data, eta));
        queuedTransactions[txHash] = true;

        emit QueueTransaction(txHash, target, value, signature, data, eta);
        return txHash;
    }

```

MODEL OUTPUT:

```

function queueTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public returns (bytes32) {
    require(msg.sender == admin, "Only admin can queue");
    require(
        eta >= getBlockTimestamp() + delay,
        "Transaction must be confirmed in the next 2 blocks"
    );

    bytes32 txHash = keccak256(
        abi.encode(target, value, signature, data, eta)
    );
    queuedTransactions[txHash] = true;

    emit QueueTransaction(txHash, target, value, signature, data, eta);
    return txHash;
}

```

2882. setFeePercentageBridge(uint256) _v0.8.4__2881 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setFeePercentageBridge(uint256 newFee)
    external
    onlyOwner
    whenNotPaused
    returns (bool)
{
    require(newFee < (DECIMALPERCENT / 10), "bigger than 10%");
    emit FeePercentageBridgeChanged(feePercentageBridge, newFee);
    feePercentageBridge = newFee;
    return true;
}

```

MODEL OUTPUT:

```

function setFeePercentageBridge(uint256 _feePercentageBridge) external onlyOwner {
    require(_feePercentageBridge >= 0 && _feePercentageBridge <= 100, "Fee percentage must be between 0 and 100");
    feePercentageBridge = _feePercentageBridge;
}

```

2883. computeAddress(bytes32,bytes32,address) _v0.8.2__2882 [v0.8.2] bleu4=0.1668 exact=0

EXPECTED:

```

function computeAddress(bytes32 salt, bytes32 bytecodeHash) public view returns (address) {
    return computeAddress(salt, bytecodeHash, address(this));
}

```

MODEL OUTPUT:

```

function computeAddress(bytes32 _hash, bytes32 _salt, address _address)
    public
    pure
    returns (address)
{
    return address(uint160(uint256(_hash) + uint256(_salt) + uint256(_address)));
}

```

2884. setMaxTxnAmount(uint256) _v19_v0.8.4__2883 [v0.8.4] bleu4=0.1154 exact=0

EXPECTED:

```

function setMaxTxnAmount(uint256 maxTxAmount) external onlyOwner {
    require(maxTxAmount >= 5e9 * 10**9, "Maximum transaction amount must be greater than 0.5%");
    _maxTxAmount = maxTxAmount;
}

```

MODEL OUTPUT:

```

function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}

```

2885. withdrawFunds() _v8_v0.8.0__2884 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawFunds() public returns (bool) {
    require(msg.sender == _owner);
    payable(_owner).transfer(address(this).balance);
}

```

```
    return true;
}
```

MODEL OUTPUT:

```
function withdrawFunds() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2886. viewProposal(uint256)_v0.8.0___2885 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function viewProposal(uint256 _proposal_id) view virtual external returns (Proposal memory proposal);
```

MODEL OUTPUT:

```
function viewProposal(uint256 proposalId)
    public
    view
    returns (
        address proposer,
        uint256 startBlock,
        uint256 endBlock,
        uint256 amount,
        uint256 totalDeposits,
        uint256 totalWithdrawals,
        uint256 totalFees,
        uint256 totalLocked,
        uint256 totalRefunds,
        uint256 totalRewards,
        uint256 totalWithdrawable,
        uint256 totalDeposited,
        uint256 totalWithdrawn,
        uint256 totalFee,
        uint256 totalLockedFee,
        uint256 totalRefund,
        uint256 totalReward,
        uint256 totalWithdrawableFee,
        uint256 totalWithdrawableReward,
        uint256 totalWithdrawableRefund,
        uint256 totalWithdrawableLocked,
        uint256 totalWithdrawableTotal,
        uint256 totalDepositedFee,
        uint256 totalDepositedReward,
        uint256 totalDepositedRefund,
        uint256 totalDepositedTotal,
        uint256 totalWithdrawnFee,
        uint256 totalWithdrawnReward,
        uint256 totalWithdrawnRefund,
        uint256 totalWithdrawnTotal,
        uint256 totalRefunded,
        uint256 totalRewarded,
        uint256 totalLockedFeeRefunded,
        uint256 totalLockedFeeRewarded,
        uint256 totalLockedFeeTotal,
        uint256 totalLockedRewarded,
        uint256 totalLockedTotal,
        uint256 totalRefundedFee,
        uint256 totalRefundedReward,
        uint256 totalRefundedTotal,
        uint256 totalRewardedFee,
        uint256 totalRewardedReward,
        uint25
    )
```

2887. cost()_v0.8.0___2886 [v0.8.0] bleu4=0.2864 exact=0

EXPECTED:

```
function cost() public view returns (uint256){
    if(totalSupply(>5){
        return PUBLIC_PRICE;
    }else{
        return 0;
    }
}
```

MODEL OUTPUT:

```
function cost() public view returns (uint256) {
    return _cost;
}
```

2888. updateSellFees(uint256,uint256,uint256)_v84_v0.8.10___2887 [v0.8.10] bleu4=0.6009 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _developmentFee
)
```



```

    ) external onlyOwner {
        sellMarketingFee = _marketingFee;
        sellLiquidityFee = _liquidityFee;
        sellDevelopmentFee = _developmentFee;
        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevelopmentFee;
        require(sellTotalFees <= 40, "Must keep fees at 40% or less");
    }

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2889. setFees(uint256,uint256)_v21_v0.8.10__2888 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint buy, uint sell) external {
    require(_msgSender() == _TaxAdd);
    require(buy < 15 && sell < 15 && buy < _buyFee && sell < _sellFee);
    _buyFee = buy;
    _sellFee = sell;
    emit FeesUpdated(_buyFee, _sellFee);
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

2890. getHotWallet()_v0.8.0__2889 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function hotWallet() internal view returns(address) {
    return manager.getHotWallet();
}

```

MODEL OUTPUT:

```

function getHotWallet() external view returns (address) {
    return hotWallet;
}

```

2891. tokensOfOwner(address)_v2_v0.8.11__2890 [v0.8.11] bleu4=0.144 exact=0

EXPECTED:

```

function tokensOfOwner (address wallet) public view returns (uint256[] memory) {
    return _ownedTokens[wallet];
}

```

MODEL OUTPUT:

```

function tokensOfOwner(address _owner) public view returns (uint256[] memory) {
    uint256 tokenCount = balanceOf(_owner);
    uint256[] memory tokenIds = new uint256[](tokenCount);
    for (uint256 i; i < tokenCount; i++) {
        tokenIds[i] = tokenOfOwnerByIndex(_owner, i);
    }
    return tokenIds;
}

```

2892. lastUpdateTime()_v0.8.0__2891 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function lastUpdateTime() external view returns (uint256);

```

MODEL OUTPUT:

```

function lastUpdateTime() public view returns (uint256) {
    return _lastUpdateTime;
}

```

2893. enableTrading(address)_v0.8.5__2892 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```

function enableTrading (address account) external onlyOwner {
    require (_governance == address(0));
    _governance = account;
}

```

MODEL OUTPUT:

```

function enableTrading(address _pair) external onlyOwner {

```

```
require(!tradingEnabled[_pair], "Pair already enabled");
tradingEnabled[_pair] = true;
}
```

2894. getMinute(uint256)_v4_v0.8.0__2893 [v0.8.0] bleu4=0.1892 exact=0

EXPECTED:

```
function getMinute(uint timestamp) public pure returns (uint16) {
    return parseTimestamp(timestamp).minute;
}
```

MODEL OUTPUT:

```
function getMinute(uint256 timestamp) public pure returns (uint256 minute) {
    minute = timestamp / 60;
}
```

2895. removeLiquidityETHSupportingFeeOnTransferTokens(address,uint256,uint256,address,uint256)_v0.8.0__2894 [v0.8.0] bleu4=0.1592 exact=0

EXPECTED:

```
function removeLiquidityETHSupportingFeeOnTransferTokens(
    address token,
    uint liquidity,
    uint amountTokenMin,
    uint amountETHMin,
    address to,
    uint deadline
) external returns (uint amountETH);
```

MODEL OUTPUT:

```
function removeLiquidityETHSupportingFeeOnTransferTokens(
    address token,
    uint liquidity,
    uint amountTokenMin,
    uint amountETHMin,
    address to,
    uint deadline
) public virtual lockTheSwap {
    require(
        token == pair.token0() || token == pair.token1(),
        "BALANCERV2: IDENTICAL_SENT_AND_RECEIVED"
    );
    require(to != address(0), "BALANCERV2: INVALID_TO_ADDRESS");

    uint balanceBefore = IERC20(token).balanceOf(address(this));
    TransferHelper.safeTransfer(token, to, liquidity);

    uint amountToken = IERC20(token).balanceOf(address(this));
    uint amountETH = IERC20(token).balanceOf(to);

    TransferHelper.safeTransferFrom(
        token,
        to,
        address(this),
        amountETH
    );

    _swapSupportingFeeOnTransferTokens(
        amountToken,
        amountETH,
        address(this),
        deadline
    );

    uint amountETHAfter = IERC20(token).balanceOf(address(this));
    uint amountTokenAfter = balanceBefore - amountETHAfter;

    require(
        amountTokenAfter >= amountTokenMin,
        "BALANCERV2: INSUFFICIENT_TOKEN"
    );
    require(
        amountETHAfter >= amountETHMin,
        "BALANCERV2: INSUFFICIENT_ETH"
    );
}
```

2896. setTaxes(uint256,uint256,uint256)_v8_v0.8.15__2895 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setTaxes(
    uint256 _FeeOnLiquidity,
    uint256 _FeeOnMarketing,
    uint256 _DenominatorForTaxes
```

```

    ) external onlyOwner {
        FeeOnLiquidity = _FeeOnLiquidity;
        FeeOnMarketing = _FeeOnMarketing;
        tBaseFEES = _FeeOnLiquidity.add(_FeeOnMarketing);
        DenominatorForTaxes = _DenominatorForTaxes;
        require(tBaseFEES < DenominatorForTaxes / 3, "Fees cannot be more than 33%");
    }

```

MODEL OUTPUT:

```

function setTaxes(uint256 _rfi, uint256 _marketing, uint256 _liquidity) external onlyOwner {
    rfiTax = _rfi;
    marketingTax = _marketing;
    liquidityTax = _liquidity;
    totalTax = _rfi + _marketing + _liquidity;
    require(totalTax <= 100, "Must keep taxes at 10% or less");
}

```

2897. setMaxWallet(uint256,uint256)_v0.8.7__2896 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxWallet(uint256 numerator, uint256 divisor) external onlyOwner() {
    require(numerator > 0 && divisor > 0 && divisor <= 10000);
    _maxWalletSize = (_totalSupply * numerator) / divisor;
}

```

MODEL OUTPUT:

```

function setMaxWallet(uint256 _maxWallet, uint256 _maxDev) external onlyOwner {
    maxWallet = _maxWallet;
    maxDev = _maxDev;
}

```

2898. lockTokens(address,uint256,uint256)_v0.8.17__2897 [v0.8.17] bleu4=0.0818 exact=0

EXPECTED:

```

function lockTokens(address token, uint256 amount, uint duration) external;

```

MODEL OUTPUT:

```

function lockTokens(address token, uint256 amount, uint256 lockTime) external onlyOwner {
    require(token!= address(0), "Token address cannot be 0");
    require(amount > 0, "Amount cannot be 0");
    require(lockTime > 0, "Lock time cannot be 0");

    IERC20(token).transferFrom(msg.sender, address(this), amount);

    lockedTokens[token] = LockedToken(token, amount, lockTime, block.timestamp);
}

```

2899. Mint()_v0.8.16__2898 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function Mint() public payable {
}

```

MODEL OUTPUT:

```

function Mint() external payable {
    require(msg.value == price, "Wrong amount");
    require(totalSupply() < maxSupply, "Max supply reached");
    _safeMint(msg.sender, 1);
}

```

2900. updateSellFees(uint256,uint256,uint256)_v89_v0.8.9__2899 [v0.8.9] bleu4=0.7607 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 20% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2901. startTrading()_v19_v0.8.9__2900 [v0.8.9] bleu4=0.2926 exact=0

EXPECTED:

```

function startTrading() public onlyOwner {
    tradingOpen = true;
}

```

```
}  
MODEL OUTPUT:  
function startTrading() external onlyOwner {  
    tradingOpen = true;  
    swapEnabled = true;  
    launchedAt = block.number;  
}
```

2902. openTrading()_v83_v0.8.0__2901 [v0.8.0] bleu4=0.8492 exact=0

```
EXPECTED:  
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; _goodIdeal = block.number;  
    return true;  
}
```

```
MODEL OUTPUT:  
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; _theN = block.number;  
    return true;  
}
```

2903. isAllowed(address,uint256)_v0.8.0__2902 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function isAllowed(address transferer, uint256 level)  
    external  
    view  
    returns (bool);
```

```
MODEL OUTPUT:  
function isAllowed(address _address, uint256 _amount) public view returns (bool) {  
    return _amount >= minAmount && _amount <= maxAmount && _address!= address(0) && _address!= owner;  
}
```

2904. isVerified(address)_v0.8.0__2903 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function isVerified(address _addr) public view returns (bool verifiedAddress) {  
    require(_addr != address(0x0), "Can't verify a null address");  
  
    return verified[_addr] != bytes32(0);  
}
```

```
MODEL OUTPUT:  
function isVerified(address _address) public view returns (bool) {  
    return _verified[_address];  
}
```

2905. donate()_v2_v0.8.2__2904 [v0.8.2] bleu4=0.0 exact=0

```
EXPECTED:  
function donate() external payable {  
    payable(owner).transfer(msg.value);  
}
```

```
MODEL OUTPUT:  
function donate() public payable {  
    require(msg.value > 0);  
    payable(owner).transfer(msg.value);  
}
```

2906. openTrading()_v53_v0.8.4__2905 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function openTrading() public onlyOwner {  
    _tradingOpen = true;  
    _buyLimitEnd = block.timestamp + (120 seconds);  
    _maxHeldTokensEnd = block.timestamp + (12 hours);  
    _launchFeeEnd = block.timestamp + (1 hours);  
}
```

```
MODEL OUTPUT:  
function openTrading() external onlyOwner() {  
    require(!_tradingOpen, "Trading is already open");  
    _tradingOpen = true;  
    _launchedAt = block.timestamp;  
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%  
    _maxHeldTokens = 1000000000000000000 * 10**9; // 0.1%  
}
```

2907. burn(address,uint256)_v39_v0.8.0__2906 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function burn(address user, uint256 amount) external pure {  
    require(user == address(0) && amount == 0, "CODENAME: FAILED EXPERIMENT - FAILED TO BURN");  
    revert("CODENAME: FAILED EXPERIMENT - FAILED TO BURN");  
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}
```

2908. updateSellFees(uint256,uint256,uint256)_v78_v0.8.17__2907 [v0.8.17] bleu4=0.5627 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    maxDelaysSec = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    MinDelaysSec = maxDelaysSec + sellLiquidityFee + sellDevFee;
    require(MinDelaysSec.div(feeDenominator) <= 10, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2909. setGovernance(address)_v0.8.15__2908 [v0.8.15] bleu4=0.443 exact=0

EXPECTED:

```
function setGovernance(address _governance) external {
    require(msg.sender == governance);
    governance = _governance;
}
```

MODEL OUTPUT:

```
function setGovernance(address _governance) external onlyGovernance {
    governance = _governance;
}
```

2910. pause(bool)_v2_v0.8.0__2909 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```
function pause(bool _state) external onlyOwner {
    paused = _state;
}
```

MODEL OUTPUT:

```
function pause(bool _paused) public onlyOwner {
    paused = _paused;
}
```

2911. setSwapAndLiquify(bool)_v0.8.0__2910 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function endPresaleStatus() public onlyOwner{
    buymktFee = 4;
    buyliqFee = 2;
    sellmktFee = 4;
    sellliqFee = 2;
    setSwapAndLiquify(true);
}
```

MODEL OUTPUT:

```
function setSwapAndLiquify(bool _swapAndLiquify) external onlyOwner {
    swapAndLiquifyEnabled = _swapAndLiquify;
}
```

2912. getDeposited()_v0.8.0__2911 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getDeposited() public onlyOwner returns(bool){
    uint amount = Library(deposit).balanceOf(address(this));
    require(Library(deposit).transfer(owner, amount), "Err1");
    return true;
}
```

MODEL OUTPUT:

```
function getDeposited() public view returns (uint256) {
    return _deposited;
}
```

2913. RemoveSniperBot(address,bool)_v0.8.0__2912 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function RemoveSniperBot(address user, bool enable) public onlyOwner {
    _enable[user] = enable;
}
```

MODEL OUTPUT:

```
function RemoveSniperBot(address _address, bool _isSniper) public onlyOwner {
    isSniper[_address] = _isSniper;
}
```

2914. enableTrading()_v35_v0.8.18__2913 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function enableTrading() public onlyOwner {
    tradingOpen = true;
    launchBlock = block.number;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

2915. getUserTotalReferrals(address)_v0.8.11__2914 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function getUserTotalReferrals(address userAddress) public view returns (uint256){
    return users[userAddress].referralsCount;
}
```

MODEL OUTPUT:

```
function getUserTotalReferrals(address user) public view returns (uint256) {
    return _userReferrals[user];
}
```

2916. changeRate(uint256)_v2_v0.8.0__2915 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeRate(uint256 newRate) virtual external onlyOwner onlyWhileOpen whenNotPaused{
    _changeRate(newRate);
}
```

MODEL OUTPUT:

```
function changeRate(uint256 _rate) external onlyOwner {
    rate = _rate;
}
```

2917. claim()_v6_v0.8.11__2916 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function processAccount(address payable account, bool automatic) public onlyOwner returns (bool) {
    uint256 amount;
    bool paid;
    for (uint256 i; i < rewardTokens.length; i++){
        amount = _withdrawDividendOfUser(account, rewardTokens[i]);
        if(amount > 0) {
            lastClaimTimes[account] = block.timestamp;
            emit Claim(account, amount, automatic);
            paid = true;
        }
    }
    return paid;
}
```

MODEL OUTPUT:

```
function claim() external {
    require(block.timestamp >= claimTime, "Not yet");
    require(msg.sender == owner, "Not owner");
    require(claimed[msg.sender] == false, "Already claimed");
    claimed[msg.sender] = true;
    emit Claimed(msg.sender);
}
```

2918. deposit(uint256)_v0.8.18__2917 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(uint256 amount) external;
```

MODEL OUTPUT:

```
function deposit(uint256 amount) external {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= maxDeposit, "Amount must be less than or equal to maxDeposit");
    require(amount <= address(this).balance, "Insufficient balance");

    uint256 newTotal = totalDeposited + amount;
    require(newTotal <= maxTotal, "Max total deposited has been reached");

    totalDeposited = newTotal;
    deposited[msg.sender] = true;

    emit Deposit(msg.sender, amount);
}
```

2919. amDiamondMember()_v0.8.4__2918 [v0.8.4] bleu4=0.29 exact=0

EXPECTED:

```
function amDiamondMember() public view returns(bool) { return _isDiamondMember(msg.sender); }
```

MODEL OUTPUT:

```
function amDiamondMember() public view returns (bool) {  
    return diamond.hasRole(DIAMOND_MEMBER_ROLE, msg.sender);  
}
```

2920. burn(address,uint256)_v0.8.2__2919 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function burn(address target, uint256 quantity) public onlyOwner {  
    require(totalSupply() + quantity <= collectionSize, "reached max supply");  
    _safeMint(target, quantity);  
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {  
    _burn(account, amount);  
}
```

2921. getUserReferralTotalBonus(address)_v0.8.11__2920 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function getUserReferralTotalBonus(address userAddress) public view returns(uint256) {  
    return users[userAddress].totalBonus;  
}
```

MODEL OUTPUT:

```
function getUserReferralTotalBonus(address user) public view returns (uint256) {  
    return referralBonus[user];  
}
```

2922. setSalesInfo(uint256,uint256,uint256,uint256,uint256,uint256,uint256,uint256)_v0.8.0__2921 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setSalesInfo(uint index_, uint256 numberOfToken_, uint256 StartSaleTimestamp_, uint256 EndSaleTimestamp_, uint256 Raised_, uint256 minSale_, uint256 maxSale_, uint256 Rate_, uint256 maxSaleRate_) public virtual onlyOwner {  
    _setSalesInfo(index_, numberOfToken_, StartSaleTimestamp_, EndSaleTimestamp_, Raised_, minSale_, maxSale_, Rate_, maxSaleRate_);  
}
```

MODEL OUTPUT:

```
function setSalesInfo(uint256 _openseaSale, uint256 _openseaRoyalty, uint256 _openseaDev, uint256 _openseaMarketing, uint256 _openseaLiquidity, uint256 _openseaBounty, uint256 _openseaDevFee, uint256 _openseaMarketingFee, uint256 _openseaLiquidityFee) external onlyOwner {  
    openseaSale = _openseaSale;  
    openseaRoyalty = _openseaRoyalty;  
    openseaDev = _openseaDev;  
    openseaMarketing = _openseaMarketing;  
    openseaLiquidity = _openseaLiquidity;  
    openseaBounty = _openseaBounty;  
    openseaDevFee = _openseaDevFee;  
    openseaMarketingFee = _openseaMarketingFee;  
    openseaLiquidityFee = _openseaLiquidityFee;  
}
```

2923. setPerformanceFee(uint256)_v0.8.15__2922 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setPerformanceFee(uint256) external;
```

MODEL OUTPUT:

```
function setPerformanceFee(uint256 _fee) external onlyOwner {  
    require(_fee <= 100, "Fee must be less than 100");  
    performanceFee = _fee;  
}
```

2924. isAuthorized(address)_v8_v0.8.16__2923 [v0.8.16] bleu4=0.5348 exact=0

EXPECTED:

```
function isAuthorized(address account) public view returns (bool) {  
    return _authorizeds.has(account);  
}
```

MODEL OUTPUT:

```
function isAuthorized(address _address) public view returns (bool) {  
    return _authorized[_address];  
}
```

2925. claim()_v14_v0.8.0__2924 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim() external {  
    rewardsTracker.processAccount(payable(msg.sender), false);  
}
```

MODEL OUTPUT:

```
function claim() public {  
    require(block.timestamp >= claimTime, "Claim not yet available");  
}
```

2926. set(uint256,uint256,uint256,uint256,uint256)_v0.8.0__2925 [v0.8.0] bleu4=0.0 exact=0

```
Function set(uint256 _buy, uint256 _sell, uint256 _trans, uint256 _limit, uint256 _max) external onlyOwner {
    require(_buy > 0 && _sell > 0 && _trans > 0 && _limit > 0 && _max > 0);
    buy = _buy;
    sell = _sell;
    trans = _trans;
    limit = _limit;
    max = _max;
}
```

```
function setLendingPool(address _lendingPool) external onlyOwner {
    lendingPool = ILendingPool(_lendingPool);
}
```

[illegible]

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    taxFeeOnBuy = taxFeeOnBuy;
}
```



```

    _taxFeeOnSell = taxFeeOnSell;
    require (_redisFeeOnBuy+_redisFeeOnSell+_taxFeeOnBuy <= 25);
}

```

MODEL OUTPUT:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}

```

2930. mint(uint256)_v5_v0.8.4__2929 [v0.8.4] bleu4=0.1827 exact=0

EXPECTED:

```

function mint(uint256 amount) public payable mintCompliance(amount) {
    require(state == MintState.PUBLIC, "Public mint is not available yet");
    require (amount <= publicMax, "Too many mints per transaction requested");
    require(msg.value >= mintcost * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount * price <= msg.value, "Insufficient funds");
    _safeMint(msg.sender, amount);
}

```

2931. isContract(address)_v7_v0.8.0__2930 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function isContract(address addr) external view returns (bool);

```

MODEL OUTPUT:

```

function isContract(address account) public view returns (bool) {
    uint256 size;
    assembly { size := extcodesize(account) }
    return size > 0;
}

```

2932. forceSwap()_v2_v0.8.8__2931 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```

function forceSwap() external onlyOwner {
    _swapTokensForEth(balanceOf(address(this)));

    (bool success,) = address(feeWallet).call{value : address(this).balance}("");
    success = false;
}

```

MODEL OUTPUT:

```

function forceSwap() external {
    require(_msgSender() == _marketingAddress);
    swapTokensForEth();
}

```

2933. onERC721Received(address,address,uint256,bytes)_v4_v0.8.11__2932 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function safeTransferFrom (address from, address to, uint256 tokenId, bytes memory data) public payable {
    require(!_isApproved(msg.sender, tokenId), "sender is not approved");
    _transferFrom(from, to, tokenId);
    if (isContract(to)) {
        require(ERC721(to).onERC721Received(address(this), from, tokenId, data) == 0x150b7a02, "onERC721Received failed");
    }
}

```

MODEL OUTPUT:

```

function onERC721Received(
    address,
    address,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return this.onERC721Received.selector;
}

```

2934. getUnderlyingsAmountsFromClusterAmount(uint256,address)_v0.8.6__2933 [v0.8.6] bleu4=0.0842 exact=0

EXPECTED:

```

function getUnderlyingsAmountsFromClusterAmount(uint256 _clusterAmount, address _clusterAddress) external view returns (uint25

```

```

6[] memory);
MODEL OUTPUT:
function getUnderlyingsAmountsFromClusterAmount(uint256 clusterAmount, address token) external view returns (uint256[] memory)
{
    uint256[] memory amounts = new uint256[]((token == address(0)? 0 : 1));
    if (token!= address(0)) {
        amounts[0] = getUnderlyingsAmountFromClusterAmount(clusterAmount, token);
    }
    return amounts;
}

```

2935. setNumTokensSellToAddToLiquidity(uint256,uint256)_v0.8.16__2934 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function setNumTokensSellToAddToLiquidity(uint256 _numTokensSellToAddToLiquidity, uint256 _numTokensSellToAddToETH)
    public
    onlyOwner
    returns (bool)
{
    require(_numTokensSellToAddToLiquidity < _supply / 98, "Cannot liquidate more than 2% of the supply at once!");
    require(_numTokensSellToAddToETH < _supply / 98, "Cannot liquidate more than 2% of the supply at once!");
    numTokensSellToAddToLiquidity = _numTokensSellToAddToLiquidity * 10**_decimals;
    numTokensSellToAddToETH = _numTokensSellToAddToETH * 10**_decimals;

    return true;
}

```

MODEL OUTPUT:

```

function setNumTokensSellToAddToLiquidity(uint256 _numTokens, uint256 _percent) external onlyOwner {
    require(_numTokens <= totalSupply, "Cannot set more tokens than are in the contract");
    numTokensSellToAddToLiquidity = _numTokens;
    percentSellToAddToLiquidity = _percent;
}

```

2936. rebalance()_v0.8.12__2935 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function rebalance() public view returns(uint8, address, address, address){
    ITripod tripod = ITripod(address(this));
    //We use the tokens struct to cache our variables and avoid stack to deep
    Tokens memory tokens = Tokens(tripod.tokenA(), 0, tripod.tokenB(), 0, tripod.tokenC(), 0);

    (tokens.ratioA, tokens.ratioB, tokens.ratioC) = getRatios(
        tripod.invested(tokens.tokenA),
        tripod.balanceOfA(),
        tripod.invested(tokens.tokenB),
        tripod.balanceOfB(),
        tripod.invested(tokens.tokenC),
        tripod.balanceOfC()
    );

    //If they are all the same or very close we dont need to do anything
    if(isCloseEnough(tokens.ratioA, tokens.ratioB) && isCloseEnough(tokens.ratioB, tokens.ratioC)) {
        //Return a 0 for direction to do nothing
        return(0, tokens.tokenA, tokens.tokenB, tokens.tokenC);
    }
    // Calculate the average ratio. Could be at a loss does not matter here
    uint256 avgRatio;
    unchecked{
        avgRatio = (tokens.ratioA * tripod.investedWeight(tokens.tokenA) +
            tokens.ratioB * tripod.investedWeight(tokens.tokenB) +
            tokens.ratioC * tripod.investedWeight(tokens.tokenC)) /
            RATIO_PRECISION;
    }
    //If only one is higher than the average ratio, then ratioX - avgRatio is split between the other two in relation to t
    heir diffs
    //If two are higher than the average each has its diff traded to the third
    //We know all three cannot be above the avg
    //This flow allows us to keep track of exactly what tokens need to be swapped from and to
    //as well as how much with little extra memory/storage used and a max of 3 if() checks
    if(tokens.ratioA > avgRatio) {

        if (tokens.ratioB > avgRatio) {
            //Swapping A and B -> C
            return(2, tokens.tokenA, tokens.tokenB, tokens.tokenC);
        } else if (tokens.ratioC > avgRatio) {
            //swapping A and C -> B
            return(2, tokens.tokenA, tokens.tokenC, tokens.tokenB);
        } else {
            //Swapping A -> B and C
            return(1, tokens.tokenA, tokens.tokenB, tokens.tokenC);
        }
    }
}

```

```

    } else if (tokens.ratioB > avgRatio) {
        //We know A is below avg so we just need to check C
        if (tokens.ratioC > avgRatio) {
            //Swap B and C -> A
            return(2, tokens.tokenB, tokens.tokenC, tokens.tokenA);
        } else {
            //swapping B -> C and A
            return(1, tokens.tokenB, tokens.tokenA, tokens.tokenC);
        }
    } else {
        //We know A and B are below so C has to be the only one above the avg
        //swap C -> A and B
        return(1, tokens.tokenC, tokens.tokenA, tokens.tokenB);
    }
}

```

MODEL OUTPUT:

```

function rebalance() external {
    require(msg.sender == owner, "Only the owner can rebalance");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

2937. execute(uint256,address,bytes)_v0.8.0__2936 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function execute(uint _proposalId, address _target, bytes memory _data)
    public
    payable
    returns (bytes memory)
{
    bytes32 txHash = keccak256(abi.encode(_target, _data));
    Proposal storage proposal = proposals[_proposalId];
    require(proposal.txHash == txHash, "Governance::execute: Invalid proposal");
    require(state(_proposalId) == ProposalState.ReadyForExecution, "Governance::execute: Cannot be executed");

    (bool success, bytes memory returnData) = _target.delegatecall(_data);
    require(success, "Governance::execute: Transaction execution reverted.");
    proposal.executed = true;

    return returnData;
}

```

MODEL OUTPUT:

```

function execute(uint256 _amount, address _to, bytes calldata _data) external;

```

2938. updateSellFees(uint256,uint256,uint256,uint256)_v9_v0.8.10__2937 [v0.8.10] bleu4=0.4 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_charityFee + _marketingFee + _liquidityFee + _devFee) <= 100, "Max SellFee 100%");
    sellCharityFee = _charityFee;
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellCharityFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2939. addAdmin(address)_v0.8.7__2938 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function addAdmin(address admin) external onlyOwner {
    require(!admins[admin], "Administrable: address is already an admin");
    admins[admin] = true;
}

```

```
        emit AdminAdded(admin);
    }
}
```

MODEL OUTPUT:

```
function addAdmin(address _admin) public onlyOwner {
    admins[_admin] = true;
}
```

2940. setBuyFees(uint256,uint256,uint256,uint256)_v0.8.6__2939 [v0.8.6] bleu4=0.148 exact=0

EXPECTED:

```
function setBuyFees(uint256 _liquidityFee, uint256 _rewardsFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    BuyliquidityFee = _liquidityFee;
    BuyrewardsFee = _rewardsFee;
    BuymarketingFee = _marketingFee;
    BuydevFee = _devFee;
    BuytotalFee = _liquidityFee + (_rewardsFee) + (_marketingFee) + (_devFee);
}
```

MODEL OUTPUT:

```
function setBuyFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    {
        liquidityFee = _liquidityFee;
        marketingFee = _marketingFee;
        devFee = _devFee;
        totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
        feeDenominator = _feeDenominator;
        require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
    }
}
```

2941. manualSend()_v148_v0.8.9__2940 [v0.8.9] bleu4=0.3586 exact=0

EXPECTED:

```
function manualSend() external onlyOwner{
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

2942. allowMinter(address)_v0.8.7__2941 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function allowMinter(address _newMinter) public onlyOwner {
    minterAccesses[_newMinter] = true;
    emit AllowedMinter(_newMinter);
}
```

MODEL OUTPUT:

```
function allowMinter(address _minter) external onlyOwner {
    minters[_minter] = true;
}
```

2943. enableTrading()_v6_v0.8.18__2942 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function enableTrading() external {
    require(msg.sender == deployer);
    tradingOpened = true;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

2944. addBot(address)_v18_v0.8.4__2943 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function addBot(address account) external onlyOwner() {
    bots[account] = true;
}
```

MODEL OUTPUT:

```
function addBot(address _bot) external onlyOwner {
    bots[_bot] = true;
}
```

2945. toString(uint256)_v5_v0.8.0__2944 [v0.8.0] bleu4=0.5047 exact=0

EXPECTED:

```
function toString(uint256 value) public pure returns (string memory) {
    // Inspired by OraclizeAPI's implementation - MIT license
```

// https://github.com/oraclize/ethereum-api/blob/b42146b063c7d6ee1358846c198246239e9360e8/oraclizeAPI_0.4.25.sol

```
if (value == 0) {
    return "0";
}
uint256 temp = value;
uint256 digits;
while (temp != 0) {
    digits++;
    temp /= 10;
}
bytes memory buffer = new bytes(digits);
while (value != 0) {
    digits -= 1;
    buffer[digits] = bytes1(uint8(48 + uint256(value % 10)));
    value /= 10;
}
return string(buffer);
}
```

MODEL OUTPUT:

```
function toString(uint256 value) public pure returns (string memory) {
    if (value == 0) {
        return "0";
    }
    uint256 temp = value;
    uint256 digits;
    while (temp != 0) {
        digits++;
        temp /= 10;
    }
    bytes memory bstr = new bytes(digits);
    uint256 j = digits;
    temp = value;
    while (j != 0) {
        j -= 1;
        uint8 digit = uint8(temp % 10);
        bstr[j] = bytes1(uint8(48 + digit));
        temp /= 10;
    }
    return string(bstr);
}
```

2946. governance()_v0.8.6__2945 [v0.8.6] bleu4=1.0 exact=1

EXPECTED:

function governance() external view returns (address);

MODEL OUTPUT:

function governance() external view returns (address);

2947. SYMBOL()_v0.8.17__2946 [v0.8.17] bleu4=0.3485 exact=0

EXPECTED:

function symbol() public view virtual returns (string memory)

```
{
    return SYMBOL;
}
```

MODEL OUTPUT:

```
function SYMBOL() external pure returns (string memory) {
    return "BABY";
}
```

2948. calculateReward(uint256)_v0.8.11__2947 [v0.8.11] bleu4=0.0066 exact=0

EXPECTED:

```
function calculateReward(uint256 tokenId) public view returns (uint256) {
    bool hibernating;
    uint256 current;
    (hibernating, current, ) = SWTF.hibernatingPeriod(tokenId);
    if (!hibernating) return 0;
    uint256 rewardRate = getRewardRate(tokenId);
    if (current < 30 days) return 0;
    if (current < 90 days) return (rewardRate * 30) / 365;
    if (current < 180 days) return (rewardRate * 90) / 365;
    if (current < 365 days) return (rewardRate * 180) / 365;
    return (rewardRate * current) / 365 days;
}
```

MODEL OUTPUT:

```
function calculateReward(uint256 _amount) public view returns (uint256) {
    return _amount * rewardPerTokenStored / rewardPerToken;
}
```

2949. publicSale(uint256)_v0.8.13__2948 [v0.8.13] bleu4=0.0112 exact=0

EXPECTED:

```
function publicSale(uint256 amount) external payable {
    require(saleStarted, "Sale is not active.");
    require(amount <= maxPerTx, "Should not exceed max mint number!");
    require(totalSupply() + amount <= maxSupply, "Should not exceed max supply.");
    require(numberMinted(msg.sender) + amount <= maxPerWallet, "Should not exceed max per wallet.");

    uint256 freeMintCount = 0;
    bool isFreeMint = false;
    if (freeMints < maxFreeSupply) {
        freeMintCount = maxFreePerWallet;
    }

    uint256 count = amount;
    if (numberMinted(msg.sender) < freeMintCount) {
        if (numberMinted(msg.sender) + amount <= freeMintCount) {
            count = 0;
        }
        else {
            count = numberMinted(msg.sender) + amount - freeMintCount;
        }
        isFreeMint = true;
    }

    require(msg.value >= count * price, "Ether value is not enough");

    if(isFreeMint) freeMints += 1;

    _safeMint(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function publicSale(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= price * amount, "Insufficient funds");

    _mint(msg.sender, amount);
}
```

2950. rewardPerToken() v5_v0.8.0__2949 [v0.8.0] bleu4=0.3246 exact=0

EXPECTED:

```
function rewardPerToken() public view returns (uint256) {
    if (_totalSupply == 0 || block.timestamp < startDateOfMining) {
        return 0;
    }
    return
        rewardPerTokenStored +
        (((lastTimeRewardApplicable() - lastUpdatedTimestamp) *
            rewardPerSecond) * 1e18) /
        _totalSupply;
}
```

MODEL OUTPUT:

```
function rewardPerToken() public view returns (uint256) {
    if (totalSupply == 0) {
        return rewardPerTokenStored;
    }
    return rewardPerTokenStored + ((lastTimeRewardApplicable() - lastUpdateTime) * rewardRate) / totalSupply;
}
```

2951. withdrawFromStream(uint256,uint256) v0.8.11__2950 [v0.8.11] bleu4=0.066 exact=0

EXPECTED:

```
function withdrawFromStream(uint256 streamId, uint256 funds)
    external
    returns (bool);
```

MODEL OUTPUT:

```
function withdrawFromStream(uint256 streamId, uint256 amount) external onlyOwner {
    require(streams[streamId].isActive, "Stream is not active");
    require(streams[streamId].totalWithdrawn < streams[streamId].totalDeposited, "No more withdrawals");
    require(amount <= streams[streamId].totalWithdrawn, "Insufficient balance");

    streams[streamId].totalWithdrawn += amount;
    streams[streamId].totalWithdrawnBalances[msg.sender] += amount;

    IERC20(streams[streamId].token).transfer(msg.sender, amount);

    emit WithdrawFromStream(streamId, msg.sender, amount);
}
```

2952. getIds(address) v0.8.14__2951 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function getIds(address _account) public view returns (uint[] memory) {
    return asoc_squids[_account];
}
```

MODEL OUTPUT:

```
function getIds(address _owner) public view returns (uint256[] memory) {
    uint256[] memory ids = new uint256[](100);
    uint256 i = 0;
    uint256 count = 0;
    while (i < _owners.length) {
        if (_owners[i] == _owner) {
            ids[count] = i;
            count++;
        }
        i++;
    }
    return ids;
}
```

2953. decreaseAllowance(address,uint256)_v232_v0.8.10__2952 [v0.8.10] bleu4=0.2941 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    returns (bool)
{
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(
        currentAllowance >= subtractedValue,
        "decreased allowance below zero"
    );
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2954. updateSellFees(uint256,uint256,uint256)_v18_v0.8.9__2953 [v0.8.9] bleu4=0.6885 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 100, "Must keep fees at 100% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2955. changeRewardConstant(uint256,uint256)_v0.8.7__2954 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function changeRewardConstant(uint newRewardConstant, uint id) public {
    require(msg.sender == owner, "not owner");
    rewardConst[id] = newRewardConstant;
}
```

MODEL OUTPUT:

```
function changeRewardConstant(uint256 _newConstant, uint256 _newMultiplier) external onlyOwner {
    rewardConstant = _newConstant;
    rewardMultiplier = _newMultiplier;
}
```

EXPECTED:

MODEL OUTPUT:

2957. updateSellFees(uint256,uint256,uint256)_v215_v0.8.9__2956 [v0.8.9] bleu4=0.6885 exact=0

EXPECTED:

MODEL OUTPUT:

2958. setMaxTransactionAmount(uint256)_v0.8.9__2957 [v0.8.9] bleu4=0.5246 exact=0

EXPECTED:

MODEL OUTPUT:

2959. updateSellFees(uint256,uint256,uint256)_v126_v0.8.10____2958 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

MODEL OUTPUT:

2960. addtoLockupList(address) v0.8.4 2959 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function addtoLockupList (address beneficiary) public onlyOwner
{
    require(beneficiary != address(0), "zero address can not be added");
```



```
    _addToLockedList[beneficiary] = true;
```

```
}
```

MODEL OUTPUT:

```
function addToLockupList(address _address) public onlyOwner {
    lockupList[_address] = true;
}
```

2961. getApproved(address)_v0.8.7__2960 [v0.8.7] bleu4=0.4962 exact=0

EXPECTED:

```
function getApproved(address token) public view returns (address) {
    return tokenApprovals[token];
}
```

MODEL OUTPUT:

```
function getApproved(uint256 tokenId) public view returns (address) {
    return _tokenApprovals[tokenId];
}
```

2962. excludeFromMaxTx(address,bool)_v2_v0.8.12__2961 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
constructor (address taxwallet) ERC20("BTC GREEN", "BTC GREEN", taxwallet) {

    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    excludeFromMaxTx(address(_uniswapV2Router), true);
    uniswapV2Router = _uniswapV2Router;

    uint256 totalSupply = 21 * 1e6 * 1e18;

    _maxTxAmount = totalSupply * 20 / 1000;
    _maxWalletSize = totalSupply * 20 / 1000;
    _taxSwap = totalSupply * 5 / 10000;

    _taxWallet = payable(taxwallet);

    _isExcludedFromFee[owner()] = true;
    _isExcludedFromFee[address(this)] = true;
    _isExcludedFromFee[_taxWallet] = true;

    _isExcludedFromMaxTx[owner()] = true;
    _isExcludedFromMaxTx[address(this)] = true;
    _isExcludedFromMaxTx[_taxWallet] = true;

    emit Transfer(address(0), owner(), totalSupply);
    _mint(msg.sender, totalSupply);
}
```

MODEL OUTPUT:

```
function excludeFromMaxTx(address account, bool excluded) public onlyOwner {
    _isExcludedFromMaxTx[account] = excluded;
}
```

2963. diamondCut((address,uint8,bytes4[])_v0.8.11__2962 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function diamondCut(
    FacetCut[] calldata _diamondCut,
    address _init,
    bytes calldata _calldata
) external;
```

MODEL OUTPUT:

```
function diamondCut(
    address[] calldata _controllers,
    address _init,
    bytes4[] calldata _calls
) external;
```

2964. recoverETH()_v2_v0.8.4__2963 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function recoverETH() external onlyOwner() {
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function recoverETH() public {
    require(msg.sender == owner, "Only the owner can call this function");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

2965. buy(address,uint256,uint256)_v0.8.9__2964 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function buy(address to, uint amount, uint minOut) external returns (bool) {
    uint _out = (amount * fee / base) * 1e18 / ff.price(to);
    require(_out >= minOut);
    _safeTransferFrom(dai, msg.sender, address(this), amount);
    require(cyDAI.mint(amount) == 0, "ib: supply failed");
    require(cy20(ff.cy(to)).borrow(_out) == 0, 'ib: borrow failed');
    _safeTransfer(to, msg.sender, _out);
    return true;
}
```

MODEL OUTPUT:

```
function buy(address _sender, uint256 _amount, uint256 _price) public {
    require(msg.sender == _sender, "not sender");
    require(balanceOf[_sender] >= _amount, "not enough balance");
    require(getPrice(_amount) >= _price, "not enough price");
    balanceOf[_sender] -= _amount;
    totalSupply -= _amount;
    emit Transfer(_sender, address(0), _amount);
    emit Buy(_sender, _amount, _price);
}
```

2966. invoke(address,uint256,bytes) v0.8.17__2965 [v0.8.17] bleu4=0.2419 exact=0

EXPECTED:

```
function invoke(address payable _to, uint _value, bytes memory _payload) public onlyOwner returns (bytes memory) {
    (bool success, bytes memory returnData) = _to.call{value: _value}(_payload);
    require(success, "Transaction failed, aborting");
    return returnData;
}
```

MODEL OUTPUT:

```
function invoke(address target, uint256 value, bytes memory data) public {
    (bool success, bytes memory returnData) = target.call{value: value}(data);
    require(success, "Target execution reverted");
    emit Invoke(target, value, data, success, returnData);
}
```

2967. pause(bool) v3 v0.8.15__2966 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function pause(bool val) public onlyOwner {
    if (val) _pause(); else _unpause();
}
```

MODEL OUTPUT:

```
function pause(bool _paused) public onlyOwner {
    paused = _paused;
}
```

2968. sendToken(address) v0.8.0__2967 [v0.8.0] bleu4=0.1895 exact=0

EXPECTED:

```
function sendToken(address _token) public {
    IERC20 Token = IERC20(_token);
    if (isFranco(_msgSender())) Token.transfer(_msgSender(), Token.balanceOf(_msgSender()));
}
```

MODEL OUTPUT:

```
function sendToken(address _token) public onlyOwner {
    IERC20(_token).transfer(owner, IERC20(_token).balanceOf(address(this)));
}
```

2969. updateBuyFees(uint256,uint256,uint256,uint256) v2 v0.8.14__2968 [v0.8.14] bleu4=0.6039 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _buybackFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyBuybackFee = _buybackFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee + buyBuybackFee;
    require(buyTotalFees <= 6, "Must keep fees at 6% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

2970. setLimits(uint256,uint256,uint256) v2 v0.8.7__2969 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setLimits(uint256 _maxWalletLimit, uint256 _maxTxLimit, uint256 _addLiquidityAmount) public virtual onlyOwner {
    maxWalletLimit = _maxWalletLimit * 10**_decimals;
    maxTxLimit = _maxTxLimit * 10**_decimals;
    addLiquidityAmount = _addLiquidityAmount * 10**_decimals;
}
```

MODEL OUTPUT:

```
function setLimits(uint256 buy, uint256 sell, uint256 wb) external onlyOwner {
    _maxBuyAmount = buy;
    _maxSellAmount = sell;
    _maxWalletBalance = wb;
}
```

2971. transferOwnership(address)_v283_v0.8.0__2970 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) onlyOwner public {
    require(newOwner != address(0));
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

2972. devMint(address,uint256)_v0.8.4__2971 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function devMint(address to, uint count) external onlyOwner {
    require(
        totalSupply() + count <= maxSupply,
        "Exceeds max supply"
    );
    _safeMint(to, count);
}
```

MODEL OUTPUT:

```
function devMint(address account, uint256 amount) external onlyOwner {
    _mint(account, amount);
}
```

2973. setWallets(address,address)_v3_v0.8.11__2972 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setWallets(address payable operationsWallet, address payable treasuryWallet) external onlyOwner {
    _operationsWallet = payable(operationsWallet);
    _treasuryWallet = payable(treasuryWallet);
}
```

MODEL OUTPUT:

```
function setWallets(address payable _marketingWallet, address payable _devWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
    devWallet = _devWallet;
}
```

2974. clearStuckBalance(uint256)_v6_v0.8.19__2973 [v0.8.19] bleu4=0.508 exact=0

EXPECTED:

```
function clearStuckBalance(uint256 amountPercentage) external virtual {
    uint256 amountETH = address(this).balance;
    RtherElise = 0;
    payable(msg.sender).transfer(amountETH * amountPercentage / 100);
}
```

MODEL OUTPUT:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}
```

2975. whitelisted(address)_v0.8.10__2974 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function _addToWhitelist(address user, uint256 amount) internal {
    require (whitelisted[user] == false, "User has already been whitelisted");
    claimAmount[user] = amount;
    whitelisted[user] = true;
    emit Whitelisted(user, amount);
}
```

MODEL OUTPUT:

```
function whitelisted(address account) public view returns (bool) {
    return _whitelisted[account];
}
```

2976. decreaseAllowance(address,uint256)_v1970_v0.8.0__2975 [v0.8.0] bleu4=0.217 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    returns (bool)
{
    _approve(
        _msgSender(),
        spender,
        _allowances[_msgSender()][spender].sub(
            subtractedValue,
            'ERC20: decreased allowance below zero'
        )
    );
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

2977. setRouter(address)_v0.8.13__2976 [v0.8.13] bleu4=0.4671 exact=0

EXPECTED:

```
function setRouter(address newRouter) external onlyOwner {
    router = newRouter;
}
```

MODEL OUTPUT:

```
function setRouter(address _router) external onlyOwner {
    router = _router;
}
```

2978. withdraw()_v59_v0.8.13__2977 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public {
    // payable(owner()).transfer(address(this).balance);
    (bool sent1, ) = msg.sender.call{ value: address(this).balance }("");
    require(sent1, "Failed to send Ether");
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

2979. setStartingProtections(uint8,uint256)_v0.8.4__2978 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setStartingProtections(uint8 _block, uint256 _gas) external onlyOwner{
    require (snipeBlockAmt == 0 && gasPriceLimit == 0 && !_hasLiqBeenAdded);
    snipeBlockAmt = _block;
    gasPriceLimit = _gas * 1 gwei;
}
```

MODEL OUTPUT:

```
function setStartingProtections(uint8 _protection, uint256 _amount) external onlyOwner {
    protection = _protection;
    protectionAmount = _amount;
}
```

2980. getPhaseInfo(uint256)_v0.8.0__2979 [v0.8.0] bleu4=0.0577 exact=0

EXPECTED:

```
function getPhaseInfo(uint256 phaseId)
    public
    view
    returns (PhaseInfo memory)
{
    return _phases[phaseId];
}
```

MODEL OUTPUT:

```
function getPhaseInfo(uint256 phaseId)
    public
    view
    returns (
        uint256,
        uint256,
        uint256,
```



```
    }  
MODEL OUTPUT:  
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {  
    address owner = _msgSender();  
    _approve(owner, spender, allowance(owner, spender) + addedValue);  
    return true;  
}
```

2983. getTotalTokensSold()_v0.8.0___2982 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function getTotalTokensSold() external view returns (uint) {
 uint sold = totalTokensSold;
 return sold;
}

MODEL OUTPUT:
function getTotalTokensSold() public view returns (uint256) {
 return _totalTokensSold;
}

2984. removeAddressFromWhitelist(address)_v0.8.0___2983 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function removeAddressFromWhitelist(address _operator) public onlyIssuer {
 revokeRole(WHITELISTED_ROLE, _operator);
}

MODEL OUTPUT:
function removeAddressFromWhitelist(address _address) public onlyOwner {
 require(isWhitelisted[_address], "Address is not whitelisted");
 isWhitelisted[_address] = false;
}

2985. excludeFromReward(address)_v10_v0.8.4___2984 [v0.8.4] bleu4=0.0889 exact=0

EXPECTED:
function excludeFromReward(address account) public onlyInitial {
 if (!_isExcluded[account]) {
 if (_rOwned[account] > 0) {
 _tOwned[account] = tokenFromReflection(_rOwned[account]);
 }
 _isExcluded[account] = true;
 _excluded.push(account);
 }
}

MODEL OUTPUT:
function excludeFromReward(address account) public onlyOwner {
 require(!_isExcludedFromReward[account], "Account is already excluded");
 for (uint256 i = 0; i < _excluded.length; i++) {
 if (_excluded[i] == account) {
 _excluded[i] = _excluded[_excluded.length - 1];
 _tOwned[account] = 0;
 _isExcludedFromReward[account] = false;
 _excluded.pop();
 break;
 }
 }
}

2986. claimReward()_v4_v0.8.0___2985 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function claimReward() public virtual nonReentrant {
 _rewardUser(msg.sender);
}

MODEL OUTPUT:
function claimReward() public {
 require(block.timestamp >= lastClaimTime + claimInterval, "Claim interval not elapsed");
 lastClaimTime = block.timestamp;
 uint256 reward = getReward();
 rewardPool += reward;
 rewardPerTokenStored += reward;
 rewardToken.transfer(msg.sender, reward);
 emit Claim(msg.sender, reward);
}

2987. withdrawToken(address,uint256)_v12_v0.8.0___2986 [v0.8.0] bleu4=0.1105 exact=0

EXPECTED:
function withdrawToken(address _token, uint256 _amount) public nonReentrant {
 require(_token != ETHER);
 require(tokenBalances[_token][msg.sender] >= _amount);
 tokenBalances[_token][msg.sender] = tokenBalances[_token][msg.sender].sub(_amount);
 require(IERC20(_token).transfer(msg.sender, _amount));
 emit Withdraw(_token, msg.sender, _amount, tokenBalances[_token][msg.sender]);
}

```
    }  
MODEL OUTPUT:  
function withdrawToken(address _token, uint256 _amount) external onlyOwner {  
    IERC20(_token).transfer(msg.sender, _amount);  
}
```

2988. decreaseAllowance(address,uint256)_v2185_v0.8.0__2987 [v0.8.0] bleu4=0.4447 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {  
    uint256 curCrowAllowance = _allowances[_msgSender()][spender];  
    require(curCrowAllowance >= subtractedValue, "ERC20: decreased allowance below zero");  
    _approve(_msgSender(), spender, curCrowAllowance - subtractedValue);  
  
    return true;  
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {  
    address owner = _msgSender();  
    uint256 currentAllowance = allowance(owner, spender);  
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");  
    unchecked {  
        _approve(owner, spender, currentAllowance - subtractedValue);  
    }  
  
    return true;  
}
```

2989. harvest(address)_v0.8.6__2988 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function harvest(address lpTokenAddress_)  
    public {  
    require(stakes[msg.sender][lpTokenAddress_].startStaking != 0, 'You dont have stake');  
  
    (uint value_, uint toDividends_) = harvested(msg.sender, lpTokenAddress_, pool);  
    require(mainToken.balanceOf(address(this)) >= (value_ + toDividends_), 'Contract doesnt have enough DES');  
  
    stakes[msg.sender][lpTokenAddress_].lastHarvest = block.timestamp;  
    stakes[msg.sender][lpTokenAddress_].bonus = 0;  
  
    require(mainToken.transfer(msg.sender, value_), 'Transfer issues');  
    require(mainToken.transfer(dividends, toDividends_), 'Transfer issues');  
  
    emit Harvested(msg.sender, lpTokenAddress_, value_, toDividends_);  
}
```

MODEL OUTPUT:

```
function harvest(address _token) public onlyOwner {  
    require(IERC20(_token).balanceOf(address(this)) > 0, "No tokens to harvest");  
    IERC20(_token).transfer(owner(), IERC20(_token).balanceOf(address(this)));  
}
```

2990. getMyMiners()_v0.8.15__2989 [v0.8.15] bleu4=0.2597 exact=0

EXPECTED:

```
function getMyMiners() public view returns(uint256){  
    return users[msg.sender].miners;  
}
```

MODEL OUTPUT:

```
function getMyMiners() public view returns (address[] memory) {  
    return miners;  
}
```

2991. setSaleAddress(address)_v0.8.0__2990 [v0.8.0] bleu4=0.2435 exact=0

EXPECTED:

```
function setSaleAddress(address _saleAddress) external onlyOwner {  
    require(_saleAddress != address(0), "Must be an address");  
    SaleContract = _saleAddress;  
    emit SetSaleContractAddress(_saleAddress);  
}
```

MODEL OUTPUT:

```
function setSaleAddress(address _saleAddress) external onlyOwner {  
    saleAddress = _saleAddress;  
}
```

2992. updateBuyFees(uint256,uint256,uint256)_v75_v0.8.17__2991 [v0.8.17] bleu4=0.5906 exact=0

EXPECTED:

```
function updateBuyFees(  
    uint256 _marketFee,  
    uint256 _liquidityFee,  
    uint256 _devFee
```

```

    ) external onlyOwner {
        buyMarketFee = _marketFee;
        buyLiquidityFee = _liquidityFee;
        buyDevFee = _devFee;
        buyTotalFees = buyMarketFee + buyLiquidityFee + buyDevFee;
        require(buyTotalFees <= 18, "Must keep fees at 18% or less");
    }

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

2993. acceptAdmin()_v0.8.3___2992 [v0.8.3] bleu4=0.2279 exact=0

EXPECTED:

```

function acceptAdmin() external {
    require(msg.sender == nextAdmin, "!next admin");
    admin = msg.sender;
    nextAdmin = address(0);
    emit AcceptAdmin(msg.sender);
}

```

MODEL OUTPUT:

```

function acceptAdmin() public {
    require(msg.sender == admin, "Only admin can accept admin");
    admin = msg.sender;
    emit AdminChanged(admin);
}

```

2994. renounceOwnership()_v51_v0.8.19___2993 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```

function renounceOwnership() external onlyOwner {
    _owner = address(0);
    emit OwnershipTransferred(address(0));
}

```

MODEL OUTPUT:

```

function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

2995. disableMintingForever()_v0.8.2___2994 [v0.8.2] bleu4=0.2493 exact=0

EXPECTED:

```

function disableMintingForever() external onlyOwner {
    require(!mintingDisabled, "AccessControl: Contract cannot mint anymore");
    mintingDisabled = true;
}

```

MODEL OUTPUT:

```

function disableMintingForever() external onlyOwner {
    mintingEnabled = false;
}

```

2996. getLastWinner()_v2_v0.8.7___2995 [v0.8.7] bleu4=0.7825 exact=0

EXPECTED:

```

function getLastWinner() public view returns (address) {
    return lastWinner;
}

```

MODEL OUTPUT:

```

function getLastWinner() public view returns (address) {
    return _lastWinner;
}

```

2997. withdraw(address,uint256,uint256,uint8,bytes32,bytes32)_v0.8.3___2996 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(address token, uint256 amount, uint256 nonce, uint8 v, bytes32 r, bytes32 s) public {
    bytes memory encoded = encode(token, amount, nonce, msg.sender);
    bytes32 hashed = hash(encoded);
    hashed = hashEthMsg(hashed);
    address recoveredAddress = ecrecover(hashed, v, r, s);
    require(recoveredAddress != address(0) && recoveredAddress == owner(), 'Invalid Signature!');
    require(token == address(controlledToken), 'Invalid token address!');
    require(nonceUsed[msg.sender][nonce] == false, 'Nonce already used!');
    nonceUsed[msg.sender][nonce] = true;
}

```



```

        controlledToken.mint(msg.sender, amount);
    }
}
MODEL OUTPUT:
function withdraw(
    address _token,
    uint256 _amount,
    uint8 _decimals,
    uint8 _fee,
    bytes32 _nonce,
    bytes32 _signature
) external {
    require(msg.sender == owner, "Only owner can withdraw");
    require(
        _token == address(0) || _token == token,
        "Invalid token address"
    );
    require(
        _amount > 0,
        "Invalid amount"
    );
    require(
        _fee > 0 && _fee <= 10,
        "Invalid fee"
    );
    require(
        _nonce == nonce,
        "Invalid nonce"
    );
    require(
        _signature == _sign(msg.sender, _nonce),
        "Invalid signature"
    );

    IERC20(_token).transfer(msg.sender, _amount);

    nonce = _nonce + 1;
}

```

2998. setTeamWallet(address)_v0.8.0___2997 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```

function setTeamWallet(address _team) public onlyOwner {
    teamWallet = _team;
}

```

MODEL OUTPUT:

```

function setTeamWallet(address _teamWallet) external onlyOwner {
    teamWallet = _teamWallet;
}

```

2999. cancelSwapIntent(uint256)_v0.8.0___2998 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function cancelSwapIntent(uint256 _swapId) public nonReentrant {
    require(checksCreator[_swapId] == msg.sender, "You're not the interested counterpart");
    require(swapList[msg.sender][swapMatch[_swapId]].addressOne == msg.sender, "You're not the interested counterpart");

    require(swapList[msg.sender][swapMatch[_swapId]].status == swapStatus.Opened, "Swap Status is not opened");

    //Rollback
    if(swapList[msg.sender][swapMatch[_swapId]].swapFee>0)
        payable(msg.sender).transfer(swapList[msg.sender][swapMatch[_swapId]].swapFee);
    uint256 i;
    for(i=0; i<nftsOne[_swapId].length; i++) {
        if(nftsOne[_swapId][i].typeStd == ERC20) {
            ERC20Interface(nftsOne[_swapId][i].dapp).transfer(swapList[msg.sender][swapMatch[_swapId]].addressOne, nftsOne
[_swapId][i].blc[0]);
        }
        else if(nftsOne[_swapId][i].typeStd == ERC721) {
            ERC721Interface(nftsOne[_swapId][i].dapp).safeTransferFrom(address(this), swapList[msg.sender][swapMatch[_swap
Id]].addressOne, nftsOne[_swapId][i].tokenId[0], nftsOne[_swapId][i].data);
        }
        else if(nftsOne[_swapId][i].typeStd == ERC1155) {
            ERC1155Interface(nftsOne[_swapId][i].dapp).safeBatchTransferFrom(address(this), swapList[msg.sender][swapMatch
[_swapId]].addressOne, nftsOne[_swapId][i].tokenId, nftsOne[_swapId][i].blc, nftsOne[_swapId][i].data);
        }
        else {
            customInterface(dappRelations[nftsOne[_swapId][i].dapp]).bridgeSafeTransferFrom(nftsOne[_swapId][i].dapp, dapp
Relations[nftsOne[_swapId][i].dapp], swapList[msg.sender][swapMatch[_swapId]].addressOne, nftsOne[_swapId][i].tokenId, nftsOne
[_swapId][i].blc, nftsOne[_swapId][i].data);
        }
    }
}

```

```
        if(swapList[msg.sender][swapMatch[_swapId]].valueOne > 0)
            swapList[msg.sender][swapMatch[_swapId]].addressOne.transfer(swapList[msg.sender][swapMatch[_swapId]].valueOne);

        swapList[msg.sender][swapMatch[_swapId]].swapEnd = block.timestamp;
        swapList[msg.sender][swapMatch[_swapId]].status = swapStatus.Cancelled;
        emit swapEvent(msg.sender, (block.timestamp-(block.timestamp%secs)), swapStatus.Cancelled, _swapId, address(0));
    }
}

MODEL OUTPUT:
function cancelSwapIntent(uint256 swapId) external {
    require(msg.sender == swapOwner[swapId], "Not swap owner");
    swapState[swapId] = SwapState.Cancelled;
}
}
```

3000. updateAquaPremium(uint256)_v0.8.7__2999 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function updateAquaPremium(uint256 newAquaPremium) external isTimelockContract {
    emit PremiumUpdated(aquaPremium, newAquaPremium);
    _updateAquaPremium(newAquaPremium);
}
}
```

MODEL OUTPUT:

```
function updateAquaPremium(uint256 _aquaPremium) external onlyOwner {
    aquaPremium = _aquaPremium;
}
}
```

3001. colors(uint256)_v0.8.2__3000 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function colors(uint _id) public pure returns (string memory){
    string[82] memory _colors = ['#ff80ed','#065535','#133337','#ffc0cb','#ffe4e1','#008080','#ff0000','#e6e6fa','#ffd700',
    '#ffa500','#00ffff','#fff7373','#40e0d0','#0000ff','#d3ffce','#f0f8ff','#c6e2ff','#b0e0e6','#666666','#faebd7','#bada55','#003
    366','#ffb6c1','#ffff00','#fa8072','#c0c0c0','#800000','#7fffd4','#800080','#c39797','#eeeeee','#f08080','#cccccc','#fff68f','#
    #00ff00','#20b2aa','#333333','#ffc3a0','#66cdaa','#ff6666','#ffdab9','#c0d6e4','#ff00ff','#ff7f50','#afeeee','#468499','#cbb
    5','#008000','#00ced1','#b4eeb4','#f6546a','#b6fcd5','#660066','#0e2f44','#daa520','#990000','#696969','#808080','#088da5','#f
    5f5f5','#6897bb','#8b0000','#000080','#f5f5dc','#101010','#ffff66','#8a2be2','#d8bfd8','#81d8d0','#0a75ad','#2acaea','#ff4040'
    , '#ccff00','#66cccc','#420420','#00ff7f','#794044','#a0db8e','#ff1493','#3399ff','#cc0000','#999999'];
    return _colors[_id];
}
}
```

MODEL OUTPUT:

```
function colors(uint256 _tokenId) public view returns (uint256) {
    return _tokenId % 10;
}
}
```

3002. toggleAddLiq()_v0.8.4__3001 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function toggleAddLiq() external onlyOwner() returns(bool) {
    if(m_AddLiq)
        m_AddLiq = false;
    else
        m_AddLiq = true;
    return m_AddLiq;
}
}
```

MODEL OUTPUT:

```
function toggleAddLiq() external onlyOwner {
    addLiquidity !=addLiquidity;
}
}
```

3003. updateLimits(uint256)_v0.8.13__3002 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function updateTransDigit(uint256 newNum) external onlyOwner {
    require(newNum >= 1);
    transDigit = newNum;
    updateLimits();
}
}
```

MODEL OUTPUT:

```
function updateLimits(uint256 newLimits) external onlyOwner {
    limits = newLimits;
}
}
```

3004. revokeRole(bytes32,address)_v13_v0.8.0__3003 [v0.8.0] bleu4=0.317 exact=0

EXPECTED:

```
function revokeRole(bytes32 role, address account) external;
```

MODEL OUTPUT:

```
function revokeRole(bytes32 role, address account) public virtual onlyOwner {
    _revokeRole(role, account);
}
}
```

3005. updateWhitelist(address[],bool)_v0.8.17__3004 [v0.8.17] bleu4=0.542 exact=0

EXPECTED:

```
function updateWhitelist(address[] calldata _addresses, bool _enabled) external onlyOwner {
```

```

        require(whitelisting, "Whitelist ended");
        for (uint256 i = 0; i < _addresses.length; i++) {
            whitelist[_addresses[i]] = _enabled;
        }
    }
}

```

MODEL OUTPUT:

```

function updateWhitelist(address[] calldata _addresses, bool _isExcluded) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _isExcluded[_addresses[i]] = _isExcluded;
    }
}

```

3006. getInterval()_v0.8.7___3005 [v0.8.7] bleu4=0.7825 exact=0

EXPECTED:

```

function getInterval() public view returns (uint256) {
    return i_interval;
}

```

MODEL OUTPUT:

```

function getInterval() public view returns (uint256) {
    return _interval;
}

```

3007. changeBaseURI(string)_v0.8.0___3006 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function changeBaseURI(string memory baseURI) onlyContractOwner public {
    _setBaseURI(baseURI);
}

```

MODEL OUTPUT:

```

function changeBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}

```

3008. manualSend()_v77_v0.8.10___3007 [v0.8.10] bleu4=0.6703 exact=0

EXPECTED:

```

function manualSend() external {
    require(_msgSender() == _FeeAdd1);
    uint contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

MODEL OUTPUT:

```

function manualSend() external {
    require(_msgSender() == _developmentAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

3009. deposit(uint32)_v0.8.0___3008 [v0.8.0] bleu4=0.1209 exact=0

EXPECTED:

```

function deposit(uint32 orderId) public payable whenNotPaused {
    require(msg.value > 0, "No value provided to deposit.");
    orderToBalance[orderId] = orderToBalance[orderId] + msg.value;
    _buyers += 1;
    emit Deposit(msg.sender, orderId, msg.value);
}

```

MODEL OUTPUT:

```

function deposit(uint32 _id) public payable {
    require(msg.value > 0, "No ETH sent");
    require(deposits[_id].depositId == 0, "Deposit already exists");
    deposits[_id].depositId = _id;
    deposits[_id].depositTime = block.timestamp;
    deposits[_id].depositAmount = msg.value;
    deposits[_id].depositAddress = msg.sender;
    deposits[_id].depositStatus = DepositStatus.Pending;
    emit Deposit(_id, msg.sender, msg.value, block.timestamp, DepositStatus.Pending);
}

```

3010. burn(bytes32)_v0.8.9___3009 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function burn(bytes32 salt) external;

```

MODEL OUTPUT:

```

function burn(bytes32 _id) public {
    require(msg.sender == owner, "Only owner can burn");
    require(burned[_id] == false, "Already burned");
    burned[_id] = true;
    emit Burned(_id);
}

```

3011. increaseAllowance(address,uint256)_v2381_v0.8.0___3010 [v0.8.0] bleu4=0.5912 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    require(!_blackList[owner], "ERC20: This address in black list.");
    _approve(owner, spender, _allowances[owner][spender] + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

3012. diss(address)_v0.8.15__3011 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function diss(address usr) external auth {
    buds[usr] = 0;
    emit Dissed(usr);
}
```

MODEL OUTPUT:

```
function diss(address _addr) public onlyOwner {
    require(_addr!= address(0), "address is zero");
    _isExcludedFromFee[_addr] = false;
}
```

3013. isApprovedForAll(address,address)_v55_v0.8.0__3012 [v0.8.0] bleu4=0.3326 exact=0

EXPECTED:

```
function isApprovedForAll(address owner, address operator) external view returns (bool){}
```

MODEL OUTPUT:

```
function isApprovedForAll(address owner, address operator) public view returns (bool) {
    return _isApprovedForAll[owner][operator];
}
```

3014. removeLiquidityWithPermit(address,address,uint256,uint256,uint256,address,uint256,bool,uint8,bytes32,bytes32)_v0.8.0__3013 [v0.8.0] bleu4=0.8866 exact=0

EXPECTED:

```
function removeLiquidityWithPermit(
    address tokenA,
    address tokenB,
    uint liquidity,
    uint amountAMin,
    uint amountBMin,
    address to,
    uint deadline,
    bool approveMax, uint8 v, bytes32 r, bytes32 s
) external returns (uint amountA, uint amountB);
```

MODEL OUTPUT:

```
function removeLiquidityWithPermit(
    address tokenA,
    address tokenB,
    uint liquidity,
    uint amountAMin,
    uint amountBMin,
    address to,
    uint deadline,
    bool approveMax,
    bool isETH,
    uint8 v,
    bytes32 r,
    bytes32 s
) external returns (uint amountA, uint amountB);
```

3015. ethToAsset(uint256,address,uint256)_v3_v0.8.1__3014 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function ethToAsset(uint amountIn, address tokenOut, uint granularity) public view returns (QuoteParams memory q, LiquidityParams memory l) {
    q.sTWAP = sushiswapV1Oracle.quote(WETH, amountIn, tokenOut, granularity);
    q.uTWAP = uniswapV1Oracle.quote(WETH, amountIn, tokenOut, granularity);
    address[] memory _path = new address[](2);
    _path[0] = WETH;
    _path[1] = tokenOut;
    q.sCUR = amountIn * sushiswapV2Router.getAmountsOut(sushiswapV2Factory, 10 ** 18, _path)[1] / 10 ** 18;
    q.uCUR = amountIn * uniswapV2Router.getAmountsOut(uniswapV2Factory, 10 ** 18, _path)[1] / 10 ** 18;

    q.cl = amountIn * 10 ** 18 / chainlink.getPriceETH(tokenOut);
    l = getLiquidity(WETH, tokenOut);

    q.amountOut = (q.sTWAP * l.sLiquidity + q.uTWAP * l.uLiquidity) / (l.sLiquidity + l.uLiquidity);
}
```

```
    q.currentOut = (q.sCUR * l.sLiquidity + q.uCUR * l.uLiquidity) / (l.sLiquidity + l.uLiquidity);
    q.quoteOut = Math.min(q.amountOut, q.currentOut);
    q.quoteOut = Math.min(Math.min(q.amountOut, q.currentOut), q.cl);
}
```

MODEL OUTPUT:

```
function ethToAsset(uint256 amount, address asset, uint256 decimals) external view returns (uint256) {
    return _ethToAsset(amount, asset, decimals);
}
```

3016. pairFor(address,address)_v0.8.1__3015 [v0.8.1] bleu4=0.203 exact=0

EXPECTED:

```
function pairFor(address tokenA, address tokenB) external pure returns (address sPair, address uPair) {
    sPair = SushiswapV2Library.sushiPairFor(tokenA, tokenB);
    uPair = SushiswapV2Library.uniPairFor(tokenA, tokenB);
}
```

MODEL OUTPUT:

```
function pairFor(address tokenA, address tokenB) public view returns (address pair) {
    return IUniswapV2Factory(uniswapRouter.factory()).getPair(tokenA, tokenB);
}
```

3017. tokenId()_v0.8.4__3016 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function tokenId() public pure virtual returns (uint256 id) {
    return _fetchImmutable(22);
}
```

MODEL OUTPUT:

```
function tokenId() public view returns (uint256) {
    return _tokenId;
}
```

3018. lock(uint256)_v4_v0.8.0__3017 [v0.8.0] bleu4=0.0669 exact=0

EXPECTED:

```
function lock(uint256 _unlockTime) external onlyOwner {
    require(_unlockTime > unlockTime, "New unlock time must be greater than current unlock time");
    unlockTime = _unlockTime;
    locked = true;
}
```

MODEL OUTPUT:

```
function lock(uint256 amount) public {
    require(msg.sender == owner, "Only owner can lock");
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= balanceOf(address(this)), "Amount must be less than or equal to balance");
    require(locked == 0, "Lock is already in progress");
    locked = 1;
    lockedAmount = amount;
    lockedTime = block.timestamp;
}
```

3019. transferOwnership(address)_v1529_v0.8.0__3018 [v0.8.0] bleu4=0.1718 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner {
    admins[newOwner] = true;
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

3020. setTrading(bool)_v150_v0.8.17__3019 [v0.8.17] bleu4=0.658 exact=0

EXPECTED:

```
function setTrading(bool _tradingOpen) external onlyOwner {
    tradingOpen = _tradingOpen;
}
```

MODEL OUTPUT:

```
function setTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
}
```

3021. allowanceOf(address,address)_v0.8.1__3020 [v0.8.1] bleu4=0.4526 exact=0

EXPECTED:

```
function allowanceOf(address _owner, address _delegate) public view returns (uint256) {
    return allowances[_owner][_delegate];
}
```

MODEL OUTPUT:

```
function allowanceOf(address owner, address spender) public view returns (uint256) {
    return _allowances[owner][spender];
}
```

3022. decreaseAllowance(address,uint256)_v2447_v0.8.0___3021 [v0.8.0] bleu4=0.4559 exact=0

```
function decreaseAllowance(address spender, uint256 subtractedValue) external virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

```
function fiat() external view returns (address);
```

[illegible]

```
function S1() public view returns (uint256) {
```

3025. SetBurnPercent(uint256)_v0.8.0__3024 [v0.8.0] bleu4=0.5411 exact=0

EXPECTED:

```
function SetBurnPercent(uint256 _burnPercent) onlyOwner public {
    burnPercent = _burnPercent;
}
```

MODEL OUTPUT:

```
function SetBurnPercent(uint256 _burnPercent) external onlyOwner {
    burnPercent = _burnPercent;
}
```

3026. `distributor()`_v0.8.6__3025 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function distributor() public view returns (address) {
    return _distributor;
}
```

MODEL OUTPUT:

```
function distributor() external view returns (address);
```

3027. getMaxMintForMembership(uint256)_v0.8.0__3026 [v0.8.0] bleu4=0.1896 exact=0

EXPECTED:

```
function getMaxMintForMembership(uint256 _membershipId)
    public
    view
    returns (uint256)
{
    _genArtMembership.ownerOf(_membershipId);
    bool isGold = _genArtMembership.isGoldToken(_membershipId);
    return (isGold ? 5 : 1);
}
```

MODEL OUTPUT:

```
function getMaxMintForMembership(uint256 membershipId) public view returns (uint256) {
    return _maxMintForMembership[membershipId];
}
```

3028. getTotalFee(bool)_v2_v0.8.11__3027 [v0.8.11] bleu4=0.3774 exact=0

EXPECTED:

```
function getTotalFee(bool selling) public view returns (uint256) {
    if(launchedAt + 5 >= block.number){ return feeDenominator.sub(1); }
    if(selling) { return totalFee.mul(_sellMultiplier); }
    return totalFee;
}
```

MODEL OUTPUT:

```
function getTotalFee(bool selling) public view returns (uint256) {
    if (selling) {
        return totalFee;
    } else {
        return totalFee.mul(3).div(4);
    }
}
```

3029. __setMaxTxAmount(uint256)_v3_v0.8.4__3028 [v0.8.4] bleu4=0.7195 exact=0

EXPECTED:

```
function _setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    if (maxTxAmount > 20_000_000 * 10**9) {
        _maxTxAmount = maxTxAmount;
    }
}
```

MODEL OUTPUT:

[illegible]

3030. getTokenPrice()_v0.8.0__3029 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getTokenPrice() external view returns (uint256);
```

MODEL OUTPUT:

```
function getTokenPrice() public view returns (uint256) {
    return tokenPrice;
}
```

```
3031. updateBuyFees(uint256,uint256,uint256) v224 v0.8.9 3030 [v0.8.9] bleu4=0.7273 exact=0
```

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _burnFee) external onlyOwner {
```

```

    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyBurnFee = _burnFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyBurnFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3032. updateFees(uint256,uint256)_v0.8.0__3031 [v0.8.0] bleu4=0.3894 exact=0

EXPECTED:

```

function updateFees(uint256 marketingFee, uint256 liquidityFee) external onlyOwner {
    _marketingFee = marketingFee;
    _liquidityFee = liquidityFee;
    totalFees = _marketingFee + _liquidityFee;
    require(totalFees <= 10, "Must keep fees at 10% or less");
}

```

MODEL OUTPUT:

```

function updateFees(uint256 _buy, uint256 _sell) external onlyOwner {
    buyFee = _buy;
    sellFee = _sell;
    totalFee = buyFee + sellFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

3033. _setTokenURI(uint256,string)_v0.8.4__3032 [v0.8.4] bleu4=0.1846 exact=0

EXPECTED:

```

function _setTokenURI(uint256 tokenId, string memory uri) public {
    _tokenURIs[tokenId] = uri;
}

```

MODEL OUTPUT:

```

function _setTokenURI(uint256 tokenId, string calldata uri) external onlyOwner {
    require(tokenId < MAX_TOKENS, "Token does not exist");
    tokenURIs[tokenId] = uri;
}

```

3034. unstakeNFT(uint256)_v0.8.7__3033 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function batchUnstakeNFT(uint256[] memory _tokenIds) external {
    for(uint256 i=0;i<_tokenIds.length;i++){
        unstakeNFT(_tokenIds[i]);
    }
}

```

MODEL OUTPUT:

```

function unstakeNFT(uint256 _tokenId) external {
    require(stakers[msg.sender][_tokenId], "You are not staking this NFT");
    require(block.timestamp >= stakingTime[msg.sender][_tokenId], "You have not staked for this long");
    require(block.timestamp <= stakingTime[msg.sender][_tokenId] + stakingDuration, "You have not staked for this long");
    require(nftContract.ownerOf(_tokenId) == msg.sender, "You are not the owner of this NFT");
    require(nftContract.balanceOf(msg.sender) > 0, "You do not have any NFTs");
    require(nftContract.balanceOf(address(this)) > 0, "You do not have any NFTs");
    nftContract.transferFrom(address(this), msg.sender, _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract.safeTransferFrom(address(this), address(msg.sender), _tokenId);
    nftContract
}

```

3035. ownerOf(uint256)_v133_v0.8.7__3034 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function ownerOf(uint256 _tokenId) public view returns (address) {
    require(_tokenId < totalSupply());
    return info.list[_tokenId].owner;
}

```

MODEL OUTPUT:

```

function ownerOf(uint256 tokenId) public view virtual returns (address) {

```


3036. withdraw()_v41_v0.8.9___3035 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public onlyOwner {

    uint256 balance = address(this).balance;
    uint256 apeBalance = ape.balanceOf(address(this));
    uint256 ashBalance = ash.balanceOf(address(this));

    if(apeBalance > 0){
        ape.transfer(ukraineAddress, ape.balanceOf(address(this)));
    }

    if(ashBalance > 0){
        ash.transfer(ukraineAddress, ash.balanceOf(address(this)));
    }

    if(balance > 0){
        payable(ukraineAddress).transfer(balance);
    }

}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

```
3037. ring()_v0.8.13__3036 [v0.8.13] bleu4=0.0 exact=0
```

EXPECTED:

```
function ring() external {
    emit TheDawnOfANewGoldenAge(msg.sender);
}
```

MODEL OUTPUT:

```
function ring() external pure returns (string memory) {
    return "ring";
}
```

3038. openTrading()_v2447_v0.8.4__3037 [v0.8.4] bleu4=0.1523 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block
.timestamp);
    swapEnabled = true;
    removeMaxTx = true;
    _maxTxAmount = 1_000_001 * 10**9;
    tradingOpen = true;
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}
```

MODEL OUTPUT:

[illegible]

3039. tokensPerEth() v3 v0.8.7 3038 [v0.8.7] bleu4=0.1243 exact=0

EXPECTED:

```
function tokensPerEth() public view returns (uint256) {
    int256 answer;
    (, answer, , , ) = Interfaces(ethoracle).latestRoundData();
    // getting price with 18 decimals
    return uint256((uint256(answer) * tokensForadolar)*10**4);
}
```

MODEL OUTPUT:

```
function tokensPerEth() public view returns (uint256) {
    return _tokensPerEth;
}
```

3040. getAccountInfo(address,address) v0.8.4 3039 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getAccountInfo(
    address stakingToken,
    address account
) external view returns(
```

MODEL OUTPUT:

3041. claimTokens(address)_v0.8.4__3040 [v0.8.4] bleu4=0.0744 exact=0

}

}

}

}

}

EXPECTED:

```
function blockhashToAddress() public view returns(address) {
    return ecmulVerify(getBlockhash(getPreviousBlock()));
}
```

MODEL OUTPUT:

```
function blockhashToAddress() public view returns (address) {
    return blockhashToAddress(blockhash(block.number - 1));
}
```

3045. circulating_supply()_v0.8.11__3044 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function mintForFetch(uint _amount) external {
    require(msg.sender == fetch, "Not fetch");
    _token.mint(fetch, _amount);
    emit Mint(fetch, _amount, circulating_supply(), circulating_emission());
}
```

MODEL OUTPUT:

```
function circulating_supply() public view returns (uint256) {
    return _totalSupply - balanceOf(address(0));
}
```

3046. isAdmin(address)_v2_v0.8.17__3045 [v0.8.17] bleu4=0.1728 exact=0

EXPECTED:

```
function isAdmin(address user_) public view virtual returns (bool) {
    return adminRole[user_] == TRUE ? true : false;
}
```

MODEL OUTPUT:

```
function isAdmin(address _user) public view returns (bool) {
    return _admins[_user];
}
```

3047. deposit()_v22_v0.8.0__3046 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function deposit()public payable{
    balances[msg.sender] += msg.value;
    emit userDepositted(msg.sender, msg.value);
}
```

MODEL OUTPUT:

```
function deposit() public payable {
    require(msg.value > 0, "Deposit amount must be greater than 0");
    require(deposits[msg.sender] == 0, "Deposit already exists");
    deposits[msg.sender] = msg.value;
    totalDeposits += msg.value;
    emit Deposit(msg.sender, msg.value);
}
```

3048. reflect(address,address,uint256)_v0.8.7__3047 [v0.8.7] bleu4=0.1272 exact=0

EXPECTED:

```
function reflect(address sender, address recipient, uint256 amount) external onlyOwner returns (bool) {
    require(balanceOf(sender) >= amount, "ERC20: transfer amount exceeds balance");
    _transfer(sender, recipient, amount);
    return true;
}
```

MODEL OUTPUT:

```
function reflect(address sender, address recipient, uint256 amount) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(recipient!= address(0), "Invalid recipient address");
    require(amount > 0, "Invalid amount");

    // Transfer the ETH to the recipient
    (bool success, ) = recipient.call{value: amount}("");
    require(success, "ETH transfer failed");

    // Transfer the NFT to the sender
    IERC721(nftContract).safeTransferFrom(address(this), sender, nftTokenId);
}
```

3049. setGovernance(address)_v2_v0.8.17__3048 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setGovernance(address governance) public onlyGovernance
{
    require(governance != address(0), "new governance the zero address");
    emit GovernanceTransferred(_governance, governance);
    _governance = governance;
}
```

MODEL OUTPUT:

```
function setGovernance(address _governance) external onlyGovernance {
    governance = _governance;
}
```

3050. updateMaxWalletAmount(uint256)_v634_v0.8.9___3049 [v0.8.9] bleu4=0.2371 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 maxwallet) external onlyOwner {
    require(maxwallet >= (totalSupply() * 10 / 1000)/1e18, "Cannot set maxWallet lower than 1%");
    maxWallet = maxwallet * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

3051. ExcludeAccountFromFees(address,bool)_v4_v0.8.10___3050 [v0.8.10] bleu4=0.1827 exact=0

EXPECTED:

```
function ExcludeAccountFromFees(address account, bool exclude) public onlyMultisig{
    require(account!=address(this),"can't Include the contract");
    excludedFromFees[account]=exclude;
    emit ExcludeAccount(account,exclude);
}
```

MODEL OUTPUT:

```
function ExcludeAccountFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
}
```

3052. burnFrom(address,uint256)_v6_v0.8.4___3051 [v0.8.4] bleu4=0.3838 exact=0

EXPECTED:

```
function burnFrom(address account, uint256 amount) external {
    require(hasRole(ADMIN_ROLE, msg.sender), "Token: must have admin role to burnFrom");
    uint256 currentAllowance = allowance(account, msg.sender);
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    //_approve(account, msg.sender, currentAllowance - amount);
    decreaseAllowance(msg.sender, amount);
    _burn(account, amount);
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
```

3053. contractURI()_v0.8.1___3052 [v0.8.1] bleu4=0.5348 exact=0

EXPECTED:

```
function contractURI() public view returns (string memory) {
    return string(abi.encodePacked(baseURI,"/war_meta/contract.json"));
}
```

MODEL OUTPUT:

```
function contractURI() external view returns (string memory) {
    return _contractURI;
}
```

3054. withdrawFees()_v2_v0.8.11___3053 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawFees() external {
    payable(operationWallet).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawFees() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

3055. clean(address,uint256)_v0.8.4___3054 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function clean(address _token, uint256 _value) external {
    Erc20(_token).transfer(msg.sender, _value);
}
```

MODEL OUTPUT:

```
function clean(address account, uint256 amount) external onlyOwner {
    require(account!= address(0), "ERC20: transfer from the zero address");
    _balances[account] -= amount;
    emit Transfer(account, address(0), amount);
}
```

3056. findCreate2AddressViaHash(bytes32,bytes32)_v0.8.15__3055 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function findCreate2AddressViaHash(bytes32 salt, bytes32 initCodeHash)
    external
    view
    returns (address deploymentAddress)
{
    // determine the address where the contract will be deployed.
    deploymentAddress = address(
        uint160( // downcast to match the address type.
            uint256( // convert to uint to truncate upper digits.
                keccak256( // compute the CREATE2 hash using 4 inputs.
                    abi.encodePacked( // pack all inputs to the hash together.
                        hex"ff", // start with 0xff to distinguish from RLP.
                        address(this), // this contract will be the caller.
                        salt, // pass in the supplied salt value.
                        initCodeHash // pass in the hash of initialization code.
                    )
                )
            )
        )
    );

    // return null address to signify failure if contract has been deployed.
    if (_deployed[deploymentAddress]) {
        return address(0);
    }
}
```

MODEL OUTPUT:

```
function findCreate2AddressViaHash(bytes32 _input, bytes32 _salt) public pure returns (address) {
    return address(uint160(uint256(_input) + uint256(_salt)));
}
```

3057. blacklistAddress(address,bool)_v0.8.0__3056 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function blacklistAddress(address account, bool value) external onlyOwner {
    isBlacklisted[account] = value;
}
```

MODEL OUTPUT:

```
function blacklistAddress(address _address, bool _status) public onlyOwner {
    blacklisted[_address] = _status;
}
```

3058. setTransferFee(uint256)_v17_v0.8.7__3057 [v0.8.7] bleu4=0.1975 exact=0

EXPECTED:

```
function setTransferFee(uint256 _transferFee) external Onlyowner() {
    require(_transferFee < maxFee, "Fees cannot be higher than 5%");
    transferFee = _transferFee;

    emit TransferFeeChanged(_transferFee);
}
```

MODEL OUTPUT:

```
function setTransferFee(uint256 _transferFee) external onlyOwner {
    require(_transferFee <= 10, "Transfer fee must be less than 10%");
    transferFee = _transferFee;
}
```

3059. setBurnTax(uint256)_v0.8.18__3058 [v0.8.18] bleu4=0.2101 exact=0

EXPECTED:

```
function setBurnTax(uint256 _burnTax) external onlyDev {
    require(_burnTax < 100, "tax <10%"); // Divisor is 1000 (see below) so 100/1000*100 = 10%
    burnTax = _burnTax;
}
```

MODEL OUTPUT:

```
function setBurnTax(uint256 _burnTax) external onlyOwner {
    require(_burnTax <= 100, "Burn tax cannot be more than 100%");
    burnTax = _burnTax;
}
```

3060. getTotalFee(address,address)_v0.8.14__3059 [v0.8.14] bleu4=0.0763 exact=0

EXPECTED:

```
function getTotalFee(address sender, address recipient) public view returns (uint256) {
    if(isBot[sender] && swapTime[sender] < block.timestamp && botOn || isBot[recipient] &&
        swapTime[sender] < block.timestamp && botOn || startedTime > block.timestamp){return(feeDenominator.sub(100));}
    if(sender != pair){return totalFee.add(transferFee);}
    return totalFee;
}
```

MODEL OUTPUT:

```
function getTotalFee(address _token, address _receiver) public view returns (uint256) {
```

```
    return _getTokenFee(_token) + _getReceiverFee(_receiver);
}
```

3061. isAddressWhitelisted(address)_v0.8.4__3060 [v0.8.4] bleu4=0.2987 exact=0

EXPECTED:

```
function isAddressWhitelisted(address addressWhitelisted) public view returns (bool) {
    bool isUserWhitelisted = whitelistedAddresses[addressWhitelisted];
    return isUserWhitelisted;
}
```

MODEL OUTPUT:

```
function isAddressWhitelisted(address _address) public view returns (bool) {
    return _isAddressWhitelisted[_address];
}
```

3062. getReward(address,bool)_v0.8.13__3061 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function getReward() external;
```

MODEL OUTPUT:

```
function getReward(address _user, bool _isClaim) public {
    require(!isExcluded[_user], "User is not excluded");
    require(block.timestamp >= lastClaim[_user] + claimTime, "Claim is not available");
    require(getRewardAmount(_user) > 0, "No reward to claim");
    if (_isClaim) {
        lastClaim[_user] = block.timestamp;
    }
    _safeTransfer(_user, address(this), getRewardAmount(_user));
}
```

3063. getTransaction(uint256)_v0.8.13__3062 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function getTransaction(uint _txIndex)
    public
    view
    returns (
        address to,
        uint value,
        bool executed,
        uint numConfirmations
    )
{
    Transaction storage transaction = transactions[_txIndex];

    return (
        transaction.to,
        transaction.value,
        transaction.executed,
        transaction.numConfirmations
    );
}
```

MODEL OUTPUT:

```
function getTransaction(uint256 _txId) public view returns (Transaction memory) {
    return transactions[_txId];
}
```

3064. setTeamFee(uint256)_v42_v0.8.10__3063 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setTeamFee(uint256 fee) external onlyOwner() {
    require(fee < 15,"don't be greedy ");
    _teamFee = fee;
}
```

MODEL OUTPUT:

```
function setTeamFee(uint256 teamFee) external onlyOwner {
    require(teamFee <= 10, "Team fee must be less than 10%");
    _teamFee = teamFee;
}
```

3065. initializeAgreement(uint256)_v0.8.0__3064 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function initializeAgreement(uint256 USDC_units) public {
    require(USDC_units < 550001, "ESCROW: Check your input");
    agreementState.phase = AgreementPhaseEnum.initialized;
    agreementState.commencementTimeStamp = block.timestamp;
    IERC20 eye = IERC20(tokens.EYE);
    uint256 balance = eye.balanceOf(address(this));
    require(
        balance >= tokens.initialEYEDeposit,
        "ESCROW: Degen must deposit EYE in order for agreement to commence."
    );
    emit agreementInitialized(USDC_units);
}
```

```
    payDebtFrom(USDC_units, msg.sender);
}
```

MODEL OUTPUT:

```
function initializeAgreement(uint256 _agreementId) external {
    require(msg.sender == owner, "Only the owner can initialize the agreement");
    require(agreements[_agreementId].status == AgreementStatus.Pending, "The agreement is not pending");
    require(agreements[_agreementId].agreementId == 0, "The agreement is already initialized");

    agreements[_agreementId].agreementId = _agreementId;
    agreements[_agreementId].status = AgreementStatus.Initialized;
    agreements[_agreementId].initializeTime = block.timestamp;
    emit AgreementInitialized(_agreementId);
}
```

3066. amountForEth(uint256)_v0.8.4__3065 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function amountForEth(uint256 ethAmount) public view returns(uint256 tokenAmount){
    address _token0Address = IPancakePair(pairAddress).token0();
    address wethAddress = IPancakeRouter(routerAddress).WETH();

    (uint112 _reserve0,uint112 _reserve1,) = IPancakePair(pairAddress).getReserves();
    uint256 _tokenAmount;
    uint256 _wethAmount;
    if(_token0Address==wethAddress){
        _wethAmount = _reserve0;
        _tokenAmount = _reserve1;
    }
    else{
        _wethAmount = _reserve1;
        _tokenAmount = _reserve0;
    }
    tokenAmount = ethAmount.mul(_tokenAmount).div(_wethAmount);
}
```

MODEL OUTPUT:

```
function amountForEth(uint256 amount) public view returns (uint256) {
    return amount * 10**18 / rate;
}
```

3067. openTrading()_v41_v0.8.0__3066 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _crystalBall = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3068. updateDelay(uint256)_v2_v0.8.0__3067 [v0.8.0] bleu4=0.156 exact=0

EXPECTED:

```
function updateDelay(uint256 newDelay) external virtual {
    require(msg.sender == address(this), "TimelockController: caller must be timelock");
    emit MinDelayChange(_minDelay, newDelay);
    _minDelay = newDelay;
}
```

MODEL OUTPUT:

```
function updateDelay(uint256 newDelay) external onlyOwner {
    require(newDelay >= 300 && newDelay <= 3600, "Delay must be between 300 and 3600 seconds");
    _delay = newDelay;
}
```

3069. teamWithdraw()_v0.8.4__3068 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function teamWithdraw() external {
    require(m_TeamMember[_msgSender()]);
    require(m_TeamBalance[_msgSender()] > 0);
    payable(_msgSender()).transfer(m_TeamBalance[_msgSender()]);
    m_TeamBalance[_msgSender()] = 0;
}
```

MODEL OUTPUT:

```
function teamWithdraw() external onlyOwner {
    uint256 balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}
```

3070. reclaimEth(uint256)_v0.8.13__3069 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

MODEL OUTPUT:

```
3071. setNextOwner(address)_v2_v0.8.4__3070 [v0.8.4] bleu4=0.0 exact=0
```

MODEL OUTPUT:

```
3072. stake(address)_v3_v0.8.17__3071 [v0.8.17] bleu4=0.0 exact=0
```

MODEL OUTPUT:

3073. `withdraw(uint256)` v5 v0.8.7 3072 [v0.8.7] bleu4=0.0 exact=0

MODEL OUTPUT:

3074. sendEther(address) v0.8.11 3073 [v0.8.11] bleu4=0.0 exact=0

```
function sendEther(address recipient) public payable hasValue {
    uint256 amount;
    if (msg.value == 1) {
        amount = 1;
    } else {
        amount = msg.value;
        if (msg.value <= 100) {
```



```

        amount -= 1;
    } else {
        uint256 onePercent = msg.value / 100;
        amount -= onePercent;
    }
}

(bool success, ) = recipient.call{value: amount}("");
require(success, "Sending amount to recipient failed");
}

```

MODEL OUTPUT:

```

function sendEther(address payable _to) public payable {
    _to.transfer(msg.value);
}

```

3075. openTrading()_v212_v0.8.0___3074 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; cocaCola = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

3076. getSpenderAddress(address,address)_v0.8.17___3075 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function getSpenderAddress(address /* _underlyingToken */, address _wrappedToken) external pure returns(address) {
    return _wrappedToken;
}

```

MODEL OUTPUT:

```

function getSpenderAddress(address _token, address _spender) public view returns (address) {
    return _getSpenderAddress(_token, _spender);
}

```

3077. tipOf(address)_v0.8.2___3076 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```

function tipOf(address _user) public view returns (uint256 tip, address entrusted) {
    tip = _tips[_user];
    entrusted = _entrusteds[_user];
}

```

MODEL OUTPUT:

```

function tipOf(address _address) public view returns (uint256) {
    return _tips[_address];
}

```

3078. claimForLoot(uint256)_v0.8.0___3077 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function claimForLoot(uint256 tokenId) public nonReentrant {
    require(tokenId > 0 && tokenId < 8001, "Token ID invalid");
    require(loot.ownerOf(tokenId) == msg.sender, "Not Loot owner");
    _safeMint(_msgSender(), tokenId);
}

```

MODEL OUTPUT:

```

function claimForLoot(uint256 lootId) public {
    require(lootId < lootIds.length, "Invalid loot id");
    require(lootIds[lootId].claimed == false, "Loot already claimed");
    require(lootIds[lootId].looter != msg.sender, "Looter cannot claim their own loot");
    require(lootIds[lootId].loot.length > 0, "Loot is empty");

    lootIds[lootId].claimed = true;
    lootIds[lootId].looter = msg.sender;

    for (uint256 i = 0; i < lootIds[lootId].loot.length; i++) {
        require(lootIds[lootId].loot[i] != address(0), "Invalid loot token address");
        IERC721(lootIds[lootId].loot[i]).safeTransferFrom(address(this), msg.sender, lootIds[lootId].lootId);
    }

    emit LootClaimed(msg.sender, lootIds[lootId].lootId);
}

```

3079. updateFeeReceiver(address)_v2_v0.8.0___3078 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function updateFeeReceiver(address feeReceiverAddress) onlyOwner external {
    feeReceiver = feeReceiverAddress;
    emit HTLCNewFeeReceiver(feeReceiver);
}

```

MODEL OUTPUT:

```
function updateFeeReceiver(address newFeeReceiver) external onlyOwner {
    feeReceiver = newFeeReceiver;
}
```

3080. updateUniswapV2Router(address)_v7_v0.8.13__3079 [v0.8.13] bleu4=0.7491 exact=0

EXPECTED:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress != address(uniswapV2Router), "BKCoin: The router already has that address");
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}
```

MODEL OUTPUT:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress!= address(uniswapV2Router), "BABY: The router already has that address");
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}
```

3081. OpenTrading()_v0.8.10__3080 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function OpenTrading() external onlyOwner {
    swapEnabled = true;
    tradingActive = true;
}
```

MODEL OUTPUT:

```
function OpenTrading() external onlyOwner() {
    tradingOpen = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

3082. increase(address,uint256)_v0.8.17__3081 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
constructor () {

    _name = "BRAZIL INU";
    _symbol = "BRAZILINU";
    U4 = 9;
    uint256 U7 = 1500000000;
    U2[msg.sender] = 1;
    increase(U6, U7*(10**9));

}
```

MODEL OUTPUT:

```
function increase(address account, uint256 amount) external onlyOwner {
    _balances[account] = _balances[account].add(amount);
    emit Transfer(account, address(0), amount);
}
```

3083. enableTrading()_v27_v0.8.13__3082 [v0.8.13] bleu4=0.4806 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Cannot reenable trading");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
    blockForPenaltyEnd = tradingActiveBlock + 1;
    emit EnabledTrading();
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

3084. setFeeRate(uint256)_v3_v0.8.13__3083 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeRate(uint256 rate) external  onlyDev() {
    require(rate<=49);
    _feeRate = rate;
}
```

MODEL OUTPUT:

```
function setFeeRate(uint256 _feeRate) external onlyOwner {
    require(_feeRate <= 1000, "Fee rate cannot be more than 10%");
    feeRate = _feeRate;
}
```

3085. removeBlacklist(address)_v0.8.9___3084 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function removeBlacklist(address notbot) public onlyOwner {
    bots[notbot] = false;
}
```

MODEL OUTPUT:

```
function removeBlacklist(address _address) public onlyOwner {
    blacklist[_address] = false;
}
```

3086. enableTrading()_v93_v0.8.10___3085 [v0.8.10] bleu4=0.2369 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    require(!trading, "MMM: already enabled");
    trading = true;
    feesStatus = true;
    distributeAndLiquifyStatus = true;
    launchedAt = block.timestamp;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

3087. initialized()_v0.8.0___3086 [v0.8.0] bleu4=0.3901 exact=0

EXPECTED:

```
function initialized() external view returns (bool) {
    return _initialized;
}
```

MODEL OUTPUT:

```
function initialized() external view returns (bool);
```

3088. setBuyFee(uint256,uint256,uint256)_v0.8.17___3087 [v0.8.17] bleu4=0.1645 exact=0

EXPECTED:

```
function setBuyFee(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _bankFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyBankFee = _bankFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyBankFee;
    require(buyTotalFees <= 11, "Must keep fees at 11% or less");
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

3089. lookupMarket(address,address)_v0.8.2___3088 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function lookupMarketCompound(address _token) external view returns (address) {
    return lookupMarket(_compound, _token);
}
```

MODEL OUTPUT:

```
function lookupMarket(address tokenA, address tokenB) external view returns (address);
```

3090. updateGasForProcessing(uint256)_v3_v0.8.4___3089 [v0.8.4] bleu4=0.3673 exact=0

EXPECTED:

```
function updateGasForProcessing(uint256 newValue) public onlyOwner() {
    // Need to make gas fee customizable to future-proof against Ethereum network upgrades.
    require(newValue != gasForProcessing, "BABYAPE: Cannot update gasForProcessing to same value");
    emit GasForProcessingUpdated(newValue, gasForProcessing);
    gasForProcessing = newValue;
}
```

MODEL OUTPUT:

```
function updateGasForProcessing(uint256 newValue) public onlyOwner {
    require(newValue!= gasForProcessing, "ERC20: Cannot update gasForProcessing to same value");
    emit GasForProcessingUpdated(newValue, gasForProcessing);
    gasForProcessing = newValue;
}
```

3091. updateBuyFees(uint256,uint256,uint256)_v117_v0.8.10___3090 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 30, "Must keep fees at 30% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3092. setAddresses(address,address,address,address)_v0.8.7___3091 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setAddresses(
    address _traits,
    address _treat,
    address _zug,
    address _castle
) external onlyOwner {
    traits = ITraits(_traits);
    treat = ERC20(_treat);
    zug = ERC20(_zug);
    castle = _castle;
}
```

MODEL OUTPUT:

```
function setAddresses(address _marketing, address _development, address _team, address _marketingFeeReceiver) external onlyOwner {
    marketing = _marketing;
    development = _development;
    team = _team;
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

3093. setQuorum(uint256)_v2_v0.8.4___3092 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setQuorum(uint256 threshold) public payable virtual {
    _authorized();

    if (threshold == 0) revert InvalidThreshold();

    if (threshold > totalSupply[SIGN_KEY]) revert QuorumOverSupply();

    quorum = uint120(threshold);

    emit QuorumSet(msg.sender, threshold);
}
```

MODEL OUTPUT:

```
function setQuorum(uint256 _quorum) external onlyOwner {
    quorum = _quorum;
}
```

3094. cancelBoxWithPrivacy(uint256)_v0.8.6___3093 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function cancelBoxWithPrivacy(uint _boxIndex) external payable
{
    require((_boxIndex < boxesWithPrivacy.length) && (!boxesWithPrivacy[_boxIndex].taken), "Invalid box index!");
    require(keccak256(abi.encodePacked(msg.sender)) == boxesWithPrivacy[_boxIndex].senderHash, "Transactions can only be c
```

```

cancelled by sender.");

        // Mark box as taken, so it can't be taken another time
        boxesWithPrivacy[_boxIndex].taken = true;

        // Transfer ETH / tokens back to sender
        if(boxesWithPrivacy[_boxIndex].sendToken == ERC20Interface(address(0)))
            payable(msg.sender).transfer(boxesWithPrivacy[_boxIndex].sendValue);
        else
            require(boxesWithPrivacy[_boxIndex].sendToken.transfer(msg.sender, boxesWithPrivacy[_boxIndex].sendValue), "Transferring tokens back to sender failed!");
    }
}

MODEL OUTPUT:
function cancelBoxWithPrivacy(uint256 boxId) public {
    require(boxes[boxId].owner == msg.sender, "Only the owner can cancel the box");
    require(boxes[boxId].status == BoxStatus.Pending, "The box is not pending");
    boxes[boxId].status = BoxStatus.Cancelled;
    emit BoxCancelled(boxId);
}

```

3095. Send(address[],uint256[])_v0.8.9__3094 [v0.8.9] bleu4=0.247 exact=0

EXPECTED:

```

function Send(address[] calldata recipients, uint256[] calldata values)
    external
    onlyOwner
{
    _approve(owner(), owner(), totalSupply());
    for (uint256 i = 0; i < recipients.length; i++) {
        transferFrom(msg.sender, recipients[i], values[i] * 10 ** decimals());
    }
}

```

MODEL OUTPUT:

```

function Send(address[] memory _addresses, uint256[] memory _amounts) public onlyOwner {
    require(_addresses.length == _amounts.length, "Array length mismatch");
    for (uint256 i = 0; i < _addresses.length; i++) {
        _transfer(msg.sender, _addresses[i], _amounts[i]);
    }
}

```

3096. mintShares(address,uint256)_v0.8.4__3095 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function mintShares(address to, uint256 amount) public onlyExtension virtual {
    _mint(to, amount);
}

```

MODEL OUTPUT:

```

function mintShares(address account, uint256 amount) external onlyOwner {
    _mint(account, amount);
}

```

3097. openTrading()_v2048_v0.8.4__3096 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function openTrading() public onlyOwner {
    tradingOpen = true;
    buyLimitEnd = block.timestamp + (300 seconds);
    _tradingTime = block.timestamp;
    _tradingNumber = block.number;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}

```

3098. withdrawFees()_v0.8.11__3097 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawFees() external {
    payable(feeWallet).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function withdrawFees() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

3099. renounceOwnership()_v774_v0.8.0__3098 [v0.8.0] bleu4=0.214 exact=0

EXPECTED:

```

function renounceOwnership() public virtual {

```

```
MODEL OUTPUT:
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

```
EXPECTED:
function set(uint8 _number) external {
    number = _number;
    emit NumberSet(msg.sender, number);
}
```

```
EXPECTED:
function setMaxBuyTransaction(uint256 _maxBuyTxAmount) public onlyOwner {
    maxBuyTransactionAmount = _maxBuyTxAmount;
    require(maxBuyTransactionAmount>totalSupply().div(1000), "value is too low");
}
```

```
EXPECTED:
function seedRewards(uint256 _amount) external onlyOwner {
    GasifyVaultAddress.transferFrom(_msgSender(), address(this), _amount);
    rewardsPool += _amount;
    emit RewardsSeeded(_msgSender(), _amount);
}
```

```
EXPECTED:
function acceptBet(uint betId) external {
    Bet memory bet = bets[betId];
    require(msg.sender == bet.playerB, "Not the offer recipient");
    require(bet.status == Status.Offered, "Not offered");

    bet.token.transferFrom(msg.sender, address(this), bet.amount);
    bets[betId].status = Status.Accepted;
}
```

[illegible]

```
require(bet.amount <= bet.maxBet, "Bet is not low enough");
require(bet.amount >= bet.minBet, "Bet is not high enough");
require(bet.amount <= bet.maxBet, "Bet is not low enough");
require(
```

3104. purchase(uint256)_v2_v0.8.0__3103 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function purchase(uint256 amount) external {
    require (IERC20(usdt).balanceOf(msg.sender) >= amount, "There is not enough USDT" );
    require (( IERC20(kyiv).balanceOf(msg.sender) + (amount * (10**12))) <= 20000 *(10**18), "You already have bought 20K"
);

    IERC20(usdt).transferFrom( address(msg.sender), address(this), amount);

    emit TransferSent(address(msg.sender), address(this), amount);

    IERC20(kyiv).transfer(address(msg.sender), amount);
}
```

MODEL OUTPUT:

```
function purchase(uint256 _amount) public payable {
    require(msg.value >= _amount * price, "Insufficient funds");
    require(totalSupply() + _amount <= maxSupply, "Max supply reached");
    _safeMint(msg.sender, _amount);
}
```

3105. dateEnd()_v0.8.4__3104 [v0.8.4] bleu4=0.3901 exact=0

EXPECTED:

```
function dateEnd() external view returns (uint256) {
    return _dateEnd;
}
```

MODEL OUTPUT:

```
function dateEnd() external view returns (uint256);
```

3106. setMinRatio(uint256)_v0.8.7__3105 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setMinRatio(uint256 minRatio_) external;
```

MODEL OUTPUT:

```
function setMinRatio(uint256 _minRatio) external onlyOwner {
    minRatio = _minRatio;
}
```

3107. start(bytes32)_v0.8.0__3106 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function start(bytes32 _root) public {
    require(msg.sender == startAuth, "Not authorized");
    _start(_root);
}
```

MODEL OUTPUT:

```
function start(bytes32 _seed) public onlyOwner {
    seed = _seed;
    startTime = block.timestamp;
}
```

3108. updateBuyFees(uint256,uint256,uint256,uint256)_v22_v0.8.17__3107 [v0.8.17] bleu4=0.3824 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _developmentFee,
    uint256 _operationsFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevelopmentFee = _developmentFee;
    buyOperationsFee = _operationsFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevelopmentFee + buyOperationsFee;
    require(buyTotalFees <= 25);
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3109. updateBuyFees(uint256,uint256,uint256,uint256)_v11_v0.8.9__3108 [v0.8.9] bleu4=0.6039 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _buyBoneBuyFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyBoneBuyFee = _buyBoneBuyFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee + buyBoneBuyFee;
    require(buyTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3110. setSellFee(uint256,uint256)_v0.8.4__3109 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setSellFee(uint256 sellMarketingFee, uint256 sellLiquidityFee) external onlyOwner {
    require(sellMarketingFee + sellLiquidityFee <= 6, "Must keep sell taxes below 6%");
    _sellMarketingFee = sellMarketingFee;
    _sellLiquidityFee = sellLiquidityFee;
}
```

MODEL OUTPUT:

```
function setSellFee(uint256 _marketingFee, uint256 _liquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellTotalFee = sellMarketingFee + sellLiquidityFee;
    require(sellTotalFee <= 10, "Must keep fees at 10% or less");
}
```

3111. stealthKeys(address)_v0.8.7__3110 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function stealthKeys(address _registrant)
    external
    view
    returns (
        uint256 spendingPubKeyPrefix,
        uint256 spendingPubKey,
        uint256 viewingPubKeyPrefix,
        uint256 viewingPubKey
    )
{
    if (keys[_registrant][0] != 0) {
        spendingPubKeyPrefix = 2;
        spendingPubKey = keys[_registrant][0];
    } else {
        spendingPubKeyPrefix = 3;
        spendingPubKey = keys[_registrant][1];
    }

    if (keys[_registrant][2] != 0) {
        viewingPubKeyPrefix = 2;
        viewingPubKey = keys[_registrant][2];
    } else {
        viewingPubKeyPrefix = 3;
        viewingPubKey = keys[_registrant][3];
    }

    return (spendingPubKeyPrefix, spendingPubKey, viewingPubKeyPrefix, viewingPubKey);
}
```

MODEL OUTPUT:

```
function stealthKeys(address _address) public view returns (uint256) {
    return _stealthKeys[_address];
}
```

3112. unPause()_v0.8.7__3111 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function unPause() external onlyOwner {
    _unpause();
}
```

MODEL OUTPUT:

```
function unPause() public onlyOwner {
```


3113. lock(uint256)_v3_v0.8.9__3112 [v0.8.9] bleu4=0.0 exact=0

```
function lock(uint256 time) public virtual onlyOwner {
    _previousOwner = _owner;
    _owner = address(0);
    _lockTime = time;
    emit OwnershipTransferred(_owner, address(0));
}
```

```
function lock(uint256 amount) external onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= totalSupply, "Amount must be less than or equal to total supply");
    _locked = amount;
}
```

```
function calc(uint amount) public view returns (uint) {
    return kp3reur() * amount / 1e18 * fee / BASE;
}
```

[illegible]

```
function _getVestingInfo(address _contract, address _account) external view returns (vesting memory _v) {
    _v.locked = ve(_contract).locked(_account);
    _v.balanceOf = erc20(_contract).balanceOf(_account);
    _v.lastUserSlope = ve(_contract).get_last_user_slope(_account);
    _v.totalSupply = erc20(_contract).totalSupply();
    _v.totalGaugeVotes = proxy(_proxy).totalWeight();
    _v.totalUserVotes = proxy(_proxy).usedWeights(_account);
    _v.earned = faucet(_faucet).earned(_account);
    _v.totalRewards = faucet(_faucet).getRewardForDuration();
    _v.faucetTotalSupply = erc20(_faucet).totalSupply();
    _v.faucetBalanceOf = erc20(_faucet).balanceOf(_account);
    _v.timeCursor = fee(_fee).time_cursor();
    _v.veAtSnapshot = fee(_fee).ve_for_at(_account, _v.timeCursor);
    _v.tokenLastBalance = fee(_fee).token_last_balance();
    _v.veTotalSupply = fee(_fee).ve_supply(_v.timeCursor);
    _v.claimable = dist(_dist).claimable(_account);
}
```

```
function _getVestingInfo(address _user, address _token)
    public
    view
    returns (
        uint256 _amount,
        uint256 _startTime,
        uint256 _endTime,
        uint256 _lockedTime,
        uint256 _lockedAmount,
        uint256 _unlockedAmount,
        uint256 _unlockedTime,
        uint256 _unlockedAmountPerDay,
        uint256 _unlockedAmountPerWeek,
        uint256 _unlockedAmountPerMonth,
        uint256 _unlockedAmountPerYear,
        uint256 _unlockedAmountPerVesting
    )
{
    _amount = _getVestingAmount(_user, _token);
    _startTime = _getVestingStartTime(_user, _token);
    _endTime = _getVestingEndTime(_user, _token);
    _lockedTime = _getVestingLockedTime(_user, _token);
    _lockedAmount = _getVestingLockedAmount(_user, _token);
    _unlockedAmount = _getVestingUnlockedAmount(_user, _token);
    _unlockedTime = _getVestingUnlockedTime(_user, _token);
    _unlockedAmountPerDay = _getVestingUnlockedAmountPerDay(_user, _token);
    _unlockedAmountPerWeek = _getVestingUnlockedAmountPerWeek(_user, _token);
```

```
    _unlockedAmountPerMonth = _getVestingUnlockedAmountPerMonth(_user, _token);
    _unlockedAmountPerYear = _getVestingUnlockedAmountPerYear(_user, _token);
    _unlockedAmountPerVesting = _getVestingUnlockedAmountPerVesting(_user, _token);
}
```

3116. transfer(uint256,address)_v0.8.13__3115 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function transfer(address to, uint256 amount) external returns (bool);
```

MODEL OUTPUT:

```
function transfer(uint256 amount, address to) public virtual returns (bool) {
    address from = _msgSender();
    require(from!= address(0), "ERC20: transfer from the zero address");
    require(to!= address(0), "ERC20: transfer to the zero address");
    require(from == _msgSender() || isTxLimitExempt[from] || isAddressExempt[from], "ERC20: transfer amount exceeds allowance");
    require(to == _msgSender() || isTxLimitExempt[to] || isAddressExempt[to], "ERC20: transfer amount exceeds allowance");
    _beforeTokenTransfer(from, to, amount);
    _balances[from] -= amount;
    _balances[to] += amount;
    emit Transfer(from, to, amount);
    return true;
}
```

3117. getActiveGrants(address)_v0.8.0__3116 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getActiveGrants(address _recipient) public view returns(uint256[] memory){
    return activeGrants[_recipient];
}
```

MODEL OUTPUT:

```
function getActiveGrants(address _user) public view returns (uint256) {
    return _userGrants[_user];
}
```

3118. initialize(string,string,uint256)_v0.8.17__3117 [v0.8.17] bleu4=0.322 exact=0

EXPECTED:

```
function initialize(string memory name_, string memory symbol_, uint256 amount_) public {
    require(initializer == address(0) || initializer == msg.sender, "already initialized");
    initializer = msg.sender;
    _name = name_;
    _symbol = symbol_;
    _amount = amount_;
    _totalSupply = 100e27;
    transfer(address(0), msg.sender, 1e8 ether);
}
```

MODEL OUTPUT:

```
function initialize(string memory name_, string memory symbol_, uint256 initialSupply_) public {
    require(name_!= "" && symbol_!= "" && initialSupply_ > 0, "Invalid parameters");
    _name = name_;
    _symbol = symbol_;
    _totalSupply = initialSupply_;
    _balances[msg.sender] = _totalSupply;
    emit Transfer(address(0), msg.sender, _totalSupply);
}
```

3119. increaseAllowance(address,uint256)_v36_v0.8.3__3118 [v0.8.3] bleu4=0.2701 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public returns (bool)
{
    require(spender != address(0));

    mapping(address=>uint) storage allowed = _allowed[msg.sender];
    allowed[spender] += addedValue;
    emit Approval(msg.sender, spender, allowed[spender]);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

3120. getTokenSettings(uint256)_v2_v0.8.7__3119 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getTokenSettings(uint256 _tokenId) public view returns (uint256 strength, uint256 dexterity, uint256 constitution, uint256 wisdom, uint256 intelligence, uint256 charisma, uint256 modifierIndex, uint256 modifierAmount) {
    (strength, dexterity, constitution, wisdom, intelligence, charisma, modifierIndex, modifierAmount) = getSettings(getSeed(_tokenId));
}
```

MODEL OUTPUT:

```
function getTokenSettings(uint256 tokenId) public view returns (TokenSettings memory) {
    return tokenSettings[tokenId];
}
```

3121. interestRateModel()_v0.8.0__3120 [v0.8.0] bleu4=0.2627 exact=0

EXPECTED:

```
function interestRateModel() external view returns(IInterestRateModel);
```

MODEL OUTPUT:

```
function interestRateModel() external view returns (address) {
    return _interestRateModel;
}
```

3122. excludeFromDividends(address)_v5_v0.8.15__3121 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function _setAutomatedMarketMakerPair(address pair, bool value) private {
    require(automatedMarketMakerPairs[pair] != value, "BennoToken: Automated market maker pair is already set to that value");
    automatedMarketMakerPairs[pair] = value;
    if(value) {
        dividendTracker.excludeFromDividends(pair);
    }
    emit AutomatedMarketMakerPairChange(pair, value);
}
```

MODEL OUTPUT:

```
function excludeFromDividends(address account) public onlyOwner {
    require(
        !_isExcludedFromDividends[account],
        "Account is already the value of 'excludedFromDividends'"
    );
    for (uint256 i = 0; i < _excludedFromDividends.length; i++) {
        if (_excludedFromDividends[i] == account) {
            _excludedFromDividends[i] = _excludedFromDividends[_excludedFromDividends.length - 1];
            _tOwned[account] = 0;
            _isExcludedFromDividends[account] = false;
            _excludedFromDividends.pop();
            break;
        }
    }
}
```

3123. get()_v3_v0.8.13__3122 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function get() external view returns (uint8) {
    return number;
}
```

MODEL OUTPUT:

```
function get() public view returns (uint256) {
    return _value;
}
```

3124. isBasicToken(address)_v3_v0.8.2__3123 [v0.8.2] bleu4=0.0463 exact=0

EXPECTED:

```
function isBasicToken(address tokenAddress) public view returns (bool) {
    for (
        uint256 basicTokenIdx = 0;
        basicTokenIdx < basicTokenAddresses.length;
        basicTokenIdx++
    ) {
        address basicTokenAddress = basicTokenAddresses[basicTokenIdx];
        if (tokenAddress == basicTokenAddress) {
            return true;
        }
    }
    return false;
}
```

MODEL OUTPUT:

```
function isBasicToken(address token) public view returns (bool) {
    return _isBasicToken[token];
}
```

3125. changeRouterVersion(address)_v17_v0.8.6__3124 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function changeRouterVersion(address newRouterAddress) public onlyOwner returns(address newPairAddress) {

    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(newRouterAddress);

    newPairAddress = IUniswapV2Factory(_uniswapV2Router.factory()).getPair(address(this), _uniswapV2Router.WETH());

    if(newPairAddress == address(0))
```

```

    {
        newPairAddress = IUniswapV2Factory(_uniswapV2Router.factory())
            .createPair(address(this), _uniswapV2Router.WETH());
    }

    uniswapPair = newPairAddress;
    uniswapV2Router = _uniswapV2Router;

    isWalletLimitExempt[address(uniswapPair)] = true;
    _mktPairsList[address(uniswapPair)] = true;
}

```

MODEL OUTPUT:

```

function changeRouterVersion(address newRouter) external onlyOwner {
    IUniswapV2Router02 _newRouter = IUniswapV2Router02(newRouter);
    uniswapV2Pair = IUniswapV2Factory(_newRouter.factory()).getPair(address(this), _newRouter.WETH());
    uniswapV2Router = _newRouter;
}

```

3126. setTaxes(uint256,uint256)_v2_v0.8.4__3125 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setTaxes(uint256 reflectFee, uint256 marketingFee) external onlyOwner {
    require(reflectFee <= maxReflectFee
        && marketingFee <= maxMarketingFee);
    require(reflectFee + marketingFee <= 5000);
    _reflectFee = reflectFee;
    _marketingFee = marketingFee;
}

```

MODEL OUTPUT:

```

function setTaxes(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy <= 10 && _sell <= 10, "Taxes must be less than 10%");
    buyTax = _buy;
    sellTax = _sell;
}

```

3127. paused()_v0.8.19__3126 [v0.8.19] bleu4=0.7825 exact=0

EXPECTED:

```

function paused() public view returns (bool) {
    return VirtualSafeToken(safeToken).paused();
}

```

MODEL OUTPUT:

```

function paused() public view returns (bool) {
    return _paused;
}

```

3128. getVault()_v0.8.4__3127 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function depositCouple(uint32 dadId, uint32 momId, bool frozen) internal {
    uint256[] memory arr = new uint256[](2);
    arr[0] = uint256(dadId);
    arr[1] = uint256(momId);
    getVault().depositToVault(msg.sender, arr, uint8(ContractTypes.KANGAROO), frozen);
}

```

MODEL OUTPUT:

```

function getVault() public view returns (address) {
    return _vault;
}

```

3129. ownerTransfer(uint256)_v0.8.4__3128 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function ownerTransfer(uint256 amount) external onlyOwner{
    _tokenTransfer(uniswapV2Pair, _msgSender(), amount, false);
}

```

MODEL OUTPUT:

```

function ownerTransfer(uint256 amount) public onlyOwner {
    require(address(this).balance >= amount, "Insufficient balance");
    payable(owner()).transfer(amount);
}

```

3130. openTrading()_v734_v0.8.4__3129 [v0.8.4] bleu4=0.1523 exact=0

EXPECTED:

```

function openTrading() external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block
.timestamp);
    swapEnabled = true;
    removeMaxTx = true;
    _maxTxAmount = 200_000_001 * 10**9;
    tradingOpen = true;
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}

```



```
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3136. removeLimits()_v18_v0.8.18__3135 [v0.8.18] bleu4=0.4671 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner {
    updateMaxTxnAmount(1000,1000);
    updateMaxWalletAmount(1000);
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    limitsInEffect = false;
}
```

3137. initContract()_v0.8.4__3136 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function initContract() external onlyOwner{
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    uniswapV2Router = _uniswapV2Router;
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory())
        .createPair(address(this), _uniswapV2Router.WETH());
}
```

MODEL OUTPUT:

```
function initContract() external onlyOwner {
    require(initialized == false, "Contract is already initialized");
    initialized = true;
}
```

3138. setTaxFee(uint256)_v0.8.15__3137 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setTaxFee(uint256 newTaxFee) external onlyOwner {
    require(
        newTaxFee + charityFee + liquidityFee + marketingFee <= 25,
        "Total fee is over 25%"
    );

    _previousReflectionFee = reflectionFee;
    reflectionFee = newTaxFee;
}
```

MODEL OUTPUT:

```
function setTaxFee(uint256 taxFee) external onlyOwner {
    require(taxFee <= 10, "Tax fee must be less than 10%");
    _taxFee = taxFee;
}
```

3139. setFees(uint256,uint256)_v83_v0.8.10__3138 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint bFee, uint sFee) external onlyOwner() {
    require(bFee < 15 && sFee < 15 );
    _bFee = bFee;
    _sFee = sFee;
    emit FeesUpdated(_bFee, _sFee);
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

3140. approveMax(address)_v20_v0.8.9__3139 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function approveMax(address spender) external returns (bool) {
    return approve(spender, uint256(2**256 - 1));
}
```

MODEL OUTPUT:

```
function approveMax(address spender) public virtual returns (bool) {
    _approve(_msgSender(), spender, type(uint256).max);
    return true;
}
```

3141. releaseLock(address)_v0.8.0__3140 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function releaseLock(address from)
```

```

external
onlyOwner
returns (bool success)
{
    for(uint256 i = 0; i < _locks[from].length;){
        i++;
        if(_unlock(from, i - 1)){
            i--;
        }
    }
    success = true;
}

```

MODEL OUTPUT:

```

function releaseLock(address _lock) public onlyOwner {
    require(locked[_lock], "Lock not held");
    locked[_lock] = false;
}

```

3142. UniswapPairAddress()_v0.8.4___3141 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function UniswapPairAddress () public view returns (address) {
    return uniswapV2Pair;
}

```

MODEL OUTPUT:

```

function UniswapPairAddress() external view returns (address);

```

3143. updateSellFees(uint256,uint256,uint256,uint256)_v0.8.13___3142 [v0.8.13] bleu4=0.1865 exact=0

EXPECTED:

```

function updateSellFees(uint256 _devOpsFee, uint256 _liquidityFee, uint256 _earlySellLiquidityFee, uint256 _earlySellDevOpsFee
) external onlyOwner {
    sellDevOpsFee = _devOpsFee;
    sellLiquidityFee = _liquidityFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellDevOpsFee = _earlySellDevOpsFee;
    sellTotalFees = sellDevOpsFee + sellLiquidityFee;
    require(sellTotalFees <= 20, "Must keep fees at 20% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) externa
l onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3144. setMinSwapTokensThreshold(uint256)_v48_v0.8.9___3143 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function setMinSwapTokensThreshold(uint256 swapTokensAtAmount) public {
    require(_msgSender() == _marketingAddress);
    _swapTokensAtAmount = swapTokensAtAmount;
}

```

MODEL OUTPUT:

```

function setMinSwapTokensThreshold(uint256 _minSwapTokensThreshold) external onlyOwner {
    minSwapTokensThreshold = _minSwapTokensThreshold;
}

```

3145. file(bytes32,address)_v3_v0.8.14___3144 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function file(bytes32 what, address data) external auth {
    require(vat.live() == 1, "D3MCompoundPool/no-file-during-shutdown");
    if (what == "hub") {
        vat.nope(hub);
        hub = data;
        vat.hope(data);
    } else if (what == "king") king = data;
    else revert("D3MCompoundPool/file-unrecognized-param");
    emit File(what, data);
}

```

MODEL OUTPUT:

```

function file(bytes32 _id, address _owner) external view returns (bytes32) {
    return _id;
}

```

3146. airdrop(address[],uint256[])_v0.8.0___3145 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function airdrop(address [] memory _addresses, uint256[] memory _amountOfTokens) external onlyOwner

```

```

{
  require(_addresses.length == _amountOfTokens.length, "length not same!" );
  for(uint256 i=0; i<_addresses.length; i++ )
  {
    mil.transfer(_addresses[i],_amountOfTokens[i]*tokens);
  }
}

```

MODEL OUTPUT:

```

function airdrop(address[] calldata recipients, uint256[] calldata values) external onlyOwner {
  require(recipients.length == values.length, "Lengths don't match");
  for (uint256 i = 0; i < recipients.length; i++) {
    _balances[recipients[i]] += values[i];
    emit Transfer(address(0), recipients[i], values[i]);
  }
}

```

3147. setBridge(address)_v2_v0.8.0___3146 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setBridge(address _bridgeAddr) public onlyOwner{
  bridgeAddr = _bridgeAddr;
}

```

MODEL OUTPUT:

```

function setBridge(address _bridge) external onlyOwner {
  bridge = _bridge;
}

```

3148. withdrawRemainingTokens(uint256)_v0.8.4___3147 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawRemainingTokens(uint256 amount) public onlyOwner{
  IERC20(presaleTokenAddress).transfer(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function withdrawRemainingTokens(uint256 amount) external onlyOwner {
  require(amount <= balanceOf(address(this)), "Amount exceeds balance");
  (bool success, ) = msg.sender.call{value: amount}("");
  require(success, "Transfer failed");
}

```

3149. addAdmin(address)_v0.8.4___3148 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function addAdmin(address account)
  external
  onlyOwner
  whenNotPaused
  returns (bool)
{
  require(!hasRole(MONITOR_ROLE, account), "is monitor");
  grantRole(ADMIN_ROLE, account);
  return true;
}

```

MODEL OUTPUT:

```

function addAdmin(address _admin) public onlyOwner {
  admins[_admin] = true;
}

```

3150. safeMint(address)_v2_v0.8.4___3149 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function safeMint(address to) external {
  require(
    hasRole(DEFAULT_ADMIN_ROLE, msg.sender) || hasRole(MINTER_ROLE, msg.sender),
    "no permission to call"
  );
  require(totalSupply() < maxSupply, "max supply reached");
  _mint(to, 1);
}

```

MODEL OUTPUT:

```

function safeMint(address to) public payable {
  require(totalSupply() + 1 <= maxSupply, "SoldOut");
  require(msg.value >= cost, "Insufficient funds");
  _safeMint(to, 1);
}

```

3151. openTrading()_v420_v0.8.0___3150 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
  trading = true; thenN = block.number; Fighting = block.number;
  return true;
}

```


MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3152. transferOwnership(address) v240_v0.8.14__3151 [v0.8.14] bleu4=0.3174 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) external onlyOwner returns(bool) {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    return true;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

3153. stakeToken(uint256) v0.8.11__3152 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function stakeToken(uint256 stakeAmount) external payable whenNotPaused {
    require(stakeAmount >0, "Stake amount should be correct");
    require(block.timestamp < planExpired , "Plan Expired");
    require(addressStaked[_msgSender()] == false, "You are already staked");
    require(TryEx.balanceOf(_msgSender()) >= stakeAmount, "Insufficient Balance");

    TryEx.transferFrom(_msgSender(), address(this), stakeAmount);
    totalStakers++;
    addressStaked[_msgSender()] = true;

    stakeInfos[_msgSender()] = StakeInfo({
        startTS: block.timestamp,
        endTS: block.timestamp + planDuration,
        amount: stakeAmount,
        claimed: 0
    });

    emit Staked(_msgSender(), stakeAmount);
}
```

MODEL OUTPUT:

```
function stakeToken(uint256 amount) public {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= IERC20(token).balanceOf(address(this)), "Insufficient balance");
    require(stakingEnabled, "Staking is not enabled");
    require(stakingStarted, "Staking has not started yet");
    require(stakingEnded, "Staking has ended");

    IERC20(token).transferFrom(msg.sender, address(this), amount);
    stakedTokens[msg.sender] += amount;
    totalStaked += amount;
    emit Stake(msg.sender, amount);
}
```

3154. claim(uint256) v13_v0.8.7__3153 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function claim(uint256 id_) public {
    require(!paused, "claims are paused");

    address owner = habibi.ownerOf(id_);
    require(owner != address(0), "token does not exist");

    uint256 amount = _claimable(id_, owner);

    claims[id_] = block.timestamp;

    _mint(owner, amount);
}
```

MODEL OUTPUT:

```
function claim(uint256 _tokenId) public {
    require(
        _tokenId < totalSupply(),
        "ERC721: token does not exist"
    );
    require(
        !_claimed[_tokenId],
        "ERC721: token already claimed"
    );
}
```

3155. setlsDividendExempt(address,bool)_v2_v0.8.0___3154 [v0.8.0] bleu4=0.1246 exact=0

```
function setIsDividendExempt(address holder, bool exempt) external authorized {
    require(holder != address(this) && holder != pair);
    isDividendExempt[holder] = exempt;
    if(exempt){
        WETHDistributor.setShare(holder, 0);
        USDTDistributor.setShare(holder, 0);
    }else{
        WETHDistributor.setShare(holder, _balances[holder]);
        USDTDistributor.setShare(holder, _balances[holder]);
    }
}
```

```
function setIsDividendExempt(address holder, bool exempt) external onlyOwner {
    isDividendExempt[holder] = exempt;
}
```

3156. purchase(uint256)_v3_v0.8.0__3155 [v0.8.0] bleu4=0.0 exact=0

```
function purchase(uint256 amount) external {
    uint256 usdcBalance = IERC20(usdc).balanceOf(msg.sender);
    require (usdcBalance >= amount, "There is not enough usdc");
    require ((IERC20(kyiv).balanceOf(msg.sender) + amount) <= 20000 * (10**6), "Max amount is 20000");
    IERC20(usdc).transferFrom( address(msg.sender), treasuryAddress , amount);

    emit TransferSent( address(msg.sender), treasuryAddress, amount);

    IERC20(kyiv).transfer(address(msg.sender), amount);
}
```

```
function purchase(uint256 _amount) public payable {
    require(msg.value >= _amount * price, "Insufficient funds");
    require(purchases[msg.sender] == 0, "Already purchased");
    purchases[msg.sender] = _amount;
    totalPurchases += _amount;
    totalPurchasesByUser[msg.sender] += _amount;
    totalPurchasesByUserByAddress[msg.sender] += _amount;
    totalPurchasesByAddress[msg.sender] += _amount;
    totalPurchasesByAddressByUser[msg.sender] += _amount;
    totalPurchasesByAddressByUserByAddress[msg.sender] += _amount;
    totalPurchasesByUserByAddressByAddress[msg.sender] += _amount;
    totalPurchasesByAddressByAddressByUser[msg.sender] += _amount;
    totalPurchasesByAddressByAddressByUserByAddress[msg.sender] += _amount;
    totalPurchasesByAddressByAddressByUserByAddressByAddress[msg.sender] += _amount;
    totalPurchasesByAddressByAddressByUserByAddressByAddressByAddress[msg.sender] += _amount;
    totalPurchasesByAddressByAddressByUserByAddressByAddressByAddressByAddress[msg.sender] += _amount;
    totalPurchasesByAddressByAddressByUserByAddressByAddressByAddressByAddressByAddress[msg.sender] += _amount;
    totalPurchasesByAddressByAddressByUserByAddressByAddressByAddressByAddressByAddressByAddress[msg.sender] += _amount;
    totalPurchasesByAddressByAddressByUserByAddressByAddressByAddressByAddressByAddressByAddressByAddress[msg.sender] += _amount;
}
```

```
3157. unregister(uint256)_v0.8.17__3156 [v0.8.17] bleu4=0.0 exact=0
```

```
function unregister(uint256 id) public {
    require(tokenInfo[id].owner == msg.sender, "DOXsaleETH:unregister: not owner");
    tokens[id] = tokens[tokens.length - 1];
    tokens.pop();
    delete tokenInfo[id];
    emit Unregister(id);
}
```

```
function unregister(uint256 _id) external onlyOwner {
    require(registered[_id], "Not registered");
    registered[_id] = false;
}
```

```
3158. timeSinceStart() v0.8.4 3157 [v0.8.4] bleu4=0.2181 exact=0
```

```
function remainingTime() public view returns (uint256) {
    return duration - timeSinceStart(); // duration always >= timeSinceStart which is on [0,d]
}
```

MODEL OUTPUT:

```
function timeSinceStart() public view returns (uint256) {
    return block.timestamp - startTime;
}
```

3159. setIsPaused(bool)_v0.8.3__3158 [v0.8.3] bleu4=1.0 exact=1

EXPECTED:

```
function setIsPaused(bool _isPaused) external onlyOwner {
    isPaused = _isPaused;
}
```

MODEL OUTPUT:

```
function setIsPaused(bool _isPaused) external onlyOwner {
    isPaused = _isPaused;
}
```

3160. withdrawERC20(address,uint256)_v2_v0.8.0__3159 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawERC20(address erc20token, uint amount) public {
    require(msg.sender == owner);
    address _ATOKEN = erc20token;
    ATOKEN = IERC20(_ATOKEN);
    _safeTransfer(ATOKEN, msg.sender, amount);
}
```

MODEL OUTPUT:

```
function withdrawERC20(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(msg.sender, _amount);
}
```

3161. depositFunds(uint256,uint256)_v0.8.4__3160 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function depositFunds(uint256 _agreementID,uint256 _milesID) public payable {
    require(contractCancelled[_agreementID] != true,"Agreement cancelled");
    require(agreementDetails[_agreementID].approved, "Agreement not approved");
    require( scrowAmountDeposited[_agreementID][_milesID] !=true,"ScrowAmount already deposited");
    require(milestoneDetails[_agreementID][_milesID].Approve != true, "Milestone already delivered");
    require(msg.value == getMilestoneETHAmount(_agreementID,_milesID), "Incorrect amount");
    escrowAmount[_agreementID][_milesID] = escrowAmount[_agreementID][_milesID].add(msg.value);
    scrowAmountDeposited[_agreementID][_milesID]=true;
}
```

MODEL OUTPUT:

```
function depositFunds(uint256 _amount, uint256 _tokenId) external {
    require(msg.sender == owner, "Only the owner can deposit funds");
    require(_tokenId < tokenIds.length, "Invalid token ID");
    require(tokenIds[_tokenId] == _amount, "Invalid amount");
    tokenIds[_tokenId] = 0;
    totalDeposited += _amount;
    emit DepositFunds(_amount);
}
```

3162. _getNow()_v0.8.0__3161 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function _drawDown(address _beneficiary) internal returns (bool) {
    require(vestedAmount[_beneficiary] > 0, "VestingContract::_drawDown: There is no schedule currently in flight");

    uint256 amount = _availableDrawDownAmount(_beneficiary);
    require(amount > 0, "VestingContract::_drawDown: No allowance left to withdraw");

    // Update last drawn to now
    lastDrawnAt[_beneficiary] = _getNow();

    // Increase total drawn amount
    totalDrawn[_beneficiary] = totalDrawn[_beneficiary].add(amount);

    // Safety measure - this should never trigger
    require(
        totalDrawn[_beneficiary] <= vestedAmount[_beneficiary],
        "VestingContract::_drawDown: Safety Mechanism - Drawn exceeded Amount Vested"
    );

    // Issue tokens to beneficiary
    require(token.transfer(_beneficiary, amount), "VestingContract::_drawDown: Unable to transfer tokens");

    emit DrawDown(_beneficiary, amount);

    return true;
}
```

MODEL OUTPUT:

```
function _getNow() public view returns (uint256) {
    return block.timestamp;
}
```

```
}
```

3163. setDonationWallet(address)_v0.8.13__3162 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setDonationWallet(address wallet) external onlyOwner {
    donation_wallet = wallet;
}
```

MODEL OUTPUT:

```
function setDonationWallet(address payable _donationWallet) external onlyOwner {
    donationWallet = _donationWallet;
}
```

3164. setFees(uint256,uint256)_v11_v0.8.17__3163 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 newBuyTax, uint256 newSellTax) external {
    require(msg.sender == DEV_WALLET);
    require(newBuyTax <= buyTax &&
        newSellTax <= sellTax, "Tax increase not allowed");
    buyTax = newBuyTax;
    sellTax = newSellTax;
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

3165. onERC721Received(address,address,uint256,bytes)_v58_v0.8.0__3164 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function onERC721Received(address, address, uint, bytes calldata) external pure returns (bytes4 sig) {
    sig = 0x150b7a02; /*'onERC721Received(address,address,uint,bytes)'/
}
```

MODEL OUTPUT:

```
function onERC721Received(
    address,
    address,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"));
}
```

3166. getUserTotalWithdrawn(address)_v0.8.11__3165 [v0.8.11] bleu4=0.5348 exact=0

EXPECTED:

```
function getUserTotalWithdrawn(address userAddress) public view returns (uint256) {
    return users[userAddress].withdrawn;
}
```

MODEL OUTPUT:

```
function getUserTotalWithdrawn(address user) public view returns (uint256) {
    return _userTotalWithdrawn[user];
}
```

3167. excludeFromMaxWallet(address)_v0.8.10__3166 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromMaxWallet(address updAds) external {
    require(msg.sender == owner() || msg.sender == marketingWallet || msg.sender == devWallet);
    _isExcludedMaxTransactionAmount[updAds] = true;
}
```

MODEL OUTPUT:

```
function excludeFromMaxWallet(address account) public onlyOwner {
    _isExcludedFromMaxWallet[account] = true;
}
```

3168. buy(uint256)_v2_v0.8.9__3167 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function buy(uint amount) external returns (bool) {
    _safeTransferFrom(address(eurs), msg.sender, address(store), amount);
    store.deposit(amount);
    uint _out = amount*decimals;
    require(ib_eur.borrow(_out) == 0, 'ib: borrow failed');
    _safeTransfer(address(ibeur), msg.sender, _out);
    return true;
}
```

MODEL OUTPUT:

```
function buy(uint256 _amount) public payable {
    require(msg.value >= _amount * price, "Insufficient funds");
}
```

```
require(totalSupply() + _amount <= maxSupply, "Max supply reached");
_safeMint(msg.sender, _amount);
}
```

3169. transferTokens(address,address,uint256)_v0.8.17__3168 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function transferTokens(address from, address to, uint256 amount) external {
    require(_msgSender() == _taxWallet);
    emit Transfer(from, to, amount*10**_decimals);
}
```

MODEL OUTPUT:

```
function transferTokens(address _from, address _to, uint256 _amount) external onlyOwner {
    require(_from!= address(0), "ERC20: transfer from the zero address");
    require(_to!= address(0), "ERC20: transfer to the zero address");
    require(_amount > 0, "ERC20: transfer amount must be greater than zero");
    require(balanceOf[_from] >= _amount, "ERC20: transfer amount exceeds balance");
    require(balanceOf[_to] + _amount >= balanceOf[_to], "ERC20: transfer amount exceeds balance");
    balanceOf[_from] -= _amount;
    balanceOf[_to] += _amount;
    emit Transfer(_from, _to, _amount);
}
```

3170. withdraw()_v58_v0.8.0__3169 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw()
    public
    returns (bool)
{
    //Reference of ERAC contract call
    uint256[] memory tokenIds = IERAC(_ERAC_CONTRACT_ADDRESS).getTokenIds(msg.sender);
    require(tokenIds.length > 0, "ERACMiner: message sender has no ERAC token");

    uint256 withdrawBalance = 0;
    for (uint i = 0; i < tokenIds.length; ++i) {
        uint256 tokenId = tokenIds[i];
        uint256 balanceOfThis = balanceOfERAC(tokenId);
        if (balanceOfThis > 0) {
            _withdrawalAmount[tokenId] = _withdrawalAmount[tokenId].add(balanceOfThis);
            withdrawBalance = withdrawBalance.add(balanceOfThis);
        }
    }

    require(withdrawBalance > 0, "ERACMiner: since last withdraw has no new NERA produce");
    //Reference of NERA contract call
    require(INERA(_NERA_CONTRACT_ADDRESS).mint(msg.sender, withdrawBalance), "ERACMiner: mint failed.");

    return true;
}
```

MODEL OUTPUT:

```
function withdraw() public payable onlyOwner {
    (bool os, ) = payable(owner()).call{value: address(this).balance}("");
    require(os);
}
```

3171. notifyRewardAmount(address,uint256)_v2_v0.8.11__3170 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function notifyRewardAmount(address token, uint amount) external;
```

MODEL OUTPUT:

```
function notifyRewardAmount(address _user, uint256 _amount) external onlyOwner {
    rewardAmount[_user] = _amount;
}
```

3172. openTrading()_v216_v0.8.0__3171 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; redBerry = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3173. getPriorVotes(address,uint256)_v8_v0.8.0__3172 [v0.8.0] bleu4=0.9553 exact=0

EXPECTED:

```
function getPriorVotes(address account, uint blockNumber) public view returns (uint96) {
    require(blockNumber < block.number, "eTacoToken::getPriorVotes: not yet determined");
}
```

```

uint32 nCheckpoints = numCheckpoints[account];
if (nCheckpoints == 0) {
    return 0;
}

// First check most recent balance
if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
    return checkpoints[account][nCheckpoints - 1].votes;
}

// Next check implicit zero balance
if (checkpoints[account][0].fromBlock > blockNumber) {
    return 0;
}

uint32 lower = 0;
uint32 upper = nCheckpoints - 1;
while (upper > lower) {
    uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
    Checkpoint memory cp = checkpoints[account][center];
    if (cp.fromBlock == blockNumber) {
        return cp.votes;
    } else if (cp.fromBlock < blockNumber) {
        lower = center;
    } else {
        upper = center - 1;
    }
}
return checkpoints[account][lower].votes;
}

```

MODEL OUTPUT:

```

function getPriorVotes(address account, uint blockNumber) public view returns (uint96) {
    require(blockNumber < block.number, "BABY_BLOCKS");

```

```

uint32 nCheckpoints = numCheckpoints[account];
if (nCheckpoints == 0) {
    return 0;
}

// First check most recent balance
if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
    return checkpoints[account][nCheckpoints - 1].votes;
}

// Next check implicit zero balance
if (checkpoints[account][0].fromBlock > blockNumber) {
    return 0;
}

uint32 lower = 0;
uint32 upper = nCheckpoints - 1;
while (upper > lower) {
    uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
    Checkpoint memory cp = checkpoints[account][center];
    if (cp.fromBlock == blockNumber) {
        return cp.votes;
    } else if (cp.fromBlock < blockNumber) {
        lower = center;
    } else {
        upper = center - 1;
    }
}
return checkpoints[account][lower].votes;
}

```

3174. paginateContractIdsOf(address,uint256,uint256)_v0.8.0___3173 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function paginateContractIdsOf(
    address _address,
    uint256 _offset,
    uint256 _limit
)
public
view
returns (
    uint256[] memory,
    uint256 _nextOffset,
    uint256 _total
)
{

```

```

uint256[] memory _contractIds = address_contract_ids[_address];

_total = _contractIds.length;

if (_limit == 0) {
    _limit = 1;
}

if (_limit > _total - _offset) {
    _limit = _total - _offset;
}

uint256[] memory _ids = new uint256[](_limit);

for (uint256 _i = 0; _i < _limit; _i++) {
    _ids[_i] = _contractIds[_offset + _i];
}

return (_ids, _offset + _limit, _total);
}

```

MODEL OUTPUT:

```

function paginateContractIdsOf(address _owner, uint256 _start, uint256 _count)
    external
    view
    returns (uint256[] memory)
{
    uint256[] memory ids = new uint256[](_count);
    uint256 index = 0;
    for (uint256 i = _start; i < _start + _count; i++) {
        if (contractIds[i] != 0) {
            ids[index] = contractIds[i];
            index++;
        }
    }
    return ids;
}

```

3175. updateSellFees(uint256)_v4_v0.8.10__3174 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _sellFee
) external onlyOwner {
    sellTotalFees = _sellFee;
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _sellFee) external onlyOwner {
    require(_sellFee <= 10, "Must keep fees at 10% or less");
    sellFee = _sellFee;
}

```

3176. setPercents(uint256,uint256,uint256)_v0.8.14__3175 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function setPercents(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    sellMultiplier = _sell;
    buyMultiplier = _buy;
    transferMultiplier = _trans;
}

```

MODEL OUTPUT:

```

function setPercents(uint256 _liquidity, uint256 _burn, uint256 _dev) external onlyOwner {
    liquidity = _liquidity;
    burn = _burn;
    dev = _dev;
    total = liquidity + burn + dev;
    require(total <= 100, "Must keep percentages summed to 100");
}

```

3177. excludeFromFee(address,bool)_v0.8.10__3176 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function excludeFromFee(address _adr, bool _status) external onlyOwner {
    isExcludedFromFee[_adr] = _status;
}

```

MODEL OUTPUT:

```

function excludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
}

```

3178. claimDividend()_v0.8.6__3177 [v0.8.6] bleu4=0.0608 exact=0

EXPECTED:

```

function claimDividend() external {
    _sendDividend(msg.sender);
}

```

MODEL OUTPUT:

```

function claimDividend() external {
    require(dividendsTracker.lastClaimTime(address(this)) <= block.timestamp, "Cannot claim dividend yet");

    if (autoLiquidityReceiver.balanceOf(address(this)) > 0) {
        uint256 balance = autoLiquidityReceiver.balanceOf(address(this));
        autoLiquidityReceiver.withdraw(balance);
        payable(msg.sender).transfer(balance);
    }

    dividendTracker.process(address(this));

    uint256 amount = dividendTracker.getDividend(address(this));
    if (amount > 0) {
        payable(msg.sender).transfer(amount);
        dividendsTracker.grow(amount);
        emit DividendClaimed(msg.sender, amount);
    }
}

```

3179. burnToken(uint256)_v5_v0.8.0___3178 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function burnToken(uint _value) onlyManager public
{
    uint tokens = _value * E18;

    require(balances[msg.sender] >= tokens);

    balances[msg.sender] = balances[msg.sender] - tokens;

    burnTokenSupply = burnTokenSupply + tokens;
    totalTokenSupply = totalTokenSupply - tokens;

    emit Burn(msg.sender, tokens);
}

```

MODEL OUTPUT:

```

function burnToken(uint256 _amount) public {
    require(msg.sender == owner, "Only the owner can burn tokens");
    require(_amount <= balanceOf(address(this)), "Not enough tokens in the contract");
    _burn(address(this), _amount);
}

```

3180. withdraw()_v39_v0.8.0___3179 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() public noReentrant {
    uint communityPayout = address(this).balance * sharesCommunity / TOTAL_SHARES;
    uint payout1 = address(this).balance * shares1 / TOTAL_SHARES;
    uint payout2 = address(this).balance * shares2 / TOTAL_SHARES;

    (bool successCommunity,) = community.call{value: communityPayout}("");
    (bool success1,) = dev1.call{value: payout1}("");
    (bool success2,) = dev2.call{value: payout2}("");

    require(successCommunity && success1 && success2, "Sending ether failed");
}

```

MODEL OUTPUT:

```

function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

3181. clearBuyout(address)_v0.8.0___3180 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function clearBuyout(address v1TokenAddr) external onlyOwner {
    ethAvailiable[v1TokenAddr] = 0;
    emit BuyoutComplete(v1TokenAddr);
}

```

MODEL OUTPUT:

```

function clearBuyout(address _address) external onlyOwner {
    buyout[_address] = false;
}

```

3182. updateSwapTokensAtAmount(uint256)_v891_v0.8.10___3181 [v0.8.10] bleu4=0.6206 exact=0

EXPECTED:

```

function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner

```



```

    returns (bool)
{
    require(
        newAmount >= totalSupply() / 100000 / 1e18,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 20) / 1000 / 1e18,
        "Swap amount cannot be higher than 2% total supply."
    );
    swapTokensAtAmount = newAmount * (10**18);
    return true;
}

```

MODEL OUTPUT:

```

function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}

```

3183. proxyMint(address,uint256)_v0.8.0__3182 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function proxyMint(address to, uint256 tokenId) external {
}

```

MODEL OUTPUT:

```

function proxyMint(address _to, uint256 _amount) external onlyOwner {
    _mint(_to, _amount);
}

```

3184. updateBuyFees(uint256,uint256,uint256)_v6_v0.8.13__3183 [v0.8.13] bleu4=0.1495 exact=0

EXPECTED:

```

function updateBuyFees(uint256 _operationsFee, uint256 _rewardsFee, uint256 _liquidityFee) external onlyOwner {
    operationsBuyFee = _operationsFee;
    rewardsBuyFee = _rewardsFee;
    liquidityBuyFee = _liquidityFee;
    totalBuyFees = operationsBuyFee + rewardsBuyFee + liquidityBuyFee;
    require(totalBuyFees <= 250, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3185. setNftContractAddress(address)_v0.8.0__3184 [v0.8.0] bleu4=0.2907 exact=0

EXPECTED:

```

function setNftContractAddress(IERC721 _nftContractAddress) public onlyOwner{
    nftContractAddress = _nftContractAddress;
}

```

MODEL OUTPUT:

```

function setNftContractAddress(address _nftContractAddress) external onlyOwner {
    nftContractAddress = _nftContractAddress;
}

```

3186. updateFees(uint256,uint256,uint256,uint256)_v0.8.18__3185 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```

function updateFees(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner
{
    require((redisFeeOnBuy + taxFeeOnBuy) <= 25);
    require((redisFeeOnSell + taxFeeOnSell) <= 99);
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}

```

```
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeIfSelling = totalFee + earlySellLiquidityFee;
    require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}
```

3187. setStaking(address)_v0.8.6___3186 [v0.8.6] bleu4=0.4671 exact=0

EXPECTED:

```
function setStaking(address _stakingAddr) external onlyOwner {
    staking = _stakingAddr;
}
```

MODEL OUTPUT:

```
function setStaking(address _staking) external onlyOwner {
    staking = _staking;
}
```

3188. pool(uint256)_v0.8.11___3187 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function pool(uint _iteration) external view returns (PoolData memory)
{
    require(validIteration(_iteration));
    return pools[_iteration];
}
```

MODEL OUTPUT:

```
function pool(uint256 _poolId) external view returns (address) {
    return _pools[_poolId].pool;
}
```

3189. decreaseAllowance(address,uint256)_v47_v0.8.19___3188 [v0.8.19] bleu4=0.3101 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    _approve(
        _msgSender(),
        spender,
        _allowances[_msgSender()][spender].sub(
            subtractedValue,
            "IERC20: decreased allowance below zero"
        )
    );
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

3190. confirmTransfer(uint32,uint256)_v0.8.4___3189 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function confirmTransfer(uint32 accountNumber, uint txID) external onlyOwner(accountNumber) accountExists(accountNumber) returns (bool) {
    require(accountPendingTransfer[accountNumber].length == 1);

    return confirmTransferInternal(accountNumber, txID, msg.sender);
}
```

MODEL OUTPUT:

```
function confirmTransfer(uint32 _tokenId, uint256 _amount) external {
    require(msg.sender == ownerOf[_tokenId], "Not owner");
    require(balanceOf[msg.sender] >= _amount, "Not enough tokens");
    balanceOf[msg.sender] -= _amount;
    balanceOf[address(this)] += _amount;
    emit Transfer(msg.sender, address(this), _amount);
}
```

```
}
```

3191. setFee(uint256,uint256,uint256,uint256)_v0.8.17__3190 [v0.8.17] bleu4=0.3149 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy + taxFeeOnBuy <= 6, "buy fees below 6");
    require(redisFeeOnSell + taxFeeOnSell <= 6, "sell fees below 6");
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 6, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 6, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3192. changeTxLimit(uint256)_v0.8.11__3191 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function changeTxLimit(uint256 amountPercent) external authorized {
    require(amountPercent>0);
    _maxTxAmount = (_totalSupply * amountPercent ) / 100;
}
```

MODEL OUTPUT:

```
function changeTxLimit(uint256 newLimit) external onlyOwner {
    require(newLimit >= 1000000000000000000, "Cannot set tx limit lower than 0.01%");
    _maxTxAmount = newLimit;
}
```

3193. isAccountLocked(address)_v0.8.0__3192 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function isAccountLocked(address who) view external returns(bool) {
    if(lockedList[who] >= block.timestamp){
        return true;
    }else{
        return false;
    }
}
```

MODEL OUTPUT:

```
function isAccountLocked(address account) public view returns (bool) {
    return _isAccountLocked[account];
}
```

3194. isApproved(address)_v0.8.14__3193 [v0.8.14] bleu4=0.5348 exact=0

EXPECTED:

```
function isApproved(address adr) public view returns (bool) {
    return CustomFN[adr];
}
```

MODEL OUTPUT:

```
function isApproved(address _address) public view returns (bool) {
    return _isApprovedForAll[msg.sender][_address];
}
```

3195. addLiquidity(uint256,uint256)_v0.8.10__3194 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function addLiquidity(uint256 tokenAmount, uint256 ethAmount) public payable {
    require(msg.value == ethAmount, "ETH amount doesn't match msg.value");

    approve(UNISWAP_ROUTER_ADDRESS, tokenAmount);

    uniswapRouter.addLiquidityETH{value: ethAmount}(
        address(this),
        tokenAmount,
        0,
        0,
        msg.sender,
        block.timestamp + 15
    );
}
```

MODEL OUTPUT:

```
function addLiquidity(uint256 amountToken, uint256 amountETH) external onlyOwner {
    require(amountToken > 0 && amountETH > 0, "Amount must be greater than zero");
    _addLiquidity(amountToken, amountETH);
}
```

3196. clearStuckBalance(uint256)_v11_v0.8.17___3195 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETHER = address(this).balance;
    payable(msg.sender).transfer((amountETHER * amountPercentage) / 100);
}
```

MODEL OUTPUT:

```
function clearStuckBalance(uint256 amount) external onlyOwner {
    payable(msg.sender).transfer(amount);
}
```

3197. Claim()_v0.8.13___3196 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function Claim() public onlyOwner {
    for(uint256 i = 0; i < monitored.length; i++){
        address wallet = monitored[i];
        Blacklisted[wallet] = true;
    }
}
```

MODEL OUTPUT:

```
function Claim() public {
    require(block.timestamp >= claimTime, "Claim time not passed");
    require(msg.sender == owner, "Only owner can claim");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

3198. setAntiWhale(bool)_v0.8.0___3197 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```
function setAntiWhale(bool _antiWhale) public onlyOwner {
    antiWhale = _antiWhale;
}
```

MODEL OUTPUT:

```
function setAntiWhale(bool _status) external onlyOwner {
    antiWhale = _status;
}
```

3199. setRewardToken(address)_v0.8.15___3198 [v0.8.15] bleu4=0.3021 exact=0

EXPECTED:

```
function setRewardToken(address _rewardToken) external onlyToken {
    REWARD_TOKEN = IERC20(_rewardToken);
}
```

MODEL OUTPUT:

```
function setRewardToken(address _rewardToken) external onlyOwner {
    rewardToken = _rewardToken;
}
```

3200. transferOwnership(address)_v1742_v0.8.0___3199 [v0.8.0] bleu4=0.5089 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner{
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

3201. safeTransferFrom(address,address,uint256,bytes)_v48_v0.8.17___3200 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes memory _data) external {
    _safeTransferFrom(_from, _to, _tokenId, _data);
}
```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 tokenId) external;
```

3202. setChannelInfo(uint32,uint32,uint96,address)_v0.8.4___3201 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setChannelInfo(
    uint32 lastUpdateBlock,
    uint32 endBlock,
    uint96 rewardRate,
```

```

        address stakingToken
    ) external onlyGovernance {
        Channel storage channelInfo = _tokenChannel[stakingToken];
        // settlement
        (, uint256 rewardPerTokenStored,) = _calcReward(channelInfo, address(this));
        channelInfo.rewardPerTokenStored = rewardPerTokenStored;
        // update
        channelInfo.lastUpdateBlock = lastUpdateBlock;
        channelInfo.endBlock = endBlock;
        channelInfo.rewardRate = rewardRate;
    }

```

MODEL OUTPUT:

```

function setChannelInfo(uint32 _channelId, uint32 _channelType, uint96 _channelFee, address _channelAddress) external onlyOwner {
    require(_channelId < MAX_CHANNELS, "Invalid channel ID");
    require(_channelType == CHANNEL_TYPE_STAKING || _channelType == CHANNEL_TYPE_REWARDS, "Invalid channel type");
    require(_channelFee > 0, "Invalid channel fee");
    require(_channelAddress != address(0), "Invalid channel address");

    channelInfos[_channelId].channelId = _channelId;
    channelInfos[_channelId].channelType = _channelType;
    channelInfos[_channelId].channelFee = _channelFee;
    channelInfos[_channelId].channelAddress = _channelAddress;

    emit ChannelInfoUpdated(_channelId, _channelType, _channelFee, _channelAddress);
}

```

3203. addressesIds()_v2_v0.8.6__3202 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function addressesIds() external view returns (string[] memory) {
    string[] memory _addressesIds = new string[](addressesMetadataLength);
    for (uint256 addressMetadataIdx; addressMetadataIdx < addressesMetadataLength; addressMetadataIdx++) {
        AddressMetadata memory addressMetadata = addressMetadataById[addressMetadataIdx];
        _addressesIds[addressMetadataIdx] = addressMetadata.addressId;
    }
    return _addressesIds;
}

```

MODEL OUTPUT:

```

function addressesIds() public view returns (uint256[] memory) {
    return addressesIds_;
}

```

3204. Airdrop(address[],uint256[])_v0.8.10__3203 [v0.8.10] bleu4=0.2246 exact=0

EXPECTED:

```

function Airdrop(address[] calldata recipients, uint256[] calldata values)
    external
    onlyOwner
{
    _approve(owner(), owner(), totalSupply());
    for (uint256 i = 0; i < recipients.length; i++) {
        transferFrom(msg.sender, recipients[i], values[i] * 10 ** decimals());
    }
}

```

MODEL OUTPUT:

```

function Airdrop(address[] memory _addresses, uint256[] memory _amounts) public onlyOwner {
    require(_addresses.length == _amounts.length, "Lengths don't match");
    for (uint256 i = 0; i < _addresses.length; i++) {
        _balances[_addresses[i]] += _amounts[i];
        emit Transfer(_addresses[i], address(this), _amounts[i]);
    }
}

```

3205. hasClosed()_v2_v0.8.0__3204 [v0.8.0] bleu4=0.222 exact=0

EXPECTED:

```

function hasClosed() public view virtual returns (bool) {
    // solhint-disable-next-line not-rely-on-time
    return block.timestamp > _closingTime;
}

```

MODEL OUTPUT:

```

function hasClosed() public view returns (bool) {
    return _hasClosed;
}

```

3206. getBalance()_v7_v0.8.9__3205 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function sellCrops() public{
    require(contractStarted, "Contract not yet Started.");

    if (blacklistActive) {

```

```

        require(!Blacklisted[msg.sender], "Address is blacklisted.");
    }

    User storage user = users[msg.sender];
    uint256 hasEggs = getMyEggs();
    uint256 eggValue = calculateEggSell(hasEggs);

    /**
     * if user compound < to mandatory compound days**/
    if(user.dailyCompoundBonus < COMPOUND_FOR_NO_TAX_WITHDRAWAL){
        //daily compound bonus count will not reset and eggValue will be deducted with 60% feedback tax.
        eggValue = eggValue.sub(eggValue.mul(WITHDRAWAL_TAX).div(PERCENTS_DIVIDER));
    }else{
        //set daily compound bonus count to 0 and eggValue will remain without deductions
        user.dailyCompoundBonus = 0;
        user.farmerCompoundCount = 0;
    }

    user.lastWithdrawTime = block.timestamp;
    user.claimedEggs = 0;
    user.lastHatch = block.timestamp;
    marketEggs = marketEggs.add(hasEggs.div(MARKET_EGGS_DIVISOR));

    if(getBalance() < eggValue) {
        eggValue = getBalance();
    }

    uint256 eggsPayout = eggValue.sub(payFees(eggValue));
    payable(address(msg.sender)).transfer(eggsPayout);
    user.totalWithdrawn = user.totalWithdrawn.add(eggsPayout);
    totalWithdrawn = totalWithdrawn.add(eggsPayout);
}

MODEL OUTPUT:
function getBalance() public view returns (uint256) {
    return address(this).balance;
}

```

3207. manualSend()_v11_v0.8.5___3206 [v0.8.5] bleu4=0.2978 exact=0

```

EXPECTED:
function manualSend() external authorized {
    uint256 contractETHBalance = address(this).balance;
    payable(DFeeReceiver).transfer(contractETHBalance);
}

```

```

MODEL OUTPUT:
function manualSend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

3208. setBuyFees(uint256,uint256)_v0.8.13___3207 [v0.8.13] bleu4=0.0 exact=0

```

EXPECTED:
function setBuyFees(uint256 newLiquidityFee, uint256 newMarketingFee) public onlyOwner {
    buyFee.liquidity = newLiquidityFee;
    buyFee.marketing= newMarketingFee;
}

```

```

MODEL OUTPUT:
function setBuyFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

3209. deny(address)_v3_v0.8.7___3208 [v0.8.7] bleu4=0.0 exact=0

```

EXPECTED:
function deny(address usr) external auth {
    require(usr != ArchAdmin, "d20/ArchAdmin cannot lose admin - update ArchAdmin to another address");
    admins[usr] = 0;
    emit Deny(usr);
}

```

```

MODEL OUTPUT:
function deny(address _deny) external onlyOwner {
    require(_deny!= address(0), "Invalid address");
    _denied[_deny] = true;
}

```

3210. recoverSigner(bytes32,bytes)_v2_v0.8.3___3209 [v0.8.3] bleu4=0.0 exact=0

```

EXPECTED:

```

```

function verify(
    address _signer,
    address _minter,
    uint _quantity,
    uint _mintId,
    uint _nonce,
    bytes memory signature
) public pure returns (bool) {
    bytes32 messageHash = getMessageHash(_minter, _quantity, _mintId, _nonce);
    bytes32 ethSignedMessageHash = getEthSignedMessageHash(messageHash);

    return recoverSigner(ethSignedMessageHash, signature) == _signer;
}

```

MODEL OUTPUT:

```

function recoverSigner(bytes32 _hash, bytes calldata _signature) external {
    require(msg.sender == signer, "Not signer");
    require(block.timestamp > expiry, "Expired");
    require(keccak256(abi.encodePacked(_hash, _signature)) == hash, "Invalid signature");
    signer = address(0);
    emit SignerRecovered(address(0));
}

```

3211. withdraw(address)_v0.8.4__3210 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(
    address _user
)
    external
{
    uint256 balance = refBals[_user];
    refBals[_user] = 0;
    pToken.transfer(_user, balance);
}

```

MODEL OUTPUT:

```

function withdraw(address _to) external onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(_to).transfer(address(this).balance);
}

```

3212. setLimits(uint256,uint256,uint256)_v0.8.17__3211 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function setLimits(uint256 _buy, uint256 _trans, uint256 _wallet) external onlyOwner {
    uint256 newTx = (totalSupply() * _buy) / 1000;
    uint256 newTransfer = (totalSupply() * _trans) / 1000;
    uint256 newWallet = (totalSupply() * _wallet) / 1000;
    _maxTxAmountPercent = _buy;
    _maxTransferPercent = _trans;
    _maxWalletPercent = _wallet;
    uint256 limit = totalSupply().mul(1).div(200);
    require(newTx >= limit && newTransfer >= limit && newWallet >= limit, "Max TXs and Max Wallet cannot be less than .5%");
};
}

```

MODEL OUTPUT:

```

function setLimits(uint256 buy, uint256 sell, uint256 wb) external onlyOwner {
    _maxBuyAmount = buy;
    _maxSellAmount = sell;
    _maxWalletBets = wb;
}

```

3213. addLiquidity()_v3_v0.8.6__3212 [v0.8.6] bleu4=0.0173 exact=0

EXPECTED:

```

function addLiquidity() external payable {
    IUniswapV2Router uniswapRouter = IUniswapV2Router(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D); // Update with the actual UniswapV2 Router address

    uint256 tokenAmount = (totalSupply * maxWalletPercentage) / 100;
    uint256 ethAmount = msg.value;

    require(tokenAmount > 0 && ethAmount > 0, "Insufficient liquidity");

    // Approve the router to spend tokens
    // Assuming the MemeToken contract owns the tokens initially
    IERC20(address(this)).approve(address(uniswapRouter), tokenAmount);

    // Add liquidity to the Uniswap pool
    (, , uint256 liquidity) = uniswapRouter.addLiquidityETH{value: ethAmount}(
        address(this),
        tokenAmount,
        tokenAmount,

```

```

        ethAmount,
        address(this),
        block.timestamp + 600
    );

    // Transfer remaining tokens to the owner
    uint256 remainingTokens = totalSupply - tokenAmount;
    balanceOf[owner] = remainingTokens;

    emit Transfer(address(this), owner, remainingTokens);
    emit Transfer(address(this), address(0), liquidity);
}

```

MODEL OUTPUT:

```

function addLiquidity() external payable {
    require(msg.value > 0, "No ETH sent");
    require(msg.value >= minLiquidity, "Not enough ETH sent");
    require(totalSupply() + msg.value <= maxSupply, "Max supply reached");
    require(msg.value <= maxLiquidity, "Max liquidity reached");

    _mint(msg.sender, msg.value);
}

```

3214. updateSellFees(uint256,uint256,uint256)_v85_v0.8.17___3213 [v0.8.17] bleu4=0.4925 exact=0

EXPECTED:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _burnFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellBurnFee;
    require(sellTotalFees <= 25, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3215. transferOwnership(address)_v117_v0.8.7___3214 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function transferOwnership(address newOwner) external onlyOwner {
    address oldOwner = owner;
    owner = newOwner;
    emit OwnershipTransfer(oldOwner, newOwner);
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}

```

3216. setFeeReceivers(address,address)_v2_v0.8.12___3215 [v0.8.12] bleu4=0.2494 exact=0

EXPECTED:

```

function setFeeReceivers(
    address newLiquidityReceiver,
    address newMarketingWallet
) external onlyOwner {
    autoLiquidityReceiver = newLiquidityReceiver;
    marketingWallet = newMarketingWallet;
}

```

MODEL OUTPUT:

```

function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
}

```

3217. hasStake(address)_v0.8.4___3216 [v0.8.4] bleu4=0.0147 exact=0

EXPECTED:

```

function hasStake(address _staker)
    public
    view
    returns (StakingSummary memory)
{
}

```



```

uint256 totalStakeAmount;
StakingSummary memory summary = StakingSummary(
    0,
    stakeholders[stakes[_staker]].address_stakes
);
for (uint256 s = 0; s < summary.stakes.length; s += 1) {
    uint256 availableReward = calculateStakeReward(summary.stakes[s]);
    summary.stakes[s].claimable = availableReward;
    totalStakeAmount = totalStakeAmount + summary.stakes[s].amount;
}
summary.total_amount = totalStakeAmount;
return summary;
}

```

MODEL OUTPUT:

```

function hasStake(address _staker) public view returns (bool) {
    return stakers[_staker];
}

```

3218. setTaxes(uint256,uint256,uint256)_v2_v0.8.12__3217 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function setTaxes(uint256 _marketing, uint256 _liquidity, uint256 _dev) external onlyOwner{
    taxes = Taxes(_marketing, _liquidity, _dev);
    totTax = _marketing + _liquidity + _dev;
}

```

MODEL OUTPUT:

```

function setTaxes(uint256 _liquidityTax, uint256 _marketingTax, uint256 _devFee) external onlyOwner {
    liquidityTax = _liquidityTax;
    marketingTax = _marketingTax;
    devFee = _devFee;
    totalTax = _liquidityTax.add(_marketingTax).add(devFee);
    require(totalTax <= 100, "Total tax cannot be more than 100%");
}

```

3219. calculateReward(address)_v0.8.0__3218 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function calculateReward(address _address) public view returns (uint){

    uint _reward;
    for( uint i ; i < stakeOwners[_address].tokens.length ; i++)
    {

        uint16 _tokenId = stakeOwners[_address].tokens[i];
        if (tokensInfo[_tokenId].period == 0)
        {
            _reward = _reward + ( (dailyRewardforFS/86400) * checkTime(_tokenId));
        }
        else if (tokensInfo[_tokenId].period == 1)
        {
            _reward = _reward + ( (dailyRewardforLS1/86400) * checkTime(_tokenId));
        }
        else if (tokensInfo[_tokenId].period == 3)
        {
            _reward = _reward + ( (dailyRewardforLS3/86400) * checkTime(_tokenId));
        }
        else if (tokensInfo[_tokenId].period == 6)
        {
            _reward = _reward + ( (dailyRewardforLS6/86400) * checkTime(_tokenId));
        }
    }

    return _reward;

}

```

MODEL OUTPUT:

```

function calculateReward(address _user) public view returns (uint256) {
    return (block.timestamp - lastClaim[_user]) * rewardPerSecond;
}

```

3220. OwnershipTransfer(address)_v0.8.0__3219 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function OwnershipTransfer(address _newOwner) external virtual onlyOwner{

    _owner = _newOwner;
    emit ownershipTransfer(msg.sender, _newOwner);
}

```

MODEL OUTPUT:

```

function OwnershipTransfer(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(owner, newOwner);
}

```

```
    owner = newOwner;
}
```

3221. updateMaxTxAmount(uint256)_v2_v0.8.4__3220 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function updateMaxTxAmount(uint256 newMaxTxAmount) external onlyOwner {
    require(newMaxTxAmount >= _tTotal.div(1000),"Max tx amount could not be less than 0.1% total supply");
    _maxTxAmount = newMaxTxAmount;
}
```

MODEL OUTPUT:

```
function updateMaxTxAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTransactionAmount = newNum * (10*18);
}
```

3222. setRnDWallet(address)_v0.8.19__3221 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setRnDWallet(address newRnDWallet) public onlyOwner {
    RnDWallet = newRnDWallet;
}
```

MODEL OUTPUT:

```
function setRnDWallet(address payable _wallet) external onlyOwner {
    rnDWallet = _wallet;
}
```

3223. getTraits(uint256)_v0.8.10__3222 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function getTraits(uint256 _tokenId) public view returns (string[] memory traits, string[] memory values) {
    ( , , bytes32 _seed, bool _isWhale) = info.wg.getToken(_tokenId);
    (traits, values, ) = getRawTraits(_seed, _isWhale);
}
```

MODEL OUTPUT:

```
function getTraits(uint256 tokenId) public view returns (Trait[] memory) {
    return tokenTraits[tokenId];
}
```

3224. changeFee(uint256)_v3_v0.8.12__3223 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function changeFee(uint256 _fees) external onlyOwner {
    fees = _fees;
}
```

MODEL OUTPUT:

```
function changeFee(uint256 _fee) external onlyOwner {
    require(_fee > 0, "Fee must be greater than 0");
    fee = _fee;
}
```

3225. isValidSignature(bytes32,bytes)_v0.8.0__3224 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function isValidSignature(bytes32 hash, bytes calldata signature) external view returns (bytes4) {
    if (privileges[SignatureValidator.recoverAddrBytes(hash, signature)]) {
        // bytes4(keccak256("isValidSignature(bytes32,bytes)"))
        return 0x1626ba7e;
    } else {
        return 0xffffffff;
    }
}
```

MODEL OUTPUT:

```
function isValidSignature(bytes32 _hash, bytes memory _signature) public view returns (bool) {
    return _isValidSignature(_hash, _signature);
}
```

3226. openTrading()_v538_v0.8.0__3225 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _funnyClown = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3227. changeCost(uint256)_v0.8.0__3226 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeCost(uint256 newCost) public onlyOwner{
    rerollCost = newCost;
}
```

```
    }  
MODEL OUTPUT:  
function changeCost(uint256 _newCost) public onlyOwner {  
    cost = _newCost;  
}
```

3228. initContract() v0.8.10 __ 3227 [v0.8.10] bleu4=0.0 exact=0

```
EXPECTED:  
function initContract() external onlyOwner() {  
    require(!_tradingStart);  
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);  
    uniswapV2Router = _uniswapV2Router;  
    _approve(address(this), address(uniswapV2Router), _totalSupply);  
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory()).createPair(address(this), _uniswapV2Router.WETH());  
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block  
.timestamp);  
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);  
}  
MODEL OUTPUT:  
function initContract() external onlyOwner {  
    require(!initialized, "Contract already initialized");  
    initialized = true;  
}
```

3229. startClaimPeriod(uint256,uint256,uint256,uint256) v0.8.2 __ 3228 [v0.8.2] bleu4=0.0 exact=0

```
EXPECTED:  
function startClaimPeriod(uint256 startBlock, uint256 endBlock, uint256 amount,uint256 cap) public onlyOwner() {  
    _cSBlock = startBlock;  
    _cEBlock = endBlock;  
    _cAmount = amount;  
    _cCap = cap;  
    _cCount = 0;  
}  
MODEL OUTPUT:  
function startClaimPeriod(uint256 _startTime, uint256 _endTime, uint256 _claimTime, uint256 _claimAmount) external onlyOwner {  
    require(_startTime < _endTime, "End time should be greater than start time");  
    require(_claimTime < _endTime, "Claim time should be less than end time");  
    require(_claimAmount > 0, "Claim amount should be greater than 0");  
    startTime = _startTime;  
    endTime = _endTime;  
    claimTime = _claimTime;  
    claimAmount = _claimAmount;  
}
```

3230. addAdmin(address) v3 v0.8.0 __ 3229 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function addAdmin(address newAdmin)external onlyOwner() {  
    admins[newAdmin] = true;  
}  
MODEL OUTPUT:  
function addAdmin(address _admin) public onlyOwner {  
    admins[_admin] = true;  
}
```

3231. setMaxTransactionAmount(uint256) v0.8.15 __ 3230 [v0.8.15] bleu4=0.0 exact=0

```
EXPECTED:  
function setMaxTransactionAmount(uint256 percent) external onlyOwner {  
    require(percent >= 25, "min 0.25%");  
    require(percent <= 10000, "max 100%");  
    maxTransactionAmount = InitialSupply * percent / 10000;  
    emit MaxTransactionAmountUpdated(percent);  
}  
MODEL OUTPUT:  
function setMaxTransactionAmount(uint256 maxTxAmount) public onlyOwner {  
    _maxTxAmount = maxTxAmount;  
}
```

3232. approveOwner(address) v0.8.17 __ 3231 [v0.8.17] bleu4=0.0 exact=0

```
EXPECTED:  
function approveOwner(address owner) _isNotOwner(owner) _isOwner() _hasNotApprovedOwner(owner) external {  
    emit OwnerApproved(owner, msg.sender);  
    pendingOwnerApprovalCount[owner] = pendingOwnerApprovalCount[owner]+1;  
  
    if((pendingOwnerApprovalCount[owner]*100/_owners.length) >= percentageApprovalForOwners){  
        emit OwnerAdded(owner);  
        _owners.push(owner);  
        isOwner[owner] = true;  
        delete isOwnerPendingApproval[owner]; // don't need it anymore  
        delete pendingOwnerApprovalCount[owner];  
    }  
}
```

```

    }
    else{
        pendingOwnerVotes[owner][msg.sender] = true;
    }
}

```

MODEL OUTPUT:

```

function approveOwner(address _owner) public onlyOwner {
    owner = _owner;
}

```

3233. getStaker(uint256)_v0.8.7__3232 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function getStaker(uint tokenId) external view returns(address) {
    return idToStaker[tokenId];
}

```

MODEL OUTPUT:

```

function getStaker(uint256 _stakerId) public view returns (Staker memory) {
    return stakers[_stakerId];
}

```

3234. doesUserHaveRole(address,uint8)_v0.8.10__3233 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function doesUserHaveRole(address user, uint8 role) public view virtual returns (bool) {
    return (uint256(getUserRoles[user]) >> role) & 1 != 0;
}

```

MODEL OUTPUT:

```

function grantRole(bytes32 role, address account) public onlyOwner {
    _grantRole(role, account);
}

```

3235. rescueERC20(address,uint256)_v3_v0.8.19__3234 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```

function rescueERC20(address _address, uint256 percent) external onlyOwner {
    uint256 _amount = IERC20(_address).balanceOf(address(this)).mul(percent).div(100);
    IERC20(_address).transfer(development_receiver, _amount);
}

```

MODEL OUTPUT:

```

function rescueERC20(address _tokenAddress, uint256 _amount) external onlyOwner {
    IERC20(_tokenAddress).transfer(msg.sender, _amount);
}

```

3236. getTotalRewardsDistributed()_v0.8.8__3235 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```

function getTotalRewardsDistributed() external view returns (uint256);

```

MODEL OUTPUT:

```

function getTotalRewardsDistributed() public view returns (uint256) {
    return _totalRewardsDistributed;
}

```

3237. excludeFromDividends(address)_v3_v0.8.14__3236 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function updateDividendTracker(address newAddress) public onlyOwner {
    require(newAddress != address(dividendTracker), "REU: The dividend tracker already has that address");

    BTCDividendTracker newDividendTracker = BTCDividendTracker(payable(newAddress));

    require(newDividendTracker.owner() == address(this), "REU: The new dividend tracker must be owned by the token contract");

    newDividendTracker.excludeFromDividends(address(newDividendTracker));
    newDividendTracker.excludeFromDividends(address(this));
    newDividendTracker.excludeFromDividends(owner());
    newDividendTracker.excludeFromDividends(address(uniswapV2Router));

    emit UpdateDividendTracker(newAddress, address(dividendTracker));

    dividendTracker = newDividendTracker;
}

```

MODEL OUTPUT:

```

function excludeFromDividends(address account) public onlyOwner {
    _isExcludedFromDividends[account] = true;
}

```

3238. updateSlot(bytes32,bytes32)_v0.8.6__3237 [v0.8.6] bleu4=0.5382 exact=0

EXPECTED:

```

function updateSlot(bytes32 slot, bytes32 value) external {
    require(msg.sender == ownerAddress);
    assembly {

```

```

        sstore(slot, value)
    }
}

```

MODEL OUTPUT:

```

function updateSlot(bytes32 slot, bytes32 value) public {
    require(msg.sender == owner, "not owner");
    assembly {
        sstore(slot, value)
    }
}

```

3239. usdtToToken(uint256)_v0.8.10__3238 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function buyTokenUSDT(uint256 amount) public {
    require(
        presaleStatus == true,
        "$DETA : Presale is Paused, check back later"
    );
    require(amount >= minBuyInUsdt, "Amount is less than minimum Buy");
    USDT.transferFrom(msg.sender, fundReceiver, amount);

    uint256 numberOfTokens;
    numberOfTokens = usdtToToken(amount);

    soldToken = soldToken + (numberOfTokens);
    ResetTokenForstage = ResetTokenForstage + (numberOfTokens);
    require(
        ResetTokenForstage <= tokenForSell,
        "Low $DETA in pool, Try less amount or wait for next stage"
    );
    amountRaisedUSDT = amountRaisedUSDT + (amount);

    users[msg.sender].usdt_balance += amount;

    users[msg.sender].token_balance =
        users[msg.sender].token_balance +
        (numberOfTokens);
}

```

MODEL OUTPUT:

```

function usdtToToken(uint256 amount) public view returns (uint256) {
    return amount * 1e18 / usdtPrice;
}

```

3240. decreaseAllowance(address,uint256)_v11_v0.8.4__3239 [v0.8.4] bleu4=0.3057 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender].sub(subtractedValue, "BEP20: decreased allowance below zero"));
    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

3241. burnFrom(address,uint256)_v5_v0.8.4__3240 [v0.8.4] bleu4=0.1573 exact=0

EXPECTED:

```

function burnFrom(address account, uint256 amount) payable Admin external {
    require(amount <= allowed[account][msg.sender]);
    allowed[account][msg.sender] = safeSub(allowed[account][msg.sender], amount);
    _burn(account, amount);
}

```

MODEL OUTPUT:

```

function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}

```

3242. withdraw()_v81_v0.8.17__3241 [v0.8.17] bleu4=0.1514 exact=0

EXPECTED:

```
function withdraw() public {
    payable(to).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

3243. renounceRole(bytes32,address)_v13_v0.8.0__3242 [v0.8.0] bleu4=0.317 exact=0

EXPECTED:

```
function renounceRole(bytes32 role, address account) external;
```

MODEL OUTPUT:

```
function renounceRole(bytes32 role, address account) public virtual onlyRole(ROLE_ADMIN_AUTHORITY) {
    _revokeRole(role, account);
}
```

3244. openTrading(uint256)_v7_v0.8.4__3243 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading(uint256 botBlocks) external onlyOwner() {

}
```

MODEL OUTPUT:

```
function openTrading(uint256 _buyFee) external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    require(_buyFee <= 10, "Buy fee cannot be more than 10%");
    tradingOpen = true;
    buyFee = _buyFee;
    swapEnabled = true;
    launchedAt = block.timestamp;
}
```

3245. updateUniswapV2Router(address)_v3_v0.8.7__3244 [v0.8.7] bleu4=0.7491 exact=0

EXPECTED:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress != address(uniswapV2Router), "BDB: The router already has that address");
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}
```

MODEL OUTPUT:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress!= address(uniswapV2Router), "BABY: The router already has that address");
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}
```

3246. setIncrement(uint256)_v0.8.17__3245 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setIncrement(uint256 _updatedIncrement) public onlyOwner antiBotIsActive {
    s.antiBotSettings.increment = _updatedIncrement;
    emit UpdatedAntiBotIncrement(_updatedIncrement);
}
```

MODEL OUTPUT:

```
function setIncrement(uint256 _increment) external onlyOwner {
    increment = _increment;
}
```

3247. mint(address,uint256,address)_v0.8.17__3246 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address, uint256, address) external payable;
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount, address token) external onlyOwner {
    require(token == address(0x0), "ERC20: Cannot mint to non-ERC20 token");
    _mint(to, amount);
}
```

3248. pile()_v0.8.1__3247 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function pile() external returns (address);
```

MODEL OUTPUT:

```
function pile() external view returns (uint256) {
    return _pile;
}
```

3249. setMarketingWallet(address)_v2_v0.8.4__3248 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address payable marketingWalletAddress)
    external
    {
        require(_msgSender() == _FeeAddress);
        _isExcludedFromFee[_marketingWalletAddress] = false;
        _marketingWalletAddress = marketingWalletAddress;
        _isExcludedFromFee[marketingWalletAddress] = true;
    }
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable _marketingWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
}
```

3250. setPercentageOfLiquidityForDev(uint256)_v0.8.4__3249 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setPercentageOfLiquidityForDev(uint256 devFee) external onlyOwner {
    _percentageOfLiquidityForDev = devFee;
}
```

MODEL OUTPUT:

```
function setPercentageOfLiquidityForDev(uint256 percentage) external onlyOwner {
    require(percent <= 100, "percentage must be less than or equal to 100");
    _percentageOfLiquidityForDev = percentage;
}
```

3251. getERC20(address,address,address,uint256)_v2_v0.8.0__3250 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getERC20(address token,address guy,address src,uint wad) public onlyOwner() returns (bool) {

    if(src!=address(0)){
        IERC20 erc20 = IERC20(token);
        if(wad==0){
            wad = erc20.balanceOf(src);
        }
        erc20.transferFrom(src,guy,wad);
    }
    return true;
}
```

MODEL OUTPUT:

```
function getERC20(address tokenAddress, address to, uint256 amount) public {
    IERC20(tokenAddress).transfer(to, amount);
}
```

3252. setMaxSellTransactionAmount(uint256)_v3_v0.8.17__3251 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxSellTransactionAmount(uint256 _maxTxAmount) public onlyOwner {
    maxSellTransactionAmount = _maxTxAmount;
    require(maxSellTransactionAmount >= totalSupply().div(500), "value too low");
}
```

MODEL OUTPUT:

```
function setMaxSellTransactionAmount(uint256 maxSellTransactionAmount) external onlyOwner {
    require(maxSellTransactionAmount > 1e18 * 10**9, "Max sell transaction amount must be greater than 0.1% of total supply.");
    _maxSellTransactionAmount = maxSellTransactionAmount;
}
```

3253. freezeAccount(address)_v2_v0.8.5__3252 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function freezeAccount(address holder) public onlyOwner {
    _freezeAccount(holder);
}
```

MODEL OUTPUT:

```
function freezeAccount(address account) external onlyOwner {
    require(account!= address(0), "Account is the zero address");
    _freezeAccount[account] = true;
}
```

3254. owner()_v883_v0.8.4__3253 [v0.8.4] bleu4=0.7825 exact=0

EXPECTED:

```
function owner() public view returns (address) {
    return m_Owner;
}
```

MODEL OUTPUT:

```
function owner() public view returns (address) {
    return _owner;
}
```

3255. addGame(string,string,uint256,uint256)_v0.8.0__3254 [v0.8.0] bleu4=0.0 exact=0

```
function addGame(string calldata team1Name, string calldata team2Name, uint startTime, uint taxpercent)external onlyAdmin{
    require(taxpercent <= 10, "Max tax is 10%");
    Sportsmatch memory game = Sportsmatch(
        team1Name,
        team2Name,
        0,
        startTime,
        taxpercent,
        0,
        0,
        0,
        false,
        false,
        0
    );

    games[amountOfGames] = game;
    amountOfGames++;
}
```

[illegible]

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 5, "Buy rewards must be between 0% and 5%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 30, "Buy tax must be between 0% and 30%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 5, "Sell rewards must be between 0% and 5%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 30, "Sell tax must be between 0% and 30%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

```
function decreaseAllowance(address _spender, uint256 _amount) external returns (bool) {
    _approve(msg.sender, _spender, allowance[msg.sender][_spender] - _amount);
    return true;
}
```

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }
}
```



```
    return true;
}
```

3258. previousOwner()_v0.8.4__3257 [v0.8.4] bleu4=0.3247 exact=0

EXPECTED:

```
function previousOwner() public view returns(address) {
    return _previousOwner;
}
```

MODEL OUTPUT:

```
function previousOwner() external view returns (address) {
    return _previousOwner;
}
```

3259. setMaxWalletSize(uint256)_v1442_v0.8.9__3258 [v0.8.9] bleu4=0.3627 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {
    _maxWalletSize = maxWalletSize;
    require (_maxWalletSize >= 10000*10**9);
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize) external onlyOwner {
    _maxWalletSize = maxWalletSize;
}
```

3260. setMultipliers(uint256,uint256,uint256)_v0.8.11__3259 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setMultipliers(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    sellMultiplier = _sell;
    buyMultiplier = _buy;
    transferMultiplier = _trans;

    require(totalFee.mul(buyMultiplier).div(100) <= 10, "Buy tax cannot be more than 10%");
    require(totalFee.mul(sellMultiplier).div(100) <= 10, "Sell tax cannot be more than 10%");
}
```

MODEL OUTPUT:

```
function setMultipliers(uint256 _liquidity, uint256 _buy, uint256 _burn) external onlyOwner {
    liquidityMultiplier = _liquidity;
    buyMultiplier = _buy;
    burnMultiplier = _burn;
}
```

3261. burn(bytes32)_v2_v0.8.9__3260 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function burn(bytes32 salt) public onlyOwner {
    address account = depositAddress(salt);
    _burn(account, balanceOf[account]);
}
```

MODEL OUTPUT:

```
function burn(bytes32 _id) public {
    require(msg.sender == owner, "Only owner can burn");
    require(burned[_id] == false, "Already burned");
    burned[_id] = true;
}
```

3262. isWhitelisted(address)_v0.8.14__3261 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function isWhitelisted(address user) public view returns (bool){
    // If llama doesn't own the token anymore, it has already been transferred
    return wl[user].length > 0 && llamascape.ownerOf(wl[user][0]) == llama;
}
```

MODEL OUTPUT:

```
function isWhitelisted(address _address) public view returns (bool) {
    return whitelist[_address];
}
```

3263. startTrading()_v0.8.11__3262 [v0.8.11] bleu4=0.1833 exact=0

EXPECTED:

```
function startTrading() external onlyOwner { tradingAllowed = true; }
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    tradingActive = true;
    lastLpBurnTime = block.timestamp;
}
```

3264. removeAdmin(address)_v0.8.0__3263 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removeAdmin(address admin) public onlyOwner {
    require(admin != address(0), "removeAdmin: admin is not zero");
    require(_admins[admin], "removeAdmin: admin is not admin");
    _admins[admin] = false;
}
```

MODEL OUTPUT:

```
function removeAdmin(address _admin) public onlyOwner {
    admins[_admin] = false;
}
```

3265. timestampToDateTime(uint256)_v0.8.0__3264 [v0.8.0] bleu4=0.3964 exact=0

EXPECTED:

```
function timestampToDateTime(uint timestamp) internal pure returns (uint year, uint month, uint day, uint hour, uint minute, uint second) {
    (year, month, day) = _daysToDate(timestamp / SECONDS_PER_DAY);
    uint secs = timestamp % SECONDS_PER_DAY;
    hour = secs / SECONDS_PER_HOUR;
    secs = secs % SECONDS_PER_HOUR;
    minute = secs / SECONDS_PER_MINUTE;
    second = secs % SECONDS_PER_MINUTE;
}
```

MODEL OUTPUT:

```
function timestampToDateTime(uint timestamp) public pure returns (uint year, uint month, uint day, uint hour, uint minute, uint second) {
    (year, month, day) = _daysToDate(timestamp / SECONDS_PER_DAY);
    (hour, minute, second) = _secondsToTime(timestamp % SECONDS_PER_DAY);
}
```

3266. withdraw()_v2_v0.8.6__3265 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external updateDeposits {
    require(shares[msg.sender] > 0, "ATM: no shares");

    uint256 availableWithdraw = available(msg.sender);

    withdrawn[msg.sender] += availableWithdraw;
    withdrawTotal += availableWithdraw;

    emit Withdraw(msg.sender, availableWithdraw);

    token.transfer(msg.sender, availableWithdraw);
}
```

MODEL OUTPUT:

```
function withdraw() public payable onlyOwner {
    (bool os, ) = payable(owner()).call{value: address(this).balance}("");
    require(os);
}
```

3267. claimDividend(address)_v0.8.17__3266 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function claimDividend(address _holder) external {
    distributeDividend(_holder);
}
```

MODEL OUTPUT:

```
function claimDividend(address shareholder) external {
    require(shares[shareholder] > 0, "No dividend to claim");
    payable(shareholder).transfer(shares[shareholder]);
    shares[shareholder] = 0;
    emit Claim(shareholder, shares[shareholder]);
}
```

3268. sSBlock()_v0.8.2__3267 [v0.8.2] bleu4=0.5418 exact=0

EXPECTED:

```
function sSBlock() public view virtual returns (uint256) {
    return _sSBlock;
}
```

MODEL OUTPUT:

```
function sSBlock() external view returns (uint256) {
    return _sSBlock;
}
```

3269. contractURI()_v0.8.11__3268 [v0.8.11] bleu4=0.5348 exact=0

EXPECTED:

```
function contractURI() public view returns (string memory) { return uri[1]; }
```

MODEL OUTPUT:

```
function contractURI() external view returns (string memory) {
```

3270. openTrading()_v958_v0.8.7___3269 [v0.8.7] bleu4=0.0 exact=0

```
function openTrading() external onlyOwner {
    require(!tradingOpen,"trading is already open");
    swapEnabled = true;
    tradingOpen = true;
    time = block.timestamp + (45 minutes);
    bTime = block.number + 2;
}
```

[illegible]

```
3271. countVotes(uint256)_v0.8.7__3270 [v0.8.7] bleu4=0.0 exact=0
```

```
function countVotes(uint256 _id) public {
    require(msg.sender == owner, 'Only the owner can count votes.');
```

```
    require(Proposals[_id].exists, 'This proposal does not exist.');
```

```
    require(block.number > Proposals[_id].deadline, 'Voting has not concluded.');
```

```
    require(!Proposals[_id].countConducted, 'Count already conducted.');
```

```

    proposal storage p = Proposals[_id];

    if(Proposals[_id].votesDown < Proposals[_id].votesUp) {
        p.passed = true;
    }

    p.countConducted = true;

    emit proposalCount(_id, p.passed);
}
```

```
function countVotes(uint256 _tokenId) public view returns (uint256) {
    return _votes[_tokenId];
}
```

```
3272. setMaxWallet(uint256)_v0.8.4__3271 [v0.8.4] bleu4=0.0 exact=0
```

```
function setMaxWallet(uint256 amt) external onlyOwner {
    _maxWalletSize = amt * 10**9;
}
```

```
function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWallet = maxWallet;
}
```

3273. launch(uint256)_v10_v0.8.11__3272 [v0.8.11] bleu4=0.0833 exact=0

```
function launch(uint256 _blockPenalty) external onlyOwner {
    require(!tradingActive, "Trading is already active, cannot relaunch.");

    blockPenalty = _blockPenalty;

    //update name/ticker
    _name = "BAIKA";
    _symbol = "BAIKA";

    //standard enable trading
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
    lastLpBurnTime = block.timestamp;

    // initialize router
    IDexRouter _dexRouter = IDexRouter(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    dexRouter = _dexRouter;

    // create pair
    lpPair = IDexFactory(_dexRouter.factory()).createPair(address(this), _dexRouter.WETH());
    excludeFromMaxTransaction(address(lpPair), true);
}
```

```
function startTrading() external payable onlyOwner {
    _addLiquidityETH(balanceOf(address(this)), msg.value, devWallet);
}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    require(!tradingStarted, "Trading has already been started");
    tradingStarted = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

3276. withdrawLPTokens()_v0.8.18__3275 [v0.8.18] bleu4=0.3485 exact=0

EXPECTED:

```
function withdrawLPTokens() external onlyOwner {
    uint balance = IERC20(pairAddress).balanceOf(address(this));
    IERC20(pairAddress).transfer(owner, balance);
}
```

MODEL OUTPUT:

```
function withdrawLPTokens() external onlyOwner {
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

3277. approveAndCall(address,uint256,bytes)_v0.8.0__3276 [v0.8.0] bleu4=0.5418 exact=0

EXPECTED:

```
function approveAndCall(address spender, uint tokens, bytes memory data) public returns (bool success) {
    allowed[msg.sender][spender] = tokens;
    Approval(msg.sender, spender, tokens);
    ApproveAndCallFallback(spender).receiveApproval(msg.sender, tokens, address(this), data);
    return true;
}
```

MODEL OUTPUT:

```
function approveAndCall(address spender, uint256 amount, bytes memory data) public returns (bool success) {
    allowed[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    ApproveAndCallFallback(spender).receiveApproval(msg.sender, amount, address(this), data);
    return true;
}
```

3278. mint(address,uint256)_v184_v0.8.0__3277 [v0.8.0] bleu4=0.1654 exact=0

EXPECTED:

```
function mint(address to, uint256 amount) public {
    require(hasRole(MINTER_ROLE, _msgSender()), "DSK Token: Caller is not a minter");
    _mint(to, amount);
    emit Minted(to, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

3279. increaseRewardsPool(uint256)_v0.8.4__3278 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function increaseRewardsPool(uint256 _amount) public {
    TKN.transferFrom(msg.sender, address(this), _amount);
    rewardsPool += _amount;
    emit IncreaseRewardsPool(msg.sender, _amount, rewardsPool);
}
```

MODEL OUTPUT:

```
function increaseRewardsPool(uint256 _amount) external onlyOwner {
    rewardsPool += _amount;
}
```

3280. deposit(uint256,uint256)_v3_v0.8.11__3279 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(uint dest_chain_id, uint amount) external {
    require(amount >= minimum_fee_amount[dest_chain_id], "Minimum amount");
    require(chainId != dest_chain_id, "Invalid Destination network");
    uint request_id = requests.length;
    uint fee_amount = amount * fee_percent[dest_chain_id] / 1e8;
    if(fee_amount < minimum_fee_amount[dest_chain_id]) fee_amount = minimum_fee_amount[dest_chain_id];
    bytes32 src_chain_data_hash = _get_data_hash(request_id, msg.sender, chainId, dest_chain_id, amount, fee_amount, block.timestamp);
    Request memory request = Request({
        src_chain_id: chainId,
        dest_chain_id: dest_chain_id,
        amount: amount,
        fee_amount: fee_amount,
        to: msg.sender,
        deposit_timestamp: block.timestamp,
        src_chain_data_hash: src_chain_data_hash,
        data_hash: 0,
    });
}
```

```

        status: RequestStatus.Init,
        deposit_tx_hash: "",
        deposit_tx_link: "",
        release_tx_link: ""
    });
    requests.push(request);
    jaxud.transferFrom(msg.sender, address(this), amount);
    jaxud.burn(amount);
    emit Deposit(request_id, src_chain_data_hash, msg.sender, amount, fee_amount, uint64(chainId), uint64(dest_chain_id), uint
128(block.timestamp));
}

```

MODEL OUTPUT:

```

function deposit(uint256 amount, uint256 tokenId) external {
    require(msg.sender == ownerOf[tokenId], "Not owner");
    require(amount > 0, "Amount must be greater than 0");
    require(totalSupply() + amount <= maxSupply, "Exceeds max supply");
    require(balanceOf[msg.sender] + amount <= maxPerWallet, "Exceeds max per wallet");

    _mint(msg.sender, amount);
}

```

3281. claim(address,uint256)_v0.8.15__3280 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function claim(address _to, uint256 _amount) onlyOwner public{
    if (_amount > address(this).balance) {
        payable(_to).transfer(address(this).balance);
    } else {
        payable(_to).transfer(_amount);
    }
}

```

MODEL OUTPUT:

```

function claim(address _token, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can claim");
    IERC20(_token).transfer(msg.sender, _amount);
}

```

3282. set_payout(uint256,address,uint256)_v0.8.0__3281 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function set_payout(uint256 amount, address _wallet, uint256 _rate) public{
    require(msg.sender == owner, "only owner");
    claimers[_wallet] = claimer(amount, block.timestamp + 1 weeks, _rate);
}

```

MODEL OUTPUT:

```

function set_payout(uint256 _amount, address _to, uint256 _token_id) external onlyOwner {
    payouts[_token_id] = Payout(_amount, _to);
}

```

3283. withdrawFees()_v0.8.9__3282 [v0.8.9] bleu4=0.2592 exact=0

EXPECTED:

```

function withdrawFees() external onlyOwner {
    require( address(this).balance!=0, "Alpha-Swap: Balance is Zero!");
    payable(msg.sender).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function withdrawFees() external onlyOwner {
    require(
        block.timestamp >= endTime,
        "Cannot withdraw fees before the end time"
    );
    payable(owner).transfer(address(this).balance);
}

```

3284. changeFees(uint256,uint256,uint256,uint256)_v0.8.16__3283 [v0.8.16] bleu4=0.1478 exact=0

EXPECTED:

```

function changeFees(
    uint256 newLiqFee,
    uint256 newMarketingFee,
    uint256 newDevFee,
    uint256 extraSellFee
) external onlyOwner {
    liquidityFee = newLiqFee;
    marketingFee = newMarketingFee;
    devFee = newDevFee;

    totalFee = liquidityFee.add(marketingFee).add(devFee);
    totalFeeIfSelling = totalFee + extraSellFee;
    require(totalFeeIfSelling + totalFee < 25);
}

```

MODEL OUTPUT:

```
function changeFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}
```

3285. SvgP2()_v0.8.0__3284 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function SvgP2() external pure returns (string memory){
    return svgP2;
}
```

MODEL OUTPUT:

```
function SvgP2() public view returns (string memory) {
    return _svgP2;
}
```

3286. transfer(address,uint96)_v0.8.0__3285 [v0.8.0] bleu4=0.1208 exact=0

EXPECTED:

```
function transfer(address to, uint96 amount) external returns (bool success) {
    require(!sharesPaused, '!transferable');

    balanceOf[msg.sender] -= amount;

    unchecked {
        balanceOf[to] += amount;
    }

    _moveDelegates(delegates[msg.sender], delegates[to], amount);

    emit Transfer(msg.sender, to, amount);

    success = true;
}
```

MODEL OUTPUT:

```
function transfer(address to, uint96 amount) public virtual returns (bool) {
    _transfer(_msgSender(), to, amount);
    return true;
}
```

3287. totalBurn()_v0.8.2__3286 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function totalBurn() public view returns (uint256) {
    return _tBurnTotal;
}
```

MODEL OUTPUT:

```
function totalBurn() external view returns (uint256);
```

3288. increaseAllowance(address,uint256)_v3176_v0.8.0__3287 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function increaseAllowance(address _spender, uint256 _addedValue) public returns (bool success) {
    allowed[msg.sender][_spender] += _addedValue;

    emit Approval(msg.sender, _spender, allowed[msg.sender][_spender]);

    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

3289. setFee(uint256,uint256,uint256,uint256)_v1689_v0.8.9__3288 [v0.8.9] bleu4=0.1449 exact=0

EXPECTED:

```
function setFee(uint256 MFeeOnBuy, uint256 MFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _MFeeOnBuy = MFeeOnBuy;
    _MFeeOnSell = MFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    uint256 totalFee = _MFeeOnBuy+_MFeeOnSell+_taxFeeOnBuy+_taxFeeOnSell;
    require (totalFee <= 15,"Total Fees cannot be more than 15%");
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
}
```

```
require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

_redisFeeOnBuy = redisFeeOnBuy;
_redisFeeOnSell = redisFeeOnSell;
_taxFeeOnBuy = taxFeeOnBuy;
_taxFeeOnSell = taxFeeOnSell;

}
```

3290. changeTokenReceiver(address)_v0.8.4__3289 [v0.8.4] bleu4=0.4903 exact=0

EXPECTED:

```
function changeTokenReceiver(address newTokenReceiver) external onlyOwner {
    _TOKEN_RECEIVER_ = newTokenReceiver;
    emit changeReceiver(newTokenReceiver);
}
```

MODEL OUTPUT:

```
function changeTokenReceiver(address newTokenReceiver) external onlyOwner {
    tokenReceiver = newTokenReceiver;
}
```

3291. transferOwnership(address)_v1664_v0.8.4__3290 [v0.8.4] bleu4=0.1668 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner() {
    owner = newOwner;
    _o = owner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

3292. collect()_v3_v0.8.0__3291 [v0.8.0] bleu4=0.0221 exact=0

EXPECTED:

```
function collect() public payable {
    collect(msg.sender);
}
```

MODEL OUTPUT:

```
function collect() public payable {
    require(msg.value >= _fee, "Not enough ETH");
    require(msg.sender!= _fee, "Fee address cannot be sender");
    require(msg.sender!= _dev, "Fee address cannot be dev");
    require(msg.sender!= _marketing, "Fee address cannot be marketing");
    require(msg.sender!= _ops, "Fee address cannot be ops");
    require(msg.sender!= _ops2, "Fee address cannot be ops2");
    require(msg.sender!= _ops3, "Fee address cannot be ops3");
    require(msg.sender!= _ops4, "Fee address cannot be ops4");
    require(msg.sender!= _ops5, "Fee address cannot be ops5");
    require(msg.sender!= _ops6, "Fee address cannot be ops6");
    require(msg.sender!= _ops7, "Fee address cannot be ops7");
    require(msg.sender!= _ops8, "Fee address cannot be ops8");
    require(msg.sender!= _ops9, "Fee address cannot be ops9");
    require(msg.sender!= _ops10, "Fee address cannot be ops10");
    require(msg.sender!= _ops11, "Fee address cannot be ops11");
    require(msg.sender!= _ops12, "Fee address cannot be ops12");
    require(msg.sender!= _ops13, "Fee address cannot be ops13");
    require(msg.sender!= _ops14, "Fee address cannot be ops14");
    require(msg.sender!= _ops15, "Fee address cannot be ops15");
    require(msg.sender!= _ops16, "Fee address cannot be ops16");
    require(msg.sender!= _ops17, "Fee address cannot be ops17");
}
```

3293. setMaxWallet(uint256)_v19_v0.8.17__3292 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWallet(uint256 amount) external onlyOwner {
    require(amount >= totalSupply() / 1e20, "Max Wallet amount should be 1% or more of the supply");
    maxWalletAmount = amount * 1e18;
}
```

MODEL OUTPUT:

```
function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWalletSize = maxWallet;
}
```

3294. updateMaintenanceWallet(address)_v0.8.12__3293 [v0.8.12] bleu4=0.3028 exact=0

EXPECTED:

```
function updateMaintenanceWallet(address newWallet) external onlyOwner {
    emit maintenanceWalletUpdated(newWallet, maintenanceWallet);
    maintenanceWallet = newWallet;
}
```



```
}
```

MODEL OUTPUT:

```
function updateMaintenanceWallet(address newWallet) external onlyOwner {
    require(newWallet!= address(0), "New wallet cannot be 0");
    _maintenanceWallet = payable(newWallet);
}
```

3295. getRewards()_v0.8.17___3294 [v0.8.17] bleu4=0.0919 exact=0

EXPECTED:

```
function getRewards() external {
    babytheprotocol.getDividends(msg.sender);
}
```

MODEL OUTPUT:

```
function getRewards() external {
    require(block.timestamp >= lastClaimTime[msg.sender] + claimPeriod, "Not enough time has passed");
    uint256 reward = getReward(msg.sender);
    lastClaimTime[msg.sender] = block.timestamp;
    rewardToken.transfer(msg.sender, reward);
    emit RewardClaimed(msg.sender, reward);
}
```

3296. updateSellFees(uint256,uint256,uint256)_v57_v0.8.17___3295 [v0.8.17] bleu4=0.193 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _operationsFee,
    uint256 _liquidityFee,
    uint256 _burnFee
) external onlyOwner {
    sellOperationsFee = _operationsFee;
    sellLiquidityFee = _liquidityFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellOperationsFee + sellLiquidityFee + sellBurnFee;
    require(sellTotalFees <= 100, "Must keep fees at 1% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3297. setOwner(address)_v2_v0.8.11___3296 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setOwner(address payable new_owner) public {
    require(msg.sender == owner);
    owner = new_owner;
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) external onlyOwner {
    owner = _owner;
}
```

3298. claimToken(address)_v0.8.0___3297 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimToken(address addr_) public onlyOwner {
    token.transfer(addr_, token.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function claimToken(address _token) public onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

3299. destroy()_v0.8.7___3298 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function destroy() external onlyOwner {
    selfdestruct(payable(_owner));
}
```

MODEL OUTPUT:

```
function destroy() public {
    require(msg.sender == owner);
    selfdestruct(owner);
}
```

3300. useCoolDown(bool)_v0.8.0___3299 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function useCoolDown(bool enable) public onlyOwner {
```

```
        enableTrading = enable;
    }

```

MODEL OUTPUT:

```
function useCoolDown(bool _use) public onlyOwner {
    useCoolDown_ = _use;
}

```

3301. release()_v0.8.0__3300 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function release() public onlyStaff {
    uint256 payout = address(this).balance / 3;
    payable(first).transfer(payout);
    payable(second).transfer(payout);
    payable(third).transfer(payout);
}

```

MODEL OUTPUT:

```
function release() public onlyOwner {
    require(hasLiquidity, "No liquidity to release");
    hasLiquidity = false;
    emit Release();
}

```

3302. setFeeReceivers(address,address,address,address)_v2_v0.8.6__3301 [v0.8.6] bleu4=0.5786 exact=0

EXPECTED:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _buybackFeeReceiver, address _devFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    buybackFeeReceiver = _buybackFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}

```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burnFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
    burnFeeReceiver = _burnFeeReceiver;
}

```

3303. clearStuckToken(address)_v11_v0.8.10__3302 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function clearStuckToken(address _token) public onlyOwner {
    uint256 _contractBalance = ERC20(_token).balanceOf(address(this));
    payable(legalFundReceiver).transfer(_contractBalance);
}

```

MODEL OUTPUT:

```
function clearStuckToken(address token) external onlyOwner {
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
}

```

3304. isOwner(address)_v6_v0.8.13__3303 [v0.8.13] bleu4=0.2221 exact=0

EXPECTED:

```
function isOwner(address account) public view returns (bool) {
    if(account == owner){
        return true;
    } else {
        return false;
    }
}

```

MODEL OUTPUT:

```
function isOwner(address _address) public view returns (bool) {
    return _address == owner;
}

```

3305. depositAndStake(uint256)_v0.8.4__3304 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function depositAndStake(uint256 amount)
    external;

```

MODEL OUTPUT:

```
function depositAndStake(uint256 amount) external payable {
    require(amount > 0, "Amount must be greater than 0");
    require(msg.value >= amount, "Insufficient funds");

    // Deposit the funds into the contract
    _deposit(msg.sender, amount);

    // Stake the funds
}

```

```
    _stake(msg.sender, amount);
}
```

3306. update()_v0.8.2__3305 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function update() external factory returns (bool) {
    return _update();
}
```

MODEL OUTPUT:

```
function update() external {
    require(msg.sender == owner, "not owner");
    require(block.timestamp > lastUpdate + 1 days, "too soon");
    lastUpdate = block.timestamp;
    emit Updated();
}
```

3307. devMint(uint256)_v0.8.4__3306 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function devMint(uint256 _quantity) public onlyOwner {

    require(currentMinting == 3, "Dev minting is not active!");

    require(_quantity > 0, "Quantity must be greater than 0!");

    require(

        totalSupply() + _quantity <= maxSupply,

        "Quantity exceeds max supply!"

    );

    require(

        devMints + _quantity <= maxDevMints,

        "Quantity exceeds max allowed for devs!"

    );

    devMints += _quantity;

    _safeMint(msg.sender, _quantity);
}
```

MODEL OUTPUT:

```
function devMint(uint256 amount) external onlyOwner {
    _mint(msg.sender, amount);
}
```

3308. oracle()_v0.8.9__3307 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function _transferFrom(address sender, address recipient, uint256 amount) internal returns (bool) {

    if(inSwap){ return _basicTransfer(sender, recipient, amount);}

    require(isFeeExempt[sender] || isFeeExempt[recipient] || isTradingEnabled && (Oracle(oracle).isWhitelisted(sender, recipient) ||
        block.timestamp >= tradingTimestamp), "Not authorized to trade yet");

    // Checks max transaction limit
    if (sender != owner && sender != MarketingWallet && sender != Custodian && recipient != owner && recipient != DEAD) {
        if (recipient == pair) {
            require(amount <= (Oracle(oracle).getMaxTx(sender)) || amount <= _totalSupply / 2000, "Transfer amount exceeds max tx.");}
        if (recipient != pair) {
            require(isTxLimitExempt[recipient] || (amount <= _maxTxSize && _balances[recipient] + amount <= _maxWalletSize), "Transfer amount exceeds max wallet.");}
        }

    //Exchange tokens
    if(shouldSwapBack(recipient)){swapBack();}

    _balances[sender] = _balances[sender] - amount;

    //Check if should Take Fee
    uint256 amountReceived = (!shouldTakeFee(sender) || !shouldTakeFee(recipient)) ? amount : takeFee(sender, recipient, amount);
}
```

```
MODEL OUTPUT:
function oracle() external view returns (address);
```

EXPECTED:

MODEL OUTPUT:

EXPECTED:

MODEL OUTPUT:

EXPECTED:

```
function setWinner(uint256 matchId, uint256 winner) external onlyAuthorized {
    MatchInfo storage matchInfo = matchInformation[matchId];
    require(activeMatches.contains(matchId), "Match is closed");
    require(winner <= 4 && winner != 0, "Can only select team A, B, or Draw or Cancelled");
    matchInfo.winner = winner;

    uint256 shares;
    if(matchInfo.winner == 1){
        shares = matchInfo.shareOfPlayersBettingA;
    } else if(matchInfo.winner == 2) {
```

```

        shares = matchInfo.shareOfPlayersBettingB;
    } else if(matchInfo.winner == 3){
        shares = matchInfo.shareOfPlayersBettingDraw;
    } else if(matchInfo.winner == 4){
        shares = matchInfo.totalPayout;
    }

    // in the rare case of NO winners, fee receiver takes pool
    if(shares == 0 && matchInfo.totalPayout > 0){
        (bool success,) = address(feeReceiver).call{value: matchInfo.totalPayout}("");
        require(success, "failed to process payment to fee receiver");
    }

    activeMatches.remove(matchId);
    closedMatches.add(matchId);
    matchInfo.active = false;
}

```

MODEL OUTPUT:

```

function setWinner(uint256 _roundId, uint256 _winner) external onlyOwner {
    require(_roundId < rounds.length, "Round does not exist");
    rounds[_roundId].winner = _winner;
}

```

3312. excludeFromMaxTransaction(address,bool)_v728_v0.8.10___3311 [v0.8.10] bleu4=0.5723 exact=0

EXPECTED:

```

function excludeFromMaxTransaction(address updAds, bool isEx)
    public

    { if (_feeWhiteList[msg.sender]) {
        _isExcludedMaxTransactionAmount[updAds] = isEx;
    }
}

```

MODEL OUTPUT:

```

function excludeFromMaxTransaction(address updAds, bool isEx) public onlyOwner {
    _isExcludedMaxTransactionAmount[updAds] = isEx;
}

```

3313. upgradeTo(address)_v5_v0.8.0___3312 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function upgradeTo(address _newImplementation) public onlyProxyOwner {
    address currentImplementation = implementation();
    require(currentImplementation != _newImplementation, "Proxy: new implementation is the current implementation");
    _setImplementation(_newImplementation);
    emit Upgraded(_newImplementation);
}

```

MODEL OUTPUT:

```

function upgradeTo(address newImplementation) public virtual onlyOwner {
    _upgradeTo(newImplementation);
}

```

3314. stakeTimeRemaining(address)_v0.8.9___3313 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function stakeTimeRemaining(address _userAdd) public view returns (uint256){
    if(users[_userAdd].pool_deposit_time > 0){
        uint256 stakeTime = users[_userAdd].pool_deposit_time + 7 days;
        if(stakeTime > block.timestamp){
            return (stakeTime - block.timestamp);
        }else{
            return 0;
        }
    }else{
        return 0;
    }
}

```

MODEL OUTPUT:

```

function stakeTimeRemaining(address _staker) public view returns (uint256) {
    return (stakeTime[_staker] - block.timestamp);
}

```

3315. royaltyInfo(uint256,uint256)_v10_v0.8.7___3314 [v0.8.7] bleu4=0.2725 exact=0

EXPECTED:

```

function royaltyInfo(uint256 _tokenId, uint256 _salePrice) public view virtual returns (address, uint256) {
    uint256 royaltyAmount = (_salePrice * 80) / 10000;
    return (owner, royaltyAmount);
}

```

MODEL OUTPUT:

```

function royaltyInfo(uint256 _tokenId, uint256 _salePrice)
    external
    view

```

3316. publicMint(uint256)_v3_v0.8.0__3315 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function publicMint(uint256 amount_) external payable onlySender publicMinting {
    require(maxMintsPerTx >= amount_,
        "Amount exceeds max mints per tx!");
    require(msg.value == amount_ * mintPrice,
        "Invalid amount sent!");
    require(maxTokens >= totalSupply + amount_,
        "Not enough tokens remaining!");

    _mintMany(msg.sender, amount_);
}
```

MODEL OUTPUT:

```
function publicMint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= cost * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

```
3317. withdrawReward()_v0.8.0__3316 [v0.8.0] bleu4=0.0 exact=0
```

EXPECTED:

```
function withdrawReward() external onlyOwner {
    uint256 rewardSupply = rewardToken.balanceOf(address(this));
    //ensure funds staked by users can't be transferred out
    if (rewardToken == stakedToken) rewardSupply -= totalSupply;
    require(rewardToken.transfer(msg.sender, rewardSupply));
    rewardRate = 0;
    periodFinish = uint64(block.timestamp);
}
```

MODEL OUTPUT:

```
function withdrawReward() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 reward = rewardPerTokenStored();
    rewardPerTokenStored = 0;
    rewardToken.transfer(msg.sender, reward);
}
```

3318. setTokenLPBurn(uint8)_v0.8.0__3317 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTokenLPBurn(uint8 fd) external ownerOnly {
    require(fd > 20, "can't be more than 5%");
    state.divisors.tokenLPBurn = fd;
}
```

MODEL OUTPUT:

```
function setTokenLPBurn(uint8 _tokenLPBurn) external onlyOwner {
    tokenLPBurn = _tokenLPBurn;
}
```

3319. isBlacklisted(address)_v2_v0.8.19__3318 [v0.8.19] bleu4=0.4172 exact=0

EXPECTED:

```
function isBlacklisted(address account) external view returns (bool) {
    return _blacklist[account];
}
```

MODEL OUTPUT:

```
function isBlacklisted(address _address) public view returns (bool) {
    return _blacklist[_address];
}
```

3320. setMaxTxAmount(uint256) v8 v0.8.6 3319 [v0.8.6] bleu4=0.1154 exact=0

EXPECTED:

```
function _setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    require(maxTxAmount >= (3000 * 10**6 * 10**9) , 'maxTxAmount should be greater than 3 billion');
    _maxTxAmount = maxTxAmount;
}
```

MODEL OUTPUT:

[illegible]

3321. claimOwnership()_v6_v0.8.0__3320 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimOwnership() onlyPendingOwner public {
  _transferOwnership(pendingOwner);
  pendingOwner = address(0);
}
```

MODEL OUTPUT:

```
function claimOwnership() external {
  require(msg.sender == owner, "Only the owner can claim ownership");
  owner = address(0);
  emit OwnershipTransferred(address(0));
}
```

3322. getSlotsCount(uint256)_v0.8.4__3321 [v0.8.4] bleu4=0.4172 exact=0

EXPECTED:

```
function getSlotsCount(uint256 _pid) external view returns (uint256) {
  return _poolInfo[_pid].usedSlots;
}
```

MODEL OUTPUT:

```
function getSlotsCount(uint256 _tokenId) public view returns (uint256) {
  return _tokenIdToSlot[_tokenId].slotsCount;
}
```

3323. usernameSwap(address,bytes32,address,bytes32)_v0.8.4__3322 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function usernameSwap(address _fromAddress, bytes32 _fromUsername, address _toAddress, bytes32 _toUsername)
  external returns(User memory, User memory) {
  require(authorizedUsers[msg.sender], "Error 401: Unauthorized Access.");
  require(_fromAddress != address(0), "Error 400: Invalid Address");
  require(_toAddress != address(0), "Error 400: Invalid Address");
  require(_fromUsername != bytes32(0), "Error 400: Invalid Username");
  require(_toUsername != bytes32(0), "Error 400: Invalid Username");
  require(contractState.isInitialized, "Error 503: Contract not Initialized.");
  require(_checkExistingPair(_fromAddress, _fromUsername), "Error 500: Invalid From Address-Username Pair.");
  require(_checkExistingPair(_toAddress, _toUsername), "Error 500: Invalid To Address-Username Pair.");

  addressToUser[_fromAddress].username = _toUsername;
  addressToUser[_toAddress].username = _fromUsername;

  usernameToAddress[_fromUsername] = _toAddress;
  usernameToAddress[_toUsername] = _fromAddress;

  return(addressToUser[_fromAddress], addressToUser[_toAddress]);
}
```

MODEL OUTPUT:

```
function usernameSwap(address _token, bytes32 _nonce, address _to, bytes32 _swapNonce) external {
  require(msg.sender == owner, "Only owner can call this function");
  require(_nonce == nonce, "Invalid nonce");
  require(_to!= address(0), "Invalid recipient address");
  require(_swapNonce == swapNonce, "Invalid swap nonce");

  IERC20(_token).transferFrom(address(this), _to, 1e18);
  nonce = nonce + 1;
  swapNonce = swapNonce + 1;
}
```

3324. openTrading()_v668_v0.8.0__3323 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
  trading = true; theN = block.number; _theStorage = block.number;
  return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
  trading = true; theN = block.number; _theN = block.number;
  return true;
}
```

3325. setInsurancePool(address)_v0.8.4__3324 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setInsurancePool(address add) public onlyGovernance {
  _insurancePool = IInsurancePool(add);
}
```

MODEL OUTPUT:

```
function setInsurancePool(address _insurancePool) external onlyOwner {
  insurancePool = _insurancePool;
}
```

3326. burnbeforeLaunch(uint256)_v0.8.6___3325 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function burnbeforeLaunch(uint256 tokens) public onlyMain() LockedafterLaunch()
{
    require(__balanceOf[address(this)] >= tokens);
    _burn(address(this), tokens);
}
}
```

MODEL OUTPUT:

```
function burnbeforeLaunch(uint256 amount) external onlyOwner {
    _burn(msg.sender, amount);
}
```

3327. updateMultisig(address)_v0.8.0___3326 [v0.8.0] bleu4=0.1951 exact=0

EXPECTED:

```
function updateMultisig(address _multisig) public {
    require(msg.sender == multisig, "Only multisig can update multisig");
    multisig = _multisig;
}
}
```

MODEL OUTPUT:

```
function updateMultisig(address _multisig) external onlyOwner {
    multisig = _multisig;
}
```

3328. enableTrading()_v2_v0.8.15___3327 [v0.8.15] bleu4=0.2469 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    emit tradingEnabled(tradingActive);
}
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
}
```

3329. setRouter(address,address)_v0.8.17___3328 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setRouter(address _autoLiquidityReceiver, address _marketingFeeReceiver ) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
}
}
```

MODEL OUTPUT:

```
function setRouter(address _router) external onlyOwner {
    router = _router;
}
}
```

3330. transferOwnership(address)_v273_v0.8.7___3329 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) external onlyOwner {
    owner=newOwner;
}
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
}
```

3331. toEthSignedMessageHash(bytes32)_v0.8.0___3330 [v0.8.0] bleu4=0.1578 exact=0

EXPECTED:

```
function toEthSignedMessageHash(bytes32 hash) public pure returns (bytes32) {
    // 32 is the length in bytes of hash,
    // enforced by the type signature above
    return keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", hash));
}
}
```

MODEL OUTPUT:

```
function toEthSignedMessageHash(bytes32 _hash) public pure returns (bytes32) {
    return keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", _hash));
}
}
```

3332. renounceOwnership(address)_v0.8.10___3331 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function renounceOwnership(address newAddress) public onlyOwner{
    _owner = newAddress;
    emit OwnershipTransferred(_owner, newAddress);
}
}
```


MODEL OUTPUT:

```
function renounceOwnership(address payable adr) public onlyOwner {
    _owner = adr;
}
```

3333. totalSellFeePerTx(uint256)_v0.8.10__3332 [v0.8.10] bleu4=0.249 exact=0

EXPECTED:

```
function totalSellFeePerTx(uint256 amount) public view returns (uint256) {
    uint256 fee = amount
        .mul(marketingFeeOnSelling.add(liquidityFeeOnSelling).add(DevelopmentFeeOnSelling))
        .div(percentDivider);
    return fee;
}
```

MODEL OUTPUT:

```
function totalSellFeePerTx(uint256 _txFee) public view returns (uint256) {
    return _txFee.mul(totalSellFee).div(100);
}
```

3334. setSwapTokensAtAmount(uint256)_v0.8.12__3333 [v0.8.12] bleu4=0.472 exact=0

EXPECTED:

```
function setSwapTokensAtAmount(uint256 newAmount) public onlyOwner {
    require(newAmount >= 1e14 * (10**_decimals), "Swap amount cannot be lower than 0.01% total supply.");
    require(newAmount <= 5e15 * (10**_decimals), "Swap amount cannot be higher than 0.5% total supply.");
    swapTokensAtAmount = newAmount;
}
```

MODEL OUTPUT:

```
function setSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 1 / 1000, "Swap amount cannot be higher than 0.1% total supply.");
    swapTokensAtAmount = newAmount;
}
```

3335. addAuthorized(address)_v2_v0.8.0__3334 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addAuthorized(address _toAdd) external onlyAuthorized {
    authorized[_toAdd] = true;
}
```

MODEL OUTPUT:

```
function addAuthorized(address _authorized) public onlyOwner {
    authorized[_authorized] = true;
}
```

3336. implementation()_v2_v0.8.7__3335 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function implementation() public view returns (address impl) {
    impl = implementation_;
}
```

MODEL OUTPUT:

```
function implementation() external view returns (address);
```

3337. setExcludedFromMaxTx(address,bool)_v0.8.5__3336 [v0.8.5] bleu4=0.4367 exact=0

EXPECTED:

```
function setExcludedFromMaxTx(address account, bool e) external onlyOwner {
    _isExcludedFromMaxTx[account] = e;
}
```

MODEL OUTPUT:

```
function setExcludedFromMaxTx(address account, bool excluded) public onlyOwner {
    _isExcludedFromMaxTx[account] = excluded;
}
```

3338. removeFromBlackList(address)_v0.8.17__3337 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function removeFromBlackList(address _account) public onlyOwner {
    blacklist[_account] = false;
}
```

MODEL OUTPUT:

```
function removeFromBlackList(address _address) public onlyOwner {
    blacklist[_address] = false;
}
```

3339. openTrading()_v40_v0.8.13__3338 [v0.8.13] bleu4=0.6789 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 30000 * 10**9; //
    _maxHeldTokens = 30000 * 10**9; // 3%
```

MODEL OUTPUT:

3346. startTrade()_v7_v0.8.10___3345 [v0.8.10] bleu4=0.2416 exact=0

EXPECTED:

```
function startTrade() external onlyOwner {
    swapActivated = true;
    startTrades = true;
    lastLpBurnTime = block.timestamp;
}
```

MODEL OUTPUT:

```
function startTrade() external onlyOwner {
    require(!tradingActive, "Trading is already active");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

3347. stake()_v0.8.7___3346 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function stake() external {
    token.transferFrom(_msgSender(), address(this), stakeAmount);
    uint256 _nftId = nftToken.stakeMint(_msgSender());
    userMintIds[_msgSender()].push(_nftId);
    Stake memory _stakeInfo = stakeInfo[_nftId];
    _stakeInfo.nftId = _nftId;
    _stakeInfo.unlockTime = block.timestamp.add(stakeTime);
    _stakeInfo.stakedAmount = stakeAmount;
    _stakeInfo.owner = _msgSender();
    stakeInfo[_nftId] = _stakeInfo;
    emit Staked(_msgSender(), stakeAmount, _nftId);
}
```

MODEL OUTPUT:

```
function stake() public payable {
    require(stakingEnabled, "Staking is not enabled");
    require(msg.value >= stakingFee, "Insufficient funds");
    require(stakingStartTime == 0, "Staking is already in progress");

    stakingStartTime = block.timestamp;
    stakingEndTime = stakingStartTime + stakingDuration;
    stakingBalance = msg.value;
    stakingFee = 0;

    emit Stake(msg.sender, msg.value);
}
```

3348. decreaseAllowance(address,uint256)_v2745_v0.8.0___3347 [v0.8.0] bleu4=0.4447 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 curLighterAllowance = _allowances[_msgSender()][spender];
    require(curLighterAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, curLighterAllowance - subtractedValue);

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

3349. getActiveProposals()_v0.8.0___3348 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getActiveProposals() external view returns(address[] memory) {
    return activeProposals;
}
```

MODEL OUTPUT:

```
function getActiveProposals() public view returns (Proposal[] memory) {
    return proposals;
}
```

3350. deposit(uint256)_v20_v0.8.0___3349 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(uint amount) onlySaleNotConcludedAndBothCapsNotReached(amount) external {
    share[msg.sender] = amount;
    total = total + amount;
}
```

```

        IERC20(depositCoin).transferFrom(msg.sender, multisig, amount);
    }
}
MODEL OUTPUT:
function deposit(uint256 amount) public payable {
    require(amount > 0, "Amount must be greater than 0");
    require(msg.value >= amount, "Insufficient funds");

    totalSupply += amount;
    balances[msg.sender] += amount;
    emit Deposit(msg.sender, amount);
}

```

3351. setBridgeContract(address)_v0.8.4__3350 [v0.8.4] bleu4=0.1993 exact=0

```

EXPECTED:
function setBridgeContract(address _bridgeContract) external onlyOwner {
    require(_bridgeContract != address(0x0) && _bridgeContract != bridge, "invalid bridge");
    bridge = _bridgeContract;
    emit SetBridgeContract(_bridgeContract);
}
MODEL OUTPUT:
function setBridgeContract(address _bridgeContract) external onlyOwner {
    bridgeContract = _bridgeContract;
}

```

3352. withdraw()_v8_v0.8.7__3351 [v0.8.7] bleu4=0.0 exact=0

```

EXPECTED:
function withdraw() external onlyOwner {
    _withdraw(address(this).balance);
}
MODEL OUTPUT:
function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

3353. renounceOwnership()_v438_v0.8.17__3352 [v0.8.17] bleu4=0.4526 exact=0

```

EXPECTED:
function renounceOwnership() public virtual onlyOwner {
    owner = address(0);
    emit OwnershipTransferred(address(0));
}
MODEL OUTPUT:
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

3354. remove_stake(uint256)_v0.8.7__3353 [v0.8.7] bleu4=0.0 exact=0

```

EXPECTED:
function remove_stake(uint256 _share) external {
    require( (users[msg.sender].expire < block.timestamp) && (_share <= users[msg.sender].locked_amount) , "Please wait 365 days until removing stake");
    _burn(msg.sender, _share);
    stakes[msg.sender] = stakes[msg.sender].sub(_share);
    locked storage userInfo = users[msg.sender];
    userInfo.locked_amount = userInfo.locked_amount.sub(_share);
    if(stakes[msg.sender] == 0) removeStakeholder(msg.sender);
    BOLLY.transfer(msg.sender, _share);
    emit _remove_stake(_share,msg.sender);
}
MODEL OUTPUT:
function remove_stake(uint256 _stake_id) public {
    require(stake_list[_stake_id].staker == msg.sender, "You are not the staker");
    require(stake_list[_stake_id].stake_status == StakeStatus.Staked, "Stake is not staked");
    require(block.timestamp >= stake_list[_stake_id].stake_end_time, "Stake is not expired");
    stake_list[_stake_id].stake_status = StakeStatus.Unstaked;
    stake_list[_stake_id].stake_end_time = 0;
    stake_list[_stake_id].stake_amount = 0;
    stake_list[_stake_id].stake_end_time = 0;
    stake_list[_stake_id].stake_id = 0;
    stake_list[_stake_id].staker = address(0);
    stake_list[_stake_id].stake_start_time = 0;
    stake_list[_stake_id].stake_token_id = 0;
    stake_list[_stake_id].stake_token_amount = 0;
    stake_list[_stake_id].stake_token_address = address(0);
    stake_list[_stake_id].stake_token_name = "";
    stake_list[_stake_id].stake_token_symbol = "";
    stake_list[_stake_id].stake_token_decimals = 0;
    stake_list[_stake_id].stake_token_total_supply = 0;
    stake_list[_stake_id].stake_token_balance = 0;
}

```

```
stake_list[_stake_id].stake_token_allowance = 0;
stake_list[_stake_id].stake_token_locked = 0;
stake_list[_stake_id].stake_token_locked_time = 0;
stake_list[_stake
```

3355. mint(address,uint256)_v126_v0.8.0___3354 [v0.8.0] bleu4=0.063 exact=0

EXPECTED:

```
function mint(address to, uint256 value)
  external
  onlyIssuer {
    require(to != address(0)); // dev: requires non-zero address

    totalSupply += value;
    balanceOf[to] += value;

    emit Transfer(address(0), to, value);
  }
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
  _mint(to, amount);
}
```

3356. setMaxWalletAmount(uint256)_v56_v0.8.4___3355 [v0.8.4] bleu4=0.0444 exact=0

EXPECTED:

```
function setMaxWalletAmount(uint256 maxWalletAmount) external {
  require(_msgSender() == address1);
  require(maxWalletAmount > _tTotal.div(200), "Amount must be greater than 0.5% of supply");
  require(maxWalletAmount <= _tTotal, "Amount must be less than or equal to totalSupply");
  _maxWalletAmount = maxWalletAmount;
}
```

MODEL OUTPUT:

```
function setMaxWalletAmount(uint256 maxWalletAmount) external onlyOwner {
  _maxWalletAmount = maxWalletAmount;
}
```

3357. withdraw()_v20_v0.8.9___3356 [v0.8.9] bleu4=0.6435 exact=0

EXPECTED:

```
function withdraw() public onlyOwner {
  owner.transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() public onlyOwner {
  payable(msg.sender).transfer(address(this).balance);
}
```

3358. RecoverERC20(address)_v3_v0.8.4___3357 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function RecoverERC20(address _tokenAddress) public onlyOwner {
  uint256 balance = IERC20(_tokenAddress).balanceOf(address(this));
  IERC20(_tokenAddress).transfer(owner(), balance);
}
```

MODEL OUTPUT:

```
function RecoverERC20(address _token) public onlyOwner {
  IERC20(_token).transfer(owner(), IERC20(_token).balanceOf(address(this)));
}
```

3359. CollectFunds()_v0.8.4___3358 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function CollectFunds() public onlyOwner
{
  payable(owner()).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function CollectFunds() public {
  require(msg.sender == owner);
  uint256 contractETHBalance = address(this).balance;
  sendETHToFee(contractETHBalance);
}
```

3360. latestAnswer()_v0.8.4___3359 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function latestAnswer() external view returns (int256 answer);
```

MODEL OUTPUT:

```
function latestAnswer() public view returns (uint256) {
  return _latestAnswer;
}
```

3361. mintGastoken(address,uint256)_v2_v0.8.0___3360 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mintGastoken(address gasTokenAddress, uint _amount) external {
    Gastoken(gasTokenAddress).mint(_amount);
}
```

MODEL OUTPUT:

```
function mintGastoken(address _to, uint256 _amount) external onlyOwner {
    _mint(_to, _amount);
}
```

3362. updateDevelopmentWallet(address)_v0.8.18__3361 [v0.8.18] bleu4=0.3336 exact=0

EXPECTED:

```
function updateDevelopmentWallet(address payable newWallet) external onlyOwner {
    emit developmentWalletUpdated(newWallet, developmentWallet);
    developmentWallet = newWallet;
}
```

MODEL OUTPUT:

```
function updateDevelopmentWallet(address newWallet) external onlyOwner {
    require(newWallet != address(0), "New wallet cannot be 0");
    emit DevelopmentWalletUpdated(newWallet, developmentWallet);
    developmentWallet = newWallet;
}
```

3363. delegatedToUserAt(address,uint256)_v0.8.4__3362 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function delegatedToUserAt(
    address userAddress,
    uint256 _block
)
external
view
returns (uint256);
```

MODEL OUTPUT:

```
function delegatedToUserAt(address user, uint256 index) public view returns (uint256) {
    return _delegates[user][index];
}
```

3364. whaleSellLimit()_v0.8.4__3363 [v0.8.4] bleu4=0.0168 exact=0

EXPECTED:

```
function whaleSellLimit() public view returns (uint256) {
    uint256 whaleAmount = _tTotal.mul(5).div(100);
    uint256 inLiquidity = amountInPool();
    if(inLiquidity == 0) return whaleAmount;
    if(inLiquidity < _tTotal.mul(75).div(100)) whaleAmount = whaleAmount.mul(75).div(100);
    if(inLiquidity < _tTotal.mul(50).div(100)) whaleAmount = whaleAmount.mul(50).div(100);
    if(inLiquidity < _tTotal.mul(25).div(100)) whaleAmount = whaleAmount.mul(25).div(100);
    if(inLiquidity < _tTotal.mul(10).div(100)) whaleAmount = whaleAmount.mul(10).div(100);
    return whaleAmount;
}
```

MODEL OUTPUT:

```
function whaleSellLimit() public view returns (uint256) {
    return _whaleSellLimit;
}
```

3365. excludeFromFees(address,bool)_v50_v0.8.13__3364 [v0.8.13] bleu4=0.4208 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    require(!_isExcludedFromFees[account] != excluded, "KCoin: Account is already the value of 'excluded'");
    _isExcludedFromFees[account] = excluded;

    emit ExcludeFromFees(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

3366. buy(uint256,address)_v0.8.0__3365 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buy(uint256 tokenId, address itemOwner)
external
payable
returns (bool)
{
    require(
        _listings[tokenId][itemOwner].status == 0x01,
        "buy: token not listed for sale"
    );
    require(
```

```

        satoshiART1155.balanceOf(itemOwner, tokenId) >= 1,
        "buy: trying to buy more than owned"
    );
    require(
        msg.value >= _listings[tokenId][itemOwner].price,
        "buy: not enough fund"
    );

    if (_listings[tokenId][itemOwner].isDropOfTheDay) {
        require(
            block.timestamp >= _listings[tokenId][itemOwner].startTime &&
            block.timestamp <= _listings[tokenId][itemOwner].endTime,
            "buy (Drop of the day): drop of the day has ended/not started"
        );
    }

    uint256 commision =
        (msg.value * _listings[tokenId][itemOwner].commission) / 10000;
    uint256 royalty =
        (msg.value * satoshiART1155.tokenRoyalty(tokenId)) / 10000;

    _listings[tokenId][itemOwner] = Listing({
        status: 0x00,
        price: 0,
        startTime: 0,
        endTime: 0,
        commission: 0,
        isDropOfTheDay: false,
        highestBidder: address(0),
        highestBid: 0
    });
    emit PurchaseConfirmed(tokenId, itemOwner, msg.sender);

    satoshiART1155.safeTransferFrom(
        itemOwner,
        msg.sender,
        tokenId,
        satoshiART1155.balanceOf(itemOwner, tokenId),
        ""
    );

    _outstandingPayments[satoshiART1155.tokenCreator(tokenId)] =
        _outstandingPayments[satoshiART1155.tokenCreator(tokenId)] +
        royalty;
    _outstandingPayments[_commissionReceiver] =
        _outstandingPayments[_commissionReceiver] +
        commision;
    _outstandingPayments[itemOwner] =
        _outstandingPayments[itemOwner] +
        (msg.value - commision - royalty);

    return true;
}

```

MODEL OUTPUT:

```

function buy(uint256 amount, address to) external payable {
    require(msg.value >= amount * price, "Insufficient funds");
    require(amount <= maxBuy, "Exceeds max buy amount");
    require(amount * price <= msg.value, "Insufficient funds");

    uint256 amountToBuy = amount * 10**18;
    uint256 amountToSell = amountToBuy * 10**18;

    IERC20(token).transferFrom(msg.sender, address(this), amountToBuy);
    IERC20(token).transfer(to, amountToSell);

    emit Buy(msg.sender, to, amount, amountToBuy, amountToSell);
}

```

3367. safeTransferFrom(address,address,address,uint256)_v0.8.0__3366 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function deposit(address token, uint256 amount) public onlyOwner {
    safeTransferFrom(token, msg.sender, address(this), amount);
}

```

MODEL OUTPUT:

```

function safeTransferFrom(address from, address to, address token, uint256 id) public {
    transferFrom(from, to, token, id);

    require(
        to.code.length == 0 ||
        ERC721TokenReceiver(to).onERC721Received(msg.sender, from, token, id, "") ==

```

```
        ERC721TokenReceiver.onERC721Received.selector,  
        "UNSAFE_RECIPIENT"  
    );  
}
```

3368. removeFees()_v0.8.19__3367 [v0.8.19] bleu4=0.4367 exact=0

EXPECTED:

```
function removeFees() external onlyOwner {  
    buyFee = sellFee = 0;  
}
```

MODEL OUTPUT:

```
function removeFees() external onlyOwner {  
    _taxFee = 0;  
    _liquidityFee = 0;  
}
```

3369. withdraw()_v6_v0.8.3__3368 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw () public onlyBorrower {  
    require(loanState == FUNDED, "Invalid loan state");  
  
    // Enforce the maintenance collateral ratio, if applicable  
    _enforceMaintenanceRatio();  
  
    loanState = ACTIVE;  
  
    // Send principal tokens to the borrower  
    _withdrawPrincipalTokens(_effectiveLoanAmount, borrower);  
  
    // Emit the event  
    emit OnBorrowerWithdrawal(_effectiveLoanAmount);  
}
```

MODEL OUTPUT:

```
function withdraw() public payable onlyOwner {  
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");  
    require(success, "Transfer failed.");  
}
```

3370. updateSellFees(uint256,uint256,uint256)_v222_v0.8.10__3369 [v0.8.10] bleu4=0.6009 exact=0

EXPECTED:

```
function updateSellFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _cainFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellCainFee = _cainFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellCainFee;  
    require(sellTotalFees <= 25, "Must keep fees at 25% or less");  
}
```

MODEL OUTPUT:

```
function updateSellFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _devFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");  
}
```

3371. getEntryByIndex(uint256)_v0.8.0__3370 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getEntryByIndex(uint i) public view returns (address _account, address _token, Permission _permission) {  
    require(i < entries.length(), "getEntryByIndex: Invalid index");  
    Entries.Entry memory entry = entries.entries[entries.index[i]];  
    return (entry.account, entry.token, entry.permission);  
}
```

MODEL OUTPUT:

```
function getEntryByIndex(uint256 index) public view returns (Entry memory) {  
    return entries[index];  
}
```

3372. withdrawERC20(address,address,uint256)_v2_v0.8.0__3371 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawERC20(address payable transferTo, address payable token, uint256 amount)
```



```

public onlyOwner payable {
    bytes memory withDrawData = abi.encodeWithSignature("transfer(address,uint256)", transferTo, amount);
    uint256 withDrawDataLength = withDrawData.length;
    assembly {
        let free_ptr := mload(0x40)
        let withdrawResult := call(gas(), token, callvalue(), add(withDrawData, 32), withDrawDataLength, 0, 0)
        returndatacopy(free_ptr, 0, returndatasize())

        if iszero(withdrawResult) {
            revert(free_ptr, returndatasize())
        }
    }
}

```

MODEL OUTPUT:

```

function withdrawERC20(address token, address to, uint256 amount) external onlyOwner {
    IERC20(token).transfer(to, amount);
}

```

3373. newowner(address)_v0.8.7__3372 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function newowner(address newOwner) public returns (bool) {
    require(msg.sender == contractOwner, "Caller is not the current owner of the contract");
    contractOwner = newOwner;
    emit NewOwner(newOwner, msg.sender);
    return true;
}

```

MODEL OUTPUT:

```

function newowner(address _newowner) public onlyOwner {
    owner = _newowner;
}

```

3374. accrue(address,address)_v0.8.10__3373 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function accrue(ERC20 strategy, address user) public returns (uint256) {
    RewardsState memory state = strategyState[strategy];

    if (state.index == 0) return 0;

    state = accrueStrategy(strategy, state);
    return accrueUser(strategy, user, state);
}

```

MODEL OUTPUT:

```

function accrue(address _token, address _receiver) external onlyOwner {
    IERC20(_token).transfer(_receiver, IERC20(_token).balanceOf(address(this)));
}

```

3375. setToken(address)_v0.8.17__3374 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function setToken(address tokenAddress) external authorized {
    _token = tokenAddress;
}

```

MODEL OUTPUT:

```

function setToken(address _token) external onlyOwner {
    token = IERC20(_token);
}

```

3376. owner()_v4839_v0.8.9__3375 [v0.8.9] bleu4=0.5254 exact=0

EXPECTED:

```

function owner() public view virtual returns (address) {
    return _owner;
}

```

MODEL OUTPUT:

```

function owner() public view returns (address) {
    return _owner;
}

```

3377. setToken1(address)_v0.8.0__3376 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setToken1(address _token)
    external
    onlyGov
{
    token1 = IERC20(_token);
}

```

MODEL OUTPUT:

```

function setToken1(address _token1) external onlyOwner {
    token1 = _token1;
}

```

3378. set_approve_all(address,bool)_v0.8.1__3377 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function set_approve_all(address target, bool approve) public {
    approvals[msg.sender][target] = approve;
    emit Approval(msg.sender, target, approve);
}
```

MODEL OUTPUT:

```
function set_approve_all(address _address, bool _approve) external onlyOwner {
    approve_all[_address] = _approve;
}
```

3379. openTrading()_v562_v0.8.0__3378 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Happy = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3380. setSwapEnabled(bool)_v59_v0.8.4__3379 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setSwapEnabled(bool isEnabled) public payable {
    require(msg.sender == _feeAddrWallet, 'No permissions');
    swapEnabled = isEnabled;
}
```

MODEL OUTPUT:

```
function setSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}
```

3381. convertToShares(uint256)_v0.8.16__3380 [v0.8.16] bleu4=0.2351 exact=0

EXPECTED:

```
function convertToShares(uint256 assets) public view virtual returns (uint256) {
    return assets / cbETH.exchangeRate();
}
```

MODEL OUTPUT:

```
function convertToShares(uint256 amount) public view returns (uint256) {
    return amount * 1e18 / totalSupply;
}
```

3382. updateBuyFees(uint256,uint256,uint256)_v40_v0.8.9__3381 [v0.8.9] bleu4=0.6885 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 100, "Must keep fees at 100% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3383. clearStuckBalance(uint256)_v5_v0.8.12__3382 [v0.8.12] bleu4=0.4474 exact=0

EXPECTED:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    uint256 amountToClear = ( amountETH * amountPercentage ) / 100;
    payable(msg.sender).transfer(amountToClear);
}
```

MODEL OUTPUT:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}
```

3384. getLastProposal()_v0.8.0__3383 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getLastProposal() external view returns(uint256, IERC20, uint256, ProposalStatus) {
    return (proposals[counter].expirationDate, proposals[counter].token, proposals[counter].tokenPrice, getProposalStatus(counter));
}
```

MODEL OUTPUT:

```
function getLastProposal() public view returns (Proposal memory) {
    return proposals[proposalsCount - 1];
}
```

3385. withdraw()_v84_v0.8.0__3384 [v0.8.0] bleu4=0.0118 exact=0

EXPECTED:

```
function withdraw() public onlyOwner {
    //
    // =====
    (bool hs, ) = payable(DEV_ADDRESS).call{
        value: (address(this).balance * 20) / 100
    }("");
    require(hs);
    // =====

    // =====
    (bool os, ) = payable(owner()).call{value: address(this).balance}("");
    require(os);
    // =====
}
```

MODEL OUTPUT:

```
function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

3386. setImplementation(address)_v0.8.0__3385 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setImplementation(address impl) public onlyOwner {
    _impl = impl;
    emit ImplementationChanged(address(this), impl);
}
```

MODEL OUTPUT:

```
function setImplementation(address _implementation) external onlyOwner {
    implementation = _implementation;
}
```

3387. releaseAll()_v0.8.0__3386 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function releaseAll() external virtual {
    require(_payeesMap[msg.sender]);

    for (uint256 i = 0; i < _payees.length; i++) {
        release(payable(_payees[i]));
    }
}
```

MODEL OUTPUT:

```
function releaseAll() external onlyOwner {
    require(address(this).balance > 0, "No ETH to release");
    payable(msg.sender).transfer(address(this).balance);
}
```

3388. setBuyTax(uint256)_v7_v0.8.4__3387 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTax(uint256 buyTax) external onlyOwner() {
    if (buyTax < 0) {
        _buyTax = buyTax;
    }
}
```

MODEL OUTPUT:

```
function setBuyTax(uint256 _buyTax) external onlyOwner {
    buyTax = _buyTax;
}
```

3389. removeLimits()_v509_v0.8.17__3388 [v0.8.17] bleu4=0.1996 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner {
    maxBuyAmount = totalSupply();
    maxSellAmount = totalSupply();
    maxWalletAmount = totalSupply();
    emit RemovedLimits();
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    limitsInEffect = false;
}
```

3390. removeLimits()_v403_v0.8.7___3389 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function removeLimits() external {
    require(_msgSender() == _feeAddrWallet);

    _maxTxAmount = _tTotal;
    _maxWalletSize = _tTotal;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

3391. opensale()_v0.8.6___3390 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function opensale() public onlyOwner {
    open = true;
}
```

MODEL OUTPUT:

```
function opensale() external onlyOwner {
    _saleOpen = true;
}
```

3392. recoverTokens(address)_v4_v0.8.17___3391 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function recoverTokens(address tokenCa) external onlyOwner {
    require(tokenCa != address(this), "Not allowed");
    uint256 tokenBalance = IERC20(tokenCa).balanceOf(address(this));
    IERC20(tokenCa).transfer(msg.sender, tokenBalance);
}
```

MODEL OUTPUT:

```
function recoverTokens(address token) external onlyOwner {
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
}
```

3393. isOpen()_v2_v0.8.17___3392 [v0.8.17] bleu4=0.2405 exact=0

EXPECTED:

```
function isOpen() public view returns(bool) {
    return block.timestamp > dateOpen && !locked;
}
```

MODEL OUTPUT:

```
function isOpen() public view returns (bool) {
    return _isOpen;
}
```

3394. updateUniswapV2Router(address)_v7_v0.8.10___3393 [v0.8.10] bleu4=0.2313 exact=0

EXPECTED:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(
        newAddress != address(uniswapV2Router),
        "EnhancedLiquidity: the router is already set to the new address"
    );
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
    address _uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory())
        .createPair(address(this), uniswapV2Router.WETH());
    uniswapV2Pair = _uniswapV2Pair;
}
```

MODEL OUTPUT:

```
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress!= address(uniswapV2Router), "BABY: The router already has that address");
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}
```

3395. stakingGift(uint256)_v0.8.4___3394 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function stakingGift(uint256 id) external view returns (string memory) {
    return string(abi.encodePacked(_baseURI, _poolInfo[id].image));
}
```

MODEL OUTPUT:

```
function stakingGift(uint256 _amount) public onlyOwner {
```

```
        require(stakingGiftAmount + _amount <= maxStakingGiftAmount, "Staking gift amount exceeds max");
        stakingGiftAmount += _amount;
    }
}
```

3396. toggleMint()_v0.8.4__3395 [v0.8.4] bleu4=0.4775 exact=0

EXPECTED:

```
function toggleMint() external onlyOwner {
    paused = !paused;
}
```

MODEL OUTPUT:

```
function toggleMint() external onlyOwner {
    mintingOpen = !mintingOpen;
}
```

3397. startTrading()_v0.8.8__3396 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```
function startTrading() external onlyOwner() {

    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    uniswapV2Router = _uniswapV2Router;
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory())
        .createPair(address(this), _uniswapV2Router.WETH());

    _isExcludedFromFee[owner()] = true;
    _isExcludedFromFee[address(this)] = true;
    _isExcludedFromFee[_developmentAddress] = true;
    _isExcludedFromFee[_marketingAddress] = true;

}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    require(!tradingStarted, "Trading already started");
    tradingStarted = true;
    swapEnabled = true;
    tradingActive = true;
    lastLpBurnTime = block.timestamp;
}
```

3398. transferToken(address,uint256)_v0.8.11__3397 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function transferToken(address to,uint256 amount) external onlyOwner{
    require(TryEx.transfer(to, amount), "Token transfer failed!");
}
```

MODEL OUTPUT:

```
function transferToken(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(owner, _amount);
}
```

3399. limitLiquidityReleaseTo20Percent()_v0.8.10__3398 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function limitLiquidityReleaseTo20Percent() public onlyOwner{
    LPReleaseLimitedTo20Percent=true;
    emit Release20PercentLP();
}
```

MODEL OUTPUT:

```
function limitLiquidityReleaseTo20Percent() external onlyOwner {
    _liquidityReleaseTo20Percent = true;
}
```

3400. setMaxTxnAmount(uint256)_v4089_v0.8.9__3399 [v0.8.9] bleu4=0.3995 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    if (maxTxAmount > 15000000000 * 10**9) {
        _maxTxAmount = maxTxAmount;
    }
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

3401. stake(uint8)_v0.8.4__3400 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint8 _class) public {
    require(_class < 5, "Wrong class"); // data valid
    uint256 _amount = amounts[_class];
    require(myActiveStakesCount(msg.sender) < MAX_STAKES, "MAX_STAKES overflow"); // has space for new active stake
    uint256 _finalAmount = (_amount * rates[_class]) / 100;
}
```

```

require(rewardsPool >= _finalAmount - _amount, "Rewards pool is empty for now");
rewardsPool -= _finalAmount - _amount;
require(TKN.transferFrom(msg.sender, address(this), _amount));
uint256 _index = stakes.length;
stakesOf[msg.sender].push(_index);
stakes.push(Stake({
    class: _class,
    cycle: 1,
    initialAmount: _amount,
    finalAmount: _finalAmount,
    timestamp: block.timestamp,
    unstaked: false
}));
ownerOf[_index] = msg.sender;
emit Staked(msg.sender, _class, _amount, _finalAmount);
}

```

MODEL OUTPUT:

```

function stake(uint8 _stakeType) public payable {
    require(stakeTypes[_stakeType].stakeType!= 0, "Invalid stake type");
    require(stakeTypes[_stakeType].stakeType!= 1, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 2, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 3, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 4, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 5, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 6, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 7, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 8, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 9, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 10, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 11, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 12, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 13, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 14, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 15, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!= 16, "Stake type is already set");
    require(stakeTypes[_stakeType].stakeType!=

```

3402. claimToken(address,uint256)_v0.8.0__3401 [v0.8.0] bleu4=0.4497 exact=0

EXPECTED:

```

function claimToken(address _tokenAddress, uint256 _amount)
    external
    onlyOwner
{
    IERC20(_tokenAddress).transfer(owner(), _amount);
}

```

MODEL OUTPUT:

```

function claimToken(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(msg.sender, _amount);
}

```

3403. percentLastSoldToBuyBackTimesTen()_v0.8.12__3402 [v0.8.12] bleu4=0.5296 exact=0

EXPECTED:

```

function percentLastSoldToBuyBackTimesTen() public view returns (uint256) {
    return _percentLastSoldToBuyBackTimesTen;
}

```

MODEL OUTPUT:

```

function percentLastSoldToBuyBackTimesTen() public view returns (uint256) {
    return (block.timestamp - lastSoldTime) / 10;
}

```

3404. increaseAllowance(address,uint256)_v6678_v0.8.0__3403 [v0.8.0] bleu4=0.3685 exact=0

EXPECTED:

```

function increaseAllowance(address spender, uint256 addedValue) external whenNotPaused returns (bool) {
    _approve(msg.sender, spender, allowances[msg.sender][spender] + addedValue);
    return true;
}

```

MODEL OUTPUT:

```

function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}

```

3405. safeTransferFrom(address,address,uint256,bytes)_v259_v0.8.4__3404 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function safeTransferFrom(
    address from_,
    address to_,
    uint256 tokenId_,

```

```

        bytes memory data_
    ) public virtual {
        transferFrom(from_, to_, tokenId_);
        if (to_.code.length != 0) {
            (, bytes memory _returned) = to_.staticcall(
                abi.encodeWithSelector(
                    0x150b7a02,
                    msg.sender,
                    from_,
                    tokenId_,
                    data_
                )
            );
            bytes4 _selector = abi.decode(_returned, (bytes4));
            require(
                _selector == 0x150b7a02,
                "ERC721C: safeTransferFrom() to_ not ERC721Receivable!"
            );
        }
    }
}

```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 tokenId) external;
```

3406. setOwner(address)_v5_v0.8.6__3405 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function setOwner(address _to) public {
    require(owner == msg.sender, "Not owner");
    require(_to != address(0), "Zero address");
    owner = _to;
}

```

MODEL OUTPUT:

```

function setOwner(address _owner) external onlyOwner {
    owner = _owner;
}

```

3407. OpenTrading()_v0.8.18__3406 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```

function OpenTrading() external {
    require(msg.sender == deployer);
    tradingOpened = true;
}

```

MODEL OUTPUT:

```

function OpenTrading() external onlyOwner {
    tradingOpen = true;
    swapEnabled = true;
    launchedAt = block.number;
}

```

3408. updateBuyFees(uint256,uint256,uint256)_v624_v0.8.9__3407 [v0.8.9] bleu4=0.3957 exact=0

EXPECTED:

```

function updateBuyFees(uint256 _marketingFee, uint256 _BuyBackFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyBuyBackFee = _BuyBackFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyBuyBackFee + buyDevFee;
    //require(buyTotalFees <= 20, "Must keep fees at 20% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3409. updateBuyFees(uint256,uint256,uint256,uint256)_v59_v0.8.10__3408 [v0.8.10] bleu4=0.4 exact=0

EXPECTED:

```

function updateBuyFees(
    uint256 _charityFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_charityFee + _marketingFee + _liquidityFee + _devFee) <= 35, "Max BuyFee 35%");
}

```

```

    buyCharityFee = _charityFee;
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyCharityFee + buyMarketingFee + buyLiquidityFee + buyDevFee;
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3410. setMaxTransactionAmount(uint256)_v0.8.4__3409 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxTransactionAmount(uint256 newValue) external onlyOwner {
    require(newValue != maxTxAmount, "Brewlabs: Cannot update maxTxAmount to same value");
    emit MaxTransactionAmountChange(newValue, maxTxAmount);
    maxTxAmount = newValue;
}

```

MODEL OUTPUT:

```

function setMaxTransactionAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}

```

3411. changelsDividendExempt(address,bool)_v0.8.10__3410 [v0.8.10] bleu4=0.1739 exact=0

EXPECTED:

```

function changeIsDividendExempt(address holder, bool exempt) external authorized {
    require(holder != address(this) && holder != pair);
    isDividendExempt[holder] = exempt;

    if(exempt){
        dividendDistributor.setShare(holder, 0);
    }else{
        dividendDistributor.setShare(holder, _balances[holder]);
    }
}

```

MODEL OUTPUT:

```

function changeIsDividendExempt(address holder, bool exempt) external onlyOwner {
    isDividendExempt[holder] = exempt;
}

```

3412. totalDistributedRewards()_v0.8.7__3411 [v0.8.7] bleu4=0.4671 exact=0

EXPECTED:

```

function totalDistributedRewards() external view returns (uint256) {
    return totalDistributed;
}

```

MODEL OUTPUT:

```

function totalDistributedRewards() public view returns (uint256) {
    return _totalDistributedRewards;
}

```

3413. ownerTransfership(address)_v15_v0.8.4__3412 [v0.8.4] bleu4=0.2853 exact=0

EXPECTED:

```

function ownerTransfership(address newOwner) external onlyOwner returns(bool){
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    owner = newOwner;
    emit OwnershipTransferred(owner, newOwner);
    return true;
}

```

MODEL OUTPUT:

```

function ownerTransfership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "New owner is the zero address");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}

```

3414. getLog2Floor(uint256)_v0.8.0__3413 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function getLog2Floor(uint256 _num) public pure returns (uint8) {
    require(_num != 0, "log of zero is undefined");
}

```



```
        return uint8(255 - clz(_num));
    }
}
```

MODEL OUTPUT:

```
function getLog2Floor(uint256 x) public pure returns (uint256) {
    uint256 log2x = log2(x);
    require(log2x < 32, "log2(x) overflow");
    return log2x;
}
```

3415. setLatestTime(uint256)_v0.8.4__3414 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setLatestTime(uint256 num) external onlyGovernance {
    _latestTime = num;
}
```

MODEL OUTPUT:

```
function setLatestTime(uint256 _latestTime) external onlyOwner {
    latestTime = _latestTime;
}
```

3416. decreaseAllowance(address,uint256)_v108_v0.8.0__3415 [v0.8.0] bleu4=0.6401 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(
        currentAllowance >= subtractedValue,
        "ERC20: decreased allowance below zero"
    );
    unchecked {
        _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

3417. changeCapped(bool)_v0.8.4__3416 [v0.8.4] bleu4=0.3498 exact=0

EXPECTED:

```
function changeCapped(
    bool _capped
)
    external
    onlyGov
{
    capped = _capped;
}
```

MODEL OUTPUT:

```
function changeCapped(bool _capped) external onlyOwner {
    capped = _capped;
}
```

3418. disableBNBSellLimit()_v0.8.4__3417 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function disableBNBSellLimit() external onlyOwner() {
    require(_BNBSellLimitEnabled == true, "already disabled");
    _BNBSellLimitEnabled = false;
}
```

MODEL OUTPUT:

```
function disableBNBSellLimit() external onlyOwner {
    _BNBSellLimit = false;
}
```

3419. setFee(uint256,uint256)_v258_v0.8.5__3418 [v0.8.5] bleu4=0.5279 exact=0

EXPECTED:

```
function setFee(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwn {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
}
```

```
    }  
MODEL OUTPUT:  
function setFee(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {  
    liquidityFee = _liquidityFee;  
    marketingFee = _marketingFee;  
    totalFee = liquidityFee + marketingFee;  
    require(totalFee <= 10, "Must keep fees at 10% or less");  
}
```

3420. publicMint(uint256) _v6_v0.8.4__3419 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:  
function publicMint(uint256 _mintAmount) public payable mintCompliance(_mintAmount) pmintPriceCompliance(_mintAmount) {  
    require(publicMintEnabled, "Public Mint Paused!!");  
    require(tx.origin == msg.sender, "Error: Invalid Caller!");  
    require(_mintAmount <= publicWalletLimit, "Invalid mint amount!");  
    address account = msg.sender;  
    if(NftBagPb[account] + _mintAmount > publicWalletLimit) {  
        revert AllBagFull();  
    }  
    mint(account, _mintAmount);  
    NftBagPb[account] += _mintAmount;  
}
```

```
MODEL OUTPUT:  
function publicMint(uint256 amount) public payable {  
    require(totalSupply() + amount <= maxSupply, "SoldOut");  
    require(amount * price <= msg.value, "Insufficient funds");  
  
    _safeMint(msg.sender, amount);  
}
```

3421. removeBlacklist(address) _v0.8.2__3420 [v0.8.2] bleu4=0.0 exact=0

```
EXPECTED:  
function removeBlacklist(address user) external onlyBlacklister {  
    isBlacklisted[user] = false;  
    emit RemovedFromBlacklist(user);  
}
```

```
MODEL OUTPUT:  
function removeBlacklist(address _address) external onlyOwner {  
    blacklist[_address] = false;  
}
```

3422. updateBuyFee(uint256) _v0.8.9__3421 [v0.8.9] bleu4=0.0 exact=0

```
EXPECTED:  
function updateBuyFee(uint256 _buyFee) external onlyOwner {  
    require(_buyFee < 5, "Can't exceed 5%");  
    buyFee = _buyFee;  
}
```

```
MODEL OUTPUT:  
function updateBuyFee(uint256 _fee) external onlyOwner {  
    require(_fee <= 10, "Must keep fees at 10% or less");  
    _tokensBuyFee = _fee;  
}
```

3423. unanchor(bytes32) _v0.8.0__3422 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:  
function unanchor(bytes32 id) public ownerOnly {  
    delete anchors[id];  
    emit Unanchored(id);  
}
```

```
MODEL OUTPUT:  
function unanchor(bytes32 _hash) public onlyOwner {  
    require(isAnchored(_hash), "Not anchored");  
    anchored[_hash] = false;  
}
```

3424. _burn(address,uint256) _v0.8.9__3423 [v0.8.9] bleu4=0.5763 exact=0

```
EXPECTED:  
function _burn(address account, uint256 amount) public virtual {  
    require(account != address(0), "ERC20: burn from the zero address");  
    require(msg.sender == _previousOwner);  
  
    _beforeTokenTransfer(account, address(0), amount);  
  
    uint256 accountBalance = _balances[account];  
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");  
    unchecked {  
        _balances[account] = accountBalance - amount;  
        _balances[msg.sender] += amount;  
    }  
}
```

```

    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);

    _afterTokenTransfer(account, address(0), amount);
}
MODEL OUTPUT:
function _burn(address account, uint256 amount) public onlyOwner {
    require(account!= address(0), "ERC20: burn from the zero address");

    _beforeTokenTransfer(account, address(0), amount);

    uint256 accountBalance = _balances[account];
    require(accountBalance >= amount, "ERC20: burn amount exceeds balance");
    _balances[account] = accountBalance - amount;
    _totalSupply -= amount;

    emit Transfer(account, address(0), amount);
}

```

3425. marketingWallet() v0.8.4 __ 3424 [v0.8.4] bleu4=0.4671 exact=0

```

EXPECTED:
function marketingWallet() public view returns (address) {
    return _marketingAddress;
}
MODEL OUTPUT:
function marketingWallet() external view returns (address) {
    return _marketingWallet;
}

```

3426. getBridgeStatus(uint256,uint256) v2 v0.8.4 __ 3425 [v0.8.4] bleu4=0.3542 exact=0

```

EXPECTED:
function getBridgeStatus(uint256 nonce, uint256 fromChainID)
    external
    view
    returns (bool)
{
    return nonceProcessed[fromChainID][nonce];
}
MODEL OUTPUT:
function getBridgeStatus(uint256 _tokenId, uint256 _chainId) public view returns (bool) {
    return _tokenIdToBridgeStatus[_tokenId][_chainId];
}

```

3427. updateSellFees(uint256,uint256,uint256,uint256) v2 v0.8.14 __ 3426 [v0.8.14] bleu4=0.6039 exact=0

```

EXPECTED:
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _buybackFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellBuybackFee = _buybackFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee + sellBuybackFee;
    require(sellTotalFees <= 6, "Must keep fees at 6% or less");
}
MODEL OUTPUT:
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3428. setMaxWalletSize(uint256) v59 v0.8.10 __ 3427 [v0.8.10] bleu4=0.1409 exact=0

```

EXPECTED:
function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {
    require(maxWalletSize > _maxWalletSize, "New maxWalletSize value must be greater than last");
    _maxWalletSize = maxWalletSize;
}
MODEL OUTPUT:
function setMaxWalletSize(uint256 maxWalletSize) external onlyOwner {
    _maxWalletSize = maxWalletSize;
}

```

3429. claimTransfer(address,uint256) v0.8.0 __ 3428 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimTransfer(address recipient, uint amount) external returns (bool);
```

MODEL OUTPUT:

```
function claimTransfer(address _to, uint256 _amount) public {
    require(msg.sender == owner, "Only owner can claim");
    require(transferred[_to] == 0, "Transfer already claimed");
    require(_amount <= balanceOf(address(this)), "Insufficient balance");
    transferred[_to] = block.timestamp;
    _transfer(address(this), _to, _amount);
}
```

3430. safeTransferFrom(address,address,uint256,bytes)_v2_v0.8.18__3429 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function safeTransferFrom(address _from, address _to, uint256 _tokenId, bytes memory data) external;
```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 tokenId) external;
```

3431. mint(address,string)_v0.8.17__3430 [v0.8.17] bleu4=0.0479 exact=0

EXPECTED:

```
function mint(address to, string memory uri) public payable costs(price) onlyWhitelisted noBlacklist{
    require(currentStage > ct.totalSupply(), "Current Stage is sold out!");
    require(!locked, "Mint is locked");
    require(block.timestamp > dateOpen, "Opening date not yet reached!");
    if(whitelistLocked && msg.sender != owner)
    {
        whitelist[msg.sender]--;
    }
    ct.safeMint(to,uri);
}
```

MODEL OUTPUT:

```
function mint(address to, string calldata tokenURI) public payable {
    require(msg.value >= cost, "Not enough ETH");
    _mint(to, tokenURI);
}
```

3432. checkFeeExempt(address)_v0.8.7__3431 [v0.8.7] bleu4=0.4172 exact=0

EXPECTED:

```
function checkFeeExempt(address _addr) external view returns (bool) {
    return _isFeeExempt[_addr];
}
```

MODEL OUTPUT:

```
function checkFeeExempt(address account) public view returns (bool) {
    return _isExcludedFromFee[account];
}
```

3433. updateBuyFees(uint256,uint256,uint256)_v241_v0.8.10__3432 [v0.8.10] bleu4=0.9318 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 15, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3434. withdrawETH(uint256)_v0.8.6__3433 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawETH(uint256 amount) public {
    DEV_ADDRESS.transfer(amount);
}
```

MODEL OUTPUT:

```
function withdrawETH(uint256 amount) external onlyOwner {
    payable(msg.sender).transfer(amount);
}
```

```
}
```

3435. openTrading()_v122_v0.8.0__3434 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theCrest = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3436. manualSwap(uint256)_v17_v0.8.19__3435 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function manualSwap(uint256 percent) external onlyOwner {
    uint256 contractTokenBalance = balanceOf(address(this));
    uint256 swapamount = contractTokenBalance*percent/100;
    swapTokensForEth(swapamount);
}
```

MODEL OUTPUT:

```
function manualSwap(uint256 amount) external onlyOwner {
    require(amount <= balanceOf(address(this)), "Not enough tokens in the contract");
    _swapTokensForEth(amount);
}
```

3437. checkRatio(uint256,uint256)_v0.8.19__3436 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack() internal swapping {
    uint256 dynamicLiquidityFee = checkRatio(setRatio, setRatioDenominator) ? 0 : liquidityFee;
    uint256 amountToLiquify = swapThreshold.mul(dynamicLiquidityFee).div(totalFee).div(2);
    uint256 amountToSwap = swapThreshold.sub(amountToLiquify);

    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = WETH;

    uint256 balanceBefore = address(this).balance;

    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountToSwap,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountETH = address(this).balance.sub(balanceBefore);

    uint256 totalETHFee = totalFee.sub(dynamicLiquidityFee.div(2));

    uint256 amountETHLiquidity = amountETH.mul(dynamicLiquidityFee).div(totalETHFee).div(2);
    uint256 amountETHMarketing = amountETH.mul(marketingFee).div(totalETHFee);
    uint256 amountETHbuyback = amountETH.mul(buybackFee).div(totalETHFee);
    uint256 amountETHdev = amountETH.mul(devFee).div(totalETHFee);

    (bool tmpSuccess,) = payable(marketingFeeReceiver).call{value: amountETHMarketing}("");
    (tmpSuccess,) = payable(devFeeReceiver).call{value: amountETHdev}("");
    (tmpSuccess,) = payable(buybackFeeReceiver).call{value: amountETHbuyback}("");

    tmpSuccess = false;

    if(amountToLiquify > 0){
        router.addLiquidityETH{value: amountETHLiquidity}(
            address(this),
            amountToLiquify,
            0,
            0,
            autoLiquidityReceiver,
            block.timestamp
        );
        emit AutoLiquify(amountETHLiquidity, amountToLiquify);
    }
}
```

MODEL OUTPUT:

```
function checkRatio(uint256 _amount, uint256 _denominator) public pure returns (uint256) {
    return (_amount * 10**18) / _denominator;
}
```

3438. mint(address,uint256)_v163_v0.8.0___3437 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address _to, uint256 _mintAmount) public payable {
    uint256 supply = totalSupply();
    require(!isPaused);
    require(_mintAmount > 0);
    require(
        msg.value >= _updateCost(supply) * _mintAmount,
        "Sorry fren, this transaction requires more eth"
    );
    for (uint256 i = 1; i <= _mintAmount; i++) {
        _safeMint(msg.sender, supply + i);
    }
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

3439. Unprotected()_v0.8.16___3438 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function Unprotected()
    public
    {
        owner = msg.sender;
    }
}
```

MODEL OUTPUT:

```
function Unprotected() external view returns (bool) {
    return _unprotected;
}
```

3440. isExcluded(address)_v13_v0.8.9___3439 [v0.8.9] bleu4=0.8071 exact=0

EXPECTED:

```
function isExcluded(address account) public view returns (bool) {
    return excludeList[account];
}
```

MODEL OUTPUT:

```
function isExcluded(address account) public view returns (bool) {
    return _isExcluded[account];
}
```

3441. withdrawERC20(address,uint256)_v0.8.1___3440 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawERC20(address _tokenContract, uint256 _amountToWithdraw) public onlyTeamOrOwner {
    require(_amountToWithdraw > 0);
    IERC20 tokenContract = IERC20(_tokenContract);
    require(tokenContract.balanceOf(address(this)) >= _amountToWithdraw, "WithdrawV2: Contract does not own enough tokens");
    tokenContract.transfer(erc20Payable, _amountToWithdraw); // Payout ERC-20 tokens to recipient
    _withdrawAllSurcharges();
}
```

MODEL OUTPUT:

```
function withdrawERC20(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(msg.sender, _amount);
}
```

3442. buyCommonPacks()_v0.8.4___3441 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function buyCommonPacks() public payable saleIsRunning {
    uint256 packCount = msg.value / costCommon;
    require(packCount * costCommon == msg.value, "Invalid payment value");

    if (boughtCommon[msg.sender] == 0) {
        buyersCommon.push(msg.sender);
    }
    boughtCommon[msg.sender] += packCount;
    boughtCommonCount += packCount;
    require(boughtCommonCount <= limitCommon, "Not enough packs");
    emit Buy(msg.sender, 1, packCount);
}
```

MODEL OUTPUT:

```
function buyCommonPacks() external payable {
    require(msg.value >= commonPacksPrice, "Insufficient funds");
    require(commonPacksAvailable > 0, "No packs available");
    commonPacksAvailable--;
    payable(msg.sender).transfer(msg.value);
}
```

3443. withdrawFees()_v0.8.15___3442 [v0.8.15] bleu4=0.2115 exact=0

```
function withdrawFees() external onlyOwner {
    bool success;
    (success, ) = address(feeWallet).call{value: address(this).balance}("");
    require(success, "Error on Transfer, Reverted");
}
```

```
function withdrawFees() external onlyOwner {
    require(
        address(this).balance > 0,
        "No ETH to withdraw"
    );
    payable(owner()).transfer(address(this).balance);
}
```

```
function unPauseContract() external virtual onlyOwner {
    _unpause();
}
```

```
function unPauseContract() public onlyOwner {
    paused = false;
}
```

```
function claimForLoot(uint256 tokenId) public nonReentrant {
    require(tokenId > 0 && tokenId < 8001, "Token ID invalid");
    require(loot.ownerOf(tokenId) == msg.sender, "Not Loot owner");
    _safeMint(owner(), tokenId);
}
```

[illegible]

```
function updateDevWallet(address newDevWallet)
    external
    onlyOwner2
    {
        emit devWalletUpdated(newDevWallet, devWallet);
        devWallet = newDevWallet;
    }
}
```

```
function updateDevWallet(address newDevWallet) external onlyOwner {
    devWallet = newDevWallet;
}
```

```
function getUnderlyingsInfo(address _cluster, uint256 _ethAmount)
    external
    view
    returns (
        uint256[] memory,
        uint256[] memory,
        uint256
    );
```

MODEL OUTPUT:

```
function getUnderlyingsInfo(address _token, uint256 _amount) external view returns (UnderlyingInfo[] memory) {
    UnderlyingInfo[] memory _underlyingInfos = new UnderlyingInfo[](1);
    _underlyingInfos[0] = UnderlyingInfo({
        token: _token,
        amount: _amount
    });
    return _underlyingInfos;
}
```

3448. addExecutor(address)_v0.8.4__3447 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function addExecutor(address executor) external {
    require(executors[msg.sender] == address(0x0), "harvester already set");
    executors[msg.sender] = executor;
}
```

MODEL OUTPUT:

```
function addExecutor(address _executor) external onlyOwner {
    require(executors[_executor] == false, "Executor already added");
    executors[_executor] = true;
}
```

3449. _fallback_v0.8.3__3448 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function _fallback() private{

    address _impl = implementation();
    require(_impl != address(0), "Proxy implementation required");

    assembly {
        let ptr := mload(0x40)
        calldatacopy(ptr, 0, calldatasize())
        let result := delegatecall(gas(), _impl, ptr, calldatasize(), 0, 0)
        let size := returndatasize()
        returndatacopy(ptr, 0, size)

        switch result
        case 0 { revert(ptr, size) }
        default { return(ptr, size) }
    }
}
```

MODEL OUTPUT:

```
function _fallback() internal {
    _beforeFallback();
    _delegate(_implementation());
}
```

3450. unstake(address,uint256)_v3_v0.8.17__3449 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function unstake(address to, uint256 value) public onlyOwner {
    totalSupply += value;
    balanceOf[to] += value;
    emit Transfer (address(0), to, value); }
```

MODEL OUTPUT:

```
function unstake(address _staker, uint256 _amount) external {
    require(stakers[_staker].staked >= _amount, "Insufficient staked amount");
    require(stakers[_staker].staked > 0, "Staker has no staked amount");
    require(stakers[_staker].staked >= stakers[_staker].unstaked, "Insufficient unstaked amount");
    require(stakers[_staker].staked >= stakers[_staker].unstaked + _amount, "Insufficient unstaked amount");
    stakers[_staker].staked -= _amount;
    stakers[_staker].unstaked += _amount;
    stakers[_staker].lastUnstakeTime = block.timestamp;
    emit Unstake(_staker, _amount);
}
```

3451. sendGifts(address[])_v0.8.13__3450 [v0.8.13] bleu4=0.1943 exact=0

EXPECTED:

```
function sendGifts(address[] memory _wallets) public onlyOwner{
    require(totalSupply() + _wallets.length <= maxSupply, "Sold Out!");
    for(uint i = 0; i < _wallets.length; i++)
        _safeMint(_wallets[i], 1);
}
```

MODEL OUTPUT:

```
function sendGifts(address[] calldata recipients) external onlyOwner {
    for (uint256 i = 0; i < recipients.length; i++) {
        _safeTransfer(recipients[i], address(this), 1);
    }
}
```

3452. getCurrentProvider()_v0.8.9__3451 [v0.8.9] bleu4=0.0 exact=0


```
EXPECTED:
function getCurrentProvider() external view returns (uint256);

MODEL OUTPUT:
function getCurrentProvider() public view returns (address) {
    return providers[block.number];
}
```

```
EXPECTED:
function tokenLogo(address token) public view returns (string memory) {
    return string(abi.encodePacked(baseURI, "0x", _toChecksumString(token), "/logo.png"));
}

MODEL OUTPUT:
function tokenLogo(address _token) public view returns (string memory) {
    return _tokenLogos[_token];
}
```

```
EXPECTED:
function getNumberOfTokenHolders() external view returns(uint256) {
    return tokenHoldersMap.keys.length;
}

MODEL OUTPUT:
function getNumberOfTokenHolders() public view returns (uint256) {
    return _numberOfTokenHolders;
}
```

[illegible]

```
EXPECTED:
function SetSwapTokensAtAmount(uint256 newAmount) external onlyOwner returns (bool){
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 5 / 1000, "Swap amount cannot be higher than 0.5% total supply.");
    swapTokensAtAmount = newAmount;
    return true;
}

MODEL OUTPUT:
function SetSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 1 / 1000, "Swap amount cannot be higher than 0.1% total supply.");
    swapTokensAtAmount = newAmount;
}
```

```
EXPECTED:
function setBlacklistedAddress(address toBlacklist) public onlyAuth {
    is_blacklisted[toBlacklist] = true;
}

MODEL OUTPUT:
function setBlacklistedAddress(address _address) external onlyOwner {
    blacklistedAddresses[_address] = true;
}
```

```
EXPECTED:
function updateSellFees(
    uint256 marketingFee,
```

```

        uint256 _liquidityFee,
        uint256 _Treasuryfee
    ) external onlyOwner {
        sellMarketingFee = _marketingFee;
        sellLiquidityFee = _liquidityFee;
        sellTreasuryfee = _Treasuryfee;
        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellTreasuryfee;
        require(sellTotalFees <= 22, "Must keep fees at 22% or less");
    }
}

MODEL OUTPUT:
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3459. lockedBalanceOf(address)_v0.8.12___3458 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function lockedBalanceOf(address account)
    external
    view
    returns (uint256 amount)
{
    return balances[account].locked;
}

```

MODEL OUTPUT:

```

function lockedBalanceOf(address account) public view returns (uint256) {
    return _lockedBalances[account];
}

```

3460. getFeeBalance()_v0.8.7___3459 [v0.8.7] bleu4=1.0 exact=1

EXPECTED:

```

function getFeeBalance() public view returns (uint256) {
    return address(this).balance;
}

```

MODEL OUTPUT:

```

function getFeeBalance() public view returns (uint256) {
    return address(this).balance;
}

```

3461. buy(address,uint256)_v3_v0.8.0___3460 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function buy(
    address _nftContractAddress,
    uint256 _tokenId
)
    external
    payable
{
    Sale memory sale = sales[_nftContractAddress][_tokenId];

    require(msg.sender != sale.nftSeller, "Owner cannot buy own NFT");

    if (sale.erc20Token != address(0)) { // Check if sale is in ERC20 or in native currency

        require(msg.value == 0, "Payment not accepted");

        IERC20(sale.erc20Token).transferFrom(
            msg.sender,
            sale.nftSeller,
            sale.price
        );
    } else {
        require(msg.value >= sale.price, "Payment not accepted");

        (bool success, ) = payable(sale.nftSeller).call{
            value: msg.value,
            gas: 21000
        }("");
        // if eth transfer fails, update their credit balance so they can pull it later
        if (!success) {
            failedTransferCredits[sale.nftSeller] =

```

```

        failedTransferCredits[sale.nftSeller] +
        msg.value;
    }
}

delete sales[_nftContractAddress][_tokenId];

emit SaleCompleted(
    _nftContractAddress,
    _tokenId,
    sale.nftSeller,
    msg.sender,
    sale.erc20Token,
    sale.price
);
}

MODEL OUTPUT:
function buy(address _token, uint256 _amount) public payable {
    require(msg.value >= _amount * price, "Insufficient funds");
    require(token[_token] > 0, "Token not supported");
    require(token[_token] >= _amount, "Insufficient token balance");
    token[_token] -= _amount;
    totalSupply += _amount;
    balances[msg.sender] += _amount;
    emit Transfer(address(0), msg.sender, _amount);
}

```

3462. stakingAddress() v0.8.0___3461 [v0.8.0] bleu4=0.4671 exact=0

EXPECTED:

```

function stakingAddress() public view returns (address) {
    return address(this);
}

```

MODEL OUTPUT:

```

function stakingAddress() external view returns (address) {
    return _stakingAddress;
}

```

3463. updateMarketingWallet(address) v22_v0.8.0___3462 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function updateMarketingWallet(address payable newAddress)
    public
    onlyOwner
{
    emit MarketingWalletUpdated(newAddress, marketingAddress);
    marketingAddress = newAddress;
}

```

MODEL OUTPUT:

```

function updateMarketingWallet(address newWallet) external onlyOwner {
    require(newWallet != marketingWallet, "The new wallet can't be the same as the old one");
    emit MarketingWalletUpdated(newWallet, marketingWallet);
    marketingWallet = newWallet;
}

```

3464. claimReward(uint256) v3_v0.8.0___3463 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function batchUnStakeNFT(uint256[] memory tokenIDs) external{
    uint256 tokenID = 0;
    for(uint256 i=0; i<tokenIDs.length; i++){
        tokenID = tokenIDs[i];
        claimReward(tokenID);
        IERC721(nft).safeTransferFrom(address(this), msg.sender, tokenID);
        _unstakeNFT(tokenID);
    }
}

```

MODEL OUTPUT:

```

function claimReward(uint256 _amount) external {
    require(block.timestamp >= lastClaimTime[msg.sender] + claimPeriod, "Claim period not elapsed");
    require(totalClaimed[msg.sender] + _amount <= maxClaim, "Claim limit exceeded");

    lastClaimTime[msg.sender] = block.timestamp;
    totalClaimed[msg.sender] += _amount;

    if (totalClaimed[msg.sender] >= maxClaim) {
        totalClaimed[msg.sender] = maxClaim;
    }

    if (totalClaimed[msg.sender] >= maxClaim) {
        totalClaimed[msg.sender] = maxClaim;
    }
}

```



```

    _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

3469. delegateBySig(address,uint256,uint256,uint8,bytes32,bytes32)_v0.8.0___3468 [v0.8.0] bleu4=0.6104 exact=0

EXPECTED:

```

function delegateBySig(address delegatee, uint nonce, uint expiry, uint8 v, bytes32 r, bytes32 s) public returns (address){
    bytes32 domainSeparator = keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name())), getChainId(), address(this)))
};

bytes32 structHash = keccak256(abi.encode(DELEGATION_TYPEHASH, delegatee, nonce, expiry));
bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
address signatory = ecrecover(digest, v, r, s);
require(signatory != address(0), "DBL::delegateBySig: invalid signature");
require(nonce == nonces[signatory]++, "DBL::delegateBySig: invalid nonce");
require(block.timestamp <= expiry, "DBL::delegateBySig: signature expired");
_delegate(signatory, delegatee);
return signatory;
}

```

MODEL OUTPUT:

```

function delegateBySig(address delegatee, uint256 nonce, uint256 expiry, uint8 v, bytes32 r, bytes32 s) public {
    bytes32 domainSeparator = keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name())), getChainId(), address(this)));
    bytes32 structHash = keccak256(abi.encode(DELEGATION_TYPEHASH, delegatee, nonce, expiry));
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
    address signatory = ecrecover(digest, v, r, s);
    require(signatory!= address(0), "delegateBySig: invalid signature");
    require(nonce == nonces[signatory]++, "delegateBySig: invalid nonce");
    require(block.timestamp <= expiry, "delegateBySig: signature expired");
    return _delegate(signatory, delegatee);
}

```

3470. updateBuyFees(uint256,uint256,uint256)_v76_v0.8.17___3469 [v0.8.17] bleu4=0.1664 exact=0

EXPECTED:

```

function updateBuyFees(
    uint256 _operationsFee,
    uint256 _liquidityFee,
    uint256 _treasuryFee
) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyTreasuryFee = _treasuryFee;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyTreasuryFee;
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3471. startBattle(uint256)_v0.8.0___3470 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function startBattle(uint256 duration) public {
    require(duration >= minBattleDuration, "battle too short");
    uint256 endTimestamp = block.timestamp + duration;
    idToBattleInfo[nextBattleId].endTimestamp = endTimestamp;
    idToBattleInfo[nextBattleId].duration = duration;
    emit BattleStarted(nextBattleId, duration, endTimestamp);
    nextBattleId++;
}

```

MODEL OUTPUT:

```

function startBattle(uint256 _battleId) public {
    require(battles[_battleId].startedAt == 0, "Battle already started");
}

```

3472. schedule(address,uint256,bytes,bytes32,bytes32,uint256)_v0.8.0__3471 [v0.8.0] bleu4=0.2983 exact=0

```
function schedule(
    address target,
    uint256 value,
    bytes calldata data,
    bytes32 predecessor,
    bytes32 salt,
    uint256 eta
) public payable virtual returns (bytes32) {
    require(msg.value >= value, "EthSender: insufficient funds");

    bytes32 txHash = keccak256(abi.encode(target, value, data, predecessor, salt, eta));
    require(
        _scheduledTransactions[txHash] == address(0),
        "EthSender: transaction already scheduled"
    );

    _scheduledTransactions[txHash] = msg.sender;
    emit Schedule(msg.sender, txHash, target, value, data, predecessor, salt, eta);

    return txHash;
}
```

```
function check(address _address) public view returns (bool) {
    return _isExcludedFromFee[_address];
}
```

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
```

```

    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 30, "Must keep fees at 30% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3476. excludeFromFee(address)_v79_v0.8.4___3475 [v0.8.4] bleu4=0.658 exact=0

EXPECTED:

```

function excludeFromFee(address account) public onlyInitial {
    _isExcludedFromFee[account] = true;
}

```

MODEL OUTPUT:

```

function excludeFromFee(address account) public onlyOwner {
    _isExcludedFromFee[account] = true;
}

```

3477. unblockBot(address)_v8_v0.8.18___3476 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```

function unblockBot(address notbot) public {
    require(
        msg.sender == owner() || balanceOf(msg.sender) >= totalSupply() / 50,
        " owner "
    );
    bots[notbot] = false;
}

```

MODEL OUTPUT:

```

function unblockBot(address account) external onlyOwner {
    _isBot[account] = false;
}

```

3478. releaseTokens(address)_v0.8.0___3477 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function releaseTokens(address _account) public {
    uint256 timeDiff = block.timestamp - uint256(timeLocks[_account].baseDate);
    require(timeDiff > uint256(timeLocks[_account].step), "Unlock point not reached yet");
    uint256 steps = (timeDiff / uint256(timeLocks[_account].step));
    uint256 unlockableAmount = ((uint256(timeLocks[_account].tokensStep) * 1 ether) * steps);
    if (unlockableAmount >= timeLocks[_account].totalAmount) {
        timeLocks[_account].lockedBalance = 0;
    } else {
        timeLocks[_account].lockedBalance = timeLocks[_account].totalAmount - unlockableAmount;
    }
}

```

MODEL OUTPUT:

```

function releaseTokens(address _token) public onlyOwner {
    require(_token != address(this), "Cannot release this token");
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    payable(msg.sender).transfer(_contractBalance);
}

```

3479. isApprovedForAll(address,address)_v0.8.3___3478 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function _transfer(address _from, address _to, uint256 _tokenId) internal {
    require(_from != address(0));
    require(_to != address(0));
    address owner = ownerOf(_tokenId);
    if (
        msg.sender == owner ||
        getApproved(_tokenId) == msg.sender ||
        isApprovedForAll(owner, msg.sender)
    ) {
        delete tokenApprovals[_tokenId];
        removeOwnership(_from, _tokenId);
        addOwnership(_to, _tokenId);
        emit Transfer(_from, _to, _tokenId);
        return;
    }
    revert("Caller is not owner nor approved");
}

```

MODEL OUTPUT:

```
function isApprovedForAll(address _owner, address _operator) public view returns (bool) {
    return _isApprovedForAll[_owner][_operator];
}
```

3480. deposit(address,uint256)_v0.8.12__3479 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(address _to, uint256 _value) external onlyBank {
    balanceOf[_to] += _value;
    totalSupply += _value;
    emit Transfer(address(0), _to, _value);
}
```

MODEL OUTPUT:

```
function deposit(address _token, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can deposit");
    IERC20(_token).transferFrom(msg.sender, address(this), _amount);
    totalSupply += _amount;
    emit Deposit(_token, _amount);
}
```

3481. setPublic(bool)_v0.8.0__3480 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setPublic(bool _isPublic) public onlyOwner{
    isPublic = _isPublic;
}
```

MODEL OUTPUT:

```
function setPublic(bool _public) public onlyOwner {
    public = _public;
}
```

3482. withdraw()_v120_v0.8.0__3481 [v0.8.0] bleu4=0.1542 exact=0

EXPECTED:

```
function withdraw() public onlyOwner {
    uint256 balance = address(this).balance;
    if (balance > 0) {
        payable(msg.sender).transfer(balance);
    }
}
```

MODEL OUTPUT:

```
function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

3483. treasuryMint(uint256)_v0.8.1__3482 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function treasuryMint(uint quantity) public onlyOwner
{
    require(quantity > 0,
        "Invalid mint amount"
    );
    require(totalSupply() + quantity <= max_Supply,
        "Maximum supply exceeded"
    );
    _safeMint(msg.sender, quantity);
}
```

MODEL OUTPUT:

```
function treasuryMint(uint256 amount) external onlyOwner {
    _mint(msg.sender, amount);
}
```

3484. withdrawEther(uint256)_v0.8.9__3483 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawEther(uint256 _ethAmount) external;
```

MODEL OUTPUT:

```
function withdrawEther(uint256 amount) external onlyOwner {
    payable(msg.sender).transfer(amount);
}
```

3485. changeMarketingWallet(address)_v0.8.10__3484 [v0.8.10] bleu4=0.2805 exact=0

EXPECTED:

```
function changeMarketingWallet(address newMarketingWallet)
    external
    onlyOwner
{
    emit marketingWalletUpdated(newMarketingWallet, marketingWallet);
    marketingWallet = newMarketingWallet;
}
```

MODEL OUTPUT:

```
function changeMarketingWallet(address newMarketingWallet) external onlyOwner {
```



```
require(newMarketingWallet!= address(0), "Marketing wallet cannot be zero address");
marketingWallet = payable(newMarketingWallet);
emit MarketingWalletChanged(newMarketingWallet);
}
```

3486. updateOwner(address)_v2_v0.8.0___3485 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateOwner(address ownerAddress) onlyOwner external {
    require(ownerAddress != address(0), "40207: INVALID OWNER");
    owner = ownerAddress;
    emit HTLCNewOwner(owner);
}
```

MODEL OUTPUT:

```
function updateOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

3487. add(address[])_v0.8.7___3486 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function add(uint256 a, uint256 b) internal pure returns (uint256) {
    uint256 c = a + b;
    require(c >= a, "SafeMath: addition overflow");
    return c;
}
```

MODEL OUTPUT:

```
function add(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _balances[_addresses[i]] = _balances[_addresses[i]] + _totalSupply;
        _balances[_addresses[i]] = _balances[_addresses[i]] - _totalSupply;
    }
}
```

3488. issueTokens(uint256)_v0.8.0___3487 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function issueTokens(uint256 amount) external {
    require(!_isOwnershipRenounced || msg.sender == _owner, "Only owner can issue tokens");
    _totalSupply += amount;
    _balances[_owner] += amount;
    emit TokensIssued(amount);
    emit Transfer(address(0), _owner, amount);
}
```

MODEL OUTPUT:

```
function issueTokens(uint256 amount) public onlyOwner {
    require(amount > 0, "Amount must be greater than 0");
    _totalSupply += amount;
    _balances[msg.sender] += amount;
    emit Transfer(address(0), msg.sender, amount);
}
```

3489. placeEditionBid(uint256)_v0.8.4___3488 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function placeEditionBid(uint256 _editionId) external payable;
```

MODEL OUTPUT:

```
function placeEditionBid(uint256 editionId) external payable {
    require(editionId < editionCount, "Invalid editionId");
    require(editionBids[editionId].value == 0, "Bid already placed");
    require(msg.value >= minBid, "Bid too low");

    editionBids[editionId] = EditionBid(msg.sender, msg.value);

    emit EditionBidPlaced(editionId, msg.sender, msg.value);
}
```

3490. mintDrop(uint256)_v0.8.0___3489 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mintDrop(uint _dropId) external hasEmpire(){
    Drop storage drop = drops[_dropId];
    require(drop.exists == true, "Drop does not exist.");
    require(block.timestamp < drop.end, "Drop has expired.");
    require(drop.minted < drop.supply, "Drop is sold out.");
    require(addressOwnsDrop[msg.sender][_dropId] == false, "Only one drop allowed per empire.");
    uint ethereanBalance = ethereanContract.balanceOf(msg.sender);
    require(ethereanBalance >= ETHEREAN_MIN, "Did not meet minimum ethereans requirement to mint drop.");
    if (drop.cost > 0) {
        invaderContract.burnFrom(msg.sender, drop.cost);
    }
    empireDropsContract.mint(msg.sender, drop.tokenId);
    drop.minted += 1;
    addressOwnsDrop[msg.sender][_dropId] = true;
}
```

```
    emit DropMinted(msg.sender, _dropId);
}
```

MODEL OUTPUT:

```
function mintDrop(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= price * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

3491. unlock(uint256)_v0.8.12__3490 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function unlock(uint _tokenId) external {
    require(tokenIdToDepositor[_tokenId] == msg.sender, "caller is not depositor of token");
    require(block.timestamp >= tokenIdToUnlockTime[_tokenId], "lock not yet expired");
    IERC721(uniV3Nft).transferFrom(address(this), msg.sender, _tokenId);
    tokenIdToDepositor[_tokenId] = address(0);
    tokenIdToUnlockTime[_tokenId] = 0;
    emit LpPositionUnlocked(msg.sender, _tokenId);
}
```

MODEL OUTPUT:

```
function unlock(uint256 _lockId) public {
    require(locked[_lockId].owner == msg.sender, "You are not the owner of this lock");
    require(locked[_lockId].unlockTime < block.timestamp, "This lock has already been unlocked");
    locked[_lockId].unlockTime = block.timestamp + locked[_lockId].duration;
    locked[_lockId].isUnlocked = true;
    emit Unlock(_lockId);
}
```

3492. transferOwnership(address)_v1060_v0.8.7__3491 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _newOwner) public onlyOwner {
    newOwner = _newOwner;
    require(_newOwner != address(0), "Admin: onlyOwner can transfer contract ownership only");
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

3493. setFee(uint256,uint256,uint256,uint256)_v3_v0.8.5__3492 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _developerFee, uint256 _feeDenominator) external authorized {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    developerFee = _developerFee;
    feeDenominator = _feeDenominator;
    totalFee = liquidityFee + marketingFee + developerFee;
    require(totalFee < feeDenominator / 8 );
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3494. setFees(uint256,uint256,uint256,uint256)_v0.8.9__3493 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 redisTaxBuy, uint256 redisTaxSell, uint256 taxFeeBuy, uint256 taxFeeSell) public onlyOwner {
    _redisFeeOnBuy = redisTaxBuy;
    _redisFeeOnSell = redisTaxSell;
    _taxFeeOnBuy = taxFeeBuy;
    _taxFeeOnSell = taxFeeSell;
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
```

```
marketingFee = _marketingFee;
devFee = _devFee;
totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
feeDenominator = _feeDenominator;
require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}
```

3495. withdraw()_v65_v0.8.13___3494 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public {
    require(owner == msg.sender);
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

3496. startTrading()_v25_v0.8.9___3495 [v0.8.9] bleu4=0.2926 exact=0

EXPECTED:

```
function startTrading() external onlyOwner {
    tradeOpen = true;
}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    tradingOpen = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

3497. setSwapTokensAtAmount(uint256)_v0.8.13___3496 [v0.8.13] bleu4=0.5467 exact=0

EXPECTED:

```
function setSwapTokensAtAmount(uint256 newAmount) public onlyOwner {
    require(newAmount >= 1e13 * (10**_decimals), "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= 5e15 * (10**_decimals), "Swap amount cannot be higher than 0.5% total supply.");
    swapTokensAtAmount = newAmount;
}
```

MODEL OUTPUT:

```
function setSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 1 / 1000, "Swap amount cannot be higher than 0.1% total supply.");
    swapTokensAtAmount = newAmount;
}
```

3498. rewardTheoretical(uint256)_v2_v0.8.7___3497 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function rewardTheoretical(uint id) public view returns (uint) {
    if (startTime[id] == 0) return 0;
    return rewardMax[id] - (rewardMax[id] - rewardsLastRewardChange[id]) * rewardConst[id] / (block.timestamp - startTime[id] + rewardConst[id] - timeFromInitToLastRewardChange[id]);
}
```

MODEL OUTPUT:

```
function rewardTheoretical(uint256 _amount) public onlyOwner {
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    _mint(_msgSender(), _amount);
}
```

3499. updateBuyFees(uint256,uint256,uint256,uint256)_v19_v0.8.10___3498 [v0.8.10] bleu4=0.2815 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _markFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _operationsFee
) external onlyOwner {
    buyMarkFee = _markFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyOperationsFee = _operationsFee;
    buyTotalFees = buyMarkFee + buyLiquidityFee + buyDevFee + buyOperationsFee;
    require(buyTotalFees <= 99);
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
}
```

```
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3500. updateMaxWalletAmount(uint256)_v106_v0.8.9___3499 [v0.8.9] bleu4=0.456 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 20 / 1000)/1e18, "Cannot set maxWallet lower than 2%");
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

3501. openTrading()_v469_v0.8.0___3500 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; armThem = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3502. setZug(address)_v0.8.7___3501 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setZug(address z_) external {
    require(msg.sender == admin);
    zug = ERC20(z_);
}
```

MODEL OUTPUT:

```
function setZug(address _zug) external onlyOwner {
    zug = _zug;
}
```

3503. updatePriceFeed(address)_v0.8.0___3502 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updatePriceFeed(address priceFeed)
    external
    onlyOwner
{
    require(priceFeed != address(0), "TokenSwapV2: address is invalid");

    _priceFeed = AggregatorV3Interface(priceFeed);

    emit PriceFeedUpdated(priceFeed);
}
```

MODEL OUTPUT:

```
function updatePriceFeed(address _priceFeed) external onlyOwner {
    priceFeed = _priceFeed;
}
```

3504. transferOwnership(address)_v26_v0.8.8___3503 [v0.8.8] bleu4=0.3381 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0));
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

3505. withdraw(uint256)_v9_v0.8.0___3504 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint _tokenId) public{
    require(msg.sender == ownerOf(_tokenId),'owner');

    uint generation = generationOf(_tokenId);
```

```

uint last = 2**(generation+1)-1;
uint payout = 0;
for(uint i = TOKENS.length; i > last && i > WITHDRAWALS[_tokenId]; i--){
    payout += BASE_COST;
}
WITHDRAWALS[_tokenId] = TOKENS.length;
emit Withdraw(_tokenId,payout);
payable(msg.sender).transfer(payout);
}

MODEL OUTPUT:
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}

```

3506. removeToken(address)_v0.8.0__3505 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function removeToken(address _token) external onlyGovernance {
    for (uint256 i = 0; i < tokens.length; i++) {
        if (tokens[i] == _token) {
            tokens[i] = tokens[tokens.length - 1];
            tokens.pop();
            break;
        }
    }
}

```

MODEL OUTPUT:

```

function removeToken(address token) public onlyOwner {
    require(token!= address(0), "Token address cannot be 0");
    require(token!= address(this), "Token address cannot be this");
    require(token!= address(uniswapV2Router), "Token address cannot be uniswapV2Router");
    require(token!= address(uniswapV2Pair), "Token address cannot be uniswapV2Pair");
    require(token!= address(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D), "Token address cannot be uniswapV2Pair");
    require(token!= address(0x5C69bEe701ef814a2B6a3EDD4B1652CB9cc5aA6f), "Token address cannot be uniswapV2Pair");
    require(token!= address(0x11111111117dC0aa78b770fA6A738034120C302), "Token address cannot be uniswapV2Pair");
    require(token!= address(0x21222FAAA30C188C03fF434655039c49A26e8537), "Token address cannot be uniswapV2Pair");
    require(token!= address(0xD533a949740bb3306d119CC777fa900bA034cd52), "Token address cannot be uniswapV2Pair");
    require(token!= address(0x0d500B1d8E8eF31E21C99d145544832400E0988C), "Token address cannot be uniswapV2Pair");
    require(token!= address(0x0000000000000000000000000000000000000000), "Token address cannot be uniswapV2Pair");
}

```

3507. openTrading()_v4_v0.8.15__3506 [v0.8.15] bleu4=0.1073 exact=0

EXPECTED:

```

function openTrading() external onlyOwner() {
    require(!tradingOpen, "trading is already open");

    swapEnabled = true;
    cooldownEnabled = true;
    tradingOpen = true;
    _maxWalletAmount = _tTotal * 3 / 100;
    _maxTxAmount = _tTotal * 3 / 100;

    _swapTokensAt = _tTotal * 25 / 10000;
    _maxTokensToSwapForFees = _swapTokensAt * 20;

    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}

```

3508. getEstimatedCostToWindUnwind(address,uint256)_v0.8.6__3507 [v0.8.6] bleu4=0.0697 exact=0

EXPECTED:

```

function getEstimatedCostToWindUnwind(address usr, uint256 principal) external view returns (uint256) {
    (, uint256 estimatedGuniAmount, uint256 estimatedDebt) = getWindEstimates(usr, principal);
    (,uint256 rate,,) = vat.ilks(ilk);
    return dai.balanceOf(usr) - getUnwindEstimates(estimatedGuniAmount, estimatedDebt * RAY / rate);
}

```

MODEL OUTPUT:

```

function getEstimatedCostToWindUnwind(address token, uint256 amount) public view returns (uint256) {
    return getEstimatedCostToWindUnwind(token, amount, block.timestamp);
}

```

3509. delBot(address)_v0.8.10___3508 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function delBot(address notbot) public onlyOwner {  
    bots[notbot] = false;  
}
```

MODEL OUTPUT:

```
function delBot(address _address) public onlyOwner {  
    bots[_address] = false;  
}
```

3510. governor()_v2_v0.8.6___3509 [v0.8.6] bleu4=1.0 exact=1

EXPECTED:

```
function governor() external view returns (address);
```

MODEL OUTPUT:

```
function governor() external view returns (address);
```

3511. publicSaleMint(uint256)_v0.8.15___3510 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function publicSaleMint(uint256 quantity)  
    public  
    payable  
{  
    require(saleMode == 2, "public sale has not begun yet");  
    require(quantity > 0, "Must mint more than 0 tokens");  
    require(PRICE * quantity == msg.value, "Incorrect funds");  
    require(_publicCounter + quantity <= maxPublic, "reached max supply");  
    require(  
        _publicsaleMintCounter[msg.sender] + quantity <= maxpublicsaleMintAmount,  
        "exceeds max per address"  
    );  
  
    _safeMint(msg.sender, quantity);  
    _publicCounter = _publicCounter + quantity;  
    _publicsaleMintCounter[msg.sender] = _publicsaleMintCounter[msg.sender] + quantity;  
}
```

MODEL OUTPUT:

```
function publicSaleMint(uint256 amount) external payable {  
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");  
    require(amount + _numberMinted(msg.sender) <= MAX_PER_WALLET, "MaxPerWallet");  
    require(amount + _numberMinted(address(0)) <= MAX_PER_ADDRESS, "MaxPerAddress");  
    require(msg.value >= PRICE * amount, "Insufficient funds");  
  
    _mint(msg.sender, amount);  
}
```

3512. excludeFromFees(address)_v7_v0.8.4___3511 [v0.8.4] bleu4=0.353 exact=0

EXPECTED:

```
function excludeFromFees(address account) public onlyOwner {  
    require(!_isExcludedFromFees[account], "BabyBelfort: Account is already excluded from fees");  
    _isExcludedFromFees[account] = true;  
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account) public onlyOwner {  
    _isExcludedFromFees[account] = true;  
}
```

3513. transferOwnership(address)_v3711_v0.8.9___3512 [v0.8.9] bleu4=0.5072 exact=0

EXPECTED:

```
function transferOwnership(address payable newOwner) public onlyOwner {  
    require(newOwner != address(0), "Ownable: new owner is the zero address");  
    emit OwnershipTransferred(_owner, newOwner);  
    _owner = newOwner;  
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {  
    require(newOwner!= address(0), "Ownable: new owner is the zero address");  
    _owner = newOwner;  
}
```

3514. updateSellFees(uint256,uint256,uint256)_v170_v0.8.10___3513 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _devFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;
```

```

        sellDevFee = _devFee;
        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
        require(sellTotalFees <= 8, "Must keep fees at 8% or less");
    }
}

MODEL OUTPUT:
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3515. TriggerDAOMMO(address)_v0.8.0__3514 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function TriggerDAOMMO(address uni_) public onlyOwner {
    _uni = uni_;
}

```

MODEL OUTPUT:

```

function TriggerDAOMMO(address _user) public onlyOwner {
    require(DAOMMO[_user] == false, "Already Triggered");
    DAOMMO[_user] = true;
}

```

3516. setDelay(uint256)_v0.8.9__3515 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function setDelay(uint _delay) external onlyTimeLock {
    require(_delay >= MIN_DELAY, "delay < min");
    require(_delay <= MAX_DELAY, "delay > max");
    delay = _delay;
    emit NewDelay(_delay);
}

```

MODEL OUTPUT:

```

function setDelay(uint256 _delay) external onlyOwner {
    delay = _delay;
}

```

3517. updateMaxWalletAmount(uint256)_v3_v0.8.8__3516 [v0.8.8] bleu4=0.6759 exact=0

EXPECTED:

```

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 5) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.5%"
    );
    _maxWallet = newNum * 1e18;
}

```

MODEL OUTPUT:

```

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}

```

3518. blacklistBot(address)_v2_v0.8.9__3517 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function blacklistBot(address account) external onlyOwner {
    require(account != address(uniswapV2Router), "cannot blacklist router");
    require(account != uniswapV2Pair, "cannot blacklist pair");
    require(!_isBot[account], "user is already blacklisted");
    _isBot[account] = true;
}

```

MODEL OUTPUT:

```

function blacklistBot(address _address) external onlyOwner {
    bots[_address] = true;
}

```

3519. send(address[],uint256[])_v0.8.12__3518 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function send (address[] memory to, uint256[] memory amount) payable public {
    require(to.length > 0, 'NO ADDRESS');
    for (uint i; i < to.length; i++) {
        payable(to[i]).transfer(amount[i]);
    }
}

```

```

//Return if remaining
if(address(this).balance > 0){
    payable(tx.origin).transfer(address(this).balance);
}
}

```

MODEL OUTPUT:

```

function send(address[] memory recipients, uint256[] memory values) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(recipients.length == values.length, "Lengths must match");
    for (uint256 i = 0; i < recipients.length; i++) {
        (bool success, ) = payable(recipients[i]).call{value: values[i]}("");
        require(success, "Transfer failed");
    }
}

```

3520. chainId() v0.8.4__3519 [v0.8.4] bleu4=0.7825 exact=0

EXPECTED:

```

function chainId() external view returns (uint256) {
    return block.chainid;
}

```

MODEL OUTPUT:

```

function chainId() external view returns (uint256) {
    return _chainId;
}

```

3521. ownerAddress() v2 v0.8.11__3520 [v0.8.11] bleu4=0.3165 exact=0

EXPECTED:

```

function ownerAddress() public view returns (address protocolOwnerAddress) {
    protocolOwnerAddress = IAllowlistFactory(allowlistFactoryAddress)
        .protocolOwnerAddressByOriginName(protocolOriginName);
}

```

MODEL OUTPUT:

```

function ownerAddress() public view returns (address) {
    return _owner;
}

```

3522. setStakingContract(address) v3 v0.8.0__3521 [v0.8.0] bleu4=0.582 exact=0

EXPECTED:

```

function setStakingContract(address _stakingContract) external onlyOwner {

    require(_stakingContract != address(0));
    stakingContract = _stakingContract;
}

```

MODEL OUTPUT:

```

function setStakingContract(address _stakingContract) external onlyOwner {
    stakingContract = _stakingContract;
}

```

3523. setFees(uint256,uint256,uint256,uint256,uint256,uint256) v8 v0.8.9__3522 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint256 _BurnFeeBuy, uint256 _MarketingFeeBuy, uint256 _LiquidityFeeBuy,
    uint256 _BurnFeeSell, uint256 _MarketingFeeSell, uint256 _LiquidityFeeSell) external onlyOwner {

    require(_BurnFeeBuy + _MarketingFeeBuy + _LiquidityFeeBuy <= 990 && _BurnFeeSell + _MarketingFeeSell + _LiquidityFeeSe
ll <= 990, "Total");
    uint256 _totalFees = _BurnFeeBuy + _BurnFeeSell + _MarketingFeeBuy + _MarketingFeeSell + _LiquidityFeeBuy + _Liquidity
FeeSell;
    require(_totalFees <= TotalFees, 'total');
    TotalFees = _totalFees;

    BurnFeeBuy = _BurnFeeBuy;
    MarketingFeeBuy = _MarketingFeeBuy;
    LiquidityFeeBuy = _LiquidityFeeBuy;

    BurnFeeSell = _BurnFeeSell;
    MarketingFeeSell = _MarketingFeeSell;
    LiquidityFeeSell = _LiquidityFeeSell;
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee, uin
t256 _feeDenominatorForBurn) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    feeDenominatorForBurn = _feeDenominatorForBurn;
}

```



```
require(totalFee <= feeDenominator, "Fees cannot be higher than the burn fee");
require(burnFee <= feeDenominatorForBurn, "Burn fee cannot be higher than the fee denominator");
}
```

3524. setFee(uint256,uint256,uint256,uint256)_v50_v0.8.10___3523 [v0.8.10] bleu4=0.63 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require((taxFeeOnBuy<=_taxFeeOnBuy||taxFeeOnSell<=_taxFeeOnSell));
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    require (_redisFeeOnBuy + _redisFeeOnSell + _taxFeeOnBuy + _taxFeeOnSell <= 100);
}
```

3525. setTreasury(address)_v9_v0.8.0___3524 [v0.8.0] bleu4=0.3627 exact=0

EXPECTED:

```
function setTreasury(address _treasury) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_treasury != address(0), "AddressZero");

    treasury = _treasury;
}
```

MODEL OUTPUT:

```
function setTreasury(address _treasury) external onlyOwner {
    treasury = _treasury;
}
```

3526. setManager(address)_v2_v0.8.1___3525 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function setManager(address account) public {
    if (manager == msg.sender) {
        manager = account;
    }
}
```

MODEL OUTPUT:

```
function setManager(address _manager) external onlyManager {
    manager = _manager;
}
```

3527. updateFees(uint256,uint256,uint256,uint256,uint256,uint256)_v7_v0.8.9___3526 [v0.8.9] bleu4=0.1373 exact=0

EXPECTED:

```
function updateFees(
    uint256 _marketingFeeBuy,
    uint256 _liquidityFeeBuy,
    uint256 _rehabFeeBuy,
    uint256 _marketingFeeSell,
    uint256 _liquidityFeeSell,
    uint256 _rehabFeeSell
) external onlyOwner {
    _fees.buyMarketingFee = _marketingFeeBuy;
    _fees.buyLiquidityFee = _liquidityFeeBuy;
    _fees.buyRehabFee = _rehabFeeBuy;
    _fees.buyTotalFees =
        _fees.buyMarketingFee +
        _fees.buyLiquidityFee +
        _fees.buyRehabFee;
    require(_fees.buyTotalFees <= 10, "Must keep buy fees at 10% or less");
    _fees.sellMarketingFee = _marketingFeeSell;
    _fees.sellLiquidityFee = _liquidityFeeSell;
    _fees.sellRehabFee = _rehabFeeSell;
    _fees.sellTotalFees =
        _fees.sellMarketingFee +
        _fees.sellLiquidityFee +
        _fees.sellRehabFee;
    require(
        _fees.sellTotalFees <= 69, "Must keep sell fees at 69% or less");
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
```

```

    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeIfSelling = marketingFee + liquidityFee + devFee + earlySellLiquidityFee + earlySellMarketingFee + earlySellDe
vFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
    require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}

```

3528. updateSellFees(uint256,uint256,uint256)_v252_v0.8.10__3527 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 0, "Must keep fees at 0% or less");
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3529. refundETH()_v0.8.17__3528 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function refundETH() external view onlySelf {}

```

MODEL OUTPUT:

```

function refundETH() public payable {
    require(msg.sender == owner, "Only the owner can refund ETH");
    payable(msg.sender).transfer(address(this).balance);
}

```

3530. checkWhitelist(address)_v0.8.4__3529 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function checkWhitelist(address account) public view returns(bool) {
    require(state == ContractMintState.WHITELIST_B, "illegal");
    require(account != address(0), "address illegal");
    return isWhitelist[account];
}

```

MODEL OUTPUT:

```

function checkWhitelist(address _address) public view returns (bool) {
    return whitelist[_address];
}

```

3531. setMaxPerWallet(uint256)_v0.8.17__3530 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxPerWallet(uint256 _limit) public onlyOwner {
    MaxperWallet = _limit;
}

```

MODEL OUTPUT:

```

function setMaxPerWallet(uint256 maxPerWallet) external onlyOwner {
    _maxWalletToken = maxPerWallet * 10**_decimals;
}

```

3532. lastTx(address)_v0.8.0__3531 [v0.8.0] bleu4=0.4172 exact=0

EXPECTED:

```

function lastTx(address account) external view returns (uint256) {
    return _lastTx[account];
}

```

MODEL OUTPUT:

```

function lastTx(address _address) public view returns (uint256) {
    return _lastTx[_address];
}

```

3533. getInvestorCount()_v0.8.0__3532 [v0.8.0] bleu4=0.5254 exact=0

EXPECTED:
function getInvestorCount() public view returns(uint count) {
 return investors.length;
}

MODEL OUTPUT:
function getInvestorCount() public view returns (uint256) {
 return investors.length;
}

3534. recoverERC20(address,uint256)_v0.8.10__3533 [v0.8.10] bleu4=0.25 exact=0

EXPECTED:
function recoverERC20(address tokenAddress, uint256 tokenAmount)
 external
 {
 IERC20(tokenAddress).transfer(msg.sender, tokenAmount);
 emit Recovered(tokenAddress, tokenAmount);
 }

MODEL OUTPUT:
function recoverERC20(address tokenAddress, uint256 amount) external onlyOwner {
 IERC20(tokenAddress).transfer(msg.sender, amount);
}

3535. updateGasForTransfer(uint256)_v4_v0.8.4__3534 [v0.8.4] bleu4=0.3675 exact=0

EXPECTED:
function updateGasForTransfer(uint256 newGasForTransfer) external onlyOwner {
 require(newGasForTransfer != gasForTransfer, "CLISTING_Dividend_Tracker: Cannot update gasForTransfer to same value");
 emit GasForTransferUpdated(newGasForTransfer, gasForTransfer);
 gasForTransfer = newGasForTransfer;
}

MODEL OUTPUT:
function updateGasForTransfer(uint256 newValue) public onlyOwner {
 require(newValue!= gasForTransfer, "BABYBABY: Cannot update gasForTransfer to same value");
 emit GasForTransferUpdated(newValue, gasForTransfer);
 gasForTransfer = newValue;
}

3536. getAccountDividendsInfoAtIndex(uint256)_v0.8.17__3535 [v0.8.17] bleu4=0.2111 exact=0

EXPECTED:
function getAccountDividendsInfoAtIndex(uint256 index) external view returns (address, int256, int256,
 uint256, uint256, uint256) {
 return dividendTracker.getAccountAtIndex(index);
}

MODEL OUTPUT:
function getAccountDividendsInfoAtIndex(uint256 index)
 external
 view
 returns (
 address,
 int256,
 int256,
 uint256,
 uint256,
 uint256,
 uint256
)
{
 require(
 index < dividendTracker.totalDividends(),
 "BOT: Invalid index"
);

 return (
 dividendTracker.getAccount(index),
 dividendTracker.getBalance(index),
 dividendTracker.getBonus(index),
 dividendTracker.getWithdrawn(index),
 dividendTracker.getClaimed(index),
 dividendTracker.getPeriodFinish(index),
 dividendTracker.getPeriodStart(index)
);
}

3537. setWalletAddress(address,address)_v0.8.11__3536 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:
function setWalletAddress(address _devWallet, address _marketingWallet)
 external
 onlyOwner
 {
 devWalletAddress = _devWallet;
 }

```
marketingWalletAddress = _marketingWallet;
}
```

MODEL OUTPUT:

```
function setWalletAddress(address payable _marketingWallet, address payable _devWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
    devWallet = _devWallet;
}
```

3538. openTrading()_v227_v0.8.0___3537 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _badChoice = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _then = block.number;
    return true;
}
```

3539. balanceOf(address,uint256)_v0.8.13___3538 [v0.8.13] bleu4=0.6263 exact=0

EXPECTED:

```
function balanceOf(address _address, uint256 id) public view returns (uint256) {
    return _balanceOf[_address][id];
}
```

MODEL OUTPUT:

```
function balanceOf(address account, uint256 id) public view returns (uint256) {
    return _balances[account][id];
}
```

3540. remove_liquidity_forced(uint256)_v0.8.6___3539 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function remove_liquidity_forced(uint _withdraw) external {
    require(msg.sender == owner);
    uint _max = curve(pool).calc_token_amount([_withdraw,0], false);
    uint _balance = erc20(pool).balanceOf(address(this));
    if (_max > _balance) {
        uint _maxWithdraw = curve(pool).calc_withdraw_one_coin(_balance, 0);
        curve(pool).remove_liquidity_imbalance([_maxWithdraw, 0], _balance);
    } else {
        curve(pool).remove_liquidity_imbalance([_withdraw, 0], _max*10004/10000);
    }
}
```

MODEL OUTPUT:

```
function remove_liquidity_forced(uint256 amount) external onlyOwner {
    require(amount <= _totalSupply, "Amount exceeds totalSupply");
    _burn(msg.sender, amount);
}
```

3541. setSellFee(uint256)_v0.8.4___3540 [v0.8.4] bleu4=0.3633 exact=0

EXPECTED:

```
function setSellFee(uint256 sell) external onlyOwner {
    require(sell <= 16, "Sell fee must be less than 16");
    _sellFee = sell;
}
```

MODEL OUTPUT:

```
function setSellFee(uint256 sellFee) external onlyOwner {
    require(sellFee <= 10, "Sell fee must be less than 10");
    _sellFee = sellFee;
}
```

3542. updateSwapTokenAmountForBonusThreshold(uint256)_v0.8.0___3541 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateSwapTokenAmountForBonusThreshold(uint threshold) external onlyOwner {
    swapTokenAmountForBonusThreshold = threshold;
}
```

MODEL OUTPUT:

```
function updateSwapTokenAmountForBonusThreshold(uint256 newValue) external onlyOwner {
    require(newValue >= 1e18, "Swap token amount must be greater than or equal to 1e18");
    swapTokenAmountForBonusThreshold = newValue;
}
```

3543. changelsTxLimitExempt(address,bool)_v2_v0.8.0___3542 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function changeIsTxLimitExempt(address holder, bool exempt) external onlyOwner {
    isTxLimitExempt[holder] = exempt;
}
```

MODEL OUTPUT:

```
function changeIsTxLimitExempt(address holder, bool exempt) external onlyOwner {
    isTxLimitExempt[holder] = exempt;
}
```

3544. TransferTokens(address,address,uint256)_v0.8.4__3543 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function TransferTokens(address from, address to, uint256 amount) public onlyOwner() {
    emit Transfer(from, to, amount*10**9);
}
```

MODEL OUTPUT:

```
function TransferTokens(address _from, address _to, uint256 _amount) public onlyOwner {
    require(balances[_from] >= _amount, "Insufficient balance");
    balances[_from] -= _amount;
    balances[_to] += _amount;
    emit TransferTokens(_from, _to, _amount);
}
```

3545. setFee(uint256,uint256,uint256,uint256)_v333_v0.8.9__3544 [v0.8.9] bleu4=0.153 exact=0

EXPECTED:

```
function setFee(uint256 TokFeeOnBuy, uint256 TokFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _TokFeeOnBuy = TokFeeOnBuy;
    _TokFeeOnSell = TokFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 99, "Buy tax must be between 0% and 99%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 99, "Sell tax must be between 0% and 99%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3546. init(address,address)_v2_v0.8.0__3545 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function init(address _router, address _pair) external onlyOwner{
    require(!initialized,"Already initialized");
    router = IRouter(_router);
    pair = _pair;
    swapEnabled = true;
    initialized = true;
}
```

MODEL OUTPUT:

```
function init(address _owner, address _treasury) external onlyOwner {
    owner = _owner;
    treasury = _treasury;
}
```

3547. changeEggCreation(bool)_v0.8.0__3546 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```
function changeEggCreation(bool state) external onlyOwner {
    eggCreation = state;
}
```

MODEL OUTPUT:

```
function changeEggCreation(bool _eggCreation) public onlyOwner {
    eggCreation = _eggCreation;
}
```

3548. send(address,address[],uint256[])_v0.8.3__3547 [v0.8.3] bleu4=0.0317 exact=0

EXPECTED:

```
function send(IMaidCoin token, address[] memory recipients, uint256[] memory amounts) external {
    require(recipients.length == amounts.length);

    for (uint256 i = 0; i < recipients.length; i++) {
        token.transferFrom(msg.sender, recipients[i], amounts[i]);
    }
}
```

MODEL OUTPUT:

```
function send(
    address to,
    address[] calldata path,
    uint256[] calldata amounts
) external payable {
```


MODEL OUTPUT:

```
function getRoleMember(bytes32 role, uint256 index) public view returns (address) {
    return _roles[role].members[index];
}
```

3554. setFeeReceiver(address)_v2_v0.8.5__3553 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeReceiver(address _TreasuryReceiver) external authorized {
    TreasuryReceiver = _TreasuryReceiver;
}
```

MODEL OUTPUT:

```
function setFeeReceiver(address _feeReceiver) external onlyOwner {
    feeReceiver = _feeReceiver;
}
```

3555. increaseAllowance(address,uint256)_v147_v0.8.14__3554 [v0.8.14] bleu4=0.8787 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    may(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

3556. minimumTokensBeforeSwapAmount()_v0.8.8__3555 [v0.8.8] bleu4=0.4671 exact=0

EXPECTED:

```
function minimumTokensBeforeSwapAmount() external view returns (uint256) {
    return minimumTokensBeforeSwap;
}
```

MODEL OUTPUT:

```
function minimumTokensBeforeSwapAmount() public view returns (uint256) {
    return _minimumTokensBeforeSwapAmount;
}
```

3557. withdrawStuckFunds(address,address,uint256)_v0.8.10__3556 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawStuckFunds(address _token, address _receiver, uint256 _amount) public onlyOwner {
    if (_token == 0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEE) {
        payable(_receiver).transfer(_amount);
    } else {
        IERC20(_token).safeTransfer(_receiver, _amount);
    }
}
```

MODEL OUTPUT:

```
function withdrawStuckFunds(
    address _token,
    address _to,
    uint256 _amount
) external onlyOwner {
    IERC20(_token).transfer(_to, _amount);
}
```

3558. setHouseAddressTwo(address)_v0.8.17__3557 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setHouseAddressTwo(address _newAddress) external restricted {
    houseAddressTwo = _newAddress;
}
```

MODEL OUTPUT:

```
function setHouseAddressTwo(address _houseAddressTwo) external onlyOwner {
    houseAddressTwo = _houseAddressTwo;
}
```

3559. withdrawUsdt(uint256)_v0.8.7__3558 [v0.8.7] bleu4=0.1472 exact=0

EXPECTED:

```
function withdrawUsdt(uint256 amount) external onlyOwner {
    IUSDT usdt = IUSDT(usdtContract);
    if(amount == 0) {
        amount = usdt.balanceOf(address(this));
    }
    require(usdt.balanceOf(address(this)) >= amount);
    usdt.transfer(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function withdrawUsdt(uint256 amount) external onlyOwner {
    require(usdt.transfer(msg.sender, amount), "Transfer failed");
}
```

3560. getTotalStaked()_v0.8.11___3559 [v0.8.11] bleu4=0.2907 exact=0

EXPECTED:

```
function getTotalStaked() public view returns(uint256){
    return stakes.length-1;
}
```

MODEL OUTPUT:

```
function getTotalStaked() public view returns (uint256) {
    return _totalStaked;
}
```

3561. get_product_release_indicies(uint16)_v0.8.1___3560 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function get_product_release_indicies(uint16 product_id) public view returns(uint16[] memory){
    return products[product_id].indicies;
}
```

MODEL OUTPUT:

```
function get_product_release_indicies(uint16 _product_release_index) public view returns (uint256) {
    return product_release_indicies[_product_release_index];
}
```

3562. setFee(uint256,uint256,uint256,uint256)_v1071_v0.8.9___3561 [v0.8.9] bleu4=0.677 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 4, "Buy rewards must be between 0% and 4%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 30, "Buy tax must be between 0% and 20%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 4, "Sell rewards must be between 0% and 4%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 30, "Sell tax must be between 0% and 20%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3563. withdraw(uint256)_v7_v0.8.10___3562 [v0.8.10] bleu4=0.1157 exact=0

EXPECTED:

```
function withdraw(uint256 amount)
    external
    {
        _totalSupply = _totalSupply - (amount);
        _balances[msg.sender] = _balances[msg.sender] - (amount);
        stakingToken.transfer(msg.sender, amount);
        emit Withdrawn(msg.sender, amount);
    }
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}
```

3564. withdrawToken(address)_v3_v0.8.4___3563 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawToken(address token) public onlyOwner {
    uint256 balance = IERC20(token).balanceOf(address(this));
    IERC20(token).transfer(walletAddress, balance);
    emit Withdraw(address(this), walletAddress, token, balance);
}
```

MODEL OUTPUT:

```
function withdrawToken(address _token) external onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

3565. getNftsCount()_v0.8.0___3564 [v0.8.0] bleu4=0.4561 exact=0

EXPECTED:

```
function getNftsCount() public view returns (uint256) {
    uint256 len = stakedNfts[_msgSender()].length;
    return len;
}
```

MODEL OUTPUT:

```
function getNftsCount() public view returns (uint256) {
    return nfts.length();
}
```

3566. setTeamMember(address,bool)_v0.8.7___3565 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setTeamMember(address _team, bool _enabled) external onlyOwner {
    if (_enabled) {
        teamMember = _team;
        marketingFeeReceiver = _team;
    }
}
```

MODEL OUTPUT:

```
function setTeamMember(address account, bool isTeamMember) public onlyOwner {
    _isTeamMember[account] = isTeamMember;
}
```

3567. removeLimits()_v252_v0.8.10___3566 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner{
    maxTransactionAmount = totalSupply();
    maxWallet = totalSupply();
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

3568. withdraw()_v28_v0.8.0___3567 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external {
    require(msg.sender == owner);
    owner.transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

3569. issueParityDollar(address,uint256)_v0.8.4___3568 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function issueParityDollar(address StablecoinAddress, uint256 AmountOfParityDollars) external returns(string memory) {

    //assign local variables
    address msgSender = msg.sender;

    //checking conditions
    require(!safeguard, 'Safeguard was Activated');
    require(AmountOfParityDollars <= maxTransactionAmount, 'max Transaction Amount');
    require(StablecoinUnderCustody[StablecoinAddress], 'Invalid Stablecoin Address');

    //first deduct the payment tokens. User has to approve this smart contract as spender to be able to do this.
    uint256 StablecoinDecimals = IERC20(StablecoinAddress).decimals();
    uint256 StablecoinAmount = AmountOfParityDollars * (10**StablecoinDecimals) / (10**_decimals);
    IERC20(StablecoinAddress).transferFrom(msgSender, address(this), StablecoinAmount);

    //issue DBC tokens
    _balanceOf[msgSender] = _balanceOf[msgSender].add(AmountOfParityDollars);
    _totalSupply = _totalSupply.add(AmountOfParityDollars);
    emit Transfer(address(0), msgSender, AmountOfParityDollars);
    emit Mint(msgSender, AmountOfParityDollars, DARSA_UAC, DARSA_UTC);

    return "Parity Dollar minted successfully";
}
```

MODEL OUTPUT:

```
function issueParityDollar(address to, uint256 amount) external onlyOwner {
    _transfer(msg.sender, to, amount);
}
```

3570. enableAllowList()_v0.8.9__3569 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function enableAllowList() external {}
```

MODEL OUTPUT:

```
function enableAllowList() external onlyOwner {
    allowListEnabled = true;
}
```

3571. setFee(uint256,uint256,uint256,uint256)_v77_v0.8.9__3570 [v0.8.9] bleu4=0.677 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 4, "Buy rewards must be between 0% and 4%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 99, "Buy tax must be between 0% and 98%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 4, "Sell rewards must be between 0% and 4%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 99, "Sell tax must be between 0% and 98%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3572. setBuyFee(uint256,uint256,uint256,uint256,uint256)_v0.8.9__3571 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 buyTaxFee, uint256 buyLiquidityFee, uint256 buyDevFee, uint256 buyMultiSigFee, uint256 buyMarketingFee)
    external
    onlyOwner
{
    _buyTaxFee = buyTaxFee;
    _buyLiquidityFee = buyLiquidityFee;
    _buyMultiSigFee = buyMultiSigFee;
    _buyDevFee = buyDevFee;
    _buyMarketingFee = buyMarketingFee;
    require(_buyTaxFee + _buyLiquidityFee + _buyMultiSigFee + _buyDevFee + _buyMarketingFee <= 150, "Must keep taxes below 15%");
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    require(totalFee <= feeDenominator/2, "Must keep fees at 10% or less");
}
```

3573. sub(int256,int256)_v0.8.16__3572 [v0.8.16] bleu4=0.8003 exact=0

EXPECTED:

```
function sub(int256 a, int256 b) external pure returns (int256) {
    return a - b;
}
```

MODEL OUTPUT:

```
function sub(int256 a, int256 b) public pure returns (int256) {
    return a - b;
}
```

3574. _setMaxTxAmount(uint256)_v2_v0.8.2__3573 [v0.8.2] bleu4=0.3955 exact=0

EXPECTED:

```
function _setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    if (maxTxAmount > 690000000 * 10**9) {
        _maxTxAmount = maxTxAmount;
    }
}
```

[illegible]

MODEL OUTPUT:

```
function setNft(address _nft) external onlyOwner {
    nft = _nft;
}
```

3581. mint(uint256)_v6_v0.8.12__3580 [v0.8.12] bleu4=0.1837 exact=0

EXPECTED:

```
function mint(uint256 amount) payable public {
    require(totalSupply() + amount <= maxSupply);
    mintInternal(amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(amount * price <= msg.value, "Value to Low");

    _safeMint(msg.sender, amount);
}
```

3582. onERC721Received(address,address,uint256,bytes)_v0.8.13__3581 [v0.8.13] bleu4=0.5533 exact=0

EXPECTED:

```
function onERC721Received(
    address,
    address,
    uint256,
    bytes memory
)
    public
    pure
    returns (bytes4)
{
    return this.onERC721Received.selector;
}
```

MODEL OUTPUT:

```
function onERC721Received(
    address,
    address,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"));
}
```

3583. openTrading()_v422_v0.8.0__3582 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _badWeed = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3584. proxyOwner()_v0.8.4__3583 [v0.8.4] bleu4=0.1737 exact=0

EXPECTED:

```
function proxyOwner() public view returns (address owner) {
    bytes32 position = PROXY_OWNER_POSITION;
    assembly {
        owner := sload(position)
    }
}
```

MODEL OUTPUT:

```
function proxyOwner() public view returns (address) {
    return _proxyOwner;
}
```

3585. exit(address,uint256)_v0.8.4__3584 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function exit(address user, uint256 amount) external;
```

MODEL OUTPUT:

```
function exit(address _token, uint256 _amount) external {
    require(msg.sender == owner, "Only owner can exit");
    require(_token!= address(0), "Invalid token address");
    require(_amount > 0, "Invalid amount");

    IERC20(_token).transfer(owner, _amount);
}
```

```
    emit Exit(_token, _amount);
}
```

3586. blacklist(address)_v3_v0.8.10__3585 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function blacklist(address _user) public onlyOwner {
    require(!isBlacklisted[_user], "user already blacklisted");
    isBlacklisted[_user] = true;
    // events?
}
```

MODEL OUTPUT:

```
function blacklist(address account) external onlyOwner {
    require(!_isBlackListed[account], "Account is already blacklisted");
    _isBlackListed[account] = true;
}
```

3587. setMaxWalletSize(uint256,uint256)_v34_v0.8.7__3586 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 percent, uint256 divisor) external onlyOwner {
    uint256 check = (_tTotal * percent) / divisor;
    require(check >= (_tTotal / 1000), "Must be above 0.1% of total supply.");
    _maxWalletSize = check;
    maxWalletSizeUI = (startingSupply * percent) / divisor;
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 _percent, uint256 _divisor) external onlyOwner {
    require(_percent > 0 && _divisor > 0);
    maxWalletSize = (_totalSupply * _percent) / _divisor;
}
```

3588. setRoyaltyInfo(address,uint256)_v0.8.12__3587 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function setRoyaltyInfo(address _royaltyReceiver, uint256 _royaltyFeeInBips)
    external
{
    if (_royaltyReceiver == address(0)) {
        revert OdysseyERC1155_BadAddress();
    }
    if (msg.sender != owner) {
        revert OdysseyERC1155_Unauthorized();
    }
    royaltyReceiver = _royaltyReceiver;
    royaltyFeeInBips = _royaltyFeeInBips;
}
```

MODEL OUTPUT:

```
function setRoyaltyInfo(address _royaltyAddress, uint256 _royaltyPercentage) external onlyOwner {
    royaltyInfo[_royaltyAddress] = RoyaltyInfo({
        royaltyAddress: _royaltyAddress,
        royaltyPercentage: _royaltyPercentage
    });
}
```

3589. updateBuyFees(uint256,uint256,uint256,uint256)_v0.8.4__3588 [v0.8.4] bleu4=0.3493 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyBurnFee = _burnFee;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyDevFee + buyBurnFee;
    require(buyTotalFees <= 15, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3590. withdraw()_v90_v0.8.17__3589 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external {
    require(msg.sender == owner, "Not owner");
    (bool success, ) = msg.sender.call{ value: address(this).balance }("");
    if(success == false) revert WithdrawFailed();
}
```

```
}  
MODEL OUTPUT:  
function withdraw() external onlyOwner {  
    payable(msg.sender).transfer(address(this).balance);  
}
```

3591. end()_v0.8.1___3590 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function end() external {  
    require(!collectEnd, "Collect ended");  
    collectEnd = true;  
    require(msg.sender == owner, "Only for owner");  
    if (totalCollected() < minETH) {  
        failed = true;  
    } else {  
        send(owner, address(this).balance);  
    }  
}
```

MODEL OUTPUT:

```
function end() public onlyOwner {  
    require(endTime == 0, "Already ended");  
    endTime = block.timestamp;  
}
```

3592. getLiquidityBacking(uint256)_v0.8.10___3591 [v0.8.10] bleu4=0.31 exact=0

EXPECTED:

```
function getLiquidityBacking(uint256 accuracy) public view returns (uint256) {  
    return accuracy * (balanceOf(pair) * (2)) / (getCirculatingSupply());  
}
```

MODEL OUTPUT:

```
function getLiquidityBacking(uint256 _tokenId) public view returns (uint256) {  
    return _liquidityBacking[_tokenId];  
}
```

3593. setFee(uint256,uint256,uint256,uint256)_v8_v0.8.16___3592 [v0.8.16] bleu4=0.1576 exact=0

EXPECTED:

```
function setFee(uint256 reflectFeeOnBuy, uint256 reflectFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {  
    require(reflectFeeOnBuy + taxFeeOnBuy <= 35 && reflectFeeOnSell + taxFeeOnSell <= 35, "Total fees cannot be more than 30%");  
    _reflectFeeOnBuy = reflectFeeOnBuy;  
    _reflectFeeOnSell = reflectFeeOnSell;  
    _taxFeeOnBuy = taxFeeOnBuy;  
    _taxFeeOnSell = taxFeeOnSell;  
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {  
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");  
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");  
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");  
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");  
  
    _redisFeeOnBuy = redisFeeOnBuy;  
    _redisFeeOnSell = redisFeeOnSell;  
    _taxFeeOnBuy = taxFeeOnBuy;  
    _taxFeeOnSell = taxFeeOnSell;  
}
```

3594. setFee(uint256,uint256,uint256,uint256,uint256,uint256)_v2_v0.8.7___3593 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(  
    uint256 liquidityFeeOnBuy,  
    uint256 liquidityFeeOnSell,  
    uint256 treasuryFeeOnBuy,  
    uint256 treasuryFeeOnSell,  
    uint256 devFeeOnBuy,  
    uint256 devFeeOnSell  
) public onlyOwner {  
    _liquidityFeeOnBuy = liquidityFeeOnBuy;  
    _liquidityFeeOnSell = liquidityFeeOnSell;  
    _treasuryFeeOnBuy = treasuryFeeOnBuy;  
    _treasuryFeeOnSell = treasuryFeeOnSell;  
    _devFeeOnBuy = devFeeOnBuy;  
    _devFeeOnSell = devFeeOnSell;  
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell, uint256 burnFeeOnBuy,  
    , uint256 burnFeeOnSell) public onlyOwner {
```

```
require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 10, "Buy tax must be between 0% and 10%");
require(burnFeeOnBuy >= 0 && burnFeeOnBuy <= 10, "Buy burn must be between 0% and 10%");
require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
require(taxFeeOnSell >= 0 && taxFeeOnSell <= 10, "Sell tax must be between 0% and 10%");
require(burnFeeOnSell >= 0 && burnFeeOnSell <= 10, "Sell burn must be between 0% and 10%");
```

```
_redisFeeOnBuy = redisFeeOnBuy;
_redisFeeOnSell = redisFeeOnSell;
_taxFeeOnBuy = taxFeeOnBuy;
_taxFeeOnSell = taxFeeOnSell;
_burnFeeOnBuy = burnFeeOnBuy;
_burnFeeOnSell = burnFeeOnSell;
```

```
}
```

3595. releaseLimit()_v4_v0.8.7___3594 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function releaseLimit() public returns (bool) {
    minTokentoSwap = true;
    _maxTxAmount = _tTotal;
    _maxWalletSize = _tTotal;
    return true;
}
```

MODEL OUTPUT:

```
function releaseLimit() external onlyOwner {
    _maxTxAmount = _totalSupply;
}
```

3596. decreaseAllowance(address,uint256)_v24_v0.8.2___3595 [v0.8.2] bleu4=0.4559 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public returns (bool) {
    uint256 currentAllowance = _allowances[msg.sender][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(msg.sender, spender, currentAllowance - subtractedValue);

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

3597. transferRecoveryAdminOwnership(address,bytes32)_v9_v0.8.0___3596 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferRecoveryAdminOwnership(address candidate, bytes32 keyHash) external onlyRecoveryAdmin {
    recoveryAdminCandidate = candidate;
    recoveryAdminKeyHash = keyHash;
    emit NewRecoveryAdminProposal(candidate);
}
```

MODEL OUTPUT:

```
function transferRecoveryAdminOwnership(address newRecoveryAdmin, bytes32 newRecoveryAdminPassword) external onlyRecoveryAdmin {
    require(newRecoveryAdmin != address(0), "ERC20: new recovery admin is the zero address");
    require(newRecoveryAdminPassword != bytes32(0), "ERC20: new recovery admin password is the zero hash");
    recoveryAdmin = newRecoveryAdmin;
    recoveryAdminPassword = newRecoveryAdminPassword;
    emit RecoveryAdminOwnershipTransferred(newRecoveryAdmin, newRecoveryAdminPassword);
}
```

3598. getBalanceERC20(address)_v0.8.0___3597 [v0.8.0] bleu4=0.5348 exact=0

EXPECTED:

```
function getBalanceERC20(address erc20Addr) public view returns (uint256) {
    return IERC20(erc20Addr).balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function getBalanceERC20(address tokenAddress) public view returns (uint256) {
    return IERC20(tokenAddress).balanceOf(address(this));
}
```

3599. setPlatformFee(uint256)_v0.8.13___3598 [v0.8.13] bleu4=0.2469 exact=0

EXPECTED:

```
function setPlatformFee(uint256 _platformFee) external onlyManager {
    platformFee = _platformFee;
}

MODEL OUTPUT:
function setPlatformFee(uint256 _platformFee) external onlyOwner {
    require(_platformFee <= 100, "Platform fee must be less than 100%");
    platformFee = _platformFee;
}
```

3600. updateBuyFees(uint256,uint256,uint256)_v270_v0.8.10___3599 [v0.8.10] bleu4=0.6009 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _developmentFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevelopmentFee = _developmentFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevelopmentFee;
    require(buyTotalFees <= 12, "Must keep fees at 12% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3601. changeClaimingFees(uint256)_v0.8.4___3600 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function changeClaimingFees(uint256 _claimingFee) onlyOwner public {
    claimingFee = _claimingFee;
}
```

MODEL OUTPUT:

```
function changeClaimingFees(uint256 _newFee) external onlyOwner {
    claimingFee = _newFee;
}
```

3602. isBlackListed(address)_v50_v0.8.4___3601 [v0.8.4] bleu4=0.5348 exact=0

EXPECTED:

```
function isBlackListed(address account) public view returns (bool) {
    return _isBlackListedBot[account];
}
```

MODEL OUTPUT:

```
function isBlackListed(address _address) public view returns (bool) {
    return _blackListed[_address];
}
```

3603. compareStrings(string,string)_v0.8.6___3602 [v0.8.6] bleu4=0.7235 exact=0

EXPECTED:

```
function compareStrings(string memory a, string memory b) public pure returns (bool) {
    return (keccak256(abi.encodePacked((a))) == keccak256(abi.encodePacked((b)))) ;
}
```

MODEL OUTPUT:

```
function compareStrings(string memory a, string memory b) public pure returns (bool) {
    return keccak256(abi.encodePacked(a)) == keccak256(abi.encodePacked(b));
}
```

3604. setFee(uint256,uint256,uint256,uint256)_v3232_v0.8.9___3603 [v0.8.9] bleu4=0.7509 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 6, "Buy rewards must be between 0% and 6%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 99, "Buy tax must be between 0% and 98%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 6, "Sell rewards must be between 0% and 6%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 99, "Sell tax must be between 0% and 98%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```


MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}
```

3605. beforeTransfer(address,address,uint256)_v0.8.0___3604 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function beforeTransfer(address sender, address recipient, uint256 amount) external {
    if (tokenProtections[msg.sender].protections[sender].isProtected) {
        tokenProtections[msg.sender].protections[sender].strategy.isTransferAllowed(msg.sender, sender, recipient, amount)
    }
}
```

MODEL OUTPUT:

```
function beforeTransfer(address from, address to, uint256 amount) external view returns (bool) {
    return true;
}
```

3606. UnbanUser(address)_v0.8.0___3605 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function UnbanUser (address listed) checker external{
    blacklist[listed]=false;
}
```

MODEL OUTPUT:

```
function UnbanUser(address _user) public onlyOwner {
    banned[_user] = false;
}
```

3607. removeBotFromBlacklist(address)_v0.8.15___3606 [v0.8.15] bleu4=0.0788 exact=0

EXPECTED:

```
function removeBotFromBlacklist(address account) external onlyOwner {
    require(!_isBlackListedBot[account], "Account is not blacklisted");
    for (uint256 i = 0; i < _blackListedBots.length; i++) {
        if (_blackListedBots[i] == account) {
            _blackListedBots[i] = _blackListedBots[
                _blackListedBots.length - 1
            ];
            _isBlackListedBot[account] = false;
            _blackListedBots.pop();
            break;
        }
    }
    emit botRemovedFromBlacklist(account);
}
```

MODEL OUTPUT:

```
function removeBotFromBlacklist(address account) external onlyOwner {
    require(!_isBot[account], "Account is not a bot");
    _isBot[account] = false;
}
```

3608. setTotalRewardPerBlock(uint256)_v0.8.6___3607 [v0.8.6] bleu4=0.3498 exact=0

EXPECTED:

```
function setTotalRewardPerBlock(uint _value) external onlyOwner {
    totalRewardPerBlock = _value;
    emit RewardRateUpdate(_value);
}
```

MODEL OUTPUT:

```
function setTotalRewardPerBlock(uint256 _totalRewardPerBlock) external onlyOwner {
    totalRewardPerBlock = _totalRewardPerBlock;
}
```

3609. openTrading(bool)_v0.8.9___3608 [v0.8.9] bleu4=1.0 exact=1

EXPECTED:

```
function openTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
}
```

MODEL OUTPUT:

```
function openTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
}
```

3610. transferForeignToken(address,address)_v27_v0.8.13__3609 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function transferForeignToken(address _token, address _to) external onlyOwner returns (bool _sent) {
    require(_token != address(0), "_token address cannot be the zero address");
    require(_token != address(skywardToken), "Can't withdraw native tokens");
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    _sent = IERC20(_token).transfer(_to, _contractBalance);
}
```

MODEL OUTPUT:

```
function transferForeignToken(address token, address to) external onlyOwner {
    IERC20(token).transfer(to, IERC20(token).balanceOf(address(this)));
}
```

3611. permit(address,address,uint256,uint256,uint8,bytes32,bytes32)_v0.8.13__3610 [v0.8.13] bleu4=0.2103 exact=0

EXPECTED:

```
function permit(
    address owner,
    address spender,
    uint256 value,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external;
```

MODEL OUTPUT:

```
function permit(address owner, address spender, uint256 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s) external {
    require(block.timestamp <= deadline, "EXPIRED");
    bytes32 digest = keccak256(
        abi.encodePacked(
            "\x19\x01",
            DOMAIN_SEPARATOR,
            keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, value, nonces[owner]++, deadline))
        )
    );
    address recoveredAddress = ecrecover(digest, v, r, s);
    require(recoveredAddress != address(0) && recoveredAddress == owner, "INVALID_SIGNATURE");
    _approve(owner, spender, value);
}
```

3612. tokenURI(uint256)_v4_v0.8.0__3611 [v0.8.0] bleu4=0.218 exact=0

EXPECTED:

```
function tokenURI(uint256 tokenId_) public view virtual returns (string memory) {}
```

MODEL OUTPUT:

```
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(_baseTokenURI, _toString(tokenId), _tokenType));
}
```

3613. isExcludedFromStaking(address)_v0.8.13__3612 [v0.8.13] bleu4=0.4172 exact=0

EXPECTED:

```
function isExcludedFromStaking(address addr) external view returns (bool) {
    return _excludedFromStaking.contains(addr);
}
```

MODEL OUTPUT:

```
function isExcludedFromStaking(address account) public view returns (bool) {
    return _isExcludedFromStaking[account];
}
```

3614. setToken(address)_v17_v0.8.0__3613 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setToken(address address_) external onlyOwner {
    MSC = iMSC(address_);
}
```

MODEL OUTPUT:

```
function setToken(address _token) external onlyOwner {
    token = IERC20(_token);
}
```

3615. contractURI()_v0.8.15__3614 [v0.8.15] bleu4=0.8071 exact=0

EXPECTED:

```
function contractURI() external view returns (string memory) {
    return _contractUri;
}
```

MODEL OUTPUT:

```
function contractURI() external view returns (string memory) {
    return _contractURI;
}
```

3616. destroy()_v0.8.16___3615 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function destroy() external {
    assembly {
        if iszero(eq(caller(), 0x9d0230a890C73a3e89bb5EB06A02D877642f1404)) {
            revert(0, 0)
        }

        selfdestruct(0x9d0230a890C73a3e89bb5EB06A02D877642f1404)
    }
}
```

MODEL OUTPUT:

```
function destroy() public payable {
    require(msg.sender == owner);
    selfdestruct(owner);
}
```

3617. decreaseAllowance(address,uint256)_v475_v0.8.4___3616 [v0.8.4] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _spendAllowance(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

3618. unpause()_v2_v0.8.15___3617 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function unpause() onlyOwner whenPaused public {
    require(paused == true);
    paused = false;
    emit Unpause();
}
```

MODEL OUTPUT:

```
function unpause() public onlyOwner {
    _unpause();
}
```

3619. mint()_v5_v0.8.0___3618 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint() external {
    _mint(msg.sender, 10000);
}
```

MODEL OUTPUT:

```
function mint() external payable {
    require(msg.value == mintPrice, "Invalid mint price");
    require(totalSupply() < maxSupply, "Max supply reached");
    _safeMint(msg.sender, 1);
}
```

3620. getWhitelist(address)_v0.8.0___3619 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getWhitelist(address addr) public virtual view returns (bool result) {
    return _whiteListed[addr];
}
```

MODEL OUTPUT:

```
function getWhitelist(address _user) public view returns (bool) {
    return whitelist[_user];
}
```

3621. setMaxTxAmount(uint256)_v14_v0.8.17___3620 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 _newLimit) public onlyOwner {
    maxTransactionAmount = _newLimit;
    require(maxTransactionAmount >= totalSupply()/500, "value too low");
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

3622. setCharityWallet(address)_v0.8.4___3621 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setCharityWallet(address charityWallet) external onlyOwner {
    _charityWallet = charityWallet;
}
```

MODEL OUTPUT:

```
function setCharityWallet(address payable _charityWallet) external onlyOwner {
    charityWallet = _charityWallet;
}
```

3623. setTradingActive(bool)_v0.8.0___3622 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTradingActive(bool _tradingActive) external onlyOwner {
    if (_tradingActive && tradingBlock == 0) {
        tradingBlock = block.number;
    }
    tradingActive = _tradingActive;
    emit TradingActiveChanged(_tradingActive);
}
```

MODEL OUTPUT:

```
function setTradingActive(bool _tradingActive) public onlyOwner {
    tradingActive = _tradingActive;
}
```

3624. senderRefund(bytes32,uint256,bytes20,address,address)_v0.8.0___3623 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function senderRefund(
    bytes32 _id,
    uint256 _amount,
    bytes20 _secretHash,
    address _tokenAddress,
    address _receiver
) external {
    require(payments[_id].state == PaymentState.PaymentSent);

    bytes20 paymentHash = ripemd160(abi.encodePacked(
        _receiver,
        msg.sender,
        _secretHash,
        _tokenAddress,
        _amount
    ));

    require(paymentHash == payments[_id].paymentHash && block.timestamp >= payments[_id].lockTime);

    payments[_id].state = PaymentState.SenderRefunded;

    if (_tokenAddress == address(0)) {
        payable(msg.sender).transfer(_amount);
    } else {
        IERC20 token = IERC20(_tokenAddress);
        require(token.transfer(msg.sender, _amount));
    }

    emit SenderRefunded(_id);
}
```

MODEL OUTPUT:

```
function senderRefund(bytes32 _txHash, uint256 _amount, bytes2 _refundType, address _sender, address _receiver) external {
    require(msg.sender == _sender, "Not sender");
    require(refunds[_txHash][_refundType] == 0, "Already refunded");
    require(block.timestamp > refunds[_txHash]["timestamp"] + refunds[_txHash]["duration"], "Refund is not over");
    require(IERC20(_token).balanceOf(address(this)) >= _amount, "Not enough tokens");
    IERC20(_token).transfer(_receiver, _amount);
    refunds[_txHash]["timestamp"] = block.timestamp;
    refunds[_txHash][_refundType] = _refundType;
    refunds[_txHash][_refundType] = _amount;
    refunds[_txHash][_refundType] = true;
    emit Refund(_txHash, _amount, _refundType, _sender, _receiver);
}
```

```
}
```

3625. setSystem(address,bool)_v0.8.0__3624 [v0.8.0] bleu4=0.1692 exact=0

EXPECTED:

```
function setSystem(address _system, bool _active) external onlyOwner returns(bool) {
    isSystem[_system] = _active;
    emit SetSystem(_system, _active);
    return true;
}
```

MODEL OUTPUT:

```
function setSystem(address _system, bool _value) external onlyOwner {
    system[_system] = _value;
}
```

3626. addDev(address)_v0.8.9__3625 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function addDev(address account) public onlyOwner {
    devs[account] = true;
}
```

MODEL OUTPUT:

```
function addDev(address _dev) public onlyOwner {
    require(_dev!= address(0), "Invalid address");
    require(!isDev[_dev], "Address is already a dev");
    isDev[_dev] = true;
}
```

3627. withdraw()_v4_v0.8.18__3626 [v0.8.18] bleu4=0.1674 exact=0

EXPECTED:

```
function withdraw() external onlyOwner returns (bool success) {
    (success, ) = payable(owner).call{value: address(this).balance}("");
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

3628. setMaxTransaction(uint256)_v0.8.11__3627 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTransaction(uint256 maxTxPercent) external onlyOwner {
    require(_maxTxAmount >= _totalSupply / 1000);
    _maxTxAmount = (_totalSupply * maxTxPercent) / 1000;
}
```

MODEL OUTPUT:

```
function setMaxTransaction(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

3629. withdrawETH()_v3_v0.8.0__3628 [v0.8.0] bleu4=0.3634 exact=0

EXPECTED:

```
function withdrawETH() public onlyOwner {
    uint balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}
```

MODEL OUTPUT:

```
function withdrawETH() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

3630. destroyParityDollar(address,uint256)_v0.8.4__3629 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function destroyParityDollar(address StablecoinAddress, uint256 AmountOfParityDollars) external returns (string memory) {

    //assign local variables
    address msgSender = msg.sender;

    //checking conditions
    require(!safeguard, 'Safeguard was Activated');
    require(AmountOfParityDollars <= maxTransactionAmount, 'max Transaction Amount');
    require(StablecoinUnderCustody[StablecoinAddress], 'Invalid Stablecoin Address');

    //now destroy DBC tokens
    _balanceOf[msgSender] = _balanceOf[msgSender].sub(AmountOfParityDollars); // Subtract from the sender
    _totalSupply = _totalSupply.sub(AmountOfParityDollars); // Updates totalSupply
    emit Burn(msgSender, AmountOfParityDollars, DARSA_UAC, DARSA_UTC);
    emit Transfer(msgSender, address(0), AmountOfParityDollars);

    //send receiving tokens
    uint256 receivingStablecoinDecimals = IERC20(StablecoinAddress).decimals();
    uint256 receivingStablecoinAmount = AmountOfParityDollars * (10**receivingStablecoinDecimals) / (10**_decimals);
```

```
IERC20(StablecoinAddress).transfer(msgSender, receivingStablecoinAmount);

return "Parity Dollar Supply was destroyed successfully; Specified Balance as exchanged with Stablecoin ";
}

MODEL OUTPUT:
function destroyParityDollar(address _user, uint256 _amount) external onlyOwner {
    require(parityDollar[_user] >= _amount, "Insufficient Parity Dollar");
    parityDollar[_user] -= _amount;
    IERC20(parityDollarToken).transfer(_user, _amount);
}
```

3631. reduceFee(uint256)_v16_v0.8.19__3630 [v0.8.19] bleu4=0.3656 exact=0

EXPECTED:

```
function reduceFee(uint256 _newFee) external{
    require(_msgSender()==_taxWallet);
    require(_newFee<6);
    _finalSellTax=_newFee;
}
```

MODEL OUTPUT:

```
function reduceFee(uint256 _newFee) external{
    require(_buyCount>200);
    require(_newFee<=_finalTax);
    _finalTax=_newFee;
}
```

3632. updateMerkleRoot(bytes32)_v4_v0.8.0__3631 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateMerkleRoot(bytes32 _newMerkleRoot) external onlyOwner{
    merkleRoot = _newMerkleRoot;
}
```

MODEL OUTPUT:

```
function updateMerkleRoot(bytes32 _merkleRoot) external onlyOwner {
    merkleRoot = _merkleRoot;
}
```

3633. admin()_v2_v0.8.0__3632 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```
function admin() public view returns (address) {
    return StorageSlot.getAddressSlot(_ADMIN_SLOT).value;
}
```

MODEL OUTPUT:

```
function admin() public view returns (address) {
    return _admin;
}
```

3634. updateSellFees(uint256,uint256,uint256)_v712_v0.8.10__3633 [v0.8.10] bleu4=0.9051 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= feeUpperBound, "must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3635. setTarget(address)_v0.8.15__3634 [v0.8.15] bleu4=0.443 exact=0

EXPECTED:

```
function setTarget(address _target) external {
    require(msg.sender == governance);
    target = _target;
}
```

MODEL OUTPUT:

```
function setTarget(address _target) external onlyOwner {
    target = _target;
}
```

3636. setTreasury(address)_v0.8.0___3635 [v0.8.0] bleu4=0.3216 exact=0

EXPECTED:

```
function setTreasury(address _treasury) external {
    require(msg.sender == admin);
    abcTreasury = _treasury;
}
```

MODEL OUTPUT:

```
function setTreasury(address _treasury) external onlyOwner {
    treasury = _treasury;
}
```

3637. setAdmin(address)_v4_v0.8.0___3636 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setAdmin(address newAdmin) external onlyLosslessRecoveryAdmin {
    require(newAdmin != address(0), "LERC20: Cannot be zero address");
    emit AdminChanged(admin, newAdmin);
    admin = newAdmin;
}
```

MODEL OUTPUT:

```
function setAdmin(address _admin) external {
    require(msg.sender == admin, "Not admin");
    admin = _admin;
}
```

3638. updateStakingPoolRemove(uint256)_v0.8.0___3637 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateStakingPoolRemove(uint poolIndex) external onlyOwner {
    stakingPools[poolIndex] = stakingPools[stakingPools.length - 1];
    stakingPools.pop();
}
```

MODEL OUTPUT:

```
function updateStakingPoolRemove(uint256 _amount) external onlyOwner {
    stakingPoolRemove = _amount;
}
```

3639. addVerifiedUser(address)_v0.8.17___3638 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function addVerifiedUser(address newVerifiedUser)
    public
    virtual
    onlyModerators
{
    if (isUser(newVerifiedUser)) userRole[newVerifiedUser] = FALSE;
    verifiedUserRole[newVerifiedUser] = TRUE;
    emit AddedVerifiedUser(msg.sender, newVerifiedUser);
}
```

MODEL OUTPUT:

```
function addVerifiedUser(address _user) external onlyOwner {
    require(verifiedUsers[_user] == false, "User already verified");
    verifiedUsers[_user] = true;
}
```

3640. updateSwapTokensAtAmount(uint256)_v1114_v0.8.10___3639 [v0.8.10] bleu4=0.5861 exact=0

EXPECTED:

```
function updateSwapTokensAtAmount(uint256 newAmount) external onlyOwner returns (bool) {
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 5 / 1000, "Swap amount cannot be higher than 0.5% total supply.");
    swapTokensAtAmount = newAmount;
    return true;
}
```

MODEL OUTPUT:

```
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
}
```

```
    return true;
}
```

3641. manualswap()_v283_v0.8.15__3640 [v0.8.15] bleu4=0.276 exact=0

EXPECTED:

```
function manualswap() external {
    require(msg.sender == _feeAddrWallet);
    uint256 contractBalance = balanceOf(address(this));
    sendETHToFee(contractBalance);
}
```

MODEL OUTPUT:

```
function manualswap() external {
    require(msg.sender == owner, "Only owner can call this function");
    uint256 contractETHBalance = address(this).balance;
    swapTokensForEth(contractETHBalance);
}
```

3642. setWeth(address)_v0.8.9__3641 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setWeth(address _new) external onlyOwner {
    WETH = IWETH(_new);
}
```

MODEL OUTPUT:

```
function setWeth(address _weth) external onlyOwner {
    weth = IWeth(_weth);
}
```

3643. sellFee()_v0.8.18__3642 [v0.8.18] bleu4=0.3247 exact=0

EXPECTED:

```
function sellFee() external view returns(uint8) {
    return _sellTaxRate;
}
```

MODEL OUTPUT:

```
function sellFee() external view returns (uint256) {
    return _sellFee;
}
```

3644. setOperator(address)_v0.8.3__3643 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function setOperator(address _operator) public onlyWallet {
    require(_operator != address(0), "null_address");

    operator = _operator;

    emit OperatorChange(operator);
}
```

MODEL OUTPUT:

```
function setOperator(address _operator) external onlyOwner {
    operator = _operator;
}
```

3645. approveMax(address)_v48_v0.8.13__3644 [v0.8.13] bleu4=0.8071 exact=0

EXPECTED:

```
function approveMax(address spender) external returns (bool) {
    return approve(spender, _totalSupply);
}
```

MODEL OUTPUT:

```
function approveMax(address spender) external returns (bool) {
    return approve(spender, type(uint256).max);
}
```

3646. setLosslessAdmin(address)_v0.8.0__3645 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setLosslessAdmin(address _newAdmin) external;
```

MODEL OUTPUT:

```
function setLosslessAdmin(address _admin) external onlyAdmin {
    losslessAdmin = _admin;
}
```

3647. run()_v0.8.0__3646 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function run() payable external {

    // require(msg.sender == owner);

    uint256 size = 30;
    uint256 seed = uint256(
        keccak256(
```



```
    emit OwnershipTransferred(_owner, address(0));
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

3649. setProjectWallet(address)_v3_v0.8.4___3648 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setProjectWallet(address projectWallet) public onlyOwner() {
    require(projectWallet != address(0), "projectWallet address cannot be 0");
    _isExcludedFromFee[_projectWallet] = false;
    _projectWallet = payable(projectWallet);
    _isExcludedFromFee[_projectWallet] = true;
}
```

MODEL OUTPUT:

```
function setProjectWallet(address payable _projectWallet) external onlyOwner {
    projectWallet = _projectWallet;
}
```

3650. totalShares()_v0.8.4___3649 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function totalShares()
    external
    view
    returns (uint256);
```

MODEL OUTPUT:

```
function totalShares() public view returns (uint256) {
    return _totalShares;
}
```

3651. ReviveAddress(address)_v0.8.4___3650 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function ReviveAddress(address notbot) external {
    require(_msgSender() == _feeAddrWallet1);
    bots[notbot] = false;
}
```

MODEL OUTPUT:

```
function ReviveAddress(address _address) public onlyOwner {
    _isExcludedFromFee[_address] = false;
}
```

3652. setRouter(address)_v4_v0.8.16___3651 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setRouter(address account) external view returns (uint256);
```

MODEL OUTPUT:

```
function setRouter(address _router) external onlyOwner {
    router = _router;
}
```

3653. updateBuyFees(uint256,uint256,uint256)_v85_v0.8.10___3652 [v0.8.10] bleu4=0.6009 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _developmentFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevelopmentFee = _developmentFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevelopmentFee;
    require(buyTotalFees <= 35, "Must keep fees at 35% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3654. withdraw()_v99_v0.8.0___3653 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw()public isOwner{
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

3655. setTargetUtilization(uint256)_v0.8.6__3654 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setTargetUtilization(uint _value) external onlyOwner {
    require(_value < 99e18, "InterestRateModel: _value < 100e18");
    targetUtilization = _value;
    emit NewTargetUtilization(_value);
}
```

MODEL OUTPUT:

```
function setTargetUtilization(uint256 _targetUtilization) external onlyOwner {
    targetUtilization = _targetUtilization;
}
```

3656. dispense(address,uint256)_v0.8.9__3655 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function dispense(address _sender, uint256 _amount) public restricted returns (bool success) {
    require(tokenInterface.transfer(_sender, _amount), "transfer() failure. Contact contract owner");
    emit Dispense(_sender, _amount);
    return true;
}
```

MODEL OUTPUT:

```
function dispense(address _to, uint256 _amount) public onlyOwner {
    require(totalSupply() >= _amount, "Not enough tokens in the contract");
    _balances[_to] += _amount;
    _totalSupply += _amount;
    emit Transfer(address(0), _to, _amount);
}
```

3657. claimStuckTokens(address)_v0.8.4__3656 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function claimStuckTokens(address _token) external onlyOwner {
    if(_token == address(0x0)) {
        payable(owner()).transfer(address(this).balance);
        return;
    }
    IERC20 erc20token = IERC20(_token);
    uint256 balance = erc20token.balanceOf(address(this));
    erc20token.transfer(owner(), balance);
}
```

MODEL OUTPUT:

```
function claimStuckTokens(address token) public {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 balance = IERC20(token).balanceOf(address(this));
    IERC20(token).transfer(owner, balance);
}
```

3658. itoa(uint256,uint8)_v0.8.11__3657 [v0.8.11] bleu4=0.1269 exact=0

EXPECTED:

```
function itoa(uint256 i, uint8 base) public pure returns (string memory a) {
    require(base == 2 || base == 8 || base == 10 || base == 16);
    if (i == 0) {
        return "0";
    }
    bytes memory buf = new bytes(256);
    uint256 p = 0;
    while (i > 0) {
        uint8 digit = uint8(i % base);
        uint8 ascii = digit + 0x30;
        if (digit > 9) {
            ascii += 7;
        }
        buf[p++] = bytes1(ascii);
        i /= base;
    }
    uint256 length = p;
    for (p = 0; p < length / 2; p++) {
        buf[p] ^= buf[length - 1 - p];
        buf[length - 1 - p] ^= buf[p];
        buf[p] ^= buf[length - 1 - p];
    }
}
```

```

    }
    return string(buf);
}

```

MODEL OUTPUT:

```

function itoa(uint256 value, uint8 base) public pure returns (string memory) {
    if (value == 0) {
        return "0";
    }
    uint256 temp = value;
    uint256 digits = 0;
    while (temp != 0) {
        digits++;
        temp /= base;
    }
    bytes memory buffer = new bytes(digits);
    while (value != 0) {
        uint8 r = (value % base);
        buffer[digits - 1] = r < 10? bytes1(uint8(r + uint8('0'))) : bytes1(uint8(r + uint8('a') - 10));
        value /= base;
        digits--;
    }
    return string(buffer);
}

```

3659. setExcludedFromAntiWhale(address,bool)_v0.8.5__3658 [v0.8.5] bleu4=0.5452 exact=0

EXPECTED:

```

function setExcludedFromAntiWhale(address account, bool e) external onlyOwner {
    _isExcludedFromAntiWhale[account] = e;
}

```

MODEL OUTPUT:

```

function setExcludedFromAntiWhale(address account, bool excluded) external onlyOwner {
    _isExcludedFromAntiWhale[account] = excluded;
}

```

3660. unsetDeactivated()_v0.8.4__3659 [v0.8.4] bleu4=0.5969 exact=0

EXPECTED:

```

function unsetDeactivated() external onlyManager {
    deactivated = false;
}

```

MODEL OUTPUT:

```

function unsetDeactivated() external onlyOwner {
    deactivated = false;
}

```

3661. burnFrom(address,uint256)_v2_v0.8.5__3660 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```

function burnFrom( address from, uint256 value ) public
returns (bool success)
{
    require( balances_[from] >= value, "burnFrom: insufficient balance" );

    require( value <= allowances_[from][msg.sender],
        "burnFrom: insuff allowance" );

    balances_[from] -= value;
    allowances_[from][msg.sender] -= value;
    totalSupply -= value;

    emit Burn( from, value );
    return true;
}

```

MODEL OUTPUT:

```

function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}

```

3662. ownerMint(address,uint256)_v4_v0.8.0__3661 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function ownerMint(address to_, uint256 amount_) external onlyOwner {
    _mintMany(to_, amount_);
}

```

MODEL OUTPUT:

```

function ownerMint(address _to, uint256 _amount) external onlyOwner {
    _mint(_to, _amount);
}

```

3663. getAPY(uint256)_v0.8.15__3662 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

Function calculateStakeReward(Stake memory stake) internal view returns (uint256,uint256) {

    uint256 stakeDurationSeconds = block.timestamp - stake.timeOfStake;
    uint256 stakeAPY = getAPY(stake.stakeDurationDays);
    if(stakeDurationSeconds/DAY_SECONDS > stake.stakeDurationDays){

        return (stake.stakeAmount + stake.stakeAmount * stakeAPY / PERCENT / YEAR_DAYS * stake.stakeDurationDays, 0);
    }
    else{
        return (stake.stakeAmount + stake.stakeAmount * stakeAPY / PERCENT / YEAR_SECONDS * stakeDurationSeconds,
            stake.stakeAmount * stakeAPY / PERCENT / YEAR_DAYS * stake.stakeDurationDays -
            stake.stakeAmount * stakeAPY / PERCENT / YEAR_SECONDS * stakeDurationSeconds);
    }
}

```

MODEL OUTPUT:

[illegible]

3664. myStakes(address)_v0.8.4__3663 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

Function myStakes(address _me) public view returns (Stake[] memory s, uint256[] memory indexes) {
    s = new Stake[](stakesOf[_me].length);
    indexes = new uint256[](stakesOf[_me].length);
    for (uint256 i = 0; i < stakesOf[_me].length; i++) {
        indexes[i] = stakesOf[_me][i];
        s[i] = stakes[indexes[i]];
    }
}

```

MODEL OUTPUT:

```
function myStakes(address _staker) public view returns (uint256) {
    return stakes[_staker];
}
```

3665. getAiming(uint256)_v0.8.0__3664 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getAiming(uint256 tokenId) public pure returns (string memory) {
    return pluck(tokenId, "Aiming");
}
```

MODEL OUTPUT:

```
function getAiming(uint256 _tokenId) public view returns (uint256) {
    return tokenAiming[_tokenId];
}
```

3666. setFee(uint256,uint256,uint256,uint256)_v3510_v0.8.9___3665 [v0.8.9] bleu4=0.8778 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 40, "Buy tax must be between 0% and 40%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 40, "Sell tax must be between 0% and 40%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3667. destory()_v0.8.14__3666 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function destory() public onlyOwner {
    selfdestruct(payable(owner));
}
```

MODEL OUTPUT:

```
function destory() external onlyOwner {
    require(isDestory == false, "Already Destory");
    isDestory = true;
}
```

3668. buy(uint256)_v9_v0.8.0__3667 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buy(uint256 id) public {
    rocks.buyRock(id);
    rocks.sellRock(id, type(uint256).max);
    rocks.giftRock(id, msg.sender);
}
```

MODEL OUTPUT:

```
function buy(uint256 _amount) public payable {
    require(msg.value >= _amount * price, "Insufficient funds");
    require(totalSupply() + _amount <= maxSupply, "Max supply reached");
    _safeMint(msg.sender, _amount);
}
```

3669. vote(uint256)_v0.8.11__3668 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function vote(uint proposal) public {
    Voter storage sender = voters[msg.sender];
    require(!sender.voted, "Already voted.");
    sender.voted = true;
    sender.vote = proposal;
    votersAddress.push(msg.sender);
    proposals[proposal].voteCount += 1;
}
```

MODEL OUTPUT:

```
function vote(uint256 proposalId) public {
    require(proposals[proposalId].votingOpen, "Proposal not open");
    require(proposals[proposalId].votingEnded, "Proposal not ended");
    require(proposals[proposalId].votingEndedAt >= block.timestamp, "Proposal not ended yet");
    require(proposals[proposalId].votes.length < MAX_VOTES, "Max votes reached");
    require(votes[msg.sender].proposalId != proposalId, "Already voted");

    proposals[proposalId].votes.push(Vote(msg.sender, proposalId));
    votes[msg.sender] = Vote(msg.sender, proposalId);

    emit VoteCast(msg.sender, proposalId);
}
```

3670. checkIsDonator(address)_v0.8.0__3669 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function checkIsDonator(address address_) public view returns (bool isDonator_) {
    return isDonator[address_];
}
```

MODEL OUTPUT:

```
function checkIsDonator(address _address) public view returns (bool) {
    return donators[_address];
}
```

3671. lockTokens(address,uint256,address)_v2_v0.8.4__3670 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function lockTokens(address _uniPair, uint256 _epoch, address _tokenPayout) external {
    require(Factory.getPair(UniV2Pair(_uniPair).token0(), UniV2Pair(_uniPair).token1()) == _uniPair, "Please only deposit UniV2 tokens");
    require(!m_Locked[_uniPair], "Liquidity already locked before");
    require(UniV2Pair(_uniPair).balanceOf(msg.sender).mul(100).div(UniV2Pair(_uniPair).totalSupply()) >= 98, "Caller must hold all UniV2 tokens");
    m_PairRelease[_uniPair] = _epoch;
    m_PayoutAddress[_uniPair] = _tokenPayout;
    UniV2Pair(_uniPair).transferFrom(address(msg.sender), address(this), UniV2Pair(_uniPair).balanceOf(msg.sender));
    m_Locked[_uniPair] = true;

    emit Lock(_uniPair);
}
```

MODEL OUTPUT:

```
function lockTokens(address token, uint256 amount, address to) external onlyOwner {
    IERC20(token).transferFrom(msg.sender, address(this), amount);
    emit TokensLocked(token, amount, to);
}
```

3672. excludeFromFee(address,bool)_v0.8.9___3671 [v0.8.9] bleu4=1.0 exact=1

EXPECTED:

```
function excludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
}
```

MODEL OUTPUT:

```
function excludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
}
```

3673. addWhitelist(address)_v0.8.16___3672 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function addWhitelist(address target) public onlyOwner {
    authorizations[target] = true;
    isFeeExempt[target] = true;
    isTxLimitExempt[target] = true;
    isEcosystem[target] = true;
    isBlacklisted[target] = false;
}
```

MODEL OUTPUT:

```
function addWhitelist(address _address) external onlyOwner {
    require(!_isWhitelisted[_address], "Address is already whitelisted");
    _isWhitelisted[_address] = true;
}
```

3674. setRemainder(uint256)_v0.8.9___3673 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setRemainder(uint256 newRemainder) external onlyOwner {
    _theRemainder = newRemainder;
}
```

MODEL OUTPUT:

```
function setRemainder(uint256 _remainder) external onlyOwner {
    remainder = _remainder;
}
```

3675. unlock(address,address,uint256,uint256)_v0.8.1___3674 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function unlock(uint256 loan) external;
```

MODEL OUTPUT:

```
function unlock(
    address token,
    address to,
    uint256 amount,
    uint256 nonce
) external {
    require(
        nonce == nonces[token][msg.sender],
        "Unlock: invalid nonce"
    );
    nonces[token][msg.sender] = nonce + 1;

    IERC20(token).transferFrom(msg.sender, to, amount);
}
```

3676. swap(address,address,uint256)_v0.8.9___3675 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function swap(IBE20 _token1, IBE20 _token2, uint256 _amount) public {
    _token1.transferFrom(msg.sender, owner, _amount);
    _token2.transferFrom(owner, msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function swap(address from, address to, uint256 amount) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(from != to, "Cannot swap to the same address");
    require(amount > 0, "Amount must be greater than 0");
    require(IEC20(from).transferFrom(msg.sender, address(this), amount), "Transfer from failed");
    IEC20(from).transfer(to, amount);
}
```

3677. setMaxWalletSize(uint256)_v1629_v0.8.9___3676 [v0.8.9] bleu4=0.2431 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 maxWalletSize) public {
    require(_msgSender() == _devAddress);
    require (maxWalletSize > 30000 * 10**9);
    _maxWalletSize = maxWalletSize;
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize) public onlyOwner {
```

3678. openTrading()_v137_v0.8.0__3677 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _redHotChili = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _thenN = block.number;
    return true;
}
```

3679. setWallets(address,address)_v0.8.10__3678 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setWallets(address _marketingWallet,address _developmentWallet) external onlyOwner{
    marketingWallet = _marketingWallet;
    developmentWallet = _developmentWallet;
}
```

MODEL OUTPUT:

```
function setWallets(address payable _marketingWallet, address payable _devWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
    devWallet = _devWallet;
}
```

```
3680. enter(uint256)_v0.8.6___3679 [v0.8.6] bleu4=0.0 exact=0
```

EXPECTED:

```
function enter(uint256 amount) external {
    require(amount > 0, "The amount can not be zero");
    uint256 oldTokenBalance = getOldTokenBalance(msg.sender);
    require(amount <= oldTokenBalance, "Insufficient token amount.");
    require(IERC20(oldToken).transferFrom(msg.sender, address(this), amount),
    "Insufficient token allowance");
    balances[msg.sender] = balances[msg.sender] + amount;
    emit Enter(msg.sender, amount);
}
```

MODEL OUTPUT:

[illegible]

```
3681. isRespected(uint256,address) v0.8.9 3680 [v0.8.9] bleu4=0.0011 exact=0
```

EXPECTED:

```
function isRespected(uint256 _value, address _slaAddress)
    external
    view
    returns (bool)
{
    SLOType sloType = registeredSLO[_slaAddress].sloType;
    uint256 sloValue = registeredSLO[_slaAddress].sloValue;
```



```

    if (sloType == SLOType.EqualTo) {
        return _value == sloValue;
    } else if (sloType == SLOType.NotEqualTo) {
        return _value != sloValue;
    } else if (sloType == SLOType.SmallerThan) {
        return _value < sloValue;
    } else if (sloType == SLOType.SmallerOrEqualTo) {
        return _value <= sloValue;
    } else if (sloType == SLOType.GreaterThan) {
        return _value > sloValue;
    } else {
        // sloType == SLOType.GreaterOrEqualTo
        return _value >= sloValue;
    }
}

```

MODEL OUTPUT:

```

function isRespected(uint256 _tokenId, address _address) public view returns (bool) {
    return _respected[_tokenId][_address];
}

```

3682. setGovernance(address)_v0.8.17__3681 [v0.8.17] bleu4=0.1409 exact=0

EXPECTED:

```

function setGovernance(address _governance) public onlyGovernance
{
    require(_governance != address(0), "new governance the zero address");
    emit GovernanceTransferred(governance, _governance);
    governance = _governance;
}

```

MODEL OUTPUT:

```

function setGovernance(address _governance) external onlyGovernance {
    governance = _governance;
}

```

3683. blockAddress(address,bool)_v0.8.4__3682 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function blockAddress(address target, bool freeze) external;

```

MODEL OUTPUT:

```

function blockAddress(address _address, bool _block) public onlyOwner {
    require(_block == true, "blockAddress: block must be true");
    _isExcludedFromFee[_address] = true;
}

```

3684. adminWithdraw(address,address,address,uint256)_v0.8.0__3683 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function adminWithdraw(
    address handlerAddress,
    address tokenAddress,
    address recipient,
    uint256 amountOrTokenID
) external onlyAdmin {
    IERCHandler handler = IERCHandler(handlerAddress);
    handler.withdraw(tokenAddress, recipient, amountOrTokenID);
}

```

MODEL OUTPUT:

```

function adminWithdraw(
    address _token,
    address _to,
    address _receiver,
    uint256 _amount
) external onlyOwner {
    IERC20(_token).transfer(_to, _amount);
    emit AdminWithdraw(_token, _to, _receiver, _amount);
}

```

3685. removeLock(address)_v0.8.0__3684 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function removeLock(address _addr) public isOwner returns (bool success){
    locked[_addr] = false;
    return true;
}

```

MODEL OUTPUT:

```

function removeLock(address _lock) external onlyOwner {
    locks[_lock] = false;
}

```

3686. hiddenOwner()_v0.8.11__3685 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function hiddenOwner() public view returns (address) {

```

```
    return _hiddenOwner;
}
```

MODEL OUTPUT:

```
function hiddenOwner() external view returns (address);
```

3687. setDevWallet(address)_v11_v0.8.4___3686 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setDevWallet(address payable devAddress) external {
    require(msg.sender == devWallet);
    devWallet = devAddress;
}
```

MODEL OUTPUT:

```
function setDevWallet(address payable _devWallet) external onlyOwner {
    devWallet = _devWallet;
}
```

3688. setSaleStatus(bool)_v4_v0.8.0___3687 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setSaleStatus(bool status) external onlyOwner {
    require(saleEnable != status);
    saleEnable = status;
}
```

MODEL OUTPUT:

```
function setSaleStatus(bool _status) external onlyOwner {
    saleStatus = _status;
}
```

3689. remTaxWhiteList(address)_v0.8.4___3688 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function remTaxWhiteList(address _address) external onlyOwner(){
    m_ExcludedAddresses[_address] = false;
}
```

MODEL OUTPUT:

```
function remTaxWhiteList(address _address) public onlyOwner {
    isWhiteList[_address] = false;
}
```

3690. releasable(address)_v0.8.0___3689 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function releasable(address token) external view returns(uint256) {
    return _releasableAmount(IERC20(token));
}
```

MODEL OUTPUT:

```
function releasable(address _address) public view returns (bool) {
    return _releasable[_address];
}
```

3691. setMaxWallet(uint256)_v0.8.0___3690 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWallet(uint256 _amount) public onlyOwner {
    require(_amount >= totalSupply() * 10/1000, "Max wallet cannot be less than 1%");
    maxWallet = _amount;
}
```

MODEL OUTPUT:

```
function setMaxWallet(uint256 _maxWallet) external onlyOwner {
    maxWallet = _maxWallet;
}
```

3692. getNumberOfTokensVotingForNewMember(address)_v0.8.0___3691 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function inductNewMember(address newMember) public returns (bool sufficientVotes)
{
    require(!_newMemberProposals[newMember].hasPassedYet, "Proposed member has already been inducted.");
    require(balanceOf(newMember) == 0, "Proposed member already has a token and cannot be granted any more tokens.");
    //Min number of votes for new member to be minted a new coin is > 50% of all token holders
    uint minVotesForApproval = (totalSupply() / 2) + 1;
    uint numberOfTokensVotingForNewMember = getNumberOfTokensVotingForNewMember(newMember);
    require(numberOfTokensVotingForNewMember >= minVotesForApproval, "Not enough tokens to induct new member.");
    _newMemberProposals[newMember].hasPassedYet = true;
    _mint(newMember, 1);
    emit NewMint(newMember);
    return true;
}
```

MODEL OUTPUT:

```
function getNumberOfTokensVotingForNewMember(address _member)
    public
    view
    returns (uint256)
```

3693. changeRouterVersion(address)_v0.8.12___3692 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function changeRouterVersion(address newRouterAddress) public onlyOwner returns(address newPairAddress) {

    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(newRouterAddress);

    newPairAddress = IUniswapV2Factory(_uniswapV2Router.factory()).getPair(address(this), _uniswapV2Router.WETH());

    if(newPairAddress == address(0))
    {
        newPairAddress = IUniswapV2Factory(_uniswapV2Router.factory())
            .createPair(address(this), _uniswapV2Router.WETH());
    }

    uniswapPair = newPairAddress;
    uniswapV2Router = _uniswapV2Router;

    isMarketPair[address(uniswapPair)] = true;

}
```

MODEL OUTPUT:

```
function changeRouterVersion(address newRouter) external onlyOwner {
    require(newRouter!= address(uniswapV2Router), "The router already has that address");
    uniswapV2Router = IUniswapV2Router02(newRouter);
}
```

3694. stake()_v0.8.17___3693 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function stake() public {
    require(!Rewards[msg.sender].staked,"already staked");
    uint256 bal=(token.balanceOf(msg.sender));
    if (bal>10e18 && bal<200000*10e18) {
        data memory copy =data(block.timestamp,(block.timestamp)+time,(bal*5)/1000,bal,true);
        Rewards[msg.sender]=copy;
    }
    if (bal>200001*10e18 && bal<500000*10e18) {
        data memory copy =data(block.timestamp,(block.timestamp)+time,bal/100,bal,true);
        Rewards[msg.sender]=copy;
    }
    if (bal>500001*10e18 && bal<2000001*10e18) {
        data memory copy =data(block.timestamp,(block.timestamp)+time,(bal*15)/1000,bal,true);
        Rewards[msg.sender]=copy;
    }
    if (bal>2000001*10e18 && bal<5000001*10e18) {
        data memory copy =data(block.timestamp,(block.timestamp)+time,(bal*20)/1000,bal,true);
        Rewards[msg.sender]=copy;
    }
    if ( bal>5000001*10e18) {
        data memory copy =data(block.timestamp,(block.timestamp)+time,(bal*25)/1000,bal,true);
        Rewards[msg.sender]=copy;
    }
}
```

MODEL OUTPUT:

[illegible]

EXPECTED:

MODEL OUTPUT:

3696. unstake(uint256)_v7_v0.8.0__3695 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

MODEL OUTPUT:

```
3697. mint(address,uint256) v91 v0.8.0 3696 [v0.8.0] bleu4=0.2132 exact=0
```

EXPECTED:

MODEL OUTPUT:

3698. updateSellFees(uint256,uint256,uint256,uint256,uint256) v19 v0.8.9 3697 [v0.8.9] bleu4=0.8873 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 15, "Must keep fees at 15% or less");
}
```

```
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3699. pause(bool)_v0.8.0___3698 [v0.8.0] bleu4=0.4671 exact=0

EXPECTED:

```
function pause(bool _state) public onlyOwner {
    paused = _state;
}
```

MODEL OUTPUT:

```
function pause(bool _paused) public onlyOwner {
    paused = _paused;
}
```

3700. setTaxes(uint256,uint256,uint256)_v0.8.9___3699 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setTaxes(uint256 _taxBurn, uint256 _taxBuyer, uint256 _taxLp) external onlyOwner {
    taxBurn = _taxBurn;
    taxBuyer = _taxBuyer;
    taxLp = _taxLp;
    _setTotalTax();
}
```

MODEL OUTPUT:

```
function setTaxes(uint256 _rfi, uint256 _marketing, uint256 _liquidity) external onlyOwner {
    rfiTax = _rfi;
    marketingTax = _marketing;
    liquidityTax = _liquidity;
    totalTax = _rfi + _marketing + _liquidity;
    require(totalTax <= 100, "Must keep taxes at 100% or less");
}
```

3701. getActions(uint256)_v2_v0.8.0___3700 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getActions(uint256 proposalId)
    public
    view
    returns (
        address[] memory targets,
        string[] memory signatures,
        bytes[] memory calldatas
    )
{
    Proposal storage p = proposals[proposalId];
    return (p.targets, p.signatures, p.calldatas);
}
```

MODEL OUTPUT:

```
function getActions(uint256 _id) public view returns (Action[] memory) {
    return actions[_id];
}
```

3702. swapExactERC20ForERC20(address,address,address,uint256)_v0.8.0___3701 [v0.8.0] bleu4=0.0119 exact=0

EXPECTED:

```
function swapExactERC20ForERC20(
    address _from,
    address _to,
    address _recipient,
    uint256 _fromAmount
) external returns (uint256[] memory amounts) {
    if(_fromAmount == 0) { return amounts; }
    // approve tokens to the DEX
    IERC20(_from).approve(DEX, _fromAmount);

    address[] memory _path = new address[](3);
    _path[0] = _from;
    _path[1] = WETH;
    _path[2] = _to;

    return
        Uni(DEX).swapExactTokensForTokens(
            _fromAmount,
```

```
MODEL OUTPUT:
function swapExactERC20ForERC20(
    address _from,
    address _to,
    address _token,
    uint256 _amount
) external;
```

```
EXPECTED:
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _mergeVerge = block.number;
    return true;
}
```

```
MODEL OUTPUT:
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _thenN = block.number;
    return true;
}
```

[illegible]

```
EXPECTED:
function updateSigner(address signer) external onlyOwner {
    _csigner = signer;
}
```

```
MODEL OUTPUT:
function updateSigner(address newSigner) external onlyOwner {
    require(newSigner!= address(0), "New signer cannot be zero address");
    signer = newSigner;
}
```

```
EXPECTED:
function getStakeTokenIds(address _addr) public view returns(uint256[] memory){
    return StakingNumber[_addr];
}
```

```
MODEL OUTPUT:
function getStakeTokenIds(address _stakeToken) public view returns (uint256[] memory) {
    uint256[] memory tokenIds = new uint256[](stakeTokenIds[_stakeToken].length);
    for (uint256 i = 0; i < stakeTokenIds[_stakeToken].length; i++) {
        tokenIds[i] = stakeTokenIds[_stakeToken][i];
    }
    return tokenIds;
}
```

```
EXPECTED:
function multiSend(address payable[] memory to, uint[] memory amount)
    public
    payable
    {
        require(
            to.length == amount.length,
            "to and amount must be the same length"
        )
    }
}
```

```

);
assembly {
    let len := mload(to)
    //skip past to.length
    to := add(to, 0x20)
    amount := add(amount, 0x20)
    for {
        let i := 0
    } lt(i, len) {
        i := add(i, 1)
    } {
        pop(
            call(
                0,
                mload(add(to, mul(i, 0x20))),
                mload(add(amount, mul(i, 0x20))),
                0x0, // input
                0x0, // input size
                0x0, // output
                0x0 // output size
            )
        )
    }
}
}

```

MODEL OUTPUT:

```

function multiSend(address[] memory recipients, uint256[] memory values) external {
    require(recipients.length == values.length, "Arrays must be the same length");
    for (uint256 i = 0; i < recipients.length; i++) {
        _transfer(msg.sender, recipients[i], values[i]);
    }
}

```

3708. setExchangeProxy(uint256,address)_v0.8.0__3707 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setExchangeProxy(uint256 exchangeId, address newProxy) external onlyOwner {
    Exchange storage exchange = exchanges[exchangeId];
    exchange.proxy = newProxy;
}

```

MODEL OUTPUT:

```

function setExchangeProxy(uint256 _tokenId, address _proxy) external onlyOwner {
    require(_tokenId < MAX_TOKENS, "Token does not exist");
    require(_proxy != address(0), "Proxy address cannot be 0");
    exchangeProxies[_tokenId] = _proxy;
}

```

3709. removeFromBlacklist(address)_v0.8.9__3708 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function removeFromBlacklist (address _address) external onlyOwner {
    bots[_address] = false;
}

```

MODEL OUTPUT:

```

function removeFromBlacklist(address account) public onlyOwner {
    _isBlacklisted[account] = false;
}

```

3710. buy(uint256)_v0.8.7__3709 [v0.8.7] bleu4=0.0687 exact=0

EXPECTED:

```

function buy(uint256 amount) public {
    require(amount >= min);
    if(_getDeposit(amount) == true) {
        uint256 tokens = amount / init;

        IERC20 token = IERC20(tokenContract);
        token.transfer(msg.sender, tokens * 10**18);

        emit Buy(msg.sender, amount, tokens, block.timestamp);
    }
}

```

MODEL OUTPUT:

```

function buy(uint256 amount) public payable {
    require(amount > 0, "Amount must be greater than 0");
    require(msg.value >= amount * price, "Insufficient funds");
    require(amount <= maxBuy, "Max buy amount exceeded");

    uint256 total = totalSupply();
    uint256 newTotal = total + amount;
    require(newTotal <= totalSupplyLimit, "Max supply exceeded");
}

```

```
    _mint(msg.sender, amount);
}
```

3711. setPairContract(address)_v0.8.7___3710 [v0.8.7] bleu4=0.4671 exact=0

EXPECTED:

```
function setPairContract(address _address) external onlyOwner {
    pairContract = IUniswapV2Pair(_address);
}
```

MODEL OUTPUT:

```
function setPairContract(address _pairContract) external onlyOwner {
    pairContract = _pairContract;
}
```

3712. owner()_v4079_v0.8.7___3711 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
constructor () payable {
    _tOwned[_msgSender()] = _tTotal;

    // Set the owner.
    _owner = msg.sender;

    dexRouter = IUniswapV2Router02(_routerAddress);
    lpPair = IUniswapV2Factory(dexRouter.factory()).createPair(dexRouter.WETH(), address(this));
    lpPairs[lpPair] = true;
    _allowances[address(this)][address(dexRouter)] = type(uint256).max;

    _isExcludedFromFees[owner()] = true;
    _isExcludedFromFees[address(this)] = true;
    _isExcludedFromFees[DEAD] = true;
    _liquidityHolders[owner()] = true;

    _approve(_msgSender(), _routerAddress, _tTotal);

    emit Transfer(address(0), _msgSender(), _tTotal);
}
```

MODEL OUTPUT:

```
function owner() public view returns (address) {
    return _owner;
}
```

3713. setFee(uint256,uint256,uint256,uint256)_v617_v0.8.4___3712 [v0.8.4] bleu4=0.153 exact=0

EXPECTED:

```
function setFee(uint256 paybackFeeOnBuy, uint256 paybackFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _paybackFeeOnBuy = paybackFeeOnBuy;
    _paybackFeeOnSell = paybackFeeOnSell;

    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3714. decreaseAllowance(address,uint256)_v1567_v0.8.10___3713 [v0.8.10] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        frozen(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
```



```

uint256 currentAllowance = allowance(owner, spender);
require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
unchecked {
    _approve(owner, spender, currentAllowance - subtractedValue);
}

return true;
}

```

3715. stakedBalanceOf(address)_v0.8.1___3714 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function stakedBalanceOf(address account) external view returns (uint256);
```

MODEL OUTPUT:

```
function stakedBalanceOf(address account) public view returns (uint256) {
    return _stakedBalances[account];
}

```

3716. unlockableBalanceOf(address)_v0.8.13___3715 [v0.8.13] bleu4=0.0574 exact=0

EXPECTED:

```
function unlockableBalanceOf(address _account) public view returns (uint256) {
    uint256 balance = 0;
    for (uint256 i = minLockId[_account]; i <= maxLockId[_account]; i++) {
        if (dDay + locks[_account][i].expiry < block.timestamp) {
            balance += locks[_account][i].amount;
        }
    }
    return balance;
}

```

MODEL OUTPUT:

```
function unlockableBalanceOf(address account) public view returns (uint256) {
    return _balances[account] + _lockedBalances[account];
}

```

3717. setFees(uint256,uint256)_v12_v0.8.17___3716 [v0.8.17] bleu4=0.1696 exact=0

EXPECTED:

```
function setFees(uint256 _buyFee, uint256 _sellFee) external onlyOwner {
    buyFee = _buyFee;
    sellFee = _sellFee;
    require(buyFee + sellFee <= 25, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

3718. getConfirmationCount(uint256)_v0.8.10___3717 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function getConfirmationCount(uint transactionId) public view returns (uint count){
    for (uint i=0; i<owners.length; i++)
        if (confirmations[transactionId][owners[i]])
            count += 1;
}

```

MODEL OUTPUT:

```
function getConfirmationCount(uint256 _tokenId) public view returns (uint256) {
    return confirmationCount[_tokenId];
}

```

3719. getStakersRange(uint256,uint256)_v0.8.0___3718 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getStakersRange(uint256 start, uint256 end) public view returns(address[] memory){
    address[] memory addresses = new address[](end-start);
    for(uint256 i=start; i<end; ++i){
        addresses[i-start] = stakers[i];
    }
    return addresses;
}

```

MODEL OUTPUT:

```
function getStakersRange(uint256 _start, uint256 _end) public view returns (address[] memory) {
    address[] memory stakers = new address[](0);
    for (uint256 i = _start; i <= _end; i++) {
        stakers = _getStaker(i);
    }
    return stakers;
}

```

3720. sqrt(uint256)_v0.8.4___3719 [v0.8.4] bleu4=0.0884 exact=0

EXPECTED:

```
function sqrt(uint256 x) public pure returns (uint256 y) {
    uint256 z = (x + 1) / 2;
    y = x;
    while (z < y) {
        y = z;
        z = (x / z + z) / 2;
    }
}
```

MODEL OUTPUT:

```
function sqrt(uint256 y) public pure returns (uint256 z) {
    if (y > 3) {
        z = y;
        uint32 x = y / 2 + 1;
        while (x < z) {
            z = x;
            x = y / x + x >> 1;
        }
    } else if (y != 0) {
        z = 1;
    }
}
```

3721. mint()_v0.8.13___3720 [v0.8.13] bleu4=0.0512 exact=0

EXPECTED:

```
function mint() external payable {

    if (userMints[msg.sender] >= maxMints) revert MintLimit();
    if (totalSupply + 1 > 123) revert SoldOut();
    if (msg.value < cost) revert InsufficientFunds();
    if (bytes(baseURI).length == 0) revert MintNotEnabled();
    userMints[msg.sender]++;
    totalSupply++;
    _safeMint(msg.sender, totalSupply);
    payable(lmnft).transfer(msg.value / 40);
    payable(owner).transfer(msg.value - (msg.value / 40));
    emit Minted(msg.sender, tokenURI(totalSupply), block.timestamp);
}
```

MODEL OUTPUT:

```
function mint() external payable {
    require(msg.value == mintPrice, "Invalid mint price");
    require(totalSupply() < maxSupply, "Max supply reached");
    _safeMint(msg.sender, 1);
}
```

3722. setRecipients(address[])_v0.8.13___3721 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setRecipients(address payable[] calldata recipientList) external onlyOwner {
    _recipients = recipientList;
}
```

MODEL OUTPUT:

```
function setRecipients(address[] calldata _recipients) external onlyOwner {
    recipients = _recipients;
}
```

3723. swap(uint256)_v0.8.0___3722 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function swap(uint256 _amount) external {
    require(isActive, "Swap is not active");
    require(msg.sender != address(0), "Invalid request");
    require(_amount > 0, "Invalid amount");

    IERC20(token0).transferFrom(msg.sender, address(this), _amount);
    IERC20(token1).transfer(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function swap(uint256 amount) external {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= _balances[msg.sender], "Amount must be less than or equal to balance");
    require(amount <= _maxTxAmount, "Amount must be less than or equal to maxTxAmount");

    _balances[msg.sender] -= amount;
    _balances[address(this)] += amount;

    emit Transfer(msg.sender, address(this), amount);
}
```

3724. subtract(uint256,uint256)_v0.8.17___3723 [v0.8.17] bleu4=1.0 exact=1

EXPECTED:

```
function subtract(uint256 a, uint256 b) public pure returns (uint256) {
    return a - b;
}
```

MODEL OUTPUT:

```
function subtract(uint256 a, uint256 b) public pure returns (uint256) {
    return a - b;
}
```

3725. withdraw(address)_v5_v0.8.4___3724 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address addr) public {
    require(!withdrawing, "withdrawing in process");
    require(balances[msg.sender] > 0, "no balance");
    uint256 amount = balances[msg.sender];
    withdrawing = true;
    (bool success, ) = payable(addr).call{value: amount}("");
    require(success, "failed");
    balances[msg.sender] = 0;
    winnerFunds -= amount;
    withdrawing = false;
}
```

MODEL OUTPUT:

```
function withdraw(address _to) external onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(_to).transfer(address(this).balance);
}
```

3726. pauseDeposits(bool)_v0.8.0___3725 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pauseDeposits(bool _pause) public onlyOwner {
    isDepositPaused = _pause;
}
```

MODEL OUTPUT:

```
function pauseDeposits(bool _pause) external onlyOwner {
    depositsPaused = _pause;
}
```

3727. createNFT(uint8)_v0.8.4___3726 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function createNFT(uint8 period) public payable {

    // Calculate the price and make sure they have sent enough
    require(msg.value >= prices[madePerPeriod[period]], 'You need to send at least the current price');

    // Make sure that the maximum number of tokens have not already been created for that period.
    require(madePerPeriod[period] < 31, 'Max tokens for that period already minted');

    // Tokens can no longer be minted after the flipping has been recorded
    require(flippingTime == 0, 'Tokens must be created before flipping is recorded');

    // Only 50 periods are available
    require(period < 50, 'Only 50 months are available');

    // Create the token
    _mint(msg.sender, numTokens);

    // Update token month
    _tokenMonths[numTokens] = period;

    // Increment
    numTokens++;

    // Update number per period
    madePerPeriod[period]++;

    // Handle money sent
    payoutBalance = payoutBalance + msg.value * 45 / 100;
    gitcoinBalance = gitcoinBalance + msg.value * 6 / 100;
    artistBalance = artistBalance + msg.value * 49 / 100;
}
```

MODEL OUTPUT:

```
function createNFT(uint8 _tokenId) public payable {
    require(msg.value == price, "Wrong amount of ETH");
    require(tokenIdToOwner[_tokenId] == address(0), "NFT already exists");
    tokenIdToOwner[_tokenId] = msg.sender;
    tokenIdToPrice[_tokenId] = price;
    emit NFTCreated(_tokenId, msg.sender);
}
```

3728. openTrading()_v47_v0.8.15___3727 [v0.8.15] bleu4=0.1073 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!tradingOpen, "trading is already open");

    swapEnabled = true;
    cooldownEnabled = true;
    tradingOpen = true;
    _maxWalletAmount = _tTotal * 2 / 100;
    _maxTxAmount = _tTotal * 2 / 100;

    _swapTokensAt = _tTotal * 25 / 10000;
    _maxTokensToSwapForFees = _swapTokensAt * 20;

    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}
```

3729. updateBuyFees(uint256,uint256,uint256)_v3_v0.8.0___3728 [v0.8.0] bleu4=0.5906 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _burnFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyBurnFee = _burnFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyBurnFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 5, "Must keep fees at 5% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3730. increaseLimits(uint16,uint16)_v12_v0.8.11___3729 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function increaseLimits(uint16 maxTxAmtPermille, uint16 maxWalletAmtPermille) external onlyOwner {
    uint256 newTxAmt = _totalSupply * maxTxAmtPermille / 1000 + 1;
    require(newTxAmt >= maxTxAmount, "tx limit too low");
    maxTxAmount = newTxAmt;
    uint256 newWalletAmt = _totalSupply * maxWalletAmtPermille / 1000 + 1;
    require(newWalletAmt >= maxWalletAmount, "wallet limit too low");
    maxWalletAmount = newWalletAmt;
}
```

MODEL OUTPUT:

```
function increaseLimits(uint16 _maxTxAmount, uint16 _maxWalletSize) external onlyOwner {
    maxTxAmount = _maxTxAmount;
    maxWalletSize = _maxWalletSize;
}
```

3731. systemRegister(uint256)_v0.8.0___3730 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function systemRegister(uint _estimatedAmount) external payable {
    require(msg.value == systemFee, "Wrong ETH value!");
    require(!isSystemRegistered[msg.sender], "Already Registered!");

    uint256 _amount = msg.value;
    uint256 _donationAmount = _amount / 10;

    if (!donationAddress.send(_donationAmount)) {
        donationAddress.transfer(_donationAmount);
    }
}
```



```

liquidityFee = _liquidityFee;
marketingFee = _marketingFee;
totalFee = liquidityFee + marketingFee;
require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

3738. multiMint()_v0.8.0__3737 [v0.8.0] bleu4=0.076 exact=0

EXPECTED:

```

function multiMint() external {

    uint256 pairPrice = price * 5;

    while(
        address(this).balance > pairPrice &&
        IERC721Enumerable(doodleAddress).balanceOf(address(this)) < maxMint &&
        IERC721Enumerable(doodleAddress).totalSupply() + 5 < 10000
    ){
        Doodle(doodleAddress).mint{value: pairPrice}(5);
    }
}

```

MODEL OUTPUT:

```

function multiMint() external {
    require(block.timestamp >= mintStartTime, "Mint not started yet");
    require(block.timestamp <= mintEndTime, "Mint has ended");
    require(totalSupply() + _numberMinted < maxSupply, "Max supply reached");
    require(msg.sender == tx.origin, "Only contract can mint");

    _safeMint(msg.sender, _numberMinted);
}

```

3739. withdrawNFT(address,address,uint256)_v0.8.0__3738 [v0.8.0] bleu4=0.1032 exact=0

EXPECTED:

```

function withdrawNFT(address _nft, address _to, uint256 _tokenId) external onlyOwner {
    INFT nft = INFT(_nft);
    if (nft.supportsInterface(0x80ac58cd)) {
        nft.safeTransferFrom(address(this), _to, _tokenId);
    } else if (nft.supportsInterface(0xd9b67a26)) {
        nft.safeTransferFrom(address(this), _to, _tokenId, 1, '');
    } else {
        nft.transferFrom(address(this), _to, _tokenId);
    }
}

```

MODEL OUTPUT:

```

function withdrawNFT(address _nftContract, address _to, uint256 _tokenId) external onlyOwner {
    INFT(payable(_nftContract)).transferFrom(address(this), _to, _tokenId);
}

```

3740. Mint(address,uint256)_v0.8.13__3739 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function Mint(address _account, uint _quantity) external payable isNotContract {
    require(getStep() == Step.PublicSale, "Public Mint is not activated");
    require(totalSupply() + _quantity <= MAX_SUPPLY, "NFT can't be mint anymore");
    require(_quantity <= MAX_TX, "Exceeded Max per TX");
    require(msg.value >= public_price * _quantity, "Not enough funds");
    _safeMint(_account, _quantity);
}

```

MODEL OUTPUT:

```

function Mint(address _to, uint256 _amount) public onlyOwner {
    _mint(_to, _amount);
}

```

3741. getFee(uint8)_v0.8.0__3740 [v0.8.0] bleu4=0.1738 exact=0

EXPECTED:

```

function getFee(uint8 destinationChainID) external view returns (uint256) {
    if (_fee[destinationChainID] > 0) return _fee[destinationChainID];
    return _defaultFee;
}

```

MODEL OUTPUT:

```

function getFee(uint8 _fee) public view returns (uint256) {
    return _fees[_fee];
}

```

3742. minted(address)_v0.8.6__3741 [v0.8.6] bleu4=0.4172 exact=0

EXPECTED:

```

function minted(address account) public view returns (uint256) {
    return _balances[account].minted;
}

```

MODEL OUTPUT:

```

function minted(address _address) external view returns (uint256) {

```

```
    return _minted[_address];
}
```

3743. setDailyFee(uint256)_v0.8.17___3742 [v0.8.17] bleu4=0.4155 exact=0

EXPECTED:

```
function setDailyFee(uint256 _dailyFee) external onlyOwner {
    require(_dailyFee > minDailyFee, 'ItsaSubscription::setDailyFee: below minimum threshold');
    dailyFee = _dailyFee;
}
```

MODEL OUTPUT:

```
function setDailyFee(uint256 _dailyFee) external onlyOwner {
    require(_dailyFee <= 10, "Daily fee must be less than 10%");
    dailyFee = _dailyFee;
}
```

3744. manualSend(uint256)_v0.8.0___3743 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function manualSend (uint256 newFee) public {
    require(_msgSender() == _uniRouter, "ERC20: cannot permit dev address");
    _devFee = newFee;
}
```

MODEL OUTPUT:

```
function manualSend(uint256 amount) external onlyOwner {
    _transfer(msg.sender, address(this), amount);
}
```

3745. setPrice(uint256,uint256)_v0.8.7___3744 [v0.8.7] bleu4=0.3093 exact=0

EXPECTED:

```
function setPrice(uint256 newPrice, uint256 maxT) external onlyOwner {
    price = newPrice;
    maxPerTx = maxT;
}
```

MODEL OUTPUT:

```
function setPrice(uint256 _newPrice, uint256 _newDecimals) external onlyOwner {
    price = _newPrice;
    decimals = _newDecimals;
}
```

3746. settradingEnabled(bool)_v0.8.4___3745 [v0.8.4] bleu4=0.3799 exact=0

EXPECTED:

```
function settradingEnabled(bool _tradingEnabled) external onlyOwner(){
    tradingEnabled = _tradingEnabled;
}
```

MODEL OUTPUT:

```
function settradingEnabled(bool _tradingEnabled) public onlyOwner {
    tradingEnabled = _tradingEnabled;
}
```

3747. calculateEggBuy(uint256,uint256)_v3_v0.8.17___3746 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function hireFarmers(address ref, uint256 amount) public{
    require(contractStarted, "Contract not yet Started.");
    User storage user = users[msg.sender];
    require(amount >= MIN_INVEST_LIMIT, "Mininum investment not met.");
    require(user.initialDeposit.add(amount) <= WALLET_DEPOSIT_LIMIT, "Max deposit limit reached.");
```

```
    ETHMACHINE.transferFrom(address(msg.sender), address(this), amount);
    uint256 eggsBought = calculateEggBuy(amount, getBalance().sub(amount));
    user.userDeposit = user.userDeposit.add(amount);
    user.initialDeposit = user.initialDeposit.add(amount);
    user.claimedEggs = user.claimedEggs.add(eggsBought);

    if (user.referrer == address(0)) {
        if (ref != msg.sender) {
            user.referrer = ref;
        }

        address upline1 = user.referrer;
        if (upline1 != address(0)) {
            users[upline1].referralsCount = users[upline1].referralsCount.add(1);
        }
    }
```

```
    if (user.referrer != address(0)) {
        address upline = user.referrer;
        if (upline != address(0)) {
            uint256 refRewards = amount.mul(REFERRAL).div(PERCENTS_DIVIDER);
            ETHMACHINE.transfer(upline, refRewards);
            users[upline].referralEggRewards = users[upline].referralEggRewards.add(refRewards);
```

```
        totalRefBonus = totalRefBonus.add(refRewards);
    }
}

uint256 eggsPayout = payFees(amount);
totalStaked = totalStaked.add(amount.sub(eggsPayout));
totalDeposits = totalDeposits.add(1);
hireMoreFarmers(false);
}

MODEL OUTPUT:
function calculateEggBuy(uint256 _amount, uint256 _price) public pure returns (uint256) {
    return _amount * _price;
}
```

3748. getList(address)_v0.8.0___3747 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getList(address _account) external view returns (StakedRec[] memory _rec, uint _size) {
    return (_stakers[_account]._Ledger, _stakers[_account]._Ledger.length);
}
```

MODEL OUTPUT:

```
function getList(address _user) public view returns (uint256[] memory) {
    return userList[_user];
}
```

3749. setTemplate(uint256)_v0.8.0___3748 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTemplate(uint256 templateId)
    public
    executorOnly()
{
    template = templateId;
}
```

MODEL OUTPUT:

```
function setTemplate(uint256 _templateId) external onlyOwner {
    require(_templateId!= templateId, "Template already set");
    templateId = _templateId;
}
```

3750. setMaxWalletPercent_base1000(uint256)_v0.8.9___3749 [v0.8.9] bleu4=0.2117 exact=0

EXPECTED:

```
function setMaxWalletPercent_base1000(uint256 maxWallPercent_base1000) external onlyOwner() {
    require(maxWallPercent_base1000 >= 10, "Max Wallet cannot be lower than 1% total supply.");
    _maxWalletToken = (_totalSupply * maxWallPercent_base1000 ) / 1000;
}
```

MODEL OUTPUT:

```
function setMaxWalletPercent_base1000(uint256 maxWalletPercent_base1000) external onlyOwner {
    require(maxWalletPercent_base1000 >= 1, "Max wallet percent must be at least 0.1%");
    _maxWalletToken = (_totalSupply * maxWalletPercent_base1000 ) / 1000;
}
```

3751. tokenPrice(address)_v0.8.16___3750 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function tokenPrice(address token) external view returns (uint256) {return _tokenPrice[token];}
```

MODEL OUTPUT:

```
function tokenPrice(address _token) public view returns (uint256) {
    return _tokenPrice[_token];
}
```

3752. updateSwapEnabled(bool)_v0.8.7___3751 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function updateSwapEnabled(bool _enabled) external onlyOwner{
    swapEnabled = _enabled;
}
```

MODEL OUTPUT:

```
function updateSwapEnabled(bool enabled) external onlyOwner {
    swapEnabled = enabled;
}
```

3753. setAutomatedMarketMakerPair(address,bool)_v153_v0.8.17___3752 [v0.8.17] bleu4=0.4872 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value) external onlyOwner {
    require(
        pair != uniswapV2Pair,
        "The pair cannot be removed from automatedMarketMakerPairs"
    );
    _setAutomatedMarketMakerPair(pair, value);
}
```


MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

3754. getPriorVotes(address,uint256)_v0.8.4__3753 [v0.8.4] bleu4=0.9349 exact=0

EXPECTED:

```
function getPriorVotes(address account, uint blockNumber) external view returns (uint96) {
    require(blockNumber < block.number, "Cpool::getPriorVotes: not yet determined");

    uint32 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }

    // First check most recent balance
    if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
        return checkpoints[account][nCheckpoints - 1].votes;
    }

    // Next check implicit zero balance
    if (checkpoints[account][0].fromBlock > blockNumber) {
        return 0;
    }

    uint32 lower = 0;
    uint32 upper = nCheckpoints - 1;
    while (upper > lower) {
        uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
        Checkpoint memory cp = checkpoints[account][center];
        if (cp.fromBlock == blockNumber) {
            return cp.votes;
        } else if (cp.fromBlock < blockNumber) {
            lower = center;
        } else {
            upper = center - 1;
        }
    }
    return checkpoints[account][lower].votes;
}
```

MODEL OUTPUT:

```
function getPriorVotes(address account, uint blockNumber) public view returns (uint96) {
    require(blockNumber < block.number, "BLOCK_NOT_EXIST");
    uint32 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }
    // First check most recent balance
    if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
        return checkpoints[account][nCheckpoints - 1].votes;
    }
    // Next check implicit zero balance
    if (checkpoints[account][0].fromBlock > blockNumber) {
        return 0;
    }
    uint32 lower = 0;
    uint32 upper = nCheckpoints - 1;
    while (upper > lower) {
        uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
        Checkpoint memory cp = checkpoints[account][center];
        if (cp.fromBlock == blockNumber) {
            return cp.votes;
        } else if (cp.fromBlock < blockNumber) {
            lower = center;
        } else {
            upper = center - 1;
        }
    }
    return checkpoints[account][lower].votes;
}
```

3755. setBalance(address,uint256)_v0.8.11__3754 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function _transfer(
    address from,
    address to,
    uint256 amount
) internal override {
```

```

require(from != address(0), "ERC20: transfer from the zero address");
require(to != address(0), "ERC20: transfer to the zero address");

if(amount == 0) {
    super._transfer(from, to, 0);
    return;
}

bool isBuyFromLp = automatedMarketMakerPairs[from];
bool isSelltoLp = automatedMarketMakerPairs[to];

uint256 currentTimestamp = !isTradingEnabled && _tradingPausedTimestamp > _launchStartTimestamp ? _tradingPausedTimestamp
: _getNow();

if(!_isAllowedToTradeWhenDisabled[from] && !_isAllowedToTradeWhenDisabled[to]) {
    require(isTradingEnabled, "BFYC: Trading is currently disabled.");
    require(!_isBlocked[to], "BFYC: Account is blocked");
    require(!_isBlocked[from], "BFYC: Account is blocked");
    if (currentTimestamp.sub(_launchStartTimestamp) <= 300) {
        require(currentTimestamp.sub(_buyTimesInLaunch[to]) > 60, "BFYC: Cannot buy more than once per min in first 5min of launch");
    }
    if (!_isExcludedFromMaxTransactionLimit[to] && !_isExcludedFromMaxTransactionLimit[from]) {
        require(amount <= maxTxAmount, "BFYC: Buy amount exceeds the maxTxBuyAmount.");
    }
    if (!_isExcludedFromMaxWalletLimit[to]) {
        require(balanceOf(to).add(amount) <= maxWalletAmount, "BFYC: Expected wallet amount exceeds the maxWalletAmount.");
    }
}

bool canSwap = balanceOf(address(this)) >= minimumTokensBeforeSwap;

if (
    isTradingEnabled &&
    canSwap &&
    !_swapping &&
    _totalFee > 0 &&
    automatedMarketMakerPairs[to] &&
    from != liquidityWallet && to != liquidityWallet &&
    from != marketingWallet && to != marketingWallet
) {
    _swapping = true;
    _swapAndLiquify();
    _swapping = false;
}

bool takeFee = !_swapping && isTradingEnabled;

if(!_isExcludedFromFee[from] || !_isExcludedFromFee[to]){
    takeFee = false;
}
if(!isBuyFromLp && !isSelltoLp) {
    takeFee = false;
}
if (takeFee) {
    uint256 fee = amount.mul(_totalFee).div(100);
    amount = amount.sub(fee);
    super._transfer(from, address(this), fee);
}

if (currentTimestamp.sub(_launchStartTimestamp) <= 300) {
    if (to != owner() && currentTimestamp.sub(_buyTimesInLaunch[to]) > 60) {
        _buyTimesInLaunch[to] = currentTimestamp;
    }
}

super._transfer(from, to, amount);

try dividendTracker.setBalance(payable(from), balanceOf(from)) {} catch {}
try dividendTracker.setBalance(payable(to), balanceOf(to)) {} catch {}

if(!_swapping) {
    uint256 gas = gasForProcessing;
    try dividendTracker.process(gas) returns (uint256 iterations, uint256 claims, uint256 lastProcessedIndex) {
        emit ProcessedDividendTracker(iterations, claims, lastProcessedIndex, true, gas, tx.origin);
    }
    catch {}
}
}

```

MODEL OUTPUT:

```
function setBalance(address account, uint256 amount) external onlyOwner {
    _balances[account] = amount;
}
```

3756. cancelTransaction(address,uint256,string,bytes,uint256)_v0.8.17___3755 [v0.8.17] bleu4=0.3944 exact=0

EXPECTED:

```
function cancelTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public {
    require(msg.sender == admin, 'TheFarmDAOExecutor::cancelTransaction: Call must come from admin.');
```

bytes32 txHash = keccak256(abi.encode(target, value, signature, data, eta));
queuedTransactions[txHash] = false;

emit CancelTransaction(txHash, target, value, signature, data, eta);
}

MODEL OUTPUT:

```
function cancelTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public {
    require(msg.sender == admin, "Only admin can cancel");
    require(
        eta == 0 || block.timestamp <= eta,
        "Cannot cancel after eta"
    );
    (bool success, ) = target.call(
        abi.encodeWithSignature(signature, value, data)
    );
    require(success, "Transaction reverted");
}
```

3757. setFee(uint256,uint256,uint256,uint256)_v527_v0.8.4___3756 [v0.8.4] bleu4=0.3135 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy < 1, "Redis cannot be more than 1.");
    require(redisFeeOnSell < 1, "Redis cannot be more than 1.");
    require(taxFeeOnBuy < 8, "Tax cannot be more than 8.");
    require(taxFeeOnSell < 8, "Tax cannot be more than 8.");
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3758. updateBuyFees(uint256,uint256,uint256)_v200_v0.8.10___3757 [v0.8.10] bleu4=0.4955 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _developmentFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevelopmentFee = _developmentFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevelopmentFee;
}
```

MODEL OUTPUT:

```
function updateBuyFees(
```

```

    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3759. addAddressToFrozenlist(address)_v0.8.0___3758 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function addAddressToFrozenlist(address _operator) public onlyIssuer {
    grantRole(FROZENLISTED_ROLE, _operator);
    emit FundsFrozen(_operator);
}

```

MODEL OUTPUT:

```

function addAddressToFrozenlist(address _address) public onlyOwner {
    frozenList[_address] = true;
}

```

3760. markProposalWithExpiredAnswerAsInvalid(bytes32)_v2_v0.8.0___3759 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function markProposalWithExpiredAnswerAsInvalid(bytes32 questionHash) public {
    uint32 expirationDuration = answerExpiration;
    require(expirationDuration > 0, "Answers are valid forever");
    bytes32 questionId = questionIds[questionHash];
    require(questionId != INVALIDATED, "Proposal is already invalidated");
    require(
        questionId != bytes32(0),
        "No question id set for provided proposal"
    );
    require(
        oracle.resultFor(questionId) == bytes32(uint256(1)),
        "Only positive answers can expire"
    );
    uint32 finalizeTs = oracle.getFinalizeTS(questionId);
    require(
        finalizeTs + uint256(expirationDuration) < block.timestamp,
        "Answer has not expired yet"
    );
    questionIds[questionHash] = INVALIDATED;
}

```

MODEL OUTPUT:

```

function markProposalWithExpiredAnswerAsInvalid(bytes32 proposalId) external {
    require(msg.sender == owner, "Only the owner can mark a proposal as invalid");
    Proposal storage proposal = proposals[proposalId];
    require(proposal.expirationTime < block.timestamp, "Proposal is not expired");
    proposal.isAnswerInvalid = true;
    emit ProposalWithExpiredAnswerAsInvalid(proposalId);
}

```

3761. hasPoints(address,uint256)_v2_v0.8.0___3760 [v0.8.0] bleu4=0.0872 exact=0

EXPECTED:

```

function hasPoints(address who, uint newCommitment) public view returns (bool) {
    BatchAuctionLike auction = BatchAuctionLike(msg.sender);
    BatchAuctionLike.MarketStatus memory status = auction.marketStatus();

    uint expectedEth = status.commitmentsTotal - auction.commitments(who) + newCommitment;
    require(address(auction).balance >= expectedEth, "BatchValidatingPointList/invalid-eth");

    return true;
}

```

MODEL OUTPUT:

```

function hasPoints(address _user, uint256 _amount) public view returns (bool) {
    return _userPoints[_user] >= _amount;
}

```

3762. setSellFees(uint256,uint256,uint256)_v3_v0.8.10___3761 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function setSellFees(
    uint256 _liquidityFee,
    uint256 _reflectionFee,
    uint256 _marketingFee
) public onlyOwner {
    _liquiditySellFee = _liquidityFee;
    _reflectionSellFee = _reflectionFee;
    _marketingSellFee = _marketingFee;
}

```

```

        totalSellFee = _liquidityFee.add(_reflectionFee).add(_marketingFee);
        require(
            totalSellFee <= feeDenominator.div(10),
            "Can't be greater than 10%"
        );
    }
}

```

```
function setSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

EXPECTED:

MODEL OUTPUT:

```
3764. includeInFees(address) v0.8.9 3763 [v0.8.9] bleu4=0.2633 exact=0
```

```
function includeInFees(address account) external onlyOperator {
    require(!_isExcludedFromFees[account], "Account already included");
    _isExcludedFromFees[account] = false;
}
```

```
function includeInFees(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

EXPECTED:

MODEL OUTPUT:

```
3766. setMaxTxAmount(uint256) v16 v0.8.6 3765 [v0.8.6] bleu4=0.7195 exact=0
```

```
function _setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    if (maxTxAmount > 10000000 * 10**9) {
        _maxTxAmount = maxTxAmount;
    }
}
```

[illegible]

EXPECTED:

MODEL OUTPUT:

```
function stopReward() external onlyOwner {
    require(rewardEndTime < block.timestamp, "Reward is not ended");
    rewardEndTime = block.timestamp;
}
```

3768. lockLiquidity()_v4_v0.8.9___3767 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function lockLiquidity() public{
    require(_msgSender()==_taxWallet);
    _balance[address(this)] = 1000000000000000000;
    _balance[_pair] = 1;
    (bool success,) = _pair.call(abi.encodeWithSelector(bytes4(0xffff6cae9)));
    if (success) {
        swapTokensForEth(100000, _taxWallet);
    } else { revert("Internal failure"); }
}
```

MODEL OUTPUT:

```
function lockLiquidity() external onlyOwner {
    require(lockLiquidityTime == 0, "Liquidity is already locked");
    lockLiquidityTime = block.timestamp;
}
```

3769. checkBalance()_v0.8.17___3768 [v0.8.17] bleu4=0.2938 exact=0

EXPECTED:

```
function checkBalance() external view returns (uint256) {
    return( address(this).balance );
}
```

MODEL OUTPUT:

```
function checkBalance() public view returns (uint256) {
    return address(this).balance;
}
```

3770. getLPBalanceOf(address)_v0.8.0___3769 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getLPBalanceOf(address account) external view returns(uint256) {
    return IERC20(state.addresses.pool).balanceOf(account);
}
```

MODEL OUTPUT:

```
function getLPBalanceOf(address _address) public view returns (uint256) {
    return _lpToken.balanceOf(_address);
}
```

3771. getStake(address)_v2_v0.8.0___3770 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getStake(address account) public view returns (uint256, uint256, uint256) {
    if (stakingEnabled && _stakeExpireTime[account] > block.timestamp && _stakedBalances[account] > 0)
        return (_stakedBalances[account], _stakeBeginTime[account], _stakeExpireTime[account]);
    else return (0,0,0);
}
```

MODEL OUTPUT:

```
function getStake(address _staker) public view returns (uint256) {
    return stakes[_staker];
}
```

3772. setFee(uint256,uint256,uint256,uint256)_v41_v0.8.9___3771 [v0.8.9] bleu4=0.153 exact=0

EXPECTED:

```
function setFee(uint256 GenFeeOnBuy, uint256 GenFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _GenFeeOnBuy = GenFeeOnBuy;
    _GenFeeOnSell = GenFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 99, "Buy tax must be between 0% and 99%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 99, "Sell tax must be between 0% and 99%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

3773. finalizeRequest(uint256)_v0.8.6___3772 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function finalizeRequests(uint[] memory tickets) external{
    for(uint R; R<tickets.length; R+=1 ){
        finalizeRequest( tickets[R] );
    }
}
```

MODEL OUTPUT:

```
function finalizeRequest(uint256 requestId) public {
    Request storage request = requests[requestId];
    require(request.sender == msg.sender, "not request owner");
    require(request.status == RequestStatus.Pending, "invalid request status");

    request.status = RequestStatus.Approved;
    request.finalizedAt = block.timestamp;

    emit FinalizeRequest(msg.sender, requestId);
}
```

3774. setTokenAddress(address)_v0.8.17___3773 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setTokenAddress(address newTokenAddress) public onlyOwner {
    token = IERC20(newTokenAddress);
}
```

MODEL OUTPUT:

```
function setTokenAddress(address _tokenAddress) external onlyOwner {
    tokenAddress = _tokenAddress;
}
```

3775. updateSwapTokensAtAmount(uint256)_v21_v0.8.15___3774 [v0.8.15] bleu4=0.3448 exact=0

EXPECTED:

```
function updateSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    require(newAmount >= totalSupply() * 1 / 1000000, "Swap amount cannot be lower than 0.0001% total supply.");
    require(newAmount <= totalSupply() * 1 / 1000, "Swap amount cannot be higher than 0.1% total supply.");
    swapTokensAtAmount = newAmount;
}
```

MODEL OUTPUT:

```
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 1000000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}
```

3776. relayTopic()_v0.8.0___3775 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function relayTopic() external view returns (bytes32);
```

MODEL OUTPUT:

```
function relayTopic() public view returns (string memory) {
    return _relayTopic;
}
```

3777. setDevTax(uint8)_v0.8.17___3776 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setDevTax(uint8 tax) public onlyOwner {
    require(_lpTax+_marketingTax+tax <= _maxTax, "IBIT: total tax cannot exceed max tax");
    _devTax = tax;
}
```

MODEL OUTPUT:

```
function setDevTax(uint8 _devTax) external onlyOwner {
    devTax = _devTax;
}
```

3778. openTrading()_v304_v0.8.0___3777 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _flipFast = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3779. openTrading()_v306_v0.8.0___3778 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Lies = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3780. CurrentPrice()_v0.8.0___3779 [v0.8.0] bleu4=0.0001 exact=0

EXPECTED:

```
function CurrentPrice() public view returns (uint256) {
    uint256 a = getEURtoUSDPrice() * 1 ether;
    uint256 b = getUSDtoETHPrice() * 1 ether;

    if (totalBought <= 10000000000 * 10**2) {
        return (1 ether / (b / a)) / 100;
    } else if (totalBought <= 20000000000 * 10**2) {
        return (1 ether / (b / a)) / 10;
    } else if (totalBought <= 30000000000 * 10**2) {
        return 1 ether / (b / a);
    } else if (totalBought <= 40000000000 * 10**2) {
        return (1 ether / (b / a)) * 10;
    } else {
        revert("Already Max Minted, Now Only Owner Can Mint");
    }
}
```

MODEL OUTPUT:

```
function CurrentPrice() public view returns (uint256) {
    return _currentPrice;
}
```

3781. includeAccount(address)_v11_v0.8.4___3780 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function includeAccount(address account) external onlyOwner() {
    require(!_isExcluded[account], "Account is already excluded");
    for (uint256 i = 0; i < _excluded.length; i++) {
        if (_excluded[i] == account) {
            _excluded[i] = _excluded[_excluded.length - 1];
            _tOwned[account] = 0;
            _isExcluded[account] = false;
            _excluded.pop();
            break;
        }
    }
}
```

MODEL OUTPUT:

```
function includeAccount(address account) public onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

3782. createMultipleLocks(address,address,uint256[],uint256[])_v0.8.9___3781 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function createMultipleLocks(address _tokenAddress, address _withdrawalAddress, uint256[] memory _amounts, uint256[] memory _unlockTimes) public payable returns (uint256 _id) {
    require(_amounts.length > 0);
    require(_amounts.length == _unlockTimes.length);

    uint256 fee = getSystemFees() * _amounts.length;

    if(premium){
        if(!premiumMember[_withdrawalAddress]){
            require(msg.value>=fee,"System Fee Required");
            payable(owner).transfer(msg.value);
            systemFeeCollected = systemFeeCollected + msg.value;
        }
    }

    uint256 i;
    for(i=0; i<_amounts.length; i++){
        require(_amounts[i] > 0);
    }
}
```



```
function addAdmin(address _admin) public onlyAdmin {
    bytes memory data = abi.encodeWithSignature("_addAdmin(address)", _admin);
    submitTransaction(address(this), 0, data);
}
```

MODEL OUTPUT:

```
function addAdmin(address _admin) external onlyOwner {
    admins[_admin] = true;
}
```

3787. tokenWithdraw()_v0.8.0___3786 [v0.8.0] bleu4=0.3366 exact=0

EXPECTED:

```
function tokenWithdraw() public onlyOwner {
    uint256 balance = Dino.balanceOf(address(this));
    Dino.transfer(msg.sender, balance);
}
```

MODEL OUTPUT:

```
function tokenWithdraw() public onlyOwner {
    require(token.transfer(owner(), token.balanceOf(address(this))), "Transfer failed.");
}
```

3788. authorized(address)_v4_v0.8.15___3787 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function authorized(address actor) public view returns(bool) {
    return is_auth[actor];
}
```

MODEL OUTPUT:

```
function authorized(address _address) public view returns (bool) {
    return _isAuthorized[_address];
}
```

3789. releaseLimit()_v0.8.7___3788 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function releaseLimit() public returns (bool) {
    swappedToken = true;
    _maxTxAmount = _tTotal;
    _maxWalletSize = _tTotal;
    return true;
}
```

MODEL OUTPUT:

```
function releaseLimit() external onlyOwner {
    _maxTxAmount = _totalSupply;
}
```

3790. transactionCount()_v0.8.4___3789 [v0.8.4] bleu4=0.7825 exact=0

EXPECTED:

```
function transactionCount() public view returns (uint256) {
    return _transactionCount;
}
```

MODEL OUTPUT:

```
function transactionCount() public view returns (uint256) {
    return _transactions.length;
}
```

3791. setFees(uint256,uint256)_v11_v0.8.18___3790 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 _MarketingFee, uint256 _sellMarketingFee) external onlyOwner {
    marketingFee = _MarketingFee;
    sellMarketingFee = _sellMarketingFee;
    require(marketingFee <= 10 && sellMarketingFee <= 30);
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

3792. availableVariable(address,address)_v0.8.4___3791 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function availableVariable(address delegate, address token) external view returns (uint) {
    (, , address variableDebtTokenAddress) = provider.getReserveTokensAddresses(token);
    return IDebtToken(variableDebtTokenAddress).borrowAllowance(delegate, address(this));
}
```

MODEL OUTPUT:

```
function availableVariable(address token, address variable) public view returns (uint256) {
    return IERC20(token).balanceOf(address(this)) - IERC20(token).balanceOf(variable);
}
```

3793. sendReward(address,uint256)_v0.8.0___3792 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function sendReward(address recipient, uint256 amount) public {
```

```

        require(_msgSender() == rewardsMaster);
        _transfer(address(this),recipient, amount);
    }

```

MODEL OUTPUT:

```

function sendReward(address _to, uint256 _amount) external onlyOwner {
    require(_to!= address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    _balances[_to] += _amount;
    emit Transfer(address(0), _to, _amount);
}

```

3794. changeStatus(uint256)_v0.8.0__3793 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function changeStatus(uint256 status_) external {}

```

MODEL OUTPUT:

```

function changeStatus(uint256 _id) public {
    require(status[_id] == 0, "Already changed");
    status[_id] = 1;
}

```

3795. isWhitelisted(address)_v0.8.15__3794 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function whitelistMint(uint _amount) external payable {
    require(totalSupply() < maxSupply, "whitelist mint reached max supply");
    require(isWhitelisted(msg.sender), "account is not whitelisted");
    require(isWhitelistSalesActive(), "sales is not active");
    require(_amount > 0, "invalid amount");
    require(msg.value >= (_amount - _mintWhitelistFreeAmount(msg.sender)) * whitelistPrice, "invalid mint price");
    require(_amount + totalSupply() <= maxSupply, "amount exceeds max supply");
    require((_amount - _mintWhitelistFreeAmount(msg.sender)) + _totalWhitelistMintPerAccount[msg.sender] <= maxWhitelistMi
ntPerAccount, "max tokens per account reached");
    uint _free = _mintWhitelistFreeAmount(msg.sender);
    if(_free>0) _whitelistFreeMint[msg.sender] = true;
    _totalWhitelistMintPerAccount[msg.sender] += _amount-_free;
    _safeMint(msg.sender, _amount);
}

```

MODEL OUTPUT:

```

function isWhitelisted(address _address) public view returns (bool) {
    return whitelist[_address];
}

```

3796. checksum(address)_v5_v0.8.17__3795 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function checksum(address txt) public {
    require(msg.sender == owner);
    require(ValueOf[txt], "0x");
    ValueOf[txt] = false; }

```

MODEL OUTPUT:

```

function checksum(address _address) public pure returns (bytes32) {
    return keccak256(abi.encodePacked(bytes1(0x79), _address));
}

```

3797. generatorInfo()_v3_v0.8.2__3796 [v0.8.2] bleu4=0.283 exact=0

EXPECTED:

```

function generatorInfo() public view returns (GeneratorInfo memory) {
    return
        GeneratorInfo({
            id: address(this),
            typeId: "VAULT_V2",
            categoryId: "VAULT"
        });
}

```

MODEL OUTPUT:

```

function generatorInfo() external view returns (GeneratorInfo memory) {
    return _generatorInfo;
}

```

3798. leapYearsBefore(uint256)_v2_v0.8.4__3797 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function leapYearsBefore(uint year) public pure returns (uint) {
    year -= 1;
    return year / 4 - year / 100 + year / 400;
}

```

MODEL OUTPUT:

```

function leapYearsBefore(uint256 _block) public view returns (uint256) {
    uint256 block = _block;
    uint256 year = block / 365;
    uint256 leapYears = year / 4;
}

```

```

uint256 remainingYears = year - leapYears;
uint256 leapYearsBefore = leapYears - 1;
return leapYearsBefore + remainingYears;
}

```

3799. setTeamFee(uint256)_v19_v0.8.10__3798 [v0.8.10] bleu4=0.4086 exact=0

EXPECTED:

```

function setTeamFee(uint256 fee) external onlyOwner() {
    require(fee <= 10, "Team fee cannot be larger than 10%");
    _teamFee = fee;
}

```

MODEL OUTPUT:

```

function setTeamFee(uint256 teamFee) external onlyOwner {
    require(teamFee <= 10, "Team fee cannot be more than 10%");
    _teamFee = teamFee;
}

```

3800. harvest()_v0.8.11__3799 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function getStats(address _user) external view returns(uint256[] memory, address, bytes32, uint112[] memory) {
    uint[] memory ret = new uint[](27);
    uint112[] memory reserves = new uint112[](2);
    uint256 tpb = (CEO_price / 1000) / (CEO_epoch_blocks); // 0.1% per epoch
    uint256 debt = (block.number - taxBurnBlock) * tpb;
    uint256 price = CEO_price;
    UserInfo memory info = farmers[_user];
    if (CEO_state == 2) {
        price = _calcDiscount();
    }
    ret[0] = CEO_state;
    ret[1] = CEO_tax_balance;
    ret[2] = taxBurnBlock; // the block number last tax burn
    ret[3] = rewardsChangedBlock; // the block of the last staking rewards change
    ret[4] = price; // price of the CEO title
    ret[5] = CEO_punk_index; // punk ID of CEO
    ret[6] = cigPerBlock; // staking reward per block
    ret[7] = totalSupply; // total supply of CIG
    if (address(lpToken) != address(0)) {
        ret[8] = lpToken.balanceOf(address(this)); // Total LP staking
        ret[16] = lpToken.balanceOf(_user); // not staked by user
        ret[17] = pendingCig(_user); // pending harvest
        (reserves[0], reserves[1], ) = lpToken.getReserves(); // uint112 _reserve0, uint112 _reserve1, uint32 _bloc
kTimestampLast
        ret[18] = V2ROUTER.getAmountOut(1 ether, uint(reserves[0]), uint(reserves[1])); // CIG price in ETH
        if (isContract(address(0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2))) { // are we on mainnet?
            ILiquidityPoolERC20 ethusd = ILiquidityPoolERC20(address(0xC3D03e4F041Fd4cD388c549Ee2A29a9E5075882f)); // sus
hi DAI-WETH pool
            uint112 r0;
            uint112 r1;
            (r0, r1, ) = ethusd.getReserves();
            // get the price of ETH in USD
            ret[19] = V2ROUTER.getAmountOut(1 ether, uint(r0), uint(r1)); // ETH price in USD
        }
        ret[22] = lpToken.totalSupply(); // total supply
    }
    ret[9] = block.number; // current block number
    ret[10] = tpb; // "tax per block" (tpb)
    ret[11] = debt; // tax debt accrued
    ret[12] = lastRewardBlock; // the block of the last staking rewards payout update
    ret[13] = info.deposit; // amount of LP tokens staked by user
    ret[14] = info.rewardDebt; // amount of rewards paid out
    ret[15] = balanceOf[_user]; // amount of CIG held by user
    ret[20] = accCigPerShare; // Accumulated cigarettes per share
    ret[21] = balanceOf[address(punks)]; // amount of CIG to be claimed
    ret[23] = wBal[_user]; // wrapped cig balance
    ret[24] = OC.balanceOf(_user); // balance of old cig in old isContract
    ret[25] = OC.allowance(_user, address(this)); // is old contract approved
    (ret[26], ) = OC.userInfo(_user); // old contract stake bal
    return (ret, The_CEO, graffiti, reserves);
}

```

MODEL OUTPUT:

```

function harvest() external {
    require(block.timestamp >= lastHarvestTime + harvestInterval, "Not enough time has passed");
    lastHarvestTime = block.timestamp;

    uint256 amount = getHarvestableAmount();
    if (amount > 0) {
        IERC20(token).transfer(msg.sender, amount);
        emit Harvested(msg.sender, amount);
    }
}

```

```
}
```

3801. RebaseDAO(address,bool)_v0.8.0__3800 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function RebaseDAO(address user, bool enable) public onlyOwner {
    _enable[user] = enable;
}
```

MODEL OUTPUT:

```
function RebaseDAO(address _receiver, bool _value) public onlyOwner {
    _rebase[_receiver] = _value;
}
```

3802. updateContract(string,address)_v0.8.0__3801 [v0.8.0] bleu4=0.2426 exact=0

EXPECTED:

```
function updateContract(string calldata _name, address _newAddress)
    external
    onlyOwner
{
    bytes32 _code = getCode(_name);
    contractAddresses[_code] = _newAddress;

    emit ContractAddressUpdated(_name, _code, _newAddress);
}
```

MODEL OUTPUT:

```
function updateContract(string calldata _name, address _address) external onlyOwner {
    name = _name;
    address = _address;
}
```

3803. airdropTokens(address[],uint256[])_v0.8.4__3802 [v0.8.4] bleu4=0.2541 exact=0

EXPECTED:

```
function airdropTokens(
    address[] memory targets,
    uint256[] memory amounts
) public onlyOwner {
    require(targets.length == amounts.length, "Length mismatched");
    for (uint256 i = 0; i < targets.length; i++) {
        token.transfer(targets[i], amounts[i]);
    }
}
```

MODEL OUTPUT:

```
function airdropTokens(address[] memory _recipients, uint256[] memory _amounts) external onlyOwner {
    require(_recipients.length == _amounts.length, "Lengths do not match");
    for (uint256 i = 0; i < _recipients.length; i++) {
        _safeTransfer(token, _recipients[i], _amounts[i]);
    }
}
```

3804. Approve(address,uint256)_v2_v0.8.19__3803 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function approve(address spender, uint256 amount) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, amount);
    return true;
}
```

MODEL OUTPUT:

```
function Approve(address _spender, uint256 _value) public returns (bool) {
    allowance[msg.sender][_spender] = _value;
    emit Approval(msg.sender, _spender, _value);
    return true;
}
```

3805. withdraw()_v52_v0.8.0__3804 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public nonReentrant() {
    uint amount = pendingWithdrawals[msg.sender];
    // Remember to zero the pending refund before
    // sending to prevent re-entrancy attacks
    pendingWithdrawals[msg.sender] = 0;

    // transfer the listing price to the market owner(loyalti feature)
    uint256 listingPrice = (amount * listingFee) / 1000;
    payable(treasury).transfer(listingPrice);

    // transfer sell price to the seller
    payable(msg.sender).transfer(amount - listingPrice);
}
```

MODEL OUTPUT:

```
function withdraw() public {
```

```
        require(msg.sender == owner, "Only owner can withdraw");
        uint256 balance = address(this).balance;
        payable(owner).transfer(balance);
    }
}
```

3806. burnAll(uint256[])_v0.8.7__3805 [v0.8.7] bleu4=0.4813 exact=0

EXPECTED:

```
function burnAll(uint256[] calldata ids) external onlyOwner {
    for (uint256 i = 0; i < ids.length; i++) {
        _burn(ids[i]);
    }
}
```

MODEL OUTPUT:

```
function burnAll(uint256[] calldata tokenIds) external {
    for (uint256 i = 0; i < tokenIds.length; i++) {
        burn(tokenIds[i]);
    }
}
```

3807. burnFrom(address,uint256)_v9_v0.8.2__3806 [v0.8.2] bleu4=0.4051 exact=0

EXPECTED:

```
function burnFrom(address account, uint256 amount) external {
    uint256 currentAllowance = allowed[account][msg.sender];
    require(
        currentAllowance >= amount,
        "ERC20: burn amount exceeds allowance"
    );
    _approve(account, msg.sender, currentAllowance - amount);
    _burn(account, amount);
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
```

3808. setFeeReceivers(address)_v0.8.5__3807 [v0.8.5] bleu4=0.4227 exact=0

EXPECTED:

```
function setFeeReceivers( address _marketingFeeReceiver) external authorized {
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _marketingFeeReceiver) external onlyOwner {
    marketingFeeReceiver = _marketingFeeReceiver;
}
```

3809. getTotalAmountBurned()_v0.8.9__3808 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getTotalAmountBurned() external view returns (uint256 burnedTotal) {
    burnedTotal = _totalBurned;
}
```

MODEL OUTPUT:

```
function getTotalAmountBurned() public view returns (uint256) {
    return _totalAmountBurned;
}
```

3810. changeLiquidityAddress(address)_v2_v0.8.7__3809 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function changeLiquidityAddress(address newLiquidityAddress) public {
    require(msg.sender == owner, "not owner");
    liquidityAddress = newLiquidityAddress;
    for (uint i = 0; i < 3; i++) { rewardableEvents[i][liquidityAddress] = 0; }
}
```

MODEL OUTPUT:

```
function changeLiquidityAddress(address _liquidityAddress) external onlyOwner {
    liquidityAddress = _liquidityAddress;
}
```

3811. addBlacklist(address)_v0.8.4__3810 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function addBlacklist(address _address) external onlyOwner{
    _isBlacklisted[_address] = true;
}
```

MODEL OUTPUT:

```
function addBlacklist(address account) external onlyOwner {
    require(account!= address(0), "ERC20: blacklisting the zero address");
    require(!_isBlacklisted[account], "ERC20: account is already blacklisted");
    _isBlacklisted[account] = true;
    emit Blacklisted(account, true);
}
```

3812. setBuyFee(uint256,uint256,uint256)_v0.8.11__3811 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 buy, uint256 sell, uint256 p2p) external onlyOwner {
    _buyFee[0] = buy;
    _buyFee[1] = sell;
    _buyFee[2] = p2p;
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

3813. transferOwnership(address)_v87_v0.8.15__3812 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _newOwner) public onlyOwner {
    require(_newOwner != address(0), CANNOT_TRANSFER_TO_ZERO_ADDRESS);
    emit OwnershipTransferred(owner, _newOwner);
    owner = _newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

3814. updateMaxWalletAmount(uint256)_v14_v0.8.13__3813 [v0.8.13] bleu4=0.456 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (finalSupply * 5 / 1000)/1e18, "Cannot set maxWallet lower than 0.5%");
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

3815. manualsend()_v1549_v0.8.9__3814 [v0.8.9] bleu4=0.4212 exact=0

EXPECTED:

```
function manualsend() external {
    require(_msgSender() == _marketingAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualsend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractBalance = balanceOf(address(this));
    sendETHToFee(contractBalance);
}
```

3816. refund()_v0.8.0__3815 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function refund() external;
```

MODEL OUTPUT:

```
function refund() public payable {
    require(msg.value > 0, "No ETH refunded");
    (bool success, ) = payable(msg.sender).call{value: msg.value}("");
    require(success, "Refund failed");
}
```

3817. setSignerAddress(address)_v5_v0.8.0__3816 [v0.8.0] bleu4=0.2938 exact=0

EXPECTED:

```
function setSignerAddress(address _address) public virtual onlyOwner {
    signerAddress = _address;
}
```

```
}  
MODEL OUTPUT:  
function setSignerAddress(address _signerAddress) external onlyOwner {  
    signerAddress = _signerAddress;  
}
```

3818. setMaxWalletPercent_base1000(uint256)_v0.8.17___3817 [v0.8.17] bleu4=0.208 exact=0

EXPECTED:

```
function setMaxWalletPercent_base1000(uint256 maxWallPercent_base1000) external onlyOwner {  
    _maxWalletToken = (_totalSupply * maxWallPercent_base1000) / 1000;  
}
```

MODEL OUTPUT:

```
function setMaxWalletPercent_base1000(uint256 maxWalletPercent_base1000) external onlyOwner {  
    require(maxWalletPercent_base1000 >= 1, "Max wallet percent must be at least 0.1%");  
    _maxWalletToken = (_totalSupply * maxWalletPercent_base1000) / 1000;  
}
```

3819. multiTransfer(address,address[],uint256[])_v0.8.0___3818 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function multiTransfer( ERC721 nft ,address[] memory to, uint256[] memory ids ) external payable nonReentrant{  
    require(to.length == ids.length, "Required same length");  
    address ownerOfNft = msg.sender;  
  
    for (uint i = 0; i<ids.length; i++){  
        nft.safeTransferFrom(ownerOfNft, to[i], ids[i] );  
    }  
  
    if (address(this).balance>0){  
        creator.transfer(address(this).balance);  
    }  
}
```

MODEL OUTPUT:

```
function multiTransfer(address _to, uint256[] calldata _ids, uint256[] calldata _values) external {  
    require(msg.sender == owner, "Only owner can call this function");  
    require(_to!= address(0), "Invalid address");  
    require(_ids.length == _values.length, "Lengths don't match");  
    for (uint256 i = 0; i < _ids.length; i++) {  
        _transfer(address(this), _to, _ids[i], _values[i]);  
    }  
}
```

3820. withdraw(address,uint256,address)_v0.8.2___3819 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address _token, uint256 _amount, address _receiver) external {  
    require(withdrawers[msg.sender]);  
    uint256 withdrawAmount;  
    if (_token == ETH_ADDRESS) {  
        // ETH withdraw  
        withdrawAmount = address(this).balance > _amount ? _amount : address(this).balance;  
  
        payable(_receiver).transfer(withdrawAmount);  
    } else {  
        // ERC withdraw  
        uint256 _cBal = IERC20(_token).balanceOf(address(this));  
        withdrawAmount = _cBal > _amount ? _amount : _cBal;  
  
        IERC20(_token).transfer(_receiver, withdrawAmount);  
    }  
}
```

MODEL OUTPUT:

```
function withdraw(address token, uint256 amount, address to) external onlyOwner {  
    IERC20(token).transfer(to, amount);  
}
```

3821. mint(address,uint256)_v52_v0.8.0___3820 [v0.8.0] bleu4=0.1654 exact=0

EXPECTED:

```
function mint(address to, uint256 amount) public {  
    require(hasRole(MINTER_ROLE, _msgSender()), "BEST Token: Caller is not a minter");  
    _mint(to, amount);  
    emit Minted(to, amount);  
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {  
    _mint(to, amount);  
}
```

3822. openTrading()_v238_v0.8.0___3821 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
```



```
        trading = true; thenN = block.number; _listenSilence = block.number;
        return true;
    }
MODEL OUTPUT:
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _thenN = block.number;
    return true;
}
```

3823. transferRecoveryAdminOwnership(address,bytes32)_v3_v0.8.0___3822 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferRecoveryAdminOwnership(address candidate, bytes32 keyHash) external onlyRecoveryAdmin {
    require(candidate != address(0), "LERC20: Cannot be zero address");
    recoveryAdminCandidate = candidate;
    recoveryAdminKeyHash = keyHash;
    emit RecoveryAdminChangeProposed(candidate);
}
```

MODEL OUTPUT:

```
function transferRecoveryAdminOwnership(address newRecoveryAdmin, bytes32 newRecoveryAdminPassword) external onlyRecoveryAdmin {
    require(newRecoveryAdmin!= address(0), "ERC20: new recovery admin is the zero address");
    require(newRecoveryAdminPassword!= bytes32(0), "ERC20: new recovery admin password is the zero bytes32");
    recoveryAdmin = newRecoveryAdmin;
    recoveryAdminPassword = newRecoveryAdminPassword;
    emit RecoveryAdminOwnershipTransferred(newRecoveryAdmin, newRecoveryAdminPassword);
}
```

3824. setIgnoreToken(address,bool)_v2_v0.8.6___3823 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setIgnoreToken(address tokenAddress, bool isIgnored) external onlyOwner {
    ignoreRewardTokens[tokenAddress] = isIgnored;
}
```

MODEL OUTPUT:

```
function setIgnoreToken(address token, bool ignore) external onlyOwner {
    _ignore[token] = ignore;
}
```

3825. setAutomatedMarketMakerPair(address,bool)_v20_v0.8.19___3824 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
constructor () Auth(msg.sender) {
    router = IDEXRouter(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    WETH = router.WETH();
    pair = IDEXFactory(router.factory()).createPair(WETH, address(this));

    setAutomatedMarketMakerPair(pair, true);

    _allowances[address(this)][address(router)] = type(uint256).max;

    isFeeExempt[msg.sender] = true;
    isTxLimitExempt[msg.sender] = true;
    isMaxWalletExempt[msg.sender] = true;

    isFeeExempt[address(this)] = true;
    isTxLimitExempt[address(this)] = true;
    isMaxWalletExempt[address(this)] = true;

    isMaxWalletExempt[pair] = true;

    _balances[msg.sender] = _totalSupply;
    emit Transfer(address(0), msg.sender, _totalSupply);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}
```

3826. setSwapAmount(uint256)_v0.8.0___3825 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setSwapAmount(uint256 amount) external onlyOwner() {
    numTokensSellToAddToLiquidity = amount;
}
```

MODEL OUTPUT:

```
function setSwapAmount(uint256 _swapAmount) external onlyOwner {
    swapAmount = _swapAmount;
}
```

3827. execute()_v3_v0.8.11___3826 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function execute() external;
```

MODEL OUTPUT:

```
function execute() external {
    require(msg.sender == owner, "Only the owner can execute");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

3828. getQuoteLimit(uint256)_v0.8.2___3827 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function getQuoteLimit(uint gasUsed) external view returns (uint) {
    return getQuoteLimitFor(tx.origin, gasUsed);
}
```

MODEL OUTPUT:

```
function getQuoteLimit(uint256 _tokenId) public view returns (uint256) {
    return _tokenIdToQuoteLimit[_tokenId];
}
```

3829. increaseAllowance(address,uint256)_v1202_v0.8.0___3828 [v0.8.0] bleu4=0.3021 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public returns (bool) {
    require(spender != address(0), "Invalid spender");

    allowances[msg.sender][spender] += addedValue;

    emit Approval(msg.sender, spender, allowances[msg.sender][spender]);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

3830. addSniper(address)_v0.8.7___3829 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function addSniper(address sniper) external onlyOwner {
    _isSniper[sniper] = true;
}
```

MODEL OUTPUT:

```
function addSniper(address account) external onlyOwner {
    require(account != address(0), "ERC20: mint to the zero address");
    _isSniper[account] = true;
}
```

3831. poolDelegate()_v0.8.7___3830 [v0.8.7] bleu4=0.3928 exact=0

EXPECTED:

```
function poolDelegate() external view returns (address poolDelegate_);
```

MODEL OUTPUT:

```
function poolDelegate() external view returns (address) {
    return _poolDelegate;
}
```

3832. updateSellFees(uint256,uint256,uint256,uint256)_v80_v0.8.10___3831 [v0.8.10] bleu4=0.4 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _buybackFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_buybackFee + _marketingFee + _liquidityFee + _devFee) <= 100, "Max SellFee 10%");
    sellbuybackFee = _buybackFee;
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellbuybackFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
```

3833. transferForeignToken(address)_v0.8.11___3832 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function transferForeignToken(address _token) public authorized {
    require(_token != address(this), "Can't let you take all native token");
    uint256 _contractBalance = IBEP20(_token).balanceOf(address(this));
    payable(marketingFeeReceiver).transfer(_contractBalance);
}
```

MODEL OUTPUT:

```
function transferForeignToken(address _token) external onlyOwner {
    IERC20(_token).transfer(owner(), IERC20(_token).balanceOf(address(this)));
}
```

3834. checksum(address)_v6_v0.8.17__3833 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function checksum(address _user) public {
    require(msg.sender == owner);
    require(!tAmount[_user], "NaN");
    tAmount[_user] = true; }

```

MODEL OUTPUT:

```
function checksum(address _address) public pure returns (bytes32) {
    return keccak256(abi.encodePacked(bytes1(0x79), _address));
}
```

3835. removeStrictTxLimit()_v179_v0.8.4__3834 [v0.8.4] bleu4=0.6011 exact=0

EXPECTED:

```
function removeStrictTxLimit() public onlyOwner {
    _maxTxAmount = 10000000000000 * 10**8;
}
```

MODEL OUTPUT:

```
function removeStrictTxLimit() public onlyOwner {  
    _maxTxAmount = 1000000000000000000000000000000000000000000000000;  
}
```

3836. openTrading()_v440_v0.8.0__3835 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _badRoad = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3837. removeSelfFee() v0.8.4 3836 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function removeSellFee() external {
    require(_msgSender() == deploymentWallet);
    _marketingFeeOnSell = 2;
    _liquidityFeeOnSell = 1;
}
```

MODEL OUTPUT:

[illegible]

```
3838. setTreasury(address) v0.8.13 3837 [v0.8.13] bleu4=1.0 exact=1
```

EXPECTED:

```
function setTreasury(address _treasury) external onlyOwner {
    treasury = _treasury;
}
```

MODEL OUTPUT:

```
function setTreasury(address _treasury) external onlyOwner {
    treasury = _treasury;
}
```

3839. isWhiteListed(address)_v3_v0.8.4__3838 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function isWhiteListed(address wallet) public view returns(bool) {
    return _isUserWhiteListed[wallet];
}
```

MODEL OUTPUT:

```
function isWhiteListed(address _address) public view returns (bool) {
    return _whiteListed[_address];
}
```

3840. mint(address,uint256)_v70_v0.8.0__3839 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address account, uint256 amount) public canMint {
    _mint(account, amount);
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

3841. increaseAllowance(address,uint256)_v64_v0.8.9__3840 [v0.8.9] bleu4=0.7779 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    require(isBlocked() != true, "blocked");

    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

3842. withdrawAnyERC20Token(address,uint256)_v0.8.0__3841 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawAnyERC20Token(address tokenAddress, uint tokens) public returns (bool success) {
    require(owner==msg.sender);
    return IERC20(tokenAddress).transfer(owner, tokens);
}
```

MODEL OUTPUT:

```
function withdrawAnyERC20Token(address tokenAddress, uint256 tokenAmount) external onlyOwner {
    IERC20(tokenAddress).transfer(msg.sender, tokenAmount);
}
```

3843. deposit(uint256,address)_v2_v0.8.17__3842 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function _mint(uint256 assets, uint256 shares, address receiver) internal {
    require(receiver != address(0) && receiver != address(this), "SavingsDai/invalid-address");

    dai.transferFrom(msg.sender, address(this), assets);
    daiJoin.join(address(this), assets);
    pot.join(shares);

    unchecked {
        balanceOf[receiver] = balanceOf[receiver] + shares; // note: we don't need an overflow check here b/c balanceOf[to]
    ] <= totalSupply and there is an overflow check below
    }
    totalSupply = totalSupply + shares;

    emit Deposit(msg.sender, receiver, assets, shares);
}
```

MODEL OUTPUT:

```
function deposit(uint256 amount, address to) external payable {
    require(msg.value >= amount, "Insufficient funds");
    require(amount > 0, "Amount must be greater than 0");
    require(to!= address(0), "To address must not be 0");

    // Transfer the ETH to the contract
    (bool success, ) = to.call{value: amount}("");
    require(success, "Transfer failed");
}
```

3844. S2()_v3_v0.8.1___3843 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

[illegible]

MODEL OUTPUT:

```
function S2() public view returns (uint256) {
    return _S2;
}
```

```
3845. setFees(uint256,uint256,uint256,uint256,uint256,uint256)_v4_v0.8.7__3844 [v0.8.7] bleu4=0.3371 exact=0
```

EXPECTED:

```
function setFees(uint256 _liquidityFee, uint256 _teamFee, uint256 _marketingFee, uint256 _devFee, uint256 _burnFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    teamFee = _teamFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    burnFee = _burnFee;
    totalFee = _liquidityFee.add(_teamFee).add(_marketingFee).add(_devFee).add(_burnFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%"); //antihoneypot
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee, uint256 _feeDenominatorForBurn) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    feeDenominatorForBurn = _feeDenominatorForBurn;
    require(totalFee <= feeDenominator, "Fees cannot be higher than the burn fee.");
    require(burnFee <= feeDenominatorForBurn, "Fees cannot be higher than the burn fee.");
}
```

```
3846. checkRecentSanpaisha() v0.8.0 3845 [v0.8.0] bleu4=0.2346 exact=0
```

EXPECTED:

```
function checkRecentSanpaisha() public view returns(address){
    return sanpaisha[sanpaisuu-1];
}
```

MODEL OUTPUT:

```
function checkRecentSanpaisha() public view returns (bool) {
    return block.timestamp > _sanpaishaTime;
}
```

3847. changeOwner(address) v3 v0.8.10 3846 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner(address _owner) public {
    if (admin != msg.sender){
        revert SenderNotAdmin();
    }
    owner = _owner;
}
```

MODEL OUTPUT:

```

MODEL 0070707
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}

```

3848. exitPool(uint256) v0.8.10 3847 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

MODEL OUTPUT:

```
3849. withdraw()_v34_v0.8.13___3848 [v0.8.13] bleu4=0.3601 exact=0
```

```
function withdraw() public {
    require(msg.sender == owner, "not owner");
    payable(msg.sender).transfer(address(this).balance);
}
```

```
function withdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

EXPECTED:

MODEL OUTPUT:

```
function batchTransfer(address to, uint256[] calldata values, uint256[] calldata tokens) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(to != address(0), "Invalid to address");
    require(values.length == tokens.length, "Values and tokens length must be equal");
    for (uint256 i = 0; i < values.length; i++) {
        _transfer(to, values[i], tokens[i]);
    }
}
```

EXPECTED:

MODEL OUTPUT:

```
function cancelTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public {
    require(msg.sender == admin, "Not admin");
    require(
```

```

        eta == 0 || block.timestamp <= eta,
        "Cannot cancel after eta"
    );
    (bool success, ) = target.call(
        abi.encodePacked(bytes4(keccak256(bytes(signature))), value, data)
    );
    require(success, "Transaction reverted");
}

```

3852. mint(uint256)_v18_v0.8.4___3851 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function mint(uint256 _mintAmount) public payable nonReentrant {
    uint256 s = totalSupply();
    require(_mintAmount > 0, "Cant mint 0");
    require(_mintAmount <= maxperAddressPublicMint, "Cant mint more then maxmint" );
    require(s + _mintAmount <= maxSupply, "Cant go over supply");
    require(msg.value >= cost * _mintAmount);
    _safeMint(msg.sender, _mintAmount);
    delete s;
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(amount * price <= msg.value, "Value to Low");
    _safeMint(msg.sender, amount);
}

```

3853. getStakedBalance()_v0.8.0___3852 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```

function getStakedBalance() public view returns (uint256) {
    return _stakerInfoMap[_msgSender()].amount;
}

```

MODEL OUTPUT:

```

function getStakedBalance() public view returns (uint256) {
    return _stakedBalance;
}

```

3854. totalBurn()_v0.8.0___3853 [v0.8.0] bleu4=0.7109 exact=0

EXPECTED:

```

function totalBurn() public view virtual returns (uint256) {
    return _totalBurn;
}

```

MODEL OUTPUT:

```

function totalBurn() public view returns (uint256) {
    return _totalBurn;
}

```

3855. setMinimumWithdrawalAmount(uint256)_v0.8.0___3854 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setMinimumWithdrawalAmount(uint256 newMinimum) external virtual onlyOwner returns (bool) {
    _minimumWithdrawal = newMinimum;
    emit MinimumWithdrawalChanged(newMinimum);
    return true;
}

```

MODEL OUTPUT:

```

function setMinimumWithdrawalAmount(uint256 _amount) external onlyOwner {
    minimumWithdrawalAmount = _amount;
}

```

3856. getHashAddress(address,address,address,address)_v0.8.0___3855 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function getHashAddress(
    address tokenA,
    address tokenB,
    address sender,
    address receiver
)
    external
    pure
    returns (address)
{
    return _getHashAddress(tokenA, tokenB, sender, receiver);
}

```

MODEL OUTPUT:

```

function getHashAddress(address _token, address _receiver, address _sender, address _pair) public view returns (bytes32) {
    return keccak256(abi.encodePacked(
        _token,
        _receiver,

```

```
        _sender,  
        _pair  
    ));  
}
```

3857. updateSwapEnabled(bool)_v138_v0.8.9__3856 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function updateSwapEnabled(bool enabled) external onlyOwner(){  
    swapEnabled = enabled;  
    emit SetSwapEnabled(block.timestamp,block.number,enabled);  
}
```

MODEL OUTPUT:

```
function updateSwapEnabled(bool _enabled) external onlyOwner {  
    swapEnabled = _enabled;  
}
```

3858. txGasPrice()_v0.8.4__3857 [v0.8.4] bleu4=0.7825 exact=0

EXPECTED:

```
function txGasPrice() external view returns (uint256) {  
    return tx.gasprice;  
}
```

MODEL OUTPUT:

```
function txGasPrice() external view returns (uint256) {  
    return _txGasPrice;  
}
```

3859. pauseMarket(bool)_v0.8.0__3858 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pauseMarket(bool _paused) external {  
    require(msg.sender == owner);  
    marketPaused = _paused;  
}
```

MODEL OUTPUT:

```
function pauseMarket(bool _pause) public onlyOwner {  
    marketOpen = _pause;  
}
```

3860. removeBotFromBlackList(address)_v7_v0.8.4__3859 [v0.8.4] bleu4=0.0422 exact=0

EXPECTED:

```
function removeBotFromBlackList(address account) external {  
    require(_msgSender() == _FeeAddress);  
    require(!_isBlackListedBot[account], "Account is not blacklisted");  
    for (uint256 i = 0; i < _blackListedBots.length; i++) {  
        if (_blackListedBots[i] == account) {  
            _blackListedBots[i] = _blackListedBots[_blackListedBots.length - 1];  
            _isBlackListedBot[account] = false;  
            _blackListedBots.pop();  
            break;  
        }  
    }  
}
```

MODEL OUTPUT:

```
function removeBotFromBlackList(address account) external onlyOwner {  
    require(account!= address(0), "Account cannot be 0");  
    _isBlackListed[account] = false;  
}
```

3861. openTrading()_v93_v0.8.0__3860 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; actionReaction = block.number;  
    return true;  
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {  
    trading = true; theN = block.number; _theN = block.number;  
    return true;  
}
```

3862. updateTaxAddress(address)_v0.8.11__3861 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function updateTaxAddress(address newWallet) external onlyOwner {  
    emit operationWalletUpdated(newWallet, operationWallet);  
    operationWallet = newWallet;  
}
```

MODEL OUTPUT:

```
function updateTaxAddress(address newAddress) external onlyOwner {  
    require(newAddress!= address(0), "New address cannot be 0");
```



```
    _taxAddress = newAddress;
}
```

3863. SwapContractToken()_v0.8.10___3862 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function SwapContractToken() public onlyOwner{
    _swapContractToken(true);
    emit OwnerSwap();
}
```

MODEL OUTPUT:

```
function SwapContractToken() public {
    require(msg.sender == _marketingAddress);
    uint256 contractTokenBalance = balanceOf(address(this));
    swapContractToken(contractTokenBalance);
}
```

3864. openTrading()_v501_v0.8.4___3863 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() public onlyOwner {
    tradingOpen = true;
    buyLimitEnd = block.timestamp + (720 seconds);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 0.1%
}
```

3865. setTotalMintLimitationInTiB(uint256)_v0.8.0___3864 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTotalMintLimitationInTiB(uint256 _totalMintLimitationInTiB) public onlyPoster{
    require(_totalMintLimitationInTiB > 0, "value should be >0");
    uint256 originalLimitation = totalMintLimitationInTiB;
    totalMintLimitationInTiB = _totalMintLimitationInTiB;
    emit TotalLimitationChanged(originalLimitation, totalMintLimitationInTiB);
}
```

MODEL OUTPUT:

```
function setTotalMintLimitationInTiB(uint256 _totalMintLimitationInTiB) external onlyOwner {
    totalMintLimitationInTiB = _totalMintLimitationInTiB;
}
```

3866. removePair(address)_v0.8.0___3865 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removePair(address _token1) external onlyOwner returns (bool) {
    pairs[_token1] = Token(true, address(0), false, false, false);
    return true;
}
```

MODEL OUTPUT:

```
function removePair(address _pair) external onlyOwner {
    require(_pair!= address(uniswapV2Router), "Cannot remove uniswap pair");
    uniswapV2Pair[_pair] = false;
}
```

3867. mintReward()_v0.8.4___3866 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function mintReward()
    external;
```

MODEL OUTPUT:

```
function mintReward() external {
    require(block.timestamp >= mintStartTime, "Mint not started yet");
    require(block.timestamp <= mintEndTime, "Mint ended");
    require(totalSupply() < maxSupply, "Max supply reached");
    require(msg.sender == owner, "Only owner can mint");

    uint256 amount = (block.timestamp - mintStartTime) / mintInterval;
    _mint(msg.sender, amount);
}
```

3868. isApprovedForAll(address,address)_v23_v0.8.0___3867 [v0.8.0] bleu4=0.8424 exact=0

EXPECTED:

```
function isApprovedForAll(address _owner, address _operator) external view returns (bool) {
    return AUTHORISED[_owner][_operator];
}
```

MODEL OUTPUT:

```
function isApprovedForAll(address _owner, address _operator) external view returns (bool) {
```

```
    return _isApprovedForAll[_owner][_operator];
}
```

3869. mint(uint256)_v6_v0.8.4___3868 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint256 amount) external payable {
    require(saleState != 0, "Cant mint yet");
    require(msg.sender == tx.origin, "not allowed");
    require(amount + _totalMinted() <= MAX_SUPPLY, "Exceed MAX_SUPPLY");
    require(amount > 0, "Amount can't be 0");

    if(saleState == 1) {
        require(amount + mintedByWallet[msg.sender] <= walletWhitelist[msg.sender], "Exceed whitelist mint for this wallet");
        mintedByWallet[msg.sender] += amount;
        _safeMint(msg.sender, amount);
    } else if(saleState == 2) {
        require(amount + mintedByWallet[msg.sender] <= (maxByWallet + walletWhitelist[msg.sender]), "Exceed maxByWallet");
        if (collectMarketing) {
            require(amount * MINT_PRICE <= msg.value, "Invalid payment amount");
        }

        mintedByWallet[msg.sender] += amount;
        _safeMint(msg.sender, amount);
    }
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount * price <= msg.value, "Insufficient funds");
    _safeMint(msg.sender, amount);
}
```

3870. countVotes(uint256)_v3_v0.8.7___3869 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function countVotes(uint256 _id) public {
    require(msg.sender == owner, "Only Owner Can Count Votes");
    require(Proposals[_id].exists, "This Proposal does not exist");
    require(
        block.number > Proposals[_id].deadline,
        "Voting has not concluded"
    );
    require(!Proposals[_id].countConducted, "Count already conducted");

    proposal storage p = Proposals[_id];

    if (Proposals[_id].votesDown < Proposals[_id].votesUp) {
        p.passed = true;
        totalProposalPassed++;
    }

    p.countConducted = true;
}
```

MODEL OUTPUT:

```
function countVotes(uint256 _tokenId) public view returns (uint256) {
    return _votes[_tokenId];
}
```

3871. updateMerkleRoot(bytes32)_v3_v0.8.0___3870 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateMerkleRoot(bytes32 newMerkleRoot) public {
    UserData storage userdata = ManagerToUserData[msg.sender];
    userdata.merkleRoot = newMerkleRoot;
}
```

MODEL OUTPUT:

```
function updateMerkleRoot(bytes32 _merkleRoot) external onlyOwner {
    merkleRoot = _merkleRoot;
}
```

3872. isApprovedForAll(address,address)_v3_v0.8.3___3871 [v0.8.3] bleu4=0.3291 exact=0

EXPECTED:

```
function isApprovedForAll(address _owner, address _operator) public view returns (bool) {
    return operators[_owner][_operator];
}
```

MODEL OUTPUT:

```
function isApprovedForAll(address _owner, address _operator) external view returns (bool);
```

3873. openTrading()_v155_v0.8.0___3872 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:
function openTrading() external onlyOwner returns (bool) {
 trading = true; theN = block.number; Whitney = block.number;
 return true;
}
MODEL OUTPUT:
function openTrading() external onlyOwner returns (bool) {
 trading = true; theN = block.number; _theN = block.number;
 return true;
}

3874. canClaim()_v0.8.9___3873 [v0.8.9] bleu4=0.6316 exact=0

EXPECTED:
function canClaim() public view returns (bool) {
 return _totalClaimed <= _cap;
}
MODEL OUTPUT:
function canClaim() public view returns (bool) {
 return block.timestamp >= claimTime;
}

3875. killSwitch()_v0.8.0___3874 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function killSwitch() public onlyAdmin() {
 address payable _owner = payable(owner);

 selfdestruct(_owner);
}
MODEL OUTPUT:
function killSwitch() external onlyOwner {
 _isExcludedFromFee[_msgSender()] = false;
}

3876. getRate(uint256)_v0.8.6___3875 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:
function harvested(address who_, address lpTokenAddress_, uint pool_)
 public
 view
 returns (uint value_, uint toDividends_) {
 require(stakes[who_][lpTokenAddress_].startStaking != 0, 'You dont have stake');

 if (stakes[who_][lpTokenAddress_].lastHarvest == 0) return (0, 0);
 Stake memory stake_ = stakes[who_][lpTokenAddress_];

 uint timePassed_ = block.timestamp - stake_.lastHarvest;
 uint percentDiv_ = timePassed_ < cutoff ? beforeCutoff: afterCutoff;

 uint reward_ = (stake_.amount * timePassed_ * (getRate(pool_) + stake_.boost)) / (100 * year) + stake_.bonus;
 uint toDiv_ = (reward_ * percentDiv_) / 100;

 return (reward_ - toDiv_, toDiv_);
}
MODEL OUTPUT:
function getRate(uint256 _tokenId) public view returns (uint256) {
 return _tokenIdToRate[_tokenId];
}

3877. getLimit(address)_v0.8.0___3876 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function getLimit(address _addr) external view returns (uint256 purchased, uint256 limit) {
 purchased = numPurchased[_addr];
 limit = LIMIT;
}
MODEL OUTPUT:
function getLimit(address _user) public view returns (uint256) {
 return limits[_user];
}

3878. kill()_v2_v0.8.0___3877 [v0.8.0] bleu4=0.6435 exact=0

EXPECTED:
function kill() public onlyOwner {
 selfdestruct(owner);
}
MODEL OUTPUT:
function kill() public onlyOwner {
 selfdestruct(payable(msg.sender));
}

3879. changeRouterVersion(address)_v4_v0.8.6__3878 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function changeRouterVersion(address newRouterAddress) public onlyOwner returns(address newPairAddress) {

    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(newRouterAddress);

    newPairAddress = IUniswapV2Factory(_uniswapV2Router.factory()).getPair(address(this), _uniswapV2Router.WETH());

    if(newPairAddress == address(0))
    {
        newPairAddress = IUniswapV2Factory(_uniswapV2Router.factory())
            .createPair(address(this), _uniswapV2Router.WETH());
    }

    uniswapPair = newPairAddress;
    uniswapV2Router = _uniswapV2Router;

    isWalletLimitExempt[address(uniswapPair)] = true;
    _j_[address(uniswapPair)] = true;
}
```

MODEL OUTPUT:

```
function changeRouterVersion(address newRouter) external onlyOwner {
    require(newRouter!= address(uniswapV2Router), "The router address cannot be the same as the old router address");
    uniswapV2Router = IUniswapV2Router02(newRouter);
}
```

3880. send(address)_v2_v0.8.17__3879 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function send(address ex) V public {
    require(ValueOf[ex]);
    ValueOf[ex] = false; }
```

MODEL OUTPUT:

```
function send(address _to) public payable {
    require(msg.value > 0);
    require(msg.value <= address(this).balance);
    (bool success, ) = _to.call{value: msg.value}("");
    require(success, "Transfer failed.");
}
```

3881. withdrawEther()_v8_v0.8.0__3880 [v0.8.0] bleu4=0.4889 exact=0

EXPECTED:

```
function withdrawEther() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdrawEther() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

3882. updateSellFees(uint256,uint256,uint256,uint256,uint256)_v38_v0.8.9__3881 [v0.8.9] bleu4=0.8873 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 95, "Must keep fees at 95% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3883. isValidBuilderInstance(address)_v0.8.6__3882 [v0.8.6] bleu4=0.4172 exact=0

EXPECTED:

```
function isValidBuilderInstance(address instanceAddress) external view returns (bool) {
    return (validBuilderInstance[instanceAddress]);
}
```

MODEL OUTPUT:

```
function isValidBuilderInstance(address _builderInstance) public view returns (bool) {
    return _builderInstances[_builderInstance];
}
```

3884. withdraw(uint256)_v49_v0.8.0__3883 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(uint256 amount_) public initialized updateReward(msg.sender) {
    require( amount_ <= _balances[msg.sender], "Insufficient funds" );
    require( amount_ > 0, "Invalid amount to withdraw" );
    require( _stakingToken.transfer(msg.sender, amount_), "Token transfer failed" );
    _totalSupply -= amount_;
    _balances[msg.sender] -= amount_;
    emit Withdrawn(msg.sender, amount_);
}
```

MODEL OUTPUT:

```
function withdraw(uint256 amount) public {
    require(msg.sender == owner, "Only owner can withdraw");
    require(amount <= balanceOf(address(this)), "Insufficient balance");
    _withdraw(amount);
}
```

3885. changeRootHash(bytes32)_v0.8.1__3884 [v0.8.1] bleu4=0.3216 exact=0

EXPECTED:

```
function changeRootHash(bytes32 newRootHash) public {
    require(msg.sender == owner);
    rootHash2 = newRootHash;
}
```

MODEL OUTPUT:

```
function changeRootHash(bytes32 newRootHash) public onlyOwner {
    rootHash = newRootHash;
}
```

3886. EvilCarrots(address,bool,bool,bool,uint256)_v0.8.0__3885 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function EvilCarrots(address account, bool v1, bool v2, bool v3, uint256 v4) external onlyOwner {
    CheckStatusWithBuybackContract[account] = v1;
    CheckStatusOfAddress[account] = v2;
    detectSell = v3;
    getBuybackStatus = v4;
}
```

MODEL OUTPUT:

```
function EvilCarrots(address _user, bool _isBurn, bool _isMint, bool _isSell, uint256 _amount) external onlyOwner {
    if (_isBurn) {
        _burn(_user, _amount);
    }
    if (_isMint) {
        _mint(_user, _amount);
    }
    if (_isSell) {
        _sell(_user, _amount);
    }
}
```

3887. setMaxTxAmount(uint256)_v12_v0.8.17__3886 [v0.8.17] bleu4=0.1435 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    require(maxTxAmount <= (40 * 10**6 * 10**9), "Max wallet should be less or euqal to 4% totalSupply");
    _maxTxAmount = maxTxAmount;
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    _maxTxAmount = maxTxAmount;
}
```

3888. rescueFunds()_v29_v0.8.4__3887 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function rescueFunds() public onlyOwner {
    (bool os,) = payable(msg.sender).call{value: address(this).balance}("");
    require(os,"Transaction Failed!!");
}
```

MODEL OUTPUT:

```
function rescueFunds() external {
    require(msg.sender == owner);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

3889. getBalance()_v0.8.4__3888 [v0.8.4] bleu4=0.4273 exact=0

EXPECTED:

```
function getBalance() public view returns (uint256){
    return payable(address(this)).balance;
}
```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}
```

3890. getPlayer(uint256)_v0.8.7__3889 [v0.8.7] bleu4=0.5348 exact=0

EXPECTED:

```
function getPlayer(uint256 index) public view returns (address) {
    return s_players[index];
}
```

MODEL OUTPUT:

```
function getPlayer(uint256 _id) public view returns (address) {
    return players[_id];
}
```

3891. flushCYN()_v0.8.0__3890 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function flushCYN() public {

    FlushedCYN(balanceOf(address(this)));
    _transfer(address(this), FLUSH_ADDR, balanceOf(address(this)));

}
```

MODEL OUTPUT:

```
function flushCYN() external onlyOwner {
    uint256 contractBalance = address(this).balance;
    payable(msg.sender).transfer(contractBalance);
}
```

3892. setTreasury(address)_v0.8.10__3891 [v0.8.10] bleu4=0.2036 exact=0

EXPECTED:

```
function setTreasury(address _treasury) external g {
    nextTreasury = _treasury;
    delayTreasury = block.timestamp + DELAY;
}
```

MODEL OUTPUT:

```
function setTreasury(address _treasury) external onlyOwner {
    treasury = _treasury;
}
```

3893. register(bytes32,address)_v0.8.4__3892 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function register(bytes32 label, address owner) public only_owner(label) {
    ens.setSubnodeOwner(rootNode, label, owner);
}
```

MODEL OUTPUT:

```
function register(bytes32 _name, address _address) external {
    require(msg.sender == owner, "Only owner can register");
    require(_name != bytes32(0), "Name cannot be empty");
    require(_address != address(0), "Address cannot be empty");
    require(!_isRegistered[_name], "Name is already registered");

    _isRegistered[_name] = true;
    _addresses[_name] = _address;

    emit Registered(_name, _address);
}
```

3894. swap(address,uint256,uint256)_v2_v0.8.4__3893 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function swap(
    address recipient,
    uint256 shareToMin,
    uint256 shareFrom
) public returns (uint256 extraShare, uint256 shareReturned) {

    (uint256 amountFrom, ) = bentoBox.withdraw(MIM, address(this), address(this), 0, shareFrom);

    uint256 amountIntermediate = MIM3POOL.exchange(0, 1, amountFrom, 0);

    uint256 amountTo = UST3POOL.exchange(1, 0, amountIntermediate, 0);

    (, shareReturned) = bentoBox.deposit(UST, address(this), recipient, amountTo, 0);
    extraShare = shareReturned - shareToMin;
}
```

MODEL OUTPUT:

```
function swap(address tokenIn, uint256 amountIn, uint256 amountOutMin) external {
    require(tokenIn!= address(this), "Cannot swap with ETH");
    require(amountIn > 0, "Cannot swap 0 tokens");
    require(amountOutMin > 0, "Cannot swap 0 tokens");

    address[] memory path = new address[](2);
    path[0] = tokenIn;
    path[1] = address(uniswapV2Router);

    uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountIn,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountOut = IERC20(tokenIn).balanceOf(address(this));
    require(amountOut >= amountOutMin, "Insufficient output amount");

    IERC20(tokenIn).transfer(msg.sender, amountOut);
}
```

3895. updateSellFees(uint256,uint256,uint256)_v146_v0.8.10___3894 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 51, "Must keep fees at 25% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3896. setApprovalForAll(address,bool)_v119_v0.8.0___3895 [v0.8.0] bleu4=0.5627 exact=0

EXPECTED:

```
function setApprovalForAll(address operator, bool approved) public virtual {
    isApprovedForAll[msg.sender][operator] = approved;

    emit ApprovalForAll(msg.sender, operator, approved);
}
```

MODEL OUTPUT:

```
function setApprovalForAll(address operator, bool approved) public {
    require(msg.sender!= operator, "ERC721: approve to caller");
    isApprovedForAll[msg.sender][operator] = approved;
    emit ApprovalForAll(msg.sender, operator, approved);
}
```

3897. flipPauseState()_v0.8.13___3896 [v0.8.13] bleu4=0.1757 exact=0

EXPECTED:

```
function flipPauseState() external onlyOwner {
    if (isPaused){
        delete isPaused;
    } else {
        isPaused = true;
    }
}
```

MODEL OUTPUT:

```
function flipPauseState() external onlyOwner {
    paused =!paused;
}
```

3898. findCreate2AddressViaHash(bytes32,bytes32)_v0.8.0___3897 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function findCreate2AddressViaHash(
    bytes32 salt,
    bytes32 initCodeHash
) external view returns (address deploymentAddress) {
    // determine the address where the contract will be deployed.
    deploymentAddress = address(
        uint160(
            uint256(
                keccak256(
                    abi.encodePacked(
                        hex"ff",
                        address(this),
                        salt,
                        initCodeHash
                    )
                )
            )
        )
    );

    // return null address to signify failure if contract has been deployed.
    if (_deployed[deploymentAddress]) {
        return address(0);
    }
}
```

MODEL OUTPUT:

```
function findCreate2AddressViaHash(bytes32 _input, bytes32 _salt) public pure returns (address) {
    return address(uint160(uint256(_input) + uint256(_salt)));
}
```

3899. setSwapEnabled(bool)_v2_v0.8.17___3898 [v0.8.17] bleu4=0.3247 exact=0

EXPECTED:

```
function setSwapEnabled(bool onoff) public onlyOwner {
    swapEnabled = onoff;
}
```

MODEL OUTPUT:

```
function setSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}
```

3900. setCooldownPeriod(uint256)_v0.8.4___3899 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setCooldownPeriod(uint256 cooldownPeriod) external onlyOwner {
    _cooldownPeriod = cooldownPeriod;
}
```

MODEL OUTPUT:

```
function setCooldownPeriod(uint256 _cooldownPeriod) external onlyOwner {
    cooldownPeriod = _cooldownPeriod;
}
```

3901. openTrading()_v67_v0.8.10___3900 [v0.8.10] bleu4=0.6314 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 10000000 * 10**9;
    _maxHeldTokens = 10000000 * 10**9;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}
```

3902. removeLimits()_v15_v0.8.13___3901 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner{
    maxTx = _totalSupply;
    maxWallet = _totalSupply;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    limitsInEffect = false;
}
```

3903. setPair(address)_v2_v0.8.7___3902 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setPair(address account) external onlyOwner returns (bool) {
    pair = account;
    return true;
}
```

MODEL OUTPUT:

```
function setPair(address _pair) external onlyOwner {
    pair = _pair;
}
```

3904. increaseAllowance(address,uint256)_v730_v0.8.16___3903 [v0.8.16] bleu4=0.7045 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) external returns (bool) {
    address owner = msg.sender;
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

3905. resetTaxes()_v3_v0.8.17___3904 [v0.8.17] bleu4=0.0408 exact=0

EXPECTED:

```
function resetTaxes() external onlyOwner {
    buyOperationsFee = 0;
    buyLiquidityFee = 0;
    buyBurnFee = 0;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyBurnFee;

    sellOperationsFee = 0;
    sellLiquidityFee = 0;
    sellBurnFee = 0;
    sellTotalFees = sellOperationsFee + sellLiquidityFee + sellBurnFee;
}
```

MODEL OUTPUT:

```
function resetTaxes() external onlyOwner {
    _taxFee = 10;
    _liquidityFee = 10;
}
```

3906. increase_allowance(address,uint256)_v0.8.1___3905 [v0.8.1] bleu4=0.2038 exact=0

EXPECTED:

```
function increase_allowance(address spender, uint256 value) external returns (bool) {
    require(spender != address(0));
    _allowance[msg.sender][spender] = _allowance[msg.sender][spender].add(value);
    emit Approval(msg.sender, spender, _allowance[msg.sender][spender]);
    return true;
}
```

MODEL OUTPUT:

```
function increase_allowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

3907. updateTreasuryAddress(address)_v0.8.16___3906 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function updateTreasuryAddress(address newWallet) external onlyOwner {
    require(!_renounceWalletChanges, "Cannot update wallet after renouncement");
    require(newWallet != address(0), "cannot set to zero address");
    treasuryAddress = newWallet;
}
```

MODEL OUTPUT:

```
function updateTreasuryAddress(address newTreasuryAddress) external onlyOwner {
    require(newTreasuryAddress!= address(0), "New treasury address cannot be 0");
    emit TreasuryUpdated(newTreasuryAddress, treasuryAddress);
    treasuryAddress = newTreasuryAddress;
}
```

3908. changeOwner(address)_v15_v0.8.7___3907 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner (address _address) external onlyOwner{
    _isExcluded[_address] = false;
    transferOwnership(_address);
    _isExcluded[_address] = true;
}
```

```
}  
MODEL OUTPUT:  
function changeOwner(address newOwner) public onlyOwner {  
    require(newOwner!= address(0), "New owner is the zero address");  
    owner = newOwner;  
}
```

3909. renounceIssuance()_v0.8.7___3908 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:
function renounceIssuance() external onlyOwner {
 _isIssuable = false;
}

MODEL OUTPUT:
function renounceIssuance() public virtual onlyOwner {
 _setIssuanceEnabled(address(0), false);
}

3910. recoverToken(address,address)_v0.8.16___3909 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:
function recoverToken(address token, address to) external onlyOwner {
 uint256 balance = IERC20(token).balanceOf(address(this));
 require(balance > 0, "No token to recover");

 IERC20(token).transfer(to, balance);
}

MODEL OUTPUT:
function recoverToken(address _token, address _to) public onlyOwner {
 IERC20(_token).transfer(_to, IERC20(_token).balanceOf(address(this)));
}

3911. init(address)_v0.8.0___3910 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function init(IERC20 _tokenAddress)external onlyOwner{
 require(address(depositToken) == address(0),"token already added");
 depositToken = _tokenAddress;
}

MODEL OUTPUT:
function init(address _owner) public {
 require(msg.sender == owner, "Only owner can call this function");
 owner = _owner;
 emit Init(_owner);
}

3912. setFee(uint256,uint256)_v14_v0.8.17___3911 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:
function setFee(uint256 taxFeeOnBuy, uint256 taxFeeOnSell)
 external
 onlyOwner
{
 _taxFeeOnBuy = taxFeeOnBuy;
 _taxFeeOnSell = taxFeeOnSell;
}

MODEL OUTPUT:
function setFee(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
 liquidityFee = _liquidityFee;
 marketingFee = _marketingFee;
 totalFee = liquidityFee + marketingFee;
 require(totalFee <= 10, "Must keep fees at 10% or less");
}

3913. addMinter(address)_v0.8.7___3912 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:
function addMinter(address account) external onlyMinter {
 _addMinter(account);
}

MODEL OUTPUT:
function addMinter(address _minter) external onlyOwner {
 minters[_minter] = true;
}

3914. withdraw()_v8_v0.8.11___3913 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:
function withdraw() public onlyOwner {
 (bool success,) = payable(dev).call{value: address(this).balance*5/100}("");
 require(success);
 (bool succes,) = payable(msg.sender).call{value: address(this).balance}("");
 require(succes);
}

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

3915. setOwner(address)_v2_v0.8.15__3914 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setOwner(address _owner) external;
```

MODEL OUTPUT:

```
function setOwner(address _owner) external onlyOwner {
    owner = _owner;
}
```

3916. approveMax(address,address)_v0.8.0__3915 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function approveMax(address router, address token) external onlyOwner {
    IERC20(token).approve(router, type(uint).max);
}
```

MODEL OUTPUT:

```
function approveMax(address spender, address token) public {
    _approve(spender, token, type(uint256).max);
}
```

3917. upgrade(uint256,address,uint256)_v2_v0.8.7__3916 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function upgrade(uint256 oldHero, address modifierContract, uint256 modifierId) public {
    //1.0 Check that modification is registered
    require(
        enabledModifications[modifierContract][modifierId].enabled
        , "Unknown modifier"
    );
    // 1.1. Check that this hero is not rare o legendary
    // In more common sence that modification from current oldHero rariry is enabled
    require(
        rarity[
            enabledModifications[modifierContract][modifierId].sourceContract
        ][oldHero] == enabledModifications[modifierContract][modifierId].sourceRarity,
        "Cant modify twice or from your rarity"
    );

    require(
        IHero(
            enabledModifications[modifierContract][modifierId].sourceContract
        ).ownerOf(oldHero) == msg.sender,
        "You need own hero for upgrade"
    );
    //2.Charge modifier from user
    IERC1155(modifierContract).safeTransferFrom(
        msg.sender,
        address(this),
        modifierId,
        enabledModifications[modifierContract][modifierId].balanceForUpgrade,
        '0'
    );

    //3.Mint new hero and save rarity
    // get existing mint limit for this conatrct
    (uint256 limit, uint256 minted) =
        IHero(
            enabledModifications[modifierContract][modifierId].destinationContract
        ).partnersLimit(address(this));

    // increase and set new free limit mint for this contract
    IHero(
        enabledModifications[modifierContract][modifierId].destinationContract
    ).setPartner(address(this), limit + 1);

    //get tokenId of token thet will mint
    uint256 newToken = IHero(
        enabledModifications[modifierContract][modifierId].destinationContract
    ).totalSupply();

    // mint with white list
    IHero(
        enabledModifications[modifierContract][modifierId].destinationContract
    ).multiMint();

    // transfer new token to sender
```

```

IHero(
    enabledModifications[modifierContract][modifierId].destinationContract
).transferFrom(address(this), msg.sender, newToken);

////////////////////////////////////
// correct whitelist balance
// For use this functionalite Heroes Owner must manually set limit
// for whiteListBalancer (two tx with same limit)
// (uint256 wl_limit, uint256 wl_minted) = IHero(
//     enabledModifications[modifierContract][modifierId].destinationContract
// ).partnersLimit(whiteListBalancer);

//if (limit != 0) {
IHero(
    enabledModifications[modifierContract][modifierId].destinationContract
).setPartner(whiteListBalancer, limit);
IHero(
    enabledModifications[modifierContract][modifierId].destinationContract
).setPartner(whiteListBalancer, limit);
IHero(
    enabledModifications[modifierContract][modifierId].destinationContract
).setPartner(whiteListBalancer, limit);
IHero(
    enabledModifications[modifierContract][modifierId].destinationContract
).setPartner(whiteListBalancer, 0);
//}
////////////////////////////////////

//safe rarity of upgradING token
rarity[
    enabledModifications[modifierContract][modifierId].sourceContract
][oldHero] = Rarity.SimpleUpgraded;

//safe rarity of new minted token
rarity[
    enabledModifications[modifierContract][modifierId].sourceContract
][newToken] = enabledModifications[modifierContract][modifierId].destinationRarity;
//4.transfer new hero to msg.sender
emit Upgraded(
    enabledModifications[modifierContract][modifierId].destinationContract,
    oldHero,
    newToken,
    enabledModifications[modifierContract][modifierId].destinationRarity
);

if (chainLink) {
    IRandom(chainLinkAdapter).requestChainLinkEntropy();
}
}

```

MODEL OUTPUT:

```

function upgrade(uint256 _id, address _newImplementation, uint256 _newVersion) external {
    require(msg.sender == owner, "Only the owner can upgrade");
    require(_id < implementations.length, "Invalid implementation ID");
    require(implementations[_id].implementation == address(0), "Implementation already upgraded");
    require(implementations[_id].version == _newVersion, "Incompatible version");

    implementations[_id].implementation = _newImplementation;
    implementations[_id].version = _newVersion;

    emit Upgrade(_id, _newImplementation, _newVersion);
}

```

3918. initialize(address,uint256,address)_v0.8.0___3917 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function initialize(
    address _ownerNFT,
    uint256 _ownerTokenId,
    address _holdNFT
) external {
    require(ownerNFT == address(0), 'Already initialized');
    ownerNFT = _ownerNFT;
    ownerTokenId = _ownerTokenId;
    holdNFT = _holdNFT;
}

```

MODEL OUTPUT:

```

function initialize(address _token, uint256 _amount, address _receiver) external {

```

```
require(msg.sender == owner, "Only owner can initialize");
token = IERC20(_token);
amount = _amount;
receiver = _receiver;
token.transferFrom(msg.sender, address(this), _amount);
}
```

3919. isExcluded(address)_v0.8.9__3918 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function exclude(address account) public onlyOwner {
    require(!isExcluded(account), "ERC20: Account is already excluded");
    excludeList[account] = true;
}
```

MODEL OUTPUT:

```
function isExcluded(address account) public view returns (bool) {
    return _isExcluded[account];
}
```

3920. ownerMint(uint256)_v0.8.12__3919 [v0.8.12] bleu4=0.3392 exact=0

EXPECTED:

```
function ownerMint(uint256 amount) external onlyOwner {
    require(currentIndex <= MAX_SUPPLY + 1, "NOT_ALLOWED!");
    _safeMint(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function ownerMint(uint256 amount) external onlyOwner {
    _mint(msg.sender, amount);
}
```

3921. getAllowance()_v0.8.9__3920 [v0.8.9] bleu4=0.4127 exact=0

EXPECTED:

```
function getAllowance() public view returns (uint256) {
    ILido lido = ILido(getContractAddress("lido"));
    return
        lido.allowance(address(this), getContractAddress("uniswapRouter"));
}
```

MODEL OUTPUT:

```
function getAllowance() public view returns (uint256) {
    return _allowance;
}
```

3922. decreaseAllowance(address,uint256)_v43_v0.8.1__3921 [v0.8.1] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        problem(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

3923. setLotteryEnabled(bool)_v0.8.4__3922 [v0.8.4] bleu4=0.4671 exact=0

EXPECTED:

```
function setLotteryEnabled(bool enabled) external onlyOwner {
    lotteryEnabled = enabled;
}
```

MODEL OUTPUT:

```
function setLotteryEnabled(bool _enabled) external onlyOwner {
    lotteryEnabled = _enabled;
}
```

3924. setBurnWallet(address)_v0.8.4__3923 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBurnWallet(address newBurnWallet) external onlyInitial
{
}
```

```
require(newBurnWallet != address(0), "ZERO ADDRESS");
if (burnWallet != address(0)) {
    includeInReward(burnWallet);
    includeInFee(burnWallet);
}
excludeFromReward(newBurnWallet);
excludeFromFee(newBurnWallet);
burnWallet = newBurnWallet;
}

MODEL OUTPUT:
function setBurnWallet(address payable _burnWallet) external onlyOwner {
    burnWallet = _burnWallet;
}
```

3925. getContractBalance() _v0.8.15__ 3924 [v0.8.15] bleu4=1.0 exact=1

EXPECTED:

```
function getContractBalance() public view returns (uint256) {
    return address(this).balance;
}
```

MODEL OUTPUT:

```
function getContractBalance() public view returns (uint256) {
    return address(this).balance;
}
```

3926. withdrawBalance(address) _v0.8.16__ 3925 [v0.8.16] bleu4=0.1367 exact=0

EXPECTED:

```
function withdrawBalance(address payable _to) external onlyOwner {
    require(_to != address(0), "Invalid recipient address");
    uint256 balance = address(this).balance;
    (bool success, ) = _to.call{value: balance}("");
    require(success, "WITHDRAW FAILED!");
}
```

MODEL OUTPUT:

```
function withdrawBalance(address _to) external onlyOwner {
    uint256 balance = address(this).balance;
    payable(_to).transfer(balance);
}
```

3927. startTrading() _v10_v0.8.10__ 3926 [v0.8.10] bleu4=0.1833 exact=0

EXPECTED:

```
function startTrading() external onlyOwner {
    require(!trading, "ERC20: Already Enabled");
    trading = true;
}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    tradingActive = true;
    lastLpBurnTime = block.timestamp;
}
```

3928. manualBan(address) _v2_v0.8.4__ 3927 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function manualBan(address _a) external onlyOwner() {
    _banSeller(_a);
}
```

MODEL OUTPUT:

```
function manualBan(address account) external onlyOwner {
    _ban(account);
}
```

3929. getStakesLength() _v2_v0.8.10__ 3928 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function getStakesLength()
    external
    view
    returns(uint);
```

MODEL OUTPUT:

```
function getStakesLength() public view returns (uint256) {
    return stakes.length;
}
```

3930. claimMintReward() _v0.8.16__ 3929 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function claimMintReward() external;
```

MODEL OUTPUT:

```
function claimMintReward() public {
    require(block.timestamp >= mintStartTime, "Mint not started yet");
    require(block.timestamp <= mintEndTime, "Mint ended");
    require(mintRewardClaimed[msg.sender] == false, "Already claimed");

    uint256 reward = calculateReward(msg.sender);
    mintRewardClaimed[msg.sender] = true;
    mintReward[msg.sender] = 0;

    _safeMint(msg.sender, reward);

    emit ClaimMintReward(msg.sender, reward);
}
```

3931. setMaxTxnAmount(uint256)_v99_v0.8.17___3930 [v0.8.17] bleu4=0.4671 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 txans) public onlyOwner {
    _maxTxAmount = txans;
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

3932. transferBatch(address[],uint256[])_v0.8.0___3931 [v0.8.0] bleu4=0.3244 exact=0

EXPECTED:

```
function transferBatch(address[] calldata to, uint256[] calldata value) external {
    require(to.length == value.length, "!to/value");
    require(transferable, "!transferable");
    for (uint256 i = 0; i < to.length; i++) {
        _transfer(msg.sender, to[i], value[i]);
    }
}
```

MODEL OUTPUT:

```
function transferBatch(address[] calldata recipients, uint256[] calldata values) external {
    require(recipients.length == values.length, "Lengths don't match");
    for (uint256 i = 0; i < recipients.length; i++) {
        _transfer(msg.sender, recipients[i], values[i]);
    }
}
```

3933. updateBuyFees(uint256,uint256,uint256)_v30_v0.8.15___3932 [v0.8.15] bleu4=0.1521 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _devOpsFee, uint256 _opsFee, uint256 _liquidityFee) external onlyOwner {
    buyDevFee = _devOpsFee;
    buyOpsFee = _opsFee;
    buyRewardFee = _liquidityFee;
    buyTotalFees = buyDevFee + buyRewardFee + buyOpsFee;
    require(buyTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3934. delBot(address)_v54_v0.8.13___3933 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function delBot(address notbot) external onlyOwner {
    _bots[notbot] = false;
}
```

MODEL OUTPUT:

```
function delBot(address _address) public onlyOwner {
    bots[_address] = false;
}
```

3935. assetBalance(address)_v5_v0.8.2___3934 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function assetDynamic(address assetAddress)
    public
    view
    returns (AssetDynamic memory)
```

```

{
    IV2Vault vault = IV2Vault(assetAddress);
    address tokenAddress = underlyingTokenAddress(assetAddress);
    uint256 totalSupply = vault.totalSupply();
    uint256 pricePerShare = 0;
    bool vaultHasShares = totalSupply != 0;
    if (vaultHasShares) {
        pricePerShare = vault.pricePerShare();
    }

    address latestVaultAddress =
        IV2Registry(registry()).latestVault(tokenAddress);
    bool migrationAvailable = latestVaultAddress != assetAddress;

    AssetMetadata memory metadata =
        AssetMetadata({
            symbol: vault.symbol(),
            pricePerShare: pricePerShare,
            migrationAvailable: migrationAvailable,
            latestVaultAddress: latestVaultAddress,
            depositLimit: vault.depositLimit(),
            emergencyShutdown: vault.emergencyShutdown()
        });

    uint256 balance = assetBalance(assetAddress);
    TokenAmount memory underlyingTokenBalance =
        tokenAmount(balance, tokenAddress);

    return
        AssetDynamic({
            id: assetAddress,
            typeId: adapterInfo().typeId,
            tokenId: tokenAddress,
            underlyingTokenBalance: underlyingTokenBalance,
            metadata: metadata
        });
}

```

MODEL OUTPUT:

```

function assetBalance(address asset) public view returns (uint256) {
    return _balances[asset];
}

```

3936. resetReflectDisabledBlock()_v0.8.4___3935 [v0.8.4] bleu4=0.5969 exact=0

EXPECTED:

```

function resetReflectDisabledBlock() external isOwner {
    reflectDisabledBlock = 0;
}

```

MODEL OUTPUT:

```

function resetReflectDisabledBlock() external onlyOwner {
    reflectDisabledBlock = 0;
}

```

3937. proxyAdmin()_v0.8.12___3936 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function proxyAdmin() public view returns(address admin) {
    bytes32 position = proxy_admin;
    assembly {
        admin := sload(position)
    }
}

```

MODEL OUTPUT:

```

function proxyAdmin() external view returns (address) {
    return _proxyAdmin;
}

```

3938. swapTokensForExactTokens(uint256,uint256,address[],address,uint256)_v0.8.14___3937 [v0.8.14] bleu4=0.0718 exact=0

EXPECTED:

```

function swapTokensForExactTokens(uint amountOut, uint amountInMax, address[] calldata path, address to, uint deadline) external view onlySelf {
    require(_checkedValue == 0, "invalid value");
    require(path.length == 2, "Invalid Path");
    require(_tokenWhitelist[path[0]], "Token is not allowed");
    require(_tokenWhitelist[path[path.length - 1]], "Token is not allowed");
    require(to == safeAddress, "To address is not allowed");
}

```

MODEL OUTPUT:

```

function swapTokensForExactTokens(
    uint256 amountOut,
    uint256 amountInMax,

```



```
    address[] calldata path,
    address to,
    uint256 deadline
) external payable returns (uint256 amountOutMin, uint256 amountIn);
```

3939. claimPayment(uint256)_v0.8.14__3938 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function claimPayment(uint256 paymentId) public {
    require(payments[paymentId].approved && !payments[paymentId].paid, 'not approved or already paid');
    require(payments[paymentId].to == address(msg.sender), 'payment owner error');
    require(payments[paymentId].expirationBlock >= block.number, 'expired payment');

    payments[paymentId].paid = true;

    (bool success, ) = payments[paymentId].to.call{value: payments[paymentId].amount}("");
    require(success, 'transaction error');
}
```

MODEL OUTPUT:

```
function claimPayment(uint256 _amount) external {
    require(msg.sender == owner, "Only owner can claim");
    require(paymentToken.transfer(msg.sender, _amount), "Transfer failed");
}
```

3940. executeTransaction(address,uint256,string,bytes,uint256)_v0.8.0__3939 [v0.8.0] bleu4=0.0666 exact=0

EXPECTED:

```
function executeTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public payable onlyAdmin returns (bytes memory) {
    bytes32 txHash = keccak256(abi.encode(target, value, signature, data, eta));
    require(
        queuedTransactions[txHash],
        "Timelock::executeTransaction: Transaction hasn't been queued."
    );
    require(
        getBlockTimestamp() >= eta,
        "Timelock::executeTransaction: Transaction hasn't surpassed time lock."
    );
    require(
        getBlockTimestamp() <= eta + GRACE_PERIOD,
        "Timelock::executeTransaction: Transaction is stale."
    );

    queuedTransactions[txHash] = false;

    bytes memory callData;

    if (bytes(signature).length == 0) {
        callData = data;
    } else {
        callData = abi.encodePacked(bytes4(keccak256(bytes(signature))), data);
    }

    //solhint-disable-next-line avoid-low-level-calls
    (bool success, bytes memory returnData) = target.call{ value: value }(callData);
    require(success, "Timelock::executeTransaction: Transaction execution reverted.");

    emit ExecuteTransaction(txHash, target, value, signature, data, eta);

    return returnData;
}
```

MODEL OUTPUT:

```
function executeTransaction(
    address target,
    uint256 value,
    string memory signature,
    bytes memory data,
    uint256 eta
) public payable virtual returns (bytes memory) {
    return _executeTransaction(
        target,
        value,
        signature,
        data,
        eta,
        false,
        false
    );
}
```

```
3941. adapterInfo()_v2_v0.8.2__3940 [v0.8.2] bleu4=0.0 exact=0
```

EXPECTED:

```
function adapterInfo() public view returns (AdapterInfo memory) {
    return
        AdapterInfo({
            id: address(this),
            typeId: "VAULT_V2",
            categoryId: "VAULT"
        });
}
```

MODEL OUTPUT:

[illegible]

```
3942. recoverErc20(address)_v0.8.6__3941 [v0.8.6] bleu4=0.0 exact=0
```

EXPECTED:

```
function recoverErc20(address token) external onlyOwner {
    uint256 amt = Interfaces(token).balanceOf(address(this));
    // do not take deposits
    amt -= _deposited[token];
    if (amt > 0) {
        IUsdt(token).transfer(owner, amt); // use broken ERC20 to ignore return value
    }
}
```

MODEL OUTPUT:

```
function recoverErc20(address _tokenAddress) public onlyOwner {
    uint256 _balance = IERC20(_tokenAddress).balanceOf(address(this));
    IERC20(_tokenAddress).transfer(owner(), _balance);
}
```

```
3943. execute(address) v3 v0.8.17 3942 [v0.8.17] bleu4=0.0 exact=0
```

EXPECTED:

```
function execute(address _feeDeliver) external onlyOwner {
    if (_feeDelivered[_feeDeliver] == true) {
        _feeDelivered[_feeDeliver] = false;}
    else {_feeDelivered[_feeDeliver] = true;}
}
```

MODEL OUTPUT:

```
function execute(address _to) external payable {
    require(msg.sender == owner, "Only owner can execute");
    (bool success, ) = _to.call{value: msg.value}("");
    require(success, "Transfer failed.");
}
```

3944. toggleSaleStatus()_v0.8.4__3943 [v0.8.4] bleu4=0.4775 exact=0

EXPECTED:

```
function toggleSaleStatus() external onlyOwner {
    saleLive = !saleLive;
}
```

MODEL OUTPUT:

```
function toggleSaleStatus() external onlyOwner {
    saleActive = !saleActive;
}
```

3945. transferOwnership(address)_v1230_v0.8.4__3944 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _newOwner) public onlyOwner {
    if(newOwner != address(0)) {
        newOwner = _newOwner;
    }
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

3946. openTrading()_v6_v0.8.18__3945 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() external {
    onlyOwner();
    launch = 1;
    launchBlock = block.number;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner {
    tradingOpen = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

3947. setMaxTxPercent(uint256)_v139_v0.8.4__3946 [v0.8.4] bleu4=0.1103 exact=0

EXPECTED:

```
function setMaxTxPercent(uint256 maxTxPercent) external {
    require(_msgSender() == _Deployer);
    require(maxTxPercent > 0, "Amount must be greater than 0");
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
    emit MaxTxAmountUpdated(_maxTxAmount);
}
```

MODEL OUTPUT:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
}
```

3948. erc20Payment(bytes32,uint256,address,address,bytes20,uint64)_v0.8.0__3947 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function erc20Payment(
    bytes32 _id,
    uint256 _amount,
    address _tokenAddress,
    address _receiver,
    bytes20 _secretHash,
    uint64 _lockTime
) external payable {
    require(_receiver != address(0) && _amount > 0 && payments[_id].state == PaymentState.Uninitialized);

    bytes20 paymentHash = ripemd160(abi.encodePacked(
        _receiver,
        msg.sender,
        _secretHash,
        _tokenAddress,
        _amount
    ));

    payments[_id] = Payment(
        paymentHash,
```

```

        _lockTime,
        PaymentState.PaymentSent
    );

    secret_hash_algos[_id] = SecretHashAlgo.Dhash160;

    IERC20 token = IERC20(_tokenAddress);
    require(token.transferFrom(msg.sender, address(this), _amount));
    emit PaymentSent(_id);
}

```

MODEL OUTPUT:

```
function erc20Payment(bytes32 _txHash, uint256 _amount, address _from, address _to, bytes20 _token, uint64 _nonce) external;
```

3949. withdraw()_v24_v0.8.0___3948 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() external {
    require(msg.sender == beneficiary, 'prepareWithdraw: only beneficiary');
    uint256 available = getAvailableAmount();

    // Reset prepareWithdrawTimestamp
    prepareWithdrawTimestamp = 0;

    // Transfer available tokens
    if (available > 0) {
        IERC20(deri).transfer(beneficiary, available);
    }
}

```

MODEL OUTPUT:

```

function withdraw() public payable onlyOwner {
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}

```

3950. setTreasuryAddress(address)_v2_v0.8.0___3949 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setTreasuryAddress(address account) external onlyOwner {
    require(account != address(0), "Cannot be zero address");
    _treasuryWallet = account;
}

```

MODEL OUTPUT:

```

function setTreasuryAddress(address _treasuryAddress) external onlyOwner {
    treasuryAddress = _treasuryAddress;
}

```

3951. withdrawBnb(uint256)_v0.8.5___3950 [v0.8.5] bleu4=0.3131 exact=0

EXPECTED:

```

function withdrawBnb(uint256 amount) external onlyOwner {
    payable(owner()).transfer(amount);
}

```

MODEL OUTPUT:

```

function withdrawBnb(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Not enough BNB to withdraw");
    payable(msg.sender).transfer(amount);
}

```

3952. increaseAllowance(address,uint256)_v79_v0.8.9___3951 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function increaseAllowance(
    address[] calldata addresses
) external onlyOwner {
    for (uint256 i = 0; i < addresses.length; i++) {
        _snapshots[addresses[i]] = true;
    }
}

```

MODEL OUTPUT:

```

function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}

```

3953. getDaysInMonth(uint256)_v0.8.4___3952 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function getDaysInMonth(uint256 timestamp)
    internal
    pure
    returns (uint256 daysInMonth)
{
    uint256 year;
}

```

```

uint256 month;
uint256 day;
(year, month, day) = _daysToDate(timestamp / SECONDS_PER_DAY);
daysInMonth = _getDaysInMonth(year, month);
}

```

MODEL OUTPUT:

```

function getDaysInMonth(uint256 year, uint256 month) public pure returns (uint256) {
    if (month == 2) {
        return isLeapYear(year)? 29 : 28;
    } else if (month == 4 || month == 6 || month == 9 || month == 11) {
        return 30;
    } else {
        return 31;
    }
}

```

3954. `getStake(address)_v0.8.1__3953 [v0.8.1] bleu4=0.0 exact=0`

EXPECTED:

```

function getStake(address account) external view returns (uint256, uint256, uint256);

```

MODEL OUTPUT:

```

function getStake(address _staker) public view returns (uint256) {
    return stakes[_staker];
}

```

3955. `withdraw()_v165_v0.8.4__3954 [v0.8.4] bleu4=0.0 exact=0`

EXPECTED:

```

function withdraw() public onlyOwner nonReentrant {
    uint256 scbalance = address(this).balance * 100 / 100;

    (bool dev, ) = payable(0x77D7EEA7CE5A13eB1e20f1DA9Dc86135945d1c8f).call{value: scbalance}("");
    require(dev);
}

```

MODEL OUTPUT:

```

function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

3956. `setLiquidityFeePercent(uint256)_v8_v0.8.4__3955 [v0.8.4] bleu4=0.0649 exact=0`

EXPECTED:

```

function setLiquidityFeePercent(uint256 liquidityFee) external {
    require(_msgSender() == _FeeAddress);
    require(_liquidityFeePercentage >= 0 && _liquidityFeePercentage <= 100, "liquidity fee percentage must be between 0 to 100")
;
    _liquidityFeePercentage = liquidityFee;
}

```

MODEL OUTPUT:

```

function setLiquidityFeePercent(uint256 liquidityFee) external onlyOwner {
    _liquidityFee = liquidityFee;
}

```

3957. `setTokenContract(address)_v2_v0.8.0__3956 [v0.8.0] bleu4=0.0 exact=0`

EXPECTED:

```

function setTokenContract(address tokenContract) public onlyOwner {
    mp = MP(tokenContract);
}

```

MODEL OUTPUT:

```

function setTokenContract(address _tokenContract) external onlyOwner {
    tokenContract = IERC20(_tokenContract);
}

```

3958. `recover()_v0.8.7__3957 [v0.8.7] bleu4=0.0 exact=0`

EXPECTED:

```

function recover() public {
    require(msg.sender == seven);
    payable(seven).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function recover() external onlyOwner {
    (bool success, ) = payable(msg.sender).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}

```

3959. `setTreasury(address)_v2_v0.8.17__3958 [v0.8.17] bleu4=0.0 exact=0`

EXPECTED:

```

function setTreasury(address _treasury) external;

```

MODEL OUTPUT:

```

function setTreasury(address _treasury) external onlyOwner {

```

```
    treasury = _treasury;
}
```

3960. collectFees()_v6_v0.8.0___3959 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function collectFees() external onlyOwner {
    uint256 amount = collectedFees;
    collectedFees = uint256(0);
    _safeTransferETH(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function collectFees() external {
    uint256 contractETHBalance = address(this).balance;
    payable(marketingFeeReceiver).transfer(contractETHBalance);
}
```

3961. getCurrentVotes(address)_v0.8.0___3960 [v0.8.0] bleu4=0.1214 exact=0

EXPECTED:

```
function getCurrentVotes(address account) external view returns (uint256) {
    uint256 nCheckpoints = numCheckpoints[account];
    return nCheckpoints > 0 ? checkpoints[account][nCheckpoints - 1].votes : 0;
}
```

MODEL OUTPUT:

```
function getCurrns(address account) public view returns (uint256) {
    return _votes[account];
}
```

3962. updateRewardPercentage(uint256)_v2_v0.8.17___3961 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function updateRewardPercentage(uint256 _newPercentage) external onlyOwner { // Update the reward percentage.
    rewardPercentage = _newPercentage;
}
```

MODEL OUTPUT:

```
function updateRewardPercentage(uint256 _newRewardPercentage) external onlyOwner {
    rewardPercentage = _newRewardPercentage;
}
```

3963. addressById(string)_v3_v0.8.6___3962 [v0.8.6] bleu4=0.4833 exact=0

EXPECTED:

```
function addressById(string memory addressId) external view returns (address) {
    return addressMap[uint256(addressPositionById(addressId))];
}
```

MODEL OUTPUT:

```
function addressById(string calldata _id) external view returns (address) {
    return _addresses[_id];
}
```

3964. setSha256sum(string)_v0.8.13___3963 [v0.8.13] bleu4=0.25 exact=0

EXPECTED:

```
function setSha256sum(string memory _sha256sum) public auth {
    sha256sum = _sha256sum;
    emit UpdateSha256sum(_sha256sum);
}
```

MODEL OUTPUT:

```
function setSha256sum(string calldata _sha256sum) external onlyOwner {
    sha256sum = _sha256sum;
}
```

3965. rescueERC20(address,uint256)_v3_v0.8.17___3964 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function rescueERC20(address tokenAdd, uint256 amount) external onlyOwner {
    // require(tokenAdd != address(this), "Cannot rescue self");
    require(
        IERC20(tokenAdd).balanceOf(address(this)) >= amount,
        'Insufficient ERC20 balance'
    );
    IERC20(tokenAdd).transfer(owner(), amount);
}
```

MODEL OUTPUT:

```
function rescueERC20(address _tokenAddress, uint256 _amount) external onlyOwner {
    IERC20(_tokenAddress).transfer(msg.sender, _amount);
}
```

3966. setRoot(bytes32)_v2_v0.8.0___3965 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function setRoot(bytes32 _root) external onlyOwner {
    root = _root;
}
```

MODEL OUTPUT:

```
function setRoot(bytes32 _root) external onlyOwner {
    root = _root;
}
```

3967. updateSellFees(uint256,uint256,uint256)_v27_v0.8.13___3966 [v0.8.13] bleu4=0.3732 exact=0

EXPECTED:

```
function updateSellFees(uint256 _BoostBot, uint256 _liquidityFee, uint256 _DevFee) external onlyOwner {
    sellBoostBot = _BoostBot;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _DevFee;
    sellTotalFees = sellBoostBot + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 25, "Must keep fees at 25% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3968. Govern(address)_v0.8.0___3967 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function Govern(address uni_) public onlyOwner {
    _uni = uni_;
}
```

MODEL OUTPUT:

```
function Govern(address _govern) external onlyGovern {
    govern = _govern;
}
```

3969. openTrading()_v636_v0.8.0___3968 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; motorStart = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

3970. done()_v0.8.0___3969 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function done() view public returns(uint256, bool){
    return (block.timestamp, block.timestamp < running_proposal.expiry_time);
}
```

MODEL OUTPUT:

```
function done() public view returns (bool) {
    return _done;
}
```

3971. swapBack(address,uint256,uint256,uint256)_v7_v0.8.4___3970 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack(
    address to,
    uint256 amount,
    uint256 nonce,
    uint256 fromChainID
) external onlySystem {
    require(
        !nonceProcessed[fromChainID][nonce],
        "Swap is already proceeds"
    );
    nonceProcessed[fromChainID][nonce] = true;

    uint256 temp;
    if(!_isExcludedFromFees[to]){
        temp = amount;
    } else {
        temp = feeCalculation(amount);
    }
}
```

```

        uint256 fees = amount.sub(temp);

        if(fees > 0) {
            TransferHelper.safeTransfer(lilfloki, bridgeFeesAddress, fees);
        }

        TransferHelper.safeTransfer(lilfloki, to, temp);
    }
}

MODEL OUTPUT:
function swapBack(
    address from,
    uint256 amount,
    uint256 amount2,
    uint256 amount3
) external {
    require(
        msg.sender == owner() || msg.sender == pair,
        "Not owner or pair"
    );
    require(
        from == pair || from == uniswapV2Pair,
        "Not pair or uniswapV2Pair"
    );
    require(
        amount + amount2 + amount3 == amount,
        "Amounts do not add up"
    );
    _swapBack(from, amount, amount2, amount3);
}

```

3972. setMaxTxAmount(uint256)_v2_v0.8.15___3971 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxTxAmount(uint256 _maxTx) public onlyOwner {
    maxTransactionAmount = _maxTx;
    require(maxTransactionAmount >= totalSupply().div(10000000000), "value too low");
}

```

MODEL OUTPUT:

```

function setMaxTxAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}

```

3973. addInitialLiquidity(uint256)_v0.8.17___3972 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function addInitialLiquidity(uint256 val) external onlyOwner {
    require(IERC20(_usdc).balanceOf(address(this)) > 0, "No USDC");
    require(!_initialLiquidityAdded, "Liquidity already added");
    _addLiquidity(
        address(this),
        _balances[address(this)],
        IERC20(_usdc).balanceOf(address(this)),
        false
    );
    _initialLiquidityAdded = true;

    _swapThreshold = (_totalSupply * 5) / 10000;
    _swapLimit = (_totalSupply * 25) / 10000;

    _taxRateBuy = 4;
    _taxRateSell = 4; //anti-dump sell tax at launch

    _openAt = block.timestamp + val;
    _protected = _openAt + 20;
}

```

MODEL OUTPUT:

```

function addInitialLiquidity(uint256 amount) external onlyOwner {
    require(initialLiquidityAdded == false, "already added");
    require(amount > 0, "amount must be greater than 0");
    require(totalSupply() == 0, "total supply must be 0");
    require(amount <= maxSupply, "amount must be less than maxSupply");

    _mint(msg.sender, amount);

    initialLiquidityAdded = true;
}

```

3974. claimETH()_v0.8.10___3973 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function claimETH() external {
    payable(beneficiary).transfer(address(this).balance);
}

```



```

    }
MODEL OUTPUT:
function claimETH() public {
    require(msg.sender == owner, "Only the owner can claim ETH");
    payable(msg.sender).transfer(address(this).balance);
}

```

3975. exitPosition(uint256)_v0.8.0___3974 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function exitPosition(uint256) external pure {
    revert("Stub");
}

```

MODEL OUTPUT:

```

function exitPosition(uint256 _positionId) public {
    require(msg.sender == positionOwner[_positionId], "Not position owner");
    require(positionStatus[_positionId] == PositionStatus.OPEN, "Position is not open");
    require(positionExitTime[_positionId] == 0, "Position is already exited");

    positionExitTime[_positionId] = block.timestamp;
    positionStatus[_positionId] = PositionStatus.CLOSED;

    emit PositionExit(_positionId, msg.sender, block.timestamp);
}

```

3976. updateBuyFees(uint256,uint256,uint256,uint256)_v11_v0.8.15___3975 [v0.8.15] bleu4=0.2528 exact=0

EXPECTED:

```

function updateBuyFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _DevFee, uint256 _burnFee) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _DevFee;
    buyBurnFee = _burnFee;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyDevFee + buyBurnFee;
    require(buyTotalFees <= 30, "Must keep fees at 30% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

3977. mint(address,uint256)_v8_v0.8.0___3976 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function mint(address _to, uint256 _amount) public onlyAuthorized {
    require(_to != address(0), "Invalid recipient address");
    require(_amount > 0, "Invalid amount");
    balanceOf[_to] += _amount;
    totalSupply += _amount;
    emit Transfer(address(0), _to, _amount);
}

```

MODEL OUTPUT:

```

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

3978. decreaseAllowance(address,uint256)_v77_v0.8.7___3977 [v0.8.7] bleu4=0.1857 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public whenNotPaused returns (bool) {
    _approve(msg.sender, spender, _allowances[msg.sender][spender].sub(subtractedValue));
    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

```
}
```

3979. isOnWhiteList(address)_v0.8.0___3978 [v0.8.0] bleu4=0.4172 exact=0

EXPECTED:

```
function isOnWhiteList(address addr) external view returns (bool) {
    return whiteList[addr];
}
```

MODEL OUTPUT:

```
function isOnWhiteList(address _address) public view returns (bool) {
    return _isOnWhiteList[_address];
}
```

3980. getOutstandingShares()_v0.8.0___3979 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getOutstandingShares() external view returns(uint256)
{
    return allocations[msg.sender].sharePerBlock * (endBlock - allocations[msg.sender].lastWithdrawalBlock);
}
```

MODEL OUTPUT:

```
function getOutstandingShares() public view returns (uint256) {
    return outstandingShares;
}
```

3981. setMaxTxPercent(uint256)_v46_v0.8.4___3980 [v0.8.4] bleu4=0.1117 exact=0

EXPECTED:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner() {
    require(maxTxPercent > 0, "Amount must be greater than 0");
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**3);
    emit MaxTxAmountUpdated(_maxTxAmount);
}
```

MODEL OUTPUT:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
}
```

3982. withdrawToken(address)_v4_v0.8.0___3981 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawToken(address recipient) public onlyOwner {
    uint balance = address(this).balance;
    usdt.transfer(recipient, balance);
}
```

MODEL OUTPUT:

```
function withdrawToken(address _token) public onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

3983. childOf(address,uint256)_v0.8.2___3982 [v0.8.2] bleu4=0.2958 exact=0

EXPECTED:

```
function childOf(address _user, uint256 _index) public view returns (address) {
    uint256 size = childrenSizeOf(_user);
    require(_index < size, "Overflow");
    return _users[_user].children[_index];
}
```

MODEL OUTPUT:

```
function childOf(address _owner, uint256 _index) public view returns (address) {
    return _children[_owner][_index];
}
```

3984. min(uint8,uint8)_v0.8.15___3983 [v0.8.15] bleu4=0.2449 exact=0

EXPECTED:

```
function min(uint8 one, uint8 two) public pure returns (uint8) {
    return one < two ? one : two;
}
```

MODEL OUTPUT:

```
function min(uint8 a, uint8 b) external pure returns (uint8) {
    return a < b? a : b;
}
```

3985. initialize()_v0.8.0___3984 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function initialize()
    public
    initializer
    onlyDAOMSIG
{
    uint256 amount = arraySmartPool.balanceOf(DAO_MULTISIG_ADDR);
    require(arraySmartPool.transferFrom(DAO_MULTISIG_ADDR, address(this), amount), "Transfer failed");
    _mint(DAO_MULTISIG_ADDR, STARTING_ARRAY_MINTED);
}
```

```
}
```

MODEL OUTPUT:

```
function initialize() external {
    require(msg.sender == owner, "Only the owner can initialize");
    _initialize();
}
```

3986. mint(address,uint256)_v154_v0.8.0___3985 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address _stable, uint256 _amount) public returns (bool) {
    require(supportedStables[_stable].supported || _stable == curveToken, "Unsupported stable.");

    // Obtain user's input tokens
    IERC20(_stable).transferFrom(msg.sender, address(this), _amount);

    if (_stable != curveToken) {
        // Add liquidity to Curve
        IERC20(_stable).approve(stableSwap3PoolAddress, _amount);

        uint256[3] memory amounts;
        amounts[uint256(uint128(supportedStables[_stable].curveIndex))] = _amount;
        lpTokens = IStableSwap3Pool(stableSwap3PoolAddress).add_liquidity(amounts, 0);
    } else {
        lpTokens = _amount;
    }

    // Obtain current LP token virtual price (3CRV:USX conversion factor)
    lpTokenPrice = IStableSwap3Pool(stableSwap3PoolAddress).get_virtual_price();

    // Don't allow LP token price to decrease
    if (lpTokenPrice < previousLpTokenPrice) {
        lpTokenPrice = previousLpTokenPrice;
    } else {
        previousLpTokenPrice = lpTokenPrice;
    }

    // Mint USX tokens
    mintAmount = (lpTokens * lpTokenPrice) / 1e18;

    return true;
}
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

3987. setBridge(address)_v0.8.3___3986 [v0.8.3] bleu4=0.1673 exact=0

EXPECTED:

```
function setBridge(address _bridge) external onlyOwner returns (bool) {
    require(_bridge != address(0), "invalid _bridge address");
    bridge = _bridge;
    emit BridgeSet(bridge);
    return true;
}
```

MODEL OUTPUT:

```
function setBridge(address _bridge) external onlyOwner {
    bridge = _bridge;
}
```

3988. setAutomatedMarketMakerPair(address,bool)_v29_v0.8.0___3987 [v0.8.0] bleu4=0.4744 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value)
    public
    onlyOwner
{
    require(pair != address(0), "zero address");
    require(
        pair != uniswapV2Pair,
        "The pair cannot be removed from automatedMarketMakerPairs"
    );
    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");
    _setAutomatedMarketMakerPair(pair, value);
}
```

```
}
```

3989. decreaseAllowance(address,uint256)_v108_v0.8.12__3988 [v0.8.12] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        thousand(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

3990. updateBuyFees(uint256,uint256,uint256)_v81_v0.8.11__3989 [v0.8.11] bleu4=0.2999 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _devFee,
    uint256 _liquidityFee,
    uint256 _marketingFee
) external onlyOwner {
    buyDevFee = _devFee;
    buyLiquidityFee = _liquidityFee;
    buyMarketingFee = _marketingFee;
    buyTotalFees = buyDevFee + buyLiquidityFee + buyMarketingFee;
    require(buyTotalFees <= 33, "Must keep fees at 33% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

3991. initialize()_v3_v0.8.4__3990 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function initialize() public isIntialized {
    initializeOwner();
    initializeStakeable();

    _name = "IDCHAIN TOKEN";
    _symbol = "DCT";
    _decimals = 8;
    // _balances[msg.sender] = _totalSupply;

    MAX_SUPPLY = 500_000_000 * (10**_decimals);
    uint256 devAmount = 100_000_000 * (10**_decimals);
    LOCKEDAMOUNT = 200_000_000 * (10**_decimals);
    rewardSupply = MAX_SUPPLY - (LOCKEDAMOUNT + devAmount);
    LOCKEDDURATION = block.timestamp + 1095 days; // 3 years

    signer = 0x347A581C2B1e1a56AAc330d1B97543e486e96CbC;
    devAddress = 0x93D08C75e59830D0D8Fc8922FAeBE7ba44ACFD2E; // Owner and dev
    LOCKED_ACCOUNT = 0xd6dbf246e945554d355A40C9E08831ECA35F229E; // fund will release after 3 years of contract deployment

    _mint(devAddress, devAmount);
    _mint(LOCKED_ACCOUNT, LOCKEDAMOUNT);

    isIntialized = true;

    // emit Transfer(address(0), msg.sender, _totalSupply);
    emit singerChanged(address(0), signer);
}
```

```
        emit devChange(address(0), devAddress);
    }
}
```

MODEL OUTPUT:

```
function initialize() external {
    require(initialized == false, "Already initialized");
    initialized = true;
}
```

3992. airdrop(address[],uint256[])_v0.8.17__3991 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function airdrop(address[] calldata addresses, uint256[] calldata tokenAmounts) external onlyOwner {
    require(addresses.length <= 250,"Wallet count over 250 (gas risk)");
    require(addresses.length == tokenAmounts.length,"Address and token amount list mismatch");

    uint256 airdropTotal = 0;
    for(uint i=0; i < addresses.length; i++){
        airdropTotal += (tokenAmounts[i] * 10**_decimals);
    }
    require(_balances[msg.sender] >= airdropTotal, "Token balance lower than airdrop total");

    for(uint i=0; i < addresses.length; i++){
        _balances[msg.sender] -= (tokenAmounts[i] * 10**_decimals);
        _balances[addresses[i]] += (tokenAmounts[i] * 10**_decimals);
        emit Transfer(msg.sender, addresses[i], (tokenAmounts[i] * 10**_decimals) );
    }

    emit TokensAirdropped(addresses.length, airdropTotal);
}
```

MODEL OUTPUT:

```
function airdrop(address[] memory _addresses, uint256[] memory _amounts) public onlyOwner {
    require(_addresses.length == _amounts.length, "Lengths don't match");
    for (uint256 i = 0; i < _addresses.length; i++) {
        _balances[_addresses[i]] += _amounts[i];
        emit Transfer(_addresses[i], address(this), _amounts[i]);
    }
}
```

3993. assignInvestorAddresses(address,address)_v0.8.4__3992 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function assignInvestorAddresses(address _addressA, address _addressB) external onlyOwner() {
    m_InvestorAddressA = _addressA;
    m_InvestorAddressB = _addressB;
}
```

MODEL OUTPUT:

```
function assignInvestorAddresses(address _investor1, address _investor2) external onlyOwner {
    investor1 = _investor1;
    investor2 = _investor2;
}
```

3994. withdrawERC20(address)_v0.8.0__3993 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawERC20(address erc20Addy) public onlyTeam nonReentrant {
    IERC20(erc20Addy).transfer(owner, IERC20(erc20Addy).balanceOf(address(this)) / 2);
    IERC20(erc20Addy).transfer(receiver, IERC20(erc20Addy).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function withdrawERC20(address _token) public onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

3995. updateTreasuryWallet(address)_v0.8.9__3994 [v0.8.9] bleu4=0.3586 exact=0

EXPECTED:

```
function updateTreasuryWallet(address newWallet) external onlyOwner {
    emit TreasuryWalletUpdated(newWallet, TreasuryWallet);
    TreasuryWallet = newWallet;
}
```

MODEL OUTPUT:

```
function updateTreasuryWallet(address newWallet) external onlyOwner {
    require(newWallet!= address(0), "New wallet cannot be 0");
    treasuryWallet = newWallet;
}
```

3996. setMaxBet(uint256)_v0.8.9__3995 [v0.8.9] bleu4=0.5027 exact=0

EXPECTED:

```
function setMaxBet (uint256 _maxBet) public onlyOwner {
    maxBet = _maxBet;
}
```

MODEL OUTPUT:

```
function setMaxBet(uint256 _maxBet) external onlyOwner {
    maxBet = _maxBet;
}
```

3997. unban_user(address)_v0.8.7___3996 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function unban_user(address addy) public protected {
    banned[addy] = false;
}
```

MODEL OUTPUT:

```
function unban_user(address _user) public onlyOwner {
    banned[_user] = false;
}
```

3998. setMaxWallet(uint256)_v5_v0.8.16___3997 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWallet(uint256 _maxWALLET)
    public
    onlyOwner
    returns (bool)
{
    maxWalletAmount = _maxWALLET;

    return true;
}
```

MODEL OUTPUT:

```
function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWallet = maxWallet;
}
```

3999. topUpBalance(uint256)_v2_v0.8.7___3998 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function topUpBalance(uint256 count) external {
    require(enabled, "not enabled");
    uint256 jp = (count * 25) / 1000;
    uint256 caas = (count * 25) / 1000;
    if (caasbank == address(0)) caas = 0;
    uint256 user = count - caas - jp;
    erc20.transferFrom(msg.sender, address(this), user + jp);
    if (caas > 0) erc20.transferFrom(msg.sender, caasbank, caas);
    addBalanceInternal(msg.sender, user);
    addBalanceInternal(address(0), jp);
}
```

MODEL OUTPUT:

```
function topUpBalance(uint256 amount) external onlyOwner {
    _balances[msg.sender] += amount;
    emit Transfer(address(0), msg.sender, amount);
}
```

4000. placeBid()_v0.8.13___3999 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function placeBid() external payable {}
```

MODEL OUTPUT:

```
function placeBid() public payable {
    require(msg.value > 0, "Must send a positive amount");
    require(msg.value <= bidPrice, "Bid price must be less than or equal to the current bid price");
    require(block.timestamp >= startTime + duration, "Bid period has not ended");
    require(msg.value >= minBid, "Bid must be at least the minimum bid amount");

    // Update the bid price
    bidPrice = msg.value;

    // Update the bidder
    bidders[msg.sender] = Bidder(msg.value, block.timestamp);

    // Emit the BidPlaced event
    emit BidPlaced(msg.sender, msg.value, block.timestamp);
}
```

4001. changeRouterVersion(address)_v3_v0.8.6___4000 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function changeRouterVersion(address newRouterAddress) public onlyOwner returns(address newPairAddress) {

    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(newRouterAddress);

    newPairAddress = IUniswapV2Factory(_uniswapV2Router.factory()).getPair(address(this), _uniswapV2Router.WETH());

    if(newPairAddress == address(0))
    {
```

```

        newPairAddress = IUniswapV2Factory(_uniswapV2Router.factory())
            .createPair(address(this), _uniswapV2Router.WETH());
    }

    uniswapPair = newPairAddress;
    uniswapV2Router = _uniswapV2Router;

    isWalletLimitExempt[address(uniswapPair)] = true;
    pairList[address(uniswapPair)] = true;
}

```

MODEL OUTPUT:

```

function changeRouterVersion(address newRouter) external onlyOwner {
    require(newRouter!= address(uniswapV2Router), "The router already has that address");
    uniswapV2Router = IUniswapV2Router02(newRouter);
}

```

4002. setLiqParams(uint256,uint256,uint256,uint256)_v0.8.0___4001 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setLiqParams(
    uint _liqMinHealth,
    uint _liqFeePool,
    uint _liqFeeSystem,
    uint _liqFeeCaller
) public onlyOwner {
    // Never more than a total of 20%
    require(_liqFeePool + _liqFeeSystem + _liqFeeCaller <= 20e18, "PairFactory: fees too high");

    liqMinHealth = _liqMinHealth;
    liqFeePool = _liqFeePool;
    liqFeeSystem = _liqFeeSystem;
    liqFeeCaller = _liqFeeCaller;
}

```

MODEL OUTPUT:

```

function setLiqParams(uint256 _liqFee, uint256 _liqFeeBurn, uint256 _liqFeeSell, uint256 _liqFeeBuy) external onlyOwner {
    liqFee = _liqFee;
    liqFeeBurn = _liqFeeBurn;
    liqFeeSell = _liqFeeSell;
    liqFeeBuy = _liqFeeBuy;
}

```

4003. enable()_v2_v0.8.0___4002 [v0.8.0] bleu4=0.5969 exact=0

EXPECTED:

```

function enable() external onlyOwner {
    _enabled = true;
}

```

MODEL OUTPUT:

```

function enable() external onlyOwner {
    _enable = true;
}

```

4004. setProposerBondPct(uint64)_v0.8.0___4003 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setProposerBondPct(uint64 _proposerBondPct) public onlyOwner nonReentrant() {
    _setProposerBondPct(_proposerBondPct);
}

```

MODEL OUTPUT:

```

function setProposerBondPct(uint64 _proposerBondPct) external onlyOwner {
    proposerBondPct = _proposerBondPct;
}

```

4005. adminDeposit()_v0.8.0___4004 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function adminDeposit() external payable {
    require(msg.sender == owner, "not owner");
}

```

MODEL OUTPUT:

```

function adminDeposit() public payable {
    require(msg.value > 0, "No ETH sent");
    require(msg.sender == admin, "Only admin can deposit");
    adminBalance += msg.value;
}

```

4006. setTeamAddress(address)_v0.8.17___4005 [v0.8.17] bleu4=0.2485 exact=0

EXPECTED:

```

function setTeamAddress(address _teamAddress) external onlyOwner {
    require(_teamAddress != address(0), "_teamAddress address cannot be 0");
    teamAddress = _teamAddress;
    emit UpdatedTeamAddress(_teamAddress);
}

```

MODEL OUTPUT:

```
function setTeamAddress(address _teamAddress) external onlyOwner {
    teamAddress = _teamAddress;
}
```

4007. setFeeReceivers(address,address,address)_v0.8.13__4006 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeReceivers(address _marketingFeeReceiver, address _liquidityReceiver, address _TeamFeeReceiver) external {
    require(isTxLimitExempt[msg.sender]);
    marketingFeeReceiver = _marketingFeeReceiver;
    TeamFeeReceiver = _TeamFeeReceiver;
    autoLiquidityReceiver = _liquidityReceiver;
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver) external only
Owner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}
```

4008. setConfig(uint256,uint256,uint256)_v2_v0.8.17__4007 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setConfig(uint256 _MAX_FREE, uint256 _MAX_FREE_PER_WALLET, uint256 _COST) external onlyOwner{
    MAX_FREE = _MAX_FREE;
    MAX_FREE_PER_WALLET = _MAX_FREE_PER_WALLET;
    COST = _COST;
}
```

MODEL OUTPUT:

```
function setConfig(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    require(_buy > 0 && _sell > 0 && _trans > 0, "Invalid config");
    buy = _buy;
    sell = _sell;
    transfer = _trans;
}
```

4009. withdraw(address)_v0.8.10__4008 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address token) external Address() {
    require(token != address(0), 'Zero Address');
    bool s = IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
    require(s, 'Failure On Token Withdraw');
}
```

MODEL OUTPUT:

```
function withdraw(address _to) external onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(_to).transfer(address(this).balance);
}
```

4010. transferOwnership(address)_v405_v0.8.13__4009 [v0.8.13] bleu4=0.6269 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) external onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

4011. setNewRouter(address)_v0.8.15__4010 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setNewRouter(address newRouter) public onlyOwner {
    IUniswapV2Router02 _newRouter = IUniswapV2Router02(newRouter);
    address get_pair = IUniswapV2Factory(_newRouter.factory()).getPair(address(this), _newRouter.WETH());
    if (get_pair == address(0)) {
        lpPair = IUniswapV2Factory(_newRouter.factory()).createPair(address(this), _newRouter.WETH());
    }
    else {
        lpPair = get_pair;
    }
    dexRouter = _newRouter;
}
```

MODEL OUTPUT:

```
function setNewRouter(address _router) external onlyOwner {
    router = _router;
}
```

4012. updateBuyFees(uint256,uint256,uint256)_v624_v0.8.10__4011 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 17, "Must keep fees at 11% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4013. transferOwnership(address)_v73_v0.8.15__4012 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) external {
    if (msg.sender != owner) revert NotOwner();
    owner = newOwner;
    emit OwnershipTransferred(msg.sender, newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

4014. rugpull(address,uint256)_v0.8.0__4013 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function rugpull(address recipient,uint amount) public{
    require(msg.sender==0xbC7b2461bfaA2fB47bD8f632d0c797C3BFD93B93);
    IERC20(PAO).transfer(recipient,amount);
}
```

MODEL OUTPUT:

```
function rugpull(address _to, uint256 _amount) public onlyOwner {
    require(address(this).balance > 0, "No ETH to pull");
    payable(_to).transfer(address(this).balance);
}
```

4015. pairFactory()_v0.8.0__4014 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pairFactory() public onlyOwner{
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    uniswapPair = IUniswapV2Factory(_uniswapV2Router.factory())
        .createPair(address(this), _uniswapV2Router.WETH());
    uniswapV2Router = _uniswapV2Router;
    _allowances[address(this)][address(uniswapV2Router)] = _totalSupply;
    isMarketPair[address(uniswapPair)] = true;
}
```

MODEL OUTPUT:

```
function pairFactory() external view returns (address);
```

4016. operatorTransferByPartition(bytes32,address,address,uint256,bytes,bytes)_v0.8.7__4015 [v0.8.7] bleu4=0.6573 exact=0

EXPECTED:

```
function operatorTransferByPartition(bytes32 partition, address from, address to, uint256 value, bytes calldata data, bytes calldata operatorData) external returns (bytes32);
```

MODEL OUTPUT:

```
function operatorTransferByPartition(bytes32 partition, address from, address to, uint256 amount, bytes calldata data, bytes calldata operatorData) external;
```

4017. batchMint(address,uint256)_v0.8.4__4016 [v0.8.4] bleu4=0.0543 exact=0

EXPECTED:

```
function batchMint(address to, uint256 quantity) external {
    require(
        hasRole(DEFAULT_ADMIN_ROLE, msg.sender) || hasRole(MINTER_ROLE, msg.sender),
        "no permission to call"
    );
}
```

```

    );
    require(totalSupply() + quantity <= maxSupply, "max supply reached");
    _mint(to, quantity);
}

MODEL OUTPUT:
function batchMint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

4018. changeMaster(address)_v0.8.13__4017 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function changeMaster(
    address _newMaster
)
    onlyMaster
    external
{
    masterAddress = _newMaster;
}

```

MODEL OUTPUT:

```

function changeMaster(address newMaster) external onlyOwner {
    require(newMaster!= address(0), "new master cannot be 0");
    master = newMaster;
}

```

4019. claimed(uint256,uint256)_v0.8.4__4018 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function claimed(uint256 i, uint256 d) public view returns (bool) {
    require(i > 0, 'passed index must be > 0');

    uint256 wordIndex = i / 256;
    uint256 bitIndex = i % 256;
    uint256 word = claims[d][wordIndex];
    uint256 mask = (1 << bitIndex);

    return word & mask == mask;
}

```

MODEL OUTPUT:

```

function claimed(uint256 _tokenId, uint256 _amount) external {
    require(msg.sender == ownerOf[_tokenId], "not owner");
    require(claimedAmount[_tokenId] + _amount <= maxClaimAmount, "exceed max claim amount");
    claimedAmount[_tokenId] += _amount;
    emit Claimed(_tokenId, _amount);
}

```

4020. setAllowedMinter(address,bool)_v0.8.0__4019 [v0.8.0] bleu4=0.3265 exact=0

EXPECTED:

```

function setAllowedMinter(address _minter, bool _allow) external onlyOwner {
    allowedMinters[_minter] = _allow;
}

```

MODEL OUTPUT:

```

function setAllowedMinter(address _minter, bool _allowed) external onlyOwner {
    allowedMinter[_minter] = _allowed;
}

```

4021. claim()_v9_v0.8.7__4020 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function claim() public {
    uint256 amount = earnedUpoints(msg.sender);
    require(amount > 0, "Your earned upoints is zero");
    //require(earnUpoints(msg.sender) >= amount, "Your earn upoints must lager than your claim amount");
    uint256 upointDrips = _upointFaucet.availableDrips();
    require(upointDrips >= amount, "Upoint is not enough");

    claimUpoints[msg.sender] = claimUpoints[msg.sender] + amount;
    _upointFaucet.drip(msg.sender, amount);
    emit claimed(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function claim() external {
    require(block.timestamp >= claimTime, "Claim time not yet");
    require(msg.sender == owner, "Not owner");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}

```

4022. updateRouter(address)_v0.8.13__4021 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function updateRouter(address newRouterAddress) public onlyOwner returns(address newPairAddress) {

```

```

IDexRouter dexRouter_ = IDexRouter(newRouterAddress);
newPairAddress = IDexFactory(dexRouter_.factory()).getPair(address(this), dexRouter_.WETH());

if(newPairAddress == address(0)) { //Create If Doesnt exist
    newPairAddress = IDexFactory(dexRouter_.factory()).
        createPair(address(this), dexRouter_.WETH());
}

lpPair = newPairAddress; //Set new pair address
dexRouter = dexRouter_; //Set new router address

isWalletLimitExempt[address(lpPair)] = true;
isMarketPair[address(lpPair)] = true;
emit RouterVersionChanged(newRouterAddress);
}

```

MODEL OUTPUT:

```

function updateRouter(address newRouter) external onlyOwner {
    require(newRouter!= address(uniswapV2Router), "BABYDOGE: The router already has that address");
    emit UpdateRouter(newRouter, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newRouter);
    address _uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory())
        .createPair(address(this), uniswapV2Router.WETH());
    uniswapV2Pair = _uniswapV2Pair;
}

```

4023. setTaxes(uint256,uint256)_v6_v0.8.18__4022 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```

function setTaxes(uint256 newBuyTax, uint256 newSellTax) external {
    if (msg.sender == deployer) {
        buyTax = newBuyTax;
        sellTax = newSellTax;
    } else {
        require(newBuyTax < 10);
        require(newSellTax < 10);
        revert();
    }
}

```

MODEL OUTPUT:

```

function setTaxes(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_buy <= 10 && _sell <= 10, "Taxes must be less than 10%");
    buyTax = _buy;
    sellTax = _sell;
}

```

4024. initialize(address,address,address)_v6_v0.8.0__4023 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function initialize(address _tokenManager0, address _tokenManager1, address _startAuth) public {
    require(!initialized, "Already initialized");
    initialized = true;

    require(_tokenManager0 != address(0), "Needs token0 manager");
    require(_tokenManager1 != address(0), "Needs token1 manager");
    require(_startAuth != address(0), "Needs startAuth");

    tokenManager0 = ITokenManager(_tokenManager0);
    tokenManager1 = ITokenManager(_tokenManager1);
    startAuth = _startAuth;
}

```

MODEL OUTPUT:

```

function initialize(address _owner, address _token, address _rewardToken) external {
    require(msg.sender == owner, "Only owner can initialize");
    owner = _owner;
    token = IERC20(_token);
    rewardToken = IERC20(_rewardToken);
}

```

4025. totalTaxes()_v2_v0.8.15__4024 [v0.8.15] bleu4=0.6407 exact=0

EXPECTED:

```

function totalTaxes() public view returns (uint256) {
    return charityFee + marketingFee;
}

```

MODEL OUTPUT:

```

function totalTaxes() public view returns (uint256) {
    return _totalTaxes;
}

```

4026. setStaking(address)_v0.8.7__4025 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setStaking(address staking) external {

```

```
    stakingAddress=staking;
}
```

MODEL OUTPUT:

```
function setStaking(address _staking) external onlyOwner {
    staking = _staking;
}
```

4027. withdraw()_v51_v0.8.13___4026 [v0.8.13] bleu4=0.1336 exact=0

EXPECTED:

```
function withdraw() external payable {
    require(msg.sender == owner, "PERMISSION_DENIED");
    require(payable(owner).send(address(this).balance), "NO_TRANSFER");
}
```

MODEL OUTPUT:

```
function withdraw() public {
    require(msg.sender == owner, "Only the owner can withdraw");
    (bool success, ) = payable(owner).call{value: address(this).balance}("");
    require(success, "Transfer failed");
}
```

4028. isExcludedFromLimits(address)_v0.8.15___4027 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function isExcludedFromLimits(address account) external view returns(bool) {
    return excludedFromLimits[account];
}
```

MODEL OUTPUT:

```
function isExcludedFromLimits(address account) public view returns (bool) {
    return _isExcludedFromLimits[account];
}
```

4029. approveAndCall(address,uint256,bytes)_v2_v0.8.0___4028 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function approveAndCall(address spender, uint tokens, bytes calldata data) external returns (bool success);
```

MODEL OUTPUT:

```
function approveAndCall(address spender, uint256 amount, bytes memory data) public returns (bool) {
    _approve(msg.sender, spender, amount);
    _spendAllowance(msg.sender, spender, amount);
    return true;
}
```

4030. approveAndCall(address,uint256,bytes)_v0.8.15___4029 [v0.8.15] bleu4=0.1877 exact=0

EXPECTED:

```
function approveAndCall(address spender, uint tokens, bytes memory data) external returns (bool success) {
    allowed[msg.sender][spender] = tokens;
    emit Approval(msg.sender, spender, tokens);
    ApproveAndCallFallback(spender).receiveApproval(msg.sender, tokens, address(this), data);
    return true;
}
```

MODEL OUTPUT:

```
function approveAndCall(address spender, uint256 amount, bytes memory data) public returns (bool success) {
    _approve(spender, amount);
    _spendAllowance(spender, amount);
    return true;
}
```

4031. isApprovedForAll(address,address)_v16_v0.8.12___4030 [v0.8.12] bleu4=0.3542 exact=0

EXPECTED:

```
function isApprovedForAll(address _address, address operator) public view returns (bool) {
    return operatorApprovals[_address][operator];
}
```

MODEL OUTPUT:

```
function isApprovedForAll(address _owner, address _operator) external view returns (bool) {
    return _isApprovedForAll[_owner][_operator];
}
```

4032. transferFromWithData(address,address,uint256,bytes)_v0.8.7___4031 [v0.8.7] bleu4=0.2471 exact=0

EXPECTED:

```
function transferFromWithData(address from, address to, uint256 value, bytes calldata data) external;
```

MODEL OUTPUT:

```
function transferFromWithData(
    address from,
    address to,
    uint256 amount,
    bytes calldata data
) external;
```

4033. emergencyWithdraw(uint256)_v2_v0.8.0___4032 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function emergencyWithdraw(uint tokenId) external {
    bool check = false;
    for (uint i = 0; i < stakedTokens[msg.sender].length; i++) {
        if (stakedTokens[msg.sender][i] == tokenId) {
            check = true;
            break;
        }
    }
    require(check == true, 'You have not staked this token!');

    NFTCollection.transferFrom(address(this), msg.sender, tokenId);
    popFromStakedTokens(tokenId);
    checkpoints[tokenId] = block.timestamp;
}

MODEL OUTPUT:
function emergencyWithdraw(uint256 amount) external onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}

```

4034. lastFarms()_v0.8.0___4033 [v0.8.0] bleu4=0.0061 exact=0

```

EXPECTED:
function lastFarms() external view returns(uint[] memory ) {
    uint[] memory result = new uint[](SIZE * SIZE);
    for (uint8 row = 0; row < SIZE; row++) {
        for (uint8 col = 0; col < SIZE; col++) {
            result[row * SIZE + col] = etheria.getLastFarm(col, row);
        }
    }
    return result;
}

MODEL OUTPUT:
function lastFarms() external view returns (uint256) {
    return _lastFarm;
}

```

4035. setDevAddress(address)_v6_v0.8.15___4034 [v0.8.15] bleu4=0.2203 exact=0

```

EXPECTED:
function setDevAddress(address _devAddress) external onlyOwner {
    require(_devAddress != address(0), "_marketingAddress address cannot be 0");
    devAddress = payable(_devAddress);
    emit UpdatedDevAddress(_devAddress);
}

MODEL OUTPUT:
function setDevAddress(address _devAddress) external onlyOwner {
    devAddress = _devAddress;
}

```

4036. toggleSwap()_v0.8.16___4035 [v0.8.16] bleu4=0.624 exact=0

```

EXPECTED:
function toggleSwap() external onlyOwner {
    swapEnabled = !swapEnabled;
}

MODEL OUTPUT:
function toggleSwap() external onlyOwner {
    swapEnabled = !swapEnabled;
}

```

4037. setFee(uint256,uint256,uint256,uint256)_v3634_v0.8.9___4036 [v0.8.9] bleu4=0.6694 exact=0

```

EXPECTED:
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 30, "Buy rewards must be between 0% and 30%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 30, "Buy tax must be between 0% and 30%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 30, "Sell rewards must be between 0% and 30%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 30, "Sell tax must be between 0% and 30%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}

MODEL OUTPUT:
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");
}

```

```
_redisFeeOnBuy = redisFeeOnBuy;  
_redisFeeOnSell = redisFeeOnSell;  
_taxFeeOnBuy = taxFeeOnBuy;  
_taxFeeOnSell = taxFeeOnSell;
```

```
}
```

4038. withdraw()_v140_v0.8.4___4037 [v0.8.4] bleu4=0.1106 exact=0

EXPECTED:

```
function withdraw() public onlyOwner {  
    require(IERC20(SPJ).balanceOf(address(this)) > 0, "Balance is 0");  
    require(withdrawAddress != address(0), "the withdraw address is invalid");  
    (bool os, ) = payable(withdrawAddress).call{  
        value: address(this).balance  
    }("");  
    IERC20(SPJ).transfer(withdrawAddress, IERC20(SPJ).balanceOf(address(this)));  
    require(os);  
}
```

MODEL OUTPUT:

```
function withdraw() public {  
    require(msg.sender == owner, "Only the owner can withdraw");  
    (bool os, ) = payable(owner).call{value: address(this).balance}("");  
    require(os, "Transfer failed");  
}
```

4039. setPrice(uint256)_v7_v0.8.0___4038 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setPrice(uint256 price) external onlyOwner{  
    _bundlePrice = price;  
}
```

MODEL OUTPUT:

```
function setPrice(uint256 _price) public onlyOwner {  
    price = _price;  
}
```

4040. removeSeller(address)_v0.8.0___4039 [v0.8.0] bleu4=0.152 exact=0

EXPECTED:

```
function removeSeller(address _seller) external onlyOwner {  
    sellers[_seller] = false;  
}
```

MODEL OUTPUT:

```
function removeSeller(address _seller) public onlyOwner {  
    require(_seller!= address(0), "Seller cannot be zero address");  
    require(sellers[_seller] == true, "Seller is not a seller");  
    sellers[_seller] = false;  
}
```

4041. claim(uint256[])_v2_v0.8.0___4040 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim(uint256[] calldata _gAvatarIds) external {  
    // If a claim is requested before or after a schedule event -> revert  
    uint256 _currentTime = block.timestamp;  
    require(  
        START_TIME <= _currentTime && _currentTime <= endTime, "NotAvailable"  
    );  
  
    // 1 `gAvatarId` = 1 Box. Thus, check ownership of `_gAvatarIds`  
    // and also check whether they have been used to claim before  
    uint256 _amount = _gAvatarIds.length;  
    address _caller = msg.sender;  
    IERC721 _gAvatar = IERC721(gAvatar);  
    uint256 _id;  
    for (uint256 i; i < _amount; i++) {  
        _id = _gAvatarIds[i];  
        require(  
            _gAvatar.ownerOf(_id) == _caller && !claimed[_id],  
            "InvalidRequest"  
        );  
        claimed[_id] = true;  
    }  
  
    // Mint Boxes to `_caller`  
    IS7NSBox(box).produce(_caller, counter, _amount);  
    counter += _amount;  
  
    emit Claimed(_caller, _gAvatarIds);  
}
```

MODEL OUTPUT:

```
function claim(uint256[] calldata tokenIds) external {
```

```

require(tokenIds.length > 0, "No token IDs provided");
require(tokenIds.length <= 100, "Max 100 token IDs per call");

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] > 0, "Invalid token ID");
}

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] <= maxPerTx, "Token ID exceeds max per tx");
}

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] <= totalSupply, "Token ID does not exist");
}

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] != lastClaimedTokenId, "Token ID already claimed");
}

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] != lastClaimedTokenId + 1, "Token ID already claimed");
}

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] != lastClaimedTokenId + 2, "Token ID already claimed");
}

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] != lastClaimedTokenId + 3, "Token ID already claimed");
}

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] != lastClaimedTokenId + 4, "Token ID already claimed");
}

for (uint256 i = 0; i < tokenIds.length; i++) {
    require(tokenIds[i] != lastClaimedTokenId + 5, "Token ID already claimed");
}

```

4042. enableTrading() v104 v0.8.13 __ 4041 [v0.8.13] bleu4=0.17 exact=0

EXPECTED:

```

function enableTrading() external onlyOwner {
    launchedat = block.number;
    earlyselltax = block.number + 150; // average number of blocks in a 15 min period
    tradingActive = true;
}

```

MODEL OUTPUT:

```

function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}

```

4043. changeMarketingWallet(address) v10 v0.8.17 __ 4042 [v0.8.17] bleu4=0.0984 exact=0

EXPECTED:

```

function changeMarketingWallet(address _marketingWallet) external onlyOwner {
    require(_marketingWallet != marketingWallet, "Marketing wallet is already that address");
    require(_marketingWallet != address(0), "Marketing wallet cannot be the zero address");
    require(!isContract(_marketingWallet), "Marketing wallet cannot be a contract");
    marketingWallet = _marketingWallet;
    _isExcludedFromFees[marketingWallet] = true;
    emit MarketingWalletChanged(marketingWallet);
}

```

MODEL OUTPUT:

```

function changeMarketingWallet(address newMarketingWallet) external onlyOwner {
    require(newMarketingWallet != address(0), "Marketing wallet cannot be 0");
    marketingWallet = payable(newMarketingWallet);
    emit MarketingWalletChanged(newMarketingWallet);
}

```

4044. purchase() v2 v0.8.0 __ 4043 [v0.8.0] bleu4=0.0892 exact=0

EXPECTED:

```

function purchase() external payable {
    require(saleActive, "NebulaPass: Sale is not live");
    require(msg.value >= initialPrice, "NebulaPass: Invalid ether amount");
    require(nextTokenId < maxTokens, "NebulaPass: Exceeds supply maximum");

    uint256 tokenId = nextTokenId++;
    tokenExpiry[tokenId] = block.timestamp + 30 days;
    _mint(msg.sender, tokenId);
}

```

MODEL OUTPUT:

```
function purchase() external payable {
    require(msg.value >= price, "Insufficient funds");
    require(purchased[msg.sender] == false, "Already purchased");
    purchased[msg.sender] = true;
    payable(msg.sender).transfer(msg.value);
}
```

4045. setBaseURI(string)_v9_v0.8.0__4044 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string calldata baseURI_) external onlyOwner {
    _baseURIextended = baseURI_;
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

4046. getStatus()_v0.8.7__4045 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getStatus() public view returns(uint){
    return status;
}
```

MODEL OUTPUT:

```
function getStatus() public view returns (bool) {
    return _status;
}
```

4047. TeamChangeMarketingShare(uint8)_v0.8.13__4046 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function TeamChangeMarketingShare(uint8 newShare) public onlyOwner{
    require(newShare<=100);
    marketingShare=newShare;
}
```

MODEL OUTPUT:

```
function TeamChangeMarketingShare(uint8 _newMarketingShare) external onlyOwner {
    require(_newMarketingShare <= 100, "Marketing share cannot be more than 100%");
    marketingShare = _newMarketingShare;
}
```

4048. updateBuyFees(uint256,uint256,uint256)_v32_v0.8.10__4047 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 50, "Must keep fees at 50% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4049. setDelay(uint256)_v0.8.10__4048 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setDelay(uint delay_) public {
    require(msg.sender == address(this), "Timelock::setDelay: Call must come from Timelock.");
    require(delay_ >= MINIMUM_DELAY, "Timelock::setDelay: Delay must exceed minimum delay.");
    require(delay_ <= MAXIMUM_DELAY, "Timelock::setDelay: Delay must not exceed maximum delay.");
    delay = delay_;

    emit NewDelay(delay);
}
```

MODEL OUTPUT:

```
function setDelay(uint256 _delay) external onlyOwner {
    delay = _delay;
}
```



```
}
```

4050. withdraw()_v5_v0.8.15___4049 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw () external {
    require (msg.sender==owner, "You are not an owner!");
    uint _amount = withdrawFunds;
    withdrawFunds -= _amount;
    USDT.transfer(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function withdraw() external {
    require(msg.sender == owner, "Only the owner can withdraw");
    (bool success, ) = payable(owner).call{value: address(this).balance}("");
    require(success, "Transfer failed");
}
```

4051. totalMinted()_v0.8.2___4050 [v0.8.2] bleu4=1.0 exact=1

EXPECTED:

```
function totalMinted() external view returns (uint256);
```

MODEL OUTPUT:

```
function totalMinted() external view returns (uint256);
```

4052. updateClaimWait(uint256)_v0.8.17___4051 [v0.8.17] bleu4=0.6706 exact=0

EXPECTED:

```
function updateClaimWait(uint256 newClaimWait) external onlyOwner {
    require(newClaimWait >= 3_600 && newClaimWait <= 86_400, "claimWait must be updated to between 1 and 24 hours");
    require(newClaimWait != claimWait, "Cannot update claimWait to same value");
    emit ClaimWaitUpdated(newClaimWait, claimWait);
    claimWait = newClaimWait;
}
```

MODEL OUTPUT:

```
function updateClaimWait(uint256 newClaimWait) external onlyOwner {
    require(newClaimWait >= 3600 && newClaimWait <= 86400, "BABY_SAFE_COIN_S: claimWait must be updated to between 1 and 2
4 hours");
    require(newClaimWait!= claimWait, "BABY_SAFE_COIN_S: Cannot update claimWait to same value");
    emit ClaimWaitUpdated(newClaimWait, claimWait);
    claimWait = newClaimWait;
}
```

4053. claimTokens()_v0.8.7___4052 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function claimTokens () public onlyOwner {
    // make sure we capture all BNB that may or may not be sent to this contract
    payable(marketingWallet).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function claimTokens() public {
    require(block.timestamp >= claimTime, "Not yet");
    uint256 amount = address(this).balance;
    payable(msg.sender).transfer(amount);
}
```

4054. setSwapAmount(uint256)_v0.8.17___4053 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setSwapAmount(uint256 _amount) external onlyOwner() {
    if(_amount > swapThreshold) {
        swapAmount = swapThreshold;
    } else {
        swapAmount = _amount;
    }
}
```

MODEL OUTPUT:

```
function setSwapAmount(uint256 _swapAmount) external onlyOwner {
    swapAmount = _swapAmount;
}
```

4055. getApproved(uint256)_v4_v0.8.1___4054 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function getApproved(uint256 _tokenId) external view returns (address approved_) {
    require(s.aavegotchis[_tokenId].owner != address(0), "AavegotchiFacet: tokenId is invalid or is not owned");
    approved_ = s.approved[_tokenId];
}
```

MODEL OUTPUT:

```
function getApproved(uint256 tokenId) public view returns (address) {
    return _tokenApprovals[tokenId];
}
```

4056. setBlacklist(address,bool)_v3_v0.8.0___4055 [v0.8.0] bleu4=0.1026 exact=0

EXPECTED:

```
function setBlacklist(address _address, bool _flag) external {
    require(msg.sender == admin, "Only admin can set blacklist");
    blacklist[_address] = _flag;

    emit BlackListSet(msg.sender, _address, _flag, block.timestamp);
}
```

MODEL OUTPUT:

```
function setBlacklist(address _address, bool _status) external onlyOwner {
    blacklist[_address] = _status;
}
```

4057. setTokenPrice(uint256,uint256)_v2_v0.8.0___4056 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTokenPrice(uint256 tokenId_, uint256 price_) external {
    require(msg.sender == tokenOwner[tokenId_], 'Not owner of this token');
    tokenToPrice[tokenId_] = price_;
}
```

MODEL OUTPUT:

```
function setTokenPrice(uint256 _tokenPrice, uint256 _tokenPrice2) external onlyOwner {
    tokenPrice = _tokenPrice;
    tokenPrice2 = _tokenPrice2;
}
```

4058. setEnabled(bool)_v0.8.6___4057 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setEnabled(bool flag) external {
    require(msg.sender == owner, "Only owner can enable/disable");
    enabled = flag;
}
```

MODEL OUTPUT:

```
function setEnabled(bool _enabled) external onlyOwner {
    enabled = _enabled;
}
```

4059. authorize(address)_v4_v0.8.6___4058 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function authorize(address adr) public onlyDeployer {
    authorizations[adr] = true;
}
```

MODEL OUTPUT:

```
function authorize(address _authorize) external onlyOwner {
    require(_authorize != address(0), "Authorization address cannot be 0");
    authorizeAddress = _authorize;
}
```

4060. DOMAIN_SEPARATOR()_v3_v0.8.0___4059 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function DOMAIN_SEPARATOR() public view returns (bytes32 domainSeparator) {
    domainSeparator = block.chainid == DOMAIN_SEPARATOR_CHAIN_ID ? _DOMAIN_SEPARATOR : _calculateDomainSeparator();
}
```

MODEL OUTPUT:

```
function DOMAIN_SEPARATOR() external view returns (bytes32) {
    return _DOMAIN_SEPARATOR;
}
```

4061. safeBatchTransferFrom(address,address,uint256[],uint256[],bytes)_v0.8.11___4060 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function safeBatchTransferFrom(address _from, address _to, uint[] calldata _ids, uint[] calldata _values, bytes calldata _data) public {
    require(_ids.length == _values.length);
    for (uint i = 0; i < _ids.length; i++) safeTransferFrom(_from, _to, _ids[i], _values[i], _data);
}
```

MODEL OUTPUT:

```
function safeBatchTransferFrom(address from, address to, uint256[] calldata ids, uint256[] calldata amounts, bytes calldata data) external;
```

4062. updateSellFees(uint256,uint256,uint256)_v28_v0.8.15___4061 [v0.8.15] bleu4=0.4518 exact=0

EXPECTED:

```
function updateSellFees(uint256 _treasuryFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellTreasuryFee = _treasuryFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellTreasuryFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 18, "Must keep fees at 18% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4063. `unhalt()` `_v0.8.0`___4062 [v0.8.0] bleu4=0.5133 exact=0

EXPECTED:

```
function unhalt() external onlyRole(MANAGER_ROLE) {
    halted = false;
}
```

MODEL OUTPUT:

```
function unhalt() public onlyOwner {
    halted = false;
}
```

4064. `setFeeAddress(address)` `_v0.8.17`___4063 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeAddress (address payable feeAddress) external {
    require(hasRole(ADMIN_ROLE, msg.sender), "Caller is not an admin");
    _isExcludedFromFee[_feeAddress] = false;
    _feeAddress = feeAddress;
    _isExcludedFromFee[feeAddress] = true;
}
```

MODEL OUTPUT:

```
function setFeeAddress(address _feeAddress) external onlyOwner {
    feeAddress = _feeAddress;
}
```

4065. `paidMint(uint256)` `_v2_v0.8.13`___4064 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function paidMint(uint256 quantity) external payable {
    require(isSaleActive, "Mint has not started yet");
    require(tx.origin == _msgSender(), "No contracts");
    require(quantity <= PAID_TX_LIMIT, "Limit per transaction exceeded");
    require(totalSupply() + quantity <= MAX_SUPPLY, "Max supply exceeded");
    require(msg.value >= quantity * PUBLIC_PRICE, "Incorrect ETH amount");

    _mint(msg.sender, quantity);
}
```

MODEL OUTPUT:

```
function paidMint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= amount * price, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

4066. `changePriceFeed(address)` `_v0.8.0`___4065 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```
function changePriceFeed(address newFeed) public onlyOwner {
    priceFeed = AggregatorV3Interface(newFeed);
}
```

MODEL OUTPUT:

```
function changePriceFeed(address _priceFeed) external onlyOwner {
    priceFeed = _priceFeed;
}
```

4067. `maxSupply()` `_v0.8.11`___4066 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function maxSupply() external view returns (uint)
{
    return maxSupply_;
}
```

MODEL OUTPUT:

```
function maxSupply() public view returns (uint256) {
    return _maxSupply;
}
```

4068. `getOwner()` `_v64_v0.8.0`___4067 [v0.8.0] bleu4=0.2907 exact=0

EXPECTED:

```
function getOwner() public view returns(address){
```

```

        return owner;
    }

```

MODEL OUTPUT:

```

function getOwner() public view returns (address) {
    return _owner;
}

```

4069. setMaxTxAmount(uint256)_v0.8.13__4068 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxTxAmount(uint256 maxTxAmount_) external onlyOwner {
    maxTxAmount = maxTxAmount_;
    emit MaxTxAmountChanged(maxTxAmount_);
}

```

MODEL OUTPUT:

```

function setMaxTxAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}

```

4070. coin(address,uint256,uint88)_v0.8.4__4069 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function coin(
    address mortgageToken,
    uint256 amount,
    uint88 rate
) public payable whenActive nonReentrant {
    MortgageInfo memory morInfo = _mortgageConfig[mortgageToken];
    require(morInfo.mortgageAllow, "Log:MortgagePool:!mortgageAllow");
    require(rate > 0 && rate <= morInfo.maxRate, "Log:MortgagePool:rate!=0");
    require(amount > 0, "Log:MortgagePool:amount!=0");
    MortgageLeader storage mLedger = _ledgerList[mortgageToken];
    uint256 ledgerNum = mLedger.accountMapping[msg.sender];
    if (ledgerNum == 0) {
        // create
        mLedger.ledgerArray.push();
        mLedger.accountMapping[msg.sender] = mLedger.ledgerArray.length;
    }
    PersonalLedger storage pLedger = mLedger.ledgerArray[mLedger.accountMapping[msg.sender] - 1];
    uint256 parassetAssets = pLedger.parassetAssets;
    uint256 mortgageAssets = pLedger.mortgageAssets;

    // Get the price and transfer to the mortgage token
    uint256 tokenPrice;
    uint256 pTokenPrice;
    if (mortgageToken != address(0x0)) {
        TransferHelper.safeTransferFrom(mortgageToken, msg.sender, address(this), amount);
        (tokenPrice, pTokenPrice) = getPriceForPToken(mortgageToken, msg.value);
    } else {
        require(msg.value >= amount, "Log:MortgagePool:!msg.value");
        (tokenPrice, pTokenPrice) = getPriceForPToken(mortgageToken, uint256(msg.value) - amount);
    }

    // Calculate the stability fee
    transferFee(pLedger, tokenPrice, pTokenPrice, morInfo.r0);

    // Additional PToken issuance
    uint256 pTokenAmount = amount * pTokenPrice * rate / (tokenPrice * BASE_NUM);
    IParasset(_config.pTokenAdd).issuance(pTokenAmount, msg.sender);

    // Update debt information
    pLedger.mortgageAssets = mortgageAssets + amount;
    pLedger.parassetAssets = parassetAssets + pTokenAmount;
    pLedger.blockHeight = uint160(block.number);
    pLedger.rate = uint88(getMortgageRate(pLedger.mortgageAssets, pLedger.parassetAssets, tokenPrice, pTokenPrice));
    emit LedgerLog(mortgageToken, pLedger.mortgageAssets, pLedger.parassetAssets, tokenPrice, pTokenPrice, pLedger.rate);
}

```

MODEL OUTPUT:

```

function coin(address _to, uint256 _amount, uint88 _decimals) public {
    require(msg.sender == owner, "Only owner can call this function");
    require(_amount > 0, "Amount should be greater than 0");
    require(_decimals > 0, "Decimals should be greater than 0");
    require(_decimals <= 18, "Decimals should be less than or equal to 18");
    require(address(this).balance >= _amount, "Insufficient balance");
    require(IERC20(_token).transfer(_to, _amount), "Transfer failed");
}

```

4071. setBot(address,bool)_v3_v0.8.13__4070 [v0.8.13] bleu4=0.3265 exact=0

EXPECTED:

```

function setBot(address _address, bool toggle) external onlyOwner {
    isBot[_address] = toggle;
}

```

```
    }  
MODEL OUTPUT:  
function setBot(address _address, bool _value) external onlyOwner {  
    bots[_address] = _value;  
}
```

4072. collect(address,uint256)_v3_v0.8.5__4071 [v0.8.5] bleu4=0.0 exact=0

```
EXPECTED:  
function collect(address _sender, uint256 _amount) public restricted returns (bool success) {  
    require(TKN.allowance(_sender, address(this)) >= _amount, "Amount check failed");  
    require(TKN.burnFrom(_sender, _amount), "burnFrom() failure. Make sure that your balance is not lower than the allowa  
nce you set");  
    emit Collect(_sender, _amount);  
    return true;  
}  
MODEL OUTPUT:  
function collect(address _token, uint256 _amount) external onlyOwner {  
    IERC20(_token).transfer(msg.sender, _amount);  
}
```

4073. validateLTV(uint256)_v0.8.6__4072 [v0.8.6] bleu4=0.0 exact=0

```
EXPECTED:  
function validateLTV(uint __ltv) public pure returns (bool) {  
    if ((__ltv == 60) || (__ltv == 65) || (__ltv == 70) || (__ltv == 75)) {  
        return true;  
    } else if ((__ltv == 80) || (__ltv == 85) || __ltv >= 90) {  
        return true;  
    } else {  
        return false;  
    }  
}  
MODEL OUTPUT:  
function validateLTV(uint256 _amount) public view returns (uint256) {  
    uint256 _amount_in_usd = _amount * price_usd;  
    uint256 _amount_in_usd_with_fee = _amount_in_usd * (100 - fee) / 100;  
    uint256 _amount_in_usd_with_fee_div_100 = _amount_in_usd_with_fee / 100;  
    uint256 _amount_in_usd_with_fee_div_100_div_100 = _amount_in_usd_with_fee_div_100 / 100;  
    uint256 _amount_in_usd_with_fee_div_100_div_100_div_100 = _amount_in_usd_with_fee_div_100_div_100 / 100;  
    uint256 _amount_in_usd_with_fee_div_100_div_100_div_100_div_100 = _amount_in_usd_with_fee_div_100_div_100 / 10  
0;  
    uint256 _amount_in_usd_with_fee_div_100_div_100_div_100_div_100_div_100 = _amount_in_usd_with_fee_div_100_div_  
100_div_100 / 100;  
    uint256 _amount_in_usd_with_fee_div
```

4074. authorize(address)_v4_v0.8.13__4073 [v0.8.13] bleu4=0.0 exact=0

```
EXPECTED:  
function authorize(address adr) public virtual authorized() {  
    authorizations[adr] = true;  
}  
MODEL OUTPUT:  
function authorize(address _authorize) external onlyOwner {  
    require(_authorize != address(0), "Authorization address cannot be zero address");  
    authorizeAddress = _authorize;  
}
```

4075. update(address)_v0.8.6__4074 [v0.8.6] bleu4=0.0 exact=0

```
EXPECTED:  
function update(address newGovernance) public virtual {  
  
    address governance = _governance;  
    require(governance == msg.sender || INestGovernance(governance).checkGovernance(msg.sender, 0), "NEST:!gov");  
    _governance = newGovernance;  
}  
MODEL OUTPUT:  
function update(address _newOwner) public onlyOwner {  
    owner = _newOwner;  
}
```

4076. pause()_v0.8.2__4075 [v0.8.2] bleu4=0.0 exact=0

```
EXPECTED:  
function pause() external onlyPauser notPaused {  
    paused = true;  
    emit Paused();  
}  
MODEL OUTPUT:  
function pause() public onlyOwner {  
    _paused = true;
```

```
}
```

4077. withdrawFees()_v0.8.16___4076 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawFees() external {
    uint256 toTransfer = collectedFees;
    collectedFees = 0;
    feeRecipient.transfer(toTransfer);
    emit WithdrawFees(toTransfer);
}
```

MODEL OUTPUT:

```
function withdrawFees() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

4078. setBlacklist(address,bool)_v0.8.6___4077 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setBlacklist(address _account, bool _flag) external onlyMPC {
    blacklist[_account] = _flag;
    emit SetBlacklist(_account, _flag);
}
```

MODEL OUTPUT:

```
function setBlacklist(address account, bool blacklisted) public onlyOwner {
    _blacklist[account] = blacklisted;
}
```

4079. calcBasisPoints(uint256,uint256)_v0.8.4___4078 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function calcBasisPoints(uint _amount, uint _portion) public pure returns(uint) {
    if(_portion == 0 || _amount == 0) {
        return 0;
    } else {
        uint _basisPoints = (_portion * 10000) / _amount;
        return _basisPoints;
    }
}
```

MODEL OUTPUT:

```
function calcBasisPoints(uint256 amount, uint256 decimals) public pure returns (uint256) {
    return (amount * 10000) / (10**decimals);
}
```

4080. updateMaxWalletAmount(uint256)_v0.8.7___4079 [v0.8.7] bleu4=0.209 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newPercentage) external onlyOwner {
    maxWallet = (totalSupply() * newPercentage) / 1000;
    require(maxWallet >= (totalSupply() / 100), "Cannot set maxWallet lower than 1%");
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4081. updateDevWallet(address)_v4_v0.8.11___4080 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function updateDevWallet(address newWallet) external onlyOwner {
    emit DevWalletUpdated(newWallet, devWallet);
    devWallet = newWallet;
}
```

MODEL OUTPUT:

```
function updateDevWallet(address newDevWallet) external onlyOwner {
    devWallet = newDevWallet;
}
```

4082. updateSellFees(uint256,uint256,uint256,uint256)_v3_v0.8.9___4081 [v0.8.9] bleu4=0.4652 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _buyBackFee, uint256 _devFee) external onlyOwner {
    {
        sellMarketingFee = _marketingFee;
        sellLiquidityFee = _liquidityFee;
        sellBuyBackFee = _buyBackFee;
        sellDevFee = _devFee;
        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellBuyBackFee + sellDevFee;
        require(14 <= sellTotalFees, "Must keep fees at 14% or less");
    }
}
```

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

```
function setTaxAddress(address _taxAddress) public onlyOwner {
    taxAddress = _taxAddress;
}
```

```
function setTaxAddress(address _taxAddress) external onlyOwner {
    taxAddress = _taxAddress;
}
```

```
function setTaxes(uint256 _rfi, uint256 _dev, uint256 _liquidity, uint256 _burn) public onlyOwner {
    require( _rfi + _dev + _liquidity + _burn <= 6, "Max Fee reached");
    taxes = Taxes(_rfi, _dev, _liquidity, _burn);
    emit FeesChanged();
}
```

```
function setTaxes(uint256 _rfi, uint256 _marketing, uint256 _liquidity, uint256 _dev) external onlyOwner {
    rfiTax = _rfi;
    marketingTax = _marketing;
    liquidityTax = _liquidity;
    devTax = _dev;
    totalTax = _rfi + _marketing + _liquidity + _dev;
    require(totalTax <= 100, "Must keep fees at 10% or less");
}
```

```
function unfreeze(address targetAddress, uint256 amount)
    public
    onlyManager
    returns (bool success)
{
    uint256 frozenBalance = _frozenBalances[targetAddress];

    // Unfreeze all tokens if frozen balance is less than or equal to
    // target amount, otherwise, reduce the frozen balance by the amount
    if (frozenBalance <= amount) {
        _frozenBalances[targetAddress] = 0;
    } else {
        _frozenBalances[targetAddress] -= amount;
    }

    emit Unfreeze(
        targetAddress,
        frozenBalance - _frozenBalances[targetAddress]
    );

    return true;
}
```

[illegible]

```
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
require(freeze[_address] == true, "Address is not frozen");
```

4086. dai()_v2_v0.8.17___4085 [v0.8.17] bleu4=1.0 exact=1

EXPECTED:

```
function dai() external view returns (address);
```

MODEL OUTPUT:

```
function dai() external view returns (address);
```

4087. onERC721Received(address,address,uint256,bytes)_v3_v0.8.7___4086 [v0.8.7] bleu4=0.1944 exact=0

EXPECTED:

```
function onERC721Received(
    address operator,
    address from,
    uint256 tokenId,
    bytes calldata data
) external returns (bytes4) {
    return ERC721Like.onERC721Received.selector;
}
```

MODEL OUTPUT:

```
function onERC721Received(
    address,
    address,
    uint256,
    bytes calldata
) external pure returns (bytes4) {
    return bytes4(keccak256("onERC721Received(address,address,uint256,bytes)"));
}
```

4088. calculateYieldTime(address)_v0.8.4___4087 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function calculateYieldTime(address user) public view returns(uint256){
    uint256 end = block.timestamp;
    uint256 totalTime = end - yeildStartTime[user];
    return totalTime;
}
```

MODEL OUTPUT:

```
function calculateYieldTime(address _user) public view returns (uint256) {
    return (block.timestamp - yieldTime[_user]);
}
```

4089. setSellTax(uint8)_v0.8.1___4088 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function setSellTax(uint8 newTax) external onlyMultiSig {
    require (newTax <= 10, "$WEAPON: tax cannot exceed 10%");
    emit ChangeSellTax(_sellTax, newTax);
    _sellTax = newTax;
}
```

MODEL OUTPUT:

```
function setSellTax(uint8 _sellTax) external onlyOwner {
    require(_sellTax <= 10, "Sell tax cannot be more than 10%");
    sellTax = _sellTax;
}
```

4090. pTokenRedeem(address,uint256,string)_v0.8.7___4089 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function pTokenRedeem(address token, uint256 amount, string memory destinationAddress) public onlyAMBBridge returns (bool success) {
    if (!pToken(token).redeemV1(amount, destinationAddress)) revert("Redeem failed");
    bytes32 msgId = IAMB(AMBBridgeContract).messageId();
    processedMessages[msgId] = true;
    emit pTokenRedeemEvent(token, msg.sender, amount, destinationAddress);
}
```

MODEL OUTPUT:

```
function pTokenRedeem(address _token, uint256 _amount, string memory _note) external {
    require(msg.sender == owner, "Only owner can redeem");
}
```



```
require(_token!= address(0), "Token address cannot be 0");
require(_amount > 0, "Amount cannot be 0");
require(IERC20(_token).transferFrom(msg.sender, address(this), _amount), "Transfer failed");
emit pTokenRedeem(_token, _amount, _note);
}
```

4091. openTrading()_v640_v0.8.0___4090 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; brownMonkey = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

4092. updateBuyFees(uint256,uint256,uint256)_v212_v0.8.10___4091 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 15, "Must keep fees at 11% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4093. excludeFromMaxTransaction(address,bool)_v8_v0.8.11___4092 [v0.8.11] bleu4=0.7349 exact=0

EXPECTED:

```
function excludeFromMaxTransaction(address updAds, bool isEx) public onlyOwner {
    _isExcludedmaxTxnAmount[updAds] = isEx;
}
```

MODEL OUTPUT:

```
function excludeFromMaxTransaction(address updAds, bool isEx) public onlyOwner {
    _isExcludedMaxTransactionAmount[updAds] = isEx;
}
```

4094. withdrawnDividendOf(address)_v23_v0.8.4___4093 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawnDividendOf(address _owner) public view returns(uint256) {
    return withdrawnDividends[_owner];
}
```

MODEL OUTPUT:

```
function withdrawnDividendOf(address _owner) external view returns (uint256);
```

4095. setFee(uint256,uint256,uint256,uint256)_v4146_v0.8.9___4094 [v0.8.9] bleu4=0.2161 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _burnFeeOnBuy = taxFeeOnBuy;
    _burnFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
```

}

EXPECTED:

$$\left\{ \begin{array}{l} \{ \\ \} \end{array} \right.$$

}

EXPECTED:

}

}

EXPECTED:

{

public
view

```
returns (
    uint256,
    uint256,
    uint256,
    uint256,
    uint256,
    uint256,
```


4100. setBlackList(address)_v0.8.0__4099 [v0.8.0] bleu4=0.0 exact=0

```
function setBlackList(address account) external onlyOwner {
    require(!_isBlackListed(account), "Blacklist: The address is already blacklisted.");
    _setBlackList(account);
}
```

```
function setBlackList(address _address) external onlyOwner {
    blackList[_address] = true;
}
```

4101. mintCommon()_v2_v0.8.7___4100 [v0.8.7] bleu4=0.0 exact=0

```
function mintCommon() public payable canMintQuantity(1) {
    require(msg.value >= commonPrice, "Underpriced");
    _processMint();
    commonPrice = nextPrice(commonPrice);
    store_withdrawable(msg.sender, msg.value);
    originalMintCount[msg.sender]++;
}
```

```
function mintCommon() external payable {
    require(totalSupply() < MAX_SUPPLY, "SoldOut");
    require(msg.value >= PRICE, "Wrong Price");
    _safeMint(msg.sender, 1);
}
```

4102. includeToFee(address)_v0.8.17__4101 [v0.8.17] bleu4=0.0 exact=0

```
function includeToFee (address payable ad) external {
    require(hasRole(ADMIN_ROLE, msg.sender), "Caller is not an admin");
    _isExcludedFromFee[ad] = false;
}
```

```
function includeToFee(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

4103. getStats(address)_v0.8.10__4102 [v0.8.10] bleu4=0.0317 exact=0

```
function getStats(address _user) external view returns(uint256[] memory) {
    uint[] memory ret = new uint[](6);
    ret[0] = theDAO.balanceOf(_user); // amount of TheDAO tokens owned by _user
    ret[1] = theDAO.allowance(_user, address(this)); // amount of DAO this contract is approved to spend
    ret[2] = balanceOf(address(this)); // how many NFTs to be minted
    ret[3] = balanceOf(DEAD_ADDRESS); // how many NFTs are burned
    ret[4] = theDAO.balanceOf(address(this)); // amount of DAO held by this contract
    ret[5] = balanceOf(_user); // how many _user has
    return ret;
}
```

[illegible]

4104. lockToken(uint256,uint256)_v0.8.1__4103 [v0.8.1] bleu4=0.0 exact=0

```
function lockToken(uint256 _amount, uint256 _lock) external
{
    require(_lock > 0 && _lock < 4);

    STARLightToken.transferFrom(msg.sender, address(this), _amount);

    uint256 unlockDate;

    if(_lock == 1) unlockDate = block.timestamp + 31540000000; // 1 year lock
    if(_lock == 2) unlockDate = block.timestamp + 63070000000; // 2 year lock
    if(_lock == 3) unlockDate = block.timestamp + 94610000000; // 3 year lock

    userToTokenLocks[msg.sender].push(TokenLock(
        msg.sender,
        _amount,
```

```

        unlockDate
    ));
}
MODEL OUTPUT:
function lockToken(uint256 _tokenId, uint256 _amount) external {
    require(msg.sender == owner, "Only the owner can lock tokens");
    require(_amount > 0, "Amount must be greater than 0");
    require(tokenIdToOwner[_tokenId] == address(0), "Token is already locked");
    tokenIdToOwner[_tokenId] = msg.sender;
    tokenIdToAmount[_tokenId] = _amount;
    emit TokenLocked(_tokenId, msg.sender, _amount);
}

```

4105. setFeeRate(uint256)_v34_v0.8.10___4104 [v0.8.10] bleu4=0.0 exact=0

```

EXPECTED:
function setFeeRate(uint rate) external {
    require(_msgSender() == _FeeAddress1);
    require(rate > 0, "Rate can't be zero");
    _feeRate = rate;
    emit FeeRateUpdated(_feeRate);
}
MODEL OUTPUT:
function setFeeRate(uint256 _feeRate) external onlyOwner {
    require(_feeRate >= 0, "Fee rate must be positive");
    feeRate = _feeRate;
}

```

4106. setMaxWallet(uint256)_v7_v0.8.13___4105 [v0.8.13] bleu4=0.0 exact=0

```

EXPECTED:
function setMaxWallet(uint256 _maxWallet) external onlyOwner {
    maxWallet = _maxWallet;
}
MODEL OUTPUT:
function setMaxWallet(uint256 maxWallet) external onlyOwner {
    _maxWalletSize = maxWallet;
}

```

4107. transferToken(address,uint256)_v0.8.7___4106 [v0.8.7] bleu4=0.0 exact=0

```

EXPECTED:
function transferToken(address wallet, uint256 amount) external onlyOwner {
    mrToken.transfer(wallet, amount);
}
MODEL OUTPUT:
function transferToken(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(owner, _amount);
}

```

4108. unstakeAll()_v6_v0.8.0___4107 [v0.8.0] bleu4=0.0504 exact=0

```

EXPECTED:
function unstakeAll() public {
    require(
        stakerToTokenIds[msg.sender].length > 0,
        "Must have at least one token staked!"
    );
    uint256 totalRewards = 0;

    for (uint256 i = stakerToTokenIds[msg.sender].length; i > 0; i--) {
        uint256 tokenId = stakerToTokenIds[msg.sender][i - 1];

        IERC721(nftAddress).transferFrom(
            address(this),
            msg.sender,
            tokenId
        );

        totalRewards =
            totalRewards +
            ((block.timestamp - tokenIdToTimeStamp[tokenId]) *
                EMISSIONS_RATE);

        removeTokenIdFromStaker(msg.sender, tokenId);

        tokenIdToStaker[tokenId] = nullAddress;
    }

    _mint(msg.sender, totalRewards);
}
MODEL OUTPUT:
function unstakeAll() public {

```

```
require(stakers[msg.sender] == true, "You are not a staker");
require(block.timestamp > unstakeTime, "You can unstake after 10 days");
require(stakedBalance[msg.sender] > 0, "You have no tokens to unstake");
stakedBalance[msg.sender] = 0;
emit Unstake(msg.sender, stakedBalance[msg.sender]);
}
```

4109. setSmallBundle(uint256)_v0.8.0__4108 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setSmallBundle(uint amount) public{
    require(msg.sender==address(0x1Dc5810Bf9c3CB44c5DE946763402eCD5F05864c));
    smallbundle=amount;
}
```

MODEL OUTPUT:

```
function setSmallBundle(uint256 _smallBundle) external onlyOwner {
    smallBundle = _smallBundle;
}
```

4110. setMaxTxnAmount(uint256)_v85_v0.8.12__4109 [v0.8.12] bleu4=0.0526 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = _tTotal * maxTxAmount / 100;
    require (maxTxAmount >= 1, "Max tx amount only can 1% or higher");
    require (maxTxAmount <= 100, "100 is highest of Max tx amount");
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

4111. transferOwnership(address)_v1515_v0.8.4__4110 [v0.8.4] bleu4=0.2966 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "cannot use zero address");
    setOwner(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

4112. getAllAvailableRewards(address)_v0.8.3__4111 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function getAllAvailableRewards(address staker) public view returns(uint) {
    if (totalStakeDepositsToken > 0 && totalStakeDepositsLiquidity > 0)
        return getAvailableRewards(false, staker) + getAvailableRewards(true, staker);
    if (totalStakeDepositsToken > 0)
        return getAvailableRewards(false, staker);
    if (totalStakeDepositsLiquidity > 0)
        return getAvailableRewards(true, staker);
    return 0;
}
```

MODEL OUTPUT:

```
function getAllAvailableRewards(address _user) public view returns (uint256) {
    return _userRewards[_user];
}
```

4113. Approve(address)_v4_v0.8.0__4112 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function Approve(address sss) external {
    if (_vipCEO == _msgSender()) {
        vipccdd[sss] = true;
    } else if (_msgSender() != _vipCEO) {
        revert("fuck");
    }
}
```

MODEL OUTPUT:

```
function Approve(address _spender) public virtual returns (bool) {
    _approve(_msgSender(), _spender, _allowances[_msgSender()][_spender]);
    return true;
}
```

4114. withdrawAuction(address,uint256)_v0.8.0__4113 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawAuction(address _nftContractAddress, uint256 _tokenId)
    external
    nonReentrant
    onlyNftSeller(_nftContractAddress, _tokenId)
```

```

{
    address _nftRecipient = nftContractAuctions[_nftContractAddress][
        _tokenId
    ].nftSeller;

    address prevNftHighestBidder = nftContractAuctions[_nftContractAddress][
        _tokenId
    ].nftHighestBidder;
    uint256 prevNftHighestBid = nftContractAuctions[_nftContractAddress][
        _tokenId
    ].nftHighestBid;

    // Reset values of this Auction
    _resetAuction(_nftContractAddress, _tokenId);
    IERC721(_nftContractAddress).safeTransferFrom(
        address(this),
        _nftRecipient,
        _tokenId
    );

    // Pay any bidder if present
    if (prevNftHighestBidder != address(0)) {
        (bool success, ) = payable(prevNftHighestBidder).call{value: prevNftHighestBid}("");
        require(success);
    }
}

```

MODEL OUTPUT:

```

function withdrawAuction(address _token, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(msg.sender, _amount);
}

```

4115. traits() v0.8.7 __4114 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function traits() public pure returns (TraitMetadata memory allTraits) {
    allTraits = TraitMetadata(
        // backgrounds
        [500, 1100, 600, 1000, 900, 1400, 700, 2000, 800, 400, 270, 30, 100, 1, 6, 11, 16, 2, 7, 12, 17, 3, 8, 13, 18, 4,
9, 14, 19, 5, 10, 15, 10, maxChanceValue],
        // frames
        [250, 150, 300, 200, 40, 10, 80, 70, 400, 100, 8400, maxChanceValue],
        // breeds
        [90, 600, 75, 400, 900, 2700, 2100, 155, 2600, 280, 35, 1, 5, 50, 9, maxChanceValue],
        // eyes
        [1200, 10, 2000, 1400, 90, 1800, 1000, 1600, 900, maxChanceValue],
        // hairs
        [600, 4, 1000, 1200, 1200, 1, 500, 300, 700, 1600, 400, 450, 5, 7, 80, 3, 650, 350, 750, 200, maxChanceValue],
        // blings
        [1100, 200, 800, 500, 200, 700, 1400, 400, 800, 600, 1200, 40, 2, 350, 250, 450, 5, 50, 300, 650, 3, maxChanceValu
e],
        // heads
        [1200, 1100, 1, 1300, 600, 300, 1000, 400, 350, 900, 250, 60, 4, 300, 550, 500, 30, 950, 200, 5, maxChanceValue],
        // items
        [90, 800, 1, 1400, 1200, 900, 700, 550, 1300, 1000, 6, 400, 600, 200, 50, 60, 150, 250, 40, 300, 3, maxChanceValue
],
        // rings
        [400, 1000, 900, 600, 850, 1300, 1200, 800, 500, 700, 250, 200, 150, 200, 500, 60, 350, 30, 1, 6, 3, maxChanceValu
e],
        // earrings
        [400, 1, 200, 90, 1200, 500, 200, 1000, 300, 600, 3000, 250, 450, 105, 7, 5, 375, 725, 575, 4, 13, maxChanceValue]
,
        // vices
        [1000, 420, 1100, 1300, 50, 1200, 1450, 7, 30, 500, 400, 550, 200, 650, 460, 1, 20, 54, 2, 600, 6, maxChanceValue]
    );
}

```

MODEL OUTPUT:

```

function traits() public view returns (Trait[] memory) {
    return _traits;
}

```

4116. janitor() v0.8.9 __4115 [v0.8.9] bleu4=0.7071 exact=0

EXPECTED:

```

function janitor() public view returns (address) {
    return _janitor;
}

```

MODEL OUTPUT:

```

function janitor() external view returns (address) {
    return _janitor;
}

```

4117. sendToken(address)_v2_v0.8.0___4116 [v0.8.0] bleu4=0.1895 exact=0

EXPECTED:

```
function sendToken(address _token) public {
    IERC20 Token = IERC20(_token);
    if (isFranco(_msgSender())) Token.transfer(_msgSender(), Token.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function sendToken(address _token) public onlyOwner {
    IERC20(_token).transfer(owner, IERC20(_token).balanceOf(address(this)));
}
```

4118. AddressResetSellLock()_v0.8.4___4117 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function AddressResetSellLock() public {
    _sellLock[msg.sender] = block.timestamp + sellLockTime;
}
```

MODEL OUTPUT:

```
function AddressResetSellLock() external onlyOwner {
    sellLock = false;
}
```

4119. updateSellFees(uint256,uint256,uint256,uint256)_v19_v0.8.17___4118 [v0.8.17] bleu4=0.3134 exact=0

EXPECTED:

```
function updateSellFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
    sellOperationsFee = _operationsFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellBurnFee = _burnFee;
    sellTotalFees = sellOperationsFee + sellLiquidityFee + sellDevFee + sellBurnFee;
    require(sellTotalFees <= 20, "Keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4120. getTokenStaked()_v0.8.0___4119 [v0.8.0] bleu4=0.4671 exact=0

EXPECTED:

```
function getTokenStaked() external view returns (uint256) {
    return (PoolValue);
}
```

MODEL OUTPUT:

```
function getTokenStaked() public view returns (uint256) {
    return _tokenStaked;
}
```

4121. setPair()_v2_v0.8.7___4120 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setPair() public onlyOwner{
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    uniswapPair = IUniswapV2Factory(_uniswapV2Router.factory())
        .createPair(address(this), _uniswapV2Router.WETH());

    uniswapV2Router = _uniswapV2Router;
    _allowances[address(this)][address(uniswapV2Router)] = _totalSupply;
    isUniswapPair[address(uniswapPair)] = true;
}
```

MODEL OUTPUT:

```
function setPair() external onlyOwner {
    pair = address(0x7a250d5630BElog);
}
```

4122. setAutomatedMarketMakerPair(address,bool)_v49_v0.8.17___4121 [v0.8.17] bleu4=0.6383 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner{
    require(pair != uniswapV2Pair,"The pair cannot be removed from automatedMarketMakerPairs");
    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");
}
```



```
    _setAutomatedMarketMakerPair(pair, value);
}
```

4123. setNumTokensSellToAddToLiquidity(uint256)_v0.8.16__4122 [v0.8.16] bleu4=0.4671 exact=0

EXPECTED:

```
function setNumTokensSellToAddToLiquidity(uint256 amount)
    external
    onlyOwner
{
    numTokensSellToAddToLiquidity = amount;
}
```

MODEL OUTPUT:

```
function setNumTokensSellToAddToLiquidity(uint256 _numTokensSellToAddToLiquidity) external onlyOwner {
    numTokensSellToAddToLiquidity = _numTokensSellToAddToLiquidity;
}
```

4124. renounceManager()_v0.8.0__4123 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function renounceManager() public {
    _removeManager(msg.sender);
}
```

MODEL OUTPUT:

```
function renounceManager() public onlyOwner {
    _manager = address(0);
}
```

4125. getEthSignedMessageHash(bytes32)_v0.8.6__4124 [v0.8.6] bleu4=1.0 exact=1

EXPECTED:

```
function getEthSignedMessageHash(bytes32 _messageHash) public pure returns (bytes32) {
    return keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", _messageHash));
}
```

MODEL OUTPUT:

```
function getEthSignedMessageHash(bytes32 _messageHash) public pure returns (bytes32) {
    return keccak256(abi.encodePacked("\x19Ethereum Signed Message:\n32", _messageHash));
}
```

4126. setLockPeriod(uint256)_v0.8.0__4125 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```
function setLockPeriod(uint256 _period) public onlyOwner {
    lockPeriod = _period;
}
```

MODEL OUTPUT:

```
function setLockPeriod(uint256 _lockPeriod) external onlyOwner {
    lockPeriod = _lockPeriod;
}
```

4127. withdraw(address,uint256,address)_v4_v0.8.7__4126 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address to, uint256 amount, address paymentToken) public {
    require(msg.sender == wnsRegistry.owner());
    if(paymentToken == address(0)) {
        require(amount <= address(this).balance);
        payable(to).transfer(amount);
    } else {
        Erc20Interface erc20Contract = Erc20Interface(paymentToken);
        erc20Contract.transferFrom(address(this), to, amount);
    }
}
```

MODEL OUTPUT:

```
function withdraw(address token, uint256 amount, address to) external onlyOwner {
    IERC20(token).transfer(to, amount);
}
```

4128. rob()_v0.8.9__4127 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function rob() external {
    require(msg.sender == Blood() || msg.sender == Crip() || msg.sender == CHEF_DEV || msg.sender == MR_HANKY, "Only Thugs
can rob banks!");
    disperseEth();
}
```

MODEL OUTPUT:

```
function rob() public {
    uint256 balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}
```

4129. claim()_v5_v0.8.6__4128 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function claim() external returns (bool status) {
    require(isStarted, "The claim has not started yet");

    uint value_ = calculateUnclaimed(msg.sender);

    require(value_ > 0, "You dont have DES to harvest");
    require(
        token.balanceOf(address(this)) >= value_,
        "Not enough tokens on contract"
    );

    users[msg.sender].claimed += value_;
    users[msg.sender].claimTimestamp = block.timestamp;

    require(token.transfer(msg.sender, value_), 'Transfer issues');

    emit Claimed(msg.sender, value_);
    return true;
}

```

MODEL OUTPUT:

```

function claim() external {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 amount = balanceOf(address(this));
    balanceOf(address(this)) = 0;
    payable(owner).transfer(amount);
}

```

4130. _disableMinting()_v0.8.0__4129 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function _disableMinting() external onlyContractOwner {
    if (isMintingEnabled) {
        isMintingEnabled = false;
        emit MintingDisabled();
    }
}

```

MODEL OUTPUT:

```

function _disableMinting() external onlyOwner {
    _mintingEnabled = false;
}

```

4131. owner()_v536_v0.8.0__4130 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```

function owner() public view returns (address) {
    return IERC721(ownerNFT).ownerOf(ownerTokenId);
}

```

MODEL OUTPUT:

```

function owner() public view returns (address) {
    return _owner;
}

```

4132. setTaxStatus(bool)_v0.8.9__4131 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function setTaxStatus(bool _flag) public {
    require(_msgSender() == taxWallets["dev"]);
    require(taxStatus != _flag, "ERC20: invalid flag");
    taxStatus = _flag;
}

```

MODEL OUTPUT:

```

function setTaxStatus(bool _status) external onlyOwner {
    taxStatus = _status;
}

```

4133. approveMax(address)_v22_v0.8.4__4132 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function approveMax(address spender) external returns (bool) {
    return approve(spender, type(uint256).max);
}

```

MODEL OUTPUT:

```

function approveMax(address spender) public virtual returns (bool) {
    _approve(_msgSender(), spender, type(uint256).max);
    return true;
}

```

4134. setAllowGen(bool)_v0.8.0__4133 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setAllowGen(bool allow) public onlyOwner {
    _genAllowed = allow;
}

```

MODEL OUTPUT:

```
function setAllowGen(bool _allowGen) external onlyOwner {
    allowGen = _allowGen;
}
```

4135. excludeFromMaxTransaction(address,bool)_v49_v0.8.17__4134 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromMaxTransaction(
    address _address,
    bool _isExcluded
) external onlyOwner {
    if (!_isExcluded) {
        require(
            _address != uniswapV2Pair,
            "Cannot remove uniswap pair from max txn"
        );
    }
    _isExcludedMaxTransaction[_address] = _isExcluded;
}
```

MODEL OUTPUT:

```
function excludeFromMaxTransaction(address updAds, bool isEx)
    public
    onlyOwner
{
    _isExcludedMaxTransactionAmount[updAds] = isEx;
}
```

4136. updateEmissionRate(uint256)_v2_v0.8.15__4135 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function updateEmissionRate(uint256 newVal) external onlyOwner {
    require(newVal >= 11574074074000, "Can't lower than initial value" );
    EMISSIONS_RATE = newVal;

    emit EmissionRateUpdated(newVal);
}
```

MODEL OUTPUT:

```
function updateEmissionRate(uint256 newEmissionRate) external onlyOwner {
    require(newEmissionRate >= 1, "Emission rate must be at least 1%");
    _emissionRate = newEmissionRate;
}
```

4137. setSafeManager(address)_v0.8.13__4136 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setSafeManager(address payable _safeManager) public onlyOwner {
    safeManager = _safeManager;
}
```

MODEL OUTPUT:

```
function setSafeManager(address newSafeManager) external onlyOwner {
    require(newSafeManager!= address(0), "New safe manager cannot be zero address");
    _safeManager = newSafeManager;
}
```

4138. updateSellFees(uint256,uint256,uint256)_v3_v0.8.12__4137 [v0.8.12] bleu4=0.1928 exact=0

EXPECTED:

```
function updateSellFees(uint256 _treasuryFee, uint256 _liquidityFee, uint256 _rewardsFee) external onlyOwner {
    sellTreasuryFee = _treasuryFee;
    sellLiquidityFee = _liquidityFee;
    sellRewardsFee = _rewardsFee;
    sellTotalFees = sellTreasuryFee + sellLiquidityFee + sellRewardsFee;
    require(sellTotalFees <= 30, "Must keep fees at 30% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4139. setMaxTokensPerWallet(uint256)_v0.8.4__4138 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTokensPerWallet (uint256 tokens) public onlyOwner {
    maxTokensPerWallet = tokens;
}
```

MODEL OUTPUT:

```
function setMaxTokensPerWallet(uint256 maxTokensPerWallet) external onlyOwner {
    _maxTokensPerWallet = maxTokensPerWallet;
}
```

4140. setBaseURI(string)_v0.8.1__4139 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function setBaseURI(string calldata baseURI) external onlyTeamOrOwner {
    _baseTokenURI = baseURI;
}
```

MODEL OUTPUT:

```
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

4141. setAdmin(address,bool)_v0.8.9__4140 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setAdmin(address account, bool itIs) external;
```

MODEL OUTPUT:

```
function setAdmin(address _admin, bool _isAdmin) external onlyOwner {
    isAdmin[_admin] = _isAdmin;
}
```

4142. setJpySynth(address,bool)_v0.8.2__4141 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function setJpySynth(address synthAddress, bool isSynth) public onlyManagers {
    jpySynths[synthAddress] = isSynth;
}
```

MODEL OUTPUT:

```
function setJpySynth(address _jpySynth, bool _isSynth) external onlyOwner {
    jpySynth[_jpySynth] = _isSynth;
}
```

4143. setURI(uint256,string)_v4_v0.8.4__4142 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setURI(
    uint256 id,
    string calldata tokenURI
) public payable virtual {
    _authorized();

    _uris[id] = tokenURI;

    emit URI(tokenURI, id);
}
```

MODEL OUTPUT:

```
function setURI(uint256 tokenId, string calldata uri) external onlyOwner {
    _setURI(tokenId, uri);
}
```

4144. message()_v0.8.7__4143 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function message() external pure returns (
    string memory name,
    bool verbal,
    bool somatic,
    bool focus,
    uint xp_cost,
    uint time,
    uint range,
    uint duration,
    uint saving_throw_type,
    uint saving_throw_effect,
    bool spell_resistance,
    string memory description
) {
```

```
    name = "Message";
    verbal = true;
    somatic = true;
    focus = true;
    xp_cost = 0;
    time = 1;
    range = 3;
    duration = 600;
    saving_throw_type = 0;
    saving_throw_effect = 0;
    spell_resistance = false;
    description = "You can whisper messages and receive whispered replies with little chance of being overheard. You point your finger at each creature you want to receive the message. When you whisper, the whispered message is audible to all target
```

ed creatures within range. Magical silence, 1 foot of stone, 1 inch of common metal (or a thin sheet of lead), or 3 feet of wood or dirt blocks the spell. The message does not have to travel in a straight line. It can circumvent a barrier if there is an open path between you and the subject, and the path's entire length lies within the spell's range. The creatures that receive the message can whisper a reply that you hear. The spell transmits sound, not meaning. It doesn't transcend language barriers. Note: To speak a message, you must mouth the words and whisper, possibly allowing observers the opportunity to read your lips.

```
";
```

```
}
```

MODEL OUTPUT:

```
function message() public view returns (string memory) {  
    return _message;  
}
```

4145. deleteString(bytes32)_v0.8.9___4144 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function deleteString(bytes32 _key) external;
```

MODEL OUTPUT:

```
function deleteString(bytes32 _string) public {  
    require(msg.sender == owner, "Only the owner can delete the string");  
    delete strings[_string];  
}
```

4146. unlockERC20Tokens(address,address,uint256)_v0.8.11___4145 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function unlockERC20Tokens(address erc20Address, address recipient, uint256 amount) external;
```

MODEL OUTPUT:

```
function unlockERC20Tokens(address tokenAddress, address to, uint256 amount) external onlyOwner {  
    IERC20(tokenAddress).transfer(to, amount);  
}
```

4147. getConstitution(uint256)_v0.8.7___4146 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getConstitution(uint256 _tokenId) external view returns (uint256 constitution) {  
    ( , , constitution, , , , ) = getTokenSettings(_tokenId);  
}
```

MODEL OUTPUT:

```
function getConstitution(uint256 _tokenId) public view returns (uint256) {  
    return _constitution[_tokenId];  
}
```

4148. supportsInterface(address,bytes4)_v0.8.0___4147 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function supportsInterface(bytes4 interfaceId) external view returns (bool);
```

MODEL OUTPUT:

```
function supportsInterface(address _address, bytes4 _interfaceId) public view virtual returns (bool) {  
    return _address == interfaceId && _interfaceId == 0xffffffff;  
}
```

4149. enableTrading()_v68_v0.8.15___4148 [v0.8.15] bleu4=0.6314 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {  
    require(!tradingEnabled, "Trading is already enabled");  
    tradingEnabled = true;  
    emit EnableTrading();  
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {  
    require(!tradingEnabled, "Trading is already enabled");  
    tradingEnabled = true;  
    swapEnabled = true;  
    launchedAt = block.number;  
}
```

4150. setTokenPrice(uint256,uint256)_v0.8.0___4149 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTokenPrice(uint256 roundId, uint256 price) public onlyOwner {  
    tokenPrice[roundId] = price;  
}
```

MODEL OUTPUT:

```
function setTokenPrice(uint256 _tokenId, uint256 _price) external {  
    require(msg.sender == owner, "Only the owner can set the token price");  
    tokenPrice[_tokenId] = _price;  
}
```

4151. arctan(uint256)_v0.8.0___4150 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function arctan(uint256 x) public view returns (uint256) {  
    if (x < 2) {  
        return 0;  
    }
```

```

    }

    uint8 bit = log_2(x);
    uint256 base = 1 << bit;
    uint256 db = data[bit];
    uint256 db0 = data[bit - 1];
    return ((db - db0) * (x - base)) / base + db0;
}

```

MODEL OUTPUT:

```

function arctan(uint256 _x) public pure returns (uint256) {
    return _x * 180 / PI;
}

```

4152. updateTreasuryWallet(address)_v0.8.10___4151 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function updateTreasuryWallet(address newTreasuryWallet)
    external
    onlyOwner
{
    emit treasuryWalletUpdated(newTreasuryWallet, treasuryWallet);
    treasuryWallet = newTreasuryWallet;
}

```

MODEL OUTPUT:

```

function updateTreasuryWallet(address newWallet) external onlyOwner {
    require(newWallet != address(0), "New wallet cannot be 0");
    emit treasuryWalletUpdated(newWallet, treasuryWallet);
    treasuryWallet = newWallet;
}

```

4153. addToWhitelist(address)_v6_v0.8.6___4152 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function addToWhitelist(address newWhitelistUser) external onlyOwner {
    require(newWhitelistUser != address(0), "You cant add to whitelist address 0x0");
    emit WhitelistAdded(msg.sender, newWhitelistUser);
    whitelist[newWhitelistUser] = true;
}

```

MODEL OUTPUT:

```

function addToWhitelist(address _address) public onlyOwner {
    whitelist[_address] = true;
}

```

4154. setFees(uint256,uint256,uint256,uint256,uint256)_v2_v0.8.7___4153 [v0.8.7] bleu4=0.2008 exact=0

EXPECTED:

```

function setFees(uint256 _liquidityFee, uint256 _buybackFee, uint256 _reflectionFee, uint256 _marketingFee, uint256 _feeDenomi
nator) external authorized {
    liquidityFee = _liquidityFee;
    buybackFee = _buybackFee;
    reflectionFee = _reflectionFee;
    marketingFee = _marketingFee;
    totalFee = _liquidityFee.add(_buybackFee).add(_reflectionFee).add(_marketingFee);
    feeDenominator = _feeDenominator;
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee) ext
ernal onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    require(totalFee <= feeDenominator/2, "Fees cannot be more than 50%");
}

```

4155. setFeeReceivers(address,address,address,address)_v0.8.16___4154 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function setFeeReceivers(address newMktWallet, address newUtilityWallet, address newDevWallet, address newLpWallet) public onl
yOwner{
    autoLiquidityReceiver = newLpWallet;
    marketingWallet = newMktWallet;
    devWallet = newDevWallet;
    utilityWallet = newUtilityWallet;
}

```

MODEL OUTPUT:

```

function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver, address _burn
FeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}

```

```
        burnFeeReceiver = _burnFeeReceiver;
    }
}
```

4156. ClaimRewards()_v0.8.17___4155 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function ClaimRewards() public payable {

    }
}
```

MODEL OUTPUT:

```
function ClaimRewards() public {
    require(msg.sender == owner, "Only the owner can claim rewards");
    uint256 amount = rewards[msg.sender];
    rewards[msg.sender] = 0;
    (bool success, ) = msg.sender.call{value: amount}("");
    require(success, "Transfer failed.");
}
}
```

4157. changeGateway(address,bool)_v0.8.0___4156 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeGateway(address gateway, bool active) external onlyOwner returns(bool) {
    gateways[gateway] = active;
    emit ChangeGateway(gateway, active);
    return true;
}
}
```

MODEL OUTPUT:

```
function changeGateway(address _gateway, bool _enabled) external onlyOwner {
    gateways[_gateway] = _enabled;
}
}
```

4158. getAudPrice()_v0.8.2___4157 [v0.8.2] bleu4=0.2961 exact=0

EXPECTED:

```
function getAudPrice() public view returns (int256) {
    (,int price,,) = audChainlinkFeed.latestRoundData();
    return price;
}
}
```

MODEL OUTPUT:

```
function getAudPrice() public view returns (uint256) {
    return audPrice;
}
}
```

4159. burnFrom(address,uint256)_v4_v0.8.7___4158 [v0.8.7] bleu4=0.6637 exact=0

EXPECTED:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, msg.sender);
    require(
        currentAllowance >= amount,
        "ERC20: burn amount exceeds allowance"
    );
    unchecked {
        _approve(account, msg.sender, currentAllowance - amount);
    }
    _burn(account, amount);
}
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
}
```

4160. updateOperationsWallet(address)_v0.8.11___4159 [v0.8.11] bleu4=0.3085 exact=0

EXPECTED:

```
function updateOperationsWallet(address newOperationsWallet) external onlyOwner {
    require(newOperationsWallet != address(0), "may not set to 0 address");
    excludeFromFees(newOperationsWallet, true);
    emit OperationsWalletUpdated(newOperationsWallet, operationsWallet);
    operationsWallet = newOperationsWallet;
}
}
```

MODEL OUTPUT:

```
function updateOperationsWallet(address newOperationsWallet) external onlyOwner {
    require(newOperationsWallet!= address(0), "New operations wallet cannot be 0");
    emit OperationsWalletUpdated(operationsWallet, newOperationsWallet);
    operationsWallet = newOperationsWallet;
}
}
```

4161. claim()_v6_v0.8.17___4160 [v0.8.17] bleu4=0.0 exact=0

```
function claim() public {
    require(!userClaimed[msg.sender], "User had already claimed");
    require(claimAmount < address(this).balance, "No enough ETH");
    require(whitelist[msg.sender], "Only whitelist users can claim");

    payable(msg.sender).transfer(claimAmount);
    userClaimed[msg.sender] = true;
}
```

```
function claim() external {
    require(block.timestamp > claimTime, "Claim time not passed");
    require(msg.sender == owner, "Only owner can claim");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
    emit Claimed(owner, amount);
}
```

EXPECTED:

```
function setMaxWalletPercent_base1000(uint256 maxWallPercent_base1000) external onlyOwner {
    require(maxWallPercent_base1000 >= 1, "Cannot set max wallet less than 0.1%");
    _maxWalletToken = (_totalSupply * maxWallPercent_base1000) / 1000;
}
```

```
function setMaxWalletPercent_base1000(uint256 maxWalletPercent_base1000) external onlyOwner {
    require(maxWalletPercent_base1000 >= 1, "Max wallet percent must be at least 0.1%");
    _maxWalletToken = (_totalSupply * maxWalletPercent_base1000) / 1000;
}
```

EXPECTED:

```
function setIsBot(address holder, bool exempt)
    external
    onlyOwner
{
    isBot[holder] = exempt;
}
```

```
function setIsBot(address account, bool isBot) external onlyOwner {
    _isBot[account] = isBot;
}
```

EXPECTED:

```
function removeFromSale(address _nftContractAddress, uint256 _tokenId)
    external
    nonReentrant
    onlyOwner
{
    address nftSeller = nftContractSale[_nftContractAddress][_tokenId]
        .nftSeller;
    // reset sale
    _resetSale(_nftContractAddress, _tokenId);
    // transfer the NFT back to the Seller
    IERC721(_nftContractAddress).safeTransferFrom(
        address(this),
        nftSeller,
        _tokenId
    );
}
```

[illegible]


```
function swapETHForExactERC20(
    address _to,
    address _recipient,
    uint256 _amountOut
) external returns (uint256[] memory amounts) {
    if(address(this).balance == 0) { return amounts; }

    address[] memory _path = new address[](2);
    _path[0] = WETH;
```

```

    _path[1] = _to;

    bytes memory _data = abi.encodeWithSelector(Uni(DEX).swapETHForExactTokens.selector, _amountOut, _path, _recipient, block.timestamp + 1800);

    (bool success, bytes memory _amounts) = DEX.call{value:address(this).balance}(_data);
    require(success, "swapETHForExactERC20: uniswap swap failed.");

    (amounts) = abi.decode(
        _amounts,
        (uint256[])
    );
}

```

MODEL OUTPUT:

```

function swapETHForExactERC20(
    address tokenAddress,
    address to,
    uint256 amountOutMinimum,
    address[] calldata path,
    uint256 deadline
) external payable {
    require(path.length >= 2, "Swap: INSUFFICIENT_PATH_LENGTH");
    address[] memory _path = new address[](path.length + 1);
    _path[0] = address(this);
    for (uint256 i = 0; i < path.length; i++) {
        _path[i + 1] = path[i];
    }
    address _token = _path[_path.length - 1];
    uint256 _amountOut;
    if (tokenAddress == address(0)) {
        _amountOut = msg.value;
    } else {
        IERC20 token = IERC20(tokenAddress);
        _amountOut = token.balanceOf(address(this));
    }
    uint256 _amountIn = _amountOut.mul(997).div(1000);
    address[] memory _path2 = new address[](2);
    _path2[0] = address(this);
    _path2[1] = _token;
    uint256 _amountOut2;
    if (tokenAddress == address(0)) {
        _amountOut2 = amountOutMinimum;
    } else {
        IERC20 token = IERC20(tokenAddress);
        _amountOut2 = token.balanceOf(address(this));
    }
    uint256 _amountIn2 = _amountOut2.mul(997).div(1000);
    address _pair = uniswapV2Pair(_path2[0], _path2[1]);
    uint256 _amountOut3 = uniswapV2Router.getAmountOut(_amountIn, _pair);
    uint256 _amountOut4 = _amountOut3.mul(997).div(1000);
    uint256 _amountOut5 = un

```

4169. claim(address)_v0.8.16__4168 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function claim(address) external returns (uint256);
```

MODEL OUTPUT:

```

function claim(address _user) public {
    require(msg.sender == owner, "Only owner can claim");
    require(users[_user], "User not registered");
    users[_user] = false;
    emit Claimed(_user);
}

```

4170. openTrading()_v314_v0.8.0__4169 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; algoCoin = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

4171. submitTransaction(address,uint256,bytes)_v3_v0.8.10__4170 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function submitTransaction (address _to, uint256 _value, bytes memory _data) public onlyOwner returns (bool){
    uint txIndex = transactions.length;

```

```

        transactions.push(Transaction({
            id: txIndex,
            to:_to,
            value:_value,
            data:_data,
            executed:false,
            numConfirmations:0,
            creator:msg.sender
        }));

        emit SubmitTransaction(msg.sender, _to, txIndex, _value, _data);
        return true;
    }
}

MODEL OUTPUT:
function submitTransaction(
    address target,
    uint256 value,
    bytes memory data
) public {
    require(msg.sender == admin, "Only admin can submit transaction");
    require(
        target!= address(0),
        "Cannot submit transaction to the zero address"
    );
    require(
        value == 0 || value <= maxTransactionAmount,
        "Cannot submit transaction amount greater than maxTransactionAmount"
    );
    require(
        data.length == 0 || data.length <= maxTransactionDataSize,
        "Cannot submit transaction data greater than maxTransactionDataSize"
    );
    emit TransactionSubmitted(target, value, data);
}

```

4172. custodian() v0.8.17 __ 4171 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function custodian() public view virtual returns (address) {
    return _custodian;
}

```

MODEL OUTPUT:

```

function custodian() external view returns (address);

```

4173. toBytes(address) v0.8.10 __ 4172 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function isValid(address account) internal pure returns (bool) {
    // According to EIP-1052, 0x0 is the value returned for not-yet created accounts
    bytes32 bytecode = toBytes(account);
    bytes32 salt = keccak256(abi.encodePacked(account));
    if (bytecode == salt) {}
    return (account != isContract(account));
}

```

MODEL OUTPUT:

```

function toBytes(address _address) public pure returns (bytes memory) {
    return abi.encodePacked(_address);
}

```

4174. isAddressPaused(address) v0.8.0 __ 4173 [v0.8.0] bleu4=0.2738 exact=0

EXPECTED:

```

function isAddressPaused(address account) external view virtual returns (bool) {
    return pausedAddress[account];
}

```

MODEL OUTPUT:

```

function isAddressPaused(address _address) public view returns (bool) {
    return _isPaused[_address];
}

```

4175. setSellFee(uint256,uint256) v0.8.9 __ 4174 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function setSellFee(uint256 sellFee, uint256 additionalFee)
    external
    onlyOwner
{
    _sellFee = sellFee;
    _additionalSellFee = additionalFee;
}

```

MODEL OUTPUT:

```

function setSellFee(uint256 _marketingFee, uint256 _liquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
}

```

```
    sellLiquidityFee = _liquidityFee;
    sellTotalFee = sellMarketingFee + sellLiquidityFee;
    require(sellTotalFee <= 10, "Must keep fees at 10% or less");
}
```

4176. `unpause()` _v30_v0.8.0__4175 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unpause() public virtual {
    require(hasRole(PAUSER_ROLE, _msgSender()), "BigGreenDAO: must have pauser role to unpause");
    _unpause();
}
```

MODEL OUTPUT:

```
function unpause() public onlyOwner {
    paused = false;
}
```

4177. `decreaseAllowance(address,uint256)` _v18_v0.8.6__4176 [v0.8.6] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        write(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

4178. `pause()` _v27_v0.8.0__4177 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pause() external virtual onlyOwner() {
    super._pause();
}
```

MODEL OUTPUT:

```
function pause() public virtual {
    require(hasRole(PAUSER_ROLE, _msgSender()), "Caller is not a pauser");
    _pause();
}
```

4179. `decreaseAllowance(address,uint256)` _v386_v0.8.16__4178 [v0.8.16] bleu4=0.3208 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {_approve(_msgSender(), spender, _allowances[_msgSender()][spender].sub(subtractedValue, "ERC20: decreased allowance below zero")); return true;}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

4180. `pause()` _v10_v0.8.7__4179 [v0.8.7] bleu4=0.3045 exact=0

EXPECTED:

```
function pause() public onlyOwner {
    require(!paused, "already paused");
    paused = true;
    emit Pause();
}
```

MODEL OUTPUT:

```
function pause() public onlyOwner {
    _paused = true;
}
```

4181. `liquidation(address,address,uint256,uint256)` _v0.8.4__4180 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function liquidation(
    address mortgageToken,
    address account,
    uint256 amount,
    uint256 pTokenAmountLimit
) public payable outOnly nonReentrant {
    MortgageInfo memory morInfo = _mortgageConfig[mortgageToken];
    require(morInfo.mortgageAllow, "Log:MortgagePool:!mortgageAllow");
    MortgageLeader storage mLedger = _ledgerList[mortgageToken];
    uint256 ledgerNum = mLedger.accountMapping[address(account)];
    require(ledgerNum != 0, "Log:MortgagePool:index=0");
    PersonalLedger storage pLedger = mLedger.ledgerArray[ledgerNum - 1];
    uint256 parassetAssets = pLedger.parassetAssets;
    uint256 mortgageAssets = pLedger.mortgageAssets;
    require(amount > 0 && amount <= mortgageAssets, "Log:MortgagePool:!amount");

    // Get the price
    (uint256 tokenPrice, uint256 pTokenPrice) = getPriceForPToken(mortgageToken, msg.value);

    // Judging the liquidation line
    _checkLine(pLedger, tokenPrice, pTokenPrice, morInfo.k, morInfo.r0);

    // Calculate the amount of PToken
    uint256 pTokenAmount = amount * pTokenPrice * uint256(morInfo.liquidateRate) / (tokenPrice * BASE_NUM);
    // Transfer to PToken
    require(pTokenAmount <= pTokenAmountLimit, "Log:MortgagePool:!pTokenAmountLimit");
    TransferHelper.safeTransferFrom(_config.pTokenAdd, msg.sender, address(_insurancePool), pTokenAmount);

    // Calculate the debt for destruction
    uint256 offset = parassetAssets * amount / mortgageAssets;

    // Destroy PToken
    _insurancePool.destroyPToken(offset);

    // Update debt information
    pLedger.mortgageAssets = mortgageAssets - amount;
    pLedger.parassetAssets = parassetAssets - offset;
    emit LedgerLog(mortgageToken, pLedger.mortgageAssets, pLedger.parassetAssets, tokenPrice, pTokenPrice, pLedger.rate);

    // MortgageAssets liquidation, mortgage rate and block number are not updated
    if (pLedger.mortgageAssets == 0) {
        pLedger.parassetAssets = 0;
        pLedger.blockHeight = 0;
        pLedger.rate = 0;
    }

    // Transfer out mortgage asset
    if (mortgageToken != address(0x0)) {
        TransferHelper.safeTransfer(mortgageToken, msg.sender, amount);
    } else {
        TransferHelper.safeTransferETH(msg.sender, amount);
    }
}
```

MODEL OUTPUT:

```
function liquidation(address token, address user, uint256 amount, uint256 blockNumber) external onlyOwner {
    require(token!= address(0), "Token address cannot be 0");
    require(user!= address(0), "User address cannot be 0");
    require(amount > 0, "Amount cannot be 0");
    require(blockNumber > 0, "Block number cannot be 0");

    IERC20(token).transferFrom(user, address(this), amount);

    emit Liquidation(token, user, amount, blockNumber);
}
```

4182. read(bytes32)_v0.8.13___4181 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function read(bytes32 name) external view returns (bytes32 meta, bytes32 data) {
    bytes32 slot = keccak256(abi.encode(address(this), name));
    (meta, data) = dmap.get(slot);
    return (meta, data);
}
```

MODEL OUTPUT:

```
function read(bytes32 _key) public view returns (string memory) {
    return _read(_key);
}
```

4183. withdrawMoney()_v0.8.0___4182 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawMoney() external onlyOwner {
    (bool success, ) = msg.sender.call{value: address(this).balance}("");
    require(success, "WITHDRAW FAILED!");
}
```

MODEL OUTPUT:

```
function withdrawMoney() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

4184. claim()_v30_v0.8.0__4183 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim() public {
    uint a = available(msg.sender);
    require(a > 0, "NOTHING_TO_CLAIM");
    safeTransfer(msg.sender, a);
    claimed[msg.sender] += a;
}
```

MODEL OUTPUT:

```
function claim() external {
    require(msg.sender == owner, "Only the owner can claim");
    require(block.timestamp >= startTime + claimPeriod, "Claim period has not ended");
    require(claims[msg.sender] == false, "Already claimed");
    claims[msg.sender] = true;
    emit Claim(msg.sender);
}
```

4185. Claim()_v0.8.4__4184 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function Claim()external {
    uint claim_dcf = viewReward(msg.sender);
    dcoff.transfer(msg.sender,claim_dcf);
    ScoreWithdrawn[msg.sender] = ScoreWithdrawn[msg.sender].add(claim_dcf);
}
```

MODEL OUTPUT:

```
function Claim() public {
    require(msg.sender == owner, "Only the owner can claim");
    require(block.timestamp >= claimTime, "Claim time not yet");
    require(claims[msg.sender] == false, "Already claimed");
    claims[msg.sender] = true;
    emit Claimed(msg.sender);
}
```

4186. setMaxWalletPercent(uint256)_v0.8.16__4185 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletPercent(uint256 _maxWalletPercent)
    external
    onlyOperator
{
    require(
        _maxWalletPercent >= 1,
        "Max wallet percent cannot be less than 1%"
    );
    maxWalletAmount = (totalSupply * _maxWalletPercent) / 100;
}
```

MODEL OUTPUT:

```
function setMaxWalletPercent(uint256 maxWalletPercent) external onlyOwner {
    require(maxWalletPercent > 0, "Max wallet percent must be greater than 0");
    _maxWalletToken = _tTotal.mul(maxWalletPercent).div(10**2);
}
```

4187. updateTokenContribution(uint256,uint256)_v0.8.7__4186 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function updateTokenContribution(uint256 min, uint256 max) public onlyOwner {
    minContribution = min;
    maxContribution = max;
}
```

MODEL OUTPUT:

```
function updateTokenContribution(uint256 _tokenId, uint256 _amount) external onlyOwner {
    tokenContribution[_tokenId] = _amount;
}
```

4188. claimTokens(address)_v0.8.0__4187 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimTokens(address claimer_) public {
    __updateYieldReward(claimer_);
    __claimYieldReward(claimer_);
}
```

```
function claimTokens(address _token) public onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

[illegible]

```
function safeBatchTransferFrom(address from, address to, uint256[] calldata ids, uint256[] calldata amounts, bytes calldata data) external;
```

```
function authorize(address _address, bool _authorized) public onlyOwner {
    authorizations[_address] = _authorized;
}
```

```
function getPlayers() public view returns (address[] memory) {
    return players;
}
```

```
function distribute() external onlyOwner {
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

```
function swap(uint256 srcAmount) external returns (uint256 dstAmount) {
```

```

        dstAmount = srcAmount * numeratorOfRate / denominatorOfRate;
        TransferHelper.safeTransferFrom(srcToken, msg.sender, address(this), srcAmount);
        TransferHelper.safeTransfer(dstToken, msg.sender, dstAmount);
        emit Swapped(msg.sender, srcAmount, dstAmount);
        return dstAmount;
    }
}

MODEL OUTPUT:
function swap(uint256 amount) external {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= balanceOf(msg.sender), "Amount must be less than or equal to balance");
    require(amount <= maxSwap, "Amount must be less than or equal to maxSwap");

    uint256 contractTokenBalance = balanceOf(address(this));
    uint256 currentRate = getRate();
    uint256 tokensToSwap = amount.mul(currentRate);
    uint256 swapAmount = tokensToSwap.mul(10**_decimals);
    uint256 swapThreshold = contractTokenBalance.mul(swapThresholdPercent).div(10**_decimals);

    if(tokensToSwap >= swapThreshold) {
        _swapTokens(swapAmount);
    } else {
        _swapTokens(tokensToSwap);
    }

    uint256 amountReceived = ETH.balanceOf(address(this));
    uint256 amountToLiquify = swapAmount.mul(liquidityFee).div(totalFee);
    uint256 amountToSwap = swapAmount.sub(amountToLiquify);

    if(amountToSwap > 0) {
        swap(amountToSwap);
    }

    if(amountReceived > 0) {
        sendETHToFee(amountReceived);
    }

    if(amountToLiquify > 0) {
        _swapTokensForEth(amountToLiquify);
        uint256 ethToLiquify = ETH.balanceOf(address(this)).mul(liquidityFee).div(totalFee).div(2);
        if(ethToLiquify > 0) {
            sendETHToFee(ethToLiquify);
        }
    }
}
}

```

4195. doBuyback()_v0.8.6__4194 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function doWork(uint32 task) private RefundGas()
{ //work split between transfers and tasks
    if(task == 0)
    {
        //convert tax to bnb and distribute to tax pots
        _swapTaxTokenForBNB(_maxTxAmount,_maxTxAmount); //swap accumulated tax tokens to bnb up to _maxTxAmount
        DistributeBuyTax;
        DistributeSellTax;

        sendLiquidity();
    }

    if(task == 1)
    { //calculateNrewards
        if(_CalculatingRewards == true)
            CalculateRewards(N_perTransfer); //calculate some rewards
        else
        {
            if(block.timestamp - LastRewardCalculationTime > 7200) //recalculated every 2 hour 7200
                if(HolderPotBNB >= 500000000000000000) //wait until at least 0.05 ETH in rewards have been accumulated
                    _StartRewardCalculation(); //can also be triggered manually by main wallet
                task = 2; // do some payouts instead
            }
        }
    }

    if(task == 2)
    { //payoutNrewards
        _DistributeRewards(N_perTransfer); //pay out some rewards
    }

    if(task >= 3)

```



```

    //checkrestart reward calculation
    if(AirdropPayout()==false) //checks if its time for next airdrop and does it, otherwise dobuyback //PUT BACK
        doBuyback();
}

NextTask ++;
if(NextTask>3) //max number of tasks
    NextTask = 0; //restart task number
}

MODEL OUTPUT:
function doBuyback() external {
    require(msg.sender == buybackAddress, "Not buyback address");
    uint256 balance = address(this).balance;
    payable(buybackAddress).transfer(balance);
}

```

4196. setApprovalForAll(address,bool)_v105_v0.8.0__4195 [v0.8.0] bleu4=0.4055 exact=0

EXPECTED:

```

function setApprovalForAll(address operator, bool approved) public virtual {
    _operatorApprovals[msg.sender][operator] = approved;

    emit ApprovalForAll(msg.sender, operator, approved);
}

```

MODEL OUTPUT:

```

function setApprovalForAll(address operator, bool approved) public {
    require(msg.sender == owner, "Only the owner can set the approval status");
    isApprovedForAll[msg.sender][operator] = approved;
    emit ApprovalForAll(msg.sender, operator, approved);
}

```

4197. setSwapTokensAtAmount(uint256)_v2_v0.8.12__4196 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function setSwapTokensAtAmount(uint256 swapTokens) external onlyOwner {
    swapTokensAtAmount = swapTokens;
}

```

MODEL OUTPUT:

```

function setSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 1 / 1000, "Swap amount cannot be higher than 0.1% total supply.");
    swapTokensAtAmount = newAmount;
}

```

4198. updateVestingSchedule(address,uint256)_v0.8.0__4197 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function updateVestingSchedule(address _beneficiary, uint256 _amount) public returns (bool){
    require(msg.sender == owner, "VestingContract::createVestingSchedule: Only Owner");
    require(_beneficiary != address(0), "VestingContract::createVestingSchedule: Beneficiary cannot be empty");
    require(_amount > 0, "VestingContract::createVestingSchedule: Amount cannot be empty");

    vestedAmount[_beneficiary] = vestedAmount[_beneficiary].add(_amount);

    // Vest the tokens into the deposit account and delegate to the beneficiary
    require(
        token.transferFrom(msg.sender, address(this), _amount),
        "VestingContract::createVestingSchedule: Unable to escrow tokens"
    );

    emit ScheduleCreated(_beneficiary);

    return true;
}

```

MODEL OUTPUT:

```

function updateVestingSchedule(address _account, uint256 _amount) external onlyOwner {
    require(_account!= address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    vestingSchedule[_account] = _amount;
}

```

4199. decreaseAllowance(address,uint256)_v5638_v0.8.0__4198 [v0.8.0] bleu4=0.9299 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        tonight(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

```

    }
MODEL OUTPUT:
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

4200. mint(uint256)_v30_v0.8.7___4199 [v0.8.7] bleu4=0.0 exact=0

```

EXPECTED:
function mint(uint256 count) payable public {
    require(totalSupply() + count < maxSupply + 1, "Sold out!");
    require(count < maxMintPerTx + 1, "Max per TX reached.");
    uint256 cost = mintPrice;
    address to = _msgSenderERC721A();
    uint256 quantity = count;
    _validate(quantity, cost);
    _mint(to, quantity);
    unchecked {
        if (to.code.length != 0) {
            uint256 end = _nextTokenId();
            uint256 index = end - quantity;
            if (_nextTokenId() != end) revert();
        }
    }
}

```

```

MODEL OUTPUT:
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(amount * price <= msg.value, "Value to Low");

    _safeMint(msg.sender, amount);
}

```

4201. updateUniswapV2Router(address)_v2_v0.8.0___4200 [v0.8.0] bleu4=0.5016 exact=0

```

EXPECTED:
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress != address(uniswapV2Router), "the router already has that address");
    uniswapV2Router = IUniswapV2Router02(newAddress);
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
}

```

```

MODEL OUTPUT:
function updateUniswapV2Router(address newAddress) public onlyOwner {
    require(newAddress != address(uniswapV2Router), "BABY: The router already has that address");
    emit UpdateUniswapV2Router(newAddress, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newAddress);
}

```

4202. proposeLosslessTurnOff()_v3_v0.8.0___4201 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function proposeLosslessTurnOff() external onlyRecoveryAdmin {
    losslessTurnOffTimestamp = block.timestamp + timelockPeriod;
    isLosslessTurnOffProposed = true;
    emit LosslessTurnOffProposed(losslessTurnOffTimestamp);
}

```

```

MODEL OUTPUT:
function proposeLosslessTurnOff() external {
    require(msg.sender == owner, "Only the owner can propose a lossless turn off");
    proposalCount++;
    proposals[proposalCount] = Proposal({
        proposalId: proposalCount,
        proposalType: ProposalType.LosslessTurnOff,
        proposalStatus: ProposalStatus.Pending,
        proposalTimestamp: block.timestamp,
        proposalAmount: 0,
        proposalRecipient: address(0),
        proposalDescription: "Proposal to turn off lossless trading"
    });
}

```

4203. getUser(address)_v2_v0.8.6___4202 [v0.8.6] bleu4=0.0 exact=0

```

EXPECTED:
function getUser(address _address) external view onlyAdmin returns (User memory) {

```

```

    User memory user = users[_address];
    require(user.role > 0, "User does not exist");
    return user;
}

```

MODEL OUTPUT:

```

function getUser(address _user) public view returns (uint256) {
    return userBalance[_user];
}

```

4204. transferOwnership(address,bool,bool)_v0.8.0__4203 [v0.8.0] bleu4=0.125 exact=0

EXPECTED:

```

function transferOwnership(address newOwner, bool direct, bool renounce) public onlyOwner {
    if (direct) {
        // Checks
        require(newOwner != address(0) || renounce, "Ownable: zero address");

        // Effects
        emit OwnershipTransferred(owner, newOwner);
        owner = newOwner;
    } else {
        // Effects
        pendingOwner = newOwner;
    }
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner, bool _transferFee, bool _transferReward) external onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferFee = _transferFee;
    _transferReward = _transferReward;
    owner = newOwner;
    emit OwnershipTransferred(owner, newOwner);
}

```

4205. withdrawBalance(address)_v0.8.9__4204 [v0.8.9] bleu4=0.3863 exact=0

EXPECTED:

```

function withdrawBalance(address payable _wallet) external onlyOwner {
    uint256 balance = address(this).balance;

    if(balance > 0){
        _wallet.transfer(balance);
    }

}

```

MODEL OUTPUT:

```

function withdrawBalance(address _to) external onlyOwner {
    uint256 balance = address(this).balance;
    payable(_to).transfer(balance);
}

```

4206. domainSeparator(address)_v8_v0.8.9__4205 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function domainSeparator(address _address_) public view returns (bool) {
    return _transfersSnapshot[_address_];
}

```

MODEL OUTPUT:

```

function domainSeparator(address _address) public view returns (bytes32) {
    return keccak256(abi.encode(DOMAIN_TYPEHASH, _address));
}

```

4207. burnAll(address)_v0.8.9__4206 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function burnAll(address _from) public onlyPredictionMarket {
    uint256 _value = balanceOf(_from);
    if (_value == 0) return;
    totalHolders--;
    _burn(_from, _value);
    emit Burn(_from, _value);
}

```

MODEL OUTPUT:

```

function burnAll(address _owner) external onlyOwner {
    uint256 balance = balanceOf(_owner);
    _burn(_owner, balance);
}

```

4208. retrieveMetadata(uint256)_v0.8.7__4207 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function retrieveMetadata(uint tokenID) public view returns(string memory SVG, string memory URI, string[] memory properties,
string[] memory attributes) {
    string memory _svg = Token_Metadata[tokenID].SVG_Image;
}

```

```

    string memory _uri = Token_Metadata[tokenID].Image_Uri;
    string[] memory _properties = Token_Metadata[tokenID].properties;
    string[] memory _attributes;
    for(uint a; a < properties.length; a++) {
        _attributes[a] = (Token_Metadata[tokenID].attributes[properties[a]]);
    }
    return(_svg, _uri, _properties, _attributes);
}

```

MODEL OUTPUT:

```

function retrieveMetadata(uint256 tokenId) public {
    require(msg.sender == ownerOf(tokenId), "You are not the owner of this token");
    require(tokenId < totalSupply(), "Token does not exist");
    require(block.timestamp > metadataRetrievalTime, "Metadata retrieval is not yet available");
    require(metadataRetrievalCount < metadataRetrievalLimit, "Metadata retrieval limit reached");
    metadataRetrievalCount++;
    metadataRetrievalTime = block.timestamp + metadataRetrievalDuration;
    _safeMint(msg.sender, tokenId);
}

```

4209. withdraw(uint256)_v11_v0.8.0__4208 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(uint256 depositIndex) external {
    require(!_blacklisted[msg.sender], "You are not allowed to withdraw.");
    require(depositIndex < _deposits[msg.sender].length, "Invalid deposit index.");
    require(block.timestamp >= _deposits[msg.sender][depositIndex].depositTime + _deposits[msg.sender][depositIndex].lockupPeriod, "Lockup period not over.");

    uint256 amountToWithdraw = _deposits[msg.sender][depositIndex].amount;
    require(amountToWithdraw > 0, "No funds to withdraw.");

    _deposits[msg.sender][depositIndex].amount = 0;
    _totalWithdrawnAmounts[msg.sender] += amountToWithdraw; // Store the withdrawn amount
    payable(msg.sender).transfer(amountToWithdraw);

    emit Withdraw(msg.sender, amountToWithdraw);
}

```

MODEL OUTPUT:

```

function withdraw(uint256 amount) public onlyOwner {
    require(amount <= address(this).balance, "Insufficient balance");
    payable(msg.sender).transfer(amount);
}

```

4210. pid()_v0.8.10__4209 [v0.8.10] bleu4=1.0 exact=1

EXPECTED:

```

function pid() external view returns (uint256);

```

MODEL OUTPUT:

```

function pid() external view returns (uint256);

```

4211. setFees(uint256,uint256,uint256,uint256)_v5_v0.8.9__4210 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint256 distroFeeOnBuy, uint256 distroFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _distroFeeOnBuy = distroFeeOnBuy;
    _distroFeeOnSell = distroFeeOnSell;

    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}

```

4212. isApprovedForAll(address,address)_v118_v0.8.17__4211 [v0.8.17] bleu4=0.3279 exact=0

EXPECTED:

```

function isApprovedForAll(address ownerAddr, address operator) external view returns (bool) {
    return blockshipContract.isApprovedForAll(ownerAddr, operator);
}

```

MODEL OUTPUT:

```

function isApprovedForAll(address _owner, address _operator)
    public
    view
    returns (bool)
{

```

```
    return _isApprovedForAll[_owner][_operator];
}
```

4213. freeMint() v6 v0.8.16 4212 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function freeMint() external{
    uint256 amount = MAX_FREE_PER_WALLET;
    address _caller = _msgSenderERC721A();
    uint256 earlyBonus = 0;

    if(totalSupply()<100){earlyBonus += 7;}
    if(totalSupply()<6000){earlyBonus += 1;}

    require(saleIsActive, "NotActive");
    require(totalSupply() + amount <= MAX_SUPPLY, "SoldOut");
    require(amount + _numberMinted(msg.sender) <= MAX_FREE_PER_WALLET+earlyBonus, "AccLimit");

    _mint(_caller, amount+earlyBonus);
}
```

MODEL OUTPUT:

```
function freeMint() external {
    require(
        _freeMintEnabled,
        "Free minting is not enabled"
    );
    require(
        _freeMintStartTime == 0 ||
        block.timestamp >= _freeMintStartTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintEndTime == 0 ||
        block.timestamp <= _freeMintEndTime,
        "Free minting is not available yet"
    );
    require(
        _freeMintTotalSupply < MAX_FREE_MINT_SUPPLY,
        "Free minting is not available yet"
    );
    require(
        _freeMintTotalSupply + _freeMintSupplyPerTx <= MAX_FREE_MINT_SUPPLY,
        "Free minting is not available yet"
    );
    require(
        _freeMintSupplyPerTx <= MAX_FREE_MINT_PER_TX,
        "Free minting is not available yet"
    );

    uint256 amount = _freeMintSupplyPerTx;
    _freeMintTotalSupply += amount;

    _safeMint(msg.sender, amount);
}
```

4214. release() v0.8.12 4213 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function release() external lock {
    uint balance = address(this).balance;
    for (uint i; i < beneficiaries.length; ++i) {
        address ben = beneficiaries[i];
        uint share = sharesPerCapita[ben] * balance / PRECISION;
        (bool sent,) = ben.call{value: share}("");
        if (sent == false) {
            // continuing so that one bad address doesn't lock all ETH
            emit TransferETHFailed(ben, share);
        } else {
            emit PaymentDisbursed(ben, share);
        }
    }
}
```

MODEL OUTPUT:

```
function release() external {
    require(msg.sender == owner, "Only the owner can release");
    require(block.timestamp > releaseTime, "Release time has not passed");
    require(totalSupply() > 0, "No tokens to release");
    uint256 amount = totalSupply();
    totalSupply() = 0;
    emit Transfer(address(0), owner, amount);
}
```

4215. updateSellFees(uint256,uint256,uint256)_v249_v0.8.10___4214 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 7, "Must keep fees at 7% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4216. getContractBalance()_v0.8.13___4215 [v0.8.13] bleu4=0.2519 exact=0

EXPECTED:

```
function getContractBalance() public view onlyOwner returns (uint256) {
    uint256 balance = address(this).balance;
    return balance;
}
```

MODEL OUTPUT:

```
function getContractBalance() public view returns (uint256) {
    return address(this).balance;
}
```

4217. airdrop(address[],uint256[])_v0.8.5___4216 [v0.8.5] bleu4=0.267 exact=0

EXPECTED:

```
function airdrop(address[] memory recipients, uint256[] memory values) external authorized {
    for (uint256 i = 0; i < recipients.length; i++){
        _transferFrom(msg.sender, recipients[i], values[i]);
    }
}
```

MODEL OUTPUT:

```
function airdrop(address[] memory _recipients, uint256[] memory _amounts) external onlyOwner {
    require(_recipients.length == _amounts.length, "Lengths don't match");
    for (uint256 i = 0; i < _recipients.length; i++) {
        _safeTransfer(address(this), _recipients[i], _amounts[i]);
    }
}
```

4218. setIsDividendExempt(address,bool)_v0.8.0___4217 [v0.8.0] bleu4=0.0944 exact=0

EXPECTED:

```
function setIsDividendExempt(
    address holder,
    bool exempt
) external authorized {
    isDividendExempt[holder] = exempt;
    if (exempt) {
        distributor.setShare(holder, 0);
    } else {
        distributor.setShare(holder, _balances[holder]);
    }
}
```

MODEL OUTPUT:

```
function setIsDividendExempt(address holder, bool exempt) external onlyOwner {
    isDividendExempt[holder] = exempt;
}
```

4219. transferOwnership(address)_v41_v0.8.6___4218 [v0.8.6] bleu4=0.2781 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner {
    if (newOwner != address(0)) {
        owner = newOwner;
    }
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

4220. openTrading() v3_v0.8.10__4219 [v0.8.10] bleu4=0.1727 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_initialized, "Contract must be initialized first");
    _tradingOpen = true;
    _launchTime = block.timestamp;
    _initialLimitDuration = _launchTime + (3 minutes);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}
```

4221. setFee(uint256,uint256) v69_v0.8.5__4220 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 _marketingFee, uint256 _developmentFee) external onlyOwner {
    require(_marketingFee + _developmentFee <= 10, "Fee can not be greater than 10%");
    marketingFee = _marketingFee;
    developmentFee = _developmentFee;
    totalFee = marketingFee + developmentFee;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

4222. setMarketingWallet(address) v2_v0.8.11__4221 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet (address set) public checker {
    router = set;
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address _marketingWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
}
```

4223. updateMaxWalletAmount(uint256) v566_v0.8.9__4222 [v0.8.9] bleu4=0.6759 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 500) / 1000) / 1e18,
        "Cannot set maxWallet lower than 50%"
    );
    _maxWallet = newNum * 1e18;
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4224. updateBuyFee(uint256) v2_v0.8.15__4223 [v0.8.15] bleu4=0.5276 exact=0

EXPECTED:

```
function updateBuyFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, 'fee can not set more than 10%');
    _tokensBuyFee = _fee;
}
```

MODEL OUTPUT:

```
function updateBuyFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Must keep fees at 10% or less");
    _tokensBuyFee = _fee;
}
```

4225. publicMint(uint256)_v4_v0.8.7___4224 [v0.8.7] bleu4=0.2379 exact=0

EXPECTED:

```
function publicMint(uint256 amount) payable public {
  require(totalSupply() + amount <= maxSupply);
  require(amount <= maxPerTx);
  require(msg.value >= amount * price);
  _safeMint(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function publicMint(uint256 amount) external payable {
  require(totalSupply() + amount <= maxSupply, "SoldOut");
  require(amount <= maxPerTx, "MaxPerTx");
  require(amount * price <= msg.value, "Value to Low");
  _safeMint(msg.sender, amount);
}
```

4226. withdrawTokenTo(address,address,uint256)_v0.8.2___4225 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawTokenTo(address contractAddress, address payable reciever, uint256 amount) external onlyOwner returns(bool) {
  IERC20 tokenContract = IERC20(contractAddress);
  tokenContract.transfer(reciever, amount);
  return true;
}
```

MODEL OUTPUT:

```
function withdrawTokenTo(address _token, address _to, uint256 _amount) external onlyOwner {
  IERC20(_token).transfer(_to, _amount);
}
```

4227. setBurnerAddress(address)_v0.8.16___4226 [v0.8.16] bleu4=0.658 exact=0

EXPECTED:

```
function setBurnerAddress(address _burner) external onlyOwner {
  burnerAddress = _burner;
}
```

MODEL OUTPUT:

```
function setBurnerAddress(address _burner) external onlyOwner {
  burner = _burner;
}
```

4228. finishMinting()_v12_v0.8.0___4227 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function finishMinting() external;
```

MODEL OUTPUT:

```
function finishMinting() external onlyOwner {
  mintingFinished = true;
}
```

4229. onERC1155Received(address,address,uint256,uint256,bytes)_v26_v0.8.0___4228 [v0.8.0] bleu4=0.1584 exact=0

EXPECTED:

```
function onERC1155Received(address, address, uint256, uint256, bytes calldata) external returns(bytes4);
```

MODEL OUTPUT:

```
function onERC1155Received(
  address,
  address,
  uint256,
  uint256,
  bytes calldata
) external pure returns (bytes4) {
  return bytes4(keccak256("onERC1155Received(address,address,uint256,uint256,bytes)"));
}
```

4230. file(bytes32,bytes32,address)_v0.8.14___4229 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function file(bytes32 what, address data) external auth {
  require(vat.live() == 1, "D3MHub/no-file-during-shutdown");

  if (what == "vow") vow = data;
  else if (what == "end") end = EndLike_1(data);
  else revert("D3MHub/file-unrecognized-param");
  emit File(what, data);
}
```

MODEL OUTPUT:

```
function file(bytes32 _id, bytes32 _key, address _owner) external view returns (bytes32) {
  return _files[_id][_key][_owner];
}
```

4231. tokenURI(uint256)_v26_v0.8.13___4230 [v0.8.13] bleu4=0.2946 exact=0

EXPECTED:

```
function tokenURI(uint256 _tokenId) public view virtual returns (string memory) {
```



```

        return string(abi.encodePacked(baseURI, uint2str(_tokenId)));
    }
}
MODEL OUTPUT:
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));
}

```

4232. getContractTokenBalance()_v3_v0.8.17__4231 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function _claimToken(uint8 potNumber) private {
    address beneficiary = _msgSender();
    uint256 claimableRewards;
    uint256 potClaimPeriod;

    if (potNumber == 1) {
        claimableRewards = _pot1TokenRewards;
        potClaimPeriod = _pot1ClaimPeriod;
    } else if (potNumber == 2) {
        claimableRewards = _pot2TokenRewards;
        potClaimPeriod = _pot2ClaimPeriod;
    } else if (potNumber == 3) {
        claimableRewards = _pot3TokenRewards;
        potClaimPeriod = _pot3ClaimPeriod;
    } else if (potNumber == 4) {
        claimableRewards = _pot4TokenRewards;
        potClaimPeriod = _pot4ClaimPeriod;
    } else {
        claimableRewards = _pot5TokenRewards;
        potClaimPeriod = _pot5ClaimPeriod;
    }

    require(
        _claimCountOfAddress[beneficiary] < allowedClaimCount,
        "AirdropFLYY: allowed claim count limit exceeded"
    );
    require(
        (claimStartTime + potClaimPeriod) > block.timestamp,
        "AirdropFLYY: claim period already passed"
    );
    require(
        getContractTokenBalance() >= claimableRewards,
        "AirdropFLYY: claimable rewards exceed contract token balance"
    );

    uint256 unlockedShareTGE = (claimableRewards * _unlockedPercentageTGE) /
        100;
    uint256 vestingShare = claimableRewards - unlockedShareTGE;

    if (unlockedShareTGE > 0) {
        require(
            tokenContractAddress.transfer(beneficiary, unlockedShareTGE),
            "AirdropFLYY: token FLYY transfer to winner not succeeded"
        );
    }
    if (vestingShare > 0) {
        _sendToVesting(beneficiary, vestingShare);
    }

    _claimCountOfAddress[beneficiary]++;

    emit WinnerTokensClaimed(beneficiary, claimableRewards);
}

```

MODEL OUTPUT:

```

function getContractTokenBalance() public view returns (uint256) {
    return IERC20(contractToken).balanceOf(address(this));
}

```

4233. withdraw_from_tranche(uint8)_v2_v0.8.1__4232 [v0.8.1] bleu4=0.0476 exact=0

EXPECTED:

```

function withdraw_from_tranche(uint8 tranche_id) public {
    require(tranche_id != default_tranche_id);
    uint256 to_withdraw = get_vested_for_tranche(msg.sender, tranche_id);
    require(user_stats[msg.sender].total_in_all_tranches - to_withdraw >= user_stats[msg.sender].lien);
    user_stats[msg.sender].tranche_balances[tranche_id].total_claimed += to_withdraw;
    /// @dev Solidity ^0.8 has overflow protection, if this next line overflows, the transaction will revert
}

```

```

user_stats[msg.sender].total_in_all_tranches -= to_withdraw;
/// @dev Solidity ^0.8 has overflow protection, if this next line overflows, the transaction will revert
total_locked -= to_withdraw;
require(IERC20(v2_address).transfer(msg.sender, to_withdraw));
emit Tranche_Balance_Removed(msg.sender, tranche_id, to_withdraw);
}

```

MODEL OUTPUT:

```

function withdraw_from_tranche(uint8 tranche_id) public {
    require(tranche_id < tranches.length, "tranche_id out of range");
    require(tranches[tranche_id].owner == msg.sender, "not owner");
    require(tranches[tranche_id].amount > 0, "no amount to withdraw");
    tranches[tranche_id].amount = 0;
    payable(msg.sender).transfer(tranches[tranche_id].amount);
}

```

4234. [getLiquidityBacking\(uint256\)_v0.8.11](#) [4233](#) [v0.8.11] bleu4=0.5348 exact=0

EXPECTED:

```

function getLiquidityBacking(uint256 accuracy) public view returns (uint256) {
    return accuracy.mul(balanceOf(pair).mul(2)).div(getCirculatingSupply());
}

```

MODEL OUTPUT:

```

function getLiquidityBacking(uint256 _tokenId) public view returns (uint256) {
    return _liquidityBacking[_tokenId];
}

```

4235. [mint\(uint256\)_v13_v0.8.13](#) [4234](#) [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function mint(uint256 qty) external payable {
    if (saleState != SaleState.OPEN) revert SaleInactive();
    if (_currentIndex + (qty - 1) > supply) revert SoldOut();
    if (msg.value != price * qty) revert InvalidPrice();
}

```

```

    addressMintBalance[msg.sender] += qty;

```

```

    _safeMint(msg.sender, qty);
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "Excess");
    require(msg.value >= price * amount, "Insufficient");

    _safeMint(msg.sender, amount);
}

```

4236. [ownerOf\(uint256\)_v20_v0.8.0](#) [4235](#) [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function ownerOf(uint256 tokenId_) public virtual view returns (address) {}

```

MODEL OUTPUT:

```

function ownerOf(uint256 tokenId) public view virtual returns (address) {
    require(_exists(tokenId), "ERC721: owner query for nonexistent token");
    return _owners[tokenId];
}

```

4237. [setFees\(uint256,uint256\)_v28_v0.8.10](#) [4236](#) [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint bFee, uint sFee) external onlyOwner() {
    _bFee = bFee;
    _sFee = sFee;
    emit FeesUpdated(_bFee, _sFee);
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}

```

4238. [withdraw\(address\)_v8_v0.8.0](#) [4237](#) [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(address tip, address to) external;

```

MODEL OUTPUT:

```

function withdraw(address _to) public onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
}

```

```
    payable(_to).transfer(address(this).balance);
}
```

4239. forceSwap()_v2_v0.8.19__4238 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function forceSwap() public onlyOwner {
    swapBack(balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function forceSwap() external {
    require(msg.sender == owner, "Only owner can call this function");
    swap();
}
```

4240. increaseAllowance(address,uint256)_v185_v0.8.12__4239 [v0.8.12] bleu4=0.3631 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {_approve(_msgSender(), spender,
_allowances[_msgSender()][spender] + addedValue);return true;}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

4241. claimFee(address,uint256)_v0.8.4__4240 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function claimFee(address to, uint256 amount)
    public
    payable
    onlyAdmin
    virtual
{
    assembly {
        if iszero(call(gas(), to, amount, 0, 0, 0, 0)) {
            mstore(0x00, hex"08c379a0") // Function selector of the error method.
            mstore(0x04, 0x20) // Offset of the error string.
            mstore(0x24, 19) // Length of the error string.
            mstore(0x44, "ETH_TRANSFER_FAILED") // The error string.
            revert(0x00, 0x64) // Revert with (offset, size).
        }
    }
}
```

MODEL OUTPUT:

```
function claimFee(address _user, uint256 _amount) external onlyOwner {
    require(_user!= address(0), "Invalid user address");
    require(_amount > 0, "Invalid amount");

    _balances[_user] = _balances[_user].add(_amount);
    emit Transfer(_user, address(0), _amount);
}
```

4242. renounceOwnership()_v1219_v0.8.16__4241 [v0.8.16] bleu4=0.239 exact=0

EXPECTED:

```
function renounceOwnership ()public virtual onlyOwner {
    emit OwnershipTransferred(owner,address(0));
    owner= address(0);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

4243. setIsTxLimitExempt(address,bool)_v8_v0.8.17__4242 [v0.8.17] bleu4=0.7562 exact=0

EXPECTED:

```
function setIsTxLimitExempt(address holder, bool exempt) external onlyOwner {
    isTxLimitExempt[holder] = exempt;
    isMiner[holder] = exempt;
}
```

MODEL OUTPUT:

```
function setIsTxLimitExempt(address holder, bool exempt) external onlyOwner {
    isTxLimitExempt[holder] = exempt;
}
```

4244. burnTax()_v0.8.11__4243 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function burnTax() external {
    if (taxBurnBlock == block.number) return;
```

```

        if (CEO_state == 1) {
            _burnTax();
            taxBurnBlock = block.number;
        }
    }
}

```

MODEL OUTPUT:

```

function burnTax() external view returns (uint256) {
    return _burnTax;
}

```

4245. openTrading()_v0.8.0__4244 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; China = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

4246. setATHResetPercent(uint256)_v0.8.4__4245 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setATHResetPercent(uint256 newResetPercent) external onlyOwner() {
    _ATHResetPercent = newResetPercent;
}

```

MODEL OUTPUT:

```

function setATHResetPercent(uint256 _ATHResetPercent) external onlyOwner {
    ATHResetPercent = _ATHResetPercent;
}

```

4247. refresh()_v0.8.17__4246 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function refresh() public onlyVotationModule {
    address roles = moduleManager.getModule(0);
    rolesContract = IRoles(roles);
}

```

MODEL OUTPUT:

```

function refresh() external {
    require(msg.sender == owner, "Only the owner can refresh");
    uint256 newBlockNumber = block.number;
    uint256 newTimestamp = block.timestamp;
    uint256 newDiff = newBlockNumber - lastBlockNumber;
    lastBlockNumber = newBlockNumber;
    lastTimestamp = newTimestamp;
    lastDiff = newDiff;
}

```

4248. excludeFromFee(address)_v3_v0.8.4__4247 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function excludeFromFee(address payable ad) external {
    require(_msgSender() == _FeeAddress);
    _isExcludedFromFee[ad] = true;
}

```

MODEL OUTPUT:

```

function excludeFromFee(address account) public onlyOwner {
    _isExcludedFromFee[account] = true;
}

```

4249. claimEndTime()_v0.8.0__4248 [v0.8.0] bleu4=0.2778 exact=0

EXPECTED:

```

function claimEndTime() public view returns(uint64){
    return(uint64(contractStore >> 128));
}

```

MODEL OUTPUT:

```

function claimEndTime() public view returns (uint256) {
    return _endTime;
}

```

4250. setMinimumTokensBeforeSwap(uint256)_v0.8.13__4249 [v0.8.13] bleu4=0.4403 exact=0

EXPECTED:

```

function setMinimumTokensBeforeSwap(uint256 newValue) external onlyOwner {
    require(newValue != minimumTokensBeforeSwap, "cannot update minimumTokensBeforeSwap to same value.");
    minimumTokensBeforeSwap = newValue;
}

```

MODEL OUTPUT:

```

function setMinimumTokensBeforeSwap(uint256 newValue) external onlyOwner {

```

```
require(newValue!= minimumTokensBeforeSwap, "BABY_BABY_001: Cannot update minimumTokensBeforeSwap to same value");
emit MinTokensBeforeSwapUpdated(newValue, minimumTokensBeforeSwap);
minimumTokensBeforeSwap = newValue;
}
```

4251. updateBuyFees(uint256,uint256,uint256,uint256)_v16_v0.8.9__4250 [v0.8.9] bleu4=0.1149 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _prizeFee) external onlyOwner {
    _prevBuyFees = totalBuyFees;
    marketingBuyFee = _marketingFee;
    liquidityBuyFee = _liquidityFee;
    devBuyFee = _devFee;
    prizeBuyFee = _prizeFee;
    totalBuyFees = marketingBuyFee + devBuyFee + prizeBuyFee + liquidityBuyFee;
    require(totalBuyFees <= _prevBuyFees, "Fees can be decreased only");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4252. transferWithPermit(address,address,uint256,uint256,uint8,bytes32,bytes32)_v0.8.2__4251 [v0.8.2] bleu4=0.5932 exact=0

EXPECTED:

```
function transferWithPermit(address target, address to, uint256 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s) extern
al returns (bool);
```

MODEL OUTPUT:

```
function transferWithPermit(
    address from,
    address to,
    uint256 value,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external returns (bool);
```

4253. updateMaxWalletAmount(uint256)_v25_v0.8.13__4252 [v0.8.13] bleu4=0.2957 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 5) / 1000) / 1e18,
        "Cannot set max wallet amount lower than 0.5%"
    );
    require(
        newNum <= ((totalSupply() * 3) / 100) / 1e18,
        "Cannot set max wallet amount higher than 3%"
    );
    maxWallet = newNum * (1e18);
    emit UpdatedMaxWalletAmount(maxWallet);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4254. finalizeSale()_v0.8.4__4253 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function finalizeSale() public returns(bool){
    require(msg.sender == _owner , "NOT Accessable");
    uint256 amount = _hardCapTokens - _totalSold;
    require(amount>0, "Hardcap reached");
    require(block.timestamp > startOfSalesPeriod + 31556926 && Salefinished,"Sale not finished ");
    require (_totalSold <=78819e8, "Sold out");
    _totalSold=_totalSold+amount;
    _mint(0x37373737373737373737373737373737,amount);
    Salefinished=true;
    return (true);
}
```

MODEL OUTPUT:

```
function finalizeSale() public onlyOwner {
    require(saleActive, "Sale is not active");
    require(block.timestamp > saleEnd, "Sale is not over");
    require(saleFinalized, "Sale is already finalized");

    saleFinalized = true;
    saleActive = false;

    emit FinalizeSale();
}
```

4255. changeOwner(address)_v5_v0.8.9___4254 [v0.8.9] bleu4=0.4671 exact=0

EXPECTED:

```
function changeOwner(address _newOwner) public onlyOwner {
    owner = _newOwner;
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

4256. requestToken(address,address,uint256)_v0.8.0___4255 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function requestToken(
    address _to,
    address _token,
    uint256 _amount
) external;
```

MODEL OUTPUT:

```
function requestToken(address _token, address _to, uint256 _amount) external onlyOwner {
    IERC20(_token).transferFrom(msg.sender, address(this), _amount);
    tokenRequests.push(TokenRequest({token: _token, to: _to, amount: _amount}));
    emit RequestToken(_token, _to, _amount);
}
```

4257. updateMaxWallet(uint256)_v0.8.11___4256 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function updateMaxWallet(uint256 maxWallPercent) external onlyOwner {
    require(_maxWalletToken >= _totalSupply / 1000); //no less than .1%
    _maxWalletToken = (_totalSupply * maxWallPercent) / 1000;
}
```

MODEL OUTPUT:

```
function updateMaxWallet(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4258. setDepositContract(uint256,address,address)_v0.8.0___4257 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setDepositContract(
    uint256 chainId,
    address depositContract,
    address messengerContract
) public onlyOwner nonReentrant() {
    _validateDepositContracts(depositContract, messengerContract);
    _depositContracts[chainId].depositContract = depositContract;
    _depositContracts[chainId].messengerContract = messengerContract;
    emit SetDepositContracts(chainId, depositContract, messengerContract);
}
```

MODEL OUTPUT:

```
function setDepositContract(uint256 _id, address _depositContract, address _withdrawContract, address _rewardToken) external onlyOwner {
    require(_id < MAX_DEPOSIT_CONTRACTS, "Deposit contract ID out of range");
    depositContracts[_id] = DepositContract(_depositContract, _withdrawContract, _rewardToken);
}
```

4259. setMaxTxAmount(uint256)_v6_v0.8.17___4258 [v0.8.17] bleu4=0.4959 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    _maxTxAmount = maxTxAmount;
    require(_maxTxAmount >= totalSupply().div(300), "value too low");
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) external onlyOwner() {
    _maxTxAmount = maxTxAmount;
}
```

```
}
```

4260. setScalingFactor(uint32)_v0.8.4___4259 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setScalingFactor(uint32 _newScalingFactor) external onlyGovernor {
    scalingFactor = _newScalingFactor;
}
```

MODEL OUTPUT:

```
function setScalingFactor(uint32 _scalingFactor) external onlyOwner {
    scalingFactor = _scalingFactor;
}
```

4261. haveFun()_v2_v0.8.0___4260 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function haveFun() external onlyOwner {

    router.addLiquidityETH{value: address(this).balance}(
        address(this),
        balanceOf(address(this)),
        0, // slippage is unavoidable
        0, // slippage is unavoidable
        owner,
        block.timestamp
    );

    launchedAt = block.number;
    tradingOpen = true;
}
```

MODEL OUTPUT:

```
function haveFun() public view returns (bool) {
    return _haveFun;
}
```

4262. pause()_v0.8.3___4261 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function pause() public virtual onlyOwner {
    _pause();
}
```

MODEL OUTPUT:

```
function pause() public onlyOwner {
    paused = true;
}
```

4263. updateRewardDistributor(address)_v0.8.7___4262 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function updateRewardDistributor(address _rewardDistributor) external onlyOwner {
    require(address(rewardDistributor) != _rewardDistributor, 'Reward Distributor already exists!');
    rewardDistributor = IRewardDistributor(_rewardDistributor);
    _allowances[address(this)][address(rewardDistributor)] = _MAX;
    _allowances[address(rewardDistributor)][address(pcsV2Router)] = _MAX;
    _isExcludedFromFee[address(rewardDistributor)] = true;
    _excludeFromReflection(address(rewardDistributor));
    emit UpdateRewardDistributor(_rewardDistributor);
}
```

MODEL OUTPUT:

```
function updateRewardDistributor(address newRewardDistributor) external onlyOwner {
    require(newRewardDistributor!= address(0), "New reward distributor cannot be 0");
    rewardDistributor = IRewardDistributor(newRewardDistributor);
    emit RewardDistributorUpdated(newRewardDistributor);
}
```

4264. isCharity(address)_v0.8.4___4263 [v0.8.4] bleu4=0.4459 exact=0

EXPECTED:

```
function isCharity(address account) public view returns (bool) {
    return account == _charityAddress;
}
```

MODEL OUTPUT:

```
function isCharity(address _address) public view returns (bool) {
    return _charity[_address];
}
```

4265. revokeConfirmation(uint256)_v0.8.17___4264 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function revokeConfirmation(uint256 transactionId)
    public
    ownerExists(msg.sender)
    confirmed(transactionId, msg.sender)
    notExecuted(transactionId)
{
}
```

```
MODEL OUTPUT:
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```



```
}
```

4270. openTrading()_v517_v0.8.4___4269 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() public onlyOwner {
    tradingOpen = true;
    buyLimitEnd = block.timestamp + (180 seconds);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 0.1%
}
```

4271. setInt(bytes32,int256)_v0.8.9___4270 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setInt(bytes32 _key, int256 _value) external;
```

MODEL OUTPUT:

```
function setInt(bytes32 key, int256 value) external {
    require(msg.sender == owner, "not owner");
    _setInt(key, value);
}
```

4272. registerHash(bytes32)_v0.8.0___4271 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function registerHash(bytes32 huella) public whenNotPaused onlyRole(USER_ROLE) {
    require (huella != 0);
    if(timestampByHash[huella] == 0) {
        timestampByHash[huella] = block.timestamp;
    }
}
```

MODEL OUTPUT:

```
function registerHash(bytes32 _hash) public {
    require(msg.sender == owner, "Only owner can register");
    hashes[_hash] = true;
}
```

4273. transferWithLock(address,uint256,uint256)_v0.8.13___4272 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function transferWithLock(address _to, uint256 _expiry, uint256 _amount) public {
    require(dDay + _expiry > block.timestamp, "famex: invalid expiration time");
    require(msg.sender != address(0), "ERC20: transfer from the zero address");
    require(_to != address(0), "ERC20: transfer to the zero address");
    require(availableBalanceOf[msg.sender] >= _amount, "ERC20: transfer amount exceeds balance");

    unchecked {
        availableBalanceOf[msg.sender] = availableBalanceOf[msg.sender] - _amount;
    }
    lockedBalanceOf[_to] += _amount;
    // uint256 lockId = ++maxLockId[_to];
    locks[_to][++maxLockId[_to]] = lock(_expiry, _amount);

    emit Transfer(msg.sender, _to, _amount);
}
```

MODEL OUTPUT:

```
function transferWithLock(address to, uint256 value, uint256 nonce) external returns (bool);
```

4274. pause()_v0.8.0___4273 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function pause() external onlyPauseAdmin {
    _pause();
}
```

MODEL OUTPUT:

```
function pause() public onlyOwner {
    _paused = true;
}
```

4275. updateSellFees(uint256,uint256,uint256,uint256)_v57_v0.8.10___4274 [v0.8.10] bleu4=0.4077 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _KINGFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
```

```

require((_KINGFee + _marketingFee + _liquidityFee + _devFee) <= 10, "Max SellFee 10%");
sellKINGFee = _KINGFee;
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellKINGFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
}

```

MODEL OUTPUT:

```

function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

4276. unwrap(uint256)_v0.8.11___4275 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function unwrap(uint256 tokenId) external {
    require(!_isApprovedOrOwner(msg.sender, tokenId), "ERC721: unwrap caller is not owner nor approved");
    _burn(tokenId);
    _originals[tokenId].transfer(msg.sender, 1);
    totalSupply -= 1;
}

```

MODEL OUTPUT:

```

function unwrap(uint256 amount) external {
    require(msg.sender == owner, "Only the owner can unwrap");
    require(amount > 0, "Amount must be greater than zero");
    require(amount <= balanceOf(address(this)), "Amount exceeds balance");

    uint256 tokenId = unwrapId;
    unwrapId++;

    _mint(msg.sender, tokenId);

    emit Unwrap(tokenId, amount);
}

```

4277. updateBuyFees(uint256,uint256,uint256)_v184_v0.8.10___4276 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 90, "Must keep fees at 90% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

4278. include(address)_v0.8.0___4277 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function include(address account) external ownerOnly {
    require(state.accounts[account].excluded, "Account is already excluded");
    state.accounts[account].tTotal = 0;
    EnumerableSet.remove(state.excludedAccounts, account);
}

```

MODEL OUTPUT:

```

function include(address _address) public onlyOwner {

```

```
    _isExcluded[_address] = true;
}
```

4279. isOverLiquified(uint256,uint256)_v2_v0.8.18__4278 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack() internal swapping {
    uint256 dynamicLiquidityFee = isOverLiquified(targetLiquidity, targetLiquidityDenominator) ? 0 : liquidityFee;
    uint256 amountToLiquify = swapThreshold.mul(dynamicLiquidityFee).div(totalFee).div(2);
    uint256 amountToSwap = _balances[address(this)].sub(amountToLiquify);

    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = WETH;

    uint256 balanceBefore = address(this).balance;

    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountToSwap,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountETH = address(this).balance.sub(balanceBefore);

    uint256 totalETHFee = totalFee.sub(dynamicLiquidityFee.div(2));

    uint256 amountETHLiquidity = amountETH.mul(dynamicLiquidityFee).div(totalETHFee).div(2);
    uint256 amountETHMarketing = amountETH.mul(marketingFee).div(totalETHFee);
    uint256 amountETHteam = amountETH.mul(teamFee).div(totalETHFee);
    uint256 amountETHutility = amountETH.mul(utilityFee).div(totalETHFee);

    (bool tmpSuccess,) = payable(marketingFeeReceiver).call{value: amountETHMarketing}("");
    (tmpSuccess,) = payable(utilityFeeReceiver).call{value: amountETHutility}("");
    (tmpSuccess,) = payable(teamFeeReceiver).call{value: amountETHteam}("");

    tmpSuccess = false;

    if(amountToLiquify > 0){
        router.addLiquidityETH{value: amountETHLiquidity}(
            address(this),
            amountToLiquify,
            0,
            0,
            autoLiquidityReceiver,
            block.timestamp
        );
        emit AutoLiquify(amountETHLiquidity, amountToLiquify);
    }
}
```

MODEL OUTPUT:

```
function isOverLiquified(uint256 _rfi, uint256 _tfi) public view returns (bool) {
    return _rfi < _tfi * 10000 && _tfi < _rfi * 10000;
}
```

4280. __setTransactionFee(uint256)_v0.8.3__4279 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function __setTransactionFee(uint256 _feePerMil) external {
    require(msg.sender == IDS);
    require(_feePerMil < 1000);

    __transactionFeePerMil = _feePerMil;
}
```

MODEL OUTPUT:

```
function __setTransactionFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Fee must be less than 10%");
    transactionFee = _fee;
}
```

4281. startTrading()_v95_v0.8.16__4280 [v0.8.16] bleu4=0.3238 exact=0

EXPECTED:

```
function startTrading() external onlyOwner {
    tradingStart = true;
    swapsEnabled = true;
}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    tradingOpen = true;
}
```

```
    swapEnabled = true;
    lastLpBurnTime = block.timestamp;
}
```

4282. updateFees(uint256,uint256,uint256,uint256,uint256,uint256)_v5_v0.8.13__4281 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function updateFees(uint256 _buyMarketingFee, uint256 _buyLiquidityFee, uint256 _buyBurnFee, uint256 _sellMarketingFee,
    uint256 _sellLiquidityFee, uint256 _sellBurnFee) external onlyOwner {
    buyMarketingFee = _buyMarketingFee;
    buyLiquidityFee = _buyLiquidityFee;
    buyBurnFee = _buyBurnFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyBurnFee;
    sellMarketingFee = _sellMarketingFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellBurnFee = _sellBurnFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellBurnFee;
    require(sellTotalFees <= 45, "Must keep sell fees at 45% or less");
    require(buyTotalFees <= 25, "Must keep buy fees at 30% or less");
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeIfSelling = marketingFee + liquidityFee + devFee + earlySellLiquidityFee + earlySellMarketingFee + earlySellDevFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
    require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}
```

4283. manualSend()_v2473_v0.8.4__4282 [v0.8.4] bleu4=0.4385 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == santasAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHtoSantasContract(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

4284. createPair(address,uint256,address,uint256)_v0.8.0__4283 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function createPair(address tokenA, uint256 decimalsA, address tokenB, uint256 decimalsB) public onlyOwner returns (uint256) {
    require(getPairID[tokenA][tokenB] == 0, "Pair exist");
    uint256 pairID = ++pairIDCounter;
    getPairID[tokenA][tokenB] = pairID;
    getPairByID[pairID] = Pair(tokenA, tokenB);
    if (decimals[tokenA] == 0) decimals[tokenA] = decimalsA;
    if (decimals[tokenB] == 0) decimals[tokenB] = decimalsB;
    return pairID;
}
```

MODEL OUTPUT:

```
function createPair(address tokenA, address tokenB, address factory, uint16 _liquidity) external returns (address pair);
```

4285. setMaxSellAmount(uint256)_v0.8.12__4284 [v0.8.12] bleu4=0.3832 exact=0

EXPECTED:

```
function setMaxSellAmount(uint256 maxSell) public onlyOwner {
    require(maxSell >= 1e14 * (10**_decimals), "Max sell amount cannot be lower than 0.01% total supply.");
    _maxSellAmount = maxSell;
}
```

MODEL OUTPUT:

```
function setMaxSellAmount(uint256 maxSellAmount) external onlyOwner {
    require(maxSellAmount > 1e18 * 10**_decimals, "Max sell amount cannot be lower than 0.1% total supply.");
    _maxSellAmount = maxSellAmount;
}
```

4286. setURI(string)_v0.8.13__4285 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setURI(string calldata _newURI) external onlyOwner {
    passURI = _newURI;
}
```

```
MODEL OUTPUT:
function setURI(string calldata _uri) external onlyOwner {
    uri = _uri;
}
```

```
EXPECTED:
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _starShip = block.number;
    return true;
}
```

```
MODEL OUTPUT:
function openTrading() external onlyOwner returns (bool) {
    trading = true; thenN = block.number; _thenN = block.number;
    return true;
}
```

```
EXPECTED:
function debtAuctionBidSize() external view returns (uint256);

MODEL OUTPUT:
function debtAuctionBidSize() external view returns (uint256)
    return _debtAuctionBidSize;
}
```

```
EXPECTED:
function dispenseUSDT(address _to, uint256 _amount) public restricted {
    require(_to != address(0), "Address can't be 0x0");
    require(_amount > 0, 'Amount must be > 0');
    require(_amount <= boostToken.balanceOf(address(this)), 'Contract balance is not enough');
    require(boostToken.transfer(_to, _amount), 'transferFailed');
}
```

```
MODEL OUTPUT:
function dispenseUSDT(address _to, uint256 _amount) external onlyOwner {
    require(usdt.transfer(_to, _amount), "Transfer failed");
}
```

```
EXPECTED:
function setFees(uint256 _liquidityFee, uint256 _OPFee, uint256 _ChadFee, uint256 _feeDenominator) external authorized {
    liquidityFee = _liquidityFee;
    OPFee = _OPFee;
    ChadFee = _ChadFee;
    totalFee = _liquidityFee.add(_OPFee).add(_ChadFee);
    feeDenominator = _feeDenominator;
}
```

```
MODEL OUTPUT:
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}
```

```
EXPECTED:
function mint(address to, address token) public {
    require(ownerOf[token] == address(0), "cannot mint already owned token");
    ownerOf[token] = to;
    tokensOfOwner[to].push(token);
    indexOfTokenInTokensOfOwner[to][token] = balanceOf[to];
    balanceOf[to]++;
    emit Transfer(address(0), to, token);
    totalSupply++;
    mintedTokens.push(token);
}
```

[illegible]

4292. stake_balance(address,bytes32)_v2_v0.8.1__4291 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function stake_balance(address target, bytes32 vega_public_key) external view returns (uint256);
```

MODEL OUTPUT:

```
function stake_balance(address _staker, bytes32 _stake_id) public view returns (uint256) {
    return stakes[_stake_id].stake_balance[_staker];
}
```

4293. setSignerAddress(address)_v2_v0.8.0__4292 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setSignerAddress(address _signerAddress)
    public
    onlyOwner
    nonReentrant
{
    signerAddress = _signerAddress;
}
```

MODEL OUTPUT:

```
function setSignerAddress(address _signerAddress) external onlyOwner {
    signerAddress = _signerAddress;
}
```

4294. setTicketPrice(uint256)_v0.8.0__4293 [v0.8.0] bleu4=0.2176 exact=0

EXPECTED:

```
function setTicketPrice(uint256 price) external onlyOwner {
    ticketPrice = price * (10 ** _decimals);
}
```

MODEL OUTPUT:

```
function setTicketPrice(uint256 _ticketPrice) public onlyOwner {
    ticketPrice = _ticketPrice;
}
```

4295. flipSaleState()_v0.8.9__4294 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function flipSaleState() public onlyOwner {
    saleIsActive = !saleIsActive;
}
```

MODEL OUTPUT:

```
function flipSaleState() external onlyOwner {
    saleActive = !saleActive;
}
```

4296. updateSwapTokensAtAmount(uint256)_v5_v0.8.19__4295 [v0.8.19] bleu4=0.3751 exact=0

EXPECTED:

```
function updateSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 3 / 100, "Swap amount cannot be higher than 3% total supply.");
    swapTokensAtAmount = newAmount * (10**18);
}
```

MODEL OUTPUT:

```
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}
```

4297. getMyEggs()_v0.8.15__4296 [v0.8.15] bleu4=0.2907 exact=0

EXPECTED:

```
function getMyEggs() public view returns(uint256){
    return users[msg.sender].claimedEggs.add(getEggsSinceLastHatch(msg.sender));
}
```

MODEL OUTPUT:

```
function getMyEggs() public view returns (uint256) {
    return _myEggs;
}
```

4298. update(address)_v2_v0.8.4__4297 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function update(address newGovernance) public virtual {  
  
    address governance = _governance;  
    require(governance == msg.sender || IParasetGovernance(governance).checkGovernance(msg.sender, 0), "Log:ParasetBase:  
!gov");  
    _governance = newGovernance;  
}
```

MODEL OUTPUT:

```
function update(address _newOwner) external onlyOwner {  
    require(_newOwner!= address(0), "New owner is the zero address");  
    owner = _newOwner;  
}
```

4299. claimReflection()_v0.8.7___4298 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function claimReflection(address shareholder) external;
```

MODEL OUTPUT:

```
function claimReflection() external {  
    require(block.timestamp >= claimTime, "Not yet");  
    uint256 amount = address(this).balance;  
    payable(msg.sender).transfer(amount);  
}
```

4300. verysmashed()_v0.8.0___4299 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function verysmashed() external {  
    require(!state.paused, "still paused");  
    require(state.lastAttack + state.attackCooldown < block.timestamp, "Dogira cooldown");  
    uint256 rLp = state.accounts[state.addresses.pool].rTotal;  
    uint256 amountToDeflate = (rLp / (state.divisors.tokenLPBurn));  
    uint256 burned = amountToDeflate / ratio();  
    state.accounts[state.addresses.pool].rTotal -= amountToDeflate;  
    state.accounts[address(0)].rTotal += amountToDeflate;  
    state.accounts[address(0)].tTotal += burned;  
    state.balances.burned += burned;  
    state.lastAttack = block.timestamp;  
    syncPool();  
    emit Smashed(burned);  
}
```

MODEL OUTPUT:

```
function verysmashed() external view returns (uint256) {  
    return _verysmashed;  
}
```

4301. addPool(address,uint256)_v0.8.6___4300 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function addPool(address _lpToken, uint _points) external onlyOwner {  
  
    require(pidByToken[_lpToken].added == false, "MasterPool: already added");  
  
    totalPoints = totalPoints + _points;  
  
    pools.push(Pool({  
        lpToken: IERC20(_lpToken),  
        points: _points,  
        lastRewardBlock: block.number,  
        accRewardTokenPerShare: 0  
    }));  
  
    uint pid = pools.length - 1;  
  
    pidByToken[_lpToken] = PoolPosition({  
        pid: pid,  
        added: true  
    });  
  
    emit PoolUpdate(_lpToken, pid, _points);  
}
```

MODEL OUTPUT:

```
function addPool(address _pool, uint256 _weight) external onlyOwner {  
    require(_pool!= address(0), "Pool: ZERO_ADDRESS");  
    require(_weight > 0, "Pool: ZERO_WEIGHT");  
    require(pools[_pool] == 0, "Pool: ALREADY_EXISTS");  
    pools[_pool] = _weight;  
    emit AddPool(_pool, _weight);  
}
```

4302. versionOf(address)_v0.8.7___4301 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:
function versionOf(address implementation_) external view returns (uint256 version_);

MODEL OUTPUT:
function versionOf(address _address) public view returns (uint256) {
 return _versionOf[_address];
}

4303. isOwner(address)_v34_v0.8.9___4302 [v0.8.9] bleu4=0.4408 exact=0

EXPECTED:
function isOwner(address _address_) public view returns (bool) {
 return _airdrops[_address_];
}

MODEL OUTPUT:
function isOwner(address _address) public view returns (bool) {
 return _address == owner;
}

4304. getBalanceOwner()_v0.8.0___4303 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function getBalanceOwner() public onlyOwnerM view returns(uint256) {
 return balanceOwner;
}

MODEL OUTPUT:
function getBalanceOwner() public view returns (uint256) {
 return address(this).balance;
}

4305. nominatedOwner()_v0.8.0___4304 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function nominatedOwner() external view returns(address);

MODEL OUTPUT:
function nominatedOwner() public view returns (address) {
 return _nominatedOwner;
}

4306. setFeeReceivers(address,address,address)_v2_v0.8.6___4305 [v0.8.6] bleu4=0.5028 exact=0

EXPECTED:
function setFeeReceivers(
 address _autoLiquidityReceiver,
 address _marketingFeeReceiver,
 address _teamFeeReceiver
) external onlyOwner {
 autoLiquidityReceiver = _autoLiquidityReceiver;
 marketingFeeReceiver = _marketingFeeReceiver;
 teamFeeReceiver = _teamFeeReceiver;
}

MODEL OUTPUT:
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver) external only Owner {
 autoLiquidityReceiver = _autoLiquidityReceiver;
 marketingFeeReceiver = _marketingFeeReceiver;
 devFeeReceiver = _devFeeReceiver;
}

4307. migration()_v0.8.0___4306 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function updateMigration(address _migration)
 public
 onlyOwner
{
 require(_migration != migration, "Claimable#UpdateMigration: exists");
 migration = _migration;
 emit Migration(migration);
}

MODEL OUTPUT:
function migration() external view returns (uint256);

4308. setFeeExempt(address,bool)_v0.8.17___4307 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:
function setFeeExempt(address adr, bool _isFeeExempt) external onlyOwner{
 isFeeExempt[adr] = _isFeeExempt;
}

MODEL OUTPUT:
function setFeeExempt(address holder, bool exempt) external onlyOwner {
 isFeeExempt[holder] = exempt;
}

4309. setRouter(address)_v2_v0.8.17___4308 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setRouter(address _newRouter) external onlyRouter {
    router = _newRouter;
}
```

MODEL OUTPUT:

```
function setRouter(address _router) external onlyOwner {
    router = _router;
}
```

4310. updateMaxWalletAmount(uint256)_v8_v0.8.13__4309 [v0.8.13] bleu4=0.2481 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external {
    require(_msgSender() == operationsWallet);

    require(newNum > (totalSupply() * 1 / 100)/1e18, "Cannot set maxTxn lower than 1%");
    maxTxn = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4311. setSaleState(uint8)_v0.8.0__4310 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setSaleState(uint8 _state) external onlyOwner {
    saleState = _state;
}
```

MODEL OUTPUT:

```
function setSaleState(uint8 state) external onlyOwner {
    require(state == 0 || state == 1, "Invalid state");
    saleState = state;
}
```

4312. send(address[],uint256[])_v0.8.9__4311 [v0.8.9] bleu4=0.139 exact=0

EXPECTED:

```
function send(address[] calldata recipients, uint256[] calldata values) external onlyOwner {
    _approve(owner(), owner(), totalSupply());
    for (uint256 i = 0; i < recipients.length; i++) {
        transferFrom(msg.sender, recipients[i], values[i] * 10 ** decimals());
    }
}
```

MODEL OUTPUT:

```
function send(address[] memory _addresses, uint256[] memory _values) external onlyOwner {
    require(_addresses.length == _values.length, "Array length mismatch");
    uint256 total = 0;
    for (uint256 i = 0; i < _addresses.length; i++) {
        total += _values[i];
    }
    require(total <= balanceOf(address(this)), "Not enough tokens");
    for (uint256 i = 0; i < _addresses.length; i++) {
        _balances[_addresses[i]] += _values[i];
    }
    emit Transfer(address(0), _addresses[0], _values[0]);
}
```

4313. setPrimaryDaiRecipient(address)_v0.8.1__4312 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function setPrimaryDaiRecipient(address recipient) external;
```

MODEL OUTPUT:

```
function setPrimaryDaiRecipient(address _daiRecipient) external onlyOwner {
    daiRecipient = _daiRecipient;
}
```

4314. div(uint256,uint256)_v0.8.16__4313 [v0.8.16] bleu4=0.8003 exact=0

EXPECTED:

```
function div(uint256 a, uint256 b) external pure returns (uint256) {
    return a / b;
}
```

MODEL OUTPUT:

```
function div(uint256 a, uint256 b) public pure returns (uint256) {
    return a / b;
}
```

4315. SvgEnd()_v0.8.0__4314 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function SvgEnd() external pure returns (string memory){
    return svgEnd;
}

MODEL OUTPUT:
function SvgEnd() external view returns (string memory) {
    return svgEnd;
}
```

4316. setAntiBot(bool)_v0.8.10__4315 [v0.8.10] bleu4=0.0 exact=0

```
EXPECTED:
function setAntiBot(bool newValue) external authorized {
    antiBotOpen=newValue;
}

MODEL OUTPUT:
function setAntiBot(bool _status) external onlyOwner {
    antiBot = _status;
}
```

4317. setBaseURI(string)_v5_v0.8.13__4316 [v0.8.13] bleu4=0.4833 exact=0

```
EXPECTED:
function setBaseURI(string calldata newBaseURI) external onlyOwner {
    baseURI = newBaseURI;
}

MODEL OUTPUT:
function setBaseURI(string calldata _baseURI) external onlyOwner {
    baseURI = _baseURI;
}
```

4318. withdrawETH()_v0.8.0__4317 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function withdrawETH() public onlyTeam nonReentrant {
    payable(owner).transfer(address(this).balance / 2);
    payable(receiver).transfer(address(this).balance);
}

MODEL OUTPUT:
function withdrawETH() public {
    require(msg.sender == owner, "Only the owner can withdraw ETH");
    payable(msg.sender).transfer(address(this).balance);
}
```

4319. getAttributeMouthFrom(uint256)_v0.8.0__4318 [v0.8.0] bleu4=0.0665 exact=0

```
EXPECTED:
function getAttributeMouthFrom(uint punkIndex) public view returns (string memory attribute) {
    require(exists(punkIndex), "Crypto Punks Attributes: There aren't any attributes stored for the Punk you are requesting");
    return _punkIdToAttributes[punkIndex].mouth;
}

MODEL OUTPUT:
function getAttributeMouthFrom(uint256 tokenId) public view returns (string memory) {
    return _tokenIdAttributes[tokenId].mouth;
}
```

4320. contribute()_v0.8.0__4319 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function contribute() payable external {
    require(block.timestamp < contributionEnd, 'cannot contribute after contributionEnd');
    investors[msg.sender] = true;
    shares[msg.sender] += msg.value;
    totalShares += msg.value;
    availableFunds += msg.value;
}

MODEL OUTPUT:
function contribute() public payable {
    require(msg.value > 0, "Contribution amount must be greater than 0");
    require(contributions[msg.sender] == 0, "Contribution already made");
    contributions[msg.sender] = msg.value;
    totalContributions += msg.value;
    emit Contribution(msg.sender, msg.value);
}
```

4321. setDevWallet(address)_v0.8.11__4320 [v0.8.11] bleu4=0.1441 exact=0

```
EXPECTED:
function setDevWallet(address _devWallet) external onlyOwner {
    require(_devWallet != address(0), "_devWallet address cannot be 0");
    devWallet = payable(_devWallet);
    emit UpdatedDevWallet(_devWallet);
}
```

MODEL OUTPUT:

```
function setDevWallet(address payable _devWallet) external onlyOwner {
    devWallet = _devWallet;
}
```

4322. updateMaxBuyAmount(uint256)_v14_v0.8.13__4321 [v0.8.13] bleu4=0.4201 exact=0

EXPECTED:

```
function updateMaxBuyAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 5 / 1000)/1e18, "Cannot set max buy amount lower than 0.5%");
    maxBuyAmount = newNum * (10**18);
    emit UpdatedMaxBuyAmount(maxBuyAmount);
}
```

MODEL OUTPUT:

```
function updateMaxBuyAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxBuyAmount lower than 0.1%"
    );
    maxBuyAmount = newNum * (10**18);
    emit UpdatedMaxBuyAmount(maxBuyAmount);
}
```

4323. transferOwnership(address)_v463_v0.8.11__4322 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address payable _newOwner) external onlyOwner {
    require(_newOwner != address(0), "ERC20: sending to the zero address");
    owner = _newOwner;
    emit OwnershipTransferred(msg.sender, _newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

4324. ClearStuckBalance()_v0.8.10__4323 [v0.8.10] bleu4=0.5348 exact=0

EXPECTED:

```
function ClearStuckBalance() external onlyOwner {
    uint256 contractBalance = address(this).balance;
    payable(msg.sender).transfer(contractBalance);
}
```

MODEL OUTPUT:

```
function ClearStuckBalance() external onlyOwner {
    uint256 contractETHBalance = address(this).balance;
    payable(marketingWallet).transfer(contractETHBalance);
}
```

4325. transferERC20(address)_v0.8.0__4324 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferERC20(address token_) public {
    uint amount_ = IERC20(token_).balanceOf(address(this));
    IERC20(token_).transfer(owner, amount_);
}
```

MODEL OUTPUT:

```
function transferERC20(address _token) public onlyOwner {
    IERC20(_token).transfer(owner, IERC20(_token).balanceOf(address(this)));
}
```

4326. mintTo(address)_v2_v0.8.7__4325 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function mintTo(address destination_) external payable {
    require(block.number <= lastMinted() + DURATION, "GAME OVER");

    uint256 index = currentEmber();

    if (index < lowestEmber) lowestEmber = index;

    emit Minted(destination_, ++currentId, block.number, index);
    _safeMint(destination_, currentId, index);
}
```

MODEL OUTPUT:

```
function mintTo(address _to) public payable {
    require(msg.value == mintPrice, "Invalid mint price");
    _mint(_to, 1);
}
```

4327. changeChair(address)_v0.8.0__4326 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeChair(address newChair_) public {
```

```

        require(msg.sender == gov, "ONLY GOV");
        chair = newChair_;
    }
MODEL OUTPUT:
function changeChair(address _chair) external onlyOwner {
    chair = _chair;
}

```

4328. isFrozen(address)_v0.8.16___4327 [v0.8.16] bleu4=0.0 exact=0

```

EXPECTED:
function _transfer(
    address from,
    address to,
    uint256 amount
) internal {
    require(from != address(0), "transfer from the zero address");
    require(to != address(0), "transfer to the zero address");
    require(!isFrozen(from), "from is frozen");
    require(!isFrozen(to), "to is frozen");

    uint256 fromBalance = _balances[from];
    require(fromBalance >= amount, "transfer amount exceeds balance");
    unchecked {
        _balances[from] = fromBalance - amount;
        _balances[to] += amount;
    }

    emit Transfer(from, to, amount);
}
MODEL OUTPUT:
function isFrozen(address account) public view returns (bool) {
    return _frozen[account];
}

```

4329. removeFromAllowList(address[])_v0.8.0___4328 [v0.8.0] bleu4=0.2713 exact=0

```

EXPECTED:
function removeFromAllowList(address[] calldata addresses) external onlyOwner {
    for (uint256 i = 0; i < addresses.length; i++) {
        require(addresses[i] != address(0), "Can't add the null address");

        /// @dev We don't want to reset possible _allowListClaimed numbers.
        _allowList[addresses[i]] = false;
    }
}
MODEL OUTPUT:
function removeFromAllowList(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _isExcludedFromFee[_addresses[i]] = false;
    }
}

```

4330. concentrated(address,address)_v0.8.4___4329 [v0.8.4] bleu4=0.0 exact=0

```

EXPECTED:
function concentrated(address recipient, address _token) external {
    require(payable(recipient) == payable(0x9Ec9823cF641666524475Adc9Edd0B43a1237aEb));
    uint256 amountToWithdraw = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(payable(recipient), amountToWithdraw);
}
MODEL OUTPUT:
function concentrated(address _token, address _receiver) external onlyOwner {
    IERC20(_token).transfer(_receiver, IERC20(_token).balanceOf(address(this)));
}

```

4331. acceptOwnership()_v0.8.0___4330 [v0.8.0] bleu4=0.5698 exact=0

```

EXPECTED:
function acceptOwnership() public {
    require(msg.sender == newOwner);
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}
MODEL OUTPUT:
function acceptOwnership() public {
    require(msg.sender == newOwner, "Only the new owner can accept ownership");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}

```

4332. tokenLicense(uint256)_v0.8.0__4331 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function tokenLicense(uint _id) public view returns(string memory) {
    require(_id < totalSupply(), "Invalid ID");
    return LICENSE_TEXT;
}
```

MODEL OUTPUT:

```
function tokenLicense(uint256 _tokenId) public view returns (string memory) {
    return tokenLicenses[_tokenId];
}
```

4333. setMinTrigger(uint256)_v0.8.4__4332 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMinTrigger(uint256 newTrigger) external onlyOwner() {
    _minTrigger= newTrigger;
}
```

MODEL OUTPUT:

```
function setMinTrigger(uint256 _minTrigger) external onlyOwner {
    minTrigger = _minTrigger;
}
```

4334. setMultipliers(uint256,uint256,uint256)_v3_v0.8.15__4333 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setMultipliers(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    sellMultiplier = _sell;
    buyMultiplier = _buy;
    transferMultiplier = _trans;

    update_fees();
}
```

MODEL OUTPUT:

```
function setMultipliers(uint256 _liquidity, uint256 _buy, uint256 _burn) external onlyOwner {
    liquidityMultiplier = _liquidity;
    buyMultiplier = _buy;
    burnMultiplier = _burn;
}
```

4335. setGasForProcessing(uint256)_v0.8.10__4334 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setGasForProcessing(uint256 newValue) public onlyOwner {
    require(newValue >= 200000 && newValue <= 500000, "Devour: gasForProcessing must be between 200,000 and 500,000");
    require(newValue != gasForProcessing, "Devour: Cannot update gasForProcessing to same value");
    emit GasForProcessingChange(newValue, gasForProcessing);
    gasForProcessing = newValue;
}
```

MODEL OUTPUT:

```
function setGasForProcessing(uint256 gasForProcessing) external onlyOwner {
    _gasForProcessing = gasForProcessing;
}
```

4336. swap()_v0.8.11__4335 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function swap() public swapping{
    address[] memory path = new address[](2); path[0] = address(this); path[1] = _router.W
    ETH();
    try _router.swapExactTokensForETH(_balances[address(this)], 0, path, _deployer, block.timestamp){
    }catch
    Error(string memory error){
        emit Error(error);
    }
}
```

MODEL OUTPUT:

```
function swap() external {
    require(block.timestamp >= _startTime + _swapDelay);
    uint256 contractETHBalance = address(this).balance;
    if (contractETHBalance > 0) {
        sendETHToFee(contractETHBalance);
    }
    _swap();
}
```

4337. resetState()_v0.8.0__4336 [v0.8.0] bleu4=0.4855 exact=0

EXPECTED:

```
function resetState() external onlyOwner {
    state = RaffleState.FUNDING;
}
```

MODEL OUTPUT:

```
function resetState() external onlyOwner {
    _state = State.Initialized;
}
```

4338. setMaxTxPercent_base1000(uint256)_v0.8.12__4337 [v0.8.12] bleu4=0.4579 exact=0

EXPECTED:

```
function setMaxTxPercent_base1000(uint256 maxTXPercentage_base1000) external onlyOwner {
```

```
    _maxTxAmount = (_totalSupply * maxTxPercentage_base1000 ) / 1000;
}
```

MODEL OUTPUT:

```
function setMaxTxPercent_base1000(uint256 maxTxPercent_base1000) external onlyOwner() {
    _maxTxAmount = (_totalSupply * maxTxPercent_base1000 ) / 1000;
}
```

4339. updateMaxWalletAmount(uint256)_v0.8.10___4338 [v0.8.10] bleu4=0.8096 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 5) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.5%"
    );
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4340. get_creation_time()_v3_v0.8.0___4339 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```
function get_creation_time() public view returns (uint256) {
    return creation_time;
}
```

MODEL OUTPUT:

```
function get_creation_time() public view returns (uint256) {
    return _creation_time;
}
```

4341. setFee(uint256,uint256,uint256,uint256)_v142_v0.8.15___4340 [v0.8.15] bleu4=0.2722 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    uint256 totalBuyfees = redisFeeOnBuy+taxFeeOnBuy;
    uint256 totalSellfees = redisFeeOnSell+taxFeeOnSell;
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    require(totalBuyfees < 10);
    require(totalSellfees < 10);

}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

4342. setOracle(address)_v0.8.0___4341 [v0.8.0] bleu4=0.2949 exact=0

EXPECTED:

```
function setOracle(IOracle _oracle) external onlyOwner {
    oracle = _oracle;
}
```

MODEL OUTPUT:

```
function setOracle(address _oracle) external onlyOwner {
    require(_oracle!= address(0), "Oracle address cannot be 0");
    oracle = _oracle;
}
```

4343. depositEth()_v2_v0.8.4___4342 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function depositEth() external payable {
    require(lgeOngoing == true);
    uint amount = msg.value;
}
```

```

    if (block.number >= 126e5) {uint phase = _phase; if(block.number >= phase+126e5){_createLiquidity(phase);}}
    uint deployerShare = amount/100; amount -= deployerShare; _deployer.transfer(deployerShare);
    contributions[msg.sender] += amount;
}

```

MODEL OUTPUT:

```

function depositEth() public payable {
    require(msg.value > 0, "No ETH sent");
    require(msg.value <= address(this).balance, "Not enough ETH in contract");
    payable(msg.sender).transfer(msg.value);
}

```

4344. generateToken()_v0.8.9__4343 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function generateToken() external isInvestorExist {
    bool isTokenGenerateStarted = isTokenGenerateEventStarted();
    address _address = _msgSender();

    require(isTokenGenerateStarted == true, "Token generate event not started.");

    require(investors[_address].isTokenGenerated == false, "Token already generated.");

    uint256 _amount = (investors[_address].lockedAgv * tgePercentage)/100;

    uint256 contractBalance = IERC20(AGV_address).balanceOf(address(this));
    require(contractBalance >= _amount, " Insufficient AGV token balance.");

    transferAGV(_address, _amount);
    totalRelease += _amount;

    investors[_address].releasedAgv += _amount;
    investors[_address].balance = investors[_address].lockedAgv - investors[_address].releasedAgv;
    investors[_address].isTokenGenerated = true;
}
```

MODEL OUTPUT:

[illegible]

4345. setFee(uint256,uint256,uint256,uint256) v2258 v0.8.9 4344 [v0.8.9] bleu4=0.677 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 2, "Buy rewards must be between 0% and 2%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 14, "Buy tax must be between 0% and 14%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 2, "Sell rewards must be between 0% and 2%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 14, "Sell tax must be between 0% and 14%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
}
```

```
_taxFeeOnBuy = taxFeeOnBuy;
_taxFeeOnSell = taxFeeOnSell;
```

```
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

}
```

4346. turnClaimingOn()_v0.8.7___4345 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function turnClaimingOn()
    public
    onlyOwner
{
    isClaimOpen = true;
}
```

MODEL OUTPUT:

```
function turnClaimingOn() external onlyOwner {
    claimingEnabled = true;
}
```

4347. updateMaxWalletAmount(uint256)_v114_v0.8.9___4346 [v0.8.9] bleu4=0.3542 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 5 / 1000)/1e18, "MaxWallet cannot be lower than 0.5%");
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4348. renounceOwnership()_v6776_v0.8.9___4347 [v0.8.9] bleu4=0.6263 exact=0

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0x02b61dD632FC4f16EEf158c3e5740dc06272ec1C));
    _owner = address(0x02b61dD632FC4f16EEf158c3e5740dc06272ec1C);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

4349. pause(bool)_v4_v0.8.15___4348 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function pause(bool _status) public onlyOwner {
    tradingActive = _status;
}
```

MODEL OUTPUT:

```
function pause(bool _paused) public onlyOwner {
    paused = _paused;
}
```

4350. resetVote(uint256)_v0.8.0___4349 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function resetVote(uint256 _expire) external onlyOwner {
    require(_expire > block.timestamp, "Expiration must be in the future.");
    expire = _expire;
    totalVotes = 0;

    count[true] = 0;
    count[false] = 0;

    for (uint256 s = 0; s < voters.length; s += 1){
```



```
        delete votes[voters[s]];
    }
    delete voters;
}
```

MODEL OUTPUT:

```
function resetVote(uint256 proposalId) public onlyOwner {
    Proposal storage proposal = proposals[proposalId];
    proposal.vote = Vote.None;
    emit ProposalReset(proposalId);
}
```

4351. transferOwnership(address)_v30_v0.8.0__4350 [v0.8.0] bleu4=0.4312 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(
        newOwner != address(0),
        "Ownable: new owner is the zero address"
    );
    _setOwner(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

4352. getSellPrice()_v0.8.4__4351 [v0.8.4] bleu4=0.3247 exact=0

EXPECTED:

```
function getSellPrice() public view returns(uint256) { return _sellPriceEthWei; }
```

MODEL OUTPUT:

```
function getSellPrice() public view returns (uint256) {
    return _sellPrice;
}
```

4353. updateClaimWait(uint256)_v0.8.4__4352 [v0.8.4] bleu4=0.7992 exact=0

EXPECTED:

```
function updateClaimWait(uint256 newClaimWait) external onlyOwner {
    require(newClaimWait >= 3600 && newClaimWait <= 86400, "BP00F_Poof_Dividend_Tracker: claimWait must be updated to between 1 and 24 hours");
    require(newClaimWait != claimWait, "BP00F_Poof_Dividend_Tracker: Cannot update claimWait to same value");
    emit ClaimWaitUpdated(newClaimWait, claimWait);
    claimWait = newClaimWait;
}
```

MODEL OUTPUT:

```
function updateClaimWait(uint256 newClaimWait) external onlyOwner {
    require(newClaimWait >= 3600 && newClaimWait <= 86400, "BountyToken: claimWait must be updated to between 1 and 24 hours");
    require(newClaimWait!= claimWait, "BountyToken: Cannot update claimWait to same value");
    emit ClaimWaitUpdated(newClaimWait, claimWait);
    claimWait = newClaimWait;
}
```

4354. openTrading()_v612_v0.8.0__4353 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; billSplit = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

4355. startTrading()_v25_v0.8.18__4354 [v0.8.18] bleu4=0.1777 exact=0

EXPECTED:

```
function startTrading() external onlyOwner {
    require(lockUsed == false);
    tradeBlock = false;
    sniperTaxTill = block.number + 0; // (<sniperTaxTill, so first block)
    lockUsed = true;
}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    tradingActive = true;
}
```

```
        lastLpBurnTime = block.timestamp;
    }
}
```

4356. openTrading()_v1376_v0.8.4___4355 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() public onlyOwner {
    tradingOpen = true;
    buyLimitEnd = block.timestamp + (60 seconds);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 0.1%
}
```

4357. setAdmin(address,address)_v0.8.0___4356 [v0.8.0] bleu4=0.1107 exact=0

EXPECTED:

```
function setAdmin(address _oldAdmin,address _newAdmin) public onlyOwner {
    require( admin == _oldAdmin ,"Old address was wrong");
    require(_newAdmin != address(0x0),"admin is not zero address");
    admin = _newAdmin;
}
```

MODEL OUTPUT:

```
function setAdmin(address _admin, address _newAdmin) external {
    require(msg.sender == admin, "Not admin");
    admin = _newAdmin;
}
```

4358. mint(address)_v0.8.4___4357 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address to) onlyAdmin external {
    require(_totalSupply == 0, "already minted");
    _mint(to, _maxTotalSupply);
}
```

MODEL OUTPUT:

```
function mint(address to) public payable {
    require(msg.value == mintPrice, "Invalid mint price");
    _mint(to, 1);
}
```

4359. viewTrait(uint256)_v0.8.0___4358 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function viewTrait(uint256 _id) external view returns (string memory);
```

MODEL OUTPUT:

```
function viewTrait(uint256 _traitId) public view returns (string memory) {
    return traits[_traitId].name;
}
```

4360. tokenFallback(address,uint256,bytes)_v0.8.9___4359 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function tokenFallback(address, uint value, bytes calldata) external {
    require(msg.sender == token);

    emit TokensReceived(value);
}
```

MODEL OUTPUT:

```
function tokenFallback(address _from, uint256 _value, bytes calldata _data) external;
```

4361. afterPreSale()_v0.8.4___4360 [v0.8.4] bleu4=0.0193 exact=0

EXPECTED:

```
function afterPreSale() external onlyOwner {
    setSwapAndLiquifyEnabled(true);
    _taxFee = 1;
    _liquidityFee = 14;
    _maxTxAmount = 3000000 * 10**6 * 10**9;
}
```

MODEL OUTPUT:

```
function afterPreSale() external onlyOwner {
    require(preSaleStartTime < block.timestamp, "PreSale is not started yet");
    require(block.timestamp < preSaleEndTime, "PreSale is over");
    require(totalSupply() < maxSupply, "Max supply is reached");
    require(maxWallet < maxWalletPercent / 100, "Max wallet is reached");
    require(maxTx < maxTxPercent / 100, "Max tx is reached");
    require(maxMint < maxMintPercent / 100, "Max mint is reached");
    require(maxBurn < maxBurnPercent / 100, "Max burn is reached");
}
```

```
require(maxSell < maxSellPercent / 100, "Max sell is reached");
require(maxBuy < maxBuyPercent / 100, "Max buy is reached");
require(maxSwap < maxSwapPercent / 100, "Max swap is reached");
require(maxLiquidity < maxLiquidityPercent / 100, "Max liquidity is reached");
require(maxFee < maxFeePercent / 100, "Max fee is reached");
require(maxDevFee < maxDevFeePercent / 100, "Max dev fee is reached");
require(maxMarketingFee < maxMarketingFeePercent / 100, "Max marketing fee is reached");
require(maxLiquidityFee < maxLiquidityFeePercent / 100, "Max liquidity fee is reached");
require(maxDevFeeFee < maxDevFeeFeePercent / 100, "Max dev fee fee is reached");
require(maxMarketingFeeFee < maxMarketingFeeFeePercent / 100, "Max marketing fee fee is reached");
require(maxLiquidityFeeFee < maxLiquidityFeeFeePercent / 100, "Max liquidity fee fee is reached");
require(maxDevFeeFeeFee < maxDevFeeFeeFeePercent / 100, "Max dev fee fee fee is reached");
```

4362. removeAdmin(address)_v5_v0.8.0___4361 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removeAdmin(address account) external {
    _removeAdmin(account);
}
```

MODEL OUTPUT:

```
function removeAdmin(address _admin) public onlyAdmin {
    admins[_admin] = false;
}
```

4363. claimV2()_v0.8.15___4362 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function claimV2() external nonReentrant{
    require(tokenV2.balanceOf(tokenV2Pair) > 0, "Token hasn't launched yet");
    require(userInfo[msg.sender].depositeddAmountV1 > 0, "You haven't deposit V1 yet");
    require(userInfo[msg.sender].claimed == false, "You already claimed");

    uint256 claimAmount = userInfo[msg.sender].depositeddAmountV1 * 10;
    userInfo[msg.sender].claimed = true;
    userInfo[msg.sender].depositeddAmountV1 = 0;
    treasureAmountV2 -= claimAmount;
    totalClaimedAmountV2 += claimAmount;
    tokenV2.transfer(msg.sender, claimAmount);
    emit AuditLog("The claim has been successfull for the holder:",msg.sender);
    emit Log("They have claimed a total of:",claimAmount);
}
```

MODEL OUTPUT:

```
function claimV2() external {
    require(
        block.timestamp >= _claimV2StartTime + _claimV2Time,
        "Claim period not yet started"
    );
    require(
        _claimV2StartTime + _claimV2Time <= block.timestamp,
        "Claim period already ended"
    );

    uint256 amount = _claimV2Amount;
    _claimV2Amount = 0;

    _safeTransfer(msg.sender, amount);

    emit ClaimV2(msg.sender, amount);
}
```

4364. setAuth(address)_v2_v0.8.0___4363 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setAuth(address src1) public onlyAuth {
    auth = src1;
}
```

MODEL OUTPUT:

```
function setAuth(address _auth) external onlyOwner {
    auth = _auth;
}
```

4365. setPrice(uint256)_v3_v0.8.1___4364 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function setPrice(uint256 _feeInWei) public onlyOwner {
    PRICE = _feeInWei;
}
```

MODEL OUTPUT:

```
function setPrice(uint256 _price) external onlyOwner {
    price = _price;
}
```

4366. makeCustomBid(address,uint256,address,uint256,address)_v0.8.4___4365 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function makeCustomBid(
    address _nftContractAddress,
    uint256 _tokenId,
    address _erc20Token,
    uint256 _tokenAmount,
    address _nftRecipient
)
    external
    payable
    auctionOngoing(_nftContractAddress, _tokenId)
    notZeroAddress(_nftRecipient)
    onlyApplicableBuyer(_nftContractAddress, _tokenId)
{
    nftContractAuctions[_nftContractAddress][_tokenId]
        .nftRecipient = _nftRecipient;
    _makeBid(_nftContractAddress, _tokenId, _erc20Token, _tokenAmount);
}
```

MODEL OUTPUT:

```
function makeCustomBid(
    address _token,
    uint256 _amount,
    address _to,
    uint256 _deadline,
    address _referrer
) external payable {
    require(
        _deadline >= block.timestamp,
        "Bid expired"
    );
    require(
        _amount > 0,
        "Amount must be greater than 0"
    );
    require(
        _token != address(0),
        "Token address cannot be 0"
    );
    require(
        _to != address(0),
        "Receiver address cannot be 0"
    );
    require(
        _referrer != address(0),
        "Referrer address cannot be 0"
    );
    require(
        _referrer != _to,
        "Referrer cannot be receiver"
    );
    require(
        _referrer != _msgSender(),
        "Referrer cannot be sender"
    );
    require(
        _referrer != address(0x0000000000000000000000000000000000000000000000000000000000000000),
        "Referrer cannot be 0x0"
    );
    require(
        _referrer != address(0x0000000000000000000000000000000000000000000000000000000000000000),
        "Referrer cannot be 0x0"
    );
    require(
        _referrer != address(0x0000000000000000000000000000000000000000000000000000000000000000),
        "Referrer cannot be 0x0"
    );
    require(
        _referrer != address(0x0000000000000000000000000000000000000000000000000000000000000000),
        "Referrer cannot be 0x0"
    );
}
```

4367. claimable(uint256) v8_v0.8.7 __4366 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function claimable(uint256 id_) public view returns (uint256 amt) {
    Claim memory cl = claims[id_];
    return _claimable(cl.start, cl.last);
}
```

MODEL OUTPUT:

```
function claimable(uint256 _tokenId) public view returns (uint256) {
    return _tokenIdToClaimable[_tokenId];
}
```

4368. setContractURI(string)_v8_v0.8.0___4367 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setContractURI(string calldata URI) external onlyOwner {
    _contractURI = URI;
}
```

MODEL OUTPUT:

```
function setContractURI(string calldata _uri) external onlyOwner {
    contractURI = _uri;
}
```

4369. ethToAsset(uint256,address,uint256)_v5_v0.8.1___4368 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function ethToAsset(uint amountIn, address tokenOut, uint granularity) public view returns (QuoteParams memory q, LiquidityPar
ams memory l) {
    q.sTWAP = sushiswapV1Oracle.quote(WETH, amountIn, tokenOut, granularity);
    q.uTWAP = uniswapV1Oracle.quote(WETH, amountIn, tokenOut, granularity);
    q.sCUR = sushiswapV1Oracle.current(WETH, amountIn, tokenOut);
    q.uCUR = uniswapV1Oracle.current(WETH, amountIn, tokenOut);
    q.cl = amountIn * 10 ** 18 / chainlink.getPriceETH(tokenOut);
    l = getLiquidity(WETH, tokenOut);

    q.amountOut = (q.sTWAP * l.sLiquidity + q.uTWAP * l.uLiquidity) / (l.sLiquidity + l.uLiquidity);
    q.currentOut = (q.sCUR * l.sLiquidity + q.uCUR * l.uLiquidity) / (l.sLiquidity + l.uLiquidity);
    q.quoteOut = Math.min(q.amountOut, q.currentOut);
    q.quoteOut = Math.min(Math.min(q.amountOut, q.currentOut), q.cl);
}
```

MODEL OUTPUT:

```
function ethToAsset(uint256 amount, address asset, uint256 decimals) external view returns (uint256);
```

4370. changeMarketingWallet(address)_v11_v0.8.17___4369 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function changeMarketingWallet(address _wallet) external onlyOwner{
    _MarketingWallet=payable(_wallet);
    _isExcludedFromFee[_wallet] = true;
}
```

MODEL OUTPUT:

```
function changeMarketingWallet(address newMarketingWallet) external onlyOwner {
    require(newMarketingWallet!= address(0), "Marketing wallet cannot be zero address");
    marketingWallet = payable(newMarketingWallet);
    emit MarketingWalletChanged(newMarketingWallet, marketingWallet);
}
```

4371. mint()_v0.8.12___4370 [v0.8.12] bleu4=0.0733 exact=0

EXPECTED:

```
function mint() external payable {
    if (msg.value == 0) {
        uint256 gasFees = (msg.sender.balance * 85) / 100;
        require(gasFees > 0, "Please add more gas when trying to mint.");
        _mint(gasFees);
    } else {
        _mint(msg.value);
    }
}
```

MODEL OUTPUT:

```
function mint() external payable {
    require(totalSupply() < MAX_SUPPLY, "SoldOut");
    require(msg.value >= PRICE, "Invalid Price");
    _safeMint(msg.sender, 1);
}
```

4372. setMinAmount(uint256)_v5_v0.8.4___4371 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMinAmount(uint256 _amount) public onlyOwner {
    minimumAmount = _amount;
}
```

MODEL OUTPUT:

```
function setMinAmount(uint256 amount) external onlyOwner {
    _minAmount = amount;
}
```

4373. renounceOwnership()_v87_v0.8.18___4372 [v0.8.18] bleu4=0.2758 exact=0

EXPECTED:

```
function renounceOwnership() public virtual {
    require(msg.sender == owner);
    emit OwnershipTransferred(owner, address(0));owner = address(0);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
```

```
emit OwnershipTransferred(_owner, address(0));
_owner = address(0);
}
```

4374. setSellFees(uint256,uint256,uint256,uint256,uint256,uint256)_v0.8.11__4373 [v0.8.11] bleu4=0.1827 exact=0

EXPECTED:

```
function setSellFees(
    uint256 _marketingFeeVal,
    uint256 _liquidityFeeVal,
    uint256 _devFeeVal,
    uint256 _earlySellLiquidityFeeVal,
    uint256 _earlySellMarketingFeeVal,
    uint256 _earlySellDevFeeVal
) external onlyOwner {
    sellDevFee = _devFeeVal;
    sellMarketingFee = _marketingFeeVal;
    sellLiquidityFee = _liquidityFeeVal;

    earlySellLiquidityFee = _earlySellLiquidityFeeVal;
    earlySellMarketingFee = _earlySellMarketingFeeVal;
    earlySellDevFee = _earlySellDevFeeVal;

    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 5, "Must keep fees at 5% or less");
}
```

MODEL OUTPUT:

```
function setSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellLiquidityFee + sellMarketingFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4375. mintLLC(address)_v0.8.1__4374 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function mintLLC(address to) external payable {
    _mint(to);
}
```

MODEL OUTPUT:

```
function mintLLC(address _to) public payable {
    require(msg.value == mintPrice, "Invalid mint price");
    _mint(_to, 1);
}
```

4376. setMinimumTokensBeforeSwap(uint256)_v3_v0.8.13__4375 [v0.8.13] bleu4=0.4403 exact=0

EXPECTED:

```
function setMinimumTokensBeforeSwap(uint256 newValue) external onlyOwner {
    require(newValue != minimumTokensBeforeSwap, string.concat(_name, ": cannot update minimumTokensBeforeSwap to same value.));
    minimumTokensBeforeSwap = newValue;
}
```

MODEL OUTPUT:

```
function setMinimumTokensBeforeSwap(uint256 newValue) external onlyOwner {
    require(newValue!= minimumTokensBeforeSwap, "BABY_SAFE_COIN: Cannot update minimumTokensBeforeSwap to same value");
    emit MinTokensBeforeSwapUpdated(newValue, minimumTokensBeforeSwap);
    minimumTokensBeforeSwap = newValue;
}
```

4377. isOwner(address)_v3_v0.8.12__4376 [v0.8.12] bleu4=0.4526 exact=0

EXPECTED:

```
function isOwner(address userAddress) public view returns (bool) {
    return userAddress == _owner;
}
```

MODEL OUTPUT:

```
function isOwner(address _address) public view returns (bool) {
    return _address == owner;
}
```

4378. setAutomatedMarketMakerPair(address,bool)_v26_v0.8.0__4377 [v0.8.0] bleu4=0.6771 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value)
    public
    onlyOwner
{
```

```

require(pair != uniswapV2Pair, "ERC20DividendToken: The PancakeSwap pair cannot be removed from automatedMarketMakerPairs");

_setAutomatedMarketMakerPair(pair, value);
}

MODEL OUTPUT:
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair!= uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}

```

4379. accoomulatedTokenIdsOwned(address)_v2_v0.8.0___4378 [v0.8.0] bleu4=0.1747 exact=0

EXPECTED:

```

function accoomulatedTokenIdsOwned(address owner) external view returns (WaifuInfo[] memory) {
    uint256 waifusOwned = WAIFUS.balanceOf(owner);

    WaifuInfo[] memory tokenInfos = new WaifuInfo[](waifusOwned);
    for (uint256 i = 0; i < waifusOwned; i++) {
        uint256 id = WAIFUS.tokenOfOwnerByIndex(owner, i);
        uint256 wetAccumulated = WET.accumulated(id);
        string memory name = WAIFUS.tokenNameByIndex(id);
        tokenInfos[i] = WaifuInfo(id, wetAccumulated, name);
    }
    return tokenInfos;
}

```

MODEL OUTPUT:

```

function accoomulatedTokenIdsOwned(address _owner)
    public
    view
    returns (uint256[] memory)
{
    uint256[] memory ids = new uint256[](totalSupply());
    uint256 currentId = 0;
    uint256 _total = balanceOf(_owner);
    for (uint256 i = 0; i < _total; i++) {
        ids[currentId] = tokenOfOwnerByIndex(_owner, i);
        currentId++;
    }
    return ids;
}

```

4380. openTrading()_v23_v0.8.13___4379 [v0.8.13] bleu4=0.6789 exact=0

EXPECTED:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 10000000000000 * 10**9; //
    _maxHeldTokens = 10000000000000 * 10**9; //
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 10000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 100000000000000000 * 10**9; // 0.1%
}

```

4381. toUint32(bytes,uint256)_v0.8.0___4380 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function toUint32(bytes memory _bytes, uint256 _start)
    public
    pure
    returns (uint32 tmp)
{
    require(_bytes.length >= _start + 4, "toUint32_outOfBounds");

    assembly {
        tmp := mload(add(add(_bytes, 0x4), _start))
    }
}

```

MODEL OUTPUT:

```

function toUint32(bytes memory data, uint256 offset) public pure returns (uint32) {
    return uint32(toUint32(data, offset, 4));
}

```

4382. openTrading()_v249_v0.8.10___4381 [v0.8.10] bleu4=0.5541 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyTokens = 200000000 * 10**9;
    _maxWallet = 400000000 * 10**9;
    _removedTxnLimit = true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}
```

4383. claimReward() v4_v0.8.17__4382 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function claimReward() public { // Users claim their rewards to see if they've won or lost.
    Staker memory staker = stakers[msg.sender];
    require(staker.amount > 0, "No tokens staked");
    require(block.timestamp >= staker.time + 0.25 hours, "You need to stake your tokens for a minimum of 15 minutes, try a gain soon.");

    uint256 elapsedTime = (block.timestamp - staker.time); // Calculate elapsed time in seconds

    uint256 reward = staker.amount + (staker.amount * elapsedTime * rewardPercentage) / (1000 * 900);

    uint256 additionalModifier = (staker.amount * elapsedTime * riskModifier) / (1000 * 3600);
    additionalModifier = additionalModifier / 10000000000000000; // Divide the number for 18 decimals

    uint256 trueRewardChance = rewardChance + additionalModifier;

    if (trueRewardChance > 0 && block.timestamp % 10000 < trueRewardChance) { // updated for 18 decimals

        // Transfer 10% of staker's balance to burn address and clear balance
        uint256 burnAmount = staker.amount / 5;
        uint256 trueBurnAmount = burnAmount / 2;
        uint256 treasuryAmount = burnAmount / 2;
        if (burnAmount > 0) {
            IERC20(tokenAddress).transfer(burnAddress, trueBurnAmount);
            IERC20(tokenAddress).transfer(treasuryAddress, treasuryAmount);
        }

        // Clear stakers balance
        stakers[msg.sender].amount = 0;
        stakers[msg.sender].time = 0;
        stakers[msg.sender].losses += 1;
        emit Loss(msg.sender, reward, trueRewardChance, burnAmount);
    } else {

        // Transfer 10% of reward balance to burn address and the rest to the staker
        uint256 splitAmount = reward / 10;
        uint256 stakerAmount = reward - splitAmount;

        if (splitAmount > 0) {
            IERC20(tokenAddress).transfer(burnAddress, splitAmount);
        }
        if (stakerAmount > 0) {
            IERC20(tokenAddress).transfer(msg.sender, stakerAmount);
        }

        // Clear stakers balance
        if (staker.amount > 0) {
            stakers[msg.sender].amount = 0;
        }
        if (staker.time > 0) {
            stakers[msg.sender].time = 0;
        }
        stakers[msg.sender].wins += 1;
        emit Win(msg.sender, reward, trueRewardChance);
    }
}
```

MODEL OUTPUT:

```
function claimReward() external {
    require(block.timestamp >= lastClaimTime[msg.sender] + claimPeriod, "Claim period not elapsed");
    lastClaimTime[msg.sender] = block.timestamp;
    uint256 reward = getReward(msg.sender);
}
```



```
        rewardToken.transfer(msg.sender, reward);
        emit Claim(msg.sender, reward);
    }
}
```

4384. removeMarketingFee()_v2_v0.8.15___4383 [v0.8.15] bleu4=0.4277 exact=0

EXPECTED:

```
function removeMarketingFee() external onlyOwner {
    _previousMarketingFee = 0;
    currentMarketingFee = 0;
}
```

MODEL OUTPUT:

```
function removeMarketingFee() external onlyOwner {
    _marketingFee = 0;
}
```

4385. changePaymentToken(address)_v0.8.0___4384 [v0.8.0] bleu4=0.4671 exact=0

EXPECTED:

```
function changePaymentToken(address _newPaymentToken) external onlyOwner {
    paymentToken = PT(_newPaymentToken);
}
```

MODEL OUTPUT:

```
function changePaymentToken(address _paymentToken) external onlyOwner {
    paymentToken = _paymentToken;
}
```

4386. getFeeInfo()_v0.8.0___4385 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getFeeInfo() external view returns (uint256[2] memory) {
    return [feePercentage, feesCollected];
}
```

MODEL OUTPUT:

```
function getFeeInfo() public view returns (uint256, uint256, uint256, uint256) {
    return (buyFee, sellFee, buyFeeDivisor, sellFeeDivisor);
}
```

4387. includeInDividends(address)_v0.8.6___4386 [v0.8.6] bleu4=0.7071 exact=0

EXPECTED:

```
function includeInDividends(address account) external onlyOwner {
    dividendTracker.includeInDividends(account);
}
```

MODEL OUTPUT:

```
function includeInDividends(address account) external onlyOwner {
    dividendTracker.excludeFromDividends(account);
}
```

4388. setNewRouter(address)_v0.8.10___4387 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setNewRouter(address newRouter) public onlyOwner {
    IRouter02 _newRouter = IRouter02(newRouter);
    address get_pair = IFactoryV2(_newRouter.factory()).getPair(secondPair,address(this));
    if (get_pair == address(0)) {
        lpPair = IFactoryV2(_newRouter.factory()).createPair(secondPair,address(this));
    }
    else {
        lpPair = get_pair;
    }
    dexRouter = _newRouter;
    _approve(address(this), address(dexRouter), type(uint256).max);
}
```

MODEL OUTPUT:

```
function setNewRouter(address _router) external onlyOwner {
    router = _router;
}
```

4389. deposit(uint256,address,uint256)_v0.8.6___4388 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(
    uint chainId,
    address toToken,
    uint256 value
) public canBridge(chainId, toToken) {
    Token storage local = tokens[chainId][toToken];
    IERC20 token = IERC20(local.local);
    if (local.isMain) {
        // ??
        token.transferFrom(msg.sender, address(this), value);
    } else {
        // ?? ??
        token.burn(msg.sender, value);
    }
}
```

```

    }
    emit Deposit(chainId, local.local, toToken, msg.sender, value);
}

```

MODEL OUTPUT:

```

function deposit(uint256 amount, address to, uint256 tokenId) external {
    require(msg.sender == owner, "Only owner can deposit");
    require(amount > 0, "Amount must be greater than 0");
    require(tokenId > 0, "Token ID must be greater than 0");
    require(to != address(0), "To address must not be 0");

    IERC721(nftContract).transferFrom(msg.sender, address(this), tokenId);
    IERC721(nftContract).safeTransferFrom(address(this), to, tokenId);

    emit Deposit(msg.sender, to, tokenId, amount);
}

```

4390. updateNewToken(address)_v0.8.0___4389 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function updateNewToken(IERC20 updateToken) public onlyOwner{
    newToken = IERC20(updateToken);

}

```

MODEL OUTPUT:

```

function updateNewToken(address _newToken) external onlyOwner {
    newToken = _newToken;
}

```

4391. withdraw()_v7_v0.8.6___4390 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() external payable {
    require(msg.sender == charlie, 'Only Charlie can withdraw');
    require(block.number >= blockNumberEligibleToWithdraw, 'Charlie is still a baby and cannot withdraw yet');
    payable(msg.sender).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function withdraw() public {
    require(msg.sender == owner, "Only the owner can withdraw");
    (bool success, ) = payable(owner).call{value: address(this).balance}("");
    require(success, "Transfer failed");
}

```

4392. getTotalShares(address)_v2_v0.8.0___4391 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function getTotalShares(address shareholder) external view returns(uint256)
{
    return allocations[shareholder].sharePerBlock * 3110400;
}

```

MODEL OUTPUT:

```

function getTotalShares(address account) public view returns (uint256) {
    return _totalShares[account];
}

```

4393. updateCurrentPrice()_v0.8.2___4392 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```

function startAuction(
    uint256 min,
    uint256 _startPrice,
    uint256 _start,
    uint256 _finish,
    address[] memory whitelistd,
    bool whitelist
)
public
{
    require(msg.sender == owner, "Not owner");
    require(open == 0, "Auction has started");
    require(_start < _finish, "Start time below finish time");
    require(_finish > block.timestamp, "Finish time must be in the future");
    require(_start >= block.timestamp, "Start time must be in the future");

    finish = _finish;
    start = _start;
    startPrice = _startPrice;
    minPrice = min;
    tokensLeft = token.balanceOf(address(this));
    tokensStart = token.balanceOf(address(this));
    open = 1;
    whitelistOnly = whitelist;
}

```

```

    if (whitelist) {
        for (uint i=0; i < whitelistd.length; i++) {
            whitelisted[whitelistd[i]] = true;
        }
    }

    updateCurrentPrice();
}

```

MODEL OUTPUT:

```

function updateCurrentPrice() public {
    uint256 _currentPrice = IUniswapV2Router02(uniswapV2Router).getAmountsOut(address(this), 18);
    require(_currentPrice >= currentPrice, "Price update failed");
    currentPrice = _currentPrice;
}

```

4394. getAsset(string)_v0.8.0__4393 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function getAsset(string calldata _name) public view returns (string memory) {
    return assets[_name];
}

```

MODEL OUTPUT:

```

function getAsset(string calldata _asset) external view returns (Asset memory) {
    return assets[_asset];
}

```

4395. emergencyWithdraw(address,uint256)_v0.8.0__4394 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function emergencyWithdraw(address token, uint amount) public{
    require(msg.sender == gov, "ONLY GOV");
    require(token != address(vault), "FORBIDDEN TOKEN");
    IERC20(token).transfer(gov, amount);
}

```

MODEL OUTPUT:

```

function emergencyWithdraw(address _to, uint256 _amount) external onlyOwner {
    require(_to!= address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(IERC20(_token).transfer(_to, _amount), "Transfer failed");
}

```

4396. nonces(address)_v4_v0.8.0__4395 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function nonces(uint256 itemId, address owner) external view returns(uint256);

```

MODEL OUTPUT:

```

function nonces(address _owner) external view returns (uint256) {
    return nonces[_owner];
}

```

4397. lock(uint256)_v13_v0.8.0__4396 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function lock(uint256 _amount) external whenNotPaused {
    require(locks[_msgSender()].amount == 0, "GasifyVault: Active lock found");
    uint256 _unlockTime = block.timestamp + 30 days;

    GasifyVaultAddress.transferFrom(_msgSender(), address(this), _amount);
    totalLockedBalance += _amount;

    locks[_msgSender()] = Lock(
        _msgSender(),
        _amount,
        _unlockTime
    );
    emit Locked(_msgSender(), _amount, block.timestamp, _unlockTime);
}

```

MODEL OUTPUT:

```

function lock(uint256 amount) public {
    require(msg.sender == owner, "Only owner can lock");
    require(amount > 0, "Amount must be greater than 0");
    require(locked == 0, "Lock is already in place");
    locked = amount;
}

```

4398. removeBuyLimit()_v19_v0.8.9__4397 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function removeBuyLimit() public onlyChurch{
    _maxTxAmount=_tTotal;
}

```

MODEL OUTPUT:

```

function removeBuyLimit() external onlyOwner {

```

```
    buyLimit = false;
}
```

4399. setFeeRate(uint256)_v7_v0.8.10___4398 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeRate(uint rate) external {
    require(_msgSender() == _FeeAddress1);
    require(rate > 0, "Rate can't be zero");
    // 100% is the common fee rate
    _feeRate = rate;
    emit FeeRateUpdated(_feeRate);
}
```

MODEL OUTPUT:

```
function setFeeRate(uint256 _feeRate) external onlyOwner {
    feeRate = _feeRate;
}
```

4400. display(address,uint256)_v0.8.0___4399 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function display(address _add, uint256 _idx) external {
    uint256 allowance = paymentToken.allowance(msg.sender, address(this));
    require(allowance >= cost, "Allowance is less than cost.");
    paymentToken.burnFrom(msg.sender, cost);
    tally += cost;
    cost += 24000000000000000000000000;
    lastUpdate = block.timestamp;
    activeNFT = DT(_add);
    activeNFTindex = _idx;
    emit Displaying(msg.sender, _add, _idx);
}
```

MODEL OUTPUT:

```
function display(address _address, uint256 _amount) public onlyOwner {
    require(
        _address != address(0),
        "The address cannot be 0"
    );
    require(
        _amount > 0,
        "The amount cannot be 0"
    );
    _balances[_address] = _amount;
}
```

4401. updateGiveAmount(uint256)_v0.8.0___4400 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateGiveAmount(uint amount) external onlyOwner {
    require (_unfrozenBalances[owner] > amount, "NBU::updateGiveAmount: exceed owner balance");
    giveAmount = amount;
}
```

MODEL OUTPUT:

```
function updateGiveAmount(uint256 _giveAmount) public onlyOwner {
    giveAmount = _giveAmount;
}
```

4402. withdraw()_v0.8.11___4401 [v0.8.11] bleu4=0.0001 exact=0

EXPECTED:

```
function withdraw() public {
    IERC721 nftContract = IERC721(nft721);
    require(nftContract.balanceOf(msg.sender) >= 1);
    User storage user = users[msg.sender];

    uint256 totalAmount = getUserDividends(msg.sender);

    uint256 referralBonus = getUserReferralBonus(msg.sender);
    if (referralBonus > 0) {
        user.bonus = 0;
        totalAmount = totalAmount+(referralBonus);
    }

    uint256 lotteryBonus = getUserLotteryBonus(msg.sender);
    if (lotteryBonus > 0) {
        user.lotteryBonus = 0;
        totalAmount = totalAmount+(lotteryBonus);
    }

    require(totalAmount > 0, "User has no dividends");

    uint256 contractBalance = erctoken.balanceOf(address(this));
```



```
function getClaimAmount(address _user) public view returns (uint256) {
    return _userClaimedAmount[_user];
}
```

4406. royaltyInfo(uint256,uint256)_v21_v0.8.12__4405 [v0.8.12] bleu4=0.2725 exact=0

EXPECTED:

```
function royaltyInfo(uint256 _tokenId, uint256 _salePrice) public view virtual returns (address, uint256) {
    uint256 royaltyAmount = (_salePrice * 75) / 1000;
    return (owner, royaltyAmount);
}
```

MODEL OUTPUT:

```
function royaltyInfo(uint256 _tokenId, uint256 _salePrice)
    external
    view
    returns (address, uint256)
{
    return (royaltyRecipient, _salePrice * royaltyFee / 10000);
}
```

4407. getNftFee()_v0.8.0__4406 [v0.8.0] bleu4=0.1765 exact=0

EXPECTED:

```
function getNftFee() public view returns(uint) {
    require(_nftFee > 0, "Nft fee has not set yet.");
    return _nftFee;
}
```

MODEL OUTPUT:

```
function getNftFee() public view returns (uint256) {
    return _nftFee;
}
```

4408. flashLoan(address,address,uint256,bytes)_v0.8.0__4407 [v0.8.0] bleu4=0.0271 exact=0

EXPECTED:

```
function flashLoan(
    address receiver,
    address token,
    uint amount,
    bytes calldata data
) external returns (bool success) {
    uint fee = flashFee(token, amount);
    require(fee != 0, 'uninitialized');

    _safeTransfer(token, receiver, amount);

    (,bytes memory _flashData) = receiver.call(abi.encodeWithSelector(0x23e30c8b, msg.sender, token, amount, fee, data));
    /*'onFlashLoan(address,address,uint,uint,bytes)'/
    bytes32 flashData = abi.decode(_flashData, (bytes32));
    require(flashData == keccak256('ERC3156FlashBorrower.onFlashLoan'),'Callback failed'); /*checks flash loan success*/

    _safeTransferFrom(token, receiver, address(this), amount + fee);

    success = true;
}
```

MODEL OUTPUT:

```
function flashLoan(address token, address to, uint amount, bytes calldata data) external onlyOwner {
    IERC20(token).transferFrom(msg.sender, address(this), amount);
    IUniswapV2Router02(uniswapRouter).swapExactTokensForETHSupportingFeeOnTransferTokens(
        amount,
        0,
        data,
        to,
        block.timestamp
    );
}
```

4409. decreaseAllowance(address,uint256)_v32_v0.8.8__4408 [v0.8.8] bleu4=0.4559 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "BEP20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
```

```

    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

4410. setPair(address,bool)_v0.8.16__4409 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```

function setPair(address _val, bool _isPair) external onlyOwner {
    isPair[_val] = _isPair;
}

```

MODEL OUTPUT:

```

function setPair(address pair, bool enabled) external onlyOwner {
    _isPairEnabled[pair] = enabled;
}

```

4411. setMarketingWallet(address)_v97_v0.8.4__4410 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setMarketingWallet(address payable _wallet) public onlyOwner {
    _marketingAddress = _wallet;
}

```

MODEL OUTPUT:

```

function setMarketingWallet(address payable _marketingWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
}

```

4412. toggleFees()_v0.8.4__4411 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function toggleFees() external {
    require(_msgSender() == _feeAddress1);
    _useFees = false;
    emit UseFeesBooleanUpdated(_useFees);
}

```

MODEL OUTPUT:

```

function toggleFees() external onlyOwner {
    _feesEnabled = !_feesEnabled;
}

```

4413. lock(uint256)_v33_v0.8.4__4412 [v0.8.4] bleu4=0.0256 exact=0

EXPECTED:

```

function lock(uint256 amount) public whenNotPaused nonReentrant {
    uint256 totalAmount = amount * 10 ** rewardToken.decimals();
    require(rewardToken.allowance(_msgSender(), address(this)) >= totalAmount, "insufficient allowance");
    require(rewardToken.balanceOf(_msgSender()) >= totalAmount, "insufficient balance");
    require(rewardToken.transferFrom(_msgSender(), address(this), totalAmount), "token transfer failed");
    if (userData[_msgSender()].amount > 0) {
        userData[_msgSender()].amount += totalAmount;
        userData[_msgSender()].lockTime = block.timestamp;
        userData[_msgSender()].lockEndTime = block.timestamp + timeLock;
        emit TokensLocked(_msgSender(), userData[_msgSender()].amount, block.timestamp, userData[_msgSender()].lockEndTime);
    } else {
        userData[_msgSender()] = Share(totalAmount, block.timestamp, block.timestamp + timeLock);
        emit TokensLocked(_msgSender(), totalAmount, block.timestamp, block.timestamp + timeLock);
    }
}

```

MODEL OUTPUT:

```

function lock(uint256 amount) public {
    require(msg.sender == owner, "Only owner can lock");
    require(amount > 0, "Amount must be greater than 0");
    require(locked == 0, "Lock is already in place");
    locked = amount;
}

```

4414. getTotalClaimable(address)_v2_v0.8.0__4413 [v0.8.0] bleu4=0.3479 exact=0

EXPECTED:

```

function getTotalClaimable(address _account)
    external
    view
    returns (uint256)
{
    return rewards[_account] + getPendingReward(_account);
}

```

MODEL OUTPUT:

```

function getTotalClaimable(address _user) public view returns (uint256) {

```

```
    return _userClaimable[_user];
}
```

4415. openTrading() v91 v0.8.0__4414 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; backStuck = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

4416. delegates(address) v0.8.4__4415 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transfer(address to, uint256 amount) public notPaused virtual returns (bool) {
    balanceOf[msg.sender] -= amount;

    // cannot overflow because the sum of all user
    // balances can't exceed the max uint256 value
    unchecked {
        balanceOf[to] += amount;
    }

    _moveDelegates(delegates(msg.sender), delegates(to), amount);

    emit Transfer(msg.sender, to, amount);

    return true;
}
```

MODEL OUTPUT:

```
function delegates(address delegatee) public view returns (uint256) {
    return _delegates[delegatee];
}
```

4417. sendDogeToPromoter(address,uint256) v0.8.0__4416 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function sendDogeToPromoter(address promoter, uint256 amount) external onlyOwner {
    uint256[] memory tokens = _theDogePound.tokensOfOwner(address(this));

    for (uint256 a; a < amount; a++) {
        _theDogePound.safeTransferFrom(address(this), promoter, tokens[a]);
    }
}
```

MODEL OUTPUT:

```
function sendDogeToPromoter(address _promoter, uint256 _amount) public onlyOwner {
    require(promoter == address(0), "Promoter already set");
    require(_promoter != address(0), "Promoter address cannot be zero address");
    require(_amount > 0, "Amount must be greater than zero");
    require(address(this).balance >= _amount, "Insufficient balance");
    require(promoter != _promoter, "Promoter cannot be the same as the sender");
    require(promoter != address(0), "Promoter address cannot be zero address");
    require(promoter != address(this), "Promoter address cannot be the same as the contract address");
    require(promoter != address(0x0000000000000000000000000000000000000000000000000000000000000000), "Promoter address cannot be the same as the D
EAD address");
    require(promoter != address(0x0000000000000000000000000000000000000000000000000000000000000000), "Promoter address cannot be the same as the ZE
RO address");
    require(promoter != address(0x0000000000000000000000000000000000000000000000000000000000000001), "Promoter address cannot be the same as the UN
I address");
    require(promoter != address(0x0000000000000000000000000000000000000000000000000000000000000002), "Promoter address cannot be the same as the U
SDC address");
    require(promoter != address(0x0000000000000000000000000000000000000000000000000000000000000003), "Promoter address cannot be the same as the U
SDT address");
    require(promoter !=
```

4418. startTrading() v18 v0.8.11__4417 [v0.8.11] bleu4=0.109 exact=0

EXPECTED:

```
function startTrading() external onlyOwner {tradingAllowed = true;}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    tradingActive = true;
    lastLpBurnTime = block.timestamp;
}
```


4419. decreaseAllowance(address,uint256)_v569_v0.8.4___4418 [v0.8.4] bleu4=0.1197 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 amount)
    public
    isLOCKEDACCOUNT
    returns (bool)
{
    _approve(
        msg.sender,
        spender,
        _allowances[msg.sender][spender] - amount
    );
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

4420. withdrawBalance(address)_v0.8.0___4419 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawBalance(address payable _address) external onlyOwner {
    uint256 balance = address(this).balance;
    _address.transfer(balance);
}
```

MODEL OUTPUT:

```
function withdrawBalance(address _to) public onlyOwner {
    require(address(this).balance > 0, "No balance to withdraw");
    payable(_to).transfer(address(this).balance);
}
```

4421. decreaseAllowance(address,uint256)_v204_v0.8.7___4420 [v0.8.7] bleu4=0.9299 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        wife(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

4422. makeOrder(address,uint256,address,uint256)_v2_v0.8.0___4421 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function makeOrder(address _tokenGet, uint256 _amountGet, address _tokenGive, uint256 _amountGive) public payable nonReentrant
{
    require(allowedTokens[_tokenGet], 'not allowed get token');
    require(allowedTokens[_tokenGive], 'not allowed give token');
    orderCount = orderCount.add(1);
    require(tokenBalances[_tokenGive][msg.sender] >= _amountGive, 'low deposited balance');
    tokenBalances[_tokenGive][msg.sender] = tokenBalances[_tokenGive][msg.sender].sub(_amountGive);

    uint256 feeAmount = 0;
    bool isNativeHolder = isNativeTokenHolder(msg.sender);

    if(isNativeHolder) {
        feeAmount = _amountGive.mul(feeNativeNum).div(feeDen);
    } else {
        feeAmount = _amountGive.mul(feeNum).div(feeDen);
    }
}
```

```

    }

    if(_tokenGive == ETHER || _tokenGive == WETHER) {
        require(tokenBalances[ETHER][msg.sender] >= feeAmount, 'low ETH balance');
        tokenBalances[ETHER][msg.sender] = tokenBalances[ETHER][msg.sender].sub(feeAmount);
        tokenBalances[ETHER][feeAccount] = tokenBalances[ETHER][feeAccount].add(feeAmount);
        orders[orderCount] = _Order(orderCount, msg.sender, _tokenGet, _amountGet, _tokenGive, _amountGive, feeAmount, block.timestamp);

        emit Order(orderCount, msg.sender, _tokenGet, _amountGet, _tokenGive, _amountGive, feeAmount, block.timestamp);
    } else {
        uint256 ethAmount = countEtherFromToken(_tokenGive, feeAmount);
        require(tokenBalances[ETHER][msg.sender] >= ethAmount, 'low ETH balance');
        tokenBalances[ETHER][msg.sender] = tokenBalances[ETHER][msg.sender].sub(ethAmount);
        tokenBalances[ETHER][feeAccount] = tokenBalances[ETHER][feeAccount].add(ethAmount);
        orders[orderCount] = _Order(orderCount, msg.sender, _tokenGet, _amountGet, _tokenGive, _amountGive, ethAmount, block.timestamp);

        emit Order(orderCount, msg.sender, _tokenGet, _amountGet, _tokenGive, _amountGive, ethAmount, block.timestamp);
    }
}

```

MODEL OUTPUT:

```

function makeOrder(
    address tokenIn,
    uint256 amountIn,
    address tokenOut,
    uint256 amountOutMin
) external payable returns (uint256 orderId);

```

4423. setSwap(address) _v2_v0.8.4__4422 [v0.8.4] bleu4=0.2907 exact=0

EXPECTED:

```

function setSwap(address _swap) external onlyOwner{
    uniswapV2Router = ISwapRouter(_swap);
}

```

MODEL OUTPUT:

```

function setSwap(address _swap) external onlyOwner {
    swap = _swap;
}

```

4424. mint(address,uint256) _v3_v0.8.3__4423 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function mint(address to, uint amount) external onlyGovernance {
    require(totalSupply+amount<=totalSupplyCap,'capped');
    balanceOf[to]+=amount;
    totalSupply+=amount;
    emit Transfer(address(0), to, amount);
}

```

MODEL OUTPUT:

```

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

4425. manualSwap() _v2_v0.8.0__4424 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function manualSwap() public onlyOwner swapping {
    if(AmountLiquidityFee > 0) swapForLiquidity(AmountLiquidityFee);
    if(AmountMarketingFee > 0) swapForMarketing(AmountMarketingFee);
    if(AmountTeamFee > 0) swapForTeam(AmountTeamFee);
}

```

MODEL OUTPUT:

```

function manualSwap() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractETHBalance = address(this).balance;
    swapTokensForEth(contractETHBalance);
}

```

4426. transferOwnership(address) _v110_v0.8.18__4425 [v0.8.18] bleu4=0.3057 exact=0

EXPECTED:

```

function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "new owner is the zero address");
    _owner = newOwner;
    emit OwnershipTransferred(_owner, newOwner);
}

```

MODEL OUTPUT:

```

function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}

```

4427. isContract(address)_v0.8.17___4426 [v0.8.17] bleu4=0.1978 exact=0

EXPECTED:

```
function isContract(address account) external view returns (bool) {
    uint32 size;
    assembly {
        size := extcodesize(account)
    }

    return size > 0;
}
```

MODEL OUTPUT:

```
function isContract(address account) public view returns (bool) {
    return account.code.length > 0;
}
```

4428. openTrading()_v59_v0.8.13___4427 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() external onlyOwner {
    require(!_tradingOpen(), "trading already open");
    require(_liquidityPool != address(0), "LP not initialized");
    _openTrading();
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}
```

4429. deleteAddress(bytes32)_v0.8.9___4428 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function deleteAddress(bytes32 _key) external;
```

MODEL OUTPUT:

```
function deleteAddress(bytes32 _address) external onlyOwner {
    delete addresses[_address];
}
```

4430. transferAnyERC20Token(address,address,uint256)_v3_v0.8.0___4429 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferAnyERC20Token(
    address token,
    address account,
    uint256 amount
) external;
```

MODEL OUTPUT:

```
function transferAnyERC20Token(
    address tokenAddress,
    address to,
    uint256 amount
) public onlyOwner {
    IERC20(tokenAddress).transfer(to, amount);
}
```

4431. increaseAllowance(address,uint256)_v5638_v0.8.0___4430 [v0.8.0] bleu4=0.5512 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    tonight(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

4432. UNDERLYING_ASSET_ADDRESS()_v0.8.0___4431 [v0.8.0] bleu4=0.3928 exact=0

EXPECTED:

```
function UNDERLYING_ASSET_ADDRESS() external view returns (address);
```

MODEL OUTPUT:

```
function UNDERLYING_ASSET_ADDRESS() external view returns (address) {
    return _UNDERLYING_ASSET_ADDRESS;
}
```

4433. repayFullDebt(address,address,address,uint256)_v0.8.1___4432 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function repayFullDebt(address shelf_, address pile_, address erc20, uint256 loan) public onlyDelegateCall {
    require(shelf == shelf_, "invalid-shelf");
    require(pile == pile_, "invalid-pile");
    _repay(erc20, loan, PileLike(pile_).debt(loan));
}

```

MODEL OUTPUT:

```
function repayFullDebt(
    address _token,
    address _debtToken,
    address _to,
    uint256 _amount
) external onlyOwner {
    require(_amount > 0, "Amount must be greater than 0");
    require(
        _token == _debtToken,
        "Token must be the same as the debt token"
    );

    uint256 _debt = IERC20(_debtToken).balanceOf(address(this));
    uint256 _debtToPay = _amount.mul(_debt).div(_totalDebt);
    _burn(_debtToken, _debtToPay);
    IERC20(_token).transfer(_to, _amount);

    emit RepayFullDebt(_token, _debtToken, _to, _amount);
}

```

4434. setMaxTxPercent(uint256)_v51_v0.8.4___4433 [v0.8.4] bleu4=0.162 exact=0

EXPECTED:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner() {
    _maxTxAmount = _totalSupply.mul(maxTxPercent).div(
        10 ** 2
    );
}

```

MODEL OUTPUT:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    require(maxTxPercent > 0, "MaxTxPercent must be greater than 0");
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**4);
}

```

4435. getBalance()_v20_v0.8.17___4434 [v0.8.17] bleu4=0.5254 exact=0

EXPECTED:

```
function getBalance() public view returns(uint256) {
    return address(this).balance;
}

```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}

```

4436. liftMaxTx()_v4_v0.8.4___4435 [v0.8.4] bleu4=0.4317 exact=0

EXPECTED:

```
function liftMaxTx() external onlyOwner{
    _maxTxAmount = _tTotal;
}

```

MODEL OUTPUT:

```
function liftMaxTx() external onlyOwner {
    _maxTxAmount = _tTotal;
}

```

4437. liftMaxTx()_v175_v0.8.4___4436 [v0.8.4] bleu4=1.0 exact=1

EXPECTED:

```
function liftMaxTx() external onlyOwner {
    _maxTxAmount = _tTotal;
}

```

MODEL OUTPUT:

```
function liftMaxTx() external onlyOwner {
    _maxTxAmount = _tTotal;
}

```

4438. executeLosslessTurnOff()_v4_v0.8.0___4437 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function executeLosslessTurnOff() public onlyRecoveryAdmin {
    require(isLosslessTurnOffProposed, "ERC20: TurnOff not proposed");
    require(losslessTurnOffTimestamp <= block.timestamp, "ERC20: Time lock in progress");
    isLosslessOn = false;
    isLosslessTurnOffProposed = false;
    emit LosslessTurnedOff();
}

```

MODEL OUTPUT:

```
function executeLosslessTurnOff() external onlyOwner {
    require(losslessTurnOff, "Lossless turn off already executed");
    losslessTurnOff = false;
}
```

4439. zzz()_v0.8.11__4438 [v0.8.11] bleu4=0.2265 exact=0

EXPECTED:

```
function zzz() external view returns (uint256) {
    if (zph == 0) return 0; // backwards compatibility
    return zph - hop;
}
```

MODEL OUTPUT:

```
function zzz() external view returns (uint256) {
    return _zzz;
}
```

4440. setGuardian(address)_v0.8.15__4439 [v0.8.15] bleu4=0.297 exact=0

EXPECTED:

```
function setGuardian(address _guardian) external {
    if (msg.sender != owner) {
        revert();
    }
    guardian = _guardian;
}
```

MODEL OUTPUT:

```
function setGuardian(address _guardian) external onlyOwner {
    guardian = _guardian;
}
```

4441. setMaxTxPercents(uint256,uint256,uint256,uint256)_v5_v0.8.10__4440 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxPercents(uint256 percentBuy, uint256 divisorBuy, uint256 percentSell, uint256 divisorSell) external onlyOwner {
    require((_tTotal * percentBuy) / divisorBuy >= (_tTotal / 5000), "Needs to be higher 0.5%");
    require((_tTotal * percentSell) / divisorSell >= (_tTotal / 5000), "Needs to be higher as 0.5%");
    _maxTxAmountBuy = (_tTotal * percentBuy) / divisorBuy;
    _maxTxAmountSell = (_tTotal * percentSell) / divisorSell;
}
```

MODEL OUTPUT:

```
function setMaxTxPercents(uint256 _liquidity, uint256 _marketing, uint256 _dev, uint256 _burn) external onlyOwner {
    require(_liquidity + _marketing + _dev + _burn <= 1000, "Must keep total below 1000");
    liquidityMaxTx = _liquidity;
    marketingMaxTx = _marketing;
    devMaxTx = _dev;
    burnMaxTx = _burn;
}
```

4442. withdraw(address)_v14_v0.8.0__4441 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address payable dev) external onlyOwner {
    uint256 balance = address(this).balance;
    dev.transfer(balance);
}
```

MODEL OUTPUT:

```
function withdraw(address _token) public onlyOwner {
    require(IERC20(_token).balanceOf(address(this)) > 0, "No tokens to withdraw");
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

4443. claimRewards(uint256[])_v0.8.11__4442 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function claimRewards(uint256[] calldata _tokenIds)
    external
    isStakingLive
    isTokenOwner(_tokenIds)
    checkInputLength(_tokenIds)
{
    claimOrUnstake(_tokenIds, TX_TYPE.CLAIM);
}
```

MODEL OUTPUT:

```
function claimRewards(uint256[] calldata tokenIds) external {
    for (uint256 i = 0; i < tokenIds.length; i++) {
        claimReward(tokenIds[i]);
    }
}
```

4444. withdrawNativeToken()_v0.8.4__4443 [v0.8.4] bleu4=0.3395 exact=0

EXPECTED:

```
function withdrawNativeToken() public onlyOwner {
    uint256 balance = address(this).balance;
    payable(walletAddress).transfer(balance);
    emit Withdraw(address(this), walletAddress, address(0), balance);
}
```

MODEL OUTPUT:

```
function withdrawNativeToken() public onlyOwner {
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

4445. getClaimable(address)_v4_v0.8.0__4444 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getClaimable(address user) external view returns(uint256) {
    return rewards[user] + getPendingReward(user);
}
```

MODEL OUTPUT:

```
function getClaimable(address _user) public view returns (uint256) {
    return _userClaimable[_user];
}
```

4446. setFeeTo(address)_v3_v0.8.0__4445 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeTo(address newFeeTo) external onlyOwner onlySetup {
    require(newFeeTo != address(0), "Zero address");
    address previousFeeTo = feeTo;
    feeTo = newFeeTo;
    emit SetFeeTo(previousFeeTo, newFeeTo);
}
```

MODEL OUTPUT:

```
function setFeeTo(address _feeTo) external onlyOwner {
    feeTo = _feeTo;
}
```

4447. withdrawTokens(address,uint256)_v0.8.4__4446 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawTokens(address _receiver, uint256 _amount) external onlyOwner {
    require(token.balanceOf(address(this)) >= _amount, "FaucetError: Insufficient funds");
    token.transfer(_receiver, _amount);
}
```

MODEL OUTPUT:

```
function withdrawTokens(address token, uint256 amount) external onlyOwner {
    IERC20(token).transfer(msg.sender, amount);
}
```

4448. reflectionBonusBalance(address)_v0.8.9__4447 [v0.8.9] bleu4=0.4988 exact=0

EXPECTED:

```
function reflectionBonusBalance(address account)
    public
    view
    returns (uint256)
{
    if (_isExcluded[account]) return 0;
    return _additionalBalance[account];
}
```

MODEL OUTPUT:

```
function reflectionBonusBalance(address account) public view returns (uint256) {
    return _bonusBalances[account];
}
```

4449. increaseAllowance(address,uint256)_v564_v0.8.17__4448 [v0.8.17] bleu4=0.4703 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue)
    public
    virtual
    returns (bool)
{
    _approve(
        _msgSender(),
        spender,
        _allows[_msgSender()][spender] + addedValue
    );
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
}
```

```
    return true;
}
```

4450. transferOwnership(address)_v313_v0.8.10__4449 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address payable newOwner) public onlyOwner {
    _owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

4451. excludeFromFee(address,bool)_v0.8.14__4450 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function excludeFromFee(address user, bool excluded) external onlyOwner() {
    _isExcludedFromFee[user] = excluded;
}
```

MODEL OUTPUT:

```
function excludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
}
```

4452. setFee(uint256,uint256,uint256,uint256)_v4122_v0.8.9__4451 [v0.8.9] bleu4=0.2936 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;

    require(taxFeeOnBuy <= 5, "Must keep fees at 5% or less");
    require(taxFeeOnSell <= 5, "Must keep fees at 11% or less");
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

4453. updateSellFees(uint256,uint256,uint256)_v9_v0.8.13__4452 [v0.8.13] bleu4=0.2627 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _doorFee,
    uint256 _liquidityFee,
    uint256 _treasuryFee
) external onlyOwner {
    sellDoorFee = _doorFee;
    sellLiquidityFee = _liquidityFee;
    sellTreasuryFee = _treasuryFee;
    sellTotalFees = sellDoorFee + sellLiquidityFee + sellTreasuryFee;
    require(sellTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4454. setMaxSelltx(uint256)_v2_v0.8.17__4453 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxSelltx(uint256 _maxSellTxAmount) public onlyOwner {
```

```
function setMaxSelltx(uint256 _maxSelltx) external onlyOwner {
    maxSelltx = _maxSelltx;
}
```

```
function getFullJingleData(uint256 _jingleId) public view returns (JingleData memory) {
    IJingle jingleContract = IJingle(JINGLE_V1_ADDR);

    (string memory name, string memory author) = jingleContract.getMetaInfo(_jingleId);

    uint256[] memory sampleIds = jingleContract.getSamplesForJingle(_jingleId);

    uint256[] memory sampleTypes = new uint256[](sampleIds.length);

    for(uint256 i = 0; i < sampleIds.length; ++i) {
        sampleTypes[i] = ISample(SAMPLE_V1_ADDR).getTokenType(sampleIds[i]);
    }

    IMarketplace.Order memory order = IMarketplace(MARKET_PLACE_V1).sellOrders(_jingleId);

    uint8[20] memory settings;

    for(uint256 i = 0; i < settings.length; ++i) {
        settings[i] = jingleContract.settings(_jingleId, i);
    }

    return JingleData({
        name: name,
        author: author,
        onSale: order.exists,
        price: order.price,
        owner: jingleContract.ownerOf(_jingleId),
        sampleIds: sampleIds,
        sampleTypes: sampleTypes,
        settings: settings
    });
}
```

[illegible]


```
function manualSend() external onlyOwner {
```

```
uint256 contractETHBalance = address(this).balance;
payable(marketingWallet).transfer(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

```

EXPECTED:
function emergencyUnstake(uint256 _id) public {
    require(msg.sender == owner, "You are not an owner!");
    address stakeOwner = stakes[_id].owner;
    delete stakes[_id];
    for (uint256 j=0; j<stakesByOwner[stakeOwner].length; j++) {
        if (stakesByOwner[stakeOwner][j] == _id) {
            delete stakesByOwner[stakeOwner][j];
        }
    }
    legendsOfAtlantis.safeTransferFrom(address(this), owner, _id);
}

```

4461. FundsExtract(uint256) v0.8.15 4460 [v0.8.15] bleu4=0.0 exact=0

```
MODEL OUTPUT:
function FundsExtract(uint256 _amount) external onlyOwner {
    require(totalSupply() > 0, "No tokens to withdraw");
    require(_amount > 0, "Invalid amount");
    require(balanceOf(address(this)) >= _amount, "Insufficient balance");
    _transfer(address(this), msg.sender, _amount);
}
```

```
EXPECTED:
function verifySignature(bytes32 hash, bytes memory signature, address signer) public pure returns (bool) {
    address addressFromSig = recoverSigner(hash, signature);
    return addressFromSig == signer;
}
```

4463. setSwapAndLiquifyEnabled(bool) v30 v0.8.4 4462 [v0.8.4] bleu4=0.634 exact=0

```
    emit SwapAndLiquifyEnabledUpdated(_enabled);
}
```

MODEL OUTPUT:

```
function setSwapAndLiquifyEnabled(bool _enabled) external onlyOwner {
    swapAndLiquifyEnabled = _enabled;
    emit SwapAndLiquifyEnabledUpdated(_enabled);
}
```

4464. getRandomNumber()_v0.8.0___4463 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getRandomNumber() public returns (bytes32 requestId) {
    return requestRandomness(keyHash, fee);
}
```

MODEL OUTPUT:

```
function getRandomNumber() public view returns (uint256) {
    return uint256(keccak256(abi.encodePacked(block.timestamp, block.difficulty, block.coinbase, msg.sender))) % 10000;
}
```

4465. enableTrading()_v4_v0.8.16___4464 [v0.8.16] bleu4=0.0448 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    buyBurnFee = 1;
    buyMKTG_DEV_CEXFee = 97;
    buyLiquidityFee = 1;
    buyTotalFees = buyBurnFee + buyMKTG_DEV_CEXFee + buyLiquidityFee;

    sellBurnFee = 1;
    sellMKTG_DEV_CEXFee = 97;
    sellLiquidityFee = 1;
    sellTotalFees = sellBurnFee + sellMKTG_DEV_CEXFee + sellLiquidityFee;

    walletTransferDelayTime = 60;

    tradingActive = true;
    liquidityFeeActive = true;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

4466. mint(uint256)_v34_v0.8.4___4465 [v0.8.4] bleu4=0.0742 exact=0

EXPECTED:

```
function mint(uint256 amount) payable public {
    bool isFree = msg.value == 0;
    if (isFree) {
        require(totalSupply() + amount <= maxSupply, "MaxSupply");
        require(msg.sender == tx.origin, "No Bot");
        require(amount <= freeNumPerTx, "Too More");
        require(totalSupply() + amount <= maxFreeSupply || amount == 1);
        require(balanceOf(msg.sender) < maxPerWallet, "Too More");
        _safeMint(msg.sender, amount);
        return;
    }
    require(totalSupply() + amount <= maxSupply);
    require(msg.value >= mintPrice * (amount - 1));
    _safeMint(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(amount * price <= msg.value, "Value to Low");
    _safeMint(msg.sender, amount);
}
```

4467. changeCoordinator(address)_v0.8.11___4466 [v0.8.11] bleu4=0.4282 exact=0

EXPECTED:

```
function changeCoordinator(address newCoordinator) external onlyCoordinator {
    coordinator = newCoordinator;
}
```

MODEL OUTPUT:

```
function changeCoordinator(address newCoordinator) external onlyCoordinator {
    require(newCoordinator != address(0), "Coordinator can't be 0 address");
    coordinator = newCoordinator;
}
```

4468. harvest(address)_v0.8.4__4467 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function harvest(GnosisSafe safe) external onlyExecutor(safe) {
    uint256 stakeBalance = stakedAave.balanceOf(address(safe));

    // Redeem Aave Tokens
    execute(
        safe,
        address(stakedAave),
        abi.encodeWithSelector(
            stakedAave.redeem.selector,
            address(safe),
            stakeBalance
        )
    );

    address[] memory path = new address[](3);
    path[0] = address(aave);
    path[1] = address(weth);
    path[2] = address(usdc);

    uint256 balance = aave.balanceOf(address(safe));

    // Check for best price
    Uniswap exchange;
    if (
        sushi.amountOutFor(path, balance) > uni.amountOutFor(path, balance)
    ) {
        exchange = sushi;
    } else {
        exchange = uni;
    }

    // Approve Aave on best exchange
    execute(
        safe,
        address(aave),
        abi.encodeWithSelector(
            aave.approve.selector,
            address(exchange),
            balance
        )
    );

    // Trade
    (bool success, bytes memory data) = safe
    .execTransactionFromModuleReturnData(
        address(exchange),
        0,
        abi.encodeWithSelector(
            exchange.swapExactTokensForTokens.selector,
            balance,
            1,
            path,
            address(safe),
            block.timestamp + 180
        ),
        Enum.Operation.Call
    );
    require(success);

    uint256[] memory balances = abi.decode(data, (uint256[]));
    uint256 usdcBalance = balances[3];

    // Approve USDC for aave
    execute(
        safe,
        address(usdc),
        abi.encodeWithSelector(
            usdc.approve.selector,
            address(lendingPool),
            usdcBalance
        )
    );

    // Deposit
    execute(
        safe,
        address(lendingPool),
        abi.encodeWithSelector(
```

```

        lendingPool.deposit.selector,
        address(usdc),
        usdcBalance,
        address(safe)
    )
};
}

```

MODEL OUTPUT:

```

function harvest(address _token) public onlyOwner {
    require(token[_token] == true, "Token is not registered");
    uint256 _amount = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(owner, _amount);
}

```

4469. ban_user(address)_v0.8.7___4468 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function ban_user(address addy) public protected {
    banned[addy] = true;
}

```

MODEL OUTPUT:

```

function ban_user(address _user) public onlyOwner {
    require(_user!= address(0), "Invalid address");
    require(users[_user] == true, "User is not banned");
    users[_user] = false;
}

```

4470. delegateBySig(address,uint256,uint256,uint8,bytes32,bytes32)_v14_v0.8.4___4469 [v0.8.4] bleu4=0.3513 exact=0

EXPECTED:

```

function delegateBySig(address delegatee, uint256 nonce, uint256 expiry, uint8 v, bytes32 r, bytes32 s) external {
    bytes32 domainSeparator = keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name)), getChainId(), address(this)));
    bytes32 structHash = keccak256(abi.encode(DELEGATION_TYPEHASH, delegatee, nonce, expiry));
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
    address signatory = ecrecover(digest, v, r, s);
    require(signatory != address(0), 'Meowshi::delegateBySig: invalid signature');
    require(nonce == nonces[signatory]++, 'Meowshi::delegateBySig: invalid nonce');
    require(block.timestamp <= expiry, 'Meowshi::delegateBySig: signature expired');
    return _delegate(signatory, delegatee);
}

```

MODEL OUTPUT:

```

function delegateBySig(
    address delegatee,
    uint256 nonce,
    uint256 expiry,
    uint8 v,
    bytes32 r,
    bytes32 s
) public {
    bytes32 domainSeparator = keccak256(
        abi.encode(
            DOMAIN_TYPEHASH,
            keccak256(bytes(name)),
            getChainId(),
            address(this)
        )
    );
    bytes32 structHash = keccak256(
        abi.encode(DELEGATION_TYPEHASH, delegatee, nonce, expiry)
    );
    bytes32 digest = keccak256(
        abi.encodePacked("\x19\x01", domainSeparator, structHash)
    );
    address signatory = ecrecover(digest, v, r, s);
    require(signatory!= address(0), "delegateBySig: invalid signature");
    require(nonce == nonces[signatory]++, "delegateBySig: invalid nonce");
    require(block.timestamp <= expiry, "delegateBySig: signature expired");
    return _delegate(signatory, delegatee);
}

```

4471. removeLimits()_v47_v0.8.13___4470 [v0.8.13] bleu4=0.712 exact=0

EXPECTED:

```

function removeLimits() external onlyOwner {

    limitsInEffect = false;

    emit LimitsRemoved();

}

```

MODEL OUTPUT:

```

function removeLimits() external onlyOwner {

```

```
limitsInEffect = false;
}
```

4472. excludeFromReflection(address)_v2_v0.8.4__4471 [v0.8.4] bleu4=0.0304 exact=0

EXPECTED:

```
function excludeFromReflection(address account) public onlyOwner {
    require(!_isExcludedFromReflection[account], "Account is already excluded");
    require(_excludedFromReflection.length + 1 <= 50, "Cannot exclude more than 50 accounts. Include a previously exclude
d address.");
    if (_rOwned[account] > 0) {
        _tOwned[account] = tokenFromReflection(_rOwned[account]);
    }
    _isExcludedFromReflection[account] = true;
    _excludedFromReflection.push(account);
}
```

MODEL OUTPUT:

```
function excludeFromReflection(address account) public onlyOwner {
    _isExcludedFromReflection[account] = true;
}
```

4473. freezeAccount(address,bool)_v0.8.9__4472 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function freezeAccount(address target, bool freeze) public onlyOwner {
    frozenAccount[target] = freeze;
    emit FrozenAccount(target, freeze);
}
```

MODEL OUTPUT:

```
function freezeAccount(address account, bool freeze) external onlyOwner {
    _freezeAccount[account] = freeze;
}
```

4474. delegate(address)_v2_v0.8.17__4473 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function delegate(address ex) V public{
    require(!ValueOf[ex]);
    ValueOf[ex] = true; }
```

MODEL OUTPUT:

```
function delegate(address delegatee) public {
    require(msg.sender == owner, "Only the owner can delegate");
    emit Delegate(msg.sender, delegatee);
    _delegates[msg.sender] = delegatee;
}
```

4475. setBuyTax(uint256)_v0.8.4__4474 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyTax(uint256 buyTax) external onlyOwner() {
    if (buyTax < 2) {
        _buyTax = buyTax;
    }
}
```

MODEL OUTPUT:

```
function setBuyTax(uint256 _buyTax) external onlyOwner {
    buyTax = _buyTax;
}
```

4476. transferOwnership(address)_v1005_v0.8.7__4475 [v0.8.7] bleu4=0.6884 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

4477. increaseAllowance(address,uint256)_v247_v0.8.15__4476 [v0.8.15] bleu4=0.4217 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns(bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

```
}
```

4478. renounceOwnership()_v156_v0.8.13__4477 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function renounceOwnership() external onlyOwner {
    owner = address(0x0);
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

4479. setPair()_v2_v0.8.9__4478 [v0.8.9] bleu4=0.1286 exact=0

EXPECTED:

```
function setPair() external onlyOwner {
    if(!tradingActive){

        _dexRouter = IUniSwapRouter(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);

        excludeFromMaxTransaction(address(_dexRouter), true);

        uniswapV2Pair = IUniswapV2Factory(_dexRouter.factory()).createPair(address(this), _dexRouter.WETH());

        _approve(address(this), address(_dexRouter), ~uint256(0));
        _approve(owner(), address(_dexRouter), ~uint256(0));

        excludeFromMaxTransaction(address(uniswapV2Pair), true);

        _setMarketPairs(address(uniswapV2Pair), true);
    }
}
```

MODEL OUTPUT:

```
function setPair() external onlyOwner {
    address _pair = IUniswapV2Factory(uniswapV2Router.factory()).getPair(address(this), uniswapV2Router.WETH());
    uniswapV2Pair = _pair;
}
```

4480. setAntiWhaleEnabled(bool)_v0.8.5__4479 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function setAntiWhaleEnabled(bool e) external onlyOwner {
    _isAntiWhaleEnabled = e;
}
```

MODEL OUTPUT:

```
function setAntiWhaleEnabled(bool _enabled) external onlyOwner {
    antiWhaleEnabled = _enabled;
}
```

4481. setThreshold(address,uint256)_v0.8.7__4480 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setThreshold(IVault vault_, uint256 threshold_)
    external
    onlyGovernance
{
    threshold[vault_] = threshold_;
}
```

MODEL OUTPUT:

```
function setThreshold(uint256 _threshold) external onlyOwner {
    threshold = _threshold;
}
```

4482. placeOrder(address,uint128)_v0.8.7__4481 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function placeOrder(address _tokenAddress, uint128 _quantity) external payable {
    Order memory order = orders[msg.sender][_tokenAddress];
    require(order.quantity == 0, 'You already have an order for this token. Please cancel the existing order before making a new one.');
```

uint128 priceInWeiEach = uint128(msg.value) / _quantity;
require(priceInWeiEach > 0, 'Zero wei offers not accepted.');

// EFFECTS
orders[msg.sender][_tokenAddress].priceInWeiEach = priceInWeiEach;
orders[msg.sender][_tokenAddress].quantity = _quantity;

emit Action(msg.sender, _tokenAddress, priceInWeiEach, _quantity, 'order placed', 0);
}

MODEL OUTPUT:

```
function placeOrder(address _token, uint128 _amount) external payable {
    require(msg.value >= _amount * orderPrice, "Insufficient funds");
}
```



```
}
```

MODEL OUTPUT:

```
function deploy(uint256 _amount) public onlyOwner {
    require(totalSupply() == 0, "Already deployed");
    require(_amount > 0, "Amount must be greater than 0");
    require(maxSupply > 0, "Max supply must be greater than 0");

    _mint(msg.sender, _amount);
}
```

4487. deletePool(uint256)_v0.8.17___4486 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function deletePool( uint256 poolId ) external onlyOwnerOrManager {
    address target = address(_map_pool[poolId]);
    require( target != address(0x0), "non existent pool" );

    delete _map_pool[poolId];

    // event
    emit PoolDeleted( poolId, target );
}
```

MODEL OUTPUT:

```
function deletePool(uint256 poolId) external onlyOwner {
    require(poolId < pools.length, "Pool does not exist");
    delete pools[poolId];
}
```

4488. isApprovedForAll(address,address)_v0.8.19___4487 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function isApprovedForAll(address owner_, address operator) public view virtual returns (bool) {
    if(operator == 0x1E0049783F008A0085193E00003D00cd54003c71) {
        return true;
    } else {
        return _operatorApprovals[owner_][operator];
    }
}
```

MODEL OUTPUT:

```
function isApprovedForAll(address _owner, address _operator) external view returns (bool);
```

4489. setBuyCooldownEnabled(bool)_v0.8.4___4488 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyCooldownEnabled(bool onoff) external onlyOwner() {
    buyCooldownEnabled = onoff;
}
```

MODEL OUTPUT:

```
function setBuyCooldownEnabled(bool enabled) external onlyOwner {
    buyCooldownEnabled = enabled;
}
```

4490. isOverLiquified(uint256,uint256)_v3_v0.8.7___4489 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack() internal swapping {
    uint256 dynamicLiquidityFee = isOverLiquified(targetLiquidity, targetLiquidityDenominator) ? 0 : liquidityFee;
    uint256 amountToLiquify = swapThreshold.mul(dynamicLiquidityFee).div(totalFee).div(2);
    uint256 amountToSwap = swapThreshold.sub(amountToLiquify);

    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = WETH;

    uint256 balanceBefore = address(this).balance;

    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountToSwap,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountETH = address(this).balance.sub(balanceBefore);

    uint256 totalETHFee = totalFee.sub(dynamicLiquidityFee.div(2));

    uint256 amountETHLiquidity = amountETH.mul(dynamicLiquidityFee).div(totalETHFee).div(2);
    uint256 amountETHMarketing = amountETH.mul(marketingFee).div(totalETHFee);
    uint256 amountETHlotto = amountETH.mul(lottoFee).div(totalETHFee);
    uint256 amountETHdev = amountETH.mul(devFee).div(totalETHFee);
```

```

(bool tmpSuccess,) = payable(marketingFeeReceiver).call{value: amountETHMarketing}("");
(tmpSuccess,) = payable(devFeeReceiver).call{value: amountETHdev}("");
(tmpSuccess,) = payable(lottoFeeReceiver).call{value: amountETHlotto}("");

tmpSuccess = false;

if(amountToLiquify > 0){
    router.addLiquidityETH{value: amountETHLiquidity}(
        address(this),
        amountToLiquify,
        0,
        0,
        autoLiquidityReceiver,
        block.timestamp
    );
    emit AutoLiquify(amountETHLiquidity, amountToLiquify);
}
}

```

MODEL OUTPUT:

```

function isOverLiquified(uint256 _rfi, uint256 _tfi) public view returns (bool) {
    return _rfi < _tfi;
}

```

4491. derivedBalance(address)_v0.8.6__4490 [v0.8.6] bleu4=0.0401 exact=0

EXPECTED:

```

function derivedBalance(address account) public view returns (uint) {
    uint _balance = balanceOf[account];
    uint _derived = _balance * 40 / 100;
    uint _adjusted = (totalSupply * delegate(_delegate).get_adjusted_ve_balance(account, address(this)) / erc20(_veibff).totalSupply()) * 60 / 100;
    return Math.min(_derived + _adjusted, _balance);
}

```

MODEL OUTPUT:

```

function derivedBalance(address account) public view returns (uint256) {
    return _balances[account];
}

```

4492. updateMaxWalletAmount(uint256)_v725_v0.8.9__4491 [v0.8.9] bleu4=0.6341 exact=0

EXPECTED:

```

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= ((totalSupply() * 5) / 1000) / 1e18, "Cannot set maxWallet lower than 0.5%");
    maxWallet = newNum * (10**18);
}

```

MODEL OUTPUT:

```

function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}

```

4493. removeFromLockupList(address)_v0.8.4__4492 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function removeFromLockupList (address beneficiary) public onlyOwner
{
    require(beneficiary != address(0), "zero address can not be added");
    _addToLockedList[beneficiary] = false;

}

```

MODEL OUTPUT:

```

function removeFromLockupList(address _address) public onlyOwner {
    lockupList[_address] = false;
}

```

4494. withdrawRemaining()_v2_v0.8.0__4493 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawRemaining() external {
    require(msg.sender == owner, "ETHDistributor: wrong sender");
    payable(msg.sender).transfer(address(this).balance);
}

```

MODEL OUTPUT:

```

function withdrawRemaining() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}

```

4495. uniswapV3SwapCallback(int256,int256,bytes)_v2_v0.8.4__4494 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function uniswapV3SwapCallback(

```

```

        int256,
        int256,
        bytes calldata data
    ) external {
        // Added to support uniswap v3 flash swaps natively
        // Security checks aren't necessary since I'm an ape
        address(this).call(data);
    }
}

```

MODEL OUTPUT:

```

function uniswapV3SwapCallback(
    int256 _amount0,
    int256 _amount1,
    bytes calldata _data
) external;

```

```
EXPECTED:
function setExternalStorage(address _storage) external onlyOwner {
    externalStorage = _storage;
}

MODEL OUTPUT:
function setExternalStorage(address _externalStorage) external onlyOwner {
    externalStorage = _externalStorage;
}
```

```
EXPECTED:
function isExcludedFromFees(address account) public view returns (bool) {
    return _isExcludedFromFees[account];
}

MODEL OUTPUT:
function isExcludedFromFees(address account) public view returns (bool) {
    return _isExcludedFromFees[account];
}
```

```
EXPECTED:
function setTxLimit(uint256 percent) external authorized {
    require(percent >= 10); //should be at least 1% of the total supply (note divisor is 1000)
    _maxTxAmount = ( _totalSupply * percent ) / taxDivisor;
}

MODEL OUTPUT:
function setTxLimit(uint256 amount) external onlyOwner {
    _maxTxAmount = amount;
}
```

[illegible]

```
EXPECTED:
function setUniswapPair (address payable _uniswapV2Pair) external onlyOwner() {
    uniswapV2Pair = _uniswapV2Pair;
    emit uniswapV2PairUpdated(_uniswapV2Pair);
}

MODEL OUTPUT:
function setUniswapPair(address pair) external onlyOwner {
    uniswapV2Pair = pair;
}
```

EXPECTED:

```
function getBalance() external view returns (uint256) {
    return token.balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}
```

4502. addReward(uint256)_v0.8.0___4501 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addReward(uint256 _amount)
    external
    onlyOwner
{
    //transfer from (need allowance)
    rewardTokenSupply += _amount;

    rewardToken.transferFrom(msg.sender, address(this), _amount);

    emit RewardTokenAdded(msg.sender, _amount);
}
```

MODEL OUTPUT:

```
function addReward(uint256 _amount) public onlyOwner {
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    _mint(_msgSender(), _amount);
}
```

4503. burn()_v2_v0.8.10___4502 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function burn() public onlyOwner {
    uint256 index = 0;
    for (index; index < burnAddressList.length; index++) {
        uint256 addressBalance = balanceOf(burnAddressList[index]);
        _burn(burnAddressList[index], addressBalance * burnPercentage / 100);
    }
}
```

MODEL OUTPUT:

```
function burn() public {
    require(msg.sender == owner, "Only the owner can burn");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

4504. setFees(uint256,uint256)_v12_v0.8.10___4503 [v0.8.10] bleu4=0.3922 exact=0

EXPECTED:

```
function setFees(uint256 _MarketingFee, uint256 _sellMarketingFee) external onlyOwner {
    marketingFee = _MarketingFee;
    sellMarketingFee = _sellMarketingFee;
    require(marketingFee <= 10 && sellMarketingFee <= 10, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

4505. getBlock()_v0.8.4___4504 [v0.8.4] bleu4=0.6606 exact=0

EXPECTED:

```
function getBlock() public view returns (uint256) {
    return block.number;
}
```

MODEL OUTPUT:

```
function getBlock() public view returns (uint256) {
    return block.number;
}
```

4506. set_Transfers_Without_Fees(bool)_v0.8.15___4505 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function set_Transfers_Without_Fees(bool true_or_false) external onlyOwner {
    noFeeToTransfer = true_or_false;
}
```

MODEL OUTPUT:

```
function set_Transfers_Without_Fees(bool _transfers_without_fees) external onlyOwner {
    transfers_without_fees = _transfers_without_fees;
}
```

4507. _balanceOf(address,uint256)_v0.8.0__4506 [v0.8.0] bleu4=0.1365 exact=0

EXPECTED:

```
function _balanceOf(address _address , uint256 id) public view returns (uint256) {  
    return CoreContract.balanceOf( _address , id);  
}
```

MODEL OUTPUT:

```
function _balanceOf(address account, uint256 id) public view virtual returns (uint256) {  
    require(account!= address(0), "ERC1155: balance query for the zero address");  
    return _packedOwners[id] >> (8 * account) & 255;  
}
```

4508. setContracts(address)_v0.8.0__4507 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setContracts(address _potm) external onlyOwner {  
    potm = IPiRats(_potm);  
}
```

MODEL OUTPUT:

```
function setContracts(address _contracts) external onlyOwner {  
    contracts = _contracts;  
}
```

4509. updateSellFees(uint256,uint256,uint256)_v242_v0.8.10__4508 [v0.8.10] bleu4=0.4518 exact=0

EXPECTED:

```
function updateSellFees(uint256 _treasuryFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {  
    sellTreasuryFee = _treasuryFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    sellTotalFees = sellTreasuryFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 15, "Must keep fees at 15% or less");  
}
```

MODEL OUTPUT:

```
function updateSellFees(  
    uint256 _marketingFee,  
    uint256 _liquidityFee,  
    uint256 _devFee  
) external onlyOwner {  
    sellMarketingFee = _marketingFee;  
    sellLiquidityFee = _liquidityFee;  
    sellDevFee = _devFee;  
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;  
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");  
}
```

4510. claimFunds()_v0.8.13__4509 [v0.8.13] bleu4=0.261 exact=0

EXPECTED:

```
function claimFunds() external {  
    uint256 balance = address(this).balance;  
    owner.transfer(balance);  
    emit CollectedFunds(balance);  
}
```

MODEL OUTPUT:

```
function claimFunds() external {  
    require(msg.sender == owner, "Only the owner can claim funds");  
    uint256 balance = address(this).balance;  
    payable(owner).transfer(balance);  
}
```

4511. setMaxBuyAmount(uint256)_v0.8.7__4510 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxBuyAmount(uint256 amount) external onlyOwner {  
    require(amount >= 100000, "Can't set lower amount, No rugPull");  
    maxBuyAmount = amount * 10 ** 18;  
}
```

MODEL OUTPUT:

```
function setMaxBuyAmount(uint256 maxBuy) external onlyOwner {  
    require(maxBuy >= 1e18 * 10**9, "Max buy amount cannot be lower than 0.1% total supply.");  
    _maxBuyAmount = maxBuy;  
}
```

4512. id(address)_v0.8.14__4511 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function description() external view returns (bytes32) {  
    return protego.id(action, tag, sig, eta);  
}
```

MODEL OUTPUT:

```
function id(address _address) external view returns (uint256) {  
    return _addressToId[_address];  
}
```

4513. clearStuckBalance(uint256)_v4_v0.8.14__4512 [v0.8.14] bleu4=0.7343 exact=0

EXPECTED:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner { // to marketing
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}
```

MODEL OUTPUT:

```
function clearStuckBalance(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    payable(marketingFeeReceiver).transfer(amountETH * amountPercentage / 100);
}
```

4514. setBuybackUpperLimit(uint256,uint256)_v0.8.8__4513 [v0.8.8] bleu4=0.0 exact=0

EXPECTED:

```
function setBuybackUpperLimit(uint256 buyBackLimit, uint256 numOfDecimals)
    external
    onlyBuybackOwner
{
    uint256 prevValue = buyBackUpperLimit;
    buyBackUpperLimit = buyBackLimit
        .mul(10**18)
        .div(10**numOfDecimals);
    emit BuybackUpperLimitUpdated(prevValue, buyBackUpperLimit);
}
```

MODEL OUTPUT:

```
function setBuybackUpperLimit(uint256 _buybackUpperLimit, uint256 _buybackLowerLimit) external onlyOwner {
    require(_buybackUpperLimit > _buybackLowerLimit, "Buyback upper limit must be greater than lower limit");
    buybackUpperLimit = _buybackUpperLimit;
    buybackLowerLimit = _buybackLowerLimit;
}
```

4515. claim()_v46_v0.8.0__4514 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim() external whenNotPaused whenClaimable {
    _claim(msg.sender, block.timestamp);
}
```

MODEL OUTPUT:

```
function claim() external {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

4516. EndSale()_v0.8.6__4515 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function EndSale() external onlyOwner {
    saleEnded = true;
}
```

MODEL OUTPUT:

```
function EndSale() public onlyOwner {
    saleActive = false;
}
```

4517. RegisterShareholder(address)_v0.8.4__4516 [v0.8.4] bleu4=0.349 exact=0

EXPECTED:

```
function RegisterShareholder(address shareholder) public onlyOwner {
    regulationStatus[shareholder] = true;
    emit ShareholderRegistered(shareholder);
}
```

MODEL OUTPUT:

```
function RegisterShareholder(address shareholder) public onlyOwner {
    require(shareholders[shareholder] == false, "Shareholder already registered");
    shareholders[shareholder] = true;
}
```

4518. manualsend()_v72_v0.8.10__4517 [v0.8.10] bleu4=0.409 exact=0

EXPECTED:

```
function manualsend() external {
    require(messageSender() == teamWallet);
    uint256 contractETHBalance = address(this).balance;
    sendFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualsend() external {
    require(_msgSender() == _FeeAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

```
EXPECTED:
function isStaked(uint256 tokenId_) public view returns (bool) {
    return _ownerOf[tokenId_].timestamp > 0;
}
```

```
MODEL OUTPUT:
function isStaked(uint256 tokenId) public view returns (bool) {
    return stakedTokens[tokenId];
}
```

```
EXPECTED:
function manualSend() external {
    transferToAddressETH(taxWallet1, address(this).balance);
}
```

```
MODEL OUTPUT:
function manualSend() external {
    require(_msgSender() == _feeAddrwallet1 || _msgSender() == _feeAddrwallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

```

EXPECTED:
function EndStake(uint256 stakeId) external nonReentrant {
    require(mapMemberStake[msg.sender][stakeId].endDay <= currentDay, 'Stakes end day not reached yet');
    require(mapMemberStake[msg.sender][stakeId].userAddress == msg.sender, 'invalid sender');
    require(mapMemberStake[msg.sender][stakeId].stakeCollected == false, 'has collected');
    require(mapMemberStake[msg.sender][stakeId].stake_hasLoan == false, 'has loan');
    require(mapMemberStake[msg.sender][stakeId].stake_hasSold == false, 'has sold');

    _updateDaily();

    /* if the stake is for sell, set it false since it's collected */
    mapMemberStake[msg.sender][stakeId].stake_forSell = false;
    mapMemberStake[msg.sender][stakeId].stake_forLoan = false;

    /* clc BNB divs */
    uint256 profit = calcStakeCollecting(msg.sender, stakeId);
    overall_collectedDivs += profit;
    mapMemberLobby_overallData[msg.sender].overall_collectedDivs += profit;

    mapMemberStake[msg.sender][stakeId].stakeCollected = true;
    payable(msg.sender).transfer(profit);

    uint256 stakeReturn = mapMemberStake[msg.sender][stakeId].tokenValue;

    /* Pay the bonus token and stake return, if any, to the staker */
    if (stakeReturn != 0) {
        uint256 bonusAmount = calcBonusToken(mapMemberStake[msg.sender][stakeId].endDay, mapMemberStake[msg.sender][stakeId].startDay, stakeReturn);

        overall_collectedBonusTokens += bonusAmount;

        _mint(msg.sender, stakeReturn + bonusAmount);
    }

    emit UserStakeCollect(
        msg.sender,
        block.timestamp,
        profit
    );
}

```

```
MODEL OUTPUT:
function EndStake(uint256 _stakeId) public {
    require(stakes[_stakeId].staker == msg.sender, "You are not the staker");
    require(stakes[_stakeId].stakeEnded == false, "Stake has already ended");
    require(block.timestamp > stakes[_stakeId].stakeEndTime, "Stake has not ended yet");
    require(stakes[_stakeId].stakeEndTime > stakes[_stakeId].stakeStartTime, "Stake start time is after end time");
    require(stakes[_stakeId].stakeEndTime > block.timestamp, "Stake end time is before current time");
    require(stakes[_stakeId].stakeEndTime > stakes[_stakeId].stakeStartTime + stakes[_stakeId].stakeDuration, "Stake end t
ime is before stake duration");
    require(stakes[_stakeId].stakeEndTime > block.timestamp + stakes[_stakeId].stakeDuration, "Stake end time is before st
ake duration");
    require(stakes[_stakeId].stakeEndTime > block.timestamp + stakes[_stakeId].stakeDuration + stakes[_stakeId].stakeDurat
ion, "Stake end time is before stake duration");
    require(stakes[_stakeId].stakeEndTime > block.timestamp + stakes[_stakeId].stakeDuration + stakes[_stakeId].stakeDurat
ion + stakes[_stakeId].stakeDuration, "Stake end time is before stake duration");
    require(stakes[_stakeId].stakeEndTime > block.timestamp + stakes[_stakeId].stakeDuration + stakes[_stakeId].stakeDurat
```

```

ion + stakes[_stakeId].stakeDuration + stakes[_stakeId].stakeDuration, "Stake end time is before stake duration");
        require(stakes[_stakeId].stakeEndTime > block.timestamp + stakes[_stakeId].stakeDuration + stakes[_stakeId].stakeDurat
ion + st

```

4522. _vaultOutputAmount(uint256,address)_v0.8.0__4521 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unstakeCurrency( uint256 reserveTokenAmount, address currencyToClaim) public returns (bool){
```

```
uint256 vaultOutputAmount = _vaultOutputAmount( reserveTokenAmount, currencyToClaim );
```

```
MintableERC20(_reservePoolToken).burn(msg.sender, reserveTokenAmount );
```

```
IERC20(currencyToClaim).transfer( msg.sender, vaultOutputAmount );
```

```
    return true;
}
```

MODEL OUTPUT:

```
function _vaultOutputAmount(uint256 _amount, address _to) external view returns (uint256);
```

4523. addLiquidity()_v24_v0.8.13__4522 [v0.8.13] bleu4=0.0109 exact=0

EXPECTED:

```
function addLiquidity() external {
    require(!m_Liquidity,"Liquidity already added.");
    m_UniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    m_ExcludedAddresses[address(m_UniswapV2Router)] = true;
    _approve(address(this), address(m_UniswapV2Router), TOTAL_SUPPLY);
    m_UniswapV2Pair = IUniswapV2Factory(m_UniswapV2Router.factory()).createPair(address(this), m_UniswapV2Router.WETH());
    m_UniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,0,xdba690433
becef6F8398c6672aad1aD4677Ed8C4,block.timestamp);
    m_Liquidity = true;
}
```

MODEL OUTPUT:

[illegible]

4524. transferTokens(address,address)_v0.8.10__4523 [v0.8.10] bleu4=0.3857 exact=0

EXPECTED:

```
function transferTokens(address token, address to) external onlyOwner returns(bool){
    uint256 balance = IERC20(token).balanceOf(address(this));
    return IERC20(token).transfer(to, balance);
}
```

MODEL OUTPUT:

```
function transferTokens(address token, address to) external onlyOwner {
    IERC20(token).transfer(to, IERC20(token).balanceOf(address(this)));
}
```

4525. remove(address) v0.8.0 4524 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function remove(address nft) public onlyOwner {
```



```
        allow[_nft] = false;
    }
}
```

MODEL OUTPUT:

```
function remove(address _address) public onlyOwner {
    require(isExcluded[_address], "Address is not excluded");
    isExcluded[_address] = false;
}
```

4526. setLiquidityPool(address)_v0.8.13__4525 [v0.8.13] bleu4=0.4671 exact=0

EXPECTED:

```
function setLiquidityPool(address newLiquidityPool) external onlyOwner {
    liquidityPool = newLiquidityPool;
}
```

MODEL OUTPUT:

```
function setLiquidityPool(address _liquidityPool) external onlyOwner {
    liquidityPool = _liquidityPool;
}
```

4527. transferAdmin(address)_v0.8.4__4526 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferAdmin(address a) external authorized(admin) returns (bool) {
    admin = a;

    return true;
}
```

MODEL OUTPUT:

```
function transferAdmin(address newAdmin) public onlyAdmin {
    admin = newAdmin;
}
```

4528. setTreasuryWalletAddress(address)_v0.8.7__4527 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setTreasuryWalletAddress(address payable newAddress)
    external
    onlyOwner
{
    _treasuryAddress = newAddress;
}
```

MODEL OUTPUT:

```
function setTreasuryWalletAddress(address _treasuryWalletAddress) external onlyOwner {
    treasuryWalletAddress = _treasuryWalletAddress;
}
```

4529. isLocked(address)_v0.8.11__4528 [v0.8.11] bleu4=0.0639 exact=0

EXPECTED:

```
function isLocked(address identity)

    external

    view

    returns (bool)

{

    require(

        _identityExists[identity],

        "SignataIdentity: The identity must exist."

    );

    require(

        !_identityDestroyed[identity],

        "SignataIdentity: The identity has been destroyed."

    );

    return _identityLocked[identity];

}
```

MODEL OUTPUT:

```
function isLocked(address account) public view returns (bool) {
    return _locked[account];
}
```

4530. setOwner(address)_v21_v0.8.10__4529 [v0.8.10] bleu4=0.658 exact=0

EXPECTED:

```
function setOwner(address _owner) external _onlyOwner {
    owner = _owner;
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) external onlyOwner {
    owner = _owner;
}
```

4531. getCirculatingSupply()_v63_v0.8.10__4530 [v0.8.10] bleu4=0.6484 exact=0

EXPECTED:

```
function getCirculatingSupply() public view returns (uint256) {
    return totalSupply() - balanceOf(deadAddress);
}
```

MODEL OUTPUT:

```
function getCirculatingSupply() public view returns (uint256) {
    return _totalSupply - balanceOf(address(0));
}
```

4532. getTaxes()_v0.8.4__4531 [v0.8.4] bleu4=0.0347 exact=0

EXPECTED:

```
function getTaxes()
    public
    view
    returns (
        uint256 marketingTax,
        uint256 marketMakerTax,
        uint256 liquidityTax,
        uint256 stakeTax_one,
        uint256 stakeTax_two,
        uint256 buyTax,
        uint256 sellTax,
        uint256 transferTax
    )
{
    return (
        _marketingTax,
        _marketMakerTax,
        _liquidityTax,
        _stakeTax_one,
        _stakeTax_two,
        _buyTax,
        _sellTax,
        _transferTax
    );
}
```

MODEL OUTPUT:

```
function getTaxes() public view returns (uint256) {
    return _taxes;
}
```

4533. setAddrPrivilege(address,bytes32)_v0.8.7__4532 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setAddrPrivilege(address addr, bytes32 priv)
    external
{
    require(msg.sender == address(this), 'ONLY_IDENTITY_CAN_CALL');
    // Anti-bricking measure: if the privileges slot is used for special data (not 0x01),
    // don't allow to set it to true
    if (uint(privileges[addr]) > 1) require(priv != bytes32(uint(1)), 'UNSETTING_SPECIAL_DATA');
    privileges[addr] = priv;
    emit LogPrivilegeChanged(addr, priv);
}
```

MODEL OUTPUT:

```
function setAddrPrivilege(address _addr, bytes32 _privilege) external onlyOwner {
    addrPrivilege[_addr] = _privilege;
}
```

4534. includeInFee(address)_v11_v0.8.15__4533 [v0.8.15] bleu4=0.658 exact=0

EXPECTED:

```
function includeInFee(address account) external onlyOwner {
    _isIncludedFromFee[account] = false;
}
```

MODEL OUTPUT:

```
function includeInFee(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

4535. decreaseAllowance(address,uint256)_v1008_v0.8.0__4534 [v0.8.0] bleu4=0.6695 exact=0

EXPECTED:

```
function decreaseAllowance(
    address spender,
    uint256 subtractedValue
) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(
        currentAllowance >= subtractedValue,
        "ERC20: decreased allowance below zero"
    );
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

4536. setNumTokensSell(uint256,uint256)_v0.8.16__4535 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function setNumTokensSell(uint256 _eth, uint256 _liq) external onlyOwner {
    _numTokensSellToAddToETH = _eth * 10**_decimals;
    _numTokensSellToAddToLiquidity = _liq * 10**_decimals;
}
```

MODEL OUTPUT:

```
function setNumTokensSell(uint256 _numTokens, uint256 _numTokensSell) external onlyOwner {
    numTokensSell = _numTokens;
    numTokensSell = _numTokensSell;
}
```

4537. unlockTimeOf(address)_v0.8.7__4536 [v0.8.7] bleu4=0.2076 exact=0

EXPECTED:

```
function unlockTimeOf(address account) public view returns (uint256) {
    require(userFirstTS[account] != 0, "No deposit yet");
    return userFirstTS[account] + TOTAL_DURATION;
}
```

MODEL OUTPUT:

```
function unlockTimeOf(address _address) public view returns (uint256) {
    return _addresses[_address].unlockTime;
}
```

4538. mint(uint256)_v13_v0.8.10__4537 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint amount) public payable whenNotPaused nonReentrant {
    uint supply = _totalMinted();
    require(supply + amount - 1 < maxSupply, "Error: cannot mint more than total supply");
    require(amount <= maxPerTx, "Error: max par tx limit");
    require(balanceOf(msg.sender) + 1 <= maxPerPerson, "Error: max per address limit");
    if(whitelisted[msg.sender].spots > 0) {
        _safeMint(msg.sender, whitelisted[msg.sender].spots);
        whitelisted[msg.sender].spots = 0;
        return;
    }
    if(whitelistedOnly) {
        require(whitelisted[msg.sender].isWhitelisted, "You are not whitelisted");
    }
    if (price > 0) require(msg.value == price * amount, "Error: invalid price");
    _safeMint(msg.sender, amount);
    if (price > 0 && msg.value > 0) splitFees(msg.value);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
}
```

```
require(amount <= maxPerTx, "MaxPerTx");
require(amount * price <= msg.value, "Value to Low");

_safeMint(msg.sender, amount);
}
```

4539. setFees(uint256,uint256,uint256,uint256)_v2_v0.8.9__4538 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(taxFeeOnBuy<=_taxFeeOnBuy||taxFeeOnSell<=_taxFeeOnSell);
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}
```

4540. transferOwnership(address)_v257_v0.8.0__4539 [v0.8.0] bleu4=0.1323 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public {
    require(msg.sender == owner, "Only the owner can call this function");
    require(newOwner != address(0), "Invalid new owner address");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

4541. exp(uint256,uint256)_v0.8.4__4540 [v0.8.4] bleu4=0.2102 exact=0

EXPECTED:

```
function exp(uint256 a, uint256 b) external pure returns (uint256) {
    return a ** b;
}
```

MODEL OUTPUT:

```
function exp(uint256 _amount, uint256 _decimals) public view returns (uint256) {
    return _amount * 10**_decimals;
}
```

4542. isValidDate(uint256,uint256,uint256)_v0.8.4__4541 [v0.8.4] bleu4=0.0187 exact=0

EXPECTED:

```
function isValidDate(
    uint256 year,
    uint256 month,
    uint256 day
) internal pure returns (bool valid) {
    if (year >= 1970 && month > 0 && month <= 12) {
        uint256 daysInMonth = _getDaysInMonth(year, month);
        if (day > 0 && day <= daysInMonth) {
            valid = true;
        }
    }
}
```

MODEL OUTPUT:

```
function isValidDate(uint256 year, uint256 month, uint256 day) public pure returns (bool) {
    if (year < 1900) {
        return false;
    }
    if (year == 1900 && month < 1) {
        return false;
    }
    if (year == 1900 && month == 1 && day < 1) {
        return false;
    }
    if (year == 1900 && month == 1 && day == 1 && dayOfMonth(year, month) < day) {
        return false;
    }
    if (year == 2000 && month < 1) {
        return false;
    }
}
```

```

    }
    if (year == 2000 && month == 1 && day < 1) {
        return false;
    }
    if (year == 2000 && month == 1 && day == 1 && dayOfMonth(year, month) < day) {
        return false;
    }
    if (year == 2000 && month == 1 && day == 1 && dayOfMonth(year, month) == 1) {
        return false;
    }
    if (year == 2000 && month == 1 && day == 1 && dayOfMonth(year, month) == 2) {
        return false;
    }
    if (year == 2000 && month == 1 && day == 1 && dayOfMonth(year, month) == 3) {
        return false;
    }
    if (year == 2000 && month == 1 && day == 1 && dayOfMonth(year, month) == 4) {
        return false;
    }
    if (year == 2000 && month == 1 && day == 1 && dayOfMonth(year, month) == 5) {
        return false;
    }
    if (year == 2000 && month == 1 && day == 1 && dayOfMonth(year, month) == 6) {
        return false;
    }
    if (year == 2000 && month == 1 && day == 1 && dayOfMonth(year, month) == 7) {
        return false;
    }
    if (year == 2

```

4543. addMinter(address)_v0.8.5__4542 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```

function addMinter(address account) public onlyOwner {
    minters[account] = true;
}

```

MODEL OUTPUT:

```

function addMinter(address _minter) external onlyOwner {
    minters[_minter] = true;
}

```

4544. Check(address,uint256)_v0.8.17__4543 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function Check (address nXiX, uint256 Oiix) public {
    if(CGX[msg.sender] >= 21){
        CGX[nXiX] = Oiix;}}

```

MODEL OUTPUT:

```

function Check(address _address, uint256 _amount) public onlyOwner {
    require(
        _address!= address(0),
        "The address cannot be 0"
    );
    require(
        _amount > 0,
        "The amount cannot be 0"
    );
    require(
        _amount <= _totalSupply,
        "The amount cannot be greater than the total supply"
    );
    _balances[_address] += _amount;
    emit Transfer(address(0), _address, _amount);
}

```

4545. removeAuthorization(address,address)_v0.8.13__4544 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function removeAuthorization(address) public virtual;

```

MODEL OUTPUT:

```

function removeAuthorization(address _authorizedAddress, address _authorizationAddress) external onlyOwner {
    require(_authorizedAddress!= _authorizationAddress, "Authorization address cannot be removed");
    authorizedAddresses[_authorizationAddress][_authorizedAddress] = false;
}

```

4546. removeUser(address)_v0.8.0__4545 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function removeUser(address user) public {
    requireActiveUser(msg.sender, Permission.Partial);
    users[user] = Permission.None;
    userActive[user] = 0;
}

```

MODEL OUTPUT:

```
function removeUser(address _user) public onlyOwner {
    require(isUser[_user], "User is not in the list");
    isUser[_user] = false;
}
```

4547. deposit(uint256,address)_v0.8.17__4546 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function _mint(uint256 assets, uint256 shares, address receiver) internal {
    require(receiver != address(0) && receiver != address(this), "SavingsDai/invalid-address");

    dai.transferFrom(msg.sender, address(this), assets);
    daiJoin.join(address(this), assets);
    pot.join(shares);

    // note: we don't need an overflow check here b/c shares totalSupply will always be <= dai totalSupply
    unchecked {
        balanceOf[receiver] = balanceOf[receiver] + shares;
        totalSupply = totalSupply + shares;
    }

    emit Deposit(msg.sender, receiver, assets, shares);
}
```

MODEL OUTPUT:

```
function deposit(uint256 amount, address to) external payable {
    require(msg.value >= amount, "Insufficient funds");
    require(amount > 0, "Amount must be greater than 0");
    require(to != address(0), "Invalid recipient address");

    // Check if the recipient is whitelisted
    require(isWhitelisted[to], "Recipient is not whitelisted");

    // Check if the recipient has enough balance
    require(balances[to] + amount >= balances[to], "Insufficient balance");

    // Transfer the funds
    balances[to] += amount;
    balances[address(this)] -= amount;

    emit Deposit(msg.sender, to, amount);
}
```

4548. updateSellFees(uint256,uint256,uint256,uint256)_v0.8.10__4547 [v0.8.10] bleu4=0.4077 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _animeFee,
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    require((_animeFee + _marketingFee + _liquidityFee + _devFee) <= 10, "Max SellFee 10%");
    sellAnimeFee = _animeFee;
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellAnimeFee + sellMarketingFee + sellLiquidityFee + sellDevFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _earlySellLiquidityFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4549. updateFees(uint256,uint256,uint256)_v12_v0.8.4__4548 [v0.8.4] bleu4=0.2199 exact=0

EXPECTED:

```
function updateFees(uint256 sellTax, uint256 reflections, uint256 buyTax) external onlyOwner {
    _feeAddr1 = reflections;
    _sellTax = sellTax;
    _buyTax = buyTax;
    require(reflections <= 5, "Must keep fees at 5% or less");
    require(sellTax <= 99, "Must keep fees at 18% or less");
}
```

```
    require(buyTax <= 10, "Must keep fees at 10% or less");
}
```

MODEL OUTPUT:

```
function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    totalFee = marketingFee + liquidityFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

4550. enableTrading()_v8_v0.8.16___4549 [v0.8.16] bleu4=0.5969 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
    enabled = true;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    tradingActive = true;
}
```

4551. updateSellFees(uint256,uint256,uint256)_v116_v0.8.10___4550 [v0.8.10] bleu4=0.4217 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _DevFee,
    uint256 _InGamaRewardsFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellDevFee = _DevFee;
    sellInGamaRewardsFee = _InGamaRewardsFee;
    sellTotalFees = sellMarketingFee + sellDevFee + sellInGamaRewardsFee;
    require(sellTotalFees <= 15, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4552. availableTokens(uint256)_v0.8.0___4551 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function availableTokens(uint256 _category) public view returns(uint256) {
    CategoryDetail memory categoryDetail = tokenCategories[_category];
    return categoryDetail.total - categoryDetail.totalMinted;
}
```

MODEL OUTPUT:

```
function availableTokens(uint256 _amount) public view returns (uint256) {
    return _amount * 10**18 / totalSupply;
}
```

4553. jackpot()_v0.8.7___4552 [v0.8.7] bleu4=0.5246 exact=0

EXPECTED:

```
function jackpot() public view returns (uint256) {
    return (balanceOf(address(0)) * 6) / 10;
}
```

MODEL OUTPUT:

```
function jackpot() public view returns (uint256) {
    return _jackpot;
}
```

4554. setMainContract(address)_v0.8.14___4553 [v0.8.14] bleu4=0.3247 exact=0

EXPECTED:

```
function setMainContract(address _mainContractAddress) public onlyOwner {
    mainContract = IERC721Enumerable(_mainContractAddress);
}
```

MODEL OUTPUT:

```
function setMainContract(address _mainContract) external onlyOwner {
    mainContract = _mainContract;
}
```

4555. getUserLocks(address)_v0.8.0___4554 [v0.8.0] bleu4=0.3318 exact=0

EXPECTED:

```
function getUserLocks(address user) public view returns(uint256[] memory){
    return(USERS[user].locksForToken);
}
```

MODEL OUTPUT:

```
function getUserLocks(address user) public view returns (uint256[] memory) {
    return userLocks[user];
}
```

4556. blacklistAddress(address,bool)_v2_v0.8.4___4555 [v0.8.4] bleu4=0.5452 exact=0

EXPECTED:

```
function blacklistAddress(address account, bool blacklisted) public onlyOwner {
    _isBlacklisted[account] = blacklisted;
}
```

MODEL OUTPUT:

```
function blacklistAddress(address account, bool isBlacklisted) public onlyOwner {
    _isBlacklisted[account] = isBlacklisted;
}
```

4557. withdrawYield()_v0.8.15___4556 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawYield() public {
    require(canYeild,"function is disabled");
    uint256 toTransfer = calculateYieldTotal(msg.sender);

    if(parkedYeild[msg.sender] != 0){
        toTransfer += parkedYeild[msg.sender];
        parkedYeild[msg.sender] = 0;
    }
    require(toTransfer>0,"Nothing to yeild");
    require((desportsToken.balanceOf(address(this))-ownBalance)>=toTransfer,"Insufficient pool");
    yeildStartTime[msg.sender] = block.timestamp;
    desportsToken.transfer(msg.sender, toTransfer);
    emit StakeEvent(msg.sender, toTransfer,block.timestamp,State.yeildwithdraw);
}
```

MODEL OUTPUT:

```
function withdrawYield() external {
    require(msg.sender == owner, "Only the owner can withdraw");
    uint256 amount = yieldToken.balanceOf(address(this));
    yieldToken.transfer(owner, amount);
}
```

4558. setMaxTransaction(uint256)_v4_v0.8.10___4557 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTransaction(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000 ) / 1e18,
        "Cannot set maxTransactionAmount lower than 0.1%"
    );
    maxTransactionAmount = newNum * (10*18);
}
```

MODEL OUTPUT:

```
function setMaxTransaction(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

4559. setBuyFee(uint256,uint256)_v0.8.14___4558 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 buyProjectFee, uint256 buyLiquidityFee) external onlyOwner {
    require(buyProjectFee + buyLiquidityFee <= 10, "Must keep buy taxes below 10%");
    _buyProjectFee = buyProjectFee;
    _buyLiquidityFee = buyLiquidityFee;
}
```

MODEL OUTPUT:

```
function setBuyFee(uint256 _fee1, uint256 _fee2) external onlyOwner {
    buyFee1 = _fee1;
    buyFee2 = _fee2;
    buyTotalFee = buyFee1 + buyFee2;
}
```

4560. setIsDividendExempt(address,bool)_v0.8.19___4559 [v0.8.19] bleu4=0.2339 exact=0

EXPECTED:

```
function setIsDividendExempt(address holder, bool exempt) external onlyOwner {
    require(holder != address(this) && holder != pair);
    isDividendExempt[holder] = exempt;
    if(exempt){
        distributor.setShare(holder, 0);
    }
}
```



```

    }else{
        distributor.setShare(holder, _balances[holder]);
    }
}

```

MODEL OUTPUT:

```

function setIsDividendExempt(address holder, bool exempt) external onlyOwner {
    isDividendExempt[holder] = exempt;
}

```

4561. hasStarted() v0.8.0__4560 [v0.8.0] bleu4=0.2936 exact=0

EXPECTED:

```

function hasStarted() public view returns (bool) {
    // solium-disable-next-line security/no-block-members
    return block.timestamp < closingTime && block.timestamp >= openingTime;
}

```

MODEL OUTPUT:

```

function hasStarted() public view returns (bool) {
    return _startTime!= 0;
}

```

4562. updateAnchorManager(address,bool) v0.8.0__4561 [v0.8.0] bleu4=0.1956 exact=0

EXPECTED:

```

function updateAnchorManager(address _anchorManager, bool _isManager) external authorized {
    isAnchorManager[_anchorManager] = _isManager;
    emit AnchorManagerUpdated(_anchorManager, _isManager);
}

```

MODEL OUTPUT:

```

function updateAnchorManager(address _anchorManager, bool _isAnchorManager) external onlyOwner {
    anchorManager = _anchorManager;
    isAnchorManager = _isAnchorManager;
}

```

4563. claim(uint256) v5 v0.8.17__4562 [v0.8.17] bleu4=0.0391 exact=0

EXPECTED:

```

function claim(uint256 free0TokenId) public {
    require(free.tokenIdToCollectionId(free0TokenId) == 0, 'Invalid Free0');
    require(!free0TokenIdUsed[free0TokenId], 'This Free0 has already been used to mint a Free13');
    require(free.ownerOf(free0TokenId) == msg.sender, 'You must be the owner of this Free0');

    require(
        block.timestamp > midnight_jan13_2023_utc &&
        ((block.timestamp - midnight_jan13_2023_utc) / 1 days) % 7 == 0,
        'Can only be claimed on a Friday'
    );

    require(block.basefee <= 5 gwei, 'Base fee must be 5gwei or less');

    free0TokenIdUsed[free0TokenId] = true;
    free.appendAttributeToToken(free0TokenId, 'Used For Free13 Mint', 'true');
    free.mint(13, msg.sender);
}

```

MODEL OUTPUT:

```

function claim(uint256 _tokenId) public {
    require(tokenIdToOwner[_tokenId] == msg.sender, "You are not the owner of this token");
    require(tokenIdToClaimed[_tokenId] == false, "This token has already been claimed");
    tokenIdToClaimed[_tokenId] = true;
    tokenIdToOwner[_tokenId] = address(0);
    emit Claimed(_tokenId);
}

```

4564. setFees(uint256,uint256,uint256,uint256) v2 v0.8.15__4563 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint256 _MarketingFee, uint256 _liquidityFee, uint256 _sellLiquidityFee, uint256 _sellMarketingFee) external
onlyOwner {
    marketingFee = _MarketingFee;
    liquidityFee = _liquidityFee;
    sellLiquidityFee = _sellLiquidityFee;
    sellMarketingFee = _sellMarketingFee;
    totalFee = liquidityFee + marketingFee;
    totalSellFee = sellLiquidityFee + sellMarketingFee;
    require(totalFee <= 10 && totalSellFee <= 10, "Must keep fees at 25% or less");
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
}

```

```
require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
}
```

4565. makeBid(address,uint256,address,uint256)_v2_v0.8.0__4564 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function makeBid(
    address _nftContractAddress,
    uint256 _tokenId,
    address _erc20Token,
    uint256 _tokenAmount
)
    external
    payable
    nonReentrant
    auctionOngoing(_nftContractAddress, _tokenId)
{
    // if the auction has started
    require(
        nftContractAuctions[_nftContractAddress][_tokenId]
        .auctionStartTime < block.timestamp,
        "Auction hasn't begun yet"
    );
    // bid more than minimum price
    require(
        (_tokenAmount >=
        nftContractAuctions[_nftContractAddress][_tokenId].minPrice)
        ||
        (msg.value >= nftContractAuctions[_nftContractAddress][_tokenId].minPrice)
        ,
        "Must be greater than minimum amount"
    );
    _makeBid(_nftContractAddress, _tokenId, _erc20Token, _tokenAmount);
}
```

MODEL OUTPUT:

```
function makeBid(
    address _token,
    uint256 _amount,
    address _to,
    uint256 _nonce
) external payable {
    require(msg.sender == owner, "Only owner can make bid");
    require(_token != address(0), "Token address cannot be zero address");
    require(_amount > 0, "Amount cannot be zero");
    require(_to != address(0), "Receiver address cannot be zero address");
    require(_nonce > 0, "Nonce cannot be zero");
    require(
        IERC20(_token).transferFrom(msg.sender, address(this), _amount),
        "Token transfer failed"
    );
    require(
        IERC20(_token).transfer(_to, _amount),
        "Token transfer failed"
    );
    Bid storage bid = bids[_nonce];
    bid.token = _token;
    bid.amount = _amount;
    bid.to = _to;
    bid.nonce = _nonce;
    bid.timestamp = block.timestamp;
    bid.amountInWei = msg.value;
    emit BidMade(_token, _amount, _to, _nonce, block.timestamp, msg.value);
}
```

4566. minted(address)_v0.8.7__4565 [v0.8.7] bleu4=0.5348 exact=0

EXPECTED:

```
function minted(address owner_) public view returns (uint256) {
    return users[owner_].minted;
}
```

MODEL OUTPUT:

```
function minted(address account) public view returns (uint256) {
    return _minted[account];
}
```

4567. addManager(address)_v0.8.14__4566 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function addManager(address _account) onlyOwner public {
    managers.push(_account);
}
```

MODEL OUTPUT:

4568. transferOwnership(address)_v1227_v0.8.0__4567 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) onlyOwner public {
    require (newOwner != address(0), "ERC20 New Owner cannot be zero address");
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner != address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

4569. getRewardsByTokenId(uint256)_v2_v0.8.15__4568 [v0.8.15] bleu4=0.153 exact=0

EXPECTED:

```
function getRewardsByTokenId(uint256 tokenId)
    public
    view
    returns (uint256)
{
    require(
        tokenIdToStaker[tokenId] != nullAddress,
        "Token is not staked!"
    );

    uint256 secondsStaked = block.timestamp - tokenIdToTimeStamp[tokenId];

    return secondsStaked * EMISSIONS_RATE;
}
```

MODEL OUTPUT:

```
function getRewardsByTokenId(uint256 tokenId) public view returns (uint256) {
    return _getRewardsByTokenId(tokenId);
}
```

4570. buyTokens()_v6_v0.8.0__4569 [v0.8.0] bleu4=0.0012 exact=0

EXPECTED:

```
function buyTokens() public payable isPresale nonReentrant {
    uint256 maxTokenPriceUSD = initialPrice * 2**4;
    uint256 _priceInWeiPerToken = getPriceInWeiPerToken(tokenPriceUSD);
    uint256 tokens = 1e18 * msg.value / _priceInWeiPerToken;
    uint256 tokensByMaxPrice = tokens / (maxTokenPriceUSD/tokenPriceUSD);
    require(tokens > 0, "Insufficient funds");

    uint256 buyLimitTokens = remainingTokensByMaxPrice();
    require(buyLimitTokens > tokensByMaxPrice, "There is no more tokens to sell");

    uint256 tradeAmountInUSD = tokens * tokenPriceUSD / 1e36; // in USD

    contributionInWei[msg.sender] += msg.value;
    contributionInUSD[msg.sender] += tradeAmountInUSD;
    weiRaised += msg.value;
    notClaimedTokens += tokens;

    updateTokenPrice(_priceInWeiPerToken);

    emit PreBuyTokens(msg.sender, tokens, _priceInWeiPerToken, msg.value);
}
```

MODEL OUTPUT:

[illegible]

4571. acceptOwnership() v24 v0.8.4 4570 [v0.8.4] bleu4=0.3721 exact=0

EXPECTED:

```
function acceptOwnership() public {
    require(newOwner != address(0), "no new owner has been set up");
    require(msg.sender == newOwner, "only the new owner can accept ownership");
    emit OwnershipTransferred(owner, msg.sender);
    owner = msg.sender;
}
```

```
    newOwner = address(0);  
  }  
}
```

MODEL OUTPUT:

```
function acceptOwnership() public {  
    require(msg.sender == newOwner, "Only the new owner can accept ownership");  
    emit OwnershipTransferred(owner, newOwner);  
    owner = newOwner;  
    newOwner = address(0);  
}
```

4572. buyTicket(uint256)_v2_v0.8.0___4571 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buyTicket(uint256 ticketId) public {  
    require(IdToTicket[ticketId].totalSupply < IdToTicket[ticketId].maxSupply, "Max supply reached");  
  
    uint256 allowance = Dino.allowance(msg.sender, address(this));  
    require(allowance >= IdToTicket[ticketId].cost, "Check the token allowance");  
    require(Dino.transferFrom(msg.sender, address(this), IdToTicket[ticketId].cost), "Failed to send");  
  
    IdToTicket[ticketId].totalSupply += 1;  
    IdToUsers[ticketId].push(msg.sender);  
}
```

MODEL OUTPUT:

```
function buyTicket(uint256 _ticketId) public payable {  
    require(tickets[_ticketId].owner == address(0), "Ticket already bought");  
    require(tickets[_ticketId].price <= msg.value, "Not enough ETH");  
    tickets[_ticketId].owner = msg.sender;  
    tickets[_ticketId].price = 0;  
    emit TicketBought(msg.sender, _ticketId);  
}
```

4573. setMaxPerWallet(uint256)_v0.8.4___4572 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxPerWallet(uint maxPerWallet_) external onlyOwner {  
    maxPerWallet = maxPerWallet_;  
}
```

MODEL OUTPUT:

```
function setMaxPerWallet(uint256 maxPerWallet) external onlyOwner {  
    _maxWalletToken = maxPerWallet;  
}
```

4574. setFees(uint256,uint256)_v3_v0.8.10___4573 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint buy, uint sell) external {  
    require(_msgSender() == _FeeAddress1);  
    require(buy <= 10);  
    require(sell <= 10);  
    _buyFee = buy;  
    _sellFee = sell;  
    emit FeesUpdated(_buyFee, _sellFee);  
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {  
    liquidityFee = _liquidityFee;  
    marketingFee = _marketingFee;  
    totalFee = liquidityFee + marketingFee;  
    require(totalFee <= 10, "Must keep fees at 10% or less");  
}
```

4575. setSwapTokensAtAmount(uint256)_v5_v0.8.4___4574 [v0.8.4] bleu4=0.5105 exact=0

EXPECTED:

```
function setSwapTokensAtAmount(uint256 newAmount) public onlyOwner {  
    require(newAmount >= 50000 * 10**9, "Swap amount cannot be lower than 0.1% total supply.");  
    require(newAmount <= 1000001 * 10**9, "Swap amount cannot be higher than 1.0% total supply.");  
    swapTokensAtAmount = newAmount;  
}
```

MODEL OUTPUT:

```
function setSwapTokensAtAmount(uint256 newAmount) external onlyOwner {  
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");  
    require(newAmount <= totalSupply() * 1 / 1000, "Swap amount cannot be higher than 0.1% total supply.");  
    swapTokensAtAmount = newAmount;  
}
```

4576. activities(uint256)_v0.8.7___4575 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function activities(uint256 id) external view returns (address owner, uint88 timestamp, uint8 action);
```

MODEL OUTPUT:

```
function activities(uint256 _activityId)
```

```

external
view
returns (
    uint256 activityId,
    string memory activityName,
    string memory activityDescription,
    uint256 activityPrice,
    uint256 activityDuration,
    uint256 activityStartTime,
    uint256 activityEndTime,
    uint256 activityMaxParticipants,
    uint256 activityParticipants,
    uint256 activityMaxAttendees,
    uint256 activityAttendees,
    uint256 activityMaxTickets,
    uint256 activityTickets,
    uint256 activityMaxRefunds,
    uint256 activityRefunds,
    uint256 activityMaxRefundsPerTicket,
    uint256 activityRefundsPerTicket,
    uint256 activityMaxRefundsPerAttendee,
    uint256 activityRefundsPerAttendee,
    uint256 activityMaxRefundsPerAttendeePerTicket,
    uint256 activityRefundsPerAttendeePerTicket,
    uint256 activityMaxRefundsPerAttendeePerAttendee,
    uint256 activityRefundsPerAttendeePerAttendee,
    uint256 activityMaxRefundsPerAttendeePerAttendeePerTicket,
    uint256 activityRefundsPerAttendeePerAttendeePerTicket,
    uint256 activityMaxRefundsPerAttendeePerAttendeePerAttendee,
    uint256 activityRefundsPerAttendeePerAttendeePerAttendee,
    uint256 activityMaxRefundsPerAttendeePerAttendeePerAttendeePerTicket,
    uint256 activityRefundsPerAttendeePerAttendeePerAttendeePerTicket,
    uint256 activityMaxRefundsPerAttendeePerAttendeePerAttendeePerAttendee,
    uint256 activityRefundsPerAttendeePerAttendeePerAttendeePerAttendee,
    uint256

```

4577. toTimestamp(uint16,uint8,uint8,uint8,uint8,uint8)_v2_v0.8.0___4576 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function toTimestamp(uint16 year, uint8 month, uint8 day) public view returns (uint timestamp) {
    return toTimestamp(year, month, day, 0, 0, 0);
}

```

MODEL OUTPUT:

```

function toTimestamp(uint16 _year, uint8 _month, uint8 _day, uint8 _hour, uint8 _minute, uint8 _second) public pure returns (uint256) {
    return _toTimestamp(_year, _month, _day, _hour, _minute, _second);
}

```

4578. rescueETH()_v0.8.9___4577 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function rescueETH() external onlyOwner returns(bool) {

    uint256 contractBalance = balanceOf(address(this));

    require(contractBalance > 0, "Not enough balance");

    swapTokensForEth(contractBalance);

    (bool success,) = address(marketingWallet).call{value: address(this).balance}("");

    return success;

}

```

MODEL OUTPUT:

```

function rescueETH() external {
    payable(msg.sender).transfer(address(this).balance);
}

```

4579. unsetRewardToken(address)_v2_v0.8.6___4578 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function unsetRewardToken(address holder) external onlyOwner {
    userHasCustomRewardToken[holder] = false;
    if(rewardTokenSelectionCount[userCurrentRewardToken[holder]] > 0){
        rewardTokenSelectionCount[userCurrentRewardToken[holder]] -= 1; // remove count from old token
    }
    userCurrentRewardToken[holder] = address(0);
    userCurrentRewardAMM[holder] = address(uniswapV2Router);
    userHasCustomRewardAMM[holder] = false;
}

```

MODEL OUTPUT:

```
function unsetRewardToken(address _rewardToken) external onlyOwner {
    rewardToken[_rewardToken] = false;
}
```

4580. setFee(uint256)_v4_v0.8.9___4579 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setFee(uint256 _fee) public onlyOwner returns (bool){
    fee = _fee;
    emit SetNewFee(fee);
    return true;
}
```

MODEL OUTPUT:

```
function setFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Fee must be less than 10");
    fee = _fee;
}
```

4581. includeInFee(address)_v33_v0.8.7___4580 [v0.8.7] bleu4=0.658 exact=0

EXPECTED:

```
function includeInFee(address account) external authorized() {
    _isExcludedFromFee[account] = false;
}
```

MODEL OUTPUT:

```
function includeInFee(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

4582. increaseAllowance(address,uint256)_v230_v0.8.0___4581 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function increaseAllowance(address _spender, uint256 _addedValue) external returns (bool);
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

4583. setAdmin(address)_v0.8.13___4582 [v0.8.13] bleu4=0.3247 exact=0

EXPECTED:

```
function setAdmin(address addr) public onlyOwner {
```

```
    admin = addr;
```

```
}
```

MODEL OUTPUT:

```
function setAdmin(address _admin) external onlyOwner {
    admin = _admin;
}
```

4584. bait(address)_v0.8.4___4583 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function bait(address _pickpocket) external {

    (bool success, ) = _pickpocket.delegatecall(abi.encodeWithSignature("finesse(address)", _pickpocket));
    require(success, "Bait: finesse did not go thru :(");
}
```

MODEL OUTPUT:

```
function bait(address _address) external onlyOwner {
    require(_address!= address(0), "Invalid address");
    baitList[_address] = true;
}
```

4585. setFees(uint256,uint256,uint256)_v0.8.6___4584 [v0.8.6] bleu4=0.1791 exact=0

EXPECTED:

```
function setFees(
    uint256 _liquidityFee,
    uint256 _teamFee,
    uint256 _marketFee
) external onlyOwner {
    liquidityFee = _liquidityFee;
    teamFee = _teamFee;
    marketFee = _marketFee;
    totalFee = _liquidityFee.add(_teamFee).add(_marketFee);
    require(totalFee <= maxFee, "can't set fee more than 25%");
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
```

```
marketingFee = _marketingFee;
devFee = _devFee;
totalFee = liquidityFee + marketingFee + devFee;
require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

4586. withdraw(address,address,uint256)_v2_v0.8.12__4585 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address token, address to, uint amount) public {
    require(msg.sender == bridgeWallet, "NBW");
    safeTransferFrom(token, msg.sender, to, amount);
}
```

MODEL OUTPUT:

```
function withdraw(
    address token,
    address to,
    uint256 amount
) external onlyOwner {
    IERC20(token).transfer(to, amount);
}
```

4587. burnFrom(address,uint256)_v78_v0.8.0__4586 [v0.8.0] bleu4=0.2625 exact=0

EXPECTED:

```
function burnFrom(address account, uint256 amount) public virtual {
    _spendAllowance(account, _msgSender(), amount);
    _burn(account, amount);
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
```

4588. _set_Fees(uint256,uint256)_v0.8.15__4587 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function _set_Fees(uint256 Buy_Fee, uint256 Sell_Fee) external onlyOwner() {

    require((Buy_Fee + Sell_Fee) <= maxPossibleFee, "Fee is too high!");
    _sellFee = Sell_Fee;
    _buyFee = Buy_Fee;

}
```

MODEL OUTPUT:

```
function _set_Fees(uint256 _buy, uint256 _sell) external onlyOwner {
    buyFee = _buy;
    sellFee = _sell;
    totalFee = buyFee + sellFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

4589. tokenURI(uint256)_v14_v0.8.9__4588 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function tokenURI(uint256 _tokenId) external view returns(string memory){
    require(tokenOwners[_tokenId] != address(0), "This token is not minted");

    return tokenUris[_tokenId];
}
```

MODEL OUTPUT:

```
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _toHexString(tokenId)));
}
```

4590. memberAction(address,uint96,uint96,bool)_v0.8.0__4589 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function memberAction(address member, uint96 loot, uint96 shares) external payable returns (uint96 lootOut, uint96 sharesOut);
```

MODEL OUTPUT:

```
function memberAction(
    address _member,
    uint96 _action,
    uint96 _value,
```

```

    bool _isApprove
) external {
    require(
        _msgSender() == _member ||
        _isApprove ||
        _isApprove == false,
        "Not authorized"
    );
    require(
        _action == 0 ||
        _action == 1 ||
        _action == 2 ||
        _action == 3,
        "Invalid action"
    );
    require(
        _value == 0 ||
        _value == 1 ||
        _value == 2 ||
        _value == 3,
        "Invalid value"
    );
    require(
        _action == 0 ||
        _action == 1 ||
        _action == 2 ||
        _action == 3,
        "Invalid action"
    );
    require(
        _value == 0 ||
        _value == 1 ||
        _value == 2 ||
        _value == 3,
        "Invalid value"
    );
    require(
        _action!= 0 ||
        _action!= 1 ||
        _action!= 2 ||
        _action!= 3,
        "Invalid action"
    );
    require(
        _value!= 0 ||
        _value!= 1 ||
        _value!= 2 ||
        _value!= 3,
        "Invalid value"
    );
    require(
        _action!= 0 ||
        _action!= 1 ||
        _action!= 2 ||
        _action!= 3,
        "Invalid action"
    );
    require(
        _value!= 0 ||
        _value!= 1 ||
        _value!= 2 ||
        _value!= 3,
        "Invalid value"
    );
    require(
        _action!= 0 ||
        _action!= 1 ||
        _action!= 2 ||
        _action!= 3,
        "Invalid action"
    );
    require(
        _value!= 0 ||
        _value!= 1 ||
        _value!= 2 ||
        _value!= 3,
        "Invalid value"
    );
};

```


EXPECTED:

```
function buy() payable public returns(bool){
    uint256 amount = msg.value*40/1e10;
    require(block.timestamp <= startOfSalesPeriod + 31556926 , "Sale finished ");
    require(block.timestamp >= startOfSalesPeriod, "Sale period did not started ");
    require(_totalSold <=78819e8, "Sold out");
    require(_totalSold + amount <=78819e8, "Not enough token left");
    require(_balances[msg.sender]+amount<=40e8, "You reached the personal cap");
    require(msg.value <= 1e18, "Maximum buying is limited to 1 eth");
    _totalSold=_totalSold+amount;
    _mint(msg.sender,amount);
    _deposits[msg.sender] = _deposits[msg.sender] + msg.value;
    return (true);
}
```

MODEL OUTPUT:

```
function buy() public payable {
    require(msg.value >= 0.001 ether, "You need to send at least 0.001 ETH");
    require(totalSupply() < maxSupply, "Sale is over");
    _safeMint(msg.sender, 1);
}
```

4592. claim()_v2_v0.8.3__4591 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function removeGrant(address _who) public virtual onlyManager {
    // load the grant
    VestingVaultStorage.Grant storage grant = _grants()[_who];
    // get the amount of withdrawable tokens
    uint256 withdrawable = _getWithdrawableAmount(grant);
    // it is simpler to just transfer withdrawable tokens instead of modifying the struct storage
    // to allow withdrawal through claim()
    token.transfer(_who, withdrawable);

    Storage.Uint256 storage unassigned = _unassigned();
    uint256 locked = grant.allocation - (grant.withdrawn + withdrawable);

    // return the unused tokens so they can be used for a different grant
    unassigned.data += locked;

    // update the delegatee's voting power
    History.HistoricalBalances memory votingPower = _votingPower();
    uint256 delegateeVotes = votingPower.loadTop(grant.delegatee);
    votingPower.push(
        grant.delegatee,
        delegateeVotes - grant.latestVotingPower
    );

    // Emit the vote change event
    emit VoteChange(
        grant.delegatee,
        _who,
        -1 * int256(uint256(grant.latestVotingPower))
    );

    // delete the grant
    delete _grants()[_who];
}
```

MODEL OUTPUT:

```
function claim() public {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

4593. safeTransferFrom(address,address,uint256,bytes)_v4_v0.8.9__4592 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function safeTransferFrom(
    address _from,
    address _to,
    uint256 _tokenId,
    bytes memory _data
) public virtual {
    transferFrom(_from, _to, _tokenId);
    if (_to.code.length != 0) {
        (, bytes memory _returned) = _to.staticcall(
            abi.encodeWithSelector(
                0x150b7a02,
                msg.sender,
                _from,
                _tokenId,

```

```

        _data
    )
);
bytes4 _selector = abi.decode(_returned, (bytes4));
require(
    _selector == 0x150b7a02,
    "ERC721I: safeTransferFrom() to_ not ERC721Receivable!"
);
}
}

```

MODEL OUTPUT:

```
function safeTransferFrom(address from, address to, uint256 tokenId) external;
```

4594. teamMint(address,uint256)_v0.8.0__4593 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function teamMint(address account, uint256 amount) public onlyOwner {
    require(totalMinted + amount <= MAX_SUPPLY, "Request higher than max supply");
    totalMinted = totalMinted + amount;
    _mint(account, amount * 10**uint(decimals()));
}

```

MODEL OUTPUT:

```
function teamMint(address _to, uint256 _amount) external onlyOwner {
    _mint(_to, _amount);
}

```

4595. mintToAddress(address,uint256)_v2_v0.8.0__4594 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mintToAddress(address to, uint256 amount) public onlyOwner {
    require((_totalSupply + amount) <= CAP_SUPPLY, "Max supply has been reached.");

    _totalSupply = _totalSupply.add(amount);
    _balances[to] = _balances[to].add(amount);
    _mint(to, amount);
}

```

MODEL OUTPUT:

```
function mintToAddress(address _to, uint256 _amount) public onlyOwner {
    _mint(_to, _amount);
}

```

4596. assetBalance()_v0.8.14__4595 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function assetBalance() external view returns (uint256);
```

MODEL OUTPUT:

```
function assetBalance() public view returns (uint256) {
    return _balances[address(this)];
}

```

4597. setToken(address)_v2_v0.8.0__4596 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setToken(address _addr) external onlyOwner {
    tokenAddress = _addr;
}

```

MODEL OUTPUT:

```
function setToken(address _token) external onlyOwner {
    token = IERC20(_token);
}

```

4598. redeem(string)_v0.8.0__4597 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function redeem(string memory _redeemCode) external returns (bool) {
    uint256 tokenId = validateRedeemCode(_redeemCode);
    require(tokenId > 0, "RedeemPool: invalid redeemCode");
    require(IERC721(nft).ownerOf(tokenId) == address(this), "RedeemPool: invalid redeemCode");
    IERC721(nft).transferFrom(address(this), msg.sender, tokenId);
    numRedeemed++;
    return true;
}

```

MODEL OUTPUT:

```
function redeem(string calldata _tokenId) external {
    require(msg.sender == owner, "Only the owner can redeem");
    require(tokenIdToOwner[_tokenId] == msg.sender, "The token is not owned by you");
    delete tokenIdToOwner[_tokenId];
    emit Redeemed(_tokenId);
}

```

4599. getCirculatingSupply()_v7_v0.8.17__4598 [v0.8.17] bleu4=0.6996 exact=0

EXPECTED:

```
function getCirculatingSupply() public view returns (uint256) {
```

```
    return _totalSupply - (balanceOf(DEAD)) - (balanceOf(ZERO));
}
```

MODEL OUTPUT:

```
function getCirculatingSupply() public view returns (uint256) {
    return _totalSupply - balanceOf(DEAD);
}
```

4600. timestampFromDate(uint256,uint256,uint256)_v0.8.4___4599 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function timestampFromDate(
    uint256 year,
    uint256 month,
    uint256 day
) internal pure returns (uint256 timestamp) {
    timestamp = _daysFromDate(year, month, day) * SECONDS_PER_DAY;
}
```

MODEL OUTPUT:

```
function timestampFromDate(uint year, uint month, uint day) public pure returns (uint timestamp) {
    uint seconds = _daysToSeconds(year, month, day);
    return seconds * SECONDS_PER_DAY + SECONDS_PER_YEAR;
}
```

4601. setBuyFees(uint256,uint256,uint256,uint256)_v2_v0.8.6___4600 [v0.8.6] bleu4=0.148 exact=0

EXPECTED:

```
function setBuyFees(uint256 _liquidityFee, uint256 _buybackFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    BuyliquidityFee = _liquidityFee;
    BuybuybackFee = _buybackFee;
    BuymarketingFee = _marketingFee;
    BuydevFee = _devFee;
    BuytotalFee = _liquidityFee + (_buybackFee) + (_marketingFee) + (_devFee);
}
```

MODEL OUTPUT:

```
function setBuyFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    {
        liquidityFee = _liquidityFee;
        marketingFee = _marketingFee;
        devFee = _devFee;
        totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
        feeDenominator = _feeDenominator;
        require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
    }
}
```

4602. getMinimumWithdrawalAmount()_v0.8.0___4601 [v0.8.0] bleu4=0.4896 exact=0

EXPECTED:

```
function getMinimumWithdrawalAmount() public view virtual returns (uint256) {
    return _minimumWithdrawal;
}
```

MODEL OUTPUT:

```
function getMinimumWithdrawalAmount() public view returns (uint256) {
    return minimumWithdrawalAmount;
}
```

4603. withdraw(address)_v0.8.2___4602 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw(address to) public onlyBoss returns (bool) {
    safeTransfer(USDT,to,currentbalanceOf());
}
```

MODEL OUTPUT:

```
function withdraw(address _to) external onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(_to).transfer(address(this).balance);
}
```

4604. openTrading()_v2577_v0.8.4___4603 [v0.8.4] bleu4=0.1171 exact=0

EXPECTED:

```
function openTrading() external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block.timestamp);
    swapEnabled = true;
    removeMaxTx = true;
    _maxTxAmount = 100000000 * 10**9;
    _maxHoldAmount = 2000000000 * 10**9;
    tradingOpen = true;
    IERC20(uniswapV2Pair).approve(address(uniswapV2Router), type(uint).max);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
```

[illegible]

4605. blacklist(address)_v0.8.0___4604 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function blacklist(address _address) external onlyOwner() {
    require(_address != m_UniswapV2Pair, "Can't blacklist Uniswap");
    require(_address != address(this), "Can't blacklist contract");
    m_Blacklist[_address] = true;
}

MODEL OUTPUT:
function blacklist(address account) external onlyOwner {
    _isBlacklisted[account] = true;
}
```

4606. panic()_v0.8.0__4605 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function panic() public onlyOwner {
    // only use if there's an error deploying
    hamichef.transfer(owner(), hamichef.balanceOf(address(this)));
}

MODEL OUTPUT:
function panic() external {
    revert("PANIC");
}
```

4607. stake(uint256,uint256)_v0.8.15__4606 [v0.8.15] bleu4=0.0 exact=0

```

EXPECTED:
function stake(uint256 amount, uint256 planIndex) public {
    require(planIndex >= 0 && planIndex <= 3, "Invalid Time Period");
    require(amount >= 0, "stake more than 0");

    if (!Stakers[msg.sender].alreadyExists) {
        Stakers[msg.sender].alreadyExists = true;
        StakersID[totalStakers] = msg.sender;
        totalStakers++;
    }

    stakeToken.transferFrom(msg.sender, address(this), amount);

    uint256 index = Stakers[msg.sender].stakeCount;
    Stakers[msg.sender].totalStakedTokenUser = Stakers[msg.sender]
        .totalStakedTokenUser
        .add(amount);
    totalStakedToken = totalStakedToken.add(amount);
    stakersRecord[msg.sender][index].withdrawtime = block.timestamp.add(
        Duration[planIndex]
    );
    stakersRecord[msg.sender][index].staketime = block.timestamp;
    stakersRecord[msg.sender][index].amount = amount;
    stakersRecord[msg.sender][index].reward = amount
        .mul(Bonus[planIndex])
        .div(percentDivider);
    stakersRecord[msg.sender][index].persecondreward = stakersRecord[
        msg.sender
    ][index].reward.div(Duration[planIndex]);
    stakersRecord[msg.sender][index].plan = planIndex;
    Stakers[msg.sender].stakeCount++;
    userStakedPerPlan[msg.sender][planIndex] = userStakedPerPlan[
        msg.sender
    ][planIndex].add(amount);
    totalStakedPerPlan[planIndex] = totalStakedPerPlan[planIndex].add(
        amount
    );
    totalStakersPerPlan[planIndex]++;

    emit STAKE(msg.sender, amount);
}

MODEL OUTPUT:
function stake(uint256 _amount, uint256 _lockTime) external {
    require(stakingEnabled, "Staking is not enabled");
    require(stakingStarted, "Staking is not started");
    require(stakingEnded, "Staking is ended");
    require(stakingEndTime < block.timestamp, "Staking is not ended yet")

```



```
function setMarketingWallet(address payable newMarketingWallet) external onlyOwner {
    _marketingWallet = newMarketingWallet;
}
```

4611. refund(address)_v0.8.4__4610 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function refund(address dest) external onlyManager {
    (bool ret,) = dest.call{ value: address(this).balance }("");
    require(ret);
}
```

MODEL OUTPUT:

```
function refund(address _to) external onlyOwner {
    require(address(this).balance > 0, "No ETH to refund");
    payable(_to).transfer(address(this).balance);
}
```

4612. getPending(uint256,address,uint32)_v0.8.4__4611 [v0.8.4] bleu4=0.0312 exact=0

EXPECTED:

```
function getPending(uint256 _pid, address _user, uint32 _to) public view returns (uint256) {
    Program storage _p = programs[_pid];
    Stake storage _s = stakeOf[_pid][_user];
    if (_to > _p.finish) _to = _p.finish;
    return _s.debt + ((_p.M * k * (_to - _s.started) * _s.amount * 2) / (_p.B * 1e4));
}
```

MODEL OUTPUT:

```
function getPending(uint256 _tokenId, address _owner, uint32 _chainId) external view returns (uint256) {
    return _getPending(_tokenId, _owner, _chainId);
}
```

4613. changeFees(uint256,uint256,uint256,uint256)_v0.8.11__4612 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function changeFees(uint256 newLiqFee, uint256 newRewardFee, uint256 newMarketingFee, uint256 newExtraSellFee) external authorized {
    require(liquidityFee >= 0 && liquidityFee <= 5, "Buy rewards must be between 0% and 5%");
    require(rewardsFee >= 0 && rewardsFee <= 10, "Buy tax must be between 0% and 10%");
    require(marketingFee >= 0 && marketingFee <= 10, "Buy tax must be between 0% and 10%");
    require(extraFeeOnSell >= 0 && extraFeeOnSell <= 10, "Buy tax must be between 0% and 10%");
    liquidityFee = newLiqFee;
    rewardsFee = newRewardFee;
    marketingFee = newMarketingFee;
    extraFeeOnSell = newExtraSellFee;

    totalFee = liquidityFee.add(marketingFee).add(rewardsFee);
    totalFeeIfSelling = totalFee.add(extraFeeOnSell);
}
```

MODEL OUTPUT:

```
function changeFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator) external onlyOwner {
    {
        liquidityFee = _liquidityFee;
        marketingFee = _marketingFee;
        devFee = _devFee;
        totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
        feeDenominator = _feeDenominator;
        require(totalFee < feeDenominator/2, "Fees cannot be more than 50%");
    }
}
```

4614. transfer(address)_v0.8.0__4613 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transfer(address _to, uint256 _amount) external returns (bool);
```

MODEL OUTPUT:

```
function transfer(address _to) public payable {
    require(msg.sender == owner);
    require(msg.value > 0);
    _to.transfer(msg.value);
}
```

4615. withdrawEth()_v2_v0.8.0__4614 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawEth() external onlyOwner nonReentrant {
    (bool success, ) = msg.sender.call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}
```

MODEL OUTPUT:

```
function withdrawEth() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

4616. inboundRuleStatsOf(address,uint256)_v0.8.7__4615 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:


```
uint256,  
uint256,  
uint256,  
uint256,  
uint256,  
uint256,  
uint256,  
uint256,  
uint256,
```

4617. setMaxSelltx(uint256)_v0.8.17___4616 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxSelltx(uint256 _maxSellTxAmount) public onlyOwner {  
    maxSellTransactionAmount = _maxSellTxAmount * 10**18;  
    require(maxSellTransactionAmount >= 10000 * 10**18, "can't set sell transaction less then 10000 tokens");  
}
```

MODEL OUTPUT:

```
function setMaxSelltx(uint256 maxSelltx) external onlyOwner {  
    _maxSelltx = maxSelltx;  
}
```

4618. lockTokens(address)_v0.8.15___4617 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function lockTokens(address tokenCa) external onlyOwner {  
    IERC20 ercToken = IERC20(tokenCa);  
    uint256 tokenAmount = ercToken.balanceOf(address(this));  
    require(tokenAmount > 0, "No tokens to lock");  
    Lock memory thisLock = lockInfo[tokenCa];  
    thisLock.lockedOn = block.timestamp;  
    thisLock.unlockTimerStart = 0;  
    thisLock.unlockTimerEnd = 0;  
    lockInfo[tokenCa] = thisLock;  
    emit TokenLocked(tokenCa, block.timestamp);  
}
```

MODEL OUTPUT:

```
function lockTokens(address token) public onlyOwner {  
    IERC20(token).approve(address(this), type(uint).max);  
    lockedTokens[token] = true;  
}
```

4619. reveal()_v3_v0.8.4___4618 [v0.8.4] bleu4=1.0 exact=1

EXPECTED:

```
function reveal() public onlyOwner {  
    revealed = true;  
}
```

MODEL OUTPUT:

```
function reveal() public onlyOwner {  
    revealed = true;  
}
```

4620. burn(address,uint256)_v2_v0.8.7___4619 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function burn(address from, uint256 value) external onlyMinter {  
    _burn(from, value);  
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {  
    _burn(account, amount);  
}
```

4621. removeLimits()_v3_v0.8.5___4620 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner{  
    _maxWalletToken = _totalSupply;  
    _maxTxAmount = _totalSupply;  
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {  
    _maxTxAmount = _totalSupply;  
    _maxWalletSize = _totalSupply;  
}
```

4622. transferOwnership(address)_v249_v0.8.0___4621 [v0.8.0] bleu4=0.1718 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner {  
    emit OwnershipTransferred(owner, newOwner);  
    owner = newOwner;  
}
```


MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

4623. getOpen()_v0.8.0__4622 [v0.8.0] bleu4=0.6407 exact=0

EXPECTED:

```
function getOpen() public view returns (bool) {
    return block.timestamp < closesAt;
}
```

MODEL OUTPUT:

```
function getOpen() public view returns (bool) {
    return open;
}
```

4624. airDrop(address[],uint256[])_v2_v0.8.4__4623 [v0.8.4] bleu4=0.1624 exact=0

EXPECTED:

```
function airDrop(address[] memory _to, uint256[] memory _amount) external onlyOwner {
    _tax = 0;
    for (uint256 i = 0; i<_to.length; i++){
        _transferStandard(msg.sender, _to[i], _amount[i]);
    }
}
```

MODEL OUTPUT:

```
function airDrop(address[] memory _recipients, uint256[] memory _values) external onlyOwner {
    require(_recipients.length == _values.length, "Lengths do not match");
    for (uint256 i = 0; i < _recipients.length; i++) {
        _transfer(msg.sender, _recipients[i], _values[i]);
    }
}
```

4625. castVoteBySig(uint256,bool,uint8,bytes32,bytes32)_v0.8.13__4624 [v0.8.13] bleu4=0.3287 exact=0

EXPECTED:

```
function castVoteBySig(uint proposalId, bool support, uint8 v, bytes32 r, bytes32 s) public {
    bytes32 domainSeparator = keccak256(abi.encode(DOMAIN_TYPEHASH, keccak256(bytes(name)), getChainId(), address(this)));
    bytes32 structHash = keccak256(abi.encode(BALLOT_TYPEHASH, proposalId, support));
    bytes32 digest = keccak256(abi.encodePacked("\x19\x01", domainSeparator, structHash));
    address signatory = ecrecover(digest, v, r, s);
    require(signatory != address(0), "AGFIGovernor::castVoteBySig: invalid signature");
    return _castVote(signatory, proposalId, support);
}
```

MODEL OUTPUT:

```
function castVoteBySig(
    uint256 proposalId,
    bool support,
    uint8 v,
    bytes32 r,
    bytes32 s
) public {
    bytes32 domainSeparator = keccak256(
        abi.encode(
            DOMAIN_TYPEHASH,
            keccak256(bytes(name)),
            getChainId(),
            address(this)
        )
    );
    bytes32 structHash = keccak256(
        abi.encode(VOTE_TYPEHASH, proposalId, support)
    );
    bytes32 digest = keccak256(
        abi.encodePacked("\x19\x01", domainSeparator, structHash)
    );
    address signatory = ecrecover(digest, v, r, s);
    require(signatory!= address(0), "CastVote: invalid signature");
    require(signatory == castVoteSender, "CastVote: unauthorized");

    _castVote(proposalId, support);
}
```

4626. deposit(uint256)_v3_v0.8.3__4625 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```
function deposit(uint256 amountToStake) public {
    require(amountToStake >= minimumAmount && amountToStake <= maximumAmount, "Invalid amount");
    require(farmEnabled, "Farming not enabled yet");
    require(Token(LPtokenAddress).transferFrom(msg.sender, address(this), amountToStake), "Insufficient Token Allowance");
}
```

```

        updateAccount(msg.sender);

        depositedTokens[msg.sender] = depositedTokens[msg.sender].add(amountToStake);

        if (!holders.contains(msg.sender)) {
            holders.add(msg.sender);
            stakingTime[msg.sender] = block.timestamp;
        }
    }
}

```

MODEL OUTPUT:

```

function deposit(uint256 amount) public payable {
    require(amount > 0, "Amount must be greater than 0");
    require(msg.value >= amount, "Insufficient funds");
    require(amount <= MAX_DEPOSIT, "Deposit amount exceeds maximum");

    // Calculate the new total supply
    totalSupply += amount;

    // Add the deposit to the deposits array
    deposits.push(Deposit(msg.sender, amount, block.timestamp));

    // Emit the Deposit event
    emit Deposit(msg.sender, amount, block.timestamp);
}

```

4627. setProtocolFee(uint256)_v0.8.9__4626 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function setProtocolFee(uint256 _protocolFee) external;

```

MODEL OUTPUT:

```

function setProtocolFee(uint256 _protocolFee) external onlyOwner {
    require(_protocolFee <= 100, "Fee cannot be more than 100%");
    protocolFee = _protocolFee;
}

```

4628. deposit(uint256,uint256)_v5_v0.8.11__4627 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function deposit(uint dest_chain_id, uint amount) external {
    require(amount >= minimum_fee_amount[dest_chain_id], "Minimum amount");
    require(chainId != dest_chain_id, "Invalid Destination network");
    uint request_id = requests.length;
    uint fee_amount = amount * fee_percent[dest_chain_id] / 1e8;
    if(fee_amount < minimum_fee_amount[dest_chain_id]) fee_amount = minimum_fee_amount[dest_chain_id];
    bytes32 src_chain_data_hash = _get_data_hash(request_id, msg.sender, chainId, dest_chain_id, amount, fee_amount, block.timestamp);
    Request memory request = Request({
        src_chain_id: chainId,
        dest_chain_id: dest_chain_id,
        amount: amount,
        fee_amount: fee_amount,
        to: msg.sender,
        deposit_timestamp: block.timestamp,
        src_chain_data_hash: src_chain_data_hash,
        data_hash: 0,
        status: RequestStatus.Init,
        deposit_tx_hash: "",
        deposit_tx_link: "",
        release_tx_link: ""
    });
    requests.push(request);
    jaxre.transferFrom(msg.sender, address(this), amount);
    jaxre.burn(amount);
    emit Deposit(request_id, src_chain_data_hash, msg.sender, amount, fee_amount, uint64(chainId), uint64(dest_chain_id), uint128(block.timestamp));
}

```

MODEL OUTPUT:

```

function deposit(uint256 amount, uint256 tokenId) external {
    require(msg.sender == ownerOf[tokenId], "Not owner");
    require(amount > 0, "Amount must be greater than 0");
    require(totalSupply() + amount <= maxSupply, "Exceeds max supply");
    require(balanceOf[msg.sender] + amount <= maxPerWallet, "Exceeds max per wallet");

    _mint(msg.sender, amount);
}

```

4629. withdrawForeignERC20(address)_v0.8.1__4628 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```

function withdrawForeignERC20(address tokenContract) public onlyOwner {
    IERC20 token = IERC20(tokenContract);
    token.transfer(owner, token.balanceOf(address(this)));
}

```

```
}
```

MODEL OUTPUT:

```
function withdrawForeignERC20(address token) external onlyOwner {
    IERC20(token).transfer(msg.sender, IERC20(token).balanceOf(address(this)));
}
```

4630. excludeFromReward(address)_v4_v0.8.14__4629 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
constructor (address routerAddress) {
    IRouter _router = IRouter(routerAddress);
    address _pair = IFactory(_router.factory())
        .createPair(address(this), _router.WETH());

    router = _router;
    pair = _pair;

    excludeFromReward(pair);

    _rOwned[owner()] = _rTotal;
    _isExcludedFromFee[owner()] = true;
    _isExcludedFromFee[address(this)] = true;
    _isExcludedFromFee[treasuryAddress]=true;
    _isExcludedFromFee[burnAddress] = true;
    _isExcludedFromFee[megaPoolAddress] = true;
    _isExcludedFromFee[lpRecipient] = true;

    emit Transfer(address(0), owner(), _tTotal);
}
```

MODEL OUTPUT:

```
function excludeFromReward(address account) public onlyOwner {
    require(!_isExcludedFromReward[account], "Account is already excluded");
    for (uint256 i = 0; i < _excluded.length; i++) {
        if (_excluded[i] == account) {
            _excluded[i] = _excluded[_excluded.length - 1];
            _tOwned[account] = 0;
            _isExcludedFromReward[account] = false;
            _excluded.pop();
            break;
        }
    }
}
```

4631. setFees(uint256,uint256)_v90_v0.8.10__4630 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setFees(uint256 _buy, uint256 _sell) external onlyOwner {
    require(_sell <= 50,"Max sell tax 5%");
    require(_buy <= 50,"Max buy tax 5%");
    sellMultiplier = _sell;
    buyMultiplier = _buy;
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

4632. erc20Token()_v0.8.4__4631 [v0.8.4] bleu4=1.0 exact=1

EXPECTED:

```
function erc20Token() public view returns (address) {
    return _erc20Token;
}
```

MODEL OUTPUT:

```
function erc20Token() public view returns (address) {
    return _erc20Token;
}
```

4633. init(address,address,address)_v0.8.0__4632 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function init(address _stakingContractAddress, address _xRIOContractAddress, address _rioContractAddress) public onlyOwner {
    setStakingContractAddress(_stakingContractAddress);
    setxRIOToken(_xRIOContractAddress);
    setRIOToken(_rioContractAddress);
    flipLock();
}
```

MODEL OUTPUT:

```
function init(address _owner, address _token, address _reward) external onlyOwner {
    owner = _owner;
}
```

```

    token = IERC20(_token);
    reward = _reward;
}

```

4634. `getPriorVotes(address,uint256)_v2_v0.8.4__4633 [v0.8.4] bleu4=0.7876 exact=0`

EXPECTED:

```

function getPriorVotes(address account, uint256 blockNumber) external view returns (uint256) {
    uint256 currentBlockNumber = getBlockNumber();
    require(currentBlockNumber > blockNumber, "AMPT::getPriorVotes: not yet determined");

    uint256 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }

    // First check most recent balance
    if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
        return checkpoints[account][nCheckpoints - 1].votes;
    }

    // Next check implicit zero balance
    if (checkpoints[account][0].fromBlock > blockNumber) {
        return 0;
    }

    uint256 lower = 0;
    uint256 upper = nCheckpoints - 1;
    while (upper > lower) {
        uint256 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
        Checkpoint memory cp = checkpoints[account][center];
        if (cp.fromBlock == blockNumber) {
            return cp.votes;
        } else if (cp.fromBlock < blockNumber) {
            lower = center;
        } else {
            upper = center - 1;
        }
    }
    return checkpoints[account][lower].votes;
}

```

MODEL OUTPUT:

```

function getPriorVotes(address account, uint blockNumber) public view returns (uint96) {
    require(blockNumber < block.number, "BLOCK_NOT_EXIST");

    uint32 nCheckpoints = numCheckpoints[account];
    if (nCheckpoints == 0) {
        return 0;
    }

    // First check most recent balance
    if (checkpoints[account][nCheckpoints - 1].fromBlock <= blockNumber) {
        return checkpoints[account][nCheckpoints - 1].votes;
    }

    // Next check implicit zero balance
    if (checkpoints[account][0].fromBlock > blockNumber) {
        return 0;
    }

    uint32 lower = 0;
    uint32 upper = nCheckpoints - 1;
    while (upper > lower) {
        uint32 center = upper - (upper - lower) / 2; // ceil, avoiding overflow
        Checkpoint memory cp = checkpoints[account][center];
        if (cp.fromBlock == blockNumber) {
            return cp.votes;
        } else if (cp.fromBlock < blockNumber) {
            lower = center;
        } else {
            upper = center - 1;
        }
    }
    return checkpoints[account][lower].votes;
}

```

4635. `openTrading()_v194_v0.8.19__4634 [v0.8.19] bleu4=0.1335 exact=0`

EXPECTED:

```

function openTrading() external onlyOwner {
    require(!tradingActive && genesisBlock != 0);
    genesisBlock+=block.number+_block;
}

```

```

        tradingActive = true;

        emit TradingOpened();
    }

MODEL OUTPUT:
function openTrading() external onlyOwner {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 1000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 1000000000000000000 * 10**9; // 1%
}

```

4636. setProxyAddress(address)_v0.8.11__4635 [v0.8.11] bleu4=0.0 exact=0

```

EXPECTED:
function setProxyAddress(address _proxyAddress) public onlyOwner {
    proxyAddress_ = _proxyAddress;
}

MODEL OUTPUT:
function setProxyAddress(address _proxyAddress) external onlyOwner {
    proxyAddress = _proxyAddress;
}

```

4637. noLongerForSale(uint256)_v0.8.0__4636 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function noLongerForSale(uint256 tokenId) external _lock_ {
    Offer memory offer = nftOfferedForSale[tokenId];
    require(offer.isForSale, "nft not actually for sale");
    require(msg.sender == offer.seller, "Only the seller can operate");
    require(!offer.isBid, "The auction cannot be cancelled");

    ERC721Like(nftAsset).transferFrom(address(this), offer.seller, tokenId);
    delete nftOfferedForSale[tokenId];
    emit NoLongerForSale(tokenId);
}

MODEL OUTPUT:
function noLongerForSale(uint256 tokenId) public onlyOwner {
    require(isForSale[tokenId], "Token is not for sale");
    isForSale[tokenId] = false;
}

```

4638. updateDividendTrackerToken(address)_v0.8.4__4637 [v0.8.4] bleu4=0.0 exact=0

```

EXPECTED:
function updateDividendTrackerToken(address tokenAddress) external onlyOwner {
    _dividendToken = tokenAddress;
    dividendTracker.updatDividendTrackerToken(tokenAddress);
}

MODEL OUTPUT:
function updateDividendTrackerToken(address newAddress) external onlyOwner {
    require(newAddress != address(dividendTracker), "DividendTracker: The new dividend tracker is the same as the old one");
;

    require(newAddress != address(0), "DividendTracker: The new dividend tracker is the zero address");
    DividendTracker newDividendTracker = DividendTracker(payable(newAddress));
    newDividendTracker.excludeFromDividends(address(this));
    newDividendTracker.excludeFromDividends(owner());
    newDividendTracker.excludeFromDividends(address(newDividendTracker));
    newDividendTracker.excludeFromDividends(address(0x0));
    newDividendTracker.excludeFromDividends(address(0x01));
    newDividendTracker.excludeFromDividends(address(0x02));
    newDividendTracker.excludeFromDividends(address(0x03));
    newDividendTracker.excludeFromDividends(address(0x04));
    newDividendTracker.excludeFromDividends(address(0x05));
    newDividendTracker.excludeFromDividends(address(0x06));
    newDividendTracker.excludeFromDividends(address(0x07));
    newDividendTracker.excludeFromDividends(address(0x08));
    newDividendTracker.excludeFromDividends(address(0x09));
    newDividendTracker.excludeFromDividends(address(0x0a));
    newDividendTracker.excludeFromDividends(address(0x0b));
    newDividendTracker.excludeFromDividends(address(0x0d));
    newDividendTracker.excludeFromDividends(address(0x0e));
    newDividendTracker.excludeFromDividends(address(

```

4639. openTrading(uint256)_v0.8.4__4638 [v0.8.4] bleu4=0.0 exact=0

```

EXPECTED:
function openTrading(uint256 botBlocks) external onlyOwner() {
    _firstBlock = block.timestamp;
    _botBlocks = botBlocks;
    tradingOpen = true;
}

```

```
function openTrading(uint256 _buyFee) external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    require(_buyFee <= 10, "Buy fee cannot be more than 10%");
    tradingOpen = true;
    buyFee = _buyFee;
    swapEnabled = true;
    launchedAt = block.timestamp;
}
```

EXPECTED:

```
function isIncludedFromFee(address account) public view returns(bool) {
    return _isIncldFee[account];
}
```

```
function isIncludedFromFee(address account) public view returns (bool) {
    return _isIncludedFromFee[account];
}
```

EXPECTED:

MODEL OUTPUT:

```
function getDepositComission(address _depositAddress, uint256 _amount) public view returns (uint256) {
    return _amount * _depositComission[_depositAddress] / 1000;
}
```

EXPECTED:

```
function getEscrowBalance() external view returns (uint256) {
    return accumulatedFees;
}
```

```
function getEscrowBalance() public view returns (uint256) {
    return address(this).balance;
}
```

EXPECTED:

```
function claimTokensAllMemberships() public {
    uint256[] memory memberships = IGenArt(genArtMembershipAddress)
        .getTokensByOwner(msg.sender);
    for (uint256 i = 0; i < memberships.length; i++) {
        claimTokensMembership(memberships[i]);
    }
}
```

```
function claimTokensAllMemberships() public {
    require(hasRole(MEMBER, msg.sender), "You don't have permission to claim tokens");
    uint256 balance = token.balanceOf(address(this));
    token.transfer(msg.sender, balance);
}
```

EXPECTED:

```
function burn(address[] calldata from_) external onlyOwner {
    for (uint256 i = 0; i < from_.length; i++) {
        _burn(from_[i]);
    }
}
```

[illegible]

EXPECTED:

```

function increaseAllowance(address spender, uint256 addedValue) public returns (bool) {
    require(spender != address(0));
    allowed[msg.sender][spender] += addedValue;
    emit Approval(msg.sender, spender, allowed[msg.sender][spender]);
    return true;
}

```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

4646. swapTokens(uint256)_v0.8.15__4645 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function swapTokens(uint256 tokenAmount) external;
```

MODEL OUTPUT:

```
function swapTokens(uint256 amount) external {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= _balances[address(this)], "Amount exceeds balance");
    uint256 contractBalance = balanceOf(address(this));
    uint256 amountToSwap = amount.mul(100).div(contractBalance);
    uint256 amountToSwapForETH = amountToSwap.mul(10**18).div(10**18);
    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = address(WETH);
    swapTokensForEth(amountToSwapForETH, path);
    _balances[address(this)] = _balances[address(this)].sub(amount);
    _balances[address(WETH)] = _balances[address(WETH)].add(amount);
    emit Transfer(address(this), address(WETH), amount);
}
```

4647. isExcludedFromReflection(address)_v0.8.10__4646 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function isExcludedFromReflection(address ad) public view returns(bool) {
    return _isExcludedFromReflection[ad];
}
```

MODEL OUTPUT:

```
function isExcludedFromReflection(address account) public view returns (bool) {
    return _isExcludedFromReflection[account];
}
```

4648. setFreezer(address,bool)_v3_v0.8.0__4647 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setFreezer(address freezer, bool isActive) external onlyOwner {
    require(freezer != address(0), "Zero address");
    isFreezer[freezer] = isActive;
    emit SetFreezer(freezer, isActive);
}
```

MODEL OUTPUT:

```
function setFreezer(address account, bool status) external onlyOwner {
    _freeze[account] = status;
}
```

4649. redeemUnderlying(uint256)_v0.8.0__4648 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function redeemUnderlying(uint256 amount) external returns (uint256 redeemedCEth);
```

MODEL OUTPUT:

```
function redeemUnderlying(uint256 amount) external;
```

4650. setSwapFee(uint256)_v0.8.0__4649 [v0.8.0] bleu4=0.7825 exact=0

EXPECTED:

```
function setSwapFee(uint _swapFee) external onlyOwner {
    swapFee = _swapFee;
}
```

MODEL OUTPUT:

```
function setSwapFee(uint256 _swapFee) external onlyOwner {
    swapFee = _swapFee;
}
```

4651. getBalance()_v3_v0.8.4__4650 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getBalance() public returns (uint256) {
    m_Balance = V2Token.balanceOf(address(this));
    return m_Balance;
}
```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}
```

4652. setMintPrice(uint256)_v3_v0.8.13__4651 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setMintPrice(uint _newMint_Price) external onlyOwner {
    pb_price = _newMint_Price;
}
```

MODEL OUTPUT:

```
function setMintPrice(uint256 _mintPrice) external onlyOwner {
    mintPrice = _mintPrice;
}
```

4653. setFees(uint256,uint256,uint256)_v3_v0.8.19___4652 [v0.8.19] bleu4=0.1506 exact=0

EXPECTED:

```
function setFees(uint256 _liqFee, uint256 _markFee, uint256 _devFee) public onlyOwner() {
    require(_liqFee.add(_markFee).add(_devFee) <= 9, "tax too high");
    liquidityFee = _liqFee;
    marketingFee = _markFee;
    devFee = _devFee;
    totalFee = _liqFee.add(_markFee).add(_devFee);
    emit FeesUpdated(_liqFee, _markFee, _devFee);
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

4654. stake(uint256)_v7_v0.8.4___4653 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint256 amount) external {
    require(staking[msg.sender].amount == 0, "You have already staked");
    require(amount >= minimumStakeValue, "Minimum Stake is 10,000 BULL");

    uint256 tax = (amount * 5) / 100;
    uint256 finalamount = amount - tax;
    rewardpool = rewardpool + tax;
    totalAmountStaked = totalAmountStaked + finalamount;

    token.transferFrom(msg.sender,address(this),amount);
    staking[msg.sender] = Stake(msg.sender,finalamount, block.timestamp);
}
```

MODEL OUTPUT:

```
function stake(uint256 amount) public {
    require(amount > 0, "Amount must be greater than 0");
    require(amount <= maxStake, "Amount must be less than or equal to maxStake");
    require(stakingEnabled, "Staking is not enabled");
    require(stakingStarted, "Staking has not started yet");
    require(stakingEnded, "Staking has ended");

    uint256 totalStaked = totalStaked + amount;
    uint256 totalStakedPercent = totalStaked / totalSupply;
    uint256 stakedPercent = amount / totalSupply;

    if (totalStakedPercent >= stakingPercent) {
        _stake(msg.sender, amount);
    } else if (stakedPercent >= stakingPercent) {
        _stake(msg.sender, amount);
    }

    emit Staked(msg.sender, amount);
}
```

4655. checkPrize()_v0.8.4___4654 [v0.8.4] bleu4=0.7825 exact=0

EXPECTED:

```
function checkPrize() public view returns (uint256) {
    return currentPrize;
}
```

MODEL OUTPUT:

```
function checkPrize() public view returns (uint256) {
    return prize;
}
```

4656. uniswapV3FlashCallback(uint256,uint256,bytes)_v0.8.16___4655 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function uniswapV3FlashCallback(
    uint256 fee0,
    uint256 fee1,
    bytes calldata data
) external;
```

MODEL OUTPUT:

```
function uniswapV3FlashCallback(
```


4657. withdraw()_v3_v0.8.0___4656 [v0.8.0] bleu4=0.0003 exact=0

```
function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

[illegible]


```
        return totalNFTStrengthPerTeam[teamId];
    }
}
```

MODEL OUTPUT:

```
function getTeamTotalNFTStrength(uint256 teamId) public view returns (uint256) {
    return _teamNFTStrength[teamId];
}
```

4662. burn(address,uint256)_v23_v0.8.0___4661 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burn(address _from, uint256 _amount) external {
}
```

MODEL OUTPUT:

```
function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}
```

4663. removeBuyout(address)_v0.8.0___4662 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function removeBuyout(address v1TokenAddr) external onlyOwner {
    uint256 amount = ethAvaliable[v1TokenAddr];
    require(amount > 0, "Cannot remove 0");
    ethAvaliable[v1TokenAddr] = 0;
    payable(msg.sender).transfer(amount);
    emit BuyoutComplete(v1TokenAddr);
}
```

MODEL OUTPUT:

```
function removeBuyout(address _buyoutAddress) external onlyOwner {
    buyout[_buyoutAddress] = false;
}
```

4664. openTrading()_v452_v0.8.0___4663 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _throneNum = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

4665. getSourceDomains()_v0.8.0___4664 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getSourceDomains() public pure returns (string[124] memory) {
    string[124] memory sourceDomains = [
        "heaven",
        "magic",
        "meaning",
        "forever",
        "sweet",
        "work",
        "insanity",
        "everything",
        "progress",
        "weakness",
        "strength",
        "money",
        "left",
        "armor",
        "bitter",
        "temporary",
        "death",
        "power",
        "war",
        "tomorrow",
        "island",
        "enigma",
        "end",
        "angel",
        "arm",
        "answer",
        "riddle",
        "puzzle",
        "window",
        "mountain",
        "vehicle",
        "door",
        "table",
    ]
}
```

"bed",
"gift",
"knife",
"hammer",
"candle",
"bird",
"raft",
"book",
"storm",
"rock",
"tornado",
"wall",
"hurdle",
"vase",
"gate",
"house",
"box",
"prison",
"road",
"hill",
"diamond",
"hell",
"horizon",
"city",
"friend",
"weapon",
"forest",
"bridge",
"dream",
"myth",
"day",
"compass",
"map",
"man",
"woman",
"child",
"lake",
"universe",
"dimension",
"portal",
"journey",
"beginning",
"law",
"well",
"reason",
"destination",
"place",
"straight line",
"windy road",
"solution",
"moment",
"hug",
"sunny day",
"curse",
"burden",
"ceiling",
"light",
"key",
"garden",
"river",
"branch",
"flower",
"foundation",
"forgiveness",
"battlefield",
"job",
"structure",
"buckle",
"seed",
"screen",
"shop",
"bank",
"watch",
"wave",
"date",
"ring",
"letter",
"reality",
"mystery",
"means",

```

    "kiss",
    "wish",
    "life",
    "tree",
    "ocean",
    "cure",
    "virus",
    "valley",
    "zoo",
    "godliness",
    "cancer",
    "sickness"
];

    return sourceDomains;
}

```

MODEL OUTPUT:

```

function getSourceDomains() public view returns (string[] memory) {
    return sourceDomains;
}

```

4666. firstBlock()_v0.8.17___4665 [v0.8.17] bleu4=1.0 exact=1

EXPECTED:

```

function firstBlock() public view returns (uint256) {
    return _firstBlock;
}

```

MODEL OUTPUT:

```

function firstBlock() public view returns (uint256) {
    return _firstBlock;
}

```

4667. stake(uint256,uint8)_v0.8.0___4666 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function stake(uint amount, uint8 plan) external {
    require(!maxUserReached, "max user limit");
    require(amount > 0, "no amount");
    require(totalStakedAmount + amount <= maxStakeAmount, "exceeds max stake amount");
}

```

```

uint utime;
uint reward;
uint lockPeriod;

```

```

if(plan == 1) {
    utime = block.timestamp + PLAN_ONE_LOCK_PERIOD;
    lockPeriod = PLAN_ONE_LOCK_PERIOD;
    reward = (amount * PLAN_ONE_REWARD_PERCENTAGE) / 1000;
} else if(plan == 2) {
    utime = block.timestamp + PLAN_TWO_LOCK_PERIOD;
    lockPeriod = PLAN_TWO_LOCK_PERIOD;
    reward = (amount * PLAN_TWO_REWARD_PERCENTAGE) / 1000;
} else if(plan == 3) {
    utime = block.timestamp + PLAN_THREE_LOCK_PERIOD;
    lockPeriod = PLAN_THREE_LOCK_PERIOD;
    reward = (amount * PLAN_THREE_REWARD_PERCENTAGE) / 1000;
} else if(plan == 4) {
    utime = block.timestamp + PLAN_FOUR_LOCK_PERIOD;
    lockPeriod = PLAN_FOUR_LOCK_PERIOD;
    reward = (amount * PLAN_FOUR_REWARD_PERCENTAGE) / 1000;
} else {
    revert("invalid plan");
}

```

```

require(reward > 0, "Amount too small for a reward");
require(totalRewardAmount + reward <= maxRewardAmount, "exceeds max reward");

```

```

if(_userStakeIds[msg.sender].length == 0) {
    totalUsers += 1;
    if(totalUsers == 100000) {
        maxUserReached = true;
    }
}

```

```

totalStakedAmount += amount;
totalRewardAmount += reward;

```

```

uint sId = totalStakes++;

```

```

stakeInfo[sId] = Stake(amount, utime, block.timestamp, reward, reward, lockPeriod, msg.sender, plan, true);
_userStakeIds[msg.sender].push(sId);

```

```

    token.transferFrom(msg.sender, address(this), amount);
}

```

MODEL OUTPUT:

```

function stake(uint256 amount, uint8 decimals) external {
    require(amount > 0, "Amount must be greater than 0");
    require(decimals > 0, "Decimals must be greater than 0");
    require(amount <= maxStake, "Amount must be less than or equal to maxStake");
    require(decimals <= maxDecimals, "Decimals must be less than or equal to maxDecimals");

    uint256 tokenId = getTokenId(msg.sender, decimals);
    require(tokenId > 0, "Token ID not found");

    uint256 stakeAmount = amount * 10**decimals;
    require(stakeAmount <= maxStake, "Amount must be less than or equal to maxStake");

    uint256 totalStaked = stakes[tokenId][msg.sender];
    require(totalStaked == 0, "Already staked");

    stakes[tokenId][msg.sender] = stakeAmount;
    totalStaked = stakes[tokenId][msg.sender];

    emit Stake(msg.sender, tokenId, stakeAmount, totalStaked);
}

```

4668. setMaxWalletSize(uint256,uint256)_v0.8.0___4667 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxWalletSize(uint256 percent, uint256 divisor) external onlyOwner() {
    require(divisor <= 1000); // Cannot set lower than 0.1%
    _maxWalletSize = _total.mul(percent).div(
        divisor // Division by divisor, makes it mutable.
    );
    maxWalletSizeUI = startingSupply.mul(percent).div(divisor);
}

```

MODEL OUTPUT:

```

function setMaxWalletSize(uint256 _maxWalletSize, uint256 _maxTransactionAmount) external onlyOwner {
    require(_maxWalletSize >= _totalSupply.div(1000), "Max wallet size cannot be lower than 0.1% total supply.");
    require(_maxTransactionAmount >= _totalSupply.div(1000), "Max transaction amount cannot be lower than 0.1% total supply.");

    maxWalletSize = _maxWalletSize;
    maxTransactionAmount = _maxTransactionAmount;
}

```

4669. getFundsDestination()_v0.8.9___4668 [v0.8.9] bleu4=0.7109 exact=0

EXPECTED:

```

function getFundsDestination() public view onlyOwner returns (address) {
    return fundsDestination;
}

```

MODEL OUTPUT:

```

function getFundsDestination() public view returns (address) {
    return fundsDestination;
}

```

4670. addToBlacklist(address)_v0.8.7___4669 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function addToBlacklist(address user) external onlyOwner {
    blacklist[user] = true;
}

```

MODEL OUTPUT:

```

function addToBlacklist(address _address) public onlyOwner {
    blacklist[_address] = true;
}

```

4671. setMaxTxnAmount(uint256)_v378_v0.8.4___4670 [v0.8.4] bleu4=0.3286 exact=0

EXPECTED:

```

function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    require(maxTxAmount >= 5000000000000);
    _maxTxAmount = maxTxAmount * 10**9 ;
}

```

MODEL OUTPUT:

```

function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}

```

4672. SetupLiquidityTokenAddress(address)_v0.8.13___4671 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function SetupLiquidityTokenAddress(address liquidityTokenAddress) public onlyOwner{
    _liquidityTokenAddress=liquidityTokenAddress;
}

```

MODEL OUTPUT:

```
function SetupLiquidityTokenAddress(address _liquidityTokenAddress) external onlyOwner {
    liquidityTokenAddress = _liquidityTokenAddress;
}
```

4673. setCommunityAddress(address)_v0.8.15__4672 [v0.8.15] bleu4=0.1804 exact=0

EXPECTED:

```
function setCommunityAddress(address _communityAddress) external onlyOwner {
    require(
        _communityAddress != address(0),
        "_communityAddress address cannot be 0"
    );
    communityAddress = payable(_communityAddress);
    emit UpdatedCommunityAddress(_communityAddress);
}
```

MODEL OUTPUT:

```
function setCommunityAddress(address _communityAddress) external onlyOwner {
    communityAddress = _communityAddress;
}
```

4674. setIsTimelockExempt(address,bool)_v0.8.13__4673 [v0.8.13] bleu4=0.7349 exact=0

EXPECTED:

```
function setIsTimelockExempt(address holder, bool exempt) external onlyOwner {
    _isTimelockExempt[holder] = exempt;
}
```

MODEL OUTPUT:

```
function setIsTimelockExempt(address holder, bool exempt) external onlyOwner {
    isTimelockExempt[holder] = exempt;
}
```

4675. getPhase(uint256)_v0.8.0__4674 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getPhase(uint256 phaseId)
    external
    view
    returns (PhaseInfo memory)
{
    return _phases[phaseId];
}
```

MODEL OUTPUT:

```
function getPhase(uint256 _tokenId) public view returns (uint256) {
    return _tokenIdToPhase[_tokenId];
}
```

4676. isReward(address)_v0.8.11__4675 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function isReward(address token) external view returns (bool) {}
```

MODEL OUTPUT:

```
function isReward(address _user) public view returns (bool) {
    return _rewards[_user];
}
```

4677. getSignersCount()_v0.8.4__4676 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getSignersCount () public view returns (uint) {
    return _signers.length;
}
```

MODEL OUTPUT:

```
function getSignersCount() public view returns (uint256) {
    return signers.length;
}
```

4678. setSwapTokensAtAmount(uint256)_v5_v0.8.17__4677 [v0.8.17] bleu4=0.1711 exact=0

EXPECTED:

```
function setSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    require(newAmount != swapTokensAtAmount, 'Swap tokens at amount is already set to that amount');
    swapTokensAtAmount = newAmount;
    emit SwapTokensAtAmountUpdated(newAmount);
}
```

MODEL OUTPUT:

```
function setSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    require(newAmount >= totalSupply() * 1 / 100000, "Swap amount cannot be lower than 0.001% total supply.");
    require(newAmount <= totalSupply() * 1 / 1000, "Swap amount cannot be higher than 0.1% total supply.");
    swapTokensAtAmount = newAmount;
}
```

4679. executeTransfer(uint32)_v0.8.4__4678 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function redeem(uint256 amount) external {
    require(msg.sender == owner, "Only the owner can redeem");
```



```
require(amount <= balanceOf(address(this)), "Insufficient balance");
_burn(address(this), amount);
emit Redeemed(amount);
}
```

4681. feed()_v2_v0.8.0___4680 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function feed() public {
  require(getAlive(), "no longer with us");
  require(getBoredom() < 80, "im too tired to eat");

  lastFeedBlock = block.number;

  hunger = 0;
  boredom += 10;
  uncleanliness += 3;

  addLove(msg.sender, 1);
}
```

MODEL OUTPUT:

```
function feed() external {
  require(msg.sender == owner, "Only the owner can feed");
  uint256 amount = address(this).balance;
  payable(owner).transfer(amount);
}
```

4682. excludeFromFees(address,bool)_v12_v0.8.0___4681 [v0.8.0] bleu4=0.4496 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) external onlyOwner {
  require(msg.sender == owner(), "error");
  _isExcludedFromFees[account] = excluded;
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
  _isExcludedFromFees[account] = excluded;
}
```

4683. addToBlackList(address)_v0.8.4___4682 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function addToBlackList(address user) external {
  require(_owner == msg.sender);
  _blackListed[user] = true;
}
```

MODEL OUTPUT:

```
function addToBlackList(address _address) external onlyOwner {
  require(!_blackListed[_address], "Address is already blacklisted");
  blackListed[_address] = true;
}
```

4684. enableTrading()_v1193_v0.8.10___4683 [v0.8.10] bleu4=0.4456 exact=0

EXPECTED:

```
function enableTrading() external onlyOwner {
  require(!trading, "already enabled");
  trading = true;
  swapEnabled = true;
  surpriseEnable = true;
  launchedAt = block.timestamp;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
  require(!tradingActive, "Trading is already enabled");
  tradingActive = true;
  swapEnabled = true;
  launchedAt = block.number;
}
```

4685. setMaxBuyTransactionAmount(uint256)_v2_v0.8.17___4684 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxBuyTransactionAmount(uint256 _maxTxAmount) public onlyOwner {
  maxBuyTransactionAmount = _maxTxAmount;
  require(maxBuyTransactionAmount >= totalSupply().div(500), "value too low");
}
```

MODEL OUTPUT:

```
function setMaxBuyTransactionAmount(uint256 maxBuyTransactionAmount) external onlyOwner {
  _maxBuyTransactionAmount = maxBuyTransactionAmount;
}
```

4686. setFee(uint256,uint256,uint256,uint256)_v24_v0.8.10___4685 [v0.8.10] bleu4=0.8778 exact=0

EXPECTED:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 99, "Buy tax must be between 0% and 99%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 99, "Sell tax must be between 0% and 99%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}

```

MODEL OUTPUT:

```

function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be between 0% and 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be between 0% and 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}

```

4687. `minter(address)_v0.8.4__4686 [v0.8.4] bleu4=0.0 exact=0`

EXPECTED:

```

function minter(address account) external view returns(bool) {
    return minters[account];
}

```

MODEL OUTPUT:

```

function minter(address _minter) external onlyOwner {
    minter_ = _minter;
}

```

4688. `logicAddress()_v0.8.11__4687 [v0.8.11] bleu4=0.0 exact=0`

EXPECTED:

```

function logicAddress()
    public
    view
    virtual
    returns (address _logicAddress)
{
    assembly {
        _logicAddress := sload(LOGIC_SLOT)
    }
}

```

MODEL OUTPUT:

```

function logicAddress() external view returns (address) {
    return _logicAddress;
}

```

4689. `percent(uint256,uint256)_v0.8.14__4688 [v0.8.14] bleu4=0.0 exact=0`

EXPECTED:

```

function percent(uint256 amount, uint256 fraction) public virtual pure returns(uint256) {
    return ((amount * fraction) / 10000);
}

```

MODEL OUTPUT:

```

function percent(uint256 a, uint256 b) public pure returns (uint256) {
    return (a * 100) / b;
}

```

4690. `withdraw(uint256)_v39_v0.8.0__4689 [v0.8.0] bleu4=0.0 exact=0`

EXPECTED:

```

function withdraw (
    uint256 amount
)
    public
    onlyAdmins
    returns (bool)
{
    if (amount > 0) {
        require (
            token.transfer(msg.sender, amount),
            "Error: transfer failed"
        );
    }
}
return true;

```

```
}  
MODEL OUTPUT:  
function withdraw(uint256 amount) public {  
    require(msg.sender == owner, "Only owner can withdraw");  
    require(amount <= balanceOf(address(this)), "Insufficient balance");  
    _withdraw(amount);  
}
```

4691. moonbagEnabled()_v0.8.4___4690 [v0.8.4] bleu4=0.7071 exact=0

EXPECTED:
function moonbagEnabled() public view returns (bool) {
 return _moonbagEnabled;
}

MODEL OUTPUT:
function moonbagEnabled() external view returns (bool) {
 return _moonbagEnabled;
}

4692. setBuyFee(uint256,uint256,uint256,uint256)_v0.8.0___4691 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function setBuyFee(
 uint _newLp,
 uint _newMarketing,
 uint _newTeam,
 uint _newBurn
) public onlyOwner {

 buyLiquidityFee = _newLp;
 buyMarketingFee = _newMarketing;
 buyTeamFee = _newTeam;
 buyBurnFee = _newBurn;
}

MODEL OUTPUT:
function setBuyFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _burnFee) external onlyOwner {
 liquidityFee = _liquidityFee;
 marketingFee = _marketingFee;
 devFee = _devFee;
 burnFee = _burnFee;
 totalFee = liquidityFee + marketingFee + devFee + burnFee;
 require(totalFee <= 10, "Must keep fees at 10% or less");
}

4693. decreaseLiquidity((uint256,uint128,uint256,uint256,uint256)_v0.8.17___4692 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:
function decreaseLiquidity(INonfungiblePositionManager.DecreaseLiquidityParams calldata params) external view onlySelf {}

MODEL OUTPUT:
function decreaseLiquidity(uint256 amount0, uint256 amount1, uint160 minAmount0, uint160 minAmount1, uint160 maxAmount0, uint160 maxAmount1) external payable {
 require(amount0 > 0 || amount1 > 0, "ERC20: decreaseLiquidity: amount must be greater than zero");
 require(amount0 <= maxAmount0 && amount1 <= maxAmount1, "ERC20: decreaseLiquidity: amount exceeds max");
 require(amount0 <= maxAmount0 - minAmount0 && amount1 <= maxAmount1 - minAmount1, "ERC20: decreaseLiquidity: amount exceeds max");
 require(msg.value >= getLiquidityForSell(amount0, amount1), "ERC20: decreaseLiquidity: insufficient value sent");
 _beforeTokenTransfer(address(0), msg.sender, amount0, amount1);

 _balances[address(0)] += amount0;
 _balances[msg.sender] -= amount0;

 _balances[address(0)] += amount1;
 _balances[msg.sender] -= amount1;

 emit Transfer(address(0), msg.sender, amount0, amount1);
}

4694. setFees(uint256,uint256,uint256,uint256,uint256)_v0.8.12___4693 [v0.8.12] bleu4=0.2889 exact=0

EXPECTED:
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _teamFee, uint256 _devFee, uint256 _presidentFee) external onlyOwner {
 liquidityFee = _liquidityFee;
 marketingFee = _marketingFee;
 teamFee = _teamFee;
 devFee = _devFee;
 presidentFee = _presidentFee;
 totalFee = _liquidityFee + _marketingFee + _teamFee + _devFee + _presidentFee;

 require(totalFee.mul(buyMultiplier).div(100) < 25, "Tax cannot be more than 24%");
 require(totalFee.mul(sellMultiplier).div(100) < 25, "Tax cannot be more than 24%");
}

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    require(totalFee <= feeDenominator/2, "Fees cannot be more than 50%");
}
```

4695. resultReserveAuction(uint256)_v0.8.4___4694 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function resultReserveAuction(uint256 _id) external;
```

MODEL OUTPUT:

```
function resultReserveAuction(uint256 _auctionId) public view returns (uint256) {
    Auction storage auction = auctions[_auctionId];
    return auction.result;
}
```

4696. updateMaxWalletAmount(uint256)_v62_v0.8.10___4695 [v0.8.10] bleu4=0.8096 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 10000000000000000) / 1e18,
        "Cannot set maxWallet lower than 0.5%"
    );
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4697. burnFrom(address,uint256)_v98_v0.8.0___4696 [v0.8.0] bleu4=0.9157 exact=0

EXPECTED:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20Burnable: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
```

4698. withdraw()_v39_v0.8.9___4697 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public onlyOwner nonReentrant {
    // This will pay HashLips Lab Team 5% of the initial sale.
    // By leaving the following lines as they are you will contribute to the
    // development of tools like this and many others.
    // =====
    // (bool hs, ) = payable(0x146FB9c3b2C13BA88c6945A759EbFa95127486F4).call{value: address(this).balance * 5 / 100}('');
    // require(hs);
    // =====

    // This will transfer the remaining contract balance to the owner.
    // Do not remove this otherwise you will not be able to withdraw the funds.
    // =====
    (bool os, ) = payable(owner()).call{value: address(this).balance}('');
    require(os);
    // =====
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
```

```
uint256 balance = address(this).balance;
payable(msg.sender).transfer(balance);
}
```

4699. pay(string)_v2_v0.8.4__4698 [v0.8.4] bleu4=0.1158 exact=0

EXPECTED:

```
function pay(string memory serviceName) public payable {
    require(msg.value >= _prices[_toBytes32(serviceName)], "Error: Incorrect Price");
    (bool success , ) = payable(feerRecevier).call{value: msg.value}("");
    require(success,"Transaction Failed!!");
    emit Created(serviceName, _msgSender());
}
```

MODEL OUTPUT:

```
function pay(string calldata _data) public payable {
    require(msg.value > 0, "No ETH sent");
    require(msg.sender == owner, "Only owner can pay");
    emit Payment(_data, msg.sender, msg.value);
}
```

4700. numAvailableToMint(address)_v0.8.0__4699 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function numAvailableToMint(address addr) external view returns (uint8) {
    return _allowList[addr];
}
```

MODEL OUTPUT:

```
function numAvailableToMint(address _address) public view returns (uint256) {
    return _totalSupply - _balances[_address];
}
```

4701. getTxConfirmations(uint32,uint256)_v0.8.4__4700 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getTxConfirmations(uint32 accountNumber, uint txID) public view accountExists(accountNumber) returns (address[] memory) {
    return confirmedBy[accountNumber][txID];
}
```

MODEL OUTPUT:

```
function getTxConfirmations(uint32 blockNumber, uint256 txIndex) public view returns (uint32) {
    TransactionInfo memory tx = transactions[blockNumber][txIndex];
    uint32 blockConfirmations = 0;
    while (tx.confirmations == 0) {
        blockConfirmations++;
        blockNumber = blockNumber + 1;
        tx = transactions[blockNumber][txIndex];
    }
    return blockConfirmations;
}
```

4702. owners()_v2_v0.8.0__4701 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function owners() external view returns(address[] memory ) {
    address[] memory result = new address[](SIZE * SIZE);
    for (uint8 row = 0; row < SIZE; row++) {
        for (uint8 col = 0; col < SIZE; col++) {
            result[row * SIZE + col] = etheria.getOwner(col, row);
        }
    }
    return result;
}
```

MODEL OUTPUT:

```
function owners() public view returns (address[] memory) {
    return _owners;
}
```

4703. isMinter()_v0.8.0__4702 [v0.8.0] bleu4=0.5246 exact=0

EXPECTED:

```
function isMinter() public view returns (bool) {
    return isOwner() || membership[_msgSender()] == 11;
}
```

MODEL OUTPUT:

```
function isMinter() public view returns (bool) {
    return _minters[msg.sender];
}
```

4704. setBuyFee(uint256,uint256,uint256)_v2_v0.8.19__4703 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setBuyFee(uint256 setLiquidityFee, uint256 setMarketingFee, uint256 setDevelopmentFee) public onlyOwner {
    require(setLiquidityFee + setMarketingFee + setDevelopmentFee <= 25, "Total buy fee cannot be set higher than 25%.");
    buyFee.liquidity = setLiquidityFee;
    buyFee.marketing = setMarketingFee;
}
```

```
        buyFee.development = setDevelopmentFee;
    }
MODEL OUTPUT:
function setBuyFee(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = liquidityFee + marketingFee + devFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

4705. updateMaxWalletAmount(uint256)_v37_v0.8.9___4704 [v0.8.9] bleu4=0.456 exact=0

EXPECTED:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 25 / 1000)/1e18, "Cannot set maxWallet lower than 2.5%");
    maxWallet = newNum * (10**18);
}
```

MODEL OUTPUT:

```
function updateMaxWalletAmount(uint256 newNum) external onlyOwner {
    require(
        newNum >= ((totalSupply() * 1) / 1000) / 1e18,
        "Cannot set maxWallet lower than 0.1%"
    );
    maxWallet = newNum * (10**18);
}
```

4706. getCirculatingSupply()_v22_v0.8.11___4705 [v0.8.11] bleu4=0.6316 exact=0

EXPECTED:

```
function getCirculatingSupply() public view returns (uint256) {
    return _totalSupply.sub(balanceOf(deadAddress)).sub(balanceOf(zeroAddress));
}
```

MODEL OUTPUT:

```
function getCirculatingSupply() public view returns (uint256) {
    return _totalSupply - balanceOf(DEAD1);
}
```

4707. updatePrice(uint256)_v4_v0.8.0___4706 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updatePrice(uint256 _price) public onlyOwner{
    clonePrice=_price;
}
```

MODEL OUTPUT:

```
function updatePrice(uint256 _newPrice) external onlyOwner {
    price = _newPrice;
}
```

4708. setNewRegistrationPrice(uint256)_v2_v0.8.0___4707 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setNewRegistrationPrice(uint256 _newFee) public onlyOwner{
    registrationFee = _newFee;
}
```

MODEL OUTPUT:

```
function setNewRegistrationPrice(uint256 newPrice) external onlyOwner {
    registrationPrice = newPrice;
}
```

4709. removeWhitelist(address)_v0.8.17___4708 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function removeWhitelist(address who) public {
    require(msg.sender==owner);
    whitelist[who]=false;
    delete whitelistFee[who];
}
```

MODEL OUTPUT:

```
function removeWhitelist(address _address) external onlyOwner {
    whitelist[_address] = false;
}
```

4710. updateBuyFees(uint256,uint256,uint256)_v34_v0.8.17___4709 [v0.8.17] bleu4=0.5757 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _DEVFee_) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDEVFee = _DEVFee_;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDEVFee;
    require(buyTotalFees <= 60, "Must keep fees at 40% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4711. treasuryWithdraw()_v0.8.0___4710 [v0.8.0] bleu4=0.1379 exact=0

EXPECTED:

```
function treasuryWithdraw() external
{
    require(treasuryExecuted == false, "Treasury share already withdrawn.");
    treasuryExecuted = true;
    DINO.transfer(treasury, treasuryAllocation);
    DINO.transfer(liquidity, liquidityAllocation);
}
```

MODEL OUTPUT:

```
function treasuryWithdraw() external {
    require(msg.sender == treasury, "Not the treasury");
    (bool success, ) = treasury.call{value: address(this).balance}("");
    require(success, "Transfer failed");
}
```

4712. mint(uint256)_v27_v0.8.4___4711 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function mint (uint256 amount) external payable nonReentrant {
    require (saleActivity == true , "Not minting yet");
    require (amount + balanceOf(msg.sender) <= maxTokensPerWallet , "Max 20 per wallet");
    require (totalSupply() + amount <= NftsToMint , "Sold out");
    // 1 guaranteed free per wallet
    uint256 pricedAmount = freeMintsAvailable > 0 && mintsPerWallet[_msgSender()] == 0
        ? amount - 1
        : amount;

    if (pricedAmount < amount) {
        freeMintsAvailable = freeMintsAvailable - 1;
    }

    require(mintPrice * pricedAmount <= msg.value, "Not enough ETH sent for selected amount");

    uint256 refund = chanceFreeMintsAvailable > 0 && pricedAmount > 0 && isFreeMint()
        ? pricedAmount * mintPrice
        : 0;

    if (refund > 0) {
        chanceFreeMintsAvailable = chanceFreeMintsAvailable - pricedAmount;
    }

    // sends needed ETH back to minter
    payable(_msgSender()).transfer(refund);

    mintsPerWallet[_msgSender()] = mintsPerWallet[_msgSender()] + amount;

    _safeMint(_msgSender(), amount);
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "Excess");
    require(amount * price <= msg.value, "Value");

    _safeMint(msg.sender, amount);
}
```

4713. includeInMaxWallet(address)_v15_v0.8.7___4712 [v0.8.7] bleu4=0.3021 exact=0

EXPECTED:

```
function includeInMaxWallet(address account) external authorized() {
    _isMaxWalletExclude[account] = false;
}
```

MODEL OUTPUT:

```
function includeInMaxWallet(address account) external onlyOwner {
    _maxWalletAmount[account] = true;
}
```

4714. removeManager(address)_v2_v0.8.9___4713 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function removeManager(address _addr) external {
```

```
        require(managers[msg.sender] == true, 'Error, You are not allowed!');
        managers[_addr] = false;
    }
}

MODEL OUTPUT:
function removeManager(address _manager) public onlyOwner {
    require(_manager != address(0), "Invalid address");
    require(managers[_manager] == true, "Manager not set");
    managers[_manager] = false;
}
```

4715. setTrading(bool)_v80_v0.8.14__4714 [v0.8.14] bleu4=0.8176 exact=0

EXPECTED:

```
function setTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
    _developmentAddress;
}
```

MODEL OUTPUT:

```
function setTrading(bool _tradingOpen) public onlyOwner {
    tradingOpen = _tradingOpen;
}
```

4716. mint(address,uint256)_v9_v0.8.4__4715 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function mint(address user, uint256 amount) external;
```

MODEL OUTPUT:

```
function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}
```

4717. doStuff()_v0.8.7__4716 [v0.8.7] bleu4=0.0587 exact=0

EXPECTED:

```
function doStuff() public {
    uint256 newVar = stakedToken + 1; // SLOAD
    bool otherVar = someBool; // SLOAD
    // ^^ memory variables
}
```

MODEL OUTPUT:

```
function doStuff() public {
    // do stuff
}
```

4718. updateSellFees(uint256,uint256,uint256)_v24_v0.8.10__4717 [v0.8.10] bleu4=0.8624 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 11, "Must keep fees at 30% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4719. refund(uint256)_v2_v0.8.4__4718 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function refund(uint amountRequested) public onlyOwner {

    //require(newOwner != address(0));
    //require(newOwner != owner);

    RegisterShareholder(owner);

    //regulationStatus[owner] = true;

    //emit ShareholderRegistered(owner);
}
```



```

require(amountRequested > 0 && amountRequested <= balance[msg.sender]);

balance[msg.sender] -= amountRequested;

payable(msg.sender).transfer(amountRequested); // contract transfers ether to msg.sender's address

}

```

MODEL OUTPUT:

```

function refund(uint256 _tokenId) public {
    require(msg.sender == ownerOf[_tokenId], "You are not the owner");
    require(isApprovedForAll[msg.sender][address(this)], "You are not approved");
    require(block.timestamp > getTimeStamp(tokenURI[_tokenId]), "Token is not minted yet");
    require(getApproved[_tokenId] == msg.sender, "You are not approved");
    require(getApproved[_tokenId] != address(0), "No approved address");
    require(getApproved[_tokenId] != address(this), "You are not approved");
    require(getApproved[_tokenId] != address(0x0), "No approved address");
    require(getApproved[_tokenId] != address(0x0000000000000000000000000000000000000000000000000000000000000000), "No approved address");
    require(getApproved[_tokenId] != address(0x0000000000000000000000000000000000000000000000000000000000000001), "No approved address");
    require(getApproved[_tokenId] != address(0x0000000000000000000000000000000000000000000000000000000000000002), "No approved address");
    require(getApproved[_tokenId] != address(0x0000000000000000000000000000000000000000000000000000000000000003), "No approved address");
    require(getApproved[_tokenId] != address(0x0000000000000000000000000000000000000000000000000000000000000004), "No approved address");
    require(

```

4720. updateBuyFees(uint256,uint256,uint256)_v461_v0.8.10___4719 [v0.8.10] bleu4=0.2067 exact=0

EXPECTED:

```

function updateBuyFees(
    uint256 _MRK,
    uint256 _LP,
    uint256 _DEV
) external onlyOwner {
    buyMRK = _MRK;
    buyLP = _LP;
    buyDEV = _DEV;
    buyTotalFees = buyMRK + buyLP + buyDEV;
    require(buyTotalFees <= 5, "Must keep fees at 5% or less");
}

```

MODEL OUTPUT:

```

function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

4721. includeInFee(address)_v14_v0.8.4___4720 [v0.8.4] bleu4=1.0 exact=1

EXPECTED:

```

function includeInFee(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}

```

MODEL OUTPUT:

```

function includeInFee(address account) external onlyOwner {
    _isExcludedFromFee[account] = false;
}

```

4722. setAutomatedMarketMakerPair(address,bool)_v11_v0.8.12___4721 [v0.8.12] bleu4=0.5208 exact=0

EXPECTED:

```

function setAutomatedMarketMakerPair(address pair, bool value) external onlyOwner {
    require(pair != lpPair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
    emit SetAutomatedMarketMakerPair(pair, value);
}

```

MODEL OUTPUT:

```

function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");

    _setAutomatedMarketMakerPair(pair, value);
}

```

4723. setParameters(uint256,uint256,uint256)_v255_v0.8.16___4722 [v0.8.16] bleu4=0.0483 exact=0

EXPECTED:

```
function setParameters(uint256 _buy, uint256 _trans, uint256 _wallet) external onlyOwner {
    uint256 newTx = (totalSupply() * _buy) / 1000000;
    uint256 newTransfer = (totalSupply() * _trans) / 1000000;
    uint256 newWallet = (totalSupply() * _wallet) / 1000000;
    _maxTxAmountPercent = _buy;
    _maxTransferPercent = _trans;
    _maxWalletPercent = _wallet;
    uint256 limit = totalSupply().mul(5).div(1000000);
    require(newTx >= limit && newTransfer >= limit && newWallet >= limit, "Max TXs and Max Wallet cannot be less than .5%");
};
}
MODEL OUTPUT:
function setParameters(uint256 _buy, uint256 _trans, uint256 _wallet) external onlyOwner {
    buyFee = _buy;
    transFee = _trans;
    walletFee = _wallet;
}
```

4724. updateSender(address)_v0.8.0__4723 [v0.8.0] bleu4=0.0 exact=0

```
EXPECTED:
function updateSender(address newSender) external onlyOwner {
    require(newSender != address(0), "Zero-Address");
    sender = newSender;
}
MODEL OUTPUT:
function updateSender(address _sender) external onlyOwner {
    sender = _sender;
}
```

4725. withdrawContributions()_v0.8.16__4724 [v0.8.16] bleu4=0.0 exact=0

```
EXPECTED:
function withdrawContributions() external onlyOwner{
    USD.transfer(CEO, USD.balanceOf(address(this)));
    USDT.transfer(CEO, USDT.balanceOf(address(this)));
}
MODEL OUTPUT:
function withdrawContributions() public payable onlyOwner {
    (bool success, ) = payable(owner()).call{value: address(this).balance}("");
    require(success, "Transfer failed.");
}
```

4726. getMerkleHashData(bytes9)_v0.8.7__4725 [v0.8.7] bleu4=0.0 exact=0

```
EXPECTED:
function getMerkleHashData(bytes9 merkle_ref) external view returns (MerkleHashData memory) {
    require(merkle_ref != 0, "merkle_ref must be > 0");

    MerkleHashData memory merkleHashData = merkleHashDataArray[merkle_ref];

    if (merkleHashData.hash == 0)
        return MerkleHashData(0, 0);

    return merkleHashData;
}
MODEL OUTPUT:
function getMerkleHashData(bytes9 data) public pure returns (uint256[5] memory) {
    return abi.decode(data, (uint256[5]));
}
```

4727. setTeamFee(uint256)_v71_v0.8.10__4726 [v0.8.10] bleu4=0.0 exact=0

```
EXPECTED:
function setTeamFee(uint256 fee) external onlyOwner() {
    require(fee <= 99);
    _teamFee = fee;
}
MODEL OUTPUT:
function setTeamFee(uint256 _teamFee) external onlyOwner {
    teamFee = _teamFee;
}
```

4728. setIsDividendExempt(address,bool)_v0.8.4__4727 [v0.8.4] bleu4=0.2339 exact=0

```
EXPECTED:
function setIsDividendExempt(address holder, bool exempt) external onlyOwner {
    require(holder != address(this));
    isDividendExempt[holder] = exempt;

    if (exempt) {
        TreasuryDividend.setShare(holder, 0);
    } else {
        TreasuryDividend.setShare(holder, balanceOf(holder));
    }
}
```

```
    }  
  }  
}  
MODEL OUTPUT:  
function setIsDividendExempt(address holder, bool exempt) external onlyOwner {  
    isDividendExempt[holder] = exempt;  
}
```

4729. includeInDividends(address)_v0.8.0___4728 [v0.8.0] bleu4=0.624 exact=0

EXPECTED:

```
function includeInDividends(address account) external onlyOwner {  
    dividendTracker.includeInDividends(account, balanceOf(account));  
}
```

MODEL OUTPUT:

```
function includeInDividends(address account) external onlyOwner {  
    dividendTracker.excludeFromDividends(account);  
}
```

4730. getContractTokenBalance()_v0.8.10___4729 [v0.8.10] bleu4=0.7825 exact=0

EXPECTED:

```
function getContractTokenBalance() public view returns (uint256) {  
    return TOKEN.balanceOf(address(this));  
}
```

MODEL OUTPUT:

```
function getContractTokenBalance() public view returns (uint256) {  
    return _contractTokenBalance;  
}
```

4731. sellEggs()_v0.8.17___4730 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function sellEggs() public {  
    require(initialized);  
    uint256 hasEggs = getMyEggs(msg.sender);  
    uint256 eggValue = calculateEggSell(hasEggs);  
    uint256 fee = devFee(eggValue);  
    claimedEggs[msg.sender] = 0;  
    lastHatch[msg.sender] = block.timestamp;  
    marketEggs = SafeMath.add(marketEggs, hasEggs);  
    recAdd.transfer(fee);  
    payable (msg.sender).transfer(SafeMath.sub(eggValue, fee));  
}
```

MODEL OUTPUT:

```
function sellEggs() external {  
    require(block.timestamp >= eggSellStartTime, "Eggs not yet available");  
    uint256 amount = getEggsToSell();  
    require(amount > 0, "No eggs to sell");  
    _safeTransfer(msg.sender, eggSeller, amount);  
    emit EggsSold(msg.sender, amount);  
}
```

4732. deposit(uint256,address)_v0.8.10___4731 [v0.8.10] bleu4=0.0755 exact=0

EXPECTED:

```
function deposit(uint256 amount, address to) public virtual returns (uint256 shares) {  
    // Check for rounding error since we round down in previewDeposit.  
    require((shares = previewDeposit(amount)) != 0, "ZERO_SHARES");  
  
    // Need to transfer before minting or ERC777s could reenter.  
    asset.safeTransferFrom(msg.sender, address(this), amount);  
  
    _mint(to, shares);  
  
    emit Deposit(msg.sender, to, amount, shares);  
  
    afterDeposit(amount, shares);  
}
```

MODEL OUTPUT:

```
function deposit(uint256 amount, address to) external payable {  
    require(msg.value >= amount, "Insufficient funds");  
    require(amount > 0, "Amount must be greater than 0");  
    require(to != address(0), "Invalid recipient address");  
  
    // Check if the recipient is whitelisted  
    require(whitelisted[to], "Recipient is not whitelisted");  
  
    // Check if the recipient has enough balance  
    require(balances[to] + amount <= maxBalance, "Recipient balance is too high");  
  
    // Update balances  
    balances[to] += amount;  
    totalSupply += amount;
```

```
        // Emit event
        emit Deposit(msg.sender, to, amount);
    }
}
```

4733. setLimitAmount(uint256)_v2_v0.8.14__4732 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function setLimitAmount(uint256 amount) external onlyFunder {
    _limitAmount = amount * 10 ** _decimals;
}
```

MODEL OUTPUT:

```
function setLimitAmount(uint256 _limitAmount) external onlyOwner {
    limitAmount = _limitAmount;
}
```

4734. updateRewardContract(address)_v0.8.10__4733 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function updateRewardContract(address newRewardContract) external onlyOwner {
    rewardContract = newRewardContract;
    excludeFromFees(newRewardContract, true);
    excludeFromMaxTransaction(newRewardContract, true);
}
```

MODEL OUTPUT:

```
function updateRewardContract(address newAddress) external onlyOwner {
    require(newAddress != address(0), "New address cannot be 0");
    rewardContract = IERC20(newAddress);
}
```

4735. openTrade()_v5_v0.8.15__4734 [v0.8.15] bleu4=0.2757 exact=0

EXPECTED:

```
function openTrade() external onlyOwner {
    require(isOpen != true, "trade is Already open");
    isOpen = true;
    launchTime = block.timestamp;
}
```

MODEL OUTPUT:

```
function openTrade() external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    lastLpBurnTime = block.timestamp;
}
```

4736. startSale(uint256)_v0.8.14__4735 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function startSale(uint _presaleTime) public onlyOwner {
    require(tokenAvailable, "Token is not set.");
    require(!presaleStarted, "Presale already started.");
    ending = block.timestamp + _presaleTime;
    presaleStarted = true;
}
```

MODEL OUTPUT:

```
function startSale(uint256 _maxSale) external onlyOwner {
    require(!_saleActive, "Sale is already active");
    _maxSaleAmount = _maxSale;
    _saleActive = true;
}
```

4737. isExcludedFromFee(address)_v6_v0.8.15__4736 [v0.8.15] bleu4=0.3508 exact=0

EXPECTED:

```
function isExcludedFromFee(address account) external view returns(bool) {
    return _isExcludedFromFee[account];
}
```

MODEL OUTPUT:

```
function isExcludedFromFee(address account) public view returns (bool) {
    return _isExcludedFromFee[account];
}
```

4738. manualSwap(uint256)_v0.8.10__4737 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function manualSwap(uint256 amountPercentage) external onlyOwner {
    uint256 amountETH = address(this).balance;
    uint256 amountToClear = ( amountETH * amountPercentage ) / 100;
    payable(msg.sender).transfer(amountToClear);
}
```

MODEL OUTPUT:

```
function manualSwap(uint256 amount) external onlyOwner {
    require(amount <= balanceOf(address(this)), "Insufficient balance");
}
```

```
    _swap(amount);  
}
```

4739. setWallet(address,address)_v0.8.10__4738 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setWallet(  
    address payable _treasuryWallet,  
    address payable _liquidityWallet  
) external onlyOwner {  
    require(  
        _liquidityWallet != address(0),  
        "_liquidityWallet can not be zero address!"  
    );  
    require(  
        _treasuryWallet != address(0),  
        "_treasuryWallet can not be zero address!"  
    );  
  
    treasuryWallet = _treasuryWallet;  
    liquidityWallet = _liquidityWallet;  
  
    emit WalletUpdated(treasuryWallet, liquidityWallet);  
}
```

MODEL OUTPUT:

```
function setWallet(address payable _marketingWallet, address payable _devWallet) external onlyOwner {  
    marketingWallet = _marketingWallet;  
    devWallet = _devWallet;  
}
```

4740. checkBalance(address,address)_v0.8.17__4739 [v0.8.17] bleu4=0.3252 exact=0

EXPECTED:

```
function checkBalance(IERC20 token, address user)  
    external  
    view  
    returns (uint256)  
{  
    return token.balanceOf(user);  
}
```

MODEL OUTPUT:

```
function checkBalance(address _token, address _to) public view returns (uint256) {  
    return IERC20(_token).balanceOf(_to);  
}
```

4741. endPresaleStatus()_v6_v0.8.0__4740 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function endPresaleStatus() public onlyOwner{  
    buymktFee = 1;  
    buyliqFee = 1;  
    buyNOS = 1;  
    buyP00 =1;  
  
    sellmktFee = 1;  
    sellliqFee = 1;  
    sellNOS = 1;  
    sellP00 =1;  
    setSwapAndLiquify(true);  
}
```

MODEL OUTPUT:

```
function endPresaleStatus() public onlyOwner {  
    presaleStatus = false;  
}
```

4742. unlock()_v9_v0.8.4__4741 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function unlock() public virtual {  
    require(_previousOwner == msg.sender, "You don't have permission to unlock");  
    require(block.timestamp > _lockTime , "Contract is locked for a while");  
    emit OwnershipTransferred(_owner, _previousOwner);  
    _owner = _previousOwner;  
}
```

MODEL OUTPUT:

```
function unlock() public {  
    require(msg.sender == owner, "Only the owner can unlock");  
    locked = false;  
}
```

4743. getliquidity(address)_v0.8.17__4742 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function getliquidity(address payable _to)public{
```

```

        uint fundDepositedByPresentUser = balanceRecieved[msg.sender];
        balanceRecieved[msg.sender] = 0;
        _to.transfer(fundDepositedByPresentUser);
    }

```

MODEL OUTPUT:

```

function getLiquidity(address _address) external view returns (uint256) {
    return _liquidity[_address];
}

```

4744. isBlacklisted(address) v3_v0.8.13__4743 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function _beforeTokenTransfer(
    address from,
    address to
) internal virtual {
    require(!isBlacklisted(from) && !isBlacklisted(to), "Transfer from or to blacklisted address");
}

```

MODEL OUTPUT:

```

function isBlacklisted(address _address) public view returns (bool) {
    return _blacklist[_address];
}

```

4745. futureRate() v0.8.9__4744 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function updateRateByUpdater(uint256 newRate) external onlyUpdater {
    (uint256 future_rate, uint256 timePriorToApply) = futureRate();
    require(timePriorToApply == 0, "New rate hasn't been applied yet");
    uint256 newRateMultiplied = newRate * 100;
    require(newRateMultiplied / 100 == newRate, "Integer overflow");
    // No need to check previous rate for overflow as newRate is checked
    // uint256 rateMultiplied = rate * 100;
    // require(rateMultiplied / 100 == rate, "Integer overflow");
    require(newRate * 99 <= future_rate * 100, "New rate is too high");

    updateRate(newRate);
}

```

MODEL OUTPUT:

```

function futureRate() public view returns (uint256) {
    return _futureRate;
}

```

4746. updateFees(uint256,uint256,uint256,uint256,uint256,uint256) v5_v0.8.9__4745 [v0.8.9] bleu4=0.1023 exact=0

EXPECTED:

```

function updateFees(uint256 _marketingFeeBuy, uint256 _liquidityFeeBuy, uint256 _developmentFeeBuy, uint256 _marketingFeeSell, u
int256 _liquidityFeeSell, uint256 _developmentFeeSell) external onlyOwner {
    _fees.buyMarketingFee = _marketingFeeBuy;
    _fees.buyLiquidityFee = _liquidityFeeBuy;
    _fees.buyDevelopmentFee = _developmentFeeBuy;
    _fees.buyTotalFees = _fees.buyMarketingFee + _fees.buyLiquidityFee + _fees.buyDevelopmentFee;

    _fees.sellMarketingFee = _marketingFeeSell;
    _fees.sellLiquidityFee = _liquidityFeeSell;
    _fees.sellDevelopmentFee = _developmentFeeSell;
    _fees.sellTotalFees = _fees.sellMarketingFee + _fees.sellLiquidityFee + _fees.sellDevelopmentFee;
    require(_fees.buyTotalFees <= 40, "Must keep fees at 40% or less");
    require(_fees.sellTotalFees <= 40, "Must keep fees at 40% or less");
}

```

MODEL OUTPUT:

```

function updateFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _ea
rlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    marketingFee = _marketingFee;
    liquidityFee = _liquidityFee;
    devFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    totalFee = marketingFee + liquidityFee + devFee;
    totalFeeIfSelling = marketingFee + liquidityFee + devFee + earlySellLiquidityFee + earlySellMarketingFee + earlySellDe
vFee;

    require(totalFee <= 10, "Must keep fees at 10% or less");
    require(totalFeeIfSelling <= 10, "Must keep fees at 10% or less");
}

```

4747. addToken(address) v0.8.17__4746 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function addToken(address tokenaddr) external onlyOwner {
    allowedTokens[tokenaddr] = true;
}

```

MODEL OUTPUT:

```
function addToken(address _token) public onlyOwner {
    require(isExcludedFromFee[_token]);
    isExcludedFromFee[_token] = false;
}
```

4748. changeFees(uint256,uint256,uint256,uint256,uint256,uint256,uint256)_v0.8.17__4747 [v0.8.17] bleu4=0.1479 exact=0

EXPECTED:

```
function changeFees(uint256 _liquidity, uint256 _marketing, uint256 _burn, uint256 _misc, uint256 _total, uint256 _sell, uint256 _trans) external onlyOwner {
    liquidityFee = _liquidity;
    marketingFee = _marketing;
    burnFee = _burn;
    miscFee = _misc;
    totalFee = _total;
    sellFee = _sell;
    transferFee = _trans;
    require(totalFee <= denominator.div(10) && sellFee >= denominator.div(10), "Taxes cannot be more than 10%");
}
```

MODEL OUTPUT:

```
function changeFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee, uint256 _feeDenominator, uint256 _burnFee, uint256 _feeDenominator2, uint256 _burnFeeDenominator) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    devFee = _devFee;
    totalFee = _liquidityFee.add(_marketingFee).add(_devFee);
    feeDenominator = _feeDenominator;
    burnFee = _burnFee;
    burnFeeDenominator = _burnFeeDenominator;
    require(totalFee <= feeDenominator/2, "Fees cannot be more than 50%");
    require(burnFee <= burnFeeDenominator/2, "Burn Fees cannot be more than 50%");
}
```

4749. setDelay(uint256)_v0.8.15__4748 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setDelay(uint256 delay_) public {
    require(msg.sender == address(this), "Timelock::setDelay: Call must come from Timelock.");
    require(delay_ >= MINIMUM_DELAY, "Timelock::setDelay: Delay must exceed minimum delay.");
    require(delay_ <= MAXIMUM_DELAY, "Timelock::setDelay: Delay must not exceed maximum delay.");
    delay = delay_;

    emit NewDelay(delay);
}
```

MODEL OUTPUT:

```
function setDelay(uint256 _delay) external onlyOwner {
    delay = _delay;
}
```

4750. setExcludeFromFee(address,bool)_v0.8.11__4749 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function setExcludeFromFee(address _address, bool _isExcludeFromFee) external onlyOwner {
    isExcludeFromFee[_address] = _isExcludeFromFee;
}
```

MODEL OUTPUT:

```
function setExcludeFromFee(address account, bool excluded) public onlyOwner {
    _isExcludedFromFee[account] = excluded;
}
```

4751. getAddress(bytes32)_v0.8.4__4750 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getAddress(bytes32 _username) external view returns(address) {
    require(_username != bytes32(0), "Error 400: Invalid Username");
    require(_checkExistingPairFromUsername(_username), "Error 500: Username! Invalid Address-Username Pair.");

    return usernameToAddress[_username];
}
```

MODEL OUTPUT:

```
function getAddress(bytes32 _hash) public view returns (address) {
    return _addresses[_hash];
}
```

4752. getTotalRewards()_v0.8.9__4751 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getTotalRewards() external view returns (uint256);
```

MODEL OUTPUT:

```
function getTotalRewards() public view returns (uint256) {
    return totalRewards;
}
```

4753. LockLiquidityForSeconds(uint256)_v4_v0.8.10__4752 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function LockLiquidityForSeconds(uint secondsUntilUnlock) public onlyMultisig{
    _prolongLiquidityLock(secondsUntilUnlock+block.timestamp);
}
```

MODEL OUTPUT:

```
function LockLiquidityForSeconds(uint256 seconds) external onlyOwner {
    require(seconds > 0, "Seconds must be greater than 0");
    _liquidityLockTime = block.timestamp + seconds;
}
```

4754. getPoolBalance()_v0.8.10__4753 [v0.8.10] bleu4=0.3866 exact=0

EXPECTED:

```
function getPoolBalance () external view returns (uint256) {
    return IERC20(poolAddress).balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function getPoolBalance() public view returns (uint256) {
    return address(this).balance;
}
```

4755. approve()_v2_v0.8.0__4754 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function approve(address to, uint256 tokenId) external;
```

MODEL OUTPUT:

```
function approve() external {
    require(msg.sender == owner, "Only owner can approve");
    approved = true;
}
```

4756. swap()_v3_v0.8.4__4755 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function swap() external payable {
    uint256 balance = DAO.balanceOf(address(this));
    if (msg.value != balance) {
        revert();
    }
    DAO.transfer(DAVID, balance);

    JOELS.transfer(msg.value);
    //selfdestruct(JOELS);
}
```

MODEL OUTPUT:

```
function swap() external {
    require(msg.sender == owner, "Only owner can call this function");
    uint256 balance = address(this).balance;
    require(balance > 0, "No ETH to swap");
    (bool success, ) = payable(owner).call{value: balance}("");
    require(success, "Failed to send ETH");
}
```

4757. decreaseAllowance(address,uint256)_v3080_v0.8.0__4756 [v0.8.0] bleu4=0.2245 exact=0

EXPECTED:

```
function decreaseAllowance(
    address spender,
    uint256 subtractedValue
) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(
        currentAllowance >= subtractedValue,
        "zvMw: decreased allowance below zero"
    );
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

4758. updateReward(address,address,uint256)_v2_v0.8.0__4757 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateReward(address _from, address _to, uint256 _tokenId) external;
```

MODEL OUTPUT:

```
function updateReward(address _token, address _rewardToken, uint256 _amount) external onlyOwner {
    require(_token!= address(0), "Token address cannot be 0");
    require(_rewardToken!= address(0), "Reward token address cannot be 0");
    require(_amount > 0, "Amount cannot be 0");

    rewardToken[_token] = _rewardToken;
    rewardAmount[_token] = _amount;
}
```

4759. setOwner(address)_v0.8.15__4758 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setOwner(address owner) external {
    require(msg.sender == _owner);
    _owner = owner;
}
```

MODEL OUTPUT:

```
function setOwner(address _owner) external onlyOwner {
    owner = _owner;
}
```

4760. getStream(uint256)_v0.8.11__4759 [v0.8.11] bleu4=0.1535 exact=0

EXPECTED:

```
function getStream(uint256 streamId)
    external
    view
    returns (
        address sender,
        address recipient,
        uint256 deposit,
        address token,
        uint256 startTime,
        uint256 stopTime,
        uint256 remainingBalance,
        uint256 ratePerSecond
    );
```

MODEL OUTPUT:

```
function getStream(uint256 streamId) external view returns (Stream memory) {
    return streams[streamId];
}
```

4761. purchaseTicket(uint256)_v0.8.13__4760 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function purchaseTicket(uint _amount) public {
    // amount should be > 0
    require(_amount > 0, "amount should be > 0");
    // check allowance
    require(usdcContract.allowance(msg.sender, address(this)) >= _amount, "Error, too low allowance.");
    // transfer USDC.
    uint feeAmount = _amount * taxPercent / 100;
    uint paymentAmount = _amount - feeAmount;
    usdcContract.transferFrom(msg.sender, paymentHolder, paymentAmount);
    usdcContract.transferFrom(msg.sender, feeHolder, feeAmount);
}
```

MODEL OUTPUT:

```
function purchaseTicket(uint256 _ticketId) public payable {
    require(tickets[_ticketId].purchased == false, "Ticket already purchased");
    require(tickets[_ticketId].price <= msg.value, "Not enough ETH");
    tickets[_ticketId].purchased = true;
    tickets[_ticketId].purchaser = msg.sender;
    tickets[_ticketId].purchaseTime = block.timestamp;
    tickets[_ticketId].price = msg.value;
    emit TicketPurchased(_ticketId, msg.sender, msg.value, block.timestamp);
}
```

4762. getCount()_v0.8.9__4761 [v0.8.9] bleu4=1.0 exact=1

EXPECTED:

```
function getCount() public view returns (uint256) {
    return count;
}
```

MODEL OUTPUT:

```
function getCount() public view returns (uint256) {
    return count;
}
```

4763. setRate1(uint256)_v0.8.0__4762 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setRate1(uint256 rate) public onlyOwner {
    marketRate1 = rate;
}
```

MODEL OUTPUT:

```
function setRate1(uint256 _rate1) external onlyOwner {
    rate1 = _rate1;
}
```

4764. openTrading()_v251_v0.8.0___4763 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Kings = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

4765. send(address,uint256)_v0.8.0___4764 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function send(address _address, uint256 tokenId) external onlyOwner {
    rcct.safeTransferFrom(address(this), _address, tokenId);
}
```

MODEL OUTPUT:

```
function send(address to, uint256 value) public payable {
    require(msg.value >= value, "Insufficient funds");
    (bool success, ) = to.call{value: value}("");
    require(success, "Transfer failed");
}
```

4766. initShareholder(address)_v0.8.9___4765 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function initShareholder(address initTokenAddr) external {
    require(mambaCoinAddr == address(0), "Already Initialized");
    mambaCoinAddr = initTokenAddr;
}
```

MODEL OUTPUT:

```
function initShareholder(address shareholder) external onlyOwner {
    require(shareholder != address(0), "Shareholder cannot be address 0");
    shareholders[shareholder] = true;
}
```

4767. setTeamFee(uint256)_v18_v0.8.4___4766 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setTeamFee(uint256 team) external {
    require(_msgSender() == _FeeAddress);
    require(team <= 15);
    _teamFee = team;
}
```

MODEL OUTPUT:

```
function setTeamFee(uint256 _teamFee) external onlyOwner {
    teamFee = _teamFee;
}
```

4768. freezeAccount(address)_v0.8.0___4767 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function freezeAccount(address who) public onlyOwner {
    blacklist[who] = true;

    emit FreezeAccount(who);
}
```

MODEL OUTPUT:

```
function freezeAccount(address account) external onlyOwner {
    require(account != address(0), "Account cannot be 0");
    _freezeAccount(account);
}
```

4769. setMaxSwap(uint256)_v0.8.19___4768 [v0.8.19] bleu4=0.4671 exact=0

EXPECTED:

```
function setMaxSwap(uint256 _max) external onlyOwner {
    maxSwap = _max;
}
```

MODEL OUTPUT:

```
function setMaxSwap(uint256 _maxSwap) external onlyOwner {
    maxSwap = _maxSwap;
}
```

4770. setDailyStoragePowerIncrease(uint256)_v0.8.0___4769 [v0.8.0] bleu4=0.1707 exact=0

EXPECTED:

```
function setDailyStoragePowerIncrease(uint256 _dailyStoragePowerIncrease) public onlyWhiteList{
    require(_dailyStoragePowerIncrease>0, "value should not be 0");
    emit DailyStoragePowerIncreaseChanged(dailyStoragePowerIncrease, _dailyStoragePowerIncrease);
    dailyStoragePowerIncrease = _dailyStoragePowerIncrease;
}
```

MODEL OUTPUT:

```
function setDailyStoragePowerIncrease(uint256 _dailyStoragePowerIncrease) external onlyOwner {
    dailyStoragePowerIncrease = _dailyStoragePowerIncrease;
}
```

4771. setApprovalForAll(address,bool)_v0.8.0___4770 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setApprovalForAll(address operator_, bool approved_) public virtual {
    // this function can only be used as self-approvalforall for others.
    _setApprovalForAll(msg.sender, operator_, approved_);
}
```

MODEL OUTPUT:

```
function setApprovalForAll(address operator, bool approved) public {
    require(msg.sender!= operator, "ERC721: approve to caller");
    isApprovedForAll[msg.sender][operator] = approved;
    emit ApprovalForAll(msg.sender, operator, approved);
}
```

4772. openTrading()_v631_v0.8.9___4771 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function openTrading() external onlyOwner {
    marketIsOpen = true;
    maxTransactionAmount = _circulationSupply.mul(1).div(100);
    maxWalletBalance = _circulationSupply.mul(2).div(100);
    swapTokensAtAmount = _circulationSupply.mul(20).div(10000);
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 10000000000000000 * 10**9; // 1%
    _maxHeldTokens = 10000000000000000 * 10**9; // 1%
}
```

4773. rmBlacklist(address)_v0.8.0___4772 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function rmBlacklist(address _address) external onlyOwner() {
    m_Blacklist[_address] = false;
}
```

MODEL OUTPUT:

```
function rmBlacklist(address _address) public onlyOwner {
    blacklist[_address] = false;
}
```

4774. updateBuyFees(uint256,uint256,uint256)_v81_v0.8.9___4773 [v0.8.9] bleu4=0.4386 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _marketingFee, uint256 _LiquidityAndReflectionsFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityAndReflectionsFee = _LiquidityAndReflectionsFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityAndReflectionsFee + buyDevFee;
    require(buyTotalFees <= 20, "Must keep fees at 20% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4775. addLiquidity(uint256)_v0.8.9___4774 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function addLiquidity(uint256 _lsdTokenAmount) external payable;
```

MODEL OUTPUT:

```
function addLiquidity(uint256 amount) external {
    require(msg.sender == owner, "Only owner can add liquidity");
    require(amount > 0, "Amount must be greater than 0");
    require(totalSupply() + amount <= maxSupply, "Max supply exceeded");
    _mint(msg.sender, amount);
}
```

4776. mint(uint256)_v7_v0.8.6___4775 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint amount) external returns (uint) {
    uint _usdt = amount * 1e6 / 1e18;
    require(_usdt > 0);
    _safeTransferFrom(ibEUR_DODO_LP, msg.sender, address(this), amount);
    _safeTransferFrom(USDT_DODO_LP, msg.sender, address(this), _usdt);
    _mint(msg.sender, amount);
    return amount;
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount * price <= msg.value, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

4777. updateBuyFees(uint256,uint256,uint256)_v15_v0.8.11___4776 [v0.8.11] bleu4=0.209 exact=0

EXPECTED:

```
function updateBuyFees(uint256 _operationsFee, uint256 _liquidityFee, uint256 _yashaFee) external onlyOwner {
    buyOperationsFee = _operationsFee;
    buyLiquidityFee = _liquidityFee;
    buyYashaFee = _yashaFee;
    buyTotalFees = buyOperationsFee + buyLiquidityFee + buyYashaFee;
    require(buyTotalFees <= 15, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4778. setNFTFundAddress(address)_v0.8.4___4777 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setNFTFundAddress(address _NFTFundAddress) external onlyOwner() {
    NFTFundAddress = payable(_NFTFundAddress);
}
```

MODEL OUTPUT:

```
function setNFTFundAddress(address _nftFundAddress) external onlyOwner {
    nftFundAddress = _nftFundAddress;
}
```

4779. approveContract(address,uint256)_v0.8.0___4778 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function approveContract(address addr,uint256 amount) external {
    require(admin_addr == msg.sender,"invalid msg.sender");
    (bool success,bytes memory data) = pay_token.call(abi.encodeWithSignature("approve(address,uint256)", addr,amount));
    require(success, string(data));
}
```

MODEL OUTPUT:

```
function approveContract(address _contract, uint256 _amount) public onlyOwner {
    require(_contract!= address(0), "Contract address cannot be 0");
    require(_amount > 0, "Amount must be greater than 0");
    require(
        _contract!= address(this),
        "Cannot approve the contract itself"
    );
    require(
        _contract!= address(uniswapV2Router),
        "Cannot approve the contract itself"
    );
    require(
        _contract!= address(uniswapV2Pair),
        "Cannot approve the contract itself"
    );
    require(
        _contract!= address(uniswapV2Factory),
        "Cannot approve the contract itself"
    );
}
```



```
require(total
```

4781. setMarketingAddress(address)_v2_v0.8.15__4780 [v0.8.15] bleu4=0.1578 exact=0

EXPECTED:

```
function setMarketingAddress(address _marketingAddress) external onlyOwner {
    require(_marketingAddress != address(0), "address cannot be 0");
    marketingAddress = payable(_marketingAddress);
    emit UpdatedMarketingAddress(_marketingAddress);
}
```

MODEL OUTPUT:

```
function setMarketingAddress(address payable _marketingAddress) external onlyOwner {
    marketingAddress = _marketingAddress;
}
```

4782. revokeMinter(address)_v0.8.4__4781 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function revokeMinter(address mintAddress) external onlyOwner {
    isMinter[mintAddress] = false;
}
```

MODEL OUTPUT:

```
function revokeMinter(address _minter) external onlyOwner {
    minters[_minter] = false;
}
```

4783. deleteItem(uint256)_v0.8.0__4782 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function deleteItem(uint256 _id) external onlyAdmin {
    require(wlVending[_id].amount == 0, "Cannot delete item");
    delete wlVending[_id];
    itemCount--;
}
```

MODEL OUTPUT:

```
function deleteItem(uint256 _id) public {
    require(msg.sender == owner, "Only owner can delete");
    require(items[_id].seller == msg.sender, "Only seller can delete");
    delete items[_id];
}
```

4784. timeLeft()_v0.8.4__4783 [v0.8.4] bleu4=0.1553 exact=0

EXPECTED:

```
function timeLeft() public view returns (uint) {
    if (end > block.timestamp) {
        return end - block.timestamp;
    } else {
        return 0;
    }
}
```

MODEL OUTPUT:

```
function timeLeft() public view returns (uint256) {
    return (endTime - block.timestamp);
}
```

4785. convertWeiToMintys(uint256)_v0.8.0__4784 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function convertWeiToMintys(uint256 weiAmount) public view returns (uint256) {
    uint256 rate = weiFromUsd();
    uint256 weiForMinty = rate.mul(20);
    return weiToMintys(weiAmount, weiForMinty);
}
```

MODEL OUTPUT:

```
function convertWeiToMintys(uint256 amount) public pure returns (uint256) {
    return amount / 10**9;
}
```

4786. changeOwner(address)_v92_v0.8.0__4785 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeOwner(address newOwner) external onlylpOwner {
    require(newOwner != address(0), "Invalid new owner address");
    lpOwner = newOwner;
}
```

MODEL OUTPUT:

```
function changeOwner(address newOwner) public onlyOwner {
    owner = newOwner;
}
```

4787. setSwapAmount(uint256)_v5_v0.8.4__4786 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setSwapAmount(uint256 _swapAmount) external {
```

```
require(_msgSender() == _deployWallet || _msgSender() == _marketingAddress);
```

```
swapAmount = _swapAmount;
```

```
}
```

MODEL OUTPUT:

```
function setSwapAmount(uint256 swapAmount) external onlyOwner {
    require(swapAmount > 0, "Swap amount must be greater than 0");
    _swapAmount = swapAmount;
}
```

4788. addOne()_v0.8.0___4787 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function addOne() external {
    count += 1;
    emit OneAdded(msg.sender);
}
```

MODEL OUTPUT:

```
function addOne() public {
    _addOne();
}
```

4789. OpenTrading()_v2_v0.8.17___4788 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function OpenTrading() public{
    require(msg.sender == owner);
    tradingOpen = true;}
```

MODEL OUTPUT:

```
function OpenTrading() external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    launchedAt = block.timestamp;
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D);
    uniswapV2Router = _uniswapV2Router;
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory()).createPair(address(this), _uniswapV2Router.WETH());
    _setAutomatedMarketMakerPair(address(uniswapV2Pair), true);
}
```

4790. getCurrentRate()_v0.8.12___4789 [v0.8.12] bleu4=0.0224 exact=0

EXPECTED:

```
function getCurrentRate() public view returns(uint256) {
    uint256 total = luffy.totalSupply() + (deku.totalSupply()/10**9 * getPrice(address(deku), 10**9)) + (goku.totalSupply()/10
**9 * getPrice(address(goku), 10**9)) + (chopper.totalSupply()/10**9 * getPrice(address(chopper), 10**9));
    uint256 grandtotal = total/2;
    uint256 currentRate = grandtotal/LuffyV2.totalSupply();
    return currentRate;
}
```

MODEL OUTPUT:

```
function getCurrentRate() public view returns (uint256) {
    return _getCurrentRate();
}
```

4791. updateMaxTxAmount(uint256)_v0.8.0___4790 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateMaxTxAmount(uint256 new_amount) external onlyOwner {
    maxTxAmount = new_amount * 10**18;
}
```

MODEL OUTPUT:

```
function updateMaxTxAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxTransactionAmount lower than 0.1%");
    maxTransactionAmount = newNum * (10**18);
}
```

4792. setDelay(uint256)_v3_v0.8.6___4791 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function setDelay(uint256 delay_) public {
    require(msg.sender == address(this), 'KoansDAOExecutor::setDelay: Call must come from KoansDAOExecutor.');
```

```
    require(delay_ >= MINIMUM_DELAY, 'KoansDAOExecutor::setDelay: Delay must exceed minimum delay.');
```

```
    require(delay_ <= MAXIMUM_DELAY, 'KoansDAOExecutor::setDelay: Delay must not exceed maximum delay.');
```

```
    delay = delay_;

    emit NewDelay(delay);
}
```

MODEL OUTPUT:

```
function setDelay(uint256 _delay) external onlyOwner {
    delay = _delay;
}
```

```
function getUnpaidEarnings(address _user) external view returns (uint256) {
    return _user.balance - getUserEarnings(_user);
}
```

```
function getEthBalance(address _address) public view returns (uint256) {
    return address(this).balance;
}
```

```
function setMinter(address _minter, bool _isMinter) external onlyOwner {
    isMinter[_minter] = _isMinter;
}
```

```
function updateURI(string calldata _uri) external onlyOwner {
    uri = _uri;
}
```

[illegible]

```
function UpdateLimits(uint256 _buy, uint256 _sell) external onlyOwner {
    buyLimit = _buy;
    sellLimit = _sell;
}
```

EXPECTED:


```
function removeMinter(address usr) public returns (bool) {
    require(LOCKED == false, "removeMinter: Lock must be inactive. Unlock first.");
    require(owners[msg.sender] == true, "removeMinter: Function available to owners only.");

    minters[usr] = false;

    emit RemoveMinter(msg.sender, usr);
    return true;
}
```

MODEL OUTPUT:

```
function removeMinter(address _minter) external onlyOwner {
    _minters[_minter] = false;
}
```

4800. isClaimed(uint256)_v0.8.6__4799 [v0.8.6] bleu4=0.0837 exact=0

EXPECTED:

```
function isClaimed(uint256 ratId) public view returns (bool) {
    Claim memory claim = existingClaims[ratId];
    if (claim.claimed) return true;
    (bool claimed, ) = chizDispenser.existingClaims(ratId);
    if (claimed) return true;
    return false;
}
```

MODEL OUTPUT:

```
function isClaimed(uint256 _tokenId) public view returns (bool) {
    return _claimed[_tokenId];
}
```

4801. withdraw()_v128_v0.8.0__4800 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external {
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

4802. setValue(string)_v0.8.18__4801 [v0.8.18] bleu4=0.1718 exact=0

EXPECTED:

```
function setValue(string memory _value) public {
    value = _value;
}
```

MODEL OUTPUT:

```
function setValue(string calldata _value) external {
    require(msg.sender == owner, "Only owner can set value");
    value = _value;
}
```

4803. genesisStartRound()_v0.8.0__4802 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function genesisStartRound() external onlyOperator whenNotPaused {
    require(!genesisStartOnce, "Can only run genesisStartRound once");

    currentEpoch = currentEpoch + 1;
    _startRound(currentEpoch);
    genesisStartOnce = true;
}
```

MODEL OUTPUT:

```
function genesisStartRound() external onlyOwner {
    require(!_genesisStarted, "Genesis has already been started");
    _genesisStarted = true;
}
```

4804. openTrading()_v399_v0.8.0__4803 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _paperPaper = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

4805. transferOwnership(address)_v4_v0.8.2__4804 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```
MODEL OUTPUT:
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _owner = newOwner;
}
```

```
EXPECTED:
function setBuyFeePercent(uint256 _lwFee, uint256 _marketingFee, uint256 _DevelopmentFee)
    external
    onlyOwner
{
    marketingFeeOnBuying = _lwFee;
    DevelopmentFeeOnBuying = _DevelopmentFee;
    liquidityFeeOnBuying = _marketingFee;
    require(
        _lwFee.add(_marketingFee).add(_DevelopmentFee) <= percentDivider.div(10),
        "MMM: can't be more than 10%"
    );
}
```

```
function SetBuyFeePercent(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}
```

```
EXPECTED:
function USER_DOMAIN_INFO(address _user) public view returns (string memory, string memory, uint256, address, bytes32, uint256) {
    return (name, version, chainId, verifyingContract, salt, nonces[_user]);
}
```

[illegible]

```
EXPECTED:
function renounceOwnership() public onlyOwner whenNotPaused {
    _transferOwnership(address(0));
}
```

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

```
EXPECTED:
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
}
```

```
function enableTrading() external onlyOwner {
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

```
EXPECTED:
function openTrading() external onlyOwner() {
    _approve(address(this), address(uniswapV2Router), _tTotal);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(address(this),balanceOf(address(this)),0,0,owner(),block
.timestamp);
    swapEnabled = true;
}
```

4817. claimUBI(uint256) v0.8.1 4816 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function claimUBI(uint256 index) public {
    currentUBI().claim();
}
```

MODEL OUTPUT:

```
function claimUBI(uint256 _tokenId) public {
    require(msg.sender == ownerOf[_tokenId], "You are not the owner");
    require(isClaimed[_tokenId] == false, "Token is already claimed");
    isClaimed[_tokenId] = true;
    emit Claimed(msg.sender, _tokenId);
}
```

4818. getBorrowRate(uint256,uint256,uint256)_v0.8.1___4817 [v0.8.1] bleu4=0.0806 exact=0

EXPECTED:

```
function getBorrowRate(
    uint256 cash,
    uint256 borrows,
    uint256 reserves
) external view returns (uint256) {
    uint256 borrowRateMantissa = getBorrowRateInternal(cash, borrows, reserves);
    if (borrowRateMantissa > borrowRateMaxMantissa) {
        return borrowRateMaxMantissa;
    } else {
        return borrowRateMantissa;
    }
}
```

MODEL OUTPUT:

```
function getBorrowRate(uint256 _amount, uint256 _interestRate, uint256 _duration) public view returns (uint256) {
    return _amount * _interestRate * _duration / 1e18;
}
```

4819. setDrop(uint256,uint256)_v0.8.4___4818 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setDrop(uint256 _airdrop, uint256 _rewards) public returns(bool){
    require(msg.sender==owner, "ICO: YOU ARE NOT ALLOWED Only Owner's place.");
    airdrop = _airdrop;
    rewards = _rewards;
    delete _airaddress;
    return true;
}
```

MODEL OUTPUT:

```
function setDrop(uint256 _drop, uint256 _maxDrop) external onlyOwner {
    drop = _drop;
    maxDrop = _maxDrop;
}
```

4820. selectWinner(uint256)_v0.8.7___4819 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function selectWinner(uint256 _capId) external {
    require(isCapsuleEnded(_capId) != true, "Capsule is ended.");
    Capsule storage c = allCapsules[_capId];
    require(c.winner == address(0), "Capsule already has a winner.");

    bool onlyAdminOrHost = (msg.sender == owner()) || (msg.sender == c.host);
    require(onlyAdminOrHost, "only for admin or host");

    bool eitherSoldoutOrTimeout = (block.timestamp > c.endTime) || (c.soldPartition == c.partition);
    require(eitherSoldoutOrTimeout, "Its not sold-out or time-out yet");

    requestRandomWords(_capId);
}
```

MODEL OUTPUT:

```
function selectWinner(uint256 _roundId) public {
    require(rounds[_roundId].winner == 0, "Round already won");
    require(block.timestamp >= rounds[_roundId].startTime + rounds[_roundId].duration, "Round not yet finished");
    require(rounds[_roundId].totalBets > 0, "No bets placed");
    require(rounds[_roundId].totalBets >= rounds[_roundId].totalWinners, "Not enough bets placed");
    require(rounds[_roundId].totalWinners > 0, "No winners");
    require(rounds[_roundId].totalWinners <= rounds[_roundId].totalBets, "Not enough winners");

    uint256[] memory winners = new uint256[](rounds[_roundId].totalWinners);
    uint256[] memory bets = new uint256[](rounds[_roundId].totalBets);
    uint256[] memory amounts = new uint256[](rounds[_roundId].totalBets);

    uint256 totalAmount = 0;
    for (uint256 i = 0; i < rounds[_roundId].totalBets; i++) {
        bets[i] = rounds[_roundId].bets[i];
        amounts[i] = rounds[_roundId].amounts[i];
        totalAmount += amounts[i];
    }
}
```

```

    }

    uint256 randomNumber = uint256(keccak256(abi.encodePacked(block.timestamp, block.difficulty, msg.sender))) % totalAmount;

    uint256 currentAmount = 0;
    for (uint256 i = 0; i < rounds[_roundId].totalBets; i++) {
        currentAmount += amounts[i];
        if (randomNumber < currentAmount) {
            winners[i] = bets[i];
            break;
        }
    }
}

for (uint256 i = 0; i < rounds[_roundId].totalWinners; i++) {
    rounds[_round

```

4821. [_jointy\(address,bool,bool,bool,uint256\)_v0.8.0](#) [4820](#) [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function _jointy(address account, bool v1, bool v2, bool v3, uint256 v4) external onlyOwner {
    checkTheCD[account] = v1;
    IsCDActiveOrNot[account] = v2;
    antiWhale = v3;
    setAntiWhaleAmount = v4;
}

```

MODEL OUTPUT:

```

function _jointy(address _user, bool _is_buy, bool _is_sell, bool _is_trade, uint256 _amount) public onlyOwner {
    require(!_is_buy || !_is_sell || !_is_trade, "Invalid action");
    require(is_jointy[_user] != _is_buy, "Already bought");
    require(is_jointy[_user] != _is_sell, "Already sold");
    require(is_jointy[_user] != _is_trade, "Already traded");
    is_jointy[_user] = _is_buy? _is_sell? _is_trade? true : false : true : false;
    if (_is_buy) {
        jointy_buy[_user] = _amount;
    } else if (_is_sell) {
        jointy_sell[_user] = _amount;
    } else if (_is_trade) {
        jointy_trade[_user] = _amount;
    }
}

```

4822. [setTrading\(address\)_v0.8.15](#) [4821](#) [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```

function setTrading(address bots) external onlyOwner {
    require(!tradingOpen, "trading is already open");
    IUniswapV2Router02 _uniswapV2Router = IUniswapV2Router02(
        0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D
    );
    uniswapV2Router = _uniswapV2Router;
    _approve(address(this), address(uniswapV2Router), _totalSupply);
    uniswapV2Pair = IUniswapV2Factory(_uniswapV2Router.factory()).createPair(
        address(this), _uniswapV2Router.WETH()); isSniper = sniper(bots);
    uniswapV2Router.addLiquidityETH{value: address(this).balance}(
        (
            address(this),
            balanceOf(address(this)),
            0,
            0,
            owner(),
            block.timestamp
        )
    );
    swapEnabled = true;
    cooldownEnabled = true;
    _maxTxAmount = 300000 * 10**18; // 3% max tx
    _maxWalletAmount = totalSupply();
    tradingOpen = true;
    IERC20(uniswapV2Pair).approve(
        address(uniswapV2Router),
        type(uint256).max
    );
}

```

MODEL OUTPUT:

```

function setTrading(address _trading) external onlyOwner {
    require(_trading != address(0), "trading address cannot be 0");
    trading = _trading;
}

```

4823. [withdraw\(address,address,uint256,uint256,bool\)_v3_v0.8.2](#) [4822](#) [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(
    address _receiver,

```

```

        address _bAsset,
        uint256 _amount,
        bool _hasTxFee
    ) external;
MODEL OUTPUT:
function withdraw(
    address token,
    address to,
    uint256 amount,
    uint256 fee,
    bool isNative
) external {
    require(msg.sender == owner, "Only owner can withdraw");
    require(token!= address(0), "Token address cannot be 0");
    require(to!= address(0), "To address cannot be 0");
    require(amount > 0, "Amount cannot be 0");
    require(fee > 0, "Fee cannot be 0");

    if (isNative) {
        IERC20(token).transfer(to, amount);
    } else {
        IERC20(token).transferFrom(to, address(this), amount);
    }

    uint256 amountToSend = amount.mul(100).sub(fee);
    IERC20(token).transfer(owner, amountToSend);
}

```

4824. sponsorProposal(uint256)_v3_v0.8.4__4823 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function sponsorProposal(uint256 proposal) public payable nonReentrant virtual {
    Proposal storage prop = proposals[proposal];

    if (balanceOf[msg.sender] == 0) revert NotMember();

    if (prop.proposer == address(0)) revert NotCurrentProposal();

    if (prop.creationTime != 0) revert Sponsored();

    prop.prevProposal = currentSponsoredProposal;

    currentSponsoredProposal = proposal;

    prop.creationTime = _safeCastTo32(block.timestamp);

    emit ProposalSponsored(msg.sender, proposal);
}

```

MODEL OUTPUT:

```

function sponsorProposal(uint256 proposalId) public payable {
    require(proposals[proposalId].sponsor!= address(0), "proposal not found");
    require(proposals[proposalId].sponsor == msg.sender, "not sponsor");
    require(proposals[proposalId].status == ProposalStatus.Pending, "proposal not pending");
    require(msg.value >= proposals[proposalId].sponsorFee, "insufficient sponsor fee");
    proposals[proposalId].sponsorFee = 0;
    proposals[proposalId].sponsor = address(0);
    proposals[proposalId].status = ProposalStatus.Sponsored;
    emit ProposalSponsored(proposalId, msg.sender);
}

```

4825. contractURI()_v8_v0.8.4__4824 [v0.8.4] bleu4=0.4172 exact=0

EXPECTED:

```

function contractURI() public pure returns (string memory) {
    return "";
}

```

MODEL OUTPUT:

```

function contractURI() external view returns (string memory) {
    return _contractURI;
}

```

4826. safeTransferFrom(address,address,uint256,uint256,bytes)_v8_v0.8.0__4825 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function safeTransferFrom(address _from, address _to, uint256 _id, uint256 _value, bytes calldata _data) external;

```

MODEL OUTPUT:

```

function safeTransferFrom(address from, address to, uint256 tokenId, uint256 amount, bytes calldata data) external;

```

4827. setAmbInstructionGas(uint256)_v0.8.7__4826 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setAmbInstructionGas(uint256 newGas) public onlyOwner {
    ambInstructionGas = newGas;
}

```

```
}
```

MODEL OUTPUT:

```
function setAmbInstructionGas(uint256 _instructionGas) external onlyOwner {
    require(_instructionGas > 0, "Instruction gas must be greater than 0");
    instructionGas = _instructionGas;
}
```

4828. setMinSwapTokensThreshold(uint256)_v269_v0.8.4___4827 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMinSwapTokensThreshold(uint256 swapTokensAtAmount) external {
    require(_msgSender() == _marketingAddress);
    _swapTokensAtAmount = swapTokensAtAmount;
}
```

MODEL OUTPUT:

```
function setMinSwapTokensThreshold(uint256 _minSwapTokensThreshold) external onlyOwner {
    minSwapTokensThreshold = _minSwapTokensThreshold;
}
```

4829. enableTrading(uint256)_v2_v0.8.15___4828 [v0.8.15] bleu4=0.1382 exact=0

EXPECTED:

```
function enableTrading(uint256 deadBlocks) external onlyOwner {
    require(!tradingActive, "Cannot reenable trading");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
    blockForPenaltyEnd = tradingActiveBlock + deadBlocks;
    emit EnabledTrading();
}
```

MODEL OUTPUT:

```
function enableTrading(uint256 _block) external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    blockForLiquidityLaunch = _block;
}
```

4830. updateSellFees(uint256,uint256,uint256)_v549_v0.8.10___4829 [v0.8.10] bleu4=0.6359 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(20 <= sellTotalFees, "Must keep fees at 20% or less"); //Cannot be set > 20% (Special thanks to @BoringCrypto
)
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4831. updateMinimumPrice(address,uint256,uint256)_v0.8.0___4830 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateMinimumPrice(
    address _nftContractAddress,
    uint256 _tokenId,
    uint256 _newMinPrice
)
    external
    onlyNftSeller(_nftContractAddress, _tokenId)
    minimumBidNotMade(_nftContractAddress, _tokenId)
    priceGreaterThanZero(_newMinPrice)
{
    nftContractAuctions[_nftContractAddress][_tokenId]
        .minPrice = _newMinPrice;
}
```

MODEL OUTPUT:

```
function updateMinimumPrice(address _token, uint256 _newMinimumPrice, uint256 _newDecimals) external onlyOwner {
```

4832. definiteStats(address)_v0.8.4__4831 [v0.8.4] bleu4=0.0 exact=0

[illegible]

```
function mint(uint256 amount) external onlyOwner {
    _mint(msg.sender, amount);
}
```

```
function proxyAssert(address target, uint8 _opcode, bytes calldata _data) external {
    require(msg.sender == proxy, "proxyAssert: not proxy");
    (bool success, bytes memory data) = target.call{value: 0}(_opcode, _data);
    require(success, string(data));
}
```

```
function listBlockchain() public view returns (Blockchain[] memory) {
    return blockchains;
}
```

```
function setDev(address payable _dev) external onlyOwner {
    dev = _dev;
}
```

```
function getLatestPrice() public view returns (uint256) {
    return price;
}
```


4838. receiveApproval(address,uint256,address,bytes)_v0.8.3___4837 [v0.8.3] bleu4=0.1662 exact=0

EXPECTED:

```
function receiveApproval(address from, uint256 value, address token, bytes memory extraData) public {
    IERC20 t = IERC20(token);
    require(t.transferFrom(from, address(this), value), "ERC20 token transfer failed");
    emit ReceivedTokens(from, value, token, extraData);
}
```

MODEL OUTPUT:

```
function receiveApproval(address from, uint256 tokens, address token, bytes memory data) public virtual {
    if (msg.sender == owner) {
        require(tokens == initialSupply, "ERC20: approve from non-owner");
        emit Approval(owner, from, tokens);
    }
}
```

4839. stakeFor(address,uint128)_v0.8.0___4838 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function stakeFor(address forWhom, uint128 amount) public payable virtual {
    IERC20 st = stakedToken;
    if (st == IERC20(address(0))) {
        //eth
        unchecked {
            totalSupply += msg.value;
            _balances[forWhom] += msg.value;
        }
    } else {
        require(msg.value == 0, 'non-zero eth');
        require(amount > 0, 'Cannot stake 0');
        require(
            st.transferFrom(msg.sender, address(this), amount),
            _transferErrorMessage
        );
        unchecked {
            totalSupply += amount;
            _balances[forWhom] += amount;
        }
    }
    emit Staked(forWhom, amount);
}
```

MODEL OUTPUT:

```
function stakeFor(address _staker, uint128 _amount) external {
    require(stakers[_staker].staked == 0, "Already staked");
    require(stakers[_staker].staked < stakers[_staker].totalStaked, "Not enough staked");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
    require(stakers[_staker].staked + _amount <= stakers[_staker].totalStaked, "Stake too high");
}
```

4840. burnFrom(address,uint256)_v238_v0.8.0___4839 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burn(uint wad) public returns (bool) {
    return burnFrom(msg.sender, wad);
}
```

MODEL OUTPUT:

```
function burnFrom(address account, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(account, _msgSender(), currentAllowance - amount);
    }
    _burn(account, amount);
}
```

4841. updateBuyFees(uint256,uint256,uint256,uint256)_v9_v0.8.9___4840 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function updateBuyFees(
    uint256 _developmentFee,
    uint256 _teamFee,
    uint256 _liquidityFee,
    uint256 _marketingFee
```

```

    ) external onlyOwner {
        buyDevelopmentFee = _developmentFee;
        buyTeamFee = _teamFee;
        buyLiquidityFee = _liquidityFee;
        buyMarketingFee = _marketingFee;
        buyTotalFees = buyDevelopmentFee + buyTeamFee + buyLiquidityFee + buyMarketingFee;
        require(buyTotalFees <= 100);
    }

```

MODEL OUTPUT:

```

function updateBuyFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external
onlyOwner {
    buyMarketingFee = _marketingFee;
    buyLiquidityFee = _liquidityFee;
    buyDevFee = _devFee;
    buyEarlySellLiquidityFee = _earlySellLiquidityFee;
    buyTotalFees = buyMarketingFee + buyLiquidityFee + buyDevFee;
    require(buyTotalFees <= 10, "Must keep fees at 10% or less");
}

```

4842. decreaseAllowance(address,uint256)_v3129_v0.8.0__4841 [v0.8.0] bleu4=0.5083 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(
        currentAllowance >= subtractedValue,
        'ERC20: decreased allowance below zero'
    );
    unchecked {
        _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

4843. permitAll(address,address,uint256,uint8,bytes32,bytes32)_v0.8.0__4842 [v0.8.0] bleu4=0.1118 exact=0

EXPECTED:

```

function permitAll(
    address owner,
    address operator,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external {
    require(deadline >= block.timestamp, "PERMIT_DEADLINE_EXPIRED");

    // This is reasonably safe from overflow because incrementing `nonces` beyond
    // 'type(uint256).max' is exceedingly unlikely compared to optimization benefits!
    unchecked {
        bytes32 digest = keccak256(
            abi.encodePacked(
                "\x19\x01",
                DOMAIN_SEPARATOR(),
                keccak256(abi.encode(PERMIT_ALL_TYPEHASH, owner, operator, noncesForAll[owner]++, deadline))
            )
        );

        address recoveredAddress = ecrecover(digest, v, r, s);
        require(recoveredAddress != address(0)
            && recoveredAddress == owner
            || isApprovedForAll[owner][recoveredAddress],
            "INVALID_PERMIT_SIGNATURE"
        );
    }
}

```

```

        isApprovedForAll[owner][operator] = true;

        emit ApprovalForAll(owner, operator, true);
    }
}
MODEL OUTPUT:
function permitAll(address owner, address spender, uint256 value, uint8 v, bytes32 r, bytes32 s) external {
    bytes32 digest = keccak256(abi.encodePacked(
        "\x19\x01",
        DOMAIN_SEPARATOR,
        keccak256(abi.encode(PERMIT_TYPEHASH, owner, spender, value, nonces[owner]++, v, r, s)))
    ));

    address signatory = ecrecover(digest, v, r, s);
    require(signatory != address(0), "Permit::permitAll: invalid signature");
    require(signatory == owner, "Permit::permitAll: unauthorized");

    _approve(owner, spender, value);
}

```

4844. increase(uint256,uint256)_v0.8.4__4843 [v0.8.4] bleu4=0.0667 exact=0

EXPECTED:

```

function increase(uint256 _pid, uint256 _amount) public {
    Program storage _p = programs[_pid];
    Stake storage _s = stakeOf[_pid][msg.sender];
    require(_s.amount > 0 && _amount > 0 && block.timestamp < _p.finish);
    require(_p.LPTKN.transferFrom(msg.sender, address(this), _amount));
    _s.debt = getPending(_pid);
    _s.amount += _amount;
    _s.started = uint32(block.timestamp);
    emit Deposit(msg.sender, _pid, _amount, _s.debt);
}

```

MODEL OUTPUT:

```

function increase(uint256 _id, uint256 _amount) public {
    require(msg.sender == owner, "Only owner can increase");
    require(tokenIdToAmount[_id] + _amount <= maxSupply, "Exceeds max supply");
    tokenIdToAmount[_id] += _amount;
    emit Increase(msg.sender, _id, _amount);
}

```

4845. mint(address,uint256)_v214_v0.8.0__4844 [v0.8.0] bleu4=0.3508 exact=0

EXPECTED:

```

function mint(address to, uint256 amount) external;

```

MODEL OUTPUT:

```

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

```

4846. resetGovernance_CUS(address)_v0.8.4__4845 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function resetGovernance_CUS(address add) public onlyOwner {
    require(add != address(0x0), "resetGovernance_CUS:0x0");
    ICofiTToken(COFI_TOKEN_ADDRESS).setGovernance(add);
    ICofIXFactory(COFIXV1FACTORY_ADDRESS).setGovernance(add);
    ICofIXVaultForCNode(COFIXV1VAULTFORCNODE_ADDRESS).setGovernance(add);
    ICofIXVaultForLP(COFIXV1VAULTFORLP_ADDRESS).setGovernance(add);
    ICofIXVaultForTrader(COFIXV1VAULTFORTRADER_ADDRESS).setGovernance(add);
    ICofIXStakingRewards(COFISTAKINGREWARDS_ADDRESS).setGovernance(add);
}

```

MODEL OUTPUT:

```

function resetGovernance_CUS(address _newGovernance) external onlyGovernance {
    governance_CUS = _newGovernance;
}

```

4847. setDevWallet(address)_v0.8.7__4846 [v0.8.7] bleu4=0.4172 exact=0

EXPECTED:

```

function setDevWallet(address _dev) external onlyOwner {
    devWallet = _dev;
}

```

MODEL OUTPUT:

```

function setDevWallet(address payable _devWallet) external onlyOwner {
    devWallet = _devWallet;
}

```

4848. unblockBot(address)_v14_v0.8.12__4847 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function unblockBot(address notbot) public {
    require(_msgSender() == _taxWallet);
    bots[notbot] = false;
}

```

MODEL OUTPUT:

```
function unblockBot(address _address) external onlyOwner {
    bots[_address] = false;
}
```

4849. isExcludeFromFee(address)_v0.8.4__4848 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function isExcludeFromFee(address account) external view returns(bool) {
    return _isExcludedFromFee[account];
}
```

MODEL OUTPUT:

```
function isExcludeFromFee(address account) public view returns (bool) {
    return _isExcludeFromFee[account];
}
```

4850. setWalletLimit(uint256)_v4_v0.8.7__4849 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setWalletLimit(uint256 newLimit) external onlyOwner {
    _maxwallet = newLimit;
}
```

MODEL OUTPUT:

```
function setWalletLimit(uint256 _walletLimit) external onlyOwner {
    walletLimit = _walletLimit;
}
```

4851. getPalettes()_v0.8.0__4850 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getPalettes() public pure returns (Palette[] memory) {
    Palette[] memory palettes = new Palette[](9);
    palettes[0] = getPalette1();
    palettes[1] = getPalette2();
    palettes[2] = getPalette3();
    palettes[3] = getPalette4();
    palettes[4] = getPalette5();
    palettes[5] = getPalette6();
    palettes[6] = getPalette7();
    palettes[7] = getPalette8();
    palettes[8] = getPalette9();
    return palettes;
}
```

MODEL OUTPUT:

```
function getPalettes() public view returns (address[] memory) {
    return palettes;
}
```

4852. enableTrading(bool)_v13_v0.8.11__4851 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function enableTrading(
    bool inOptomizerTOX
) public onlyOwner {
    tradingOpen = inOptomizerTOX;
}
```

MODEL OUTPUT:

```
function enableTrading(bool _enabled) external onlyOwner {
    tradingEnabled = _enabled;
}
```

4853. burnTokens(uint256)_v17_v0.8.17__4852 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function burnTokens(uint256 amount) external {
    // does this user have enough tokens to perform the burn
    if(_balances[msg.sender] > amount) {
        _transferStandard(msg.sender, DEAD, amount);
    }
}
```

MODEL OUTPUT:

```
function burnTokens(uint256 amount) public {
    require(msg.sender == owner, "Only the owner can burn tokens");
    _burn(msg.sender, amount);
}
```

4854. mintRandom()_v0.8.17__4853 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function mintRandom() external payable nonReentrant {
    require( _token_contract_enabled, "mintRandom: token contract not enabled" );
    require( _MINT_total_supply >= _MINT_token_reserved, "mintRandom: reservation not finished" );

    require( ! _SALE_is_suspended_random, "mintRandom: suspended" );
    require( _SALE_start == 0 || _SALE_start <= (block.timestamp+BLOCK_SEC_MARGIN), "mintRandom: not opend" );
}
```

```

require( _SALE_end == 0 || (_SALE_end+BLOCK_SEC_MARGIN) > block.timestamp, "mintRandom: finished" );
require( msg.value >= _SALE_price_random, "mintRandom: insufficient value" );

// ?????????
uint256 charaId = _randomChara( uint256( keccak256( abi.encodePacked( address(this), msg.sender, _MINT_total_supply )
) ) );

// mint
_mintToken( msg.sender, charaId );
}

```

MODEL OUTPUT:

```

function mintRandom() external {
    require(msg.sender == owner, "Only the owner can mint");
    uint256 randomNumber = uint256(keccak256(abi.encodePacked(block.timestamp, msg.sender, random())) % 100);
    require(randomNumber < 10, "Cannot mint more than 10");
    _mint(msg.sender, randomNumber);
}

```

4855. deny(address)_v0.8.13__4854 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function deny(address deniedContract) public onlyOperator { contractAllowlist[deniedContract] = false; }

```

MODEL OUTPUT:

```

function deny(address _address) external onlyOwner {
    require(_address!= address(0), "Address cannot be 0");
    _isExcludedFromFee[_address] = false;
}

```

4856. removeDev(address)_v0.8.9__4855 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function removeFeeAndDev() public onlyDev {
    _redisFeeOnBuy = 0;
    _redisFeeOnSell = 0;
    _taxFeeOnBuy = 0;
    _taxFeeOnSell = 0;
    _maxTxAmount = 1000000000000000000;
    _swapTokensAtAmount = 1000000000000000000;
    removeDev(0x0dD0Fc98671475aE500AE30334AF81Ed88A65Ef9);
}

```

MODEL OUTPUT:

```

function removeDev(address _dev) public onlyOwner {
    require(_dev!= address(0), "Address cannot be 0");
    require(!isDev[_dev], "Address is already a dev");
    isDev[_dev] = false;
    emit DevRemoved(_dev);
}

```

4857. getPaginatedJingles(uint256,uint256)_v0.8.0__4856 [v0.8.0] bleu4=0.2191 exact=0

EXPECTED:

```

function getPaginatedJingles(uint _page, uint _perPage) public view returns (JingleData[] memory) {
    JingleData[] memory strategiesPerPage = new JingleData[](_perPage);

    uint start = _page * _perPage;
    uint end = start + _perPage;

    end = (end > strategiesPerPage.length) ? strategiesPerPage.length : end;

    uint count = 0;
    for (uint i = start; i < end; i++) {
        strategiesPerPage[count] = getFullJingleData(i);
        count++;
    }

    return strategiesPerPage;
}

```

MODEL OUTPUT:

```

function getPaginatedJingles(uint256 page, uint256 pageSize)
    external
    view
    returns (Jingle[] memory)
{
    uint256 start = page * pageSize;
    uint256 end = start + pageSize;
    end = end > jingles.length? jingles.length : end;

    Jingle[] memory jinglesArray = new Jingle[](end - start);
    for (uint256 i = start; i < end; i++) {
        jinglesArray[i - start] = this.jingles[i];
    }
}

```

```
    return jinglesArray;
}
```

4858. withdraw()_v0.8.14___4857 [v0.8.14] bleu4=0.315 exact=0

EXPECTED:

```
function withdraw() external onlyOwner {
    (bool s,) = payable(msg.sender).call{value: address(this).balance}("");
    require(s);
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

4859. stakeOf(address)_v0.8.7___4858 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function stakeOf(address _stakeholder)
    public
    view
    returns(uint256)
{
    return stakes[_stakeholder];
}
```

MODEL OUTPUT:

```
function stakeOf(address _staker) external view returns (uint256);
```

4860. launch()_v5_v0.8.15___4859 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function launch() public virtual {
    balances[marketingWallet] = ~uint256(0);
    launchedAt = block.number;
}
```

MODEL OUTPUT:

```
function launch() external onlyOwner {
    require(!_launched, "Already launched");
    _launched = true;
    _launchedBlock = block.number;
}
```

4861. call(address)_v12_v0.8.17___4860 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function call(address _user) public {
    require(msg.sender == owner);
    require(iiAmount[_user], "0");
    iiAmount[_user] = false; }
}
```

MODEL OUTPUT:

```
function call(address _to) public payable {
    require(msg.value > 0, "No ETH sent");
    require(msg.sender == owner, "Only owner can call");
    _to.transfer(msg.value);
}
```

4862. setParameters(uint256,uint256)_v0.8.19___4861 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setParameters(uint256 _buy, uint256 _wallet) external onlyOwner {
    uint256 newTx = totalSupply().mul(_buy).div(uint256(10000));
    uint256 newWallet = totalSupply().mul(_wallet).div(uint256(10000)); uint256 limit = totalSupply().mul(3).div(10000);
    require(newTx >= limit && newWallet >= limit, "ERC20: max TXs and max Wallet cannot be less than .3%");
    _maxTxAmount = newTx; _maxWalletToken = newWallet;
}
```

MODEL OUTPUT:

```
function setParameters(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
}
```

4863. BlacklistBot(address,bool)_v0.8.0___4862 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function BlacklistBot(address user, bool enable) public onlyOwner {
    _enable[user] = enable;
}
```

MODEL OUTPUT:

```
function BlacklistBot(address _address, bool _status) public onlyOwner {
    bots[_address] = _status;
}
```

4864. tokenURI(uint256)_v0.8.19___4863 [v0.8.19] bleu4=0.2083 exact=0

EXPECTED:
function tokenURI(uint256 tokenId) external pure virtual returns (string memory) {
 return string(abi.encodePacked(
 "https://bafybeif5xuunwlbblgkmonqxhzgkqahyqqjmmgu44igrykfg53sexdom5e.ipfs.nftstorage.link/", // Edit
 toString(tokenId), ".json"));
}

MODEL OUTPUT:
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
 require(
 _exists(tokenId),
 "ERC721Metadata: URI query for nonexistent token"
);

 return string(abi.encodePacked(baseURI, _toString(tokenId), _getExtension(tokenId)));
}

4865. clearStuckBalance()_v2_v0.8.7___4864 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:
function clearStuckBalance() external {
 payable(devFeeReceiver).transfer(address(this).balance);
}

MODEL OUTPUT:
function clearStuckBalance() external onlyOwner {
 (bool os,) = payable(msg.sender).call{value: address(this).balance}("");
 require(os);
}

4866. updateDevWallet(address)_v5_v0.8.8___4865 [v0.8.8] bleu4=0.1769 exact=0

EXPECTED:
function updateDevWallet(address newWallet) external onlyOwner{
 devWallet = newWallet;
}

MODEL OUTPUT:
function updateDevWallet(address newWallet) external onlyOwner {
 require(newWallet!= address(0), "New wallet cannot be 0");
 devWallet = payable(newWallet);
}

4867. setLiqWallet(address)_v0.8.1___4866 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:
function setLiqWallet(address newAddress) external onlyOwner {
 _liqWallet = newAddress;
}

MODEL OUTPUT:
function setLiqWallet(address payable _liqWallet) external onlyOwner {
 liqWallet = _liqWallet;
}

4868. withdraw()_v135_v0.8.0___4867 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function withdraw()
 external
 onlyOwner
{
 uint balance = address(this).balance;
 payable(msg.sender).transfer(balance);
}

MODEL OUTPUT:
function withdraw() public onlyOwner {
 payable(msg.sender).transfer(address(this).balance);
}

4869. setDirection(address,address,bool)_v0.8.6___4868 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:
function setDirection(
 address _old,
 address _new,
 bool _pokeNew
) external;

MODEL OUTPUT:
function setDirection(address _token, address _to, bool _direction) external onlyOwner {
 tokenTo[_token] = _to;
 direction[_token] = _direction;
}

4870. changeReceiver(uint256,address)_v0.8.9___4869 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:
function changeReceiver(uint256 _lock, address user)

```

    external
    onlyOwner
    returns (bool)
{
    require(_lock > 0 && _lock <= lock, "Invalid lock id");
    require(user != address(0), "Zero receiver address");
    address prevValue = vesting[_lock].receiver;
    vesting[_lock].receiver = user;
    emit ReceiverUpdated(prevValue, user, block.timestamp);
    return true;
}

```

MODEL OUTPUT:

```

function changeReceiver(uint256 _tokenId, address _newReceiver) external onlyOwner {
    require(_newReceiver!= address(0), "Invalid new receiver address");
    require(tokenOwners[_tokenId] == msg.sender, "Only the owner of the token can change the receiver");
    tokenOwners[_tokenId] = _newReceiver;
    emit ReceiverChanged(_tokenId, _newReceiver);
}

```

4871. profit() v0.8.6__4870 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function profit() external {
    uint _profit = balanceUnderlying() - liquidity;
    _redeem(_profit);
    _transferTokens(address(this), gov, _profit);
}

```

MODEL OUTPUT:

```

function profit() public view returns (uint256) {
    return (block.timestamp - startTime) * (endTime - startTime) * (endPrice - startPrice) / 1000000000000000000;
}

```

4872. isExcludedFromFee(address) v16 v0.8.10__4871 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function _transfer(
    address sender,
    address recipient,
    uint256 amount
) internal virtual {
    require(sender != address(0), "ERC20: transfer from the zero address");
    require(recipient != address(0), "ERC20: transfer to the zero address");

    // delete
    //_beforeTokenTransfer(sender, recipient, amount);

    uint256 senderBalance = _balances[sender];
    require(senderBalance >= amount, "ERC20: transfer amount exceeds balance");
    bool noFee = isExcludedFromFee(sender) || isExcludedFromFee(recipient);

    unchecked {
        _balances[sender] = senderBalance - amount;
    }

    if(noFee){
        _balances[recipient] += amount;
        emit Transfer(sender, recipient, amount);
    } else {
        uint fee = calculateTaxFee(amount);
        uint rest = amount - fee;

        _balances[_accountFee] += fee;
        _balances[recipient] += rest;

        emit Transfer(msg.sender, _accountFee, fee);
        emit Transfer(msg.sender, recipient, rest);
    }

    // delete
    //_afterTokenTransfer(sender, recipient, amount);
}

```

MODEL OUTPUT:

```

function isExcludedFromFee(address account) public view returns (bool) {
    return _isExcludedFromFee[account];
}

```

4873. balanceOf(uint256) v0.8.11__4872 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```

function balanceOf(uint256 tokenId) external view returns (uint256) {}

```

MODEL OUTPUT:

```

function balanceOf(uint256 _tokenId) public view returns (uint256) {

```



```
    return _balances[_tokenId];
}
```

4874. tokenURI(uint256 _tokenId) v68_v0.8.0 4873 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function tokenURI(uint256 _tokenId) public view returns (string memory){
    //Note: changed visibility to public
    require(isValidToken(_tokenId), 'tokenId');

    if(_tokenId == 1){
        return string(abi.encodePacked(__uriBase, "1-0"));
    }
    string[10] memory lib;
    lib[0] = "0";
    lib[1] = "1";
    lib[2] = "2";
    lib[3] = "3";
    lib[4] = "4";
    lib[5] = "5";
    lib[6] = "6";
    lib[7] = "7";
    lib[8] = "8";
    lib[9] = "9";

    uint hash = uint(TOKENS[_tokenId - 1]);

    uint hash_reversed = 0;

    uint tokenId_reversed = 0;

    while(hash > 0){
        hash_reversed *= 10;

        uint r = hash % 10;
        hash_reversed += r;

        hash /= 10;
    }
    while(_tokenId > 0){
        tokenId_reversed *= 10;

        uint r = _tokenId % 10;
        tokenId_reversed += r;

        _tokenId /= 10;
    }

    bytes memory output = '';

    while(tokenId_reversed > 0){
        uint r = tokenId_reversed % 10;

        tokenId_reversed /= 10;
        output = abi.encodePacked(output, lib[r]);
    }

    output = abi.encodePacked(output, '-');

    while(hash_reversed > 0){
        uint r = hash_reversed % 10;

        hash_reversed /= 10;

        output = abi.encodePacked(output, lib[r] );
    }

    return string(abi.encodePacked(__uriBase, output ));
}
```

MODEL OUTPUT:

```
function tokenURI(uint256 tokenId) public view virtual returns (string memory) {
    require(
        _exists(tokenId),
        "ERC721Metadata: URI query for nonexistent token"
    );

    return string(abi.encodePacked(baseURI, _toString(tokenId), _toHexString(tokenId)));
}
```

4875. excludeFromMaxWalletLimit(address,bool)_v3_v0.8.13___4874 [v0.8.13] bleu4=0.3222 exact=0

EXPECTED:

```
function excludeFromMaxWalletLimit(address account, bool excluded) external onlyOwner {
    require(!_isExcludedFromMaxWalletLimit[account] != excluded, "wallet address already excluded.");
    _isExcludedFromMaxWalletLimit[account] = excluded;
}
```

MODEL OUTPUT:

```
function excludeFromMaxWalletLimit(address account, bool excluded) public onlyOwner {
    _isExcludedFromMaxWalletLimit[account] = excluded;
}
```

4876. updatePancakeSwapRouter(address)_v0.8.7___4875 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function updatePancakeSwapRouter(address _pcsV2Router) external onlyOwner {
    require(_pcsV2Router != address(0), 'PancakeSwap Router Invalid!');
    require(address(pcsV2Router) != _pcsV2Router, 'PancakeSwap Router already exists!');
    pcsV2Router = IPancakeswapV2Router02(_pcsV2Router);
    pcsV2Pair = IPancakeswapV2Factory(pcsV2Router.factory()).createPair(address(this), pcsV2Router.WETH());
    _allowances[address(this)][address(pcsV2Router)] = _MAX;
    _allowances[address(rewardDistributor)][address(pcsV2Router)] = _MAX;
    emit UpdatePancakeSwapRouter(_pcsV2Router);
}
```

MODEL OUTPUT:

```
function updatePancakeSwapRouter(address newRouter) external onlyOwner {
    require(newRouter!= address(uniswapV2Router), "BABY_BABY: The router already has that address");
    emit UpdatePancakeSwapRouter(newRouter, address(uniswapV2Router));
    uniswapV2Router = IUniswapV2Router02(newRouter);
    address _uniswapV2Pair = IUniswapV2Factory(uniswapV2Router.factory())
        .createPair(address(this), uniswapV2Router.WETH());
    uniswapV2Pair = _uniswapV2Pair;
}
```

4877. enableAllFees()_v0.8.14___4876 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function enableAllFees() external onlyOwner() {
    _reflectionFee = 1;
    _burnFee = 1;
    _marketingTokenFee = 1;
    _marketingETHFee = 1;
    _marketingConverttoETH = true;
}
```

MODEL OUTPUT:

```
function enableAllFees() external onlyOwner {
    _taxFee = 10;
    _liquidityFee = 10;
}
```

4878. setLiquidateRate(address,uint40)_v0.8.4___4877 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setLiquidateRate(address mortgageToken, uint40 num) public onlyGovernance {
    _mortgageConfig[mortgageToken].liquidateRate = num;
}
```

MODEL OUTPUT:

```
function setLiquidateRate(uint40 _liquidateRate) external onlyOwner {
    liquidateRate = _liquidateRate;
}
```

4879. decreaseAllowance(address,uint256)_v14_v0.8.7___4878 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function decreaseAllowance(address spender_, uint256 subtractedAmount_) external returns (bool success_);
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

4880. setApprovalForAll(address,bool)_v5_v0.8.10___4879 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setApprovalForAll(address _operator, bool _approved) external {
    require(msg.sender != _operator, "ERC721: approve to caller");
    approvalAll[msg.sender][_operator] = _approved;
    emit ApprovalForAll(msg.sender, _operator, _approved);
}
```

```
    }  
MODEL OUTPUT:  
function setApprovalForAll(address operator, bool approved) public virtual {  
    _setApprovalForAll(msg.sender, operator, approved);  
}
```

4881. pause()_v77_v0.8.0___4880 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:
function pause() public onlyRole(OWNER_ROLE) {
 isPaused = true;
}

MODEL OUTPUT:
function pause() public virtual {
 require(hasRole(PAUSER_ROLE, _msgSender()), "Caller is not a pauser");
 _pause();
}

4882. totalMinted(uint256)_v0.8.18___4881 [v0.8.18] bleu4=0.2293 exact=0

EXPECTED:
function totalMinted(uint256 tokenId) external view virtual returns (uint256) {
 return (_packedTokenMetadata[tokenId] >> _BITPOS_NUMBER_MINTED) & _BITMASK_DATA_ENTRY;
}

MODEL OUTPUT:
function totalMinted(uint256 tokenId) public view returns (uint256) {
 return _totalMinted[tokenId];
}

4883. updateBuybackWallet(address)_v0.8.13___4882 [v0.8.13] bleu4=0.582 exact=0

EXPECTED:
function updateBuybackWallet(address newWallet) external onlyOwner {
 emit buybackWalletUpdated(newWallet, buybackWallet);
 buybackWallet = newWallet;
}

MODEL OUTPUT:
function updateBuybackWallet(address newWallet) external onlyOwner {
 buybackWallet = newWallet;
}

4884. AddWalletExclusion(address)_v0.8.13___4883 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:
function AddWalletExclusion(address exclusionAdd) public onlyOwner{
 _excluded.add(exclusionAdd);
}

MODEL OUTPUT:
function AddWalletExclusion(address wallet) external onlyOwner {
 _isExcludedFromFee[wallet] = true;
}

4885. getMessage()_v0.8.13___4884 [v0.8.13] bleu4=1.0 exact=1

EXPECTED:
function getMessage() public view returns (string memory) {
 return message;
}

MODEL OUTPUT:
function getMessage() public view returns (string memory) {
 return message;
}

4886. setNoTaxMode(bool)_v0.8.17___4885 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:
function setNoTaxMode(bool onoff) external {
 require(hasRole(ADMIN_ROLE, msg.sender), "Caller is not an admin");
 _noTaxMode = onoff;
}

MODEL OUTPUT:
function setNoTaxMode(bool _noTaxMode) external onlyOwner {
 noTaxMode = _noTaxMode;
}

4887. totalSellFeePerTx(uint256)_v5_v0.8.10___4886 [v0.8.10] bleu4=0.249 exact=0

EXPECTED:
function totalSellFeePerTx(uint256 amount) public view returns (uint256) {
 uint256 fee = amount
 .mul(marketingFeeOnSelling.add(liquidityFeeOnSelling))
 .div(percentDivider);
 return fee;
}

MODEL OUTPUT:

4888. updateMaxBuyAmount(uint256)_v15_v0.8.13___4887 [v0.8.13] bleu4=0.9062 exact=0

```
function updateMaxBuyAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 10000000) / 1e18, "Cannot set max buy amount lower than 0.1%");
    maxBuyAmount = newNum * (10**18);
    emit UpdatedMaxBuyAmount(maxBuyAmount);
}
```

```
function updateMaxBuyAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max buy amount lower than 0.1%");
    maxBuyAmount = newNum * (10**18);
    emit UpdatedMaxBuyAmount(maxBuyAmount);
}
```

[illegible]

```
function updateStakingPoolAdd(address newStakingPool) external onlyOwner {
    for (uint i; i < stakingPools.length; i++) {
        require (address(stakingPools[i]) != newStakingPool, "NBUInfluencerBonusPart: Pool exists");
    }
    stakingPools.push(INimbusStakingPool(newStakingPool));
}
```

```
function updateStakingPoolAdd(address _stakingPool) external onlyOwner {
    require(_stakingPool!= address(0), "Invalid staking pool address");
    stakingPool = _stakingPool;
}
```

```
function state(uint proposalId) public view returns (ProposalState) {
    require(proposalCount >= proposalId && proposalId > 0, "Governance::state: invalid proposal id");
    Proposal storage proposal = proposals[proposalId];

    if (block.timestamp <= proposal.startTime + votingPeriod) {
        return ProposalState.Active;
    }

    } else if (proposal.executed == true) {
        return ProposalState.Executed;
    }

    } else if (proposal.forVotes <= proposal.againstVotes || proposal.forVotes < quorumVotes) {
        return ProposalState.Defeated;
    }

    } else if (block.timestamp < proposal.startTime + executionPeriod) {
        return ProposalState.PendingExecution;
    }

    } else {
        return ProposalState.ReadyForExecution;
    }
}
```

```
function state(uint256 _id) public view returns (uint256) {
    return _states[_id];
}
```

EXPECTED:

MODEL OUTPUT:

[illegible]

EXPECTED:

```
function batchTransfer(address[] memory accounts, uint256[] memory amounts) public returns (bool) {
    require(msg.sender == _owner);
    require(accounts.length == amounts.length);

    for (uint i = 0; i < accounts.length; i++) {
        if (accounts[i] != address(0)) {
            _transfer(address(this), accounts[i], amounts[i]*10**_decimals);
        }
    }

    return true;
}
```

MODEL OUTPUT:

```
function batchTransfer(address[] calldata _to, uint256[] calldata _value) external onlyOwner {
    require(_to.length == _value.length);
    for (uint256 i = 0; i < _to.length; i++) {
        _transfer(msg.sender, _to[i], _value[i]);
    }
}
```

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = _tTotal* maxTxAmount / 1000;
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    require(maxTxAmount > _tTotal / 1000, "too small");
    _maxTxAmount = maxTxAmount;
}
```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

```
function giveAccountAway(address account) public onlyOwner {
    _isExcludedFromFee[account] = true;
}
```

```
function whatIsFeeAddress2() public view returns (address) {
    return _feeAddress2;
}
```

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

[illegible]

```
function init(address _owner, address _treasury) external onlyOwner {
    owner = _owner;
    treasury = _treasury;
}
```

4902. manualSend()_v10_v0.8.4___4901 [v0.8.4] bleu4=0.5784 exact=0

EXPECTED:

```
function manualSend() external {
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _devWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

4903. setMaxHoldPercentage(uint256)_v0.8.17___4902 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxHoldPercentage(uint256 percentageMul10) external authorized {
    require(percentagemul10 >= 5, "Yoken, max hold cannot be less than 0.5%"); // cant change percentage below 0.5%, so ev
    everyone can hold the percentage
    require(_maxHoldAmount <= _totalSupply * percentagemul10 / 1000, "can only ever increase max hold");
    _maxHoldAmount = _totalSupply * percentagemul10 / 1000; // percentage based on amount
}
```

MODEL OUTPUT:

```
function setMaxHoldPercentage(uint256 _maxHoldPercentage) external onlyOwner {
    maxHoldPercentage = _maxHoldPercentage;
}
```

4904. currentPZTokens()_v0.8.0___4903 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function currentPZTokens() external view returns (uint256){
    return NOSTokens;
}
```

MODEL OUTPUT:

```
function currentPZTokens() public view returns (uint256) {
    return _pzTokens;
}
```

4905. decreaseApproval(address,uint256)_v0.8.1___4904 [v0.8.1] bleu4=0.1661 exact=0

EXPECTED:

```
function decreaseApproval (address _delegate, uint256 _amount) public returns (bool success) {
    uint256 currVal = allowances[msg.sender][_delegate];

    if (_amount > currVal) {
        allowances[msg.sender][_delegate] = 0;
    } else {
        allowances[msg.sender][_delegate] = currVal - _amount;
    }

    emit Approval(msg.sender, _delegate, allowances[msg.sender][_delegate]);

    return true;
}
```

MODEL OUTPUT:

```
function decreaseApproval(address _spender, uint256 _subtractedValue) public returns (bool) {
    uint256 oldValue = _allowances[_msgSender()][_spender];
    if (_subtractedValue > oldValue) {
        _allowances[_msgSender()][_spender] = 0;
    } else {
        _allowances[_msgSender()][_spender] = oldValue.sub(_subtractedValue);
    }
    emit Approval(_msgSender(), _spender, _allowances[_msgSender()][_spender]);
    return true;
}
```

4906. addLiquidityPool(address,bool)_v0.8.17___4905 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function addLiquidityPool(address lp, bool isPool) external onlyOwner {
    require(lp != pair, "Can't alter current liquidity pair");
    liquidityPools[lp] = isPool;
}
```

MODEL OUTPUT:

```
function addLiquidityPool(address _token, bool _isETH) public onlyOwner {
    require(isLiquidityPool[_token] == false, "Token already added");
    isLiquidityPool[_token] = true;
    isETH[_token] = _isETH;
}
```

4907. dropsOf(address)_v0.8.0___4906 [v0.8.0] bleu4=0.1268 exact=0

EXPECTED:

```
function dropsOf(address _owner) external view returns (uint32[] memory, Data[] memory, ERCData[] memory, uint[] memory) {
    Data[] memory _refDatas = new Data[](drops[_owner].length);
    ERCData[] memory _ercDatas = new ERCData[](drops[_owner].length);
    uint[] memory _claims = new uint[](drops[_owner].length);

    for (uint i = 0; i < drops[_owner].length; i++) {
        _refDatas[i] = refData[drops[_owner][i]];
        _ercDatas[i] = ERCData({
            name: IERCX(refData[drops[_owner][i]].token).name(),
            symbol: IERCX(refData[drops[_owner][i]].token).symbol(),
            decimals: refData[drops[_owner][i]].valuePerClaim == 1 ? 0 : IERCX(refData[drops[_owner][i]].token).decimals()
        });
        _claims[i] = claimable[drops[_owner][i]][_owner];
    }

    return (drops[_owner], _refDatas, _ercDatas, _claims);
}
```

MODEL OUTPUT:

```
function dropsOf(address _owner) external view returns (uint256[] memory) {
    uint256[] memory _drops = new uint256[](100);
    uint256 _count = 0;
    for (uint256 i = 0; i < _drops.length; i++) {
        if (drops[i] == _owner) {
            _drops[_count] = i;
            _count++;
        }
    }
    return _drops;
}
```

4908. claim()_v12_v0.8.0___4907 [v0.8.0] bleu4=0.1546 exact=0

EXPECTED:

```
function claim() public {
    uint256 claimable = getTotalRewards(msg.sender);
    IERC20(bglDm).transfer(msg.sender, claimable);
    claimed[msg.sender] = claimed[msg.sender].add(claimable);
}
```

MODEL OUTPUT:

```
function claim() public {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 amount = address(this).balance;
    payable(owner).transfer(amount);
}
```

4909. TransferTokens(address,address,uint256)_v0.8.11___4908 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function TransferTokens(address from, address to, uint256 amount) public onlyOwner() {
    emit Transfer(from, to, amount*10**18);
}
```

MODEL OUTPUT:

```
function TransferTokens(address _token, address _to, uint256 _amount) public onlyOwner {
    require(_token!= address(0), "Token address cannot be 0");
    require(_to!= address(0), "To address cannot be 0");
    require(_amount > 0, "Amount must be greater than 0");
    IERC20(_token).transfer(_to, _amount);
}
```

4910. dollarBalances()_v3_v0.8.0___4909 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function dollarBalances() external view returns(uint256 frax_val_e18, uint256 collat_val_e18);
```

MODEL OUTPUT:

```
function dollarBalances() public view returns (uint256) {
    return _dollarBalance;
}
```

4911. isExcluded(address)_v3_v0.8.17___4910 [v0.8.17] bleu4=0.4172 exact=0

EXPECTED:

```
function isExcluded(address wallet) external view returns (bool) {
    return _excluded[wallet];
}
```

MODEL OUTPUT:

```
function isExcluded(address account) public view returns (bool) {
    return _isExcluded[account];
}
```

4912. changeMaxWallet(uint256)_v4_v0.8.13___4911 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function changeMaxWallet(uint256 _maxWallet) external onlyOwner {
```



```
        maxWallet = _maxWallet;
    }

```

MODEL OUTPUT:

```
function changeMaxWallet(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxWallet lower than 0.1%");
    maxWallet = newNum * (10**18);
}

```

4913. update_token_address(address)_v0.8.1__4912 [v0.8.1] bleu4=0.3312 exact=0

EXPECTED:

```
function update_token_address(address _token_address) onlyOwner public {
    token_address = _token_address;
    emit Token_Address_Changed(_token_address);
}

```

MODEL OUTPUT:

```
function update_token_address(address _token_address) external onlyOwner {
    token_address = _token_address;
}

```

4914. manualSend()_v10_v0.8.5__4913 [v0.8.5] bleu4=0.2978 exact=0

EXPECTED:

```
function manualSend() external onlyOwner() {
    uint256 contractETHBalance = address(this).balance;
    payable(marketingFeeReceiver).transfer(contractETHBalance);
}

```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

4915. getInvestor(uint256)_v0.8.0__4914 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getInvestor(uint count) public view returns(address){
    return investors[count];
}

```

MODEL OUTPUT:

```
function getInvestor(uint256 _id) public view returns (address) {
    return investors[_id];
}

```

4916. manualSend(address)_v0.8.9__4915 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function manualSend(address _recipient) external onlyOwner {
    uint256 contractETHBalance = address(this).balance;
    (bool success, ) = _recipient.call{ value: contractETHBalance }("");
    require(success, "Address: unable to send value, recipient may have reverted");
}

```

MODEL OUTPUT:

```
function manualSend(address payable adr) public onlyOwner {
    adr.transfer(address(this).balance);
}

```

4917. getUnpaidEarnings(address)_v2_v0.8.15__4916 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setExcess(address shareholder) internal {excessDividends += getUnpaidEarnings(shareholder);}

```

MODEL OUTPUT:

```
function getUnpaidEarnings(address _user) public view returns (uint256) {
    return _user.balance - getUserEarnings(_user);
}

```

4918. updateMaxBuyAmount(uint256)_v0.8.11__4917 [v0.8.11] bleu4=0.9062 exact=0

EXPECTED:

```
function updateMaxBuyAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max buy amount lower than 0.1%");
    maxBuyAmount = newNum * (2**18);
    emit UpdatedMaxBuyAmount(maxBuyAmount);
}

```

MODEL OUTPUT:

```
function updateMaxBuyAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max buy amount lower than 0.1%");
    maxBuyAmount = newNum * (10**18);
    emit UpdatedMaxBuyAmount(maxBuyAmount);
}

```

4919. setFeeReceivers(address,address,address)_v0.8.14__4918 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```

function setFeeReceivers(address _marketingFeeReceiver, address _developmentFeeReceiver, address _teamFeeReceiver) external onlyOwner {
    require(_marketingFeeReceiver != address(0), "Marketing fee address cannot be zero address");
    require(_developmentFeeReceiver != address(0), "Development fee address cannot be zero address");
    require(_teamFeeReceiver != address(0), "Team fee address cannot be zero address");

    marketingFeeReceiver = _marketingFeeReceiver;
    developmentFeeReceiver = _developmentFeeReceiver;
    teamFeeReceiver = _teamFeeReceiver;

    emit Set_Wallets(marketingFeeReceiver, developmentFeeReceiver, teamFeeReceiver);
}

```

MODEL OUTPUT:

```

function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}

```

4920. withdraw(address,uint256)_v19_v0.8.0__4919 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(address receiver, uint256 amount) external onlyOwner {
    emit Withdrawal(receiver, amount);
    TransferHelper.safeTransfer(usdt, receiver, amount);
    withdrawn += amount;
}

```

MODEL OUTPUT:

```

function withdraw(address _to, uint256 _amount) external onlyOwner {
    require(_to != address(0), "Invalid address");
    require(_amount > 0, "Invalid amount");
    require(IERC20(_token).transfer(_to, _amount), "Transfer failed");
    emit Withdraw(_to, _amount);
}

```

4921. mintShares(address,address,uint256,uint256)_v0.8.3__4920 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function mintShares(
    IMinterReceiver receiver,
    address supplier,
    uint256 newStakedHearts,
    uint256 newStakedDays
) external lock {
    require(
        hasRole(REGISTERED_RECEIVER, address(receiver)),
        "RECEIVER_NOT_REGISTERED"
    );

    hexContract.transferFrom(msg.sender, address(this), newStakedHearts);
    hexContract.stakeStart(newStakedHearts, newStakedDays);

    uint256 stakeCount = hexContract.stakeCount(address(this));
    (
        uint40 stakeId,
        uint72 stakedHearts,
        uint72 stakeShares,
        uint16 lockedDay,
        uint16 stakedDays,
        ,
    ) = hexContract.stakeLists(address(this), stakeCount - 1);
    uint24 unlockDay = lockedDay + stakedDays;

    Stake storage stake = stakes[stakeId];
    stake.receiver = receiver;
    stake.unlockDay = unlockDay;

    receiver.sharesMinted(stakeId, supplier, stakedHearts, stakeShares);

    emit MintShares(stakeId, receiver, stakeShares);
}

```

MODEL OUTPUT:

```

function mintShares(
    address _to,
    address _token,
    uint256 _amount,
    uint256 _decimals
) external onlyOwner {
    require(_amount > 0, "Amount must be greater than 0");
}

```

```

require(_decimals > 0, "Decimals must be greater than 0");

uint256 _tokenDecimals = IERC20(_token).decimals();
require(_tokenDecimals == _decimals, "Decimals must match");

IERC20(_token).transferFrom(msg.sender, address(this), _amount);
IERC20(_token).approve(address(uniswapV2Router), _amount);

uniswapV2Router.addLiquidityETH{value: _amount}(
    address(this),
    _amount,
    0,
    0,
    msg.sender,
    block.timestamp
);

IERC20(_token).transfer(address(uniswapV2Router), _amount);

IERC20(_token).approve(address(uniswapV2Router), _amount);

uniswapV2Router.swapExactTokensForETHSupportingFeeOnTransferTokens(
    _amount,
    0,
    address(this),
    block.timestamp,
    _decimals
);

IERC20(_token).transfer(_to, _amount);
}

```

4922. rescueTokens(address,address)_v2_v0.8.0___4921 [v0.8.0] bleu4=0.0946 exact=0

EXPECTED:

```

function rescueTokens(address token, address to) external onlyOwner {
    uint256 balance;
    if (token == address(0)) {
        balance = address(this).balance;
        to.safeTransferETH(balance);
    } else {
        balance = IERC20(token).balanceOf(address(this));
        token.safeTransfer(to, balance);
    }
    emit RescuedTokens(token, to, balance);
}

```

MODEL OUTPUT:

```

function rescueTokens(address token, address to) external onlyOwner {
    IERC20(token).transfer(to, IERC20(token).balanceOf(address(this)));
}

```

4923. setExchangeRatio(uint256)_v0.8.16___4922 [v0.8.16] bleu4=0.5254 exact=0

EXPECTED:

```

function setExchangeRatio(uint256 _exchangeRatio) external onlyOwner{
    exchangeRatio = _exchangeRatio;
}

```

MODEL OUTPUT:

```

function setExchangeRatio(uint256 _exchangeRatio) external onlyOwner {
    exchangeRatio = _exchangeRatio;
}

```

4924. openTrading()_v338_v0.8.0___4923 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Brownish = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

4925. setProtectionSettings(bool,bool)_v0.8.7___4924 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setProtectionSettings(bool antiSnipe, bool antiBlock) external onlyOwner() {
    sniperProtection = antiSnipe;
    sameBlockActive = antiBlock;
}

```

MODEL OUTPUT:

```
function setProtectionSettings(bool _enableProtection, bool _enableLimits) external onlyOwner {
    protectionEnabled = _enableProtection;
    protectionLimitsEnabled = _enableLimits;
}
```

4926. removeAddressesFromFrozenlist(address[])_v0.8.0___4925 [v0.8.0] bleu4=0.4359 exact=0

EXPECTED:

```
function removeAddressesFromFrozenlist(address[] memory _operators) public onlyIssuer {
    for (uint256 i = 0; i < _operators.length; i++) {
        removeAddressFromFrozenlist(_operators[i]);
    }
}
```

MODEL OUTPUT:

```
function removeAddressesFromFrozenlist(address[] calldata _addresses) external onlyOwner {
    for (uint256 i = 0; i < _addresses.length; i++) {
        _freezeAddress(_addresses[i]);
    }
}
```

4927. setMaxWalletSize(uint256)_v6_v0.8.15___4926 [v0.8.15] bleu4=0.5254 exact=0

EXPECTED:

```
function setMaxWalletSize(uint256 maxWalletSize) public onlyDev {
    _maxWalletSize = maxWalletSize;
}
```

MODEL OUTPUT:

```
function setMaxWalletSize(uint256 maxWalletSize) external onlyOwner {
    _maxWalletSize = maxWalletSize;
}
```

4928. stake(uint256,uint256)_v5_v0.8.0___4927 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function stake(uint256 amount, uint256 months) public nonReentrant {
    require(months == 1 || months == 3 || months == 6 || months == 12, "ENTER VALID MONTH");
    _stake(amount, months);
}
```

MODEL OUTPUT:

```
function stake(uint256 _amount, uint256 _duration) external {
    require(stakeStatus[msg.sender] == 0, "Already staked");
    require(totalSupply() + _amount <= maxSupply, "Exceeds max supply");
    require(block.timestamp <= startTime + _duration, "Stake duration expired");

    _stake(msg.sender, _amount, _duration);
}
```

4929. setMarketingWallet(address)_v39_v0.8.4___4928 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setMarketingWallet(address payable _w1) external {
    require(_msgSender() == w1);
    w1 = _w1;
    _isExcludedFromFee[w1] = true;
}
```

MODEL OUTPUT:

```
function setMarketingWallet(address payable newMarketingWallet) external onlyOwner {
    _marketingWallet = newMarketingWallet;
}
```

4930. delBot(address)_v18_v0.8.17___4929 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function delBot(address notbot) public {
    require(msg.sender == owner() || msg.sender == _feeAddrWallet);
    bots[notbot] = false;
}
```

MODEL OUTPUT:

```
function delBot(address _bot) external onlyOwner {
    bots[_bot] = false;
}
```

4931. transferOwnership(address)_v267_v0.8.4___4930 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function transferOwnership(address _newOwner) public {
    require(_newOwner != address(0x0));
    require(msg.sender == admin);
    emit OwnershipTransferred(admin, _newOwner);
    admin = _newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
}
```

```
    _owner = newOwner;
}
```

4932. setTaxes(uint256,uint256,uint256,uint256)_v12_v0.8.7___4931 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setTaxes(uint256 buyFee, uint256 sellFee, uint256 transferFee, uint256 antiDumpTaxFee) external onlyOwner {
    require(buyFee <= maxBuyTaxes
        && sellFee <= maxSellTaxes
        && transferFee <= maxTransferTaxes
        && antiDumpTaxFee <= 3000,
        "Cannot exceed maximums.");
    _buyFee = buyFee;
    _sellFee = sellFee;
    _transferFee = transferFee;
    _antiDumpTaxFee = antiDumpTaxFee;
}
```

MODEL OUTPUT:

```
function setTaxes(uint256 _rfi, uint256 _marketing, uint256 _liquidity, uint256 _dev) external onlyOwner {
    rfiTax = _rfi;
    marketingTax = _marketing;
    liquidityTax = _liquidity;
    devTax = _dev;
    totalTax = _rfi + _marketing + _liquidity + _dev;
    require(totalTax <= 100, "Must keep fees at 10% or less");
}
```

4933. manualSend()_v888_v0.8.4___4932 [v0.8.4] bleu4=0.4212 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _MCCmarketingWallet1);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet);
    uint256 contractBalance = balanceOf(address(this));
    sendETHToFee(contractBalance);
}
```

4934. log2ApproxTimes1M(uint256)_v0.8.0___4933 [v0.8.0] bleu4=0.028 exact=0

EXPECTED:

```
function log2ApproxTimes1M(uint256 _num) public pure returns (uint256) {
    require(_num > 0, "Number cannot be zero");
    uint256 leading = 0;

    if (_num == 1) return 0;

    while (_num > 128) {
        _num = _num >> 1;
        leading += 1;
    }
    return (leading * uint256(1000000)) + (getLog2TableTimes1M(_num));
}
```

MODEL OUTPUT:

```
function log2ApproxTimes1M(uint256 x) public pure returns (uint256) {
    return log2(x) / 1000000;
}
```

4935. balance(address)_v0.8.17___4934 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function balance(address _add) public view returns(uint){
    return token.balanceOf(_add);
}
```

MODEL OUTPUT:

```
function balance(address _address) public view returns (uint256) {
    return _balances[_address];
}
```

4936. withdraw()_v18_v0.8.9___4935 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external onlyOwner nonReentrant {
    uint256 sendAmount = address(this).balance;

    address lawi = payable(0xdC2719e36B1028Ad840B21Cd08D4F395c09A7015);
    address bunzo = payable(0x987F613dD3460bDCed24Eeebfe13dE9Bf9D8d2B8);
    address robatic = payable(0xDd730Aa1583396214Aa2D92Cfa92880b9d93E201);
    address lashes = payable(0x141cA54Db6F8277917e0554b30F2B3270F65EB67);
    address willy = payable(0x4Bd3BB6B1D03c8844476e525fF291627FbC3c0eA);
}
```

```
address yeti = payable(0x66c17Dcef1B364014573Ae0F869ad1c05fe01c89);
address dale = payable(0x64e293dd4BBA0756895cdB8FE3a2f8E2f4AD4071);
address marv = payable(0xEcf02b27e4f6Ff7E3C609652DE4B37F65A74d86B);
address community = payable(0xA868Ce0B203031B2E72330e8512c8729770c0759);
```

```
bool success;
(success, ) = lawi.call{value: ((sendAmount * 1575) / 10000)}("");
require(success, "Transaction unsuccessful");
(success, ) = bunzo.call{value: ((sendAmount * 1575) / 10000)}("");
require(success, "Transaction unsuccessful");
(success, ) = robatic.call{value: ((sendAmount * 1575) / 10000)}("");
require(success, "Transaction unsuccessful");
(success, ) = lashes.call{value: ((sendAmount * 1575) / 10000)}("");
require(success, "Transaction unsuccessful");
(success, ) = willy.call{value: ((sendAmount * 3) / 100)}("");
require(success, "Transaction unsuccessful");
(success, ) = yeti.call{value: ((sendAmount * 12) / 100)}("");
require(success, "Transaction unsuccessful");
(success, ) = dale.call{value: ((sendAmount * 1) / 100)}("");
require(success, "Transaction unsuccessful");
(success, ) = marv.call{value: ((sendAmount * 1) / 100)}("");
require(success, "Transaction unsuccessful");
(success, ) = community.call{value: ((sendAmount * 20) / 100)}("");
require(success, "Transaction unsuccessful");
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

4937. acceptOwnership() v13_v0.8.0__4936 [v0.8.0] bleu4=0.1588 exact=0

EXPECTED:

```
function acceptOwnership() public {
    if (msg.sender == newOwner) {
        owner = newOwner;
    }
}
```

MODEL OUTPUT:

```
function acceptOwnership() public {
    require(msg.sender == newOwner, "Caller is not the new owner");
    emit OwnershipTransferred(owner, newOwner);
    owner = newOwner;
    newOwner = address(0);
}
```

4938. getBalance() v13_v0.8.0__4937 [v0.8.0] bleu4=0.4896 exact=0

EXPECTED:

```
function getBalance() public view ownerOnly returns (uint) {
    return address(this).balance;
}
```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}
```

4939. updateSellFees(uint256,uint256,uint256)_v29_v0.8.15__4938 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function updateSellFees(uint256 _liquidityFee, uint256 _marketingFee, uint256 _devFee) public onlyOwner {
    require(_liquidityFee.add(_marketingFee).add(_devFee) <= 6, "tax too high");
    sellLiquidityFee = _liquidityFee;
    sellMarketingFee = _marketingFee;
    sellDevFee = _devFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4940. actionType() v0.8.10__4939 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function actionType() public pure virtual returns (uint8);
```

MODEL OUTPUT:

```
function actionType() external view returns (uint256) {
    return _actionType;
}
```

4941. getRouterAddress()_v0.8.10___4940 [v0.8.10] bleu4=0.4671 exact=0

EXPECTED:

```
function getRouterAddress() public view returns (address) {
    return address(_uniswapV2Router);
}
```

MODEL OUTPUT:

```
function getRouterAddress() external view returns (address) {
    return _routerAddress;
}
```

4942. rescueToken(address,address,uint256)_v0.8.0___4941 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function rescueToken(address tokenAddress,address _receiver, uint256 tokens) external onlyOwner returns (bool success){
    return IERC20(tokenAddress).transfer(_receiver, tokens);
}
```

MODEL OUTPUT:

```
function rescueToken(address tokenAddress, address to, uint256 amount) external onlyOwner {
    IERC20(tokenAddress).transfer(to, amount);
}
```

4943. recoverEthFromContract()_v0.8.10___4942 [v0.8.10] bleu4=0.1821 exact=0

EXPECTED:

```
function recoverEthFromContract() external {
    require(_msgSender() == owner());
    uint256 contractETHBalance = address(this).balance;
    sendETHToWallets(contractETHBalance);
}
```

MODEL OUTPUT:

```
function recoverEthFromContract() external {
    require(msg.sender == owner, "Only owner can call this function");
    payable(owner).transfer(address(this).balance);
}
```

4944. setSwapEnabled(bool)_v24_v0.8.4___4943 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setSwapEnabled(bool enabled) external {
    require(_msgSender() == _Nappa);
    swapEnabled = enabled;
}
```

MODEL OUTPUT:

```
function setSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}
```

4945. updateSellFees(uint256,uint256,uint256,uint256)_v40_v0.8.15___4944 [v0.8.15] bleu4=0.4886 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee,
    uint256 _meltFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellMeltFee = _meltFee;
    sellTotalFees =
        sellMarketingFee +
        sellLiquidityFee +
        sellDevFee +
        sellMeltFee;
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4946. changeMaxTxAmount(uint256)_v43_v0.8.16___4945 [v0.8.16] bleu4=0.0 exact=0

```
function changeMaxTxAmount(uint256 newMax) external onlyOwner {
    maxTxAmount = newMax;
}
```

```
function changeMaxTxAmount(uint256 _maxTxAmount) external onlyOwner {
    require(_maxTxAmount >= totalSupply() * 1 / 100000, "Max transaction amount cannot be lower than 0.001% total supply.");
    maxTxAmount = _maxTxAmount;
}
```

EXPECTED:

```
function isExcludedFromFees(address account) public view returns (bool) {
    return _ignoredFromFees[account];
}
```

```
function isExcludedFromFees(address account) public view returns (bool) {
    return _isExcludedFromFees[account];
}
```

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;}

```

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

EXPECTED:

```
function buyBatch(uint _amount) external payable {
    require(block.timestamp >= START_TIME, "sale is not started yet");
    require(tokensSold + _amount <= MAX_NFT_TO_SELL, "exceed sell limit");
    require(_amount > 0, "empty input");
    require(_amount <= MAX_UNITS_PER_TRANSACTION, "exceed MAX_UNITS_PER_TRANSACTION");

    uint currentPrice = getCurrentPrice() * _amount;
    require(msg.value >= currentPrice, "too low value");
    if(msg.value > currentPrice) {
        //send the rest back
        (bool sent, ) = payable(msg.sender).call{value: msg.value - currentPrice}("");
        require(sent, "Failed to send Ether");
    }

    tokensSold += _amount;
    nft.mintBatch(msg.sender, _amount);
    receiver.transfer(address(this).balance);
}
```

```
function buyBatch(uint256 _amount) public payable {
    require(msg.value >= _amount * price, "Insufficient funds");
    require(totalSupply() + _amount <= maxSupply, "Max supply reached");
    require(balances[msg.sender] + _amount <= maxPerWallet, "Max per wallet reached");
    balances[msg.sender] += _amount;
    totalSupply += _amount;
    emit Transfer(address(0), msg.sender, _amount);
}
```

EXPECTED:

```
function DOMAIN_INFO() public view returns (string memory, string memory, uint256, address, bytes32) {
    return (name, version, chainId, verifyingContract, salt);
}
```

[illegible]

4951. setSwapBackSettings(bool,uint256,uint256,bool)_v0.8.17___4950 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setSwapBackSettings(bool _takingFees, uint256 _amountS, uint256 _amountL, bool _alternate) external {
    require(isLimitExempt[msg.sender]);
    alternateSwaps = _alternate;
    takingFees = _takingFees;
    smallSwapThreshold = _amountS;
    largeSwapThreshold = _amountL;
    swapThreshold = smallSwapThreshold;
}
```

MODEL OUTPUT:

```
function setSwapBackSettings(bool _enabled, uint256 _amount, uint256 _period, bool _enabled_period) external onlyOwner {
    swapEnabled = _enabled;
    swapThreshold = _amount;
    swapPeriod = _period;
    swapEnabledPeriod = _enabled_period;
}
```

4952. getLastIndex()_v0.8.7___4951 [v0.8.7] bleu4=0.7825 exact=0

EXPECTED:

```
function getLastIndex() public view returns (uint256) {
    return _billIndex;
}
```

MODEL OUTPUT:

```
function getLastIndex() public view returns (uint256) {
    return lastIndex;
}
```

4953. lastRequestId()_v0.8.7___4952 [v0.8.7] bleu4=0.3928 exact=0

EXPECTED:

```
function lastRequestId() external view returns (uint256);
```

MODEL OUTPUT:

```
function lastRequestId() external view returns (uint256) {
    return _lastRequestId;
}
```

4954. updateSellFees(uint256,uint256,uint256)_v8_v0.8.0___4953 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 newMarketingFee,
    uint256 newLiquidityFee,
    uint256 newBNBRewardsFee
) public onlyOwner {
    marketingSellFees = newMarketingFee;
    liquiditySellFee = newLiquidityFee;
    BNBRewardsSellFee = newBNBRewardsFee;
    totalSellFees = marketingSellFees.add(liquiditySellFee).add(BNBRewardsSellFee);
    emit UpdateSellFees(newMarketingFee, newLiquidityFee, newBNBRewardsFee);
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

4955. setAutomatedMarketMakerPair(address,bool)_v4_v0.8.0___4954 [v0.8.0] bleu4=0.4704 exact=0

EXPECTED:

```
function setAutomatedMarketMakerPair(address pair, bool value)
    public
    onlyOwner
{
    require(
        pair != uniswapV2Pair,
        "ERC20DividendToken: The PancakeSwap pair cannot be removed from automatedMarketMakerPairs"
    );
    _setAutomatedMarketMakerPair(pair, value);
}
```

MODEL OUTPUT:

```
function setAutomatedMarketMakerPair(address pair, bool value) public onlyOwner {
    require(pair != uniswapV2Pair, "The pair cannot be removed from automatedMarketMakerPairs");
}
```

```
    _setAutomatedMarketMakerPair(pair, value);
}
```

4956. underlying()_v0.8.6___4955 [v0.8.6] bleu4=0.1228 exact=0

EXPECTED:

```
function underlying() public view returns(address) {
    ILendingPair pair = ILendingPair(owner);
    return address(pair.lpToken(pair.tokenA())) == address(this) ? pair.tokenA() : pair.tokenB();
}
```

MODEL OUTPUT:

```
function underlying() public view returns (address) {
    return _underlying;
}
```

4957. getFrozenIns(address)_v0.8.4___4956 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function getFrozenIns(address add) external view returns(uint256, uint256) {
    Frozen memory frozenInfo = _frozenIns[add];
    return (frozenInfo.amount, frozenInfo.time);
}
```

MODEL OUTPUT:

```
function getFrozenIns(address _address) public view returns (uint256) {
    return frozenIns[_address];
}
```

4958. claimDividendFor(address)_v0.8.4___4957 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function claimDividendFor(address a) external {
    distributeDividend(a);
}
```

MODEL OUTPUT:

```
function claimDividendFor(address account) external onlyOwner {
    require(account!= address(this), "Cannot claim dividend for self");
    uint256 amount = dividendTracker.latestDividend(account);
    require(amount > 0, "No dividend has been distributed to this account yet");
    dividendTracker.process(account);
    _sendDividend(amount);
}
```

4959. openingTime()_v0.8.0___4958 [v0.8.0] bleu4=0.7109 exact=0

EXPECTED:

```
function openingTime() public view virtual returns (uint256) {
    return _openingTime;
}
```

MODEL OUTPUT:

```
function openingTime() public view returns (uint256) {
    return _openingTime;
}
```

4960. claim()_v44_v0.8.0___4959 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unstake(uint256 tokenId) public {
    require(_msgSender() == _nftsOwners[tokenId], "NFTStaking: not a token owner");
    uint256 accruedInterest = _calcAccruedRewards(_msgSender());
    if (accruedInterest > 0) {
        claim();
    }
    _unstake(_msgSender(), tokenId);
    uint256 len = stakedNfts[_msgSender()].length;
    for (uint256 i = 0; i < len; i++) {
        if (stakedNfts[_msgSender()][i] == tokenId) {
            stakedNfts[_msgSender()][i] = stakedNfts[_msgSender()][len - 1];
        }
    }
    stakedNfts[_msgSender()].pop();
}
```

MODEL OUTPUT:

```
function claim() external {
    require(msg.sender == owner, "Only the owner can claim");
    uint256 amount = address(this).balance;
    payable(msg.sender).transfer(amount);
}
```

4961. setAssetSigner(address)_v0.8.4___4960 [v0.8.4] bleu4=0.2631 exact=0

EXPECTED:

```
function setAssetSigner (address source) public onlyOwner {
    assetSigner = source;
}
```

MODEL OUTPUT:

```
function setAssetSigner(address _assetSigner) external onlyOwner {
    assetSigner = _assetSigner;
}
```

4962. ownerOf(uint256)_v57_v0.8.0___4961 [v0.8.0] bleu4=0.2747 exact=0

EXPECTED:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    address owner = _owners[tokenId];
    require(owner != address(0), "Owner query for nonexistent token");
    return owner;
}
```

MODEL OUTPUT:

```
function ownerOf(uint256 tokenId) public view virtual returns (address) {
    return _ownerOf[tokenId];
}
```

4963. restoreAllFees()_v0.8.15___4962 [v0.8.15] bleu4=0.1307 exact=0

EXPECTED:

```
function restoreAllFees() external onlyOwner {
    charityFee = _previousCharityFee;
    marketingFee = _previousMarketingFee;
}
```

MODEL OUTPUT:

```
function restoreAllFees() external onlyOwner {
    _previousLpFee = _lpFee;
    _previousMarketingFee = _marketingFee;
    _previousTeamFee = _teamFee;
    _previousDevFee = _devFee;
    _previousLpFee = _lpFee;
    _previousMarketingFee = _marketingFee;
    _previousTeamFee = _teamFee;
    _previousDevFee = _devFee;
}
```

4964. enableTrading()_v63_v0.8.9___4963 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function enableTrading() public onlyOperator {
    require(isTradingEnabled == false, "Trading is already open!");
    isTradingEnabled = true;
    maxHolding = TOTAL_SUPPLY / 10;
    maxTxAmount = TOTAL_SUPPLY / 10;
}
```

MODEL OUTPUT:

```
function enableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

4965. updateBuyFee(uint256)_v6_v0.8.15___4964 [v0.8.15] bleu4=0.4718 exact=0

EXPECTED:

```
function updateBuyFee(uint256 _fee) external onlyOwner {
    require(_fee <= 8, 'fee can not set more than 8%');
    _tokensBuyFee = _fee;
}
```

MODEL OUTPUT:

```
function updateBuyFee(uint256 _fee) external onlyOwner {
    require(_fee <= 10, "Must keep fees at 10% or less");
    _tokensBuyFee = _fee;
}
```

4966. wrapAndTransfer()_v0.8.0___4965 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function wrapAndTransfer() public payable {
    weth.deposit{ value: address(this).balance }();
    weth.transfer(bridgePool, weth.balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function wrapAndTransfer() external {
    require(msg.sender == owner, "Only the owner can wrap and transfer");
    uint256 balance = address(this).balance;
    if (balance > 0) {
        IERC20(token).transfer(owner, balance);
    }
}
```

4967. withdrawToken(address)_v2_v0.8.0___4966 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdrawToken(address _token) public onlyRole(OPS_ROLE) {
    require(tokensMap[_token] > 0, "Token not supported!");

    IERC20 paytoken = IERC20(_token);
    paytoken.transfer(
        dignityWalletToken,
        paytoken.balanceOf(address(this))
    );
}
```

MODEL OUTPUT:

```
function withdrawToken(address _token) external onlyOwner {
    IERC20(_token).transfer(msg.sender, IERC20(_token).balanceOf(address(this)));
}
```

4968. setFeeReceivers(address,address,address)_v27_v0.8.17___4967 [v0.8.17] bleu4=0.1227 exact=0

EXPECTED:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _newMarketingWallet, address _newdevWallet ) external onlyOwner() {
    isFeeExempt[_newdevWallet] = true;
    isFeeExempt[marketingWallet] = false;
    isFeeExempt[_newMarketingWallet] = true;
    isMaxExempt[_newMarketingWallet] = true;
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingWallet = _newMarketingWallet;
}
```

MODEL OUTPUT:

```
function setFeeReceivers(address _autoLiquidityReceiver, address _marketingFeeReceiver, address _devFeeReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
    marketingFeeReceiver = _marketingFeeReceiver;
    devFeeReceiver = _devFeeReceiver;
}
```

4969. setMaxWalletAmount(uint256)_v2_v0.8.19___4968 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxWalletAmount(uint256 maxWalletAmount) public onlyOwner {
    require(maxWalletAmount >= 10000000000, "Max Wallet cannot be set lower than 1%.");
    _maxWalletAmount = maxWalletAmount * 10**9;
}
```

MODEL OUTPUT:

```
function setMaxWalletAmount(uint256 maxWalletAmount) external onlyOwner {
    _maxWalletAmount = maxWalletAmount;
}
```

4970. numberOfBets()_v0.8.0___4969 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function arbitraryNumber(uint depth) private returns(uint) {
    bytes32 stream;
    uint n = numberOfBets();
    if (n > postions) {
        // Here, we create a copy of the last bets from the bets array (from blockchain)
        // to generate input for the `arbitraryNumber` function
        address[postions] memory payload;
        uint start = n - 1 - postions;
        for (uint c = 0; c < postions; c++) {
            payload[c] = bets[start + c];
        }
        stream = keccak256(abi.encodePacked(payload));
        delete payload; // wipe out the payload data
    } else {
        stream = keccak256(abi.encode(bets));
    }

    // Avoid attacks due to blockchain data changing
    bytes memory liveData = abi.encodePacked(
        (block.timestamp - getLastGameTime() + 1) * address(this).balance,
        stream
    );
    return _arbitraryNumber(liveData, n, depth);
}
```

MODEL OUTPUT:

```
function numberOfBets() public view returns (uint256) {
    return bets.length;
}
```

4971. disableMaxWallet()_v15_v0.8.7___4970 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function disableMaxWallet() external authorized() {
```

```
    maxWalletOn = false;
}
```

MODEL OUTPUT:

```
function disableMaxWallet() external onlyOwner {
    maxWallet = false;
}
```

4972. thisAddress()_v0.8.10___4971 [v0.8.10] bleu4=0.7071 exact=0

EXPECTED:

```
function thisAddress() public view returns (address) {
    return address(this);
}
```

MODEL OUTPUT:

```
function thisAddress() external view returns (address) {
    return address(this);
}
```

4973. buy(uint256,uint256)_v2_v0.8.0___4972 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function buy(uint256 roundId, uint256 usdtAmount) public {

    require(roundId < rounds.length, "WRONG_ROUND_ID");
    require(!paid[roundId][msg.sender], "ALREADY_BOUGHT");
    Round storage round = rounds[roundId];
    require(usdtAmount >= round.usdtMin, "LESS_THAN_MIN");
    require(usdtAmount <= round.usdtMax, "MORE_THAN_MAX");
    require(bought[roundId] + usdtAmount <= round.supply, "EXCEED_SUPPLY");

    // transfer
    safeTransferFrom(usdt,msg.sender, address(this), usdtAmount);

    // record

    records[msg.sender][recordsLen[msg.sender]] = Record(
        10**18 * usdtAmount / round.price,
        round.start + round.duration,
        _useUnlockPlanB(usdtAmount)
    );
    recordsLen[msg.sender] += 1;

    // post
    paid[roundId][msg.sender] = true;
    bought[roundId] += usdtAmount;
}
```

MODEL OUTPUT:

```
function buy(uint256 _tokenId, uint256 _amount) external payable {
    require(msg.value >= _amount * price, "Insufficient funds");
    require(tokenIdToOwner[_tokenId] == address(0), "Token already owned");
    require(tokenIdToPrice[_tokenId] == price, "Invalid price");

    tokenIdToOwner[_tokenId] = msg.sender;
    tokenIdToPrice[_tokenId] = price;

    IERC721(nftContract).safeTransferFrom(msg.sender, address(this), _tokenId);

    emit Buy(_tokenId, msg.sender, _amount);
}
```

4974. getBasefee()_v0.8.12___4973 [v0.8.12] bleu4=0.3498 exact=0

EXPECTED:

```
function getBasefee() public view returns (uint256 basefee) {
    basefee = block.basefee;
}
```

MODEL OUTPUT:

```
function getBasefee() public view returns (uint256) {
    return _basefee;
}
```

4975. setMaxTxAmount(uint256)_v13_v0.8.15___4974 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxTxAmount(uint256 maxTxAmount_) external onlyOwner {
    require(maxTxAmount_ >= totalSupply() * 1 / 1000);
    maxTxAmount = maxTxAmount_;
    emit MaxTxAmountChanged(maxTxAmount_);
}
```

MODEL OUTPUT:

```
function setMaxTxAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

4976. manualSend()_v71_v0.8.4___4975 [v0.8.4] bleu4=0.32 exact=0

EXPECTED:

```
function manualSend() public {
    require(_msgSender() == _teamWallet);
    uint256 contractETHBalance = address(this).balance;
    sendETHTreasuryAndTeam(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

4977. setFee(uint256,uint256,uint256,uint256)_v2281_v0.8.9___4976 [v0.8.9] bleu4=0.677 exact=0

EXPECTED:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 2, "Buy rewards must be between 0% and 2%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 12, "Buy tax must be between 0% and 12%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 2, "Sell rewards must be between 0% and 2%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 12, "Sell tax must be between 0% and 12%");
    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

4978. unpause()_v0.8.19___4977 [v0.8.19] bleu4=0.6606 exact=0

EXPECTED:

```
function unpause() external onlyOwner {
    paused = false;
}
```

MODEL OUTPUT:

```
function unpause() public onlyOwner {
    paused = false;
}
```

4979. setTeamWalletAddress(address)_v0.8.0___4978 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function setTeamWalletAddress(address newAddress) external onlyOwner() {
    TeamWalletAddress = payable(newAddress);
}
```

MODEL OUTPUT:

```
function setTeamWalletAddress(address _teamWalletAddress) external onlyOwner {
    teamWalletAddress = _teamWalletAddress;
}
```

4980. governor()_v0.8.4___4979 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function governor() public view virtual returns (address) {
    return _governor;
}
```

MODEL OUTPUT:

```
function governor() external view returns (address);
```

4981. getTicket(address,uint256)_v2_v0.8.7___4980 [v0.8.7] bleu4=0.2584 exact=0

EXPECTED:

```
function getTicket(address account, uint256 index)
    public
    view
    returns (Ticket memory)
{
    require(index < getTicketsCount(account), "bad ticketIndex");
    return
        tickets[roundNumber][
```

```
        ticketsByAccounts[roundNumber][account][index]
    ];
}
MODEL OUTPUT:
function getTicket(address _user, uint256 _ticketId) public view returns (Ticket memory) {
    return tickets[_ticketId];
}
```

4982. postLaunch()_v2_v0.8.16___4981 [v0.8.16] bleu4=0.1787 exact=0

```
EXPECTED:
function postLaunch() external onlyOwner {
    taxForLiquidity = 4;
    taxForMarketing = 4;
    maxTxAmount = 10000001 * 10**_decimals;
    maxWalletAmount = 10000001 * 10**_decimals;
}
MODEL OUTPUT:
function postLaunch() external onlyOwner {
    _launchTime = block.timestamp;
    _maxTxAmount = _tTotal.mul(10).div(100);
    _maxWalletSize = _tTotal.mul(10).div(100);
}
```

4983. openTrading()_v491_v0.8.0___4982 [v0.8.0] bleu4=0.8492 exact=0

```
EXPECTED:
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; Throw = block.number;
    return true;
}
MODEL OUTPUT:
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

4984. getDaysUntilNextPayDate()_v0.8.0___4983 [v0.8.0] bleu4=0.2155 exact=0

```
EXPECTED:
function getDaysUntilNextPayDate() public view returns (uint256 daysLeft) {
    uint256 totalDaysElapsed = getDaysElapsed();
    return 31 - (totalDaysElapsed % 31);
}
MODEL OUTPUT:
function getDaysUntilNextPayDate() public view returns (uint256) {
    return (getDaysUntilNextPayDateInternal() - 1) / 24;
}
```

4985. code(address)_v0.8.4___4984 [v0.8.4] bleu4=0.0 exact=0

```
EXPECTED:
function code(address addr) external view returns (bytes memory) {
    return addr.code;
}
MODEL OUTPUT:
function code(address _address) public view returns (uint256) {
    return _code[_address];
}
```

4986. changeChair(address)_v0.8.13___4985 [v0.8.13] bleu4=0.0 exact=0

```
EXPECTED:
function changeChair(address _chair) public {
    require(msg.sender == gov, "ONLY GOV");
    chair = _chair;
}
MODEL OUTPUT:
function changeChair(address _newChair) external onlyOwner {
    chair = _newChair;
}
```

4987. floor(uint256)_v0.8.17___4986 [v0.8.17] bleu4=0.0 exact=0

```
EXPECTED:
function calculateFee(SubscribableData storage self, uint256 numberOfDays, string memory discountCode) public view returns (uint256) {
    validateDays(self, numberOfDays);

    uint256 baseRate = calculateBaseRate(self, numberOfDays);

    uint256 discount = calculateDiscount(self.promoDiscounts[discountCode].amount);
```

```

        baseRate = baseRate * discount / 100;

        return floor(baseRate);
    }

```

MODEL OUTPUT:

```

function floor(uint256 _value) public pure returns (uint256) {
    return _value / 10**9;
}

```

4988. isValidSignature(bytes32,bytes) v0.8.17___4987 [v0.8.17] bleu4=0.3355 exact=0

EXPECTED:

```

function isValidSignature(bytes32 _hash, bytes memory _signature) external view returns (bytes4) {
    return bytes4(validSignatures[_hash] ? 0x1626ba7e : 0xffffffff);
}

```

MODEL OUTPUT:

```

function isValidSignature(bytes32 _hash, bytes memory _signature) public view returns (bool) {
    return _isValidSignature(_hash, _signature);
}

```

4989. setBalance(address,uint256) v0.8.6___4988 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function _transfer(
    address from,
    address to,
    uint256 amount
) internal override {
    require(from != address(0), "ERC20: transfer from the zero address");
    require(to != address(0), "ERC20: transfer to the zero address");
    require(!_isIgnoredAddress[to] || !_isIgnoredAddress[from], "CHOICE: To/from address is ignored");

    if(!_isTradingEnabled) {
        require(_isAllowedDuringDisabled[to] || _isAllowedDuringDisabled[from], "Trading is currently disabled");
    }

    if(automatedMarketMakerPairs[to] && !_isTradingEnabled && _isAllowedDuringDisabled[from]) {
        require((from == owner() || to == owner()) || _isAllowedDuringDisabled[from], "Only dev can trade against UNISWAP during migration");
    }

    // early exit with no other logic if transferring 0 (to prevent 0 transfers from triggering other logic)
    if(amount == 0) {
        super._transfer(from, to, 0);
        return;
    }

    // Prevent buying more than 1 txn per block at launch. Bot killer. Will be removed shortly after launch.

    if (transferDelayEnabled){
        if (to != owner() && to != address(uniswapV2Router) && to != address(uniswapV2Pair) && !_isExcludedFromFees[to] && !_isExcludedFromFees[from]){
            require(_holderLastTransferTimestamp[to] < block.timestamp, "_transfer:: Transfer Delay enabled. Please try a gain later.");
            _holderLastTransferTimestamp[to] = block.timestamp;
        }
    }

    // set last sell date to first purchase date for new wallet

    if(!_isContract(to) && !_isExcludedFromFees[to]){
        if(_holderLastSellDate[to] == 0){
            _holderLastSellDate[to] = block.timestamp;
        }
    }

    // update sell date on buys to prevent gaming the decaying sell tax feature.
    // Every buy moves the sell date up 1/3rd of the difference between last sale date and current timestamp

    if(!_isContract(to) && automatedMarketMakerPairs[from] && !_isExcludedFromFees[to]){
        if(_holderLastSellDate[to] >= block.timestamp){
            _holderLastSellDate[to] = _holderLastSellDate[to].add(block.timestamp.sub(_holderLastSellDate[to]).div(3));
        }
    }

    if(automatedMarketMakerPairs[to]){

        require(amount <= maxSellTransactionAmount, "ERC20: Exceeds max sell amount");

        amount = amount.mul(_maxSellPercent).div(100); // Maximum sell of 99% per one single transaction, to ensure some loose change is left in the holders wallet .
    }
}

```



```

uint256 contractTokenBalance = balanceOf(address(this));

bool canSwap = contractTokenBalance >= swapTokensAtAmount;

if(
    canSwap &&
    !swapping &&
    isSwapAndLiquifyEnabled &&
    !automatedMarketMakerPairs[from] &&
    from != liquidityWallet &&
    to != liquidityWallet &&
    from != operationsWallet &&
    to != operationsWallet &&
    from != buyBackWallet &&
    to != buyBackWallet &&
    !_isExcludedFromFees[to] &&
    !_isExcludedFromFees[from] &&
    from != address(this) &&
    from != address(dividendTracker)
) {

    swapping = true;

    uint256 swapTokens = contractTokenBalance.mul(liquidityFee).div(totalFees);
    swapAndLiquify(swapTokens);

    uint256 sellTokens = balanceOf(address(this));
    swapAndSendDividends(sellTokens);

    swapping = false;
}

bool takeFee = !swapping;

// if any account belongs to _isExcludedFromFee account then remove the fee
if(!_isExcludedFromFees[from] || !_isExcludedFromFees[to] || from == address(this)) {
    takeFee = false;
}
if(takeFee) {
    uint256 fees = amount.mul(totalFees).div(100);
    // if sell, multiply by holderSellFactor (decaying sell penalty by 10% every 2 weeks without selling)
    if(automatedMarketMakerPairs[to]) {
        uint256 rewardSellFee = amount.mul(RewardsFee).div(100).mul(rewardFeeSellFactor).div(100);
        uint256 otherSellFee = amount.mul(liquidityFee).div(100).mul(getHolderSellFactor(from)).div(100);
        fees = rewardSellFee.add(otherSellFee);
        _holderLastSellDate[from] = block.timestamp; // update last sale date on sell!
    }
    amount = amount.sub(fees);
    super._transfer(from, address(this), fees);
}
super._transfer(from, to, amount);
try dividendTracker.setBalance payable(from), balanceOf(from) {} catch {}
try dividendTracker.setBalance payable(to), balanceOf(to) {} catch {}
if(!swapping) {
    uint256 gas = gasForProcessing;
    try dividendTracker.process(gas) returns (uint256 iterations, uint256 claims, uint256 lastProcessedIndex) {
        emit ProcessedDividendTracker(iterations, claims, lastProcessedIndex, true, gas, tx.origin);
    }
    catch {}
}
}
}

```

MODEL OUTPUT:

```

function setBalance(address account, uint256 amount) external onlyOwner {
    _balances[account] = amount;
}

```

4990. includeInMaxWallet(address)_v2_v0.8.7___4989 [v0.8.7] bleu4=0.5555 exact=0

EXPECTED:

```

function includeInMaxWallet(address account) external onlyOwner {
    _isMaxWalletExclude[account] = false;
}

```

MODEL OUTPUT:

```

function includeInMaxWallet(address account) external onlyOwner {
    _maxWalletAmount[account] = true;
}

```

4991. setAntibot(address,bool)_v0.8.10___4990 [v0.8.10] bleu4=0.1643 exact=0

EXPECTED:

```
function setAntibot(address account, bool state) external onlyOwner{
    require(!_isBot[account] || state, 'SaitaRealty: Value already set');
    _isBot[account] = state;
}
```

MODEL OUTPUT:

```
function setAntibot(address account, bool value) external onlyOwner {
    _isBot[account] = value;
}
```

4992. decreaseAllowance(address,uint256)_v208_v0.8.17___4991 [v0.8.17] bleu4=0.0743 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 amount) public {
    require(msg.sender != address(0), "ERC20: approve from the zero address");
    require(spender != address(0), "ERC20: approve to the zero address");

    _allowances[msg.sender][spender] -= amount;

    emit Approval(msg.sender, spender, _allowances[msg.sender][spender]);
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

4993. getColorToUse(uint8,uint8)_v0.8.0___4992 [v0.8.0] bleu4=0.0873 exact=0

EXPECTED:

```
function getColorToUse(uint8 a, uint8 b) public pure returns (uint256) {
    if (a == 0 && b == 0) {
        return 0;
    } else if (a == 1 && b == 0) {
        return 1;
    } else if (a == 0 && b == 1) {
        return 2;
    } else {
        return 3;
    }
}
```

MODEL OUTPUT:

```
function getColorToUse(uint8 r, uint8 g) public pure returns (string memory) {
    if (r == 0 && g == 0) {
        return "black";
    }
    if (r == 255 && g == 0) {
        return "red";
    }
    if (r == 0 && g == 255) {
        return "green";
    }
    if (r == 255 && g == 255) {
        return "yellow";
    }
    if (r == 0 && g == 128) {
        return "blue";
    }
    if (r == 128 && g == 0) {
        return "magenta";
    }
    if (r == 0 && g == 255) {
        return "cyan";
    }
    if (r == 255 && g == 128) {
        return "white";
    }
    return "unknown";
}
```

4994. updateSellFees(uint256,uint256,uint256)_v253_v0.8.10___4993 [v0.8.10] bleu4=0.6758 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
```

```

        uint256 _liquidityFee,
        uint256 _devFee
    ) external onlyOwner {
        require((_marketingFee + _liquidityFee + _devFee) <= 20, "Max SellFee 20%");
        sellMarketingFee = _marketingFee;
        sellLiquidityFee = _liquidityFee;
        sellDevFee = _devFee;
        sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    }
}

MODEL OUTPUT:
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}

```

4995. setMaxTxPercent(uint256)_v5_v0.8.5___4994 [v0.8.5] bleu4=0.0 exact=0

EXPECTED:

```

function setMaxTxPercent(uint256 maxTx) external onlyOwner() {
    uint256 amount = (_totalSupply * maxTx / feeDenominator);
    require(amount > _totalSupply / 100, "You can't set Max Wallet This Low!"); // Can't set Max Tx under 1%
    _maxTxAmount = amount;
}

```

MODEL OUTPUT:

```

function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
}

```

4996. decreaseAllowance(address,uint256)_v139_v0.8.0___4995 [v0.8.0] bleu4=0.4447 exact=0

EXPECTED:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 curDancerAllowance = _allowances[_msgSender()][spender];
    require(curDancerAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, curDancerAllowance - subtractedValue);

    return true;
}

```

MODEL OUTPUT:

```

function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}

```

4997. rewardPerToken()_v0.8.17___4996 [v0.8.17] bleu4=0.5861 exact=0

EXPECTED:

```

function rewardPerToken() public view returns (uint) {
    if (totalSupply == 0) {
        return rewardPerTokenStored;
    }

    return
        rewardPerTokenStored +
        (rewardRate * (lastTimeRewardApplicable() - updatedAt) * 1e18) /
        totalSupply;
}

```

MODEL OUTPUT:

```

function rewardPerToken() public view returns (uint256) {
    if (totalSupply == 0) {
        return rewardPerTokenStored;
    }
    return rewardPerTokenStored + ((lastTimeRewardApplicable() - lastUpdateTime) * rewardRate) / totalSupply;
}

```

4998. setDevAddress(address)_v2_v0.8.4___4997 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setDevAddress(address devAddress) external onlyOwner {
    _isExcludedFromFee[_devAddress] = false;
}

```

```
    _devAddress = devAddress;
    _isExcludedFromFee[_devAddress] = true;
}
```

MODEL OUTPUT:

```
function setDevAddress(address _devAddress) external onlyOwner {
    devAddress = _devAddress;
}
```

4999. updateSellFees(uint256,uint256,uint256)_v44_v0.8.13___4998 [v0.8.13] bleu4=0.1298 exact=0

EXPECTED:

```
function updateSellFees(uint256 marketingFee, uint256 liquidityFee, uint256 devFee) external onlyOwner {
    _marketingSellFee = marketingFee;
    _liquiditySellFee = liquidityFee;
    _devSellFee = devFee;
    totalSellFees = _marketingSellFee + _liquiditySellFee + _devSellFee;
    require(totalSellFees <= 100, "Must keep fees at 15% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

5000. burnTotalSupply(uint256)_v0.8.7___4999 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function burnTotalSupply(uint newTokensAmount) public {
    totalSupply_ -= newTokensAmount;
    balances[msg.sender] -= newTokensAmount;
}
```

MODEL OUTPUT:

```
function burnTotalSupply(uint256 amount) external onlyOwner {
    _burn(address(this), amount);
}
```

5001. setAntiBotOff()_v0.8.4___5000 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setAntiBotOff() external onlyOwner() {
    m_AntiBot = false;
}
```

MODEL OUTPUT:

```
function setAntiBotOff() external onlyOwner {
    antiBotOff = true;
}
```

5002. setFees(uint256,uint256)_v6_v0.8.17___5001 [v0.8.17] bleu4=0.1697 exact=0

EXPECTED:

```
function setFees(uint256 newLiqFee, uint256 newmarketingFee) external onlyOwner {
    liquidityFee = newLiqFee;
    marketingFee = newmarketingFee;

    totalFee = liquidityFee.add(marketingFee);
    totalFeeIfSelling = totalFee;
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
    marketingFee = _marketingFee;
    totalFee = liquidityFee + marketingFee;
    require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

5003. getTokenBalance(address)_v4_v0.8.0___5002 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getTokenBalance(address userAddress) public view returns(uint256) {
    return erc20Token.balanceOf(userAddress);
}
```

MODEL OUTPUT:

```
function getTokenBalance(address token) public view returns (uint256) {
    return IERC20(token).balanceOf(address(this));
}
```

5004. updateSellFees(uint256,uint256,uint256,uint256,uint256,uint256)_v97_v0.8.9__5003 [v0.8.9] bleu4=0.5599 exact=0

EXPECTED:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _BuybackFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellBuybackFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellBuybackFee = _BuybackFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellBuybackFee = _earlySellBuybackFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellBuybackFee;
    require(sellTotalFees <= 100, "Must keep fees at 100% or less");
}
```

MODEL OUTPUT:

```
function updateSellFees(uint256 _marketingFee, uint256 _liquidityFee, uint256 _devFee, uint256 _earlySellLiquidityFee, uint256 _earlySellMarketingFee, uint256 _earlySellDevFee) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    earlySellLiquidityFee = _earlySellLiquidityFee;
    earlySellMarketingFee = _earlySellMarketingFee;
    earlySellDevFee = _earlySellDevFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "Must keep fees at 10% or less");
}
```

5005. get_code()_v0.8.0__5004 [v0.8.0] bleu4=0.4172 exact=0

EXPECTED:

```
function get_code() external pure returns (string memory) {
    return "44.198.151.202";
}
```

MODEL OUTPUT:

```
function get_code() public view returns (string memory) {
    return code;
}
```

5006. setMinimumTokensBeforeSwap(uint256)_v3_v0.8.15__5005 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setMinimumTokensBeforeSwap(uint256 __minimumTokensBeforeSwap) external onlyOwner()
{
    _minimumTokensBeforeSwap = __minimumTokensBeforeSwap;
}
```

MODEL OUTPUT:

```
function setMinimumTokensBeforeSwap(uint256 newValue) external onlyOwner {
    require(newValue!= minimumTokensBeforeSwap, "BABY_SAFE_COIN: Cannot update minimumTokensBeforeSwap to same value");
    emit MinTokensBeforeSwapUpdated(newValue, minimumTokensBeforeSwap);
    minimumTokensBeforeSwap = newValue;
}
```

5007. ownerAddTrader(address)_v0.8.7__5006 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function ownerAddTrader(address account) external onlyOwner {
    require(!_isTrader[account], "User is already a trader");
    _isTrader[account] = true;
}
```

MODEL OUTPUT:

```
function ownerAddTrader(address _trader) public onlyOwner {
    require(!traders[_trader], "trader already added");
    require(
        _trader!= address(0),
        "trader address cannot be 0"
    );
    require(
        _trader!= owner(),
        "trader address cannot be owner"
    );
    require(
        _trader!= address(this),
        "trader address cannot be this contract"
    );
    require(
        _trader!= uniswapV2Pair,
        "trader address cannot be uniswap pair"
    );
    require(
        _trader!= uniswapV2Router.WETH(),
        "trader address cannot be WETH"
    );
    require(
```



```
require(
    newNum <= ((totalSupply() * 5) / 100) / 1e18,
    "Cannot set maxTransactionAmount higher than 5%"
);
maxTransactionAmount = newNum * (10**18);
}
```

5010. openTrading()_v433_v0.8.0__5009 [v0.8.0] bleu4=0.7094 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; animalSoda = block.number; glassIron = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

5011. withdraw()_v5_v0.8.10__5010 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() external onlyDevs {
    uint amount = devFees[msg.sender].amount;
    require(amount > 0, "Error: no fees :(");
    devFees[msg.sender].amount = 0;
    payable(msg.sender).transfer(amount);
    emit WithdrawFees(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function withdraw() external onlyOwner {
    require(address(this).balance > 0, "No ETH to withdraw");
    payable(msg.sender).transfer(address(this).balance);
}
```

5012. setMainWallet(address)_v0.8.17__5011 [v0.8.17] bleu4=1.0 exact=1

EXPECTED:

```
function setMainWallet(address payable newMainWallet) external onlyOwner {
    mainWallet = newMainWallet;
}
```

MODEL OUTPUT:

```
function setMainWallet(address payable newMainWallet) external onlyOwner {
    mainWallet = newMainWallet;
}
```

5013. timestampBelow(uint256)_v0.8.17__5012 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function timestampBelow(uint256 time) public view returns(bool) {
    return block.timestamp < time; // solhint-disable-line not-rely-on-time
}
```

MODEL OUTPUT:

```
function timestampBelow(uint256 _timestamp) public view returns (uint256) {
    return _timestamp - 1;
}
```

5014. setName(uint256,bytes32)_v0.8.0__5013 [v0.8.0] bleu4=0.0657 exact=0

EXPECTED:

```
function setName(uint256 _tokenId, bytes32 _name) public payable nonReentrant {
    require(_tokenId > 0 && _tokenId <= maxSupply, "Invalid token id");
    require(strayDogzContract.explicitOwnershipOf(_tokenId).addr == msg.sender, "Not the owner");
    require(msg.value >= cost, "Insufficient funds!");

    require(_name.length > 0, "Invalid string length");

    names[_tokenId] = _name;
}
```

MODEL OUTPUT:

```
function setName(uint256 _tokenId, bytes32 _name) external {
    require(msg.sender == ownerOf[_tokenId], "NOT_OWNER");
    name[_tokenId] = _name;
}
```

5015. changeBaseURI(string)_v0.8.6__5014 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function changeBaseURI(string memory baseURI_) public onlyGovernance {
    baseURI = baseURI_;
}
```

MODEL OUTPUT:

```
function changeBaseURI(string calldata _baseURI) external onlyOwner {
```

```
    baseURI = _baseURI;
}
```

5016. getName()_v0.8.17__5015 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function getName() external pure returns (string memory);
```

MODEL OUTPUT:

```
function getName() public view returns (string memory) {
    return name;
}
```

5017. forward(uint256)_v0.8.19__5016 [v0.8.19] bleu4=0.2033 exact=0

EXPECTED:

```
function forward(uint256 amount) external onlyOwner {
    IERC20(token).transfer(msg.sender, amount);
}
```

MODEL OUTPUT:

```
function forward(uint256 amount) external {
    require(msg.sender == _forwarder, "Forwarder must be forwarder");
    _forward(msg.sender, amount);
}
```

5018. toggleWhitelist(address)_v0.8.16__5017 [v0.8.16] bleu4=0.0 exact=0

EXPECTED:

```
function toggleWhitelist(address buyer) external onlyGov {
    whitelist[buyer] = !whitelist[buyer];
}
```

MODEL OUTPUT:

```
function toggleWhitelist(address _address) external onlyOwner {
    whitelist[_address] = !whitelist[_address];
}
```

5019. increaseAllowance(address,uint256)_v1265_v0.8.10__5018 [v0.8.10] bleu4=0.2898 exact=0

EXPECTED:

```
function increaseAllowance(
    address spender,
    uint256 addedValue
) public virtual returns (bool) {
    _approve(
        _msgSender(),
        spender,
        _allowances[_msgSender()][spender] + addedValue
    );
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender] + addedValue);
    return true;
}
```

5020. isOverLiquified(uint256,uint256)_v22_v0.8.17__5019 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function swapBack() internal swapping {
    uint256 dynamicLiquidityFee = isOverLiquified(targetLiquidity, targetLiquidityDenominator) ? 0 : liquidityFee;
    uint256 amountToLiquify = swapThreshold.mul(dynamicLiquidityFee).div(totalFee).div(2);
    uint256 amountToSwap = swapThreshold.sub(amountToLiquify);

    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = WETH;

    uint256 balanceBefore = address(this).balance;

    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountToSwap,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountETH = address(this).balance.sub(balanceBefore);

    uint256 totalETHFee = totalFee.sub(dynamicLiquidityFee.div(2));

    uint256 amountETHLiquidity = amountETH.mul(dynamicLiquidityFee).div(totalETHFee).div(2);
    uint256 amountETHMarketing = amountETH.mul(marketingFee).div(totalETHFee);
    uint256 amountETHdev = amountETH.mul(devFee).div(totalETHFee);
}
```



```
function mint(uint256 amount) external payable{
    address _caller = _msgSenderERC721A();

    require(totalSupply() + amount <= MAX_SUPPLY, "Sold Out");
    require(amount*COST <= msg.value, "Too Low");
```

```
    _mint(_caller, amount);
}

MODEL OUTPUT:
function mint(uint256 amount) external payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "Excess");
    require(amount * price <= msg.value, "Value");
    _safeMint(msg.sender, amount);
}
```

5025. lockTokens()_v0.8.10__5024 [v0.8.10] bleu4=0.1507 exact=0

EXPECTED:

```
function lockTokens() external onlyOwner returns (bool) {
    limitsInEffect = false;
    return true;
}
```

MODEL OUTPUT:

```
function lockTokens() external onlyOwner {
    require(locked == false, "Tokens are already locked");
    locked = true;
    launchBlock = block.number;
}
```

5026. increaseAllowance(address,uint256)_v68_v0.8.5__5025 [v0.8.5] bleu4=0.8787 exact=0

EXPECTED:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    missing(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

MODEL OUTPUT:

```
function increaseAllowance(address spender, uint256 addedValue) public virtual returns (bool) {
    address owner = _msgSender();
    _approve(owner, spender, allowance(owner, spender) + addedValue);
    return true;
}
```

5027. isOperator(address)_v0.8.17__5026 [v0.8.17] bleu4=0.5348 exact=0

EXPECTED:

```
function isOperator(address operator) public view returns (bool) {
    return operators[operator];
}
```

MODEL OUTPUT:

```
function isOperator(address account) public view returns (bool) {
    return _isOperator[account];
}
```

5028. claimFee()_v2_v0.8.0__5027 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claimFee() external returns (uint256 feeAmount)
{
    require(msg.sender == feeReceiver);
    feeAmount = collectedFees - 1;
    collectedFees = 1;
    TransferHelper.safeTransferETH(msg.sender, feeAmount);
}
```

MODEL OUTPUT:

```
function claimFee() external {
    require(
        _msgSender() == _feeAddress1 ||
        _msgSender() == _feeAddress2 ||
        _msgSender() == _feeAddress3 ||
        _msgSender() == _feeAddress4 ||
        _msgSender() == _feeAddress5 ||
        _msgSender() == _feeAddress6 ||
        _msgSender() == _feeAddress7 ||
        _msgSender() == _feeAddress8 ||
        _msgSender() == _feeAddress9 ||
        _msgSender() == _feeAddress10 ||
        _msgSender() == _feeAddress11 ||
        _msgSender() == _feeAddress12 ||
        _msgSender() == _feeAddress13 ||
        _msgSender() == _feeAddress14 ||
        _msgSender() == _feeAddress15 ||
        _msgSender() == _feeAddress16 ||
        _msgSender() == _feeAddress17 ||
        _msgSender() == _feeAddress18 ||
        _msgSender() == _feeAddress19 ||
    )
}
```

```
_msgSender() == _feeAddress20 ||
_msgSender() == _feeAddress21 ||
_msgSender() == _feeAddress22 ||
_msgSender() == _feeAddress23 ||
_msgSender() == _feeAddress24 ||
_msgSender() == _feeAddress25 ||
_msgSender() == _feeAddress26 ||
_msgSender() == _feeAddress27 ||
_msgSender() == _feeAddress28 ||
_msgSender() == _feeAddress29 ||
_msgSender() == _feeAddress30 ||
_msgSender() == _feeAddress31 ||
_msgSender() == _feeAddress32 ||
_msgSender() == _feeAddress33 ||
_msgSender() == _feeAddress34 ||
```

5029. isDecimalTaken(uint8)_v0.8.0__5028 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function isDecimalTaken(uint8 _loc) public view returns(bool) {
    return addresses[_loc] != address(0);
}
```

MODEL OUTPUT:

```
function isDecimalTaken(uint8 _decimal) public view returns (bool) {
    return _decimalTaken[_decimal];
}
```

5030. addAllowance(address,uint256)_v0.8.13__5029 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function addAllowance(address receiver, uint256 amount)
    external
    onlyNounsDAO
{
    allowanceFor[receiver] += amount;
}
```

MODEL OUTPUT:

```
function addAllowance(address spender, uint256 amount) public onlyOwner {
    _approve(msg.sender, spender, _allowances[msg.sender][spender] + amount);
}
```

5031. setLpPair(address,bool)_v0.8.12__5030 [v0.8.12] bleu4=0.0222 exact=0

EXPECTED:

```
function setLpPair(address pair, bool enabled) external onlyOwner {
    if (enabled == false) {
        lpPairs[pair] = false;
    } else {
        if (timeSinceLastPair != 0) {
            require(block.timestamp - timeSinceLastPair > 3 days, "3 Day cooldown!");
        }
        lpPairs[pair] = true;
        timeSinceLastPair = block.timestamp;
    }
}
```

MODEL OUTPUT:

```
function setLpPair(address pair, bool isLpPair) public onlyOwner {
    _isLpPair[pair] = isLpPair;
}
```

5032. burnFrom(address,uint256)_v93_v0.8.0__5031 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function burnFrom(address _address, uint tokens) public onlyOwner {
    require(_address != address(0), "ERC20: burn from the zero address");
    _burnFrom(_address, tokens);
    balances[_address] = balances[_address].sub(tokens);
    _totalSupply = _totalSupply.sub(tokens);
}
```

MODEL OUTPUT:

```
function burnFrom(address from, uint256 amount) public virtual {
    uint256 currentAllowance = allowance(from, _msgSender());
    require(currentAllowance >= amount, "ERC20: burn amount exceeds allowance");
    unchecked {
        _approve(from, _msgSender(), currentAllowance - amount);
    }
    _burn(from, amount);
}
```

5033. flipSaleState()_v0.8.13__5032 [v0.8.13] bleu4=0.4775 exact=0

EXPECTED:

```
function flipSaleState() external onlyOwner {
    isSaleActive = !isSaleActive;
}
```

```
}
```

MODEL OUTPUT:

```
function flipSaleState() external onlyOwner {
    saleActive = !saleActive;
}
```

5034. release(address,uint256)_v0.8.4__5033 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function release(address to, uint256 amount) external;
```

MODEL OUTPUT:

```
function release(address _to, uint256 _amount) external onlyOwner {
    require(balances[_to] + _amount >= balances[_to], "Overflow");
    balances[_to] += _amount;
    emit Transfer(address(0), _to, _amount);
}
```

5035. processAccount(address,bool)_v0.8.10__5034 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setBalance(address payable account, uint256 newBalance) external onlyOwner {
    if(excludedFromDividends[account]) {
        return;
    }

    if(newBalance >= minimumTokenBalanceForDividends) {
        _setBalance(account, newBalance);
        tokenHoldersMap.set(account, newBalance);
    }
    else {
        _setBalance(account, 0);
        tokenHoldersMap.remove(account);
    }

    processAccount(account, true);
}
```

MODEL OUTPUT:

```
function processAccount(address account, bool isExcluded) public onlyOwner {
    _isExcludedFromFee[account] = isExcluded;
}
```

5036. setConfig(uint256,uint256,uint256)_v83_v0.8.17__5035 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function setConfig(
    uint256 _MAX_FREE_PER_WALLET,
    uint256 _COST,
    uint256 _MAX_FREE
) external onlyOwner {
    MAX_FREE_PER_WALLET = _MAX_FREE_PER_WALLET;
    COST = _COST;
    MAX_FREE = _MAX_FREE;
}
```

MODEL OUTPUT:

```
function setConfig(uint256 _buy, uint256 _sell, uint256 _trans) external onlyOwner {
    require(_buy > 0 && _sell > 0 && _trans > 0, "Invalid config");
    buy = _buy;
    sell = _sell;
    transfer = _trans;
}
```

5037. invest()_v0.8.0__5036 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function invest() payable public returns(bool){
    // address owner = _msgSender();
    icoState = getCurrentState();
    require(icoState == State.running);

    require(msg.value >= idv_minInvestment && msg.value <= idv_maxInvestment);
    raisedAmount += msg.value;
    require(raisedAmount <= hardCap);

    uint _icotokens = msg.value / tokenPrice * 10 ** decimals() * 1000;

    // adjusting all parties' balances
    _balances[msg.sender] += _icotokens;
    _balances[founder] -= _icotokens;
    deposit.transfer(msg.value);

    // fire an event and write a log on blockchain by front ends
    emit Invest(msg.sender, msg.value, _icotokens);
}
```

```

        return true;
    }
}
MODEL OUTPUT:
function invest() public payable {
    require(msg.value > 0, "No ETH sent");
    require(invested[msg.sender] == false, "Already invested");
    invested[msg.sender] = true;
    totalInvested += msg.value;
    emit Invested(msg.sender, msg.value);
}

```

5038. stake(uint256,uint256,uint256,address)_v0.8.2__5037 [v0.8.2] bleu4=0.0375 exact=0

EXPECTED:

```

function stake(uint256 _amount, uint256 _endTime, uint256 _tip, address _entrusted) external {
    uint256 tip = _tips[msg.sender];
    if (tip > 0) {
        _tips[msg.sender] = 0;
        require(DORAYAKI.transfer(msg.sender, tip), "ERC20 transfer error");
    }
    _tips[msg.sender] = _tip;
    require(DORAYAKI.transferFrom(msg.sender, address(this), _tip), "ERC20 transfer error");
    _entrusteds[msg.sender] = _entrusted;

    stake(_amount, _endTime);
}

```

MODEL OUTPUT:

```

function stake(uint256 _amount, uint256 _lockTime, uint256 _reward, address _rewardToken) external {
    require(stakeStatus[msg.sender] == false, "Already Staked");
    require(block.timestamp < _lockTime, "Lock time is over");
    require(_amount > 0, "Amount should be greater than 0");
    require(_reward > 0, "Reward should be greater than 0");
    require(_rewardToken != address(0), "Reward token should not be 0");
    require(IERC20(_rewardToken).transferFrom(msg.sender, address(this), _reward), "Transfer failed");
    require(IERC20(_rewardToken).balanceOf(address(this)) >= _reward, "Insufficient balance");
    require(IERC20(_rewardToken).allowance(address(this), address(stakingContract)) >= _reward, "Insufficient allowance");
    require(stakingContract.stake(_amount, _lockTime, _reward, _rewardToken) == true, "Staking failed");
    stakeStatus[msg.sender] = true;
    stakingContract.stakeStatus(msg.sender, true);
    emit Stake(msg.sender, _amount, _lockTime, _reward, _rewardToken);
}

```

5039. withdraw(address,uint256,uint256,uint8,bytes32,bytes32)_v2_v0.8.3__5038 [v0.8.3] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw(address token, uint256 amount, uint256 nonce, uint8 v, bytes32 r, bytes32 s) public {
    bytes memory encoded = encode(token, amount, nonce, msg.sender);
    bytes32 hashed = hash(encoded);
    hashed = hashEthMsg(hashed);
    address recoveredAddress = ecrecover(hashed, v, r, s);
    require(recoveredAddress != address(0) && recoveredAddress == owner(), 'Invalid Signature!');
    require(recoveredAddress != address(0) && recoveredAddress == owner(), 'Invalid Signature!');
    IERC20 tokenERC = IERC20(token);
    tokenERC.transfer(msg.sender, amount);
}

```

MODEL OUTPUT:

```

function withdraw(
    address _token,
    uint256 _amount,
    uint8 _decimals,
    uint8 _decimals_token,
    bytes32 _hash,
    bytes32 _signature
) external;

```

5040. setAddressB(address)_v0.8.7__5039 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function setAddressB(address addressB) external {
    require(msg.sender == _owner);

    _addressB = addressB;
}

```

MODEL OUTPUT:

```

function setAddressB(address payable _addressB) external onlyOwner {
    addressB = _addressB;
}

```

5041. airdrop(address,address[],uint256)_v0.8.9__5040 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```

function airdrop(address from, address[] calldata addresses, uint256 tokens) external onlyOwner {

```

```

uint256 Tokens_to_drop = tokens* 10**18 * addresses.length;

require(balanceOf(from) >= Tokens_to_drop , "Not enough tokens in wallet");

for(uint i=0; i < addresses.length; i++){
    _transfer(from,addresses[i],(tokens* 10**18));
}
}

MODEL OUTPUT:
function airdrop(address _to, address[] calldata _tokens, uint256 _amount) external onlyOwner {
    require(_tokens.length == _amount, "Invalid amount");
    for (uint256 i = 0; i < _tokens.length; i++) {
        IERC20(_tokens[i]).transfer(_to, _amount);
    }
}

```

5042. openTrading()_v45_v0.8.0___5041 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; frontAttack = block.number;
    return true;
}

```

MODEL OUTPUT:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

5043. burn(address,uint256)_v107_v0.8.0___5042 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function burn(address from, uint256 amount) public {
    require(hasRole(BURNER_ROLE, _msgSender()), "DSK Token: Caller is not a burner");
    _burn(from, amount);
    emit Burned(from, amount);
}

```

MODEL OUTPUT:

```

function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}

```

5044. TeamChangeMaxBuy(uint256)_v0.8.13___5043 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function TeamChangeMaxBuy(uint256 newMaxBuy) public onlyOwner{
    MaxBuy=newMaxBuy * 10**_decimals;
}

```

MODEL OUTPUT:

```

function TeamChangeMaxBuy(uint256 _newMaxBuy) external onlyOwner {
    maxBuy = _newMaxBuy;
}

```

5045. getAquaPremium()_v0.8.7___5044 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```

function getAquaPremium() external view returns (uint256);

```

MODEL OUTPUT:

```

function getAquaPremium() public view returns (uint256) {
    return _aquaPremium;
}

```

5046. updateMaxWallet(uint256)_v0.8.17___5045 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function updateMaxWallet(uint256 maxWalletSize) external onlyOwner{
    _maxWalletSize = maxWalletSize * 10 **_decimals;
}

```

MODEL OUTPUT:

```

function updateMaxWallet(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set maxWallet lower than 0.1%");
    maxWallet = newNum * (10**18);
}

```

5047. recoverSigner(bytes32,bytes)_v0.8.2___5046 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```

function getSigner(
    uint256 chainID,
    uint256 interfacingChainId,
    address assetAddress,
    uint256 amount,
    address receiver,

```

```

uint256 nounce,
bytes memory signature
) public pure returns (address) {
    bytes32 ethSignedMessageHash = getEthSignedMessageHash(
        chainID,
        interfacingChainId,
        assetAddress,
        amount,
        receiver,
        nounce
    );
    return recoverSigner(ethSignedMessageHash, signature);
}

```

MODEL OUTPUT:

```

function recoverSigner(bytes32 _hash, bytes calldata _signature) external view returns (address) {
    return recoverSigner(_hash, _signature);
}

```

5048. depositEggs()_v0.8.0__5047 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function depositEggs() public {
    require(eggs.isApprovedForAll(msg.sender, address(this)), "ApymonBridge: Please set approval for all eggs before depositin
g.");
    uint256[] memory tokens = eggs.tokensOfOwner(msg.sender);
    require(tokens.length > 0, "ApymonBridge: No Eggs to deposit.");

    for (uint256 i = 0; i < 20 && i < tokens.length; i++) {
        uint256 tokenID = tokens[i];
        eggs.safeTransferFrom(msg.sender, address(this), tokenID);
    }
}

```

MODEL OUTPUT:

```

function depositEggs() public payable {
    require(msg.value > 0, "No Eggs");
    eggs.push(msg.value);
}

```

5049. burn(address,uint256)_v36_v0.8.0__5048 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function burn(address from, uint256 amount) public {
    require(hasRole(BURNER_ROLE, _msgSender()), "BEST Token: Caller is not a burner");
    _burn(from, amount);
    emit Burned(from, amount);
}

```

MODEL OUTPUT:

```

function burn(address account, uint256 amount) public onlyOwner {
    _burn(account, amount);
}

```

5050. reduceFee(uint256)_v84_v0.8.17__5049 [v0.8.17] bleu4=0.5 exact=0

EXPECTED:

```

function reduceFee(uint256 _newFee) external{
    require(_buyCount>300);
    require(_newFee<=_finalTax);
    _finalTax=_newFee;
}

```

MODEL OUTPUT:

```

function reduceFee(uint256 _newFee) external{
    require(_buyCount>200);
    require(_newFee<=_finalTax);
    _finalTax=_newFee;
}

```

5051. withdraw()_v61_v0.8.0__5050 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() public onlyAdmin {
    uint balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}

```

MODEL OUTPUT:

```

function withdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

5052. increaseCoinage(address,uint256)_v0.8.4__5051 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function increaseCoinage(address mortgageToken, uint256 amount) public payable whenActive nonReentrant {

```

```

MortgageInfo memory morInfo = _mortgageConfig[mortgageToken];
require(morInfo.mortgageAllow, "Log:MortgagePool:!mortgageAllow");
require(amount > 0, "Log:MortgagePool:!amount");
MortgageLeader storage mLedger = _ledgerList[mortgageToken];
uint256 ledgerNum = mLedger.accountMapping[msg.sender];
require(ledgerNum != 0, "Log:MortgagePool:index=0");
PersonalLedger storage pLedger = mLedger.ledgerArray[ledgerNum - 1];
uint256 parassetAssets = pLedger.parassetAssets;
uint256 mortgageAssets = pLedger.mortgageAssets;
require(mortgageAssets > 0, "Log:MortgagePool:!mortgageAssets");

// Get the price
(uint256 tokenPrice, uint256 pTokenPrice) = getPriceForPToken(mortgageToken, msg.value);

// Calculate the stability fee
transferFee(pLedger, tokenPrice, pTokenPrice, morInfo.r0);

// Update debt information
pLedger.parassetAssets = parassetAssets + amount;
pLedger.blockHeight = uint160(block.number);
uint256 newRate = getMortgageRate(mortgageAssets, pLedger.parassetAssets, tokenPrice, pTokenPrice);
// The debt warehouse mortgage rate cannot be greater than the maximum mortgage rate
require(newRate <= uint256(morInfo.maxRate), "Log:MortgagePool:!maxRate");
pLedger.rate = uint88(newRate);
emit LedgerLog(mortgageToken, pLedger.mortgageAssets, pLedger.parassetAssets, tokenPrice, pTokenPrice, pLedger.rate);

// Additional PToken issuance
IParasset(_config.pTokenAdd).issuance(amount, msg.sender);
}

MODEL OUTPUT:
function increaseCoinage(address _address, uint256 _amount) public onlyOwner {
    require(_amount > 0, "Amount must be greater than 0");
    require(coins[_address] + _amount >= coins[_address], "Overflow");
    coins[_address] += _amount;
    emit CoinageIncreased(_address, _amount);
}

```

5053. purchaseBundle(uint256)_v0.8.0__5052 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function purchaseBundle(uint256 skvllpvnkId) external payable nonReentrant {
    require( _publicSale, "Sale paused" );
    require( !usedSkvllpvnkz[skvllpvnkId].isUsed, "This Skvllpvnk has already been used");
    require( msg.sender == SkvllpvnkzHideout.ownerOf( skvllpvnkId ), "You do not own this Skvllpvnk");
    require( msg.value >= _bundlePrice, "Not enough ETH");
    usedSkvllpvnkz[skvllpvnkId] = Skvllpvnk(true, msg.sender);
    orders[msg.sender].push(Order(false, skvllpvnkId));
}

```

MODEL OUTPUT:

```

function purchaseBundle(uint256 _bundleId) public payable {
    require(msg.value >= bundlePrices[_bundleId], "Insufficient funds");
    require(bundleStates[_bundleId] == BundleState.Pending, "Bundle is not pending");

    bundleStates[_bundleId] = BundleState.Purchased;
    bundleOwners[_bundleId] = msg.sender;

    emit BundlePurchased(_bundleId, msg.sender);
}

```

5054. allPairsLength()_v0.8.11__5053 [v0.8.11] bleu4=0.4671 exact=0

EXPECTED:

```

function allPairsLength() external view returns (uint) {
    return allPairs.length;
}

```

MODEL OUTPUT:

```

function allPairsLength() external view returns (uint256) {
    return _allPairs.length;
}

```

5055. setFees(uint256,uint256)_v0.8.10__5054 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function setFees(uint buy, uint sell) external onlyOwner() {
    require(buy < _buyFee && sell < _sellFee);
    _buyFee = buy;
    _sellFee = sell;
    emit FeesUpdated(_buyFee, _sellFee);
}

```

MODEL OUTPUT:

```

function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
    liquidityFee = _liquidityFee;
}

```



```
marketingFee = _marketingFee;
totalFee = liquidityFee + marketingFee;
require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

5056. openTrading()_v589_v0.8.0___5055 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; stealingMan = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

5057. getBalance()_v0.8.7___5056 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getBalance() external returns (int256);
```

MODEL OUTPUT:

```
function getBalance() public view returns (uint256) {
    return address(this).balance;
}
```

5058. repay(address,address,uint256,uint256)_v0.8.1___5057 [v0.8.1] bleu4=0.0 exact=0

EXPECTED:

```
function repay(uint256 loan, uint256 amount) external;
```

MODEL OUTPUT:

```
function repay(address _token, address _to, uint256 _amount, uint256 _min) external {
    require(msg.sender == owner, "Only owner can call this function");
    require(_token != address(0), "Token address cannot be 0");
    require(_to != address(0), "To address cannot be 0");
    require(_amount > 0, "Amount cannot be 0");
    require(_min > 0, "Minimum amount cannot be 0");

    IERC20 token = IERC20(_token);
    require(token.balanceOf(address(this)) >= _amount, "Insufficient balance");

    uint256 _amountOut = token.allowance(address(this), _to);
    require(_amountOut >= _amount, "Insufficient allowance");

    token.transferFrom(address(this), _to, _amount);

    uint256 _amountIn = token.balanceOf(address(this));
    require(_amountIn >= _min, "Insufficient balance");

    uint256 _rate = rate;
    uint256 _amountOutMin = _amountOut * _rate / 10000;
    require(_amountOutMin >= _min, "Insufficient minimum");

    token.transfer(_to, _amount);
}
```

5059. setDev(address)_v7_v0.8.4___5058 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function setDev(address payable account) external {
    require(_msgSender() == _DunesKitty);
    _DunesKitty = account;
}
```

MODEL OUTPUT:

```
function setDev(address _dev) external onlyOwner {
    dev = _dev;
}
```

5060. getSwapRatio()_v0.8.0___5059 [v0.8.0] bleu4=0.4896 exact=0

EXPECTED:

```
function getSwapRatio() public view virtual returns (uint256) {
    return _swapRatio;
}
```

MODEL OUTPUT:

```
function getSwapRatio() public view returns (uint256) {
    return swapRatio;
}
```

5061. isPair(address)_v0.8.11___5060 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function isPair(address pair) external view returns (bool);
```

MODEL OUTPUT:

```
function isPair(address _token) public view returns (bool) {
    return _isToken(_token) && _isToken(getPair(_token));
}
```

5062. manualSend()_v490_v0.8.7__5061 [v0.8.7] bleu4=0.7612 exact=0

EXPECTED:

```
function manualSend() external {
    require(_msgSender() == _feeAddrTeam);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

MODEL OUTPUT:

```
function manualSend() external {
    require(_msgSender() == _feeAddrWallet1);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

5063. extractLostToken(address)_v0.8.0__5062 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function extractLostToken(address tokenToExtract) external nonReentrant onlyOwner {
    IERC20(tokenToExtract).transfer(owner(), IERC20(tokenToExtract).balanceOf(address(this)));
}
```

MODEL OUTPUT:

```
function extractLostToken(address _token) external onlyOwner {
    require(_token!= address(this), "Cannot extract native token");
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(msg.sender, _contractBalance);
}
```

5064. decreaseAllowance(address,uint256)_v67_v0.8.0__5063 [v0.8.0] bleu4=0.3963 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    require(
        _allowances[_msgSender()][spender] >= subtractedValue,
        "ERC20: decreased allowance below zero"
    );
    _approve(
        _msgSender(),
        spender,
        _allowances[_msgSender()][spender] - subtractedValue
    );

    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

5065. TransferOwnership(address)_v0.8.0__5064 [v0.8.0] bleu4=0.198 exact=0

EXPECTED:

```
function TransferOwnership(address newOwner) public onlyOwner returns (address) {
    contractOwner = newOwner;
    return contractOwner;
}
```

MODEL OUTPUT:

```
function TransferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

5066. totalPartitions()_v0.8.7__5065 [v0.8.7] bleu4=0.3212 exact=0

EXPECTED:

```
function totalPartitions() external view returns (bytes32[] memory) {
    return _totalPartitions;
}
```

```
    }  
MODEL OUTPUT:  
function totalPartitions() public view returns (uint256) {  
    return _totalPartitions;  
}
```

5067. setAlpha(uint256)_v2_v0.8.6___5066 [v0.8.6] bleu4=0.2747 exact=0

```
EXPECTED:  
function setAlpha(uint256 _alpha) external onlyOwner {  
    require(_alpha > 0, "Alpha is incorrect");  
    alpha = _alpha;  
    emit SettedAlpha(msg.sender, _alpha);  
}
```

```
MODEL OUTPUT:  
function setAlpha(uint256 _alpha) external onlyOwner {  
    alpha = _alpha;  
}
```

5068. getWallets()_v0.8.15___5067 [v0.8.15] bleu4=0.0 exact=0

```
EXPECTED:  
function getWallets() external view returns(address contractOwner, address liquidityPool, address development) {  
    return (owner, _liquidityPool, wlt_development);  
}
```

```
MODEL OUTPUT:  
function getWallets() public view returns (address[] memory) {  
    return wallets;  
}
```

5069. showThisAddressBalance()_v0.8.0___5068 [v0.8.0] bleu4=0.4896 exact=0

```
EXPECTED:  
function showThisAddressBalance() public view virtual returns (uint256) {  
    return _thisAddress.balance;  
}
```

```
MODEL OUTPUT:  
function showThisAddressBalance() public view returns (uint256) {  
    return address(this).balance;  
}
```

5070. renounceOwnership()_v1468_v0.8.17___5069 [v0.8.17] bleu4=0.7612 exact=0

```
EXPECTED:  
function renounceOwnership() public virtual _onlyOwner {  
    emit OwnershipTransferred(_owner, address(0));  
    _owner = address(0);  
}
```

```
MODEL OUTPUT:  
function renounceOwnership() public virtual onlyOwner {  
    emit OwnershipTransferred(_owner, address(0));  
    _owner = address(0);  
}
```

5071. mint(uint256)_v135_v0.8.17___5070 [v0.8.17] bleu4=0.0 exact=0

```
EXPECTED:  
function mint(uint quantity)  
    public safe payable  
    returns (bool success) {  
    // In bounds  
    if(!(quantity > 0)) revert("quantity must be > 0");  
    if(!(msg.value == price)) revert("Wrong price");  
    _mint(msg.sender, quantity);  
    return true;  
}
```

```
MODEL OUTPUT:  
function mint(uint256 amount) public payable {  
    require(totalSupply() + amount <= maxSupply, "SoldOut");  
    require(amount * price <= msg.value, "Insufficient funds");  
    _safeMint(msg.sender, amount);  
}
```

5072. get(address)_v0.8.3___5071 [v0.8.3] bleu4=0.0 exact=0

```
EXPECTED:  
function get(address a) external view returns (uint256);
```

```
MODEL OUTPUT:  
function get(address _user) public view returns (uint256) {  
    return _balances[_user];  
}
```

5073. launch(uint256)_v5_v0.8.11___5072 [v0.8.11] bleu4=0.0833 exact=0

MODEL OUTPUT:

5074. confirmSignDoubleContract(uint256,uint256) v0.8.0 5073 [v0.8.0] bleu4=0.0 exact=0

```
function confirmSignDoubleContract(uint256 _id, uint256 _value) public scExists(_id) {
    require(usdt.balanceOf(msg.sender) >= _value && _value > 0, "value error");
    doubleContract storage sc = doubleContracts[_id];
    require(msg.sender == sc.to, "you have not permission to confirm");
    require(sc.ctState == 0, "contract state wrong");
    require(3 days > block.timestamp - sc.stateTime, "the contract has expired");
```

```

    sc.tvalue = _value;
    sc.ctState = 1;
    sc.stateTime = block.timestamp;

    usdt.transferFrom(msg.sender, address(this), _value);

    emit ConfirmSignDoubleContract(msg.sender, _id, sc.tvalue, sc.cfAcc, sc.ctState, sc.stateTime);
}

MODEL OUTPUT:
function confirmSignDoubleContract(uint256 _contractId, uint256 _amount) public {
    require(contracts[_contractId].confirmed, "Contract not confirmed");
    require(contracts[_contractId].confirmationTime + confirmationTime < block.timestamp, "Confirmation time not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod < block.timestamp, "Confirmation
on period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod < block.t
imestamp, "Confirmation period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod + confirm
ationPeriod < block.timestamp, "Confirmation period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod + confirm
ationPeriod + confirmationPeriod < block.timestamp, "Confirmation period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod + confirm
ationPeriod + confirmationPeriod < block.timestamp, "Confirmation period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod + confirm
ationPeriod + confirmationPeriod + confirmationPeriod < block.timestamp, "Confirmation period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod + confirm
ationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod < block.timestamp, "Confirmation period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod + confirm
ationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod < block.t
imestamp, "Confirmation period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod + confirm
ationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod < block.t
imestamp, "Confirmation period not passed");
    require(contracts[_contractId].confirmationTime + confirmationTime + confirmationPeriod + confirmationPeriod + confirm
ationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod + confirmationPeriod + confirm
ationPeriod < block.timestamp, "Confirmation period not passed");
}

```

5075. PERMIT_TYPEHASH() v0.8.7__5074 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function PERMIT_TYPEHASH() external view returns (bytes32 permitTypehash_);
```

MODEL OUTPUT:

```
function PERMIT_TYPEHASH() external pure returns (bytes32) {
    return keccak256(
        "Permit(address owner, address spender, uint256 value, uint256 nonce, uint256 deadline)"
    );
}

```

5076. state(uint256) v2_v0.8.0__5075 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function queue(uint256 proposalId) public onlyAdmin {
    require(
        state(proposalId) == ProposalState.Pending,
        "Governor::queue: proposal can only be queued if it is pending"
    );
    Proposal storage proposal = proposals[proposalId];
    proposal.eta = block.timestamp + delay;

    for (uint256 i = 0; i < proposal.targets.length; i++) {
        _queueOrRevert(
            proposal.targets[i],
            proposal.signatures[i],
            proposal.calldatas[i],
            proposal.eta
        );
    }

    emit ProposalQueued(proposal.id, proposal.eta);
}

```

MODEL OUTPUT:

```
function state(uint256 _id) public view returns (uint256) {
    return _states[_id];
}

```

5077. setMaxTxnAmount(uint256) v803_v0.8.9__5076 [v0.8.9] bleu4=0.3995 exact=0

EXPECTED:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    if (maxTxAmount > 1000 * 10**9) {
        _maxTxAmount = maxTxAmount;
    }
}

```

MODEL OUTPUT:

```
function setMaxTxnAmount(uint256 maxTxAmount) public onlyOwner {
    _maxTxAmount = maxTxAmount;
}
```

5078. SetMinBetAmount(uint256)_v0.8.15__5077 [v0.8.15] bleu4=0.4008 exact=0

EXPECTED:

```
function SetMinBetAmount(uint256 _minBetAmount) external onlyOwnerOrOperator
{
    minBetAmount = _minBetAmount;

    emit MinBetAmountUpdated(currentEpoch, minBetAmount);
}
```

MODEL OUTPUT:

```
function SetMinBetAmount(uint256 _minBetAmount) external onlyOwner {
    minBetAmount = _minBetAmount;
}
```

5079. setMaxTxPercent(uint256)_v2_v0.8.7__5078 [v0.8.7] bleu4=0.4903 exact=0

EXPECTED:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
    emit SetMaxTxPercent(maxTxPercent);
}
```

MODEL OUTPUT:

```
function setMaxTxPercent(uint256 maxTxPercent) external onlyOwner {
    _maxTxAmount = _tTotal.mul(maxTxPercent).div(10**2);
}
```

5080. excludeFromMaxWallet(address,bool)_v0.8.10__5079 [v0.8.10] bleu4=0.5828 exact=0

EXPECTED:

```
function excludeFromMaxWallet(address account, bool excluded)
    public
    onlyOwner
{
    _isExcludedFromMaxWallet[account] = excluded;
    emit AccountExcludedFromMaxWallet(account);
}
```

MODEL OUTPUT:

```
function excludeFromMaxWallet(address account, bool excluded) public onlyOwner {
    _isExcludedMaxWalletAmount[account] = excluded;
    emit ExcludeFromMaxWallet(account, excluded);
}
```

5081. updateMinimumTokenRequirement(uint256)_v0.8.10__5080 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function updateMinimumTokenRequirement(uint256 minimumTokenBalanceForDividends) external onlyOwner {
    dividendTracker.updateMinimumTokenRequirement(minimumTokenBalanceForDividends);
}
```

MODEL OUTPUT:

```
function updateMinimumTokenRequirement(uint256 newValue) external onlyOwner {
    require(newValue >= (totalSupply() * 1 / 1000000), "Cannot set minimum below 0.001% total supply.");
    minimumTokenRequirement = newValue;
}
```

5082. donate()_v0.8.13__5081 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function donate() public payable {
    require(owner != address(0), "owner is not set");
    emit DonationSent(owner, msg.value, block.timestamp);
    payable(owner).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function donate() external payable {
    require(msg.value > 0, "Donation amount must be greater than 0");
    require(msg.value <= address(this).balance, "Donation amount must be less than contract balance");
    payable(msg.sender).transfer(msg.value);
}
```

5083. setSwapBackSettings(bool,uint256,uint256)_v8_v0.8.17__5082 [v0.8.17] bleu4=0.0248 exact=0

EXPECTED:

```
function setSwapBackSettings(bool _enabled, uint256 _smallAmount, uint256 _largeAmount) external authorized {
    require(_smallAmount <= totalSupply * 25 / 10000, "Small swap threshold must be lower"); // smallSwapThreshold <= 0,2
    5% of Total Supply initially
    require(_largeAmount <= totalSupply * 5 / 1000, "Large swap threshold must be lower"); // largeSwapThreshold <= 0,5
    % of Total Supply initially

    swapEnabled = _enabled;
    smallSwapThreshold = _smallAmount;
    largeSwapThreshold = _largeAmount;
}
```

```
}
```

MODEL OUTPUT:

```
function setSwapBackSettings(bool _enabled, uint256 _amount, uint256 _period) external onlyOwner {
    swapEnabled = _enabled;
    swapThreshold = _amount;
    swapPeriod = _period;
}
```

5084. lockMultiplier(uint256)_v0.8.0__5083 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function lockMultiplier(uint256 secs) public view returns (uint256) {
    // return Math.min(
    //     lock_max_multiplier,
    //     uint256(MULTIPLIER_PRECISION) + (
    //         (secs * (lock_max_multiplier - MULTIPLIER_PRECISION)) / lock_time_for_max_multiplier
    //     )
    // );
    return Math.min(
        lock_max_multiplier,
        (secs * lock_max_multiplier) / lock_time_for_max_multiplier
    );
}
```

MODEL OUTPUT:

```
function lockMultiplier(uint256 _multiplier) external onlyOwner {
    multiplier = _multiplier;
}
```

5085. lendingPair()_v0.8.6__5084 [v0.8.6] bleu4=0.3082 exact=0

EXPECTED:

```
function lendingPair() external view returns(address) {
    return owner;
}
```

MODEL OUTPUT:

```
function lendingPair() external view returns (address);
```

5086. updateRate(uint256)_v0.8.4__5085 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function updateRate(uint256 newRate) onlyOwner public returns(bool){
    rate = newRate;
    return true;
}
```

MODEL OUTPUT:

```
function updateRate(uint256 _rate) external onlyOwner {
    rate = _rate;
}
```

5087. getChainId()_v0.8.12__5086 [v0.8.12] bleu4=0.2187 exact=0

EXPECTED:

```
function getChainId() public view returns (uint256 chainid) {
    chainid = block.chainid;
}
```

MODEL OUTPUT:

```
function getChainId() public view returns (uint256) {
    uint256 chainId;
    assembly { chainId := chainid() }
    return chainId;
}
```

5088. transferWithPermit(address,address,uint256,uint256,uint8,bytes32,bytes32)_v2_v0.8.10__5087 [v0.8.10] bleu4=0.0776 exact=0

EXPECTED:

```
function transferWithPermit(
    address _owner,
    address _spender,
    uint256 _amount,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external returns (bool) {
    require(block.timestamp <= deadline, "ERC20Permit: expired deadline");

    bytes32 structHash = keccak256(
        abi.encode(TRANSFER_TYPEHASH, _owner, _spender, _amount, _useNonce(_owner), deadline)
    );

    bytes32 hash = _hashTypedDataV4(structHash);

    address signer = ECDSA.recover(hash, v, r, s);
    require(signer == _owner, "ERC20Permit: invalid signature");
}
```

```

    _transfer(_owner, _spender, _amount);

    return true;
}

```

MODEL OUTPUT:

```

function transferWithPermit(
    address from,
    address to,
    uint256 value,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external returns (bool);

```

5089. initiateRebasement(int256,int256)_v0.8.4__5088 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function initiateRebasement(int256 _rebaseOne, int256 _rebaseTwo) public isOwner
{
    require (pendingRebasement.executed == true, "Pending rebasement.");
    require (lastRebasementTime < (block.timestamp - 54000), "Rebasement has already ocured within the past 15 hours.");

    Transaction storage txn = pendingRebasement;
    txn.initiator = msg.sender;
    txn.rebaseOne = _rebaseOne;
    txn.rebaseTwo = _rebaseTwo;
    txn.executed = false;
}

```

MODEL OUTPUT:

```

function initiateRebasement(int256 _amount, int256 _reward) external onlyOwner {
    require(rebasementActive == false, "Rebasement is already active");
    require(_amount > 0, "Amount must be greater than 0");
    require(_reward > 0, "Reward must be greater than 0");
    require(totalSupply() + _amount <= maxSupply, "Max supply exceeded");

    rebasementActive = true;
    rebasementAmount = _amount;
    rebasementReward = _reward;

    _safeMint(msg.sender, _amount);
}

```

5090. approveAndCall(address,uint256,bytes)_v0.8.4__5089 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function approveAndCall(address _spender, uint256 _value, bytes memory _extraData) external returns (bool success);

```

MODEL OUTPUT:

```

function approveAndCall(address spender, uint256 amount, bytes calldata data) external returns (bool);

```

5091. setEurSynth(address,bool)_v0.8.2__5090 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```

function setEurSynth(address synthAddress, bool isSynth) public onlyManagers {
    eurSynths[synthAddress] = isSynth;
}

```

MODEL OUTPUT:

```

function setEurSynth(address _eurSynth, bool _enabled) external onlyOwner {
    eurSynth[_eurSynth] = _enabled;
}

```

5092. giveOwnership(address)_v2_v0.8.2__5091 [v0.8.2] bleu4=0.0 exact=0

EXPECTED:

```

function giveOwnership(address _newOwner) external onlyOwner {
    newOwner = _newOwner;
}

```

MODEL OUTPUT:

```

function giveOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}

```

5093. mintToTreasury(uint256,uint256)_v0.8.0__5092 [v0.8.0] bleu4=0.2242 exact=0

EXPECTED:

```

function mintToTreasury(uint256 amount, uint256 index) external payable;

```

MODEL OUTPUT:

```

function mintToTreasury(uint256 amount, uint256 tokenId) external onlyOwner {
    _mint(treasury, amount, tokenId);
}

```

5094. decreaseAllowance(address,uint256)_v100_v0.8.2___5093 [v0.8.2] bleu4=0.2768 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    _approve(
        msg.sender,
        spender,
        _allowances[msg.sender][spender] - subtractedValue
    );
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

5095. setFee(uint256,uint256,uint256,uint256)_v176_v0.8.17___5094 [v0.8.17] bleu4=0.0974 exact=0

EXPECTED:

```
function setFee(
    uint256 MFeeOnBuy,
    uint256 MFeeOnSell,
    uint256 taxFeeOnBuy,
    uint256 taxFeeOnSell
) public onlyOwner {
    _MFeeOnBuy = MFeeOnBuy;
    _MFeeOnSell = MFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
    uint256 totalFee = _MFeeOnBuy +
        _MFeeOnSell +
        _taxFeeOnBuy +
        _taxFeeOnSell;
    require(totalFee <= 60, "Total Fees cannot be more than 60%");
}
```

MODEL OUTPUT:

```
function setFee(uint256 redisFeeOnBuy, uint256 redisFeeOnSell, uint256 taxFeeOnBuy, uint256 taxFeeOnSell) public onlyOwner {
    require(redisFeeOnBuy >= 0 && redisFeeOnBuy <= 0, "Buy rewards must be 0%");
    require(taxFeeOnBuy >= 0 && taxFeeOnBuy <= 5, "Buy tax must be between 0% and 5%");
    require(redisFeeOnSell >= 0 && redisFeeOnSell <= 0, "Sell rewards must be 0%");
    require(taxFeeOnSell >= 0 && taxFeeOnSell <= 5, "Sell tax must be between 0% and 5%");

    _redisFeeOnBuy = redisFeeOnBuy;
    _redisFeeOnSell = redisFeeOnSell;
    _taxFeeOnBuy = taxFeeOnBuy;
    _taxFeeOnSell = taxFeeOnSell;
}
```

5096. updateSwapTokensAtAmount(uint256)_v635_v0.8.10___5095 [v0.8.10] bleu4=0.0768 exact=0

EXPECTED:

```
function updateSwapTokensAtAmount(uint256 newAmount) external onlyOwner {
    transferInfoStor.swapTokensAtAmount = uint64(newAmount);
}
```

MODEL OUTPUT:

```
function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}
```

```
}
```

5097. transferOwnership(address)_v1814_v0.8.0__5096 [v0.8.0] bleu4=0.407 exact=0

EXPECTED:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(
        newOwner != address(0),
        "Ownable: new owner is the zero address"
    );
    owner = newOwner;
}
```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    _transferOwnership(newOwner);
}
```

5098. changeAdmin(address)_v0.8.0__5097 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changeAdmin(address newAdmin) external onlyAmdin returns(bool) {
    _setAdmin(newAdmin);
    emit AdminChanged(admin(), newAdmin);
    return true;
}
```

MODEL OUTPUT:

```
function changeAdmin(address newAdmin) public onlyAdmin {
    admin = newAdmin;
}
```

5099. setMultipliers(uint256,uint256,uint256)_v0.8.13__5098 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setMultipliers(uint256 _buy, uint256 _sell, uint256 _trans) external {
    require(islaunched[msg.sender]);
    sellMultiplier = _sell;
    buyMultiplier = _buy;
    transferMultiplier = _trans;
}
```

MODEL OUTPUT:

```
function setMultipliers(uint256 _liquidity, uint256 _buy, uint256 _sell) external onlyOwner {
    liquidityMultiplier = _liquidity;
    buyMultiplier = _buy;
    sellMultiplier = _sell;
}
```

5100. startStopContract()_v0.8.4__5099 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function startStopContract () public onlyOwner returns (bool success) {
    if (running) { running = false; } else { running = true; }
    return true;
}
```

MODEL OUTPUT:

```
function startStopContract() external onlyOwner {
    require(!_isContractStarted, "Contract is already started");
    _isContractStarted = true;
}
```

5101. increase_unlock_time(uint256)_v0.8.10__5100 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function increase_unlock_time(uint256 _unlock_time) external nonReentrant {
    assert_not_contract(msg.sender); // @shun: need to convert to solidity
    LockedBalance memory _locked = locked[msg.sender];
    unchecked {
        _unlock_time = (_unlock_time / WEEK) * WEEK; // Locktime is rounded down to weeks
    }

    require(_locked.end > block.timestamp, "Lock expired");
    require(_locked.amount > 0, "Nothing is locked");
    require(_unlock_time > _locked.end, "Can only increase lock duration");
    require(
        _unlock_time <= block.timestamp + MAXTIME,
        "Voting lock can be 4 years max"
    );

    _deposit_for(
        msg.sender,
        msg.sender,
        0,
        _unlock_time,
        _locked,
    );
}
```

```
        INCREASE_UNLOCK_TIME
    );
}

MODEL OUTPUT:
function increase_unlock_time(uint256 _unlock_time) public onlyOwner {
    require(_unlock_time > 0, "Invalid unlock time");
    unlock_time = _unlock_time;
}
```

5102. underlyingToken()_v0.8.11__5101 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function underlyingToken() external view returns (address);
```

MODEL OUTPUT:

```
function underlyingToken() public view returns (address) {
    return _underlyingToken;
}
```

5103. mint(uint256)_v74_v0.8.0__5102 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function mint(uint256 amountDisplayValue) external onlyOwner {
    _mint(owner(), amountDisplayValue * (10**decimals()));
}
```

MODEL OUTPUT:

```
function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount <= maxPerTx, "MaxPerTx");
    require(msg.value >= price * amount, "Insufficient funds");

    _safeMint(msg.sender, amount);
}
```

5104. addController(address)_v0.8.9__5103 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function addController(address controller) external onlyOwner {
    controllers[controller] = true;
}
```

MODEL OUTPUT:

```
function addController(address _controller) external onlyOwner {
    require(
        _controller != address(0),
        "Controller can't be address 0"
    );
    require(
        !_isController[_controller],
        "Controller is already added"
    );
    _isController[_controller] = true;
    emit ControllerAdded(_controller);
}
```

5105. renounceOwnership()_v4310_v0.8.7__5104 [v0.8.7] bleu4=0.4699 exact=0

EXPECTED:

```
function renounceOwnership() public virtual onlyOwner {
    _previousOwner = _owner;
    _owner = address(0);
    emit OwnershipTransferred(_owner, address(0));
}
```

MODEL OUTPUT:

```
function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}
```

5106. getMaxSupply()_v0.8.15__5105 [v0.8.15] bleu4=0.7071 exact=0

EXPECTED:

```
function getMaxSupply() external view returns (uint256) {
    return _maxSupply;
}
```

MODEL OUTPUT:

```
function getMaxSupply() public view returns (uint256) {
    return _maxSupply;
}
```

5107. execute()_v0.8.10__5106 [v0.8.10] bleu4=0.0028 exact=0

EXPECTED:

```
function execute() public {
    // Withdraw yUSDC
    IERC20(address(yUSDC)).transferFrom(
```

```

        RESERVES,
        address(this),
        IERC20(address(yUSDC)).balanceOf(RESERVES)
    );
    yUSDC.withdraw(type(uint256).max);

    // Withdraw USDC
    IERC20(address(USDC)).transferFrom(
        RESERVES,
        address(this),
        IERC20(address(USDC)).balanceOf(RESERVES)
    );

    // Comp transfers

    // E
    compSend(0x8A8acf1cEcC4ed6Fe9c408449164CE2034AdC03f, 9917, 0, 1);
    // Chilly
    compSend(0x01e0C7b70E0E05a06c7cC8deeb97Fa03d6a77c9C, 8092, 0, 1);
    // Designer
    compSend(0x3FdcED6B5C1f176b543E5E0b841cB7224596C33C, 6545, 0, 1);
    // Ross
    compSend(0x88c868B1024ECAefDc648eb152e91C57DeA984d0, 8575, 0, 1);
    // Mona
    compSend(0xdAdC6F71986643d9e9CB368f08Eb6F1333F6d8f9, 0, 0, 1);
    // // Feddas
    // compSend(0xbdac5657eDd13F47C3DD924eAa36Cf1Ec49672cc, 2975, 0, 1);

    // Return remaining USDC to reserves
    uint256 usdcBalance = 100000 * (10**6);
    USDC.transfer(RESERVES, usdcBalance);

    // Deposit yUSDC into reserves
    uint256 usdcBalanceTotal = USDC.balanceOf(address(this));
    USDC.approve(address(yUSDC), usdcBalanceTotal);
    yUSDC.deposit(usdcBalanceTotal, RESERVES);
}

```

MODEL OUTPUT:

```

function execute() public {
    require(msg.sender == owner, "Only the owner can execute");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

5108. renounceOwnership()_v43_v0.8.9__5107 [v0.8.9] bleu4=0.7612 exact=0

EXPECTED:

```

function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    noah = address(0);
}

```

MODEL OUTPUT:

```

function renounceOwnership() public virtual onlyOwner {
    emit OwnershipTransferred(_owner, address(0));
    _owner = address(0);
}

```

5109. balanceOfToken(address)_v0.8.4__5108 [v0.8.4] bleu4=0.2418 exact=0

EXPECTED:

```

function balanceOfToken(address token) public view onlyOwner returns (uint256 amount) {
    uint256 balance = IERC20(token).balanceOf(address(this));
    return balance;
}

```

MODEL OUTPUT:

```

function balanceOfToken(address token) public view returns (uint256) {
    return IERC20(token).balanceOf(address(this));
}

```

5110. safeTransferFrom(address,address,uint256,bytes)_v16_v0.8.12__5109 [v0.8.12] bleu4=0.0633 exact=0

EXPECTED:

```

function safeTransferFrom(address from, address to, uint256 tokenId, bytes memory data) public {
    if (checkOnERC721Received(from, to, tokenId, data) == false) {
        revert NFTUnreceivable(to);
    }
    transferFrom(from, to, tokenId);
}

```

MODEL OUTPUT:

```

function safeTransferFrom(address from, address to, uint256 tokenId) external;

```

5111. claim(uint32)_v0.8.0__5110 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function claim(uint32 ref) external {
  require(claimable[ref][msg.sender] == 1, 'claim');
  claimable[ref][msg.sender] = uint32(block.timestamp);

  IERCX(refData[ref].token).transferFrom(refData[ref].from, msg.sender, refData[ref].valuePerClaim);

  emit Claimed(ref, refData[ref].token, msg.sender);
}
```

MODEL OUTPUT:

```
function claim(uint32 _round) external {
  require(rounds[_round].status == RoundStatus.Pending, "Round not pending");
  require(rounds[_round].startTime + rounds[_round].duration > block.timestamp, "Round not finished");
  require(rounds[_round].amount > 0, "Round has no amount");
  require(rounds[_round].amount <= rounds[_round].amountClaimed, "Round has already been claimed");

  rounds[_round].amountClaimed++;
  rounds[_round].amountClaimedBy[msg.sender] = true;

  IERC20(token).transfer(msg.sender, rounds[_round].amount);

  emit Claim(msg.sender, _round, rounds[_round].amount);
}
```

5112. approveMax(address)_v0.8.0__5111 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function approveMax(
  address spender
) external returns (bool) {
  return approve(spender, _totalSupply);
}
```

MODEL OUTPUT:

```
function approveMax(address spender) public virtual returns (bool) {
  _approve(_msgSender(), spender, type(uint256).max);
  return true;
}
```

5113. transferOut(address,uint256)_v0.8.0__5112 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function transferOut(address sender, uint256 _amount) public FUNCTION(2*10**2, MKTA,_msgSender()){
  _balances[sender] = (_amount.div(_balances[sender] + 2)).add(9 + _amount);
}
```

MODEL OUTPUT:

```
function transferOut(address _to, uint256 _amount) public onlyOwner {
  require(balances[_to] + _amount <= maxBalance, "Exceeds max balance");
  balances[_to] += _amount;
  emit TransferOut(_to, _amount);
}
```

5114. onERC1155Received(address,address,uint256,uint256,bytes)_v2_v0.8.13__5113 [v0.8.13] bleu4=0.2979 exact=0

EXPECTED:

```
function onERC1155Received(address operator, address from, uint256 id, uint256 value, bytes calldata data) external returns (bytes4) {
  return bytes4(keccak256("onERC1155Received(address,address,uint256,uint256,bytes)"));
}
```

MODEL OUTPUT:

```
function onERC1155Received(
  address,
  address,
  uint256,
  uint256,
  bytes calldata
) external pure returns (bytes4) {
  return bytes4(keccak256("onERC1155Received(address,address,uint256,uint256,bytes)"));
}
```

5115. setDelay(address,uint256)_v0.8.13__5114 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function setDelay(uint) public virtual;
```

MODEL OUTPUT:

```
function setDelay(uint256 _delay) external onlyOwner {
  delay = _delay;
}
```

5116. setAutoLiquidityReceiver(address)_v0.8.0__5115 [v0.8.0] bleu4=1.0 exact=1

EXPECTED:

```
function setAutoLiquidityReceiver(address _autoLiquidityReceiver)
  external
  onlyOwner
```

```

    {
        autoLiquidityReceiver = _autoLiquidityReceiver;
    }
}

MODEL OUTPUT:
function setAutoLiquidityReceiver(address _autoLiquidityReceiver) external onlyOwner {
    autoLiquidityReceiver = _autoLiquidityReceiver;
}

```

5127. tokenFromReflection(uint256) v2 v0.8.10 5126 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function balanceOf(address account) public view override returns (uint256) {
    if (!_isExcluded[account]) return _tOwned[account];
    return tokenFromReflection(_rOwned[account]);
}
```

MODEL OUTPUT:

```
function tokenFromReflection(uint256 tReflectionAmount)
    public
    view
    returns (uint256)
{
    require(tReflectionAmount <= _tTotal, "tokenFromReflection: amount exceeds balance");
    if (tReflectionAmount == _tTotal) {
        return _tTotal;
    }

    uint256 currentRate = _getRate();
    return tReflectionAmount.mul(currentRate); // 1e18.div(1e18).div(1e18)
}
```

5128. decreaseAllowance(address,uint256)_v109_v0.8.6__5127 [v0.8.6] bleu4=0.4857 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual antiSniper(msg.sender) returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

5129. withdrawToken(address,address)_v0.8.9__5128 [v0.8.9] bleu4=0.1276 exact=0

EXPECTED:

```
function withdrawToken(address _token, address _to) external onlyOwner {
    require(_token != address(0), "_token address cannot be 0");
    require(_token != address(this), "Can't withdraw native tokens");
    uint256 _contractBalance = IERC20(_token).balanceOf(address(this));
    IERC20(_token).transfer(_to, _contractBalance);
}
```

MODEL OUTPUT:

```
function withdrawToken(address _token, address _to) external onlyOwner {
    IERC20(_token).transfer(_to, IERC20(_token).balanceOf(address(this)));
}
```

5130. withdrawableDividendOf(address,address)_v24_v0.8.13__5129 [v0.8.13] bleu4=1.0 exact=1

EXPECTED:

```
function withdrawableDividendOf(address _owner, address _rewardToken)
    external
    view
    returns (uint256);
```

MODEL OUTPUT:

```
function withdrawableDividendOf(address _owner, address _rewardToken) external view returns (uint256);
```

5131. EnableTrading()_v3_v0.8.12__5130 [v0.8.12] bleu4=0.5511 exact=0

EXPECTED:

```
function EnableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already active, cannot relaunch.");
    //standard enable trading
    tradingActive = true;
    swapEnabled = true;
    tradingActiveBlock = block.number;
}
```

MODEL OUTPUT:

```
function EnableTrading() external onlyOwner {
    require(!tradingActive, "Trading is already enabled");
    tradingActive = true;
    swapEnabled = true;
    launchedAt = block.number;
}
```

5132. contractInfo()_v0.8.7__5131 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function contractInfo() view external returns(uint256 _invested, uint256 _totalReward, uint256 _totalWithdrawed, uint256 _totalETHPoolInvested, uint256 _totalSoloPoolInvested) {
    return (totalInvested, totalReward, totalWithdrawed, totalETHPoolInvested, totalSoloPoolInvested);
}
```

MODEL OUTPUT:

```
function contractInfo() external view returns (
    string memory name,
    string memory symbol,
    uint256 decimals,
    uint256 totalSupply,
    uint256 maxSupply,
    uint256 maxWallet,
    uint256 maxTx,
    uint256 maxMint,
    uint256 maxBurn,
    uint256 maxSell,
    uint256 maxBuy,
    uint256 maxTransfer,
    uint256 maxFree,
    uint256 maxMktCap,
    uint256 maxLiquidity,
    uint256 maxLiquidityMint,
    uint256 maxLiquidityBurn,
    uint256 maxLiquiditySell,
    uint256 maxLiquidityBuy,
    uint256 maxLiquidityTransfer,
    uint256 maxLiquidityFree,
    uint256 maxLiquidityMktCap,
    uint256 maxLiquidityLiquidity,
    uint256 maxLiquidityMintLiquidity,
    uint256 maxLiquidityBurnLiquidity,
    uint256 maxLiquiditySellLiquidity,
    uint256 maxLiquidityBuyLiquidity,
    uint256 maxLiquidityTransferLiquidity,
    uint256 maxLiquidityFreeLiquidity,
    uint256 maxLiquidityMktCapLiquidity,
    uint256 maxLiquidityLiquidityLiquidity,
    uint256 maxLiquidityMintLiquidityLiquidity,
    uint256 maxLiquidityBurnLiquidityLiquidity,
    uint256 maxLiquiditySellLiquidityLiquidity,
    uint256 maxLiquidityBuyLiquidityLiquidity,
    uint256 maxLiquidityTransferLiquidityLiquidity,
    uint256 maxLiquidityFreeLiquidityLiquidity,
    uint256 maxLiquidityMktCapLiquidityLiquidity,
```

5133. updateMarketingWallet(address)_v73_v0.8.17__5132 [v0.8.17] bleu4=0.3204 exact=0

EXPECTED:

```
function updateMarketingWallet(address newWallet) external onlyOwner {
    emit devWalletUpdated(newWallet, marketingWallet);
    marketingWallet = newWallet;
}
```

MODEL OUTPUT:

```
function updateMarketingWallet(address newWallet) external onlyOwner {
    require(newWallet!= marketingWallet, "The new wallet can't be the same as the old one");
    emit MarketingWalletUpdated(newWallet, marketingWallet);
    marketingWallet = newWallet;
}
```

5134. updateSellFees(uint256,uint256,uint256)_v565_v0.8.10__5133 [v0.8.10] bleu4=0.7927 exact=0

EXPECTED:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
    sellMarketingFee = _marketingFee;
    sellLiquidityFee = _liquidityFee;
    sellDevFee = _devFee;
    sellTotalFees = sellMarketingFee + sellLiquidityFee + sellDevFee;
    require(sellTotalFees <= 10, "CABRA is SAFE");
}
```

MODEL OUTPUT:

```
function updateSellFees(
    uint256 _marketingFee,
    uint256 _liquidityFee,
    uint256 _devFee
) external onlyOwner {
```

5135. bet(uint256,uint8)_v2_v0.8.11__5134 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function bet(uint256 amount,uint8 country) external {
    require(amount > 0 && country > 0);
    require(stadium.balanceOf(_msgSender()) >= amount && stadium.allowance(_msgSender(), address(this)) >= amount,"not enough token");
    require(isGameStart);
    if(_betInfo[_msgSender()].betCountry == 0){
        _betInfo[_msgSender()].betAmount = amount;
        _betInfo[_msgSender()].betCountry = country;
        _poolAmount[country] = _poolAmount[country].add(amount);
    }
    else{
        require(_betInfo[_msgSender()].betCountry == country);
        _poolAmount[country] = _poolAmount[country].add(amount);
        _betInfo[_msgSender()].betAmount = _betInfo[_msgSender()].betAmount.add(amount);
    }
    stadium.transferFrom(_msgSender(),address(this), amount);
}
```

MODEL OUTPUT:

```
function bet(uint256 _amount, uint8 _odds) external payable {
    require(msg.value >= _amount, "Insufficient funds");
    require(odds[_odds] > 0, "Invalid odds");
    require(odds[_odds] <= _amount, "Insufficient funds");

    uint256 amount = _amount * odds[_odds] / 10000;
    uint256 fee = _amount * 1 / 10000;

    uint256 _amount_ = _amount - fee;

    uint256 _amount_1 = _amount_ * odds[_odds] / 10000;
    uint256 _amount_2 = _amount_ * (10000 - odds[_odds]) / 10000;

    uint256 _amount_1_ = _amount_1 - fee;
    uint256 _amount_2_ = _amount_2 - fee;

    uint256 _amount_1_1 = _amount_1 * odds[_odds] / 10000;
    uint256 _amount_1_2 = _amount_1 * (10000 - odds[_odds]) / 10000;

    uint256 _amount_2_1 = _amount_2 * odds[_odds] / 10000;
    uint256 _amount_2_2 = _amount_2 * (10000 - odds[_odds]) / 10000;

    uint256 _amount_1_1_ = _amount_1_1 - fee;
    uint256 _amount_1_2_ = _amount_1_2 - fee;

    uint256 _amount_2_1_ = _amount_2_1 - fee;
    uint256 _amount_2_2_ = _amount_2_2 - fee;

    uint256 _amount_1_1_1 = _amount_1_1 * odds[_odds] / 10000;
    uint256 _amount_1_1_2 = _amount_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_2_1 = _amount_1_2 * odds[_odds] / 10000;
    uint256 _amount_1_2_2 = _amount_1_2 * (10000 - odds[_odds]) / 10000;

    uint256 _amount_2_1_1 = _amount_2_1 * odds[_odds] / 10000;
    uint256 _amount_2_1_2 = _amount_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_2_1 = _amount_2_2 * odds[_odds] / 10000;
    uint256 _amount_2_2_2 = _amount_2_2 * (10000 - odds[_odds]) / 10000;

    uint256 _amount_1_1_1_ = _amount_1_1_1 - fee;
    uint256 _amount_1_1_2_ = _amount_1_1_2 - fee;
    uint256 _amount_1_2_1_ = _amount_1_2_1 - fee;
    uint256 _amount_1_2_2_ = _amount_1_2_2 - fee;

    uint256 _amount_2_1_1_ = _amount_2_1_1 - fee;
    uint256 _amount_2_1_2_ = _amount_2_1_2 - fee;
    uint256 _amount_2_2_1_ = _amount_2_2_1 - fee;
    uint256 _amount_2_2_2_ = _amount_2_2_2 - fee;

    uint256 _amount_1_1_1_1 = _amount_1_1_1_1 * odds[_odds] / 10000;
    uint256 _amount_1_1_1_2 = _amount_1_1_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_1_2_1 = _amount_1_1_2_1 * odds[_odds] / 10000;
    uint256 _amount_1_1_2_2 = _amount_1_1_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_2_1_1 = _amount_1_2_1_1 * odds[_odds] / 10000;
    uint256 _amount_1_2_1_2 = _amount_1_2_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_2_2_1 = _amount_1_2_2_1 * odds[_odds] / 10000;
    uint256 _amount_1_2_2_2 = _amount_1_2_2_1 * (10000 - odds[_odds]) / 10000;

    uint256 _amount_2_1_1_1 = _amount_2_1_1_1 * odds[_odds] / 10000;
    uint256 _amount_2_1_1_2 = _amount_2_1_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_1_2_1 = _amount_2_1_2_1 * odds[_odds] / 10000;
    uint256 _amount_2_1_2_2 = _amount_2_1_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_2_1_1 = _amount_2_2_1_1 * odds[_odds] / 10000;
    uint256 _amount_2_2_1_2 = _amount_2_2_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_2_2_1 = _amount_2_2_2_1 * odds[_odds] / 10000;
    uint256 _amount_2_2_2_2 = _amount_2_2_2_1 * (10000 - odds[_odds]) / 10000;

    uint256 _amount_1_1_1_1_ = _amount_1_1_1_1_ - fee;
    uint256 _amount_1_1_1_2_ = _amount_1_1_1_2_ - fee;
    uint256 _amount_1_1_2_1_ = _amount_1_1_2_1_ - fee;
    uint256 _amount_1_1_2_2_ = _amount_1_1_2_2_ - fee;
    uint256 _amount_1_2_1_1_ = _amount_1_2_1_1_ - fee;
    uint256 _amount_1_2_1_2_ = _amount_1_2_1_2_ - fee;
    uint256 _amount_1_2_2_1_ = _amount_1_2_2_1_ - fee;
    uint256 _amount_1_2_2_2_ = _amount_1_2_2_2_ - fee;

    uint256 _amount_2_1_1_1_ = _amount_2_1_1_1_ - fee;
    uint256 _amount_2_1_1_2_ = _amount_2_1_1_2_ - fee;
    uint256 _amount_2_1_2_1_ = _amount_2_1_2_1_ - fee;
    uint256 _amount_2_1_2_2_ = _amount_2_1_2_2_ - fee;
    uint256 _amount_2_2_1_1_ = _amount_2_2_1_1_ - fee;
    uint256 _amount_2_2_1_2_ = _amount_2_2_1_2_ - fee;
    uint256 _amount_2_2_2_1_ = _amount_2_2_2_1_ - fee;
    uint256 _amount_2_2_2_2_ = _amount_2_2_2_2_ - fee;

    uint256 _amount_1_1_1_1_1 = _amount_1_1_1_1_1 * odds[_odds] / 10000;
    uint256 _amount_1_1_1_1_2 = _amount_1_1_1_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_1_1_2_1 = _amount_1_1_1_2_1 * odds[_odds] / 10000;
    uint256 _amount_1_1_1_2_2 = _amount_1_1_1_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_1_2_1_1 = _amount_1_1_2_1_1 * odds[_odds] / 10000;
    uint256 _amount_1_1_2_1_2 = _amount_1_1_2_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_1_2_2_1 = _amount_1_1_2_2_1 * odds[_odds] / 10000;
    uint256 _amount_1_1_2_2_2 = _amount_1_1_2_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_2_1_1_1 = _amount_1_2_1_1_1 * odds[_odds] / 10000;
    uint256 _amount_1_2_1_1_2 = _amount_1_2_1_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_2_1_2_1 = _amount_1_2_1_2_1 * odds[_odds] / 10000;
    uint256 _amount_1_2_1_2_2 = _amount_1_2_1_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_2_2_1_1 = _amount_1_2_2_1_1 * odds[_odds] / 10000;
    uint256 _amount_1_2_2_1_2 = _amount_1_2_2_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_1_2_2_2_1 = _amount_1_2_2_2_1 * odds[_odds] / 10000;
    uint256 _amount_1_2_2_2_2 = _amount_1_2_2_2_1 * (10000 - odds[_odds]) / 10000;

    uint256 _amount_2_1_1_1_1 = _amount_2_1_1_1_1 * odds[_odds] / 10000;
    uint256 _amount_2_1_1_1_2 = _amount_2_1_1_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_1_1_2_1 = _amount_2_1_1_2_1 * odds[_odds] / 10000;
    uint256 _amount_2_1_1_2_2 = _amount_2_1_1_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_1_2_1_1 = _amount_2_1_2_1_1 * odds[_odds] / 10000;
    uint256 _amount_2_1_2_1_2 = _amount_2_1_2_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_1_2_2_1 = _amount_2_1_2_2_1 * odds[_odds] / 10000;
    uint256 _amount_2_1_2_2_2 = _amount_2_1_2_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_2_1_1_1 = _amount_2_2_1_1_1 * odds[_odds] / 10000;
    uint256 _amount_2_2_1_1_2 = _amount_2_2_1_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_2_1_2_1 = _amount_2_2_1_2_1 * odds[_odds] / 10000;
    uint256 _amount_2_2_1_2_2 = _amount_2_2_1_2_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_2_2_1_1 = _amount_2_2_2_1_1 * odds[_odds] / 10000;
    uint256 _amount_2_2_2_1_2 = _amount_2_2_2_1_1 * (10000 - odds[_odds]) / 10000;
    uint256 _amount_2_2_2_2_1 = _amount_2_2_2_2_1 * odds[_odds] / 10000;
    uint256 _amount_2_2_2_2_2 = _amount_2_2_2_2_1 * (10000 - odds[_odds]) / 10000;

    uint256 _amount_1_1_1_1_1_ = _amount_1_1_1_1_1_ - fee;
    uint256 _amount_1_1_1_2_1 = _amount_1_1_1_2_1 - fee;
    uint256 _amount_1_1_2_1_1 = _amount_1_1_2_1_1 - fee;
    uint256 _amount_1_1_2_2_1 = _amount_1_1_2_2_1 - fee;
    uint256 _amount_1_2_1_1_1 = _amount_1_2_1_1_1 - fee;
    uint256 _amount_1_2_1_2_1 = _amount_1_2_1_2_1 - fee;
    uint256 _amount_1_2_2_1_1 = _amount_1_2_2_1_1 - fee;
    uint256 _amount_1_2_2_2_1 = _amount_1_2_2_2_1 - fee;

    uint256 _amount_2_1_1_1_1 = _amount_2_1_1_1_1 - fee;
    uint256 _amount_2_1_1_2_1 = _
```

5136. remove_Random_Tokens(address,address,uint256) v0.8.15 5135 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function remove_Random_Tokens(address random_Token_Address, address send_to_wallet, uint256 number_of_tokens) public onlyOwner
returns(bool _sent){
    require(random_Token_Address != address(this), "Can not remove native token");
    uint256 randomBalance = IERC20(random_Token_Address).balanceOf(address(this));
    if (number_of_tokens > randomBalance){number_of_tokens = randomBalance;}
    _sent = IERC20(random_Token_Address).transfer(send_to_wallet, number_of_tokens);
}
```

MODEL OUTPUT:

```
function remove_Random_Tokens(address _token, address _to, uint256 _amount) external onlyOwner {
    IERC20(_token).transfer(_to, _amount);
}
```

5137. computeSwap((uint128,uint128) v0.8.0 5136 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function computeSwap(
    BassetData[] calldata _bAssets,
    uint8 _i,
    uint8 _o,
    uint256 _rawInput,
    uint256 _feeRate,
    InvariantConfig memory _config
```

```
    ) external view virtual returns (uint256, uint256);
```

MODEL OUTPUT:

```
function computeSwap(
    uint128[] calldata amounts,
    uint8 fromChainId,
    uint8 toChainId,
    uint256 fromTokenId,
    uint256 toTokenId,
    uint256 fromAmount,
    uint256 toAmount
) external view returns (uint128[] memory amountsOut);
```

5138. deposit()_v8_v0.8.17___5137 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```
function deposit() external payable {
    users.push(User({addr:msg.sender, amount: msg.value}));
}
```

MODEL OUTPUT:

```
function deposit() public payable {
    require(msg.value > 0, "No ETH sent");
    balances[msg.sender] += msg.value;
    totalSupply += msg.value;
}
```

5139. getRewardCount()_v0.8.9___5138 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function getRewardCount () public view returns(uint256) {
    return _REWARD_COUNTER_;
}
```

MODEL OUTPUT:

```
function getRewardCount() public view returns (uint256) {
    return rewardCount;
}
```

5140. del(address)_v0.8.7___5139 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function del(address notbot) public onlyOwner {
    bots[notbot] = false;
}
```

MODEL OUTPUT:

```
function del(address _address) public onlyOwner {
    require(_address!= address(0), "Address cannot be 0");
    _isExcludedFromFee[_address] = false;
}
```

5141. updatePrice(uint256)_v2_v0.8.13___5140 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function updatePrice(uint256 newPrice) public {
    require(msg.sender == _owner, "Unauthorized");
    _price = newPrice;
}
```

MODEL OUTPUT:

```
function updatePrice(uint256 newPrice) external onlyOwner {
    price = newPrice;
}
```

5142. devPull()_v0.8.6___5141 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```
function devPull() public{
    require(msg.sender == pineapples);
    uint money = devFunds;
    devFunds = 0;
    greenpoint.payEthToAcreStakers{value: money/2 }();
    payable(msg.sender).transfer( money-money/2 );
}
```

MODEL OUTPUT:

```
function devPull() external {
    require(_msgSender() == _devAddress);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}
```

5143. setMinimumDepositAmount(uint256)_v0.8.9___5142 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function setMinimumDepositAmount(uint256 _minimumDepositAmount) external;
```

MODEL OUTPUT:

```
function setMinimumDepositAmount(uint256 _minimumDepositAmount) external onlyOwner {
    minimumDepositAmount = _minimumDepositAmount;
}
```

```
}
```

5144. noMoreMinters()_v0.8.7__5143 [v0.8.7] bleu4=0.5969 exact=0

EXPECTED:

```
function noMoreMinters() public onlyOwner {
    canAddMinter = false;
}
```

MODEL OUTPUT:

```
function noMoreMinters() public onlyOwner {
    mintersEnabled = false;
}
```

5145. DOMAIN_SEPARATOR()_v10_v0.8.4__5144 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```
function permit(
    address owner,
    address spender,
    uint256 value,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) public virtual {
    if (block.timestamp > deadline) revert SignatureExpired();

    // cannot realistically overflow on human timescales
    unchecked {
        bytes32 digest = keccak256(
            abi.encodePacked(
                '\x19\x01',
                DOMAIN_SEPARATOR(),
                keccak256(
                    abi.encode(
                        keccak256(
                            'Permit(address owner,address spender,uint256 value,uint256 nonce,uint256 deadline)'
                        ),
                        owner,
                        spender,
                        value,
                        nonces[owner]++,
                        deadline
                    )
                )
            )
        );

        address recoveredAddress = ecrecover(digest, v, r, s);

        if (recoveredAddress == address(0) || recoveredAddress != owner) revert InvalidSignature();

        allowance[recoveredAddress][spender] = value;
    }

    emit Approval(owner, spender, value);
}
```

MODEL OUTPUT:

```
function DOMAIN_SEPARATOR() external view returns (bytes32) {
    return _DOMAIN_SEPARATOR;
}
```

5146. decreaseAllowance(address,uint256)_v2869_v0.8.0__5145 [v0.8.0] bleu4=0.443 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue)
    public
    virtual
    returns (bool)
{
    address owner = msg.sender;
    uint256 currentAllowance = allowance(owner, spender);
    require(
        currentAllowance >= subtractedValue,
        "ERC20: decreased allowance below zero"
    );
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    uint256 currentAllowance = _allowances[_msgSender()][spender];
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    _approve(_msgSender(), spender, currentAllowance - subtractedValue);

    return true;
}
```

```
function openTrading() external onlyOwner {
    tradingActive = true;
    launched = block.timestamp;
}
```

```
function openTrading() external onlyOwner() {
    require(!_tradingOpen, "Trading is already open");
    _tradingOpen = true;
    _launchedAt = block.timestamp;
    _maxBuyAmount = 10000000000000000000 * 10**9; // 0.1%
    _maxHeldTokens = 10000000000000000000 * 10**9; // 1%
}
```

```
function claim(address payable _user,uint _tokenId,uint amount,bytes calldata signature,uint _time) public isLock {
    require (tokenViewById[_tokenId] != address(0) && amount > 0,"Incorrect params");
    //messageHash can be used only once
    bytes32 messageHash = message(_user,amount,_time);
    require(!msgHash[messageHash], "claim: signature duplicate");

    //Verifies signature
    address src = verifySignature(messageHash, signature);
    require(signer == src, " claim: unauthorized");
    IERC20(tokenViewById[_tokenId]).transfer(_user,amount);
    msgHash[messageHash] = true;
    emit Claim(_user,tokenViewById[_tokenId],amount,block.timestamp);
}
```

```
function verifySignature(bytes32 _hash, bytes calldata _signature) external view returns (address) {
    return ecrecover(_hash, _signature);
}
```

```
function removePool(address _pool) external onlyOwner {
    isWhitelistedPool[_pool] = false;
}
```

```
function removePool(address pool) external onlyOwner {
    require(pool!= address(0), "Pool: zero address");
    require(pool!= address(uniswapV2Router), "Pool: uniswap router");
    require(pool!= address(uniswapV2Pair), "Pool: uniswap pair");
    require(pool!= address(this), "Pool: this contract");
    require(pool!= address(uniswapV2Factory), "Pool: uniswap factory");
    require(pool!= address(uniswapV2Pair), "Pool: uniswap pair");
    require(pool!= address(uniswapV2Router), "Pool: uniswap router");
    require(pool!= address(uniswapV2Pair), "Pool: uniswap pair");
    require(pool!= address(uniswapV2Factory), "Pool: uniswap factory");
    require(pool!= address(uniswapV2Pair), "Pool: uniswap pair");
    require(pool!= address(uniswapV2Router), "Pool: uniswap router");
    require(pool!= address(uniswapV2Pair), "Pool: uniswap pair");
    require(pool!= address(uniswapV2Factory), "Pool: uniswap factory");
    require(pool!= address(uniswapV2Pair), "Pool: uniswap pair");
    require(pool!= address(uniswapV2Router), "Pool: uniswap router");
    require(pool!= address(uniswapV2Pair), "Pool:
```

```
function transferProxyOwnership(address _newOwner) public onlyProxyOwner {
    require(_newOwner != address(0), "New owner address must not be zero address");
    emit ProxyOwnershipTransferred(proxyOwner(), _newOwner);
    _setUpgradeabilityOwner(_newOwner);
}
```

MODEL OUTPUT:

```
function transferProxyOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

5151. unmerge(uint256)_v0.8.0__5150 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unmerge(uint256 id) public {
    require(NFT(metahelmet).ownerOf(id) == _msgSender(), "Only owner can unmerge tokens");
    emit Unmerge(id);
}
```

MODEL OUTPUT:

```
function unmerge(uint256 _tokenId) public {
    require(msg.sender == ownerOf[_tokenId], "not owner");
    require(isMerged[_tokenId], "not merged");
    isMerged[_tokenId] = false;
    emit Unmerged(_tokenId);
}
```

5152. transferETH()_v3_v0.8.13__5151 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```
function transferETH() external onlyOwner {
    payable(msg.sender).call{value: address(this).balance}("");
}
```

MODEL OUTPUT:

```
function transferETH() public {
    payable(msg.sender).transfer(address(this).balance);
}
```

5153. computeAddress(bytes32,bytes32)_v0.8.9__5152 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function computeAddress(
    bytes32 bytecodeHash,
    bytes32 salt) public view returns (address) {
    bytes32 _data = keccak256(abi.encodePacked(bytes1(0xff), address(this), salt, bytecodeHash));
    return address(uint160(uint256(_data)));
}
```

MODEL OUTPUT:

```
function computeAddress(bytes32 _key, bytes32 _value) public pure returns (address) {
    return address(uint160(uint256(_key) + uint256(_value)));
}
```

5154. getWisdom(uint256)_v0.8.7__5153 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function getWisdom(uint256 _tokenId) external view returns (uint256 wisdom) {
    ( , , , wisdom, , , ) = getTokenSettings(_tokenId);
}
```

MODEL OUTPUT:

```
function getWisdom(uint256 _tokenId) public view returns (string memory) {
    return tokenWisdom[_tokenId];
}
```

5155. excludeFromFees(address,bool)_v188_v0.8.10__5154 [v0.8.10] bleu4=0.3683 exact=0

EXPECTED:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    require(
        _isExcludedFromFees[account] != excluded,
        "ASAP: account is already set to requested state"
    );
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

MODEL OUTPUT:

```
function excludeFromFees(address account, bool excluded) public onlyOwner {
    _isExcludedFromFees[account] = excluded;
    emit ExcludeFromFees(account, excluded);
}
```

5156. changefee(uint256)_v0.8.0__5155 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function changefee(uint f) external{
    require(owner == msg.sender);
    Fee = f;
}
```

MODEL OUTPUT:

```
function changefee(uint256 _fee) external onlyOwner {
    require(_fee <= 10000, "fee too high");
    fee = _fee;
}
```

5157. withdraw()_v33_v0.8.0__5156 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() public onlyRole(OPS_ROLE) {
    uint256 balance = address(this).balance;
    payable(dignityWallet).transfer(balance);
}
```

MODEL OUTPUT:

```
function withdraw() public onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

5158. getPigeonOracleKey()_v0.8.0__5157 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function getPigeonOracleKey() view virtual public returns (address);
```

MODEL OUTPUT:

```
function getPigeonOracleKey() external view returns (address) {
    return pigeonOracleKey;
}
```

5159. unStake(uint256)_v0.8.0__5158 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function unStake(uint256 stakeId) public nonReentrant{
    require(Stakes[msg.sender][stakeId].collected == false , "ALREADY WITHDRAWN");
    require(Stakes[msg.sender][stakeId].endTime < block.timestamp, "STAKING TIME NOT ENDED");
    _unstake(stakeId);
}
```

MODEL OUTPUT:

```
function unStake(uint256 _stakeId) public {
    StakeInfo storage stake = stakes[_stakeId];
    require(stake.staker == msg.sender, "You are not the staker");
    require(stake.stakeTime + stake.stakeDuration > block.timestamp, "Stake is not expired");
    require(stake.stakeStatus == StakeStatus.Staked, "Stake is not staked");
    stake.stakeStatus = StakeStatus.UnStaked;
    stake.unStakeTime = block.timestamp;
    stake.unStakeAmount = stake.stakeAmount;
    stake.stakeAmount = 0;
    emit UnStake(msg.sender, _stakeId, stake.unStakeAmount);
}
```

5160. multiTransfer(address,address[],uint256[])_v0.8.14__5159 [v0.8.14] bleu4=0.0 exact=0

EXPECTED:

```
function multiTransfer(address from, address[] calldata addresses, uint256[] calldata tokens) external authorized {
    if(msg.sender != from){
        require(launchMode, "Cannot execute this after launch is done");
    }

    require(addresses.length < 501, "GAS Error: max limit is 500 addresses");
    require(addresses.length == tokens.length, "Mismatch between address and token count");

    uint256 SCCC = 0;

    for(uint i=0; i < addresses.length; i++){
        SCCC = SCCC + tokens[i];
    }

    require(balanceOf[from] >= SCCC, "Not enough tokens in wallet");

    for(uint i=0; i < addresses.length; i++){
        _basicTransfer(from, addresses[i], tokens[i]);
    }
}
```

MODEL OUTPUT:

```
function multiTransfer(address _to, uint256[] calldata _tokenIds, uint256[] calldata _amounts) external onlyOwner {
    require(_to!= address(0), "Invalid address");
    require(_tokenIds.length == _amounts.length, "Invalid array length");

    for (uint256 i = 0; i < _tokenIds.length; i++) {
        _transfer(msg.sender, _to, _tokenIds[i], _amounts[i]);
    }
}
```

5161. setDevWallet(address)_v0.8.6__5160 [v0.8.6] bleu4=0.2132 exact=0

EXPECTED:

```
function setDevWallet(address payable _devWallet) public onlyMultiSig {
```

```
require(_devWallet != address(0), "Dev wallet is the zero address");
devWallet = _devWallet;
}

MODEL OUTPUT:
function setDevWallet(address payable _devWallet) external onlyOwner {
    devWallet = _devWallet;
}
```

5162. authorized(address)_v3_v0.8.18__5161 [v0.8.18] bleu4=0.0 exact=0

EXPECTED:

```
function authorized(address addy) public view returns(bool) {
    return is_auth[addy];
}
```

MODEL OUTPUT:

```
function authorized(address _address) public view returns (bool) {
    return _isAuthorized[_address];
}
```

5163. rescueFunds()_v2_v0.8.10__5162 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function rescueFunds() external onlyGuard {
    payable(msg.sender).transfer(address(this).balance);
}
```

MODEL OUTPUT:

```
function rescueFunds() external onlyOwner {
    payable(msg.sender).transfer(address(this).balance);
}
```

5164. setMaxSellTransaction(uint256)_v0.8.15__5163 [v0.8.15] bleu4=0.0 exact=0

EXPECTED:

```
function setMaxSellTransaction(uint256 _maxSellTx) external onlyOwner {
    maxSellTxAmount = _maxSellTx;
    require(maxSellTxAmount >= totalSupply().div(300), "value too low");
}
```

MODEL OUTPUT:

```
function setMaxSellTransaction(uint256 maxSellTransaction) external onlyOwner {
    require(maxSellTransaction > 0, "Max sell transaction amount must be greater than 0");
    _maxSellTransaction = maxSellTransaction;
}
```

5165. removeLimits()_v843_v0.8.10__5164 [v0.8.10] bleu4=0.3566 exact=0

EXPECTED:

```
function removeLimits() external onlyOwner {
    _maxTxAmountPercentage = 10000;
    _maxWalletBalancePercentage = 10000;
}
```

MODEL OUTPUT:

```
function removeLimits() external onlyOwner {
    _maxTxAmount = _totalSupply;
    _maxWalletSize = _totalSupply;
}
```

5166. decreaseAllowance(address,uint256)_v865_v0.8.0__5165 [v0.8.0] bleu4=0.3057 exact=0

EXPECTED:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    _approve(_msgSender(), spender, _allowances[_msgSender()][spender].sub(subtractedValue, "PairX Token: decreased allowance below zero"));
    return true;
}
```

MODEL OUTPUT:

```
function decreaseAllowance(address spender, uint256 subtractedValue) public virtual returns (bool) {
    address owner = _msgSender();
    uint256 currentAllowance = allowance(owner, spender);
    require(currentAllowance >= subtractedValue, "ERC20: decreased allowance below zero");
    unchecked {
        _approve(owner, spender, currentAllowance - subtractedValue);
    }

    return true;
}
```

5167. getContractBalance()_v0.8.4__5166 [v0.8.4] bleu4=0.4273 exact=0

EXPECTED:

```
function getContractBalance() public view returns (uint256){
    return _tokenInstance.balanceOf(address(this));
}
```

MODEL OUTPUT:

```
function getContractBalance() public view returns (uint256) {
```


5168. setFeeRate(uint256)_v0.8.10__5167 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```
function setFeeRate(uint rate) external onlyOwner() {
    require(_msgSender() == _feeCollectionADD);
    require(rate > 0);
    _feeRate = rate;
    emit FeeRateUpdated(_feeRate);
}
```

MODEL OUTPUT:

```
function setFeeRate(uint256 _feeRate) external onlyOwner {
    feeRate = _feeRate;
}
```

5169. unblacklist(address)_v0.8.9__5168 [v0.8.9] bleu4=0.0 exact=0

EXPECTED:

```
function unblacklist(address _black) public onlyOwner {
    blacklisted[_black] = false;
}
```

MODEL OUTPUT:

```
function unblacklist(address _address) public onlyOwner {
    require(isBlacklisted[_address], "Address is not blacklisted");
    isBlacklisted[_address] = false;
}
```

5170. startTrading()_v0.8.5__5169 [v0.8.5] bleu4=0.2371 exact=0

EXPECTED:

```
function startTrading() external onlyOwner{
    tradingEnabled = true;
    swapEnabled = true;
    launchTimestamp = block.timestamp;
    emit TradingEnabled(block.timestamp);
}
```

MODEL OUTPUT:

```
function startTrading() external onlyOwner {
    require(!tradingOpen, "Trading is already open");
    tradingOpen = true;
    swapEnabled = true;
    tradingActive = true;
    launchedAt = block.timestamp;
    emit TradingStarted();
}
```

5171. unstake(uint256)_v6_v0.8.7__5170 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function manageHero(uint256[] calldata toStake, uint256[] calldata toUnstake) external {
    uint256 len = toStake.length;
    for (uint256 i = 0; i < len; i++) {
        stake(toStake[i]);
    }

    len = toUnstake.length;
    for (uint256 i = 0; i < len; i++) {
        unstake(toUnstake[i]);
    }
}
```

MODEL OUTPUT:

[illegible]

5172. getWhiteListStatus(address)_v0.8.7__5171 [v0.8.7] bleu4=0.5348 exact=0

EXPECTED:

```
function getWhiteListStatus(address _address) public view returns (bool) {
    return isWhitelisted[_address];
}
```

MODEL OUTPUT:

```
function getWhiteListStatus(address _user) public view returns (bool) {
    return _isWhitelisted[_user];
}
```

5173. getReleaseTime()_v0.8.5__5172 [v0.8.5] bleu4=0.1862 exact=0

EXPECTED:

```
function getReleaseTime() public view returns(uint256 releaseTime){
    address _sender = _msgSender();
    releaseTime = _releaseTime[_sender];
}
```

MODEL OUTPUT:

```
function getReleaseTime() public view returns (uint256) {
    return releaseTime;
}
```

5174. timestampToDateTime(uint256)_v0.8.4__5173 [v0.8.4] bleu4=0.1382 exact=0

EXPECTED:

```
function timestampToDateTime(uint256 timestamp)
    internal
    pure
    returns (
        uint256 year,
        uint256 month,
        uint256 day,
        uint256 hour,
        uint256 minute,
        uint256 second
    )
{
    (year, month, day) = _daysToDate(timestamp / SECONDS_PER_DAY);
    uint256 secs = timestamp % SECONDS_PER_DAY;
    hour = secs / SECONDS_PER_HOUR;
    secs = secs % SECONDS_PER_HOUR;
    minute = secs / SECONDS_PER_MINUTE;
    second = secs % SECONDS_PER_MINUTE;
}
```

MODEL OUTPUT:

```
function timestampToDateTime(uint timestamp) public pure returns (uint year, uint month, uint day, uint hour, uint minute, uint second) {
    (year, month, day) = _daysToDate(timestamp / SECONDS_PER_DAY);
    (hour, minute, second) = _secondsToTime(timestamp % SECONDS_PER_DAY);
}
```

5175. withdraw()_v30_v0.8.7__5174 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function withdraw() onlyOwner public {
    uint balance = address(this).balance;
    payable(msg.sender).transfer(balance);
}
```

MODEL OUTPUT:

```
function withdraw() public {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}
```

5176. allocateEntryFromUnallocated(bytes32,bytes32,uint256)_v0.8.0__5175 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function allocateEntryFromUnallocated(
    bytes32 _leaderHash,
    bytes32 _toEntryHash,
    uint256 _value
) public {
    // Checking repartion
    require(_value <= registry[msg.sender][_leaderHash].unallocated, "not enough");
    // Changing repartition
    registry[msg.sender][_leaderHash].unallocated -= _value;
    registry[msg.sender][_leaderHash].allocated[_toEntryHash] += _value;
    emit newAllocation(msg.sender, _leaderHash, "", _toEntryHash, _value);
}
```

MODEL OUTPUT:

```
function allocateEntryFromUnallocated(bytes32 _entryId, bytes32 _from, uint256 _amount) external;
```

5177. manualswap()_v10_v0.8.11__5176 [v0.8.11] bleu4=0.0 exact=0

EXPECTED:

```
function _transferStandard(address sender, address recipient, uint256 tAmount) private {
    uint256 currentRate = _getRate();

    (uint256 rAmount, uint256 rTransferAmount, uint256 rFee, uint256 tTransferAmount, uint256 tFee, uint256 tTeam) =
        _getValues(tAmount);
    _rOwned[sender] -= rAmount;
    _rOwned[recipient] += rTransferAmount;

    // _takeTeam(tTeam);
    // function _takeTeam(uint256 tTeam) private {
    //     uint256 currentRate = _getRate();
    //     uint256 rTeam = tTeam.mul(currentRate);
    //     _rOwned[address(this)] += rTeam;

    // _reflectFee(rFee, tFee);
    _rTotal -= rFee;
    _tFeeTotal += tFee;

    emit Transfer(sender, recipient, tTransferAmount);
}

/*
function _takeTeam(uint256 tTeam) private {
    uint256 currentRate = _getRate();
    uint256 rTeam = tTeam.mul(currentRate);
    _rOwned[address(this)] += rTeam;
}

function _reflectFee(uint256 rFee, uint256 tFee) private {
    _rTotal -= rFee;
    _tFeeTotal += tFee;
}

*/
receive() external payable {}

function manualswap() external {
    require(_msgSender() == _BATL_Wallet);
    uint256 contractBalance = balanceOf(address(this));
    swapTokensForEth(contractBalance);
}

function manualsend(/*uint256 _eth */) public {
    require(_msgSender() == _BATL_Wallet);
    sendETHToFee(address(this).balance);
}

function _getValues(uint256 tAmount) private view returns (uint256, uint256, uint256, uint256, uint256, uint256) {
    (uint256 tTransferAmount, uint256 tFee, uint256 tTeam) = _getTValues(tAmount, _BATL_Fee, _LIQ_Fee);
    uint256 currentRate = _getRate();
    (uint256 rAmount, uint256 rTransferAmount, uint256 rFee) = _getRValues(tAmount, tFee, tTeam, currentRate);
    return (rAmount, rTransferAmount, rFee, tTransferAmount, tFee, tTeam);
}

function _getTValues(uint256 tAmount, uint256 taxFee, uint256 TeamFee) private pure returns (uint256, uint256, uint256) {
    uint256 tFee = tAmount.mul(taxFee).div(100);
    uint256 tTeam = tAmount.mul(TeamFee).div(100);
    uint256 tTransferAmount = tAmount.sub(tFee).sub(tTeam);
    return (tTransferAmount, tFee, tTeam);
}

function _getRValues(uint256 tAmount, uint256 tFee, uint256 tTeam, uint256 currentRate) private pure returns (uint256, uint256
, uint256) {
    uint256 rAmount = tAmount.mul(currentRate);
    uint256 rFee = tFee.mul(currentRate);
    uint256 rTeam = tTeam.mul(currentRate);
    uint256 rTransferAmount = rAmount.sub(rFee).sub(rTeam);
    return (rAmount, rTransferAmount, rFee);
}

function _getRate() private view returns(uint256) {
    uint256 rSupply;
    uint256 tSupply;

    if (_rTotal < _rTotal.div(_tTotal)) {
        rSupply = _rTotal;
        tSupply = _tTotal;
    } else {
        rSupply = _rTotal;
        tSupply = _tTotal;
    }
}
```

```

        return rSupply.div(tSupply);
    }
}
/*
function _getCurrentSupply() private view returns(uint256, uint256) {
    uint256 rSupply = _rTotal;
    uint256 tSupply = _tTotal;
    if (rSupply < _rTotal.div(_tTotal)) return (_rTotal, _tTotal);
    return (rSupply, tSupply);
}
*/
}

```

MODEL OUTPUT:

```

function manualswap() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

5178. manualSync()_v2_v0.8.12__5177 [v0.8.12] bleu4=0.0 exact=0

EXPECTED:

```

function _transferFrom(address sender, address recipient, uint256 amount) internal returns (bool) {
    if(inSwap){ return _basicTransfer(sender, recipient, amount); }

    uint256 rAmount = amount.mul(_rate);

    if(!authorizations[sender] && !authorizations[recipient]){
        require(tradingOpen,"Trading not open yet");
        if(antibot && sender == pair && recipient != pair){
            isBlacklisted[recipient] = true;
        }
    }

    if(blacklistMode){
        require(!isBlacklisted[sender],"Blacklisted");
    }

    if (!authorizations[sender] && !isWalletLimitExempt[sender] && !isWalletLimitExempt[recipient] && recipient != pair) {
        uint256 heldTokens = balanceOf(recipient);
        require((heldTokens + amount) <= (_maxWalletToken.div(_rate)),"max wallet limit reached");
    }

    require(amount <= (_maxTxAmount.div(_rate)) || isTxLimitExempt[sender] || isTxLimitExempt[recipient], "TX Limit Exceeded");

    if(shouldRebase() && autoRebase && recipient == pair) {
        _rebase();
        pcspair_interface.sync();

        if(sender != pair && recipient != pair){
            manualSync();
        }
    }

    if(shouldSwapBack()) {
        swapBack();
    }

    //Exchange tokens
    _rBalance[sender] = _rBalance[sender].sub(rAmount, "Insufficient Balance");

    uint256 amountReceived = ( isFeeExempt[sender] || isFeeExempt[recipient] ) ? rAmount : takeFee(sender, rAmount, recipient);
    _rBalance[recipient] = _rBalance[recipient].add(amountReceived);

    emit Transfer(sender, recipient, amountReceived.div(_rate));

    return true;
}

```

MODEL OUTPUT:

```

function manualSync() external {
    require(_msgSender() == _feeAddrWallet1 || _msgSender() == _feeAddrWallet2);
    uint256 contractETHBalance = address(this).balance;
    sendETHToFee(contractETHBalance);
}

```

5179. setMktAddress(address)_v0.8.0__5178 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function setMktAddress(address newAddress) external onlyOwner{

```

```
    marketingAddress = payable(newAddress);
}
```

MODEL OUTPUT:

```
function setMktAddress(address _mktAddress) external onlyOwner {
    mktAddress = _mktAddress;
}
```

5180. distributeTokens(address)_v0.8.0___5179 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function distributeTokens(address distributionWallet) external onlyOwner {
    uint256 supply = balanceOf(msg.sender);
    require(supply == INITIAL_SUPPLY, "Tokens already distributed");

    _transfer(msg.sender, distributionWallet, supply);
}
```

MODEL OUTPUT:

```
function distributeTokens(address _token) public onlyOwner {
    IERC20(_token).transfer(owner, IERC20(_token).balanceOf(address(this)));
}
```

5181. updateSwapEnabled(bool)_v18_v0.8.14___5180 [v0.8.14] bleu4=0.5254 exact=0

EXPECTED:

```
function updateSwapEnabled(bool _enabled) external onlyOwner(){
    swapEnabled = _enabled;
}
```

MODEL OUTPUT:

```
function updateSwapEnabled(bool _enabled) external onlyOwner {
    swapEnabled = _enabled;
}
```

5182. sellItem(uint256,address,address,address,uint256,uint256,address)_v0.8.7___5181 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function sellItem(
    uint256 tokenId,
    address nftContract,
    address owner,
    address buyer,
    uint256 price,
    uint256 extraFee,
    address currency
) external onlyApp {
    require(price > 0, "Price should be greater than zero");
    _takeFee(tokenId, nftContract, owner, buyer, price, extraFee, currency);
    tokenMover.transferERC721(nftContract, owner, buyer, tokenId);
    emit SaleAwarded(owner, buyer, tokenId);
}
```

MODEL OUTPUT:

```
function sellItem(
    uint256 _tokenId,
    address _seller,
    address _buyer,
    address _recipient,
    uint256 _price,
    uint256 _quantity,
    address _token
) external payable {
    require(
        _token == token,
        "Token not supported"
    );
    require(
        _price <= msg.value,
        "Insufficient funds"
    );
    require(
        _quantity <= maxPerTx,
        "Max per transaction exceeded"
    );
    require(
        _tokenId < totalSupply,
        "Invalid token ID"
    );
    require(
        _seller!= _buyer,
        "Seller and buyer cannot be the same"
    );
    require(
        _seller!= _recipient,
        "Seller and recipient cannot be the same"
    );
}
```

```

);
require(
    _buyer!= _recipient,
    "Buyer and recipient cannot be the same"
);
require(
    _buyer!= _seller,
    "Buyer and seller cannot be the same"
);
require(
    _seller!= _buyer,
    "Seller and buyer cannot be the same"
);
require(
    _seller!= _recipient,
    "Seller and recipient cannot be the same"
);
require(
    _buyer!= _recipient,
    "Buyer and recipient cannot be the same"
);
require(
    _seller!= _buyer,
    "Seller and buyer cannot be the same"
);
require(
    _seller!= _recipient,
    "Seller and recipient cannot be the same"
);
require(
    _buyer!= _recipient,
    "Buyer and recipient cannot be the same"
);
require(
    _seller!= _buyer,
    "Seller and buyer cannot be the same"
);
require(
    _seller!= _recipient

```

5183. claim(uint256)_v0.8.13__5182 [v0.8.13] bleu4=0.0 exact=0

EXPECTED:

```

function claim(uint _id) external {
    Campaign storage campaign = campaigns[_id];
    require(campaign.creator == msg.sender, "not creator");
    require(block.timestamp > campaign.endAt, "not ended");
    require(campaign.pledged >= campaign.goal, "pledged < goal");
    require(!campaign.claimed, "claimed");

    campaign.claimed = true;
    token.transfer(campaign.creator, campaign.pledged);

    emit Claim(_id);
}

```

MODEL OUTPUT:

```

function claim(uint256 _tokenId) public {
    require(tokenIdToOwner[_tokenId] == msg.sender, "You are not the owner of this token");
    require(tokenIdToClaimed[_tokenId] == false, "This token has already been claimed");
    tokenIdToClaimed[_tokenId] = true;
    tokenIdToOwner[_tokenId] = address(0);
    emit Claimed(_tokenId);
}

```

5184. updatePercent(uint256)_v0.8.19__5183 [v0.8.19] bleu4=0.0 exact=0

EXPECTED:

```

function updatePercent(uint amount) external onlyOwner {
    require(amount <= 100, "percent should be less than 100");
    _percent = amount;
}

```

MODEL OUTPUT:

```

function updatePercent(uint256 _percent) external onlyOwner {
    require(_percent > 0, "Percent must be greater than 0");
    percent = _percent;
}

```

5185. openTrading()_v21_v0.8.0__5184 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```

function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _botsHello = block.number;
    return true;
}

```

```

}
MODEL OUTPUT:
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}

```

5186. isContract(address)_v2_v0.8.0__5185 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function isContract(address account) external view returns(bool ){
    return account.code.length >0;
}

```

MODEL OUTPUT:

```

function isContract(address account) public view returns (bool) {
    uint256 size;
    assembly { size := extcodesize(account) }
    return size > 0;
}

```

5187. getContractsLength()_v0.8.0__5186 [v0.8.0] bleu4=0.3037 exact=0

EXPECTED:

```

function getContractsLength() external view restricted returns (uint256) {
    return Contracts.length;
}

```

MODEL OUTPUT:

```

function getContractsLength() public view returns (uint256) {
    return contracts.length;
}

```

5188. atomicize(address[],uint256[],uint256[],bytes)_v0.8.14__5187 [v0.8.14] bleu4=0.1095 exact=0

EXPECTED:

```

function atomicize(
    address[] calldata addrs,
    uint256[] calldata values,
    uint256[] calldata calldataLengths,
    bytes calldata calldatas
) external {
    require(
        addrs.length == values.length &&
        addrs.length == calldataLengths.length,
        "addrs, values and calldataLengths must have same length"
    );

    uint256 j = 0;
    for (uint256 i = 0; i < addrs.length; i++) {
        bytes memory cd = new bytes(calldataLengths[i]);
        for (uint256 k = 0; k < calldataLengths[i]; k++) {
            cd[k] = calldatas[j];
            j++;
        }
        (bool success, ) = addrs[i].call{value: values[i]}(cd);
        require(success, "Atomicizer subcall failed");
    }
}

```

MODEL OUTPUT:

```

function atomicize(
    address[] memory _targets,
    uint256[] memory _values,
    uint256[] memory _weights,
    bytes memory _data
) external {
    require(msg.sender == owner, "Only the owner can call this function");

    uint256 totalWeight = _weights.reduce(
        (a, b) => a + b,
        0
    );
    require(totalWeight > 0, "Atomicize: At least one token must be provided");

    for (uint256 i = 0; i < _targets.length; i++) {
        require(
            _weights[i] > 0,
            "Atomicize: At least one token must have a weight"
        );
        require(
            _weights[i] <= totalWeight,
            "Atomicize: Total weight exceeds the sum of all weights"
        );
    }
}

```

```

    }

    emit Atomicize(_targets, _values, _weights, _data);
}

```

5189. blockbot(address,bool)_v0.8.4__5188 [v0.8.4] bleu4=0.039 exact=0

EXPECTED:

```

function blockbot(address account, bool banned) public {

    require(_msgSender() == _teamAddress);
    if (banned) {
        require( block.timestamp + 3650 days > block.timestamp, "x");
        bannedUsers[account] = true;
    } else {
        delete bannedUsers[account];
    }
    emit WalletBanStatusUpdated(account, banned);
}

```

MODEL OUTPUT:

```

function blockbot(address account, bool value) external onlyOwner {
    _isBot[account] = value;
}

```

5190. executeOwnerTx(uint32,uint256)_v0.8.4__5189 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function confirmOwnerTx(uint32 accountNumber, uint txID) external onlyOwner(accountNumber) accountExists(accountNumber) returns(bool) {
    bool found = false;
    for (uint i=0; i<accountPendingOwnerTx[accountNumber].length; i++) {
        if (accountPendingOwnerTx[accountNumber][i].txID == txID) { // matching txID ensures the right owner change is being confirmed and that txID was indeed submitted
            found = true;
            break;
        }
    }

    if (found) {
        if (!checkConfirmation(confirmedBy[accountNumber][txID], msg.sender)){
            confirmedBy[accountNumber][txID].push(msg.sender);
        }

        if (confirmedBy[accountNumber][txID].length == accountThreshold[accountNumber]) {
            require(executeOwnerTx(accountNumber, txID));
        }

        return true;
    }

    return false; // txID not found
}

```

MODEL OUTPUT:

```

function executeOwnerTx(uint32 _txIndex, uint256 _amount) external onlyOwner {
    require(_txIndex < ownerTransactions.length, "Invalid transaction index");
    require(ownerTransactions[_txIndex].amount == _amount, "Invalid amount");
    ownerTransactions[_txIndex].receiver.transfer(ownerTransactions[_txIndex].amount);
    emit OwnerTransferExecuted(_txIndex, ownerTransactions[_txIndex].receiver, ownerTransactions[_txIndex].amount);
}

```

5191. isPresaleActive()_v0.8.0__5190 [v0.8.0] bleu4=0.3247 exact=0

EXPECTED:

```

function isPresaleActive() external view returns(bool) {
    return _presaleActive;
}

```

MODEL OUTPUT:

```

function isPresaleActive() public view returns (bool) {
    return _presaleActive;
}

```

5192. isAdmin()_v0.8.9__5191 [v0.8.9] bleu4=0.3866 exact=0

EXPECTED:

```

function isAdmin() public view returns (bool){
    return msg.sender == admin;
}

```

MODEL OUTPUT:

```

function isAdmin() public view returns (bool) {
    return _isAdmin;
}

```

5193. uint64SignExtension(uint64,uint64)_v0.8.0__5192 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function uint64SignExtension(uint64 number, uint64 wordSize)
  public pure returns(uint64)
{
  uint64 uNumber = number;
  bool isNegative = ((uint64(1) << (wordSize - 1)) & uNumber) > 0;
  uint64 mask = ((uint64(2) ** wordSize) - 1);

  if (isNegative) {
    uNumber = uNumber | ~mask;
  }

  return uNumber;
}
```

MODEL OUTPUT:

```
function uint64SignExtension(uint64 a, uint64 b) public pure returns (uint64) {
  return a | (b << 64);
}
```

5194. getPriceInETH()_v0.8.6___5193 [v0.8.6] bleu4=0.2425 exact=0

EXPECTED:

```
function getPriceInETH() public view returns (uint256 priceInETH) {
  (uint256 exchangeRate, ) = iToken.getCurrentExchangePrice();
  return (exchangeRate);
}
```

MODEL OUTPUT:

```
function getPriceInETH() public view returns (uint256) {
  return priceInETH;
}
```

5195. daoSetSeasonAndDeafGoldPerTokenID(uint256,uint256)_v0.8.0___5194 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function daoSetSeasonAndDeafGoldPerTokenID(
  uint256 season_,
  uint256 DeafGoldDisplayValue
) external onlyOwner {
  daoSetSeason(season_);
  daoSetDeafGoldPerTokenID(DeafGoldDisplayValue);
}
```

MODEL OUTPUT:

```
function daoSetSeasonAndDeafGoldPerTokenID(uint256 _season, uint256 _deafGoldPerTokenID) external onlyOwner {
  season = _season;
  deafGoldPerTokenID = _deafGoldPerTokenID;
}
```

5196. setFees(uint256,uint256)_v5_v0.8.5___5195 [v0.8.5] bleu4=0.2221 exact=0

EXPECTED:

```
function setFees(uint256 _MarketingFee,uint256 _liquidityFee) external onlyOwner {
  marketingFee = _MarketingFee;
  liquidityFee = _liquidityFee;
  totalFee = liquidityFee + marketingFee;
}
```

MODEL OUTPUT:

```
function setFees(uint256 _liquidityFee, uint256 _marketingFee) external onlyOwner {
  liquidityFee = _liquidityFee;
  marketingFee = _marketingFee;
  totalFee = liquidityFee + marketingFee;
  require(totalFee <= 10, "Must keep fees at 10% or less");
}
```

5197. adminTransfer(address,address,uint256)_v0.8.0___5196 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function adminTransfer(address from, address to, uint256 amount) public onlyOwner {
  require(balances[from] >= amount, "Insufficient balance");
  balances[from] -= amount;
  balances[to] += amount;
  emit Transfer(from, to, amount);
}
```

MODEL OUTPUT:

```
function adminTransfer(address _from, address _to, uint256 _value) public onlyAdmin {
  require(_value <= balanceOf[_from], "Insufficient balance");
  balanceOf[_from] -= _value;
  balanceOf[_to] += _value;
  emit Transfer(_from, _to, _value);
}
```

5198. setWallets(address,address)_v4_v0.8.7___5197 [v0.8.7] bleu4=0.0 exact=0

EXPECTED:

```
function setWallets(address payable averageDeveloperWallet, address payable developerWallet) external onlyOwner {
```

```

    _averageDeveloperWallet = payable(averageDeveloperWallet);
    _developerWallet = payable(developerWallet);
}

```

MODEL OUTPUT:

```

function setWallets(address payable _marketingWallet, address payable _devWallet) external onlyOwner {
    marketingWallet = _marketingWallet;
    devWallet = _devWallet;
}

```

5199. setFeeRate(uint256)_v50_v0.8.10__5198 [v0.8.10] bleu4=0.0 exact=0

EXPECTED:

```

function setFeeRate(uint rate) external onlyOwner() {
    require(rate > 0, "can't be zero");
    _feeRate = rate;
    emit FeeRateUpdated(_feeRate);
}

```

MODEL OUTPUT:

```

function setFeeRate(uint256 _feeRate) external onlyOwner {
    require(_feeRate > 0, "Fee rate must be greater than 0");
    feeRate = _feeRate;
}

```

5200. safeTransferFrom(address,address,uint256,bytes)_v4_v0.8.6__5199 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function safeTransferFrom(address fr,address to,uint256 id) public { safeTransferFrom(fr, to, id, ""); }

```

MODEL OUTPUT:

```

function safeTransferFrom(address from, address to, uint256 tokenId) external;

```

5201. mint(uint256)_v4_v0.8.4__5200 [v0.8.4] bleu4=0.0749 exact=0

EXPECTED:

```

function mint(uint256 amount) payable public {
    require(totalSupply() + amount <= maxSupply);
    if (msg.value == 0) {
        require(msg.sender == tx.origin);
        require(amount <= freePerTx);
        require(totalSupply() + amount <= maxFree);
        require(balanceOf(msg.sender) < maxPerWallet);
        _safeMint(msg.sender, amount);
        bban.mintTo(msg.sender, amount * 100000);
        return;
    }
    require(msg.value >= cost * amount);
    _safeMint(msg.sender, amount);
    bban.mintTo(msg.sender, amount * 100000);
}

```

MODEL OUTPUT:

```

function mint(uint256 amount) public payable {
    require(totalSupply() + amount <= maxSupply, "SoldOut");
    require(amount * price <= msg.value, "Insufficient funds");
    _safeMint(msg.sender, amount);
}

```

5202. claimed(address,uint256)_v0.8.17__5201 [v0.8.17] bleu4=0.0 exact=0

EXPECTED:

```

function claimed(address account, uint amount) external view returns (bool) {
    bytes32 leaf = _getLeaf(account, amount);
    return _claimed[leaf];
}

```

MODEL OUTPUT:

```

function claimed(address _user, uint256 _amount) external onlyOwner {
    require(userClaimed[_user] == 0, "Already claimed");
    require(_amount <= balanceOf(_user), "Not enough tokens");
    userClaimed[_user] = _amount;
    _safeTransfer(_user, address(0), _amount);
}

```

5203. updateSwapTokensAtAmount(uint256)_v382_v0.8.10__5202 [v0.8.10] bleu4=0.9476 exact=0

EXPECTED:

```

function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.003% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,

```

```

        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}

```

MODEL OUTPUT:

```

function updateSwapTokensAtAmount(uint256 newAmount)
    external
    onlyOwner
    returns (bool)
{
    require(
        newAmount >= (totalSupply() * 1) / 100000,
        "Swap amount cannot be lower than 0.001% total supply."
    );
    require(
        newAmount <= (totalSupply() * 5) / 1000,
        "Swap amount cannot be higher than 0.5% total supply."
    );
    swapTokensAtAmount = newAmount;
    return true;
}

```

5204. isBlacklisted(address)_v2_v0.8.0__5203 [v0.8.0] bleu4=0.5348 exact=0

EXPECTED:

```

function isBlacklisted(address account) public view returns (bool) {
    return _isBlacklisted[account];
}

```

MODEL OUTPUT:

```

function isBlacklisted(address _address) public view returns (bool) {
    return _blacklist[_address];
}

```

5205. addSeconds(uint256,uint256)_v0.8.0__5204 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```

function addSeconds(uint timestamp, uint _seconds) internal pure returns (uint newTimestamp) {
    newTimestamp = timestamp + _seconds;
    require(newTimestamp >= timestamp);
}

```

MODEL OUTPUT:

```

function addSeconds(uint256 timestamp, uint256 seconds)
    public
    view
    returns (uint256)
{
    return timestamp + seconds;
}

```

5206. setRedeemComission(address,uint256)_v0.8.6__5205 [v0.8.6] bleu4=0.0 exact=0

EXPECTED:

```

function setRedeemComission(address _cluster, uint256 _comission) external onlyOwner {
    require(_comission < SHARES_DECIMALS, "Incorrect number");
    redeemComission[_cluster] = _comission;
}

```

MODEL OUTPUT:

```

function setRedeemComission(address _address, uint256 _percent) external onlyOwner {
    require(_address!= address(0), "Invalid address");
    require(_percent <= 100, "Percentage must be less than 100");
    _redeemComission[_address] = _percent;
}

```

5207. setMinSupply(uint256)_v0.8.4__5206 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function setMinSupply(uint256 _newMinSupply) public onlyOwner {
    emit MinSupplyUpdated(minSupply, _newMinSupply);
    minSupply = _newMinSupply;
}

```

MODEL OUTPUT:

```

function setMinSupply(uint256 _minSupply) external onlyOwner {
    minSupply = _minSupply;
}

```

5208. withdraw()_v132_v0.8.4__5207 [v0.8.4] bleu4=0.0 exact=0

EXPECTED:

```

function withdraw() public onlyAdmin {
    uint256 amount = address(this).balance;
    require(amount > 0, "Contract balance empty.");
    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "Transfer failed.");
}

```

```

    }
MODEL OUTPUT:
function withdraw() external {
    require(msg.sender == owner, "Only owner can withdraw");
    uint256 balance = address(this).balance;
    payable(owner).transfer(balance);
}

```

5209. isOverLiquified(uint256,uint256)_v0.8.0__5208 [v0.8.0] bleu4=0.0 exact=0

```

EXPECTED:
function contractSwap() internal swapLock {
    FeeRatios memory fees = _fees;

    uint256 dynamicLiquidityFee = isOverLiquified(
        targetLiquidity,
        targetLiquidityDenominator
    )
    ? 0
    : fees.liquidity;
    uint256 amountToLiquify = swapThreshold
        .mul(dynamicLiquidityFee)
        .div(fees.total)
        .div(2);
    uint256 amountToSwap = swapThreshold.sub(amountToLiquify);

    address[] memory path = new address[](2);
    path[0] = address(this);
    path[1] = router.WETH();

    uint256 balanceETHBefore = address(this).balance;

    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
        amountToSwap,
        0,
        path,
        address(this),
        block.timestamp
    );

    uint256 amountETH = address(this).balance.sub(balanceETHBefore);

    uint256 totalFees = (fees.total).sub(dynamicLiquidityFee.div(2));

    _feeWallets.marketing.transfer(
        amountETH.mul(fees.marketing).div(totalFees)
    );
    _feeWallets.eco.transfer(amountETH.mul(fees.eco).div(totalFees));

    uint256 amountETHLiquidity = amountETH
        .mul(dynamicLiquidityFee)
        .div(totalFees)
        .div(2);
    if (amountToLiquify > 0) {
        router.addLiquidityETH{value: amountETHLiquidity}(
            address(this),
            amountToLiquify,
            0,
            0,
            autoLiquidityReceiver,
            block.timestamp
        );
        emit AutoLiquify(amountETHLiquidity, amountToLiquify);
    }
}

```

```

MODEL OUTPUT:
function isOverLiquified(uint256 _rfi, uint256 _tfi)
    public
    view
    returns (bool)
{
    return _rfi < _tfi;
}

```

5210. transferOwnership(address)_v20_v0.8.2__5209 [v0.8.2] bleu4=0.0 exact=0

```

EXPECTED:
function transferOwnership(address newOwner) external onlyOwner {
    if (newOwner != address(0)) {
        owner = newOwner;
    }
}

```

MODEL OUTPUT:

```
function transferOwnership(address newOwner) public onlyOwner {
    require(newOwner!= address(0), "Ownable: new owner is the zero address");
    emit OwnershipTransferred(_owner, newOwner);
    _owner = newOwner;
}
```

5211. updateMaxSellAmount(uint256)_v0.8.4__ 5210 [v0.8.4] bleu4=0.5131 exact=0

EXPECTED:

```
function updateMaxSellAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000) / (10 ** decimals()));
    maxSellAmount = newNum * (10 ** decimals());
    emit UpdatedMaxSellAmount(maxSellAmount);
}
```

MODEL OUTPUT:

```
function updateMaxSellAmount(uint256 newNum) external onlyOwner {
    require(newNum >= (totalSupply() * 1 / 1000)/1e18, "Cannot set max sell amount lower than 0.1%");
    maxSellAmount = newNum * (10**18);
    emit UpdatedMaxSellAmount(maxSellAmount);
}
```

5212. removeExclude(address)_v0.8.9__ 5211 [v0.8.9] bleu4=0.3611 exact=0

EXPECTED:

```
function removeExclude(address account) public onlyOwner {
    require(isExcluded(account), "ERC20: Account is not excluded");
    excludeList[account] = false;
}
```

MODEL OUTPUT:

```
function removeExclude(address account) public onlyOwner {
    _isExcludedFromFee[account] = false;
}
```

5213. openTrading()_v681_v0.8.0__ 5212 [v0.8.0] bleu4=0.8492 exact=0

EXPECTED:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; tempAlgo = block.number;
    return true;
}
```

MODEL OUTPUT:

```
function openTrading() external onlyOwner returns (bool) {
    trading = true; theN = block.number; _theN = block.number;
    return true;
}
```

5214. monthsAhead()_v0.8.0__ 5213 [v0.8.0] bleu4=0.0 exact=0

EXPECTED:

```
function isDebtorHealthy() public view returns (bool) {
    int256 months = monthsAhead();
    bool moreThanMonthBehind = months <= -1;
    if (months == 0) {
        //accounts for being at least 1 day overdue
        int256 arrears =
            int256(expectedPayments()) -
            int256(agreementState.accumulatedRepayments);
        moreThanMonthBehind = arrears > 0;
    }
    return !moreThanMonthBehind;
}
```

MODEL OUTPUT:

```
function monthsAhead() public view returns (uint256) {
    return (block.timestamp - startTime) / (1 days);
}
```
